录友们，估计大家之前都下载过代码随想录PDF，但是那是我两年前整理的了，如今，很多题目的描述和表达都改了上十遍了。

而且之前的PDF一直都没有全集，而是重点章节的整理。

所以这次我把代码随想录最新的内容，以及全集全部整理成PDF分享出来。

不过，这里依然建议大家尽量去代码随想录网站学习，因为PDF更新难免不及时以及动图无法显示，而网站的更新是最及时的。

代码随想录网站 和 代码随想录算法公开课 都是免费开源的，足够大家学习算法之用，希望大家可以利用好这些资料。

最后也祝愿录友们都能拿到合适的offer，稳稳上岸，前程似锦，加油💪

-------- 程序员Carl 2023年8月15日 晚 18:03

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 数组理论基础

数组是非常基础的数据结构，在面试中，考察数组的题目一般在思维上都不难，主要是考察对代码的掌控能力

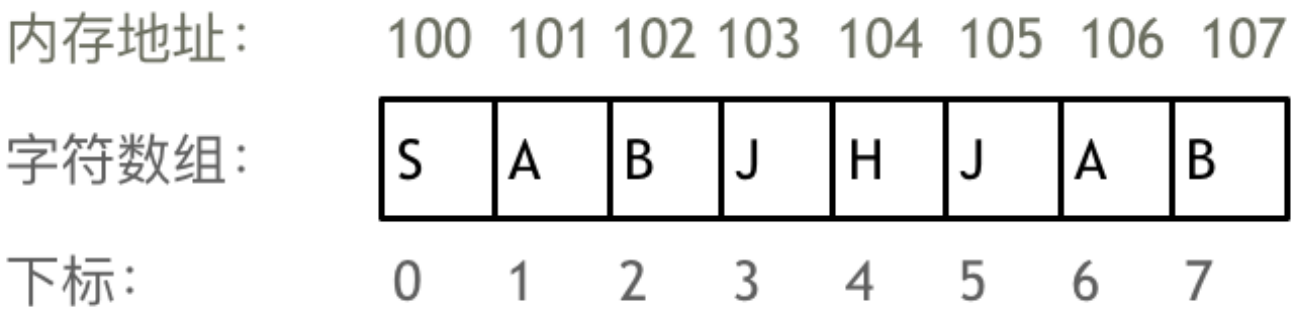
也就是说，想法很简单，但实现起来 可能就不是那么回事了。

首先要知道数组在内存中的存储方式，这样才能真正理解数组相关的面试题

数组是存放在连续内存空间上的相同类型数据的集合。

数组可以方便的通过下标索引的方式获取到下标下对应的数据。

举一个字符数组的例子，如图所示：

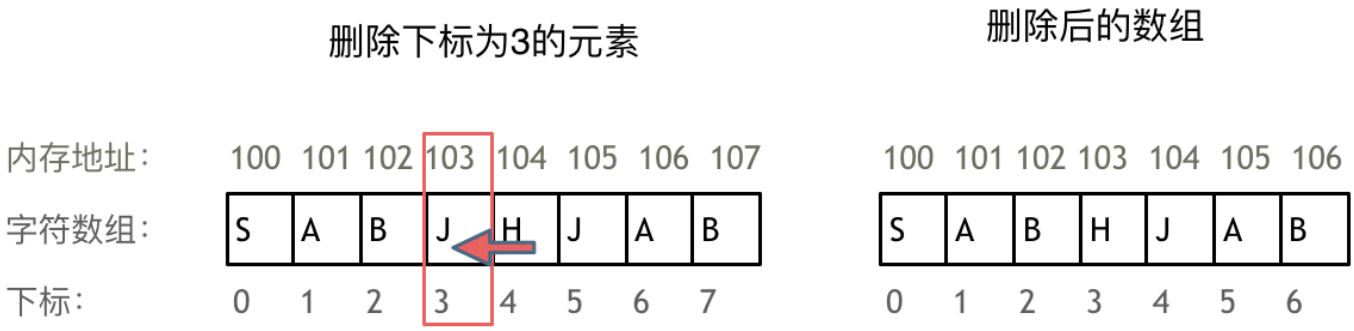


需要两点注意的是

- **数组下标都是从0开始的。**
- **数组内存空间的地址是连续的**

正是因为数组的在内存空间的地址是连续的，所以我们在删除或者增添元素的时候，就难免要移动其他元素的地址。

例如删除下标为3的元素，需要对下标为3的元素后面的所有元素都要做移动操作，如图所示：



而且大家如果使用C++的话，要注意vector 和 array的区别，vector的底层实现是array，严格来讲vector是容器，不是数组。

数组的元素是不能删的，只能覆盖。

那么二维数组直接上图，大家应该就知道怎么回事了

那么二维数组在内存的空间地址是连续的么？

不同编程语言的内存管理是不一样的，以C++为例，在C++中二维数组是连续分布的。

我们来做一个实验，C++测试代码如下：

```
void test_arr() {
    int array[2][3] = {
    {0, 1, 2},
    {3, 4, 5}
    };
    cout << &array[0][0] << " " << &array[0][1] << " " << &array[0][2] << endl;
    cout << &array[1][0] << " " << &array[1][1] << " " << &array[1][2] << endl;
}

int main() {
    test_arr();
}
```

测试地址为

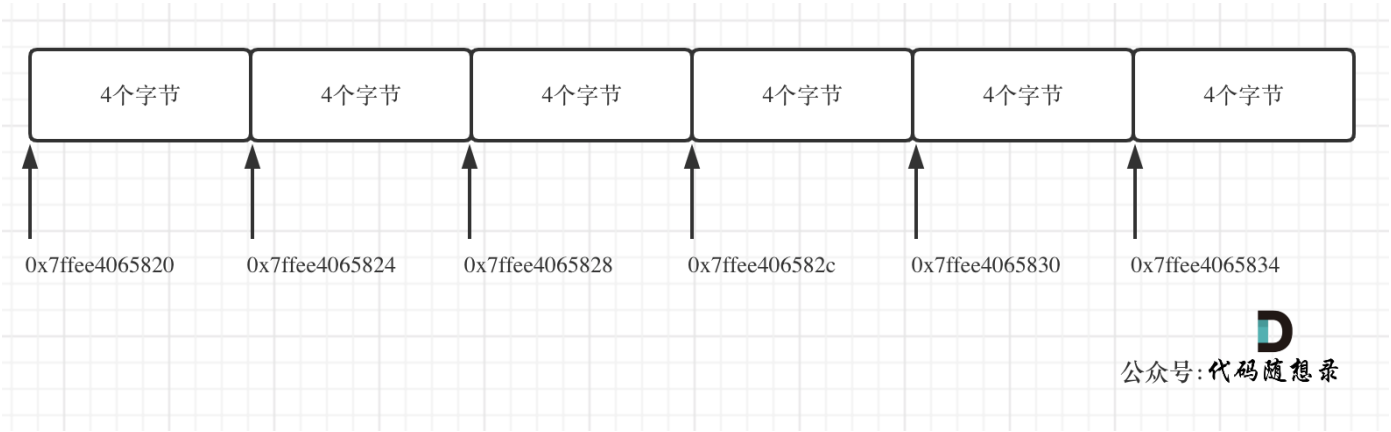
```
0x7ffee4065820 0x7ffee4065824 0x7ffee4065828
0x7ffee406582c 0x7ffee4065830 0x7ffee4065834
```

注意地址为16进制，可以看出二维数组地址是连续一条线的。

一些录友可能看不懂内存地址，我就简单介绍一下， 0x7ffee4065820 与 0x7ffee4065824 差了一个4，就是4个字节，因为这是一个int型的数组，所以两个相邻数组元素地址差4个字节。

0x7ffee4065828 与 0x7ffee406582c 也是差了4个字节，在16进制里8 + 4 = c，c就是12。

如图：



所以可以看出在C++中二维数组在地址空间上是连续的。

像Java是没有指针的，同时也不对程序员暴露其元素的地址，寻址操作完全交给虚拟机。

所以看不到每个元素的地址情况，这里我以Java为例，也做一个实验。

```
public static void test_arr() {
    int[][] arr = {{1, 2, 3}, {3, 4, 5}, {6, 7, 8}, {9,9,9}};
    System.out.println(arr[0]);
    System.out.println(arr[1]);
    System.out.println(arr[2]);
    System.out.println(arr[3]);
}
```

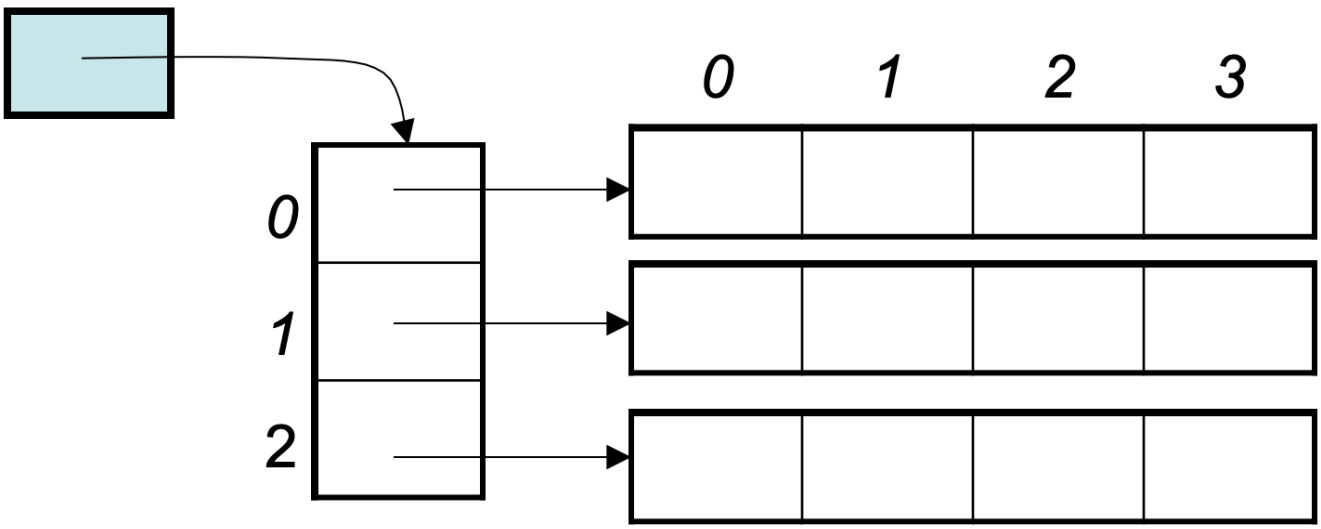
输出的地址为：

```
[I@7852e922
[I@4e25154f
[I@70dea4e
[I@5c647e05
```

这里的数值也是16进制，这不是真正的地址，而是经过处理过后的数值了，我们也可以看出，二维数组的每一行头结点的地址是没有规则的，更谈不上连续。

所以Java的二维数组可能是如下排列的方式：

int[][] rating = new int[3][4];



这里面试中数组相关的理论知识就介绍完了。

2. 二分查找

力扣题目链接

给定一个 n 个元素有序的（升序）整型数组 nums 和一个目标值 target ，写一个函数搜索 nums 中的 target，如果目标值存在返回下标，否则返回 -1。

示例 1:

```
输入：nums = [-1,0,3,5,9,12], target = 9
输出：4
解释：9 出现在 nums 中并且下标为 4
```

示例 2:

```
输入：nums = [-1,0,3,5,9,12], target = 2
输出：-1
解释：2 不存在 nums 中因此返回 -1
```

提示：

- 你可以假设 nums 中的所有元素是不重复的。
- n 将在 [1, 10000]之间。
- nums 的每个元素都将在 [-9999, 9999]之间。

算法公开课

《代码随想录》算法视频公开课：手把手带你撕出正确的二分法，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目的前提是数组为有序数组，同时题目还强调**数组中无重复元素**，因为一旦有重复元素，使用二分查找法返回的元素下标可能不是唯一的，这些都是使用二分法的前提条件，当大家看到题目描述满足如上条件的时候，可要想一想是不是可以用二分法了。

二分查找涉及的很多的边界条件，逻辑比较简单，但就是写不好。例如到底是`while(left < right)`还是`while(left <= right)`，到底是`right = middle`呢，还是要`right = middle - 1`呢?

大家写二分法经常写乱，主要是因为**对区间的定义没有想清楚，区间的定义就是不变量**。要在二分查找的过程中，保持不变量，就是在while寻找中每一次边界的处理都要坚持根据区间的定义来操作，这就是**循环不变量**规则。

写二分法，区间的定义一般为两种，左闭右闭即[left, right]，或者左闭右开即[left, right)。

下面我用这两种区间的定义分别讲解两种不同的二分写法。

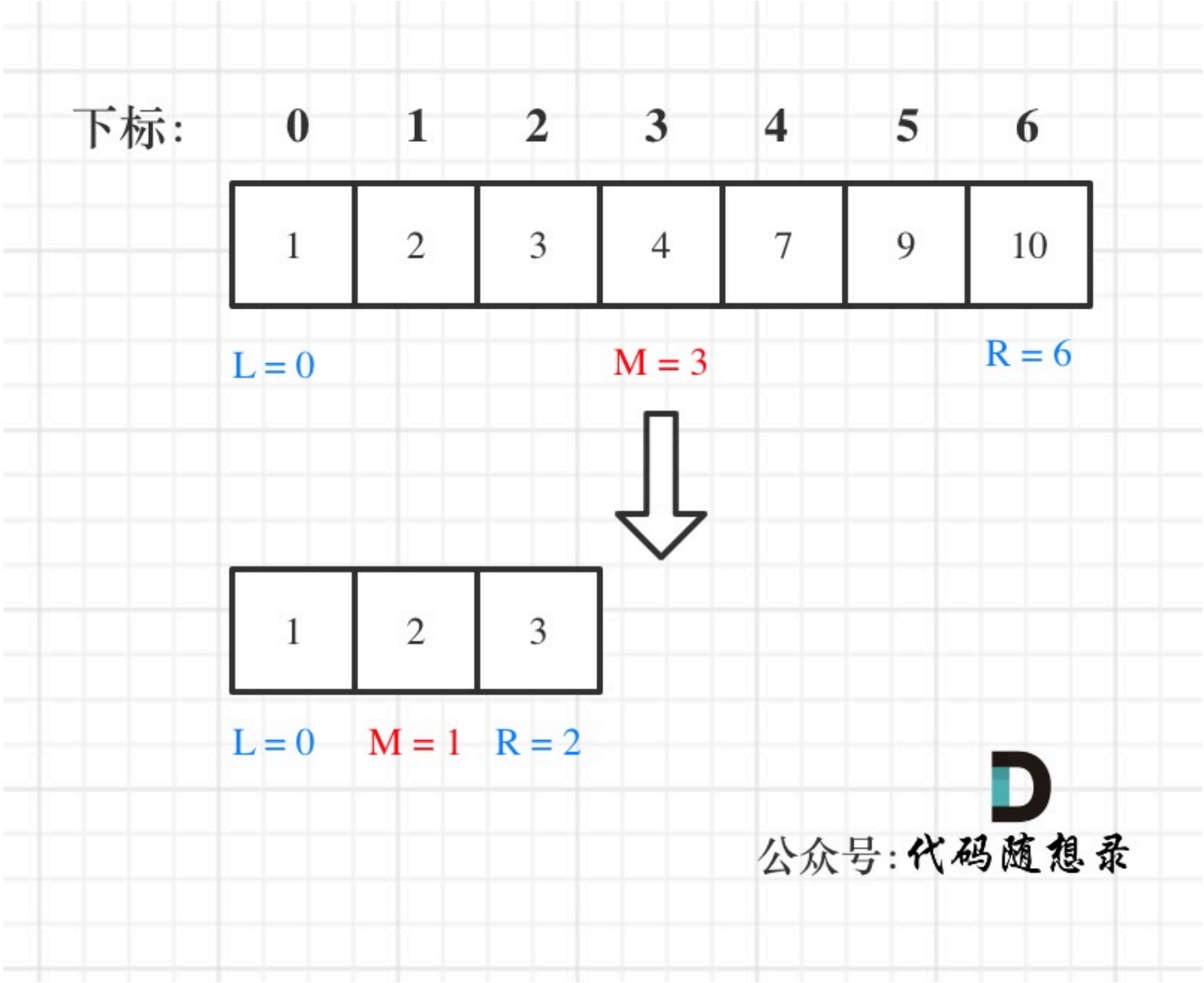
二分法第一种写法

第一种写法，我们定义 target 是在一个在左闭右闭的区间里，**也就是[left, right]（这个很重要非常重要）**。

区间的定义这就决定了二分法的代码应该如何写，**因为定义target在[left, right]区间，所以有如下两点：**

- while (left <= right) 要使用 <= ，因为left == right是有意义的，所以使用 <=
- if (nums[middle] > target) right 要赋值为 middle - 1，因为当前这个nums[middle]一定不是target，那么接下来要查找的左区间结束下标位置就是 middle - 1

例如在数组：1,2,3,4,7,9,10中查找元素2，如图所示：



代码如下：（详细注释）

```
// 版本一
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1; // 定义target在左闭右闭的区间里，[left, right]
        while (left <= right) { // 当left==right，区间[left, right]依然有效，所以用 <=
            int middle = left + ((right - left) / 2);// 防止溢出 等同于(left + right)/2
            if (nums[middle] > target) {
                right = middle - 1; // target 在左区间，所以[left, middle - 1]
            } else if (nums[middle] < target) {
                left = middle + 1; // target 在右区间，所以[middle + 1, right]
            } else { // nums[middle] == target
                return middle; // 数组中找到目标值，直接返回下标
            }
        }
        // 未找到目标值
        return -1;
```

```
    }
};
```

- 时间复杂度：O(log n)
- 空间复杂度：O(1)

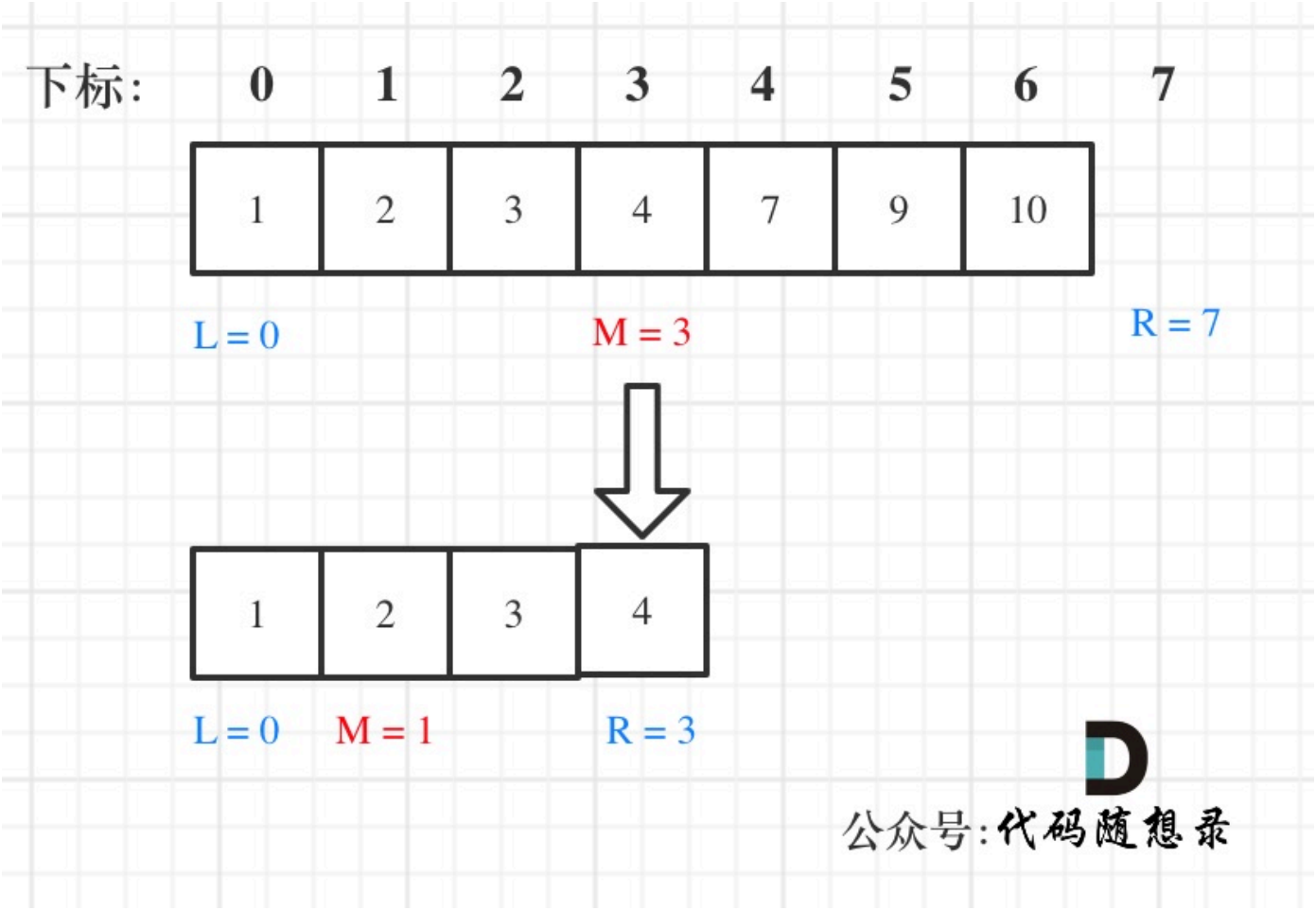
二分法第二种写法

如果说定义 target 是在一个在左闭右开的区间里，也就是[left, right) ，那么二分法的边界处理方式则截然不同。

有如下两点：

- while (left < right)，这里使用 < ,因为left == right在区间[left, right)是没有意义的
- if (nums[middle] > target) right 更新为 middle，因为当前nums[middle]不等于target，去左区间继续寻找，而寻找区间是左闭右开区间，所以right更新为middle，即：下一个查询区间不会去比较nums[middle]

在数组：1,2,3,4,7,9,10中查找元素2，如图所示：（**注意和方法一的区别**）



代码如下：（详细注释）

```
// 版本二
class Solution {
public:
    int search(vector<int>& nums, int target) {
```

```
        int left = 0;
        int right = nums.size(); // 定义target在左闭右开的区间里，即：[left, right)
        while (left < right) { // 因为left == right的时候，在[left, right)是无效的空间，所以使用 <
            int middle = left + ((right - left) >> 1);
            if (nums[middle] > target) {
                right = middle; // target 在左区间，在[left, middle)中
            } else if (nums[middle] < target) {
                left = middle + 1; // target 在右区间，在[middle + 1, right)中
            } else { // nums[middle] == target
                return middle; // 数组中找到目标值，直接返回下标
            }
        }
        // 未找到目标值
        return -1;
    }
};
```

- 时间复杂度：O(log n)
- 空间复杂度：O(1)

总结

二分法是非常重要的基础算法，为什么很多同学对于二分法都是**一看就会，一写就废**?

其实主要就是对区间的定义没有理解清楚，在循环中没有始终坚持根据查找区间的定义来做边界处理。

区间的定义就是不变量，那么在循环中坚持根据查找区间的定义来做边界处理，就是循环不变量规则。

本篇根据两种常见的区间定义，给出了两种二分法的写法，每一个边界为什么这么处理，都根据区间的定义做了详细介绍。

相信看完本篇应该对二分法有更深刻的理解了。

相关题目推荐

- 35.搜索插入位置
- 34.在排序数组中查找元素的第一个和最后一个位置
- 69.x 的平方根
- 367.有效的完全平方数

3. 移除元素

力扣题目链接

给你一个数组 nums 和一个值 val，你需要 原地 移除所有数值等于 val 的元素，并返回移除后数组的新长度。

不要使用额外的数组空间，你必须仅使用 O(1) 额外空间并**原地**修改输入数组。

元素的顺序可以改变。你不需要考虑数组中超出新长度后面的元素。

示例 1:
给定 nums = [3,2,2,3], val = 3,
函数应该返回新的长度 2, 并且 nums 中的前两个元素均为 2。
你不需要考虑数组中超出新长度后面的元素。

示例 2:
给定 nums = [0,1,2,2,3,0,4,2], val = 2,
函数应该返回新的长度 5, 并且 nums 中的前五个元素为 0, 1, 3, 0, 4。

你不需要考虑数组中超出新长度后面的元素。

算法公开课

[《代码随想录》算法视频公开课](#)：[数组中移除元素并不容易！LeetCode：27. 移除元素](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

有的同学可能说了，多余的元素，删掉不就得了。

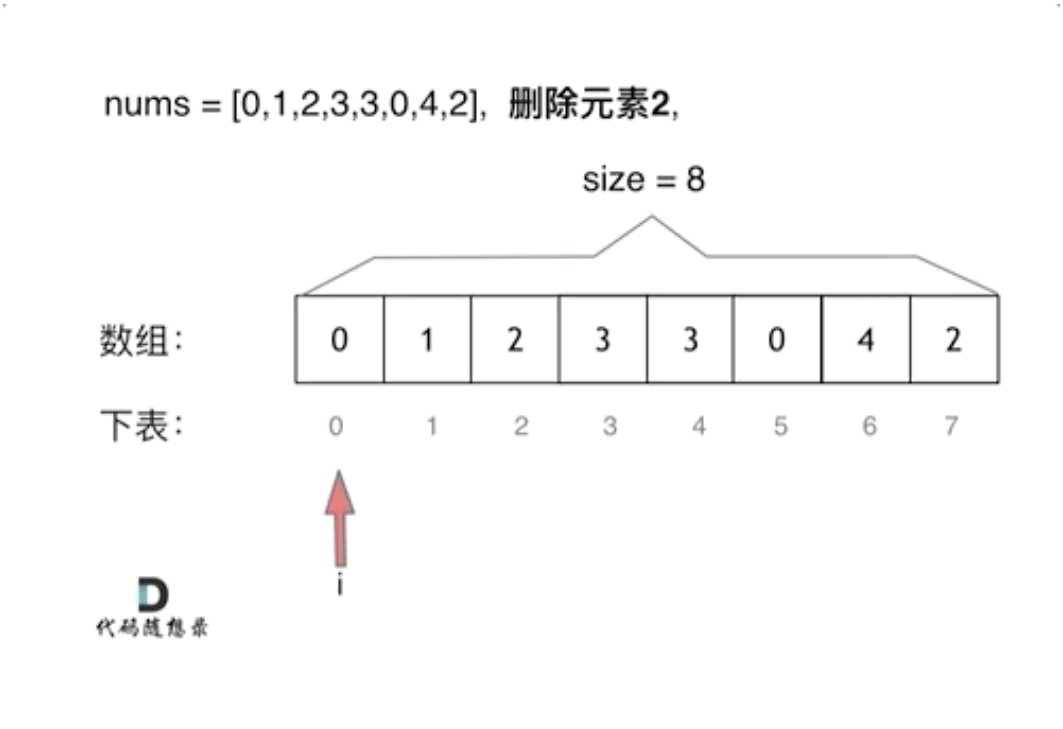
要知道数组的元素在内存地址中是连续的，不能单独删除数组中的某个元素，只能覆盖。

数组的基础知识可以看这里[程序员算法面试中，必须掌握的数组理论知识](#)。

暴力解法

这个题目暴力的解法就是两层for循环，一个for循环遍历数组元素 ，第二个for循环更新数组。

删除过程如下：



很明显暴力解法的时间复杂度是O(n^2)，这道题目暴力解法在leetcode上是可以过的。

代码如下：

```
// 时间复杂度：O(n^2)
// 空间复杂度：O(1)
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int size = nums.size();
        for (int i = 0; i < size; i++) {
            if (nums[i] == val) { // 发现需要移除的元素，就将数组集体向前移动一位
                for (int j = i + 1; j < size; j++) {
                    nums[j - 1] = nums[j];
                }
                i--; // 因为下标i以后的数值都向前移动了一位，所以i也向前移动一位
                size--; // 此时数组的大小-1
            }
        }
        return size;

    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

双指针法

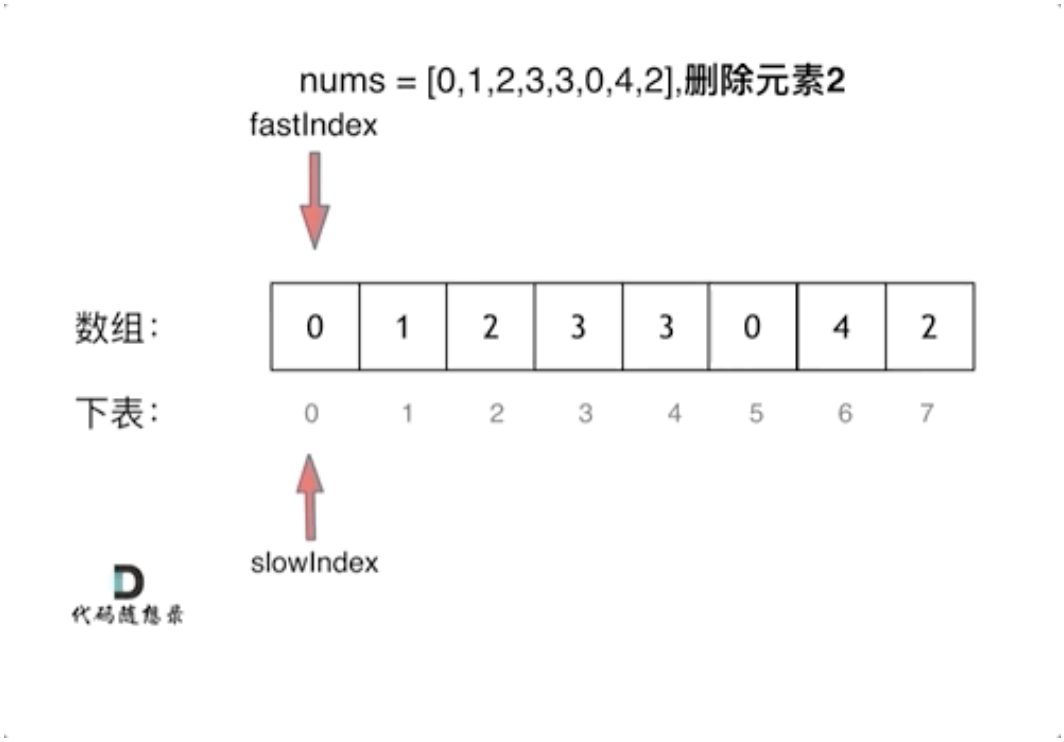
双指针法（快慢指针法）： **通过一个快指针和慢指针在一个for循环下完成两个for循环的工作。**

定义快慢指针

- 快指针：寻找新数组的元素 ，新数组就是不含有目标元素的数组
- 慢指针：指向更新 新数组下标的位置

很多同学这道题目做的很懵，就是不理解 快慢指针究竟都是什么含义，所以一定要明确含义，后面的思路就更容易理解了。

删除过程如下：



很多同学不了解

双指针法（快慢指针法）在数组和链表的操作中是非常常见的，很多考察数组、链表、字符串等操作的面试题，都使用双指针法。

后续都会一一介绍到，本题代码如下：

```
// 时间复杂度：O(n)
// 空间复杂度：O(1)
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int slowIndex = 0;
        for (int fastIndex = 0; fastIndex < nums.size(); fastIndex++) {
            if (val != nums[fastIndex]) {
                nums[slowIndex++] = nums[fastIndex];
            }
        }
        return slowIndex;
    }
};
```

注意这些实现方法并没有改变元素的相对位置！

- 时间复杂度：O(n)
- 空间复杂度：O(1)

```
/**
* 相向双指针方法，基于元素顺序可以改变的题目描述改变了元素相对位置，确保了移动最少元素
* 时间复杂度：O(n)
```

```
* 空间复杂度: O(1)
*/
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int leftIndex = 0;
        int rightIndex = nums.size() - 1;
        while (leftIndex <= rightIndex) {
            // 找左边等于val的元素
            while (leftIndex <= rightIndex && nums[leftIndex] != val){
                ++leftIndex;
            }
            // 找右边不等于val的元素
            while (leftIndex <= rightIndex && nums[rightIndex] == val) {
                -- rightIndex;
            }
            // 将右边不等于val的元素覆盖左边等于val的元素
            if (leftIndex < rightIndex) {
                nums[leftIndex++] = nums[rightIndex--];
            }
        }
        return leftIndex;   // leftIndex一定指向了最终数组末尾的下一个元素
    }
};
```

相关题目推荐

- 26.删除排序数组中的重复项
- 283.移动零
- 844.比较含退格的字符串
- 977.有序数组的平方

> 双指针风骚起来，也是无敌

4.有序数组的平方

力扣题目链接

给你一个按 非递减顺序 排序的整数数组 nums，返回 每个数字的平方 组成的新数组，要求也按 非递减顺序 排序。

示例 1：

- 输入：nums = [-4,-1,0,3,10]
- 输出：[0,1,9,16,100]
- 解释：平方后，数组变为 [16,1,0,9,100]，排序后，数组变为 [0,1,9,16,100]

示例 2：

- 输入：nums = [-7,-3,2,3,11]
- 输出：[4,9,9,49,121]

算法公开课

《代码随想录》算法视频公开课：双指针法经典题目!LeetCode:977.有序数组的平方，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

暴力排序

最直观的想法，莫过于：每个数平方之后，排个序，美滋滋，代码如下：

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& A) {
        for (int i = 0; i < A.size(); i++) {
            A[i] *= A[i];
        }
        sort(A.begin(), A.end()); // 快速排序
        return A;
    }
};
```

这个时间复杂度是 O(n + nlogn)， 可以说是O(nlogn)的时间复杂度，但为了和下面双指针法算法时间复杂度有鲜明对比，我记为 O(n + nlog n)。

双指针法

数组其实是有序的， 只不过负数平方之后可能成为最大数了。

那么数组平方的最大值就在数组的两端，不是最左边就是最右边，不可能是中间。

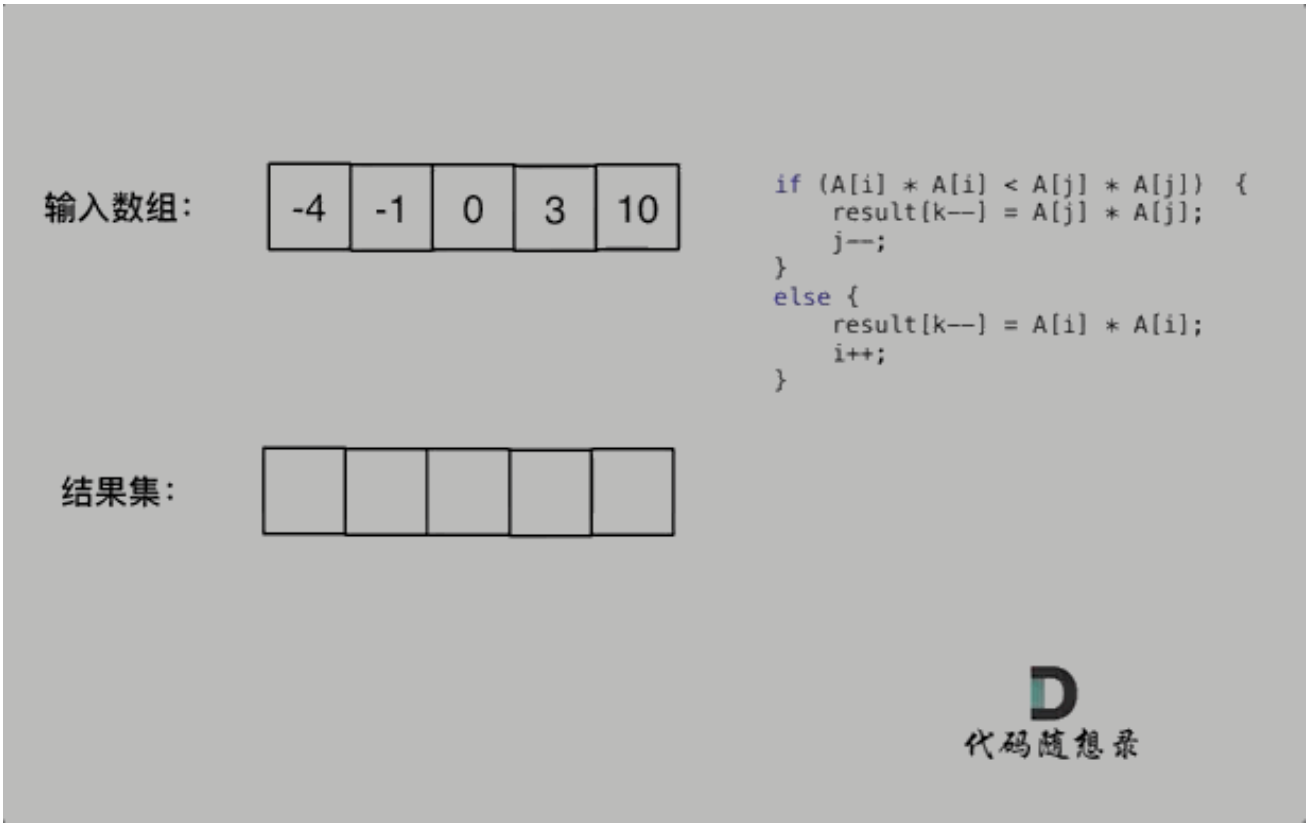
此时可以考虑双指针法了，i指向起始位置，j指向终止位置。

定义一个新数组result，和A数组一样的大小，让k指向result数组终止位置。

如果`A[i] * A[i] < A[j] * A[j]` 那么`result[k--] = A[j] * A[j];` 。

如果`A[i] * A[i] >= A[j] * A[j]` 那么`result[k--] = A[i] * A[i];` 。

如动画所示：



不难写出如下代码：

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& A) {
        int k = A.size() - 1;
        vector<int> result(A.size(), 0);
        for (int i = 0, j = A.size() - 1; i <= j;) { // 注意这里要i <= j，因为最后要处理两个元素
            if (A[i] * A[i] < A[j] * A[j])  {
                result[k--] = A[j] * A[j];
                j--;
            }
            else {
                result[k--] = A[i] * A[i];
                i++;
            }
        }
        return result;
    }
};
```

此时的时间复杂度为O(n)，相对于暴力排序的解法O(n + nlog n)还是提升不少的。

这里还是说一下，大家不必太在意leetcode上执行用时，打败多少多少用户，这个就是一个玩具，非常不准确。

做题的时候自己能分析出来时间复杂度就可以了，至于leetcode上执行用时，大概看一下就行，只要达到最优的时间复杂度就可以了，

一样的代码多提交几次可能就击败百分之百了.....

5.长度最小的子数组

力扣题目链接

给定一个含有 n 个正整数的数组和一个正整数 s ，找出该数组中满足其和 ≥ s 的长度最小的 连续 子数组，并返回其长度。如果不存在符合条件的子数组，返回 0。

示例：

- 输入：s = 7, nums = [2,3,1,2,4,3]
- 输出：2
- 解释：子数组 [4,3] 是该条件下的长度最小的子数组。

提示：

- 1 <= target <= 10^9
- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^5

算法公开课

《代码随想录》算法视频公开课：拿下滑动窗口！ | LeetCode 209 长度最小的子数组，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

暴力解法

这道题目暴力解法当然是 两个for循环，然后不断的寻找符合条件的子序列，时间复杂度很明显是O(n^2)。

代码如下：

```
class Solution {
public:
    int minSubArrayLen(int s, vector<int>& nums) {
        int result = INT32_MAX; // 最终的结果
        int sum = 0; // 子序列的数值之和
        int subLength = 0; // 子序列的长度
        for (int i = 0; i < nums.size(); i++) { // 设置子序列起点为i
            sum = 0;
            for (int j = i; j < nums.size(); j++) { // 设置子序列终止位置为j
                sum += nums[j];
                if (sum >= s) { // 一旦发现子序列和超过了s，更新result
                    subLength = j - i + 1; // 取子序列的长度
                    result = result < subLength ? result : subLength;
                    break; // 因为我们是找符合条件最短的子序列，所以一旦符合条件就break
                }
```

```
                }
            }
            // 如果result没有被赋值的话，就返回0，说明没有符合条件的子序列
            return result == INT32_MAX ? 0 : result;
        }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

后面力扣更新了数据，暴力解法已经超时了。

滑动窗口

接下来就开始介绍数组操作中另一个重要的方法：**滑动窗口**。

所谓滑动窗口，**就是不断的调节子序列的起始位置和终止位置，从而得出我们要想的结果**。

在暴力解法中，是一个for循环滑动窗口的起始位置，一个for循环为滑动窗口的终止位置，用两个for循环 完成了一个不断搜索区间的过程。

那么滑动窗口如何用一个for循环来完成这个操作呢。

首先要思考 如果用一个for循环，那么应该表示 滑动窗口的起始位置，还是终止位置。

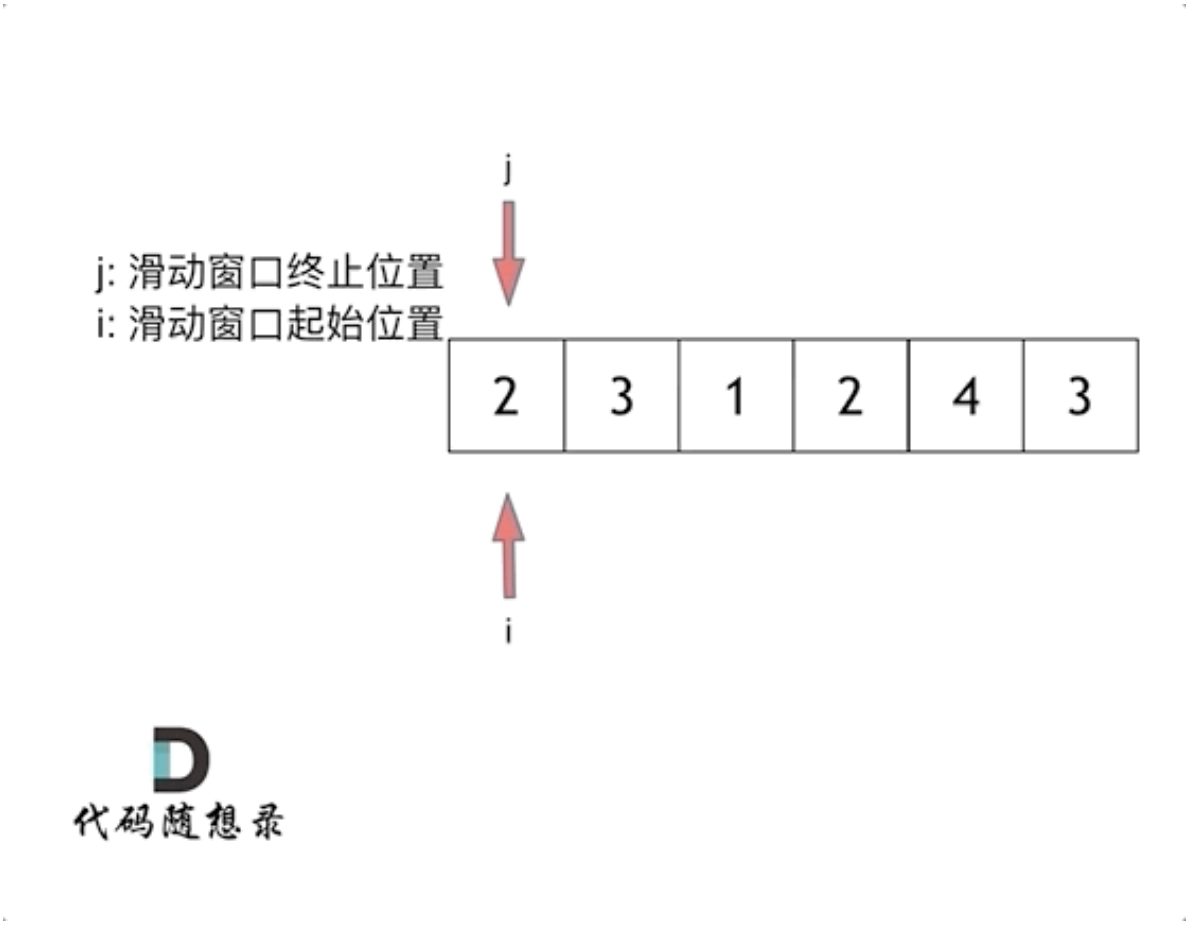
如果只用一个for循环来表示 滑动窗口的起始位置，那么如何遍历剩下的终止位置?

此时难免再次陷入 暴力解法的怪圈。

所以 只用一个for循环，那么这个循环的索引，一定是表示 滑动窗口的终止位置。

那么问题来了， 滑动窗口的起始位置如何移动呢?

这里还是以题目中的示例来举例，s=7， 数组是 2，3，1，2，4，3，来看一下查找的过程：



最后找到 4，3 是最短距离。

其实从动画中可以发现滑动窗口也可以理解为双指针法的一种！只不过这种解法更像是一个窗口的移动，所以叫做滑动窗口更适合一些。

在本题中实现滑动窗口，主要确定如下三点：

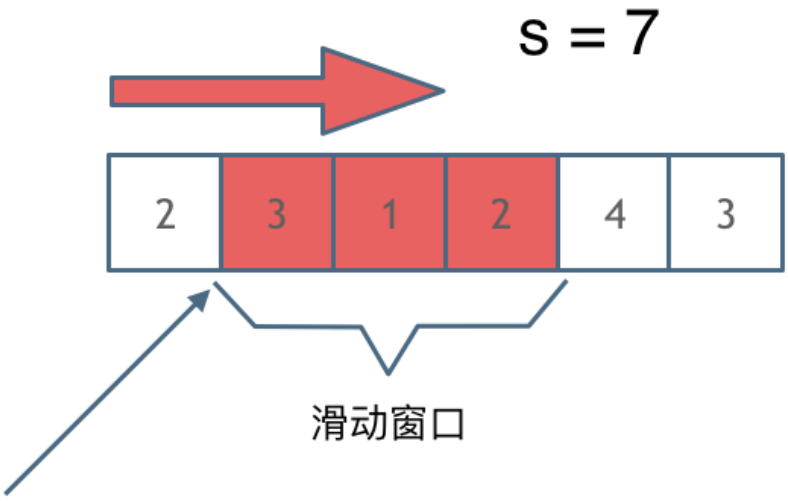
- 窗口内是什么？
- 如何移动窗口的起始位置？
- 如何移动窗口的结束位置？

窗口就是 满足其和 ≥ s 的长度最小的 连续 子数组。

窗口的起始位置如何移动：如果当前窗口的值大于s了，窗口就要向前移动了（也就是该缩小了）。

窗口的结束位置如何移动：窗口的结束位置就是遍历数组的指针，也就是for循环里的索引。

解题的关键在于 窗口的起始位置如何移动，如图所示：



此块代码的精髓就是动态调节滑动窗口的起始位置

```
while (sum >= s) {
    subLength = (j - i + 1); // 取子序列的长度
    result = result < subLength ? result : subLength;
    sum -= nums[i++]; // 这里体现出滑动窗口的精髓之处，不断变更i（子序列的起始位置）
}
```

可以发现**滑动窗口的精妙之处在于根据当前子序列和大小的情况，不断调节子序列的起始位置。从而将O(n^2)暴力解法降为O(n)。**

C++代码如下：

```
class Solution {
public:
    int minSubArrayLen(int s, vector<int>& nums) {
        int result = INT32_MAX;
        int sum = 0; // 滑动窗口数值之和
        int i = 0; // 滑动窗口起始位置
        int subLength = 0; // 滑动窗口的长度
        for (int j = 0; j < nums.size(); j++) {
            sum += nums[j];
            // 注意这里使用while，每次更新 i（起始位置），并不断比较子序列是否符合条件
            while (sum >= s) {
                subLength = (j - i + 1); // 取子序列的长度
                result = result < subLength ? result : subLength;
                sum -= nums[i++]; // 这里体现出滑动窗口的精髓之处，不断变更i（子序列的起始位置）
            }
        }
        // 如果result没有被赋值的话，就返回0，说明没有符合条件的子序列
        return result == INT32_MAX ? 0 : result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

一些录友会疑惑为什么时间复杂度是O(n)。

不要以为for里放一个while就以为是O(n^2)啊， 主要是看每一个元素被操作的次数，每个元素在滑动窗后进来操作一次，出去操作一次，每个元素都是被操作两次，所以时间复杂度是 2 × n 也就是O(n)。

相关题目推荐

- 904.水果成篮
- 76.最小覆盖子串

6.螺旋矩阵II

力扣题目链接

给定一个正整数 n，生成一个包含 1 到 n^2 所有元素，且元素按顺时针顺序螺旋排列的正方形矩阵。

示例:

输入: 3
输出:
[
 [1, 2, 3],
 [8, 9, 4],
 [7, 6, 5]
]

算法公开课

《代码随想录》算法视频公开课：拿下螺旋矩阵！LeetCode：59.螺旋矩阵II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目可以说在面试中出现频率较高的题目，**本题并不涉及到什么算法，就是模拟过程，但却十分考察对代码的掌控能力。**

要如何画出这个螺旋排列的正方形矩阵呢?

相信很多同学刚开始做这种题目的时候，上来就是一波判断猛如虎。

结果运行的时候各种问题，然后开始各种修修补补，最后发现改了这里那里有问题，改了那里这里又跑不起来了。

大家还记得我们在这篇文章数组：每次遇到二分法，都是一看就会，一写就废中讲解了二分法，提到如果要写出正确的二分法一定要坚持**循环不变量原则**。

而求解本题依然是要坚持循环不变量原则。

模拟顺时针画矩阵的过程:

- 填充上行从左到右
- 填充右列从上到下
- 填充下行从右到左

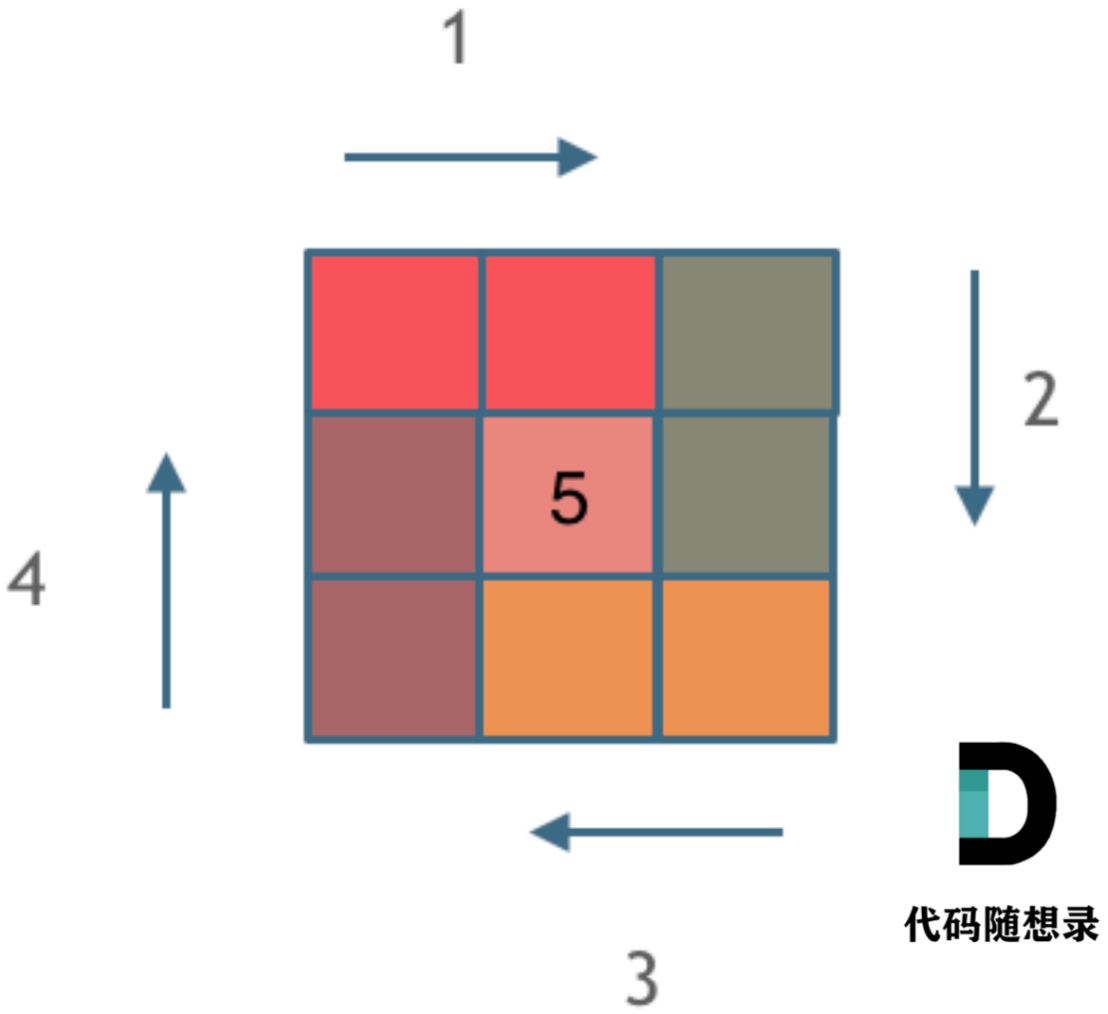
- 填充左列从下到上

由外向内一圈一圈这么画下去。

可以发现这里的边界条件非常多，在一个循环中，如此多的边界条件，如果不按照固定规则来遍历，那就是**一进循环深似海，从此offer是路人**。

这里一圈下来，我们要画每四条边，这四条边怎么画，每画一条边都要坚持一致的左闭右开，或者左开右闭的原则，这样这一圈才能按照统一的规则画下来。

那么我按照左闭右开的原则，来画一圈，大家看一下：



这里每一种颜色，代表一条边，我们遍历的长度，可以看出每一个拐角处的处理规则，拐角处让给新的一条边来继续画。

这也是坚持了每条边左闭右开的原则。

一些同学做这道题目之所以一直写不好，代码越写越乱。

就是因为在画每一条边的时候，一会左开右闭，一会左闭右闭，一会又来左闭右开，岂能不乱。

代码如下，已经详细注释了每一步的目的，可以看出while循环里判断的情况是很多的，代码里处理的原则也是统一的左闭右开。

整体C++代码如下：

```
class Solution {
public:
    vector<vector<int>> generateMatrix(int n) {
        vector<vector<int>> res(n, vector<int>(n, 0)); // 使用vector定义一个二维数组
        int startx = 0, starty = 0; // 定义每循环一个圈的起始位置
        int loop = n / 2; // 每个圈循环几次，例如n为奇数3，那么loop = 1 只是循环一圈，矩阵中间的值需要单独处理
        int mid = n / 2; // 矩阵中间的位置，例如：n为3， 中间的位置就是(1, 1)，n为5，中间位置为(2, 2)
        int count = 1; // 用来给矩阵中每一个空格赋值
        int offset = 1; // 需要控制每一条边遍历的长度，每次循环右边界收缩一位
        int i,j;
        while (loop --) {
            i = startx;
            j = starty;

            // 下面开始的四个for就是模拟转了一圈
            // 模拟填充上行从左到右(左闭右开)
            for (j = starty; j < n - offset; j++) {
                res[startx][j] = count++;
            }
            // 模拟填充右列从上到下(左闭右开)
            for (i = startx; i < n - offset; i++) {
                res[i][j] = count++;
            }
            // 模拟填充下行从右到左(左闭右开)
            for (; j > starty; j--) {
                res[i][j] = count++;
            }
            // 模拟填充左列从下到上(左闭右开)
            for (; i > startx; i--) {
                res[i][j] = count++;
            }

            // 第二圈开始的时候，起始位置要各自加1， 例如：第一圈起始位置是(0, 0)，第二圈起始位置是(1, 1)
            startx++;
            starty++;

            // offset 控制每一圈里每一条边遍历的长度
            offset += 1;
        }

        // 如果n为奇数的话，需要单独给矩阵最中间的位置赋值
        if (n % 2) {
            res[mid][mid] = count;
        }
```

```
        return res;
    }
};
```

- 时间复杂度 O(n^2): 模拟遍历二维矩阵的时间
- 空间复杂度 O(1)

类似题目

- 54.螺旋矩阵
- 剑指Offer 29.顺时针打印矩阵

7. 数组总结篇

数组理论基础

数组是非常基础的数据结构，在面试中，考察数组的题目一般在思维上都不难，主要是考察对代码的掌控能力

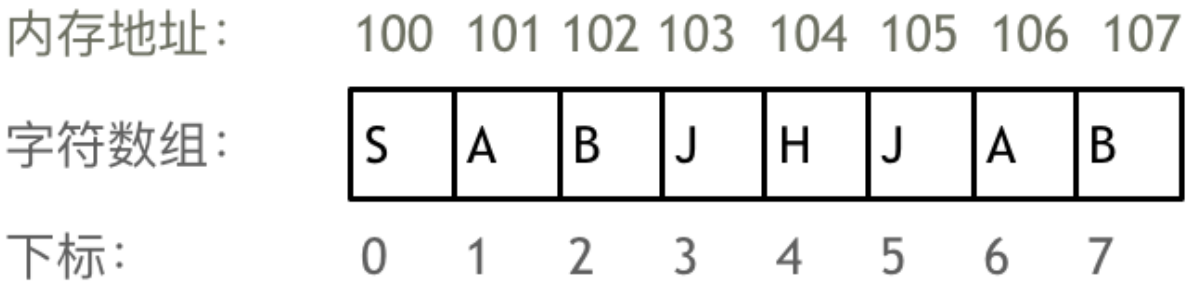
也就是说，想法很简单，但实现起来 可能就不是那么回事了。

首先要知道数组在内存中的存储方式，这样才能真正理解数组相关的面试题

数组是存放在连续内存空间上的相同类型数据的集合。

数组可以方便的通过下标索引的方式获取到下标下对应的数据。

举一个字符数组的例子，如图所示：

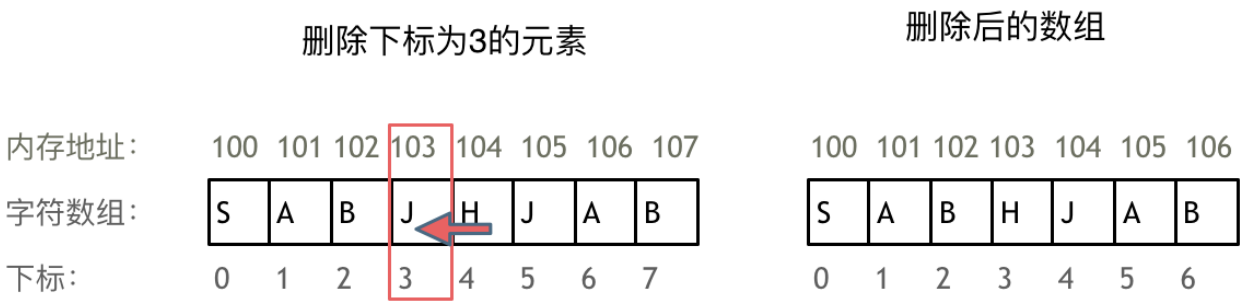


需要两点注意的是

- **数组下标都是从0开始的。**
- **数组内存空间的地址是连续的**

正是**因为数组的在内存空间的地址是连续的，所以我们在删除或者增添元素的时候，就难免要移动其他元素的地址。**

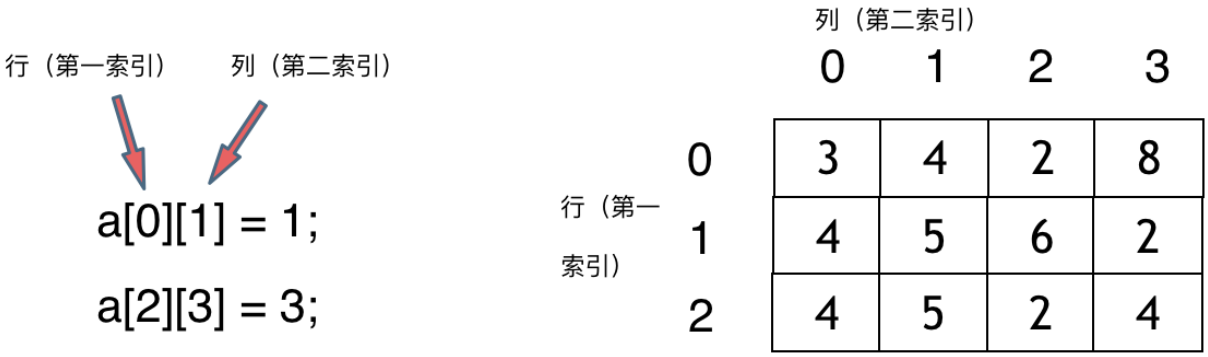
例如删除下标为3的元素，需要对下标为3的元素后面的所有元素都要做移动操作，如图所示：



而且大家如果使用C++的话，要注意vector 和 array的区别，vector的底层实现是array，严格来讲vector是容器，不是数组。

数组的元素是不能删的，只能覆盖。

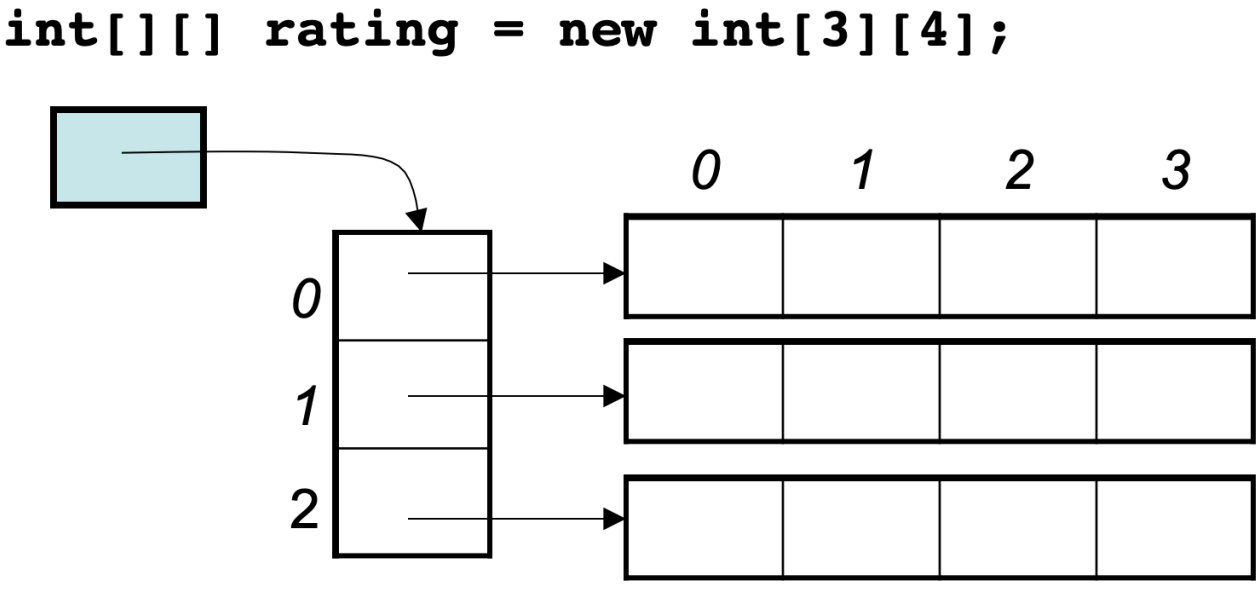
那么二维数组直接上图，大家应该就知道怎么回事了



那么二维数组在内存的空间地址是连续的么?

我们来举一个Java的例子，例如：`int[][] rating = new int[3][4];`， 这个二维数组在内存空间可不是一个`3*4`的连续地址空间

看了下图，就应该明白了：



所以**Java的二维数组在内存中不是`3*4`的连续地址空间，而是四条连续的地址空间组成!**

数组的经典题目

在面试中，数组是必考的基础数据结构。

其实数组的题目在思想上一般比较简单的，但是如果想高效，并不容易。

我们之前一共讲解了四道经典数组题目，每一道题目都代表一个类型，一种思想。

二分法

数组：每次遇到二分法，都是一看就会，一写就废

这道题目呢，考察数组的基本操作，思路很简单，但是通过率在简单题里并不高，不要轻敌。

可以使用暴力解法，通过这道题目，如果追求更优的算法，建议试一试用二分法，来解决这道题目

- 暴力解法时间复杂度：O(n)
- 二分法时间复杂度：O(logn)

在这道题目中我们讲到了**循环不变量原则**，只有在循环中坚持对区间的定义，才能清楚的把握循环中的各种细节。

二分法是算法面试中的常考题，建议通过这道题目，锻炼自己手撕二分的能力。

双指针法

- 数组：就移除个元素很难么？

双指针法（快慢指针法）：**通过一个快指针和慢指针在一个for循环下完成两个for循环的工作。**

- 暴力解法时间复杂度：O(n^2)
- 双指针时间复杂度：O(n)

这道题目迷惑了不少同学，纠结于数组中的元素为什么不能删除，主要是因为以下两点：

- 数组在内存中是连续的地址空间，不能释放单一元素，如果要释放，就是全释放（程序运行结束，回收内存栈空间）。
- C++中vector和array的区别一定要弄清楚，vector的底层实现是array，封装后使用更友好。

双指针法（快慢指针法）在数组和链表的操作中是非常常见的，很多考察数组和链表操作的面试题，都使用双指针法。

滑动窗口

- 数组：滑动窗口拯救了你

本题介绍了数组操作中的另一个重要思想：滑动窗口。

- 暴力解法时间复杂度：O(n^2)
- 滑动窗口时间复杂度：O(n)

本题中，主要要理解滑动窗口如何移动 窗口起始位置，达到动态更新窗口大小的，从而得出长度最小的符合条件的长度。

滑动窗口的精妙之处在于根据当前子序列和大小的情况，不断调节子序列的起始位置。从而将O(n^2)的暴力解法降为O(n)。

如果没有接触过这一类的方法，很难想到类似的解题思路，滑动窗口方法还是很巧妙的。

模拟行为

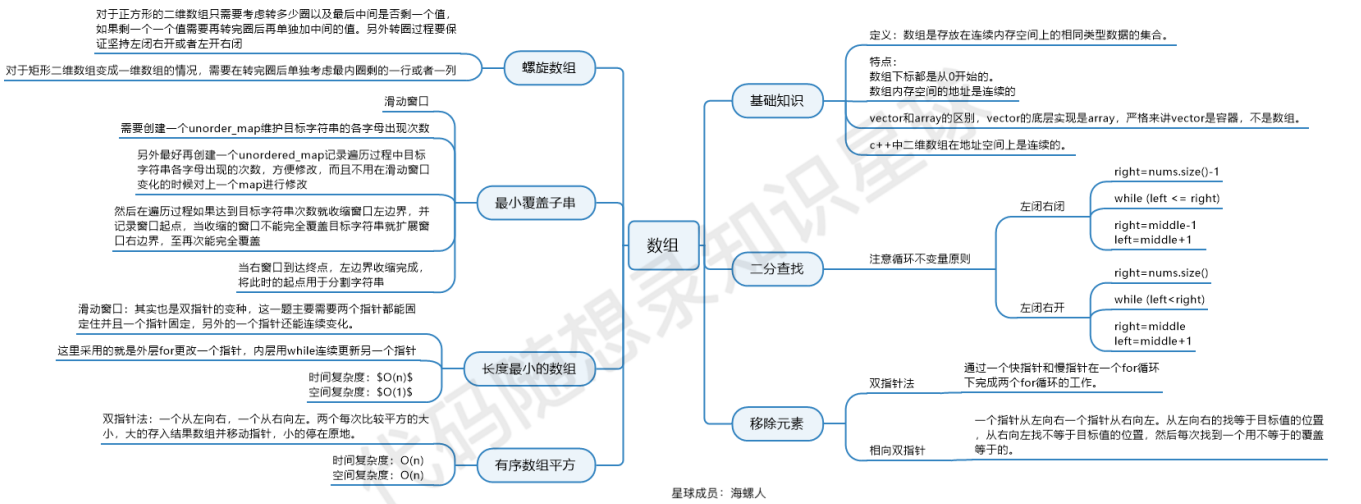
- 数组：这个循环可以转懵很多人！

模拟类的题目在数组中很常见，不涉及到什么算法，就是单纯的模拟，十分考察大家对代码的掌控能力。

在这道题目中，我们再一次介绍到了**循环不变量原则**，其实这也是写程序中的重要原则。

相信大家有遇到过这种情况： 感觉题目的边界调节超多，一波接着一波的判断，找边界，拆了东墙补西墙，好不容易运行通过了，代码写的十分冗余，毫无章法，其实**真正解决题目的代码都是简洁的，或者有原则性的**，大家可以在这道题目中体会到这一点。

总结



这个图是 代码随想录知识星球 成员：海螺人，所画，总结的非常好，分享给大家。

从二分法到双指针，从滑动窗口到螺旋矩阵，相信如果大家真的认真做了「代码随想录」每日推荐的题目，定会有所收获。

推荐的题目即使大家之前做过了，再读一遍文章，也会帮助你提炼出解题的精髓所在。

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

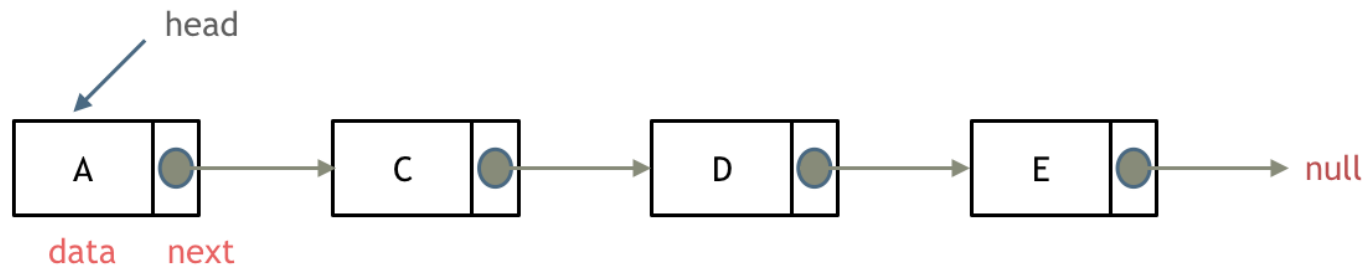
特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 关于链表，你该了解这些！

什么是链表，链表是一种通过指针串联在一起的线性结构，每一个节点由两部分组成，一个是数据域一个是指针域（存放指向下一个节点的指针），最后一个节点的指针域指向null（空指针的意思）。

链表的入口节点称为链表的头结点也就是head。

如图所示：



链表的类型

接下来说一下链表的几种类型:

单链表

刚刚说的就是单链表。

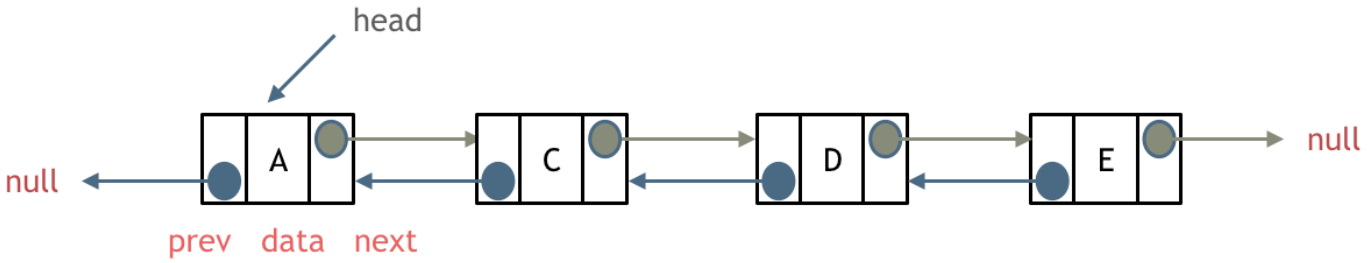
双链表

单链表中的指针域只能指向节点的下一个节点。

双链表：每一个节点有两个指针域，一个指向下一个节点，一个指向上一个节点。

双链表 既可以向前查询也可以向后查询。

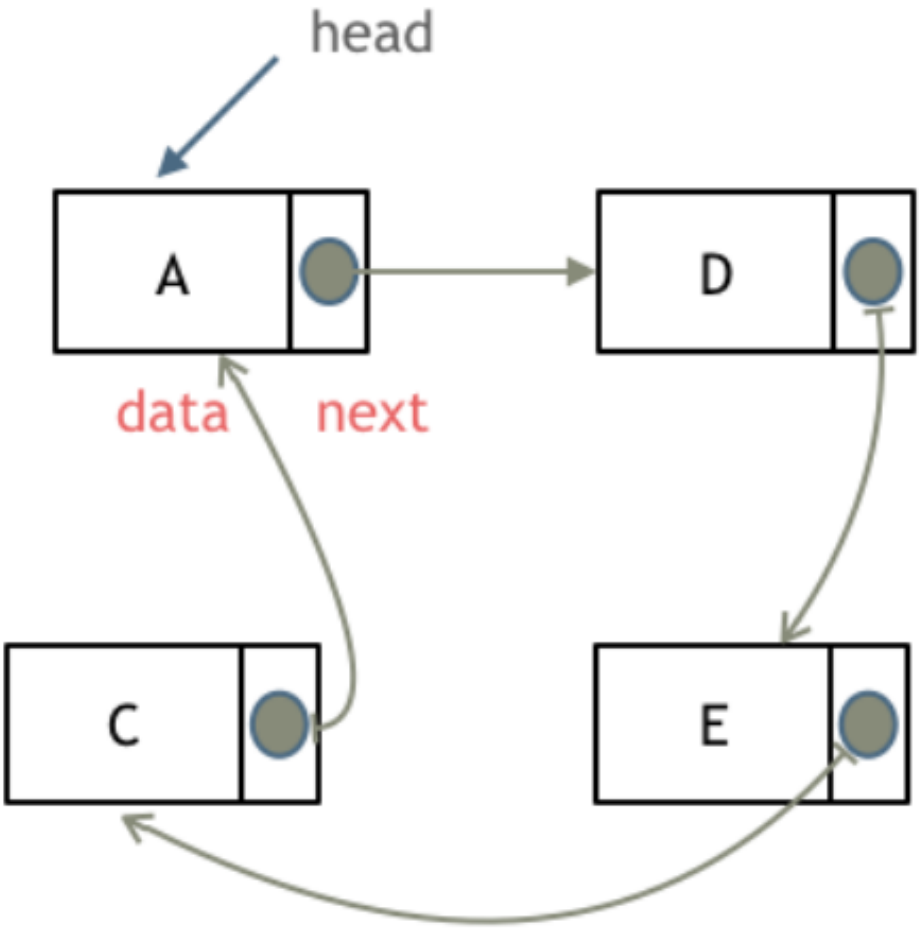
如图所示：



循环链表

循环链表，顾名思义，就是链表首尾相连。

循环链表可以用来解决约瑟夫环问题。



链表的存储方式

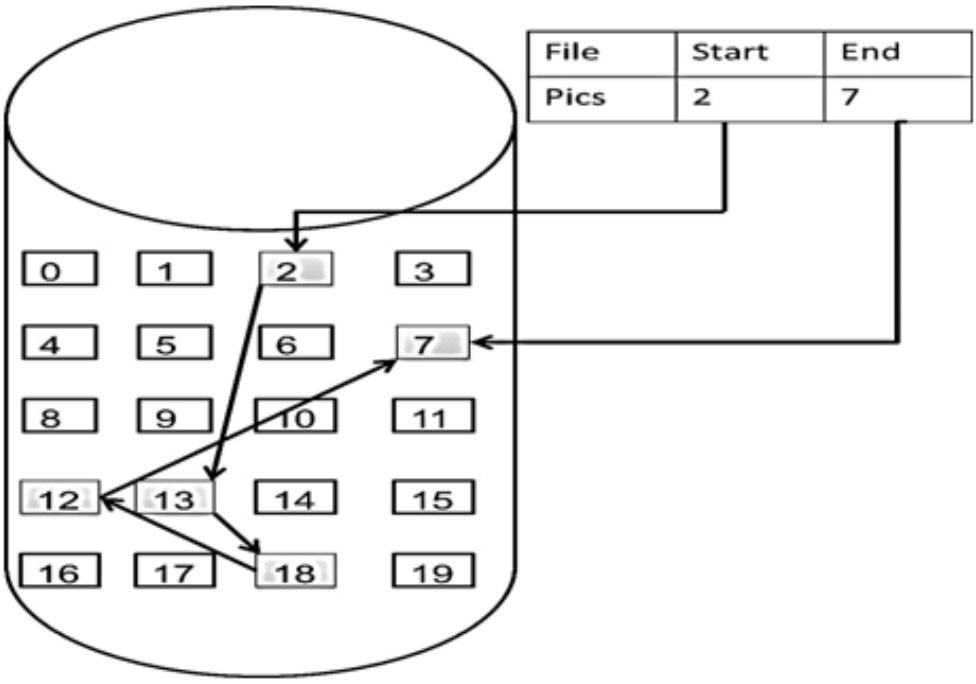
了解完链表的类型，再来说一说链表在内存中的存储方式。

数组是在内存中是连续分布的，但是链表在内存中可不是连续分布的。

链表是通过指针域的指针链接在内存中各个节点。

所以链表中的节点在内存中不是连续分布的 ，而是散乱分布在内存中的某地址上，分配机制取决于操作系统的内存管理。

如图所示：



这个链表起始节点为2， 终止节点为7， 各个节点分布在内存的不同地址空间上，通过指针串联在一起。

链表的定义

接下来说一说链表的定义。

链表节点的定义，很多同学在面试的时候都写不好。

这是因为平时在刷leetcode的时候，链表的节点都默认定义好了，直接用就行了，所以同学们都没有注意到链表的节点是如何定义的。

而在面试的时候，一旦要自己手写链表，就写的错漏百出。

这里我给出C/C++的定义链表节点方式，如下所示：

```
// 单链表
struct ListNode {
    int val;  // 节点上存储的元素
    ListNode *next;  // 指向下一个节点的指针
    ListNode(int x) : val(x), next(NULL) {}  // 节点的构造函数
};
```

有同学说了，我不定义构造函数行不行，答案是可以的，C++默认生成一个构造函数。

但是这个构造函数不会初始化任何成员变量，下面我来举两个例子：

通过自己定义构造函数初始化节点：

```
ListNode* head = new ListNode(5);
```

使用默认构造函数初始化节点：

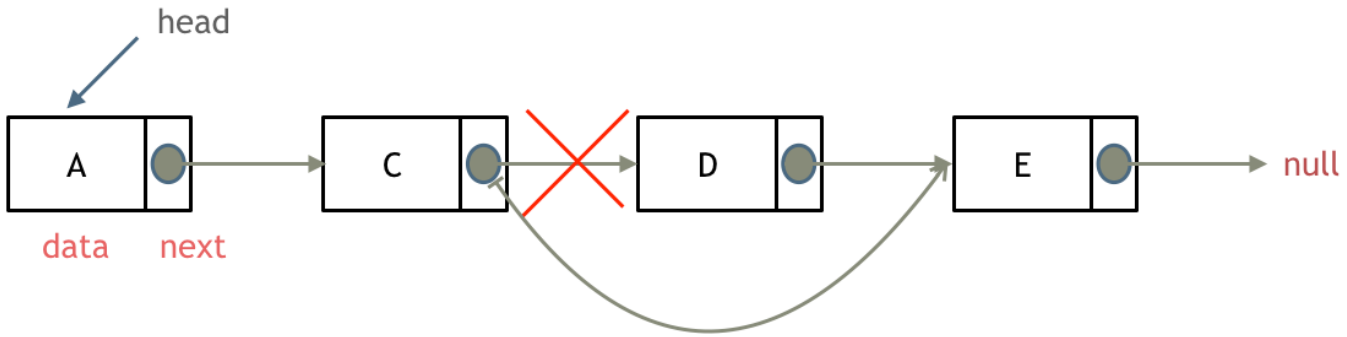
```
ListNode* head = new ListNode();
head->val = 5;
```

所以如果不定义构造函数使用默认构造函数的话，在初始化的时候就不能直接给变量赋值！

链表的操作

删除节点

删除D节点，如图所示：



只要将C节点的next指针 指向E节点就可以了。

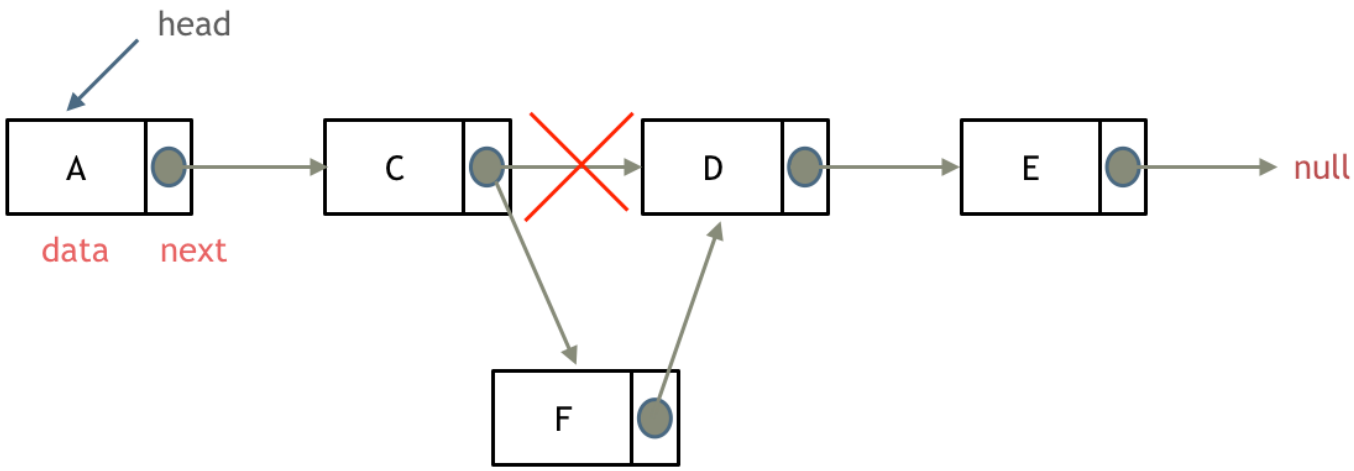
那有同学说了，D节点不是依然存留在内存里么？只不过是没有在这个链表里而已。

是这样的，所以在C++里最好是再手动释放这个D节点，释放这块内存。

其他语言例如Java、Python，就有自己的内存回收机制，就不用自己手动释放了。

添加节点

如图所示：



可以看出链表的增添和删除都是O(1)操作，也不会影响到其他节点。

但是要注意，要是删除第五个节点，需要从头节点查找到第四个节点通过next指针进行删除操作，查找的时间复杂度是O(n)。

性能分析

再把链表的特性和数组的特性进行一个对比，如图所示：

	插入/删除（时间复杂度）	查询（时间复杂度）	适用场景
数组	O(n)	O(1)	数据量固定，频繁查询，较少增删
链表	O(1)	O(n)	数据量不固定，频繁增删，较少查询

数组在定义的时候，长度就是固定的，如果想改动数组的长度，就需要重新定义一个新的数组。

链表的长度可以是不固定的，并且可以动态增删， 适合数据量不固定，频繁增删，较少查询的场景。

相信大家已经对链表足够的了解，后面我会讲解关于链表的高频面试题目，我们下期见！

> 链表操作中，可以使用原链表来直接进行删除操作，也可以设置一个虚拟头结点再进行删除操作，接下来看一看哪种方式更方便。

2.移除链表元素

力扣题目链接

题意：删除链表中等于给定值 val 的所有节点。

示例 1：
输入：head = [1,2,6,3,4,5,6], val = 6
输出：[1,2,3,4,5]

示例 2：
输入：head = [], val = 1
输出：[]

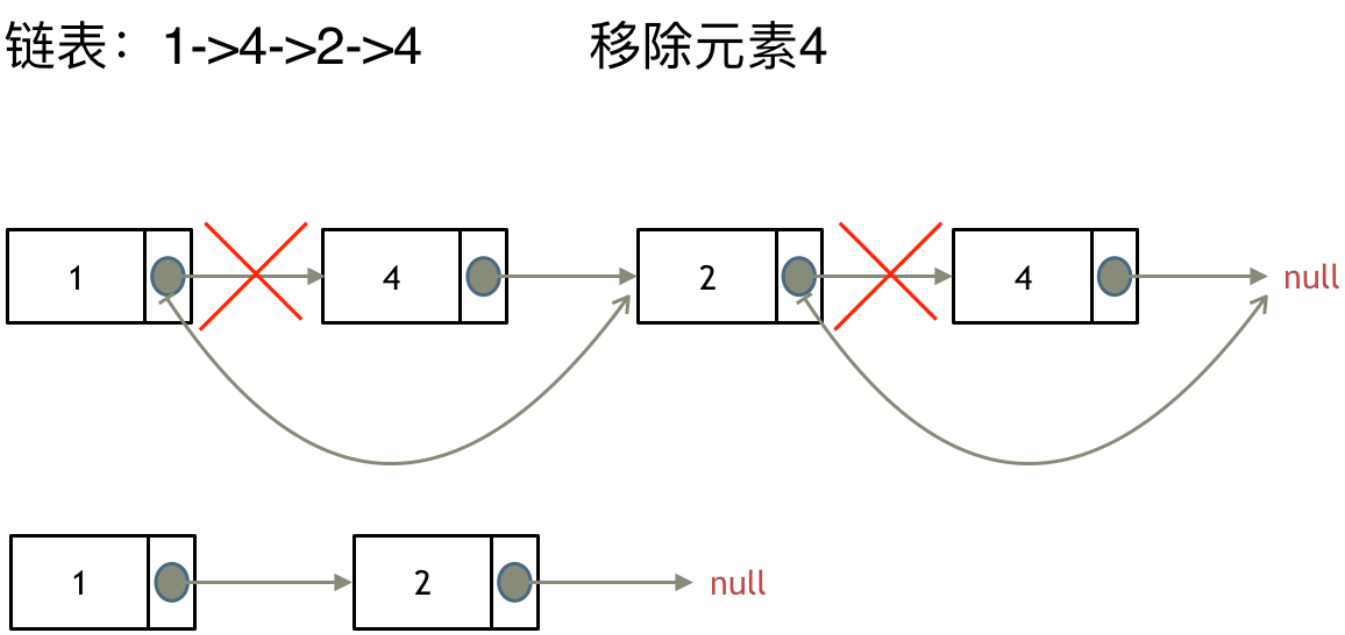
示例 3：
输入：head = [7,7,7,7], val = 7
输出：[]

算法公开课

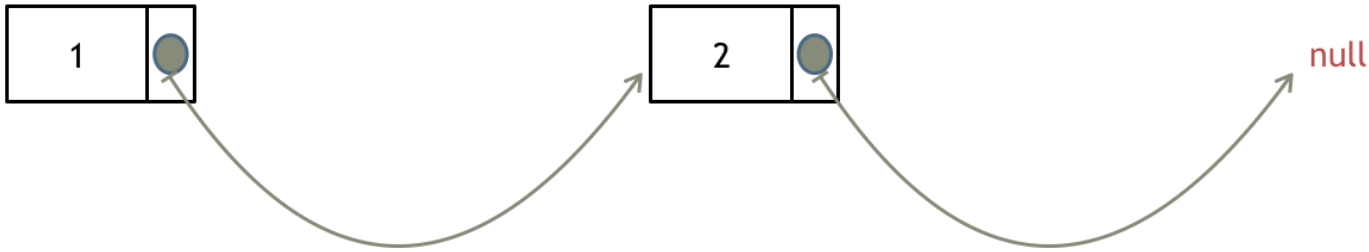
《代码随想录》算法视频公开课：链表基础操作| LeetCode：203.移除链表元素，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这里以链表 1 4 2 4 来举例，移除元素4。



如果使用C，C++编程语言的话，不要忘了还要从内存中删除这两个移除的节点， 清理节点内存之后如图：



当然如果使用java ，python的话就不用手动管理内存了。

还要说明一下，就算使用C++来做leetcode，如果移除一个节点之后，没有手动在内存中删除这个节点，leetcode依然也是可以通过的，只不过，内存使用的空间大一些而已，但建议依然要养成手动清理内存的习惯。

这种情况下的移除操作，就是让节点next指针直接指向下下一个节点就可以了，

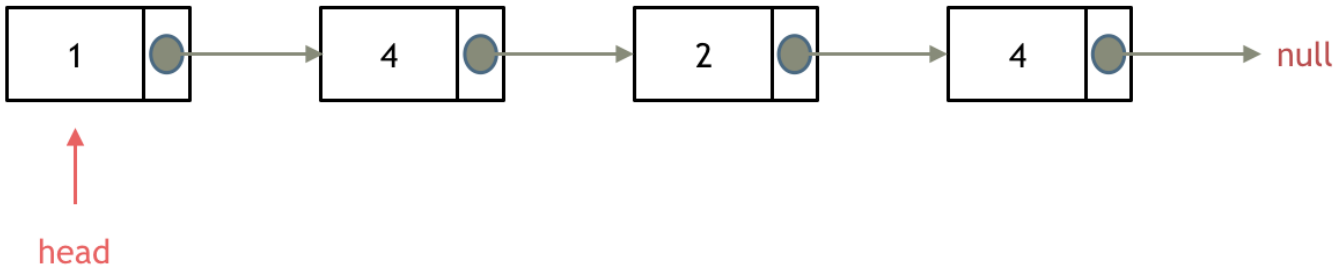
那么因为单链表的特殊性，只能指向下一个节点，刚刚删除的是链表的中第二个，和第四个节点，那么如果删除的是头结点又该怎么办呢?

这里就涉及如下链表操作的两种方式：

- **直接使用原来的链表来进行删除操作。**
- **设置一个虚拟头结点在进行删除操作。**

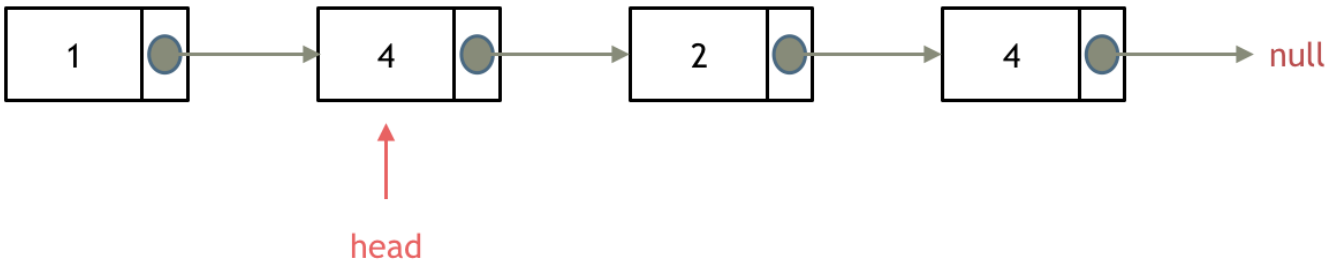
来看第一种操作：直接使用原来的链表来进行移除。

链表：1->4->2->4　　移除元素1

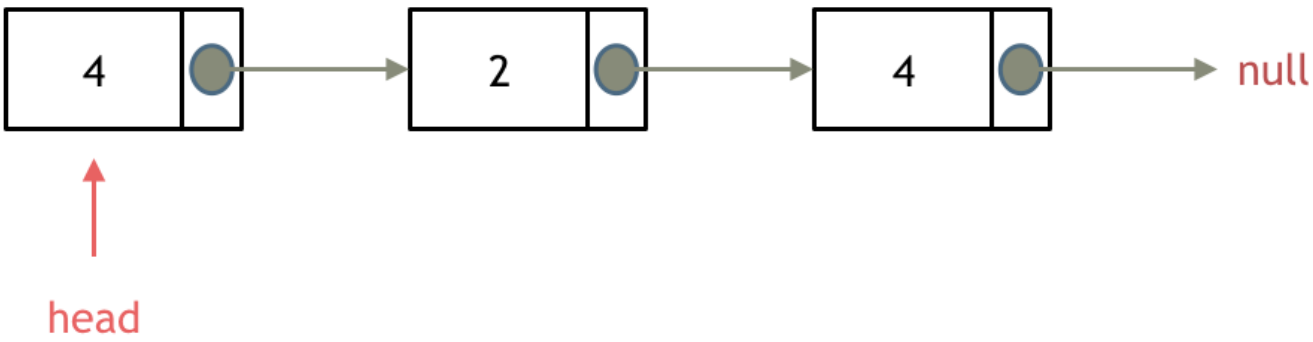


移除头结点和移除其他节点的操作是不一样的，因为链表的其他节点都是通过前一个节点来移除当前节点，而头结点没有前一个节点。

所以头结点如何移除呢，其实只要将头结点向后移动一位就可以，这样就从链表中移除了一个头结点。



依然别忘将原头结点从内存中删掉。



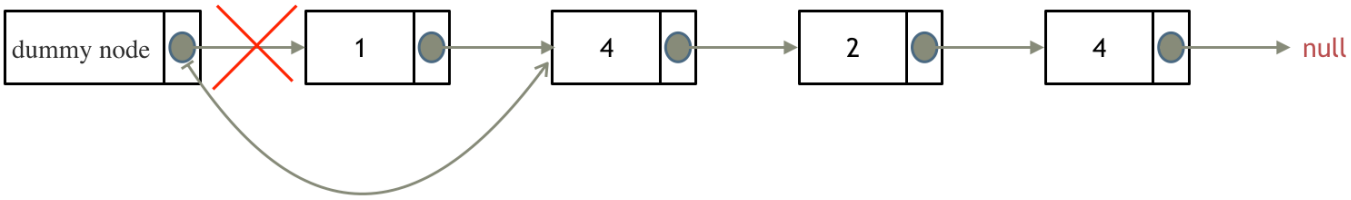
这样移除了一个头结点，是不是发现，在单链表中移除头结点 和 移除其他节点的操作方式是不一样，其实在写代码的时候也会发现，需要单独写一段逻辑来处理移除头结点的情况。

那么可不可以 以一种统一的逻辑来移除 链表的节点呢。

其实**可以设置一个虚拟头结点**，这样原链表的所有节点就都可以按照统一的方式进行移除了。

来看看如何设置一个虚拟头。依然还是在这个链表中，移除元素1。

链表：1->4->2->4　　移除元素1



这里来给链表添加一个虚拟头结点为新的头结点，此时要移除这个旧头结点元素1。

这样是不是就可以使用和移除链表其他节点的方式统一了呢?

来看一下，如何移除元素1 呢，还是熟悉的方式，然后从内存中删除元素1。

最后呢在题目中，return 头结点的时候，别忘了 `return dummyNode->next;`， 这才是新的头结点

直接使用原来的链表来进行移除节点操作：

```
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        // 删除头结点
        while (head != NULL && head->val == val) { // 注意这里不是if
            ListNode* tmp = head;
            head = head->next;
            delete tmp;
        }

        // 删除非头结点
        ListNode* cur = head;
        while (cur != NULL && cur->next!= NULL) {
            if (cur->next->val == val) {
                ListNode* tmp = cur->next;
                cur->next = cur->next->next;
                delete tmp;
            } else {
                cur = cur->next;
            }
        }
        return head;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

设置一个虚拟头结点在进行移除节点操作：

```
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        ListNode* dummyHead = new ListNode(0); // 设置一个虚拟头结点
        dummyHead->next = head; // 将虚拟头结点指向head，这样方面后面做删除操作
        ListNode* cur = dummyHead;
        while (cur->next != NULL) {
            if(cur->next->val == val) {
                ListNode* tmp = cur->next;
                cur->next = cur->next->next;
                delete tmp;
            } else {
                cur = cur->next;
            }
        }
        head = dummyHead->next;
        delete dummyHead;
        return head;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

> 听说这道题目把链表常见的五个操作都覆盖了?

3.设计链表

[力扣题目链接]

题意：

在链表类中实现这些功能：

- get(index)：获取链表中第 index 个节点的值。如果索引无效，则返回-1。
- addAtHead(val)：在链表的第一个元素之前添加一个值为 val 的节点。插入后，新节点将成为链表的第一个节点。
- addAtTail(val)：将值为 val 的节点追加到链表的最后一个元素。
- addAtIndex(index,val)：在链表中的第 index 个节点之前添加值为 val 的节点。如果 index 等于链表的长度，则该节点将附加到链表的末尾。如果 index 大于链表长度，则不会插入节点。如果index小于0，则在头部插入节点。
- deleteAtIndex(index)：如果索引 index 有效，则删除链表中的第 index 个节点。

示例：

```
MyLinkedList linkedList = new MyLinkedList();
linkedList.addAtHead(1);
linkedList.addAtTail(3);
linkedList.addAtIndex(1,2);   //链表变为1-> 2-> 3
linkedList.get(1);            //返回2
linkedList.deleteAtIndex(1);  //现在链表是1-> 3
linkedList.get(1);            //返回3
```

算法公开课

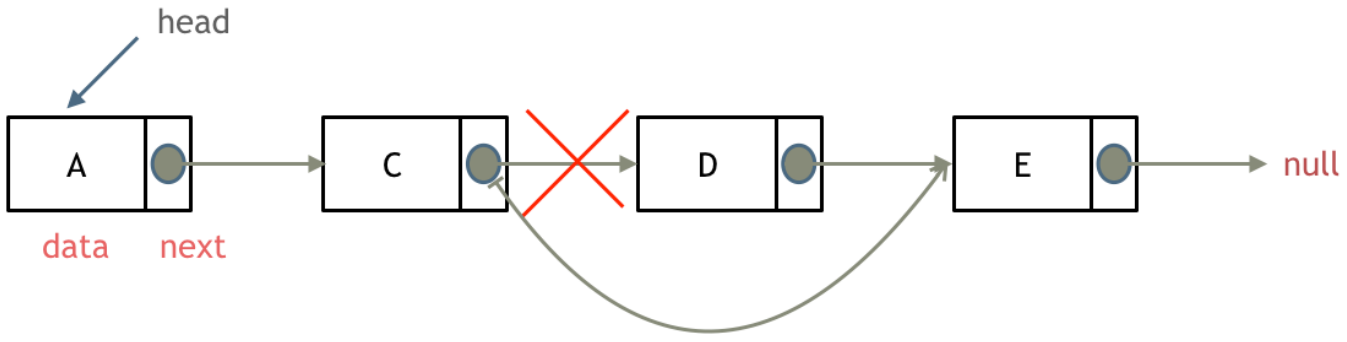
[《代码随想录》算法视频公开课](#)：[帮你把链表操作学个通透！LeetCode：707.设计链表](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

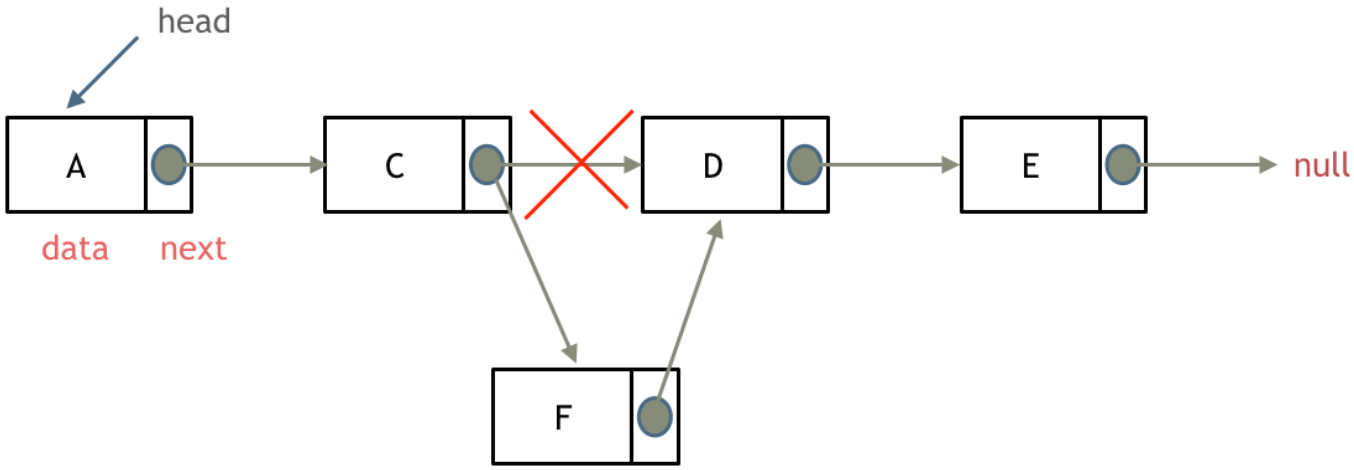
如果对链表的基础知识还不太懂，可以看这篇文章：[关于链表，你该了解这些！](#)

如果对链表的虚拟头结点不清楚，可以看这篇文章：[链表：听说用虚拟头节点会方便很多？](#)

删除链表节点：



添加链表节点：



这道题目设计链表的五个接口：

- 获取链表第index个节点的数值
- 在链表的最前面插入一个节点
- 在链表的最后面插入一个节点
- 在链表第index个节点前面插入一个节点
- 删除链表的第index个节点

可以说这五个接口，已经覆盖了链表的常见操作，是练习链表操作非常好的一道题目

链表操作的两种方式：

1. 直接使用原来的链表来进行操作。
2. 设置一个虚拟头结点在进行操作。

下面采用的设置一个虚拟头结点（这样更方便一些，大家看代码就会感受出来）。

```
class MyLinkedList {
public:
    // 定义链表节点结构体
    struct LinkedNode {
        int val;
        LinkedNode* next;
        LinkedNode(int val):val(val), next(nullptr){}
    };

    // 初始化链表
    MyLinkedList() {
        _dummyHead = new LinkedNode(0); // 这里定义的头结点 是一个虚拟头结点，而不是真正的链表头结点
        _size = 0;
    }

    // 获取到第index个节点数值，如果index是非法数值直接返回-1， 注意index是从0开始的，第0个节点就是头结点
```

```
    int get(int index) {
        if (index > (_size - 1) || index < 0) {
            return -1;
        }
        LinkedNode* cur = _dummyHead->next;
        while(index--){ // 如果--index 就会陷入死循环
            cur = cur->next;
        }
        return cur->val;
    }

    // 在链表最前面插入一个节点，插入完成后，新插入的节点为链表的新的头结点
    void addAtHead(int val) {
        LinkedNode* newNode = new LinkedNode(val);
        newNode->next = _dummyHead->next;
        _dummyHead->next = newNode;
        _size++;
    }

    // 在链表最后面添加一个节点
    void addAtTail(int val) {
        LinkedNode* newNode = new LinkedNode(val);
        LinkedNode* cur = _dummyHead;
        while(cur->next != nullptr){
            cur = cur->next;
        }
        cur->next = newNode;
        _size++;
    }

    // 在第index个节点之前插入一个新节点，例如index为0，那么新插入的节点为链表的新头节点。
    // 如果index 等于链表的长度，则说明是新插入的节点为链表的尾结点
    // 如果index大于链表的长度，则返回空
    // 如果index小于0，则在头部插入节点
    void addAtIndex(int index, int val) {

        if(index > _size) return;
        if(index < 0) index = 0;
        LinkedNode* newNode = new LinkedNode(val);
        LinkedNode* cur = _dummyHead;
        while(index--) {
            cur = cur->next;
        }
        newNode->next = cur->next;
        cur->next = newNode;
        _size++;
    }

    // 删除第index个节点，如果index 大于等于链表的长度，直接return，注意index是从0开始的
```

```
    void deleteAtIndex(int index) {
        if (index >= _size || index < 0) {
            return;
        }
        LinkedNode* cur = _dummyHead;
        while(index--) {
            cur = cur ->next;
        }
        LinkedNode* tmp = cur->next;
        cur->next = cur->next->next;
        delete tmp;
        //delete命令指示释放了tmp指针原本所指的那部分内存，
        //被delete后的指针tmp的值（地址）并非就是NULL，而是随机值。也就是被delete后，
        //如果不再加上一句tmp=nullptr,tmp会成为乱指的野指针
        //如果之后的程序不小心使用了tmp，会指向难以预想的内存空间
        tmp=nullptr;
        _size--;
    }

    // 打印链表
    void printLinkedList() {
        LinkedNode* cur = _dummyHead;
        while (cur->next != nullptr) {
            cout << cur->next->val << " ";
            cur = cur->next;
        }
        cout << endl;
    }
private:
    int _size;
    LinkedNode* _dummyHead;

};
```

- 时间复杂度: 涉及 `index` 的相关操作为 O(index), 其余为 O(1)
- 空间复杂度: O(n)

> 反转链表的写法很简单，一些同学甚至可以背下来但过一阵就忘了该咋写，主要是因为没有理解真正的反转过程。

4.反转链表

[力扣题目链接]

题意：反转一个单链表。

示例:
输入: 1->2->3->4->5->NULL
输出: 5->4->3->2->1->NULL

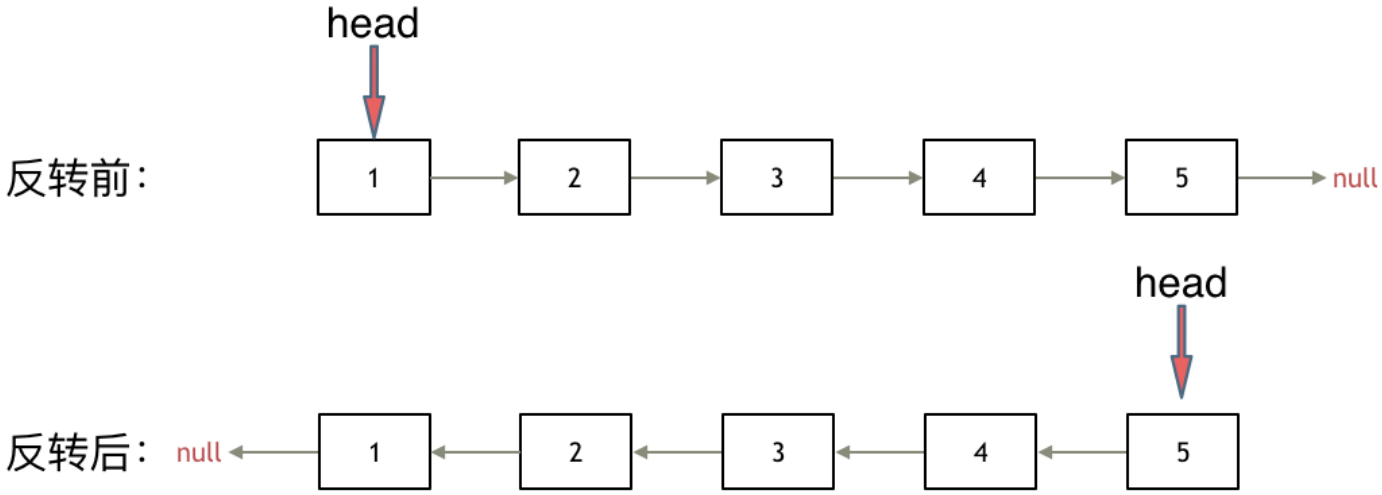
算法公开课

[《代码随想录》算法视频公开课](#)：[帮你拿下反转链表 | LeetCode：206.反转链表](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

如果再定义一个新的链表，实现链表元素的反转，其实这是对内存空间的浪费。

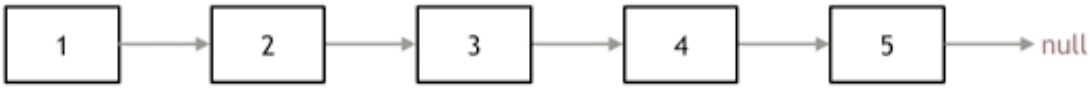
其实只需要改变链表的next指针的指向，直接将链表反转 ，而不用重新定义一个新的链表，如图所示:



之前链表的头节点是元素1， 反转之后头结点就是元素5 ，这里并没有添加或者删除节点，仅仅是改变next指针的方向。

那么接下来看一看是如何反转的呢?

我们拿有示例中的链表来举例，如动画所示：（纠正：动画应该是先移动pre，在移动cur）



首先定义一个cur指针，指向头结点，再定义一个pre指针，初始化为null。

然后就要开始反转了，首先要把 cur->next 节点用tmp指针保存一下，也就是保存一下这个节点。

为什么要保存一下这个节点呢，因为接下来要改变 cur->next 的指向了，将cur->next 指向pre ，此时已经反转了第一个节点了。

接下来，就是循环走如下代码逻辑了，继续移动pre和cur指针。

最后，cur 指针已经指向了null，循环结束，链表也反转完毕了。 此时我们return pre指针就可以了，pre指针就指向了新的头结点。

双指针法

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* temp; // 保存cur的下一个节点
        ListNode* cur = head;
        ListNode* pre = NULL;
        while(cur) {
            temp = cur->next;  // 保存一下 cur的下一个节点，因为接下来要改变cur->next
            cur->next = pre; // 翻转操作
            // 更新pre 和 cur指针
            pre = cur;
            cur = temp;
        }
```

```
        return pre;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

递归法

递归法相对抽象一些，但是其实和双指针法是一样的逻辑，同样是当cur为空的时候循环结束，不断将cur指向pre的过程。

关键是初始化的地方，可能有的同学会不理解， 可以看到双指针法中初始化 cur = head，pre = NULL，在递归法中可以从如下代码看出初始化的逻辑也是一样的，只不过写法变了。

具体可以看代码（已经详细注释），**双指针法写出来之后，理解如下递归写法就不难了，代码逻辑都是一样的。**

```
class Solution {
public:
    ListNode* reverse(ListNode* pre,ListNode* cur){
        if(cur == NULL) return pre;
        ListNode* temp = cur->next;
        cur->next = pre;
        // 可以和双指针法的代码进行对比，如下递归的写法，其实就是做了这两步
        // pre = cur;
        // cur = temp;
        return reverse(cur,temp);
    }
    ListNode* reverseList(ListNode* head) {
        // 和双指针法初始化是一样的逻辑
        // ListNode* cur = head;
        // ListNode* pre = NULL;
        return reverse(NULL, head);
    }

};
```

- 时间复杂度: O(n), 要递归处理链表的每个节点
- 空间复杂度: O(n), 递归调用了 n 层栈空间

我们可以发现，上面的递归写法和双指针法实质上都是从前往后翻转指针指向，其实还有另外一种与双指针法不同思路的递归写法：从后往前翻转指针指向。

具体代码如下（带详细注释）：

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        // 边缘条件判断
```

```
        if(head == NULL) return NULL;
        if (head->next == NULL) return head;

        // 递归调用，翻转第二个节点开始往后的链表
        ListNode *last = reverseList(head->next);
        // 翻转头节点与第二个节点的指向
        head->next->next = head;
        // 此时的 head 节点为尾节点，next 需要指向 NULL
        head->next = NULL;
        return last;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

其他解法

使用虚拟头结点解决链表反转

> 使用虚拟头结点，通过头插法实现链表的反转（不需要栈）

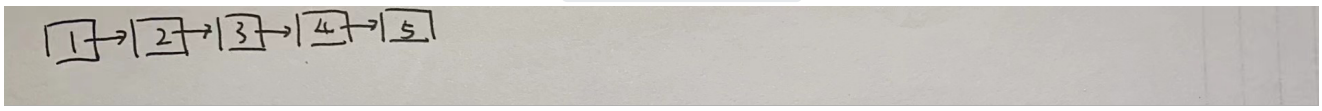
```
// 迭代方法：增加虚头结点，使用头插法实现链表翻转
public static ListNode reverseList1(ListNode head) {
    // 创建虚头结点
    ListNode dumpyHead = new ListNode(-1);
    dumpyHead.next = null;
    // 遍历所有节点
    ListNode cur = head;
    while(cur != null){
        ListNode temp = cur.next;
        // 头插法
        cur.next = dumpyHead.next;
        dumpyHead.next = cur;
        cur = temp;
    }
    return dumpyHead.next;
}
```

使用栈解决反转链表的问题

- 首先将所有的结点入栈
- 然后创建一个虚拟虚拟头结点，让cur指向虚拟头结点。然后开始循环出栈，每出来一个元素，就把它加入到以虚拟头结点为头结点的链表当中，最后返回即可。

```
public ListNode reverseList(ListNode head) {
```

```
    // 如果链表为空，则返回空
    if (head == null) return null;
    // 如果链表中只有只有一个元素，则直接返回
    if (head.next == null) return head;
    // 创建栈 每一个结点都入栈
    Stack<ListNode> stack = new Stack<>();
    ListNode cur = head;
    while (cur != null) {
        stack.push(cur);
        cur = cur.next;
    }
    // 创建一个虚拟头结点
    ListNode pHead = new ListNode(0);
    cur = pHead;
    while (!stack.isEmpty()) {
        ListNode node = stack.pop();
        cur.next = node;
        cur = cur.next;
    }
    // 最后一个元素的next要赋值为空
    cur.next = null;
    return pHead.next;
}
```

> 采用这种方法需要注意一点。就是当整个出栈循环结束以后，cur正好指向原来链表的第一个结点，而此时结点1中的next指向的是结点2，因此最后还需要`cur.next = null`
>
> 
>

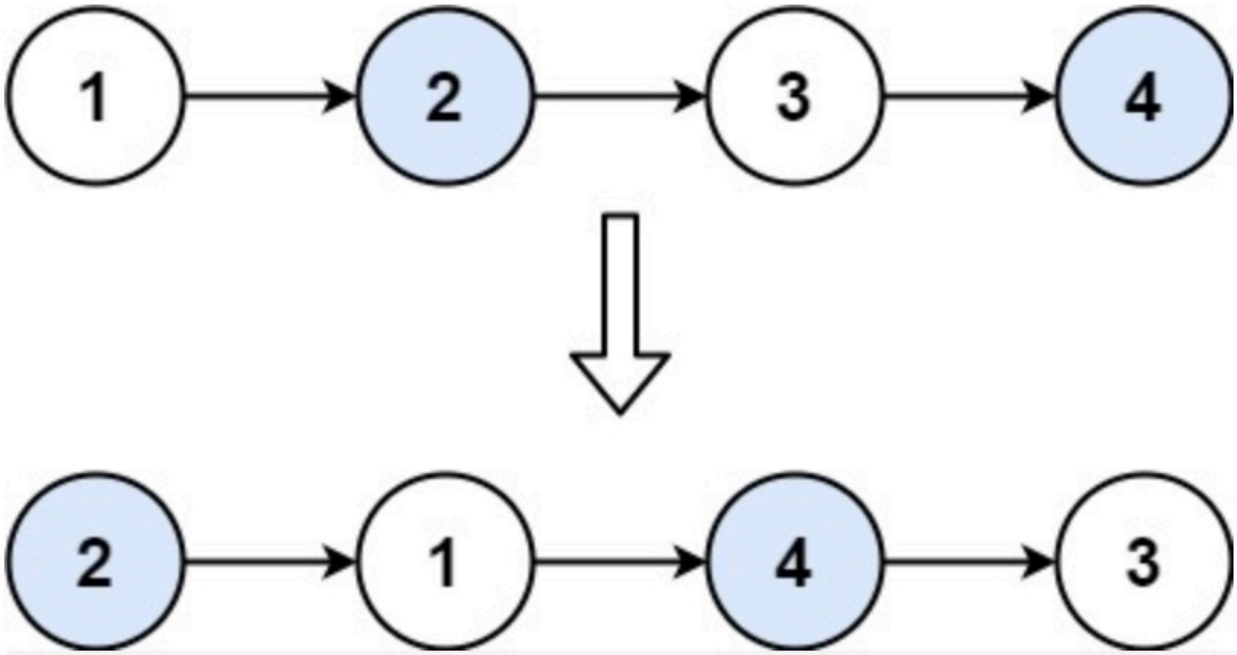
5. 两两交换链表中的节点

[力扣题目链接]

给定一个链表，两两交换其中相邻的节点，并返回交换后的链表。

你不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

示例 1：




```
输入：head = [1,2,3,4]
输出：[2,1,4,3]
```

示例 2：

```
输入：head = []
输出：[]
```

示例 3：

```
输入：head = [1]
输出：[1]
```

算法公开课

[《代码随想录》算法视频公开课](#)：[帮你把链表细节学清楚！ | LeetCode：24. 两两交换链表中的节点](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

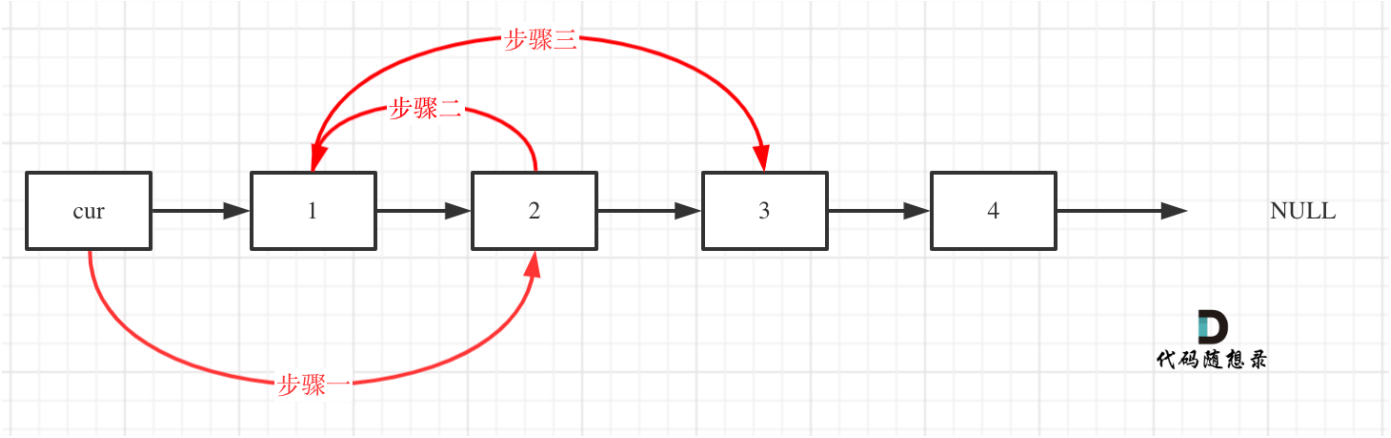
这道题目正常模拟就可以了。

建议使用虚拟头结点，这样会方便很多，要不然每次针对头结点（没有前一个指针指向头结点），还要单独处理。

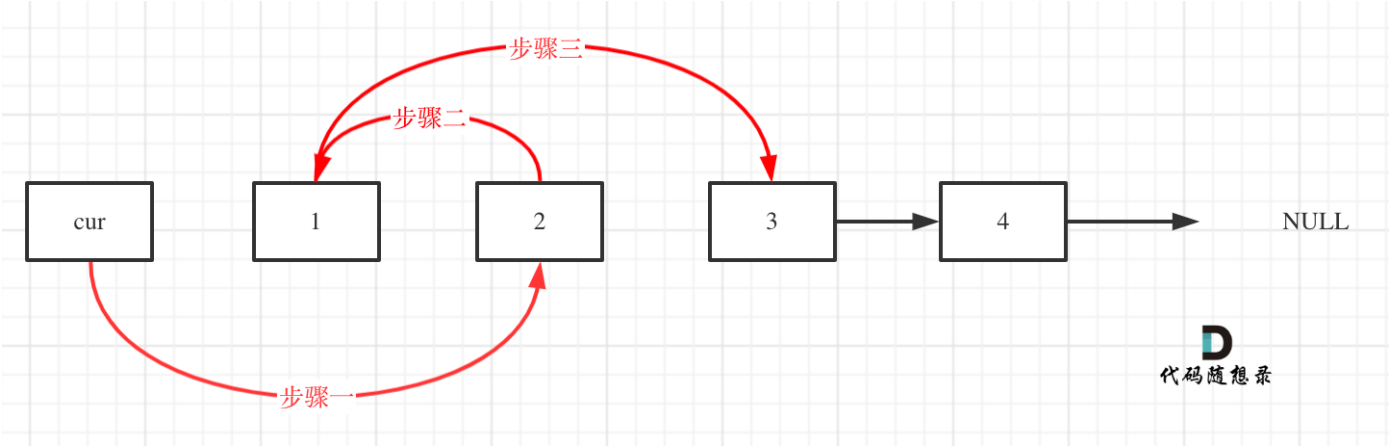
对虚拟头结点的操作，还不熟悉的话，可以看这篇链表：听说用虚拟头节点会方便很多？。

接下来就是交换相邻两个元素了，**此时一定要画图，不画图，操作多个指针很容易乱，而且要操作的先后顺序**

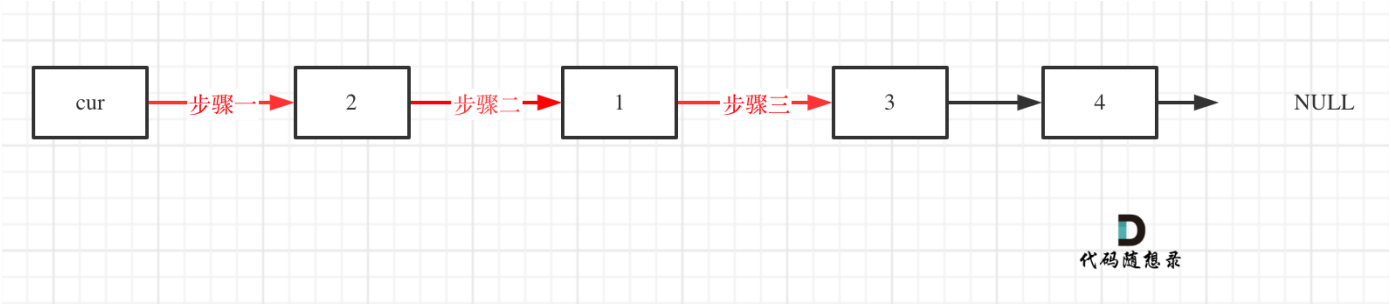
初始时，cur指向虚拟头结点，然后进行如下三步：



操作之后，链表如下：



看这个可能就更直观一些了：



对应的C++代码实现如下：（注释中详细和如上图中的三步做对应）

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummyHead = new ListNode(0); // 设置一个虚拟头结点
        dummyHead->next = head; // 将虚拟头结点指向head，这样方面后面做删除操作
        ListNode* cur = dummyHead;
        while(cur->next != nullptr && cur->next->next != nullptr) {
            ListNode* tmp = cur->next; // 记录临时节点
```

```
            ListNode* tmp1 = cur->next->next->next; // 记录临时节点

            cur->next = cur->next->next;    // 步骤一
            cur->next->next = tmp;          // 步骤二
            cur->next->next->next = tmp1;   // 步骤三

            cur = cur->next->next; // cur移动两位，准备下一轮交换
        }
        return dummyHead->next;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

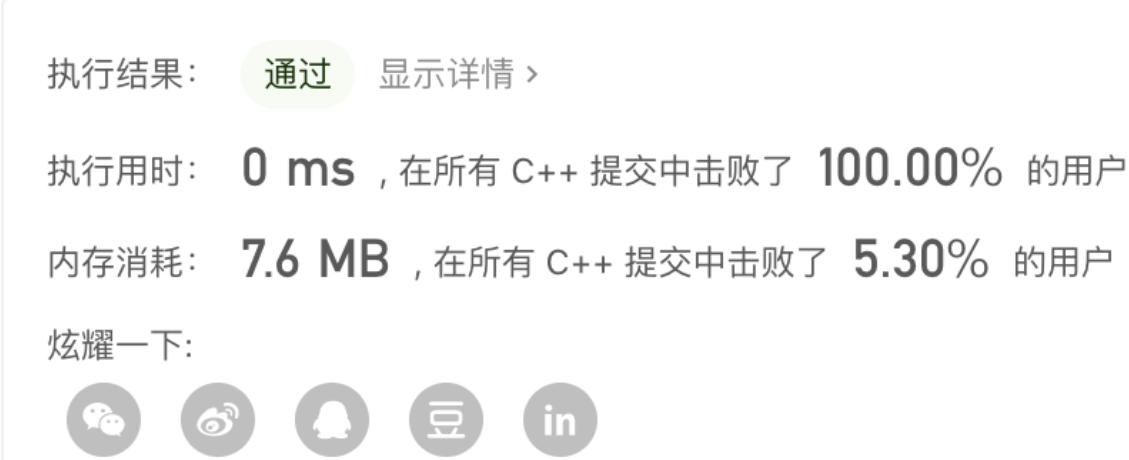
拓展

这里还是说一下，大家不必太在意力扣上执行用时，打败多少多少用户，这个统计不准确的。

做题的时候自己能分析出来时间复杂度就可以了，至于力扣上执行用时，大概看一下就行。

上面的代码我第一次提交执行用时8ms，打败6.5%的用户，差点吓到我了。

心想应该没有更好的方法了吧，也就$O(n)$的时间复杂度，重复提交几次，这样了：



力扣上的统计如果两份代码是 100ms 和 300ms的耗时，其实是需要注意的。

如果一个是 4ms 一个是 12ms，看上去好像是一个打败了80%，一个打败了20%，其实是没有差别的。 只不过是力扣上统计的误差而已。

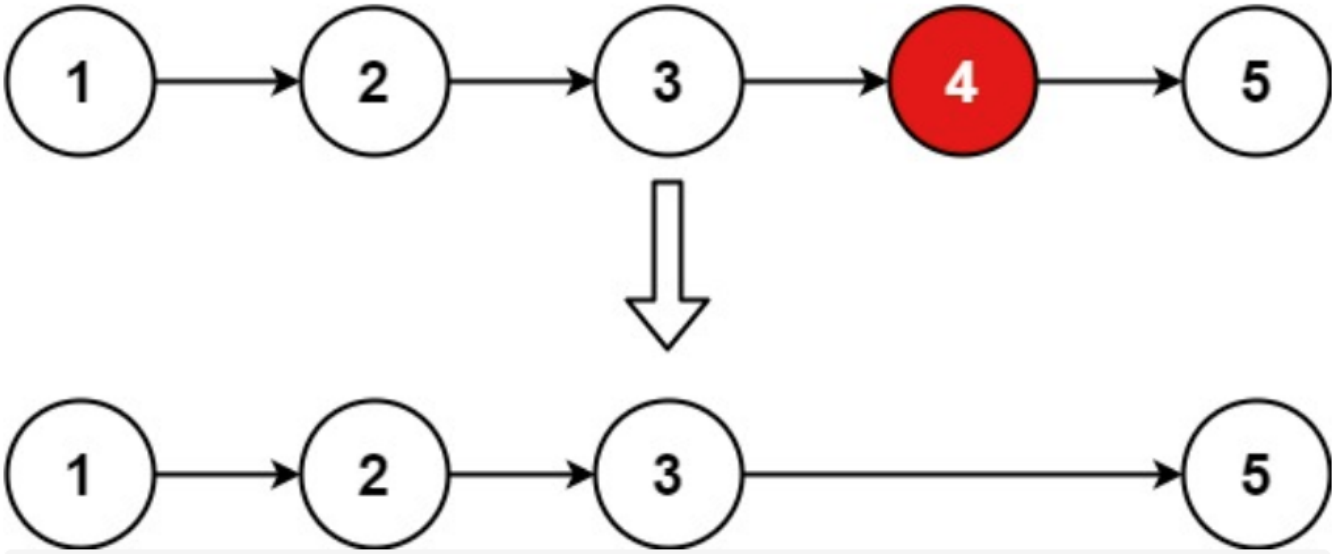
6.删除链表的倒数第N个节点

[力扣题目链接]

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

进阶：你能尝试使用一趟扫描实现吗?

示例 1：



输入：head = [1,2,3,4,5], n = 2
输出：[1,2,3,5]
示例 2：

输入：head = [1], n = 1
输出：[]
示例 3：

输入：head = [1,2], n = 1
输出：[1]

算法公开课

《代码随想录》算法视频公开课：：链表遍历学清楚！ | LeetCode：19.删除链表倒数第N个节点，相信结合视频再看本篇题解，更有助于大家对链表的理解。

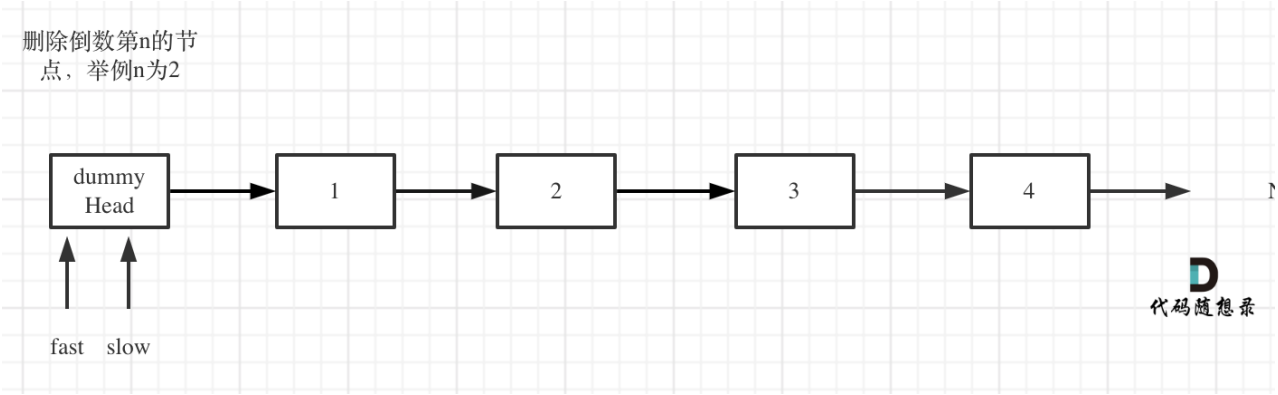
思路

双指针的经典应用，如果要删除倒数第n个节点，让fast移动n步，然后让fast和slow同时移动，直到fast指向链表末尾。删掉slow所指向的节点就可以了。

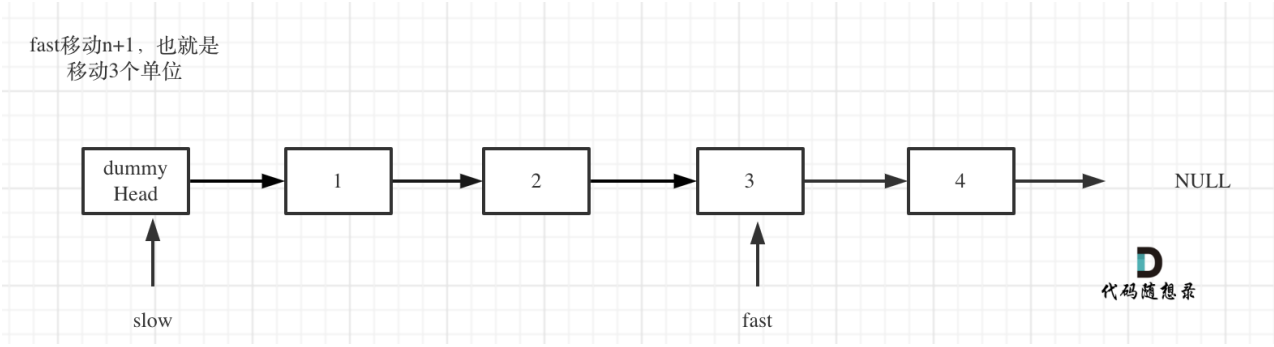
思路是这样的，但要注意一些细节。

分为如下几步：

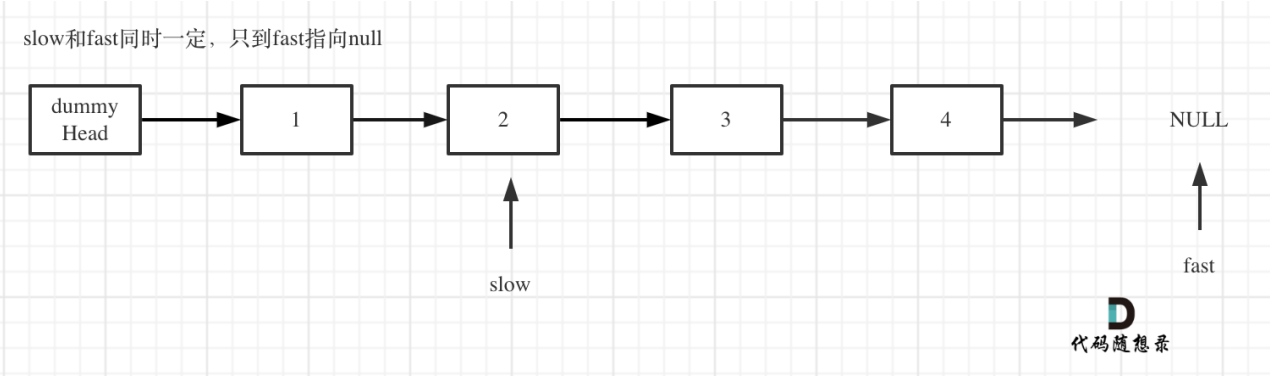
- 首先这里我推荐大家使用虚拟头结点，这样方便处理删除实际头结点的逻辑，如果虚拟头结点不清楚，可以看这篇： 链表：听说用虚拟头节点会方便很多？
- 定义fast指针和slow指针，初始值为虚拟头结点，如图：



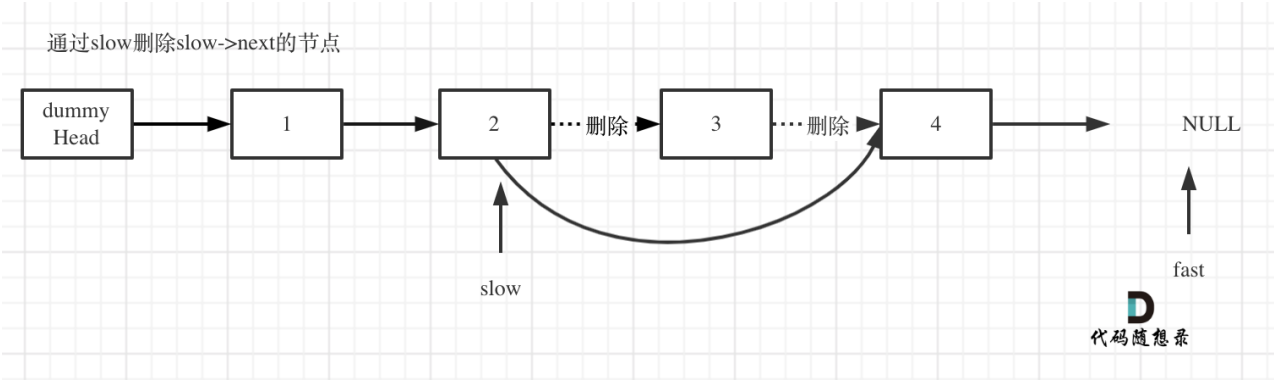
- fast首先走n + 1步 ，为什么是n+1呢，因为只有这样同时移动的时候slow才能指向删除节点的上一个节点（方便做删除操作），如图：



- fast和slow同时移动，直到fast指向末尾，如题：



- 删除slow指向的下一个节点，如图：



此时不难写出如下C++代码：

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* dummyHead = new ListNode(0);
```

```
        dummyHead->next = head;
        ListNode* slow = dummyHead;
        ListNode* fast = dummyHead;
        while(n-- && fast != NULL) {
            fast = fast->next;
        }
        fast = fast->next; // fast再提前走一步，因为需要让slow指向删除节点的上一个节点
        while (fast != NULL) {
            fast = fast->next;
            slow = slow->next;
        }
        slow->next = slow->next->next;

        // ListNode *tmp = slow->next;  C++释放内存的逻辑
        // slow->next = tmp->next;
        // delete nth;

        return dummyHead->next;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

---

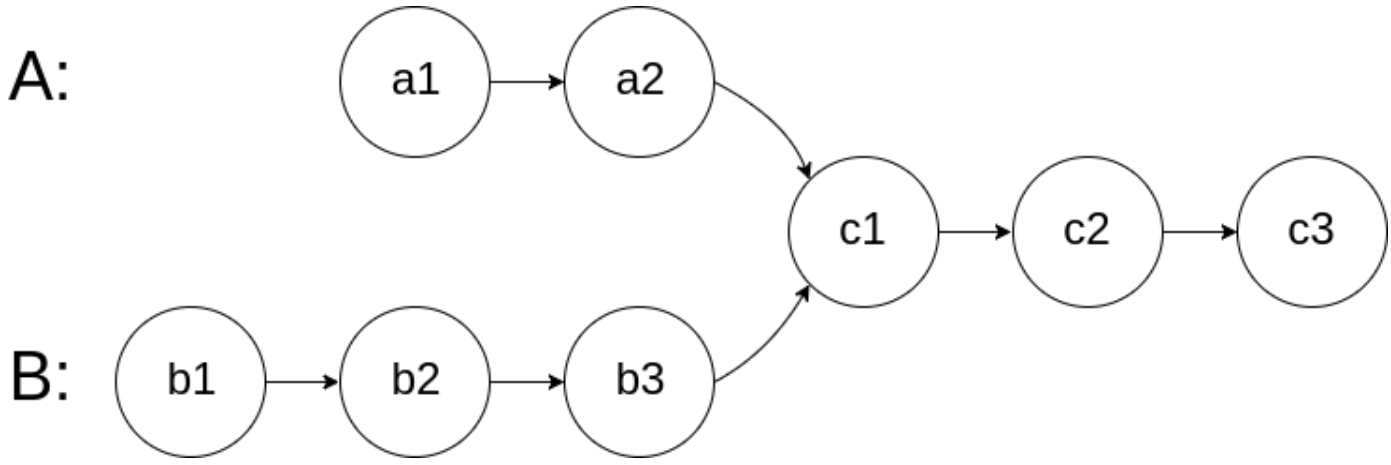
7. 面试题 02.07. 链表相交

同：160.链表相交

[力扣题目链接]

给你两个单链表的头节点 headA 和 headB ， 请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 null 。

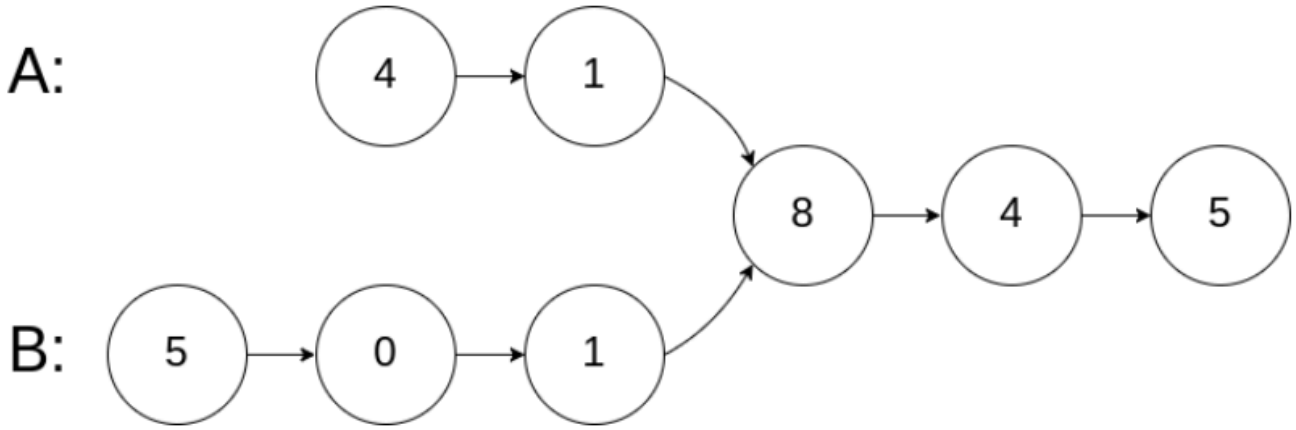
图示两个链表在节点 c1 开始相交：



题目数据 保证 整个链式结构中不存在环。

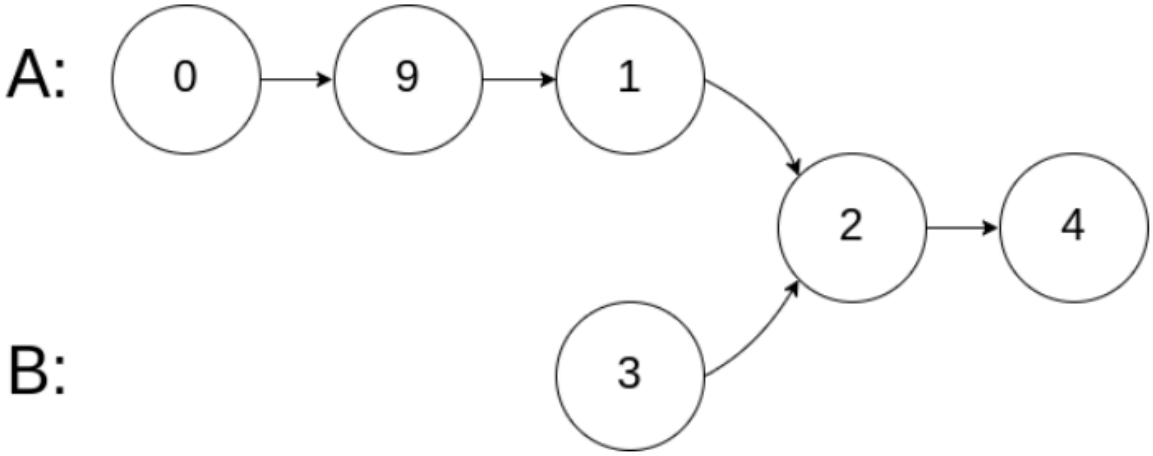
注意，函数返回结果后，链表必须 保持其原始结构 。

示例 1：



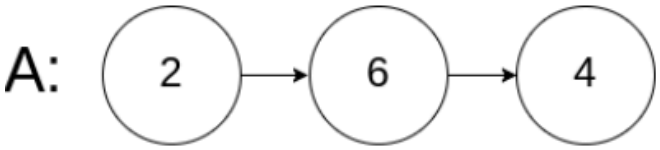
输入： intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3
输出： Intersected at '8'
解释： 相交节点的值为 8 （注意，如果两个链表相交则不能为 0）。
从各自的表头开始算起，链表 A 为 [4,1,8,4,5]，链表 B 为 [5,0,1,8,4,5]。
在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。

示例 2：

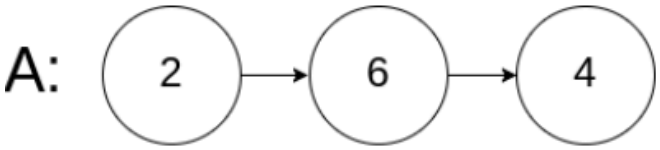


输入： intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
输出： Intersected at '2'
解释： 相交节点的值为 2 （注意，如果两个链表相交则不能为 0）。
从各自的表头开始算起，链表 A 为 [0,9,1,2,4]，链表 B 为 [3,2,4]。
在 A 中，相交节点前有 3 个节点；在 B 中，相交节点前有 1 个节点。

示例 3：



输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
输出：null
解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。
由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA 和 skipB 可以是任意值。
这两个链表不相交，因此返回 null 。



输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
输出：null
解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。
由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA

和 `skipB` 可以是任意值。

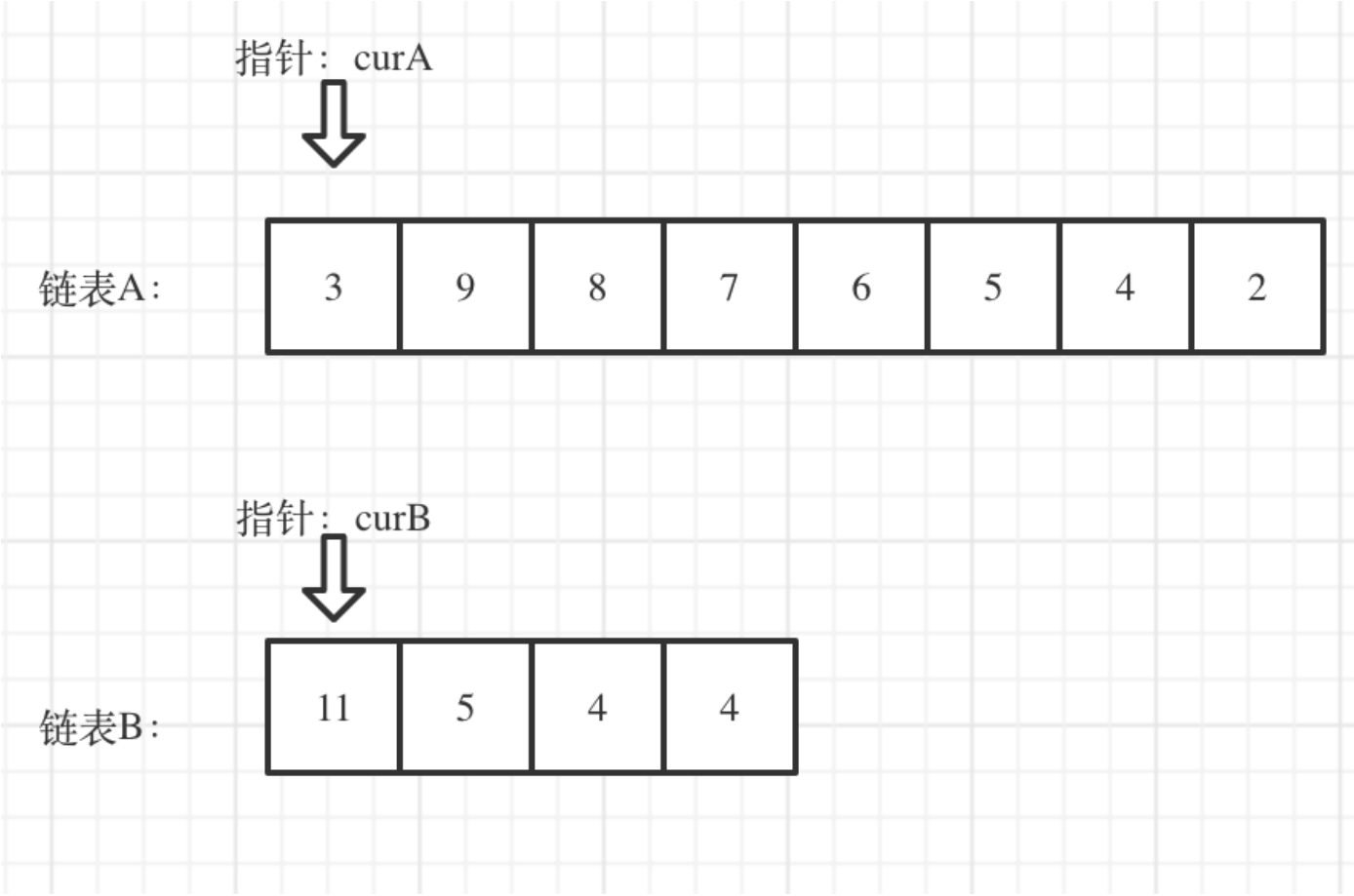
这两个链表不相交，因此返回 `null` 。

思路

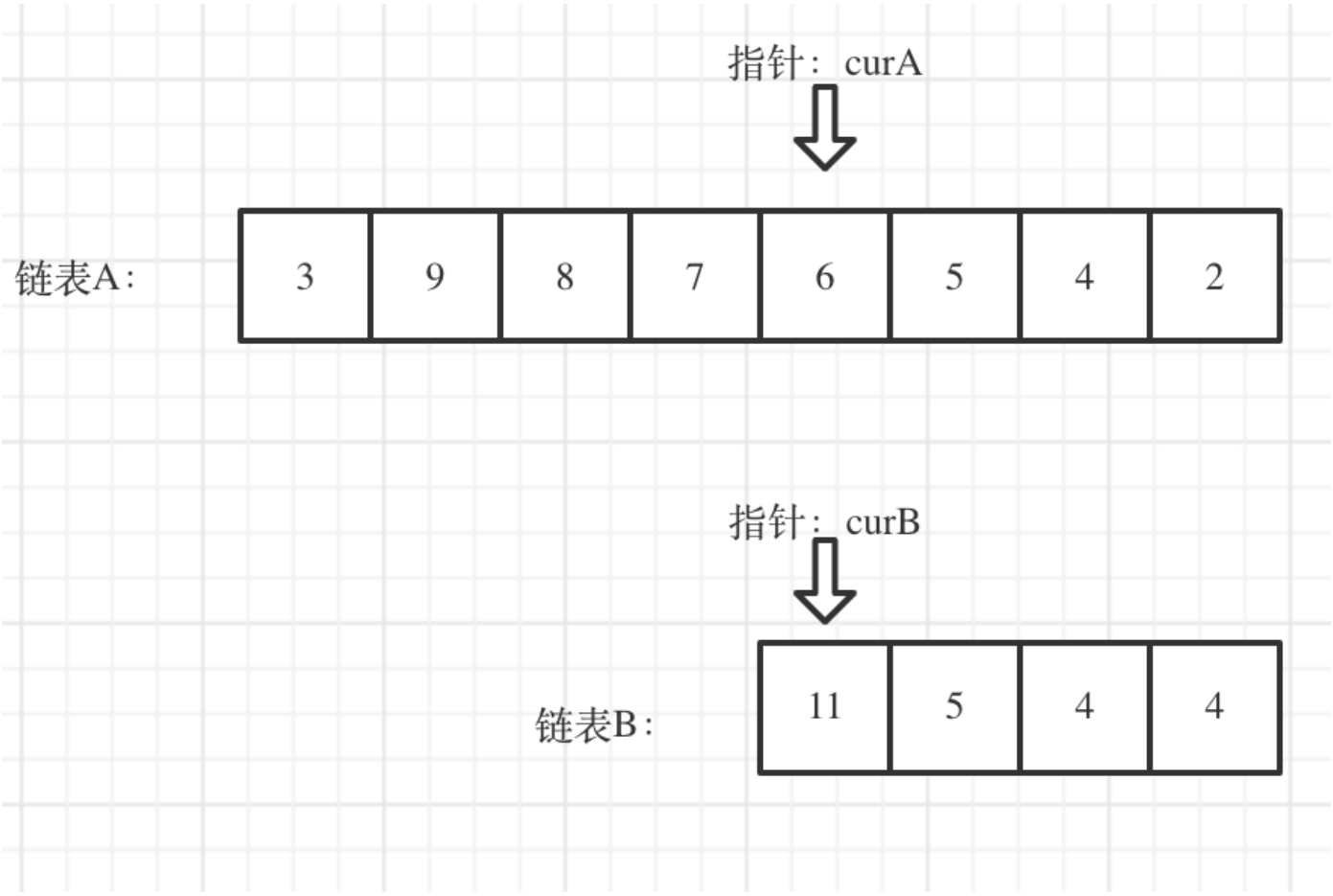
简单来说，就是求两个链表交点节点的**指针**。 这里同学们要注意，交点不是数值相等，而是指针相等。

为了方便举例，假设节点元素数值相等，则节点指针相等。

看如下两个链表，目前curA指向链表A的头结点，curB指向链表B的头结点：



我们求出两个链表的长度，并求出两个链表长度的差值，然后让curA移动到，和curB 末尾对齐的位置，如图：



此时我们就可以比较curA和curB是否相同，如果不相同，同时向后移动curA和curB，如果遇到curA == curB，则找到交点。

否则循环退出返回空指针。

C++代码如下：

```
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        ListNode* curA = headA;
        ListNode* curB = headB;
        int lenA = 0, lenB = 0;
        while (curA != NULL) { // 求链表A的长度
            lenA++;
            curA = curA->next;
        }
        while (curB != NULL) { // 求链表B的长度
            lenB++;
            curB = curB->next;
        }
        curA = headA;
        curB = headB;
        // 让curA为最长链表的头，lenA为其长度
        if (lenB > lenA) {
            swap (lenA, lenB);
```

```
            swap (curA, curB);
        }
        // 求长度差
        int gap = lenA - lenB;
        // 让curA和curB在同一起点上（末尾位置对齐）
        while (gap--) {
            curA = curA->next;
        }
        // 遍历curA 和 curB，遇到相同则直接返回
        while (curA != NULL) {
            if (curA == curB) {
                return curA;
            }
            curA = curA->next;
            curB = curB->next;
        }
        return NULL;
    }
};
```

- 时间复杂度：O(n + m)
- 空间复杂度：O(1)

> 找到有没有环已经很不容易了，还要让我找到环的入口?

8.环形链表II

[力扣题目链接]

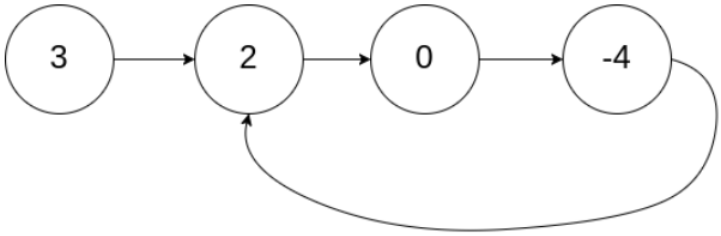
题意：
给定一个链表，返回链表开始入环的第一个节点。 如果链表无环，则返回 null。

为了表示给定链表中的环，使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。 如果 pos 是 -1，则在该链表中没有环。

说明：不允许修改给定的链表。

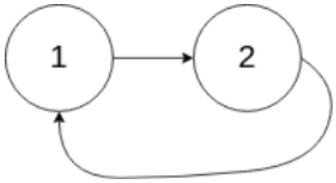
示例 1：

```
输入：head = [3,2,0,-4], pos = 1
输出：tail connects to node index 1
解释：链表中有一个环，其尾部连接到第二个节点。
```



示例 2：

```
输入：head = [1,2], pos = 0
输出：tail connects to node index 0
解释：链表中有一个环，其尾部连接到第一个节点。
```



算法公开课

[《代码随想录》算法视频公开课](#)：[把环形链表讲清楚！ | LeetCode:142.环形链表II](#)，**相信结合视频在看本篇题解，更有助于大家对链表的理解。**

思路

这道题目，不仅考察对链表的操作，而且还需要一些数学运算。

主要考察两知识点：

- 判断链表是否环
- 如果有环，如何找到这个环的入口

判断链表是否有环

可以使用快慢指针法，分别定义 fast 和 slow 指针，从头结点出发，fast指针每次移动两个节点，slow指针每次移动一个节点，如果 fast 和 slow指针在途中相遇 ，说明这个链表有环。

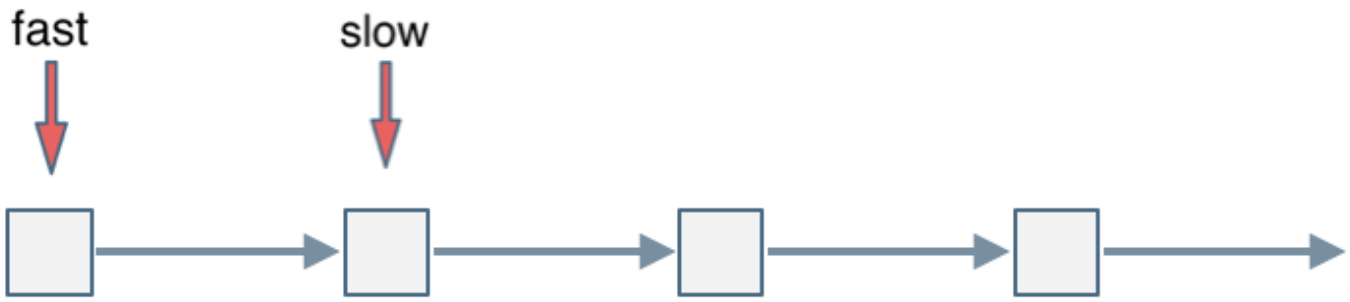
为什么fast 走两个节点，slow走一个节点，有环的话，一定会在环内相遇呢，而不是永远的错开呢

首先第一点：**fast指针一定先进入环中，如果fast指针和slow指针相遇的话，一定是在环中相遇，这是毋庸置疑的。**

那么来看一下，**为什么fast指针和slow指针一定会相遇呢?**

可以画一个环，然后让 fast指针在任意一个节点开始追赶slow指针。

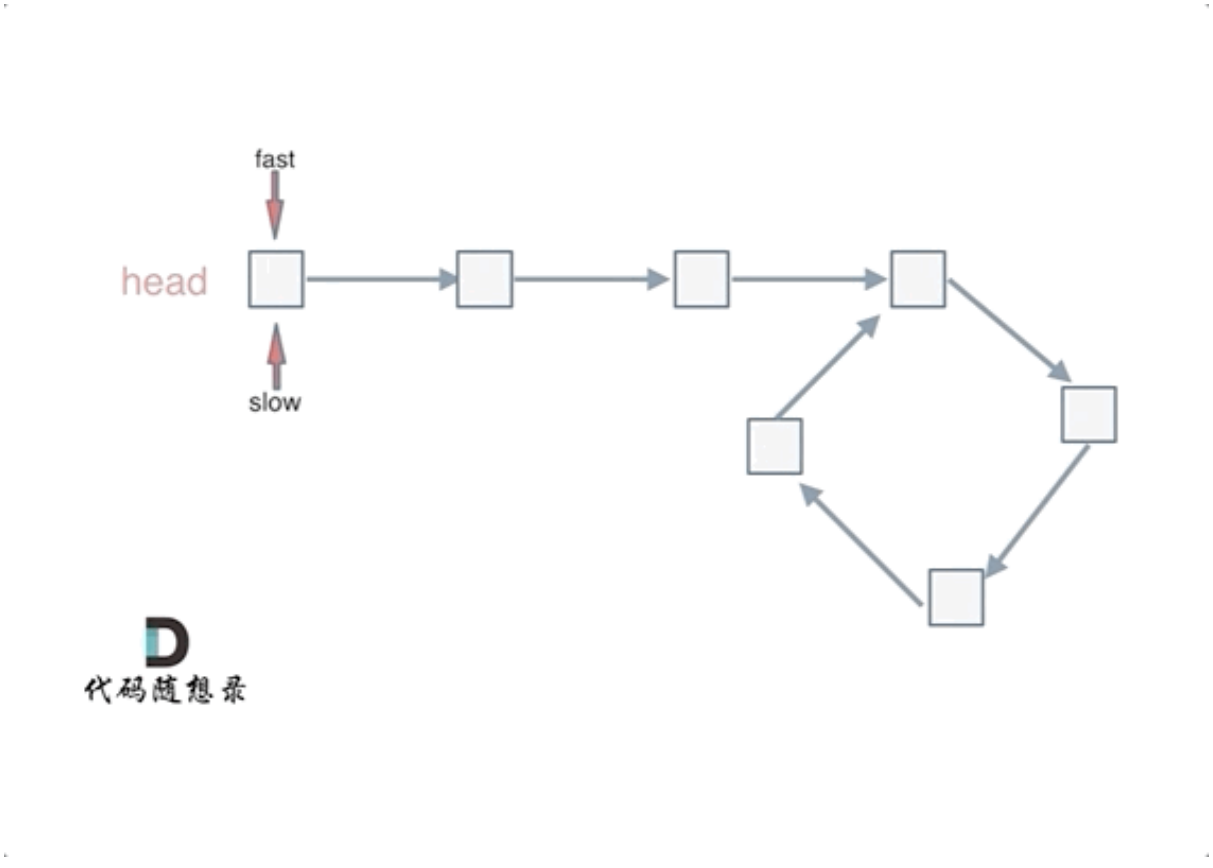
会发现最终都是这种情况， 如下图：



fast和slow各自再走一步， fast和slow就相遇了

这是因为fast是走两步，slow是走一步，**其实相对于slow来说，fast是一个节点一个节点的靠近slow的**，所以fast一定可以和slow重合。

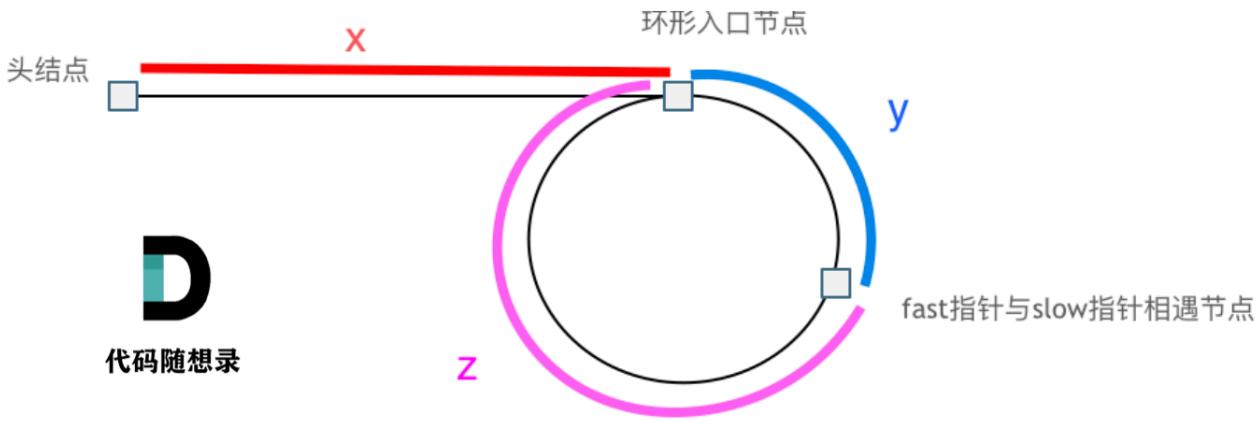
动画如下：



如果有环，如何找到这个环的入口

此时已经可以判断链表是否有环了，那么接下来要找这个环的入口了。

假设从头结点到环形入口节点 的节点数为x。
环形入口节点到 fast指针与slow指针相遇节点 节点数为y。
从相遇节点 再到环形入口节点节点数为 z。 如图所示：



那么相遇时：
slow指针走过的节点数为: `x + y`，
fast指针走过的节点数： `x + y + n (y + z)`，n为fast指针在环内走了n圈才遇到slow指针， （y+z）为 一圈内节点的个数A。

因为fast指针是一步走两个节点，slow指针一步走一个节点， 所以 fast指针走过的节点数 = slow指针走过的节点数 * 2：

`(x + y) * 2 = x + y + n (y + z)`

两边消掉一个（x+y）：`x + y  = n (y + z)`

因为要找环形的入口，那么要求的是x，因为x表示 头结点到 环形入口节点的的距离。

所以要求x ，将x单独放在左面：`x = n (y + z) - y`，

再从n(y+z)中提出一个 （y+z）来，整理公式之后为如下公式：`x = (n - 1) (y + z) + z` 注意这里n一定是大于等于1的，因为 fast指针至少要多走一圈才能相遇slow指针。

这个公式说明什么呢?

先拿n为1的情况来举例，意味着fast指针在环形里转了一圈之后，就遇到了 slow指针了。

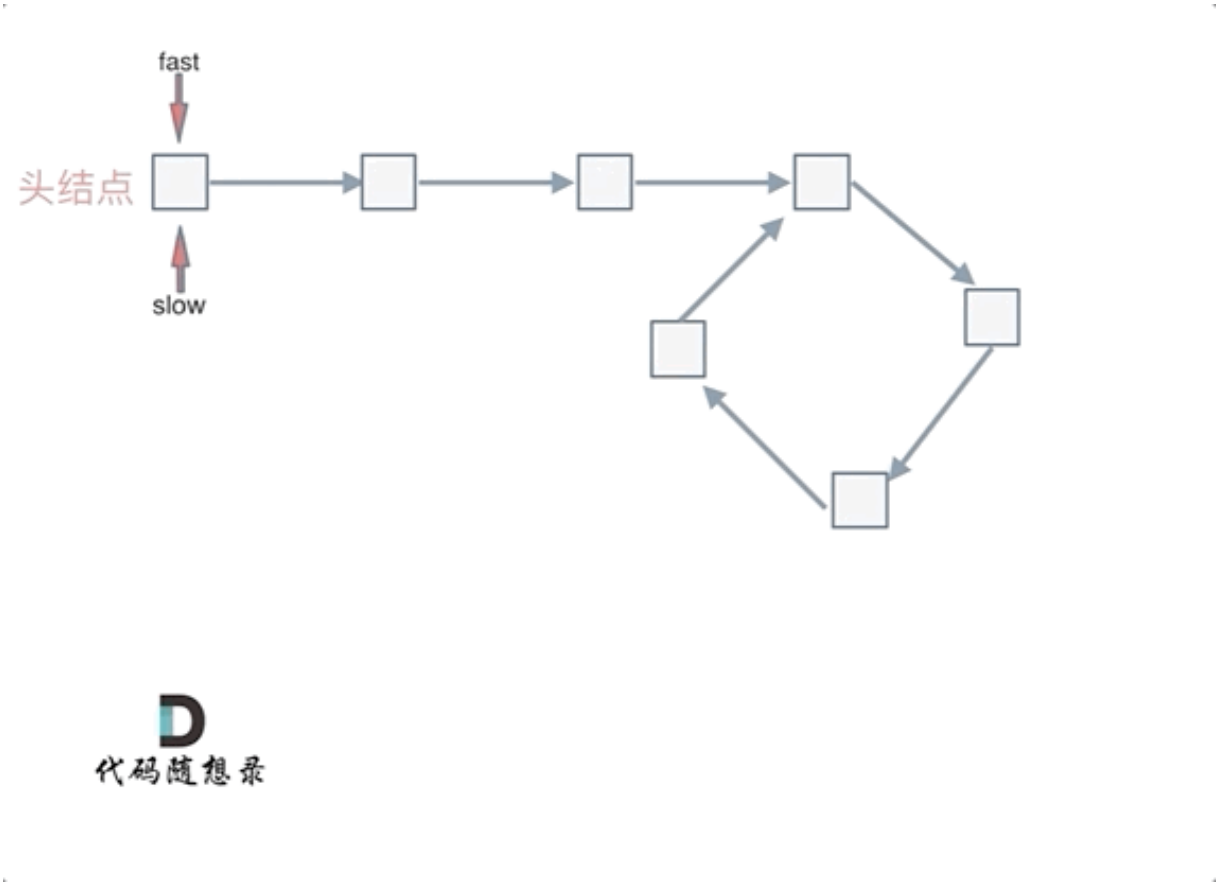
当 n为1的时候，公式就化解为 `x = z`，

这就意味着，**从头结点出发一个指针，从相遇节点 也出发一个指针，这两个指针每次只走一个节点， 那么当这两个指针相遇的时候就是 环形入口的节点。**

也就是在相遇节点处，定义一个指针index1，在头结点处定一个指针index2。

让index1和index2同时移动，每次移动一个节点， 那么他们相遇的地方就是 环形入口的节点。

动画如下：



那么 n如果大于1是什么情况呢，就是fast指针在环形转n圈之后才遇到 slow指针。

其实这种情况和n为1的时候 效果是一样的，一样可以通过这个方法找到 环形的入口节点，只不过，index1 指针在环里 多转了(n-1)圈，然后再遇到index2，相遇点依然是环形的入口节点。

代码如下：

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        ListNode* fast = head;
        ListNode* slow = head;
        while(fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
            // 快慢指针相遇，此时从head 和 相遇点，同时查找直至相遇
            if (slow == fast) {
                ListNode* index1 = fast;
                ListNode* index2 = head;
```

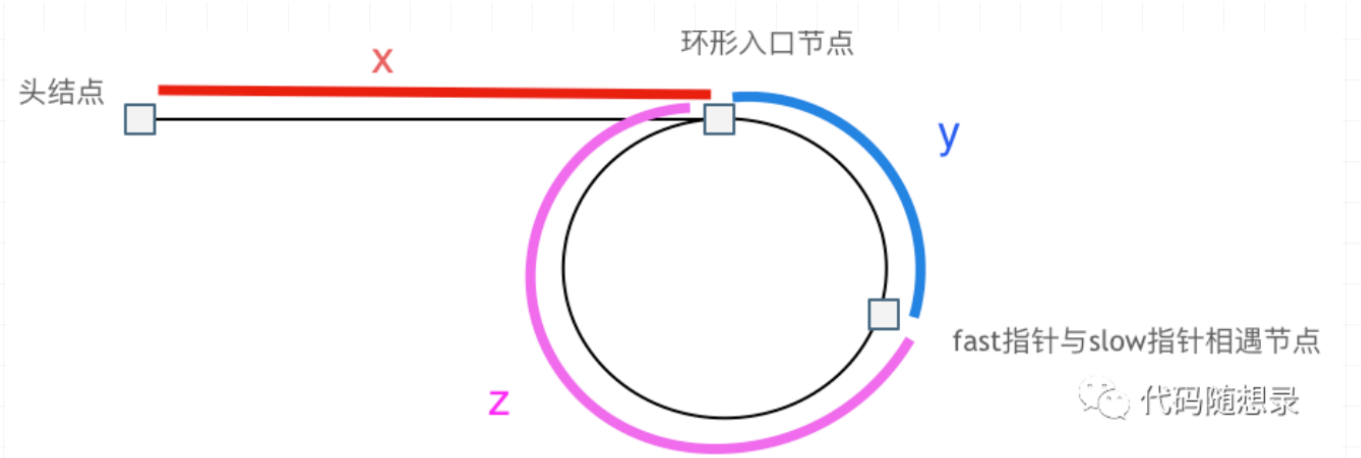
```
                while (index1 != index2) {
                    index1 = index1->next;
                    index2 = index2->next;
                }
                return index2; // 返回环的入口
            }
        }
        return NULL;
    }
};
```

- 时间复杂度: O(n)，快慢指针相遇前，指针走的次数小于链表长度，快慢指针相遇后，两个index指针走的次数也小于链表长度，总体为走的次数小于 2n
- 空间复杂度: O(1)

补充

在推理过程中，大家可能有一个疑问就是：**为什么第一次在环中相遇，slow的 步数 是 x+y 而不是 x + 若干环的长度 + y 呢?**

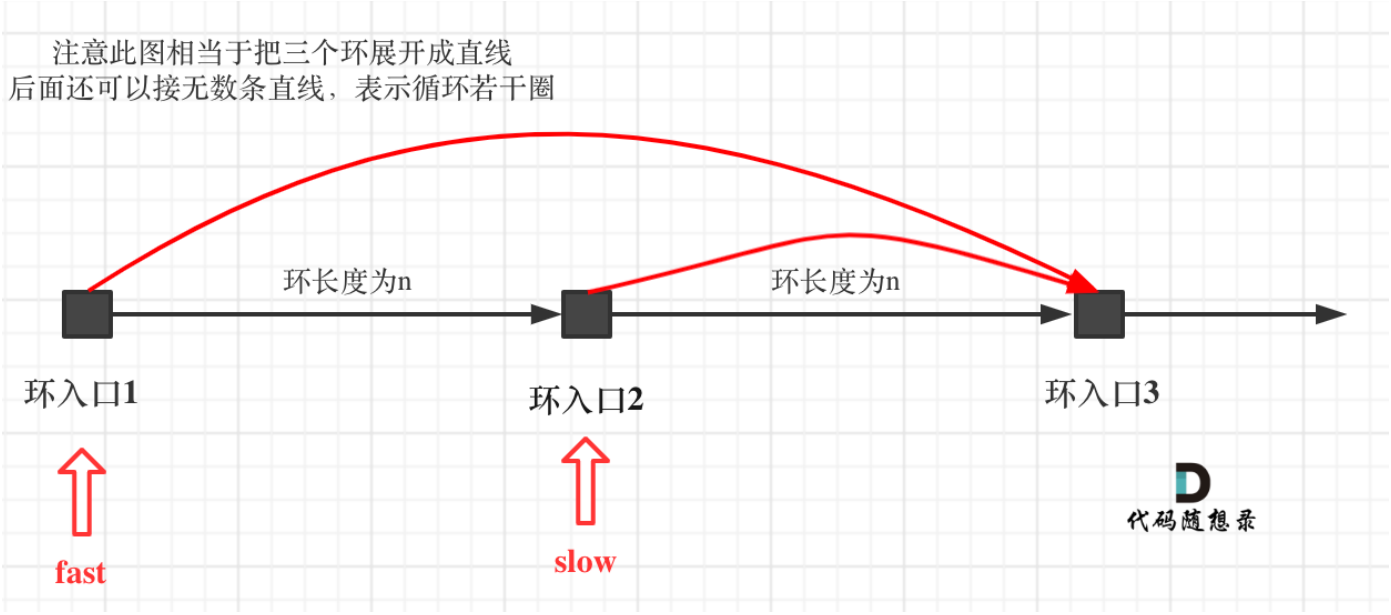
即文章链表：环找到了，那入口呢? 中如下的地方：



那么相遇时：slow指针走过的节点数为：`x + y`，fast指针走过的节点数：`x + y + n (y + z)`，n为fast指针在环内走了n圈才遇到slow指针，（y+z）为 一圈内节点的个数A。

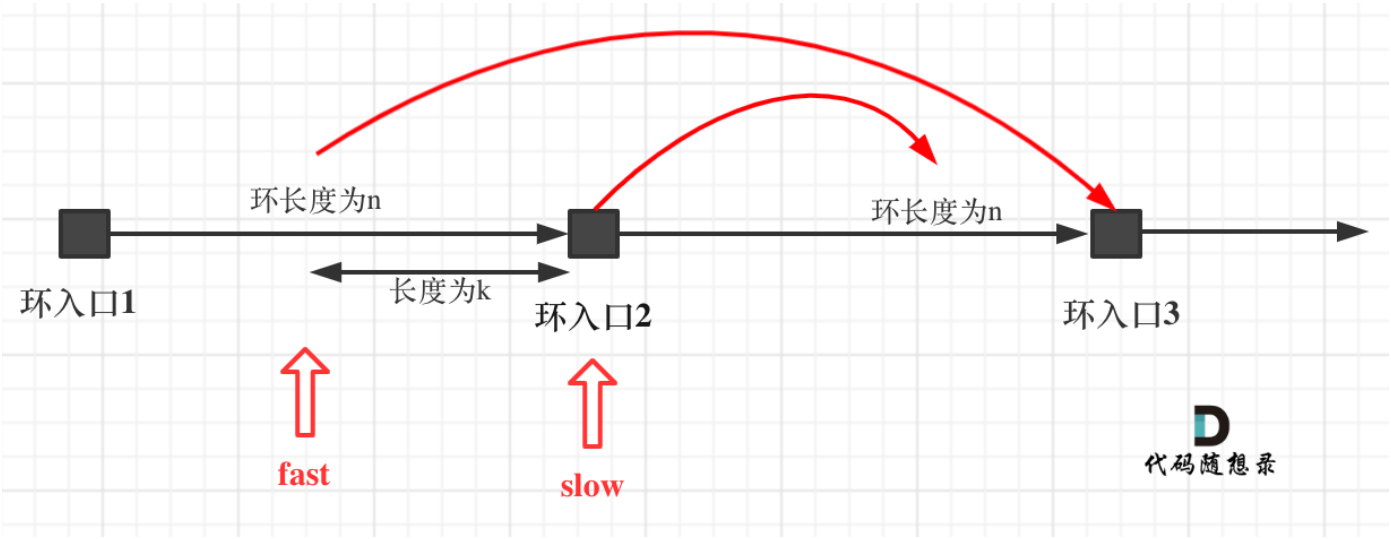
首先slow进环的时候，fast一定是先进环来了。

如果slow进环入口，fast也在环入口，那么把这个环展开成直线，就是如下图的样子：



可以看出如果slow 和 fast同时在环入口开始走，一定会在环入口3相遇，slow走了一圈，fast走了两圈。

重点来了，slow进环的时候，fast一定是在环的任意一个位置，如图：



那么fast指针走到环入口3的时候，已经走了k + n 个节点，slow相应的应该走了(k + n) / 2 个节点。

因为k是小于n的（图中可以看出），所以(k + n) / 2 一定小于n。

也就是说slow一定没有走到环入口3，而fast已经到环入口3了。

这说明什么呢?

在slow开始走的那一环已经和fast相遇了。

那有同学又说了，为什么fast不能跳过去呢？ 在刚刚已经说过一次了，**fast相对于slow是一次移动一个节点，所以不可能跳过去。**

好了，这次把为什么第一次在环中相遇，slow的 步数 是 x+y 而不是 x + 若干环的长度 + y ，用数学推理了一下，算是对链表：环找到了，那入口呢？的补充。

总结

这次可以说把环形链表这道题目的各个细节，完完整整的证明了一遍，说这是全网最详细讲解不为过吧，哈哈。

9. 链表总结篇

链表的理论基础

在这篇文章关于链表，你该了解这些！中，介绍了如下几点：

- 链表的种类主要为：单链表，双链表，循环链表
- 链表的存储方式：链表的节点在内存中是分散存储的，通过指针连在一起。
- 链表是如何进行增删改查的。
- 数组和链表在不同场景下的性能分析。

可以说把链表基础的知识都概括了，但又不像教科书那样的繁琐。

链表经典题目

虚拟头结点

在链表：听说用虚拟头节点会方便很多？中，我们讲解了链表操作中一个非常总要的技巧：虚拟头节点。

链表的一大问题就是操作当前节点必须要找前一个节点才能操作。这就造成了，头结点的尴尬，因为头结点没有前一个节点了。

每次对应头结点的情况都要单独处理，所以使用虚拟头结点的技巧，就可以解决这个问题。

在链表：听说用虚拟头节点会方便很多？中，我给出了用虚拟头结点和没用虚拟头结点的代码，大家对比一下就会发现，使用虚拟头结点的好处。

链表的基本操作

在链表：一道题目考察了常见的五个操作！中，我们通设计链表把链表常见的五个操作练习了一遍。

这是练习链表基础操作的非常好的一道题目，考察了：

- 获取链表第index个节点的数值
- 在链表的最前面插入一个节点
- 在链表的最后面插入一个节点
- 在链表第index个节点前面插入一个节点
- 删除链表的第index个节点的数值

可以说把这道题目做了，链表基本操作就OK了，再也不用担心链表增删改查整不明白了。

这里我依然使用了虚拟头结点的技巧，大家复习的时候，可以去看一下代码。

反转链表

在链表：听说过两天反转链表又写不出来了？ 中，讲解了如何反转链表。

因为反转链表的代码相对简单，有的同学可能直接背下来了，但一写还是容易出问题。

反转链表是面试中高频题目，很考察面试者对链表操作的熟练程度。

我在文章中，给出了两种反转的方式，迭代法和递归法。

建议大家先学透迭代法，然后再看递归法，因为递归法比较绕，如果迭代还写不明白，递归基本也写不明白了。

可以先通过迭代法，彻底弄清楚链表反转的过程！

删除倒数第N个节点

在链表：删除链表倒数第N个节点，怎么删？ 中我们结合虚拟头结点 和 双指针法来移除链表倒数第N个节点。

链表相交

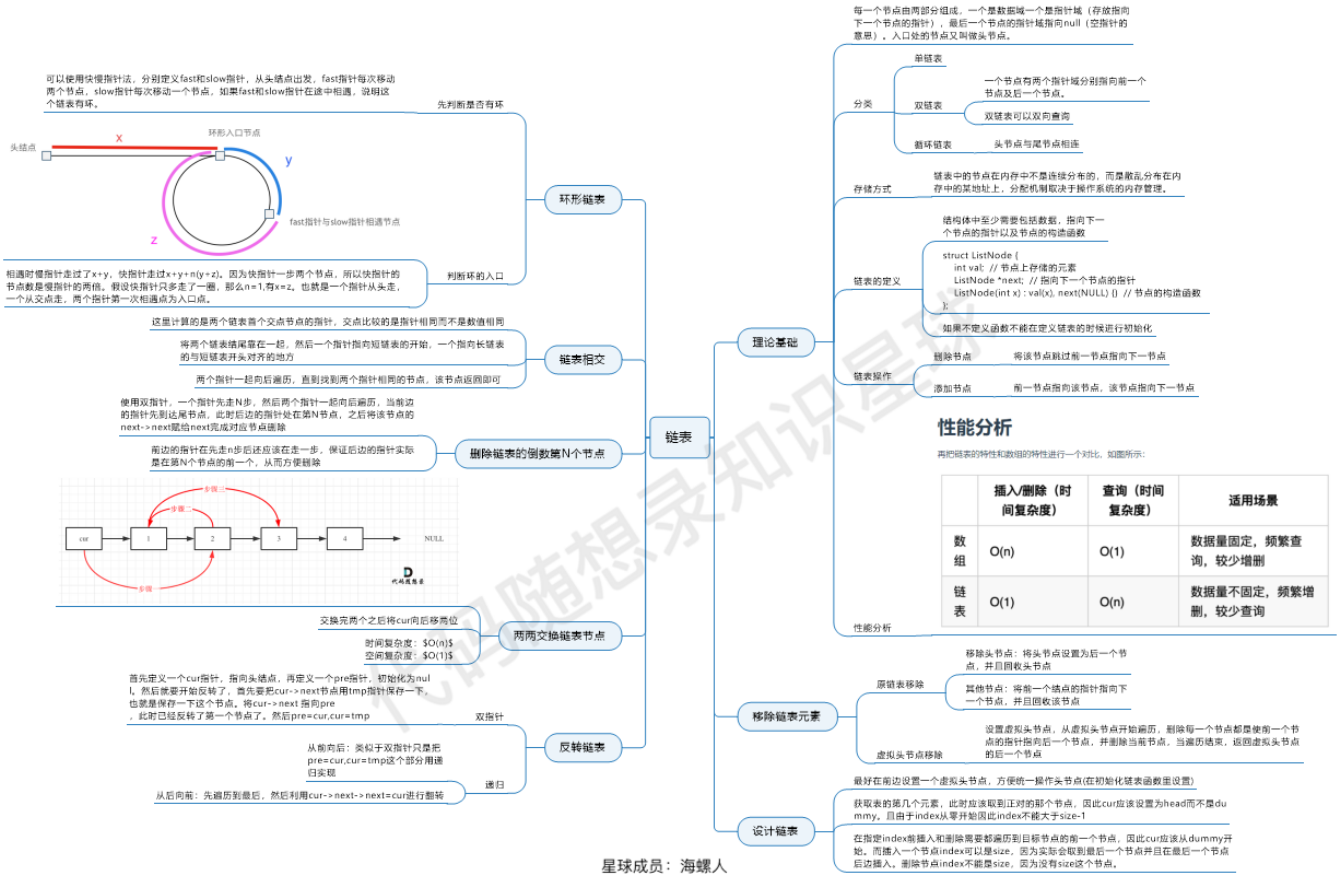
链表：链表相交使用双指针来找到两个链表的交点（引用完全相同，即：内存地址完全相同的交点）

环形链表

在链表：环找到了，那入口呢？ 中，讲解了在链表如何找环，以及如何找环的入口位置。

这道题目可以说是链表的比较难的题目了。 但代码却十分简洁，主要在于一些数学证明。

总结



这个图是 代码随想录知识星球 成员：海螺人，所画，总结的非常好，分享给大家。

考察链表的操作其实就是考察指针的操作，是面试中的常见类型。

链表篇中开头介绍链表理论知识，然后分别通过经典题目介绍了如下知识点：

1. 关于链表，你该了解这些！
2. 虚拟头结点的技巧
3. 链表的增删改查
4. 反转一个链表
5. 删除倒数第N个节点
6. 链表相交
7. 有否环形，以及环的入口

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 哈希表理论基础

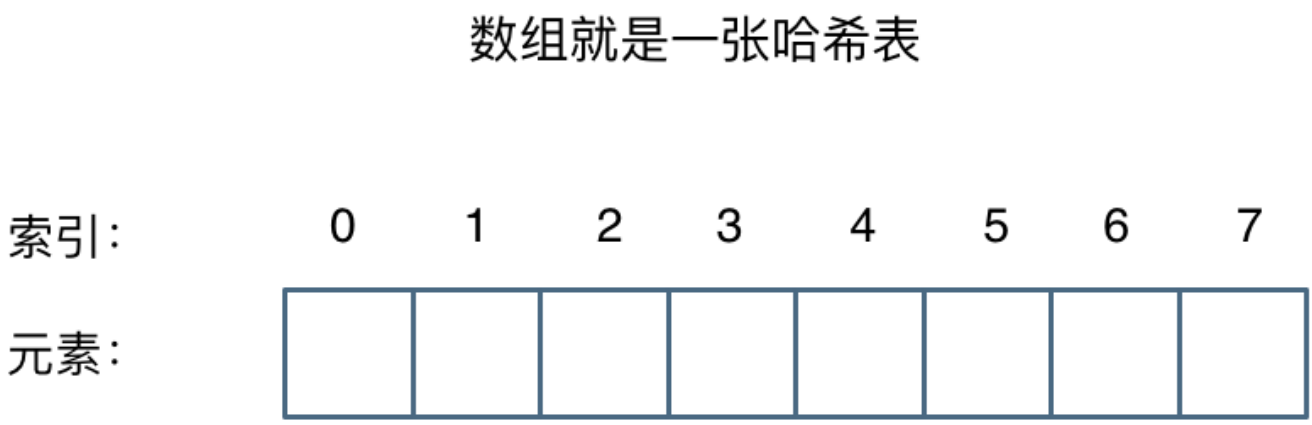
哈希表

首先什么是 哈希表，哈希表（英文名字为Hash table，国内也有一些算法书籍翻译为散列表，大家看到这两个名称知道都是指hash table就可以了）。

> 哈希表是根据关键码的值而直接进行访问的数据结构。

这么这官方的解释可能有点懵，其实直白来讲其实数组就是一张哈希表。

哈希表中关键码就是数组的索引下标，然后通过下标直接访问数组中的元素，如下图所示：



那么哈希表能解决什么问题呢，**一般哈希表都是用来快速判断一个元素是否出现集合里。**

例如要查询一个名字是否在这所学校里。

要枚举的话时间复杂度是O(n)，但如果使用哈希表的话， 只需要O(1)就可以做到。

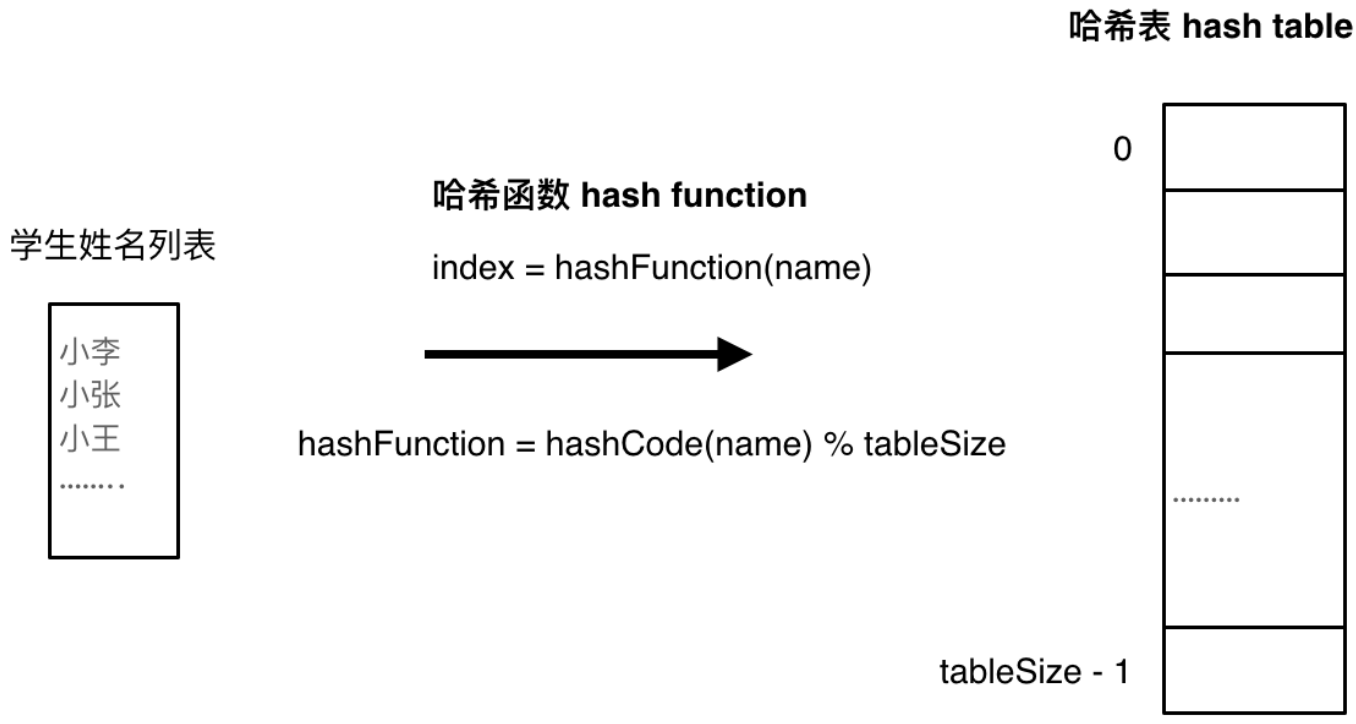
我们只需要初始化把这所学校里学生的名字都存在哈希表里，在查询的时候通过索引直接就可以知道这位同学在不在这所学校里了。

将学生姓名映射到哈希表上就涉及到了**hash function ，也就是哈希函数**。

哈希函数

哈希函数，把学生的姓名直接映射为哈希表上的索引，然后就可以通过查询索引下标快速知道这位同学是否在这所学校里了。

哈希函数如下图所示，通过hashCode把名字转化为数值，一般hashcode是通过特定编码方式，可以将其他数据格式转化为不同的数值，这样就把学生名字映射为哈希表上的索引数字了。



如果hashCode得到的数值大于 哈希表的大小了，也就是大于tableSize了，怎么办呢?

此时为了保证映射出来的索引数值都落在哈希表上，我们会在再次对数值做一个取模的操作，就要我们就保证了学生姓名一定可以映射到哈希表上了。

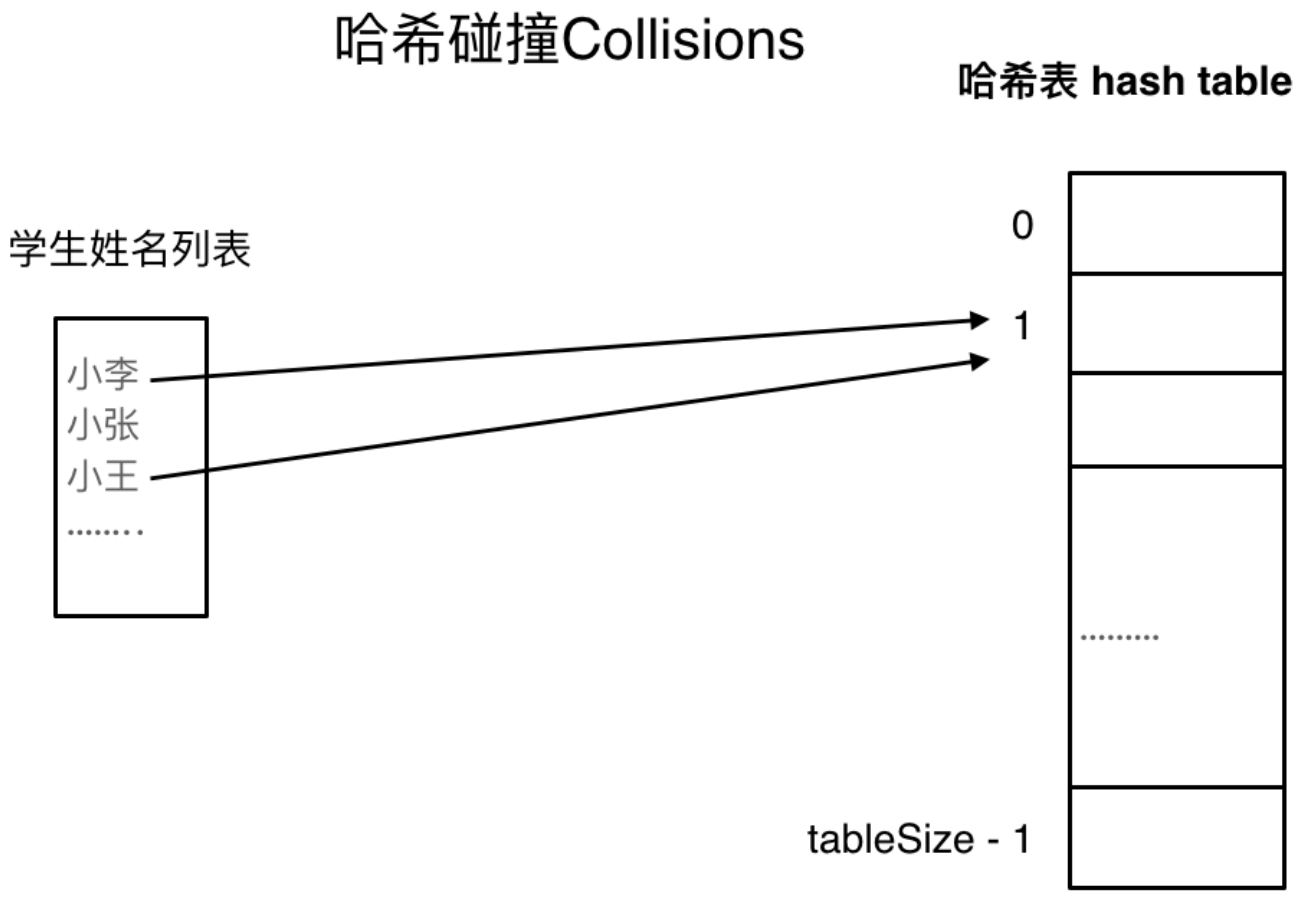
此时问题又来了，哈希表我们刚刚说过，就是一个数组。

如果学生的数量大于哈希表的大小怎么办，此时就算哈希函数计算的再均匀，也避免不了会有几位学生的名字同时映射到哈希表 同一个索引下标的位置。

接下来**哈希碰撞**登场

哈希碰撞

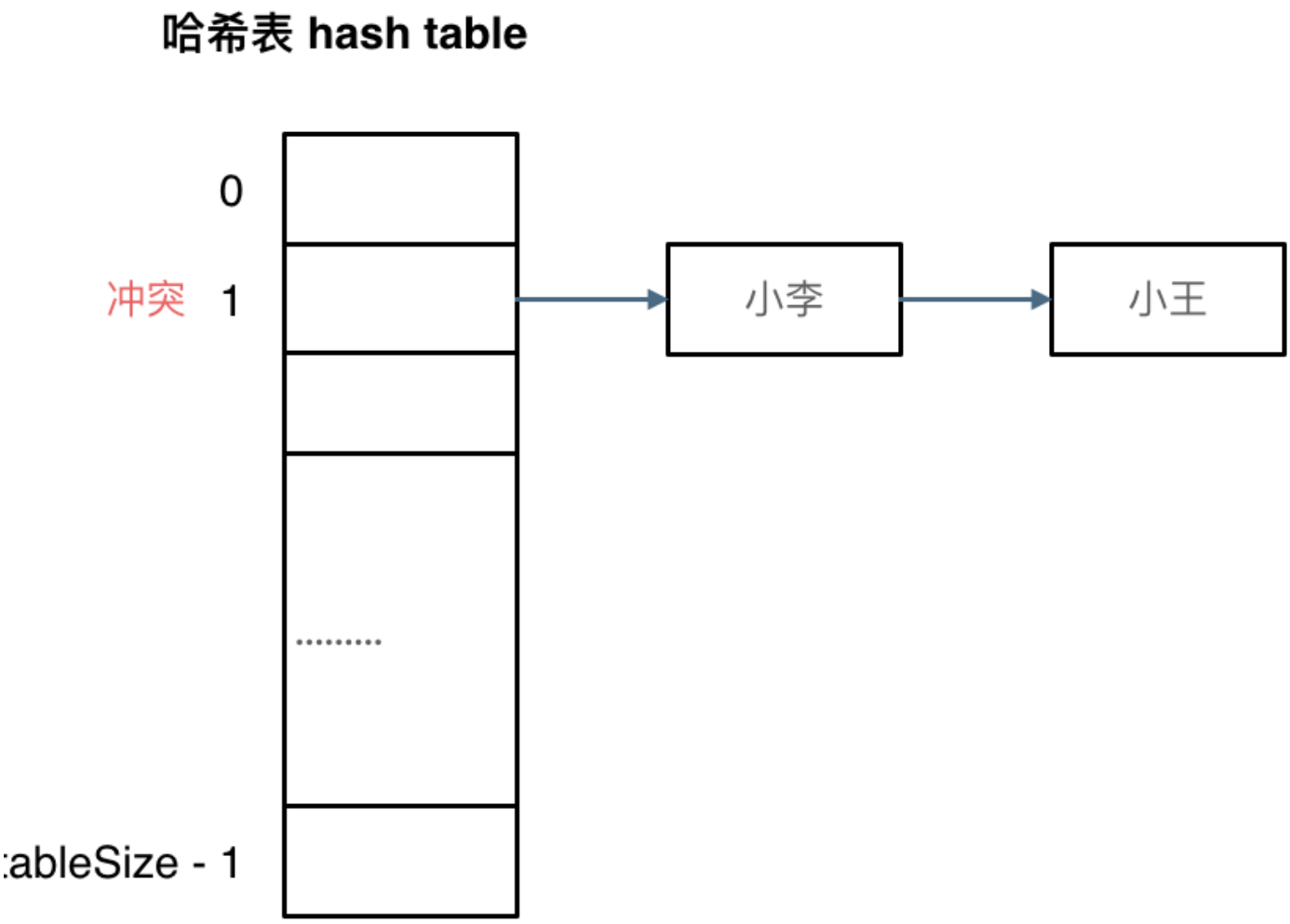
如图所示，小李和小王都映射到了索引下标 1 的位置，**这一现象叫做哈希碰撞**。



一般哈希碰撞有两种解决方法， 拉链法和线性探测法。

拉链法

刚刚小李和小王在索引1的位置发生了冲突，发生冲突的元素都被存储在链表中。 这样我们就可以通过索引找到小李和小王了



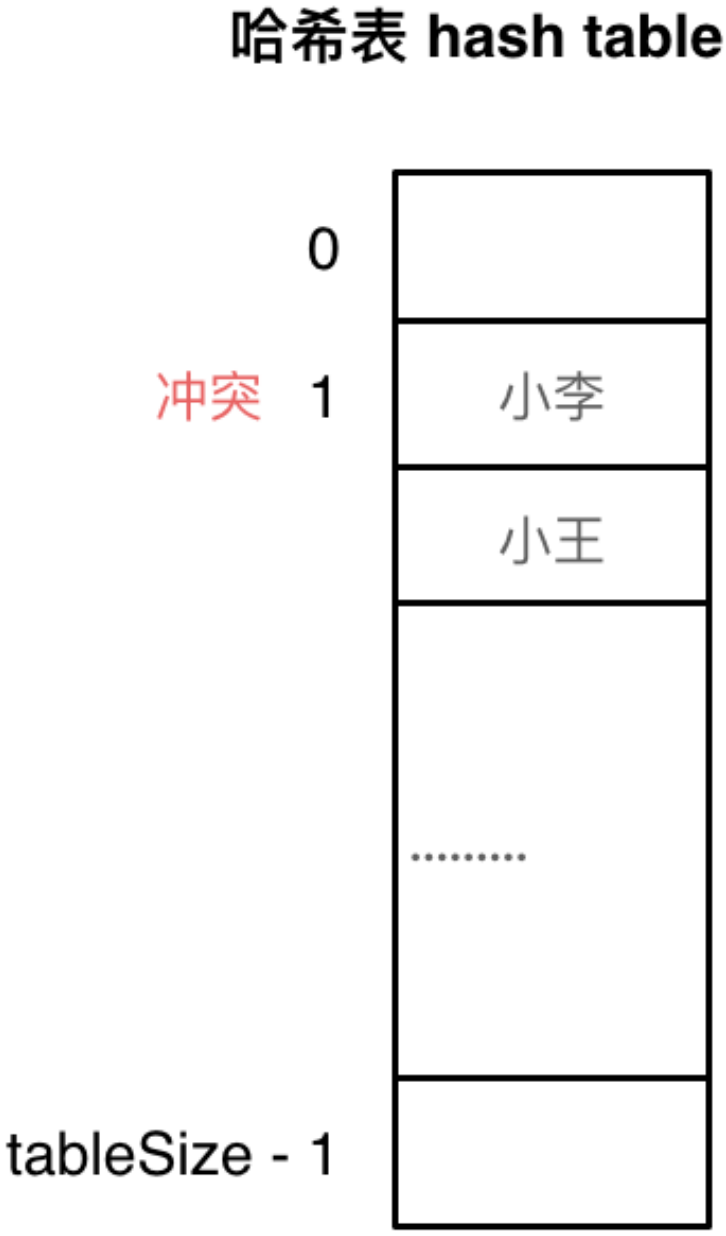
（数据规模是dataSize， 哈希表的大小为tableSize）

其实拉链法就是要选择适当的哈希表的大小，这样既不会因为数组空值而浪费大量内存，也不会因为链表太长而在查找上浪费太多时间。

线性探测法

使用线性探测法，一定要保证tableSize大于dataSize。 我们需要依靠哈希表中的空位来解决碰撞问题。

例如冲突的位置，放了小李，那么就向下找一个空位放置小王的信息。所以要求tableSize一定要大于dataSize ，要不然哈希表上就没有空置的位置来存放 冲突的数据了。如图所示：



其实关于哈希碰撞还有非常多的细节，感兴趣的同学可以再好好研究一下，这里我就不再赘述了。

常见的三种哈希结构

当我们想使用哈希法来解决问题的时候，我们一般会选择如下三种数据结构。

- 数组
- set （集合）
- map(映射)

这里数组就没啥可说的了，我们来看一下set。

在C++中，set 和 map 分别提供以下三种数据结构，其底层实现以及优劣如下表所示：

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	O(log n)	O(log n)
std::multiset	红黑树	有序	是	否	O(logn)	O(logn)
std::unordered_set	哈希表	无序	否	否	O(1)	O(1)

std::unordered_set底层实现为哈希表，std::set 和std::multiset 的底层实现是红黑树，红黑树是一种平衡二叉搜索树，所以key值是有序的，但key不可以修改，改动key值会导致整棵树的错乱，所以只能删除和增加。

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	O(logn)	O(logn)
std::multimap	红黑树	key有序	key可重复	key不可修改	O(log n)	O(log n)
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	O(1)	O(1)

std::unordered_map 底层实现为哈希表，std::map 和std::multimap 的底层实现是红黑树。同理，std::map 和 std::multimap 的key也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解）。

当我们要使用集合来解决哈希问题的时候，优先使用unordered_set，因为它的查询和增删效率是最优的，如果需要集合是有序的，那么就用set，如果要求不仅有序还要有重复数据的话，那么就用multiset。

那么再来看一下map ，在map 是一个key value 的数据结构，map中，对key是有限制，对value没有限制的，因为key的存储方式使用红黑树实现的。

其他语言例如：java里的HashMap ，TreeMap 都是一样的原理。可以灵活贯通。

虽然std::set、std::multiset 的底层实现是红黑树，不是哈希表，std::set、std::multiset 使用红黑树来索引和存储，不过给我们的使用方式，还是哈希法的使用方式，即key和value。所以使用这些数据结构来解决映射问题的方法，我们依然称之为哈希法。 map也是一样的道理。

这里在说一下，一些C++的经典书籍上 例如STL源码剖析，说到了hash_set hash_map，这个与unordered_set，unordered_map又有什么关系呢?

实际上功能都是一样一样的， 但是unordered_set在C++11的时候被引入标准库了，而hash_set并没有，所以建议还是使用unordered_set比较好，这就好比一个是官方认证的，hash_set，hash_map 是C++11标准之前民间高手自发造的轮子。

总结

总结一下，**当我们遇到了要快速判断一个元素是否出现集合里的时候，就要考虑哈希法。**

但是哈希法也是**牺牲了空间换取了时间**，因为我们要使用额外的数组，set或者是map来存放数据，才能实现快速的查找。

如果在做面试题目的时候遇到需要判断一个元素是否出现过的场景也应该第一时间想到哈希法！

> 数组就是简单的哈希表，但是数组的大小可不是无限开辟的

2.有效的字母异位词

力扣题目链接

给定两个字符串 s 和 t ，编写一个函数来判断 t 是否是 s 的字母异位词。

示例 1:
输入: s = "anagram", t = "nagaram"
输出: true

示例 2:
输入: s = "rat", t = "car"
输出: false

说明:
你可以假设字符串只包含小写字母。

算法公开课

《代码随想录》算法视频公开课：学透哈希表，数组使用有技巧！Leetcode：242.有效的字母异位词，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

先看暴力的解法，两层for循环，同时还要记录字符是否重复出现，很明显时间复杂度是 O(n^2)。

暴力的方法这里就不做介绍了，直接看一下有没有更优的方式。

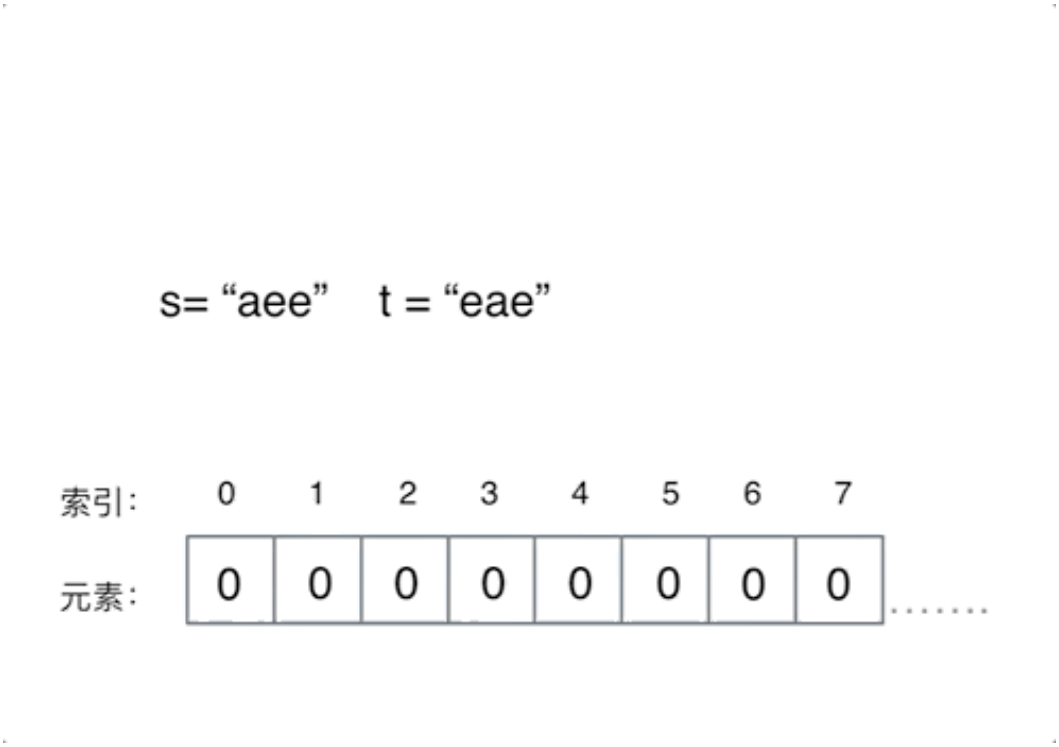
数组其实就是一个简单哈希表，而且这道题目中字符串只有小写字符，那么就可以定义一个数组，来记录字符串s里字符出现的次数。

如果对哈希表的理论基础关于数组，set，map不了解的话可以看这篇：关于哈希表，你该了解这些！

需要定义一个多大的数组呢，定一个数组叫做record，大小为26 就可以了，初始化为0，因为字符a到字符z的ASCII也是26个连续的数值。

为了方便举例，判断一下字符串s= "aee", t = "eae"。

操作动画如下：



定义一个数组叫做record用来上记录字符串s里字符出现的次数。

需要把字符映射到数组也就是哈希表的索引下标上，**因为字符a到字符z的ASCII是26个连续的数值，所以字符a映射为下标0，相应的字符z映射为下标25。**

再遍历 字符串s的时候，**只需要将 s[i] - ‘a’ 所在的元素做+1 操作即可，并不需要记住字符a的ASCII，只要求出一个相对数值就可以了。** 这样就将字符串s中字符出现的次数，统计出来了。

那看一下如何检查字符串t中是否出现了这些字符，同样在遍历字符串t的时候，对t中出现的字符映射哈希表索引上的数值再做-1的操作。

那么最后检查一下，**record数组如果有的元素不为零0，说明字符串s和t一定是谁多了字符或者谁少了字符，return false。**

最后如果record数组所有元素都为零0，说明字符串s和t是字母异位词，return true。

时间复杂度为O(n)，空间上因为定义是的一个常量大小的辅助数组，所以空间复杂度为O(1)。

C++ 代码如下：

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        int record[26] = {0};
        for (int i = 0; i < s.size(); i++) {
            // 并不需要记住字符a的ASCII，只要求出一个相对数值就可以了
            record[s[i] - 'a']++;
        }
        for (int i = 0; i < t.size(); i++) {
            record[t[i] - 'a']--;
        }
        for (int i = 0; i < 26; i++) {
            if (record[i] != 0) {
                // record数组如果有的元素不为零0，说明字符串s和t 一定是谁多了字符或者谁少了字符。
                return false;
            }
        }
        // record数组所有元素都为零0，说明字符串s和t是字母异位词
        return true;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

> 如果哈希值比较少、特别分散、跨度非常大，使用数组就造成空间的极大浪费！

3. 两个数组的交集

力扣题目链接

题意：给定两个数组，编写一个函数来计算它们的交集。

示例 1：

```
输入：nums1 = [1,2,2,1], nums2 = [2,2]
输出：[2]
```

示例 2：

```
输入：nums1 = [4,9,5], nums2 = [9,4,9,8,4]
输出：[9,4]
```

说明：
输出结果中的每个元素一定是唯一的。
我们可以不考虑输出结果的顺序。

算法公开课

《代码随想录》算法视频公开课：：学透哈希表，set使用有技巧！Leetcode：349. 两个数组的交集，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目，主要要学会使用一种哈希数据结构：unordered_set，这个数据结构可以解决很多类似的问题。

注意题目特意说明：**输出结果中的每个元素一定是唯一的，也就是说输出的结果的去重的， 同时可以不考虑输出结果的顺序**

这道题用暴力的解法时间复杂度是O(n^2)，那来看看使用哈希法进一步优化。

那么用数组来做哈希表也是不错的选择，例如242. 有效的字母异位词

但是要注意，**使用数组来做哈希的题目，是因为题目都限制了数值的大小。**

而这道题目没有限制数值的大小，就无法使用数组来做哈希表了。

而且如果哈希值比较少、特别分散、跨度非常大，使用数组就造成空间的极大浪费。

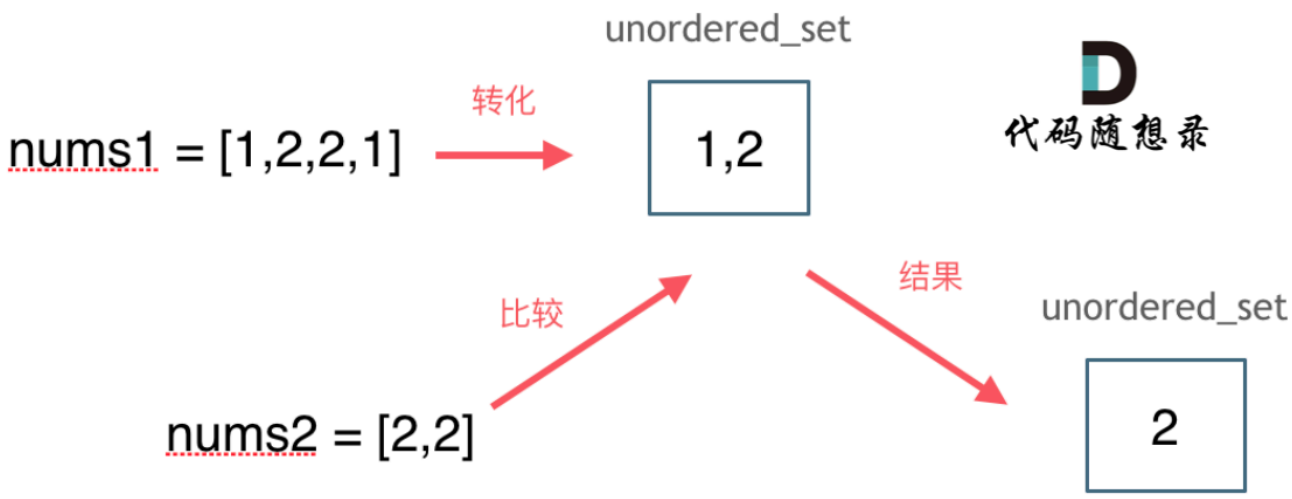
此时就要使用另一种结构体了，set ，关于set，C++ 给提供了如下三种可用的数据结构：

- std::set
- std::multiset

- std::unordered_set

std::set和std::multiset底层实现都是红黑树，std::unordered_set的底层实现是哈希表， 使用unordered_set 读写效率是最高的，并不需要对数据进行排序，而且还不要让数据重复，所以选择unordered_set。

思路如图所示：



C++代码如下：

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        unordered_set<int> result_set; // 存放结果，之所以用set是为了给结果集去重
        unordered_set<int> nums_set(nums1.begin(), nums1.end());
        for (int num : nums2) {
            // 发现nums2的元素 在nums_set里又出现过
            if (nums_set.find(num) != nums_set.end()) {
                result_set.insert(num);
            }
        }
        return vector<int>(result_set.begin(), result_set.end());
    }
};
```

- 时间复杂度: O(mn)
- 空间复杂度: O(n)

拓展

那有同学可能问了，遇到哈希问题我直接都用set不就得了，用什么数组啊。

直接使用set 不仅占用空间比数组大，而且速度要比数组慢，set把数值映射到key上都要做hash计算的。

不要小瞧 这个耗时，在数据量大的情况，差距是很明显的。

后记

本题后面 力扣改了 题目描述 和 后台测试数据，增添了 数值范围：

- 1 <= nums1.length, nums2.length <= 1000
- 0 <= nums1[i], nums2[i] <= 1000

所以就可以 使用数组来做哈希表了， 因为数组都是 1000以内的。

对应C++代码如下：

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        unordered_set<int> result_set; // 存放结果，之所以用set是为了给结果集去重
        int hash[1005] = {0}; // 默认数值为0
        for (int num : nums1) { // nums1中出现的字母在hash数组中做记录
            hash[num] = 1;
        }
        for (int num : nums2) { // nums2中出现话，result记录
            if (hash[num] == 1) {
                result_set.insert(num);
            }
        }
        return vector<int>(result_set.begin(), result_set.end());
    }
};
```

- 时间复杂度: O(m + n)
- 空间复杂度: O(n)

相关题目

- 350.两个数组的交集 II

> 该用set的时候，还是得用set

4. 快乐数

力扣题目链接

编写一个算法来判断一个数 n 是不是快乐数。

「快乐数」定义为：对于一个正整数，每一次将该数替换为它每个位置上的数字的平方和，然后重复这个过程直到这个数变为 1，也可能是 无限循环 但始终变不到 1。如果 可以变为 1，那么这个数就是快乐数。

如果 n 是快乐数就返回 True ； 不是，则返回 False 。

示例：

输入：19
输出：true
解释：
1^2 + 9^2 = 82
8^2 + 2^2 = 68
6^2 + 8^2 = 100
1^2 + 0^2 + 0^2 = 1

思路

这道题目看上去貌似一道数学问题，其实并不是！

题目中说了会 **无限循环**，那么也就是说**求和的过程中，sum会重复出现，这对解题很重要！**

正如：关于哈希表，你该了解这些！中所说，**当我们遇到了要快速判断一个元素是否出现集合里的时候，就要考虑哈希法了。**

所以这道题目使用哈希法，来判断这个sum是否重复出现，如果重复了就是return false， 否则一直找到sum为1为止。

判断sum是否重复出现就可以使用unordered_set。

还有一个难点就是求和的过程，如果对取数值各个位上的单数操作不熟悉的话，做这道题也会比较艰难。

C++代码如下：

```
class Solution {
public:
    // 取数值各个位上的单数之和
    int getSum(int n) {
        int sum = 0;
        while (n) {
            sum += (n % 10) * (n % 10);
            n /= 10;
        }
        return sum;
    }
    bool isHappy(int n) {
        unordered_set<int> set;
        while(1) {
            int sum = getSum(n);
            if (sum == 1) {
                return true;
            }
            // 如果这个sum曾经出现过，说明已经陷入了无限循环了，立刻return false
            if (set.find(sum) != set.end()) {
                return false;
            } else {
```

```
                set.insert(sum);
            }
            n = sum;
        }
    }
};
```

- 时间复杂度: O(logn)
- 空间复杂度: O(logn)

---

5. 两数之和

力扣题目链接

给定一个整数数组 nums 和一个目标值 target，请你在该数组中找出和为目标值的那 两个 整数，并返回他们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素不能使用两遍。

示例:

给定 nums = [2, 7, 11, 15], target = 9

因为 nums[0] + nums[1] = 2 + 7 = 9

所以返回 [0, 1]

算法公开课

《代码随想录》算法视频公开课：梦开始的地方，Leetcode：1.两数之和,相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

很明显暴力的解法是两层for循环查找，时间复杂度是O(n^2)。

建议大家做这道题目之前，先做一下这两道

- 242. 有效的字母异位词
- 349. 两个数组的交集

242. 有效的字母异位词 这道题目是用数组作为哈希表来解决哈希问题，349. 两个数组的交集这道题目是通过set作为哈希表来解决哈希问题。

首先我在强调一下 **什么时候使用哈希法**，当我们需要查询一个元素是否出现过，或者一个元素是否在集合里的时候，就要第一时间想到哈希法。

本题呢，我就需要一个集合来存放我们遍历过的元素，然后在遍历数组的时候去询问这个集合，某元素是否遍历过，也就是 是否出现在这个集合。

那么我们就应该想到使用哈希法了。

因为本地，我们不仅要知道元素有没有遍历过，还要知道这个元素对应的下标，**需要使用 key value结构来存放，key来存元素，value来存下标，那么使用map正合适**。

再来看一下使用数组和set来做哈希法的局限。

- 数组的大小是受限制的，而且如果元素很少，而哈希值太大会造成内存空间的浪费。
- set是一个集合，里面放的元素只能是一个key，而两数之和这道题目，不仅要判断y是否存在而且还要记录y的下标位置，因为要返回x 和 y的下标。所以set 也不能用。

此时就要选择另一种数据结构：map ，map是一种key value的存储结构，可以用key保存数值，用value在保存数值所在的下标。

C++中map，有三种类型：

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	O(log n)	O(log n)
std::multimap	红黑树	key有序	key可重复	key不可修改	O(log n)	O(log n)
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	O(1)	O(1)

std::unordered_map 底层实现为哈希表，std::map 和std::multimap 的底层实现是红黑树。

同理，std::map 和std::multimap 的key也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解）。 更多哈希表的理论知识请看关于哈希表，你该了解这些！。

这道题目中并不需要key有序，选择std::unordered_map 效率更高！ 使用其他语言的录友注意了解一下自己所用语言的数据结构就行。

接下来需要明确两点：

- **map用来做什么**
- **map中key和value分别表示什么**

map目的用来存放我们访问过的元素，因为遍历数组的时候，需要记录我们之前遍历过哪些元素和对应的下标，这样才能找到与当前元素相匹配的（也就是相加等于target）

接下来是map中key和value分别表示什么。

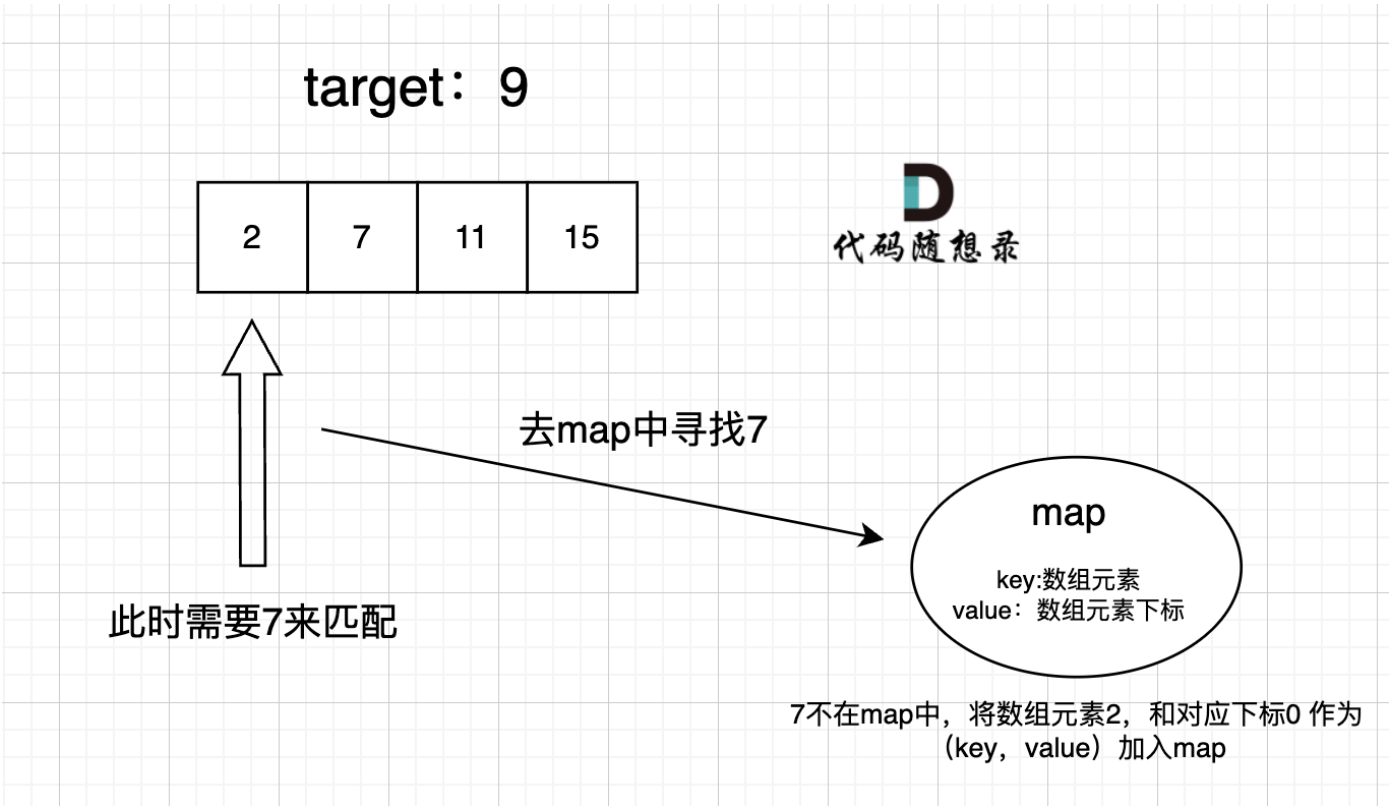
这道题 我们需要 给出一个元素，判断这个元素是否出现过，如果出现过，返回这个元素的下标。

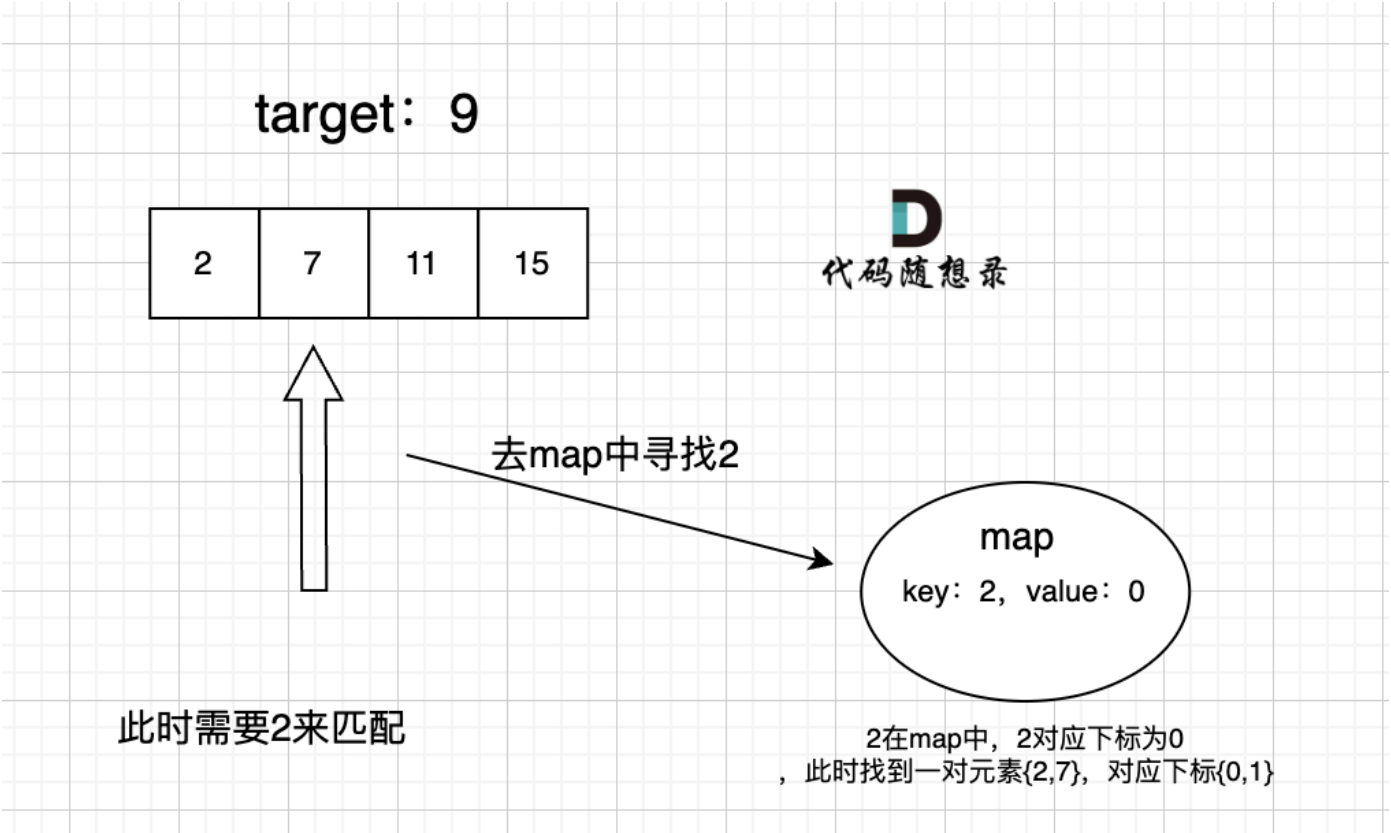
那么判断元素是否出现，这个元素就要作为key，所以数组中的元素作为key，有key对应的就是value，value用来存下标。

所以 map中的存储结构为 {key：数据元素，value：数组元素对应的下标}。

在遍历数组的时候，只需要向map去查询是否有和目前遍历元素匹配的数值，如果有，就找到的匹配对，如果没有，就把目前遍历的元素放进map中，因为map存放的就是我们访问过的元素。

过程如下：





C++代码：

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        std::unordered_map <int,int> map;
        for(int i = 0; i < nums.size(); i++) {
```

```
            // 遍历当前元素，并在map中寻找是否有匹配的key
            auto iter = map.find(target - nums[i]);
            if(iter != map.end()) {
                return {iter->second, i};
            }
            // 如果没找到匹配对，就把访问过的元素和下标加入到map中
            map.insert(pair<int, int>(nums[i], i));
        }
        return {};
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

总结

本题其实有四个重点：

- 为什么会想到用哈希表
- 哈希表为什么用map
- 本题map是用来存什么的
- map中的key和value用来存什么的

把这四点想清楚了，本题才算是理解透彻了。

很多录友把这道题目 通过了，但都没想清楚map是用来做什么的，以至于对代码的理解其实是 一知半解的。

> 需要哈希的地方都能找到map的身影

6.四数相加II

力扣题目链接

给定四个包含整数的数组列表 A , B , C , D ,计算有多少个元组 (i, j, k, l) ，使得 A[i] + B[j] + C[k] + D[l] = 0。

为了使问题简单化，所有的 A, B, C, D 具有相同的长度 N，且 0 ≤ N ≤ 500 。所有整数的范围在 -2^28 到 2^28 - 1 之间，最终结果不会超过 2^31 - 1 。

例如:

输入:

- A = [1, 2]
- B = [-2,-1]
- C = [-1, 2]

- D = [0, 2]

输出:

2

解释:

两个元组如下:

1. (0, 0, 0, 1) -> A[0] + B[0] + C[0] + D[1] = 1 + (-2) + (-1) + 2 = 0
2. (1, 1, 0, 0) -> A[1] + B[1] + C[0] + D[0] = 2 + (-1) + (-1) + 0 = 0

算法公开课

《代码随想录》算法视频公开课：学透哈希表，map使用有技巧！LeetCode：454.四数相加II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题咋眼一看好像和0015.三数之和，0018.四数之和差不多，其实差很多。

本题是使用哈希法的经典题目，而0015.三数之和，0018.四数之和并不合适使用哈希法，因为三数之和和四数之和这两道题目使用哈希法在不超时的情况下做到对结果去重是很困难的，很有多细节需要处理。

而这道题目是四个独立的数组，只要找到A[i] + B[j] + C[k] + D[l] = 0就可以，不用考虑有重复的四个元素相加等于0的情况，所以相对于题目18. 四数之和，题目15.三数之和，还是简单了不少！

如果本题想难度升级：就是给出一个数组（而不是四个数组），在这里找出四个元素相加等于0，答案中不可以包含重复的四元组，大家可以思考一下，后续的文章我也会讲到的。

本题解题步骤：

1. 首先定义 一个unordered_map，key放a和b两数之和，value 放a和b两数之和出现的次数。
2. 遍历大A和大B数组，统计两个数组元素之和，和出现的次数，放到map中。
3. 定义int变量count，用来统计 a+b+c+d = 0 出现的次数。
4. 在遍历大C和大D数组，找到如果 0-(c+d) 在map中出现过的话，就用count把map中key对应的value也就是出现次数统计出来。
5. 最后返回统计值 count 就可以了

C++代码:

```
class Solution {
public:
    int fourSumCount(vector<int>& A, vector<int>& B, vector<int>& C, vector<int>& D) {
        unordered_map<int, int> umap; //key:a+b的数值，value:a+b数值出现的次数
        // 遍历大A和大B数组，统计两个数组元素之和，和出现的次数，放到map中
        for (int a : A) {
            for (int b : B) {
                umap[a + b]++;
            }
```

```
        }
        int count = 0; // 统计a+b+c+d = 0 出现的次数
        // 在遍历大C和大D数组，找到如果 0-(c+d) 在map中出现过的话，就把map中key对应的value也就是出现次数统计出来。
        for (int c : C) {
            for (int d : D) {
                if (umap.find(0 - (c + d)) != umap.end()) {
                    count += umap[0 - (c + d)];
                }
            }
        }
        return count;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(n^2)，最坏情况下A和B的值各不相同，相加产生的数字个数为 n^2

> 在哈希法中有一些场景就是为数组量身定做的。

7. 赎金信

力扣题目链接

给定一个赎金信 (ransom) 字符串和一个杂志(magazine)字符串，判断第一个字符串 ransom 能不能由第二个字符串 magazines 里面的字符构成。如果可以构成，返回 true ； 否则返回 false。

(题目说明：为了不暴露赎金信字迹，要从杂志上搜索各个需要的字母，组成单词来表达意思。杂志字符串中的每个字符只能在赎金信字符串中使用一次。)

注意：

你可以假设两个字符串均只含有小写字母。

canConstruct("a", "b") -> false
canConstruct("aa", "ab") -> false
canConstruct("aa", "aab") -> true

思路

这道题目和242.有效的字母异位词很像，242.有效的字母异位词相当于求 字符串a 和 字符串b 是否可以相互组成，而这道题目是求 字符串a能否组成字符串b，而不用管字符串b 能不能组成字符串a。

本题判断第一个字符串ransom能不能由第二个字符串magazines里面的字符构成，但是这里需要注意两点。

- 第一点“为了不暴露赎金信字迹，要从杂志上搜索各个需要的字母，组成单词来表达意思” 这里*说明杂志里面的字母不可重复使用。*

- 第二点 “你可以假设两个字符串均只含有小写字母。” *说明只有小写字母*，这一点很重要

暴力解法

那么第一个思路其实就是暴力枚举了，两层for循环，不断去寻找，代码如下：

```
class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        for (int i = 0; i < magazine.length(); i++) {
            for (int j = 0; j < ransomNote.length(); j++) {
                // 在ransomNote中找到和magazine相同的字符
                if (magazine[i] == ransomNote[j]) {
                    ransomNote.erase(ransomNote.begin() + j); // ransomNote删除这个字符
                    break;
                }
            }
        }
        // 如果ransomNote为空，则说明magazine的字符可以组成ransomNote
        if (ransomNote.length() == 0) {
            return true;
        }
        return false;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(1)

这里时间复杂度是比较高的，而且里面还有一个字符串删除也就是erase的操作，也是费时的，当然这段代码也可以过这道题。

哈希解法

因为题目所只有小写字母，那可以采用空间换取时间的哈希策略， 用一个长度为26的数组还记录magazine里字母出现的次数。

然后再用ransomNote去验证这个数组是否包含了ransomNote所需要的所有字母。

依然是数组在哈希法中的应用。

一些同学可能想，用数组干啥，都用map完事了，**其实在本题的情况下，使用map的空间消耗要比数组大一些的，因为map要维护红黑树或者哈希表，而且还要做哈希函数，是费时的！数据量大的话就能体现出来差别了。所以数组更加简单直接有效！**

代码如下：

```
class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
```

```
        int record[26] = {0};
        //add
        if (ransomNote.size() > magazine.size()) {
            return false;
        }
        for (int i = 0; i < magazine.length(); i++) {
            // 通过record数据记录 magazine里各个字符出现次数
            record[magazine[i]-'a'] ++;
        }
        for (int j = 0; j < ransomNote.length(); j++) {
            // 遍历ransomNote，在record里对应的字符个数做--操作
            record[ransomNote[j]-'a']--;
            // 如果小于零说明ransomNote里出现的字符，magazine没有
            if(record[ransomNote[j]-'a'] < 0) {
                return false;
            }
        }
        return true;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

> 用哈希表解决了两数之和，那么三数之和呢?

8. 三数之和

力扣题目链接

给你一个包含 n 个整数的数组 nums，判断 nums 中是否存在三个元素 a，b，c ，使得 a + b + c = 0 ？请你找出所有满足条件且不重复的三元组。

注意： 答案中不可以包含重复的三元组。

示例：

给定数组 nums = [-1, 0, 1, 2, -1, -4]，

满足要求的三元组集合为：
[
  [-1, 0, 1],
  [-1, -1, 2]
]

算法公开课

《代码随想录》算法视频公开课：梦破碎的地方！| LeetCode：15.三数之和，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

注意[0， 0， 0， 0] 这组数据

思路

哈希解法

两层for循环就可以确定 a 和b 的数值了，可以使用哈希法来确定 0-(a+b) 是否在 数组里出现过，其实这个思路是正确的，但是我们有一个非常棘手的问题，就是题目中说的不可以包含重复的三元组。

把符合条件的三元组放进vector中，然后再去重，这样是非常费时的，很容易超时，也是这道题目通过率如此之低的根源所在。

去重的过程不好处理，有很多小细节，如果在面试中很难想到位。

时间复杂度可以做到O(n^2)，但还是比较费时的，因为不好做剪枝操作。

大家可以尝试使用哈希法写一写，就知道其困难的程度了。

哈希法C++代码:

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        // 找出a + b + c = 0
        // a = nums[i], b = nums[j], c = -(a + b)
        for (int i = 0; i < nums.size(); i++) {
            // 排序之后如果第一个元素已经大于零，那么不可能凑成三元组
            if (nums[i] > 0) {
                break;
            }
            if (i > 0 && nums[i] == nums[i - 1]) { //三元组元素a去重
                continue;
            }
            unordered_set<int> set;
            for (int j = i + 1; j < nums.size(); j++) {
                if (j > i + 2
                        && nums[j] == nums[j-1]
                        && nums[j-1] == nums[j-2]) { // 三元组元素b去重
                    continue;
                }
                int c = 0 - (nums[i] + nums[j]);
                if (set.find(c) != set.end()) {
                    result.push_back({nums[i], nums[j], c});
                    set.erase(c);// 三元组元素c去重
                } else {
                    set.insert(nums[j]);
```

```
                }
            }
        }
        return result;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(n)，额外的 set 开销

双指针

其实这道题目使用哈希法并不十分合适，因为在去重的操作中有很多细节需要注意，在面试中很难直接写出没有bug的代码。

而且使用哈希法 在使用两层for循环的时候，能做的剪枝操作很有限，虽然时间复杂度是O(n^2)，也是可以在leetcode上通过，但是程序的执行时间依然比较长 。

接下来我来介绍另一个解法：双指针法，**这道题目使用双指针法 要比哈希法高效一些**，那么来讲解一下具体实现的思路。

动画效果如下：

拿这个nums数组来举例，首先将数组排序，然后有一层for循环，i从下标0的地方开始，同时定一个下标left 定义在i+1的位置上，定义下标right 在数组结尾的位置上。

依然还是在数组中找到 abc 使得a + b +c =0，我们这里相当于 a = nums[i]，b = nums[left]，c = nums[right]。

接下来如何移动left 和right呢， 如果nums[i] + nums[left] + nums[right] > 0 就说明 此时三数之和大了，因为数组是排序后了，所以right下标就应该向左移动，这样才能让三数之和小一些。

如果 nums[i] + nums[left] + nums[right] < 0 说明 此时 三数之和小了，left 就向右移动，才能让三数之和大一些，直到left与right相遇为止。

时间复杂度：O(n^2)。

C++代码代码如下：

```
```

```cpp
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        // 找出a + b + c = 0
        // a = nums[i], b = nums[left], c = nums[right]
        for (int i = 0; i < nums.size(); i++) {
            // 排序之后如果第一个元素已经大于零，那么无论如何组合都不可能凑成三元组，直接返回结果就可以了
            if (nums[i] > 0) {
                return result;
            }
            // 错误去重a方法，将会漏掉-1,-1,2 这种情况
            /*
            if (nums[i] == nums[i + 1]) {
                continue;
            }
            */
            // 正确去重a方法
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }
            int left = i + 1;
            int right = nums.size() - 1;
            while (right > left) {
                // 去重复逻辑如果放在这里，0, 0, 0 的情况，可能直接导致 right<=left 了，从而漏掉了 0,0,0 这种三元组
                /*
                while (right > left && nums[right] == nums[right - 1]) right--;
                while (right > left && nums[left] == nums[left + 1]) left++;
                */
                if (nums[i] + nums[left] + nums[right] > 0) right--;
                else if (nums[i] + nums[left] + nums[right] < 0) left++;
                else {
                    result.push_back(vector<int>{nums[i], nums[left], nums[right]});
                    // 去重逻辑应该放在找到一个三元组之后，对b 和 c去重
                    while (right > left && nums[right] == nums[right - 1]) right--;
                    while (right > left && nums[left] == nums[left + 1]) left++;

                    // 找到答案时，双指针同时收缩
                    right--;
                    left++;
                }
            }

        }
        return result;
    }
```

```
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(1)

去重逻辑的思考

a的去重

说道去重，其实主要考虑三个数的去重。 a, b ,c，对应的就是 nums[i]，nums[left]，nums[right]

a 如果重复了怎么办，a是nums里遍历的元素，那么应该直接跳过去。

但这里有一个问题，是判断 nums[i] 与 nums[i + 1]是否相同，还是判断 nums[i] 与 nums[i-1] 是否相同。

有同学可能想，这不都一样吗。

其实不一样！

都是和 nums[i]进行比较，是比较它的前一个，还是比较他的后一个。

如果我们的写法是 这样：

```
if (nums[i] == nums[i + 1]) { // 去重操作
    continue;
}
```

那就我们就把 三元组中出现重复元素的情况直接pass掉了。 例如{-1, -1 ,2} 这组数据，当遍历到第一个-1 的时候，判断 下一个也是-1，那这组数据就pass了。

我们要做的是 不能有重复的三元组，但三元组内的元素是可以重复的！

所以这里是有两个重复的维度。

那么应该这么写：

```
if (i > 0 && nums[i] == nums[i - 1]) {
    continue;
}
```

这么写就是当前使用 nums[i]，我们判断前一位是不是一样的元素，在看 {-1, -1 ,2} 这组数据，当遍历到 第一个 -1 的时候，只要前一位没有-1，那么 {-1, -1 ,2} 这组数据一样可以收录到 结果集里。

这是一个非常细节的思考过程。

b与c的去重

很多同学写本题的时候，去重的逻辑多加了 对right 和left 的去重：（代码中注释部分）

```
while (right > left) {
    if (nums[i] + nums[left] + nums[right] > 0) {
        right--;
        // 去重 right
        while (left < right && nums[right] == nums[right + 1]) right--;
    } else if (nums[i] + nums[left] + nums[right] < 0) {
        left++;
        // 去重 left
        while (left < right && nums[left] == nums[left - 1]) left++;
    } else {
    }
}
```

但细想一下，这种去重其实对提升程序运行效率是没有帮助的。

拿right去重为例，即使不加这个去重逻辑，依然根据 `while (right > left)` 和 `if (nums[i] + nums[left] + nums[right] > 0)` 去完成right-- 的操作。

多加了 `while (left < right && nums[right] == nums[right + 1]) right--;` 这一行代码，其实就是把需要执行的逻辑提前执行了，但并没有减少 判断的逻辑。

最直白的思考过程，就是right还是一个数一个数的减下去的，所以在哪里减的都是一样的。

所以这种去重 是可以不加的。 仅仅是 把去重的逻辑提前了而已。

思考题

既然三数之和可以使用双指针法，我们之前讲过的1.两数之和，可不可以使用双指针法呢?

如果不能，题意如何更改就可以使用双指针法呢？ **大家留言说出自己的想法吧!**

两数之和 就不能使用双指针法，因为1.两数之和要求返回的是索引下标， 而双指针法一定要排序，一旦排序之后原数组的索引就被改变了。

如果1.两数之和要求返回的是数值的话，就可以使用双指针法了。

> 一样的道理，能解决四数之和
> 那么五数之和、六数之和、N数之和呢?

9. 四数之和

力扣题目链接

题意：给定一个包含 n 个整数的数组 nums 和一个目标值 target，判断 nums 中是否存在四个元素 a，b，c 和 d ，使得 a + b + c + d 的值与 target 相等？找出所有满足条件且不重复的四元组。

注意：

答案中不可以包含重复的四元组。

示例：
给定数组 nums = [1, 0, -1, 0, -2, 2]，和 target = 0。
满足要求的四元组集合为：
[
  [-1, 0, 0, 1],
  [-2, -1, 1, 2],
  [-2, 0, 0, 2]
]

算法公开课

《代码随想录》算法视频公开课：难在去重和剪枝！| LeetCode：18. 四数之和，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

四数之和，和15.三数之和是一个思路，都是使用双指针法, 基本解法就是在15.三数之和 的基础上再套一层for循环。

但是有一些细节需要注意，例如： 不要判断`nums[k] > target` 就返回了，三数之和 可以通过 `nums[i] > 0` 就返回了，因为 0 已经是确定的数了，四数之和这道题目 target是任意值。比如：数组是`[-4, -3, -2, -1]`，`target`是`-10`，不能因为`-4 > -10`而跳过。但是我们依旧可以去做剪枝，逻辑变成`nums[i] > target && (nums[i] >=0 || target >= 0)`就可以了。

15.三数之和的双指针解法是一层for循环num[i]为确定值，然后循环内有left和right下标作为双指针，找到nums[i] + nums[left] + nums[right] == 0。

四数之和的双指针解法是两层for循环nums[k] + nums[i]为确定值，依然是循环内有left和right下标作为双指针，找出nums[k] + nums[i] + nums[left] + nums[right] == target的情况，三数之和的时间复杂度是O(n^2)，四数之和的时间复杂度是O(n^3) 。

那么一样的道理，五数之和、六数之和等等都采用这种解法。

对于15.三数之和双指针法就是将原本暴力O(n^3)的解法，降为O(n^2)的解法，四数之和的双指针解法就是将原本暴力O(n^4)的解法，降为O(n^3)的解法。

之前我们讲过哈希表的经典题目：454.四数相加II，相对于本题简单很多，因为本题是要求在一个集合中找出四个数相加等于target，同时四元组不能重复。

而454.四数相加II是四个独立的数组，只要找到A[i] + B[j] + C[k] + D[l] = 0就可以，不用考虑有重复的四个元素相加等于0的情况，所以相对于本题还是简单了不少！

我们来回顾一下，几道题目使用了双指针法。

双指针法将时间复杂度：O(n^2)的解法优化为 O(n)的解法。也就是降一个数量级，题目如下：

- 27.移除元素
- 15.三数之和
- 18.四数之和

链表相关双指针题目：

- 206.反转链表
- 19.删除链表的倒数第N个节点
- 面试题 02.07. 链表相交
- 142题.环形链表II

双指针法在字符串题目中还有很多应用，后面还会介绍到。

C++代码

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        for (int k = 0; k < nums.size(); k++) {
            // 剪枝处理
            if (nums[k] > target && nums[k] >= 0) {
              break; // 这里使用break，统一通过最后的return返回
            }
            // 对nums[k]去重
            if (k > 0 && nums[k] == nums[k - 1]) {
                continue;
            }
            for (int i = k + 1; i < nums.size(); i++) {
                // 2级剪枝处理
                if (nums[k] + nums[i] > target && nums[k] + nums[i] >= 0) {
                    break;
                }

                // 对nums[i]去重
                if (i > k + 1 && nums[i] == nums[i - 1]) {
                    continue;
                }
                int left = i + 1;
                int right = nums.size() - 1;
                while (right > left) {
                    // nums[k] + nums[i] + nums[left] + nums[right] > target 会溢出
                    if ((long) nums[k] + nums[i] + nums[left] + nums[right] > target) {
                        right--;
                    // nums[k] + nums[i] + nums[left] + nums[right] < target 会溢出
                    } else if ((long) nums[k] + nums[i] + nums[left] + nums[right]  <
target) {
                        left++;
                    } else {
                        result.push_back(vector<int>{nums[k], nums[i], nums[left],
nums[right]});
                        // 对nums[left]和nums[right]去重
                        while (right > left && nums[right] == nums[right - 1]) right--;
                        while (right > left && nums[left] == nums[left + 1]) left++;
```

```
                        // 找到答案时，双指针同时收缩
                        right--;
                        left++;
                    }
                }

            }
        }
        return result;
    }
};
```

- 时间复杂度: O(n^3)
- 空间复杂度: O(1)

补充

二级剪枝的部分：

```
if (nums[k] + nums[i] > target && nums[k] + nums[i] >= 0) {
    break;
}
```

可以优化为：

```
if (nums[k] + nums[i] > target && nums[i] >= 0) {
    break;
}
```

因为只要 nums[k] + nums[i] > target，那么 nums[i] 后面的数都是正数的话，就一定 不符合条件了。

不过这种剪枝 其实有点 小绕，大家能够理解 文章给的完整代码的剪枝 就够了。

> 哈希表总结篇如约而至

10. 哈希表总结篇

哈希表理论基础

在关于哈希表，你该了解这些！中，我们介绍了哈希表的基础理论知识，不同于枯燥的讲解，这里介绍了都是对刷题有帮助的理论知识点。

一般来说哈希表都是用来快速判断一个元素是否出现集合里。

对于哈希表，要知道**哈希函数**和**哈希碰撞**在哈希表中的作用.

哈希函数是把传入的key映射到符号表的索引上。

哈希碰撞处理有多个key映射到相同索引上时的情景，处理碰撞的普遍方式是拉链法和线性探测法。

接下来是常见的三种哈希结构：

- 数组
- set（集合）
- map（映射）

在C++语言中，set 和 map 都分别提供了三种数据结构，每种数据结构的底层实现和用途都有所不同，在关于哈希表，你该了解这些！中我给出了详细分析，这一知识点很重要！

例如什么时候用std::set，什么时候用std::multiset，什么时候用std::unordered_set，都是很有考究的。

只有对这些数据结构的底层实现很熟悉，才能灵活使用，否则很容易写出效率低下的程序。

哈希表经典题目

数组作为哈希表

一些应用场景就是为数组量身定做的。

在242.有效的字母异位词中，我们提到了数组就是简单的哈希表，但是数组的大小是受限的！

这道题目包含小写字母，那么使用数组来做哈希最合适不过。

在383.赎金信中同样要求只有小写字母，那么就给我们浓浓的暗示，用数组！

本题和242.有效的字母异位词很像，242.有效的字母异位词是求 字符串a 和 字符串b 是否可以相互组成，在383.赎金信中是求字符串a能否组成字符串b，而不用管字符串b 能不能组成字符串a。

一些同学可能想，用数组干啥，都用map不就完事了。

上面两道题目用map确实可以，但使用map的空间消耗要比数组大一些，因为map要维护红黑树或者符号表，而且还要做哈希函数的运算。所以数组更加简单直接有效！

set作为哈希表

在349. 两个数组的交集中我们给出了什么时候用数组就不行了，需要用set。

这道题目没有限制数值的大小，就无法使用数组来做哈希表了。

主要因为如下两点：

- 数组的大小是有限的，受到系统栈空间（不是数据结构的栈）的限制。
- 如果数组空间够大，但哈希值比较少、特别分散、跨度非常大，使用数组就造成空间的极大浪费。

所以此时一样的做映射的话，就可以使用set了。

关于set，C++ 给提供了如下三种可用的数据结构：（详情请看关于哈希表，你该了解这些！）

- std::set

- std::multiset
- std::unordered_set

std::set和std::multiset底层实现都是红黑树，std::unordered_set的底层实现是哈希， 使用unordered_set 读写效率是最高的，本题并不需要对数据进行排序，而且还不要让数据重复，所以选择unordered_set。

在202.快乐数中，我们再次使用了unordered_set来判断一个数是否重复出现过。

map作为哈希表

在1.两数之和中map正式登场。

来说一说：使用数组和set来做哈希法的局限。

- 数组的大小是受限制的，而且如果元素很少，而哈希值太大会造成内存空间的浪费。
- set是一个集合，里面放的元素只能是一个key，而两数之和这道题目，不仅要判断y是否存在而且还要记录y的下标位置，因为要返回x 和 y的下标。所以set 也不能用。

map是一种`<key, value>`的结构，本题可以用key保存数值，用value在保存数值所在的下标。所以使用map最为合适。

C++提供如下三种map：：（详情请看关于哈希表，你该了解这些！）

- std::map
- std::multimap
- std::unordered_map

std::unordered_map 底层实现为哈希，std::map 和std::multimap 的底层实现是红黑树。

同理，std::map 和std::multimap 的key也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解），1.两数之和中并不需要key有序，选择std::unordered_map 效率更高！

在454.四数相加中我们提到了其实需要哈希的地方都能找到map的身影。

本题咋眼一看好像和18. 四数之和，15.三数之和差不多，其实差很多！

关键差别是本题为四个独立的数组，只要找到A[i] + B[j] + C[k] + D[l] = 0就可以，不用考虑重复问题，而18. 四数之和，15.三数之和是一个数组（集合）里找到和为0的组合，可就难很多了！

用哈希法解决了两数之和，很多同学会感觉用哈希法也可以解决三数之和，四数之和。

其实是可以解决，但是非常麻烦，需要去重导致代码效率很低。

在15.三数之和中我给出了哈希法和双指针两个解法，大家就可以体会到，使用哈希法还是比较麻烦的。

所以18. 四数之和，15.三数之和都推荐使用双指针法！

总结

对于哈希表的知识相信很多同学都知道，但是没有成体系。

本篇我们从哈希表的理论基础到数组、set和map的经典应用，把哈希表的整个全貌完整的呈现给大家。

同时也强调虽然map是万能的，详细介绍了什么时候用数组，什么时候用set。

相信通过这个总结篇，大家可以对哈希表有一个全面的了解。

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

> 打基础的时候，不要太迷恋于库函数。

1.反转字符串

力扣题目链接

编写一个函数，其作用是将输入的字符串反转过来。输入字符串以字符数组 char[] 的形式给出。

不要给另外的数组分配额外的空间，你必须原地修改输入数组、使用 O(1) 的额外空间解决这一问题。

你可以假设数组中的所有字符都是 ASCII 码表中的可打印字符。

示例 1：
输入：["h","e","l","l","o"]
输出：["o","l","l","e","h"]

示例 2：
输入：["H","a","n","n","a","h"]
输出：["h","a","n","n","a","H"]

算法公开课

《代码随想录》算法视频公开课：字符串基础操作！ | LeetCode：344.反转字符串，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

先说一说题外话：

对于这道题目一些同学直接用C++里的一个库函数 reverse，调一下直接完事了， 相信每一门编程语言都有这样的库函数。

如果这么做题的话，这样大家不会清楚反转字符串的实现原理了。

但是也不是说库函数就不能用，是要分场景的。

如果在现场面试中，我们什么时候使用库函数，什么时候不要用库函数呢?

如果题目关键的部分直接用库函数就可以解决，建议不要使用库函数。

毕竟面试官一定不是考察你对库函数的熟悉程度， 如果使用python和java 的同学更需要注意这一点，因为python、java提供的库函数十分丰富。

如果库函数仅仅是 解题过程中的一小部分，并且你已经很清楚这个库函数的内部实现原理的话，可以考虑使用库函数。

建议大家平时在leetcode上练习算法的时候本着这样的原则去练习，这样才有助于我们对算法的理解。

不要沉迷于使用库函数一行代码解决题目之类的技巧，不是说这些技巧不好，而是说这些技巧可以用来娱乐一下。

真正自己写的时候，要保证理解可以实现是相应的功能。

接下来再来讲一下如何解决反转字符串的问题。

大家应该还记得，我们已经讲过了206.反转链表。

在反转链表中，使用了双指针的方法。

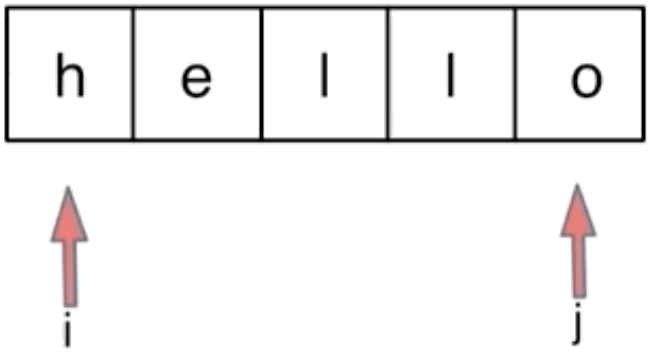
那么反转字符串依然是使用双指针的方法，只不过对于字符串的反转，其实要比链表简单一些。

因为字符串也是一种数组，所以元素在内存中是连续分布，这就决定了反转链表和反转字符串方式上还是有所差异的。

如果对数组和链表原理不清楚的同学，可以看这两篇，关于链表，你该了解这些！，必须掌握的数组理论知识。

对于字符串，我们定义两个指针（也可以说是索引下标），一个从字符串前面，一个从字符串后面，两个指针同时向中间移动，并交换元素。

以字符串`hello`为例，过程如下：



不难写出如下C++代码:

```
void reverseString(vector<char>& s) {
    for (int i = 0, j = s.size() - 1; i < s.size()/2; i++, j--) {
        swap(s[i],s[j]);
    }
}
```

循环里只要做交换s[i] 和s[j]操作就可以了，那么我这里使用了swap 这个库函数。大家可以使用。

因为相信大家都知道交换函数如何实现，而且这个库函数仅仅是解题中的一部分， 所以这里使用库函数也是可以的。

swap可以有两种实现。

一种就是常见的交换数值:

```
int tmp = s[i];
s[i] = s[j];
s[j] = tmp;
```

一种就是通过位运算:

```
s[i] ^= s[j];
s[j] ^= s[i];
s[i] ^= s[j];
```

这道题目还是比较简单的，但是我正好可以通过这道题目说一说在刷题的时候，使用库函数的原则。

如果题目关键的部分直接用库函数就可以解决，建议不要使用库函数。

如果库函数仅仅是 解题过程中的一小部分，并且你已经很清楚这个库函数的内部实现原理的话，可以考虑使用库函数。

本着这样的原则，我没有使用reverse库函数，而使用swap库函数。

在字符串相关的题目中，库函数对大家的诱惑力是非常大的，因为会有各种反转，切割取词之类的操作，这也是为什么字符串的库函数这么丰富的原因。

相信大家本着我所讲述的原则来做字符串相关的题目，在选择库函数的角度上会有所原则，也会有所收获。

C++代码如下：

```
class Solution {
public:
    void reverseString(vector<char>& s) {
        for (int i = 0, j = s.size() - 1; i < s.size()/2; i++, j--) {
            swap(s[i],s[j]);
        }
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

> 简单的反转还不够，我要花式反转

2. 反转字符串II

力扣题目链接

给定一个字符串 s 和一个整数 k，从字符串开头算起, 每计数至 2k 个字符，就反转这 2k 个字符中的前 k 个字符。

如果剩余字符少于 k 个，则将剩余字符全部反转。

如果剩余字符小于 2k 但大于或等于 k 个，则反转前 k 个字符，其余字符保持原样。

示例:

输入: s = "abcdefg", k = 2
输出: "bacdfeg"

算法公开课

《代码随想录》算法视频公开课：字符串操作进阶！ | LeetCode：541. 反转字符串II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目其实也是模拟，实现题目中规定的反转规则就可以了。

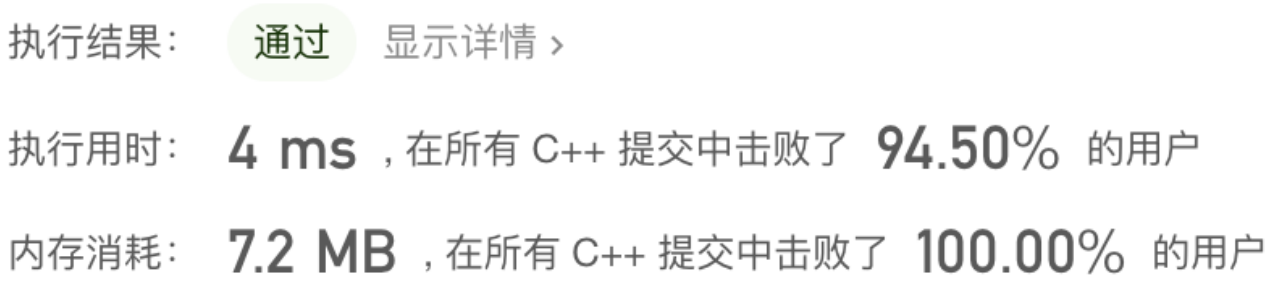
一些同学可能为了处理逻辑：每隔2k个字符的前k的字符，写了一堆逻辑代码或者再搞一个计数器，来统计2k，再统计前k个字符。

其实在遍历字符串的过程中，只要让 i += (2 * k)，i 每次移动 2 * k 就可以了，然后判断是否需要有反转的区间。

因为要找的也就是每2 * k 区间的起点，这样写，程序会高效很多。

所以当需要固定规律一段一段去处理字符串的时候，要想想在在for循环的表达式上做做文章。

性能如下：



那么这里具体反转的逻辑我们要不要使用库函数呢，其实用不用都可以，使用reverse来实现反转也没毛病，毕竟不是解题关键部分。

使用C++库函数reverse的版本如下：

```
class Solution {
public:
    string reverseStr(string s, int k) {
        for (int i = 0; i < s.size(); i += (2 * k)) {
            // 1. 每隔 2k 个字符的前 k 个字符进行反转
            // 2. 剩余字符小于 2k 但大于或等于 k 个，则反转前 k 个字符
            if (i + k <= s.size()) {
                reverse(s.begin() + i, s.begin() + i + k );
            } else {
                // 3. 剩余字符少于 k 个，则将剩余字符全部反转。
                reverse(s.begin() + i, s.end());
            }
        }
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

那么我们也可以实现自己的reverse函数，其实和题目344. 反转字符串道理是一样的。

下面我实现的reverse函数区间是左闭右闭区间，代码如下：

```
class Solution {
public:
    void reverse(string& s, int start, int end) {
        for (int i = start, j = end; i < j; i++, j--) {
            swap(s[i], s[j]);
        }
    }
    string reverseStr(string s, int k) {
        for (int i = 0; i < s.size(); i += (2 * k)) {
            // 1. 每隔 2k 个字符的前 k 个字符进行反转
            // 2. 剩余字符小于 2k 但大于或等于 k 个，则反转前 k 个字符
            if (i + k <= s.size()) {
                reverse(s, i, i + k - 1);
                continue;
            }
            // 3. 剩余字符少于 k 个，则将剩余字符全部反转。
            reverse(s, i, s.size() - 1);
        }
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)或O(n), 取决于使用的语言中字符串是否可以修改.

另一种思路的解法

```
class Solution {
public:
    string reverseStr(string s, int k) {
        int n = s.size(),pos = 0;
        while(pos < n){
            //剩余字符串大于等于k的情况
            if(pos + k < n) reverse(s.begin() + pos, s.begin() + pos + k);
            //剩余字符串不足k的情况
            else reverse(s.begin() + pos,s.end());
            pos += 2 * k;
        }
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

3.剑指Offer 05.替换空格

[力扣题目链接]

请实现一个函数，把字符串 s 中的每个空格替换成"%20"。

示例 1：
输入：s = "We are happy."
输出："We%20are%20happy."

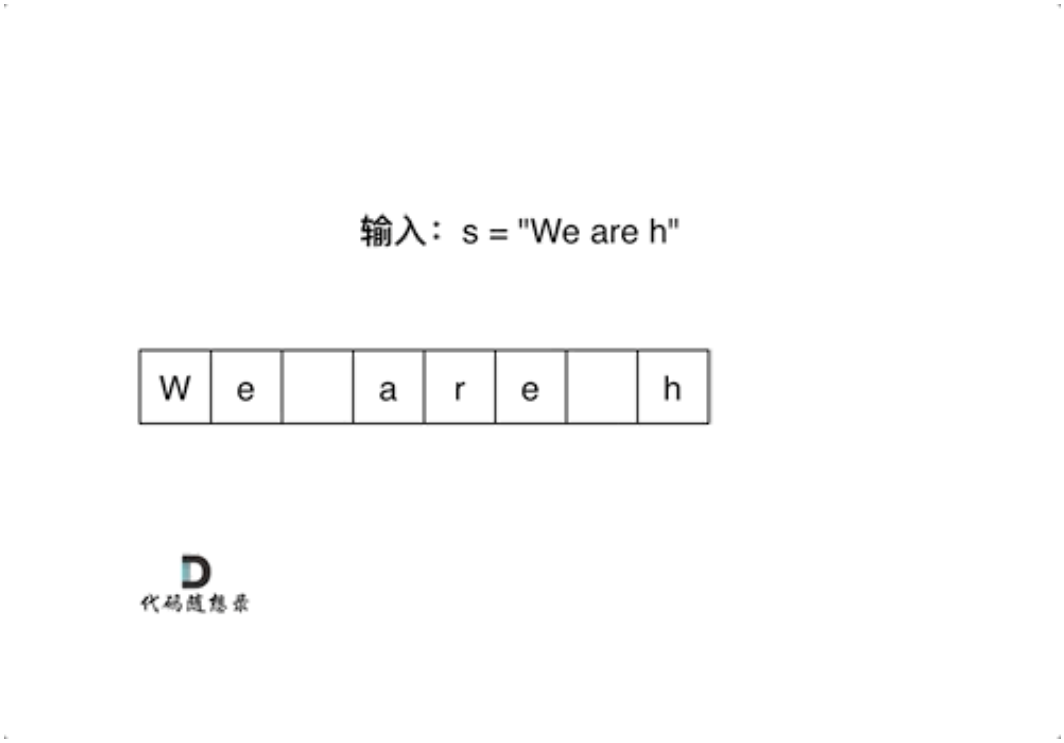
思路

如果想把这道题目做到极致，就不要只用额外的辅助空间了！

首先扩充数组到每个空格替换成"%20"之后的大小。

然后从后向前替换空格，也就是双指针法，过程如下：

i指向新长度的末尾，j指向旧长度的末尾。



有同学问了，为什么要从后向前填充，从前向后填充不行么？

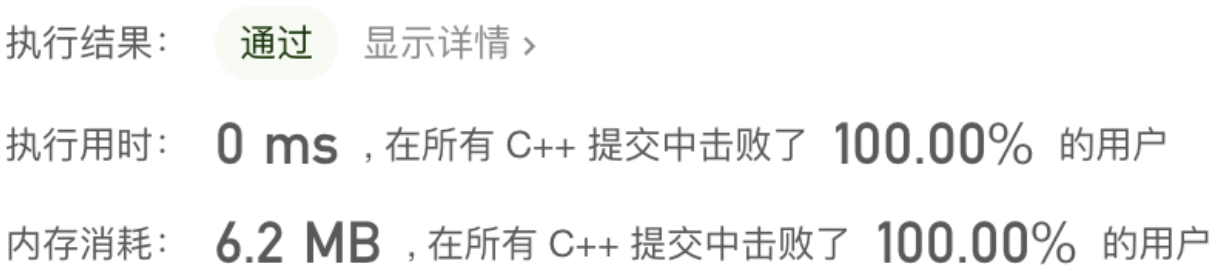
从前向后填充就是O(n^2)的算法了，因为每次添加元素都要将添加元素之后的所有元素向后移动。

其实很多数组填充类的问题，都可以先预先给数组扩容带填充后的大小，然后在从后向前进行操作。

这么做有两个好处：

1. 不用申请新数组。
2. 从后向前填充元素，避免了从前向后填充元素时，每次添加元素都要将添加元素之后的所有元素向后移动的问题。

时间复杂度，空间复杂度均超过100%的用户。



C++代码如下：

```
class Solution {
public:
    string replaceSpace(string s) {
        int count = 0; // 统计空格的个数
        int sOldSize = s.size();
        for (int i = 0; i < s.size(); i++) {
            if (s[i] == ' ') {
                count++;
            }
        }
        // 扩充字符串s的大小，也就是每个空格替换成"%20"之后的大小
        s.resize(s.size() + count * 2);
        int sNewSize = s.size();
        // 从后先前将空格替换为"%20"
        for (int i = sNewSize - 1, j = sOldSize - 1; j < i; i--, j--) {
            if (s[j] != ' ') {
                s[i] = s[j];
            } else {
                s[i] = '0';
                s[i - 1] = '2';
                s[i - 2] = '%';
                i -= 2;
            }
        }
        return s;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

此时算上本题，我们已经做了七道双指针相关的题目了分别是：

- 27.移除元素
- 15.三数之和
- 18.四数之和
- 206.翻转链表

- 142.环形链表II
- 344.反转字符串

拓展

这里也给大家拓展一下字符串和数组有什么差别，

字符串是若干字符组成的有限序列，也可以理解为是一个字符数组，但是很多语言对字符串做了特殊的规定，接下来我来说一说C/C++中的字符串。

在C语言中，把一个字符串存入一个数组时，也把结束符 '\0'存入数组，并以此作为该字符串是否结束的标志。

例如这段代码：

```
char a[5] = "asd";
for (int i = 0; a[i] != '\0'; i++) {
}
```

在C++中，提供一个string类，string类会提供 size接口，可以用来判断string类字符串是否结束，就不用'\0'来判断是否结束。

例如这段代码:

```
string a = "asd";
for (int i = 0; i < a.size(); i++) {
}
```

那么vector< char > 和 string 又有什么区别呢?

其实在基本操作上没有区别，但是 string提供更多的字符串处理的相关接口，例如string 重载了+，而vector却没有。

所以想处理字符串，我们还是会定义一个string类型。

> 综合考察字符串操作的好题。

4.翻转字符串里的单词

力扣题目链接

给定一个字符串，逐个翻转字符串中的每个单词。

示例 1：
输入: "the sky is blue"
输出: "blue is sky the"

示例 2：
输入: " hello world! "
输出: "world! hello"
解释: 输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。

示例 3：
输入: "a good   example"
输出: "example good a"
解释: 如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。

算法公开课

[《代码随想录》算法视频公开课](#)：[字符串复杂操作拿捏了！ | LeetCode:151.翻转字符串里的单词](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

这道题目可以说是综合考察了字符串的多种操作。

一些同学会使用split库函数，分隔单词，然后定义一个新的string字符串，最后再把单词倒序相加，那么这道题题目就是一道水题了，失去了它的意义。

所以这里我还是提高一下本题的难度：**不要使用辅助空间，空间复杂度要求为O(1)。**

不能使用辅助空间之后，那么只能在原字符串上下功夫了。

想一下，我们将整个字符串都反转过来，那么单词的顺序指定是倒序了，只不过单词本身也倒序了，那么再把单词反转一下，单词不就正过来了。

所以解题思路如下：

- 移除多余空格
- 将整个字符串反转
- 将每个单词反转

举个例子，源字符串为："the sky is blue "

- 移除多余空格 : "the sky is blue"
- 字符串反转："eulb si yks eht"
- 单词反转："blue is sky the"

这样我们就完成了翻转字符串里的单词。

思路很明确了，我们说一说代码的实现细节，就拿移除多余空格来说，一些同学会上来写如下代码：

```
void removeExtraSpaces(string& s) {
    for (int i = s.size() - 1; i > 0; i--) {
        if (s[i] == s[i - 1] && s[i] == ' ') {
            s.erase(s.begin() + i);
        }
    }
```

```
    // 删除字符串最后面的空格
    if (s.size() > 0 && s[s.size() - 1] == ' ') {
        s.erase(s.begin() + s.size() - 1);
    }
    // 删除字符串最前面的空格
    if (s.size() > 0 && s[0] == ' ') {
        s.erase(s.begin());
    }
}
```

逻辑很简单，从前向后遍历，遇到空格了就erase。

如果不仔细琢磨一下erase的时间复杂度，还以为以上的代码是O(n)的时间复杂度呢。

想一下真正的时间复杂度是多少，一个erase本来就是O(n)的操作。

erase操作上面还套了一个for循环，那么以上代码移除冗余空格的代码时间复杂度为O(n^2)。

那么使用双指针法来去移除空格，最后resize（重新设置）一下字符串的大小，就可以做到O(n)的时间复杂度。

```
//版本一
void removeExtraSpaces(string& s) {
    int slowIndex = 0, fastIndex = 0; // 定义快指针，慢指针
    // 去掉字符串前面的空格
    while (s.size() > 0 && fastIndex < s.size() && s[fastIndex] == ' ') {
        fastIndex++;
    }
    for (; fastIndex < s.size(); fastIndex++) {
        // 去掉字符串中间部分的冗余空格
        if (fastIndex - 1 > 0
                && s[fastIndex - 1] == s[fastIndex]
                && s[fastIndex] == ' ') {
            continue;
        } else {
            s[slowIndex++] = s[fastIndex];
        }
    }
    if (slowIndex - 1 > 0 && s[slowIndex - 1] == ' ') { // 去掉字符串末尾的空格
        s.resize(slowIndex - 1);
    } else {
        s.resize(slowIndex); // 重新设置字符串大小
    }
}
```

有的同学可能发现用erase来移除空格，在leetcode上性能也还行。主要是以下几点；：

1. leetcode上的测试集里，字符串的长度不够长，如果足够长，性能差距会非常明显。
2. leetcode的测程序耗时不是很准确的。

版本一的代码是一般的思考过程，就是 先移除字符串前的空格，再移除中间的，再移除后面部分。

不过其实还可以优化，这部分和27.移除元素的逻辑是一样一样的，本题是移除空格，而 27.移除元素 就是移除元素。

所以代码可以写的很精简，大家可以看 如下 代码 removeExtraSpaces 函数的实现：

```
// 版本二
void removeExtraSpaces(string& s) {//去除所有空格并在相邻单词之间添加空格，快慢指针。
    int slow = 0;   //整体思想参考https://programmercarl.com/0027.移除元素.html
    for (int i = 0; i < s.size(); ++i) { //
        if (s[i] != ' ') { //遇到非空格就处理，即删除所有空格。
            if (slow != 0) s[slow++] = ' '; //手动控制空格，给单词之间添加空格。slow != 0说明不是第一个单词，需要在单词前添加空格。
            while (i < s.size() && s[i] != ' ') { //补上该单词，遇到空格说明单词结束。
                s[slow++] = s[i++];
            }
        }
    }
    s.resize(slow); //slow的大小即为去除多余空格后的大小。
}
```

如果以上代码看不懂，建议先把 27.移除元素这道题目做了，或者看视频讲解：数组中移除元素并不容易！ LeetCode：27. 移除元素 。

此时我们已经实现了removeExtraSpaces函数来移除冗余空格。

还要实现反转字符串的功能，支持反转字符串子区间，这个实现我们分别在344.反转字符串和541.反转字符串II里已经讲过了。

代码如下：

```
// 反转字符串s中左闭右闭的区间[start, end]
void reverse(string& s, int start, int end) {
    for (int i = start, j = end; i < j; i++, j--) {
        swap(s[i], s[j]);
    }
}
```

整体代码如下：

```
class Solution {
public:
    void reverse(string& s, int start, int end){ //翻转，区间写法：左闭右闭 []
        for (int i = start, j = end; i < j; i++, j--) {
            swap(s[i], s[j]);
        }
    }

    void removeExtraSpaces(string& s) {//去除所有空格并在相邻单词之间添加空格，快慢指针。
        int slow = 0;   //整体思想参考https://programmercarl.com/0027.移除元素.html
```

```
        for (int i = 0; i < s.size(); ++i) { //
            if (s[i] != ' ') { //遇到非空格就处理，即删除所有空格。
                if (slow != 0) s[slow++] = ' '; //手动控制空格，给单词之间添加空格。slow != 0
说明不是第一个单词，需要在单词前添加空格。
                while (i < s.size() && s[i] != ' ') { //补上该单词，遇到空格说明单词结束。
                    s[slow++] = s[i++];
                }
            }
        }
        s.resize(slow); //slow的大小即为去除多余空格后的大小。
    }

    string reverseWords(string s) {
        removeExtraSpaces(s); //去除多余空格，保证单词之间之只有一个空格，且字符串首尾没空格。
        reverse(s, 0, s.size() - 1);
        int start = 0; //removeExtraSpaces后保证第一个单词的开始下标一定是0。
        for (int i = 0; i <= s.size(); ++i) {
            if (i == s.size() || s[i] == ' ') { //到达空格或者串尾，说明一个单词结束。进行翻
转。
                reverse(s, start, i - 1); //翻转，注意是左闭右闭 []的翻转。
                start = i + 1; //更新下一个单词的开始下标start
            }
        }
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1) 或 O(n)，取决于语言中字符串是否可变

> 反转个字符串还有这么多用处?

5. 剑指Offer58-II.左旋转字符串

力扣题目链接

字符串的左旋转操作是把字符串前面的若干个字符转移到字符串的尾部。请定义一个函数实现字符串左旋转操作的功能。比如，输入字符串"abcdefg"和数字2，该函数将返回左旋转两位得到的结果"cdefgab"。

示例 1：
输入: s = "abcdefg", k = 2
输出: "cdefgab"

示例 2：
输入: s = "lrloseumgh", k = 6
输出: "umghlrlose"

限制：
1 <= k < s.length <= 10000

思路

为了让本题更有意义，提升一下本题难度：**不能申请额外空间，只能在本串上操作。**

不能使用额外空间的话，模拟在本串操作要实现左旋转字符串的功能还是有点困难的。

那么我们可以想一下上一题目字符串：花式反转还不够！中讲过，使用整体反转+局部反转就可以实现反转单词顺序的目的。

这道题目也非常类似，依然可以通过局部反转+整体反转 达到左旋转的目的。

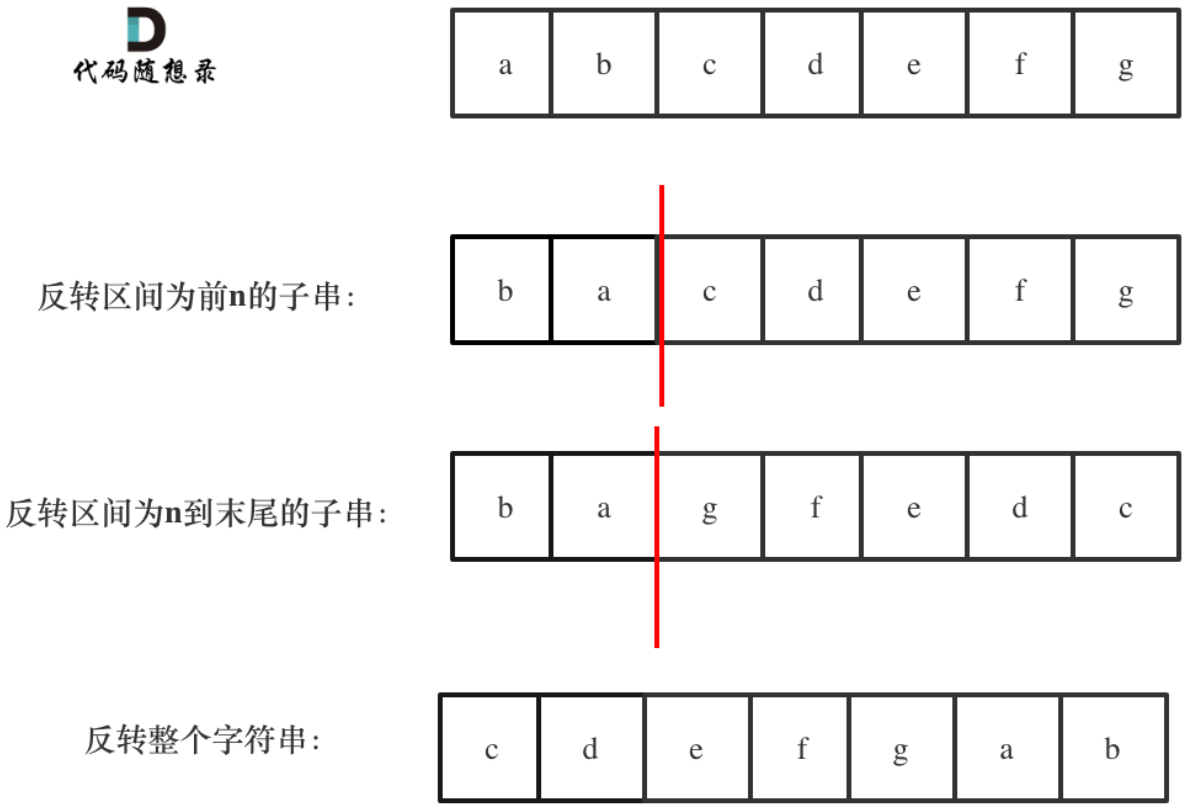
具体步骤为：

1. 反转区间为前n的子串
2. 反转区间为n到末尾的子串
3. 反转整个字符串

最后就可以达到左旋n的目的，而不用定义新的字符串，完全在本串上操作。

例如：示例1中 输入：字符串abcdefg，n=2

如图：



最终得到左旋2个单元的字符串：cdefgab

思路明确之后，那么代码实现就很简单了

C++代码如下：

```
class Solution {
public:
    string reverseLeftWords(string s, int n) {
        reverse(s.begin(), s.begin() + n);
        reverse(s.begin() + n, s.end());
        reverse(s.begin(), s.end());
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度：O(1)

是不是发现这代码也太简单了，哈哈。

总结

此时我们已经反转好多次字符串了，来一起回顾一下吧。

在这篇文章344.反转字符串，第一次讲到反转一个字符串应该怎么做，使用了双指针法。

然后发现541. 反转字符串II，这里开始给反转加上了一些条件，当需要固定规律一段一段去处理字符串的时候，要想想在for循环的表达式上做做文章。

后来在151.翻转字符串里的单词中，要对一句话里的单词顺序进行反转，发现先整体反转再局部反转 是一个很妙的思路。

最后再讲到本题，本题则是先局部反转再 整体反转，与151.翻转字符串里的单词类似，但是也是一种新的思路。

好了，反转字符串一共就介绍到这里，相信大家此时对反转字符串的常见操作已经很了解了。

题外话

一些同学热衷于使用substr，来做这道题。
其实使用substr 和 反转 时间复杂度是一样的 ，都是O(n)，但是使用substr申请了额外空间，所以空间复杂度是O(n)，而反转方法的空间复杂度是O(1)。

如果想让这套题目有意义，就不要申请额外空间。

> 在一个串中查找是否出现过另一个串，这是KMP的看家本领。

6. 实现 strStr()

力扣题目链接

实现 strStr() 函数。

给定一个 haystack 字符串和一个 needle 字符串，在 haystack 字符串中找出 needle 字符串出现的第一个位置 (从0开始)。如果不存在，则返回 -1。

示例 1:
输入: haystack = "hello", needle = "ll"
输出: 2

示例 2:
输入: haystack = "aaaaa", needle = "bba"
输出: -1

说明:
当 needle 是空字符串时，我们应当返回什么值呢？这是一个在面试中很好的问题。
对于本题而言，当 needle 是空字符串时我们应当返回 0 。这与C语言的 strstr() 以及 Java的 indexOf() 定义相符。

思路

本题是KMP 经典题目。

以下文字如果看不进去，可以看我的B站视频：

- 帮你把KMP算法学个通透！B站（理论篇）
- 帮你把KMP算法学个通透！（求next数组代码篇）

KMP的经典思想就是:**当出现字符串不匹配时，可以记录一部分之前已经匹配的文本内容，利用这些信息避免从头再去做匹配。**

本篇将以如下顺序来讲解KMP，

- 什么是KMP
- KMP有什么用
- 什么是前缀表
- 为什么一定要用前缀表
- 如何计算前缀表
- 前缀表与next数组
- 使用next数组来匹配
- 时间复杂度分析
- 构造next数组
- 使用next数组来做匹配
- 前缀表统一减一 C++代码实现
- 前缀表（不减一）C++实现
- 总结

读完本篇可以顺便把leetcode上28.实现strStr()题目做了。

什么是KMP

说到KMP，先说一下KMP这个名字是怎么来的，为什么叫做KMP呢。

因为是由这三位学者发明的：Knuth，Morris和Pratt，所以取了三位学者名字的首字母。所以叫做KMP

KMP有什么用

KMP主要应用在字符串匹配上。

KMP的主要思想是**当出现字符串不匹配时，可以知道一部分之前已经匹配的文本内容，可以利用这些信息避免从头再去做匹配了。**

所以如何记录已经匹配的文本内容，是KMP的重点，也是next数组肩负的重任。

其实KMP的代码不好理解，一些同学甚至直接把KMP代码的模板背下来。

没有彻底搞懂，懵懵懂懂就把代码背下来太容易忘了。

不仅面试的时候可能写不出来，如果面试官问：**next数组里的数字表示的是什么，为什么这么表示?**

估计大多数候选人都是懵逼的。

下面Carl就带大家把KMP的精髓，next数组弄清楚。

什么是前缀表

写过KMP的同学，一定都写过next数组，那么这个next数组究竟是个啥呢?

next数组就是一个前缀表（prefix table）。

前缀表有什么作用呢?

前缀表是用来回退的，它记录了模式串与主串(文本串)不匹配的时候，模式串应该从哪里开始重新匹配。

为了清楚地了解前缀表的来历，我们来举一个例子：

要在文本串：aabaabaafa 中查找是否出现过一个模式串：aabaaf。

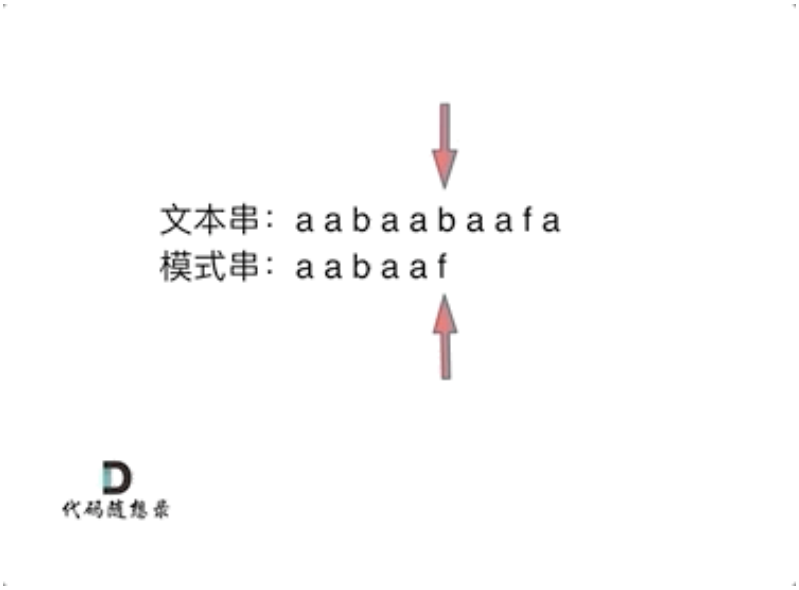
请记住文本串和模式串的作用，对于理解下文很重要，要不然容易看懵。所以说三遍：

要在文本串：aabaabaafa 中查找是否出现过一个模式串：aabaaf。

要在文本串：aabaabaafa 中查找是否出现过一个模式串：aabaaf。

要在文本串：aabaabaafa 中查找是否出现过一个模式串：aabaaf。

如动画所示：



动画里，我特意把 子串`aa` 标记上了，这是有原因的，大家先注意一下，后面还会说到。

可以看出，文本串中第六个字符b 和 模式串的第六个字符f，不匹配了。如果暴力匹配，发现不匹配，此时就要从头匹配了。

但如果使用前缀表，就不会从头匹配，而是从上次已经匹配的内容开始匹配，找到了模式串中第三个字符b继续开始匹配。

此时就要问了**前缀表是如何记录的呢?**

首先要知道前缀表的任务是当前位置匹配失败，找到之前已经匹配上的位置，再重新匹配，此也意味着在某个字符失配时，前缀表会告诉你下一步匹配中，模式串应该跳到哪个位置。

那么什么是前缀表：**记录下标i之前（包括i）的字符串中，有多大长度的相同前缀后缀。**

最长公共前后缀

文章中字符串的**前缀是指不包含最后一个字符的所有以第一个字符开头的连续子串**。

后缀是指不包含第一个字符的所有以最后一个字符结尾的连续子串。

正确理解什么是前缀什么是后缀很重要!

那么网上清一色都说 “kmp 最长公共前后缀” 又是什么回事呢?

我查了一遍 算法导论 和 算法4里KMP的章节，都没有提到 “最长公共前后缀”这个词，也不知道从哪里来了，我理解是用“最长相等前后缀” 更准确一些。

因为前缀表要求的就是相同前后缀的长度。

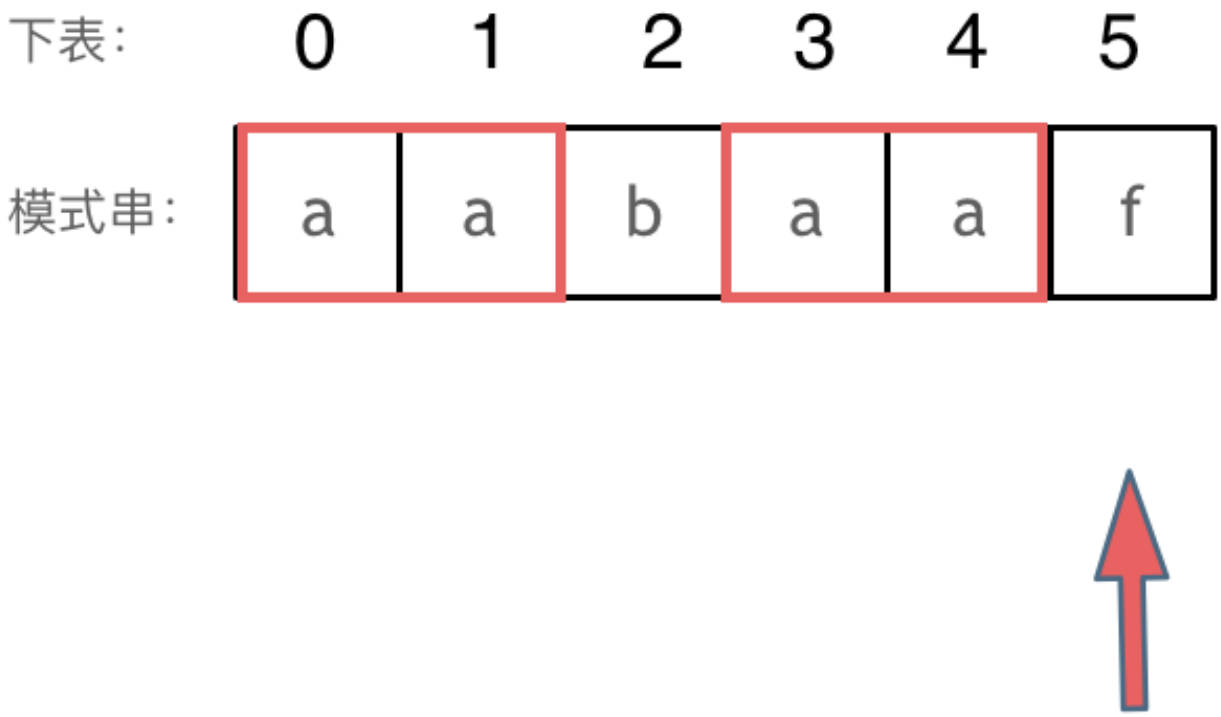
而最长公共前后缀里面的“公共”，更像是说前缀和后缀公共的长度。这其实并不是前缀表所需要的。

所以字符串a的最长相等前后缀为0。
字符串aa的最长相等前后缀为1。
字符串aaa的最长相等前后缀为2。
等等.....。

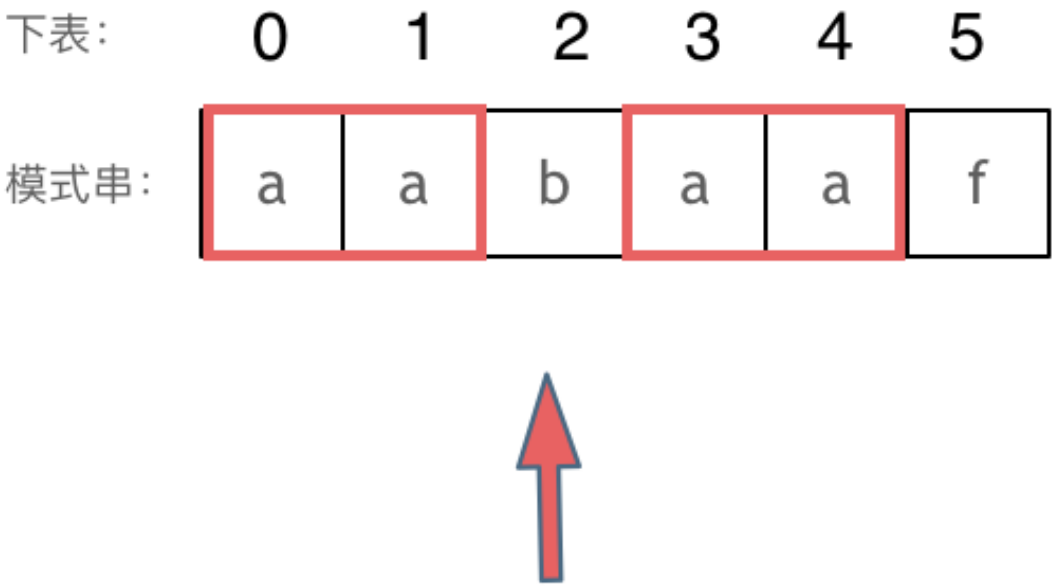
为什么一定要用前缀表

这就是前缀表，那为啥就能告诉我们 上次匹配的位置，并跳过去呢?

回顾一下，刚刚匹配的过程在下标5的地方遇到不匹配，模式串是指向f，如图：



然后就找到了下标2，指向b，继续匹配：如图：



以下这句话，对于理解为什么使用前缀表可以告诉我们匹配失败之后跳到哪里重新匹配 非常重要！

下标5之前这部分的字符串（也就是字符串aabaa）的最长相等的前缀 和 后缀字符串是 子字符串aa ，因为找到了最长相等的前缀和后缀，匹配失败的位置是后缀子串的后面，那么我们找到与其相同的前缀的后面重新匹配就可以了。

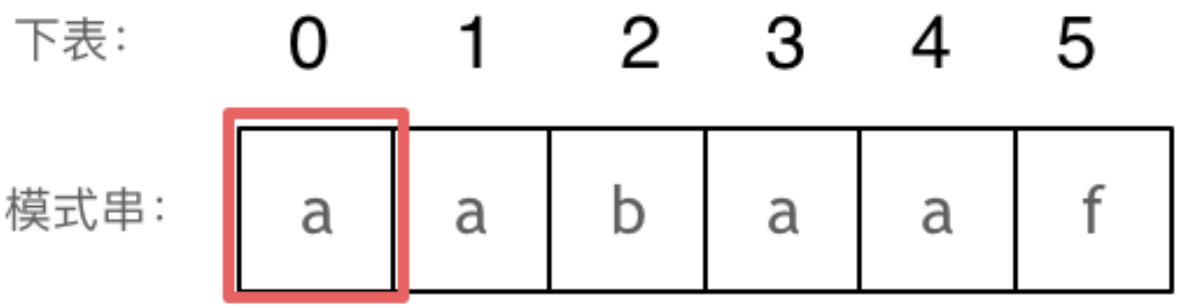
所以前缀表具有告诉我们当前位置匹配失败，跳到之前已经匹配过的地方的能力。

很多介绍KMP的文章或者视频并没有把为什么要用前缀表？这个问题说清楚，而是直接默认使用前缀表。

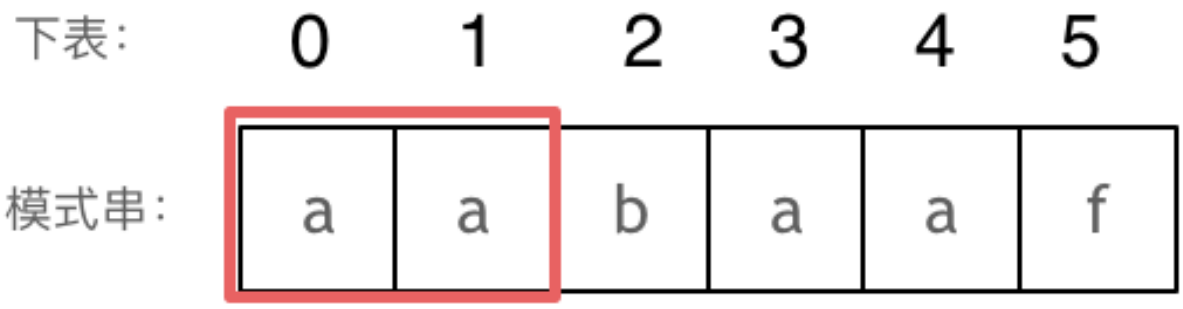
如何计算前缀表

接下来就要说一说怎么计算前缀表。

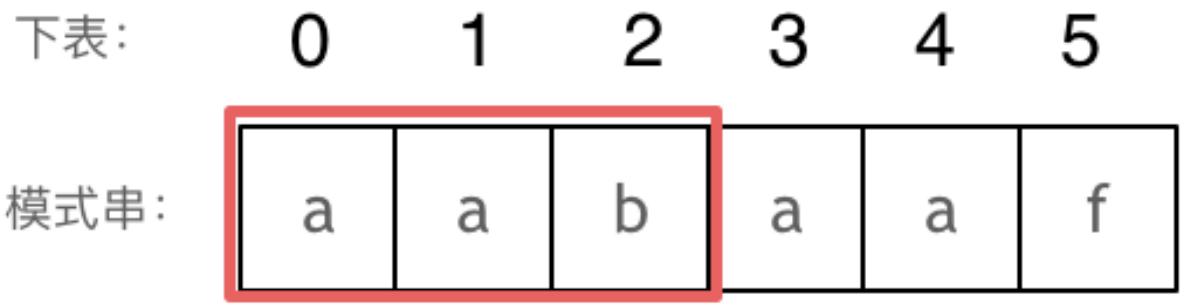
如图：



长度为前1个字符的子串`a`，最长相同前后缀的长度为0。（注意字符串的**前缀是指不包含最后一个字符的所有以第一个字符开头的连续子串；后缀是指不包含第一个字符的所有以最后一个字符结尾的连续子串。**）



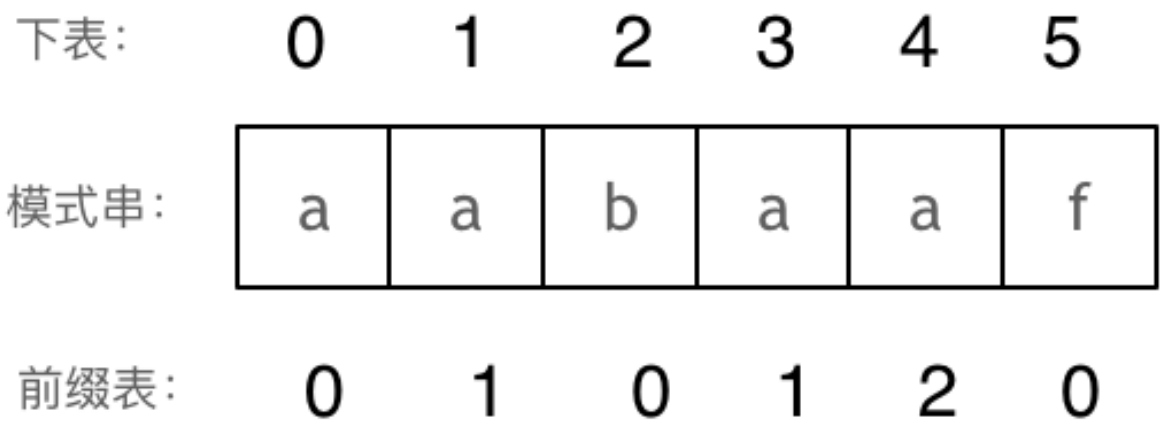
长度为前2个字符的子串`aa`，最长相同前后缀的长度为1。



长度为前3个字符的子串`aab`，最长相同前后缀的长度为0。

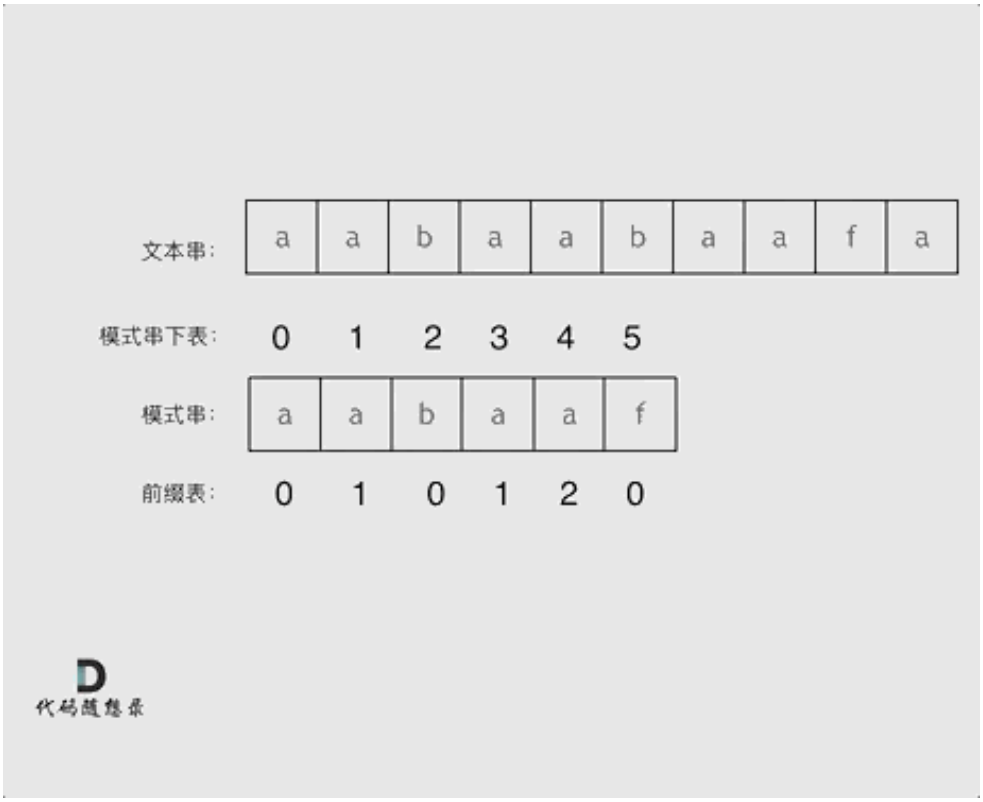
以此类推：
长度为前4个字符的子串`aaba`，最长相同前后缀的长度为1。
长度为前5个字符的子串`aabaa`，最长相同前后缀的长度为2。
长度为前6个字符的子串`aabaaf`，最长相同前后缀的长度为0。

那么把求得的最长相同前后缀的长度就是对应前缀表的元素，如图：



可以看出模式串与前缀表对应位置的数字表示的就是：**下标i之前（包括i）的字符串中，有多大长度的相同前缀后缀。**

再来看一下如何利用 前缀表找到 当字符不匹配的时候应该指针应该移动的位置。如动画所示：



找到的不匹配的位置， 那么此时我们要看它的前一个字符的前缀表的数值是多少。

为什么要前一个字符的前缀表的数值呢，因为要找前面字符串的最长相同的前缀和后缀。

所以要看前一位的 前缀表的数值。

前一个字符的前缀表的数值是2， 所以把下标移动到下标2的位置继续比配。 可以再反复看一下上面的动画。

最后就在文本串中找到了和模式串匹配的子串了。

前缀表与next数组

很多KMP算法的时间都是使用next数组来做回退操作，那么next数组与前缀表有什么关系呢?

next数组就可以是前缀表，但是很多实现都是把前缀表统一减一（右移一位，初始位置为-1）之后作为next数组。

为什么这么做呢，其实也是很多文章视频没有解释清楚的地方。

其实**这并不涉及到KMP的原理，而是具体实现，next数组既可以就是前缀表，也可以是前缀表统一减一（右移一位，初始位置为-1）。**

后面我会提供两种不同的实现代码，大家就明白了。

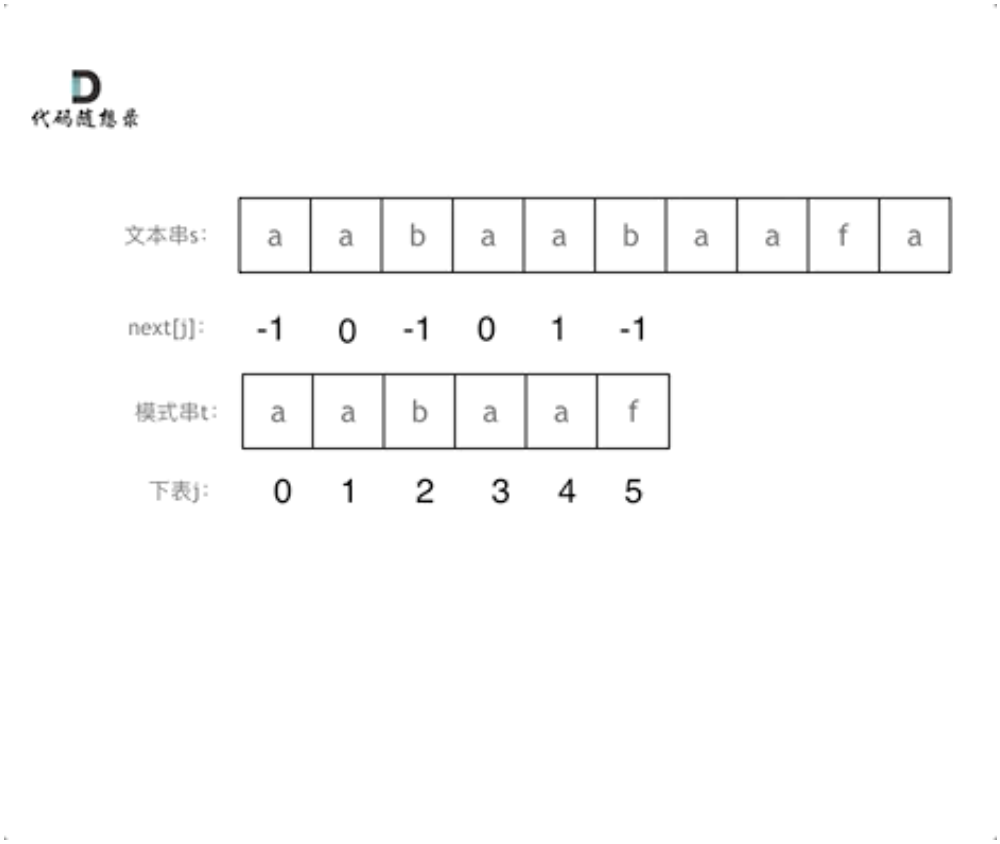
使用next数组来匹配

以下我们以前缀表统一减一之后的next数组来做演示。

有了next数组，就可以根据next数组来 匹配文本串s，和模式串t了。

注意next数组是新前缀表（旧前缀表统一减一了）。

匹配过程动画如下：



时间复杂度分析

其中n为文本串长度，m为模式串长度，因为在匹配的过程中，根据前缀表不断调整匹配的位置，可以看出匹配的过程是O(n)，之前还要单独生成next数组，时间复杂度是O(m)。所以整个KMP算法的时间复杂度是O(n+m)的。

暴力的解法显而易见是O(n × m)，所以**KMP在字符串匹配中极大地提高了搜索的效率。**

为了和力扣题目28.实现strStr保持一致，方便大家理解，以下文章统称haystack为文本串, needle为模式串。

都知道使用KMP算法，一定要构造next数组。

构造next数组

我们定义一个函数getNext来构建next数组，函数参数为指向next数组的指针，和一个字符串。 代码如下：

```
void getNext(int* next, const string& s)
```

构造next数组其实就是计算模式串s，前缀表的过程。 主要有如下三步：

1. 初始化
2. 处理前后缀不相同的情况
3. 处理前后缀相同的情况

接下来我们详解一下。

1. 初始化：

定义两个指针i和j，j指向前缀末尾位置，i指向后缀末尾位置。

然后还要对next数组进行初始化赋值，如下：

```
int j = -1;
next[0] = j;
```

j 为什么要初始化为 -1呢，因为之前说过 前缀表要统一减一的操作仅仅是其中的一种实现，我们这里选择j初始化为-1，下文我还会给出j不初始化为-1的实现代码。

next[i] 表示 i（包括i）之前最长相等的前后缀长度（其实就是j）

所以初始化next[0] = j 。

2. 处理前后缀不相同的情况

因为j初始化为-1，那么i就从1开始，进行s[i] 与 s[j+1]的比较。

所以遍历模式串s的循环下标i 要从 1开始，代码如下：

```
for (int i = 1; i < s.size(); i++) {
```

如果 s[i] 与 s[j+1]不相同，也就是遇到 前后缀末尾不相同的情况，就要向前回退。

怎么回退呢?

next[j]就是记录着j（包括j）之前的子串的相同前后缀的长度。

那么 s[i] 与 s[j+1] 不相同，就要找 j+1前一个元素在next数组里的值（就是next[j]）。

所以，处理前后缀不相同的情况代码如下：

```
while (j >= 0 && s[i] != s[j + 1]) { // 前后缀不相同了
    j = next[j]; // 向前回退
}
```

3. 处理前后缀相同的情况

如果 s[i] 与 s[j + 1] 相同，那么就同时向后移动i 和j 说明找到了相同的前后缀，同时还要将j（前缀的长度）赋给next[i], 因为next[i]要记录相同前后缀的长度。

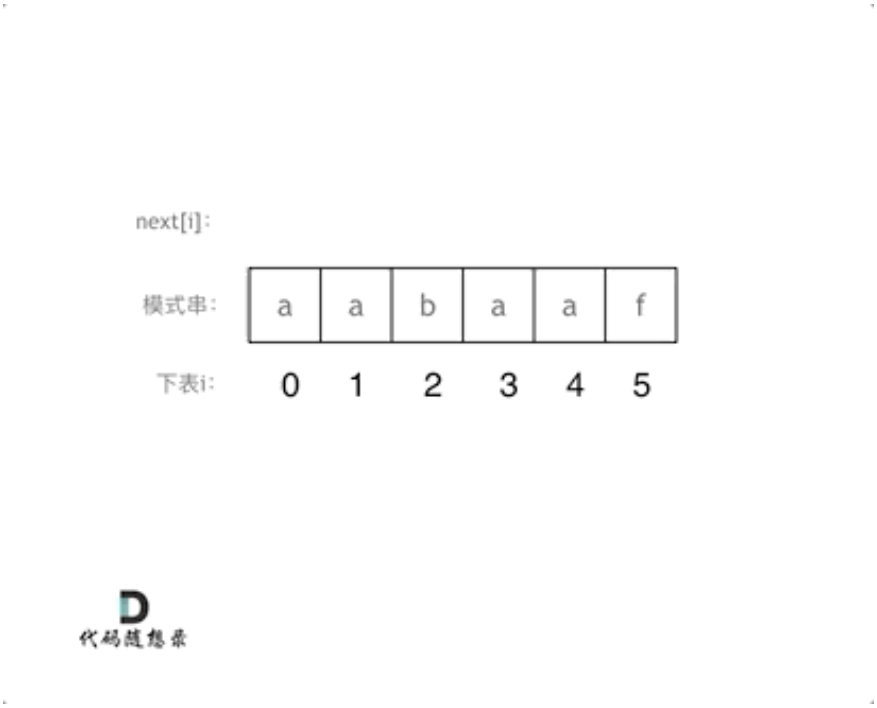
代码如下：

```
if (s[i] == s[j + 1]) { // 找到相同的前后缀
    j++;
}
next[i] = j;
```

最后整体构建next数组的函数代码如下：

```
void getNext(int* next, const string& s){
    int j = -1;
    next[0] = j;
    for(int i = 1; i < s.size(); i++) { // 注意i从1开始
        while (j >= 0 && s[i] != s[j + 1]) { // 前后缀不相同了
            j = next[j]; // 向前回退
        }
        if (s[i] == s[j + 1]) { // 找到相同的前后缀
            j++;
        }
        next[i] = j; // 将j（前缀的长度）赋给next[i]
    }
}
```

代码构造next数组的逻辑流程动画如下：



得到了next数组之后，就要用这个来做匹配了。

使用next数组来做匹配

在文本串s里 找是否出现过模式串t。

定义两个下标j 指向模式串起始位置，i指向文本串起始位置。

那么j初始值依然为-1，为什么呢？ **依然因为next数组里记录的起始位置为-1。**

i就从0开始，遍历文本串，代码如下：

```
for (int i = 0; i < s.size(); i++)
```

接下来就是 s[i] 与 t[j + 1] （因为j从-1开始的） 进行比较。

如果 s[i] 与 t[j + 1] 不相同，j就要从next数组里寻找下一个匹配的位置。

代码如下：

```
while(j >= 0 && s[i] != t[j + 1]) {
    j = next[j];
}
```

如果 s[i] 与 t[j + 1] 相同，那么i 和 j 同时向后移动， 代码如下：

```
if (s[i] == t[j + 1]) {
    j++; // i的增加在for循环里
}
```

如何判断在文本串s里出现了模式串t呢，如果j指向了模式串t的末尾，那么就说明模式串t完全匹配文本串s里的某个子串了。

本题要在文本串字符串中找出模式串出现的第一个位置 (从0开始)，所以返回当前在文本串匹配模式串的位置i 减去模式串的长度，就是文本串字符串中出现模式串的第一个位置。

代码如下：

```
if (j == (t.size() - 1) ) {
    return (i - t.size() + 1);
}
```

那么使用next数组，用模式串匹配文本串的整体代码如下：

```
int j = -1; // 因为next数组里记录的起始位置为-1
for (int i = 0; i < s.size(); i++) { // 注意i就从0开始
    while(j >= 0 && s[i] != t[j + 1]) { // 不匹配
        j = next[j]; // j 寻找之前匹配的位置
    }
    if (s[i] == t[j + 1]) { // 匹配，j和i同时向后移动
        j++; // i的增加在for循环里
    }
    if (j == (t.size() - 1) ) { // 文本串s里出现了模式串t
        return (i - t.size() + 1);
    }
}
```

此时所有逻辑的代码都已经写出来了，力扣 28.实现strStr 题目的整体代码如下：

前缀表统一减一 C++代码实现

```
class Solution {
public:
    void getNext(int* next, const string& s) {
        int j = -1;
        next[0] = j;
        for(int i = 1; i < s.size(); i++) { // 注意i从1开始
            while (j >= 0 && s[i] != s[j + 1]) { // 前后缀不相同了
                j = next[j]; // 向前回退
            }
            if (s[i] == s[j + 1]) { // 找到相同的前后缀
                j++;
            }
            next[i] = j; // 将j（前缀的长度）赋给next[i]
        }
    }
    int strStr(string haystack, string needle) {
        if (needle.size() == 0) {
            return 0;
        }
        int next[needle.size()];
        getNext(next, needle);
        int j = -1; // // 因为next数组里记录的起始位置为-1
        for (int i = 0; i < haystack.size(); i++) { // 注意i就从0开始
            while(j >= 0 && haystack[i] != needle[j + 1]) { // 不匹配
                j = next[j]; // j 寻找之前匹配的位置
            }
            if (haystack[i] == needle[j + 1]) { // 匹配，j和i同时向后移动
                j++; // i的增加在for循环里
            }
            if (j == (needle.size() - 1) ) { // 文本串s里出现了模式串t
                return (i - needle.size() + 1);
            }
        }
        return -1;
    }
};
```

- 时间复杂度: O(n + m)
- 空间复杂度: O(m), 只需要保存字符串needle的前缀表

前缀表（不减一）C++实现

那么前缀表就不减一了，也不右移的，到底行不行呢?

行!

我之前说过，这仅仅是KMP算法实现上的问题，如果就直接使用前缀表可以换一种回退方式，找j=next[j-1] 来进行回退。

主要就是j=next[x]这一步最为关键!

我给出的getNext的实现为：（前缀表统一减一）

```
void getNext(int* next, const string& s) {
    int j = -1;
    next[0] = j;
    for(int i = 1; i < s.size(); i++) { // 注意i从1开始
        while (j >= 0 && s[i] != s[j + 1]) { // 前后缀不相同了
            j = next[j]; // 向前回退
        }
        if (s[i] == s[j + 1]) { // 找到相同的前后缀
            j++;
        }
        next[i] = j; // 将j（前缀的长度）赋给next[i]
    }
}
```

此时如果输入的模式串为aabaaf，对应的next为-1 0 -1 0 1 -1。

这里j和next[0]初始化为-1，整个next数组是以 前缀表减一之后的效果来构建的。

那么前缀表不减一来构建next数组，代码如下：

```
    void getNext(int* next, const string& s) {
        int j = 0;
        next[0] = 0;
        for(int i = 1; i < s.size(); i++) {
            while (j > 0 && s[i] != s[j]) { // j要保证大于0，因为下面有取j-1作为数组下标的操作
                j = next[j - 1]; // 注意这里，是要找前一位的对应的回退位置了
            }
            if (s[i] == s[j]) {
                j++;
            }
            next[i] = j;
        }
    }
```

此时如果输入的模式串为aabaaf，对应的next为 0 1 0 1 2 0，（其实这就是前缀表的数值了）。

那么用这样的next数组也可以用来做匹配，代码要有所改动。

实现代码如下：

```
class Solution {
public:
    void getNext(int* next, const string& s) {
        int j = 0;
        next[0] = 0;
        for(int i = 1; i < s.size(); i++) {
            while (j > 0 && s[i] != s[j]) {
                j = next[j - 1];
            }
            if (s[i] == s[j]) {
                j++;
            }
            next[i] = j;
        }
    }
    int strStr(string haystack, string needle) {
        if (needle.size() == 0) {
            return 0;
        }
        int next[needle.size()];
        getNext(next, needle);
        int j = 0;
        for (int i = 0; i < haystack.size(); i++) {
            while(j > 0 && haystack[i] != needle[j]) {
                j = next[j - 1];
            }
            if (haystack[i] == needle[j]) {
                j++;
            }
            if (j == needle.size() ) {
                return (i - needle.size() + 1);
            }
        }
        return -1;
    }
};
```

- 时间复杂度: O(n + m)
- 空间复杂度: O(m)

总结

我们介绍了什么是KMP，KMP可以解决什么问题，然后分析KMP算法里的next数组，知道了next数组就是前缀表，再分析为什么要是前缀表而不是什么其他表。

接着从给出的模式串中，我们一步一步的推导出了前缀表，得出前缀表无论是统一减一还是不减一得到的next数组仅仅是kmp的实现方式的不同。

其中还分析了KMP算法的时间复杂度，并且和暴力方法做了对比。

然后先用前缀表统一减一得到的next数组，求得文本串s里是否出现过模式串t，并给出了具体分析代码。

又给出了直接用前缀表作为next数组，来做匹配的实现代码。

可以说把KMP的每一个细微的细节都扣了出来，毫无遮掩的展示给大家了！

> KMP算法还能干这个

7.重复的子字符串

力扣题目链接

给定一个非空的字符串，判断它是否可以由它的一个子串重复多次构成。给定的字符串只含有小写英文字母，并且长度不超过10000。

示例 1:

- 输入: "abab"
- 输出: True
- 解释: 可由子字符串 "ab" 重复两次构成。

示例 2:

- 输入: "aba"
- 输出: False

示例 3:

- 输入: "abcabcabcabc"
- 输出: True
- 解释: 可由子字符串 "abc" 重复四次构成。 (或者子字符串 "abcabc" 重复两次构成。)

算法公开课

《代码随想录》算法视频公开课：字符串这么玩，可有点难度！ | LeetCode：459.重复的子字符串，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

暴力的解法， 就是一个for循环获取 子串的终止位置， 然后判断子串是否能重复构成字符串，又嵌套一个for循环，所以是O(n^2)的时间复杂度。

有的同学可以想，怎么一个for循环就可以获取子串吗？ 至少得一个for获取子串起始位置，一个for获取子串结束位置吧。

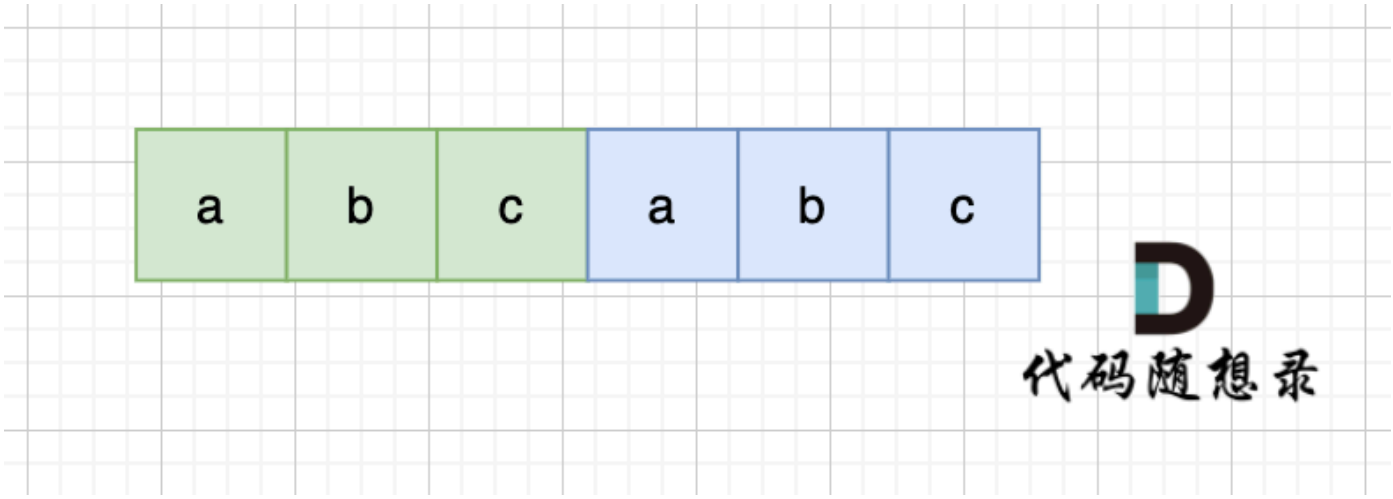
其实我们只需要判断，以第一个字母为开始的子串就可以，所以一个for循环获取子串的终止位置就行了。 而且遍历的时候 都不用遍历结束，只需要遍历到中间位置，因为子串结束位置大于中间位置的话，一定不能重复组成字符串。

暴力的解法，这里就不详细讲解了。

主要讲一讲移动匹配 和 KMP两种方法。

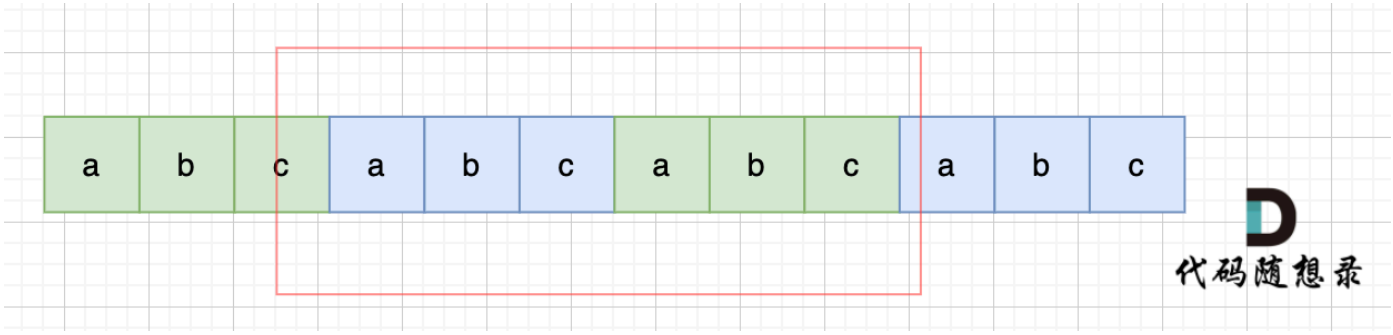
移动匹配

当一个字符串s：abcabc，内部由重复的子串组成，那么这个字符串的结构一定是这样的：



也就是由前后相同的子串组成。

那么既然前面有相同的子串，后面有相同的子串，用 s + s，这样组成的字符串中，后面的子串做前串，前后的子串做后串，就一定还能组成一个s，如图：



所以判断字符串s是否由重复子串组成，只要两个s拼接在一起，里面还出现一个s的话，就说明是由重复子串组成。

当然，我们在判断 s + s 拼接的字符串里是否出现一个s的的时候，**要刨除 s + s 的首字符和尾字符**，这样避免在s+s中搜索出原来的s，我们要搜索的是中间拼接出来的s。

代码如下：

```
class Solution {
public:
    bool repeatedSubstringPattern(string s) {
        string t = s + s;
        t.erase(t.begin()); t.erase(t.end() - 1); // 掐头去尾
        if (t.find(s) != std::string::npos) return true; // r
        return false;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

不过这种解法还有一个问题，就是 我们最终还是要判断 一个字符串（s + s）是否出现过 s 的过程，大家可能直接用contains，find 之类的库函数。 却忽略了实现这些函数的时间复杂度（暴力解法是m * n，一般库函数实现为O(m + n)）。

如果我们做过 28.实现strStr 题目的话，其实就知道，**实现一个 高效的算法来判断 一个字符串中是否出现另一个字符串是很复杂的**，这里就涉及到了KMP算法。

KMP

为什么会使用KMP

以下使用KMP方式讲解，强烈建议大家先把以下两个视频看了，理解KMP算法，再来看下面讲解，否则会很懵。

- 视频讲解版：帮你把KMP算法学个通透！（理论篇）
- 视频讲解版：帮你把KMP算法学个通透！（求next数组代码篇）
- 文字讲解版：KMP算法

在一个串中查找是否出现过另一个串，这是KMP的看家本领。那么寻找重复子串怎么也涉及到KMP算法了呢?

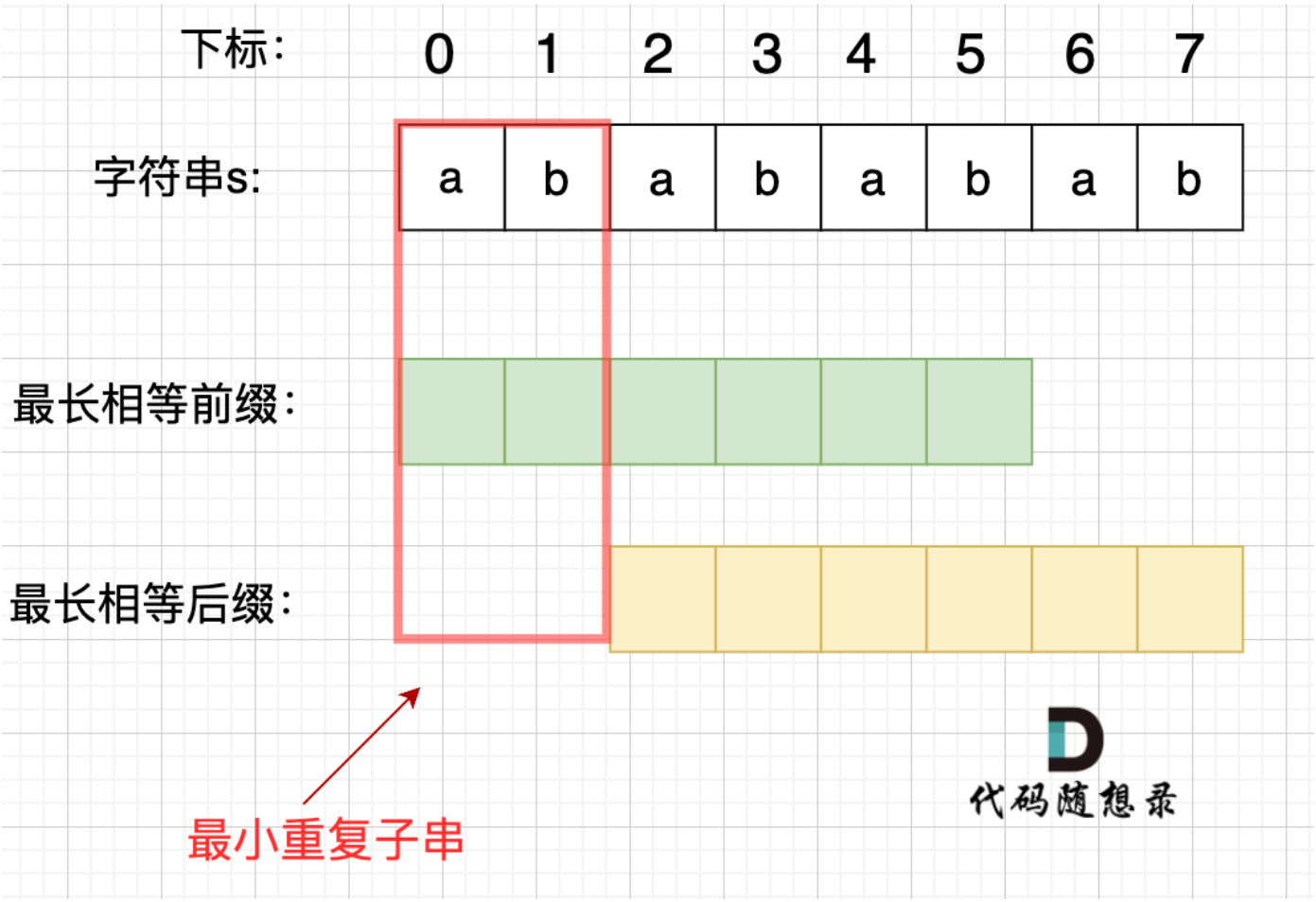
KMP算法中next数组为什么遇到字符不匹配的时候可以找到上一个匹配过的位置继续匹配，靠的是有计算好的前缀表。 前缀表里，统计了各个位置为终点字符串的最长相同前后缀的长度。

那么 最长相同前后缀和重复子串的关系又有什么关系呢。

可能很多录友又忘了 前缀和后缀的定义，再回顾一下：

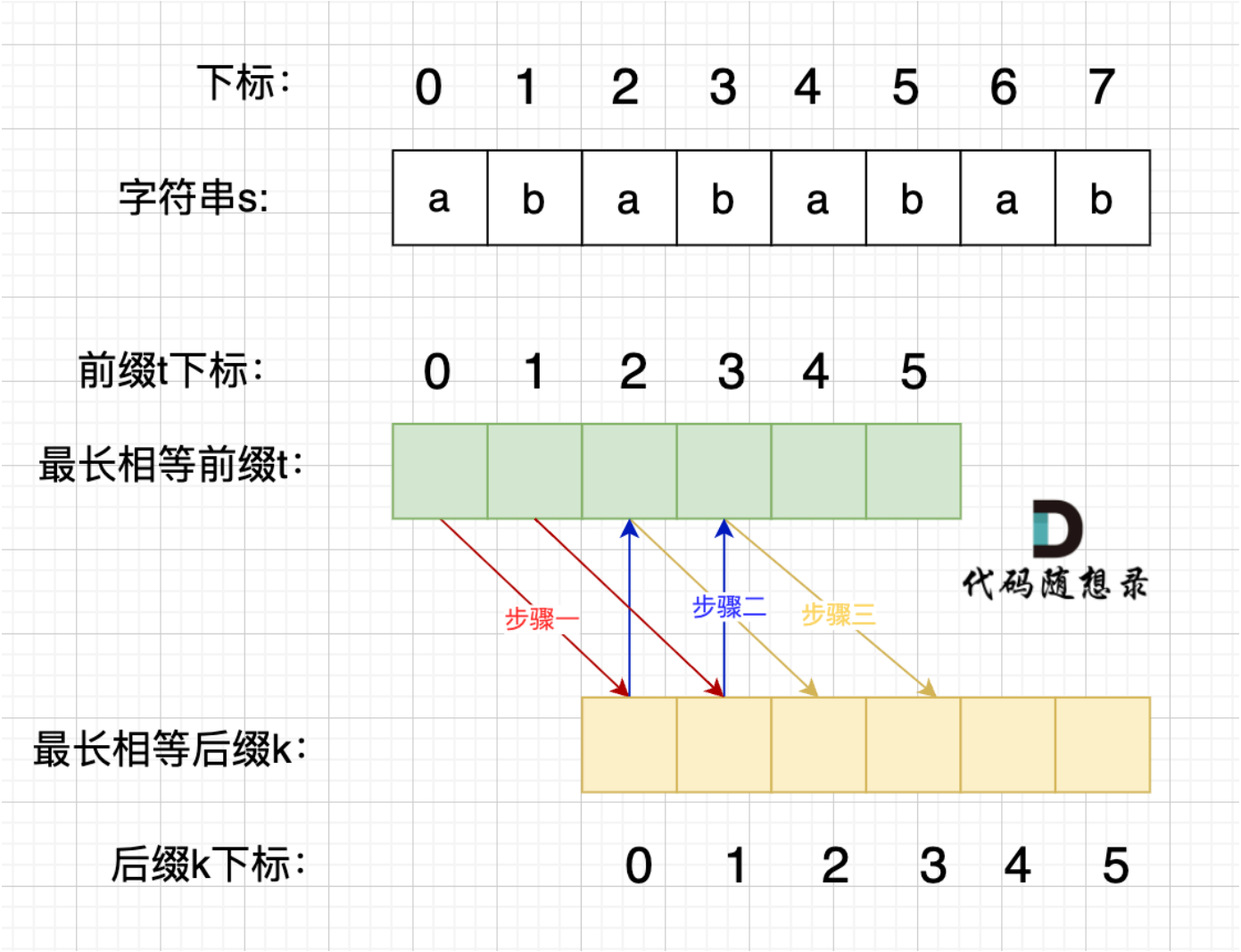
- 前缀是指不包含最后一个字符的所有以第一个字符开头的连续子串；
- 后缀是指不包含第一个字符的所有以最后一个字符结尾的连续子串

在由重复子串组成的字符串中，最长相等前后缀不包含的子串就是最小重复子串，这里拿字符串s：abababab 来举例，ab就是最小重复单位，如图所示：



如何找到最小重复子串

这里有同学就问了，为啥一定是开头的ab呢。 其实最关键还是要理解 最长相等前后缀，如图：



步骤一：因为 这是相等的前缀和后缀，t[0] 与 k[0]相同， t[1] 与 k[1]相同，所以 s[0] 一定和 s[2]相同，s[1] 一定和 s[3]相同，即：，s[0]s[1]与s[2]s[3]相同 。

步骤二： 因为在同一个字符串位置，所以 t[2] 与 k[0]相同，t[3] 与 k[1]相同。

步骤三： 因为 这是相等的前缀和后缀，t[2] 与 k[2]相同 ，t[3]与k[3] 相同，所以，s[2]一定和s[4]相同，s[3]一定和s[5]相同，即：s[2]s[3] 与 s[4]s[5]相同。

步骤四：循环往复。

所以字符串s，s[0]s[1]与s[2]s[3]相同， s[2]s[3] 与 s[4]s[5]相同，s[4]s[5] 与 s[6]s[7] 相同。

正是因为 最长相等前后缀的规则，当一个字符串由重复子串组成的，最长相等前后缀不包含的子串就是最小重复子串。

简单推理

这里再给出一个数学推导，就容易理解很多。

假设字符串s使用多个重复子串构成（这个子串是最小重复单位），重复出现的子字符串长度是x，所以s是由n * x组成。

因为字符串s的最长相同前后缀的长度一定是不包含s本身，所以 最长相同前后缀长度必然是m * x，而且 n - m = 1，（这里如果不懂，看上面的推理）

所以如果 nx % (n - m)x = 0，就可以判定有重复出现的子字符串。

next 数组记录的就是最长相同前后缀 字符串：KMP算法精讲 这里介绍了什么是前缀，什么是后缀，什么又是最长相同前后缀)， 如果 next[len - 1] != -1，则说明字符串有最长相同的前后缀（就是字符串里的前缀子串和后缀子串相同的最长长度）。

最长相等前后缀的长度为：next[len - 1] + 1。(这里的next数组是以统一减一的方式计算的，因此需要+1，两种计算next数组的具体区别看这里：字符串：KMP算法精讲)

数组长度为：len。

如果len % (len - (next[len - 1] + 1)) == 0 ，则说明数组的长度正好可以被 (数组长度-最长相等前后缀的长度) 整除 ，说明该字符串有重复的子字符串。

数组长度减去最长相同前后缀的长度相当于是第一个周期的长度，也就是一个周期的长度，如果这个周期可以被整除，就说明整个数组就是这个周期的循环。

强烈建议大家把next数组打印出来，看看next数组里的规律，有助于理解KMP算法

如图：

字符串	a	s	d	f	a	s	d	f	a	s	d	f
对应的next数组的值:	-1	-1	-1	-1	0	1	2	3	4	5	6	7

next[len - 1] = 7，next[len - 1] + 1 = 8，8就是此时字符串asdfasdfasdf的最长相同前后缀的长度。

(len - (next[len - 1] + 1)) 也就是： 12(字符串的长度) - 8(最长公共前后缀的长度) = 4， 4正好可以被 12(字符串的长度) 整除，所以说明有重复的子字符串（asdf）。

C++代码如下：（这里使用了前缀表统一减一的实现方式）

```
class Solution {
public:
    void getNext (int* next, const string& s){
        next[0] = -1;
        int j = -1;
        for(int i = 1;i < s.size(); i++){
            while(j >= 0 && s[i] != s[j + 1]) {
                j = next[j];
            }
            if(s[i] == s[j + 1]) {
                j++;
            }
            next[i] = j;
        }
    }
    bool repeatedSubstringPattern (string s) {
        if (s.size() == 0) {
```

```
            return false;
        }
        int next[s.size()];
        getNext(next, s);
        int len = s.size();
        if (next[len - 1] != -1 && len % (len - (next[len - 1] + 1)) == 0) {
            return true;
        }
        return false;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

前缀表（不减一）的C++代码实现：

```
class Solution {
public:
    void getNext (int* next, const string& s){
        next[0] = 0;
        int j = 0;
        for(int i = 1;i < s.size(); i++){
            while(j > 0 && s[i] != s[j]) {
                j = next[j - 1];
            }
            if(s[i] == s[j]) {
                j++;
            }
            next[i] = j;
        }
    }
    bool repeatedSubstringPattern (string s) {
        if (s.size() == 0) {
            return false;
        }
        int next[s.size()];
        getNext(next, s);
        int len = s.size();
        if (next[len - 1] != 0 && len % (len - (next[len - 1] )) == 0) {
            return true;
        }
        return false;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

8. 字符串总结篇

其实我们已经学习了十天的字符串了，从字符串的定义到库函数的使用原则，从各种反转到KMP算法，相信大家应该对字符串有比较深刻的认识了。

那么这次我们来做一个总结。

什么是字符串

字符串是若干字符组成的有限序列，也可以理解为是一个字符数组，但是很多语言对字符串做了特殊的规定，接下来我来说一说C/C++中的字符串。

在C语言中，把一个字符串存入一个数组时，也把结束符 '\0'存入数组，并以此作为该字符串是否结束的标志。

例如这段代码：

```
char a[5] = "asd";
for (int i = 0; a[i] != '\0'; i++) {
}
```

在C++中，提供一个string类，string类会提供 size接口，可以用来判断string类字符串是否结束，就不用'\0'来判断是否结束。

例如这段代码:

```
string a = "asd";
for (int i = 0; i < a.size(); i++) {
}
```

那么vector< char > 和 string 又有什么区别呢?

其实在基本操作上没有区别，但是 string提供更多的字符串处理的相关接口，例如string 重载了+，而vector却没有。

所以想处理字符串，我们还是会定义一个string类型。

要不要使用库函数

在文章344.反转字符串中强调了**打基础的时候，不要太迷恋于库函数。**

甚至一些同学习惯于调用substr，split，reverse之类的库函数，却不知道其实现原理，也不知道其时间复杂度，这样实现出来的代码，如果在面试现场，面试官问：“分析其时间复杂度”的话，一定会一脸懵逼！

所以建议**如果题目关键的部分直接用库函数就可以解决，建议不要使用库函数。**

如果库函数仅仅是 解题过程中的一小部分，并且你已经很清楚这个库函数的内部实现原理的话，可以考虑使用库函数。

双指针法

在344.反转字符串 ，我们使用双指针法实现了反转字符串的操作，**双指针法在数组，链表和字符串中很常用。**

接着在字符串：替换空格，同样还是使用双指针法在时间复杂度O(n)的情况下完成替换空格。

其实很多数组填充类的问题，都可以先预先给数组扩容带填充后的大小，然后在从后向前进行操作。

那么针对数组删除操作的问题，其实在27. 移除元素中就已经提到了使用双指针法进行移除操作。

同样的道理在151.翻转字符串里的单词中我们使用O(n)的时间复杂度，完成了删除冗余空格。

一些同学会使用for循环里调用库函数erase来移除元素，这其实是O(n^2)的操作，因为erase就是O(n)的操作，所以这也是典型的不知道库函数的时间复杂度，上来就用的案例了。

反转系列

在反转上还可以在加一些玩法，其实考察的是对代码的掌控能力。

541. 反转字符串II中，一些同学可能为了处理逻辑：每隔2k个字符的前k的字符，写了一堆逻辑代码或者再搞一个计数器，来统计2k，再统计前k个字符。

其实**当需要固定规律一段一段去处理字符串的时候，要想想在在for循环的表达式上做做文章**。

只要让 i += (2 * k)，i 每次移动 2 * k 就可以了，然后判断是否需要有反转的区间。

因为要找的也就是每2 * k 区间的起点，这样写程序会高效很多。

在151.翻转字符串里的单词中要求翻转字符串里的单词，这道题目可以说是综合考察了字符串的多种操作。是考察字符串的好题。

这道题目通过 **先整体反转再局部反转**，实现了反转字符串里的单词。

后来发现反转字符串还有一个牛逼的用处，就是达到左旋的效果。

在字符串：反转个字符串还有这个用处？ 中，我们通过**先局部反转再整体反转**达到了左旋的效果。

KMP

KMP的主要思想是**当出现字符串不匹配时，可以知道一部分之前已经匹配的文本内容，可以利用这些信息避免从头再去做匹配了。**

KMP的精髓所在就是前缀表，在KMP精讲中提到了，什么是KMP，什么是前缀表，以及为什么要用前缀表。

前缀表：起始位置到下标i之前（包括i）的子串中，有多大长度的相同前缀后缀。

那么使用KMP可以解决两类经典问题：

1. 匹配问题：28. 实现 strStr()
2. 重复子串问题：459.重复的子字符串

再一次强调了什么是前缀，什么是后缀，什么又是最长相等前后缀。

前缀：指不包含最后一个字符的所有以第一个字符开头的连续子串。

后缀：指不包含第一个字符的所有以最后一个字符结尾的连续子串。

然后**针对前缀表到底要不要减一，这其实是不同KMP实现的方式**，我们在KMP精讲中针对之前两个问题，分别给出了两个不同版本的的KMP实现。

其中主要**理解j=next[x]这一步最为关键！**

总结

字符串类类型的题目，往往想法比较简单，但是实现起来并不容易，复杂的字符串题目非常考验对代码的掌控能力。

双指针法是字符串处理的常客。

KMP算法是字符串查找最重要的算法，但彻底理解KMP并不容易，我们已经写了五篇KMP的文章，不断总结和完善，最终才把KMP讲清楚。

好了字符串相关的算法知识就介绍到了这里了，明天开始新的征程，大家加油！

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 移除元素

力扣题目链接

给你一个数组 nums 和一个值 val，你需要 原地 移除所有数值等于 val 的元素，并返回移除后数组的新长度。

不要使用额外的数组空间，你必须仅使用 O(1) 额外空间并**原地**修改输入数组。

元素的顺序可以改变。你不需要考虑数组中超出新长度后面的元素。

示例 1:
给定 nums = [3,2,2,3], val = 3,
函数应该返回新的长度 2, 并且 nums 中的前两个元素均为 2。
你不需要考虑数组中超出新长度后面的元素。

示例 2:
给定 nums = [0,1,2,2,3,0,4,2], val = 2,
函数应该返回新的长度 5, 并且 nums 中的前五个元素为 0, 1, 3, 0, 4。

你不需要考虑数组中超出新长度后面的元素。

算法公开课

[《代码随想录》算法视频公开课](#)：[数组中移除元素并不容易！LeetCode：27. 移除元素](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

有的同学可能说了，多余的元素，删掉不就得了。

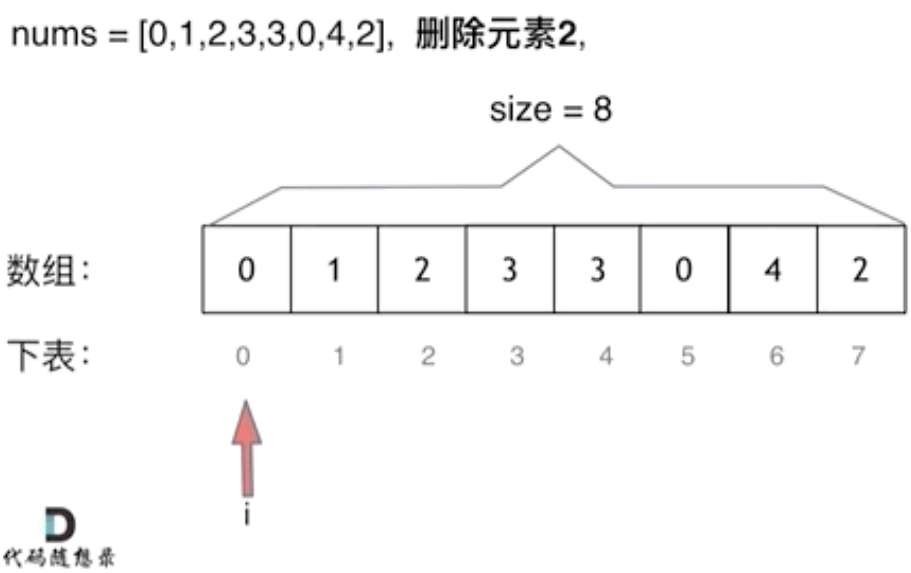
要知道数组的元素在内存地址中是连续的，不能单独删除数组中的某个元素，只能覆盖。

数组的基础知识可以看这里[程序员算法面试中，必须掌握的数组理论知识](#)。

暴力解法

这个题目暴力的解法就是两层for循环，一个for循环遍历数组元素 ，第二个for循环更新数组。

删除过程如下：



很明显暴力解法的时间复杂度是O(n^2)，这道题目暴力解法在leetcode上是可以过的。

代码如下：

```
// 时间复杂度：O(n^2)
// 空间复杂度：O(1)
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int size = nums.size();
        for (int i = 0; i < size; i++) {
```

```
            if (nums[i] == val) { // 发现需要移除的元素，就将数组集体向前移动一位
                for (int j = i + 1; j < size; j++) {
                    nums[j - 1] = nums[j];
                }
                i--; // 因为下标i以后的数值都向前移动了一位，所以i也向前移动一位
                size--; // 此时数组的大小-1
            }
        }
        return size;

    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

双指针法

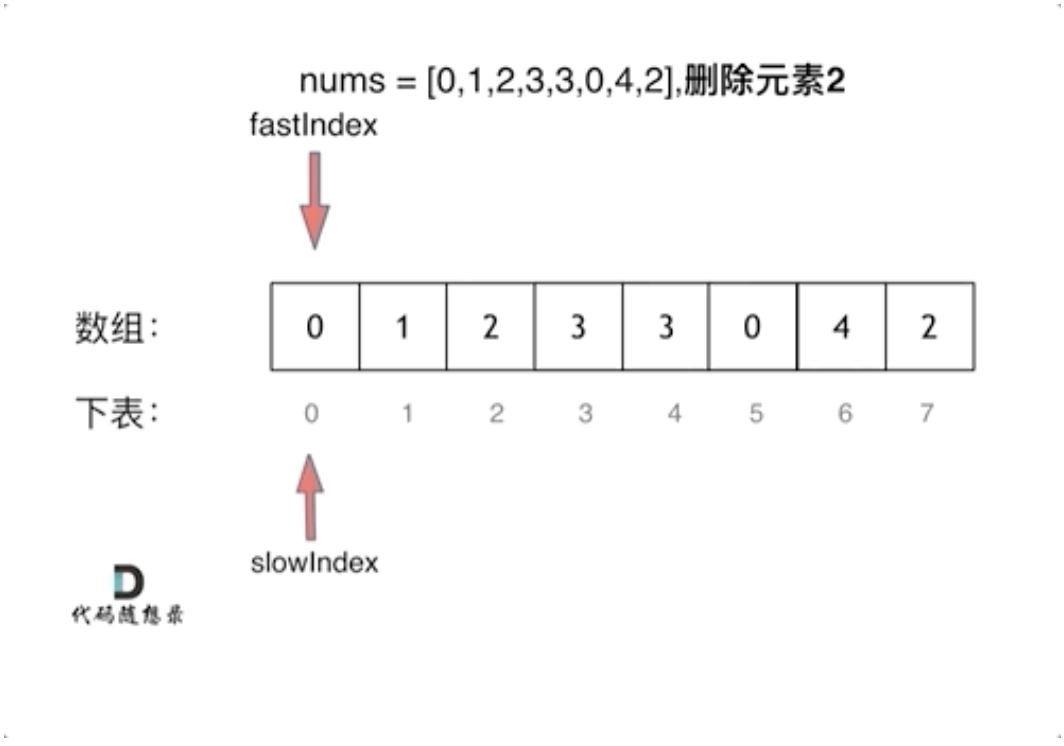
双指针法（快慢指针法）： **通过一个快指针和慢指针在一个for循环下完成两个for循环的工作。**

定义快慢指针

- 快指针：寻找新数组的元素 ，新数组就是不含有目标元素的数组
- 慢指针：指向更新 新数组下标的位置

很多同学这道题目做的很懵，就是不理解 快慢指针究竟都是什么含义，所以一定要明确含义，后面的思路就更容易理解了。

删除过程如下：



很多同学不了解

双指针法（快慢指针法）在数组和链表的操作中是非常常见的，很多考察数组、链表、字符串等操作的面试题，都使用双指针法。

后续都会一一介绍到，本题代码如下：

```
// 时间复杂度：O(n)
// 空间复杂度：O(1)
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int slowIndex = 0;
        for (int fastIndex = 0; fastIndex < nums.size(); fastIndex++) {
            if (val != nums[fastIndex]) {
                nums[slowIndex++] = nums[fastIndex];
            }
        }
        return slowIndex;
    }
};
```

注意这些实现方法并没有改变元素的相对位置！

- 时间复杂度：O(n)
- 空间复杂度：O(1)

```
/**
* 相向双指针方法，基于元素顺序可以改变的题目描述改变了元素相对位置，确保了移动最少元素
* 时间复杂度：O(n)
* 空间复杂度：O(1)
*/
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int leftIndex = 0;
        int rightIndex = nums.size() - 1;
        while (leftIndex <= rightIndex) {
            // 找左边等于val的元素
            while (leftIndex <= rightIndex && nums[leftIndex] != val){
                ++leftIndex;
            }
            // 找右边不等于val的元素
            while (leftIndex <= rightIndex && nums[rightIndex] == val) {
                -- rightIndex;
            }
            // 将右边不等于val的元素覆盖左边等于val的元素
            if (leftIndex < rightIndex) {
                nums[leftIndex++] = nums[rightIndex--];
            }
        }
```

```
        return leftIndex;   // leftIndex一定指向了最终数组末尾的下一个元素
    }
};
```

相关题目推荐

- 26.删除排序数组中的重复项
- 283.移动零
- 844.比较含退格的字符串
- 977.有序数组的平方

> 打基础的时候，不要太迷恋于库函数。

2.反转字符串

力扣题目链接

编写一个函数，其作用是将输入的字符串反转过来。输入字符串以字符数组 char[] 的形式给出。

不要给另外的数组分配额外的空间，你必须原地修改输入数组、使用 O(1) 的额外空间解决这一问题。

你可以假设数组中的所有字符都是 ASCII 码表中的可打印字符。

示例 1：
输入：["h","e","l","l","o"]
输出：["o","l","l","e","h"]

示例 2：
输入：["H","a","n","n","a","h"]
输出：["h","a","n","n","a","H"]

算法公开课

《代码随想录》算法视频公开课：字符串基础操作！ | LeetCode：344.反转字符串，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

先说一说题外话：

对于这道题目一些同学直接用C++里的一个库函数 reverse，调一下直接完事了， 相信每一门编程语言都有这样的库函数。

如果这么做题的话，这样大家不会清楚反转字符串的实现原理了。

但是也不是说库函数就不能用，是要分场景的。

如果在现场面试中，我们什么时候使用库函数，什么时候不要用库函数呢?

如果题目关键的部分直接用库函数就可以解决，建议不要使用库函数。

毕竟面试官一定不是考察你对库函数的熟悉程度， 如果使用python和java 的同学更需要注意这一点，因为python、java提供的库函数十分丰富。

如果库函数仅仅是 解题过程中的一小部分，并且你已经很清楚这个库函数的内部实现原理的话，可以考虑使用库函数。

建议大家平时在leetcode上练习算法的时候本着这样的原则去练习，这样才有助于我们对算法的理解。

不要沉迷于使用库函数一行代码解决题目之类的技巧，不是说这些技巧不好，而是说这些技巧可以用来娱乐一下。

真正自己写的时候，要保证理解可以实现是相应的功能。

接下来再来讲一下如何解决反转字符串的问题。

大家应该还记得，我们已经讲过了206.反转链表。

在反转链表中，使用了双指针的方法。

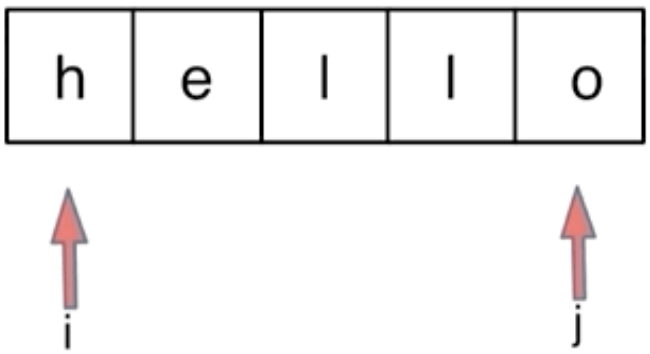
那么反转字符串依然是使用双指针的方法，只不过对于字符串的反转，其实要比链表简单一些。

因为字符串也是一种数组，所以元素在内存中是连续分布，这就决定了反转链表和反转字符串方式上还是有所差异的。

如果对数组和链表原理不清楚的同学，可以看这两篇，关于链表，你该了解这些！，必须掌握的数组理论知识。

对于字符串，我们定义两个指针（也可以说是索引下标），一个从字符串前面，一个从字符串后面，两个指针同时向中间移动，并交换元素。

以字符串`hello`为例，过程如下：



不难写出如下C++代码:

```
void reverseString(vector<char>& s) {
    for (int i = 0, j = s.size() - 1; i < s.size()/2; i++, j--) {
        swap(s[i],s[j]);
    }
}
```

循环里只要做交换s[i] 和s[j]操作就可以了，那么我这里使用了swap 这个库函数。大家可以使用。

因为相信大家都知道交换函数如何实现，而且这个库函数仅仅是解题中的一部分， 所以这里使用库函数也是可以的。

swap可以有两种实现。

一种就是常见的交换数值：

```
int tmp = s[i];
s[i] = s[j];
s[j] = tmp;
```

一种就是通过位运算：

```
s[i] ^= s[j];
s[j] ^= s[i];
s[i] ^= s[j];
```

这道题目还是比较简单的，但是我正好可以通过这道题目说一说在刷题的时候，使用库函数的原则。

如果题目关键的部分直接用库函数就可以解决，建议不要使用库函数。

如果库函数仅仅是 解题过程中的一小部分，并且你已经很清楚这个库函数的内部实现原理的话，可以考虑使用库函数。

本着这样的原则，我没有使用reverse库函数，而使用swap库函数。

在字符串相关的题目中，库函数对大家的诱惑力是非常大的，因为会有各种反转，切割取词之类的操作，这也是为什么字符串的库函数这么丰富的原因。

相信大家本着我所讲述的原则来做字符串相关的题目，在选择库函数的角度上会有所原则，也会有所收获。

C++代码如下：

```
class Solution {
public:
    void reverseString(vector<char>& s) {
        for (int i = 0, j = s.size() - 1; i < s.size()/2; i++, j--) {
            swap(s[i],s[j]);
        }
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

---

3. 剑指Offer 05.替换空格

力扣题目链接

请实现一个函数，把字符串 s 中的每个空格替换成"%20"。

示例 1：
输入：s = "We are happy."
输出："We%20are%20happy."

思路

如果想把这道题目做到极致，就不要只用额外的辅助空间了！

首先扩充数组到每个空格替换成"%20"之后的大小。

然后从后向前替换空格，也就是双指针法，过程如下：

i指向新长度的末尾，j指向旧长度的末尾。

有同学问了，为什么要从后向前填充，从前向后填充不行么？

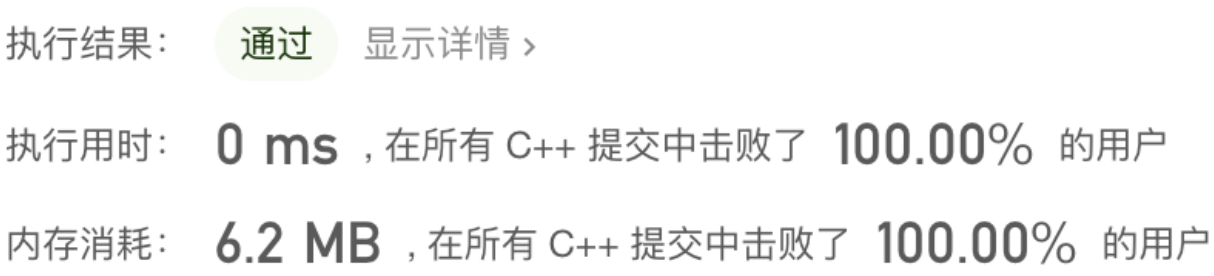
从前向后填充就是O(n^2)的算法了，因为每次添加元素都要将添加元素之后的所有元素向后移动。

其实很多数组填充类的问题，都可以先预先给数组扩容带填充后的大小，然后在从后向前进行操作。

这么做有两个好处：

1. 不用申请新数组。
2. 从后向前填充元素，避免了从前向后填充元素时，每次添加元素都要将添加元素之后的所有元素向后移动的问题。

时间复杂度，空间复杂度均超过100%的用户。



C++代码如下：

```
class Solution {
public:
    string replaceSpace(string s) {
        int count = 0; // 统计空格的个数
        int sOldSize = s.size();
        for (int i = 0; i < s.size(); i++) {
            if (s[i] == ' ') {
                count++;
            }
        }
        // 扩充字符串s的大小，也就是每个空格替换成"%20"之后的大小
        s.resize(s.size() + count * 2);
        int sNewSize = s.size();
        // 从后先前将空格替换为"%20"
        for (int i = sNewSize - 1, j = sOldSize - 1; j < i; i--, j--) {
            if (s[j] != ' ') {
                s[i] = s[j];
            } else {
                s[i] = '0';
                s[i - 1] = '2';
                s[i - 2] = '%';
                i -= 2;
            }
        }
        return s;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

此时算上本题，我们已经做了七道双指针相关的题目了分别是：

- 27.移除元素
- 15.三数之和
- 18.四数之和
- 206.翻转链表
- 142.环形链表II
- 344.反转字符串

拓展

这里也给大家拓展一下字符串和数组有什么差别，

字符串是若干字符组成的有限序列，也可以理解为是一个字符数组，但是很多语言对字符串做了特殊的规定，接下来我来说一说C/C++中的字符串。

在C语言中，把一个字符串存入一个数组时，也把结束符 '\0'存入数组，并以此作为该字符串是否结束的标志。

例如这段代码：

```
char a[5] = "asd";
for (int i = 0; a[i] != '\0'; i++) {
}
```

在C++中，提供一个string类，string类会提供 size接口，可以用来判断string类字符串是否结束，就不用'\0'来判断是否结束。

例如这段代码:

```
string a = "asd";
for (int i = 0; i < a.size(); i++) {
}
```

那么vector< char > 和 string 又有什么区别呢?

其实在基本操作上没有区别，但是 string提供更多的字符串处理的相关接口，例如string 重载了+，而vector却没有。

所以想处理字符串，我们还是会定义一个string类型。

> 综合考察字符串操作的好题。

4.翻转字符串里的单词

力扣题目链接

给定一个字符串，逐个翻转字符串中的每个单词。

示例 1：
输入: "the sky is blue"
输出: "blue is sky the"

示例 2：
输入: " hello world! "
输出: "world! hello"
解释: 输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。

示例 3：
输入: "a good   example"
输出: "example good a"
解释: 如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。

算法公开课

[《代码随想录》算法视频公开课](#)：[字符串复杂操作拿捏了！ | LeetCode:151.翻转字符串里的单词](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

这道题目可以说是综合考察了字符串的多种操作。

一些同学会使用split库函数，分隔单词，然后定义一个新的string字符串，最后再把单词倒序相加，那么这道题题目就是一道水题了，失去了它的意义。

所以这里我还是提高一下本题的难度：**不要使用辅助空间，空间复杂度要求为O(1)。**

不能使用辅助空间之后，那么只能在原字符串上下功夫了。

想一下，我们将整个字符串都反转过来，那么单词的顺序指定是倒序了，只不过单词本身也倒序了，那么再把单词反转一下，单词不就正过来了。

所以解题思路如下：

- 移除多余空格
- 将整个字符串反转
- 将每个单词反转

举个例子，源字符串为："the sky is   blue   "

- 移除多余空格 : "the sky is blue"
- 字符串反转："eulb si yks eht"
- 单词反转："blue is sky the"

这样我们就完成了翻转字符串里的单词。

思路很明确了，我们说一说代码的实现细节，就拿移除多余空格来说，一些同学会上来写如下代码：

```
void removeExtraSpaces(string& s) {
    for (int i = s.size() - 1; i > 0; i--) {
        if (s[i] == s[i - 1] && s[i] == ' ') {
```

```
            s.erase(s.begin() + i);
        }
    }
    // 删除字符串最后面的空格
    if (s.size() > 0 && s[s.size() - 1] == ' ') {
        s.erase(s.begin() + s.size() - 1);
    }
    // 删除字符串最前面的空格
    if (s.size() > 0 && s[0] == ' ') {
        s.erase(s.begin());
    }
}
```

逻辑很简单，从前向后遍历，遇到空格了就erase。

如果不仔细琢磨一下erase的时间复杂度，还以为以上的代码是O(n)的时间复杂度呢。

想一下真正的时间复杂度是多少，一个erase本来就是O(n)的操作。

erase操作上面还套了一个for循环，那么以上代码移除冗余空格的代码时间复杂度为O(n^2)。

那么使用双指针法来去移除空格，最后resize（重新设置）一下字符串的大小，就可以做到O(n)的时间复杂度。

```
//版本一
void removeExtraSpaces(string& s) {
    int slowIndex = 0, fastIndex = 0; // 定义快指针，慢指针
    // 去掉字符串前面的空格
    while (s.size() > 0 && fastIndex < s.size() && s[fastIndex] == ' ') {
        fastIndex++;
    }
    for (; fastIndex < s.size(); fastIndex++) {
        // 去掉字符串中间部分的冗余空格
        if (fastIndex - 1 > 0
                && s[fastIndex - 1] == s[fastIndex]
                && s[fastIndex] == ' ') {
            continue;
        } else {
            s[slowIndex++] = s[fastIndex];
        }
    }
    if (slowIndex - 1 > 0 && s[slowIndex - 1] == ' ') { // 去掉字符串末尾的空格
        s.resize(slowIndex - 1);
    } else {
        s.resize(slowIndex); // 重新设置字符串大小
    }
}
```

有的同学可能发现用erase来移除空格，在leetcode上性能也还行。主要是以下几点；：

1. leetcode上的测试集里，字符串的长度不够长，如果足够长，性能差距会非常明显。

2. leetcode的测程序耗时不是很准确的。

版本一的代码是一般的思考过程，就是 先移除字符串前的空格，再移除中间的，再移除后面部分。

不过其实还可以优化，这部分和27.移除元素的逻辑是一样一样的，本题是移除空格，而 27.移除元素 就是移除元素。

所以代码可以写的很精简，大家可以看 如下 代码 removeExtraSpaces 函数的实现：

```
// 版本二
void removeExtraSpaces(string& s) {//去除所有空格并在相邻单词之间添加空格，快慢指针。
    int slow = 0;   //整体思想参考https://programmercarl.com/0027.移除元素.html
    for (int i = 0; i < s.size(); ++i) { //
        if (s[i] != ' ') { //遇到非空格就处理，即删除所有空格。
            if (slow != 0) s[slow++] = ' '; //手动控制空格，给单词之间添加空格。slow != 0说明不是第一个单词，需要在单词前添加空格。
            while (i < s.size() && s[i] != ' ') { //补上该单词，遇到空格说明单词结束。
                s[slow++] = s[i++];
            }
        }
    }
    s.resize(slow); //slow的大小即为去除多余空格后的大小。
}
```

如果以上代码看不懂，建议先把 27.移除元素这道题目做了，或者看视频讲解：数组中移除元素并不容易！ LeetCode：27. 移除元素 。

此时我们已经实现了removeExtraSpaces函数来移除冗余空格。

还要实现反转字符串的功能，支持反转字符串子区间，这个实现我们分别在344.反转字符串和541.反转字符串II里已经讲过了。

代码如下：

```
// 反转字符串s中左闭右闭的区间[start, end]
void reverse(string& s, int start, int end) {
    for (int i = start, j = end; i < j; i++, j--) {
        swap(s[i], s[j]);
    }
}
```

整体代码如下：

```
class Solution {
public:
    void reverse(string& s, int start, int end){ //翻转，区间写法：左闭右闭 []
        for (int i = start, j = end; i < j; i++, j--) {
            swap(s[i], s[j]);
        }
    }
```

```
    void removeExtraSpaces(string& s) {//去除所有空格并在相邻单词之间添加空格，快慢指针。
        int slow = 0;   //整体思想参考https://programmercarl.com/0027.移除元素.html
        for (int i = 0; i < s.size(); ++i) { //
            if (s[i] != ' ') { //遇到非空格就处理，即删除所有空格。
                if (slow != 0) s[slow++] = ' '; //手动控制空格，给单词之间添加空格。slow != 0 说明不是第一个单词，需要在单词前添加空格。
                while (i < s.size() && s[i] != ' ') { //补上该单词，遇到空格说明单词结束。
                    s[slow++] = s[i++];
                }
            }
        }
        s.resize(slow); //slow的大小即为去除多余空格后的大小。
    }

    string reverseWords(string s) {
        removeExtraSpaces(s); //去除多余空格，保证单词之间之只有一个空格，且字符串首尾没空格。
        reverse(s, 0, s.size() - 1);
        int start = 0; //removeExtraSpaces后保证第一个单词的开始下标一定是0。
        for (int i = 0; i <= s.size(); ++i) {
            if (i == s.size() || s[i] == ' ') { //到达空格或者串尾，说明一个单词结束。进行翻转。
                reverse(s, start, i - 1); //翻转，注意是左闭右闭 []的翻转。
                start = i + 1; //更新下一个单词的开始下标start
            }
        }
        return s;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1) 或 O(n)，取决于语言中字符串是否可变

> 反转链表的写法很简单，一些同学甚至可以背下来但过一阵就忘了该咋写，主要是因为没有理解真正的反转过程。

5.反转链表

[力扣题目链接]

题意：反转一个单链表。

示例:
输入: 1->2->3->4->5->NULL
输出: 5->4->3->2->1->NULL

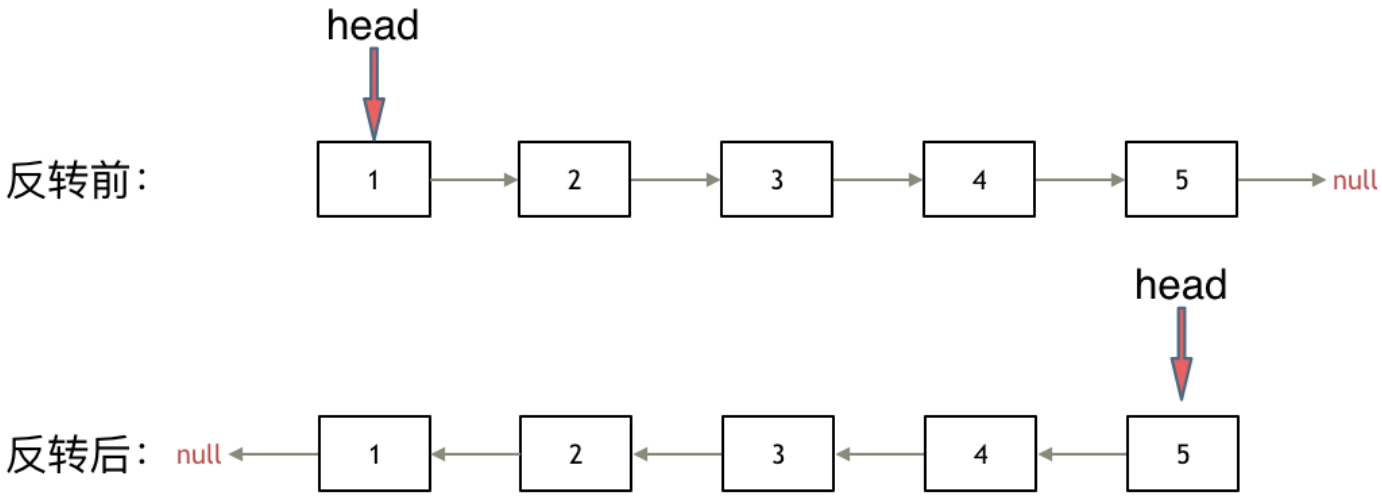
算法公开课

[《代码随想录》算法视频公开课](#)：[帮你拿下反转链表 | LeetCode：206.反转链表](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

如果再定义一个新的链表，实现链表元素的反转，其实这是对内存空间的浪费。

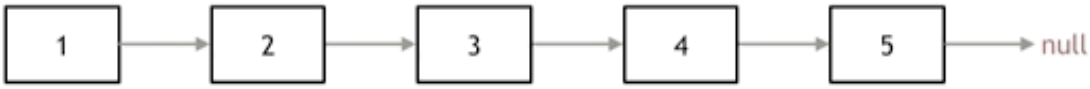
其实只需要改变链表的next指针的指向，直接将链表反转 ，而不用重新定义一个新的链表，如图所示:



之前链表的头节点是元素1， 反转之后头结点就是元素5 ，这里并没有添加或者删除节点，仅仅是改变next指针的方向。

那么接下来看一看是如何反转的呢?

我们拿有示例中的链表来举例，如动画所示：（纠正：动画应该是先移动pre，在移动cur）



首先定义一个cur指针，指向头结点，再定义一个pre指针，初始化为null。

然后就要开始反转了，首先要把 cur->next 节点用tmp指针保存一下，也就是保存一下这个节点。

为什么要保存一下这个节点呢，因为接下来要改变 cur->next 的指向了，将cur->next 指向pre ，此时已经反转了第一个节点了。

接下来，就是循环走如下代码逻辑了，继续移动pre和cur指针。

最后，cur 指针已经指向了null，循环结束，链表也反转完毕了。 此时我们return pre指针就可以了，pre指针就指向了新的头结点。

双指针法

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* temp; // 保存cur的下一个节点
        ListNode* cur = head;
        ListNode* pre = NULL;
        while(cur) {
            temp = cur->next;  // 保存一下 cur的下一个节点，因为接下来要改变cur->next
            cur->next = pre; // 翻转操作
            // 更新pre 和 cur指针
            pre = cur;
            cur = temp;
        }
```

```
        return pre;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

递归法

递归法相对抽象一些，但是其实和双指针法是一样的逻辑，同样是当cur为空的时候循环结束，不断将cur指向pre的过程。

关键是初始化的地方，可能有的同学会不理解， 可以看到双指针法中初始化 cur = head，pre = NULL，在递归法中可以从如下代码看出初始化的逻辑也是一样的，只不过写法变了。

具体可以看代码（已经详细注释），**双指针法写出来之后，理解如下递归写法就不难了，代码逻辑都是一样的。**

```
class Solution {
public:
    ListNode* reverse(ListNode* pre,ListNode* cur){
        if(cur == NULL) return pre;
        ListNode* temp = cur->next;
        cur->next = pre;
        // 可以和双指针法的代码进行对比，如下递归的写法，其实就是做了这两步
        // pre = cur;
        // cur = temp;
        return reverse(cur,temp);
    }
    ListNode* reverseList(ListNode* head) {
        // 和双指针法初始化是一样的逻辑
        // ListNode* cur = head;
        // ListNode* pre = NULL;
        return reverse(NULL, head);
    }

};
```

- 时间复杂度: O(n), 要递归处理链表的每个节点
- 空间复杂度: O(n), 递归调用了 n 层栈空间

我们可以发现，上面的递归写法和双指针法实质上都是从前往后翻转指针指向，其实还有另外一种与双指针法不同思路的递归写法：从后往前翻转指针指向。

具体代码如下（带详细注释）：

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        // 边缘条件判断
```

```
        if(head == NULL) return NULL;
        if (head->next == NULL) return head;

        // 递归调用，翻转第二个节点开始往后的链表
        ListNode *last = reverseList(head->next);
        // 翻转头节点与第二个节点的指向
        head->next->next = head;
        // 此时的 head 节点为尾节点，next 需要指向 NULL
        head->next = NULL;
        return last;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

其他解法

使用虚拟头结点解决链表反转

> 使用虚拟头结点，通过头插法实现链表的反转（不需要栈）

```
// 迭代方法：增加虚头结点，使用头插法实现链表翻转
public static ListNode reverseList1(ListNode head) {
    // 创建虚头结点
    ListNode dumpyHead = new ListNode(-1);
    dumpyHead.next = null;
    // 遍历所有节点
    ListNode cur = head;
    while(cur != null){
        ListNode temp = cur.next;
        // 头插法
        cur.next = dumpyHead.next;
        dumpyHead.next = cur;
        cur = temp;
    }
    return dumpyHead.next;
}
```

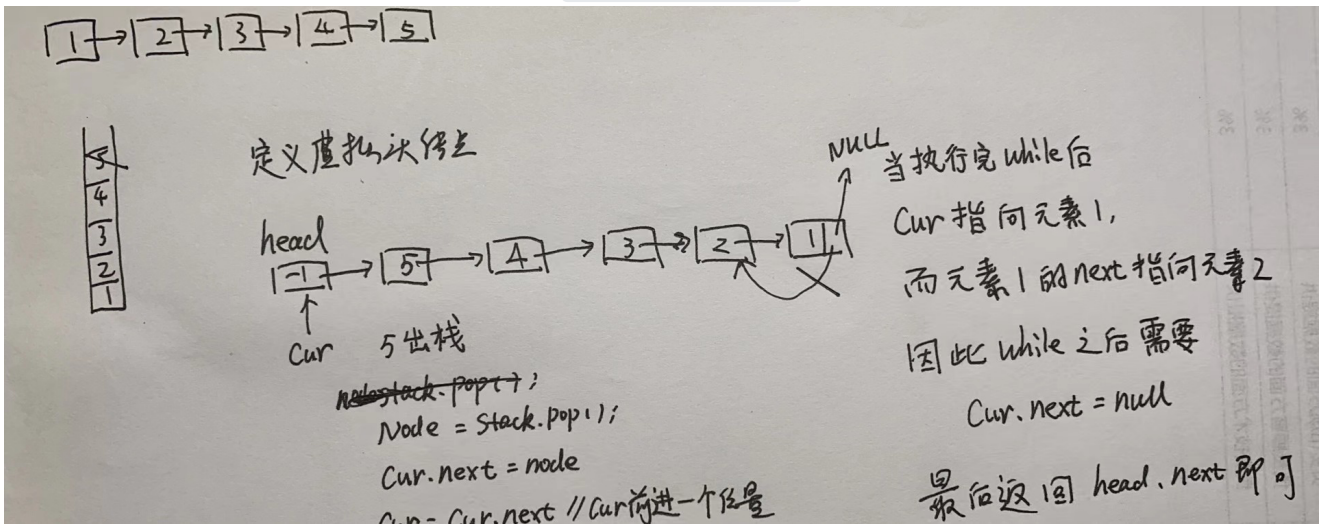
使用栈解决反转链表的问题

- 首先将所有的结点入栈
- 然后创建一个虚拟虚拟头结点，让cur指向虚拟头结点。然后开始循环出栈，每出来一个元素，就把它加入到以虚拟头结点为头结点的链表当中，最后返回即可。

```
public ListNode reverseList(ListNode head) {
```

```
        // 如果链表为空，则返回空
        if (head == null) return null;
        // 如果链表中只有只有一个元素，则直接返回
        if (head.next == null) return head;
        // 创建栈 每一个结点都入栈
        Stack<ListNode> stack = new Stack<>();
        ListNode cur = head;
        while (cur != null) {
            stack.push(cur);
            cur = cur.next;
        }
        // 创建一个虚拟头结点
        ListNode pHead = new ListNode(0);
        cur = pHead;
        while (!stack.isEmpty()) {
            ListNode node = stack.pop();
            cur.next = node;
            cur = cur.next;
        }
        // 最后一个元素的next要赋值为空
        cur.next = null;
        return pHead.next;
    }
```

> 采用这种方法需要注意一点。就是当整个出栈循环结束以后，cur正好指向原来链表的第一个结点，而此时结点1中的next指向的是结点2，因此最后还需要`cur.next = null`



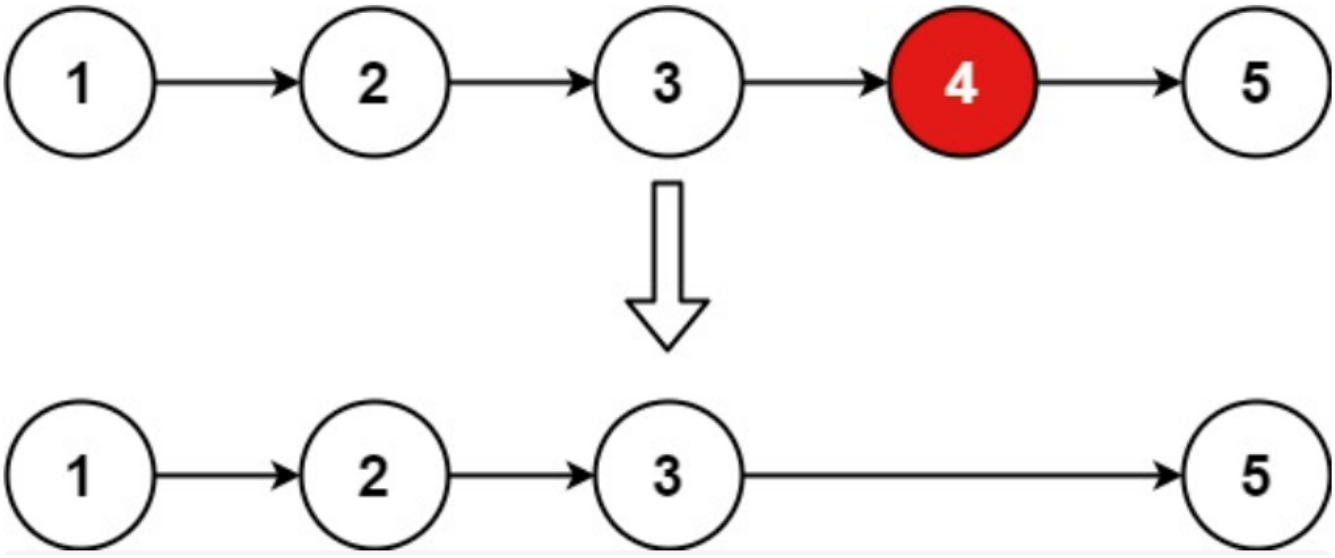
6.删除链表的倒数第N个节点

[力扣题目链接]

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

进阶：你能尝试使用一趟扫描实现吗?

示例 1：



输入：head = [1,2,3,4,5], n = 2
输出：[1,2,3,5]
示例 2：

输入：head = [1], n = 1
输出：[]
示例 3：

输入：head = [1,2], n = 1
输出：[1]

算法公开课

《代码随想录》算法视频公开课：：链表遍历学清楚！ | LeetCode：19.删除链表倒数第N个节点，相信结合视频再看本篇题解，更有助于大家对链表的理解。

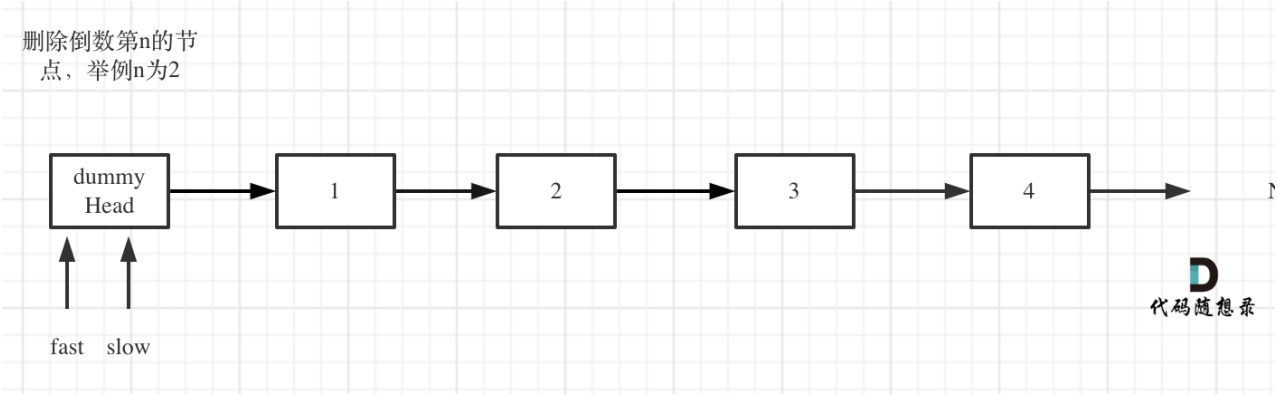
思路

双指针的经典应用，如果要删除倒数第n个节点，让fast移动n步，然后让fast和slow同时移动，直到fast指向链表末尾。删掉slow所指向的节点就可以了。

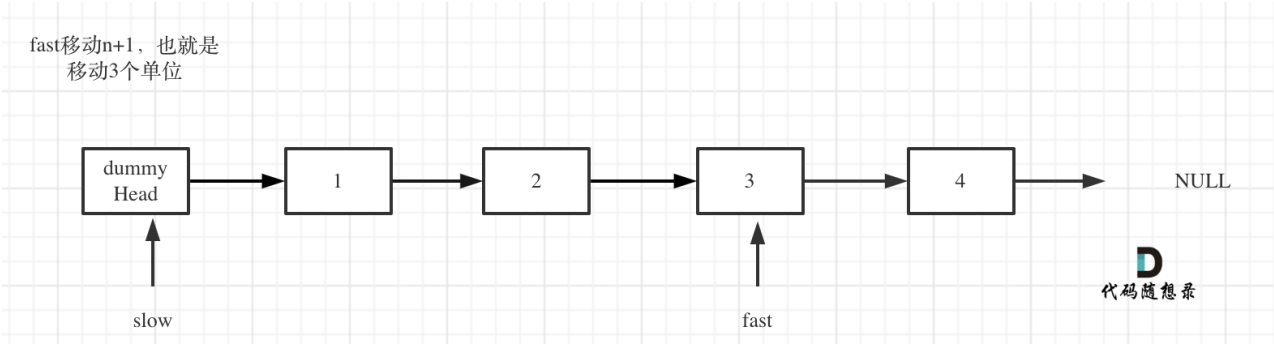
思路是这样的，但要注意一些细节。

分为如下几步：

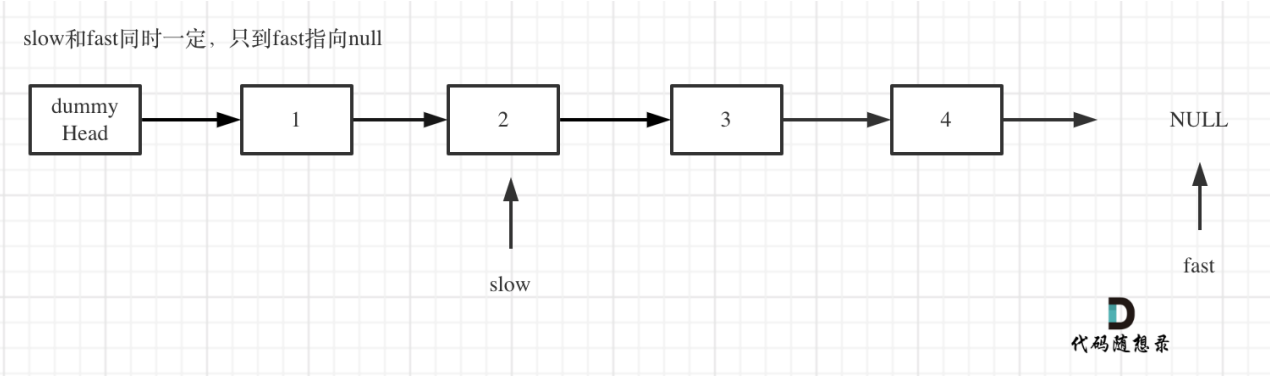
- 首先这里我推荐大家使用虚拟头结点，这样方便处理删除实际头结点的逻辑，如果虚拟头结点不清楚，可以看这篇： 链表：听说用虚拟头节点会方便很多?
- 定义fast指针和slow指针，初始值为虚拟头结点，如图：



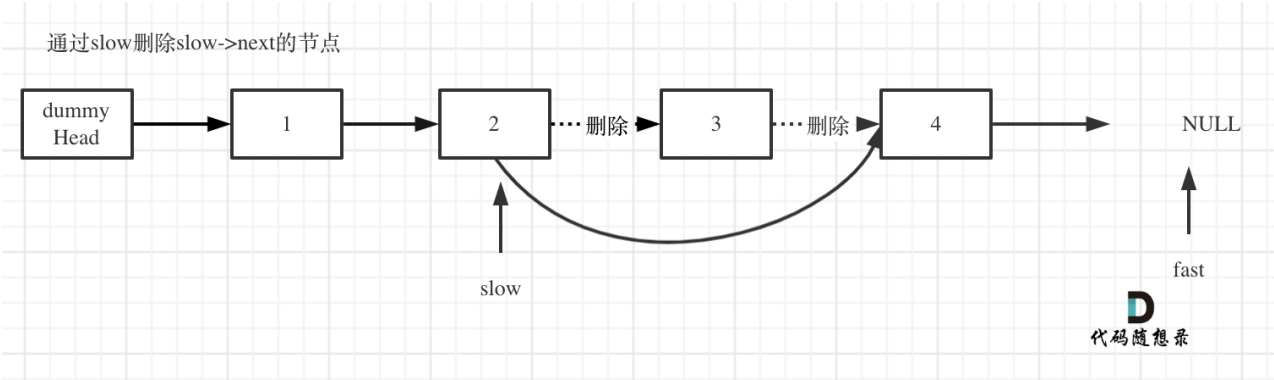
- fast首先走n + 1步 ，为什么是n+1呢，因为只有这样同时移动的时候slow才能指向删除节点的上一个节点（方便做删除操作），如图：



- fast和slow同时移动，直到fast指向末尾，如题：



- 删除slow指向的下一个节点，如图：



此时不难写出如下C++代码：

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* dummyHead = new ListNode(0);
```

```
        dummyHead->next = head;
        ListNode* slow = dummyHead;
        ListNode* fast = dummyHead;
        while(n-- && fast != NULL) {
            fast = fast->next;
        }
        fast = fast->next; // fast再提前走一步，因为需要让slow指向删除节点的上一个节点
        while (fast != NULL) {
            fast = fast->next;
            slow = slow->next;
        }
        slow->next = slow->next->next;

        // ListNode *tmp = slow->next;  C++释放内存的逻辑
        // slow->next = tmp->next;
        // delete nth;

        return dummyHead->next;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

---

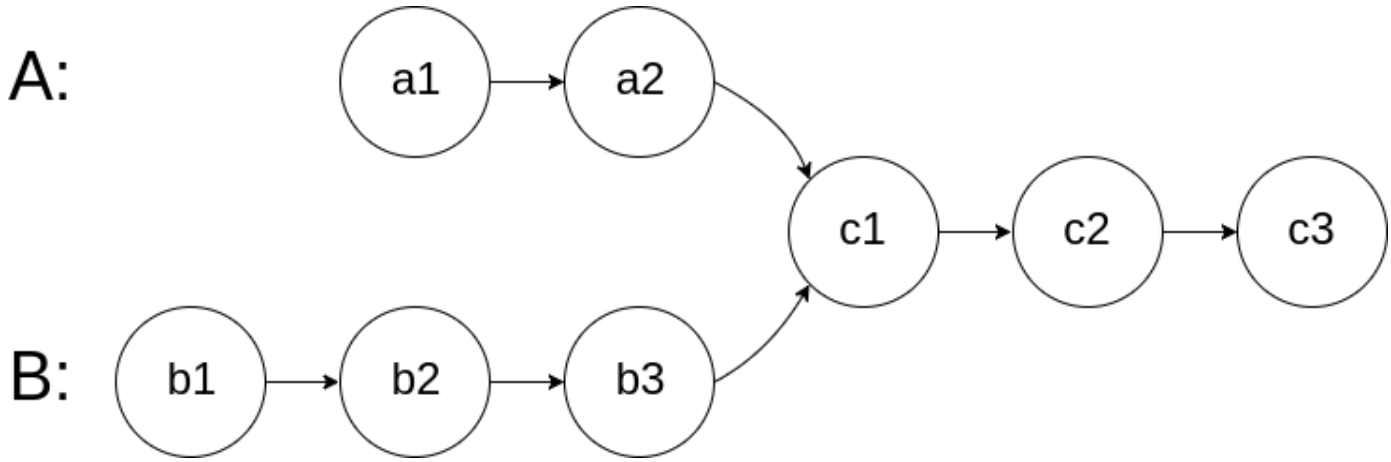
7. 面试题 02.07. 链表相交

同：160.链表相交

[力扣题目链接]

给你两个单链表的头节点 headA 和 headB ，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 null 。

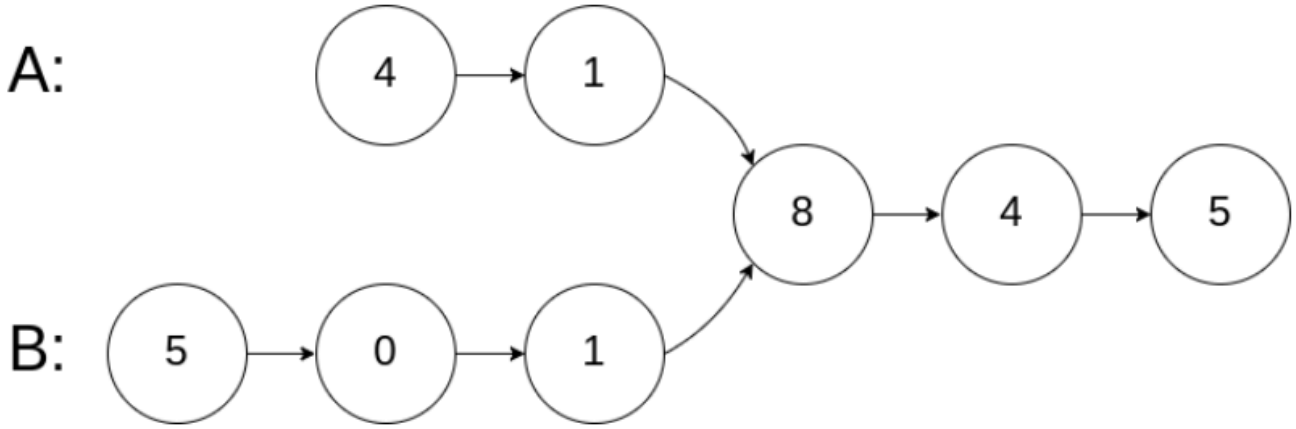
图示两个链表在节点 c1 开始相交：



题目数据 保证 整个链式结构中不存在环。

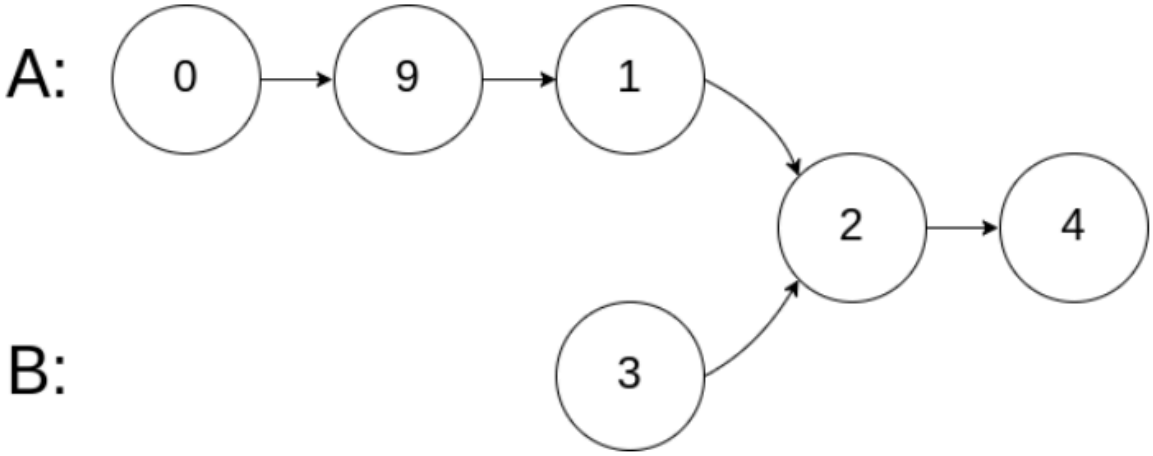
注意，函数返回结果后，链表必须 保持其原始结构 。

示例 1：



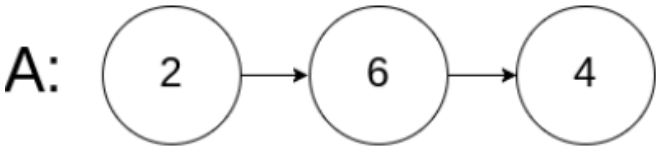
输入：intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3
输出：Intersected at '8'
解释：相交节点的值为 8 （注意，如果两个链表相交则不能为 0）。
从各自的表头开始算起，链表 A 为 [4,1,8,4,5]，链表 B 为 [5,0,1,8,4,5]。
在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。

示例 2：

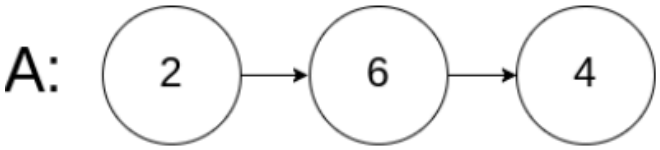


输入： intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
输出： Intersected at '2'
解释： 相交节点的值为 2 （注意，如果两个链表相交则不能为 0）。
从各自的表头开始算起，链表 A 为 [0,9,1,2,4]，链表 B 为 [3,2,4]。
在 A 中，相交节点前有 3 个节点；在 B 中，相交节点前有 1 个节点。

示例 3：



输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
输出：null
解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。
由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA 和 skipB 可以是任意值。
这两个链表不相交，因此返回 null 。



输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
输出：null
解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。
由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA

和 `skipB` 可以是任意值。

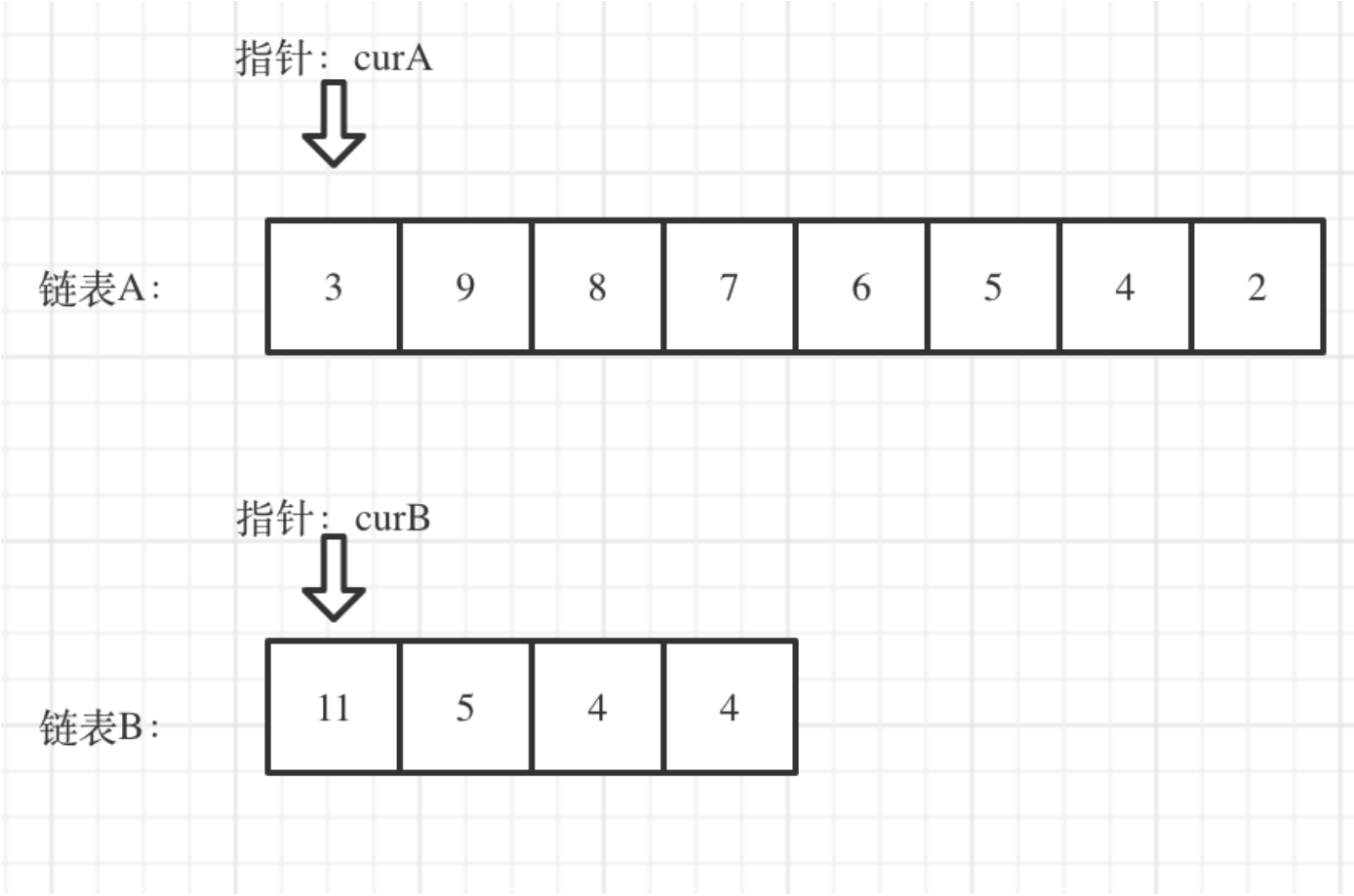
这两个链表不相交，因此返回 `null` 。

思路

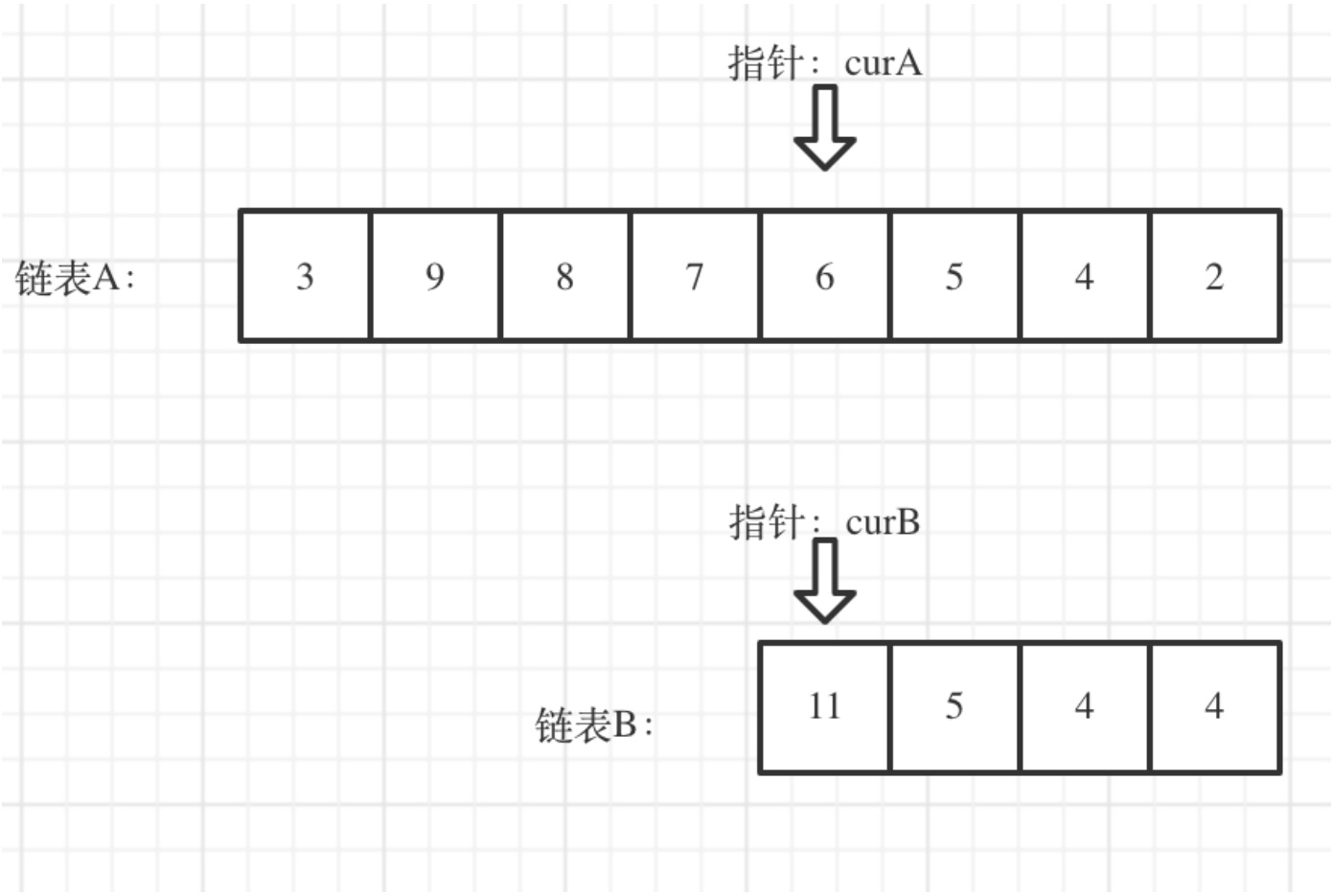
简单来说，就是求两个链表交点节点的**指针**。 这里同学们要注意，交点不是数值相等，而是指针相等。

为了方便举例，假设节点元素数值相等，则节点指针相等。

看如下两个链表，目前curA指向链表A的头结点，curB指向链表B的头结点：



我们求出两个链表的长度，并求出两个链表长度的差值，然后让curA移动到，和curB 末尾对齐的位置，如图：



此时我们就可以比较curA和curB是否相同，如果不相同，同时向后移动curA和curB，如果遇到curA == curB，则找到交点。

否则循环退出返回空指针。

C++代码如下：

```
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        ListNode* curA = headA;
        ListNode* curB = headB;
        int lenA = 0, lenB = 0;
        while (curA != NULL) { // 求链表A的长度
            lenA++;
            curA = curA->next;
        }
        while (curB != NULL) { // 求链表B的长度
            lenB++;
            curB = curB->next;
        }
        curA = headA;
        curB = headB;
        // 让curA为最长链表的头，lenA为其长度
        if (lenB > lenA) {
            swap (lenA, lenB);
```

```
            swap (curA, curB);
        }
        // 求长度差
        int gap = lenA - lenB;
        // 让curA和curB在同一起点上（末尾位置对齐）
        while (gap--) {
            curA = curA->next;
        }
        // 遍历curA 和 curB，遇到相同则直接返回
        while (curA != NULL) {
            if (curA == curB) {
                return curA;
            }
            curA = curA->next;
            curB = curB->next;
        }
        return NULL;
    }
};
```

- 时间复杂度：O(n + m)
- 空间复杂度：O(1)

> 找到有没有环已经很不容易了，还要让我找到环的入口?

8.环形链表II

[力扣题目链接]

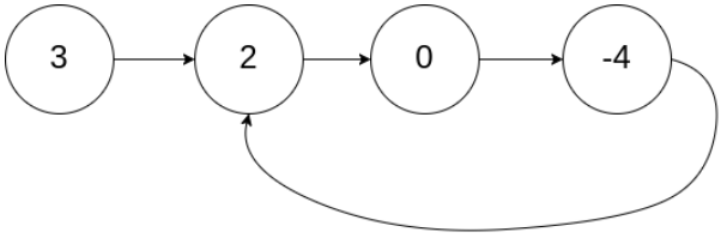
题意：
给定一个链表，返回链表开始入环的第一个节点。 如果链表无环，则返回 null。

为了表示给定链表中的环，使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。 如果 pos 是 -1，则在该链表中没有环。

说明：不允许修改给定的链表。

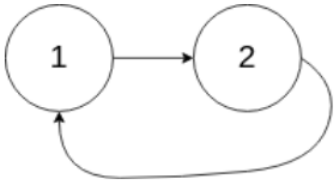
示例 1：

```
输入：head = [3,2,0,-4], pos = 1
输出：tail connects to node index 1
解释：链表中有一个环，其尾部连接到第二个节点。
```



示例 2：

```
输入：head = [1,2], pos = 0
输出：tail connects to node index 0
解释：链表中有一个环，其尾部连接到第一个节点。
```



算法公开课

[《代码随想录》算法视频公开课](#)：[把环形链表讲清楚！ | LeetCode:142.环形链表II](#)，**相信结合视频在看本篇题解，更有助于大家对链表的理解。**

思路

这道题目，不仅考察对链表的操作，而且还需要一些数学运算。

主要考察两知识点：

- 判断链表是否环
- 如果有环，如何找到这个环的入口

判断链表是否有环

可以使用快慢指针法，分别定义 fast 和 slow 指针，从头结点出发，fast指针每次移动两个节点，slow指针每次移动一个节点，如果 fast 和 slow指针在途中相遇 ，说明这个链表有环。

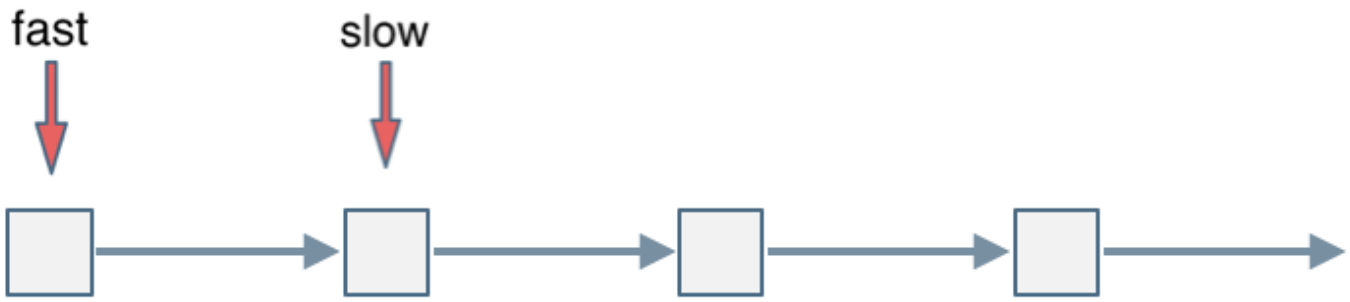
为什么fast 走两个节点，slow走一个节点，有环的话，一定会在环内相遇呢，而不是永远的错开呢

首先第一点：**fast指针一定先进入环中，如果fast指针和slow指针相遇的话，一定是在环中相遇，这是毋庸置疑的。**

那么来看一下，**为什么fast指针和slow指针一定会相遇呢?**

可以画一个环，然后让 fast指针在任意一个节点开始追赶slow指针。

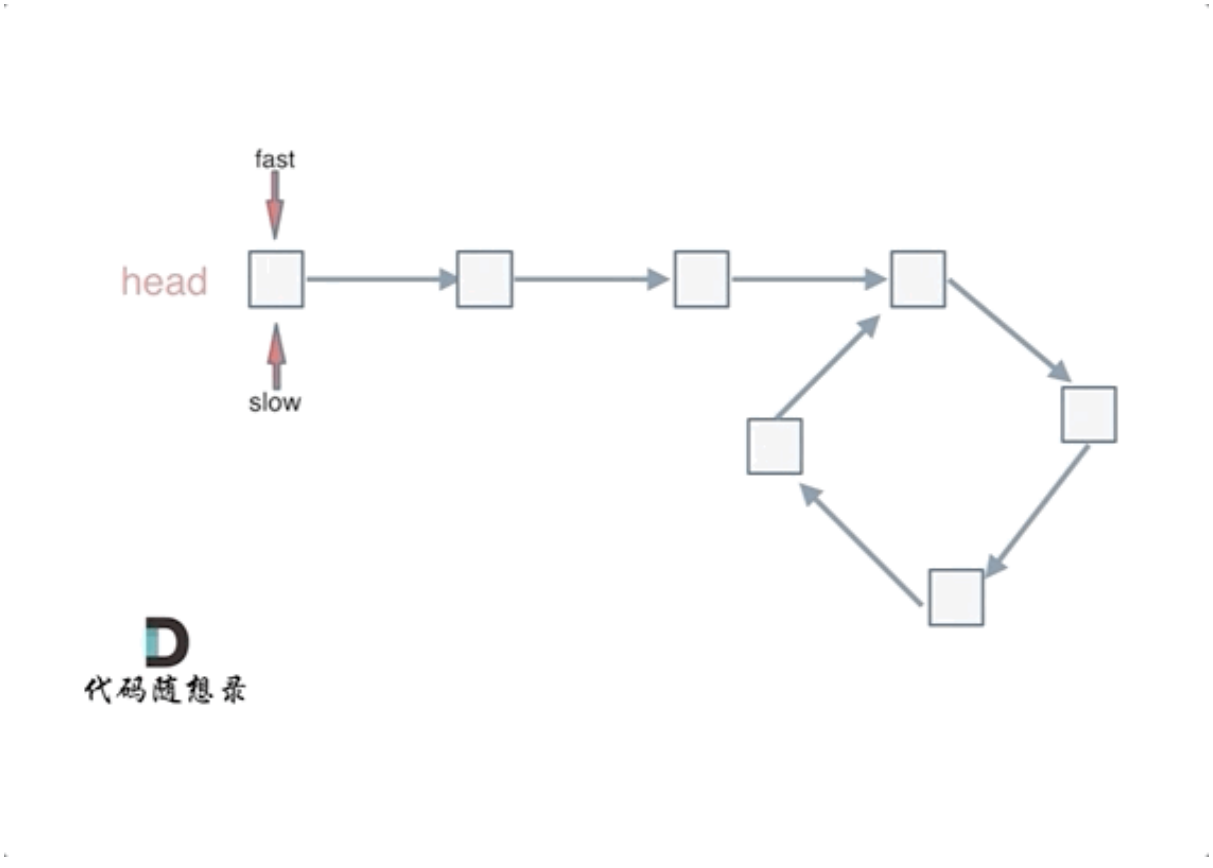
会发现最终都是这种情况， 如下图：



fast和slow各自再走一步， fast和slow就相遇了

这是因为fast是走两步，slow是走一步，**其实相对于slow来说，fast是一个节点一个节点的靠近slow的**，所以fast一定可以和slow重合。

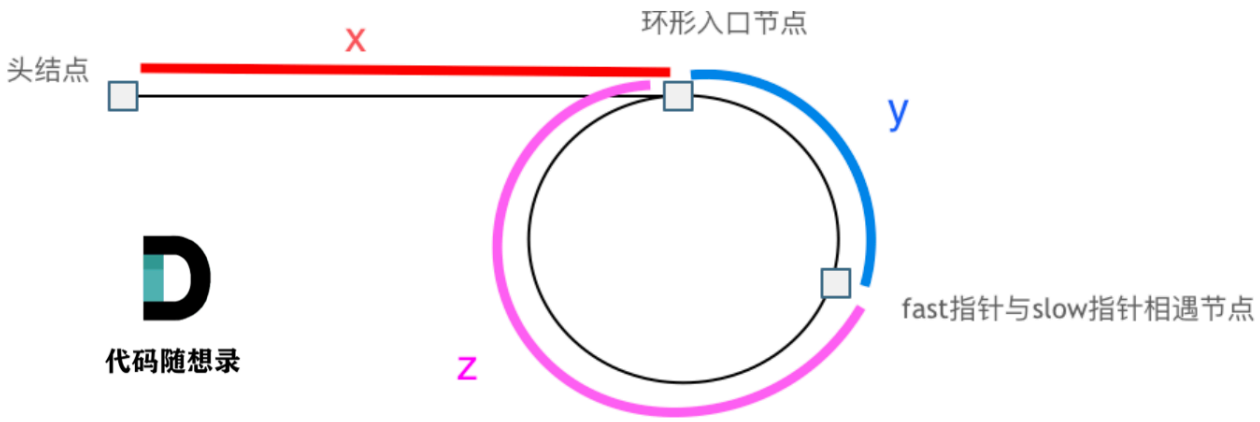
动画如下：



如果有环，如何找到这个环的入口

此时已经可以判断链表是否有环了，那么接下来要找这个环的入口了。

假设从头结点到环形入口节点 的节点数为x。
环形入口节点到 fast指针与slow指针相遇节点 节点数为y。
从相遇节点 再到环形入口节点节点数为 z。 如图所示：



那么相遇时：
slow指针走过的节点数为: `x + y`，
fast指针走过的节点数： `x + y + n (y + z)`，n为fast指针在环内走了n圈才遇到slow指针， （y+z）为 一圈内节点的个数A。

因为fast指针是一步走两个节点，slow指针一步走一个节点， 所以 fast指针走过的节点数 = slow指针走过的节点数 * 2：

`(x + y) * 2 = x + y + n (y + z)`

两边消掉一个（x+y）：`x + y  = n (y + z)`

因为要找环形的入口，那么要求的是x，因为x表示 头结点到 环形入口节点的的距离。

所以要求x ，将x单独放在左面：`x = n (y + z) - y`，

再从n(y+z)中提出一个 （y+z）来，整理公式之后为如下公式：`x = (n - 1) (y + z) + z` 注意这里n一定是大于等于1的，因为 fast指针至少要多走一圈才能相遇slow指针。

这个公式说明什么呢?

先拿n为1的情况来举例，意味着fast指针在环形里转了一圈之后，就遇到了 slow指针了。

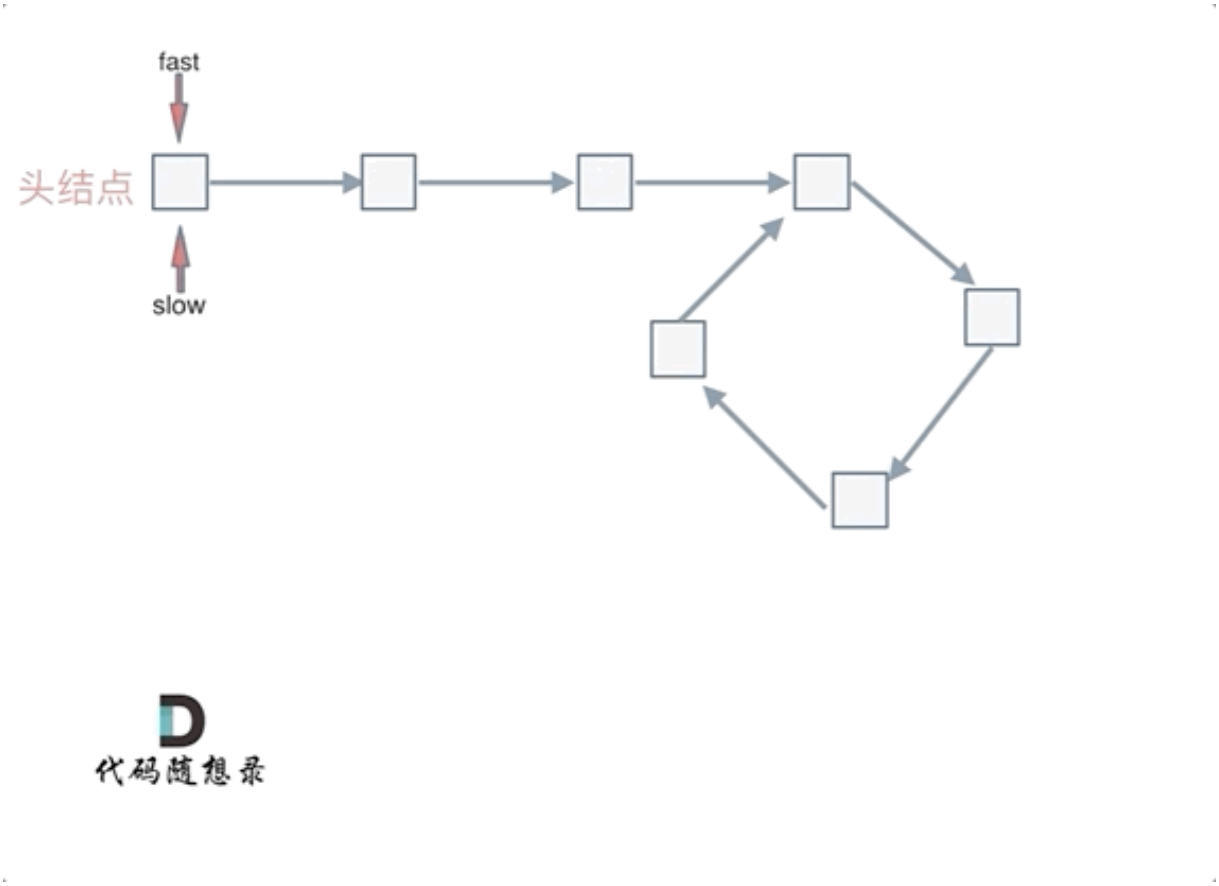
当 n为1的时候，公式就化解为 `x = z`，

这就意味着，**从头结点出发一个指针，从相遇节点 也出发一个指针，这两个指针每次只走一个节点， 那么当这两个指针相遇的时候就是 环形入口的节点。**

也就是在相遇节点处，定义一个指针index1，在头结点处定一个指针index2。

让index1和index2同时移动，每次移动一个节点， 那么他们相遇的地方就是 环形入口的节点。

动画如下：



那么 n如果大于1是什么情况呢，就是fast指针在环形转n圈之后才遇到 slow指针。

其实这种情况和n为1的时候 效果是一样的，一样可以通过这个方法找到 环形的入口节点，只不过，index1 指针在环里 多转了(n-1)圈，然后再遇到index2，相遇点依然是环形的入口节点。

代码如下：

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        ListNode* fast = head;
        ListNode* slow = head;
        while(fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
            // 快慢指针相遇，此时从head 和 相遇点，同时查找直至相遇
            if (slow == fast) {
                ListNode* index1 = fast;
                ListNode* index2 = head;
```

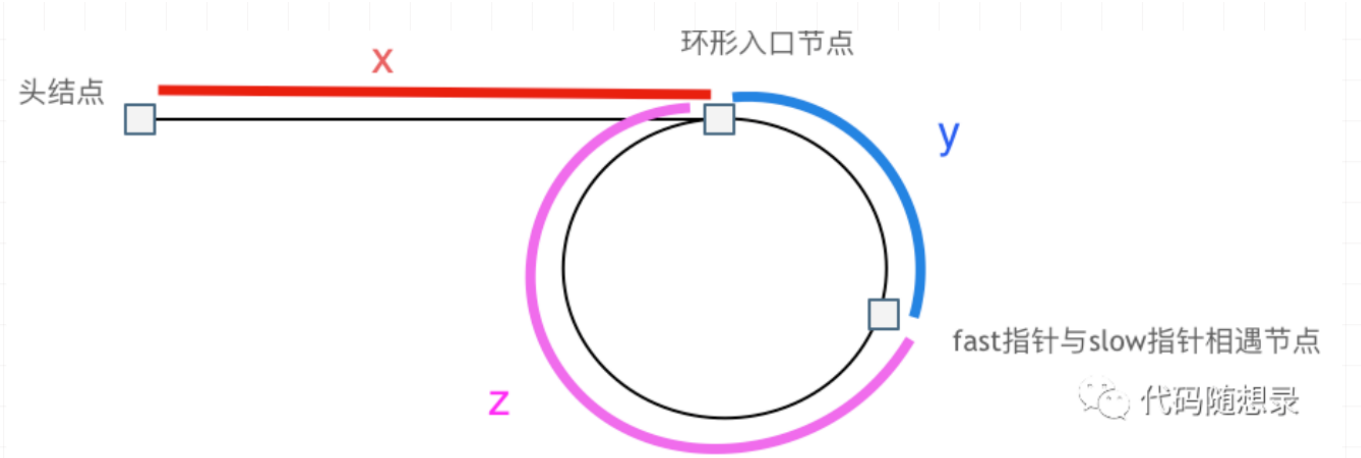
```
                while (index1 != index2) {
                    index1 = index1->next;
                    index2 = index2->next;
                }
                return index2; // 返回环的入口
            }
        }
        return NULL;
    }
};
```

- 时间复杂度: O(n)，快慢指针相遇前，指针走的次数小于链表长度，快慢指针相遇后，两个index指针走的次数也小于链表长度，总体为走的次数小于 2n
- 空间复杂度: O(1)

补充

在推理过程中，大家可能有一个疑问就是：**为什么第一次在环中相遇，slow的 步数 是 x+y 而不是 x + 若干环的长度 + y 呢?**

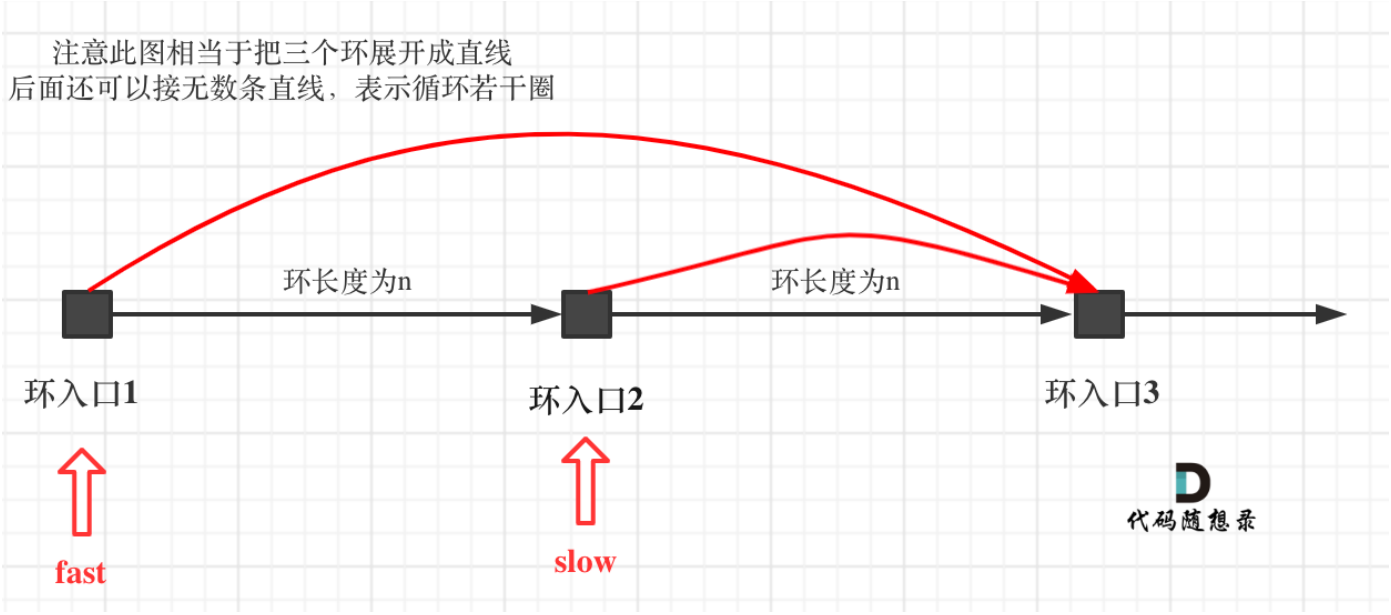
即文章链表：环找到了，那入口呢? 中如下的地方：



那么相遇时：slow指针走过的节点数为：`x + y`，fast指针走过的节点数：`x + y + n (y + z)`，n为fast指针在环内走了n圈才遇到slow指针，（y+z）为一圈内节点的个数A。

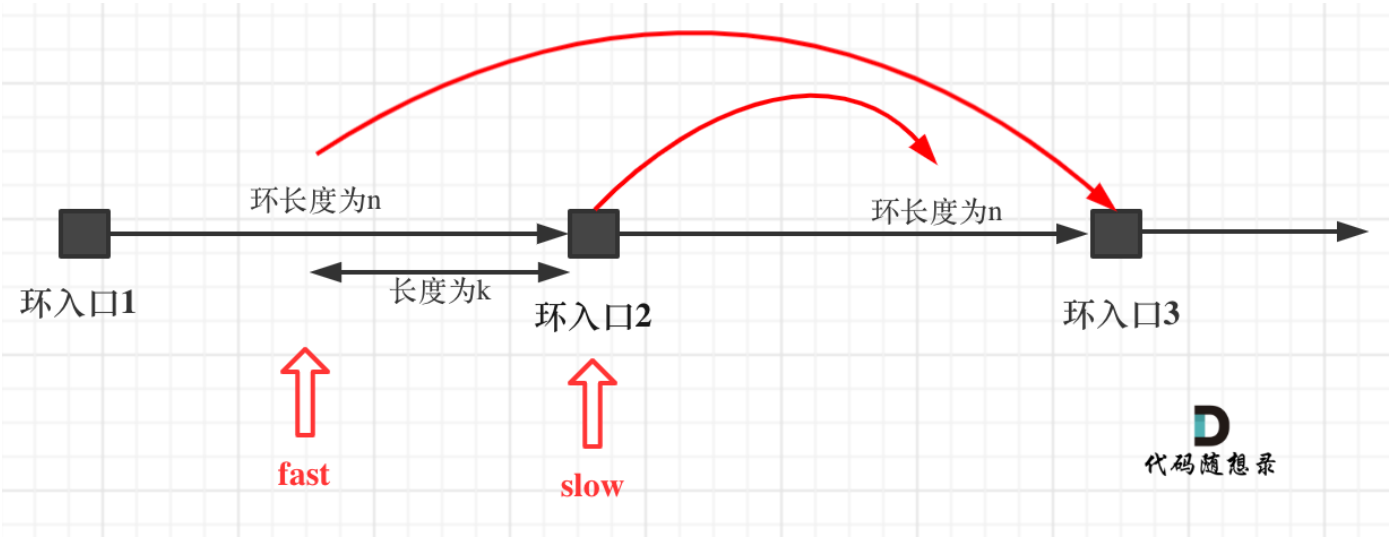
首先slow进环的时候，fast一定是先进环来了。

如果slow进环入口，fast也在环入口，那么把这个环展开成直线，就是如下图的样子：



可以看出如果slow 和 fast同时在环入口开始走，一定会在环入口3相遇，slow走了一圈，fast走了两圈。

重点来了，slow进环的时候，fast一定是在环的任意一个位置，如图：



那么fast指针走到环入口3的时候，已经走了k + n 个节点，slow相应的应该走了(k + n) / 2 个节点。

因为k是小于n的（图中可以看出），所以(k + n) / 2 一定小于n。

也就是说slow一定没有走到环入口3，而fast已经到环入口3了。

这说明什么呢?

在slow开始走的那一环已经和fast相遇了。

那有同学又说了，为什么fast不能跳过去呢？ 在刚刚已经说过一次了，**fast相对于slow是一次移动一个节点，所以不可能跳过去。**

好了，这次把为什么第一次在环中相遇，slow的 步数 是 x+y 而不是 x + 若干环的长度 + y ，用数学推理了一下，算是对链表：环找到了，那入口呢？的补充。

总结

这次可以说把环形链表这道题目的各个细节，完完整整的证明了一遍，说这是全网最详细讲解不为过吧，哈哈。

> 用哈希表解决了两数之和，那么三数之和呢?

9. 三数之和

力扣题目链接

给你一个包含 n 个整数的数组 nums，判断 nums 中是否存在三个元素 a，b，c ，使得 a + b + c = 0 ？请你找出所有满足条件且不重复的三元组。

注意： 答案中不可以包含重复的三元组。

示例：

给定数组 nums = [-1, 0, 1, 2, -1, -4]，

满足要求的三元组集合为：
[
  [-1, 0, 1],
  [-1, -1, 2]
]

算法公开课

《代码随想录》算法视频公开课：梦破碎的地方！| LeetCode：15.三数之和，相信结合视频再看本篇题解，更有助于大家对本题的理解。

注意[0， 0， 0， 0] 这组数据

思路

哈希解法

两层for循环就可以确定 a 和b 的数值了，可以使用哈希法来确定 0-(a+b) 是否在 数组里出现过，其实这个思路是正确的，但是我们有一个非常棘手的问题，就是题目中说的不可以包含重复的三元组。

把符合条件的三元组放进vector中，然后再去重，这样是非常费时的，很容易超时，也是这道题目通过率如此之低的根源所在。

去重的过程不好处理，有很多小细节，如果在面试中很难想到位。

时间复杂度可以做到O(n^2)，但还是比较费时的，因为不好做剪枝操作。

大家可以尝试使用哈希法写一写，就知道其困难的程度了。

哈希法C++代码:

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;
```

```
        sort(nums.begin(), nums.end());
        // 找出a + b + c = 0
        // a = nums[i], b = nums[j], c = -(a + b)
        for (int i = 0; i < nums.size(); i++) {
            // 排序之后如果第一个元素已经大于零，那么不可能凑成三元组
            if (nums[i] > 0) {
                break;
            }
            if (i > 0 && nums[i] == nums[i - 1]) { //三元组元素a去重
                continue;
            }
            unordered_set<int> set;
            for (int j = i + 1; j < nums.size(); j++) {
                if (j > i + 2
                        && nums[j] == nums[j-1]
                        && nums[j-1] == nums[j-2]) { // 三元组元素b去重
                    continue;
                }
                int c = 0 - (nums[i] + nums[j]);
                if (set.find(c) != set.end()) {
                    result.push_back({nums[i], nums[j], c});
                    set.erase(c);// 三元组元素c去重
                } else {
                    set.insert(nums[j]);
                }
            }
        }
        return result;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(n)，额外的 set 开销

双指针

其实这道题目使用哈希法并不十分合适，因为在去重的操作中有很多细节需要注意，在面试中很难直接写出没有bug的代码。

而且使用哈希法 在使用两层for循环的时候，能做的剪枝操作很有限，虽然时间复杂度是O(n^2)，也是可以在leetcode上通过，但是程序的执行时间依然比较长 。

接下来我来介绍另一个解法：双指针法，**这道题目使用双指针法 要比哈希法高效一些**，那么来讲解一下具体实现的思路。

动画效果如下：

拿这个nums数组来举例，首先将数组排序，然后有一层for循环，i从下标0的地方开始，同时定一个下标left 定义在i+1的位置上，定义下标right 在数组结尾的位置上。

依然还是在数组中找到 abc 使得a + b +c =0，我们这里相当于 a = nums[i]，b = nums[left]，c = nums[right]。

接下来如何移动left 和right呢， 如果nums[i] + nums[left] + nums[right] > 0 就说明 此时三数之和大了，因为数组是排序后了，所以right下标就应该向左移动，这样才能让三数之和小一些。

如果 nums[i] + nums[left] + nums[right] < 0 说明 此时 三数之和小了，left 就向右移动，才能让三数之和大一些，直到left与right相遇为止。

时间复杂度：O(n^2)。

C++代码代码如下：

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        // 找出a + b + c = 0
        // a = nums[i], b = nums[left], c = nums[right]
        for (int i = 0; i < nums.size(); i++) {
            // 排序之后如果第一个元素已经大于零，那么无论如何组合都不可能凑成三元组，直接返回结果就可以了
            if (nums[i] > 0) {
                return result;
            }
            // 错误去重a方法，将会漏掉-1,-1,2 这种情况
            /*
            if (nums[i] == nums[i + 1]) {
                continue;
            }
            */
            // 正确去重a方法
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
```

```
            }
            int left = i + 1;
            int right = nums.size() - 1;
            while (right > left) {
                // 去重复逻辑如果放在这里，0, 0, 0 的情况，可能直接导致 right<=left 了，从而漏掉了 0,0,0 这种三元组
                /*
                while (right > left && nums[right] == nums[right - 1]) right--;
                while (right > left && nums[left] == nums[left + 1]) left++;
                */
                if (nums[i] + nums[left] + nums[right] > 0) right--;
                else if (nums[i] + nums[left] + nums[right] < 0) left++;
                else {
                    result.push_back(vector<int>{nums[i], nums[left], nums[right]});
                    // 去重逻辑应该放在找到一个三元组之后，对b 和 c去重
                    while (right > left && nums[right] == nums[right - 1]) right--;
                    while (right > left && nums[left] == nums[left + 1]) left++;

                    // 找到答案时，双指针同时收缩
                    right--;
                    left++;
                }
            }

        }
        return result;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(1)

去重逻辑的思考

a的去重

说道去重，其实主要考虑三个数的去重。 a, b ,c，对应的就是 nums[i]，nums[left]，nums[right]

a 如果重复了怎么办，a是nums里遍历的元素，那么应该直接跳过去。

但这里有一个问题，是判断 nums[i] 与 nums[i + 1]是否相同，还是判断 nums[i] 与 nums[i-1] 是否相同。

有同学可能想，这不都一样吗。

其实不一样！

都是和 nums[i]进行比较，是比较它的前一个，还是比较他的后一个。

如果我们的写法是 这样：

```
if (nums[i] == nums[i + 1]) { // 去重操作
    continue;
}
```

那就我们就把 三元组中出现重复元素的情况直接pass掉了。 例如{-1, -1 ,2} 这组数据，当遍历到第一个-1 的时候，判断 下一个也是-1，那这组数据就pass了。

我们要做的是 不能有重复的三元组，但三元组内的元素是可以重复的！

所以这里是有两个重复的维度。

那么应该这么写：

```
if (i > 0 && nums[i] == nums[i - 1]) {
    continue;
}
```

这么写就是当前使用 nums[i]，我们判断前一位是不是一样的元素，在看 {-1, -1 ,2} 这组数据，当遍历到 第一个 -1 的时候，只要前一位没有-1，那么 {-1, -1 ,2} 这组数据一样可以收录到 结果集里。

这是一个非常细节的思考过程。

b与c的去重

很多同学写本题的时候，去重的逻辑多加了 对right 和left 的去重：（代码中注释部分）

```
while (right > left) {
    if (nums[i] + nums[left] + nums[right] > 0) {
        right--;
        // 去重 right
        while (left < right && nums[right] == nums[right + 1]) right--;
    } else if (nums[i] + nums[left] + nums[right] < 0) {
        left++;
        // 去重 left
        while (left < right && nums[left] == nums[left - 1]) left++;
    } else {
    }
}
```

但细想一下，这种去重其实对提升程序运行效率是没有帮助的。

拿right去重为例，即使不加这个去重逻辑，依然根据 `while (right > left)` 和 `if (nums[i] + nums[left] + nums[right] > 0)` 去完成right-- 的操作。

多加了 `while (left < right && nums[right] == nums[right + 1]) right--;` 这一行代码，其实就是把需要执行的逻辑提前执行了，但并没有减少 判断的逻辑。

最直白的思考过程，就是right还是一个数一个数的减下去的，所以在哪里减的都是一样的。

所以这种去重 是可以不加的。 仅仅是 把去重的逻辑提前了而已。

思考题

既然三数之和可以使用双指针法，我们之前讲过的1.两数之和，可不可以使用双指针法呢?

如果不能，题意如何更改就可以使用双指针法呢？ **大家留言说出自己的想法吧！**

两数之和 就不能使用双指针法，因为1.两数之和要求返回的是索引下标， 而双指针法一定要排序，一旦排序之后原数组的索引就被改变了。

如果1.两数之和要求返回的是数值的话，就可以使用双指针法了。

> 一样的道理，能解决四数之和
> 那么五数之和、六数之和、N数之和呢?

10. 四数之和

力扣题目链接

题意：给定一个包含 n 个整数的数组 nums 和一个目标值 target，判断 nums 中是否存在四个元素 a，b，c 和 d ，使得 a + b + c + d 的值与 target 相等？找出所有满足条件且不重复的四元组。

注意：

答案中不可以包含重复的四元组。

示例：
给定数组 nums = [1, 0, -1, 0, -2, 2]，和 target = 0。
满足要求的四元组集合为：
```
[
  [-1,  0, 0, 1],
  [-2, -1, 1, 2],
  [-2,  0, 0, 2]
]
```

算法公开课

《代码随想录》算法视频公开课：难在去重和剪枝！| LeetCode：18. 四数之和，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

四数之和，和15.三数之和是一个思路，都是使用双指针法, 基本解法就是在15.三数之和 的基础上再套一层for循环。

但是有一些细节需要注意，例如： 不要判断`nums[k] > target` 就返回了，三数之和 可以通过 `nums[i] > 0` 就返回了，因为 0 已经是确定的数了，四数之和这道题目 target是任意值。比如：数组是`[-4, -3, -2, -1]`，`target`是`-10`，不能因为`-4 > -10`而跳过。但是我们依旧可以去做剪枝，逻辑变成`nums[i] > target && (nums[i] >=0 || target >= 0)`就可以了。

15.三数之和的双指针解法是一层for循环num[i]为确定值，然后循环内有left和right下标作为双指针，找到nums[i] + nums[left] + nums[right] == 0。

四数之和的双指针解法是两层for循环nums[k] + nums[i]为确定值，依然是循环内有left和right下标作为双指针，找出nums[k] + nums[i] + nums[left] + nums[right] == target的情况，三数之和的时间复杂度是O(n^2)，四数之和的时间复杂度是O(n^3) 。

那么一样的道理，五数之和、六数之和等等都采用这种解法。

对于15.三数之和双指针法就是将原本暴力O(n^3)的解法，降为O(n^2)的解法，四数之和的双指针解法就是将原本暴力O(n^4)的解法，降为O(n^3)的解法。

之前我们讲过哈希表的经典题目：454.四数相加II，相对于本题简单很多，因为本题是要求在一个集合中找出四个数相加等于target，同时四元组不能重复。

而454.四数相加II是四个独立的数组，只要找到A[i] + B[j] + C[k] + D[l] = 0就可以，不用考虑有重复的四个元素相加等于0的情况，所以相对于本题还是简单了不少！

我们来回顾一下，几道题目使用了双指针法。

双指针法将时间复杂度：O(n^2)的解法优化为 O(n)的解法。也就是降一个数量级，题目如下：

- 27.移除元素
- 15.三数之和
- 18.四数之和

链表相关双指针题目：

- 206.反转链表
- 19.删除链表的倒数第N个节点
- 面试题 02.07. 链表相交
- 142题.环形链表II

双指针法在字符串题目中还有很多应用，后面还会介绍到。

C++代码

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        for (int k = 0; k < nums.size(); k++) {
            // 剪枝处理
            if (nums[k] > target && nums[k] >= 0) {
              break; // 这里使用break，统一通过最后的return返回
            }
            // 对nums[k]去重
            if (k > 0 && nums[k] == nums[k - 1]) {
                continue;
            }
```

```
            for (int i = k + 1; i < nums.size(); i++) {
                // 2级剪枝处理
                if (nums[k] + nums[i] > target && nums[k] + nums[i] >= 0) {
                    break;
                }

                // 对nums[i]去重
                if (i > k + 1 && nums[i] == nums[i - 1]) {
                    continue;
                }
                int left = i + 1;
                int right = nums.size() - 1;
                while (right > left) {
                    // nums[k] + nums[i] + nums[left] + nums[right] > target 会溢出
                    if ((long) nums[k] + nums[i] + nums[left] + nums[right] > target) {
                        right--;
                    // nums[k] + nums[i] + nums[left] + nums[right] < target 会溢出
                    } else if ((long) nums[k] + nums[i] + nums[left] + nums[right]  <
target) {
                        left++;
                    } else {
                        result.push_back(vector<int>{nums[k], nums[i], nums[left],
nums[right]});
                        // 对nums[left]和nums[right]去重
                        while (right > left && nums[right] == nums[right - 1]) right--;
                        while (right > left && nums[left] == nums[left + 1]) left++;

                        // 找到答案时，双指针同时收缩
                        right--;
                        left++;
                    }
                }

            }
        }
        return result;
    }
};
```

- 时间复杂度: O(n^3)
- 空间复杂度: O(1)

补充

二级剪枝的部分：

```
if (nums[k] + nums[i] > target && nums[k] + nums[i] >= 0) {
    break;
}
```

可以优化为：

```
if (nums[k] + nums[i] > target && nums[i] >= 0) {
    break;
}
```

因为只要 nums[k] + nums[i] > target，那么 nums[i] 后面的数都是正数的话，就一定 不符合条件了。

不过这种剪枝 其实有点 小绕，大家能够理解 文章给的完整代码的剪枝 就够了。

> 又是一波总结

相信大家已经对双指针法很熟悉了，但是双指针法并不隶属于某一种数据结构，我们在讲解数组，链表，字符串都用到了双指针法，所有有必要针对双指针法做一个总结。

11. 双指针总结篇

数组篇

在数组：就移除个元素很难么？ 中，原地移除数组上的元素，我们说到了数组上的元素，不能真正的删除，只能覆盖。

一些同学可能会写出如下代码（伪代码）：

```
for (int i = 0; i < array.size(); i++) {
    if (array[i] == target) {
        array.erase(i);
    }
}
```

这个代码看上去好像是O(n)的时间复杂度，其实是O(n^2)的时间复杂度，因为erase操作也是O(n)的操作。

所以此时使用双指针法才展现出效率的优势：**通过两个指针在一个for循环下完成两个for循环的工作。**

字符串篇

在字符串：这道题目，使用库函数一行代码搞定中讲解了反转字符串，注意这里强调要原地反转，要不然就失去了题目的意义。

使用双指针法，**定义两个指针（也可以说是索引下标），一个从字符串前面，一个从字符串后面，两个指针同时向中间移动，并交换元素。**，时间复杂度是O(n)。

在替换空格 中介绍使用双指针填充字符串的方法，如果想把这道题目做到极致，就不要只用额外的辅助空间了！

思路就是**首先扩充数组到每个空格替换成"%20"之后的大小。然后双指针从后向前替换空格。**

有同学问了，为什么要从后向前填充，从前向后填充不行么？

从前向后填充就是O(n^2)的算法了，因为每次添加元素都要将添加元素之后的所有元素向后移动。

其实很多数组（字符串）填充类的问题，都可以先预先给数组扩容带填充后的大小，然后在从后向前进行操作。

那么在字符串：花式反转还不够！中，我们使用双指针法，用O(n)的时间复杂度完成字符串删除类的操作，因为题目要删除冗余空格。

在删除冗余空格的过程中，如果不注意代码效率，很容易写成了O(n^2)的时间复杂度。其实使用双指针法O(n)就可以搞定。

主要还是大家用erase用的比较随意，一定要注意for循环下用erase的情况，一般可以用双指针写效率更高！

链表篇

翻转链表是现场面试，白纸写代码的好题，考察了候选者对链表以及指针的熟悉程度，而且代码也不长，适合在白纸上写。

在链表：听说过两天反转链表又写不出来了？中，讲如何使用双指针法来翻转链表，**只需要改变链表的next指针的指向，直接将链表反转 ，而不用重新定义一个新的链表。**

思路还是很简单的，代码也不长，但是想在白纸上一次性写出bugfree的代码，并不是容易的事情。

在链表中求环，应该是双指针在链表里最经典的应用，在链表：环找到了，那入口呢？中讲解了如何通过双指针判断是否有环，而且还要找到环的入口。

使用快慢指针（双指针法），分别定义 fast 和 slow指针，从头结点出发，fast指针每次移动两个节点，slow指针每次移动一个节点，如果 fast 和 slow指针在途中相遇 ，说明这个链表有环。

那么找到环的入口，其实需要点简单的数学推理，我在文章中把找环的入口清清楚楚的推理的一遍，如果对找环入口不够清楚的同学建议自己看一看链表：环找到了，那入口呢？。

N数之和篇

在哈希表：解决了两数之和，那么能解决三数之和么？中，讲到使用哈希法可以解决1.两数之和的问题

其实使用双指针也可以解决1.两数之和的问题，只不过1.两数之和求的是两个元素的下标，没法用双指针，如果改成求具体两个元素的数值就可以了，大家可以尝试用双指针做一个leetcode上两数之和的题目，就可以体会到我说的意思了。

使用了哈希法解决了两数之和，但是哈希法并不使用于三数之和！

使用哈希法的过程中要把符合条件的三元组放进vector中，然后在去去重，这样是非常费时的，很容易超时，也是三数之和通过率如此之低的根源所在。

去重的过程不好处理，有很多小细节，如果在面试中很难想到位。

时间复杂度可以做到O(n^2)，但还是比较费时的，因为不好做剪枝操作。

所以这道题目使用双指针法才是最为合适的，用双指针做这道题目才能就能真正体会到，**通过前后两个指针不算向中间逼近，在一个for循环下完成两个for循环的工作。**

只用双指针法时间复杂度为O(n^2)，但比哈希法的O(n^2)效率高得多，哈希法在使用两层for循环的时候，能做的剪枝操作很有限。

在双指针法：一样的道理，能解决四数之和中，讲到了四数之和，其实思路是一样的，**在三数之和的基础上再套一层for循环，依然是使用双指针法。**

对于三数之和使用双指针法就是将原本暴力O(n^3)的解法，降为O(n^2)的解法，四数之和的双指针解法就是将原本暴力O(n^4)的解法，降为O(n^3)的解法。

同样的道理，五数之和，n数之和都是在这个基础上累加。

总结

本文中一共介绍了leetcode上九道使用双指针解决问题的经典题目，除了链表一些题目一定要使用双指针，其他题目都是使用双指针来提高效率，一般是将O(n^2)的时间复杂度，降为$O(n)$。

建议大家可以把文中涉及到的题目在好好做一做，琢磨琢磨，基本对双指针法就不在话下了。

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

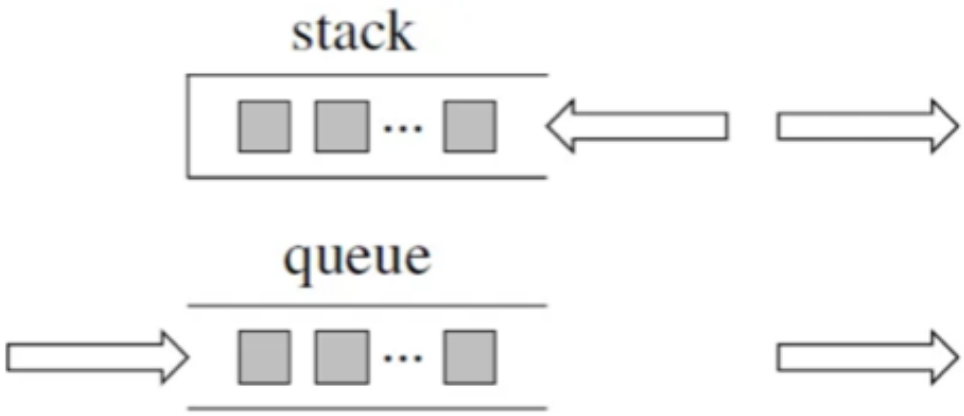
特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

> 来看看栈和队列不为人知的一面

1. 栈与队列理论基础

我想栈和队列的原理大家应该很熟悉了，队列是先进先出，栈是先进后出。

如图所示：



那么我这里再列出四个关于栈的问题，大家可以思考一下。以下是以C++为例，使用其他编程语言的同学也对应思考一下，自己使用的编程语言里栈和队列是什么样的。

1. C++中stack 是容器么?
2. 我们使用的stack是属于哪个版本的STL?
3. 我们使用的STL中stack是如何实现的?
4. stack 提供迭代器来遍历stack空间么?

相信这四个问题并不那么好回答， 因为一些同学使用数据结构会停留在非常表面上的应用，稍稍往深一问，就会有好像懂，好像也不懂的感觉。

有的同学可能仅仅知道有栈和队列这么个数据结构，却不知道底层实现，也不清楚所使用栈和队列和STL是什么关系。

所以这里我再给大家扫一遍基础知识，

首先大家要知道 栈和队列是STL（C++标准库）里面的两个数据结构。

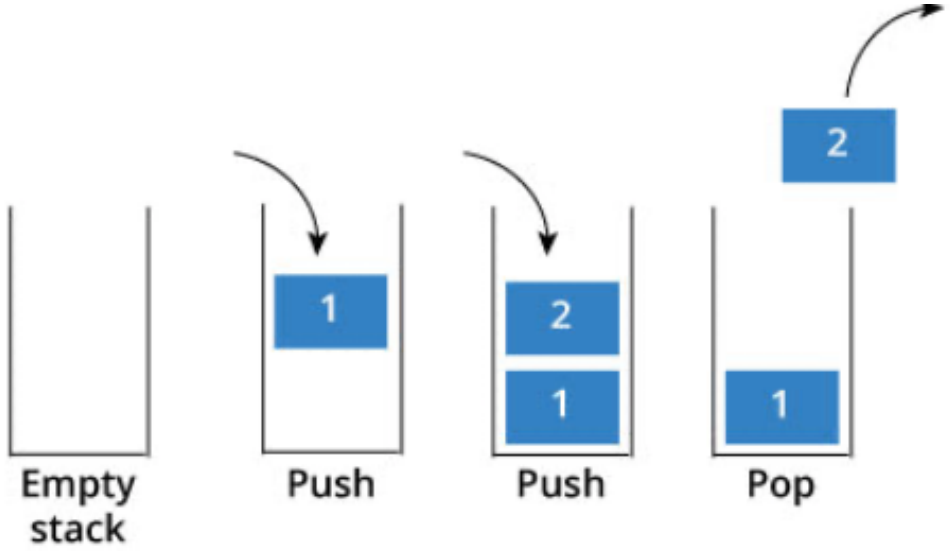
C++标准库是有多个版本的，要知道我们使用的STL是哪个版本，才能知道对应的栈和队列的实现原理。

那么来介绍一下，三个最为普遍的STL版本：

1. HP STL
   其他版本的C++ STL，一般是以HP STL为蓝本实现出来的，HP STL是C++ STL的第一个实现版本，而且开放源代码。
2. P.J.Plauger STL
   由P.J.Plauger参照HP STL实现出来的，被Visual C++编译器所采用，不是开源的。
3. SGI STL
   由Silicon Graphics Computer Systems公司参照HP STL实现，被Linux的C++编译器GCC所采用，SGI STL是开源软件，源码可读性甚高。

接下来介绍的栈和队列也是SGI STL里面的数据结构， 知道了使用版本，才知道对应的底层实现。

来说一说栈，栈先进后出，如图所示：



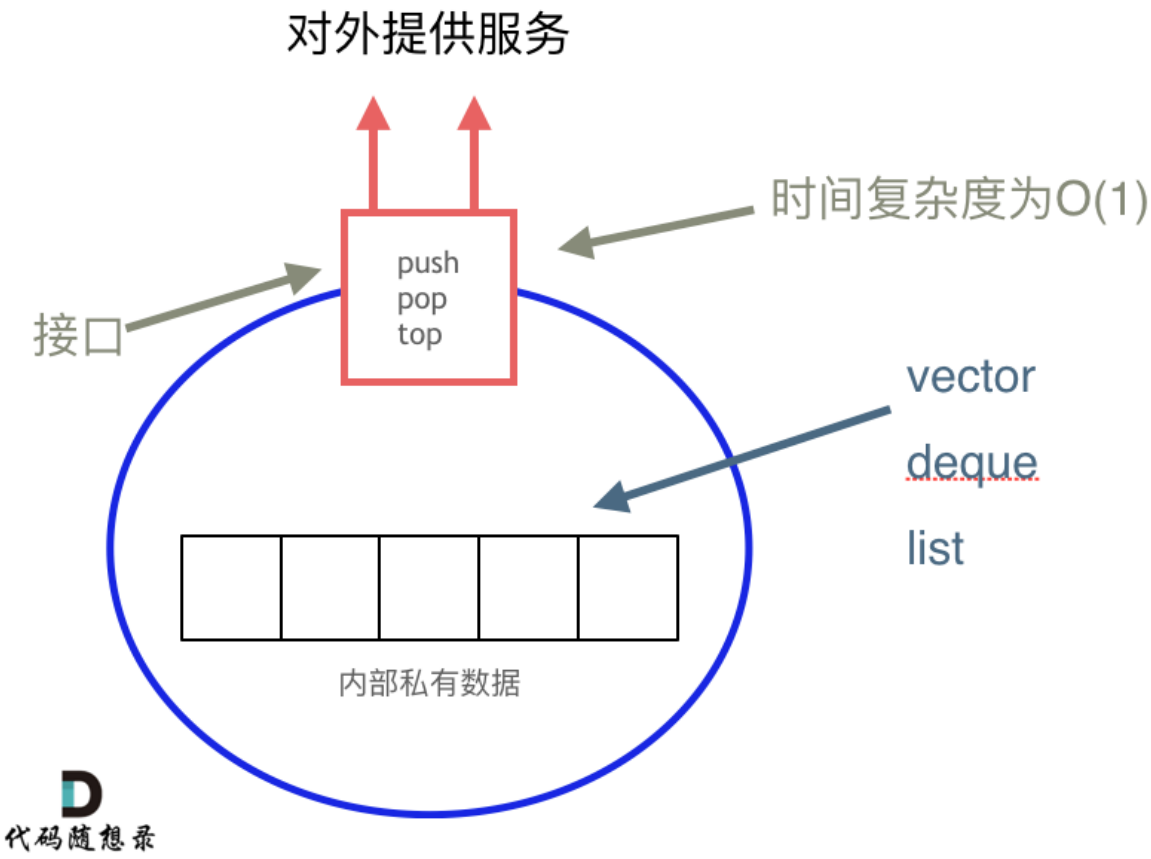
栈提供push 和 pop 等等接口，所有元素必须符合先进后出规则，所以栈不提供走访功能，也不提供迭代器(iterator)。 不像是set 或者map 提供迭代器iterator来遍历所有元素。

栈是以底层容器完成其所有的工作，对外提供统一的接口，底层容器是可插拔的（也就是说我们可以控制使用哪种容器来实现栈的功能）。

所以STL中栈往往不被归类为容器，而被归类为container adapter（容器适配器）。

那么问题来了，STL 中栈是用什么容器实现的?

从下图中可以看出，栈的内部结构，栈的底层实现可以是vector，deque，list 都是可以的， 主要就是数组和链表的底层实现。



我们常用的SGI STL，如果没有指定底层实现的话，默认是以deque为缺省情况下栈的底层结构。

deque是一个双向队列，只要封住一段，只开通另一端就可以实现栈的逻辑了。

SGI STL中 队列底层实现缺省情况下一样使用deque实现的。

我们也可以指定vector为栈的底层实现，初始化语句如下：

```
std::stack<int, std::vector<int> > third;  // 使用vector为底层容器的栈
```

刚刚讲过栈的特性，对应的队列的情况是一样的。

队列中先进先出的数据结构，同样不允许有遍历行为，不提供迭代器, **SGI STL中队列一样是以deque为缺省情况下的底部结构。**

也可以指定list 为起底层实现，初始化queue的语句如下：

```
std::queue<int, std::list<int>> third; // 定义以list为底层容器的队列
```

所以STL 队列也不被归类为容器，而被归类为container adapter（ 容器适配器）。

我这里讲的都是C++ 语言中的情况， 使用其他语言的同学也要思考栈与队列的底层实现问题， 不要对数据结构的使用浅尝辄止，而要深挖其内部原理，才能夯实基础。

> 工作上一定没人这么搞，但是考察对栈、队列理解程度的好题

2.用栈实现队列

力扣题目链接

使用栈实现队列的下列操作：

push(x) -- 将一个元素放入队列的尾部。
pop() -- 从队列首部移除元素。
peek() -- 返回队列首部的元素。
empty() -- 返回队列是否为空。

示例:

```
MyQueue queue = new MyQueue();
queue.push(1);
queue.push(2);
queue.peek();  // 返回 1
queue.pop();   // 返回 1
queue.empty(); // 返回 false
```

说明:

- 你只能使用标准的栈操作 -- 也就是只有 push to top, peek/pop from top, size, 和 is empty 操作是合法的。
- 你所使用的语言也许不支持栈。你可以使用 list 或者 deque（双端队列）来模拟一个栈，只要是标准的栈操作即可。
- 假设所有操作都是有效的 （例如，一个空的队列不会调用 pop 或者 peek 操作）。

算法公开课

《代码随想录》算法视频公开课：栈的基本操作！ | LeetCode：232.用栈实现队列，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

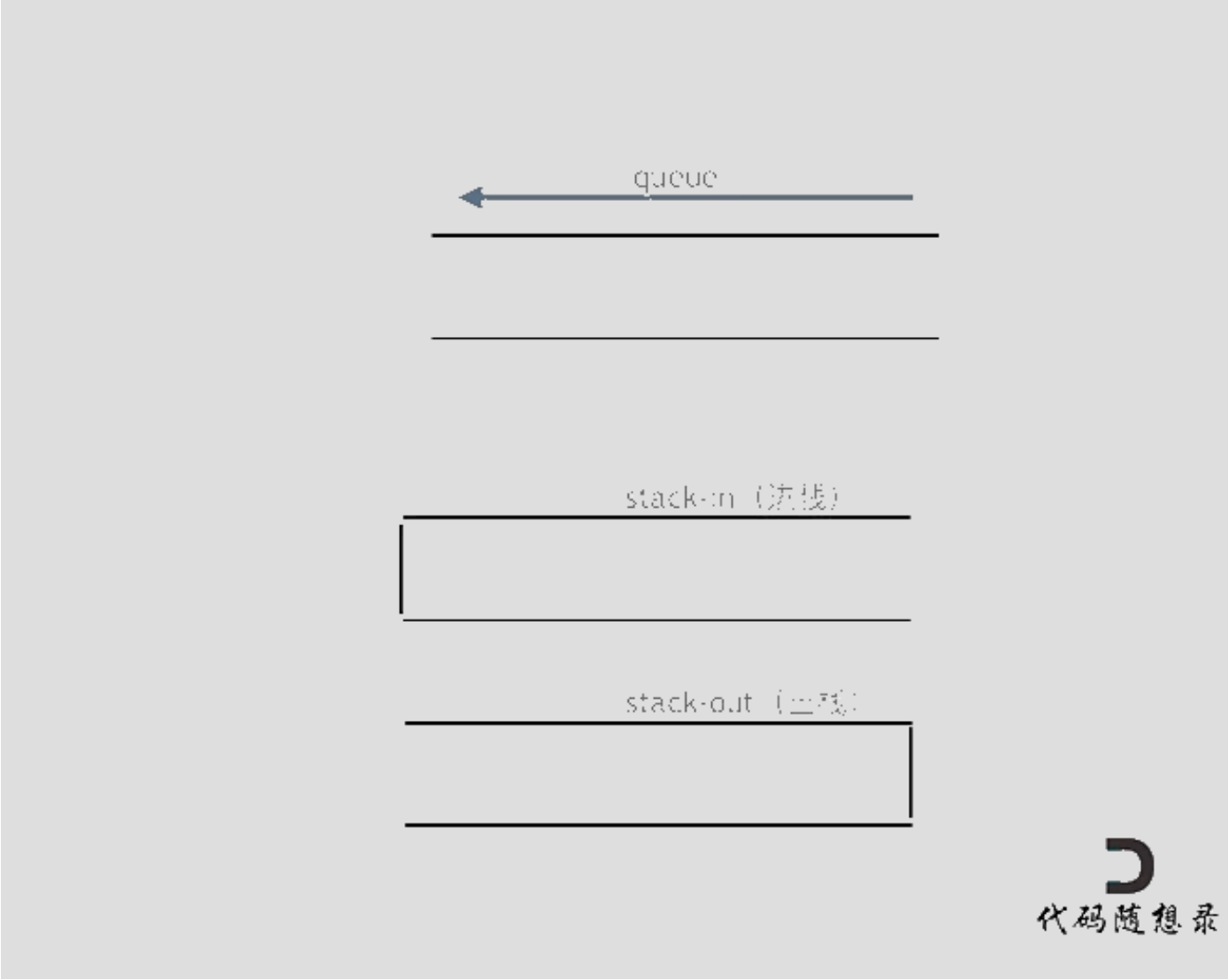
这是一道模拟题，不涉及到具体算法，考察的就是对栈和队列的掌握程度。

使用栈来模式队列的行为，如果仅仅用一个栈，是一定不行的，所以需要两个栈**一个输入栈，一个输出栈**，这里要注意输入栈和输出栈的关系。

下面动画模拟以下队列的执行过程：

执行语句：
queue.push(1);
queue.push(2);
queue.pop(); **注意此时的输出栈的操作**
queue.push(3);
queue.push(4);
queue.pop();

queue.pop();**注意此时的输出栈的操作**
queue.pop();
queue.empty();



在push数据的时候，只要数据放进输入栈就好，**但在pop的时候，操作就复杂一些，输出栈如果为空，就把进栈数据全部导入进来（注意是全部导入）**，再从出栈弹出数据，如果输出栈不为空，则直接从出栈弹出数据就可以了。

最后如何判断队列为空呢？**如果进栈和出栈都为空的话，说明模拟的队列为空了。**

在代码实现的时候，会发现pop() 和 peek()两个函数功能类似，代码实现上也是类似的，可以思考一下如何把代码抽象一下。

C++代码如下：

```
class MyQueue {
public:
    stack<int> stIn;
    stack<int> stOut;
    /** Initialize your data structure here. */
    MyQueue() {

    }
```

```
    /** Push element x to the back of queue. */
    void push(int x) {
        stIn.push(x);
    }

    /** Removes the element from in front of queue and returns that element. */
    int pop() {
        // 只有当stOut为空的时候，再从stIn里导入数据（导入stIn全部数据）
        if (stOut.empty()) {
            // 从stIn导入数据直到stIn为空
            while(!stIn.empty()) {
                stOut.push(stIn.top());
                stIn.pop();
            }
        }
        int result = stOut.top();
        stOut.pop();
        return result;
    }

    /** Get the front element. */
    int peek() {
        int res = this->pop(); // 直接使用已有的pop函数
        stOut.push(res); // 因为pop函数弹出了元素res，所以再添加回去
        return res;
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return stIn.empty() && stOut.empty();
    }
};
```

- 时间复杂度: push和empty为O(1), pop和peek为O(n)
- 空间复杂度: O(n)

拓展

可以看出peek()的实现，直接复用了pop()， 要不然，对stOut判空的逻辑又要重写一遍。

再多说一些代码开发上的习惯问题，在工业级别代码开发中，最忌讳的就是 实现一个类似的函数，直接把代码粘过来改一改就完事了。

这样的项目代码会越来越乱，**一定要懂得复用，功能相近的函数要抽象出来，不要大量的复制粘贴，很容易出问题！（踩过坑的人自然懂）**

工作中如果发现某一个功能自己要经常用，同事们可能也会用到，自己就花点时间把这个功能抽象成一个好用的函数或者工具类，不仅自己方便，也方便了同事们。

同事们就会逐渐认可你的工作态度和工作能力，自己的口碑都是这么一点一点积累起来的！在同事圈里口碑起来了之后，你就发现自己走上了一个正循环，以后的升职加薪才少不了你！哈哈哈

> 用队列实现栈还是有点别扭。

3. 用队列实现栈

力扣题目链接

使用队列实现栈的下列操作：

- push(x) -- 元素 x 入栈
- pop() -- 移除栈顶元素
- top() -- 获取栈顶元素
- empty() -- 返回栈是否为空

注意:

- 你只能使用队列的基本操作-- 也就是 push to back, peek/pop from front, size, 和 is empty 这些操作是合法的。
- 你所使用的语言也许不支持队列。 你可以使用 list 或者 deque（双端队列）来模拟一个队列 , 只要是标准的队列操作即可。
- 你可以假设所有操作都是有效的（例如, 对一个空的栈不会调用 pop 或者 top 操作）。

算法公开课

《代码随想录》算法视频公开课：队列的基本操作！ | LeetCode：225. 用队列实现栈，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

（这里要强调是单向队列）

有的同学可能疑惑这种题目有什么实际工程意义，**其实很多算法题目主要是对知识点的考察和教学意义远大于其工程实践的意义，所以面试题也是这样！**

刚刚做过栈与队列：我用栈来实现队列怎么样？的同学可能依然想着用一个输入队列，一个输出队列，就可以模拟栈的功能，仔细想一下还真不行！

队列模拟栈，其实一个队列就够了，那么我们先说一说两个队列来实现栈的思路。

队列是先进先出的规则，把一个队列中的数据导入另一个队列中，数据的顺序并没有变，并没有变成先进后出的顺序。

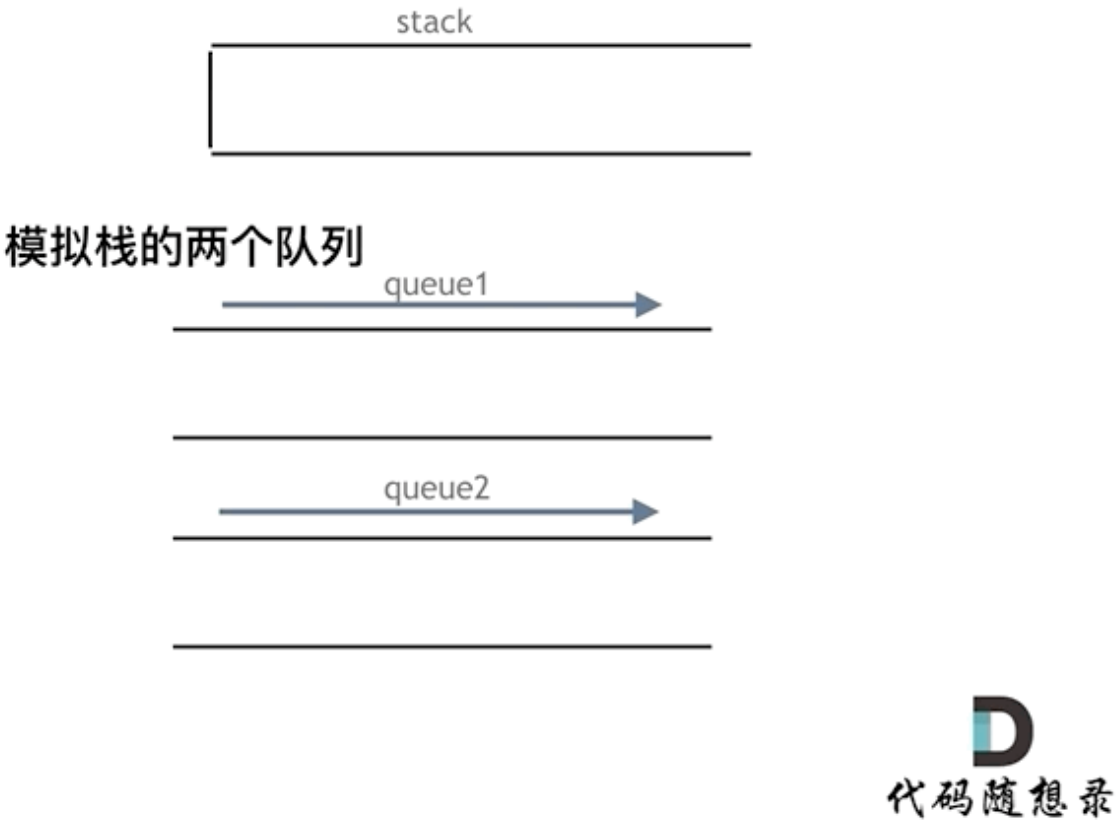
所以用栈实现队列， 和用队列实现栈的思路还是不一样的，这取决于这两个数据结构的性质。

但是依然还是要用两个队列来模拟栈，只不过没有输入和输出的关系，而是另一个队列完全用来备份的！

如下面动画所示，**用两个队列que1和que2实现队列的功能，que2其实完全就是一个备份的作用**，把que1最后面的元素以外的元素都备份到que2，然后弹出最后面的元素，再把其他元素从que2导回que1。

模拟的队列执行语句如下：

```
queue.push(1);
queue.push(2);
queue.pop();   // 注意弹出的操作
queue.push(3);
queue.push(4);
queue.pop();  // 注意弹出的操作
queue.pop();
queue.pop();
queue.empty();
```



详细如代码注释所示：

```
class MyStack {
public:
    queue<int> que1;
    queue<int> que2; // 辅助队列，用来备份
```

```
    /** Initialize your data structure here. */
    MyStack() {

    }

    /** Push element x onto stack. */
    void push(int x) {
        que1.push(x);
    }

    /** Removes the element on top of the stack and returns that element. */
    int pop() {
        int size = que1.size();
        size--;
        while (size--) { // 将que1 导入que2，但要留下最后一个元素
            que2.push(que1.front());
            que1.pop();
        }

        int result = que1.front(); // 留下的最后一个元素就是要返回的值
        que1.pop();
        que1 = que2;              // 再将que2赋值给que1
        while (!que2.empty()) { // 清空que2
            que2.pop();
        }
        return result;
    }

    /** Get the top element. */
    int top() {
        return que1.back();
    }

    /** Returns whether the stack is empty. */
    bool empty() {
        return que1.empty();
    }
};
```

- 时间复杂度: push为O(n)，其他为O(1)
- 空间复杂度: O(n)

优化

其实这道题目就是用一个队列就够了。

一个队列在模拟栈弹出元素的时候只要将队列头部的元素（除了最后一个元素外） 重新添加到队列尾部，此时再去弹出元素就是栈的顺序了。

C++优化代码

```
class MyStack {
public:
    queue<int> que;
    /** Initialize your data structure here. */
    MyStack() {

    }
    /** Push element x onto stack. */
    void push(int x) {
        que.push(x);
    }
    /** Removes the element on top of the stack and returns that element. */
    int pop() {
        int size = que.size();
        size--;
        while (size--) { // 将队列头部的元素（除了最后一个元素外） 重新添加到队列尾部
            que.push(que.front());
            que.pop();
        }
        int result = que.front(); // 此时弹出的元素顺序就是栈的顺序了
        que.pop();
        return result;
    }

    /** Get the top element. */
    int top() {
        return que.back();
    }

    /** Returns whether the stack is empty. */
    bool empty() {
        return que.empty();
    }
};
```

- 时间复杂度: push为O(n)，其他为O(1)
- 空间复杂度: O(n)

> 数据结构与算法应用往往隐藏在我们看不到的地方

4. 有效的括号

[力扣题目链接](#)

给定一个只包括 '('，')'，'{'，'}'，'['，']' 的字符串，判断字符串是否有效。

有效字符串需满足：

- 左括号必须用相同类型的右括号闭合。
- 左括号必须以正确的顺序闭合。
- 注意空字符串可被认为是有效字符串。

示例 1:

- 输入: "()"
- 输出: true

示例 2:

- 输入: "()[]{}"
- 输出: true

示例 3:

- 输入: "(]"
- 输出: false

示例 4:

- 输入: "([)]"
- 输出: false

示例 5:

- 输入: "{[]}"
- 输出: true

算法公开课

[《代码随想录》算法视频公开课](#)：[栈的拿手好戏！| LeetCode：20. 有效的括号](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

题外话

括号匹配是使用栈解决的经典问题。

题意其实就像我们在写代码的过程中，要求括号的顺序是一样的，有左括号，相应的位置必须要有右括号。

如果还记得编译原理的话，编译器在 词法分析的过程中处理括号、花括号等这个符号的逻辑，也是使用了栈这种数据结构。

再举个例子，linux系统中，cd这个进入目录的命令我们应该再熟悉不过了。

```
cd a/b/c/../../
```

这个命令最后进入a目录，系统是如何知道进入了a目录呢，这就是栈的应用（其实可以出一道相应的面试题了）

所以栈在计算机领域中应用是非常广泛的。

有的同学经常会想学的这些数据结构有什么用，也开发不了什么软件，大多数同学说的软件应该都是可视化的软件例如APP、网站之类的，那都是非常上层的应用了，底层很多功能的实现都是基础的数据结构和算法。

所以数据结构与算法的应用往往隐藏在我们看不到的地方！

这里我就不过多展开了，先来看题。

进入正题

由于栈结构的特殊性，非常适合做对称匹配类的题目。

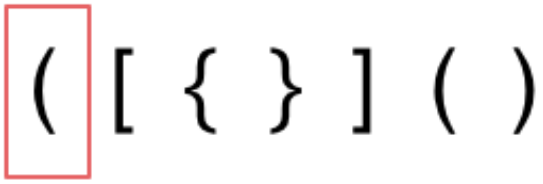
首先要弄清楚，字符串里的括号不匹配有几种情况。

一些同学，在面试中看到这种题目上来就开始写代码，然后就越写越乱。

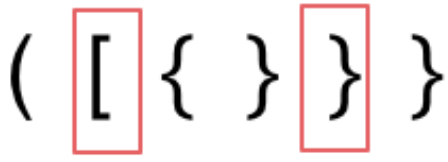
建议在写代码之前要分析好有哪几种不匹配的情况，如果不在动手之前分析好，写出的代码也会有很多问题。

先来分析一下 这里有三种不匹配的情况，

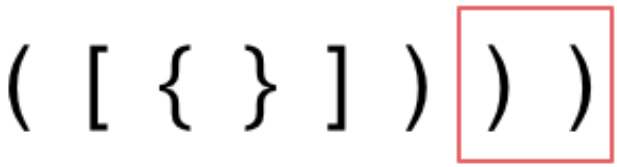
1. 第一种情况，字符串里左方向的括号多余了，所以不匹配。



2. 第二种情况，括号没有多余，但是 括号的类型没有匹配上。



3. 第三种情况，字符串里右方向的括号多余了，所以不匹配。



我们的代码只要覆盖了这三种不匹配的情况，就不会出问题，可以看出 动手之前分析好题目的重要性。

动画如下：

第一种情况：已经遍历完了字符串，但是栈不为空，说明有相应的左括号没有右括号来匹配，所以return false

第二种情况：遍历字符串匹配的过程中，发现栈里没有要匹配的字符。所以return false

第三种情况：遍历字符串匹配的过程中，栈已经为空了，没有匹配的字符了，说明右括号没有找到对应的左括号return false

那么什么时候说明左括号和右括号全都匹配了呢，就是字符串遍历完之后，栈是空的，就说明全都匹配了。

分析完之后，代码其实就比较好写了，

但还有一些技巧，在匹配左括号的时候，右括号先入栈，就只需要比较当前元素和栈顶相不相等就可以了，比左括号先入栈代码实现要简单的多了！

实现C++代码如下：

```
class Solution {
public:
    bool isValid(string s) {
        if (s.size() % 2 != 0) return false; // 如果s的长度为奇数，一定不符合要求
        stack<char> st;
        for (int i = 0; i < s.size(); i++) {
            if (s[i] == '(') st.push(')');
            else if (s[i] == '{') st.push('}');
            else if (s[i] == '[') st.push(']');
```

```
                // 第三种情况：遍历字符串匹配的过程中，栈已经为空了，没有匹配的字符了，说明右括号没有找到对应的左括号 return false
                // 第二种情况：遍历字符串匹配的过程中，发现栈里没有我们要匹配的字符。所以return false
                else if (st.empty() || st.top() != s[i]) return false;
                else st.pop(); // st.top() 与 s[i]相等，栈弹出元素
            }
            // 第一种情况：此时我们已经遍历完了字符串，但是栈不为空，说明有相应的左括号没有右括号来匹配，所以return false，否则就return true
            return st.empty();
        }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

技巧性的东西没有固定的学习方法，还是要多看多练，自己灵活运用了。

> 匹配问题都是栈的强项

5. 删除字符串中的所有相邻重复项

[力扣题目链接]

给出由小写字母组成的字符串 S，重复项删除操作会选择两个相邻且相同的字母，并删除它们。

在 S 上反复执行重复项删除操作，直到无法继续删除。

在完成所有重复项删除操作后返回最终的字符串。答案保证唯一。

示例：

- 输入："abbaca"
- 输出："ca"
- 解释：例如，在 "abbaca" 中，我们可以删除 "bb" 由于两字母相邻且相同，这是此时唯一可以执行删除操作的重复项。之后我们得到字符串 "aaca"，其中又只有 "aa" 可以执行重复项删除操作，所以最后的字符串为 "ca"。

提示：

- 1 <= S.length <= 20000
- S 仅由小写英文字母组成。

算法公开课

《代码随想录》算法视频公开课：栈的好戏还要继续！| LeetCode：1047. 删除字符串中的所有相邻重复项，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

正题

本题要删除相邻相同元素，相对于20. 有效的括号来说其实也是匹配问题，20. 有效的括号 是匹配左右括号，本题是匹配相邻元素，最后都是做消除的操作。

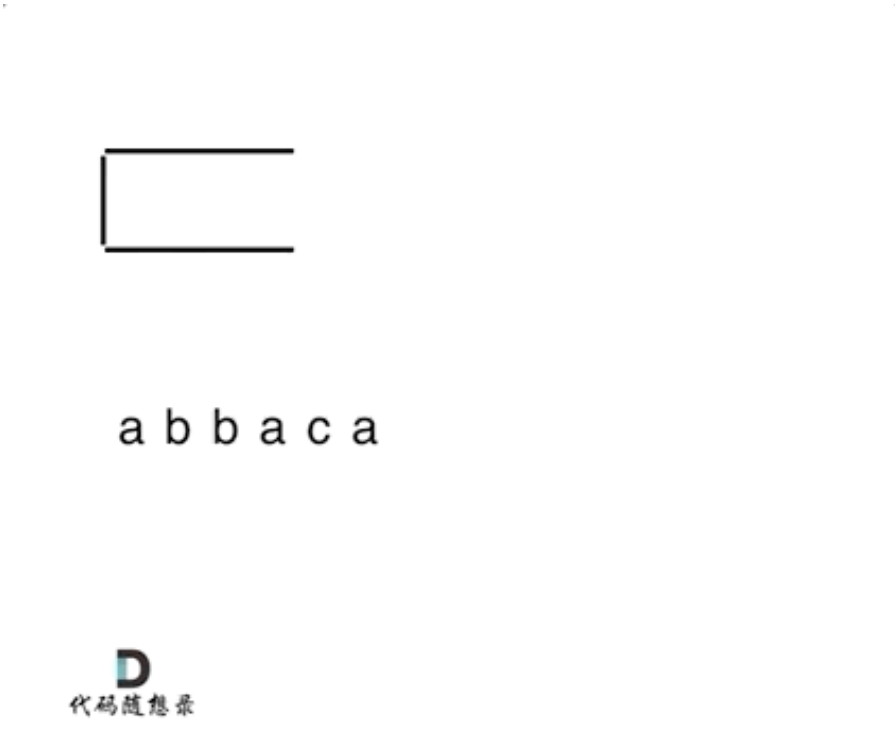
本题也是用栈来解决的经典题目。

那么栈里应该放的是什么元素呢?

我们在删除相邻重复项的时候，其实就是要知道当前遍历的这个元素，我们在前一位是不是遍历过一样数值的元素，那么如何记录前面遍历过的元素呢?

所以就是用栈来存放，那么栈的目的，就是存放遍历过的元素，当遍历当前的这个元素的时候，去栈里看一下我们是不是遍历过相同数值的相邻元素。

然后再去做对应的消除操作。 如动画所示：



从栈中弹出剩余元素，此时是字符串ac，因为从栈里弹出的元素是倒序的，所以再对字符串进行反转一下，就得到了最终的结果。

C++代码：

```
class Solution {
public:
    string removeDuplicates(string S) {
        stack<char> st;
        for (char s : S) {
```

```
            if (st.empty() || s != st.top()) {
                st.push(s);
            } else {
                st.pop(); // s 与 st.top()相等的情况
            }
        }
        string result = "";
        while (!st.empty()) { // 将栈中元素放到result字符串汇总
            result += st.top();
            st.pop();
        }
        reverse (result.begin(), result.end()); // 此时字符串需要反转一下
        return result;

    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

当然可以拿字符串直接作为栈，这样省去了栈还要转为字符串的操作。

代码如下：

```
class Solution {
public:
    string removeDuplicates(string S) {
        string result;
        for(char s : S) {
            if(result.empty() || result.back() != s) {
                result.push_back(s);
            }
            else {
                result.pop_back();
            }
        }
        return result;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)，返回值不计空间复杂度

题外话

这道题目就像是我们玩过的游戏对对碰，如果相同的元素挨在一起就要消除。

可能我们在玩游戏的时候感觉理所当然应该消除，但程序又怎么知道该如何消除呢，特别是消除之后又有新的元素可能挨在一起。

此时游戏的后端逻辑就可以用一个栈来实现（我没有实际考察对对碰或者爱消除游戏的代码实现，仅从原理上进行推断）。

游戏开发可能使用栈结构，编程语言的一些功能实现也会使用栈结构，实现函数递归调用就需要栈，但不是每种编程语言都支持递归，例如：

递归的实现就是：每一次递归调用都会把函数的局部变量、参数值和返回地址等压入调用栈中，然后递归返回的时候，从栈顶弹出上一次递归的各项参数，所以这就是递归为什么可以返回上一层位置的原因。

相信大家应该遇到过一种错误就是栈溢出，系统输出的异常是`Segmentation fault`（当然不是所有的`Segmentation fault`都是栈溢出导致的），如果你使用了递归，就要想一想是不是无限递归了，那么系统调用栈就会溢出。

而且**在企业项目开发中，尽量不要使用递归**！在项目比较大的时候，由于参数多，全局变量等等，使用递归很容易判断不充分return的条件，非常容易无限递归（或者递归层级过深），**造成栈溢出错误（这种问题还不好排查！）**

> 这不仅仅是一道好题，也展现出计算机的思考方式

6. 逆波兰表达式求值

力扣题目链接

根据 逆波兰表示法，求表达式的值。

有效的运算符包括 + , - , * , / 。每个运算对象可以是整数，也可以是另一个逆波兰表达式。

说明：

整数除法只保留整数部分。
给定逆波兰表达式总是有效的。换句话说，表达式总会得出有效数值且不存在除数为 0 的情况。

示例 1：

- 输入: ["2", "1", "+", "3", " * "]
- 输出: 9
- 解释: 该算式转化为常见的中缀算术表达式为：((2 + 1) * 3) = 9

示例 2：

- 输入: ["4", "13", "5", "/", "+"]
- 输出: 6

- 解释: 该算式转化为常见的中缀算术表达式为：(4 + (13 / 5)) = 6

示例 3：

- 输入: ["10", "6", "9", "3", "+", "-11", " * ", "/", " * ", "17", "+", "5", "+"]
- 输出: 22
- 解释:该算式转化为常见的中缀算术表达式为：

```
((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22
```

逆波兰表达式：是一种后缀表达式，所谓后缀就是指运算符写在后面。

平常使用的算式则是一种中缀表达式，如 (1 + 2) * (3 + 4) 。

该算式的逆波兰表达式写法为 ((1 2 +) (3 4 +) *) 。

逆波兰表达式主要有以下两个优点：

- 去掉括号后表达式无歧义，上式即便写成 1 2 + 3 4 + * 也可以依据次序计算出正确结果。
- 适合用栈操作运算：遇到数字则入栈；遇到运算符则取出栈顶两个数字进行计算，并将结果压入栈中。

算法公开课

[《代码随想录》算法视频公开课](#)：[栈的最后表演！ | LeetCode：150. 逆波兰表达式求值](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

正题

在上一篇文章中[1047.删除字符串中的所有相邻重复项](#)提到了 递归就是用栈来实现的。

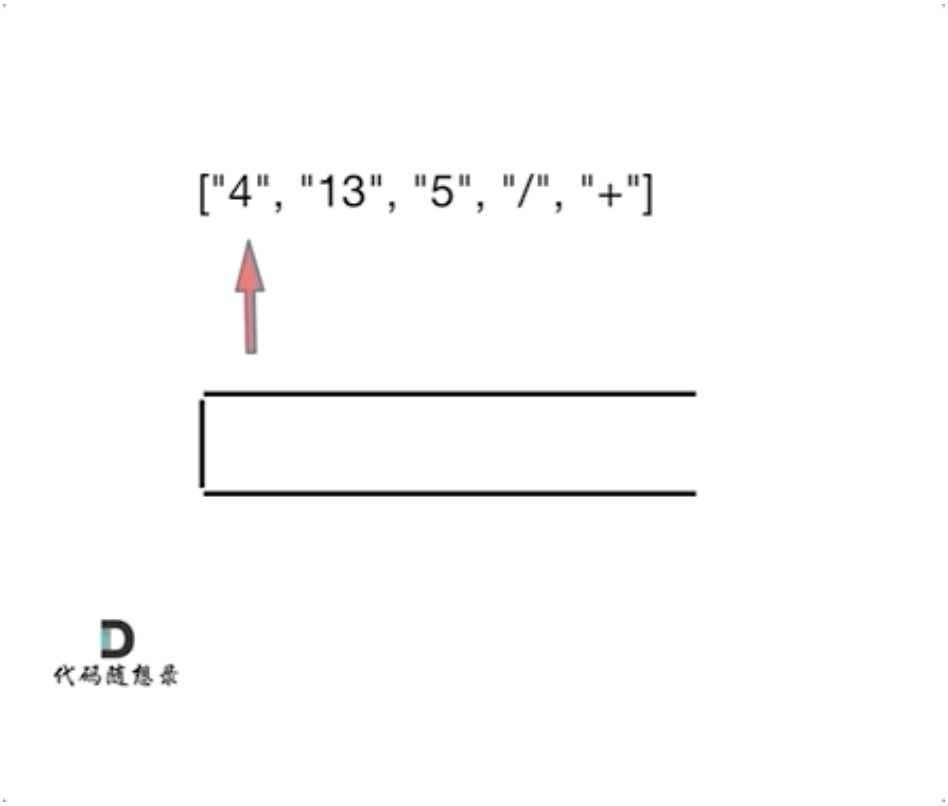
所以**栈与递归之间在某种程度上是可以转换的！** 这一点我们在后续讲解二叉树的时候，会更详细的讲解到。

那么来看一下本题，**其实逆波兰表达式相当于是二叉树中的后序遍历**。 大家可以把运算符作为中间节点，按照后序遍历的规则画出一个二叉树。

但我们没有必要从二叉树的角度去解决这个问题，只要知道逆波兰表达式是用后序遍历的方式把二叉树序列化了，就可以了。

在进一步看，本题中每一个子表达式要得出一个结果，然后拿这个结果再进行运算，那么**这岂不就是一个相邻字符串消除的过程，和[1047.删除字符串中的所有相邻重复项](#)中的对对碰游戏是不是就非常像了。**

如动画所示：



相信看完动画大家应该知道，这和1047. 删除字符串中的所有相邻重复项是差不错的，只不过本题不要相邻元素做消除了，而是做运算！

C++代码如下：

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        // 力扣修改了后台测试数据，需要用longlong
        stack<long long> st;
        for (int i = 0; i < tokens.size(); i++) {
            if (tokens[i] == "+" || tokens[i] == "-" || tokens[i] == "*" || tokens[i]
== "/") {
                long long num1 = st.top();
                st.pop();
                long long num2 = st.top();
                st.pop();
                if (tokens[i] == "+") st.push(num2 + num1);
                if (tokens[i] == "-") st.push(num2 - num1);
                if (tokens[i] == "*") st.push(num2 * num1);
                if (tokens[i] == "/") st.push(num2 / num1);
            } else {
                st.push(stoll(tokens[i]));
            }
        }

        int result = st.top();
        st.pop(); // 把栈里最后一个元素弹出（其实不弹出也没事）
```

```
        return result;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

题外话

我们习惯看到的表达式都是中缀表达式，因为符合我们的习惯，但是中缀表达式对于计算机来说就不是很友好了。

例如：4 + 13 / 5，这就是中缀表达式，计算机从左到右去扫描的话，扫到13，还要判断13后面是什么运算符，还要比较一下优先级，然后13还和后面的5做运算，做完运算之后，还要向前回退到 4 的位置，继续做加法，你说麻不麻烦！

那么将中缀表达式，转化为后缀表达式之后：["4", "13", "5", "/", "+"] ，就不一样了，计算机可以利用栈来顺序处理，不需要考虑优先级了。也不用回退了， **所以后缀表达式对计算机来说是非常友好的。**

可以说本题不仅仅是一道好题，也展现出计算机的思考方式。

在1970年代和1980年代，惠普在其所有台式和手持式计算器中都使用了RPN（后缀表达式），直到2020年代仍在某些模型中使用了RPN。

参考维基百科如下：

> During the 1970s and 1980s, Hewlett-Packard used RPN in all of their desktop and hand-held calculators, and continued to use it in some models into the 2020s.

> 要用啥数据结构呢?

7. 滑动窗口最大值

力扣题目链接

给定一个数组 nums，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。

返回滑动窗口中的最大值。

进阶：

你能在线性时间复杂度内解决此题吗?

示例：

输入： *nums* = [1,3,-1,-3,5,3,6,7], 和 *k* = 3
输出： [3,3,5,5,6,7]
解释：

```
  滑动窗口的位置                最大值
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

提示：

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4
- 1 <= k <= nums.length

算法公开课

[《代码随想录》算法视频公开课](#)：[单调队列正式登场！| LeetCode：239. 滑动窗口最大值](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这是使用单调队列的经典题目。

难点是如何求一个区间里的最大值呢？ （这好像是废话），暴力一下不就得了。

暴力方法，遍历一遍的过程中每次从窗口中再找到最大的数值，这样很明显是O(n × k)的算法。

有的同学可能会想用一个大顶堆（优先级队列）来存放这个窗口里的k个数字，这样就可以知道最大的最大值是多少了， **但是问题是这个窗口是移动的，而大顶堆每次只能弹出最大值，我们无法移除其他数值，这样就造成大顶堆维护的不是滑动窗口里面的数值了。所以不能用大顶堆。**

此时我们需要一个队列，这个队列呢，放进去窗口里的元素，然后随着窗口的移动，队列也一进一出，每次移动之后，队列告诉我们里面的最大值是什么。

这个队列应该长这个样子：

```
class MyQueue {
public:
    void pop(int value) {
    }
    void push(int value) {
    }
    int front() {
        return que.front();
    }
};
```

每次窗口移动的时候，调用que.pop(滑动窗口中移除元素的数值)，que.push(滑动窗口添加元素的数值)，然后que.front()就返回我们要的最大值。

这么个队列香不香，要是有现成的这种数据结构是不是更香了！

可惜了，没有！ 我们需要自己实现这么个队列。

然后再分析一下，队列里的元素一定是要排序的，而且要最大值放在出队口，要不然怎么知道最大值呢。

但如果把窗口里的元素都放进队列里，窗口移动的时候，队列需要弹出元素。

那么问题来了，已经排序之后的队列 怎么能把窗口要移除的元素（这个元素可不一定是最大值）弹出呢。

大家此时应该陷入深思.....

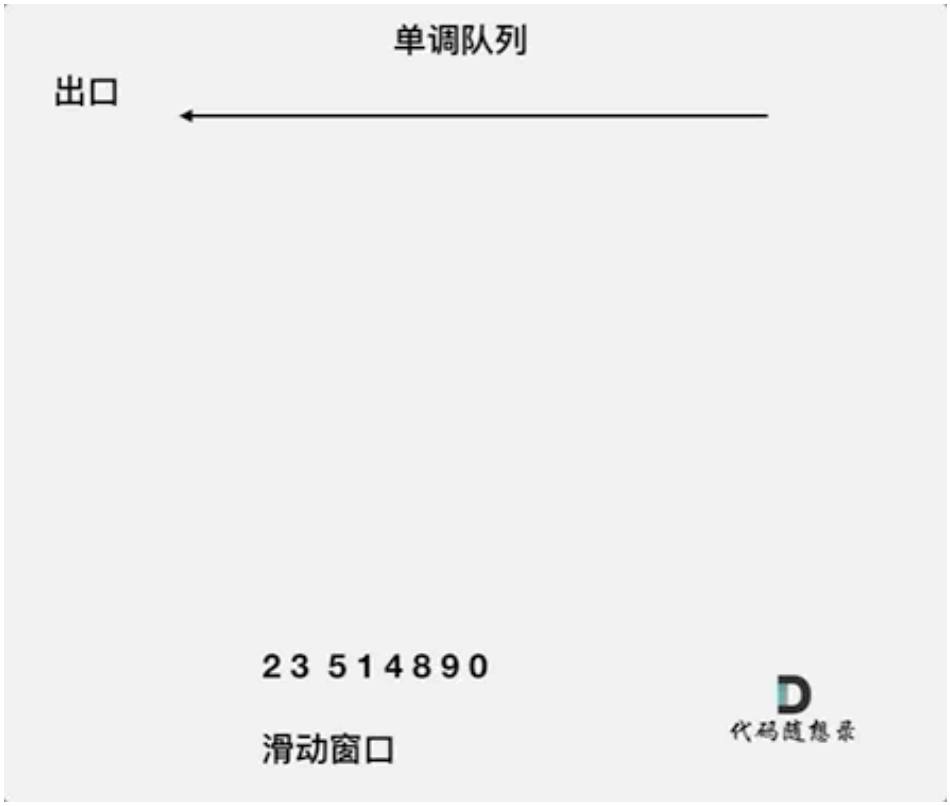
其实队列没有必要维护窗口里的所有元素，只需要维护有可能成为窗口里最大值的元素就可以了，同时保证队列里的元素数值是由大到小的。

那么这个维护元素单调递减的队列就叫做**单调队列，即单调递减或单调递增的队列。C++中没有直接支持单调队列，需要我们自己来实现一个单调队列**

不要以为实现的单调队列就是 对窗口里面的数进行排序，如果排序的话，那和优先级队列又有什么区别了呢。

来看一下单调队列如何维护队列里的元素。

动画如下：



对于窗口里的元素{2, 3, 5, 1 ,4}，单调队列里只维护{5, 4} 就够了，保持单调队列里单调递减，此时队列出口元素就是窗口里最大元素。

此时大家应该怀疑单调队列里维护着{5, 4} 怎么配合窗口进行滑动呢?

设计单调队列的时候，pop，和push操作要保持如下规则：

1. pop(value)：如果窗口移除的元素value等于单调队列的出口元素，那么队列弹出元素，否则不用任何操作
2. push(value)：如果push的元素value大于入口元素的数值，那么就将队列入口的元素弹出，直到push元素的数值小于等于队列入口元素的数值为止

保持如上规则，每次窗口移动的时候，只要问que.front()就可以返回当前窗口的最大值。

为了更直观的感受到单调队列的工作过程，以题目示例为例，输入: nums = [1,3,-1,-3,5,3,6,7], 和 k = 3，动画如下：

那么我们用什么数据结构来实现这个单调队列呢?

使用deque最为合适，在文章[栈与队列：来看看栈和队列不为人知的一面]中，我们就提到了常用的queue在没有指定容器的情况下，deque就是默认底层容器。

基于刚刚说过的单调队列pop和push的规则，代码不难实现，如下：

```
class MyQueue { //单调队列（从大到小）
public:
    deque<int> que; // 使用deque来实现单调队列
    // 每次弹出的时候，比较当前要弹出的数值是否等于队列出口元素的数值，如果相等则弹出。
    // 同时pop之前判断队列当前是否为空。
    void pop(int value) {
        if (!que.empty() && value == que.front()) {
            que.pop_front();
        }
    }
    // 如果push的数值大于入口元素的数值，那么就将队列后端的数值弹出，直到push的数值小于等于队列入口元素的数值为止。
    // 这样就保持了队列里的数值是单调从大到小的了。
    void push(int value) {
        while (!que.empty() && value > que.back()) {
            que.pop_back();
        }
        que.push_back(value);

    }
    // 查询当前队列里的最大值 直接返回队列前端也就是front就可以了。
    int front() {
```

```
        return que.front();
    }
};
```

这样我们就用deque实现了一个单调队列，接下来解决滑动窗口最大值的问题就很简单了，直接看代码吧。

C++代码如下：

```
class Solution {
private:
    class MyQueue { //单调队列（从大到小）
    public:
        deque<int> que; // 使用deque来实现单调队列
        // 每次弹出的时候，比较当前要弹出的数值是否等于队列出口元素的数值，如果相等则弹出。
        // 同时pop之前判断队列当前是否为空。
        void pop(int value) {
            if (!que.empty() && value == que.front()) {
                que.pop_front();
            }
        }
        // 如果push的数值大于入口元素的数值，那么就将队列后端的数值弹出，直到push的数值小于等于队列入口元素的数值为止。
        // 这样就保持了队列里的数值是单调从大到小的了。
        void push(int value) {
            while (!que.empty() && value > que.back()) {
                que.pop_back();
            }
            que.push_back(value);

        }
        // 查询当前队列里的最大值 直接返回队列前端也就是front就可以了。
        int front() {
            return que.front();
        }
    };
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        MyQueue que;
        vector<int> result;
        for (int i = 0; i < k; i++) { // 先将前k的元素放进队列
            que.push(nums[i]);
        }
        result.push_back(que.front()); // result 记录前k的元素的最大值
        for (int i = k; i < nums.size(); i++) {
            que.pop(nums[i - k]); // 滑动窗口移除最前面元素
            que.push(nums[i]); // 滑动窗口前加入最后面的元素
            result.push_back(que.front()); // 记录对应的最大值
        }
        return result;
```

```
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(k)

再来看一下时间复杂度，使用单调队列的时间复杂度是 O(n)。

有的同学可能想了，在队列中 push元素的过程中，还有pop操作呢，感觉不是纯粹的O(n)。

其实，大家可以自己观察一下单调队列的实现，nums 中的每个元素最多也就被 push_back 和 pop_back 各一次，没有任何多余操作，所以整体的复杂度还是 O(n)。

空间复杂度因为我们定义一个辅助队列，所以是O(k)。

扩展

大家貌似对单调队列 都有一些疑惑，首先要明确的是，题解中单调队列里的pop和push接口，仅适用于本题哈。单调队列不是一成不变的，而是不同场景不同写法，总之要保证队列里单调递减或递增的原则，所以叫做单调队列。 不要以为本题中的单调队列实现就是固定的写法哈。

大家貌似对deque也有一些疑惑，C++中deque是stack和queue默认的底层实现容器（这个我们之前已经讲过啦），deque是可以两边扩展的，而且deque里元素并不是严格的连续分布的。

> 前K个大数问题，老生常谈，不得不谈

8.前 K 个高频元素

[力扣题目链接]

给定一个非空的整数数组，返回其中出现频率前 k 高的元素。

示例 1:

- 输入: nums = [1,1,1,2,2,3], k = 2
- 输出: [1,2]

示例 2:

- 输入: nums = [1], k = 1
- 输出: [1]

提示：

- 你可以假设给定的 k 总是合理的，且 1 ≤ k ≤ 数组中不相同的元素的个数。
- 你的算法的时间复杂度必须优于 $O(n \log n)$, n 是数组的大小。
- 题目数据保证答案唯一，换句话说，数组中前 k 个高频元素的集合是唯一的。

- 你可以按任意顺序返回答案。

算法公开课

[《代码随想录》算法视频公开课](#)：[优先级队列正式登场！大顶堆、小顶堆该怎么用？ | LeetCode：347.前 K 个高频元素](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目主要涉及到如下三块内容：

1. 要统计元素出现频率
2. 对频率排序
3. 找出前K个高频元素

首先统计元素出现的频率，这一类的问题可以使用map来进行统计。

然后是对频率进行排序，这里我们可以使用一种 容器适配器就是**优先级队列**。

什么是优先级队列呢？

其实**就是一个披着队列外衣的堆**，因为优先级队列对外接口只是从队头取元素，从队尾添加元素，再无其他取元素的方式，看起来就是一个队列。

而且优先级队列内部元素是自动依照元素的权值排列。那么它是如何有序排列的呢？

缺省情况下priority_queue利用max-heap（大顶堆）完成对元素的排序，这个大顶堆是以vector为表现形式的complete binary tree（完全二叉树）。

什么是堆呢？

堆是一棵完全二叉树，树中每个结点的值都不小于（或不大于）其左右孩子的值。 如果父亲结点是大于等于左右孩子就是大顶堆，小于等于左右孩子就是小顶堆。

所以大家经常说的大顶堆（堆头是最大元素），小顶堆（堆头是最小元素），如果懒得自己实现的话，就直接用priority_queue（优先级队列）就可以了，底层实现都是一样的，从小到大排就是小顶堆，从大到小排就是大顶堆。

本题我们就要使用优先级队列来对部分频率进行排序。

为什么不用快排呢， 使用快排要将map转换为vector的结构，然后对整个数组进行排序， 而这种场景下，我们其实只需要维护k个有序的序列就可以了，所以使用优先级队列是最优的。

此时要思考一下，是使用小顶堆呢，还是大顶堆？

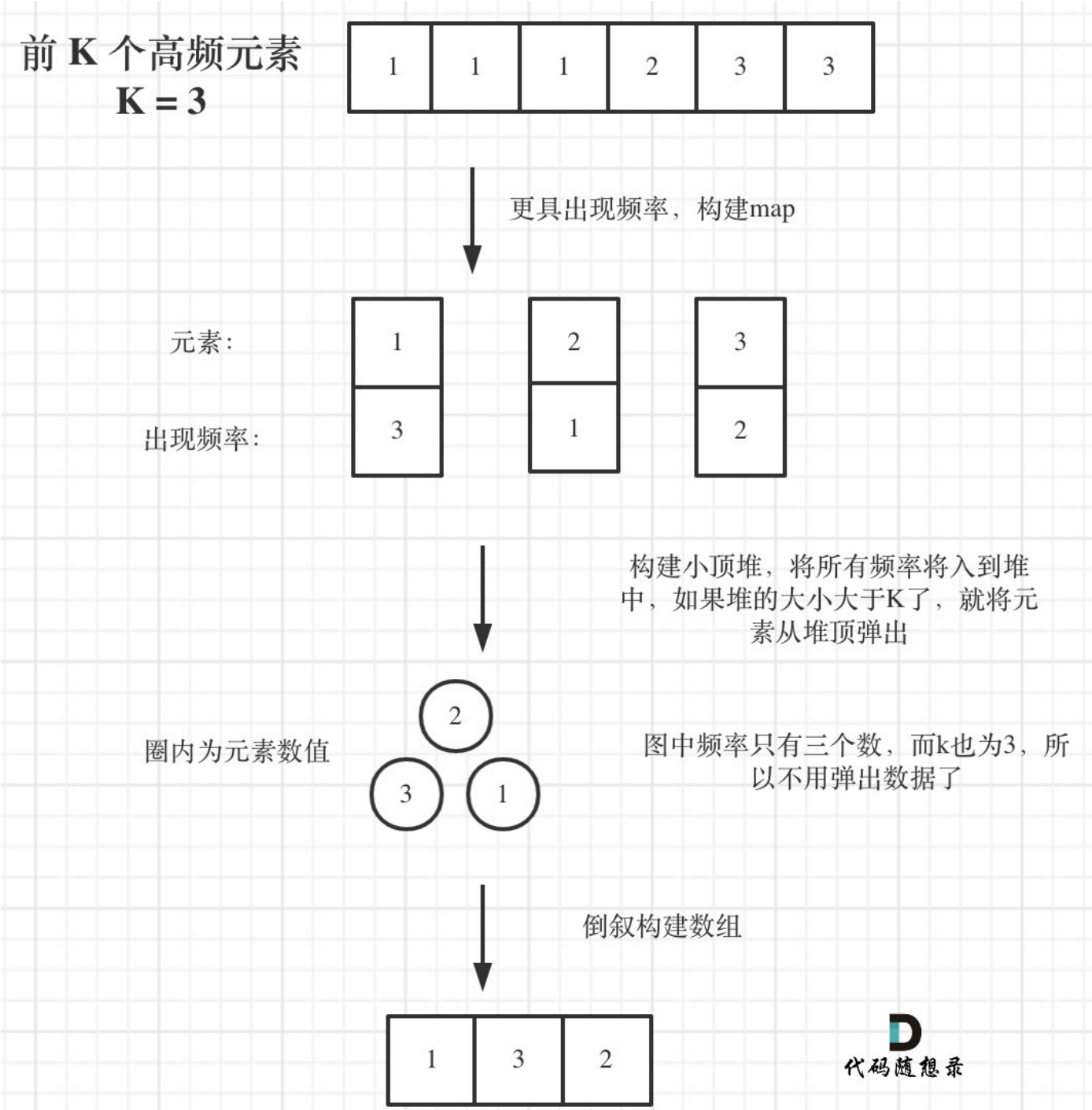
有的同学一想，题目要求前 K 个高频元素，那么果断用大顶堆啊。

那么问题来了，定义一个大小为k的大顶堆，在每次移动更新大顶堆的时候，每次弹出都把最大的元素弹出去了，那么怎么保留下来前K个高频元素呢。

而且使用大顶堆就要把所有元素都进行排序，那能不能只排序k个元素呢？

所以我们要用小顶堆，因为要统计最大前k个元素，只有小顶堆每次将最小的元素弹出，最后小顶堆里积累的才是前k个最大元素。

寻找前k个最大元素流程如图所示：（图中的频率只有三个，所以正好构成一个大小为3的小顶堆，如果频率更多一些，则用这个小顶堆进行扫描）



我们来看一下C++代码：

```
class Solution {
public:
    // 小顶堆
    class mycomparison {
    public:
        bool operator()(const pair<int, int>& lhs, const pair<int, int>& rhs) {
            return lhs.second > rhs.second;
        }
    };
```

```
    vector<int> topKFrequent(vector<int>& nums, int k) {
        // 要统计元素出现频率
        unordered_map<int, int> map; // map<nums[i],对应出现的次数>
        for (int i = 0; i < nums.size(); i++) {
            map[nums[i]]++;
        }

        // 对频率排序
        // 定义一个小顶堆，大小为k
        priority_queue<pair<int, int>, vector<pair<int, int>>, mycomparison> pri_que;

        // 用固定大小为k的小顶堆，扫面所有频率的数值
        for (unordered_map<int, int>::iterator it = map.begin(); it != map.end(); it++)
{
            pri_que.push(*it);
            if (pri_que.size() > k) { // 如果堆的大小大于了K，则队列弹出，保证堆的大小一直为k
                pri_que.pop();
            }
        }

        // 找出前K个高频元素，因为小顶堆先弹出的是最小的，所以倒序来输出到数组
        vector<int> result(k);
        for (int i = k - 1; i >= 0; i--) {
            result[i] = pri_que.top().first;
            pri_que.pop();
        }
        return result;

    }
};
```

- 时间复杂度: O(nlogk)
- 空间复杂度: O(n)

拓展

大家对这个比较运算在建堆时是如何应用的，为什么左大于右就会建立小顶堆，反而建立大顶堆比较困惑。

确实 例如我们在写快排的cmp函数的时候，`return left>right` 就是从大到小，`return left<right` 就是从小到大。

优先级队列的定义正好反过来了，可能和优先级队列的源码实现有关（我没有仔细研究），我估计是底层实现上优先队列队首指向后面，队尾指向最前面的缘故！

9. 栈与队列总结篇

栈与队列的理论基础

首先我们在栈与队列：来看看栈和队列不为人知的一面中讲解了栈和队列的理论基础。

里面提到了灵魂四问：

1. C++中stack，queue 是容器么?
2. 我们使用的stack，queue是属于那个版本的STL?
3. 我们使用的STL中stack，queue是如何实现的?
4. stack，queue 提供迭代器来遍历空间么?

相信不仅仅是C++中有这些问题，那么大家使用其他编程语言，也可以考虑一下这四个问题，栈和队列是如何实现的。

栈与队列是我们熟悉的不能再熟悉的数据结构，但它们的底层实现，很多同学都比较模糊，这其实就是基础所在。

可以出一道面试题：栈里面的元素在内存中是连续分布的么?

这个问题有两个陷阱：

- 陷阱1：栈是容器适配器，底层容器使用不同的容器，导致栈内数据在内存中是不是连续分布。
- 陷阱2：缺省情况下，默认底层容器是deque，那么deque的在内存中的数据分布是什么样的呢？ 答案是：不连续的，下文也会提到deque。

所以这就是考察候选者基础知识扎不扎实的好问题。

大家还是要多多重视起来！

了解了栈与队列基础之后，那么可以用栈与队列：栈实现队列 和 栈与队列：队列实现栈 来练习一下栈与队列的基本操作。

值得一提的是，用栈与队列：用队列实现栈还有点别扭中，其实只用一个队列就够了。

一个队列在模拟栈弹出元素的时候只要将队列头部的元素（除了最后一个元素外） 重新添加到队列尾部，此时在去弹出元素就是栈的顺序了。

栈经典题目

栈在系统中的应用

如果还记得编译原理的话，编译器在 词法分析的过程中处理括号、花括号等这个符号的逻辑，就是使用了栈这种数据结构。

再举个例子，linux系统中，cd这个进入目录的命令我们应该再熟悉不过了。

```
cd a/b/c/../../
```

这个命令最后进入a目录，系统是如何知道进入了a目录呢 ，这就是栈的应用。**这在leetcode上也是一道题目，编号：71. 简化路径，大家有空可以做一下。**

递归的实现是栈：每一次递归调用都会把函数的局部变量、参数值和返回地址等压入调用栈中，然后递归返回的时候，从栈顶弹出上一次递归的各项参数，所以这就是递归为什么可以返回上一层位置的原因。

所以栈在计算机领域中应用是非常广泛的。

有的同学经常会想学的这些数据结构有什么用，也开发不了什么软件，大多数同学说的软件应该都是可视化的软件例如APP、网站之类的，那都是非常上层的应用了，底层很多功能的实现都是基础的数据结构和算法。

所以数据结构与算法的应用往往隐藏在我们看不到的地方！

括号匹配问题

在栈与队列：系统中处处都是栈的应用中我们讲解了括号匹配问题。

括号匹配是使用栈解决的经典问题。

建议要写代码之前要分析好有哪几种不匹配的情况，如果不动手之前分析好，写出的代码也会有很多问题。

先来分析一下 这里有三种不匹配的情况，

1. 第一种情况，字符串里左方向的括号多余了 ，所以不匹配。
2. 第二种情况，括号没有多余，但是 括号的类型没有匹配上。
3. 第三种情况，字符串里右方向的括号多余了，所以不匹配。

这里还有一些技巧，在匹配左括号的时候，右括号先入栈，就只需要比较当前元素和栈顶相不相等就可以了，比左括号先入栈代码实现要简单的多了！

字符串去重问题

在栈与队列：匹配问题都是栈的强项中讲解了字符串去重问题。

047. 删除字符串中的所有相邻重复项

思路就是可以把字符串顺序放到一个栈中，然后如果相同的话 栈就弹出，这样最后栈里剩下的元素都是相邻不相同的元素了。

逆波兰表达式问题

在栈与队列：有没有想过计算机是如何处理表达式的？ 中讲解了求逆波兰表达式。

本题中每一个子表达式要得出一个结果，然后拿这个结果再进行运算，那么**这岂不就是一个相邻字符串消除的过程，和栈与队列：匹配问题都是栈的强项中的对对碰游戏是不是就非常像了。**

队列的经典题目

滑动窗口最大值问题

在栈与队列：滑动窗口里求最大值引出一个重要数据结构中讲解了一种数据结构：单调队列。

这道题目还是比较绕的，如果第一次遇到这种题目，需要反复琢磨琢磨

主要思想是**队列没有必要维护窗口里的所有元素，只需要维护有可能成为窗口里最大值的元素就可以了，同时保证队列里的元素数值是由大到小的。**

那么这个维护元素单调递减的队列就叫做**单调队列，即单调递减或单调递增的队列。C++中没有直接支持单调队列，需要我们自己来一个单调队列**

而且**不要以为实现的单调队列就是 对窗口里面的数进行排序，如果排序的话，那和优先级队列又有什么区别了呢。**

设计单调队列的时候，pop，和push操作要保持如下规则：

1. pop(value)：如果窗口移除的元素value等于单调队列的出口元素，那么队列弹出元素，否则不用任何操作
2. push(value)：如果push的元素value大于入口元素的数值，那么就将队列出口的元素弹出，直到push元素的数值小于等于队列入口元素的数值为止

保持如上规则，每次窗口移动的时候，只要问que.front()就可以返回当前窗口的最大值。

一些同学还会对单调队列都有一些困惑，首先要明确的是，**题解中单调队列里的pop和push接口，仅适用于本题。**

单调队列不是一成不变的，而是不同场景不同写法，总之要保证队列里单调递减或递增的原则，所以叫做单调队列。

不要以为本地中的单调队列实现就是固定的写法。

我们用deque作为单调队列的底层数据结构，C++中deque是stack和queue默认的底层实现容器（这个我们之前已经讲过），deque是可以两边扩展的，而且deque里元素并不是严格的连续分布的。

求前 K 个高频元素

在栈与队列：求前 K 个高频元素和队列有啥关系？ 中讲解了求前 K 个高频元素。

通过求前 K 个高频元素，引出另一种队列就是**优先级队列**。

什么是优先级队列呢?

其实**就是一个披着队列外衣的堆**，因为优先级队列对外接口只是从队头取元素，从队尾添加元素，再无其他取元素的方式，看起来就是一个队列。

而且优先级队列内部元素是自动依照元素的权值排列。那么它是如何有序排列的呢?

缺省情况下priority_queue利用max-heap（大顶堆）完成对元素的排序，这个大顶堆是以vector为表现形式的complete binary tree（完全二叉树）。

什么是堆呢?

堆是一棵完全二叉树，树中每个结点的值都不小于（或不大于）其左右孩子的值。 如果父亲结点是大于等于左右孩子就是大顶堆，小于等于左右孩子就是小顶堆。

所以大家经常说的大顶堆（堆头是最大元素），小顶堆（堆头是最小元素），如果懒得自己实现的话，就直接用priority_queue（优先级队列）就可以了，底层实现都是一样的，从小到大排就是小顶堆，从大到小排就是大顶堆。

本题就要**使用优先级队列来对部分频率进行排序。** 注意这里是对部分数据进行排序而不需要对所有数据排序！

所以排序的过程的时间复杂度是$O(\log k)$，整个算法的时间复杂度是$O(n\log k)$。

总结

在栈与队列系列中，我们强调栈与队列的基础，也是很多同学容易忽视的点。

使用抽象程度越高的语言，越容易忽视其底层实现，而C++相对来说是比较接近底层的语言。

我们用栈实现队列，用队列实现栈来掌握的栈与队列的基本操作。

接着，通过括号匹配问题、字符串去重问题、逆波兰表达式问题来系统讲解了栈在系统中的应用，以及使用技巧。

通过求滑动窗口最大值，以及前K个高频元素介绍了两种队列：单调队列和优先级队列，这是特殊场景解决问题的利器，是一定要掌握的。

好了，栈与队列我们就总结到这里了，接下来Carl就要带大家开启新的篇章了，大家加油！

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

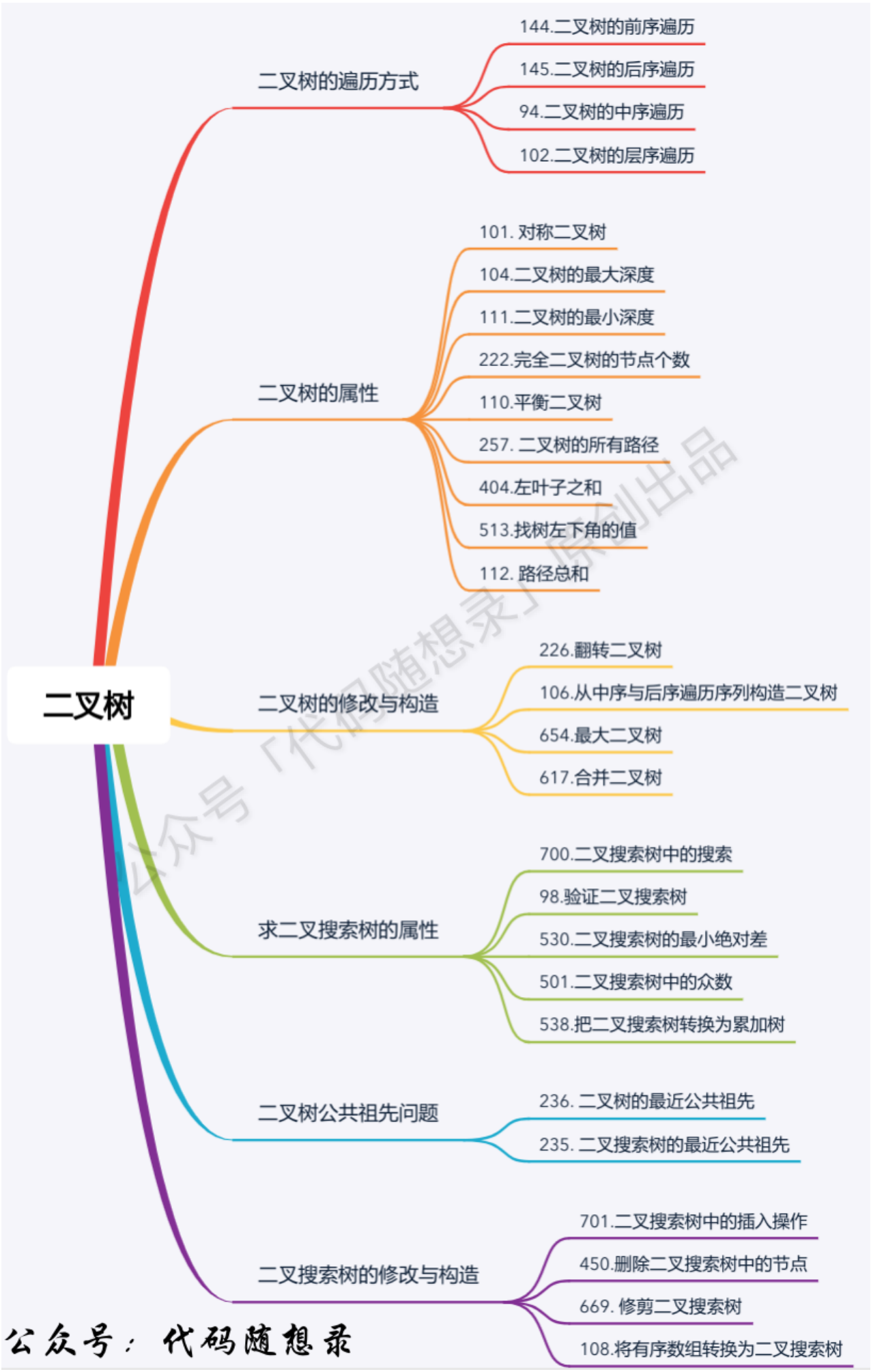
1. 二叉树理论基础篇

算法公开课

《代码随想录》算法视频公开课：关于二叉树，你该了解这些！，相信结合视频再看本篇题解，更有助于大家对本题的理解。

题目分类

题目分类大纲如下：


二叉树
二叉树的遍历方式
144.二叉树的前序遍历
145.二叉树的后序遍历
94.二叉树的中序遍历
102.二叉树的层序遍历
二叉树的属性
101. 对称二叉树
104.二叉树的最大深度
111.二叉树的最小深度
222.完全二叉树的节点个数
110.平衡二叉树
257. 二叉树的所有路径
404.左叶子之和
513.找树左下角的值
112. 路径总和
二叉树的修改与构造
226.翻转二叉树
106.从中序与后序遍历序列构造二叉树
654.最大二叉树
617.合并二叉树
求二叉搜索树的属性
700.二叉搜索树中的搜索
98.验证二叉搜索树
530.二叉搜索树的最小绝对差
501.二叉搜索树中的众数
538.把二叉搜索树转换为累加树
二叉树公共祖先问题
236. 二叉树的最近公共祖先
235. 二叉搜索树的最近公共祖先
二叉搜索树的修改与构造
701.二叉搜索树中的插入操作
450.删除二叉搜索树中的节点
669. 修剪二叉搜索树
108.将有序数组转换为二叉搜索树
公众号「代码随想录」原创出品
公众号：代码随想录

说到二叉树，大家对于二叉树其实都很熟悉了，本文呢我也不想教科书式的把二叉树的基础内容再啰嗦一遍，所以以下我讲的都是一些比较重点的内容。

相信只要耐心看完，都会有所收获。

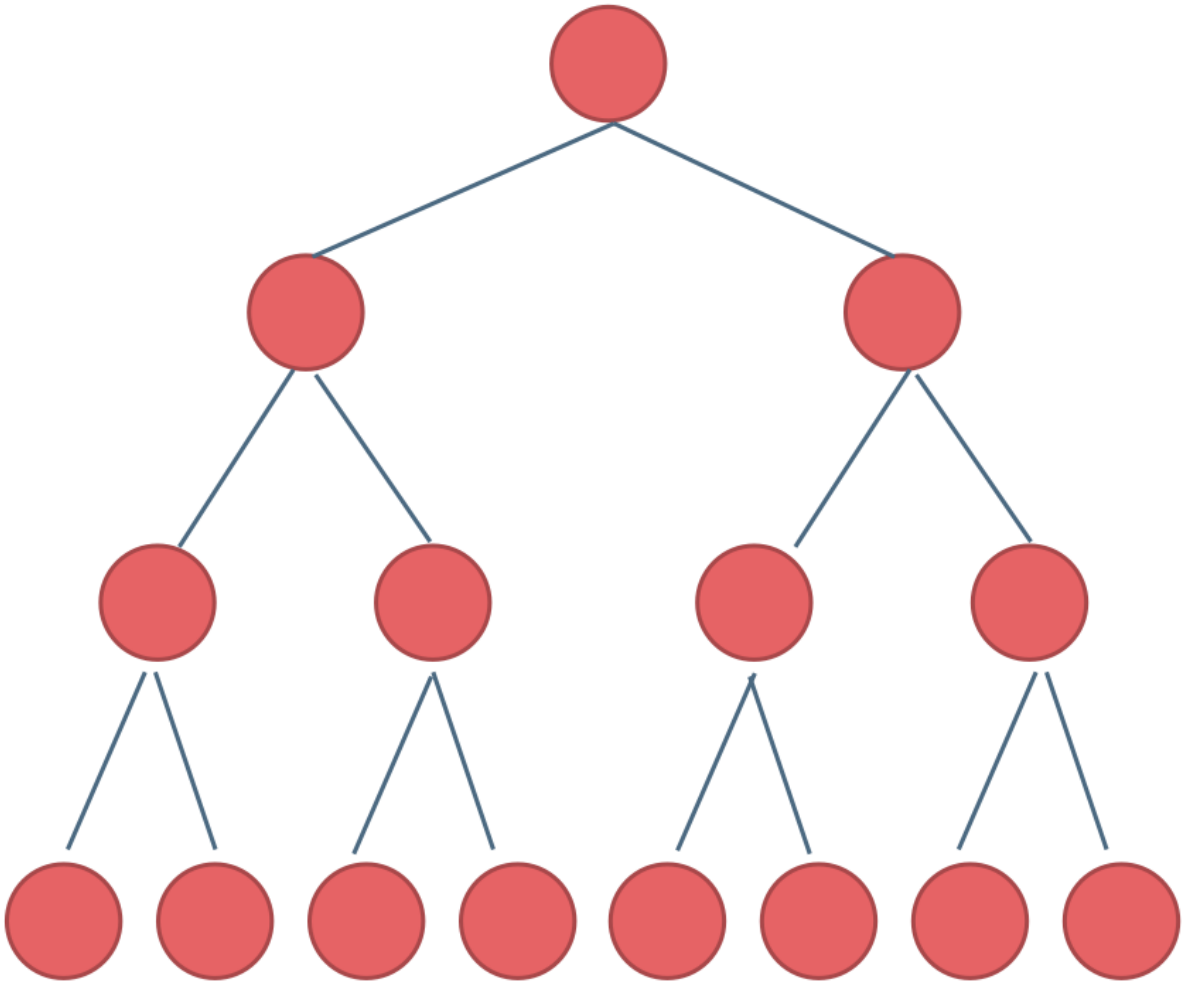
二叉树的种类

在我们解题过程中二叉树有两种主要的形式：满二叉树和完全二叉树。

满二叉树

满二叉树：如果一棵二叉树只有度为0的结点和度为2的结点，并且度为0的结点在同一层上，则这棵二叉树为满二叉树。

如图所示：



这棵二叉树为满二叉树，也可以说深度为k，有2^k-1个节点的二叉树。

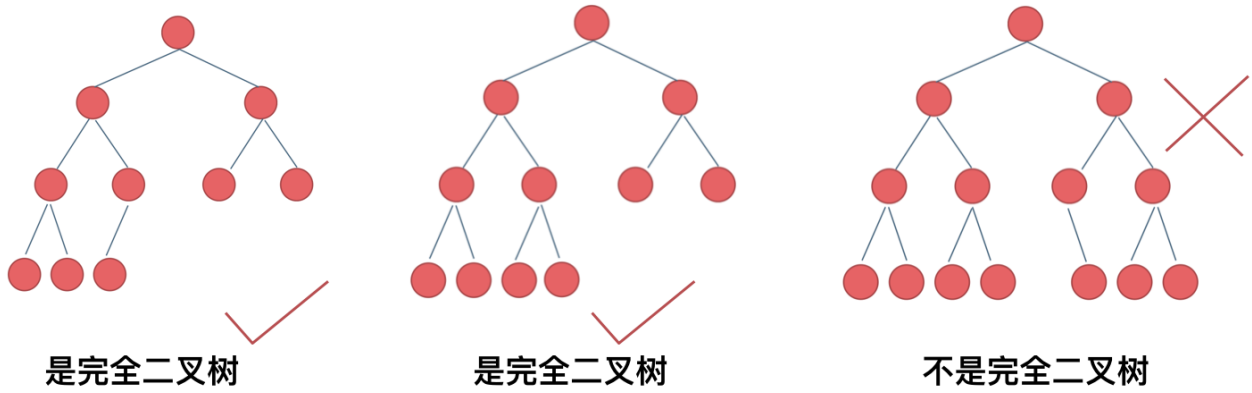
完全二叉树

什么是完全二叉树?

完全二叉树的定义如下：在完全二叉树中，除了最底层节点可能没填满外，其余每层节点数都达到最大值，并且最下面一层的节点都集中在该层最左边的若干位置。若最底层为第 h 层（h从1开始），则该层包含 1~ 2^(h-1) 个节点。

大家要自己看完全二叉树的定义，很多同学对完全二叉树其实不是真正的懂了。

我来举一个典型的例子如题：



相信不少同学最后一个二叉树是不是完全二叉树都中招了。

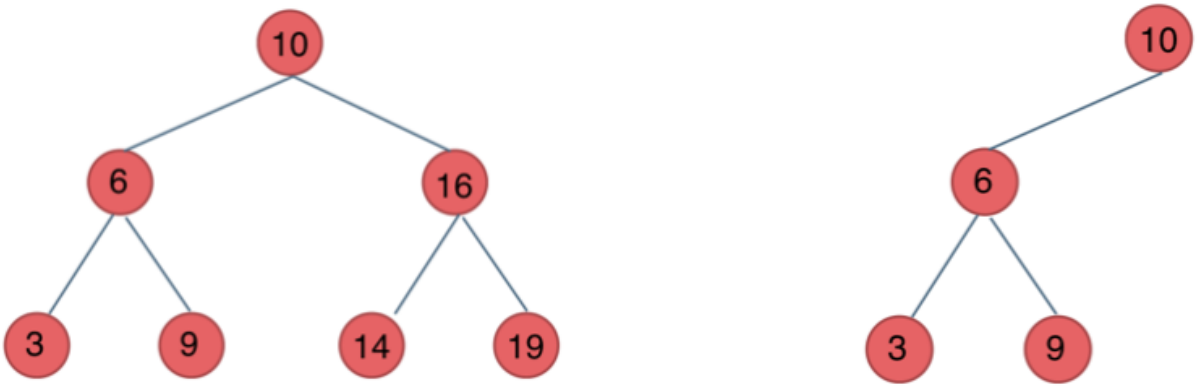
之前我们刚刚讲过优先级队列其实是一个堆，堆就是一棵完全二叉树，同时保证父子节点的顺序关系。

二叉搜索树

前面介绍的树，都没有数值的，而二叉搜索树是有数值的了，**二叉搜索树是一个有序树**。

- 若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值；
- 若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；
- 它的左、右子树也分别为二叉排序树

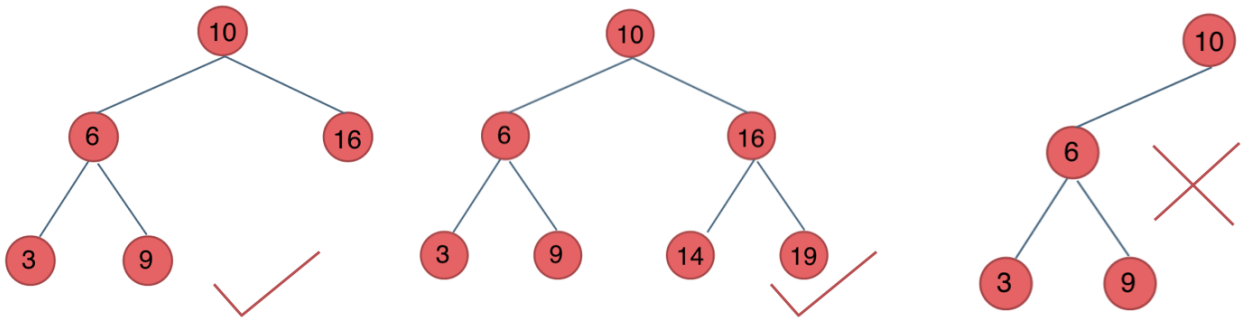
下面这两棵树都是搜索树



平衡二叉搜索树

平衡二叉搜索树：又被称为AVL（Adelson-Velsky and Landis）树，且具有以下性质：它是一棵空树或它的左右两个子树的高度差的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树。

如图：



最后一棵 不是平衡二叉树，因为它的左右两个子树的高度差的绝对值超过了1。

C++中map、set、multimap，multiset的底层实现都是平衡二叉搜索树，所以map、set的增删操作时间时间复杂度是logn，注意我这里没有说unordered_map、unordered_set，unordered_map、unordered_set底层实现是哈希表。

所以大家使用自己熟悉的编程语言写算法，一定要知道常用的容器底层都是如何实现的，最基本的就是map、set等等，否则自己写的代码，自己对其性能分析都分析不清楚！

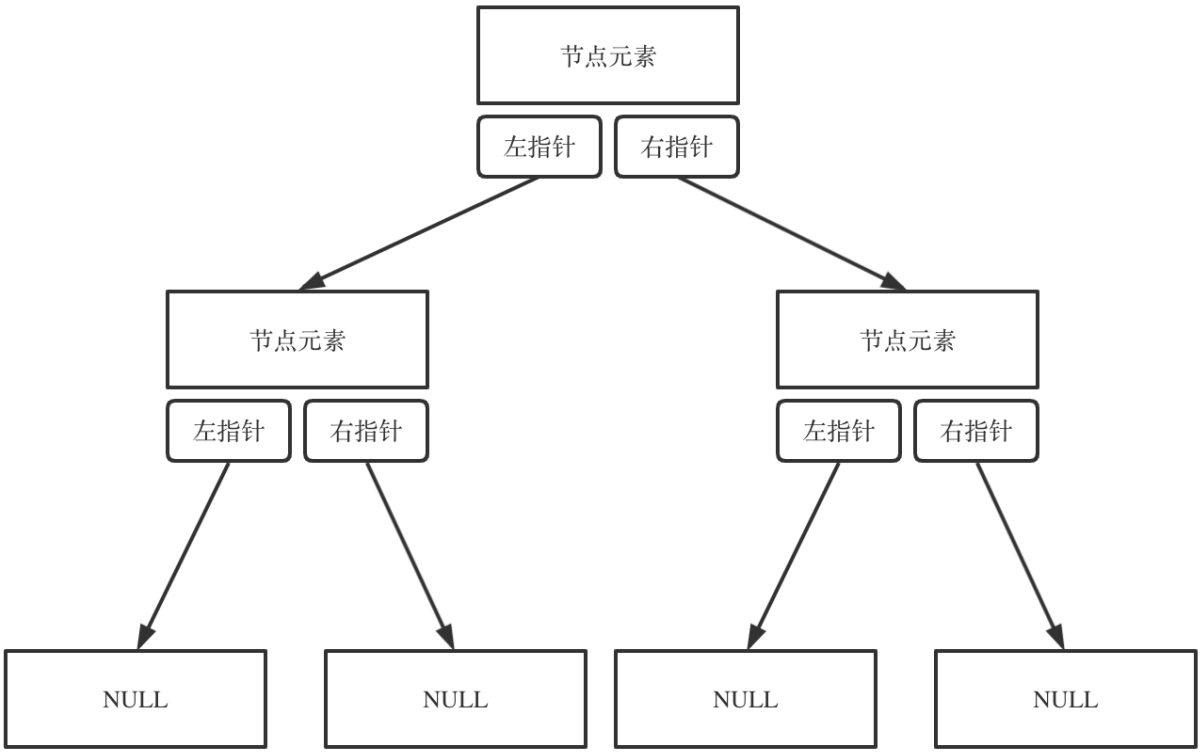
二叉树的存储方式

二叉树可以链式存储，也可以顺序存储。

那么链式存储方式就用指针， 顺序存储的方式就是用数组。

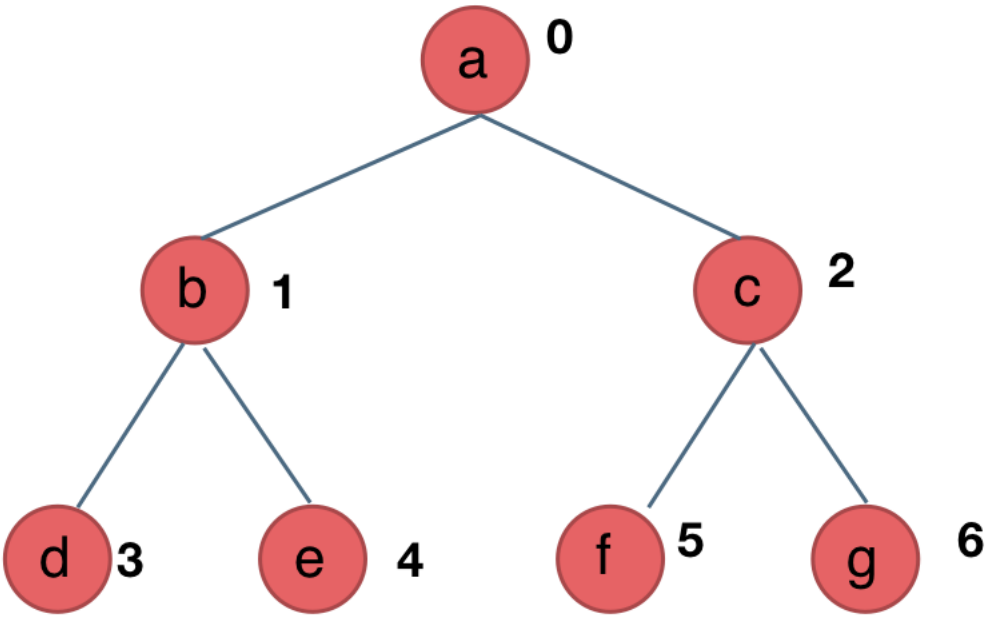
顾名思义就是顺序存储的元素在内存是连续分布的，而链式存储则是通过指针把分布在各个地址的节点串联一起。

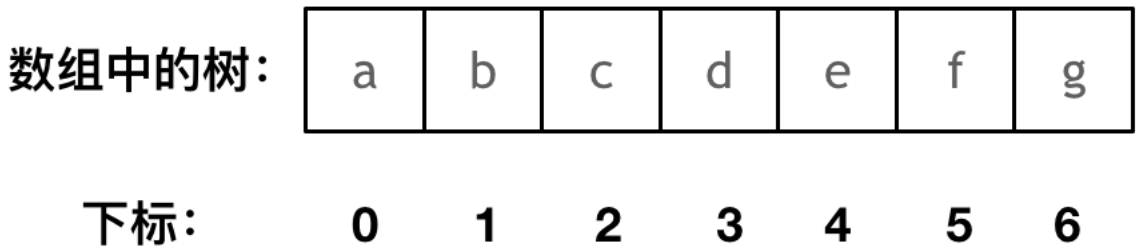
链式存储如图：



链式存储是大家很熟悉的一种方式，那么我们来看看如何顺序存储呢?

其实就是用数组来存储二叉树，顺序存储的方式如图：





用数组来存储二叉树如何遍历的呢?

如果父节点的数组下标是 i，那么它的左孩子就是 i * 2 + 1，右孩子就是 i * 2 + 2。

但是用链式表示的二叉树，更有利于我们理解，所以一般我们都是用链式存储二叉树。

所以大家要了解，用数组依然可以表示二叉树。

二叉树的遍历方式

关于二叉树的遍历方式，要知道二叉树遍历的基本方式都有哪些。

一些同学用做了很多二叉树的题目了，可能知道前中后序遍历，可能知道层序遍历，但是却没有框架。

我这里把二叉树的几种遍历方式列出来，大家就可以一一串起来了。

二叉树主要有两种遍历方式：

1. 深度优先遍历：先往深走，遇到叶子节点再往回走。
2. 广度优先遍历：一层一层的去遍历。

这两种遍历是图论中最基本的两种遍历方式，后面在介绍图论的时候 还会介绍到。

那么从深度优先遍历和广度优先遍历进一步拓展，才有如下遍历方式：

- 深度优先遍历
    - 前序遍历（递归法，迭代法）
    - 中序遍历（递归法，迭代法）
    - 后序遍历（递归法，迭代法）
- 广度优先遍历
    - 层次遍历（迭代法）

在深度优先遍历中：有三个顺序，前中后序遍历， 有同学总分不清这三个顺序，经常搞混，我这里教大家一个技巧。

这里前中后，其实指的就是中间节点的遍历顺序，只要大家记住 前中后序指的就是中间节点的位置就可以了。

看如下中间节点的顺序，就可以发现，中间节点的顺序就是所谓的遍历方式

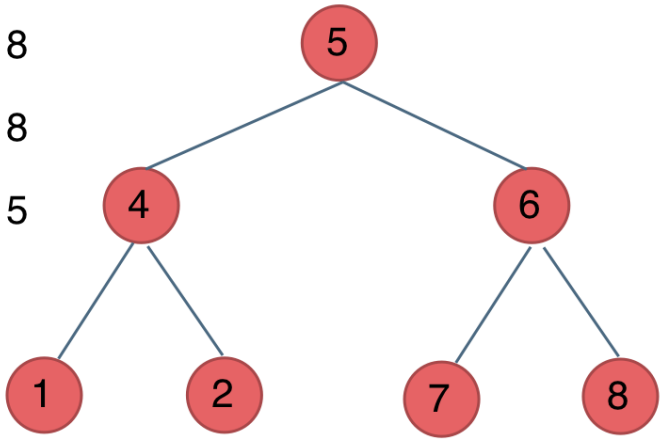
- 前序遍历：中左右
- 中序遍历：左中右
- 后序遍历：左右中

大家可以对着如下图，看看自己理解的前后中序有没有问题。

前序遍历（中左右）：5 4 1 2 6 7 8

中序遍历（左中右）：1 4 2 5 7 6 8

后序遍历（左右中）：1 2 4 7 8 6 5



最后再说一说二叉树中深度优先和广度优先遍历实现方式，我们做二叉树相关题目，经常会使用递归的方式来实现深度优先遍历，也就是实现前中后序遍历，使用递归是比较方便的。

之前我们讲栈与队列的时候，就说过栈其实就是递归的一种实现结构，也就说前中后序遍历的逻辑其实都是可以借助栈使用非递归的方式来实现的。

而广度优先遍历的实现一般使用队列来实现，这也是队列先进先出的特点所决定的，因为需要先进先出的结构，才能一层一层的来遍历二叉树。

这里其实我们又了解了栈与队列的一个应用场景了。

具体的实现我们后面都会讲的，这里大家先要清楚这些理论基础。

二叉树的定义

刚刚我们说过了二叉树有两种存储方式顺序存储，和链式存储，顺序存储就是用数组来存，这个定义没啥可说的，我们来看看链式存储的二叉树节点的定义方式。

C++代码如下：

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
```

大家会发现二叉树的定义 和链表是差不多的，相对于链表 ，二叉树的节点里多了一个指针， 有两个指针，指向左右孩子。

这里要提醒大家要注意二叉树节点定义的书写方式。

在现场面试的时候 面试官可能要求手写代码，所以数据结构的定义以及简单逻辑的代码一定要锻炼白纸写出来。

因为我们在刷leetcode的时候，节点的定义默认都定义好了，真到面试的时候，需要自己写节点定义的时候，有时候会一脸懵逼！

总结

二叉树是一种基础数据结构，在算法面试中都是常客，也是众多数据结构的基石。

本篇我们介绍了二叉树的种类、存储方式、遍历方式以及定义，比较全面的介绍了二叉树各个方面的重点，帮助大家扫一遍基础。

说到二叉树，就不得不说递归，很多同学对递归都是又熟悉又陌生，递归的代码一般很简短，但每次都是一看就会，一写就废。

> 一看就会，一写就废！

2. 二叉树的递归遍历

算法公开课

[《代码随想录》算法视频公开课](#)：[每次写递归都要靠直觉？ 这次带你学透二叉树的递归遍历！](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

这次我们要好好谈一谈递归，为什么很多同学看递归算法都是“一看就会，一写就废”。

主要是对递归不成体系，没有方法论，**每次写递归算法，都是靠玄学来写代码**，代码能不能编过都靠运气。

本篇将介绍前后中序的递归写法，一些同学可能会感觉很简单，其实不然，我们要通过简单题目把方法论确定下来，有了方法论，后面才能应付复杂的递归。

这里帮助大家确定下来递归算法的三个要素。**每次写递归，都按照这三要素来写，可以保证大家写出正确的递归算法！**

1. **确定递归函数的参数和返回值：**
   确定哪些参数是递归的过程中需要处理的，那么就在递归函数里加上这个参数， 并且还要明确每次递归的返回值是什么进而确定递归函数的返回类型。
2. **确定终止条件：**
   写完了递归算法，运行的时候，经常会遇到栈溢出的错误，就是没写终止条件或者终止条件写的不对，操作系统也是用一个栈的结构来保存每一层递归的信息，如果递归没有终止，操作系统的内存栈必然就会溢出。
3. **确定单层递归的逻辑：**
   确定每一层递归需要处理的信息。在这里也就会重复调用自己来实现递归的过程。

好了，我们确认了递归的三要素，接下来就来练练手：

以下以前序遍历为例：

1. **确定递归函数的参数和返回值**：因为要打印出前序遍历节点的数值，所以参数里需要传入vector来放节点的数值，除了这一点就不需要再处理什么数据了也不需要有返回值，所以递归函数返回类型就是void，代码如下：

```
void traversal(TreeNode* cur, vector<int>& vec)
```

2. **确定终止条件**：在递归的过程中，如何算是递归结束了呢，当然是当前遍历的节点是空了，那么本层递归就要结束了，所以如果当前遍历的这个节点是空，就直接return，代码如下：

```
if (cur == NULL) return;
```

3. **确定单层递归的逻辑**：前序遍历是中左右的循序，所以在单层递归的逻辑，是要先取中节点的数值，代码如下：

```
vec.push_back(cur->val);    // 中
traversal(cur->left, vec);  // 左
traversal(cur->right, vec); // 右
```

单层递归的逻辑就是按照中左右的顺序来处理的，这样二叉树的前序遍历，基本就写完了，再看一下完整代码：

前序遍历：

```
class Solution {
public:
    void traversal(TreeNode* cur, vector<int>& vec) {
        if (cur == NULL) return;
        vec.push_back(cur->val);    // 中
        traversal(cur->left, vec);  // 左
        traversal(cur->right, vec); // 右
    }
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> result;
        traversal(root, result);
        return result;
    }
};
```

那么前序遍历写出来之后，中序和后序遍历就不难理解了，代码如下：

中序遍历：

```
void traversal(TreeNode* cur, vector<int>& vec) {
    if (cur == NULL) return;
    traversal(cur->left, vec);  // 左
    vec.push_back(cur->val);    // 中
    traversal(cur->right, vec); // 右
}
```

后序遍历：

```
void traversal(TreeNode* cur, vector<int>& vec) {
    if (cur == NULL) return;
    traversal(cur->left, vec);  // 左
    traversal(cur->right, vec); // 右
    vec.push_back(cur->val);    // 中
}
```

此时大家可以做一做leetcode上三道题目，分别是：

- 144.二叉树的前序遍历
- 145.二叉树的后序遍历
- 94.二叉树的中序遍历

可能有同学感觉前后中序遍历的递归太简单了，要打迭代法（非递归），别急，我们明天打迭代法，打个通透！

> 听说还可以用非递归的方式

3. 二叉树的迭代遍历

算法公开课

《代码随想录》算法视频公开课：

- **写出二叉树的非递归遍历很难么？（前序和后序）**
- **写出二叉树的非递归遍历很难么？（中序）**
  相信结合视频在看本篇题解，更有助于大家对本题的理解。

看完本篇大家可以使用迭代法，再重新解决如下三道leetcode上的题目：

- 144.二叉树的前序遍历
- 94.二叉树的中序遍历

- 145.二叉树的后序遍历

思路

为什么可以用迭代法（非递归的方式）来实现二叉树的前后中序遍历呢?

我们在栈与队列：匹配问题都是栈的强项中提到了，**递归的实现就是：每一次递归调用都会把函数的局部变量、参数值和返回地址等压入调用栈中**，然后递归返回的时候，从栈顶弹出上一次递归的各项参数，所以这就是递归为什么可以返回上一层位置的原因。

此时大家应该知道我们用栈也可以是实现二叉树的前后中序遍历了。

前序遍历（迭代法）

我们先看一下前序遍历。

前序遍历是中左右，每次先处理的是中间节点，那么先将根节点放入栈中，然后将右孩子加入栈，再加入左孩子。

为什么要先加入 右孩子，再加入左孩子呢？ 因为这样出栈的时候才是中左右的顺序。

动画如下：

不难写出如下代码: （**注意代码中空节点不入栈**）

```
class Solution {
public:
```

```
    vector<int> preorderTraversal(TreeNode* root) {
        stack<TreeNode*> st;
        vector<int> result;
        if (root == NULL) return result;
        st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();                       // 中
            st.pop();
            result.push_back(node->val);
            if (node->right) st.push(node->right);           // 右（空节点不入栈）
            if (node->left) st.push(node->left);             // 左（空节点不入栈）
        }
        return result;
    }
};
```

此时会发现貌似使用迭代法写出前序遍历并不难，确实不难。

此时是不是想改一点前序遍历代码顺序就把中序遍历搞出来了？

其实还真不行！

但接下来，**再用迭代法写中序遍历的时候，会发现套路又不一样了，目前的前序遍历的逻辑无法直接应用到中序遍历上。**

中序遍历（迭代法）

为了解释清楚，我说明一下 刚刚在迭代的过程中，其实我们有两个操作：

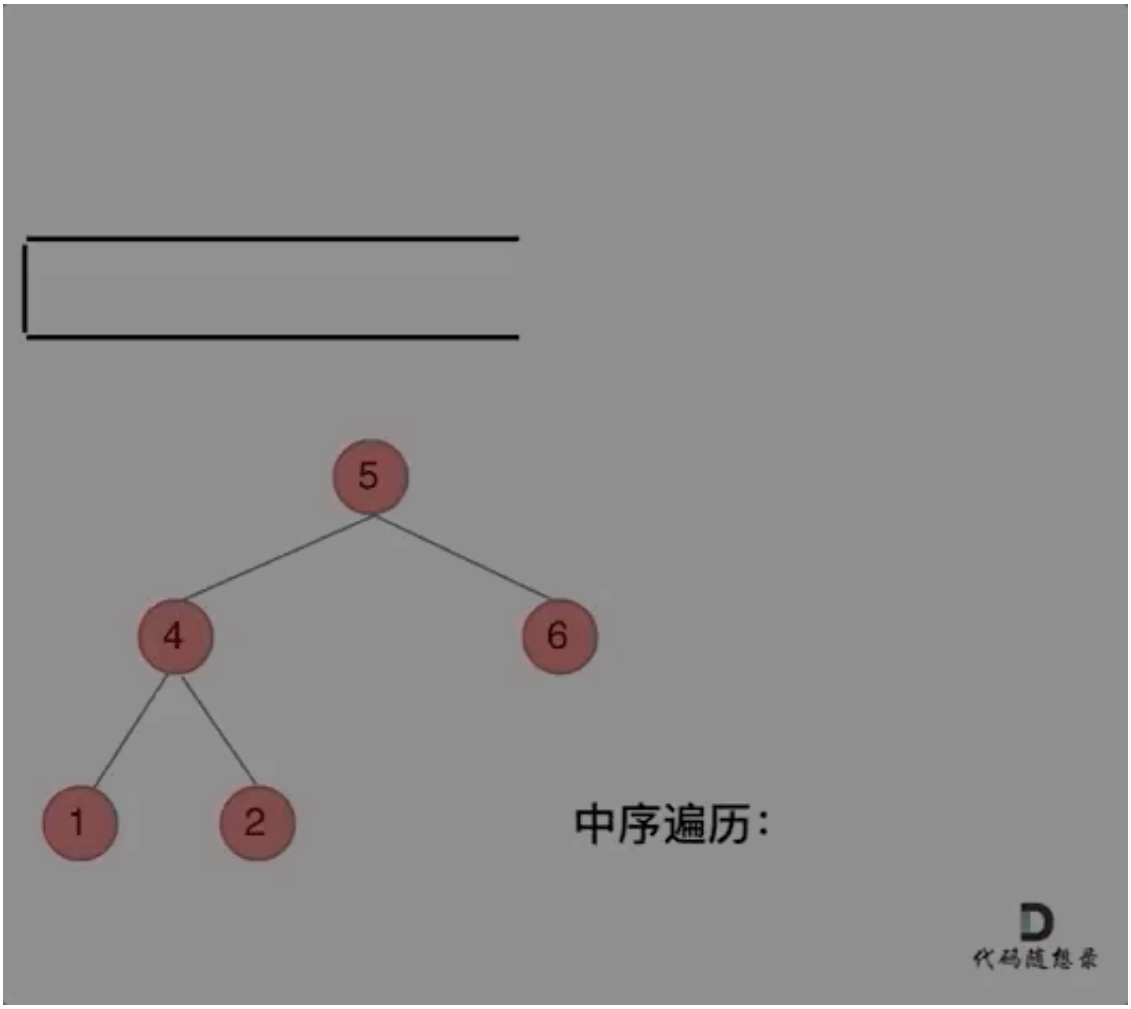
1. **处理：将元素放进result数组中**
2. **访问：遍历节点**

分析一下为什么刚刚写的前序遍历的代码，不能和中序遍历通用呢，因为前序遍历的顺序是中左右，先访问的元素是中间节点，要处理的元素也是中间节点，所以刚刚才能写出相对简洁的代码，**因为要访问的元素和要处理的元素顺序是一致的，都是中间节点。**

那么再看看中序遍历，中序遍历是左中右，先访问的是二叉树顶部的节点，然后一层一层向下访问，直到到达树左面的最底部，再开始处理节点（也就是在把节点的数值放进result数组中），这就造成了**处理顺序和访问顺序是不一致的。**

那么**在使用迭代法写中序遍历，就需要借用指针的遍历来帮助访问节点，栈则用来处理节点上的元素。**

动画如下：

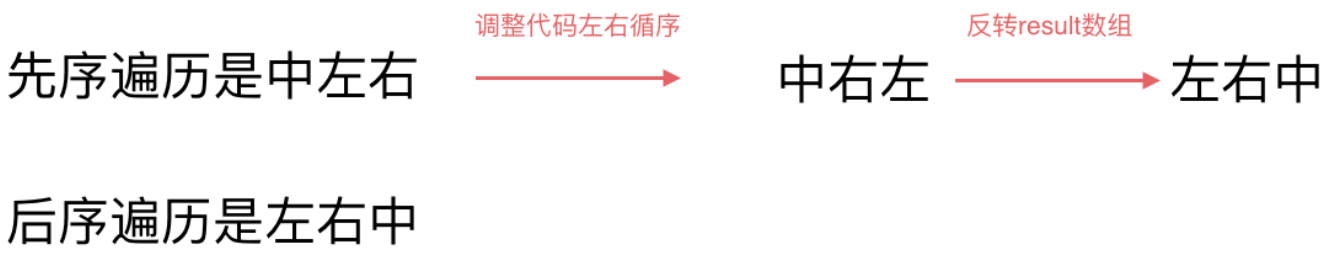


中序遍历，可以写出如下代码：

```
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> result;
        stack<TreeNode*> st;
        TreeNode* cur = root;
        while (cur != NULL || !st.empty()) {
            if (cur != NULL) { // 指针来访问节点，访问到最底层
                st.push(cur); // 将访问的节点放进栈
                cur = cur->left;                // 左
            } else {
                cur = st.top(); // 从栈里弹出的数据，就是要处理的数据（放进result数组里的数据）
                st.pop();
                result.push_back(cur->val);     // 中
                cur = cur->right;               // 右
            }
        }
        return result;
    }
};
```

后序遍历（迭代法）

再来看后序遍历，先序遍历是中左右，后续遍历是左右中，那么我们只需要调整一下先序遍历的代码顺序，就变成中右左的遍历顺序，然后在反转result数组，输出的结果顺序就是左右中了，如下图：



后序遍历是左右中

所以后序遍历只需要前序遍历的代码稍作修改就可以了，代码如下：

```
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        stack<TreeNode*> st;
        vector<int> result;
        if (root == NULL) return result;
        st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            st.pop();
            result.push_back(node->val);
            if (node->left) st.push(node->left); // 相对于前序遍历，这更改一下入栈顺序 （空节点不入栈）
            if (node->right) st.push(node->right); // 空节点不入栈
        }
        reverse(result.begin(), result.end()); // 将结果反转之后就是左右中的顺序了
        return result;
    }
};
```

总结

此时我们用迭代法写出了二叉树的前后中序遍历，大家可以看出前序和中序是完全两种代码风格，并不像递归写法那样代码稍做调整，就可以实现前后中序。

这是因为前序遍历中访问节点（遍历节点）和处理节点（将元素放进result数组中）可以同步处理，但是中序就无法做到同步！

上面这句话，可能一些同学不太理解，建议自己亲手用迭代法，先写出来前序，再试试能不能写出中序，就能理解了。

那么问题又来了，难道 二叉树前后中序遍历的迭代法实现，就不能风格统一么（即前序遍历 改变代码顺序就可以实现中序 和 后序）？

当然可以，这种写法，还不是很好理解，我们将在下一篇文章里重点讲解，敬请期待！

> 统一写法是一种什么感觉

4. 二叉树的统一迭代法

思路

此时我们在二叉树：一入递归深似海，从此offer是路人中用递归的方式，实现了二叉树前中后序的遍历。

在二叉树：听说递归能做的，栈也能做！中用栈实现了二叉树前后中序的迭代遍历（非递归）。

之后我们发现**迭代法实现的先中后序，其实风格也不是那么统一，除了先序和后序，有关联，中序完全就是另一个风格了，一会用栈遍历，一会又用指针来遍历。**

实践过的同学，也会发现使用迭代法实现先中后序遍历，很难写出统一的代码，不像是递归法，实现了其中的一种遍历方式，其他两种只要稍稍改一下节点顺序就可以了。

其实**针对三种遍历方式，使用迭代法是可以写出统一风格的代码！**

重头戏来了，接下来介绍一下统一写法。

我们以中序遍历为例，在二叉树：听说递归能做的，栈也能做！中提到说使用栈的话，**无法同时解决访问节点（遍历节点）和处理节点（将元素放进结果集）不一致的情况。**

那我们就将访问的节点放入栈中，把要处理的节点也放入栈中但是要做标记。

如何标记呢，**就是要处理的节点放入栈之后，紧接着放入一个空指针作为标记。** 这种方法也可以叫做标记法。

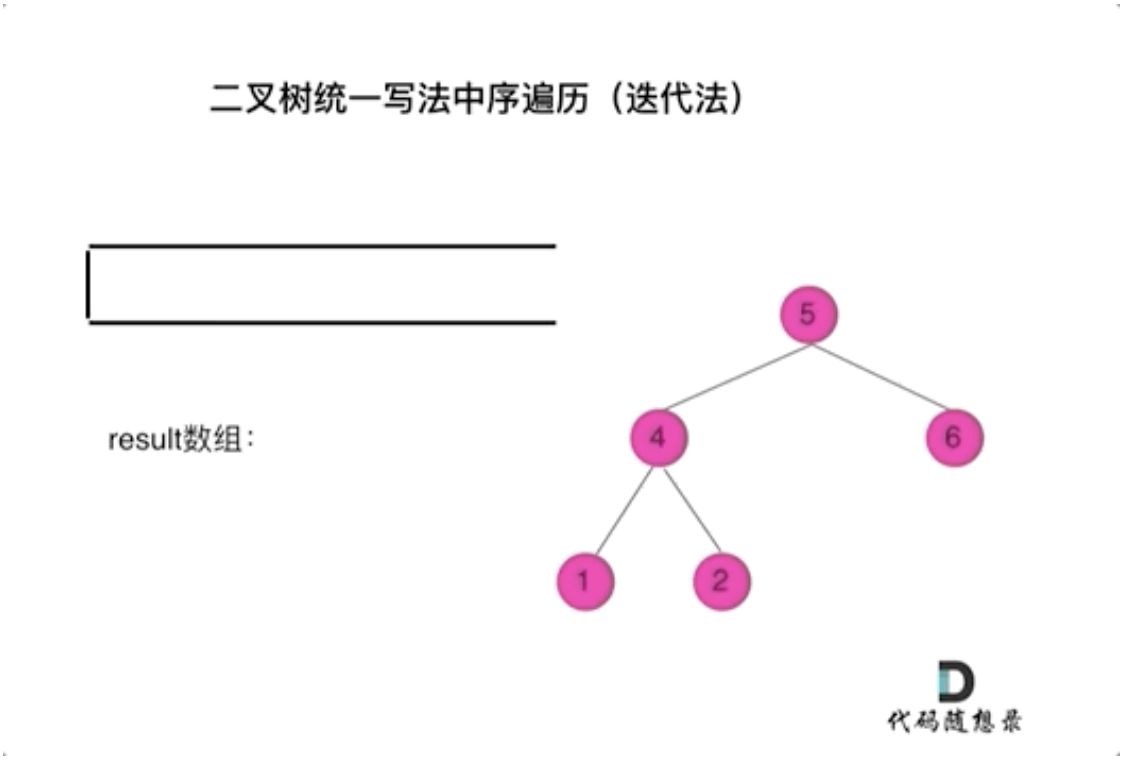
迭代法中序遍历

中序遍历代码如下：（详细注释）

```
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> result;
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop(); // 将该节点弹出，避免重复操作，下面再将右中左节点添加到栈中
                if (node->right) st.push(node->right);  // 添加右节点（空节点不入栈）

                st.push(node);                          // 添加中节点
                st.push(NULL); // 中节点访问过，但是还没有处理，加入空节点做为标记。
```

```
                if (node->left) st.push(node->left);     // 添加左节点（空节点不入栈）
            } else { // 只有遇到空节点的时候，才将下一个节点放进结果集
                st.pop();           // 将空节点弹出
                node = st.top();    // 重新取出栈中元素
                st.pop();
                result.push_back(node->val); // 加入到结果集
            }
        }
        return result;
    }
};
```

看代码有点抽象我们来看一下动画(中序遍历)：



动画中，result数组就是最终结果集。

可以看出我们将访问的节点直接加入到栈中，但如果是处理的节点则后面放入一个空节点， 这样只有空节点弹出的时候，才将下一个节点放进结果集。

此时我们再来看前序遍历代码。

迭代法前序遍历

迭代法前序遍历代码如下：（**注意此时我们和中序遍历相比仅仅改变了两行代码的顺序**）

```
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> result;
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
```

```
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                if (node->right) st.push(node->right);  // 右
                if (node->left) st.push(node->left);    // 左
                st.push(node);                          // 中
                st.push(NULL);
            } else {
                st.pop();
                node = st.top();
                st.pop();
                result.push_back(node->val);
            }
        }
        return result;
    }
};
```

迭代法后序遍历

后续遍历代码如下：(**注意此时我们和中序遍历相比仅仅改变了两行代码的顺序**)

```
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> result;
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                st.push(node);                          // 中
                st.push(NULL);

                if (node->right) st.push(node->right);  // 右
                if (node->left) st.push(node->left);    // 左

            } else {
                st.pop();
                node = st.top();
                st.pop();
                result.push_back(node->val);
            }
        }
        return result;
    }
};
```

总结

此时我们写出了统一风格的迭代法，不用在纠结于前序写出来了，中序写不出来的情况了。

但是统一风格的迭代法并不好理解，而且想在面试直接写出来还有难度的。

所以大家根据自己的个人喜好，对于二叉树的前中后序遍历，选择一种自己容易理解的递归和迭代法。

---

5. 叉树层序遍历登场！

算法公开课

《代码随想录》算法视频公开课：讲透二叉树的层序遍历 | 广度优先搜索 | LeetCode：102.二叉树的层序遍历，相信结合视频再看本篇题解，更有助于大家对本题的理解。

学会二叉树的层序遍历，可以一口气打完以下十题：

- 102.二叉树的层序遍历
- 107.二叉树的层次遍历II
- 199.二叉树的右视图
- 637.二叉树的层平均值
- 429.N叉树的层序遍历
- 515.在每个树行中找最大值
- 116.填充每个节点的下一个右侧节点指针
- 117.填充每个节点的下一个右侧节点指针II
- 104.二叉树的最大深度
- 111.二叉树的最小深度

二叉树的层序遍历

[力扣题目链接]

给你一个二叉树，请你返回其按 层序遍历 得到的节点值。 （即逐层地，从左到右访问所有节点）。

示例：

二叉树： `[3,9,20,null,null,15,7]`，

```
    3
   / \
  9  20
    /  \
   15   7
```

返回其层次遍历结果：

```
[
  [3],
  [9,20],
  [15,7]
]
```

思路

我们之前讲过了三篇关于二叉树的深度优先遍历的文章：

- 二叉树：前中后序递归法
- 二叉树：前中后序迭代法
- 二叉树：前中后序迭代方式统一写法

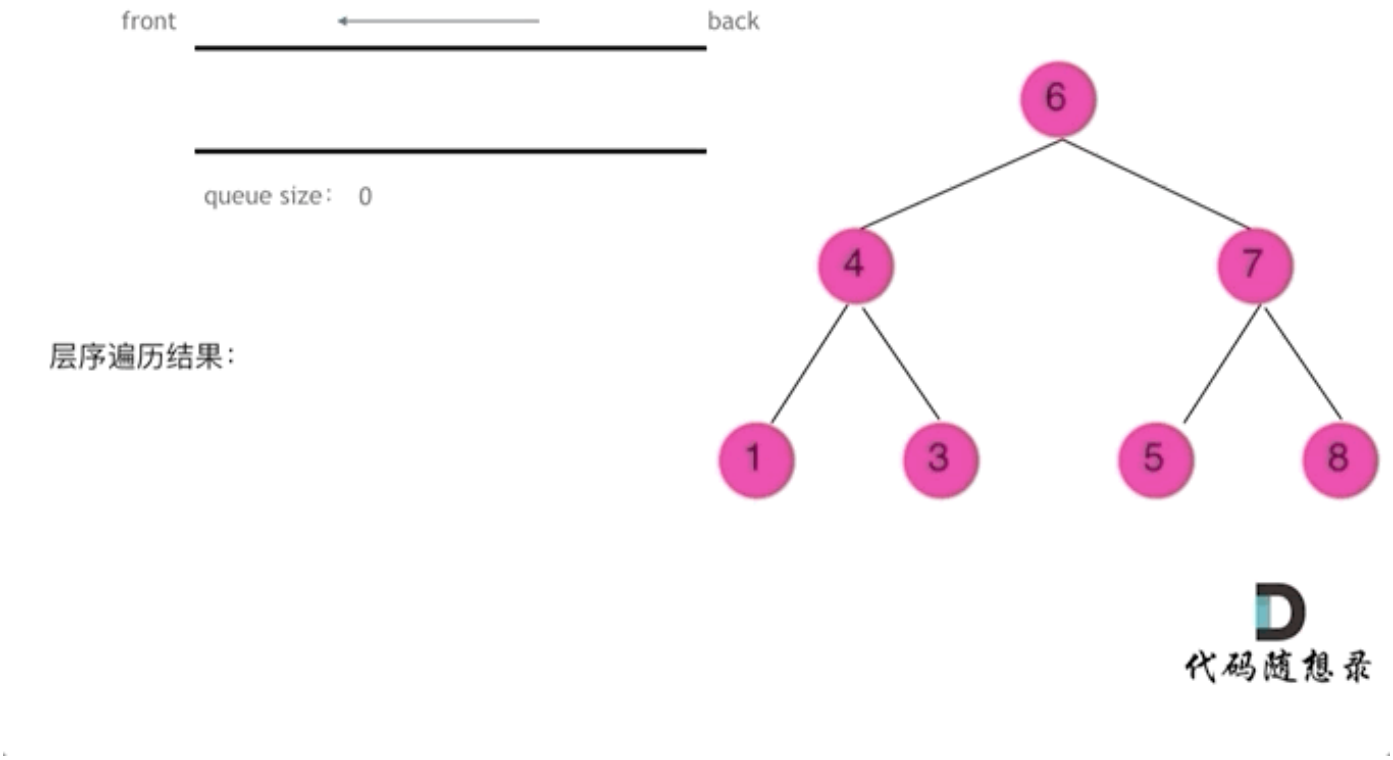
接下来我们再来介绍二叉树的另一种遍历方式：层序遍历。

层序遍历一个二叉树。就是从左到右一层一层的去遍历二叉树。这种遍历的方式和我们之前讲过的都不太一样。

需要借用一个辅助数据结构即队列来实现，**队列先进先出，符合一层一层遍历的逻辑，而用栈先进后出适合模拟深度优先遍历也就是递归的逻辑。**

而这种层序遍历方式就是图论中的广度优先遍历，只不过我们应用在二叉树上。

使用队列实现二叉树广度优先遍历，动画如下：



这样就实现了层序从左到右遍历二叉树。

代码如下：**这份代码也可以作为二叉树层序遍历的模板，打十个就靠它了。**

c++代码如下：

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        vector<vector<int>> result;
        while (!que.empty()) {
            int size = que.size();
            vector<int> vec;
            // 这里一定要使用固定大小size，不要使用que.size()，因为que.size是不断变化的
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
```

```
                vec.push_back(node->val);
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
            result.push_back(vec);
        }
        return result;
    }
};
```

```
# 递归法
class Solution {
public:
    void order(TreeNode* cur, vector<vector<int>>& result, int depth)
    {
        if (cur == nullptr) return;
        if (result.size() == depth) result.push_back(vector<int>());
        result[depth].push_back(cur->val);
        order(cur->left, result, depth + 1);
        order(cur->right, result, depth + 1);
    }
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> result;
        int depth = 0;
        order(root, result, depth);
        return result;
    }
};
```

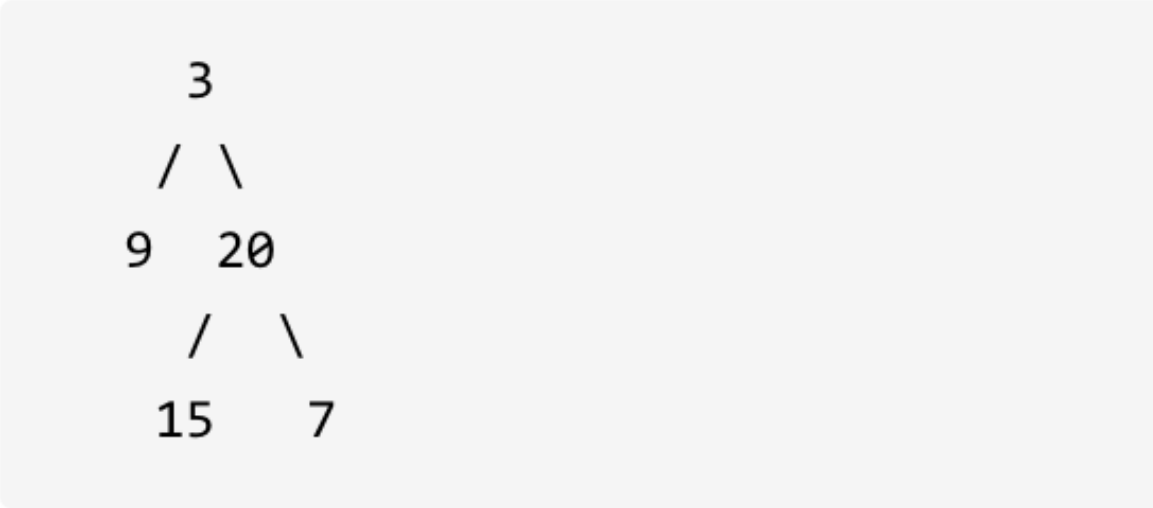
此时我们就掌握了二叉树的层序遍历了，那么如下九道力扣上的题目，只需要修改模板的两三行代码（不能再多了），便可打倒！

二叉树的层次遍历 II

[力扣题目链接]

给定一个二叉树，返回其节点值自底向上的层次遍历。 （即按从叶子节点所在层到根节点所在的层，逐层从左向右遍历）

例如：
给定二叉树 `[3,9,20,null,null,15,7]`，



返回其自底向上的层次遍历为：

```
[
  [15,7],
  [9,20],
  [3]
]
```

思路

相对于102.二叉树的层序遍历，就是最后把result数组反转一下就可以了。

C++代码：

```
class Solution {
public:
    vector<vector<int>> levelOrderBottom(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        vector<vector<int>> result;
        while (!que.empty()) {
            int size = que.size();
```

```
            vector<int> vec;
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                vec.push_back(node->val);
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
            result.push_back(vec);
        }
        reverse(result.begin(), result.end()); // 在这里反转一下数组即可
        return result;

    }
};
```

二叉树的右视图

[力扣题目链接]

给定一棵二叉树，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

> 示例:
>
> 输入: [1,2,3,null,5,null,4]
> 输出: [1, 3, 4]
> 解释:
>
>
>

思路

层序遍历的时候，判断是否遍历到单层的最后面的元素，如果是，就放进result数组中，随后返回result就可以了。

C++代码：

```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        vector<int> result;
        while (!que.empty()) {
            int size = que.size();
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                if (i == (size - 1)) result.push_back(node->val); // 将每一层的最后元素放入
result数组中
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return result;
    }
};
```

二叉树的层平均值

力扣题目链接

给定一个非空二叉树, 返回一个由每层节点平均值组成的数组。

示例 1：

```
输入：
    3
   / \
  9  20
    /  \
   15   7
输出：[3, 14.5, 11]
解释：
第 0 层的平均值是 3 ,  第1层是 14.5 , 第2层是 11 。因此返回 [3, 14.5, 11] 。
```

思路

本题就是层序遍历的时候把一层求个总和在取一个均值。

C++代码:

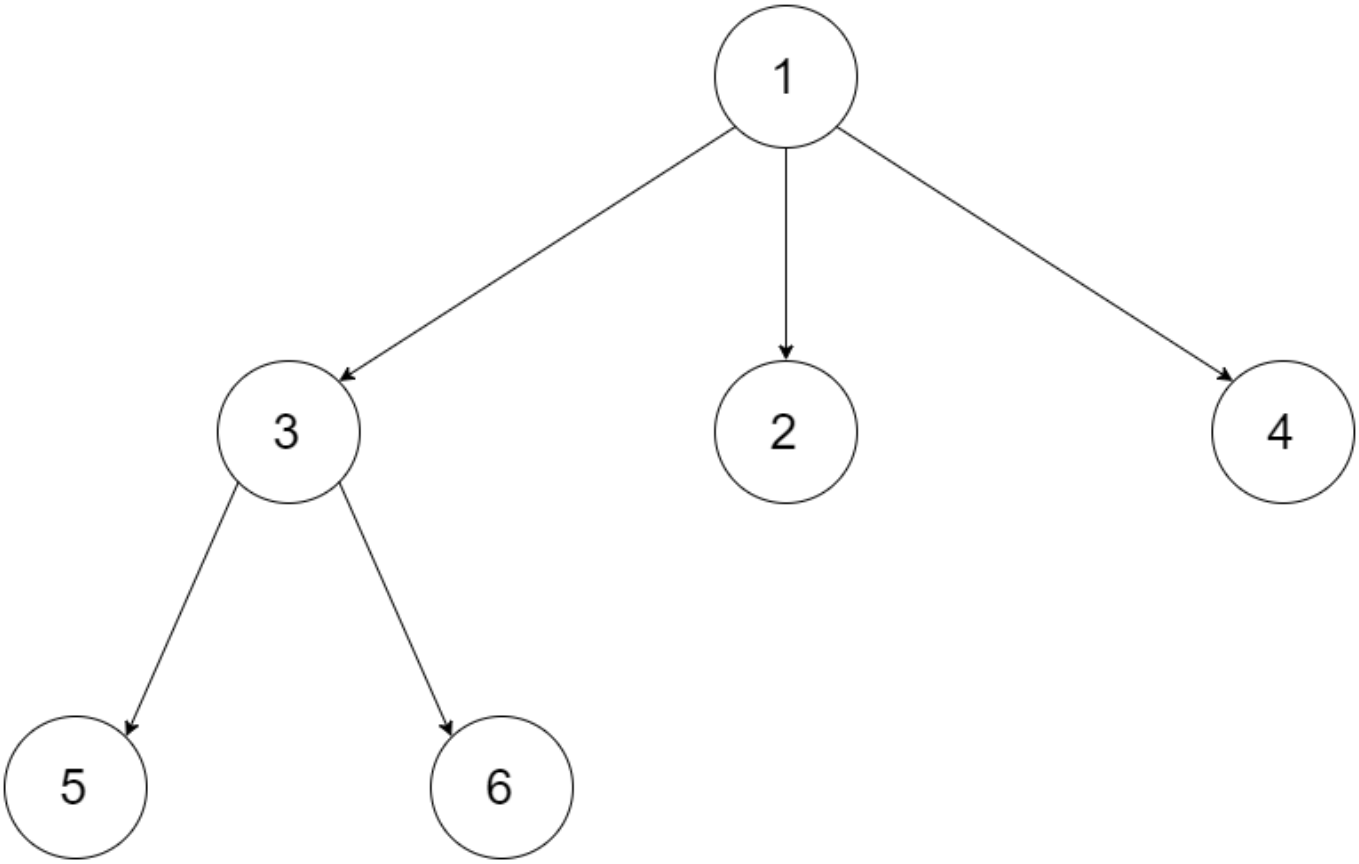
```
class Solution {
public:
    vector<double> averageOfLevels(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        vector<double> result;
        while (!que.empty()) {
            int size = que.size();
            double sum = 0; // 统计每一层的和
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                sum += node->val;
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
            result.push_back(sum / size); // 将每一层均值放进结果集
        }
        return result;
    }
};
```

N叉树的层序遍历

力扣题目链接

给定一个 N 叉树，返回其节点值的层序遍历。 (即从左到右，逐层遍历)。

例如，给定一个 3叉树 :



返回其层序遍历:

```
[
    [1],
    [3,2,4],
    [5,6]
]
```

思路

这道题依旧是模板题，只不过一个节点有多个孩子了

C++代码:

```
class Solution {
public:
    vector<vector<int>> levelOrder(Node* root) {
        queue<Node*> que;
        if (root != NULL) que.push(root);
        vector<vector<int>> result;
```

```
        while (!que.empty()) {
            int size = que.size();
            vector<int> vec;
            for (int i = 0; i < size; i++) {
                Node* node = que.front();
                que.pop();
                vec.push_back(node->val);
                for (int i = 0; i < node->children.size(); i++) { // 将节点孩子加入队列
                    if (node->children[i]) que.push(node->children[i]);
                }
            }
            result.push_back(vec);
        }
        return result;

    }
};
```

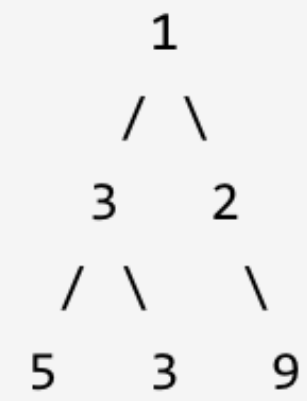
在每个树行中找最大值

[力扣题目链接]

您需要在二叉树的每一行中找到最大的值。

示例：

输入：



输出：[1, 3, 9]

思路

层序遍历，取每一层的最大值

C++代码：

```
class Solution {
public:
    vector<int> largestValues(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        vector<int> result;
        while (!que.empty()) {
            int size = que.size();
            int maxValue = INT_MIN; // 取每一层的最大值
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                maxValue = node->val > maxValue ? node->val : maxValue;
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
            result.push_back(maxValue); // 把最大值放进数组
        }
        return result;
    }
};
```

填充每个节点的下一个右侧节点指针

[力扣题目链接]

给定一个完美二叉树，其所有叶子节点都在同一层，每个父节点都有两个子节点。二叉树定义如下：

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

填充它的每个 next 指针，让这个指针指向其下一个右侧节点。如果找不到下一个右侧节点，则将 next 指针设置为 NULL。

初始状态下，所有 next 指针都被设置为 NULL。

示例：

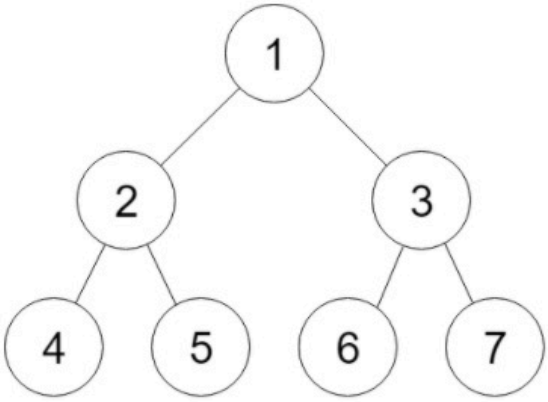


Figure A

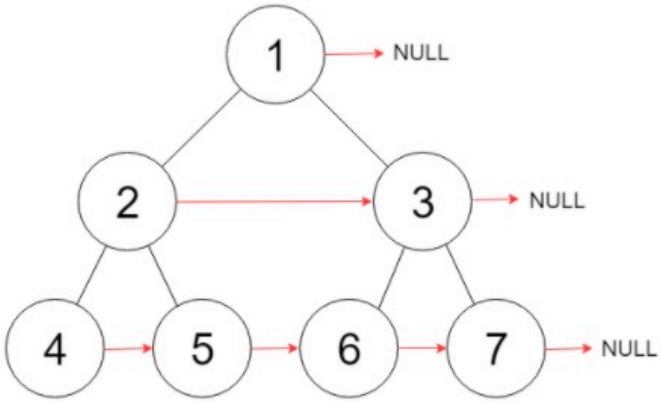


Figure B

输入： root = [1,2,3,4,5,6,7]
输出： [1,#,2,3,#,4,5,6,7,#]
解释： 给定二叉树如图 A 所示，你的函数应该填充它的每个 next 指针，以指向其下一个右侧节点，如图 B 所示。序列化的输出按层序遍历排列，同一层节点由 next 指针连接，'#' 标志着每一层的结束。

思路

本题依然是层序遍历，只不过在单层遍历的时候记录一下本层的头部节点，然后在遍历的时候让前一个节点指向本节点就可以了

C++代码：

```
class Solution {
public:
    Node* connect(Node* root) {
        queue<Node*> que;
        if (root != NULL) que.push(root);
        while (!que.empty()) {
            int size = que.size();
            // vector<int> vec;
            Node* nodePre;
            Node* node;
            for (int i = 0; i < size; i++) {
                if (i == 0) {
                    nodePre = que.front(); // 取出一层的头结点
                    que.pop();
                    node = nodePre;
                } else {
```

```
                    node = que.front();
                    que.pop();
                    nodePre->next = node; // 本层前一个节点next指向本节点
                    nodePre = nodePre->next;
                }
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
            nodePre->next = NULL; // 本层最后一个节点指向NULL
        }
        return root;

    }
};
```

填充每个节点的下一个右侧节点指针II

[力扣题目链接]

思路

这道题目说是二叉树，但116题目说是完整二叉树，其实没有任何差别，一样的代码一样的逻辑一样的味道

C++代码：

```
class Solution {
public:
    Node* connect(Node* root) {
        queue<Node*> que;
        if (root != NULL) que.push(root);
        while (!que.empty()) {
            int size = que.size();
            vector<int> vec;
            Node* nodePre;
            Node* node;
            for (int i = 0; i < size; i++) {
                if (i == 0) {
                    nodePre = que.front(); // 取出一层的头结点
                    que.pop();
                    node = nodePre;
                } else {
                    node = que.front();
                    que.pop();
                    nodePre->next = node; // 本层前一个节点next指向本节点
                    nodePre = nodePre->next;
                }
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
```

```
            nodePre->next = NULL; // 本层最后一个节点指向NULL
        }
        return root;
    }
};
```

二叉树的最大深度

[力扣题目链接]

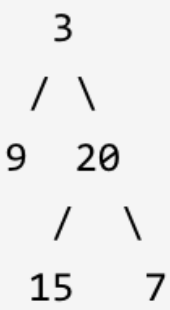
给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

说明: 叶子节点是指没有子节点的节点。

示例：

给定二叉树 [3,9,20,null,null,15,7],

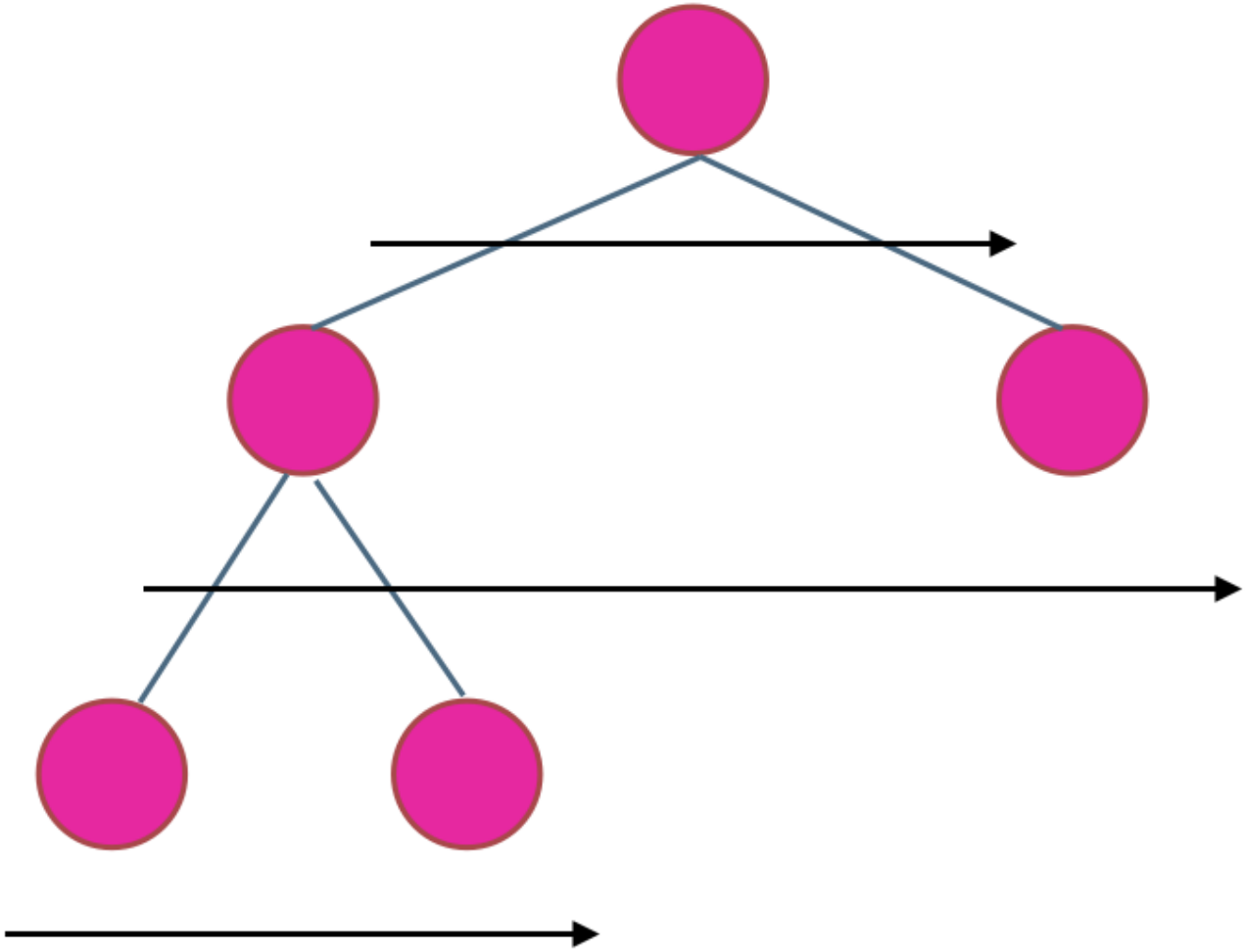


返回它的最大深度 3 。

思路

使用迭代法的话，使用层序遍历是最为合适的，因为最大的深度就是二叉树的层数，和层序遍历的方式极其吻合。

在二叉树中，一层一层的来遍历二叉树，记录一下遍历的层数就是二叉树的深度，如图所示：



所以这道题的迭代法就是一道模板题，可以使用二叉树层序遍历的模板来解决的。

C++代码如下：

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == NULL) return 0;
        int depth = 0;
        queue<TreeNode*> que;
        que.push(root);
        while(!que.empty()) {
            int size = que.size();
            depth++; // 记录深度
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return depth;
    }
```

```
};
```

二叉树的最小深度

力扣题目链接

思路

相对于 104.二叉树的最大深度 ，本题还也可以使用层序遍历的方式来解决，思路是一样的。

需要注意的是，只有当左右孩子都为空的时候，才说明遍历的最低点了。如果其中一个孩子为空则不是最低点

代码如下：（详细注释）

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == NULL) return 0;
        int depth = 0;
        queue<TreeNode*> que;
        que.push(root);
        while(!que.empty()) {
            int size = que.size();
            depth++; // 记录最小深度
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
                if (!node->left && !node->right) { // 当左右孩子都为空的时候，说明是最低点的一层了，退出
                    return depth;
                }
            }
        }
        return depth;
    }
};
```

总结

二叉树的层序遍历，**就是图论中的广度优先搜索在二叉树中的应用**，需要借助队列来实现（此时又发现队列的一个应用了）。

来吧，一口气打十个：

- 102.二叉树的层序遍历
- 107.二叉树的层次遍历II
- 199.二叉树的右视图

- 637.二叉树的层平均值
- 429.N叉树的层序遍历
- 515.在每个树行中找最大值
- 116.填充每个节点的下一个右侧节点指针
- 117.填充每个节点的下一个右侧节点指针II
- 104.二叉树的最大深度
- 111.二叉树的最小深度

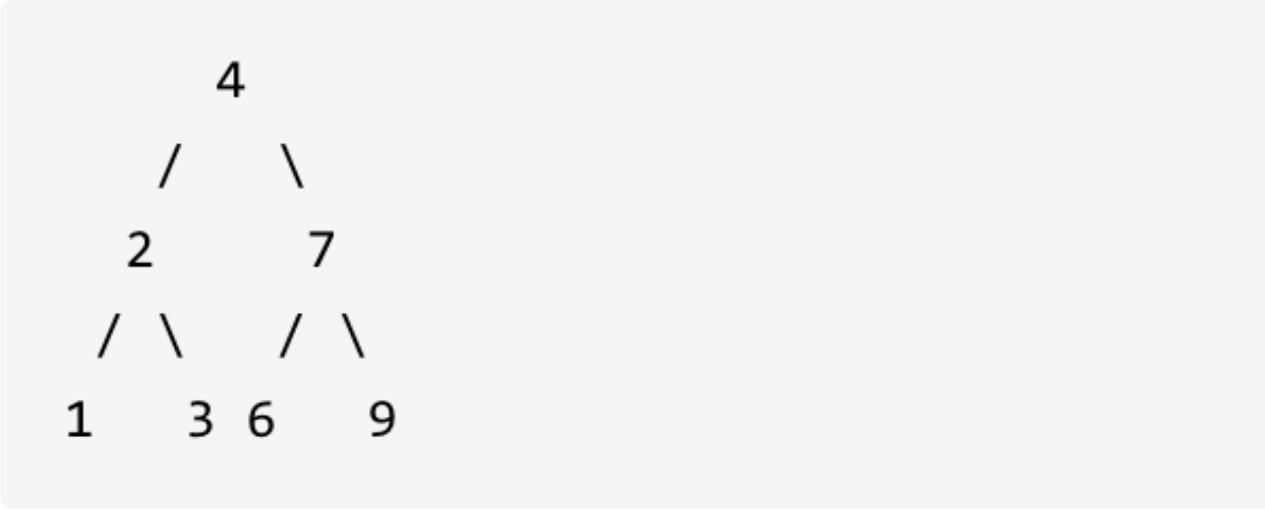
6.翻转二叉树

力扣题目链接

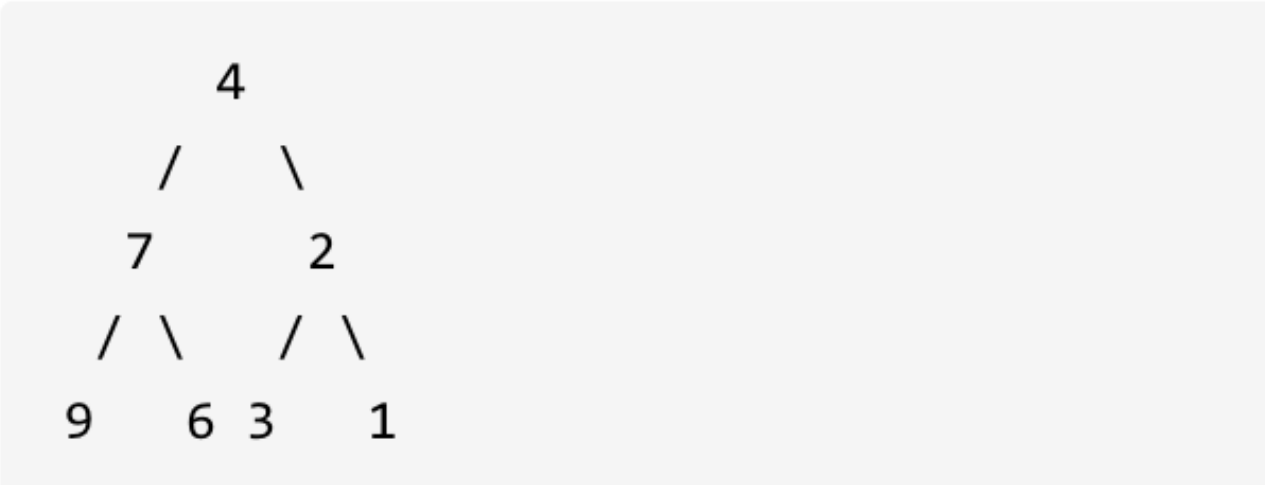
翻转一棵二叉树。

示例：

输入：



输出：



这道题目背后有一个让程序员心酸的故事，听说 Homebrew的作者Max Howell，就是因为没在白板上写出翻转二叉树，最后被Google拒绝了。（真假不做判断，权当一个乐子哈）

算法公开课

《代码随想录》算法视频公开课：听说一位巨佬面Google被拒了，因为没写出翻转二叉树 | LeetCode：226.翻转二叉树，相信结合视频再看本篇题解，更有助于大家对本题的理解。

题外话

这道题目是非常经典的题目，也是比较简单的题目（至少一看就会）。

但正是因为这道题太简单，一看就会，一些同学都没有抓住起本质，稀里糊涂的就把这道题目过了。

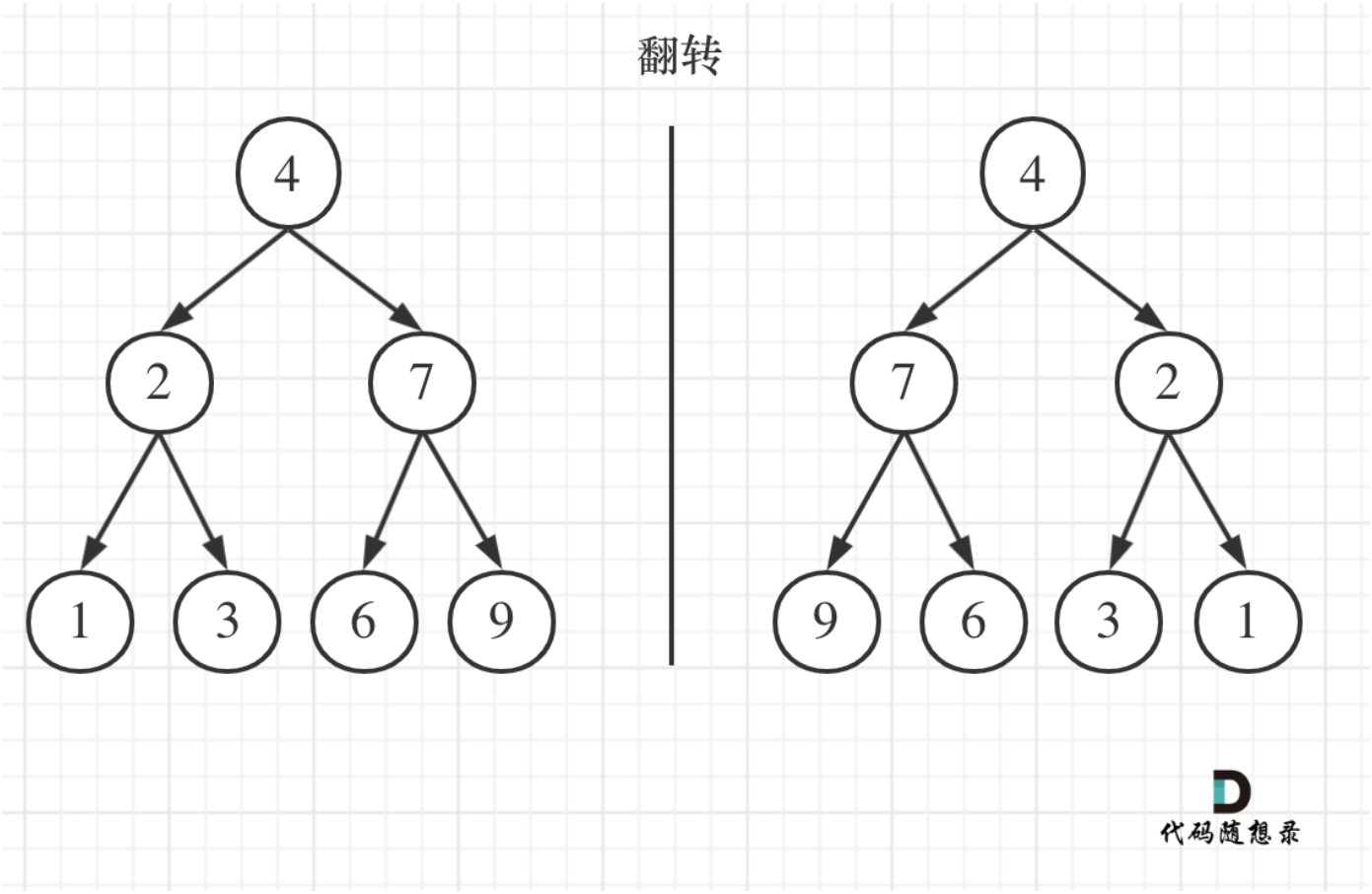
如果做过这道题的同学也建议认真看完，相信一定有所收获！

思路

我们之前介绍的都是各种方式遍历二叉树，这次要翻转了，感觉还是有点懵逼。

这得怎么翻转呢？

如果要从整个树来看，翻转还真的挺复杂，整个树以中间分割线进行翻转，如图：



可以发现想要翻转它，其实就把每一个节点的左右孩子交换一下就可以了。

关键在于遍历顺序，前中后序应该选哪一种遍历顺序？ （一些同学这道题都过了，但是不知道自己用的是什么顺序）

遍历的过程中去翻转每一个节点的左右孩子就可以达到整体翻转的效果。

注意只要把每一个节点的左右孩子翻转一下，就可以达到整体翻转的效果

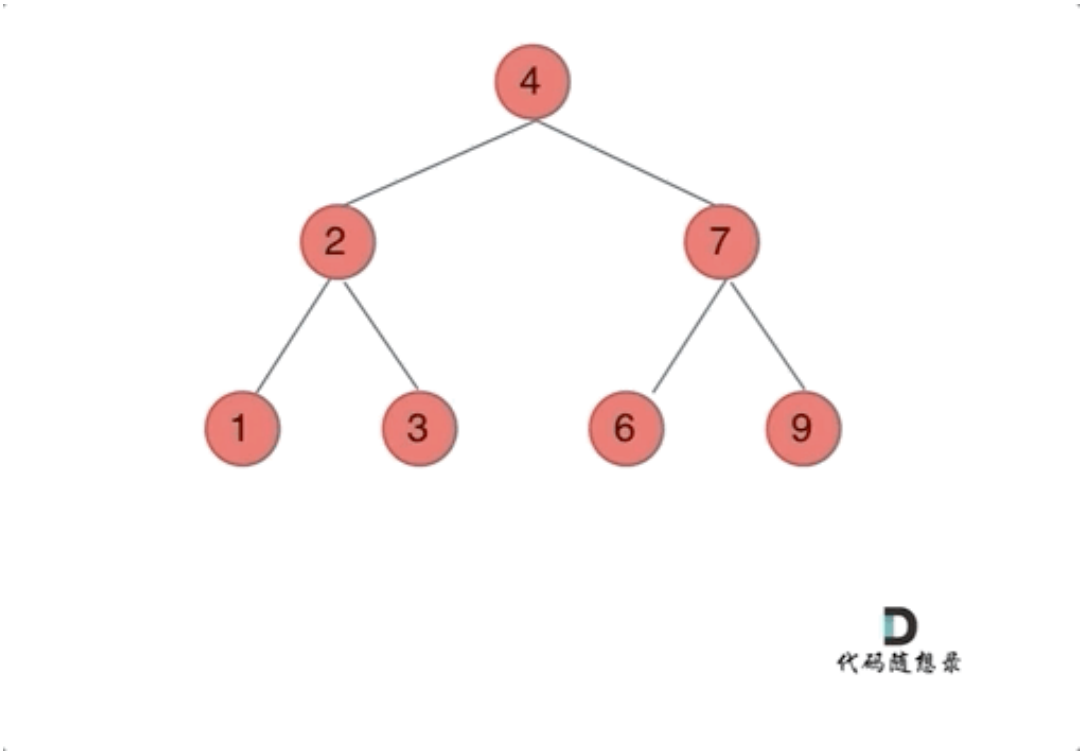
这道题目使用前序遍历和后序遍历都可以，唯独中序遍历不方便，因为中序遍历会把某些节点的左右孩子翻转了两次！建议拿纸画一画，就理解了

那么层序遍历可以不可以呢？**依然可以的！只要把每一个节点的左右孩子翻转一下的遍历方式都是可以的！**

递归法

对于二叉树的递归法的前中后序遍历，已经在二叉树：前中后序递归遍历详细讲解了。

我们下文以前序遍历为例，通过动画来看一下翻转的过程：



我们来看一下递归三部曲：

1. 确定递归函数的参数和返回值

参数就是要传入节点的指针，不需要其他参数了，通常此时定下来主要参数，如果在写递归的逻辑中发现还需要其他参数的时候，随时补充。

返回值的话其实也不需要，但是题目中给出的要返回root节点的指针，可以直接使用题目定义好的函数，所以就函数的返回类型为`TreeNode*`。

```
TreeNode* invertTree(TreeNode* root)
```

2. 确定终止条件

当前节点为空的时候，就返回

```
if (root == NULL) return root;
```

3. 确定单层递归的逻辑

因为是先前序遍历，所以先进行交换左右孩子节点，然后反转左子树，反转右子树。

```
swap(root->left, root->right);
invertTree(root->left);
invertTree(root->right);
```

基于这递归三步法，代码基本写完，C++代码如下：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (root == NULL) return root;
        swap(root->left, root->right);  // 中
        invertTree(root->left);         // 左
        invertTree(root->right);        // 右
        return root;
    }
};
```

迭代法

深度优先遍历

二叉树：听说递归能做的，栈也能做！中给出了前中后序迭代方式的写法，所以本题可以很轻松的写出如下迭代法的代码：

C++代码迭代法（前序遍历）

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (root == NULL) return root;
        stack<TreeNode*> st;
        st.push(root);
        while(!st.empty()) {
            TreeNode* node = st.top();              // 中
            st.pop();
            swap(node->left, node->right);
            if(node->right) st.push(node->right);   // 右
            if(node->left) st.push(node->left);     // 左
        }
        return root;
    }
};
```

如果这个代码看不懂的话可以再回顾一下二叉树：听说递归能做的，栈也能做！。

我们在二叉树：前中后序迭代方式的统一写法中介绍了统一的写法，所以，本题也只需将文中的代码少做修改便可。

C++代码如下迭代法（前序遍历）

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
```

```
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                if (node->right) st.push(node->right);  // 右
                if (node->left) st.push(node->left);    // 左
                st.push(node);                          // 中
                st.push(NULL);
            } else {
                st.pop();
                node = st.top();
                st.pop();
                swap(node->left, node->right);          // 节点处理逻辑
            }
        }
        return root;
    }
};
```

如果上面这个代码看不懂，回顾一下文章二叉树：前中后序迭代方式的统一写法。

广度优先遍历

也就是层序遍历，层数遍历也是可以翻转这棵树的，因为层序遍历也可以把每个节点的左右孩子都翻转一遍，代码如下：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        while (!que.empty()) {
            int size = que.size();
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                swap(node->left, node->right); // 节点处理
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return root;
    }
};
```

如果对以上代码不理解，或者不清楚二叉树的层序遍历，可以看这篇二叉树：层序遍历登场！

拓展

文中我指的是递归的中序遍历是不行的，因为使用递归的中序遍历，某些节点的左右孩子会翻转两次。

如果非要使用递归中序的方式写，也可以，如下代码就可以避免节点左右孩子翻转两次的情况：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (root == NULL) return root;
        invertTree(root->left);         // 左
        swap(root->left, root->right);  // 中
        invertTree(root->left);         // 注意 这里依然要遍历左孩子，因为中间节点已经翻转了
        return root;
    }
};
```

代码虽然可以，但这毕竟不是真正的递归中序遍历了。

但使用迭代方式统一写法的中序是可以的。

代码如下：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                if (node->right) st.push(node->right);  // 右
                st.push(node);                          // 中
                st.push(NULL);
                if (node->left) st.push(node->left);    // 左

            } else {
                st.pop();
                node = st.top();
                st.pop();
                swap(node->left, node->right);          // 节点处理逻辑
            }
        }
        return root;
    }
};
```

为什么这个中序就是可以的呢，因为这是用栈来遍历，而不是靠指针来遍历，避免了递归法中翻转了两次的情况，大家可以画图理解一下，这里有点意思的。

总结

针对二叉树的问题，解题之前一定要想清楚究竟是前中后序遍历，还是层序遍历。

二叉树解题的大忌就是自己稀里糊涂的过了（因为这道题相对简单），但是也不知道自己是怎么遍历的。

这也是造成了二叉树的题目“一看就会，一写就废”的原因。

针对翻转二叉树，我给出了一种递归，三种迭代（两种模拟深度优先遍历，一种层序遍历）的写法，都是之前我们讲过的写法，融汇贯通一下而已。

大家一定也有自己的解法，但一定要成方法论，这样才能通用，才能举一反三！

---

7. 本周小结！（二叉树系列一）

周日我做一个针对本周的打卡留言疑问以及在刷题群里的讨论内容做一下梳理吧。，这样也有助于大家补一补本周的内容，消化消化。

注意这个周末总结和系列总结还是不一样的（二叉树还远没有结束），这个总结是针对留言疑问以及刷题群里讨论内容的归纳。

周一

本周我们开始讲解了二叉树，在关于二叉树，你该了解这些！中讲解了二叉树的理论基础。

有同学会把红黑树和二叉平衡搜索树弄分开了，其实红黑树就是一种二叉平衡搜索树，这两个树不是独立的，所以C++中map、multimap、set、multiset的底层实现机制是二叉平衡搜索树，再具体一点是红黑树。

对于二叉树节点的定义，C++代码如下：

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
```

对于这个定义中`TreeNode(int x) : val(x), left(NULL), right(NULL) {}`有同学不清楚干什么的。

这是构造函数，这么说吧C语言中的结构体是C++中类的祖先，所以C++结构体也可以有构造函数。

构造函数也可以不写，但是new一个新的节点的时候就比较麻烦。

例如有构造函数，定义初始值为9的节点：

```
TreeNode* a = new TreeNode(9);
```

没有构造函数的话就要这么写：

```
TreeNode* a = new TreeNode();
a->val = 9;
a->left = NULL;
a->right = NULL;
```

在介绍前中后序遍历的时候，有递归和迭代（非递归），还有一种牛逼的遍历方式：morris遍历。

morris遍历是二叉树遍历算法的超强进阶算法，morris遍历可以将非递归遍历中的空间复杂度降为O(1)，感兴趣大家就去查一查学习学习，比较小众，面试几乎不会考。我其实也没有研究过，就不做过多介绍了。

周二

在二叉树：一入递归深似海，从此offer是路人中讲到了递归三要素，以及前中后序的递归写法。

文章中我给出了leetcode上三道二叉树的前中后序题目，但是看完二叉树：一入递归深似海，从此offer是路人，依然可以解决n叉树的前后序遍历，在leetcode上分别是

- 589. N叉树的前序遍历
- 590. N叉树的后序遍历

大家可以再去把这两道题目做了。

周三

在二叉树：听说递归能做的，栈也能做！中我们开始用栈来实现递归的写法，也就是所谓的迭代法。

细心的同学发现文中前后序遍历空节点是否入栈写法是不同的

其实空节点入不入栈都差不多，但感觉空节点不入栈确实清晰一些，符合文中动画的演示。

拿前序遍历来举例，空节点入栈：

```
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        stack<TreeNode*> st;
        vector<int> result;
        st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();                       // 中
            st.pop();
            if (node != NULL) result.push_back(node->val);
            else continue;
            st.push(node->right);                            // 右
            st.push(node->left);                             // 左
        }
```

```
        return result;
    }
};
```

前序遍历空节点不入栈的代码：（注意注释部分和上文的区别）

```
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        stack<TreeNode*> st;
        vector<int> result;
        if (root == NULL) return result;
        st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();                       // 中
            st.pop();
            result.push_back(node->val);
            if (node->right) st.push(node->right);           // 右（空节点不入栈）
            if (node->left) st.push(node->left);             // 左（空节点不入栈）
        }
        return result;
    }
};
```

在实现迭代法的过程中，有同学问了：递归与迭代究竟谁优谁劣呢?

从时间复杂度上其实迭代法和递归法差不多（在不考虑函数调用开销和函数调用产生的堆栈开销），但是空间复杂度上，递归开销会大一些，因为递归需要系统堆栈存参数返回值等等。

递归更容易让程序员理解，但收敛不好，容易栈溢出。

这么说吧，递归是方便了程序员，难为了机器（各种保存参数，各种进栈出栈）。

在实际项目开发的过程中我们是要尽量避免递归！因为项目代码参数、调用关系都比较复杂，不容易控制递归深度，甚至会栈溢出。

周四

在二叉树：前中后序迭代方式的写法就不能统一一下么？中我们使用空节点作为标记，给出了统一的前中后序迭代法。

此时又多了一种前中后序的迭代写法，那么有同学问了：前中后序迭代法是不是一定要统一来写，这样才算是规范。

其实没必要，还是自己感觉哪一种更好记就用哪种。

但是**一定要掌握前中后序一种迭代的写法，并不因为某种场景的题目一定要用迭代，而是现场面试的时候，面试官看你顺畅的写出了递归，一般会进一步考察能不能写出相应的迭代。**

周五

在二叉树：层序遍历登场！中我们介绍了二叉树的另一种遍历方式（图论中广度优先搜索在二叉树上的应用）即：层序遍历。

看完这篇文章，去leetcode上怒刷五题，文章中 编号107题目的样例图放错了（原谅我匆忙之间总是手抖），但不影响大家理解。

只有同学发现leetcode上“515. 在每个树行中找最大值”，也是层序遍历的应用，依然可以分分钟解决，所以就是一鼓作气解决六道了，哈哈。

层序遍历遍历相对容易一些，只要掌握基本写法（也就是框架模板），剩下的就是在二叉树每一行遍历的时候做做逻辑修改。

周六

在二叉树：你真的会翻转二叉树么？中我们把翻转二叉树这么一道简单又经典的问题，充分的剖析了一波，相信就算做过这道题目的同学，看完本篇之后依然有所收获！

文中我指的是递归的中序遍历是不行的，因为使用递归的中序遍历，某些节点的左右孩子会翻转两次。

如果非要使用递归中序的方式写，也可以，如下代码就可以避免节点左右孩子翻转两次的情况：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (root == NULL) return root;
        invertTree(root->left);         // 左
        swap(root->left, root->right);  // 中
        invertTree(root->left);         // 注意 这里依然要遍历左孩子，因为中间节点已经翻转了
        return root;
    }
};
```

代码虽然可以，但这毕竟不是真正的递归中序遍历了。

但使用迭代方式统一写法的中序是可以的。

代码如下：

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        stack<TreeNode*> st;
        if (root != NULL) st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                if (node->right) st.push(node->right);  // 右
                st.push(node);                          // 中
                st.push(NULL);
                if (node->left) st.push(node->left);    // 左
```

```
            } else {
                st.pop();
                node = st.top();
                st.pop();
                swap(node->left, node->right);          // 节点处理逻辑
            }
        }
        return root;
    }
};
```

为什么这个中序就是可以的呢，因为这是用栈来遍历，而不是靠指针来遍历，避免了递归法中翻转了两次的情况，大家可以画图理解一下，这里有点意思的。

总结

本周我们都是讲解了二叉树，从理论基础到遍历方式，从递归到迭代，从深度遍历到广度遍历，最后再用了一个翻转二叉树的题目把我们之前讲过的遍历方式都串了起来。

---

8. 对称二叉树

力扣题目链接

给定一个二叉树，检查它是否是镜像对称的。

例如，二叉树 `[1,2,2,3,4,4,3]` 是对称的。

```
    1
   / \
  2   2
 / \ / \
3  4 4  3
```

但是下面这个 `[1,2,2,null,3,null,3]` 则不是镜像对称的:

```
    1
   / \
  2   2
   \   \
   3    3
```

算法公开课

[《代码随想录》算法视频公开课](#)：[同时操作两个二叉树 | LeetCode：101. 对称二叉树](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

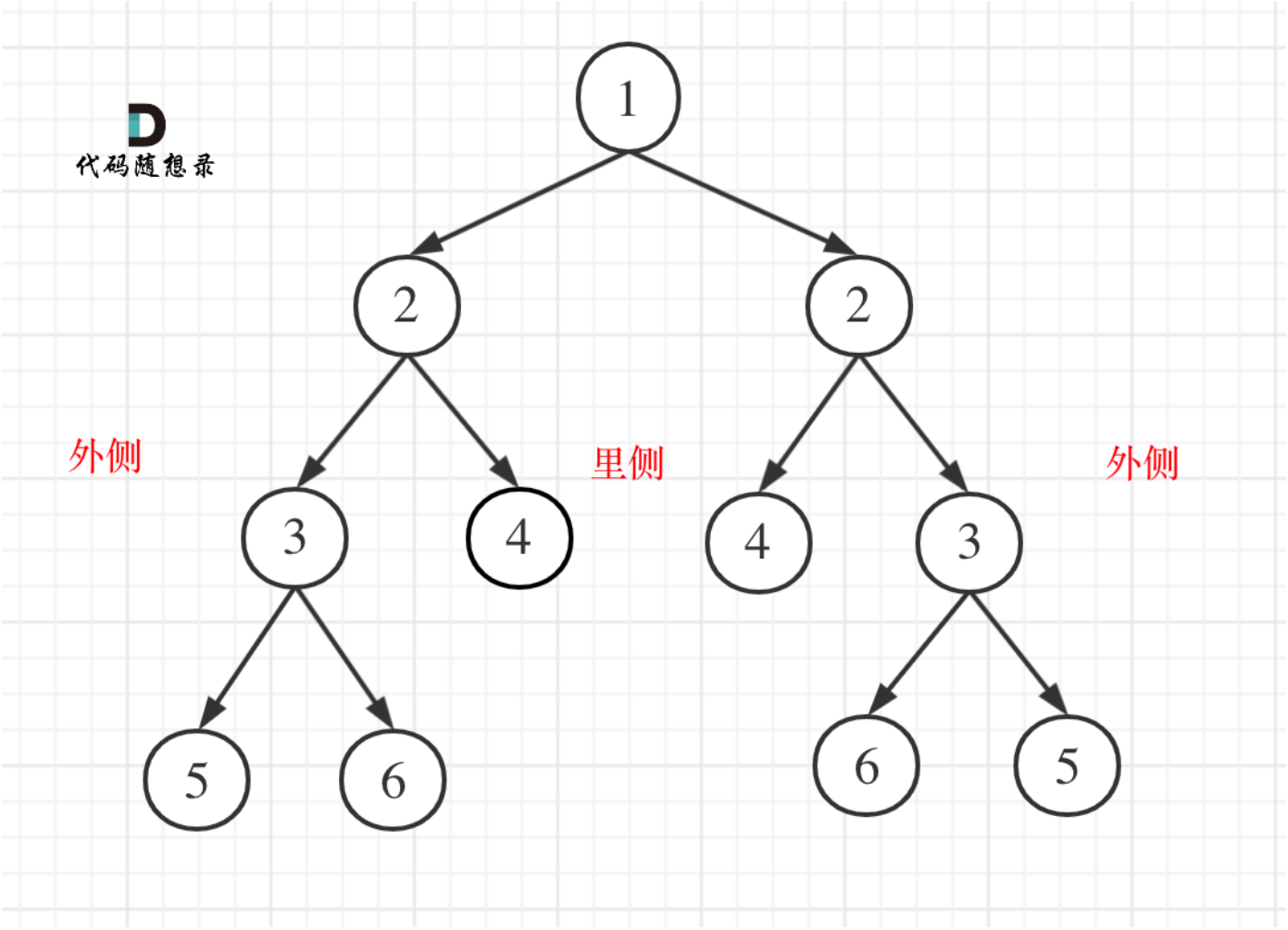
思路

首先想清楚，判断对称二叉树要比较的是哪两个节点，要比较的可不是左右节点！

对于二叉树是否对称，要比较的是根节点的左子树与右子树是不是相互翻转的，理解这一点就知道了**其实我们要比较的是两个树（这两个树是根节点的左右子树）**，所以在递归遍历的过程中，也是要同时遍历两棵树。

那么如何比较呢？

比较的是两个子树的里侧和外侧的元素是否相等。如图所示：



那么遍历的顺序应该是什么样的呢?

本题遍历只能是“后序遍历”，因为我们要通过递归函数的返回值来判断两个子树的内侧节点和外侧节点是否相等。

正是因为要遍历两棵树而且要比较内侧和外侧节点，所以准确的来说是一个树的遍历顺序是左右中，一个树的遍历顺序是右左中。

但都可以理解算是后序遍历，尽管已经不是严格上在一个树上进行遍历的后序遍历了。

其实后序也可以理解为是一种回溯，当然这是题外话，讲回溯的时候会重点讲的。

说到这大家可能感觉我有点啰嗦，哪有这么多道理，上来就干就完事了。别急，我说的这些在下面的代码讲解中都有身影。

那么我们先来看看递归法的代码应该怎么写。

递归法

递归三部曲

1. 确定递归函数的参数和返回值

因为我们要比较的是根节点的两个子树是否是相互翻转的，进而判断这个树是不是对称树，所以要比较的是两个树，参数自然也是左子树节点和右子树节点。

返回值自然是bool类型。

代码如下：

```
bool compare(TreeNode* left, TreeNode* right)
```

2. 确定终止条件

要比较两个节点数值相不相同，首先要把两个节点为空的情况弄清楚！否则后面比较数值的时候就会操作空指针了。

节点为空的情况有：（**注意我们比较的其实不是左孩子和右孩子，所以如下我称之为左节点右节点**）

- 左节点为空，右节点不为空，不对称，return false
- 左不为空，右为空，不对称 return false
- 左右都为空，对称，返回true

此时已经排除掉了节点为空的情况，那么剩下的就是左右节点不为空：

- 左右都不为空，比较节点数值，不相同就return false

此时左右节点不为空，且数值也不相同的情况我们也处理了。

代码如下：

```
if (left == NULL && right != NULL) return false;
else if (left != NULL && right == NULL) return false;
else if (left == NULL && right == NULL) return true;
else if (left->val != right->val) return false; // 注意这里我没有使用else
```

注意上面最后一种情况，我没有使用else，而是else if， 因为我们把以上情况都排除之后，剩下的就是 左右节点都不为空，且数值相同的情况。

3. 确定单层递归的逻辑

此时才进入单层递归的逻辑，单层递归的逻辑就是处理 左右节点都不为空，且数值相同的情况。

- 比较二叉树外侧是否对称：传入的是左节点的左孩子，右节点的右孩子。
- 比较内侧是否对称，传入左节点的右孩子，右节点的左孩子。
- 如果左右都对称就返回true ，有一侧不对称就返回false 。

代码如下：

```
bool outside = compare(left->left, right->right);   // 左子树：左、 右子树：右
bool inside = compare(left->right, right->left);    // 左子树：右、 右子树：左
bool isSame = outside && inside;                    // 左子树：中、 右子树：中（逻辑处理）
return isSame;
```

如上代码中，我们可以看出使用的遍历方式，左子树左右中，右子树右左中，所以我把这个遍历顺序也称之为“后序遍历”（尽管不是严格的后序遍历）。

最后递归的C++整体代码如下：

```
class Solution {
```

```
public:
    bool compare(TreeNode* left, TreeNode* right) {
        // 首先排除空节点的情况
        if (left == NULL && right != NULL) return false;
        else if (left != NULL && right == NULL) return false;
        else if (left == NULL && right == NULL) return true;
        // 排除了空节点，再排除数值不相同的情况
        else if (left->val != right->val) return false;

        // 此时就是：左右节点都不为空，且数值相同的情况
        // 此时才做递归，做下一层的判断
        bool outside = compare(left->left, right->right);   // 左子树：左、 右子树：右
        bool inside = compare(left->right, right->left);    // 左子树：右、 右子树：左
        bool isSame = outside && inside;                    // 左子树：中、 右子树：中 （逻辑处理）
        return isSame;

    }
    bool isSymmetric(TreeNode* root) {
        if (root == NULL) return true;
        return compare(root->left, root->right);
    }
};
```

我给出的代码并不简洁，但是把每一步判断的逻辑都清楚的描绘出来了。

如果上来就看网上各种简洁的代码，看起来真的很简单，但是很多逻辑都掩盖掉了，而题解可能也没有把掩盖掉的逻辑说清楚。

盲目的照着抄，结果就是：发现这是一道“简单题”，稀里糊涂的就过了，但是真正的每一步判断逻辑未必想到清楚。

当然我可以把如上代码整理如下：

```
class Solution {
public:
    bool compare(TreeNode* left, TreeNode* right) {
        if (left == NULL && right != NULL) return false;
        else if (left != NULL && right == NULL) return false;
        else if (left == NULL && right == NULL) return true;
        else if (left->val != right->val) return false;
        else return compare(left->left, right->right) && compare(left->right, right->left);

    }
    bool isSymmetric(TreeNode* root) {
        if (root == NULL) return true;
        return compare(root->left, root->right);
    }
};
```

这个代码就很简洁了，但隐藏了很多逻辑，条理不清晰，而且递归三部曲，在这里完全体现不出来。

所以建议大家做题的时候，一定要想清楚逻辑，每一步做什么。把题目所有情况想到位，相应的代码写出来之后，再去追求简洁代码的效果。

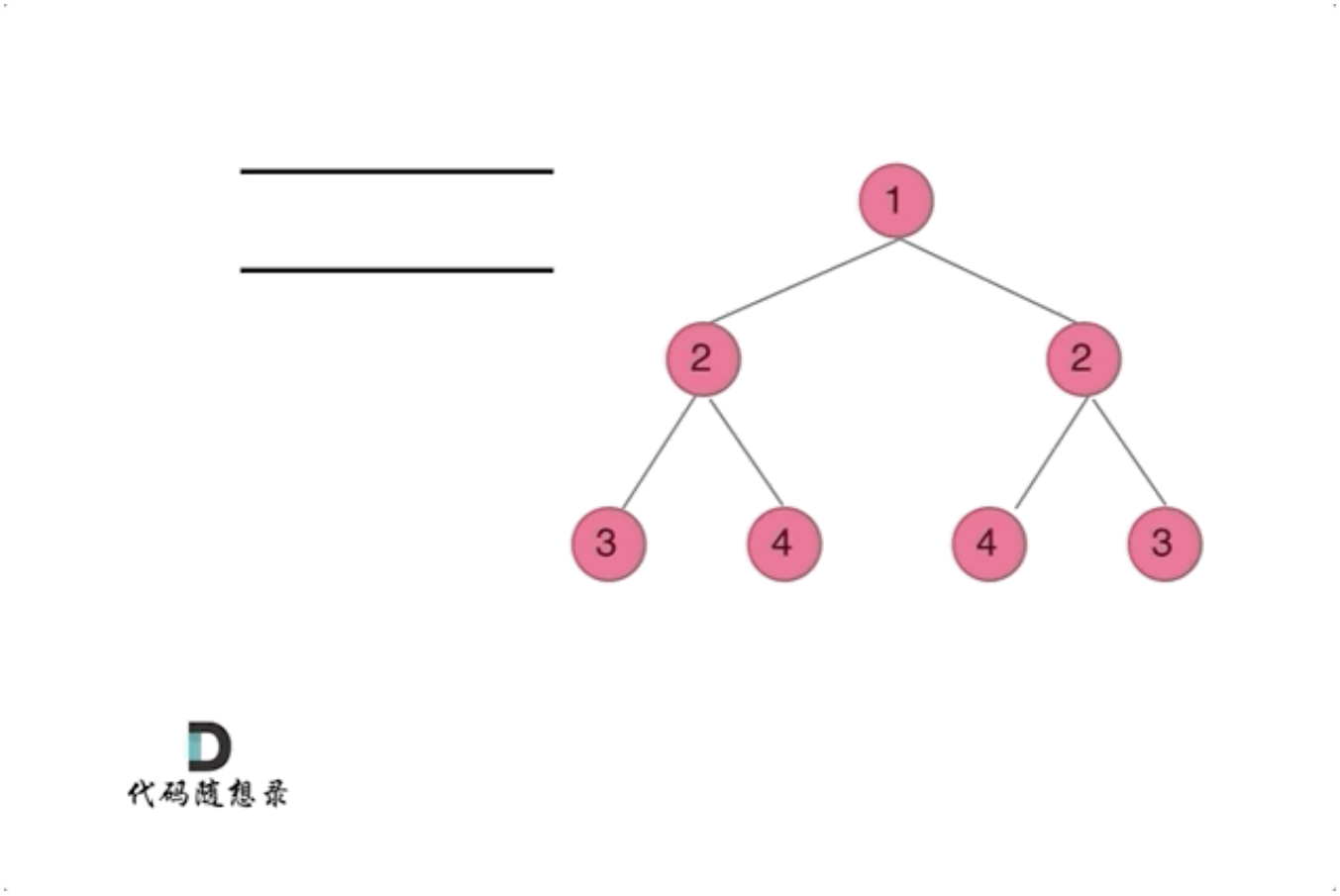
迭代法

这道题目我们也可以使用迭代法，但要注意，这里的迭代法可不是前中后序的迭代写法，因为本题的本质是判断两个树是否是相互翻转的，其实已经不是所谓二叉树遍历的前中后序的关系了。

这里我们可以使用队列来比较两个树（根节点的左右子树）是否相互翻转，（**注意这不是层序遍历**）

使用队列

通过队列来判断根节点的左子树和右子树的内侧和外侧是否相等，如动画所示：



如下的条件判断和递归的逻辑是一样的。

代码如下：

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if (root == NULL) return true;
        queue<TreeNode*> que;
        que.push(root->left);   // 将左子树头结点加入队列
        que.push(root->right);  // 将右子树头结点加入队列
```

```
        while (!que.empty()) {  // 接下来就要判断这两个树是否相互翻转
            TreeNode* leftNode = que.front(); que.pop();
            TreeNode* rightNode = que.front(); que.pop();
            if (!leftNode && !rightNode) {  // 左节点为空、右节点为空，此时说明是对称的
                continue;
            }

            // 左右一个节点不为空，或者都不为空但数值不相同，返回false
            if ((!leftNode || !rightNode || (leftNode->val != rightNode->val))) {
                return false;
            }
            que.push(leftNode->left);   // 加入左节点左孩子
            que.push(rightNode->right); // 加入右节点右孩子
            que.push(leftNode->right);  // 加入左节点右孩子
            que.push(rightNode->left);  // 加入右节点左孩子
        }
        return true;
    }
};
```

使用栈

细心的话，其实可以发现，这个迭代法，其实是把左右两个子树要比较的元素顺序放进一个容器，然后成对成对的取出来进行比较，那么其实使用栈也是可以的。

只要把队列原封不动的改成栈就可以了，我下面也给出了代码。

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if (root == NULL) return true;
        stack<TreeNode*> st; // 这里改成了栈
        st.push(root->left);
        st.push(root->right);
        while (!st.empty()) {
            TreeNode* leftNode = st.top(); st.pop();
            TreeNode* rightNode = st.top(); st.pop();
            if (!leftNode && !rightNode) {
                continue;
            }
            if ((!leftNode || !rightNode || (leftNode->val != rightNode->val))) {
                return false;
            }
            st.push(leftNode->left);
            st.push(rightNode->right);
            st.push(leftNode->right);
            st.push(rightNode->left);
        }
```

```
            return true;
        }
    };
```

总结

这次我们又深度剖析了一道二叉树的“简单题”，大家会发现，真正的把题目搞清楚其实并不简单，leetcode上accept了和真正掌握了还是有距离的。

我们介绍了递归法和迭代法，递归依然通过递归三部曲来解决了这道题目，如果只看精简的代码根本看不出来递归三部曲是如何解题的。

在迭代法中我们使用了队列，需要注意的是这不是层序遍历，而且仅仅通过一个容器来成对的存放我们要比较的元素，知道这一本质之后就发现，用队列，用栈，甚至用数组，都是可以的。

如果已经做过这道题目的同学，读完文章可以再去看看这道题目，思考一下，会有不一样的发现！

相关题目推荐

这两道题目基本和本题是一样的，只要稍加修改就可以AC。

- 100.相同的树
- 572.另一个树的子树

9.二叉树的最大深度

力扣题目链接

给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

说明: 叶子节点是指没有子节点的节点。

示例：
给定二叉树 [3,9,20,null,null,15,7],

```
    3
   / \
  9  20
    /  \
   15   7
```

返回它的最大深度 3 。

算法公开课

[《代码随想录》算法视频公开课](): [二叉树的高度和深度有啥区别？究竟用什么遍历顺序？很多录友搞不懂 | 104.二叉树的最大深度](), 相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

看完本篇可以一起做了如下两道题目：

- 104.二叉树的最大深度
- 559.n叉树的最大深度

递归法

本题可以使用前序（中左右），也可以使用后序遍历（左右中），使用前序求的就是深度，使用后序求的是高度。

- 二叉树节点的深度：指从根节点到该节点的最长简单路径边的条数或者节点数（取决于深度从0开始还是从1开始）
- 二叉树节点的高度：指从该节点到叶子节点的最长简单路径边的条数或者节点数（取决于高度从0开始还是从1开始）

而根节点的高度就是二叉树的最大深度，所以本题中我们通过后序求的根节点高度来求的二叉树最大深度。

这一点其实是很多同学没有想清楚的，很多题解同样没有讲清楚。

我先用后序遍历（左右中）来计算树的高度。

1. 确定递归函数的参数和返回值：参数就是传入树的根节点，返回就返回这棵树的深度，所以返回值为int类型。

代码如下：

```
int getdepth(TreeNode* node)
```

2. 确定终止条件：如果为空节点的话，就返回0，表示高度为0。

代码如下：

```
if (node == NULL) return 0;
```

3. 确定单层递归的逻辑：先求它的左子树的深度，再求右子树的深度，最后取左右深度最大的数值 再+1 （加1是因为算上当前中间节点）就是目前节点为根节点的树的深度。

代码如下：

```
int leftdepth = getdepth(node->left);       // 左
int rightdepth = getdepth(node->right);     // 右
int depth = 1 + max(leftdepth, rightdepth); // 中
return depth;
```

所以整体c++代码如下：

```
class solution {
public:
    int getdepth(TreeNode* node) {
        if (node == NULL) return 0;
        int leftdepth = getdepth(node->left);       // 左
        int rightdepth = getdepth(node->right);     // 右
        int depth = 1 + max(leftdepth, rightdepth); // 中
        return depth;
    }
    int maxDepth(TreeNode* root) {
        return getdepth(root);
    }
};
```

代码精简之后c++代码如下：

```
class solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == null) return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

精简之后的代码根本看不出是哪种遍历方式，也看不出递归三部曲的步骤，所以如果对二叉树的操作还不熟练，尽量不要直接照着精简代码来学。

本题当然也可以使用前序，代码如下：(**充分表现出求深度回溯的过程**)

```
class solution {
public:
    int result;
    void getdepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中

        if (node->left == NULL && node->right == NULL) return ;

        if (node->left) { // 左
            depth++;    // 深度+1
            getdepth(node->left, depth);
```

```
            depth--;     // 回溯，深度-1
        }
        if (node->right) { // 右
            depth++;     // 深度+1
            getdepth(node->right, depth);
            depth--;     // 回溯，深度-1
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
        if (root == NULL) return result;
        getdepth(root, 1);
        return result;
    }
};
```

可以看出使用了前序（中左右）的遍历顺序，这才是真正求深度的逻辑！

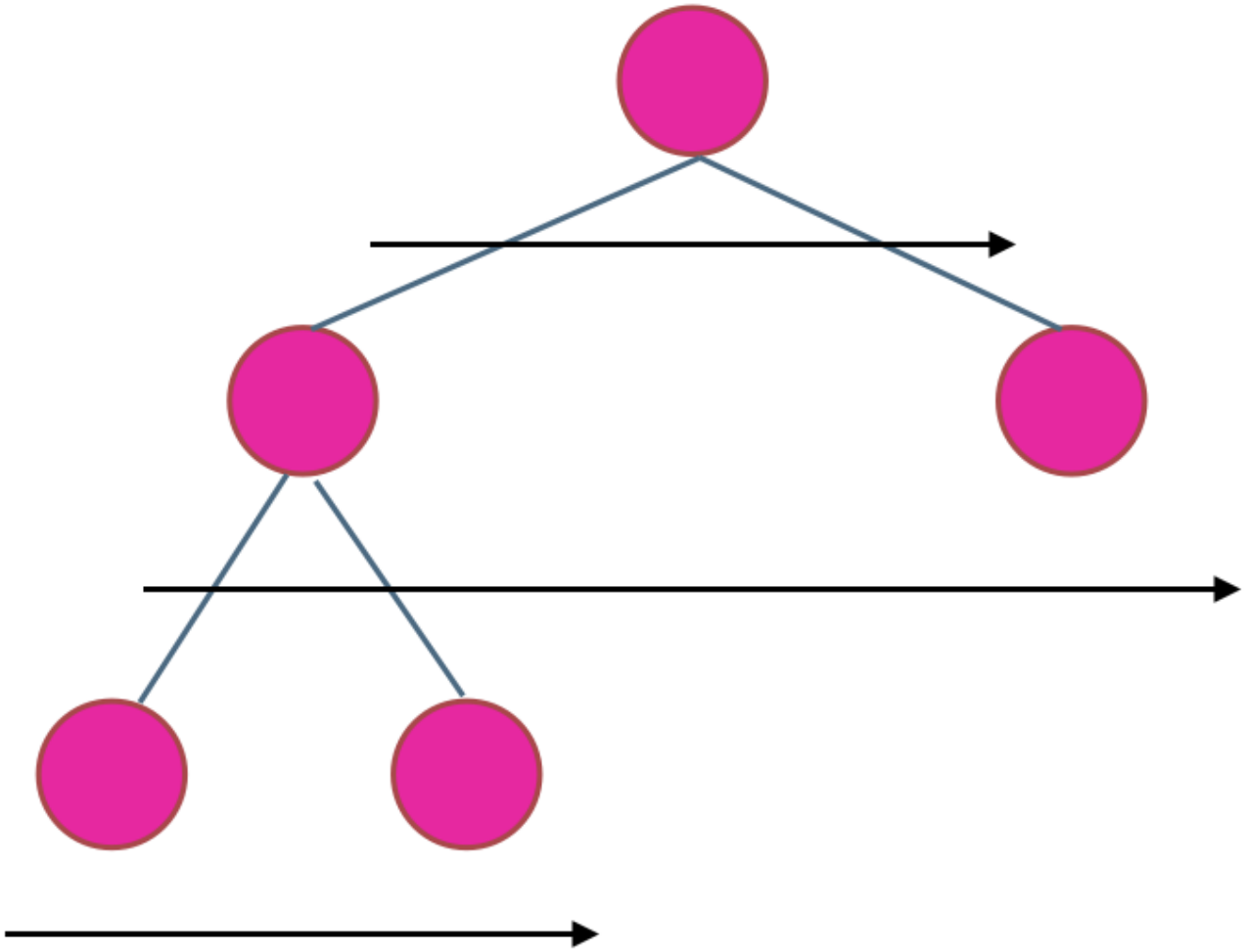
注意以上代码是为了把细节体现出来，简化一下代码如下：

```
class solution {
public:
    int result;
    void getdepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中
        if (node->left == NULL && node->right == NULL) return ;
        if (node->left) { // 左
            getdepth(node->left, depth + 1);
        }
        if (node->right) { // 右
            getdepth(node->right, depth + 1);
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
        if (root == 0) return result;
        getdepth(root, 1);
        return result;
    }
};
```

迭代法

使用迭代法的话，使用层序遍历是最为合适的，因为最大的深度就是二叉树的层数，和层序遍历的方式极其吻合。

在二叉树中，一层一层的来遍历二叉树，记录一下遍历的层数就是二叉树的深度，如图所示：



所以这道题的迭代法就是一道模板题，可以使用二叉树层序遍历的模板来解决的。

如果对层序遍历还不清楚的话，可以看这篇：二叉树：层序遍历登场！

c++代码如下：

```
class solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == NULL) return 0;
        int depth = 0;
        queue<TreeNode*> que;
        que.push(root);
        while(!que.empty()) {
            int size = que.size();
            depth++; // 记录深度
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
```

```
                que.pop();
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return depth;
    }
};
```

那么我们可以顺便解决一下n叉树的最大深度问题

相关题目推荐

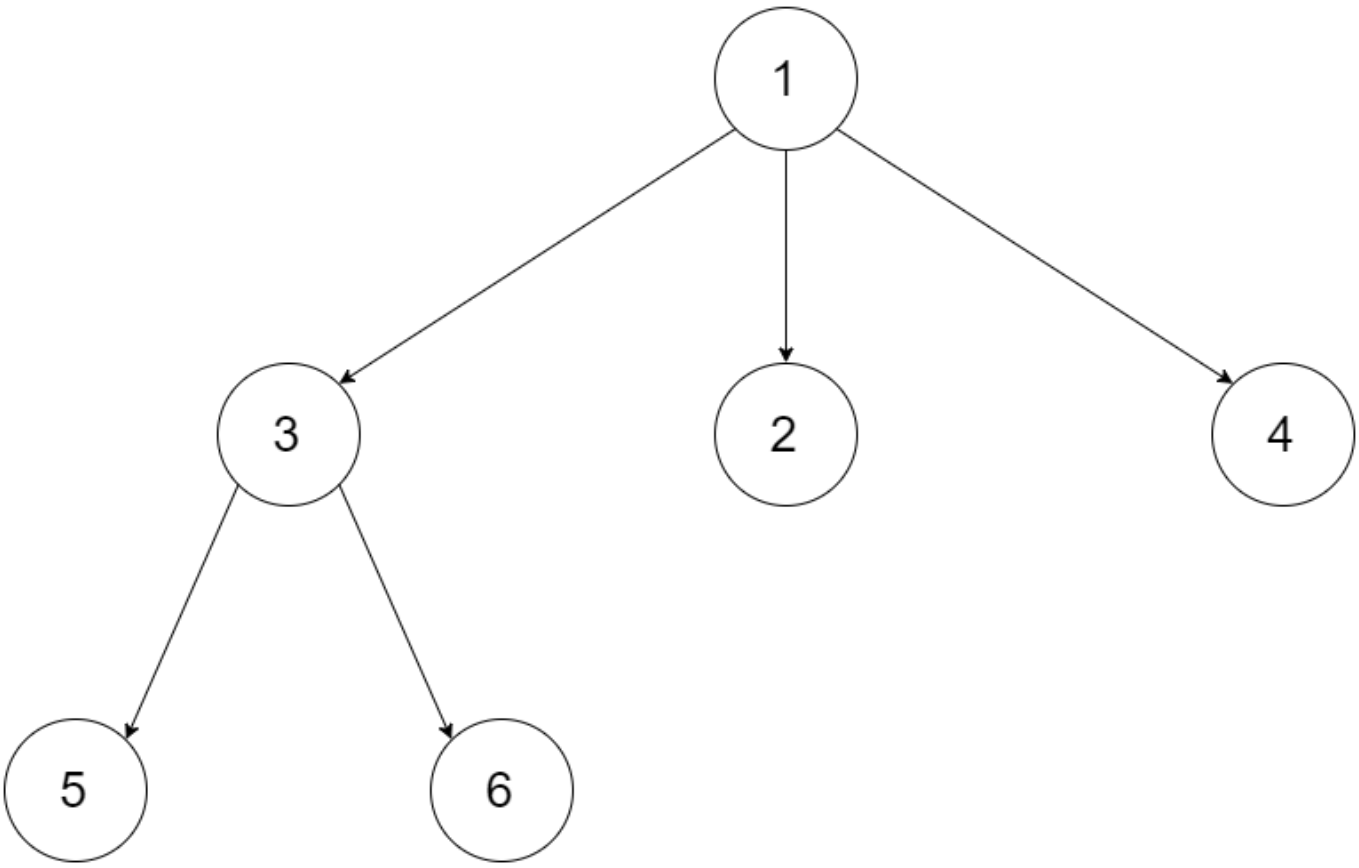
n叉树的最大深度

力扣题目链接

给定一个 n 叉树，找到其最大深度。

最大深度是指从根节点到最远叶子节点的最长路径上的节点总数。

例如，给定一个 3叉树 :



我们应返回其最大深度，3。

思路

依然可以提供递归法和迭代法，来解决这个问题，思路是和二叉树思路一样的，直接给出代码如下：

递归法

c++代码：

```
class solution {
public:
    int maxDepth(Node* root) {
        if (root == 0) return 0;
        int depth = 0;
        for (int i = 0; i < root->children.size(); i++) {
            depth = max (depth, maxDepth(root->children[i]));
        }
        return depth + 1;
    }
};
```

迭代法

依然是层序遍历，代码如下：

```
class solution {
public:
    int maxDepth(Node* root) {
        queue<Node*> que;
        if (root != NULL) que.push(root);
        int depth = 0;
        while (!que.empty()) {
            int size = que.size();
            depth++; // 记录深度
            for (int i = 0; i < size; i++) {
                Node* node = que.front();
                que.pop();
                for (int j = 0; j < node->children.size(); j++) {
                    if (node->children[j]) que.push(node->children[j]);
                }
            }
        }
        return depth;
    }
};
```

> 和求最大深度一个套路?

10.二叉树的最小深度

力扣题目链接

给定一个二叉树，找出其最小深度。

最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

说明: 叶子节点是指没有子节点的节点。

示例:

给定二叉树 [3,9,20,null,null,15,7],

```
    3
   / \
  9  20
    /  \
   15   7
```

返回它的最小深度 2.

算法公开课

《代码随想录》算法视频公开课：看起来好像做过，一写就错！ | LeetCode：111.二叉树的最小深度，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

看完了这篇104.二叉树的最大深度，再来看看如何求最小深度。

直觉上好像和求最大深度差不多，其实还是差不少的。

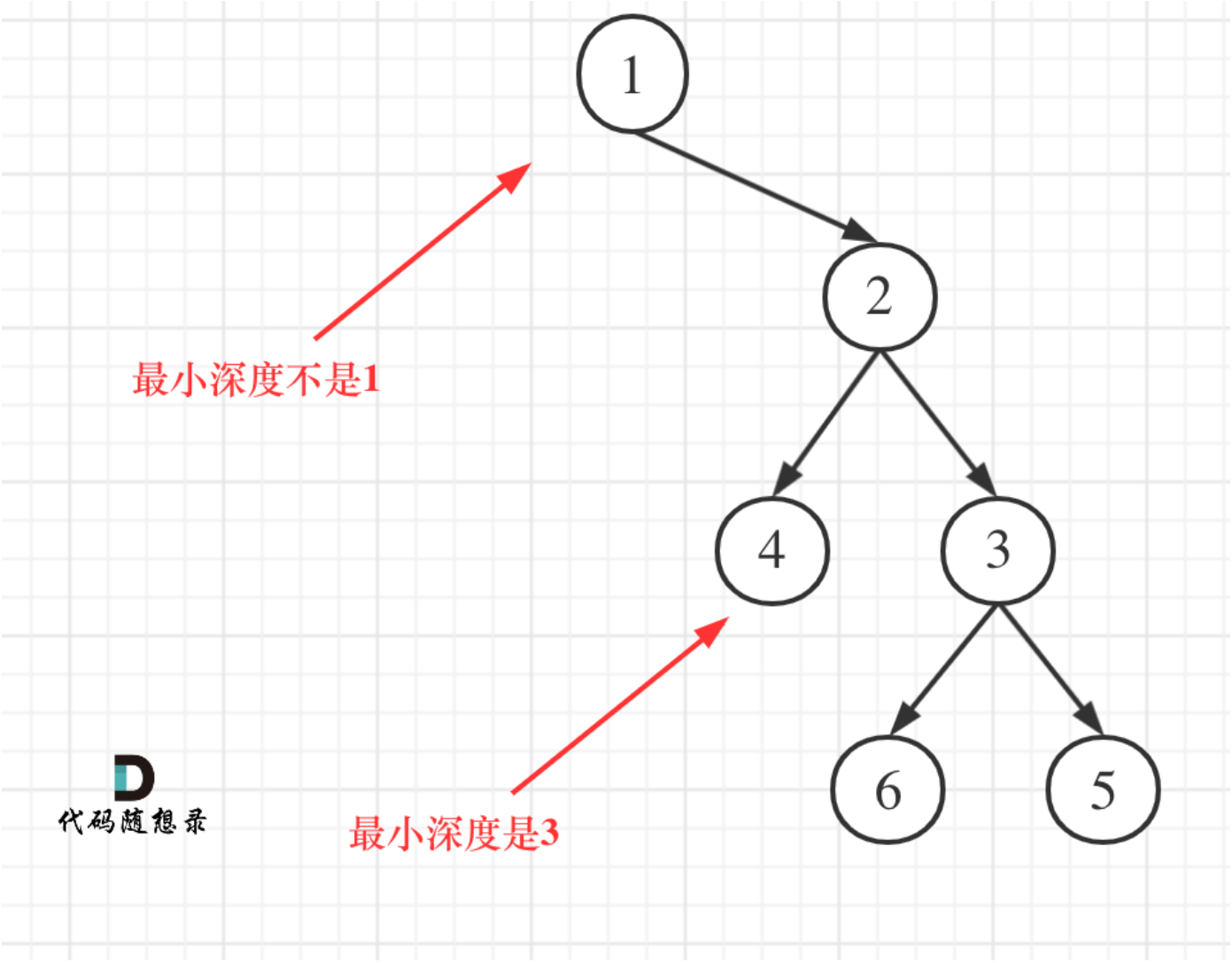
本题依然是前序遍历和后序遍历都可以，前序求的是深度，后序求的是高度。

- 二叉树节点的深度：指从根节点到该节点的最长简单路径边的条数或者节点数（取决于深度从0开始还是从1开始）
- 二叉树节点的高度：指从该节点到叶子节点的最长简单路径边的条数后者节点数（取决于高度从0开始还是从1开始）

那么使用后序遍历，其实求的是根节点到叶子节点的最小距离，就是求高度的过程，不过这个最小距离 也同样是最小深度。

以下讲解中遍历顺序上依然采用后序遍历（因为要比较递归返回之后的结果，本文我也给出前序遍历的写法）。

本题还有一个误区，在处理节点的过程中，最大深度很容易理解，最小深度就不那么好理解，如图：



这就重新审题了，题目中说的是：**最小深度是从根节点到最近叶子节点的最短路径上的节点数量。**，注意是**叶子节点**。

什么是叶子节点，左右孩子都为空的节点才是叶子节点！

递归法

来来来，一起递归三部曲：

1. 确定递归函数的参数和返回值

参数为要传入的二叉树根节点，返回的是int类型的深度。

代码如下：

```
int getDepth(TreeNode* node)
```

2. 确定终止条件

终止条件也是遇到空节点返回0，表示当前节点的高度为0。

代码如下：

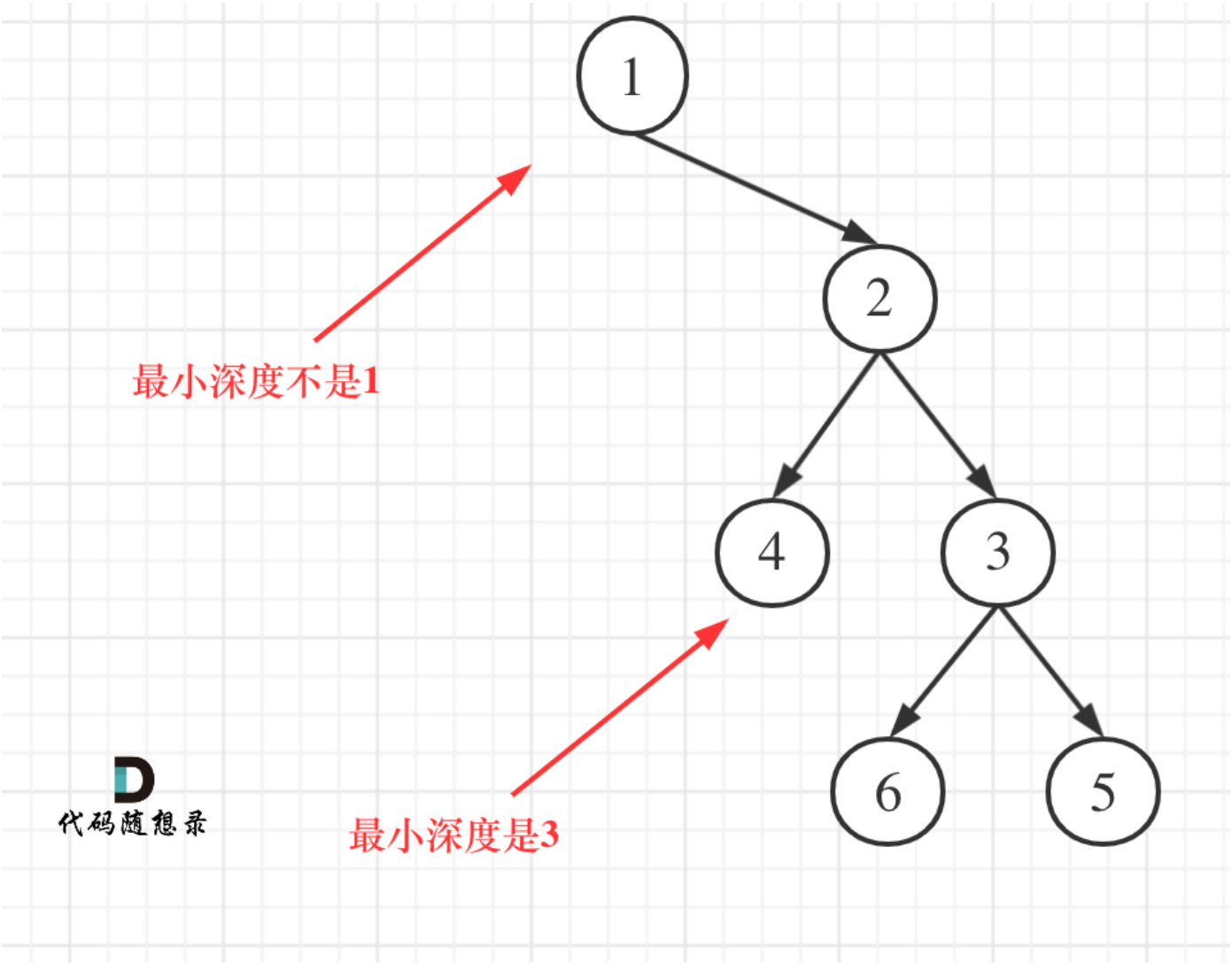
```
if (node == NULL) return 0;
```

3. 确定单层递归的逻辑

这块和求最大深度可就不一样了，一些同学可能会写如下代码：

```
int leftDepth = getDepth(node->left);
int rightDepth = getDepth(node->right);
int result = 1 + min(leftDepth, rightDepth);
return result;
```

这个代码就犯了此图中的误区：



如果这么求的话，没有左孩子的分支会算为最短深度。

所以，如果左子树为空，右子树不为空，说明最小深度是 1 + 右子树的深度。

反之，右子树为空，左子树不为空，最小深度是 1 + 左子树的深度。 最后如果左右子树都不为空，返回左右子树深度最小值 + 1 。

代码如下：

```
int leftDepth = getDepth(node->left);           // 左
int rightDepth = getDepth(node->right);         // 右
                                                // 中
// 当一个左子树为空，右不为空，这时并不是最低点
if (node->left == NULL && node->right != NULL) {
    return 1 + rightDepth;
}
// 当一个右子树为空，左不为空，这时并不是最低点
if (node->left != NULL && node->right == NULL) {
    return 1 + leftDepth;
}
int result = 1 + min(leftDepth, rightDepth);
return result;
```

遍历的顺序为后序（左右中），可以看出：**求二叉树的最小深度和求二叉树的最大深度的差别主要在于处理左右孩子不为空的逻辑。**

整体递归代码如下：

```
class Solution {
public:
    int getDepth(TreeNode* node) {
        if (node == NULL) return 0;
        int leftDepth = getDepth(node->left);           // 左
        int rightDepth = getDepth(node->right);         // 右
                                                        // 中
        // 当一个左子树为空，右不为空，这时并不是最低点
        if (node->left == NULL && node->right != NULL) {
            return 1 + rightDepth;
        }
        // 当一个右子树为空，左不为空，这时并不是最低点
        if (node->left != NULL && node->right == NULL) {
            return 1 + leftDepth;
        }
        int result = 1 + min(leftDepth, rightDepth);
        return result;
    }

    int minDepth(TreeNode* root) {
        return getDepth(root);
    }
};
```

精简之后代码如下：

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right != NULL) {
            return 1 + minDepth(root->right);
        }
        if (root->left != NULL && root->right == NULL) {
            return 1 + minDepth(root->left);
        }
        return 1 + min(minDepth(root->left), minDepth(root->right));
    }
};
```

精简之后的代码根本看不出是哪种遍历方式，所以依然还要强调一波：如果对二叉树的操作还不熟练，尽量不要直接照着精简代码来学。

前序遍历的方式：

```
class Solution {
private:
    int result;
    void getdepth(TreeNode* node, int depth) {
        // 函数递归终止条件
        if (root == nullptr) {
            return;
        }
        // 中，处理逻辑：判断是不是叶子结点
        if (root -> left == nullptr && root->right == nullptr) {
            res = min(res, depth);
        }
        if (node->left) { // 左
            getdepth(node->left, depth + 1);
        }
        if (node->right) { // 右
            getdepth(node->right, depth + 1);
        }
        return ;
    }

public:
    int minDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }
        result = INT_MAX;
        getdepth(root, 1);
        return result;
    }
```

```
};
```

迭代法

相对于104.二叉树的最大深度，本题还可以使用层序遍历的方式来解决，思路是一样的。

如果对层序遍历还不清楚的话，可以看这篇：二叉树：层序遍历登场！

需要注意的是，只有当左右孩子都为空的时候，才说明遍历到最低点了。如果其中一个孩子不为空则不是最低点

代码如下：（详细注释）

```
class Solution {
public:

    int minDepth(TreeNode* root) {
        if (root == NULL) return 0;
        int depth = 0;
        queue<TreeNode*> que;
        que.push(root);
        while(!que.empty()) {
            int size = que.size();
            depth++; // 记录最小深度
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
                if (!node->left && !node->right) { // 当左右孩子都为空的时候，说明是最低点的一
层了，退出
                    return depth;
                }
            }
        }
        return depth;
    }
};
```

11.完全二叉树的节点个数

力扣题目链接

给出一个完全二叉树，求出该树的节点个数。

示例 1：

- 输入：root = [1,2,3,4,5,6]
- 输出：6

示例 2：

- 输入：root = []
- 输出：0

示例 3：

- 输入：root = [1]
- 输出：1

提示：

- 树中节点的数目范围是[0, 5 * 10^4]
- 0 <= Node.val <= 5 * 10^4
- 题目数据保证输入的树是 完全二叉树

算法公开课

《代码随想录》算法视频公开课：要理解普通二叉树和完全二叉树的区别！ | LeetCode：222.完全二叉树节点的数量，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本篇给出按照普通二叉树的求法以及利用完全二叉树性质的求法。

普通二叉树

首先按照普通二叉树的逻辑来求。

这道题目的递归法和求二叉树的深度写法类似， 而迭代法，二叉树：层序遍历登场！遍历模板稍稍修改一下，记录遍历的节点数量就可以了。

递归遍历的顺序依然是后序（左右中）。

递归

如果对求二叉树深度还不熟悉的话，看这篇：二叉树：看看这些树的最大深度。

1. 确定递归函数的参数和返回值：参数就是传入树的根节点，返回就返回以该节点为根节点二叉树的节点数量，所以返回值为int类型。

代码如下：

```
int getNodesNum(TreeNode* cur) {
```

2. 确定终止条件：如果为空节点的话，就返回0，表示节点数为0。

代码如下：

```
if (cur == NULL) return 0;
```

3. 确定单层递归的逻辑：先求它的左子树的节点数量，再求右子树的节点数量，最后取总和再加一 （加1是因为算上当前中间节点）就是目前节点为根节点的节点数量。

代码如下：

```
int leftNum = getNodesNum(cur->left);      // 左
int rightNum = getNodesNum(cur->right);    // 右
int treeNum = leftNum + rightNum + 1;      // 中
return treeNum;
```

所以整体C++代码如下：

```
// 版本一
class Solution {
private:
    int getNodesNum(TreeNode* cur) {
        if (cur == NULL) return 0;
        int leftNum = getNodesNum(cur->left);      // 左
        int rightNum = getNodesNum(cur->right);    // 右
        int treeNum = leftNum + rightNum + 1;      // 中
        return treeNum;
    }
public:
    int countNodes(TreeNode* root) {
        return getNodesNum(root);
    }
};
```

代码精简之后C++代码如下：

```
// 版本二
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == NULL) return 0;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(log n)，算上了递归系统栈占用的空间

网上基本都是这个精简的代码版本，其实不建议大家照着这个来写，代码确实精简，但隐藏了一些内容，连遍历的顺序都看不出来，所以初学者建议学习版本一的代码，稳稳的打基础。

迭代

如果对求二叉树层序遍历还不熟悉的话，看这篇：二叉树：层序遍历登场！。

那么只要模板少做改动，加一个变量result，统计节点数量就可以了

```
class Solution {
public:
    int countNodes(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        int result = 0;
        while (!que.empty()) {
            int size = que.size();
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                result++;    // 记录节点数量
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

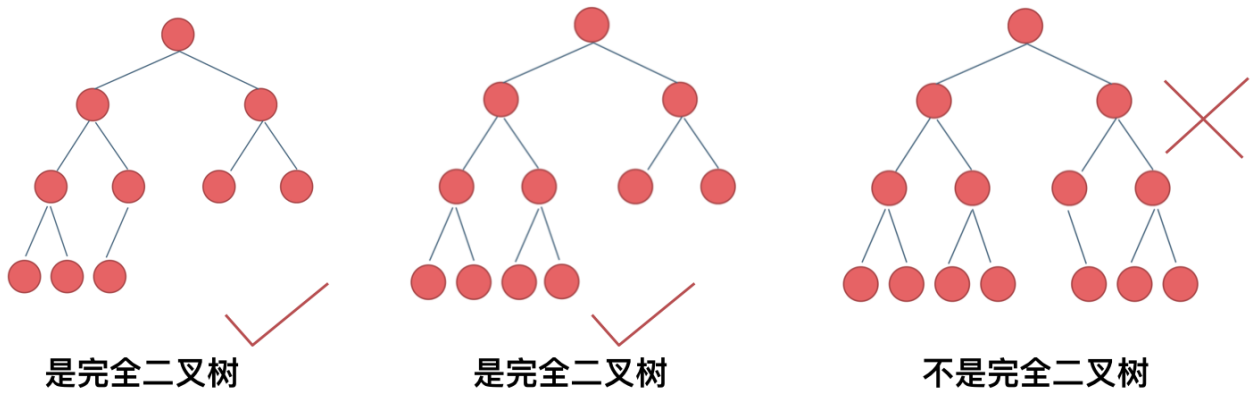
完全二叉树

以上方法都是按照普通二叉树来做的，对于完全二叉树特性不了解的同学可以看这篇 关于二叉树，你该了解这些！，这篇详细介绍了各种二叉树的特性。

在完全二叉树中，除了最底层节点可能没填满外，其余每层节点数都达到最大值，并且最下面一层的节点都集中在该层最左边的若干位置。若最底层为第 h 层，则该层包含 1~ 2^(h-1) 个节点。

大家要自己看完全二叉树的定义，很多同学对完全二叉树其实不是真正的懂了。

我来举一个典型的例子如题：

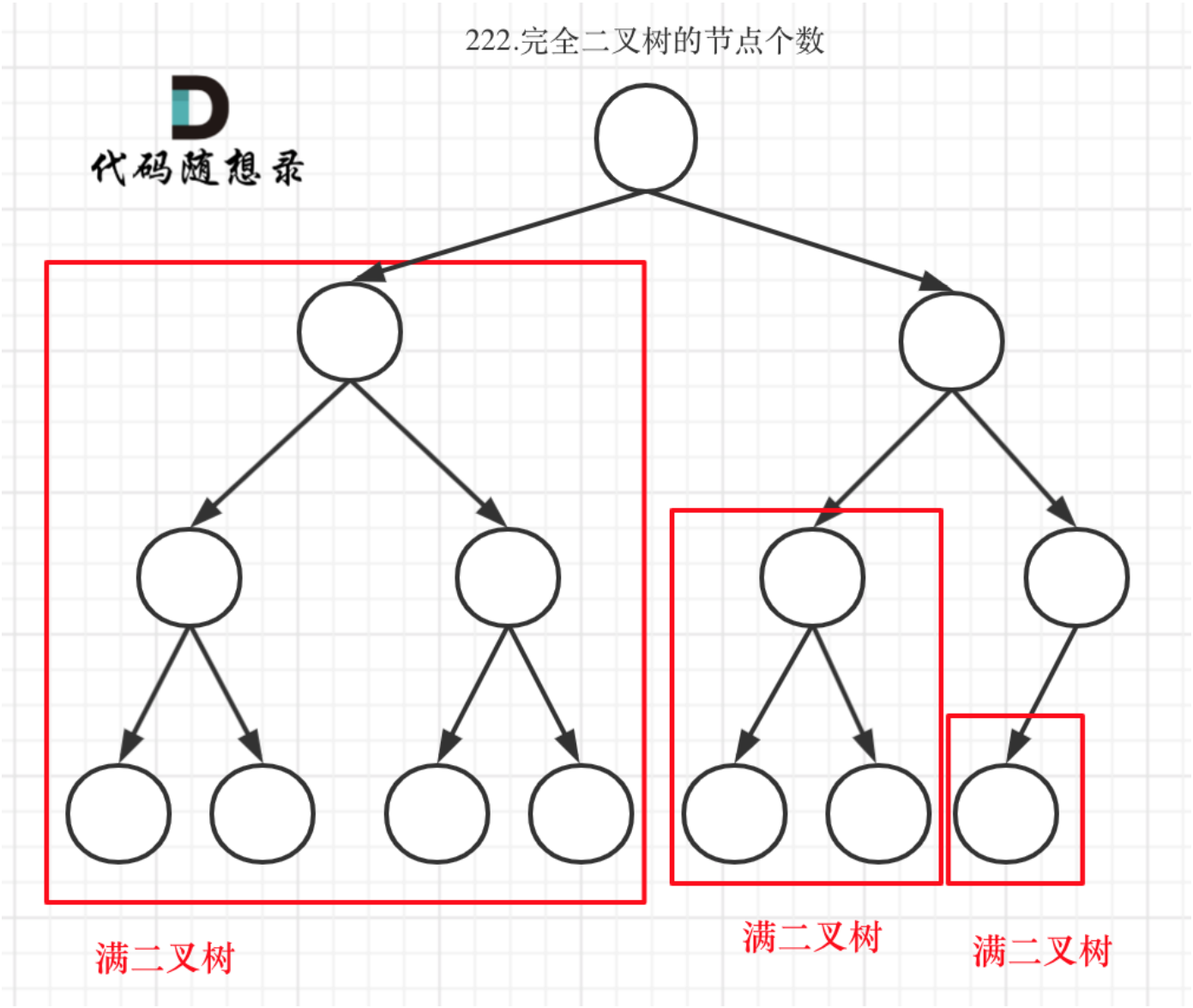


完全二叉树只有两种情况，情况一：就是满二叉树，情况二：最后一层叶子节点没有满。

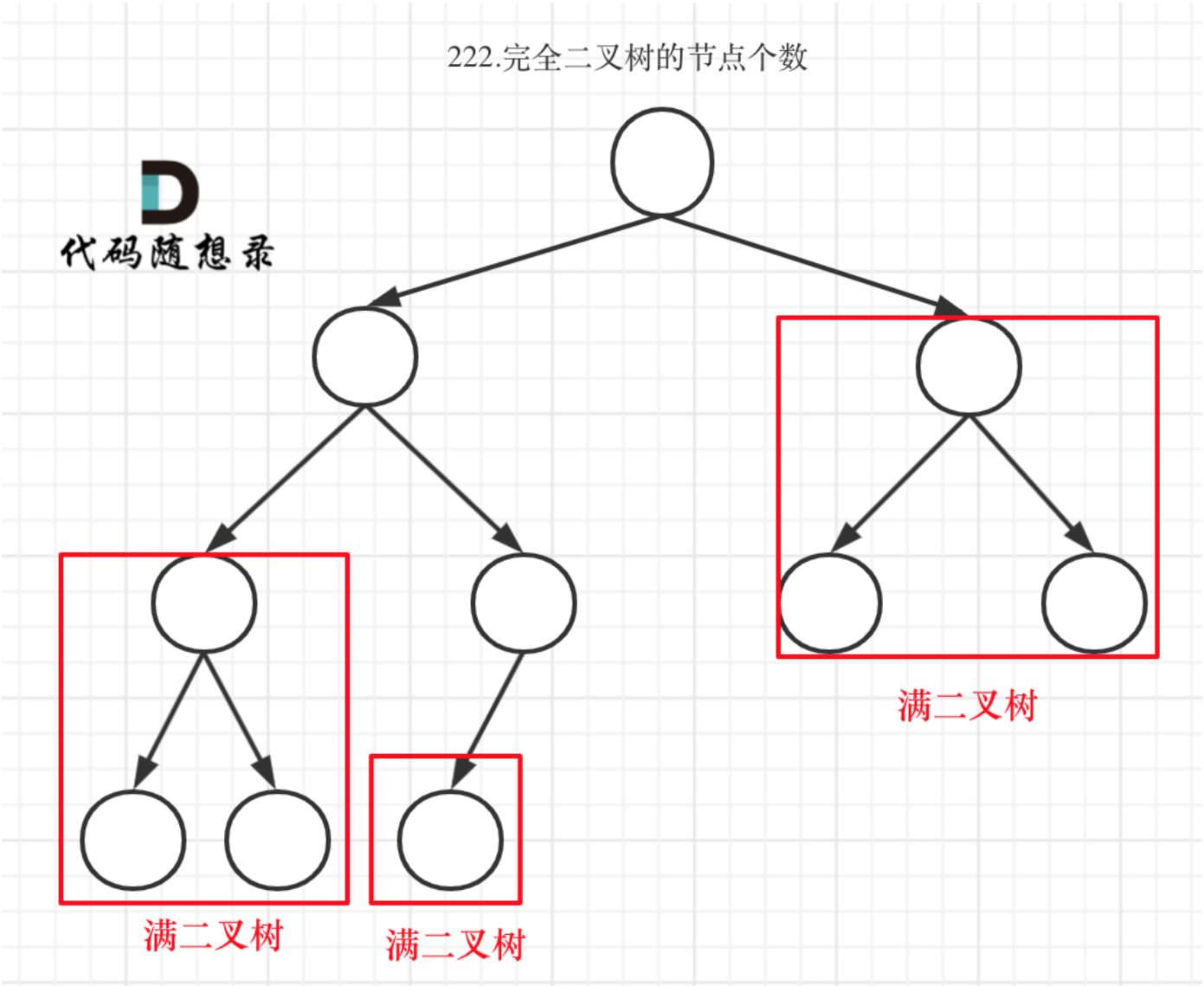
对于情况一，可以直接用 2^树深度 - 1 来计算，注意这里根节点深度为1。

对于情况二，分别递归左孩子，和右孩子，递归到某一深度一定会有左孩子或者右孩子为满二叉树，然后依然可以按照情况1来计算。

完全二叉树（一）如图：



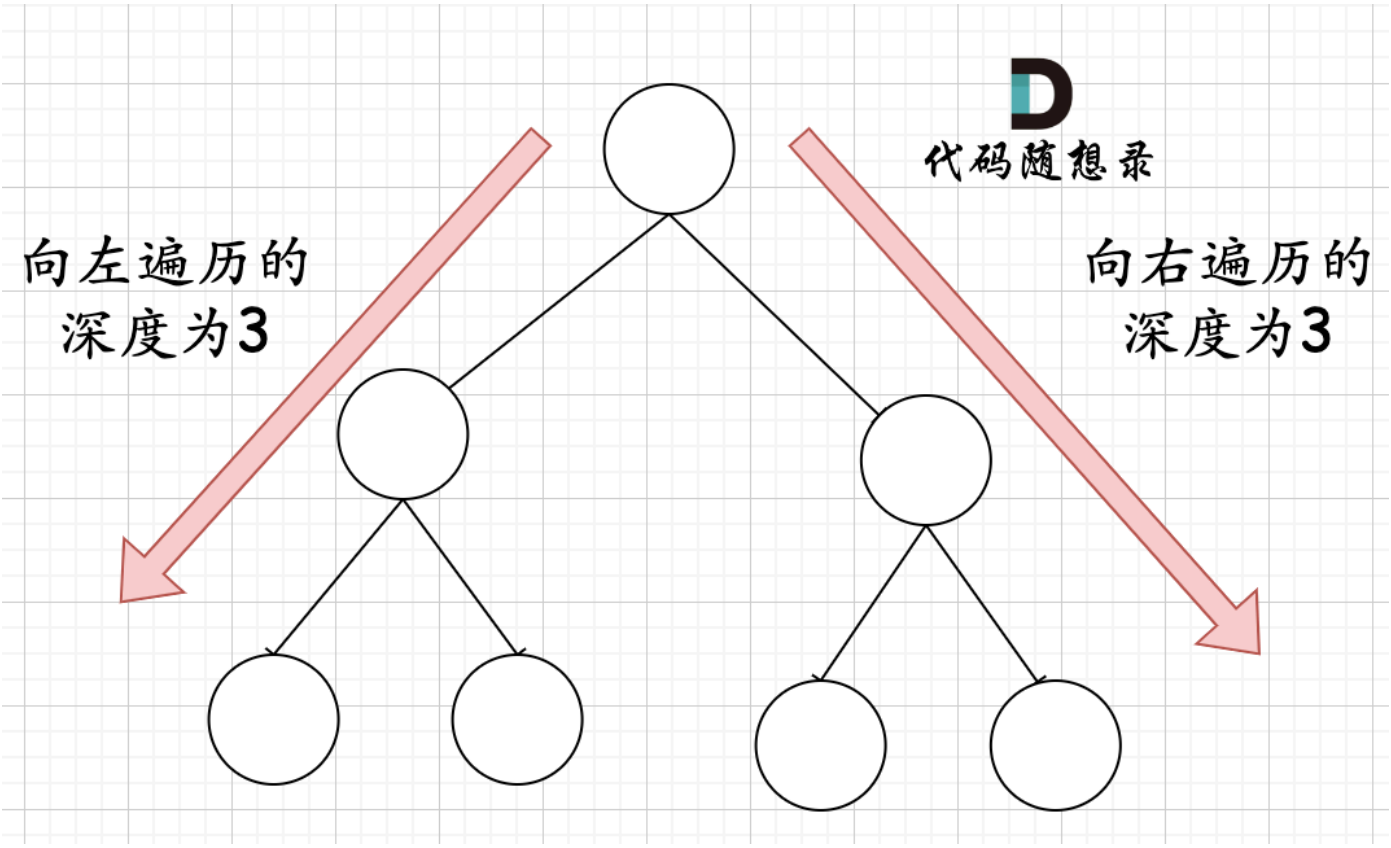
完全二叉树（二）如图：



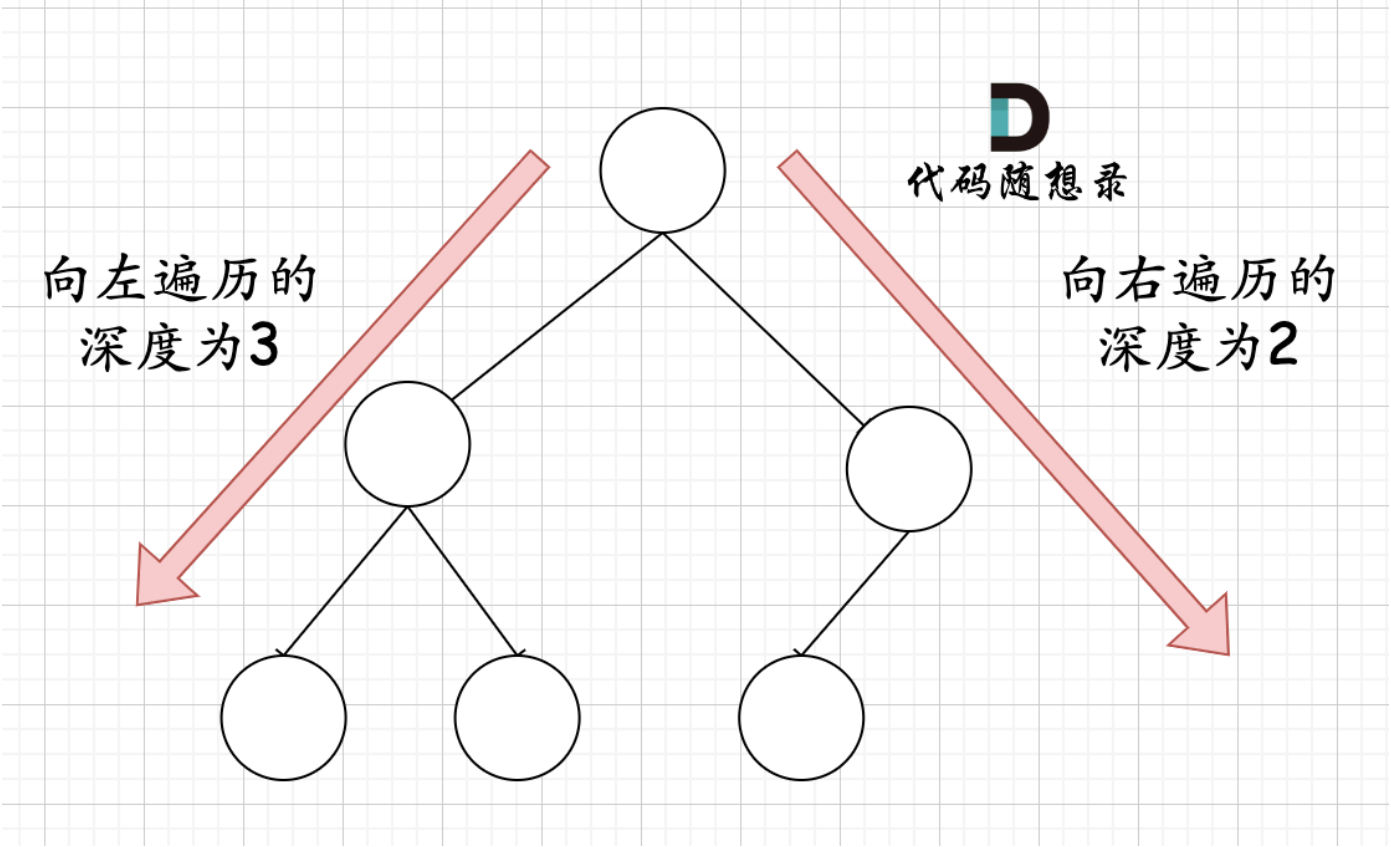
可以看出如果整个树不是满二叉树，就递归其左右孩子，直到遇到满二叉树为止，用公式计算这个子树（满二叉树）的节点数量。

这里关键在于如何去判断一个左子树或者右子树是不是满二叉树呢?

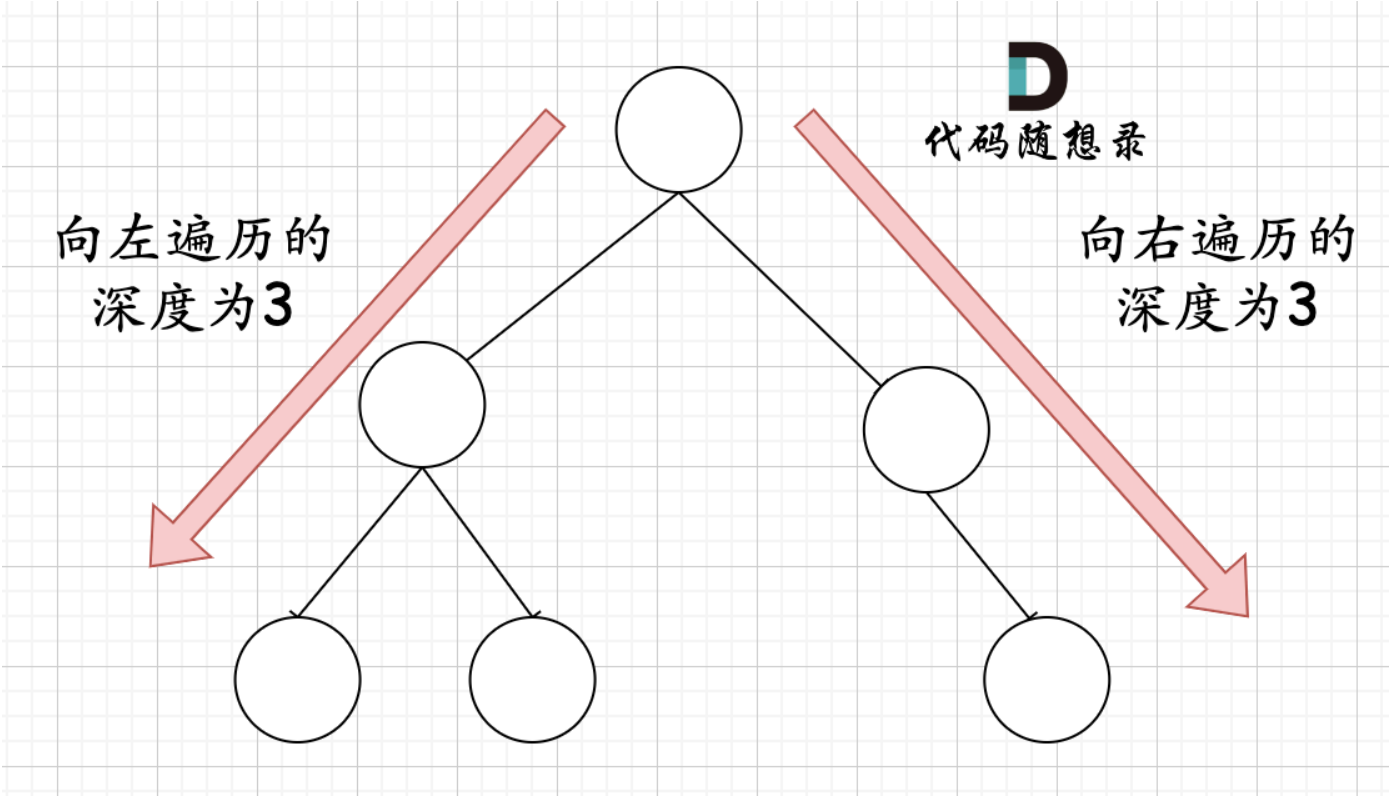
在完全二叉树中，如果递归向左遍历的深度等于递归向右遍历的深度，那说明就是满二叉树。如图：



在完全二叉树中，如果递归向左遍历的深度不等于递归向右遍历的深度，则说明不是满二叉树，如图：



那有录友说了，这种情况，递归向左遍历的深度等于递归向右遍历的深度，但也不是满二叉树，如题：



如果这么想，大家就是对 完全二叉树理解有误区了，**以上这棵二叉树，它根本就不是一个完全二叉树**！

判断其子树是不是满二叉树，如果是则利用公式计算这个子树（满二叉树）的节点数量，如果不是则继续递归，那么 在递归三部曲中，第二部：终止条件的写法应该是这样的：

```
if (root == nullptr) return 0;
// 开始根据左深度和右深度是否相同来判断该子树是不是满二叉树
TreeNode* left = root->left;
TreeNode* right = root->right;
int leftDepth = 0, rightDepth = 0; // 这里初始为0是有目的的，为了下面求指数方便
while (left) {  // 求左子树深度
    left = left->left;
    leftDepth++;
}
while (right) { // 求右子树深度
    right = right->right;
    rightDepth++;
}
if (leftDepth == rightDepth) {
    return (2 << leftDepth) - 1; // 注意(2<<1) 相当于2^2，返回满足满二叉树的子树节点数量
}
```

递归三部曲，第三部，单层递归的逻辑：（可以看出使用后序遍历）

```
int leftTreeNum = countNodes(root->left);       // 左
int rightTreeNum = countNodes(root->right);     // 右
int result = leftTreeNum + rightTreeNum + 1;    // 中
return result;
```

该部分精简之后代码为：

```
return countNodes(root->left) + countNodes(root->right) + 1;
```

最后整体C++代码如下：

```
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == nullptr) return 0;
        TreeNode* left = root->left;
        TreeNode* right = root->right;
        int leftDepth = 0, rightDepth = 0; // 这里初始为0是有目的的，为了下面求指数方便
        while (left) {  // 求左子树深度
            left = left->left;
            leftDepth++;
        }
        while (right) { // 求右子树深度
            right = right->right;
            rightDepth++;
        }
        if (leftDepth == rightDepth) {
            return (2 << leftDepth) - 1; // 注意(2<<1) 相当于2^2，所以leftDepth初始为0
        }
        return countNodes(root->left) + countNodes(root->right) + 1;
    }
};
```

- 时间复杂度：O(log n × log n)
- 空间复杂度：O(log n)

> 求高度还是求深度，你搞懂了不?

12.平衡二叉树

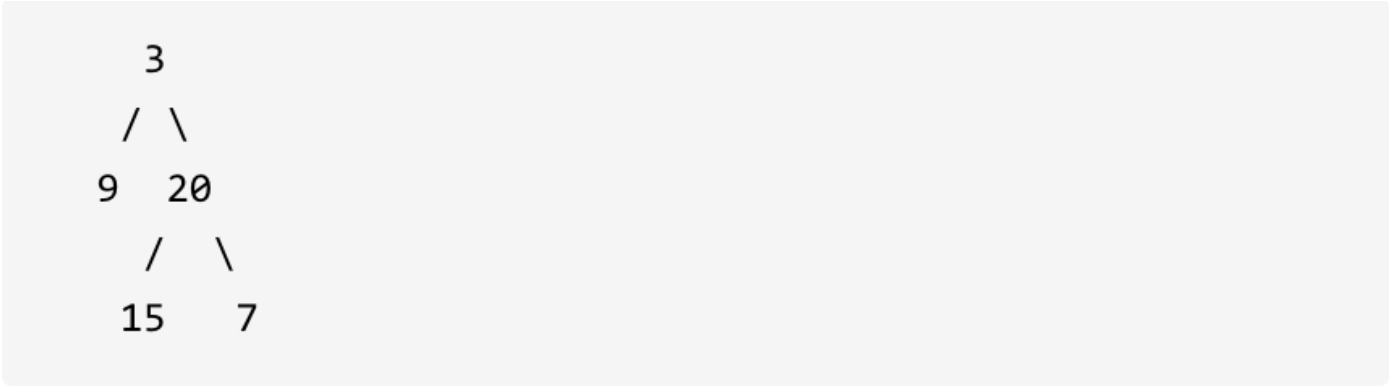
[力扣题目链接]

给定一个二叉树，判断它是否是高度平衡的二叉树。

本题中，一棵高度平衡二叉树定义为：一个二叉树每个节点 的左右两个子树的高度差的绝对值不超过1。

示例 1:

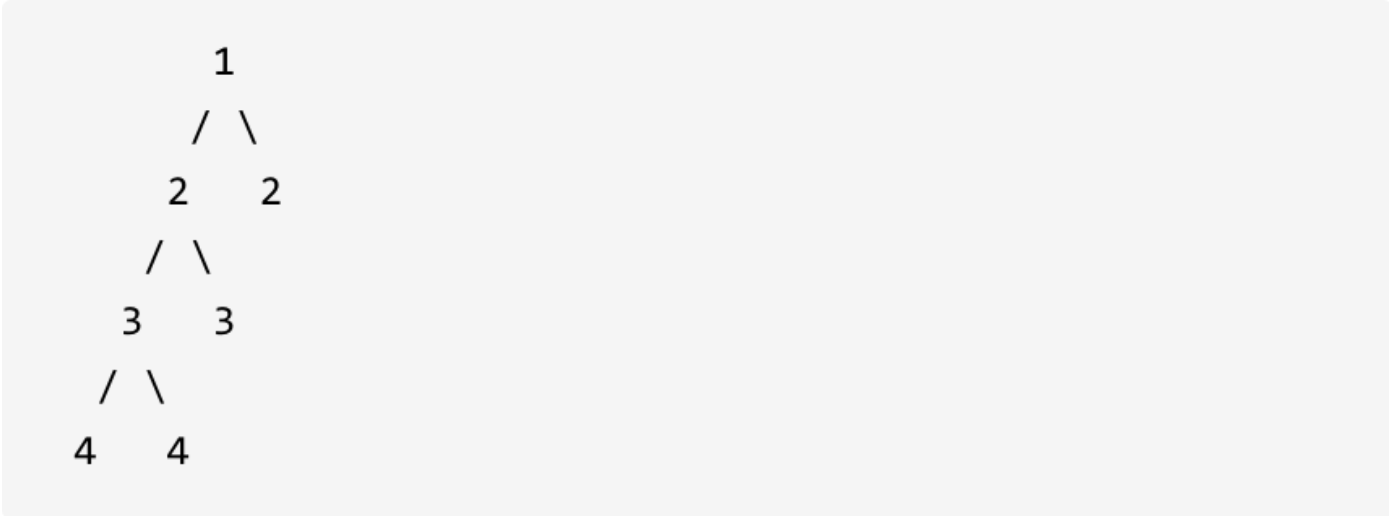
给定二叉树 [3,9,20,null,null,15,7]



返回 true 。

示例 2:

给定二叉树 [1,2,2,3,3,null,null,4,4]



返回 false 。

算法公开课

[《代码随想录》算法视频公开课](): [后序遍历求高度，高度判断是否平衡 | LeetCode：110.平衡二叉树](), 相信结合视频在看本篇题解，更有助于大家对本题的理解。

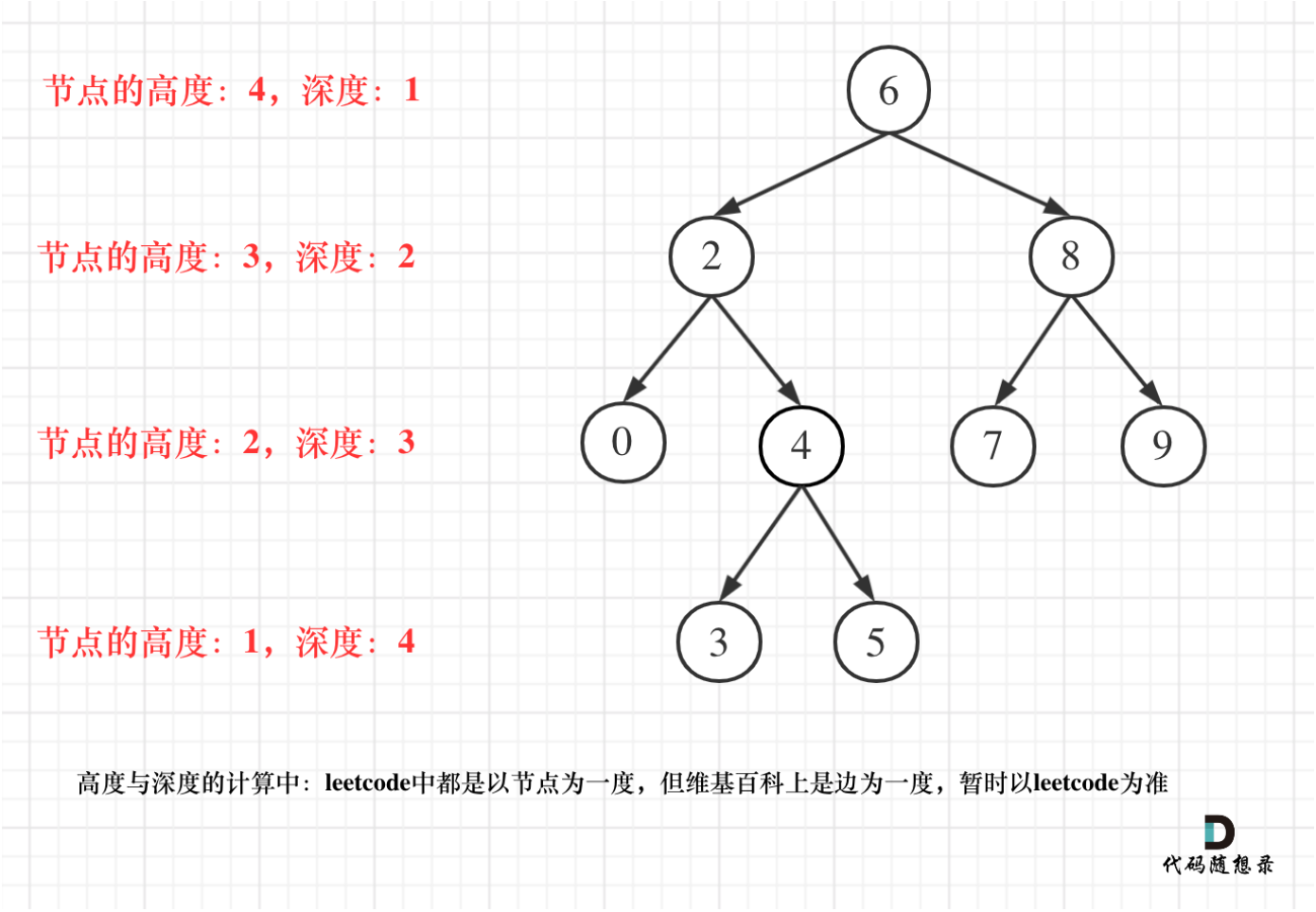
题外话

咋眼一看这道题目和104.二叉树的最大深度很像，其实有很大区别。

这里强调一波概念：

- 二叉树节点的深度：指从根节点到该节点的最长简单路径边的条数。
- 二叉树节点的高度：指从该节点到叶子节点的最长简单路径边的条数。

但leetcode中强调的深度和高度很明显是按照节点来计算的，如图：



关于根节点的深度究竟是1 还是 0，不同的地方有不一样的标准，leetcode的题目中都是以节点为一度，即根节点深度是1。但维基百科上定义用边为一度，即根节点的深度是0，我们暂时以leetcode为准（毕竟要在这上面刷题）。

因为求深度可以从上到下去查 所以需要前序遍历（中左右），而高度只能从下到上去查，所以只能后序遍历（左右中）

有的同学一定疑惑，为什么104.二叉树的最大深度中求的是二叉树的最大深度，也用的是后序遍历。

那是因为代码的逻辑其实是求的根节点的高度，而根节点的高度就是这棵树的最大深度，所以才可以使用后序遍历。

在104.二叉树的最大深度中，如果真正求取二叉树的最大深度，代码应该写成如下：（前序遍历）

```
class Solution {
public:
    int result;
    void getDepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中

        if (node->left == NULL && node->right == NULL) return ;

        if (node->left) { // 左
            depth++;    // 深度+1
            getDepth(node->left, depth);
            depth--;    // 回溯，深度-1
```

```
        }
        if (node->right) { // 右
            depth++;    // 深度+1
            getDepth(node->right, depth);
            depth--;    // 回溯，深度-1
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
        if (root == NULL) return result;
        getDepth(root, 1);
        return result;
    }
};
```

可以看出使用了前序（中左右）的遍历顺序，这才是真正求深度的逻辑！

注意以上代码是为了把细节体现出来，简化一下代码如下：

```
class Solution {
public:
    int result;
    void getDepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中
        if (node->left == NULL && node->right == NULL) return ;
        if (node->left) { // 左
            getDepth(node->left, depth + 1);
        }
        if (node->right) { // 右
            getDepth(node->right, depth + 1);
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
        if (root == 0) return result;
        getDepth(root, 1);
        return result;
    }
};
```

本题思路

递归

此时大家应该明白了既然要求比较高度，必然是要后序遍历。

递归三步曲分析：

1. 明确递归函数的参数和返回值

参数：当前传入节点。
返回值：以当前传入节点为根节点的树的高度。

那么如何标记左右子树是否差值大于1呢?

如果当前传入节点为根节点的二叉树已经不是二叉平衡树了，还返回高度的话就没有意义了。

所以如果已经不是二叉平衡树了，可以返回-1 来标记已经不符合平衡树的规则了。

代码如下：

```
// -1 表示已经不是平衡二叉树了，否则返回值是以该节点为根节点树的高度
int getHeight(TreeNode* node)
```

2. 明确终止条件

递归的过程中依然是遇到空节点了为终止，返回0，表示当前节点为根节点的树高度为0

代码如下：

```
if (node == NULL) {
    return 0;
}
```

3. 明确单层递归的逻辑

如何判断以当前传入节点为根节点的二叉树是否是平衡二叉树呢？当然是其左子树高度和其右子树高度的差值。

分别求出其左右子树的高度，然后如果差值小于等于1，则返回当前二叉树的高度，否则返回-1，表示已经不是二叉平衡树了。

代码如下：

```
int leftHeight = getHeight(node->left); // 左
if (leftHeight == -1) return -1;
int rightHeight = getHeight(node->right); // 右
if (rightHeight == -1) return -1;

int result;
if (abs(leftHeight - rightHeight) > 1) {  // 中
    result = -1;
} else {
    result = 1 + max(leftHeight, rightHeight); // 以当前节点为根节点的树的最大高度
}

return result;
```

代码精简之后如下：

```
int leftHeight = getHeight(node->left);
if (leftHeight == -1) return -1;
int rightHeight = getHeight(node->right);
if (rightHeight == -1) return -1;
return abs(leftHeight - rightHeight) > 1 ? -1 : 1 + max(leftHeight, rightHeight);
```

此时递归的函数就已经写出来了，这个递归的函数传入节点指针，返回以该节点为根节点的二叉树的高度，如果不是二叉平衡树，则返回-1。

getHeight整体代码如下：

```
int getHeight(TreeNode* node) {
    if (node == NULL) {
        return 0;
    }
    int leftHeight = getHeight(node->left);
    if (leftHeight == -1) return -1;
    int rightHeight = getHeight(node->right);
    if (rightHeight == -1) return -1;
    return abs(leftHeight - rightHeight) > 1 ? -1 : 1 + max(leftHeight, rightHeight);
}
```

最后本题整体递归代码如下：

```
class Solution {
public:
    // 返回以该节点为根节点的二叉树的高度，如果不是平衡二叉树了则返回-1
    int getHeight(TreeNode* node) {
        if (node == NULL) {
            return 0;
        }
        int leftHeight = getHeight(node->left);
```

```
        if (leftHeight == -1) return -1;
        int rightHeight = getHeight(node->right);
        if (rightHeight == -1) return -1;
        return abs(leftHeight - rightHeight) > 1 ? -1 : 1 + max(leftHeight,
rightHeight);
    }
    bool isBalanced(TreeNode* root) {
        return getHeight(root) == -1 ? false : true;
    }
};
```

迭代

在104.二叉树的最大深度中我们可以使用层序遍历来求深度，但是就不能直接用层序遍历来求高度了，这就体现出求高度和求深度的不同。

本题的迭代方式可以先定义一个函数，专门用来求高度。

这个函数通过栈模拟的后序遍历找每一个节点的高度（其实是通过求传入节点为根节点的最大深度来求的高度）

代码如下：

```
// cur节点的最大深度，就是cur的高度
int getDepth(TreeNode* cur) {
    stack<TreeNode*> st;
    if (cur != NULL) st.push(cur);
    int depth = 0; // 记录深度
    int result = 0;
    while (!st.empty()) {
        TreeNode* node = st.top();
        if (node != NULL) {
            st.pop();
            st.push(node);                          // 中
            st.push(NULL);
            depth++;
            if (node->right) st.push(node->right);  // 右
            if (node->left) st.push(node->left);    // 左

        } else {
            st.pop();
            node = st.top();
            st.pop();
            depth--;
        }
        result = result > depth ? result : depth;
    }
    return result;
}
```

然后再用栈来模拟后序遍历，遍历每一个节点的时候，再去判断左右孩子的高度是否符合，代码如下：

```
bool isBalanced(TreeNode* root) {
    stack<TreeNode*> st;
    if (root == NULL) return true;
    st.push(root);
    while (!st.empty()) {
        TreeNode* node = st.top();                       // 中
        st.pop();
        if (abs(getDepth(node->left) - getDepth(node->right)) > 1) { // 判断左右孩子高度是否符合
            return false;
        }
        if (node->right) st.push(node->right);           // 右（空节点不入栈）
        if (node->left) st.push(node->left);             // 左（空节点不入栈）
    }
    return true;
}
```

整体代码如下：

```
class Solution {
private:
    int getDepth(TreeNode* cur) {
        stack<TreeNode*> st;
        if (cur != NULL) st.push(cur);
        int depth = 0; // 记录深度
        int result = 0;
        while (!st.empty()) {
            TreeNode* node = st.top();
            if (node != NULL) {
                st.pop();
                st.push(node);                           // 中
                st.push(NULL);
                depth++;
                if (node->right) st.push(node->right);   // 右
                if (node->left) st.push(node->left);     // 左

            } else {
                st.pop();
                node = st.top();
                st.pop();
                depth--;
            }
            result = result > depth ? result : depth;
        }
        return result;
    }

public:
```

```
    bool isBalanced(TreeNode* root) {
        stack<TreeNode*> st;
        if (root == NULL) return true;
        st.push(root);
        while (!st.empty()) {
            TreeNode* node = st.top();                       // 中
            st.pop();
            if (abs(getDepth(node->left) - getDepth(node->right)) > 1) {
                return false;
            }
            if (node->right) st.push(node->right);           // 右（空节点不入栈）
            if (node->left) st.push(node->left);             // 左（空节点不入栈）
        }
        return true;
    }
};
```

当然此题用迭代法，其实效率很低，因为没有很好的模拟回溯的过程，所以迭代法有很多重复的计算。

虽然理论上所有的递归都可以用迭代来实现，但是有的场景难度可能比较大。

例如：都知道回溯法其实就是递归，但是很少人用迭代的方式去实现回溯算法！

因为对于回溯算法已经是非常复杂的递归了，如果再用迭代的话，就是自己给自己找麻烦，效率也并不一定高。

总结

通过本题可以了解求二叉树深度 和 二叉树高度的差异，求深度适合用前序遍历，而求高度适合用后序遍历。

本题迭代法其实有点复杂，大家可以有一个思路，也不一定说非要写出来。

但是递归方式是一定要掌握的！

> 只用了递归，其实还用了回溯

13. 二叉树的所有路径

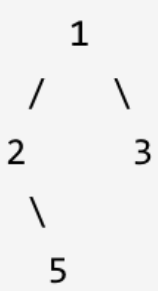
[力扣题目链接]

给定一个二叉树，返回所有从根节点到叶子节点的路径。

说明: 叶子节点是指没有子节点的节点。

示例:

输入：



输出： ["1->2->5", "1->3"]

解释： 所有根节点到叶子节点的路径为：1->2->5，1->3

算法公开课

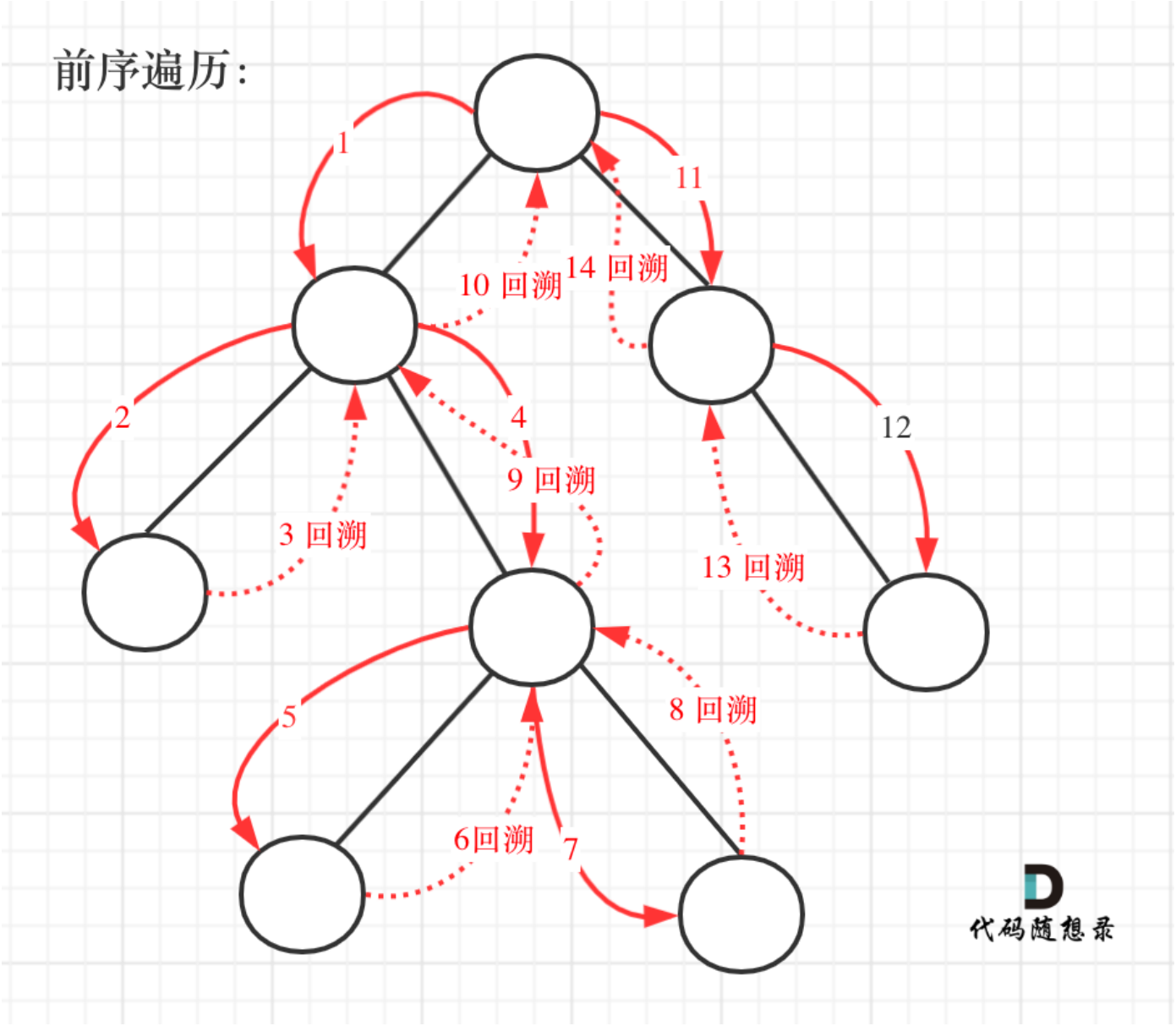
[《代码随想录》算法视频公开课](#)：：[递归中带着回溯，你感受到了没？| LeetCode：257. 二叉树的所有路径](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

这道题目要求从根节点到叶子的路径，所以需要前序遍历，这样才方便让父节点指向孩子节点，找到对应的路径。

在这道题目中将第一次涉及到回溯，因为我们要把路径记录下来，需要回溯来回退一个路径再进入另一个路径。

前序遍历以及回溯的过程如图：



我们先使用递归的方式，来做前序遍历。**要知道递归和回溯就是一家的，本题也需要回溯。**

递归

1. 递归函数参数以及返回值

要传入根节点，记录每一条路径的path，和存放结果集的result，这里递归不需要返回值，代码如下：

```
void traversal(TreeNode* cur, vector<int>& path, vector<string>& result)
```

2. 确定递归终止条件

在写递归的时候都习惯了这么写：

```
if (cur == NULL) {
    终止处理逻辑
}
```

但是本题的终止条件这样写会很麻烦，因为本题要找到叶子节点，就开始结束的处理逻辑了（把路径放进result里）。

那么什么时候算是找到了叶子节点？ 是当 cur不为空，其左右孩子都为空的时候，就找到叶子节点。

所以本题的终止条件是：

```
if (cur->left == NULL && cur->right == NULL) {
    终止处理逻辑
}
```

为什么没有判断cur是否为空呢，因为下面的逻辑可以控制空节点不入循环。

再来看一下终止处理的逻辑。

这里使用vector 结构path来记录路径，所以要把vector 结构的path转为string格式，再把这个string 放进 result里。

那么为什么使用了vector 结构来记录路径呢？ 因为在下面处理单层递归逻辑的时候，要做回溯，使用vector方便来做回溯。

可能有的同学问了，我看有些人的代码也没有回溯啊。

其实是有回溯的，只不过隐藏在函数调用时的参数赋值里，下文我还会提到。

这里我们先使用vector结构的path容器来记录路径，那么终止处理逻辑如下：

```
if (cur->left == NULL && cur->right == NULL) { // 遇到叶子节点
    string sPath;
    for (int i = 0; i < path.size() - 1; i++) { // 将path里记录的路径转为string格式
        sPath += to_string(path[i]);
        sPath += "->";
    }
    sPath += to_string(path[path.size() - 1]); // 记录最后一个节点（叶子节点）
    result.push_back(sPath); // 收集一个路径
    return;
}
```

3. 确定单层递归逻辑

因为是前序遍历，需要先处理中间节点，中间节点就是我们要记录路径上的节点，先放进path中。

`path.push_back(cur->val);`

然后是递归和回溯的过程，上面说过没有判断cur是否为空，那么在这里递归的时候，如果为空就不进行下一层递归了。

所以递归前要加上判断语句，下面要递归的节点是否为空，如下

```
if (cur->left) {
    traversal(cur->left, path, result);
}
if (cur->right) {
    traversal(cur->right, path, result);
}
```

此时还没完，递归完，要做回溯啊，因为path 不能一直加入节点，它还要删节点，然后才能加入新的节点。

那么回溯要怎么回溯呢，一些同学会这么写，如下：

```
if (cur->left) {
    traversal(cur->left, path, result);
}
if (cur->right) {
    traversal(cur->right, path, result);
}
path.pop_back();
```

这个回溯就有很大的问题，我们知道，**回溯和递归是一一对应的，有一个递归，就要有一个回溯**，这么写的话相当于把递归和回溯拆开了， 一个在花括号里，一个在花括号外。

所以回溯要和递归永远在一起，世界上最遥远的距离是你在花括号里，而我在花括号外！

那么代码应该这么写：

```
if (cur->left) {
    traversal(cur->left, path, result);
    path.pop_back(); // 回溯
}
if (cur->right) {
    traversal(cur->right, path, result);
    path.pop_back(); // 回溯
}
```

那么本题整体代码如下：

```
// 版本一
class Solution {
private:

    void traversal(TreeNode* cur, vector<int>& path, vector<string>& result) {
        path.push_back(cur->val); // 中，中为什么写在这里，因为最后一个节点也要加入到path中
        // 这才到了叶子节点
        if (cur->left == NULL && cur->right == NULL) {
            string sPath;
            for (int i = 0; i < path.size() - 1; i++) {
                sPath += to_string(path[i]);
                sPath += "->";
```

```
            }
            sPath += to_string(path[path.size() - 1]);
            result.push_back(sPath);
            return;
        }
        if (cur->left) { // 左
            traversal(cur->left, path, result);
            path.pop_back(); // 回溯
        }
        if (cur->right) { // 右
            traversal(cur->right, path, result);
            path.pop_back(); // 回溯
        }
    }

public:
    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> result;
        vector<int> path;
        if (root == NULL) return result;
        traversal(root, path, result);
        return result;
    }
};
```

如上的C++代码充分体现了回溯。

那么如上代码可以精简成如下代码：

```
class Solution {
private:

    void traversal(TreeNode* cur, string path, vector<string>& result) {
        path += to_string(cur->val); // 中
        if (cur->left == NULL && cur->right == NULL) {
            result.push_back(path);
            return;
        }
        if (cur->left) traversal(cur->left, path + "->", result); // 左
        if (cur->right) traversal(cur->right, path + "->", result); // 右
    }

public:
    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> result;
        string path;
        if (root == NULL) return result;
        traversal(root, path, result);
        return result;
```

```
    }
};
```

如上代码精简了不少，也隐藏了不少东西。

注意在函数定义的时候`void traversal(TreeNode* cur, string path, vector<string>& result)`，定义的是`string path`，每次都是复制赋值，不用使用引用，否则就无法做到回溯的效果。（这里涉及到C++语法知识）

那么在如上代码中，**貌似没有看到回溯的逻辑，其实不然，回溯就隐藏在`traversal(cur->left, path + "->", result);`中的 `path + "->"`。** 每次函数调用完，path依然是没有加上"->" 的，这就是回溯了。

为了把这份精简代码的回溯过程展现出来，大家可以试一试把：

```
if (cur->left) traversal(cur->left, path + "->", result); // 左   回溯就隐藏在这里
```

改成如下代码：

```
path += "->";
traversal(cur->left, path, result); // 左
```

即：

```
if (cur->left) {
    path += "->";
    traversal(cur->left, path, result); // 左
}
if (cur->right) {
    path += "->";
    traversal(cur->right, path, result); // 右
}
```

此时就没有回溯了，这个代码就是通过不了的了。

如果想把回溯加上，就要 在上面代码的基础上，加上回溯，就可以AC了。

```
if (cur->left) {
    path += "->";
    traversal(cur->left, path, result); // 左
    path.pop_back(); // 回溯 '>'
    path.pop_back(); // 回溯 '-'
}
if (cur->right) {
    path += "->";
    traversal(cur->right, path, result); // 右
    path.pop_back(); // 回溯 '>'
    path.pop_back(); //  回溯 '-'
}
```

整体代码如下：

```
//版本二
class Solution {
private:
    void traversal(TreeNode* cur, string path, vector<string>& result) {
        path += to_string(cur->val); // 中，中为什么写在这里，因为最后一个节点也要加入到path中
        if (cur->left == NULL && cur->right == NULL) {
            result.push_back(path);
            return;
        }
        if (cur->left) {
            path += "->";
            traversal(cur->left, path, result); // 左
            path.pop_back(); // 回溯 '>'
            path.pop_back(); // 回溯 '-'
        }
        if (cur->right) {
            path += "->";
            traversal(cur->right, path, result); // 右
            path.pop_back(); // 回溯'>'
            path.pop_back(); // 回溯 '-'
        }
    }

public:
    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> result;
        string path;
        if (root == NULL) return result;
        traversal(root, path, result);
        return result;

    }
};
```

大家应该可以感受出来，如果把 `path + "->"` 作为函数参数就是可以的，因为并没有改变path的数值，执行完递归函数之后，path依然是之前的数值（相当于回溯了）

综合以上，第二种递归的代码虽然精简但把很多重要的点隐藏在了代码细节里，第一种递归写法虽然代码多一些，但是把每一个逻辑处理都完整的展现出来了。

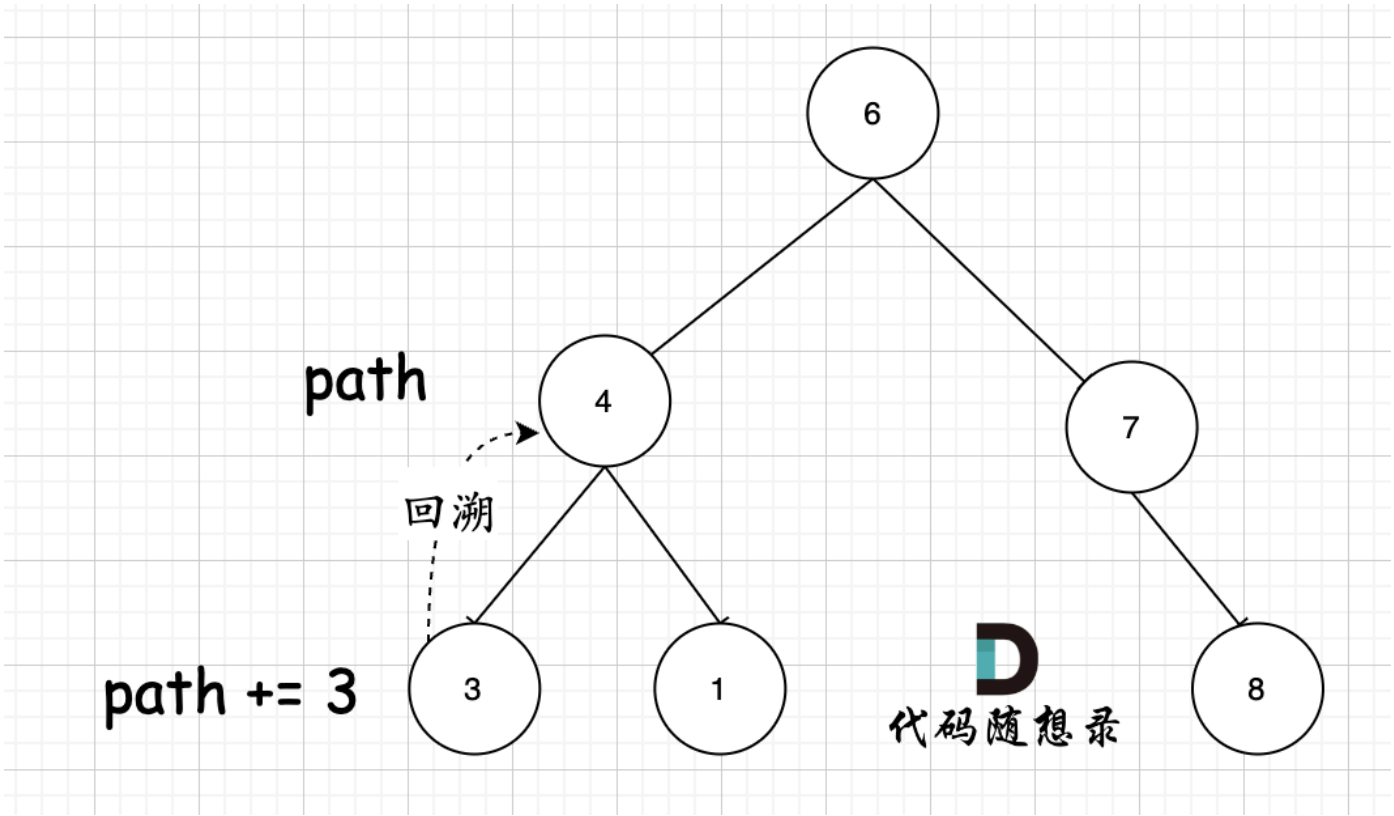
拓展

这里讲解本题解的写法逻辑以及一些更具体的细节，下面的讲解中，涉及到C++语法特性，如果不是C++的录友，就可以不看了，避免越看越晕。

如果是C++的录友，建议本题独立刷过两遍，再看下面的讲解，同样避免越看越晕，造成不必要的负担。

在第二版本的代码中，其实仅仅是回溯了 `->` 部分（调用两次pop_back，一个pop`>` 一次pop`-`），大家应该疑惑那么 `path += to_string(cur->val);` 这一步为什么没有回溯呢？ 一条路径能持续加节点 不做回溯吗?

其实关键还在于 参数，使用的是 `string path`，这里并没有加上引用`&` ，即本层递归中，path + 该节点数值，但该层递归结束，上一层path的数值并不会受到任何影响。 如图所示：



节点4 的path，在遍历到节点3，path+3，遍历节点3的递归结束之后，返回节点4（回溯的过程），path并不会把3加上。

所以这是参数中，不带引用，不做地址拷贝，只做内容拷贝的效果。（这里涉及到C++引用方面的知识）

在第一个版本中，函数参数我就使用了引用，即 `vector<int>& path` ，这是会拷贝地址的，所以 本层递归逻辑如果有`path.push_back(cur->val);` 就一定要有对应的 `path.pop_back()`

那有同学可能想，为什么不去定义一个 `string& path` 这样的函数参数呢，然后也可能在递归函数中展现回溯的过程，但关键在于，`path += to_string(cur->val);` 每次是加上一个数字，这个数字如果是个位数，那好说，就调用一次`path.pop_back()`，但如果是 十位数，百位数，千位数呢？ 百位数就要调用三次`path.pop_back()`，才能实现对应的回溯操作，这样代码实现就太冗余了。

所以，第一个代码版本中，我才使用 vector 类型的path，这样方便给大家演示代码中回溯的操作。 vector类型的path，不管 每次 路径收集的数字是几位数，总之一定是int，所以就一次 pop_back就可以。

迭代法

至于非递归的方式，我们可以依然可以使用前序遍历的迭代方式来模拟遍历路径的过程，对该迭代方式不了解的同学，可以看文章二叉树：听说递归能做的，栈也能做！和二叉树：前中后序迭代方式统一写法。

这里除了模拟递归需要一个栈，同时还需要一个栈来存放对应的遍历路径。

C++代码如下：

```
class Solution {
public:
    vector<string> binaryTreePaths(TreeNode* root) {
        stack<TreeNode*> treeSt;// 保存树的遍历节点
        stack<string> pathSt;   // 保存遍历路径的节点
        vector<string> result;  // 保存最终路径集合
        if (root == NULL) return result;
        treeSt.push(root);
        pathSt.push(to_string(root->val));
        while (!treeSt.empty()) {
            TreeNode* node = treeSt.top(); treeSt.pop(); // 取出节点 中
            string path = pathSt.top();pathSt.pop();    // 取出该节点对应的路径
            if (node->left == NULL && node->right == NULL) { // 遇到叶子节点
                result.push_back(path);
            }
            if (node->right) { // 右
                treeSt.push(node->right);
                pathSt.push(path + "->" + to_string(node->right->val));
            }
            if (node->left) { // 左
                treeSt.push(node->left);
                pathSt.push(path + "->" + to_string(node->left->val));
            }
        }
        return result;
    }
};
```

当然，使用java的同学，可以直接定义一个成员变量为object的栈`Stack<Object> stack = new Stack<>();`，这样就不用定义两个栈了，都放到一个栈里就可以了。

总结

本文我们开始初步涉及到了回溯，很多同学过了这道题目，可能都不知道自己其实使用了回溯，回溯和递归都是相伴相生的。

我在第一版递归代码中，把递归与回溯的细节都充分的展现了出来，大家可以自己感受一下。

第二版递归代码对于初学者其实非常不友好，代码看上去简单，但是隐藏细节于无形。

最后我依然给出了迭代法。

对于本题充分了解递归与回溯的过程之后，有精力的同学可以再去实现迭代法。

---

14. 本周小结！（二叉树系列二）

本周赶上了十一国庆，估计大家已经对本周末没什么概念了，但是我们该做总结还是要做总结的。

本周的主题其实是**简单但并不简单**，本周所选的题目大多是看一下就会的题目，但是大家看完本周的文章估计也发现了，二叉树的简单题目其实里面都藏了很多细节。 这些细节我都给大家展现了出来。

周一

本周刚开始我们讲解了判断二叉树是否对称的写法， 二叉树：我对称么？ 。

这道题目的本质是要比较两个树（这两个树是根节点的左右子树），遍历两棵树而且要比较内侧和外侧节点，所以准确的来说是一个树的遍历顺序是左右中，一个树的遍历顺序是右左中。

而本题的迭代法中我们使用了队列，需要注意的是这不是层序遍历，而且仅仅通过一个容器来成对的存放我们要比较的元素，认识到这一点之后就发现：用队列，用栈，甚至用数组，都是可以的。

那么做完本题之后，在看如下两个题目。

- 100.相同的树
- 572.另一个树的子树

二叉树：我对称么？中的递归法和迭代法只需要稍作修改其中一个树的遍历顺序，便可刷了100.相同的树。

100.相同的树的递归代码如下：

```
class Solution {
public:
    bool compare(TreeNode* left, TreeNode* right) {
        // 首先排除空节点的情况
        if (left == NULL && right != NULL) return false;
        else if (left != NULL && right == NULL) return false;
        else if (left == NULL && right == NULL) return true;
        // 排除了空节点，再排除数值不相同的情况
        else if (left->val != right->val) return false;

        // 此时就是：左右节点都不为空，且数值相同的情况
```

```
        // 此时才做递归，做下一层的判断
        bool outside = compare(left->left, right->left);   // 左子树：左、 右子树：左  （相对于求对称二叉树，只需改一下这里的顺序）
        bool inside = compare(left->right, right->right);    // 左子树：右、 右子树：右
        bool isSame = outside && inside;                    // 左子树：中、 右子树：中  （逻辑处理）
        return isSame;

    }
    bool isSymmetric(TreeNode* p, TreeNode* q) {
        return compare(p, q);
    }
};
```

100.相同的树，精简之后代码如下：

```
class Solution {
public:
    bool compare(TreeNode* left, TreeNode* right) {
        if (left == NULL && right != NULL) return false;
        else if (left != NULL && right == NULL) return false;
        else if (left == NULL && right == NULL) return true;
        else if (left->val != right->val) return false;
        else return compare(left->left, right->left) && compare(left->right, right->right);

    }
    bool isSameTree(TreeNode* p, TreeNode* q) {
        return compare(p, q);
    }
};
```

100.相同的树，迭代法代码如下：

```
class Solution {
public:

    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == NULL && q == NULL) return true;
        if (p == NULL || q == NULL) return false;
        queue<TreeNode*> que;
        que.push(p);
        que.push(q);
        while (!que.empty()) {
            TreeNode* leftNode = que.front(); que.pop();
            TreeNode* rightNode = que.front(); que.pop();
            if (!leftNode && !rightNode) {
                continue;
```

```
            }
            if ((!leftNode || !rightNode || (leftNode->val != rightNode->val))) {
                return false;
            }
            // 相对于求对称二叉树，这里两个树都要保持一样的遍历顺序
            que.push(leftNode->left);
            que.push(rightNode->left);
            que.push(leftNode->right);
            que.push(rightNode->right);
        }
        return true;
    }
};
```

而572.另一个树的子树，则和 100.相同的树几乎一样的了，大家可以直接AC了。

周二

在二叉树：看看这些树的最大深度中，我们讲解了如何求二叉树的最大深度。

本题可以使用前序，也可以使用后序遍历（左右中），使用前序求的就是深度，使用后序呢求的是高度。

而根节点的高度就是二叉树的最大深度，所以本题中我们通过后序求的根节点高度来求的二叉树最大深度，所以二叉树：看看这些树的最大深度中使用的是后序遍历。

本题当然也可以使用前序，代码如下：**(充分表现出求深度回溯的过程)**

```
class Solution {
public:
    int result;
    void getDepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中

        if (node->left == NULL && node->right == NULL) return ;

        if (node->left) { // 左
            depth++;    // 深度+1
            getDepth(node->left, depth);
            depth--;    // 回溯，深度-1
        }
        if (node->right) { // 右
            depth++;    // 深度+1
            getDepth(node->right, depth);
            depth--;    // 回溯，深度-1
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
```

```
        if (root == 0) return result;
        getDepth(root, 1);
        return result;
    }
};
```

可以看出使用了前序（中左右）的遍历顺序，这才是真正求深度的逻辑！

注意以上代码是为了把细节体现出来，简化一下代码如下：

```
class Solution {
public:
    int result;
    void getDepth(TreeNode* node, int depth) {
        result = depth > result ? depth : result; // 中
        if (node->left == NULL && node->right == NULL) return ;
        if (node->left) { // 左
            getDepth(node->left, depth + 1);
        }
        if (node->right) { // 右
            getDepth(node->right, depth + 1);
        }
        return ;
    }
    int maxDepth(TreeNode* root) {
        result = 0;
        if (root == 0) return result;
        getDepth(root, 1);
        return result;
    }
};
```

周三

在二叉树：看看这些树的最小深度中，我们讲解如何求二叉树的最小深度， 这道题目要是稍不留心很容易犯错。

注意这里最小深度是从根节点到最近叶子节点的最短路径上的节点数量。注意是叶子节点。

什么是叶子节点，左右孩子都为空的节点才是叶子节点！

求二叉树的最小深度和求二叉树的最大深度的差别主要在于处理左右孩子不为空的逻辑。

注意到这一点之后 递归法和迭代法 都可以参照二叉树：看看这些树的最大深度写出来。

周四

我们在二叉树：我有多少个节点？ 中，讲解了如何求二叉树的节点数量。

这一天是十一长假的第一天，又是双节，所以简单一些，只要把之前两篇二叉树：看看这些树的最大深度， 二叉树：看看这些树的最小深度都认真看了的话，这道题目可以分分钟刷掉了。

估计此时大家对这一类求二叉树节点数量以及求深度应该非常熟练了。

周五

在二叉树：我平衡么？ 中讲解了如何判断二叉树是否是平衡二叉树

今天讲解一道判断平衡二叉树的题目，其实 方法上我们之前讲解深度的时候都讲过了，但是这次我们通过这道题目彻底搞清楚二叉树高度与深度的问题，以及对应的遍历方式。

二叉树节点的深度：指从根节点到该节点的最长简单路径边的条数。
二叉树节点的高度：指从该节点到叶子节点的最长简单路径边的条数。

但leetcode中强调的深度和高度很明显是按照节点来计算的。

关于根节点的深度究竟是1 还是 0，不同的地方有不一样的标准，leetcode的题目中都是以节点为一度，即根节点深度是1。但维基百科上定义用边为一度，即根节点的深度是0，我们暂时以leetcode为准（毕竟要在这上面刷题）。

当然此题用迭代法，其实效率很低，因为没有很好的模拟回溯的过程，所以迭代法有很多重复的计算。

虽然理论上所有的递归都可以用迭代来实现，但是有的场景难度可能比较大。

例如：都知道回溯法其实就是递归，但是很少人用迭代的方式去实现回溯算法！

讲了这么多二叉树题目的迭代法，有的同学会疑惑，迭代法中究竟什么时候用队列，什么时候用栈？

如果是模拟前中后序遍历就用栈，如果是适合层序遍历就用队列，当然还是其他情况，那么就是 先用队列试试行不行，不行就用栈。

周六

在二叉树：找我的所有路径？ 中正式涉及到了回溯，很多同学过了这道题目，可能都不知道自己使用了回溯，其实回溯和递归都是相伴相生的。最后我依然给出了迭代法的版本。

我在题解中第一个版本的代码会把回溯的过程充分体现出来，如果大家直接看简洁的代码版本，很可能就会忽略的回溯的存在。

我在文中也强调了这一点。

有的同学还不理解 ，文中精简之后的递归代码，回溯究竟隐藏在哪里了。

文中我明确的说了：**回溯就隐藏在traversal(cur->left, path + "->", result);中的 path + "->"。 每次函数调用完，path依然是没有加上"->" 的，这就是回溯了。**

如果还不理解的话，可以把

```
traversal(cur->left, path + "->", result);
```

改成

```
string tmp = path + "->";
traversal(cur->left, tmp, result);
```

看看还行不行了，答案是这么写就不行了，因为没有回溯了。

总结

二叉树的题目，我都是使用了递归三部曲一步一步的把整个过程分析出来，而不是上来就给出简洁的代码。

一些同学可能上来就能写出代码，大体上也知道是为啥，可以自圆其说，但往细节一扣，就不知道了。

所以刚接触二叉树的同学，建议按照文章分析的步骤一步一步来，不要上来就照着精简的代码写（那样写完了也很容易忘的，知其然不知其所以然）。

简短的代码看不出遍历的顺序，也看不出分析的逻辑，还会把必要的回溯的逻辑隐藏了，所以尽量按照原理分析一步一步来，写出来之后，再去优化代码。

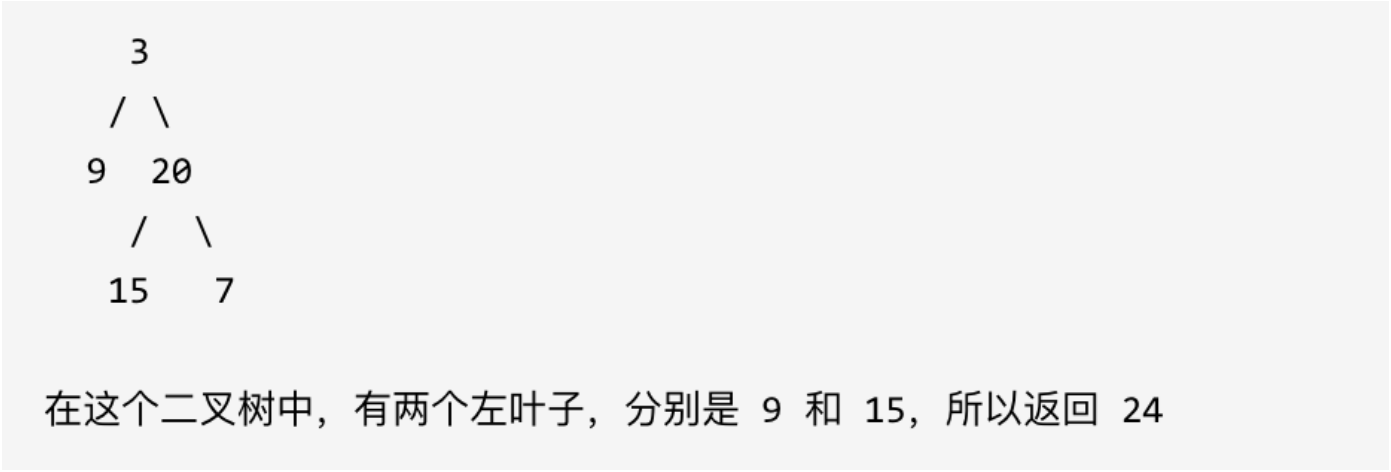
大家加个油！！

15.左叶子之和

力扣题目链接

计算给定二叉树的所有左叶子之和。

示例：



算法公开课

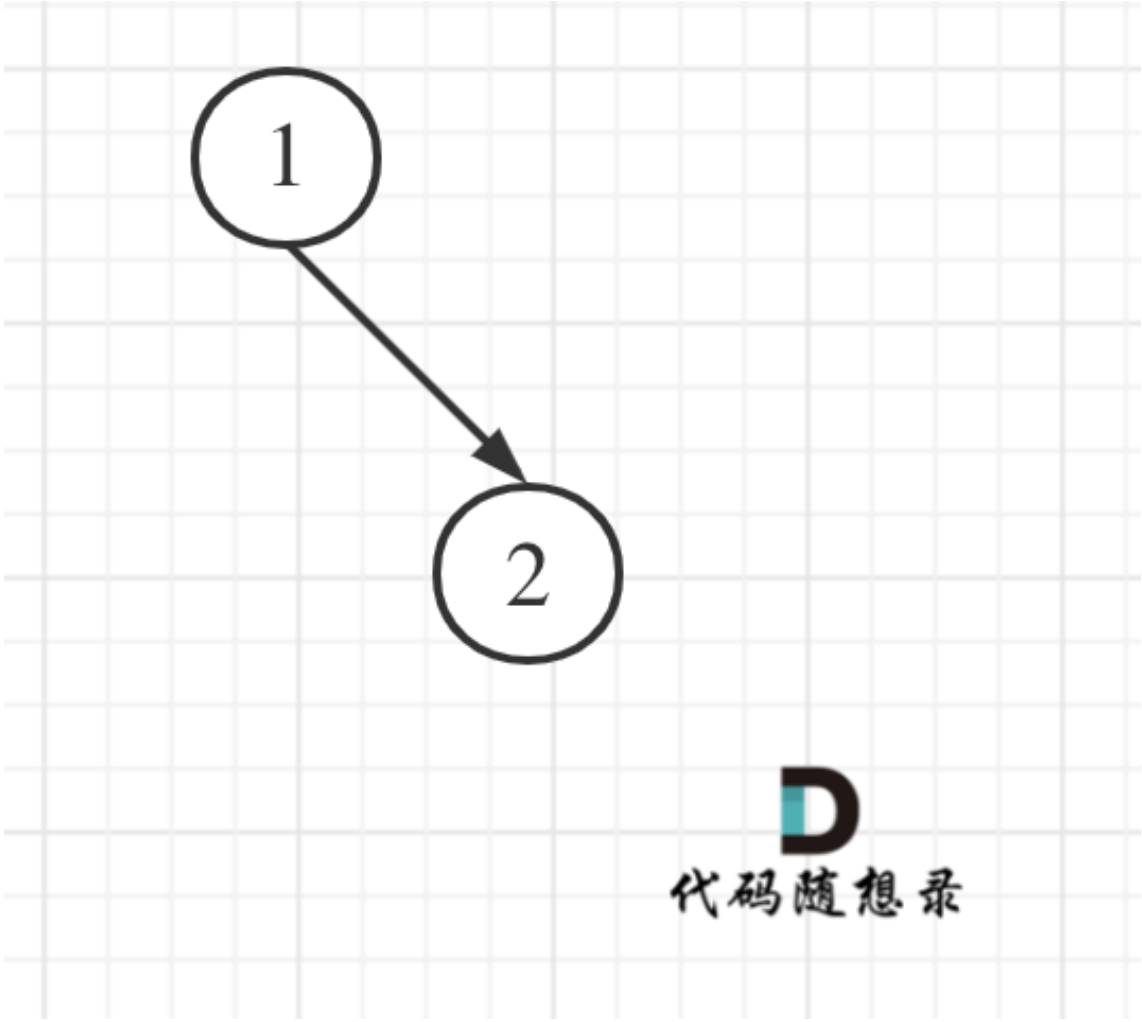
《代码随想录》算法视频公开课：：二叉树的题目中，总有一些规则让你找不到北 | LeetCode：404.左叶子之和，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

首先要注意是判断左叶子，不是二叉树左侧节点，所以不要上来想着层序遍历。

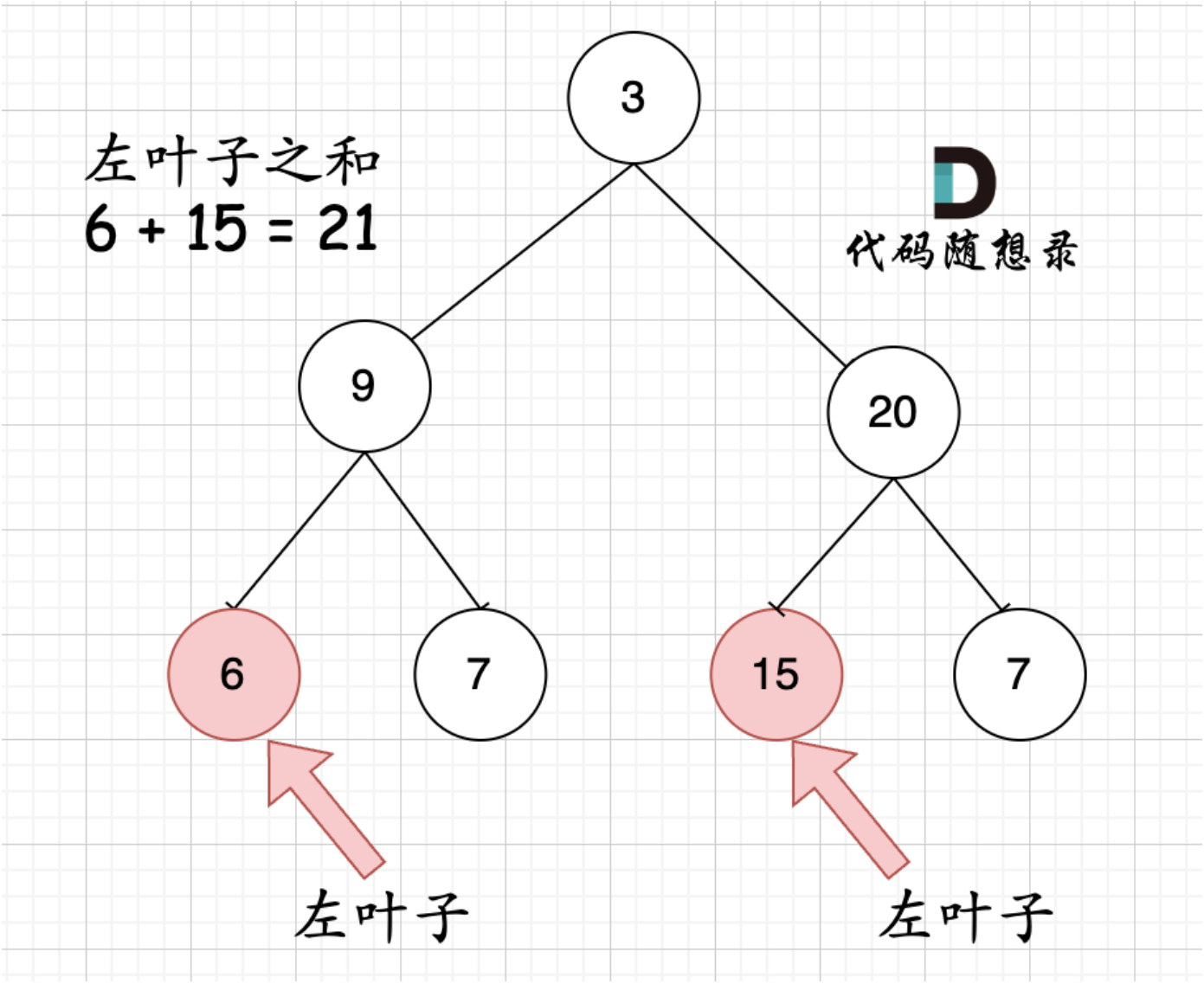
因为题目中其实没有说清楚左叶子究竟是什么节点，那么我来给出左叶子的明确定义：**节点A的左孩子不为空，且左孩子的左右孩子都为空（说明是叶子节点），那么A节点的左孩子为左叶子节点**

大家思考一下如下图中二叉树，左叶子之和究竟是多少？



其实是0，因为这棵树根本没有左叶子！

但看这个图的左叶子之和是多少？



相信通过这两个图，大家对最左叶子的定义有明确理解了。

那么**判断当前节点是不是左叶子是无法判断的，必须要通过节点的父节点来判断其左孩子是不是左叶子。**

如果该节点的左节点不为空，该节点的左节点的左节点为空，该节点的左节点的右节点为空，则找到了一个左叶子，判断代码如下：

```
if (node->left != NULL && node->left->left == NULL && node->left->right == NULL) {
    左叶子节点处理逻辑
}
```

递归法

递归的遍历顺序为后序遍历（左右中），是因为要通过递归函数的返回值来累加求取左叶子数值之和。

递归三部曲：

1. 确定递归函数的参数和返回值

判断一个树的左叶子节点之和，那么一定要传入树的根节点，递归函数的返回值为数值之和，所以为int

使用题目中给出的函数就可以了。

2. 确定终止条件

如果遍历到空节点，那么左叶子值一定是0

```
if (root == NULL) return 0;
```

注意，只有当前遍历的节点是父节点，才能判断其子节点是不是左叶子。 所以如果当前遍历的节点是叶子节点，那其左叶子也必定是0，那么终止条件为：

```
if (root == NULL) return 0;
if (root->left == NULL && root->right== NULL) return 0; //其实这个也可以不写，如果不写不影响结果，但就会让递归多进行了一层。
```

3. 确定单层递归的逻辑

当遇到左叶子节点的时候，记录数值，然后通过递归求取左子树左叶子之和，和 右子树左叶子之和，相加便是整个树的左叶子之和。

代码如下：

```
int leftValue = sumOfLeftLeaves(root->left);    // 左
if (root->left && !root->left->left && !root->left->right) {
    leftValue = root->left->val;
}
int rightValue = sumOfLeftLeaves(root->right);  // 右

int sum = leftValue + rightValue;               // 中
return sum;
```

整体递归代码如下：

```
class Solution {
public:
    int sumOfLeftLeaves(TreeNode* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right== NULL) return 0;

        int leftValue = sumOfLeftLeaves(root->left);    // 左
        if (root->left && !root->left->left && !root->left->right) { // 左子树就是一个左叶子的情况
            leftValue = root->left->val;
        }
        int rightValue = sumOfLeftLeaves(root->right);  // 右

        int sum = leftValue + rightValue;               // 中
        return sum;
    }
};
```

以上代码精简之后如下：

```
class Solution {
public:
    int sumOfLeftLeaves(TreeNode* root) {
        if (root == NULL) return 0;
        int leftValue = 0;
        if (root->left != NULL && root->left->left == NULL && root->left->right ==
NULL) {
            leftValue = root->left->val;
        }
        return leftValue + sumOfLeftLeaves(root->left) + sumOfLeftLeaves(root->right);
    }
};
```

精简之后的代码其实看不出来用的是什么遍历方式了，对于算法初学者以上根据第一个版本来学习。

迭代法

本题迭代法使用前中后序都是可以的，只要把左叶子节点统计出来，就可以了，那么参考文章 二叉树：听说递归能做的，栈也能做！和二叉树：迭代法统一写法中的写法，可以写出一个前序遍历的迭代法。

判断条件都是一样的，代码如下：

```
class Solution {
public:
    int sumOfLeftLeaves(TreeNode* root) {
        stack<TreeNode*> st;
        if (root == NULL) return 0;
        st.push(root);
        int result = 0;
        while (!st.empty()) {
            TreeNode* node = st.top();
            st.pop();
            if (node->left != NULL && node->left->left == NULL && node->left->right ==
NULL) {
                result += node->left->val;
            }
            if (node->right) st.push(node->right);
            if (node->left) st.push(node->left);
        }
        return result;
    }
};
```

总结

这道题目要求左叶子之和，其实是比较绕的，因为不能判断本节点是不是左叶子节点。

此时就要通过节点的父节点来判断其左孩子是不是左叶子了。

平时我们解二叉树的题目时，已经习惯了通过节点的左右孩子判断本节点的属性，而本题我们要通过节点的父节点判断本节点的属性。

希望通过这道题目，可以扩展大家对二叉树的解题思路。

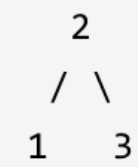
16.找树左下角的值

[力扣题目链接]

给定一个二叉树，在树的最后一行找到最左边的值。

示例 1:

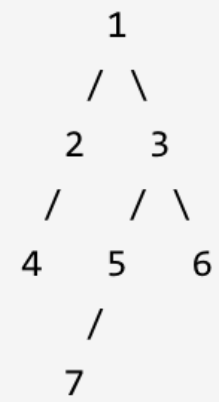
输入：



输出：
1

示例 2:

输入：



输出：
7

算法公开课

[《代码随想录》算法视频公开课](#)：[怎么找二叉树的左下角？ 递归中又带回溯了，怎么办？ | LeetCode：513.找二叉树左下角的值](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

本题要找出树的最后一行的最左边的值。此时大家应该想起用层序遍历是非常简单的了，反而用递归的话会比较难一点。

我们依然还是先介绍递归法。

递归

咋眼一看，这道题目用递归的话就就一直向左遍历，最后一个就是答案呗?

没有这么简单，一直向左遍历到最后一个，它未必是最后一行啊。

我们来分析一下题目：在树的**最后一行**找到**最左边的值**。

首先要是最后一行，然后是最左边的值。

如果使用递归法，如何判断是最后一行呢，其实就是深度最大的叶子节点一定是最后一行。

如果对二叉树深度和高度还有点疑惑的话，请看：[110.平衡二叉树](#)。

所以要找深度最大的叶子节点。

那么如何找最左边的呢？可以使用前序遍历（当然中序，后序都可以，因为本题没有 中间节点的处理逻辑，只要左优先就行），保证优先左边搜索，然后记录深度最大的叶子节点，此时就是树的最后一行最左边的值。

递归三部曲：

1. 确定递归函数的参数和返回值

参数必须有要遍历的树的根节点，还有就是一个int型的变量用来记录最长深度。 这里就不需要返回值了，所以递归函数的返回类型为void。

本题还需要类里的两个全局变量，maxLen用来记录最大深度，result记录最大深度最左节点的数值。

代码如下：

```
int maxDepth = INT_MIN;    // 全局变量 记录最大深度
int result;        // 全局变量 最大深度最左节点的数值
void traversal(TreeNode* root, int depth)
```

2. 确定终止条件

当遇到叶子节点的时候，就需要统计一下最大的深度了，所以需要遇到叶子节点来更新最大深度。

代码如下：

```
if (root->left == NULL && root->right == NULL) {
    if (depth > maxDepth) {
        maxDepth = depth;            // 更新最大深度
        result = root->val;    // 最大深度最左面的数值
    }
    return;
}
```

3. 确定单层递归的逻辑

在找最大深度的时候，递归的过程中依然要使用回溯，代码如下：

```
                    // 中
if (root->left) {    // 左
    depth++; // 深度加一
    traversal(root->left, depth);
    depth--; // 回溯，深度减一
}
if (root->right) { // 右
    depth++; // 深度加一
    traversal(root->right, depth);
    depth--; // 回溯，深度减一
}
return;
```

完整代码如下：

```
class Solution {
public:
    int maxDepth = INT_MIN;
    int result;
    void traversal(TreeNode* root, int depth) {
        if (root->left == NULL && root->right == NULL) {
```

```
            if (depth > maxDepth) {
                maxDepth = depth;
                result = root->val;
            }
            return;
        }
        if (root->left) {
            depth++;
            traversal(root->left, depth);
            depth--; // 回溯
        }
        if (root->right) {
            depth++;
            traversal(root->right, depth);
            depth--; // 回溯
        }
        return;
    }
    int findBottomLeftValue(TreeNode* root) {
        traversal(root, 0);
        return result;
    }
};
```

当然回溯的地方可以精简，精简代码如下：

```
class Solution {
public:
    int maxDepth = INT_MIN;
    int result;
    void traversal(TreeNode* root, int depth) {
        if (root->left == NULL && root->right == NULL) {
            if (depth > maxDepth) {
                maxDepth = depth;
                result = root->val;
            }
            return;
        }
        if (root->left) {
            traversal(root->left, depth + 1); // 隐藏着回溯
        }
        if (root->right) {
            traversal(root->right, depth + 1); // 隐藏着回溯
        }
        return;
    }
    int findBottomLeftValue(TreeNode* root) {
        traversal(root, 0);
        return result;
```

```
    }
};
```

如果对回溯部分精简的代码 不理解的话，可以看这篇257. 二叉树的所有路径

迭代法

本题使用层序遍历再合适不过了，比递归要好理解得多！

只需要记录最后一行第一个节点的数值就可以了。

如果对层序遍历不了解，看这篇二叉树：层序遍历登场！，这篇里也给出了层序遍历的模板，稍作修改就一过刷了这道题了。

代码如下：

```
class Solution {
public:
    int findBottomLeftValue(TreeNode* root) {
        queue<TreeNode*> que;
        if (root != NULL) que.push(root);
        int result = 0;
        while (!que.empty()) {
            int size = que.size();
            for (int i = 0; i < size; i++) {
                TreeNode* node = que.front();
                que.pop();
                if (i == 0) result = node->val; // 记录最后一行第一个元素
                if (node->left) que.push(node->left);
                if (node->right) que.push(node->right);
            }
        }
        return result;
    }
};
```

总结

本题涉及如下几点：

- 递归求深度的写法，我们在110.平衡二叉树中详细的分析了深度应该怎么求，高度应该怎么求。
- 递归中其实隐藏了回溯，在257. 二叉树的所有路径中讲解了究竟哪里使用了回溯，哪里隐藏了回溯。
- 层次遍历，在二叉树：层序遍历登场！深度讲解了二叉树层次遍历。
  所以本题涉及到的点，我们之前都讲解过，这些知识点需要同学们灵活运用，这样就举一反三了。

---

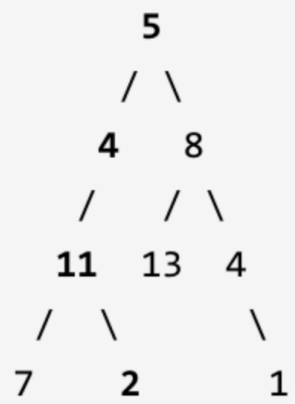
17. 路径总和

力扣题目链接

给定一个二叉树和一个目标和，判断该树中是否存在根节点到叶子节点的路径，这条路径上所有节点值相加等于目标和。

说明: 叶子节点是指没有子节点的节点。

示例:
给定如下二叉树，以及目标和 sum = 22，



返回 true, 因为存在目标和为 22 的根节点到叶子节点的路径 5->4->11->2。

算法公开课

《代码随想录》算法视频公开课：拿不准的遍历顺序，搞不清的回溯过程，我太难了！ | LeetCode：112. 路径总和，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

相信很多同学都会疑惑，递归函数什么时候要有返回值，什么时候没有返回值，特别是有的时候递归函数返回类型为bool类型。

那么接下来我通过详细讲解如下两道题，来回答这个问题：

- 112.路径总和
- 113.路径总和ii

这道题我们要遍历从根节点到叶子节点的路径看看总和是不是目标和。

递归

可以使用深度优先遍历的方式（本题前中后序都可以，无所谓，因为中节点也没有处理逻辑）来遍历二叉树

1. 确定递归函数的参数和返回类型

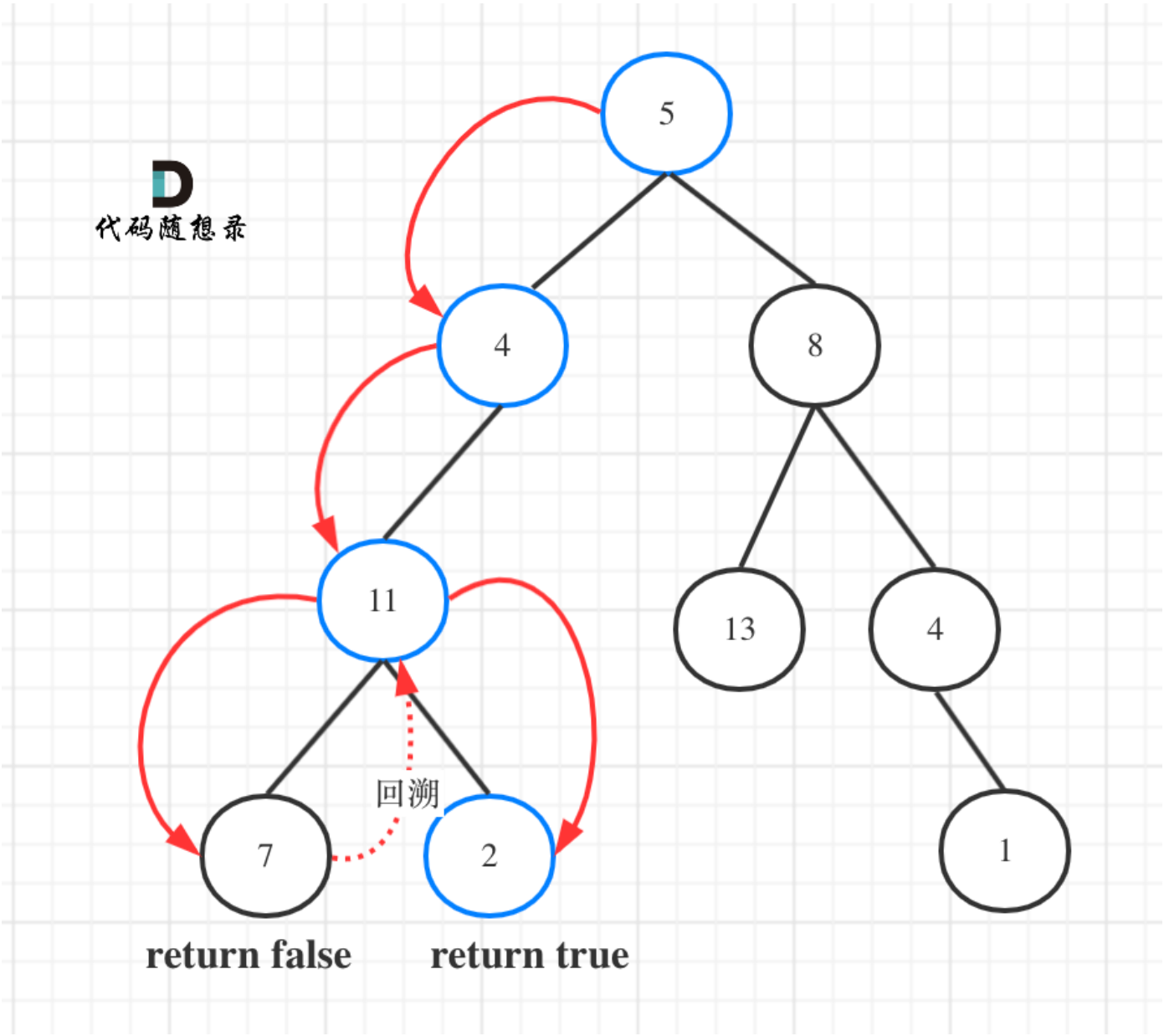
参数：需要二叉树的根节点，还需要一个计数器，这个计数器用来计算二叉树的一条边之和是否正好是目标和，计数器为int型。

再来看返回值，递归函数什么时候需要返回值？什么时候不需要返回值？这里总结如下三点：

- 如果需要搜索整棵二叉树且不用处理递归返回值，递归函数就不要返回值。（这种情况就是本文下半部分介绍的113.路径总和ii）
- 如果需要搜索整棵二叉树且需要处理递归返回值，递归函数就需要返回值。（这种情况我们在236. 二叉树的最近公共祖先中介绍）
- 如果要搜索其中一条符合条件的路径，那么递归一定需要返回值，因为遇到符合条件的路径了就要及时返回。（本题的情况）

而本题我们要找一条符合条件的路径，所以递归函数需要返回值，及时返回，那么返回类型是什么呢？

如图所示：



图中可以看出，遍历的路线，并不要遍历整棵树，所以递归函数需要返回值，可以用bool类型表示。

所以代码如下：

```
bool traversal(treenode* cur, int count)   // 注意函数的返回类型
```

2. 确定终止条件

首先计数器如何统计这一条路径的和呢?

不要去累加然后判断是否等于目标和，那么代码比较麻烦，可以用递减，让计数器count初始为目标和，然后每次减去遍历路径节点上的数值。

如果最后count == 0，同时到了叶子节点的话，说明找到了目标和。

如果遍历到了叶子节点，count不为0，就是没找到。

递归终止条件代码如下：

```
if (!cur->left && !cur->right && count == 0) return true; // 遇到叶子节点，并且计数为0
if (!cur->left && !cur->right) return false; // 遇到叶子节点而没有找到合适的边，直接返回
```

3. 确定单层递归的逻辑

因为终止条件是判断叶子节点，所以递归的过程中就不要让空节点进入递归了。

递归函数是有返回值的，如果递归函数返回true，说明找到了合适的路径，应该立刻返回。

代码如下：

```
if (cur->left) { // 左 （空节点不遍历）
    // 遇到叶子节点返回true，则直接返回true
    if (traversal(cur->left, count - cur->left->val)) return true; // 注意这里有回溯的逻辑
}
if (cur->right) { // 右 （空节点不遍历）
    // 遇到叶子节点返回true，则直接返回true
    if (traversal(cur->right, count - cur->right->val)) return true; // 注意这里有回溯的逻
辑
}
return false;
```

以上代码中是包含着回溯的，没有回溯，如何后撤重新找另一条路径呢。

回溯隐藏在`traversal(cur->left, count - cur->left->val)`这里， 因为把`count - cur->left->val`直接作为参数传进去，函数结束，count的数值没有改变。

为了把回溯的过程体现出来，可以改为如下代码：

```
if (cur->left) { // 左
    count -= cur->left->val; // 递归，处理节点;
    if (traversal(cur->left, count)) return true;
    count += cur->left->val; // 回溯，撤销处理结果
}
if (cur->right) { // 右
    count -= cur->right->val;
    if (traversal(cur->right, count)) return true;
    count += cur->right->val;
}
return false;
```

整体代码如下：

```
class Solution {
private:
    bool traversal(TreeNode* cur, int count) {
        if (!cur->left && !cur->right && count == 0) return true; // 遇到叶子节点，并且计数为0
        if (!cur->left && !cur->right) return false; // 遇到叶子节点直接返回

        if (cur->left) { // 左
            count -= cur->left->val; // 递归，处理节点;
            if (traversal(cur->left, count)) return true;
            count += cur->left->val; // 回溯，撤销处理结果
        }
        if (cur->right) { // 右
            count -= cur->right->val; // 递归，处理节点;
            if (traversal(cur->right, count)) return true;
            count += cur->right->val; // 回溯，撤销处理结果
        }
        return false;
    }

public:
    bool hasPathSum(TreeNode* root, int sum) {
        if (root == NULL) return false;
        return traversal(root, sum - root->val);
    }
};
```

以上代码精简之后如下：

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int sum) {
        if (!root) return false;
        if (!root->left && !root->right && sum == root->val) {
            return true;
        }
        return hasPathSum(root->left, sum - root->val) || hasPathSum(root->right, sum -
root->val);
    }
};
```

是不是发现精简之后的代码，已经完全看不出分析的过程了，所以我们要把题目分析清楚之后，再追求代码精简。
这一点我已经强调很多次了！

迭代

如果使用栈模拟递归的话，那么如果做回溯呢？

此时栈里一个元素不仅要记录该节点指针，还要记录从头结点到该节点的路径数值总和。

c++就我们用pair结构来存放这个栈里的元素。

定义为：`pair<TreeNode*, int>` pair<节点指针，路径数值>

这个为栈里的一个元素。

如下代码是使用栈模拟的前序遍历，如下：（详细注释）

```
class solution {

public:
    bool haspathsum(TreeNode* root, int sum) {
        if (root == null) return false;
        // 此时栈里要放的是pair<节点指针，路径数值>
        stack<pair<TreeNode*, int>> st;
        st.push(pair<TreeNode*, int>(root, root->val));
        while (!st.empty()) {
            pair<TreeNode*, int> node = st.top();
            st.pop();
            // 如果该节点是叶子节点了，同时该节点的路径数值等于sum，那么就返回true
            if (!node.first->left && !node.first->right && sum == node.second) return
true;

            // 右节点，压进去一个节点的时候，将该节点的路径数值也记录下来
            if (node.first->right) {
                st.push(pair<TreeNode*, int>(node.first->right, node.second +
node.first->right->val));
            }
```

```
            // 左节点，压进去一个节点的时候，将该节点的路径数值也记录下来
            if (node.first->left) {
                st.push(pair<TreeNode*, int>(node.first->left, node.second +
node.first->left->val));
            }
        }
        return false;
    }
};
```

如果大家完全理解了本题的递归方法之后，就可以顺便把leetcode上113. 路径总和ii做了。

相关题目推荐

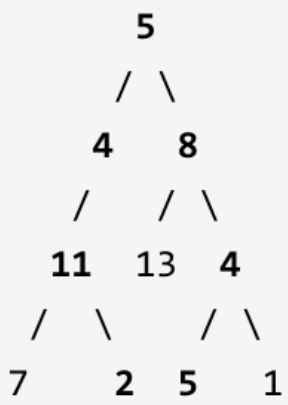
路径总和ii

[力扣题目链接]

给定一个二叉树和一个目标和，找到所有从根节点到叶子节点路径总和等于给定目标和的路径。

说明: 叶子节点是指没有子节点的节点。

示例:
给定如下二叉树，以及目标和 sum = 22，



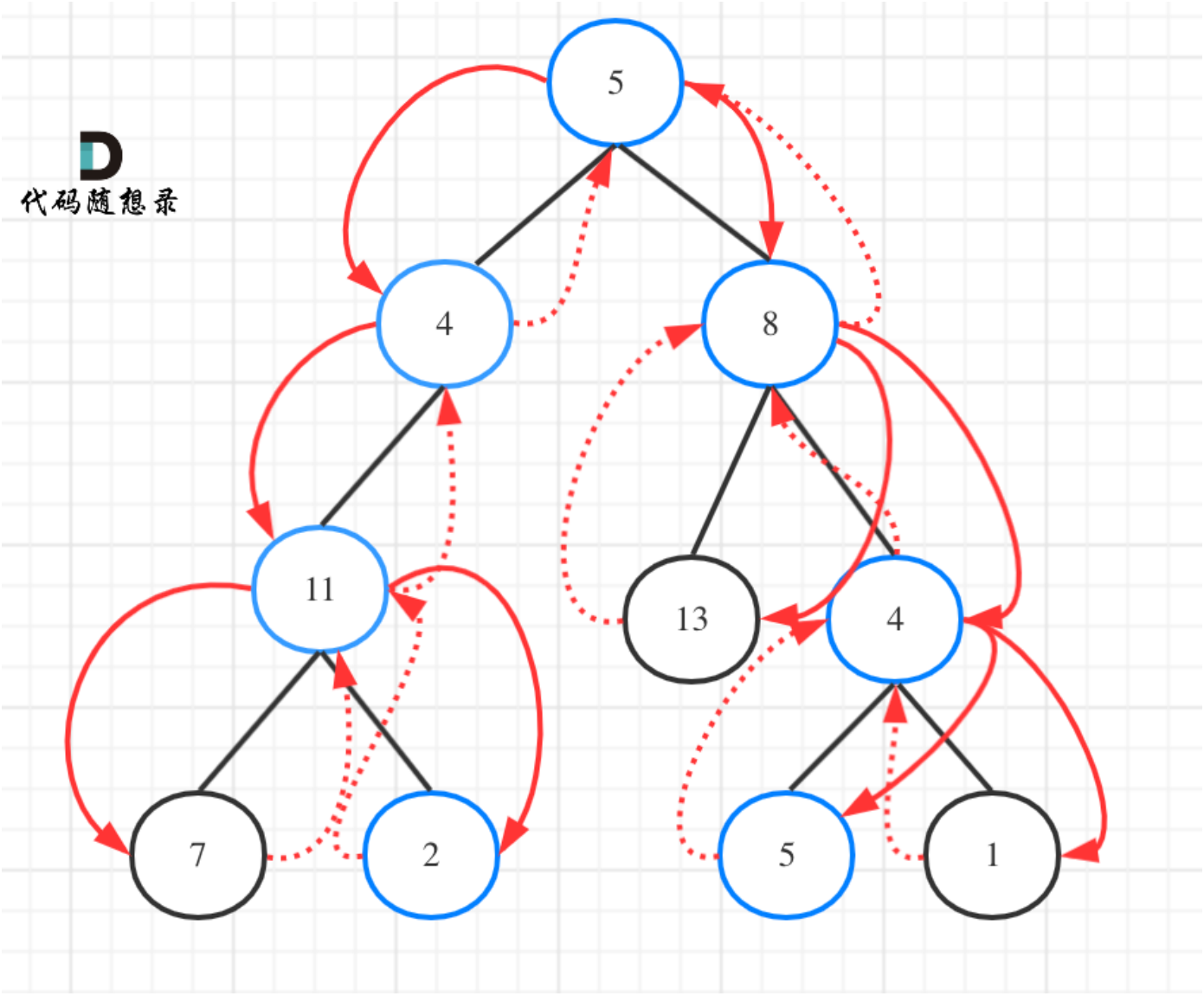
返回:

```
[
   [5,4,11,2],
   [5,8,4,5]
]
```

思路

113.路径总和ii要遍历整个树，找到所有路径，**所以递归函数不要返回值！**

如图：



为了尽可能的把细节体现出来，我写出如下代码（**这份代码并不简洁，但是逻辑非常清晰**）

```
class solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    // 递归函数不需要返回值，因为我们要遍历整个树
    void traversal(TreeNode* cur, int count) {
        if (!cur->left && !cur->right && count == 0) { // 遇到了叶子节点且找到了和为sum的路径
            result.push_back(path);
            return;
        }

        if (!cur->left && !cur->right) return ; // 遇到叶子节点而没有找到合适的边，直接返回
```

```
        if (cur->left) { // 左 （空节点不遍历）
            path.push_back(cur->left->val);
            count -= cur->left->val;
            traversal(cur->left, count);    // 递归
            count += cur->left->val;        // 回溯
            path.pop_back();                // 回溯
        }
        if (cur->right) { // 右 （空节点不遍历）
            path.push_back(cur->right->val);
            count -= cur->right->val;
            traversal(cur->right, count);   // 递归
            count += cur->right->val;       // 回溯
            path.pop_back();                // 回溯
        }
        return ;
    }

public:
    vector<vector<int>> pathSum(TreeNode* root, int sum) {
        result.clear();
        path.clear();
        if (root == NULL) return result;
        path.push_back(root->val); // 把根节点放进路径
        traversal(root, sum - root->val);
        return result;
    }
};
```

至于113. 路径总和ii 的迭代法我并没有写，用迭代方式记录所有路径比较麻烦，也没有必要，如果大家感兴趣的话，可以再深入研究研究。

总结

本篇通过leetcode上112. 路径总和 和 113. 路径总和ii 详细的讲解了 递归函数什么时候需要返回值，什么不需要返回值。

这两道题目是掌握这一知识点非常好的题目，大家看完本篇文章再去做题，就会感受到搜索整棵树和搜索某一路径的差别。

对于112. 路径总和，我依然给出了递归法和迭代法，这种题目其实用迭代法会复杂一些，能掌握递归方式就够了！

看完本文，可以一起解决如下两道题目

- 106.从中序与后序遍历序列构造二叉树
- 105.从前序与中序遍历序列构造二叉树

18.从中序与后序遍历序列构造二叉树

力扣题目链接

根据一棵树的中序遍历与后序遍历构造二叉树。

注意:
你可以假设树中没有重复的元素。

例如，给出

- 中序遍历 inorder = [9,3,15,20,7]
- 后序遍历 postorder = [9,15,7,20,3]
  返回如下的二叉树：

```
    3
   / \
  9  20
    /  \
   15   7
```

算法公开课

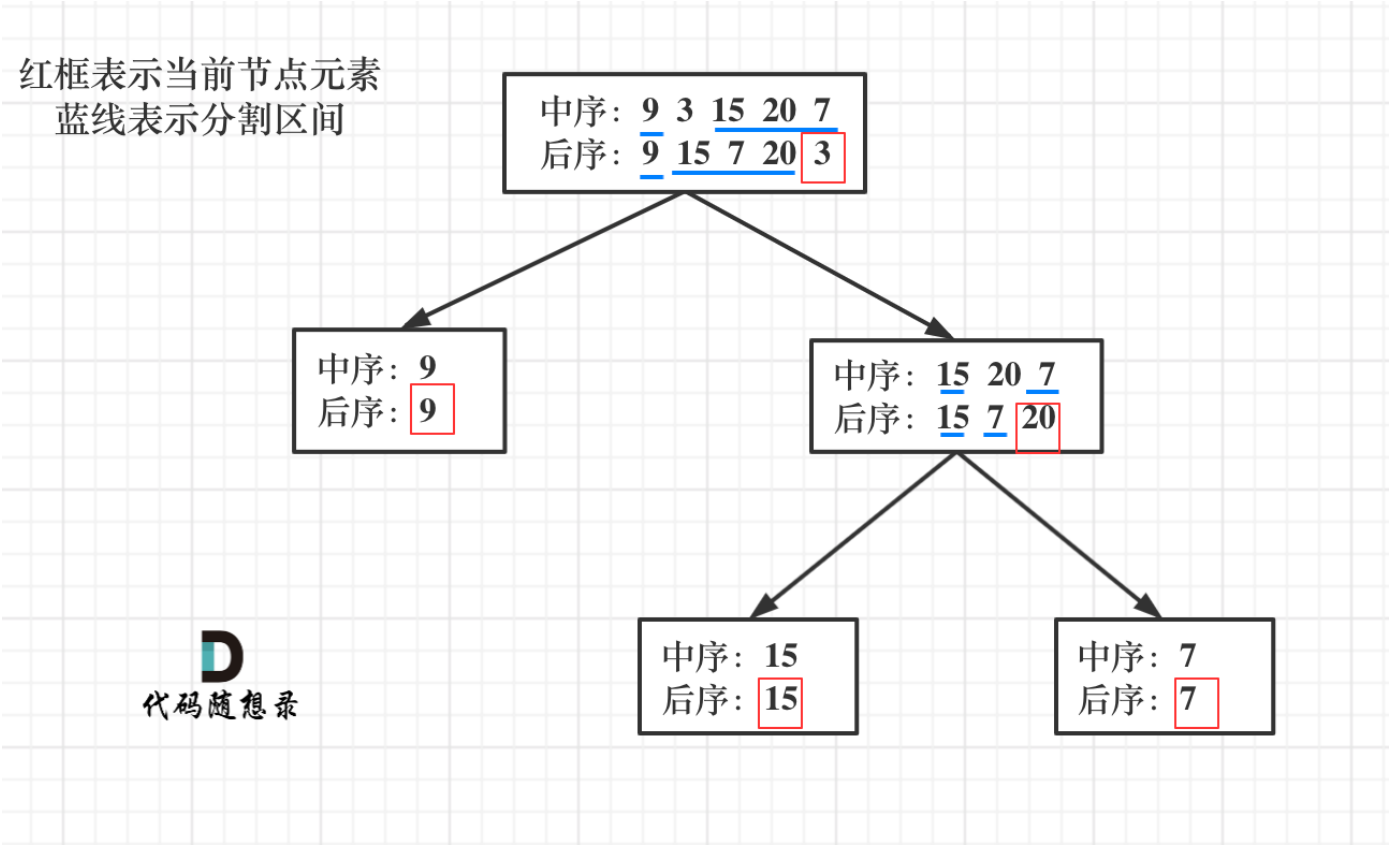
《代码随想录》算法视频公开课：坑很多！来看看你掉过几次坑 | LeetCode：106.从中序与后序遍历序列构造二叉树，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

首先回忆一下如何根据两个顺序构造一个唯一的二叉树，相信理论知识大家应该都清楚，就是以 后序数组的最后一个元素为切割点，先切中序数组，根据中序数组，反过来再切后序数组。一层一层切下去，每次后序数组最后一个元素就是节点元素。

如果让我们肉眼看两个序列，画一棵二叉树的话，应该分分钟都可以画出来。

流程如图：



那么代码应该怎么写呢?

说到一层一层切割，就应该想到了递归。

来看一下一共分几步：

- 第一步：如果数组大小为零的话，说明是空节点了。
- 第二步：如果不为空，那么取后序数组最后一个元素作为节点元素。
- 第三步：找到后序数组最后一个元素在中序数组的位置，作为切割点
- 第四步：切割中序数组，切成中序左数组和中序右数组 （顺序别搞反了，一定是先切中序数组）
- 第五步：切割后序数组，切成后序左数组和后序右数组
- 第六步：递归处理左区间和右区间

不难写出如下代码：（先把框架写出来）

```
TreeNode* traversal (vector<int>& inorder, vector<int>& postorder) {

    // 第一步
    if (postorder.size() == 0) return NULL;

    // 第二步：后序遍历数组最后一个元素，就是当前的中间节点
    int rootValue = postorder[postorder.size() - 1];
    TreeNode* root = new TreeNode(rootValue);

    // 叶子节点
    if (postorder.size() == 1) return root;
```

```
    // 第三步：找切割点
    int delimiterIndex;
    for (delimiterIndex = 0; delimiterIndex < inorder.size(); delimiterIndex++) {
        if (inorder[delimiterIndex] == rootValue) break;
    }

    // 第四步：切割中序数组，得到 中序左数组和中序右数组
    // 第五步：切割后序数组，得到 后序左数组和后序右数组

    // 第六步
    root->left = traversal(中序左数组, 后序左数组);
    root->right = traversal(中序右数组, 后序右数组);

    return root;
}
```

难点大家应该发现了，就是如何切割，以及边界值找不好很容易乱套。

此时应该注意确定切割的标准，是左闭右开，还有左开右闭，还是左闭右闭，这个就是不变量，要在递归中保持这个不变量。

在切割的过程中会产生四个区间，把握不好不变量的话，一会左闭右开，一会左闭右闭，必然乱套！

我在数组：每次遇到二分法，都是一看就会，一写就废和数组：这个循环可以转懵很多人！中都强调过循环不变量的重要性，在二分查找以及螺旋矩阵的求解中，坚持循环不变量非常重要，本题也是。

首先要切割中序数组，为什么先切割中序数组呢？

切割点在后序数组的最后一个元素，就是用这个元素来切割中序数组的，所以必要先切割中序数组。

中序数组相对比较好切，找到切割点（后序数组的最后一个元素）在中序数组的位置，然后切割，如下代码中我坚持左闭右开的原则：

```
// 找到中序遍历的切割点
int delimiterIndex;
for (delimiterIndex = 0; delimiterIndex < inorder.size(); delimiterIndex++) {
    if (inorder[delimiterIndex] == rootValue) break;
}

// 左闭右开区间：[0, delimiterIndex)
vector<int> leftInorder(inorder.begin(), inorder.begin() + delimiterIndex);
// [delimiterIndex + 1, end)
vector<int> rightInorder(inorder.begin() + delimiterIndex + 1, inorder.end() );
```

接下来就要切割后序数组了。

首先后序数组的最后一个元素指定不能要了，这是切割点 也是 当前二叉树中间节点的元素，已经用了。

后序数组的切割点怎么找？

后序数组没有明确的切割元素来进行左右切割，不像中序数组有明确的切割点，切割点左右分开就可以了。

此时有一个很重的点，就是中序数组大小一定是和后序数组的大小相同的（这是必然）。

中序数组我们都切成了左中序数组和右中序数组了，那么后序数组就可以按照左中序数组的大小来切割，切成左后序数组和右后序数组。

代码如下：

```
// postorder 舍弃末尾元素，因为这个元素就是中间节点，已经用过了
postorder.resize(postorder.size() - 1);

// 左闭右开，注意这里使用了左中序数组大小作为切割点：[0, leftInorder.size)
vector<int> leftPostorder(postorder.begin(), postorder.begin() + leftInorder.size());
// [leftInorder.size(), end)
vector<int> rightPostorder(postorder.begin() + leftInorder.size(), postorder.end());
```

此时，中序数组切成了左中序数组和右中序数组，后序数组切割成左后序数组和右后序数组。

接下来可以递归了，代码如下：

```
root->left = traversal(leftInorder, leftPostorder);
root->right = traversal(rightInorder, rightPostorder);
```

完整代码如下：

```
class Solution {
private:
    TreeNode* traversal (vector<int>& inorder, vector<int>& postorder) {
        if (postorder.size() == 0) return NULL;

        // 后序遍历数组最后一个元素，就是当前的中间节点
        int rootValue = postorder[postorder.size() - 1];
        TreeNode* root = new TreeNode(rootValue);

        // 叶子节点
        if (postorder.size() == 1) return root;

        // 找到中序遍历的切割点
        int delimiterIndex;
        for (delimiterIndex = 0; delimiterIndex < inorder.size(); delimiterIndex++) {
            if (inorder[delimiterIndex] == rootValue) break;
        }

        // 切割中序数组
        // 左闭右开区间：[0, delimiterIndex)
        vector<int> leftInorder(inorder.begin(), inorder.begin() + delimiterIndex);
        // [delimiterIndex + 1, end)
        vector<int> rightInorder(inorder.begin() + delimiterIndex + 1, inorder.end() );

        // postorder 舍弃末尾元素
```

```
        postorder.resize(postorder.size() - 1);

        // 切割后序数组
        // 依然左闭右开，注意这里使用了左中序数组大小作为切割点
        // [0, leftInorder.size)
        vector<int> leftPostorder(postorder.begin(), postorder.begin() +
leftInorder.size());
        // [leftInorder.size(), end)
        vector<int> rightPostorder(postorder.begin() + leftInorder.size(),
postorder.end());

        root->left = traversal(leftInorder, leftPostorder);
        root->right = traversal(rightInorder, rightPostorder);

        return root;
    }
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() == 0 || postorder.size() == 0) return NULL;
        return traversal(inorder, postorder);
    }
};
```

相信大家自己就算是思路清晰， 代码写出来一定是各种问题，所以一定要加日志来调试，看看是不是按照自己思路来切割的，不要大脑模拟，那样越想越糊涂。

加了日志的代码如下：（加了日志的代码不要在leetcode上提交，容易超时）

```
class Solution {
private:
    TreeNode* traversal (vector<int>& inorder, vector<int>& postorder) {
        if (postorder.size() == 0) return NULL;

        int rootValue = postorder[postorder.size() - 1];
        TreeNode* root = new TreeNode(rootValue);

        if (postorder.size() == 1) return root;

        int delimiterIndex;
        for (delimiterIndex = 0; delimiterIndex < inorder.size(); delimiterIndex++) {
            if (inorder[delimiterIndex] == rootValue) break;
        }

        vector<int> leftInorder(inorder.begin(), inorder.begin() + delimiterIndex);
        vector<int> rightInorder(inorder.begin() + delimiterIndex + 1, inorder.end() );

        postorder.resize(postorder.size() - 1);
```

```
        vector<int> leftPostorder(postorder.begin(), postorder.begin() +
leftInorder.size());
        vector<int> rightPostorder(postorder.begin() + leftInorder.size(),
postorder.end());

        // 以下为日志
        cout << "----------" << endl;

        cout << "leftInorder :";
        for (int i : leftInorder) {
            cout << i << " ";
        }
        cout << endl;

        cout << "rightInorder :";
        for (int i : rightInorder) {
            cout << i << " ";
        }
        cout << endl;

        cout << "leftPostorder :";
        for (int i : leftPostorder) {
            cout << i << " ";
        }
        cout << endl;
         cout << "rightPostorder :";
        for (int i : rightPostorder) {
            cout << i << " ";
        }
        cout << endl;

        root->left = traversal(leftInorder, leftPostorder);
        root->right = traversal(rightInorder, rightPostorder);

        return root;
    }
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() == 0 || postorder.size() == 0) return NULL;
        return traversal(inorder, postorder);
    }
};
```

此时应该发现了，如上的代码性能并不好，因为每层递归定义了新的vector（就是数组），既耗时又耗空间，但上面的代码是最好理解的，为了方便读者理解，所以用如上的代码来讲解。

下面给出用下标索引写出的代码版本：（思路是一样的，只不过不用重复定义vector了，每次用下标索引来分割）

```
class Solution {
```

```
private:
    // 中序区间：[inorderBegin, inorderEnd)，后序区间[postorderBegin, postorderEnd)
    TreeNode* traversal (vector<int>& inorder, int inorderBegin, int inorderEnd,
vector<int>& postorder, int postorderBegin, int postorderEnd) {
        if (postorderBegin == postorderEnd) return NULL;

        int rootValue = postorder[postorderEnd - 1];
        TreeNode* root = new TreeNode(rootValue);

        if (postorderEnd - postorderBegin == 1) return root;

        int delimiterIndex;
        for (delimiterIndex = inorderBegin; delimiterIndex < inorderEnd;
delimiterIndex++) {
            if (inorder[delimiterIndex] == rootValue) break;
        }
        // 切割中序数组
        // 左中序区间，左闭右开[leftInorderBegin, leftInorderEnd)
        int leftInorderBegin = inorderBegin;
        int leftInorderEnd = delimiterIndex;
        // 右中序区间，左闭右开[rightInorderBegin, rightInorderEnd)
        int rightInorderBegin = delimiterIndex + 1;
        int rightInorderEnd = inorderEnd;

        // 切割后序数组
        // 左后序区间，左闭右开[leftPostorderBegin, leftPostorderEnd)
        int leftPostorderBegin =  postorderBegin;
        int leftPostorderEnd = postorderBegin + delimiterIndex - inorderBegin; // 终止位
置是 需要加上 中序区间的大小size
        // 右后序区间，左闭右开[rightPostorderBegin, rightPostorderEnd)
        int rightPostorderBegin = postorderBegin + (delimiterIndex - inorderBegin);
        int rightPostorderEnd = postorderEnd - 1; // 排除最后一个元素，已经作为节点了

        root->left = traversal(inorder, leftInorderBegin, leftInorderEnd,  postorder,
leftPostorderBegin, leftPostorderEnd);
        root->right = traversal(inorder, rightInorderBegin, rightInorderEnd, postorder,
rightPostorderBegin, rightPostorderEnd);

        return root;
    }
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() == 0 || postorder.size() == 0) return NULL;
        // 左闭右开的原则
        return traversal(inorder, 0, inorder.size(), postorder, 0, postorder.size());
    }
};
```

那么这个版本写出来依然要打日志进行调试，打日志的版本如下：（**该版本不要在leetcode上提交，容易超时**）

```
class Solution {
private:
    TreeNode* traversal (vector<int>& inorder, int inorderBegin, int inorderEnd,
vector<int>& postorder, int postorderBegin, int postorderEnd) {
        if (postorderBegin == postorderEnd) return NULL;

        int rootValue = postorder[postorderEnd - 1];
        TreeNode* root = new TreeNode(rootValue);

        if (postorderEnd - postorderBegin == 1) return root;

        int delimiterIndex;
        for (delimiterIndex = inorderBegin; delimiterIndex < inorderEnd;
delimiterIndex++) {
            if (inorder[delimiterIndex] == rootValue) break;
        }
        // 切割中序数组
        // 左中序区间，左闭右开[leftInorderBegin, leftInorderEnd)
        int leftInorderBegin = inorderBegin;
        int leftInorderEnd = delimiterIndex;
        // 右中序区间，左闭右开[rightInorderBegin, rightInorderEnd)
        int rightInorderBegin = delimiterIndex + 1;
        int rightInorderEnd = inorderEnd;

        // 切割后序数组
        // 左后序区间，左闭右开[leftPostorderBegin, leftPostorderEnd)
        int leftPostorderBegin =  postorderBegin;
        int leftPostorderEnd = postorderBegin + delimiterIndex - inorderBegin; // 终止位
置是 需要加上 中序区间的大小size
        // 右后序区间，左闭右开[rightPostorderBegin, rightPostorderEnd)
        int rightPostorderBegin = postorderBegin + (delimiterIndex - inorderBegin);
        int rightPostorderEnd = postorderEnd - 1; // 排除最后一个元素，已经作为节点了

        cout << "----------" << endl;
        cout << "leftInorder :";
        for (int i = leftInorderBegin; i < leftInorderEnd; i++) {
            cout << inorder[i] << " ";
        }
        cout << endl;

        cout << "rightInorder :";
        for (int i = rightInorderBegin; i < rightInorderEnd; i++) {
            cout << inorder[i] << " ";
        }
        cout << endl;

        cout << "leftpostorder :";
        for (int i = leftPostorderBegin; i < leftPostorderEnd; i++) {
            cout << postorder[i] << " ";
```

```
        }
        cout << endl;

        cout << "rightpostorder :";
        for (int i = rightPostorderBegin; i < rightPostorderEnd; i++) {
            cout << postorder[i] << " ";
        }
        cout << endl;

        root->left = traversal(inorder, leftInorderBegin, leftInorderEnd,  postorder,
leftPostorderBegin, leftPostorderEnd);
        root->right = traversal(inorder, rightInorderBegin, rightInorderEnd, postorder,
rightPostorderBegin, rightPostorderEnd);

        return root;
    }
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() == 0 || postorder.size() == 0) return NULL;
        return traversal(inorder, 0, inorder.size(), postorder, 0, postorder.size());
    }
};
```

相关题目推荐

从前序与中序遍历序列构造二叉树

[力扣题目链接]

根据一棵树的前序遍历与中序遍历构造二叉树。

注意:
你可以假设树中没有重复的元素。

例如，给出

前序遍历 preorder = [3,9,20,15,7]
中序遍历 inorder = [9,3,15,20,7]
返回如下的二叉树：

```
    3
   / \
  9  20
    /  \
   15   7
```

思路

本题和106是一样的道理。

我就直接给出代码了。

带日志的版本C++代码如下： **（带日志的版本仅用于调试，不要在leetcode上提交，会超时）**

```
class Solution {
private:
        TreeNode* traversal (vector<int>& inorder, int inorderBegin, int inorderEnd,
vector<int>& preorder, int preorderBegin, int preorderEnd) {
        if (preorderBegin == preorderEnd) return NULL;

        int rootValue = preorder[preorderBegin]; // 注意用preorderBegin 不要用0
        TreeNode* root = new TreeNode(rootValue);

        if (preorderEnd - preorderBegin == 1) return root;

        int delimiterIndex;
        for (delimiterIndex = inorderBegin; delimiterIndex < inorderEnd;
delimiterIndex++) {
            if (inorder[delimiterIndex] == rootValue) break;
        }
        // 切割中序数组
        // 中序左区间，左闭右开[leftInorderBegin, leftInorderEnd)
        int leftInorderBegin = inorderBegin;
        int leftInorderEnd = delimiterIndex;
        // 中序右区间，左闭右开[rightInorderBegin, rightInorderEnd)
        int rightInorderBegin = delimiterIndex + 1;
        int rightInorderEnd = inorderEnd;

        // 切割前序数组
        // 前序左区间，左闭右开[leftPreorderBegin, leftPreorderEnd)
        int leftPreorderBegin =  preorderBegin + 1;
        int leftPreorderEnd = preorderBegin + 1 + delimiterIndex - inorderBegin; // 终止
位置是起始位置加上中序左区间的大小size
        // 前序右区间，左闭右开[rightPreorderBegin, rightPreorderEnd)
        int rightPreorderBegin = preorderBegin + 1 + (delimiterIndex - inorderBegin);
        int rightPreorderEnd = preorderEnd;

        cout << "----------" << endl;
        cout << "leftInorder :";
        for (int i = leftInorderBegin; i < leftInorderEnd; i++) {
            cout << inorder[i] << " ";
        }
        cout << endl;

        cout << "rightInorder :";
        for (int i = rightInorderBegin; i < rightInorderEnd; i++) {
```

```
            cout << inorder[i] << " ";
        }
        cout << endl;

        cout << "leftPreorder :";
        for (int i = leftPreorderBegin; i < leftPreorderEnd; i++) {
            cout << preorder[i] << " ";
        }
        cout << endl;

        cout << "rightPreorder :";
        for (int i = rightPreorderBegin; i < rightPreorderEnd; i++) {
            cout << preorder[i] << " ";
        }
        cout << endl;


        root->left = traversal(inorder, leftInorderBegin, leftInorderEnd,  preorder,
leftPreorderBegin, leftPreorderEnd);
        root->right = traversal(inorder, rightInorderBegin, rightInorderEnd, preorder,
rightPreorderBegin, rightPreorderEnd);

        return root;
    }

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        if (inorder.size() == 0 || preorder.size() == 0) return NULL;
        return traversal(inorder, 0, inorder.size(), preorder, 0, preorder.size());

    }
};
```

105.从前序与中序遍历序列构造二叉树，最后版本，C++代码：

```
class Solution {
private:
        TreeNode* traversal (vector<int>& inorder, int inorderBegin, int inorderEnd,
vector<int>& preorder, int preorderBegin, int preorderEnd) {
        if (preorderBegin == preorderEnd) return NULL;

        int rootValue = preorder[preorderBegin]; // 注意用preorderBegin 不要用0
        TreeNode* root = new TreeNode(rootValue);

        if (preorderEnd - preorderBegin == 1) return root;

        int delimiterIndex;
        for (delimiterIndex = inorderBegin; delimiterIndex < inorderEnd;
delimiterIndex++) {
```

```
            if (inorder[delimiterIndex] == rootValue) break;
        }
        // 切割中序数组
        // 中序左区间，左闭右开[leftInorderBegin, leftInorderEnd)
        int leftInorderBegin = inorderBegin;
        int leftInorderEnd = delimiterIndex;
        // 中序右区间，左闭右开[rightInorderBegin, rightInorderEnd)
        int rightInorderBegin = delimiterIndex + 1;
        int rightInorderEnd = inorderEnd;

        // 切割前序数组
        // 前序左区间，左闭右开[leftPreorderBegin, leftPreorderEnd)
        int leftPreorderBegin =  preorderBegin + 1;
        int leftPreorderEnd = preorderBegin + 1 + delimiterIndex - inorderBegin; // 终止位置是起始位置加上中序左区间的大小size
        // 前序右区间，左闭右开[rightPreorderBegin, rightPreorderEnd)
        int rightPreorderBegin = preorderBegin + 1 + (delimiterIndex - inorderBegin);
        int rightPreorderEnd = preorderEnd;

        root->left = traversal(inorder, leftInorderBegin, leftInorderEnd,  preorder, leftPreorderBegin, leftPreorderEnd);
        root->right = traversal(inorder, rightInorderBegin, rightInorderEnd, preorder, rightPreorderBegin, rightPreorderEnd);

        return root;
    }

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        if (inorder.size() == 0 || preorder.size() == 0) return NULL;

        // 参数坚持左闭右开的原则
        return traversal(inorder, 0, inorder.size(), preorder, 0, preorder.size());
    }
};
```

思考题

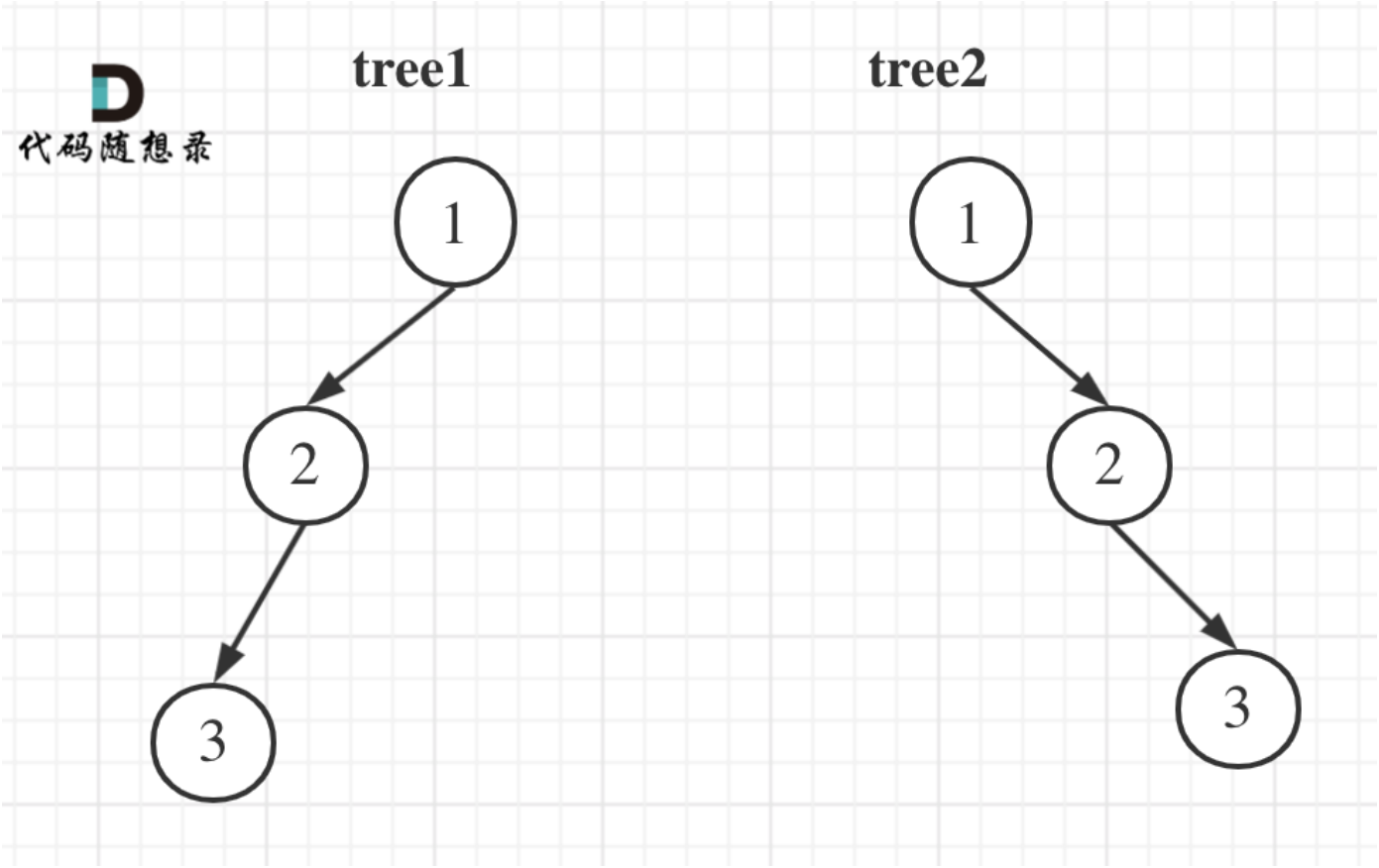
前序和中序可以唯一确定一棵二叉树。

后序和中序可以唯一确定一棵二叉树。

那么前序和后序可不可以唯一确定一棵二叉树呢?

前序和后序不能唯一确定一棵二叉树！，因为没有中序遍历无法确定左右部分，也就是无法分割。

举一个例子：



tree1 的前序遍历是[1 2 3]， 后序遍历是[3 2 1]。

tree2 的前序遍历是[1 2 3]， 后序遍历是[3 2 1]。

那么tree1 和 tree2 的前序和后序完全相同，这是一棵树么，很明显是两棵树！

所以前序和后序不能唯一确定一棵二叉树！

总结

之前我们讲的二叉树题目都是各种遍历二叉树，这次开始构造二叉树了，思路其实比较简单，但是真正代码实现出来并不容易。

所以要避免眼高手低，踏实地把代码写出来。

我同时给出了添加日志的代码版本，因为这种题目是不太容易写出来调一调就能过的，所以一定要把流程日志打出来，看看符不符合自己的思路。

大家遇到这种题目的时候，也要学会打日志来调试（如何打日志有时候也是个技术活），不要脑动模拟，脑动模拟很容易越想越乱。

最后我还给出了为什么前序和中序可以唯一确定一棵二叉树，后序和中序可以唯一确定一棵二叉树，而前序和后序却不行。

认真研究完本篇，相信大家对二叉树的构造会清晰很多。

19.最大二叉树

力扣题目地址

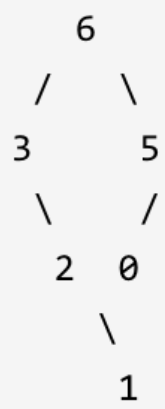
给定一个不含重复元素的整数数组。一个以此数组构建的最大二叉树定义如下：

- 二叉树的根是数组中的最大元素。
- 左子树是通过数组中最大值左边部分构造出的最大二叉树。
- 右子树是通过数组中最大值右边部分构造出的最大二叉树。

通过给定的数组构建最大二叉树，并且输出这个树的根节点。

示例：

输入：[3,2,1,6,0,5]
输出：返回下面这棵树的根节点：



提示：

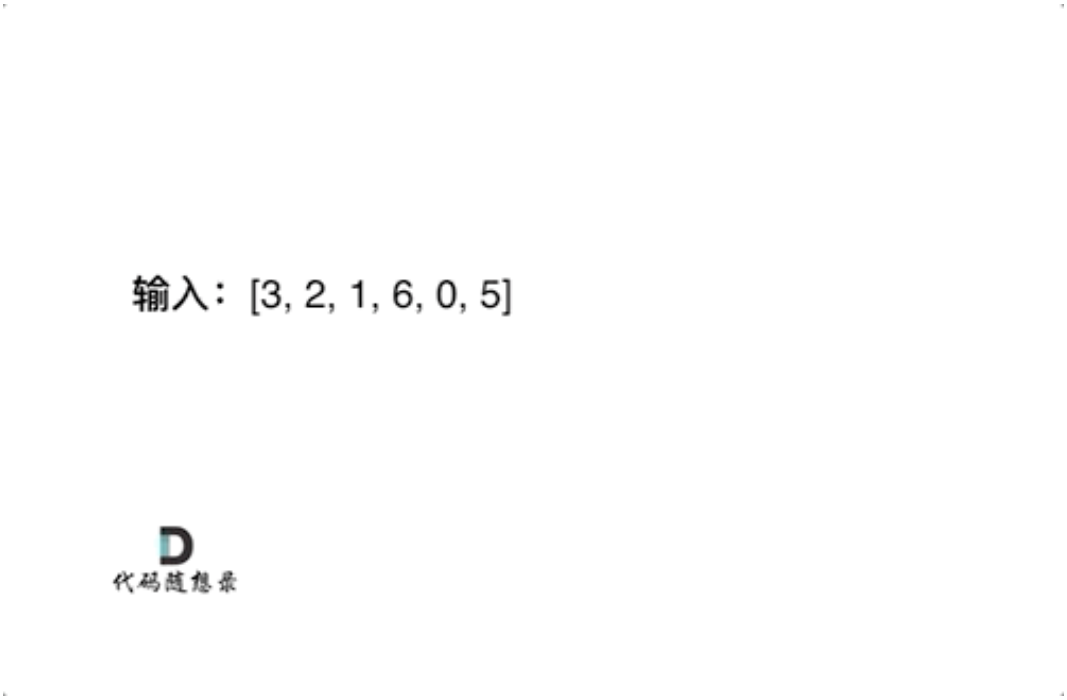
给定的数组的大小在 [1, 1000] 之间。

算法公开课

《代码随想录》算法视频公开课：又是构造二叉树，又有很多坑！| LeetCode：654.最大二叉树，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

最大二叉树的构建过程如下：



构造树一般采用的是前序遍历，因为先构造中间节点，然后递归构造左子树和右子树。

- 确定递归函数的参数和返回值

参数传入的是存放元素的数组，返回该数组构造的二叉树的头结点，返回类型是指向节点的指针。

代码如下：

```
TreeNode* constructMaximumBinaryTree(vector<int>& nums)
```

- 确定终止条件

题目中说了输入的数组大小一定是大于等于1的，所以我们不用考虑小于1的情况，那么当递归遍历的时候，如果传入的数组大小为1，说明遍历到了叶子节点了。

那么应该定义一个新的节点，并把这个数组的数值赋给新的节点，然后返回这个节点。 这表示一个数组大小是1的时候，构造了一个新的节点，并返回。

代码如下：

```
TreeNode* node = new TreeNode(0);
if (nums.size() == 1) {
    node->val = nums[0];
    return node;
}
```

- 确定单层递归的逻辑

这里有三步工作

1. 先要找到数组中最大的值和对应的下标， 最大的值构造根节点，下标用来下一步分割数组。

代码如下：

```
int maxValue = 0;
int maxValueIndex = 0;
for (int i = 0; i < nums.size(); i++) {
    if (nums[i] > maxValue) {
        maxValue = nums[i];
        maxValueIndex = i;
    }
}
TreeNode* node = new TreeNode(0);
node->val = maxValue;
```

2. 最大值所在的下标左区间 构造左子树

这里要判断maxValueIndex > 0，因为要保证左区间至少有一个数值。

代码如下：

```
if (maxValueIndex > 0) {
    vector<int> newVec(nums.begin(), nums.begin() + maxValueIndex);
    node->left = constructMaximumBinaryTree(newVec);
}
```

3. 最大值所在的下标右区间 构造右子树

判断maxValueIndex < (nums.size() - 1)，确保右区间至少有一个数值。

代码如下：

```
if (maxValueIndex < (nums.size() - 1)) {
    vector<int> newVec(nums.begin() + maxValueIndex + 1, nums.end());
    node->right = constructMaximumBinaryTree(newVec);
}
```

这样我们就分析完了，整体代码如下：（详细注释）

```
class Solution {
public:
    TreeNode* constructMaximumBinaryTree(vector<int>& nums) {
        TreeNode* node = new TreeNode(0);
        if (nums.size() == 1) {
            node->val = nums[0];
            return node;
        }
        // 找到数组中最大的值和对应的下标
        int maxValue = 0;
        int maxValueIndex = 0;
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] > maxValue) {
                maxValue = nums[i];
```

```
                maxValueIndex = i;
            }
        }
        node->val = maxValue;
        // 最大值所在的下标左区间 构造左子树
        if (maxValueIndex > 0) {
            vector<int> newVec(nums.begin(), nums.begin() + maxValueIndex);
            node->left = constructMaximumBinaryTree(newVec);
        }
        // 最大值所在的下标右区间 构造右子树
        if (maxValueIndex < (nums.size() - 1)) {
            vector<int> newVec(nums.begin() + maxValueIndex + 1, nums.end());
            node->right = constructMaximumBinaryTree(newVec);
        }
        return node;
    }
};
```

以上代码比较冗余，效率也不高，每次还要切割的时候每次都要定义新的vector（也就是数组），但逻辑比较清晰。

和文章二叉树：构造二叉树登场！中一样的优化思路，就是每次分隔不用定义新的数组，而是通过下标索引直接在原数组上操作。

优化后代码如下：

```
class Solution {
private:
    // 在左闭右开区间[left, right)，构造二叉树
    TreeNode* traversal(vector<int>& nums, int left, int right) {
        if (left >= right) return nullptr;

        // 分割点下标：maxValueIndex
        int maxValueIndex = left;
        for (int i = left + 1; i < right; ++i) {
            if (nums[i] > nums[maxValueIndex]) maxValueIndex = i;
        }

        TreeNode* root = new TreeNode(nums[maxValueIndex]);

        // 左闭右开：[left, maxValueIndex)
        root->left = traversal(nums, left, maxValueIndex);

        // 左闭右开：[maxValueIndex + 1, right)
        root->right = traversal(nums, maxValueIndex + 1, right);

        return root;
    }
public:
    TreeNode* constructMaximumBinaryTree(vector<int>& nums) {
```

```
        return traversal(nums, 0, nums.size());
    }
};
```

拓展

可以发现上面的代码看上去简洁一些，**主要是因为第二版其实是允许空节点进入递归，所以不用在递归的时候加判断节点是否为空**

第一版递归过程：（加了if判断，为了不让空节点进入递归）

```
if (maxValueIndex > 0) { // 这里加了判断是为了不让空节点进入递归
    vector<int> newVec(nums.begin(), nums.begin() + maxValueIndex);
    node->left = constructMaximumBinaryTree(newVec);
}

if (maxValueIndex < (nums.size() - 1)) { // 这里加了判断是为了不让空节点进入递归
    vector<int> newVec(nums.begin() + maxValueIndex + 1, nums.end());
    node->right = constructMaximumBinaryTree(newVec);
}
```

第二版递归过程：（如下代码就没有加if判断）

```
root->left = traversal(nums, left, maxValueIndex);

root->right = traversal(nums, maxValueIndex + 1, right);
```

第二版代码是允许空节点进入递归，所以没有加if判断，当然终止条件也要有相应的改变。

第一版终止条件，是遇到叶子节点就终止，因为空节点不会进入递归。

第二版相应的终止条件，是遇到空节点，也就是数组区间为0，就终止了。

总结

这道题目其实和 二叉树：构造二叉树登场！ 是一个思路，比二叉树：构造二叉树登场！ 还简单一些。

注意类似用数组构造二叉树的题目，每次分隔尽量不要定义新的数组，而是通过下标索引直接在原数组上操作，这样可以节约时间和空间上的开销。

一些同学也会疑惑，什么时候递归函数前面加if，什么时候不加if，这个问题我在最后也给出了解释。

其实就是不同代码风格的实现，**一般情况来说：如果让空节点（空指针）进入递归，就不加if，如果不让空节点进入递归，就加if限制一下， 终止条件也会相应的调整。**

20.本周小结！（二叉树系列三）

周一

在二叉树：以为使用了递归，其实还隐藏着回溯中，通过leetcode 257.二叉树的所有路径这道题目，讲解了递归如何隐藏着回溯，一些代码会把回溯的过程都隐藏了起来了，甚至刷过这道题的同学可能都不知道自己用了回溯。

文章中第一版代码把每一个细节都展示了输出来了，大家可以清晰的看到回溯的过程。

然后给出了第二版优化后的代码，分析了其回溯隐藏在了哪里，如果要把这个回溯扣出来的话，在第二版的基础上应该怎么改。

主要需要理解：**回溯隐藏在traversal(cur->left, path + "->", result);中的 path + "->"。 每次函数调用完，path依然是没有加上"->" 的，这就是回溯了。**

周二

在文章二叉树：做了这么多题目了，我的左叶子之和是多少？ 中提供了另一个判断节点属性的思路，平时我们习惯了使用通过节点的左右孩子判断本节点的属性，但发现使用这个思路无法判断左叶子。

此时需要相连的三层之间构成的约束条件，也就是要通过节点的父节点以及孩子节点来判断本节点的属性。

这道题目可以扩展大家对二叉树的解题思路。

周三

在二叉树：我的左下角的值是多少？ 中的题目如果使用递归的写法还是有点难度的，层次遍历反而很简单。

题目其实就是要在树的**最后一行**找到**最左边的值**。

如何判断是最后一行呢，其实就是深度最大的叶子节点一定是最后一行。

在这篇文章中，我们使用递归算法实实在在的求了一次深度，然后使用靠左的遍历，保证求得靠左的最大深度，而且又一次使用了回溯。

如果对二叉树的高度与深度又有点模糊了，在看这里二叉树：我平衡么？，回忆一下吧。

二叉树：我的左下角的值是多少？ 中把我们之前讲过的内容都过了一遍，此外，还用前序遍历的技巧求得了靠左的最大深度。

求二叉树的各种最值，就想应该采用什么样的遍历顺序，确定了遍历循序，其实就和数组求最值一样容易了。

周四

在二叉树：递归函数究竟什么时候需要返回值，什么时候不要返回值？ 中通过两道题目，彻底说清楚递归函数的返回值问题。

一般情况下：**如果需要搜索整棵二叉树，那么递归函数就不要返回值，如果要搜索其中一条符合条件的路径，递归函数就需要返回值，因为遇到符合条件的路径了就要及时返回。**

特别是有些时候 递归函数的返回值是bool类型，一些同学会疑惑为啥要加这个，其实就是为了找到一条边立刻返回。

其实还有一种就是后序遍历需要根据左右递归的返回值推出中间节点的状态，这种需要有返回值，例如222.完全二叉树，110.平衡二叉树，这几道我们之前也讲过。

周五

之前都是讲解遍历二叉树，这次该构造二叉树了，在二叉树：构造二叉树登场！ 中，我们通过前序和中序，后序和中序，构造了唯一的一棵二叉树。

构造二叉树有三个注意的点：

- 分割时候，坚持区间不变量原则，左闭右开，或者左闭又闭。
- 分割的时候，注意后序 或者 前序已经有一个节点作为中间节点了，不能继续使用了。
- 如何使用切割后的后序数组来切合中序数组？利用中序数组大小一定是和后序数组的大小相同这一特点来进行切割。

这道题目代码实现并不简单，大家啃下来之后，二叉树的构造应该不是问题了。

最后我还给出了为什么前序和后序不能唯一构成一棵二叉树，因为没有中序遍历就无法确定左右部分，也就无法分割。

周六

知道了如何构造二叉树，那么使用一个套路就可以解决文章二叉树：构造一棵最大的二叉树中的问题。

注意类似用数组构造二叉树的题目，每次分隔尽量不要定义新的数组，而是通过下标索引直接在原数组上操作，这样可以节约时间和空间上的开销。

文章中我还给出了递归函数什么时候加if，什么时候不加if，其实就是控制空节点（空指针）是否进入递归，是不同的代码实现方式，都是可以的。

一般情况来说：如果让空节点（空指针）进入递归，就不加if，如果不让空节点进入递归，就加if限制一下， 终止条件也会相应的调整。

总结

本周我们深度讲解了如下知识点：

1. 递归中如何隐藏着回溯
2. 如何通过三层关系确定左叶子
3. 如何通过二叉树深度来判断左下角的值
4. 递归函数究竟什么时候需要返回值，什么时候不要返回值？
5. 前序和中序，后序和中序构造唯一二叉树
6. 使用数组构造某一特性的二叉树

如果大家一路跟下来，一定收获满满，如果周末不做这个总结，大家可能都不知道自己收获满满，啊哈！

21.合并二叉树

[力扣题目链接]

给定两个二叉树，想象当你将它们中的一个覆盖到另一个上时，两个二叉树的一些节点便会重叠。

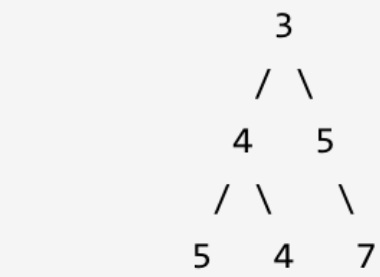
你需要将他们合并为一个新的二叉树。合并的规则是如果两个节点重叠，那么将他们的值相加作为节点合并后的新值，否则不为 NULL 的节点将直接作为新二叉树的节点。

示例 1:

输入：

输出：

合并后的树：



注意: 合并必须从两个树的根节点开始。

算法公开课

[《代码随想录》算法视频公开课]：[一起操作两个二叉树？有点懵！ | LeetCode：617.合并二叉树]，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

相信这道题目很多同学疑惑的点是如何同时遍历两个二叉树呢?

其实和遍历一个树逻辑是一样的，只不过传入两个树的节点，同时操作。

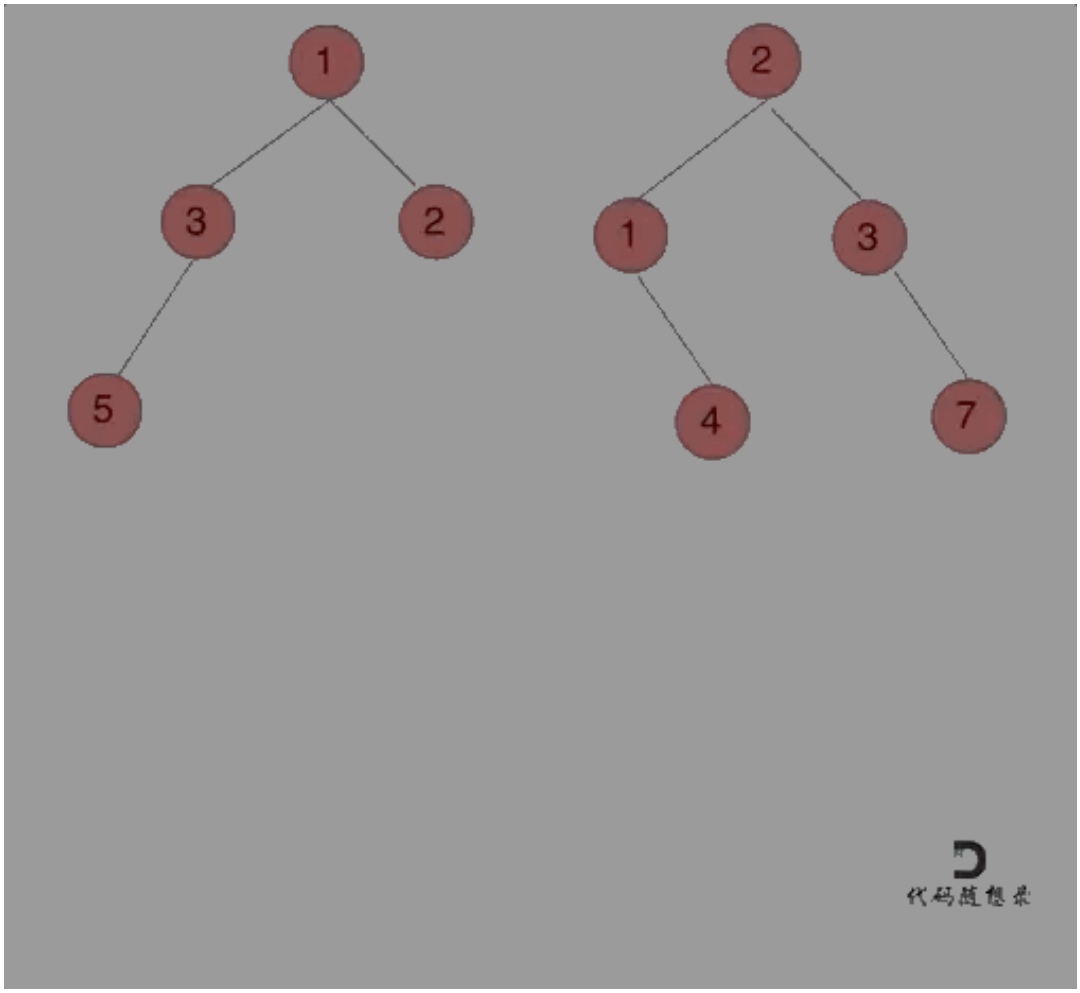
递归

二叉树使用递归，就要想使用前中后哪种遍历方式?

本题使用哪种遍历都是可以的!

我们下面以前序遍历为例。

动画如下：



那么我们来按照递归三部曲来解决：

1. **确定递归函数的参数和返回值：**

首先要合入两个二叉树，那么参数至少是要传入两个二叉树的根节点，返回值就是合并之后二叉树的根节点。

代码如下：

```
TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
```

2. **确定终止条件：**

因为是传入了两个树，那么就有两个树遍历的节点t1 和 t2，如果t1 == NULL 了，两个树合并就应该是 t2 了（如果t2也为NULL也无所谓，合并之后就是NULL）。

反过来如果t2 == NULL，那么两个数合并就是t1（如果t1也为NULL也无所谓，合并之后就是NULL）。

代码如下：

```
if (t1 == NULL) return t2; // 如果t1为空，合并之后就应该是t2
if (t2 == NULL) return t1; // 如果t2为空，合并之后就应该是t1
```

3. **确定单层递归的逻辑：**

单层递归的逻辑就比较好写了，这里我们重复利用一下t1这个树，t1就是合并之后树的根节点（就是修改了原来树的结构）。

那么单层递归中，就要把两棵树的元素加到一起。

```
t1->val += t2->val;
```

接下来t1 的左子树是：合并 t1左子树 t2左子树之后的左子树。

t1 的右子树：是 合并 t1右子树 t2右子树之后的右子树。

最终t1就是合并之后的根节点。

代码如下：

```
t1->left = mergeTrees(t1->left, t2->left);
t1->right = mergeTrees(t1->right, t2->right);
return t1;
```

此时前序遍历，完整代码就写出来了，如下：

```
class Solution {
public:
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        if (t1 == NULL) return t2; // 如果t1为空，合并之后就应该是t2
        if (t2 == NULL) return t1; // 如果t2为空，合并之后就应该是t1
        // 修改了t1的数值和结构
        t1->val += t2->val;                             // 中
        t1->left = mergeTrees(t1->left, t2->left);      // 左
        t1->right = mergeTrees(t1->right, t2->right);   // 右
        return t1;
    }
};
```

那么中序遍历也是可以的，代码如下：

```
class Solution {
public:
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        if (t1 == NULL) return t2; // 如果t1为空，合并之后就应该是t2
        if (t2 == NULL) return t1; // 如果t2为空，合并之后就应该是t1
        // 修改了t1的数值和结构
        t1->left = mergeTrees(t1->left, t2->left);      // 左
        t1->val += t2->val;                             // 中
        t1->right = mergeTrees(t1->right, t2->right);   // 右
        return t1;
    }
};
```

后序遍历依然可以，代码如下：

```
class Solution {
public:
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        if (t1 == NULL) return t2; // 如果t1为空，合并之后就应该是t2
        if (t2 == NULL) return t1; // 如果t2为空，合并之后就应该是t1
        // 修改了t1的数值和结构
        t1->left = mergeTrees(t1->left, t2->left);      // 左
        t1->right = mergeTrees(t1->right, t2->right);   // 右
        t1->val += t2->val;                             // 中
        return t1;
    }
};
```

但是前序遍历是最好理解的，我建议大家用前序遍历来做就OK。

如上的方法修改了t1的结构，当然也可以不修改t1和t2的结构，重新定义一个树。

不修改输入树的结构，前序遍历，代码如下：

```
class Solution {
public:
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        if (t1 == NULL) return t2;
        if (t2 == NULL) return t1;
        // 重新定义新的节点，不修改原有两个树的结构
        TreeNode* root = new TreeNode(0);
        root->val = t1->val + t2->val;
        root->left = mergeTrees(t1->left, t2->left);
        root->right = mergeTrees(t1->right, t2->right);
        return root;
    }
};
```

迭代法

使用迭代法，如何同时处理两棵树呢?

思路我们在二叉树：我对称么？中的迭代法已经讲过一次了，求二叉树对称的时候就是把两个树的节点同时加入队列进行比较。

本题我们也使用队列，模拟的层序遍历，代码如下：

```
class Solution {
public:
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        if (t1 == NULL) return t2;
        if (t2 == NULL) return t1;
        queue<TreeNode*> que;
        que.push(t1);
        que.push(t2);
        while(!que.empty()) {
            TreeNode* node1 = que.front(); que.pop();
            TreeNode* node2 = que.front(); que.pop();
            // 此时两个节点一定不为空，val相加
            node1->val += node2->val;

            // 如果两棵树左节点都不为空，加入队列
            if (node1->left != NULL && node2->left != NULL) {
                que.push(node1->left);
                que.push(node2->left);
            }
            // 如果两棵树右节点都不为空，加入队列
            if (node1->right != NULL && node2->right != NULL) {
                que.push(node1->right);
                que.push(node2->right);
            }

            // 当t1的左节点 为空 t2左节点不为空，就赋值过去
            if (node1->left == NULL && node2->left != NULL) {
                node1->left = node2->left;
            }
            // 当t1的右节点 为空 t2右节点不为空，就赋值过去
            if (node1->right == NULL && node2->right != NULL) {
                node1->right = node2->right;
            }
        }
        return t1;
    }
};
```

拓展

当然也可以秀一波指针的操作，这是我写的野路子，大家就随便看看就行了，以防带跑偏了。

如下代码中，想要更改二叉树的值，应该传入指向指针的指针。

代码如下：（前序遍历）

```
class Solution {
public:
    void process(TreeNode** t1, TreeNode** t2) {
        if ((*t1) == NULL && (*t2) == NULL) return;
        if ((*t1) != NULL && (*t2) != NULL) {
            (*t1)->val += (*t2)->val;
        }
        if ((*t1) == NULL && (*t2) != NULL) {
            *t1 = *t2;
            return;
        }
        if ((*t1) != NULL && (*t2) == NULL) {
            return;
        }
        process(&((*t1)->left), &((*t2)->left));
        process(&((*t1)->right), &((*t2)->right));
    }
    TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
        process(&t1, &t2);
        return t1;
    }
};
```

总结

合并二叉树，也是二叉树操作的经典题目，如果没有接触过的话，其实并不简单，因为我们习惯了操作一个二叉树，一起操作两个二叉树，还会有点懵懵的。

这不是我们第一次操作两棵二叉树了，在二叉树：我对称么？中也一起操作了两棵二叉树。

迭代法中，一般一起操作两个树都是使用队列模拟类似层序遍历，同时处理两个树的节点，这种方式最好理解，如果用模拟递归的思路的话，要复杂一些。

最后拓展中，我给了一个操作指针的野路子，大家随便看看就行了，如果学习C++的话，可以再去研究研究。

22.二叉搜索树中的搜索

力扣题目地址

给定二叉搜索树（BST）的根节点和一个值。 你需要在BST中找到节点值等于给定值的节点。 返回以该节点为根的子树。 如果节点不存在，则返回 NULL。

例如，

```
给定二叉搜索树：

        4
       / \
      2   7
     / \
    1   3

和值：2
```

你应该返回如下子树:

```
      2
     / \
    1   3
```

在上述示例中，如果要找的值是 5，但因为没有节点值为 5，我们应该返回 NULL。

算法公开课

《代码随想录》算法视频公开课：不愧是搜索树，这次搜索有方向了！ | LeetCode：700.二叉搜索树中的搜索，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

之前我们讲的都是普通二叉树，那么接下来看看二叉搜索树。

在关于二叉树，你该了解这些！ 中，我们已经讲过了二叉搜索树。

二叉搜索树是一个有序树：

- 若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值；
- 若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；
- 它的左、右子树也分别为二叉搜索树

这就决定了，二叉搜索树，递归遍历和迭代遍历和普通二叉树都不一样。

本题，其实就是在二叉搜索树中搜索一个节点。那么我们来看看应该如何遍历。

递归法

1. 确定递归函数的参数和返回值

递归函数的参数传入的就是根节点和要搜索的数值，返回的就是以这个搜索数值所在的节点。

代码如下：

```
TreeNode* searchBST(TreeNode* root, int val)
```

2. 确定终止条件

如果root为空，或者找到这个数值了，就返回root节点。

```
if (root == NULL || root->val == val) return root;
```

3. 确定单层递归的逻辑

看看二叉搜索树的单层递归逻辑有何不同。

因为二叉搜索树的节点是有序的，所以可以有方向的去搜索。

如果root->val > val，搜索左子树，如果root->val < val，就搜索右子树，最后如果都没有搜索到，就返回NULL。

代码如下：

```
TreeNode* result = NULL;
if (root->val > val) result = searchBST(root->left, val);
if (root->val < val) result = searchBST(root->right, val);
return result;
```

很多录友写递归函数的时候 习惯直接写 `searchBST(root->left, val)`，却忘了 递归函数还有返回值。

递归函数的返回值是什么？是 左子树如果搜索到了val，要将该节点返回。 如果不用一个变量将其接住，那么返回值不就没了。

所以要 `result = searchBST(root->left, val)`。

整体代码如下：

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        if (root == NULL || root->val == val) return root;
        TreeNode* result = NULL;
        if (root->val > val) result = searchBST(root->left, val);
        if (root->val < val) result = searchBST(root->right, val);
        return result;
    }
};
```

或者我们也可以这么写

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        if (root == NULL || root->val == val) return root;
        if (root->val > val) return searchBST(root->left, val);
        if (root->val < val) return searchBST(root->right, val);
        return NULL;
    }
};
```

迭代法

一提到二叉树遍历的迭代法，可能立刻想起使用栈来模拟深度遍历，使用队列来模拟广度遍历。

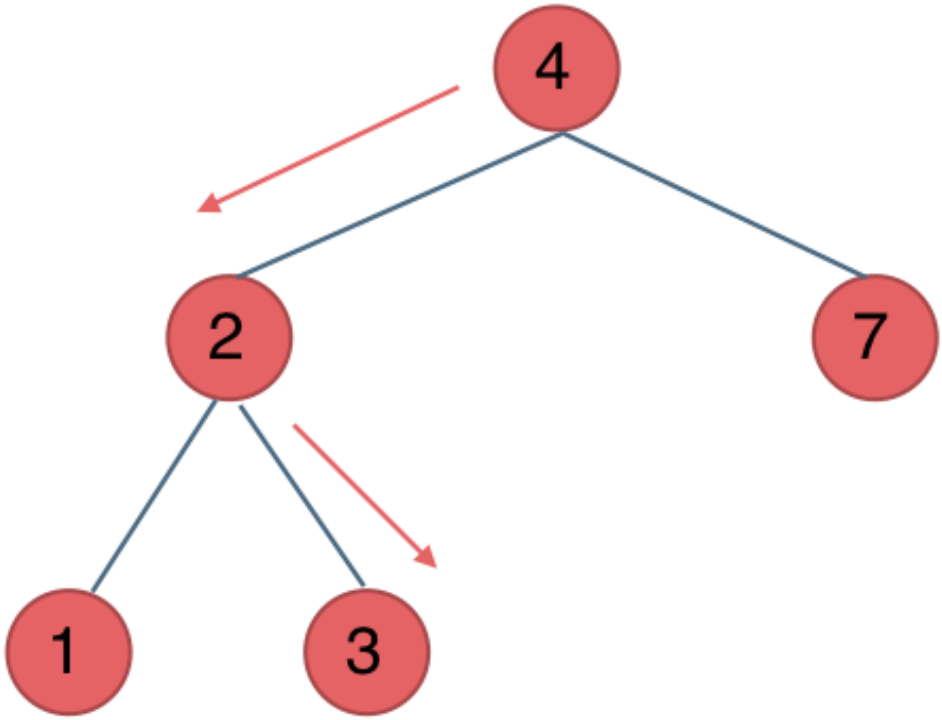
对于二叉搜索树可就不一样了，因为二叉搜索树的特殊性，也就是节点的有序性，可以不使用辅助栈或者队列就可以写出迭代法。

对于一般二叉树，递归过程中还有回溯的过程，例如走一个左方向的分支走到头了，那么要调头，在走右分支。

而对于二叉搜索树，不需要回溯的过程，因为节点的有序性就帮我们确定了搜索的方向。

例如要搜索元素为3的节点，**我们不需要搜索其他节点，也不需要做回溯，查找的路径已经规划好了。**

中间节点如果大于3就向左走，如果小于3就向右走，如图：



所以迭代法代码如下：

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        while (root != NULL) {
            if (root->val > val) root = root->left;
            else if (root->val < val) root = root->right;
            else return root;
        }
        return NULL;
    }
};
```

第一次看到了如此简单的迭代法，是不是感动的痛哭流涕，哭一会~

总结

本篇我们介绍了二叉搜索树的遍历方式，因为二叉搜索树的有序性，遍历的时候要比普通二叉树简单很多。

但是一些同学很容易忽略二叉搜索树的特性，所以写出遍历的代码就未必真的简单了。

所以针对二叉搜索树的题目，一样要利用其特性。

文中我依然给出递归和迭代两种方式，可以看出写法都非常简单，就是利用了二叉搜索树有序的特点。

23.验证二叉搜索树

[力扣题目链接]

给定一个二叉树，判断其是否是一个有效的二叉搜索树。

假设一个二叉搜索树具有如下特征：

- 节点的左子树只包含小于当前节点的数。
- 节点的右子树只包含大于当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

示例 1:

```
输入:
    2
   / \
  1   3
输出: true
```

示例 2:

```
输入:
    5
   / \
  1   4
     / \
    3   6
输出: false
解释: 输入为: [5,1,4,null,null,3,6]。
     根节点的值为 5 ，但是其右子节点值为 4 。
```

算法公开课

[《代码随想录》算法视频公开课](): [你对二叉搜索树了解的还不够！ | LeetCode：98.验证二叉搜索树](), **相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

要知道中序遍历下，输出的二叉搜索树节点的数值是有序序列。

有了这个特性，**验证二叉搜索树，就相当于变成了判断一个序列是不是递增的了。**

递归法

可以递归中序遍历将二叉搜索树转变成一个数组，代码如下：

```
vector<int> vec;
void traversal(TreeNode* root) {
    if (root == NULL) return;
    traversal(root->left);
    vec.push_back(root->val); // 将二叉搜索树转换为有序数组
    traversal(root->right);
}
```

然后只要比较一下，这个数组是否是有序的，**注意二叉搜索树中不能有重复元素**。

```
traversal(root);
for (int i = 1; i < vec.size(); i++) {
    // 注意要小于等于，搜索树里不能有相同元素
    if (vec[i] <= vec[i - 1]) return false;
}
return true;
```

整体代码如下：

```
class Solution {
private:
    vector<int> vec;
    void traversal(TreeNode* root) {
        if (root == NULL) return;
        traversal(root->left);
        vec.push_back(root->val); // 将二叉搜索树转换为有序数组
        traversal(root->right);
    }
public:
    bool isValidBST(TreeNode* root) {
        vec.clear(); // 不加这句在leetcode上也可以过，但最好加上
        traversal(root);
        for (int i = 1; i < vec.size(); i++) {
            // 注意要小于等于，搜索树里不能有相同元素
            if (vec[i] <= vec[i - 1]) return false;
        }
        return true;
    }
};
```

以上代码中，我们把二叉树转变为数组来判断，是最直观的，但其实不用转变成数组，可以在递归遍历的过程中直接判断是否有序。

这道题目比较容易陷入两个陷阱：

- 陷阱1

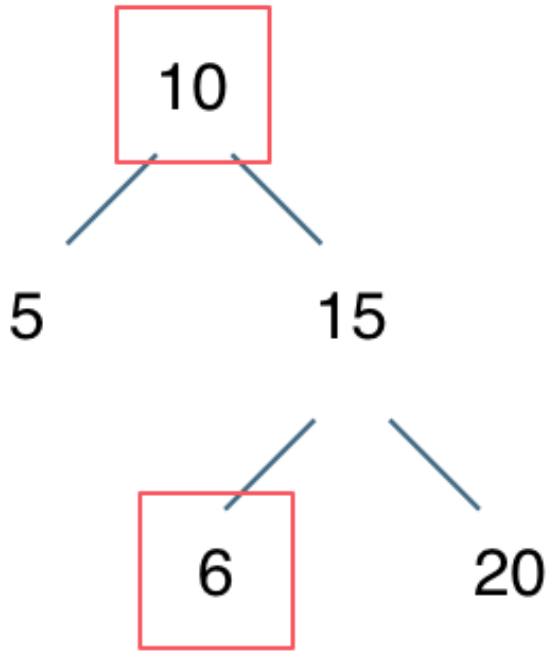
不能单纯的比较左节点小于中间节点，右节点大于中间节点就完事了。

写出了类似这样的代码：

```
if (root->val > root->left->val && root->val < root->right->val) {
    return true;
} else {
    return false;
}
```

我们要比较的是 左子树所有节点小于中间节点，右子树所有节点大于中间节点。所以以上代码的判断逻辑是错误的。

例如： [10,5,15,null,null,6,20] 这个case：



节点10大于左节点5，小于右节点15，但右子树里出现了一个6 这就不符合了！

- 陷阱2

样例中最小节点 可能是int的最小值，如果这样使用最小的int来比较也是不行的。

此时可以初始化比较元素为longlong的最小值。

问题可以进一步演进：如果样例中根节点的val 可能是longlong的最小值 又要怎么办呢？文中会解答。

了解这些陷阱之后我们来看一下代码应该怎么写：

递归三部曲：

- 确定递归函数，返回值以及参数

要定义一个longlong的全局变量，用来比较遍历的节点是否有序，因为后台测试数据中有int最小值，所以定义为longlong的类型，初始化为longlong最小值。

注意递归函数要有bool类型的返回值， 我们在二叉树：递归函数究竟什么时候需要返回值，什么时候不要返回值？ 中讲了，只有寻找某一条边（或者一个节点）的时候，递归函数会有bool类型的返回值。

其实本题是同样的道理，我们在寻找一个不符合条件的节点，如果没有找到这个节点就遍历了整个树，如果找到不符合的节点了，立刻返回。

代码如下：

```
long long maxVal = LONG_MIN; // 因为后台测试数据中有int最小值
bool isValidBST(TreeNode* root)
```

- 确定终止条件

如果是空节点 是不是二叉搜索树呢?

是的，二叉搜索树也可以为空！

代码如下：

```
if (root == NULL) return true;
```

- 确定单层递归的逻辑

中序遍历，一直更新maxVal，一旦发现maxVal >= root->val，就返回false，注意元素相同时候也要返回false。

代码如下：

```
bool left = isValidBST(root->left);         // 左

// 中序遍历，验证遍历的元素是不是从小到大
if (maxVal < root->val) maxVal = root->val; // 中
else return false;

bool right = isValidBST(root->right);       // 右
return left && right;
```

整体代码如下：

```
class Solution {
public:
    long long maxVal = LONG_MIN; // 因为后台测试数据中有int最小值
    bool isValidBST(TreeNode* root) {
        if (root == NULL) return true;

        bool left = isValidBST(root->left);
        // 中序遍历，验证遍历的元素是不是从小到大
        if (maxVal < root->val) maxVal = root->val;
        else return false;
        bool right = isValidBST(root->right);
```

```
        return left && right;
    }
};
```

以上代码是因为后台数据有int最小值测试用例，所以都把maxVal改成了longlong最小值。

如果测试数据中有 longlong的最小值，怎么办?

不可能在初始化一个更小的值了吧。 建议避免 初始化最小值，如下方法取到最左面节点的数值来比较。

代码如下：

```
class Solution {
public:
    TreeNode* pre = NULL; // 用来记录前一个节点
    bool isValidBST(TreeNode* root) {
        if (root == NULL) return true;
        bool left = isValidBST(root->left);

        if (pre != NULL && pre->val >= root->val) return false;
        pre = root; // 记录前一个节点

        bool right = isValidBST(root->right);
        return left && right;
    }
};
```

最后这份代码看上去整洁一些，思路也清晰。

迭代法

可以用迭代法模拟二叉树中序遍历，对前中后序迭代法生疏的同学可以看这两篇二叉树：听说递归能做的，栈也能做！，二叉树：前中后序迭代方式统一写法

迭代法中序遍历稍加改动就可以了，代码如下：

```
class Solution {
public:
    bool isValidBST(TreeNode* root) {
        stack<TreeNode*> st;
        TreeNode* cur = root;
        TreeNode* pre = NULL; // 记录前一个节点
        while (cur != NULL || !st.empty()) {
            if (cur != NULL) {
                st.push(cur);
                cur = cur->left;                // 左
            } else {
                cur = st.top();                 // 中
                st.pop();
                if (pre != NULL && cur->val <= pre->val)
```

```
                return false;
                pre = cur; //保存前一个访问的结点

                cur = cur->right;                // 右
            }
        }
        return true;
    }
};
```

在[二叉树：二叉搜索树登场！](#)中我们分明写出了痛哭流涕的简洁迭代法，怎么在这里不行了呢，因为本题是要验证二叉搜索树啊。

总结

这道题目是一个简单题，但对于没接触过的同学还是有难度的。

所以初学者刚开始学习算法的时候，看到简单题目没有思路很正常，千万别怀疑自己智商，学习过程都是这样的，大家智商都差不多，哈哈。

只要把基本类型的题目都做过，总结过之后，思路自然就开阔了。

> 利用二叉搜索树的特性搞起！

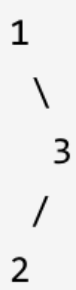
24.二叉搜索树的最小绝对差

[力扣题目链接](#)

给你一棵所有节点为非负值的二叉搜索树，请你计算树中任意两节点的差的绝对值的最小值。

示例：

输入：



输出：
1

解释：
最小绝对差为 1，其中 2 和 1 的差的绝对值为 1（或者 2 和 3）。

提示：树中至少有 2 个节点。

算法公开课

[《代码随想录》算法视频公开课](#)：[二叉搜索树中，需要掌握如何双指针遍历！| LeetCode：530.二叉搜索树的最小绝对差](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

题目中要求在二叉搜索树上任意两节点的差的绝对值的最小值。

注意是二叉搜索树，二叉搜索树可是有序的。

遇到在二叉搜索树上求什么最值啊，差值之类的，就把它想成在一个有序数组上求最值，求差值，这样就简单多了。

递归

那么二叉搜索树采用中序遍历，其实就是一个有序数组。

在一个有序数组上求两个数最小差值，这是不是就是一道送分题了。

最直观的想法，就是把二叉搜索树转换成有序数组，然后遍历一遍数组，就统计出来最小差值了。

代码如下：

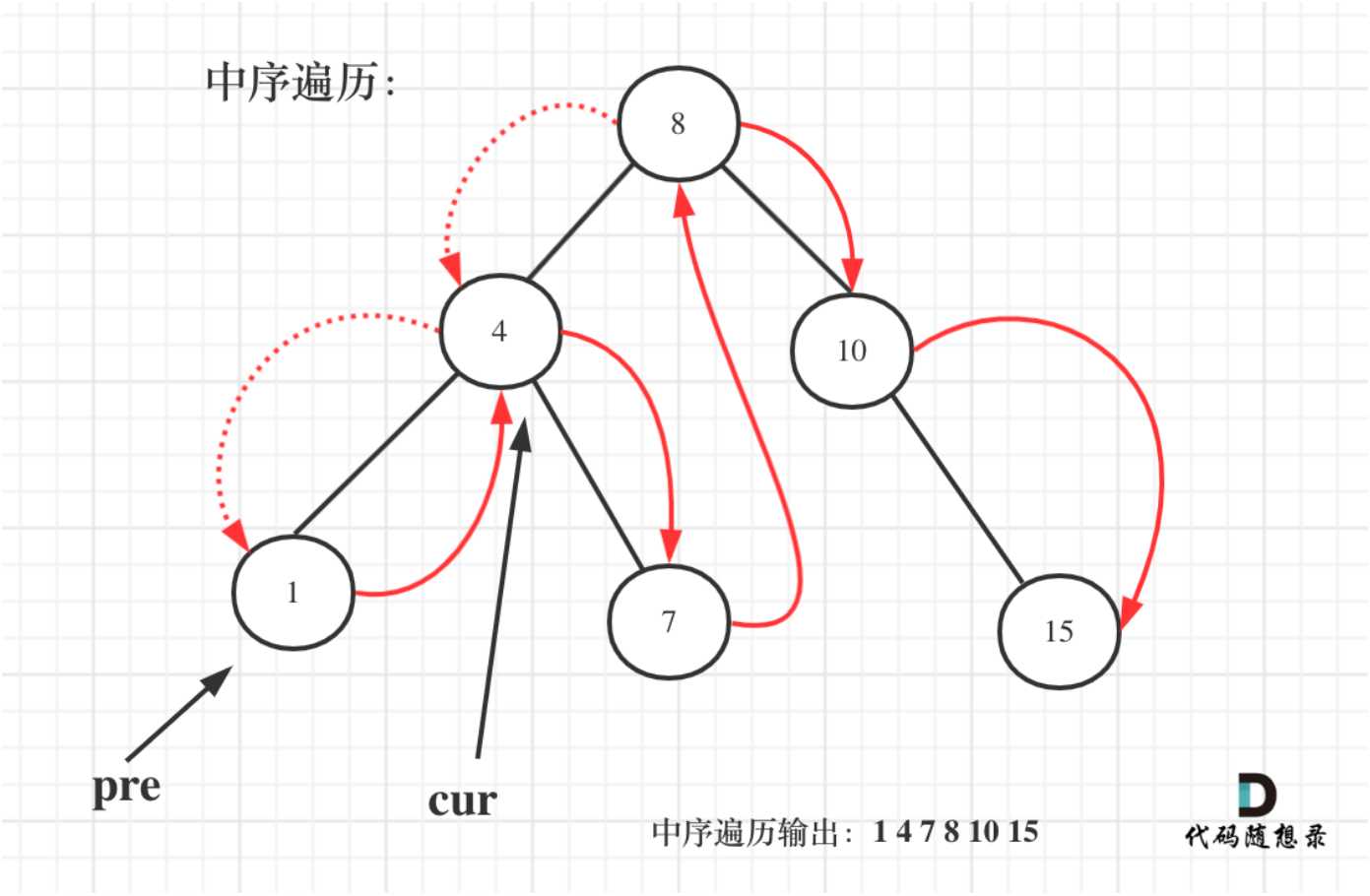
```
class Solution {
private:
vector<int> vec;
void traversal(TreeNode* root) {
```

```
    if (root == NULL) return;
    traversal(root->left);
    vec.push_back(root->val); // 将二叉搜索树转换为有序数组
    traversal(root->right);
}
public:
    int getMinimumDifference(TreeNode* root) {
        vec.clear();
        traversal(root);
        if (vec.size() < 2) return 0;
        int result = INT_MAX;
        for (int i = 1; i < vec.size(); i++) { // 统计有序数组的最小差值
            result = min(result, vec[i] - vec[i-1]);
        }
        return result;
    }
};
```

以上代码是把二叉搜索树转化为有序数组了，其实在二叉搜素树中序遍历的过程中，我们就可以直接计算了。

需要用一个pre节点记录一下cur节点的前一个节点。

如图：



一些同学不知道在递归中如何记录前一个节点的指针，其实实现起来是很简单的，大家只要看过一次，写过一次，就掌握了。

代码如下：

```
class Solution {
private:
int result = INT_MAX;
TreeNode* pre = NULL;
void traversal(TreeNode* cur) {
    if (cur == NULL) return;
    traversal(cur->left);   // 左
    if (pre != NULL){       // 中
        result = min(result, cur->val - pre->val);
    }
    pre = cur; // 记录前一个
    traversal(cur->right);  // 右
}
public:
    int getMinimumDifference(TreeNode* root) {
        traversal(root);
        return result;
    }
};
```

是不是看上去也并不复杂!

迭代

看过这两篇二叉树：听说递归能做的，栈也能做！，二叉树：前中后序迭代方式的写法就不能统一一下么？文章之后，不难写出两种中序遍历的迭代法。

下面我给出其中的一种中序遍历的迭代法，代码如下：

```
class Solution {
public:
    int getMinimumDifference(TreeNode* root) {
        stack<TreeNode*> st;
        TreeNode* cur = root;
        TreeNode* pre = NULL;
        int result = INT_MAX;
        while (cur != NULL || !st.empty()) {
            if (cur != NULL) { // 指针来访问节点，访问到最底层
                st.push(cur); // 将访问的节点放进栈
                cur = cur->left;                // 左
            } else {
                cur = st.top();
                st.pop();
                if (pre != NULL) {              // 中
                    result = min(result, cur->val - pre->val);
                }
                pre = cur;
                cur = cur->right;               // 右
            }
```

```
        }
        return result;
    }
};
```

总结

遇到在二叉搜索树上求什么最值，求差值之类的，都要思考一下二叉搜索树可是有序的，要利用好这一特点。

同时要学会在递归遍历的过程中如何记录前后两个指针，这也是一个小技巧，学会了还是很受用的。

后面我将继续介绍一系列利用二叉搜索树特性的题目。

> 二叉树上应该怎么求，二叉搜索树上又应该怎么求？

25.二叉搜索树中的众数

力扣题目链接

给定一个有相同值的二叉搜索树（BST），找出 BST 中的所有众数（出现频率最高的元素）。

假定 BST 有如下定义：

- 结点左子树中所含结点的值小于等于当前结点的值
- 结点右子树中所含结点的值大于等于当前结点的值
- 左子树和右子树都是二叉搜索树

例如：

给定 BST [1,null,2,2],

```
   1
    \
     2
    /
   2
```

返回[2].

提示：如果众数超过1个，不需考虑输出顺序

进阶：你可以不使用额外的空间吗？（假设由递归产生的隐式调用栈的开销不被计算在内）

算法公开课

[《代码随想录》算法视频公开课](#)：[不仅双指针，还有代码技巧可以惊艳到你！ | LeetCode：501.二叉搜索树中的众数](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

这道题目呢，递归法我从两个维度来讲。

首先如果不是二叉搜索树的话，应该怎么解题，是二叉搜索树，又应该如何解题，两种方式做一个比较，可以加深大家对二叉树的理解。

递归法

如果不是二叉搜索树

如果不是二叉搜索树，最直观的方法一定是把这个树都遍历了，用map统计频率，把频率排个序，最后取前面高频的元素的集合。

具体步骤如下：

1. 这个树都遍历了，用map统计频率

至于用前中后序哪种遍历也不重要，因为就是要全遍历一遍，怎么个遍历法都行，层序遍历都没毛病！

这里采用前序遍历，代码如下：

```
// map<int, int> key:元素，value:出现频率
void searchBST(TreeNode* cur, unordered_map<int, int>& map) { // 前序遍历
    if (cur == NULL) return ;
    map[cur->val]++; // 统计元素频率
    searchBST(cur->left, map);
    searchBST(cur->right, map);
    return ;
}
```

2. 把统计的出来的出现频率（即map中的value）排个序

有的同学可能可以想直接对map中的value排序，还真做不到，C++中如果使用std::map或者std::multimap可以对key排序，但不能对value排序。

所以要把map转化数组即vector，再进行排序，当然vector里面放的也是`pair<int, int>`类型的数据，第一个int为元素，第二个int为出现频率。

代码如下：

```
bool static cmp (const pair<int, int>& a, const pair<int, int>& b) {
    return a.second > b.second; // 按照频率从大到小排序
}

vector<pair<int, int>> vec(map.begin(), map.end());
sort(vec.begin(), vec.end(), cmp); // 给频率排个序
```

3. 取前面高频的元素

此时数组vector中已经是存放着按照频率排好序的pair，那么把前面高频的元素取出来就可以了。

代码如下：

```
result.push_back(vec[0].first);
for (int i = 1; i < vec.size(); i++) {
    // 取最高的放到result数组中
    if (vec[i].second == vec[0].second) result.push_back(vec[i].first);
    else break;
}
return result;
```

整体C++代码如下：

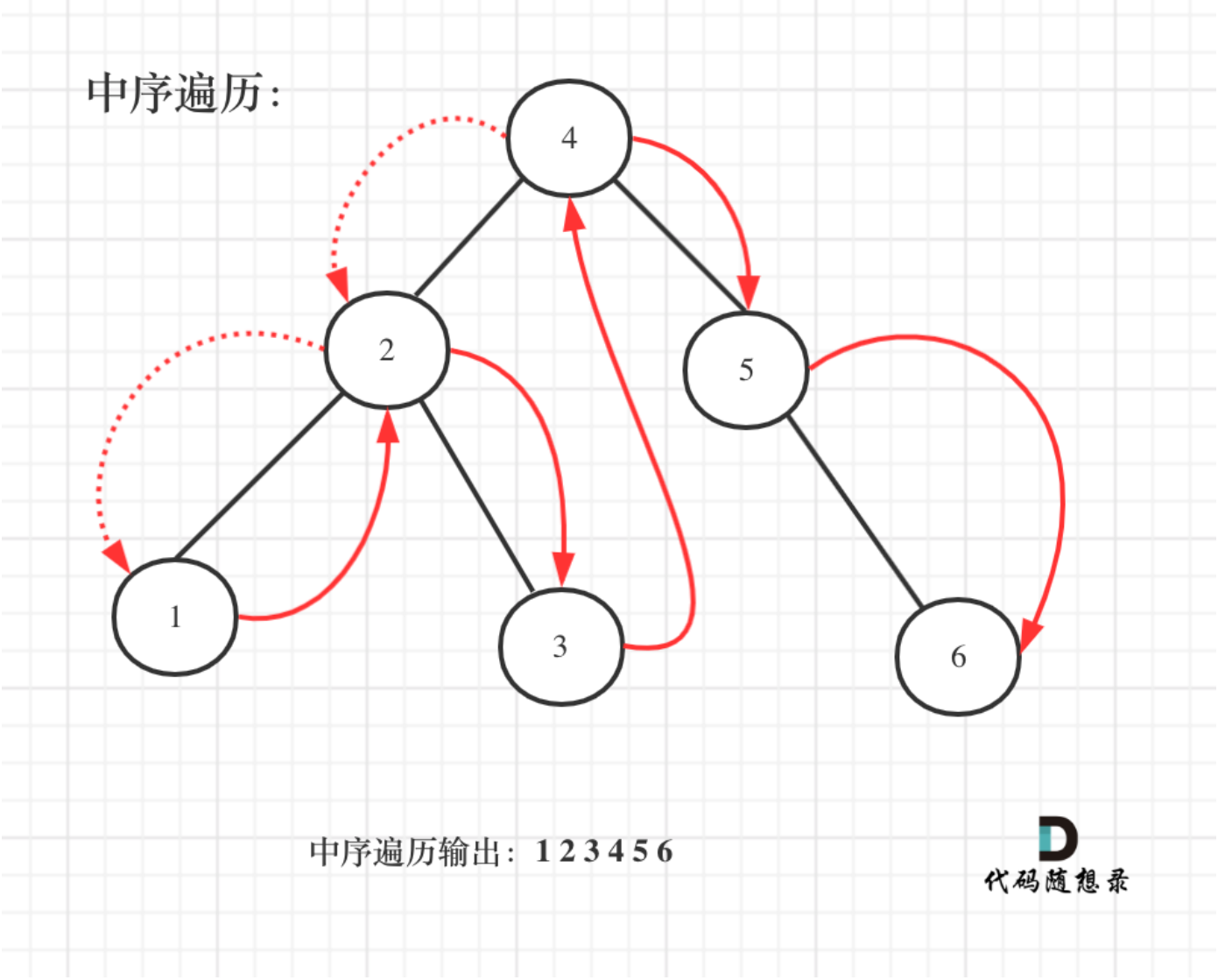
```
class Solution {
private:

void searchBST(TreeNode* cur, unordered_map<int, int>& map) { // 前序遍历
    if (cur == NULL) return ;
    map[cur->val]++; // 统计元素频率
    searchBST(cur->left, map);
    searchBST(cur->right, map);
    return ;
}
bool static cmp (const pair<int, int>& a, const pair<int, int>& b) {
    return a.second > b.second;
}
public:
    vector<int> findMode(TreeNode* root) {
        unordered_map<int, int> map; // key:元素，value:出现频率
        vector<int> result;
        if (root == NULL) return result;
        searchBST(root, map);
        vector<pair<int, int>> vec(map.begin(), map.end());
        sort(vec.begin(), vec.end(), cmp); // 给频率排个序
        result.push_back(vec[0].first);
        for (int i = 1; i < vec.size(); i++) {
            // 取最高的放到result数组中
            if (vec[i].second == vec[0].second) result.push_back(vec[i].first);
            else break;
        }
        return result;
    }
};
```

所以如果本题没有说是二叉搜索树的话，那么就按照上面的思路写！

是二叉搜索树

既然是搜索树，它中序遍历就是有序的。

如图：



中序遍历代码如下：

```
void searchBST(TreeNode* cur) {
    if (cur == NULL) return ;
    searchBST(cur->left);       // 左
     (处理节点)                 // 中
    searchBST(cur->right);      // 右
    return ;
}
```

遍历有序数组的元素出现频率，从头遍历，那么一定是相邻两个元素作比较，然后就把出现频率最高的元素输出就可以了。

关键是在有序数组上的话，好搞，在树上怎么搞呢?

这就考察对树的操作了。

在二叉树：搜索树的最小绝对差中我们就使用了pre指针和cur指针的技巧，这次又用上了。

弄一个指针指向前一个节点，这样每次cur（当前节点）才能和pre（前一个节点）作比较。

而且初始化的时候pre = NULL，这样当pre为NULL时候，我们就知道这是比较的第一个元素。

代码如下：

```
if (pre == NULL) { // 第一个节点
    count = 1; // 频率为1
} else if (pre->val == cur->val) { // 与前一个节点数值相同
    count++;
} else { // 与前一个节点数值不同
    count = 1;
}
pre = cur; // 更新上一个节点
```

此时又有问题了，因为要求最大频率的元素集合（注意是集合，不是一个元素，可以有多个众数），如果是数组上大家一般怎么办？

应该是先遍历一遍数组，找出最大频率（maxCount），然后再重新遍历一遍数组把出现频率为maxCount的元素放进集合。（因为众数有多个）

这种方式遍历了两遍数组。

那么我们遍历两遍二叉搜索树，把众数集合算出来也是可以的。

但这里其实只需要遍历一次就可以找到所有的众数。

那么如何只遍历一遍呢？

如果 频率count 等于 maxCount（最大频率），当然要把这个元素加入到结果集中（以下代码为result数组），代码如下：

```
if (count == maxCount) { // 如果和最大值相同，放进result中
    result.push_back(cur->val);
}
```

是不是感觉这里有问题，result怎么能轻易就把元素放进去了呢，万一，这个maxCount此时还不是真正最大频率呢。

所以下面要做如下操作：

频率count 大于 maxCount的时候，不仅要更新maxCount，而且要清空结果集（以下代码为result数组），因为结果集之前的元素都失效了。

```
if (count > maxCount) { // 如果计数大于最大值
    maxCount = count;   // 更新最大频率
    result.clear();     // 很关键的一步，不要忘记清空result，之前result里的元素都失效了
    result.push_back(cur->val);
}
```

关键代码都讲完了，完整代码如下：（**只需要遍历一遍二叉搜索树，就求出了众数的集合**）

```
class Solution {
private:
    int maxCount = 0; // 最大频率
    int count = 0; // 统计频率
    TreeNode* pre = NULL;
    vector<int> result;
    void searchBST(TreeNode* cur) {
        if (cur == NULL) return ;

        searchBST(cur->left);       // 左
                                    // 中
        if (pre == NULL) { // 第一个节点
            count = 1;
        } else if (pre->val == cur->val) { // 与前一个节点数值相同
            count++;
        } else { // 与前一个节点数值不同
            count = 1;
        }
        pre = cur; // 更新上一个节点

        if (count == maxCount) { // 如果和最大值相同，放进result中
            result.push_back(cur->val);
        }

        if (count > maxCount) { // 如果计数大于最大值频率
            maxCount = count;   // 更新最大频率
            result.clear();     // 很关键的一步，不要忘记清空result，之前result里的元素都失效了
            result.push_back(cur->val);
        }

        searchBST(cur->right);      // 右
        return ;
    }

public:
    vector<int> findMode(TreeNode* root) {
        count = 0;
        maxCount = 0;
        TreeNode* pre = NULL; // 记录前一个节点
        result.clear();

        searchBST(root);
        return result;
    }
};
```

迭代法

只要把中序遍历转成迭代，中间节点的处理逻辑完全一样。

二叉树前中后序转迭代，传送门：

- 二叉树：前中后序迭代法
- 二叉树：前中后序统一风格的迭代方式

下面我给出其中的一种中序遍历的迭代法，其中间处理逻辑一点都没有变（我从递归法直接粘过来的代码，连注释都没改，哈哈）

代码如下：

```
class Solution {
public:
    vector<int> findMode(TreeNode* root) {
        stack<TreeNode*> st;
        TreeNode* cur = root;
        TreeNode* pre = NULL;
        int maxCount = 0; // 最大频率
        int count = 0; // 统计频率
        vector<int> result;
        while (cur != NULL || !st.empty()) {
            if (cur != NULL) { // 指针来访问节点，访问到最底层
                st.push(cur); // 将访问的节点放进栈
                cur = cur->left;                // 左
            } else {
                cur = st.top();
                st.pop();                       // 中
                if (pre == NULL) { // 第一个节点
                    count = 1;
                } else if (pre->val == cur->val) { // 与前一个节点数值相同
                    count++;
                } else { // 与前一个节点数值不同
                    count = 1;
                }
                if (count == maxCount) { // 如果和最大值相同，放进result中
                    result.push_back(cur->val);
                }

                if (count > maxCount) { // 如果计数大于最大值频率
                    maxCount = count;   // 更新最大频率
                    result.clear();     // 很关键的一步，不要忘记清空result，之前result里的元素都失效了
                    result.push_back(cur->val);
                }
                pre = cur;
                cur = cur->right;               // 右
            }
```

```
        }
        return result;
    }
};
```

总结

本题在递归法中，我给出了如果是普通二叉树，应该怎么求众数。

知道了普通二叉树的做法时候，我再进一步给出二叉搜索树又应该怎么求众数，这样鲜明的对比，相信会对二叉树又有更深层次的理解了。

在递归遍历二叉搜索树的过程中，我还介绍了一个统计最高出现频率元素集合的技巧， 要不然就要遍历两次二叉搜索树才能把这个最高出现频率元素的集合求出来。

为什么没有这个技巧一定要遍历两次呢？ 因为要求的是集合，会有多个众数，如果规定只有一个众数，那么就遍历一次稳稳的了。

最后我依然给出对应的迭代法，其实就是迭代法中序遍历的模板加上递归法中中间节点的处理逻辑，分分钟就可以写出来，中间逻辑的代码我都是从递归法中直接粘过来的。

求二叉搜索树中的众数其实是一道简单题，但大家可以发现我写了这么一大篇幅的文章来讲解，主要是为了尽量从各个角度对本题进剖析，帮助大家更快更深入理解二叉树。

> **需要强调的是 leetcode上的耗时统计是非常不准确的，看个大概就行，一样的代码耗时可以差百分之50以上**，所以leetcode的耗时统计别太当回事，知道理论上的效率优劣就行了。

> 本来是打算将二叉树和二叉搜索树的公共祖先问题一起讲，后来发现篇幅过长了，只能先说一说二叉树的公共祖先问题。

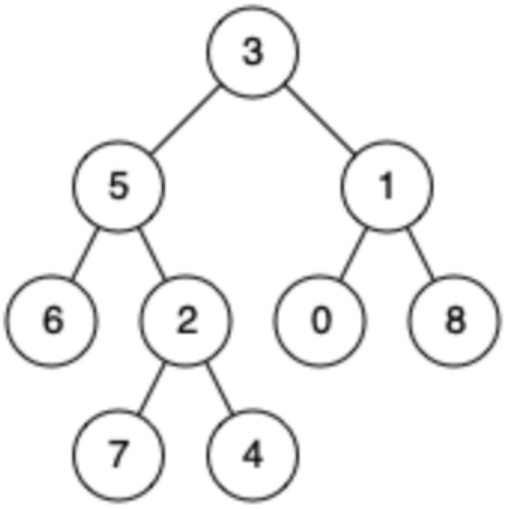
26. 二叉树的最近公共祖先

[力扣题目链接]

给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个结点 p、q，最近公共祖先表示为一个结点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

例如，给定如下二叉树: root = [3,5,1,6,2,0,8,null,null,7,4]



示例 1:
输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
输出: 3
解释: 节点 5 和节点 1 的最近公共祖先是节点 3。

示例 2:
输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
输出: 5
解释: 节点 5 和节点 4 的最近公共祖先是节点 5。因为根据定义最近公共祖先节点可以为节点本身。

说明:

- 所有节点的值都是唯一的。
- p、q 为不同节点且均存在于给定的二叉树中。

算法公开课

《代码随想录》算法视频公开课：自底向上查找，有点难度！ | LeetCode：236. 二叉树的最近公共祖先，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

遇到这个题目首先想的是要是能自底向上查找就好了，这样就可以找到公共祖先了。

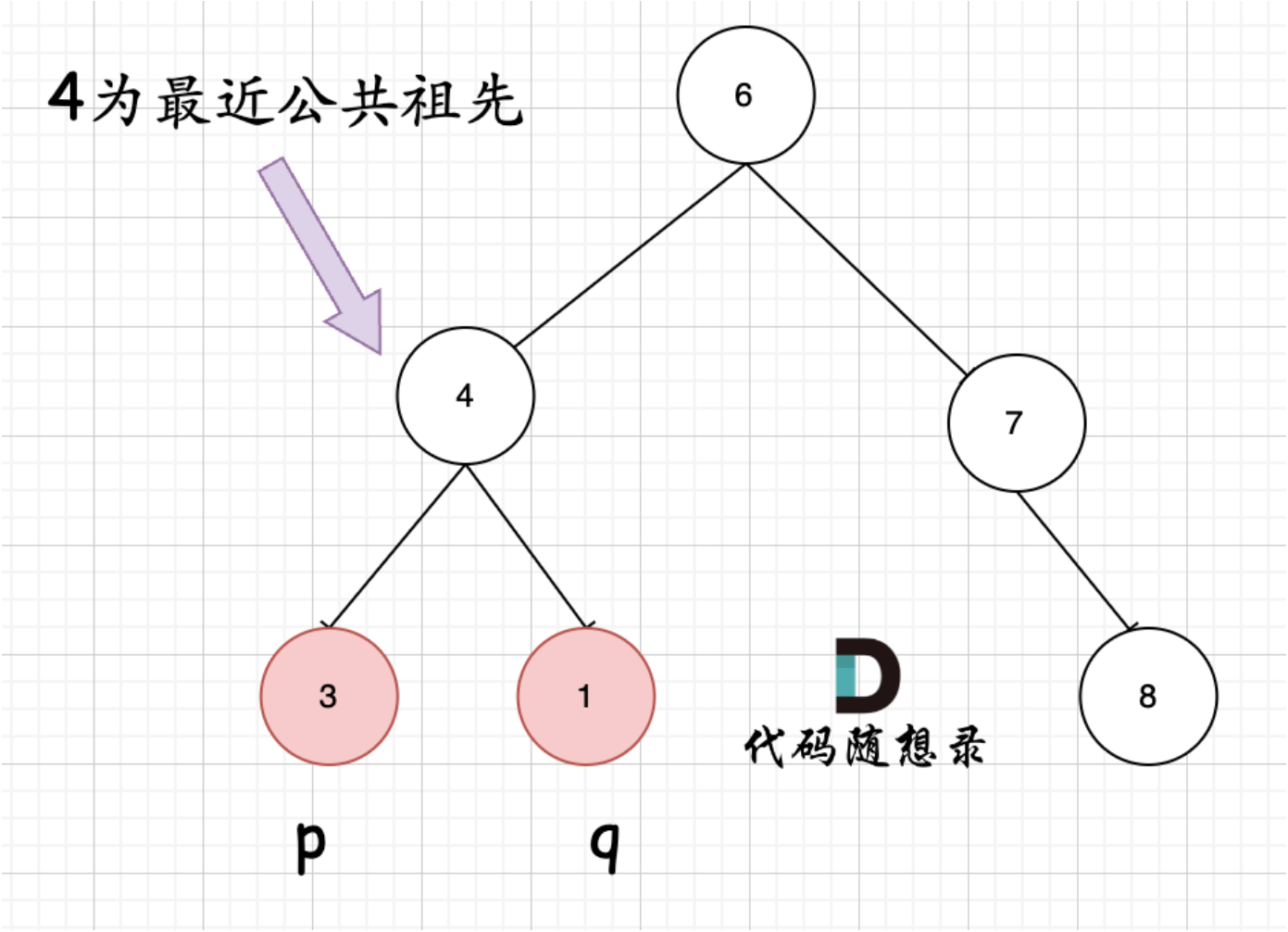
那么二叉树如何可以自底向上查找呢?

回溯啊，二叉树回溯的过程就是从低到上。

后序遍历（左右中）就是天然的回溯过程，可以根据左右子树的返回值，来处理中节点的逻辑。

接下来就看如何判断一个节点是节点q和节点p的公共祖先呢。

首先最容易想到的一个情况：如果找到一个节点，发现左子树出现结点p，右子树出现节点q，或者 左子树出现结点q，右子树出现节点p，那么该节点就是节点p和q的最近公共祖先。 即情况一：

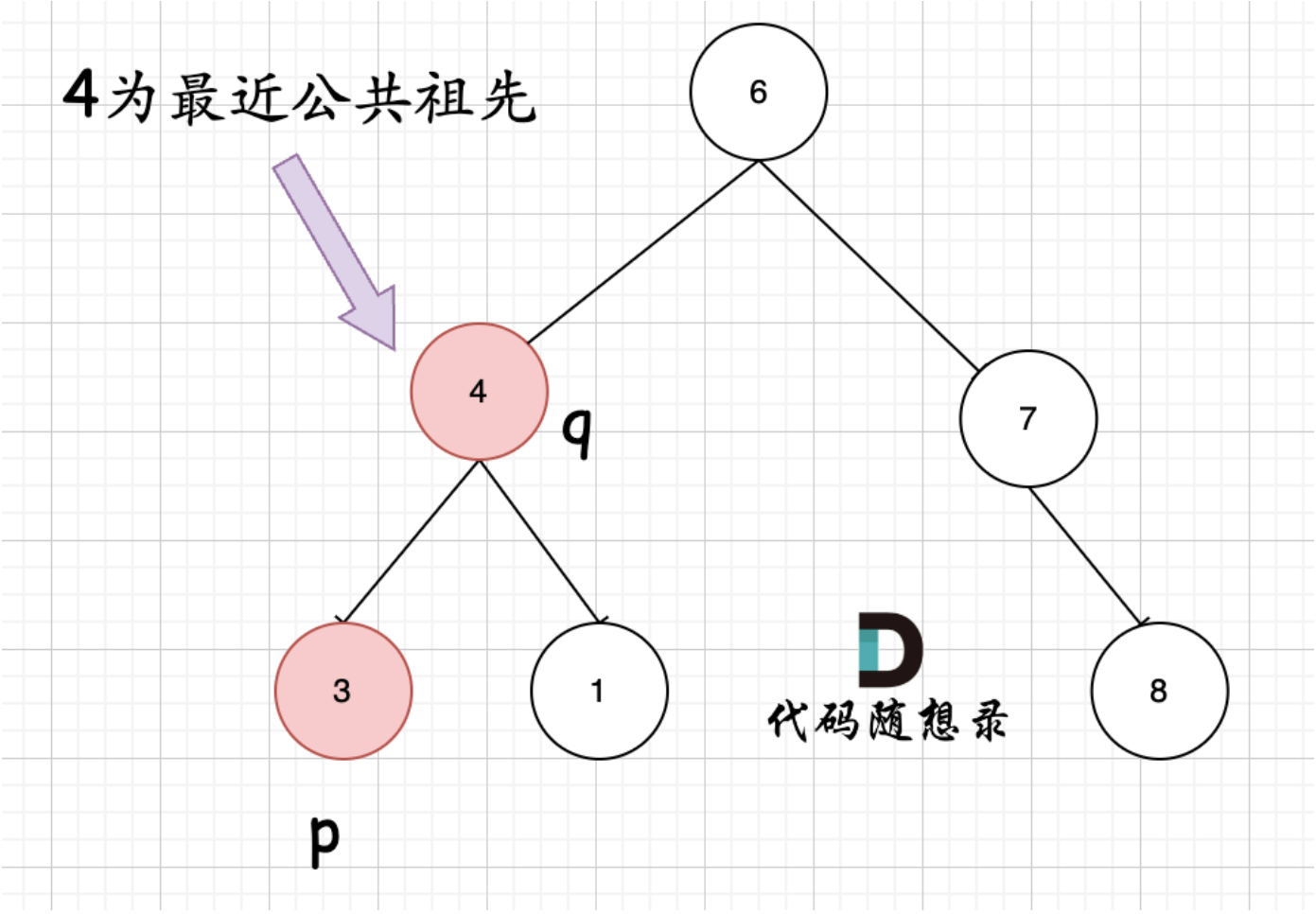


判断逻辑是 如果递归遍历遇到q，就将q返回，遇到p 就将p返回，那么如果 左右子树的返回值都不为空，说明此时的中节点，一定是q 和p 的最近祖先。

那么有录友可能疑惑，会不会左子树 遇到q 返回，右子树也遇到q返回，这样并没有找到 q 和p的最近祖先。

这么想的录友，要审题了，题目强调：**二叉树节点数值是不重复的，而且一定存在 q 和 p**。

但是很多人容易忽略一个情况，就是节点本身p(q)，它拥有一个子孙节点q(p)。 情况二：



其实情况一 和 情况二 代码实现过程都是一样的，也可以说，实现情况一的逻辑，顺便包含了情况二。

因为遇到 q 或者 p 就返回，这样也包含了 q 或者 p 本身就是 公共祖先的情况。

这一点是很多录友容易忽略的，在下面的代码讲解中，可以再去体会。

递归三部曲：

- 确定递归函数返回值以及参数

需要递归函数返回值，来告诉我们是否找到节点q或者p，那么返回值为bool类型就可以了。

但我们还要返回最近公共节点，可以利用上题目中返回值是TreeNode * ，那么如果遇到p或者q，就把q或者p返回，返回值不为空，就说明找到了q或者p。

代码如下：

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q)
```

- 确定终止条件

遇到空的话，因为树都是空了，所以返回空。

那么我们来说一说，如果 root == q，或者 root == p，说明找到 q p ，则将其返回，这个返回值，后面在中节点的处理过程中会用到，那么中节点的处理逻辑，下面讲解。

代码如下：

```
if (root == q || root == p || root == NULL) return root;
```

- 确定单层递归逻辑

值得注意的是 本题函数有返回值，是因为回溯的过程需要递归函数的返回值做判断，但本题我们依然要遍历树的所有节点。

我们在二叉树：递归函数究竟什么时候需要返回值，什么时候不要返回值？中说了 递归函数有返回值就是要遍历某一条边，但有返回值也要看如何处理返回值！

如果递归函数有返回值，如何区分要搜索一条边，还是搜索整个树呢？

搜索一条边的写法：

```
if (递归函数(root->left)) return ;

if (递归函数(root->right)) return ;
```

搜索整个树写法：

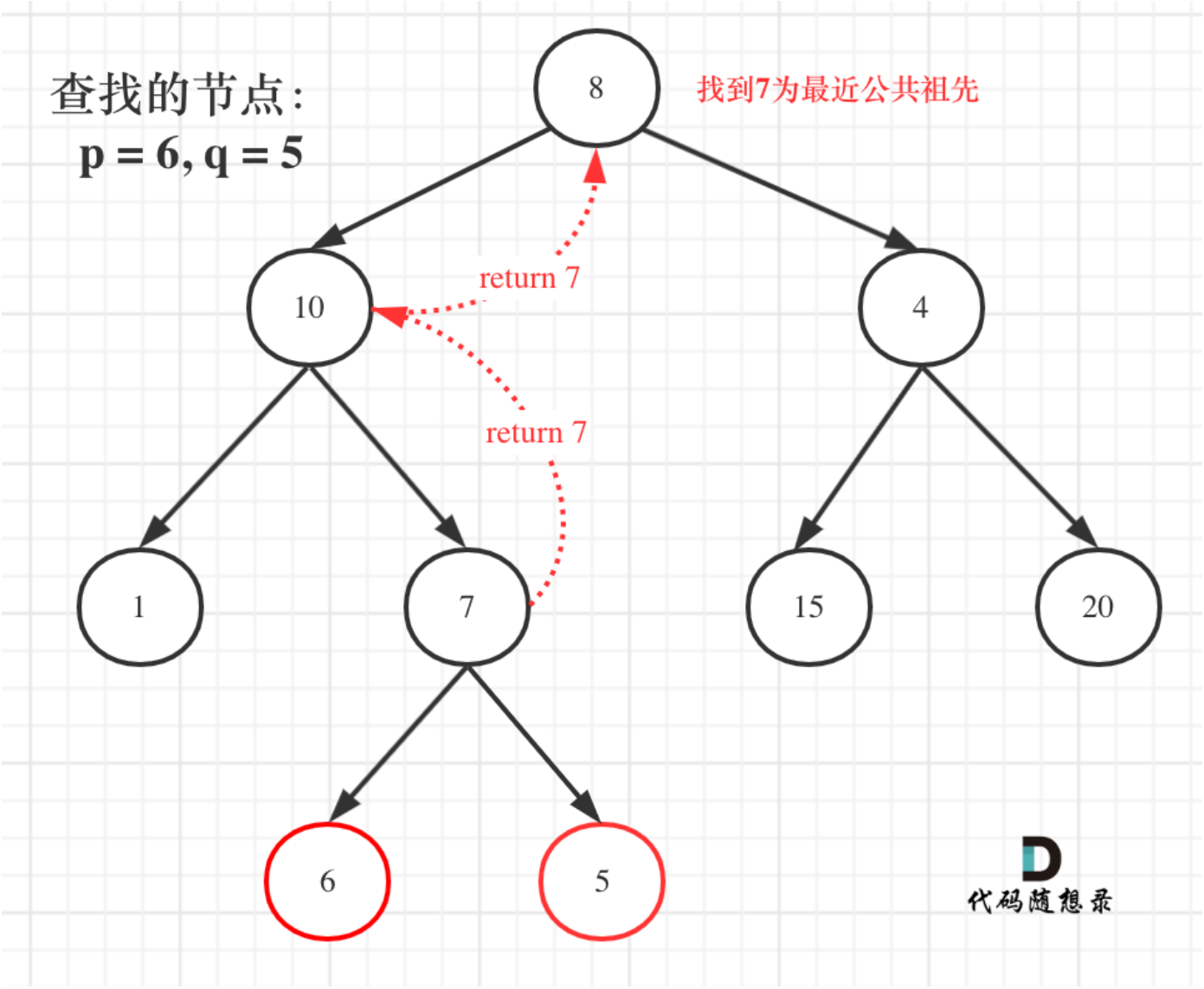
```
left = 递归函数(root->left);  // 左
right = 递归函数(root->right); // 右
left与right的逻辑处理;         // 中
```

看出区别了没？

在递归函数有返回值的情况下：如果要搜索一条边，递归函数返回值不为空的时候，立刻返回，如果搜索整个树，直接用一个变量left、right接住返回值，这个left、right后序还有逻辑处理的需要，也就是后序遍历中处理中间节点的逻辑（也是回溯）。

那么为什么要遍历整棵树呢？直观上来看，找到最近公共祖先，直接一路返回就可以了。

如图：



就像图中一样直接返回7，多美滋滋。

但事实上还要遍历根节点右子树（即使此时已经找到了目标节点了），也就是图中的节点4、15、20。

因为在如下代码的后序遍历中，如果想利用left和right做逻辑处理， 不能立刻返回，而是要等left与right逻辑处理完之后才能返回。

```
left = 递归函数(root->left);  // 左
right = 递归函数(root->right); // 右
left与right的逻辑处理;         // 中
```

所以此时大家要知道我们要遍历整棵树。知道这一点，对本题就有一定深度的理解了。

那么先用left和right接住左子树和右子树的返回值，代码如下：

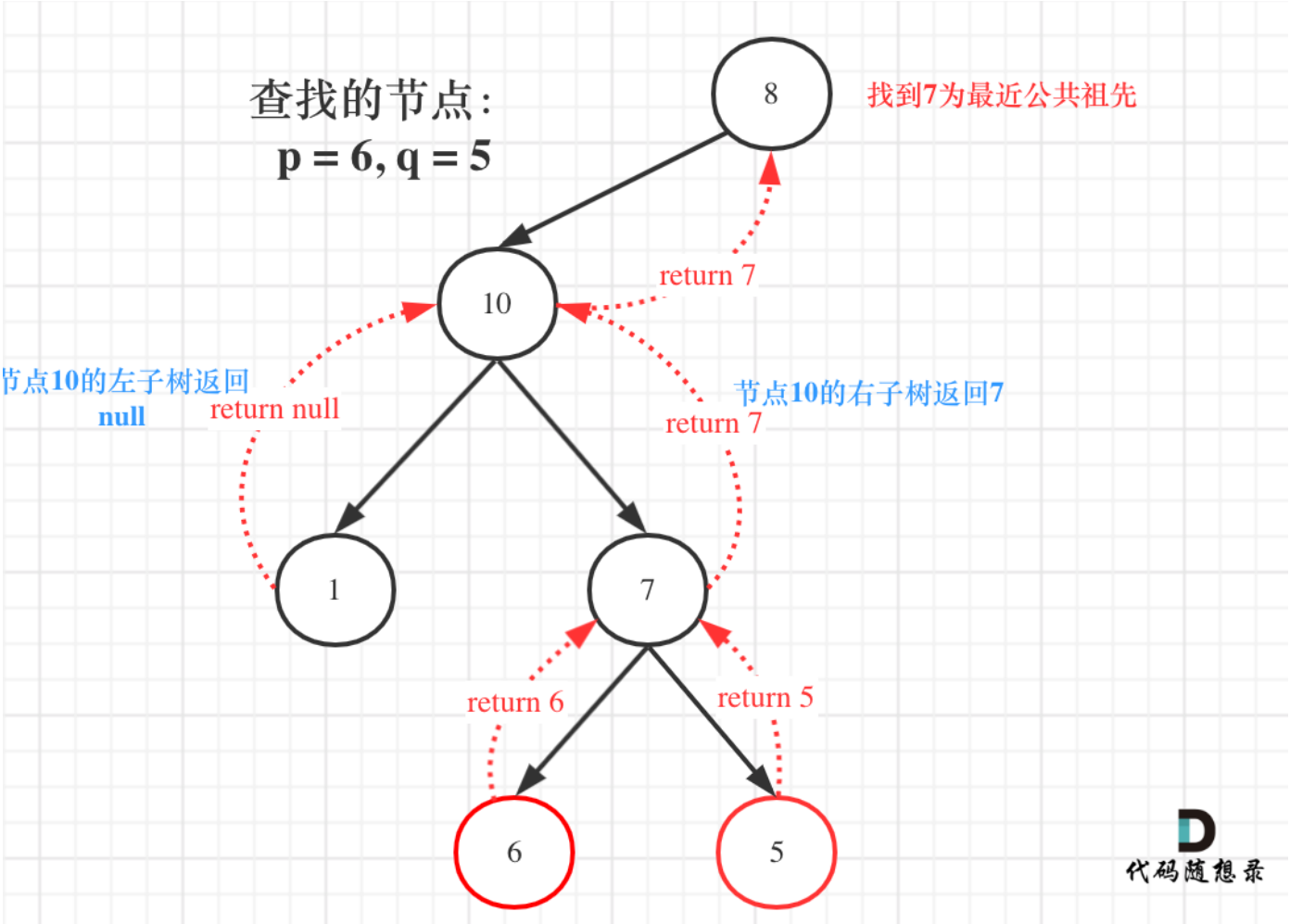
```
TreeNode* left = lowestCommonAncestor(root->left, p, q);
TreeNode* right = lowestCommonAncestor(root->right, p, q);
```

如果left 和 right都不为空，说明此时root就是最近公共节点。这个比较好理解

如果left为空，right不为空，就返回right，说明目标节点是通过right返回的，反之依然。

这里有的同学就理解不了了，为什么left为空，right不为空，目标节点通过right返回呢?

如图：



图中节点10的左子树返回null，右子树返回目标值7，那么此时节点10的处理逻辑就是把右子树的返回值（最近公共祖先7）返回上去！

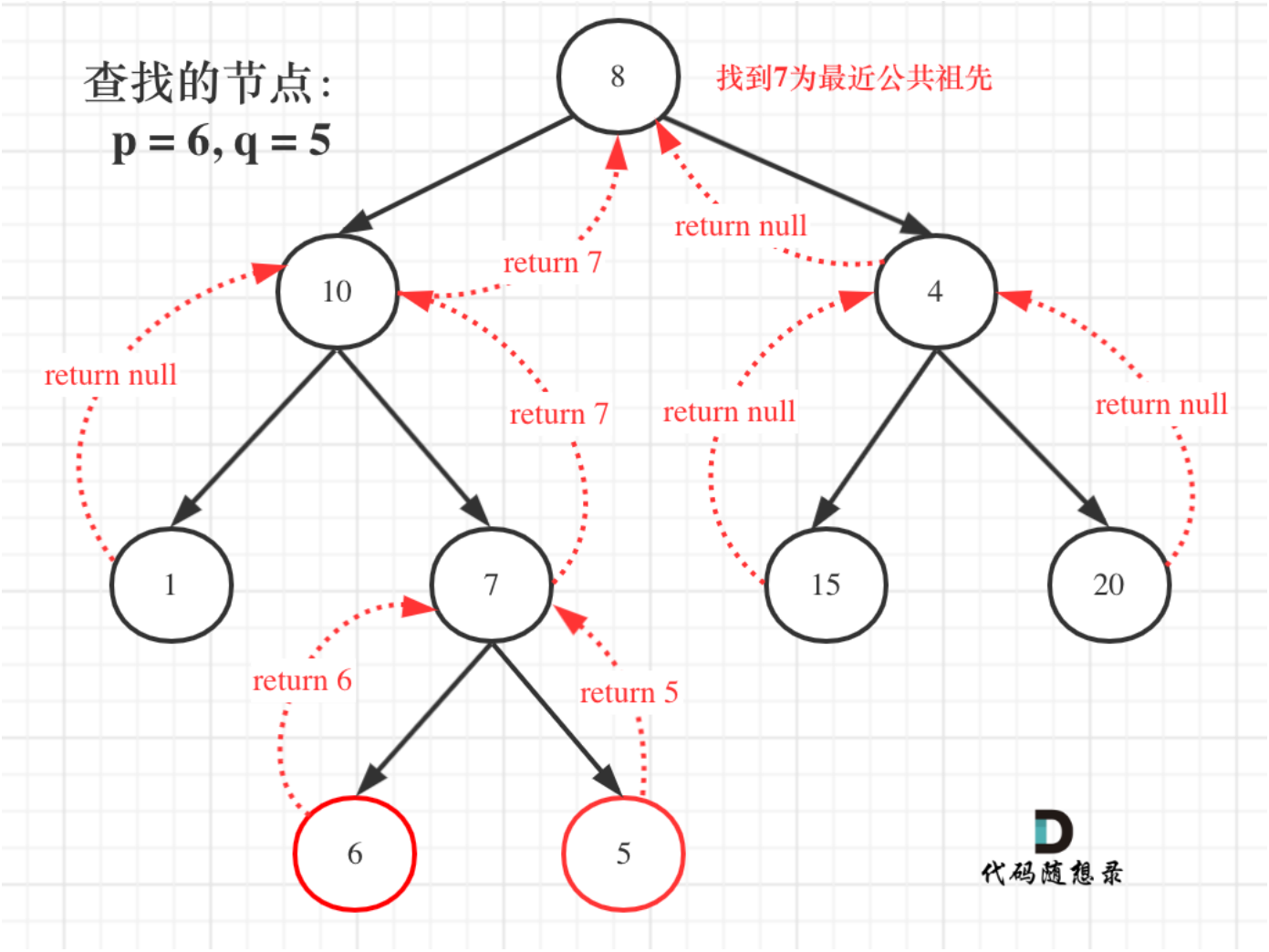
这里也很重要，可能刷过这道题目的同学，都不清楚结果究竟是如何从底层一层一层传到头结点的。

那么如果left和right都为空，则返回left或者right都是可以的，也就是返回空。

代码如下：

```
if (left == NULL && right != NULL) return right;
else if (left != NULL && right == NULL) return left;
else  { //  (left == NULL && right == NULL)
    return NULL;
}
```

那么寻找最小公共祖先，完整流程图如下：



从图中，大家可以看到，我们是如何回溯遍历整棵二叉树，将结果返回给头结点的！

整体代码如下：

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == q || root == p || root == NULL) return root;
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);
        if (left != NULL && right != NULL) return root;

        if (left == NULL && right != NULL) return right;
        else if (left != NULL && right == NULL) return left;
        else  { //  (left == NULL && right == NULL)
            return NULL;
        }

    }
};
```

稍加精简，代码如下：

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == q || root == p || root == NULL) return root;
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);
        if (left != NULL && right != NULL) return root;
        if (left == NULL) return right;
        return left;
    }
};
```

总结

这道题目刷过的同学未必真正了解这里面回溯的过程，以及结果是如何一层一层传上去的。

那么我给大家归纳如下三点：

1. 求最小公共祖先，需要从底向上遍历，那么二叉树，只能通过后序遍历（即：回溯）实现从底向上的遍历方式。
2. 在回溯的过程中，必然要遍历整棵二叉树，即使已经找到结果了，依然要把其他节点遍历完，因为要使用递归函数的返回值（也就是代码中的left和right）做逻辑判断。
3. 要理解如果返回值left为空，right不为空为什么要返回right，为什么可以用返回right传给上一层结果。

可以说这里每一步，都是有难度的，都需要对二叉树，递归和回溯有一定的理解。

本题没有给出迭代法，因为迭代法不适合模拟回溯的过程。理解递归的解法就够了。

---

27. 本周小结！（二叉树系列四）

> 这已经是二叉树的第四周总结了，二叉树是非常重要的数据结构，也是面试中的常客，所以有必要一步一步帮助大家彻底掌握二叉树！

周一

在二叉树：合并两个二叉树中讲解了如何合并两个二叉树，平时我们都习惯了操作一个二叉树，一起操作两个树可能还有点陌生。

其实套路是一样，只不过一起操作两个树的指针，我们之前讲过求 二叉树：我对称么？ 的时候，已经初步涉及到了一起遍历两棵二叉树了。

迭代法中，一般一起操作两个树都是使用队列模拟类似层序遍历，同时处理两个树的节点，这种方式最好理解，如果用模拟递归的思路的话，要复杂一些。

周二

周二开始讲解一个新的树，二叉搜索树，开始要换一个思路了，如果没有利用好二叉搜索树的特性，就容易把简单题做成了难题了。

学习二叉搜索树的特性，还是比较容易的。

大多是二叉搜索树的题目，其实都离不开中序遍历，因为这样就是有序的。

至于迭代法，相信大家看到文章中如此简单的迭代法的时候，都会感动的痛哭流涕。

周三

了解了二搜索树的特性之后， 开始验证一棵二叉树是不是二叉搜索树。

首先在此强调一下二叉搜索树的特性：

- 节点的左子树只包含小于当前节点的数。
- 节点的右子树只包含大于当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

那么我们在验证二叉搜索树的时候，有两个陷阱：

- 陷阱一

不能单纯的比较左节点小于中间节点，右节点大于中间节点就完事了，而是左子树都小于中间节点，右子树都大于中间节点。

- 陷阱二

在一个有序序列求最值的时候，不要定义一个全局遍历，然后遍历序列更新全局变量求最值。因为最值可能就是int或者 longlong的最小值。

推荐要通过前一个数值（pre）和后一个数值比较（cur），得出最值。

在二叉树中通过两个前后指针作比较，会经常用到。

本文二叉树：我是不是一棵二叉搜索树中迭代法中为什么没有周一那篇那么简洁了呢，因为本篇是验证二叉搜索树，前提默认它是一棵普通二叉树，所以还是要回归之前老办法。

周四

了解了二叉搜索树，并且知道如何判断二叉搜索树，本篇就很简单了。

要知道二叉搜索树和中序遍历是好朋友！

在二叉树：搜索树的最小绝对差中强调了要利用搜索树的特性，把这道题目想象成在一个有序数组上求两个数最小差值，这就是一道送分题了。

需要明确：在有序数组求任意两数最小值差等价于相邻两数的最小值差。

同样本题也需要用pre节点记录cur节点的前一个节点。（这种写法一定要掌握）

周五

此时大家应该知道遇到二叉搜索树，就想是有序数组，那么在二叉搜索树中求二叉搜索树众数就很简单了。

在二叉树：我的众数是多少？ 中我给出了如果是普通二叉树，应该如何求众数的集合，然后进一步讲解了二叉搜索树应该如何求众数集合。

在求众数集合的时候有一个技巧，因为题目中众数是可以有多个的，所以一般的方法需要遍历两遍才能求出众数的集合。

但可以遍历一遍就可以求众数集合，使用了适时清空结果集的方法，这个方法还是很巧妙的。相信仔细读了文章的同学会惊呼其巧妙！

所以大家不要看题目简单了，就不动手做了，我选的题目，一般不会简单到不用动手的程度，哈哈。

周六

在二叉树：公共祖先问题中，我们开始讲解如何在二叉树中求公共祖先的问题，本来是打算和二叉搜索树一起讲的，但发现篇幅过长，所以先讲二叉树的公共祖先问题。

如果找到一个节点，发现左子树出现结点p，右子树出现节点q，或者 左子树出现结点q，右子树出现节点p，那么该节点就是节点p和q的最近公共祖先。

这道题目的看代码比较简单，而且好像也挺好理解的，但是如果把每一个细节理解到位，还是不容易的。

主要思考如下几点：

- 如何从底向上遍历?
- 遍历整棵树，还是遍历局部树?
- 如何把结果传到根节点的?

这些问题都需要弄清楚，上来直接看代码的话，是可能想不到这些细节的。

公共祖先问题，还是有难度的，初学者还是需要慢慢消化！

总结

本周我们讲了如何合并两个二叉树，了解了如何操作两个二叉树。

然后开始另一种树：二叉搜索树，了解二叉搜索树的特性，然后判断一棵二叉树是不是二叉搜索树。

了解以上知识之后，就开始利用其特性，做一些二叉搜索树上的题目，求最小绝对差，求众数集合。

接下来，开始求二叉树与二叉搜索树的公共祖先问题，单篇篇幅原因，先单独介绍普通二叉树如何求最近公共祖先。

现在已经讲过了几种二叉树了，二叉树，二叉平衡树，完全二叉树，二叉搜索树，后面还会有平衡二叉搜索树。 那么一些同学难免会有混乱了，我针对如下三个问题，帮大家在捋顺一遍：

1. 平衡二叉搜索树是不是二叉搜索树和平衡二叉树的结合?

是的，是二叉搜索树和平衡二叉树的结合。

2. 平衡二叉树与完全二叉树的区别在于底层节点的位置?

是的，完全二叉树底层必须是从左到右连续的，且次底层是满的。

3. 堆是完全二叉树和排序的结合，而不是平衡二叉搜索树?

堆是一棵完全二叉树，同时保证父子节点的顺序关系（有序）。**但完全二叉树一定是平衡二叉树，堆的排序是父节点大于子节点，而搜索树是父节点大于左孩子，小于右孩子，所以堆不是平衡二叉搜索树。**

大家如果每天坚持跟下来，会发现又是充实的一周！

---

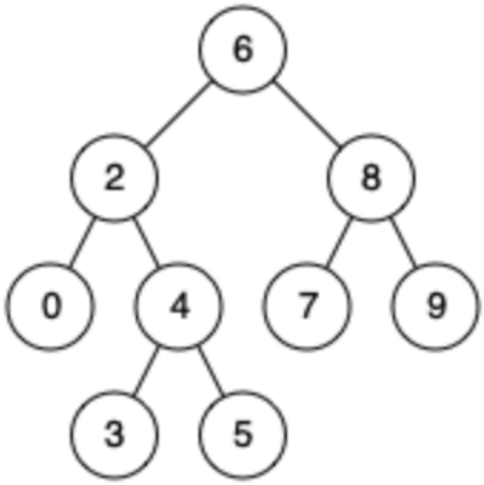
28. 二叉搜索树的最近公共祖先

力扣题目链接

给定一个二叉搜索树, 找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个结点 p、q，最近公共祖先表示为一个结点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

例如，给定如下二叉搜索树: root = [6,2,8,0,4,7,9,null,null,3,5]



示例 1:

- 输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
- 输出: 6
- 解释: 节点 2 和节点 8 的最近公共祖先是 6。

示例 2:

- 输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
- 输出: 2
- 解释: 节点 2 和节点 4 的最近公共祖先是 2, 因为根据定义最近公共祖先节点可以为节点本身。

说明:

- 所有节点的值都是唯一的。
- p、q 为不同节点且均存在于给定的二叉搜索树中。

算法公开课

《代码随想录》算法视频公开课：二叉搜索树找祖先就有点不一样了！| 235. 二叉搜索树的最近公共祖先，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

做过二叉树：公共祖先问题题目的同学应该知道，利用回溯从底向上搜索，遇到一个节点的左子树里有p，右子树里有q，那么当前节点就是最近公共祖先。

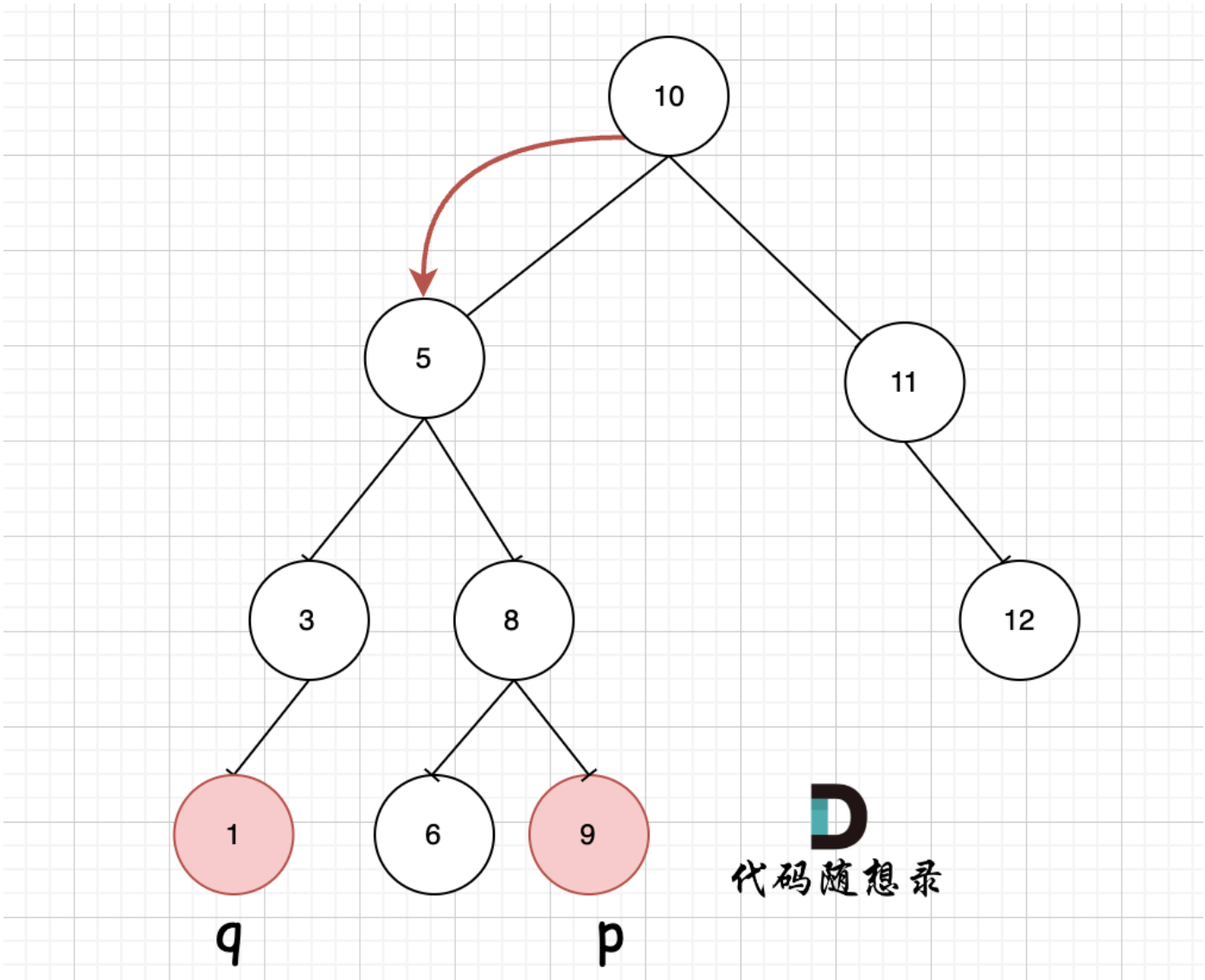
那么本题是二叉搜索树，二叉搜索树是有序的，那得好好利用一下这个特点。

在有序树里，如果判断一个节点的左子树里有p，右子树里有q呢?

因为是有序树，所有 如果 中间节点是 q 和 p 的公共祖先，那么 中节点的数组 一定是在 [p, q]区间的。即 中节点 > p && 中节点 < q 或者 中节点 > q && 中节点 < p。

那么只要从上到下去遍历，遇到 cur节点是数值在[p, q]区间中则一定可以说明该节点cur就是q 和 p的公共祖先。那问题来了，**一定是最近公共祖先吗**?

如图，我们从根节点搜索，第一次遇到 cur节点是数值在[p, q]区间中，即 节点5，此时可以说明 p 和 q 一定分别存在于 节点 5的左子树，和右子树中。



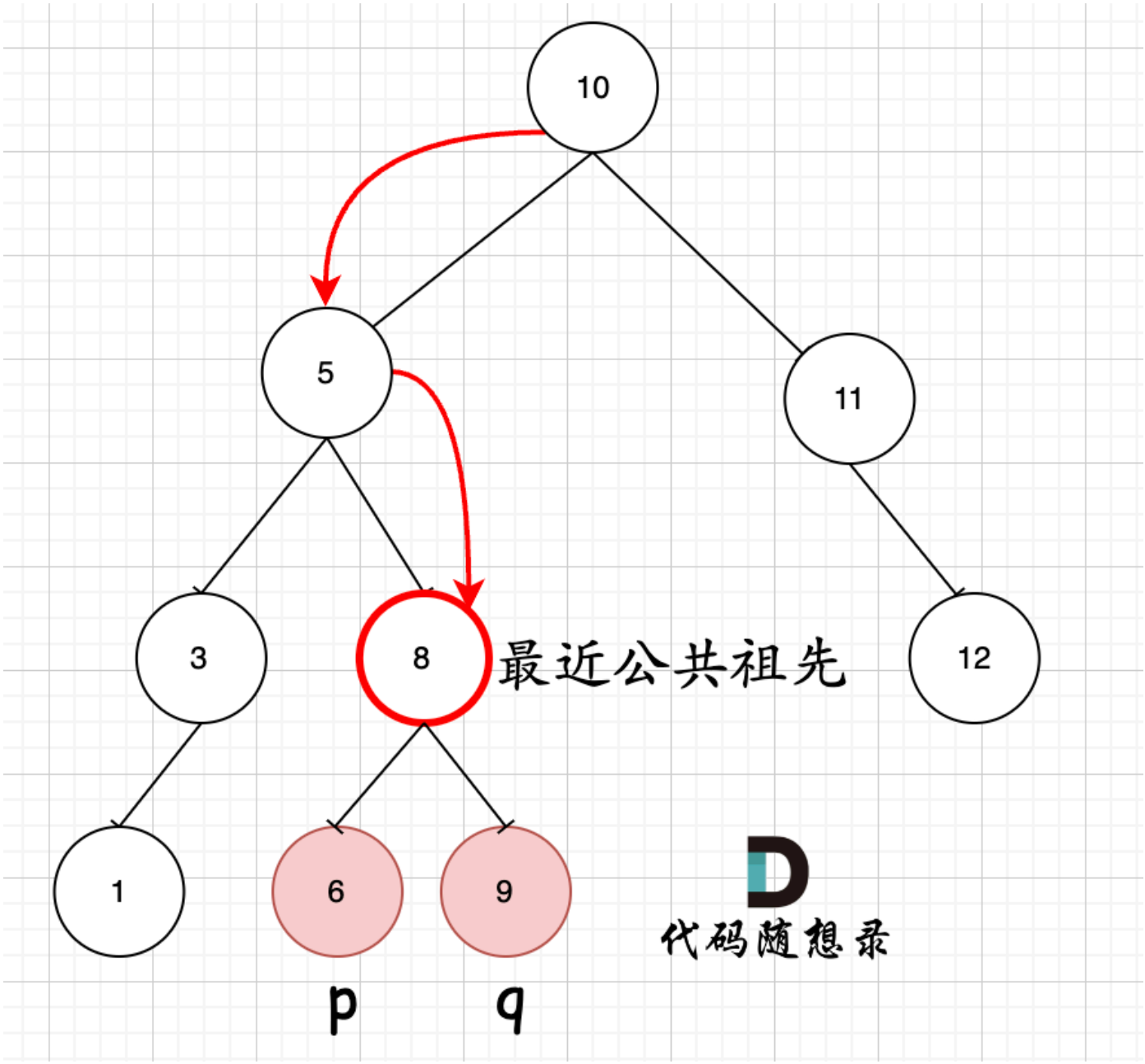
此时节点5是不是最近公共祖先？ 如果 从节点5继续向左遍历，那么将错过成为q的祖先， 如果从节点5继续向右遍历则错过成为p的祖先。

所以当我们从上向下去递归遍历，第一次遇到 cur节点是数值在[p, q]区间中，那么cur就是 p和q的最近公共祖先。

理解这一点，本题就很好解了。

而递归遍历顺序，本题就不涉及到 前中后序了（这里没有中节点的处理逻辑，遍历顺序无所谓了）。

如图所示：p为节点6，q为节点9



可以看出直接按照指定的方向，就可以找到节点8，为最近公共祖先，而且不需要遍历整棵树，找到结果直接返回！

递归法

递归三部曲如下：

- 确定递归函数返回值以及参数

参数就是当前节点，以及两个结点 p、q。

返回值是要返回最近公共祖先，所以是TreeNode * 。

代码如下：

```
TreeNode* traversal(TreeNode* cur, TreeNode* p, TreeNode* q)
```

- 确定终止条件

遇到空返回就可以了，代码如下：

```
if (cur == NULL) return cur;
```

其实都不需要这个终止条件，因为题目中说了p、q 为不同节点且均存在于给定的二叉搜索树中。也就是说一定会找到公共祖先的，所以并不存在遇到空的情况。

- 确定单层递归的逻辑

在遍历二叉搜索树的时候就是寻找区间[p->val, q->val]（注意这里是左闭又闭）

那么如果 cur->val 大于 p->val，同时 cur->val 大于q->val，那么就应该向左遍历（说明目标区间在左子树上）。

需要注意的是此时不知道p和q谁大，所以两个都要判断

代码如下：

```
if (cur->val > p->val && cur->val > q->val) {
    TreeNode* left = traversal(cur->left, p, q);
    if (left != NULL) {
        return left;
    }
}
```

细心的同学会发现，在这里调用递归函数的地方，把递归函数的返回值left，直接return。

在二叉树：公共祖先问题中，如果递归函数有返回值，如何区分要搜索一条边，还是搜索整个树。

搜索一条边的写法：

```
if (递归函数(root->left)) return ;
if (递归函数(root->right)) return ;
```

搜索整个树写法：

```
left = 递归函数(root->left);
right = 递归函数(root->right);
left与right的逻辑处理;
```

本题就是标准的搜索一条边的写法，遇到递归函数的返回值，如果不为空，立刻返回。

如果 cur->val 小于 p->val，同时 cur->val 小于 q->val，那么就应该向右遍历（目标区间在右子树）。

```
if (cur->val < p->val && cur->val < q->val) {
    TreeNode* right = traversal(cur->right, p, q);
    if (right != NULL) {
        return right;
    }
}
```

剩下的情况，就是cur节点在区间（p->val <= cur->val && cur->val <= q->val）或者 （q->val <= cur->val && cur->val <= p->val）中，那么cur就是最近公共祖先了，直接返回cur。

代码如下：

```
return cur;
```

那么整体递归代码如下:

```
class Solution {
private:
    TreeNode* traversal(TreeNode* cur, TreeNode* p, TreeNode* q) {
        if (cur == NULL) return cur;
                                                        // 中
        if (cur->val > p->val && cur->val > q->val) {   // 左
            TreeNode* left = traversal(cur->left, p, q);
            if (left != NULL) {
                return left;
            }
        }

        if (cur->val < p->val && cur->val < q->val) {   // 右
            TreeNode* right = traversal(cur->right, p, q);
            if (right != NULL) {
                return right;
            }
        }
        return cur;
    }
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        return traversal(root, p, q);
    }
```

```
};
```

精简后代码如下：

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root->val > p->val && root->val > q->val) {
            return lowestCommonAncestor(root->left, p, q);
        } else if (root->val < p->val && root->val < q->val) {
            return lowestCommonAncestor(root->right, p, q);
        } else return root;
    }
};
```

迭代法

对于二叉搜索树的迭代法，大家应该在二叉树：二叉搜索树登场！就了解了。

利用其有序性，迭代的方式还是比较简单的，解题思路在递归中已经分析了。

迭代代码如下：

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        while(root) {
            if (root->val > p->val && root->val > q->val) {
                root = root->left;
            } else if (root->val < p->val && root->val < q->val) {
                root = root->right;
            } else return root;
        }
        return NULL;
    }
};
```

灵魂拷问：是不是又被简单的迭代法感动到痛哭流涕?

总结

对于二叉搜索树的最近祖先问题，其实要比普通二叉树公共祖先问题简单的多。

不用使用回溯，二叉搜索树自带方向性，可以方便的从上向下查找目标区间，遇到目标区间内的节点，直接返回。

最后给出了对应的迭代法，二叉搜索树的迭代法甚至比递归更容易理解，也是因为其有序性（自带方向性），按照目标区间找就行了。

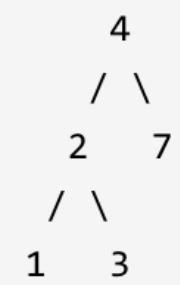
29.二叉搜索树中的插入操作

力扣题目链接

给定二叉搜索树（BST）的根节点和要插入树中的值，将值插入二叉搜索树。 返回插入后二叉搜索树的根节点。 输入数据保证，新值和原始二叉搜索树中的任意节点值都不同。

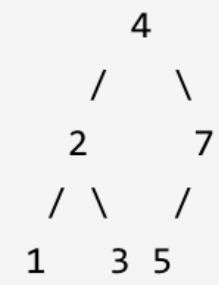
注意，可能存在多种有效的插入方式，只要树在插入后仍保持为二叉搜索树即可。 你可以返回任意有效的结果。

给定二叉搜索树：

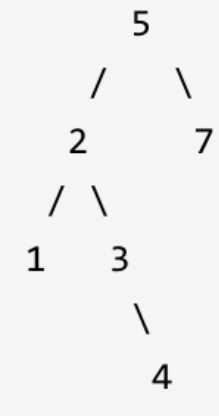


和 插入的值：5

你可以返回这个二叉搜索树：



或者这个树也是有效的：



提示：

- 给定的树上的节点数介于 0 和 10^4 之间
- 每个节点都有一个唯一整数值，取值范围从 0 到 10^8
- -10^8 <= val <= 10^8
- 新值和原始二叉搜索树中的任意节点值都不同

算法公开课

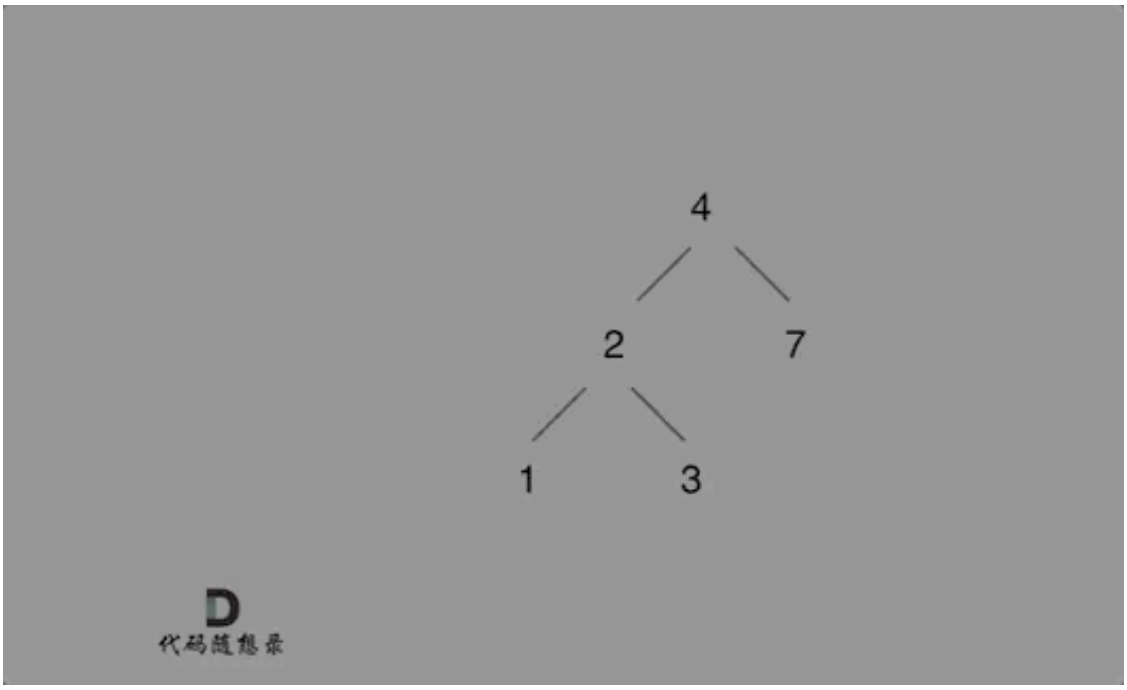
[《代码随想录》算法视频公开课](#)：[原来这么简单？ | LeetCode：701.二叉搜索树中的插入操作](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

这道题目其实是一道简单题目，**但是题目中的提示：有多种有效的插入方式，还可以重构二叉搜索树，一下子吓退了不少人**，瞬间感觉题目复杂了很多。

其实**可以不考虑题目中提示所说的改变树的结构的插入方式。**

如下演示视频中可以看出：只要按照二叉搜索树的规则去遍历，遇到空节点就插入节点就可以了。



例如插入元素10 ，需要找到末尾节点插入便可，一样的道理来插入元素15，插入元素0，插入元素6，**需要调整二叉树的结构么？ 并不需要。**。

只要遍历二叉搜索树，找到空节点 插入元素就可以了，那么这道题其实就简单了。

接下来就是遍历二叉搜索树的过程了。

递归

递归三部曲：

- 确定递归函数参数以及返回值

参数就是根节点指针，以及要插入元素，这里递归函数要不要有返回值呢?

可以有，也可以没有，但递归函数如果没有返回值的话，实现是比较麻烦的，下面也会给出其具体实现代码。

有返回值的话，可以利用返回值完成新加入的节点与其父节点的赋值操作。（下面会进一步解释）

递归函数的返回类型为节点类型TreeNode * 。

代码如下：

```
TreeNode* insertIntoBST(TreeNode* root, int val)
```

- 确定终止条件

终止条件就是找到遍历的节点为null的时候，就是要插入节点的位置了，并把插入的节点返回。

代码如下：

```
if (root == NULL) {
    TreeNode* node = new TreeNode(val);
    return node;
}
```

这里把添加的节点返回给上一层，就完成了父子节点的赋值操作了，详细再往下看。

- 确定单层递归的逻辑

此时要明确，需要遍历整棵树么？

别忘了这是搜索树，遍历整棵搜索树简直是对搜索树的侮辱，哈哈。

搜索树是有方向了，可以根据插入元素的数值，决定递归方向。

代码如下：

```
if (root->val > val) root->left = insertIntoBST(root->left, val);
if (root->val < val) root->right = insertIntoBST(root->right, val);
return root;
```

到这里，大家应该能感受到，如何通过递归函数返回值完成了新加入节点的父子关系赋值操作了，下一层将加入节点返回，本层用root->left或者root->right将其接住。

整体代码如下：

```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        if (root == NULL) {
            TreeNode* node = new TreeNode(val);
            return node;
        }
        if (root->val > val) root->left = insertIntoBST(root->left, val);
        if (root->val < val) root->right = insertIntoBST(root->right, val);
        return root;
    }
};
```

可以看出代码并不复杂。

刚刚说了递归函数不用返回值也可以，找到插入的节点位置，直接让其父节点指向插入节点，结束递归，也是可以的。

那么递归函数定义如下：

```
TreeNode* parent; // 记录遍历节点的父节点
void traversal(TreeNode* cur, int val)
```

没有返回值，需要记录上一个节点（parent），遇到空节点了，就让parent左孩子或者右孩子指向新插入的节点。然后结束递归。

代码如下：

```
class Solution {
private:
    TreeNode* parent;
    void traversal(TreeNode* cur, int val) {
        if (cur == NULL) {
            TreeNode* node = new TreeNode(val);
            if (val > parent->val) parent->right = node;
            else parent->left = node;
            return;
        }
        parent = cur;
        if (cur->val > val) traversal(cur->left, val);
        if (cur->val < val) traversal(cur->right, val);
        return;
    }

public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        parent = new TreeNode(0);
        if (root == NULL) {
            root = new TreeNode(val);
        }
        traversal(root, val);
        return root;
    }
};
```

可以看出还是麻烦一些的。

我之所以举这个例子，是想说明通过递归函数的返回值完成父子节点的赋值是可以带来便利的。

网上千篇一律的代码，可能会误导大家认为通过递归函数返回节点 这样的写法是天经地义，其实这里是有优化的！

迭代

再来看看迭代法，对二叉搜索树迭代写法不熟悉，可以看这篇：二叉树：二叉搜索树登场！

在迭代法遍历的过程中，需要记录一下当前遍历的节点的父节点，这样才能做插入节点的操作。

在二叉树：搜索树的最小绝对差和二叉树：我的众数是多少？ 中，都是用了记录pre和cur两个指针的技巧，本题也是一样的。

代码如下：

```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        if (root == NULL) {
            TreeNode* node = new TreeNode(val);
            return node;
        }
        TreeNode* cur = root;
        TreeNode* parent = root; // 这个很重要，需要记录上一个节点，否则无法赋值新节点
        while (cur != NULL) {
            parent = cur;
            if (cur->val > val) cur = cur->left;
            else cur = cur->right;
        }
        TreeNode* node = new TreeNode(val);
        if (val < parent->val) parent->left = node;// 此时是用parent节点的进行赋值
        else parent->right = node;
        return root;
    }
};
```

总结

首先在二叉搜索树中的插入操作，大家不用恐惧其重构搜索树，其实根本不用重构。

然后在递归中，我们重点讲了如何通过递归函数的返回值完成新加入节点和其父节点的赋值操作，并强调了搜索树的有序性。

最后依然给出了迭代的方法，迭代的方法就需要记录当前遍历节点的父节点了，这个和没有返回值的递归函数实现的代码逻辑是一样的。

> 二叉搜索树删除节点就涉及到结构调整了

30.删除二叉搜索树中的节点

力扣题目链接

给定一个二叉搜索树的根节点 root 和一个值 key，删除二叉搜索树中的 key 对应的节点，并保证二叉搜索树的性质不变。返回二叉搜索树（有可能被更新）的根节点的引用。

一般来说，删除节点可分为两个步骤：

首先找到需要删除的节点；
如果找到了，删除它。
说明： 要求算法时间复杂度为 $O(h)$，h 为树的高度。

示例:

```
root = [5,3,6,2,4,null,7]
key = 3

    5
   / \
  3   6
 / \   \
2   4   7

给定需要删除的节点值是 3，所以我们首先找到 3 这个节点，然后删除它。

一个正确的答案是 [5,4,6,2,null,null,7]，如下图所示。

    5
   / \
  4   6
 /     \
2       7

另一个正确答案是 [5,2,6,null,4,null,7]。

    5
   / \
  2   6
   \   \
    4   7
```

算法公开课

《代码随想录》算法视频公开课：调整二叉树的结构最难！ | LeetCode：450.删除二叉搜索树中的节点，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

搜索树的节点删除要比节点增加复杂的多，有很多情况需要考虑，做好心理准备。

递归

递归三部曲：

- 确定递归函数参数以及返回值

说到递归函数的返回值，在二叉树：搜索树中的插入操作中通过递归返回值来加入新节点， 这里也可以通过递归返回值删除节点。

代码如下：

```
TreeNode* deleteNode(TreeNode* root, int key)
```

- 确定终止条件

遇到空返回，其实这也说明没找到删除的节点，遍历到空节点直接返回了

```
if (root == nullptr) return root;
```

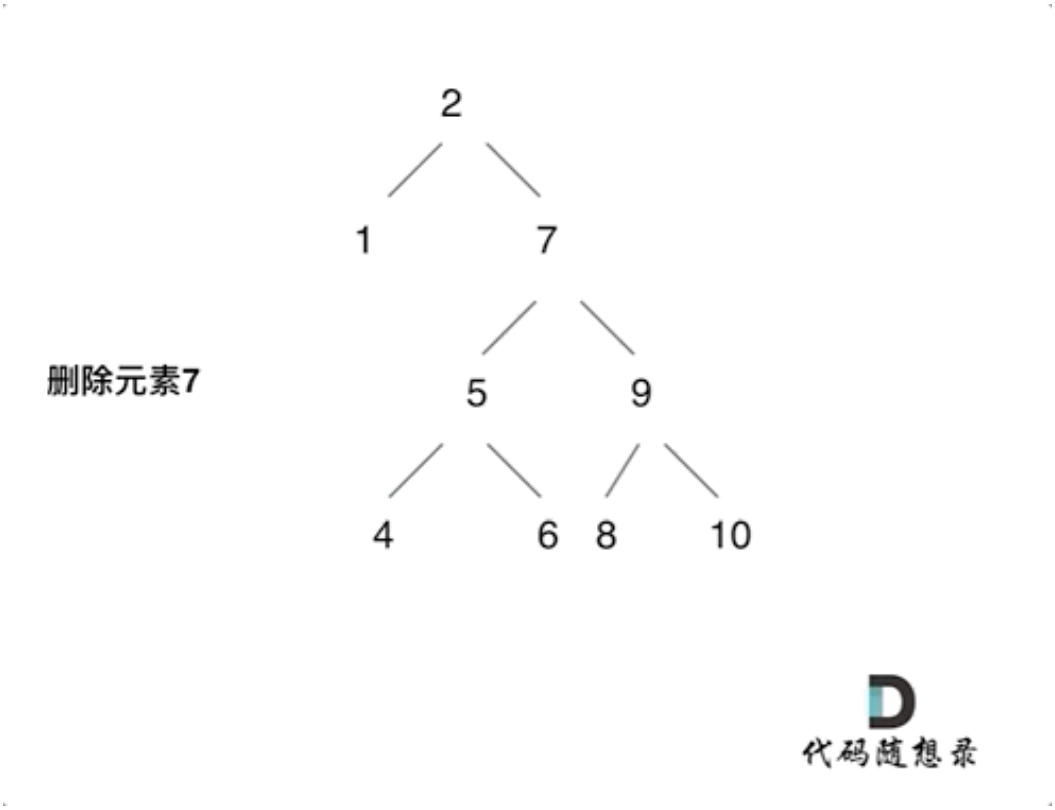
- 确定单层递归的逻辑

这里就把二叉搜索树中删除节点遇到的情况都搞清楚。

有以下五种情况：

- 第一种情况：没找到删除的节点，遍历到空节点直接返回了
- 找到删除的节点
    - 第二种情况：左右孩子都为空（叶子节点），直接删除节点， 返回NULL为根节点
    - 第三种情况：删除节点的左孩子为空，右孩子不为空，删除节点，右孩子补位，返回右孩子为根节点
    - 第四种情况：删除节点的右孩子为空，左孩子不为空，删除节点，左孩子补位，返回左孩子为根节点
    - 第五种情况：左右孩子节点都不为空，则将删除节点的左子树头结点（左孩子）放到删除节点的右子树的最左面节点的左孩子上，返回删除节点右孩子为新的根节点。

第五种情况有点难以理解，看下面动画：



动画中的二叉搜索树中，删除元素7， 那么删除节点（元素7）的左孩子就是5，删除节点（元素7）的右子树的最左面节点是元素8。

将删除节点（元素7）的左孩子放到删除节点（元素7）的右子树的最左面节点（元素8）的左孩子上，就是把5为根节点的子树移到了8的左孩子的位置。

要删除的节点（元素7）的右孩子（元素9）为新的根节点。.

这样就完成删除元素7的逻辑，最好动手画一个图，尝试删除一个节点试试。

代码如下：

```
if (root->val == key) {
    // 第二种情况：左右孩子都为空（叶子节点），直接删除节点， 返回NULL为根节点
    // 第三种情况：其左孩子为空，右孩子不为空，删除节点，右孩子补位 ，返回右孩子为根节点
    if (root->left == nullptr) return root->right;
    // 第四种情况：其右孩子为空，左孩子不为空，删除节点，左孩子补位，返回左孩子为根节点
    else if (root->right == nullptr) return root->left;
    // 第五种情况：左右孩子节点都不为空，则将删除节点的左子树放到删除节点的右子树的最左面节点的左孩子的
位置
    // 并返回删除节点右孩子为新的根节点。
    else {
        TreeNode* cur = root->right; // 找右子树最左面的节点
        while(cur->left != nullptr) {
            cur = cur->left;
        }
        cur->left = root->left; // 把要删除的节点（root）左子树放在cur的左孩子的位置
        TreeNode* tmp = root;   // 把root节点保存一下，下面来删除
        root = root->right;     // 返回旧root的右孩子作为新root
        delete tmp;             // 释放节点内存（这里不写也可以，但C++最好手动释放一下吧）
```

```
        return root;
    }
}
```

这里相当于把新的节点返回给上一层，上一层就要用 root->left 或者 root->right接住，代码如下：

```
if (root->val > key) root->left = deleteNode(root->left, key);
if (root->val < key) root->right = deleteNode(root->right, key);
return root;
```

整体代码如下：（注释中：情况1，2，3，4，5和上面分析严格对应）

```
class Solution {
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == nullptr) return root; // 第一种情况：没找到删除的节点，遍历到空节点直接返回了
        if (root->val == key) {
            // 第二种情况：左右孩子都为空（叶子节点），直接删除节点， 返回NULL为根节点
            if (root->left == nullptr && root->right == nullptr) {
                ///! 内存释放
                delete root;
                return nullptr;
            }
            // 第三种情况：其左孩子为空，右孩子不为空，删除节点，右孩子补位 ，返回右孩子为根节点
            else if (root->left == nullptr) {
                auto retNode = root->right;
                ///! 内存释放
                delete root;
                return retNode;
            }
            // 第四种情况：其右孩子为空，左孩子不为空，删除节点，左孩子补位，返回左孩子为根节点
            else if (root->right == nullptr) {
                auto retNode = root->left;
                ///! 内存释放
                delete root;
                return retNode;
            }
            // 第五种情况：左右孩子节点都不为空，则将删除节点的左子树放到删除节点的右子树的最左面节点的左孩子的位置
            // 并返回删除节点右孩子为新的根节点。
            else {
                TreeNode* cur = root->right; // 找右子树最左面的节点
                while(cur->left != nullptr) {
                    cur = cur->left;
                }
                cur->left = root->left; // 把要删除的节点（root）左子树放在cur的左孩子的位置
                TreeNode* tmp = root;   // 把root节点保存一下，下面来删除
```

```
                root = root->right;     // 返回旧root的右孩子作为新root
                delete tmp;             // 释放节点内存（这里不写也可以，但C++最好手动释放一下吧）
                return root;
            }
        }
        if (root->val > key) root->left = deleteNode(root->left, key);
        if (root->val < key) root->right = deleteNode(root->right, key);
        return root;
    }
};
```

普通二叉树的删除方式

这里我在介绍一种通用的删除，普通二叉树的删除方式（没有使用搜索树的特性，遍历整棵树），用交换值的操作来删除目标节点。

代码中目标节点（要删除的节点）被操作了两次：

- 第一次是和目标节点的右子树最左面节点交换。
- 第二次直接被NULL覆盖了。

思路有点绕，感兴趣的同学可以画图自己理解一下。

代码如下：（关键部分已经注释）

```
class Solution {
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == nullptr) return root;
        if (root->val == key) {
            if (root->right == nullptr) { // 这里第二次操作目标值：最终删除的作用
                return root->left;
            }
            TreeNode *cur = root->right;
            while (cur->left) {
                cur = cur->left;
            }
            swap(root->val, cur->val); // 这里第一次操作目标值：交换目标值其右子树最左面节点。
        }
        root->left = deleteNode(root->left, key);
        root->right = deleteNode(root->right, key);
        return root;
    }
};
```

这个代码是简短一些，思路也巧妙，但是不太好想，实操性不强，推荐第一种写法！

迭代法

删除节点的迭代法还是复杂一些的，但其本质我在递归法里都介绍了，最关键就是删除节点的操作（动画模拟的过程）

代码如下：

```
class Solution {
private:
    // 将目标节点（删除节点）的左子树放到 目标节点的右子树的最左面节点的左孩子位置上
    // 并返回目标节点右孩子为新的根节点
    // 是动画里模拟的过程
    TreeNode* deleteOneNode(TreeNode* target) {
        if (target == nullptr) return target;
        if (target->right == nullptr) return target->left;
        TreeNode* cur = target->right;
        while (cur->left) {
            cur = cur->left;
        }
        cur->left = target->left;
        return target->right;
    }
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == nullptr) return root;
        TreeNode* cur = root;
        TreeNode* pre = nullptr; // 记录cur的父节点，用来删除cur
        while (cur) {
            if (cur->val == key) break;
            pre = cur;
            if (cur->val > key) cur = cur->left;
            else cur = cur->right;
        }
        if (pre == nullptr) { // 如果搜索树只有头结点
            return deleteOneNode(cur);
        }
        // pre 要知道是删左孩子还是右孩子
        if (pre->left && pre->left->val == key) {
            pre->left = deleteOneNode(cur);
        }
        if (pre->right && pre->right->val == key) {
            pre->right = deleteOneNode(cur);
        }
        return root;
    }
};
```

总结

读完本篇，大家会发现二叉搜索树删除节点比增加节点复杂的多。

因为二叉搜索树添加节点只需要在叶子上添加就可以的，不涉及到结构的调整，而删除节点操作涉及到结构的调整。

这里我们依然使用递归函数的返回值来完成把节点从二叉树中移除的操作。

这里最关键的逻辑就是第五种情况（删除一个左右孩子都不为空的节点），这种情况一定要想清楚。

而且就算想清楚了，对应的代码也未必可以写出来，所以**这道题目既考察思维逻辑，也考察代码能力。**

递归中我给出了两种写法，推荐大家学会第一种（利用搜索树的特性）就可以了，第二种递归写法其实是比较绕的。

最后我也给出了相应的迭代法，就是模拟递归法中的逻辑来删除节点，但需要一个pre记录cur的父节点，方便做删除操作。

迭代法其实不太容易写出来，所以如果是初学者的话，彻底掌握第一种递归写法就够了。

> 如果不对递归有深刻的理解，本题有点难
> 单纯移除一个节点那还不够，要修剪！

31. 修剪二叉搜索树

力扣题目链接

给定一个二叉搜索树，同时给定最小边界L 和最大边界 R。通过修剪二叉搜索树，使得所有节点的值在[L, R]中 (R>=L) 。你可能需要改变树的根节点，所以结果应当返回修剪好的二叉搜索树的新的根节点。

示例 1：

```
输入：
    1
   / \
  0   2

  L = 1
  R = 2

输出：
    1
      \
       2
```

示例 2:

```
输入:
    3
   / \
  0   4
   \
    2
   /
  1

  L = 1
  R = 3

输出:
      3
     /
   2
  /
 1
```

算法公开课

[《代码随想录》算法视频公开课](#)：[你修剪的方式不对，我来给你纠正一下！| LeetCode：669. 修剪二叉搜索树](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

相信看到这道题目大家都感觉是一道简单题（事实上leetcode上也标明是简单）。

但还真的不简单！

递归法

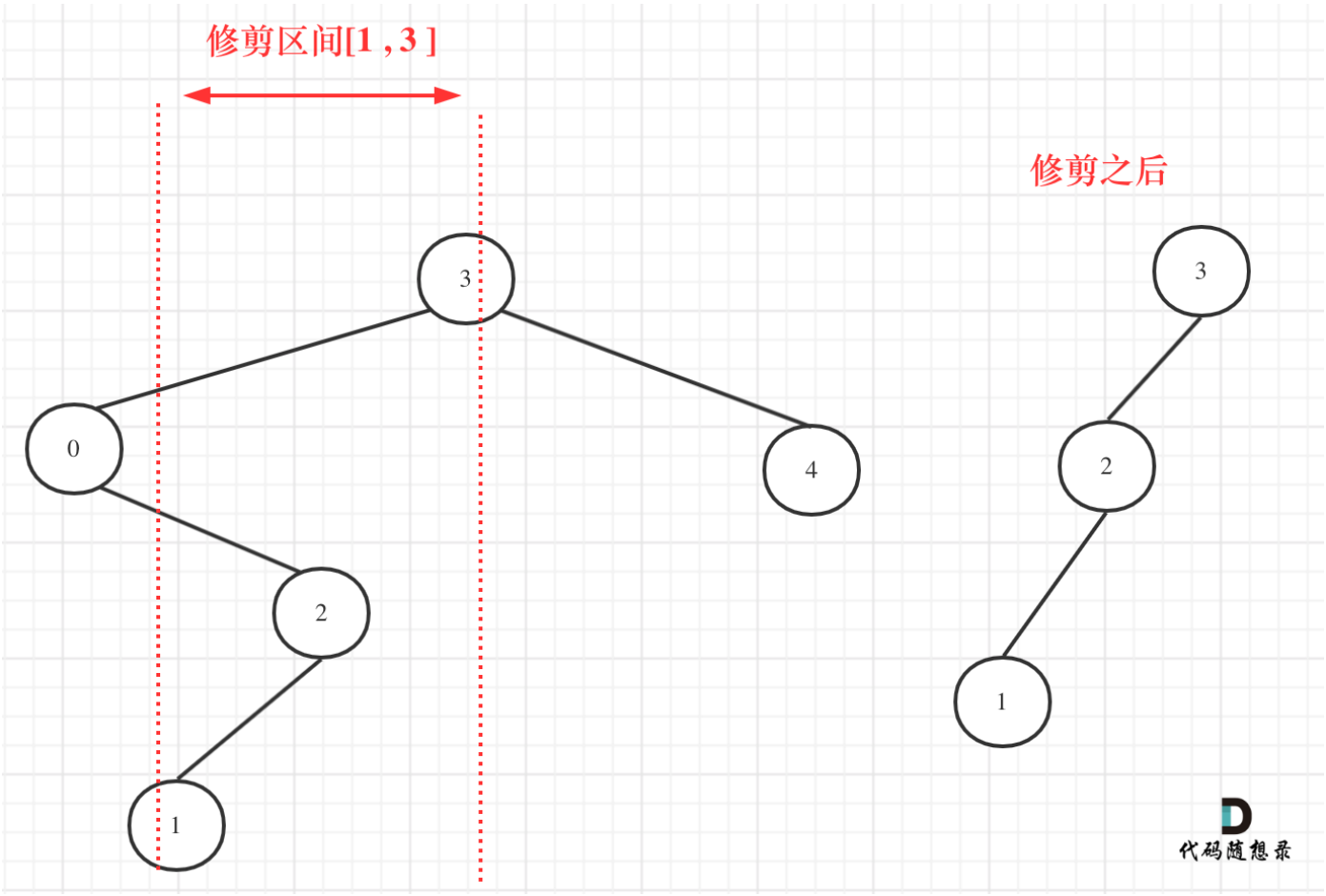
直接想法就是：递归处理，然后遇到 `root->val < low || root->val > high` 的时候直接return NULL，一波修改，赶紧利落。

不难写出如下代码：

```
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int low, int high) {
        if (root == nullptr || root->val < low || root->val > high) return nullptr;
        root->left = trimBST(root->left, low, high);
        root->right = trimBST(root->right, low, high);
        return root;
    }
};
```

然而[1, 3]区间在二叉搜索树的中可不是单纯的节点3和左孩子节点0就决定的，还要考虑节点0的右子树。

我们在重新关注一下第二个示例，如图：

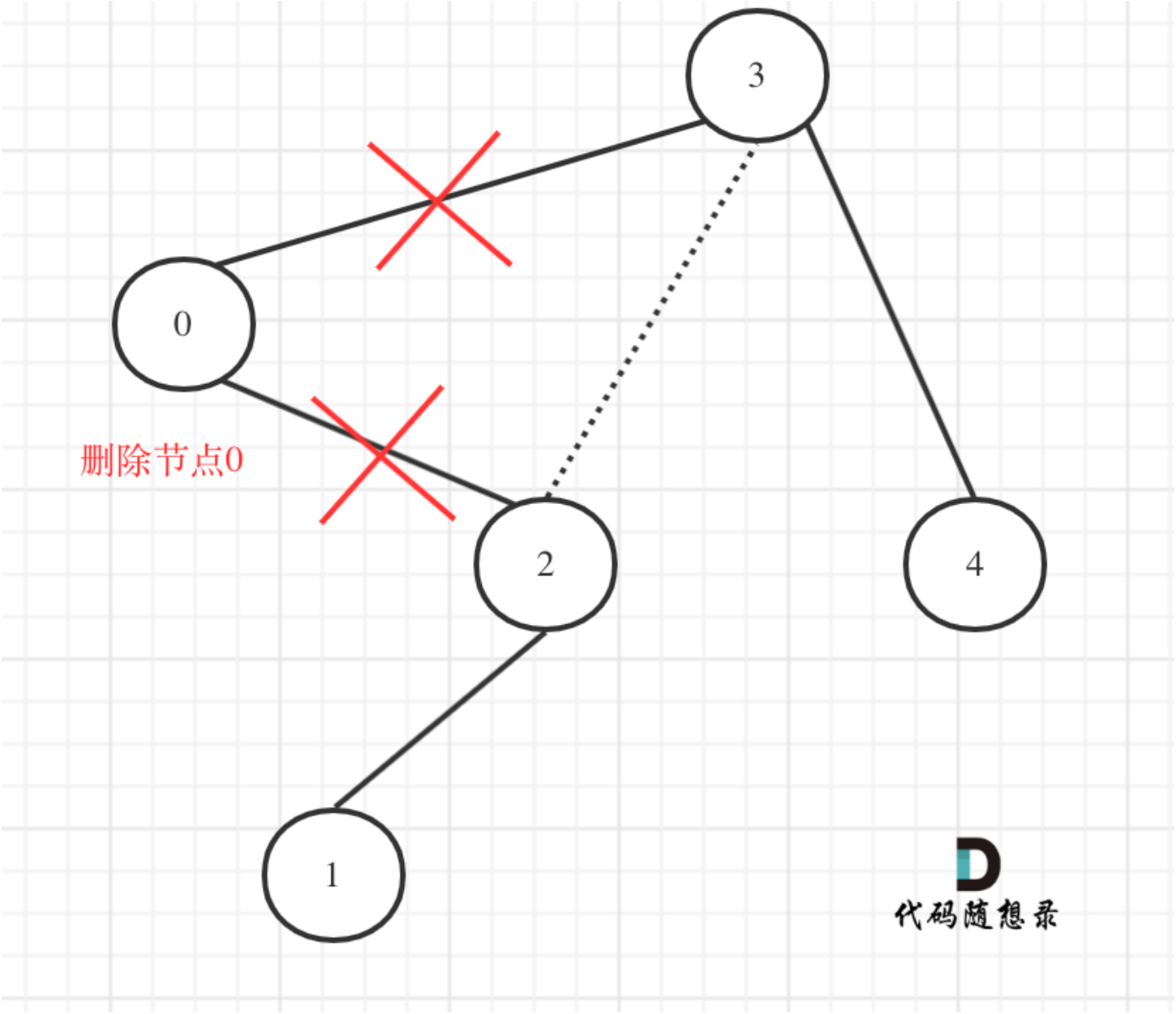


所以以上的代码是不可行的！

从图中可以看出需要重构二叉树，想想是不是本题就有点复杂了。

其实不用重构那么复杂。

在上图中我们发现节点0并不符合区间要求，那么将节点0的右孩子 节点2 直接赋给 节点3的左孩子就可以了（就是把节点0从二叉树中移除），如图：



理解了最关键部分了我们再递归三部曲：

- 确定递归函数的参数以及返回值

这里我们为什么需要返回值呢？

因为是要遍历整棵树，做修改，其实不需要返回值也可以，我们也可以完成修剪（其实就是从二叉树中移除节点）的操作。

但是有返回值，更方便，可以通过递归函数的返回值来移除节点。

这样的做法在二叉树：搜索树中的插入操作和二叉树：搜索树中的删除操作中大家已经了解过了。

代码如下：

```
TreeNode* trimBST(TreeNode* root, int low, int high)
```

- 确定终止条件

修剪的操作并不是在终止条件上进行的，所以就是遇到空节点返回就可以了。

```
if (root == nullptr ) return nullptr;
```

- 确定单层递归的逻辑

如果root（当前节点）的元素小于low的数值，那么应该递归右子树，并返回右子树符合条件的头结点。

代码如下：

```
if (root->val < low) {
    TreeNode* right = trimBST(root->right, low, high); // 寻找符合区间[low, high]的节点
    return right;
}
```

如果root(当前节点)的元素大于high的，那么应该递归左子树，并返回左子树符合条件的头结点。

代码如下：

```
if (root->val > high) {
    TreeNode* left = trimBST(root->left, low, high); // 寻找符合区间[low, high]的节点
    return left;
}
```

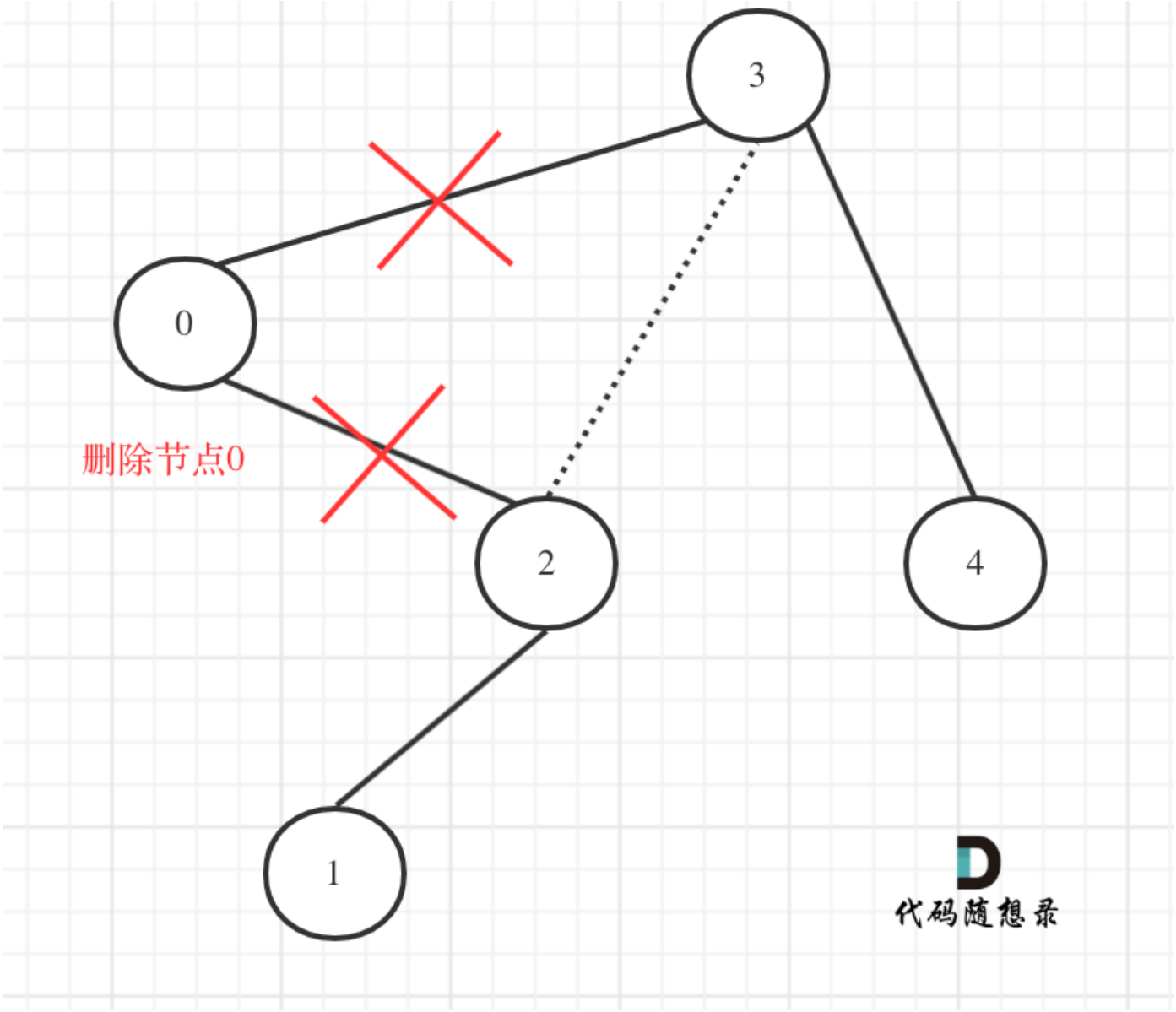
接下来要将下一层处理完左子树的结果赋给root->left，处理完右子树的结果赋给root->right。

最后返回root节点，代码如下：

```
root->left = trimBST(root->left, low, high); // root->left接入符合条件的左孩子
root->right = trimBST(root->right, low, high); // root->right接入符合条件的右孩子
return root;
```

此时大家是不是还没发现这多余的节点究竟是如何从二叉树中移除的呢?

在回顾一下上面的代码，针对下图中二叉树的情况：



如下代码相当于把节点0的右孩子（节点2）返回给上一层，

```
if (root->val < low) {
    TreeNode* right = trimBST(root->right, low, high); // 寻找符合区间[low, high]的节点
    return right;
}
```

然后如下代码相当于用节点3的左孩子 把下一层返回的 节点0的右孩子（节点2） 接住。

```
root->left = trimBST(root->left, low, high);
```

此时节点3的左孩子就变成了节点2，将节点0从二叉树中移除了。

最后整体代码如下：

```
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int low, int high) {
```

```
        if (root == nullptr ) return nullptr;
        if (root->val < low) {
            TreeNode* right = trimBST(root->right, low, high); // 寻找符合区间[low, high]
的节点
            return right;
        }
        if (root->val > high) {
            TreeNode* left = trimBST(root->left, low, high); // 寻找符合区间[low, high]的
节点
            return left;
        }
        root->left = trimBST(root->left, low, high); // root->left接入符合条件的左孩子
        root->right = trimBST(root->right, low, high); // root->right接入符合条件的右孩子
        return root;
    }
};
```

精简之后代码如下：

```
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int low, int high) {
        if (root == nullptr) return nullptr;
        if (root->val < low) return trimBST(root->right, low, high);
        if (root->val > high) return trimBST(root->left, low, high);
        root->left = trimBST(root->left, low, high);
        root->right = trimBST(root->right, low, high);
        return root;
    }
};
```

只看代码，其实不太好理解节点是如何移除的，这一块大家可以自己再模拟模拟！

迭代法

因为二叉搜索树的有序性，不需要使用栈模拟递归的过程。

在剪枝的时候，可以分为三步：

- 将root移动到[L, R] 范围内，注意是左闭右闭区间
- 剪枝左子树
- 剪枝右子树

代码如下：

```
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int L, int R) {
        if (!root) return nullptr;
```

```
        // 处理头结点，让root移动到[L, R] 范围内，注意是左闭右闭
        while (root != nullptr && (root->val < L || root->val > R)) {
            if (root->val < L) root = root->right; // 小于L往右走
            else root = root->left; // 大于R往左走
        }
        TreeNode *cur = root;
        // 此时root已经在[L, R] 范围内，处理左孩子元素小于L的情况
        while (cur != nullptr) {
            while (cur->left && cur->left->val < L) {
                cur->left = cur->left->right;
            }
            cur = cur->left;
        }
        cur = root;

        // 此时root已经在[L, R] 范围内，处理右孩子大于R的情况
        while (cur != nullptr) {
            while (cur->right && cur->right->val > R) {
                cur->right = cur->right->left;
            }
            cur = cur->right;
        }
        return root;
    }
};
```

总结

修剪二叉搜索树其实并不难，但在递归法中大家可看出我费了很大的功夫来讲解如何删除节点的，这个思路其实是比较绕的。

最终的代码倒是很简洁。

如果不对递归有深刻的理解，这道题目还是有难度的！

本题我依然给出递归法和迭代法，初学者掌握递归就可以了，如果想进一步学习，就把迭代法也写一写。

> 构造二叉搜索树，一不小心就平衡了

32.将有序数组转换为二叉搜索树

[力扣题目链接]

将一个按照升序排列的有序数组，转换为一棵高度平衡二叉搜索树。

本题中，一个高度平衡二叉树是指一个二叉树每个节点 的左右两个子树的高度差的绝对值不超过 1。

示例:

```
给定有序数组: [-10,-3,0,5,9],

一个可能的答案是：[0,-3,9,-10,null,5]，它可以表示下面这个高度平衡二叉搜索树：

      0
     / \
   -3   9
   /   /
 -10  5
```

算法公开课

[《代码随想录》算法视频公开课](#)：[构造平衡二叉搜索树！| LeetCode：108.将有序数组转换为二叉搜索树](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

做这道题目之前大家可以了解一下这几道：

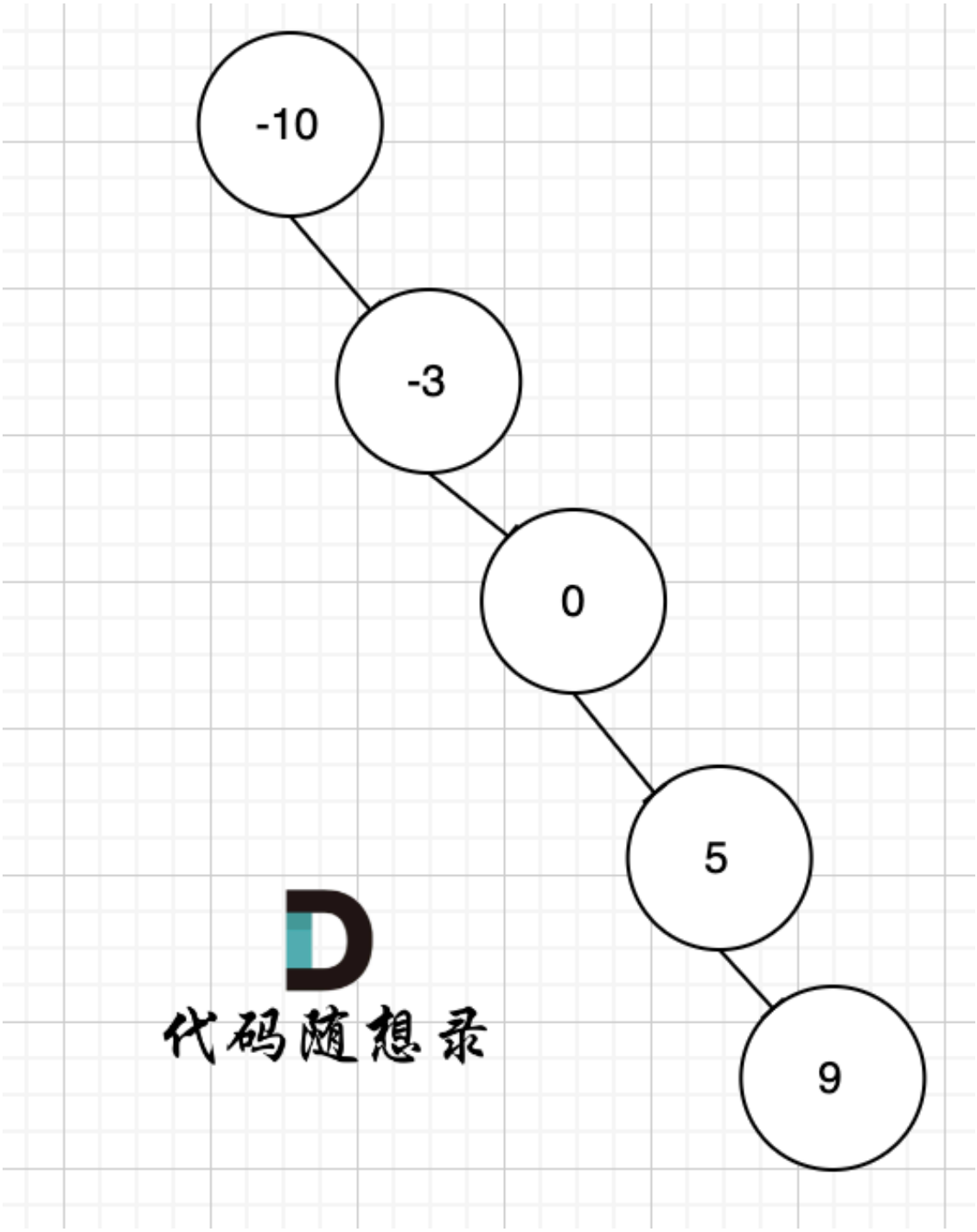
- [106.从中序与后序遍历序列构造二叉树](#)
- [654.最大二叉树](#)中其实已经讲过了，如果根据数组构造一棵二叉树。
- [701.二叉搜索树中的插入操作](#)
- [450.删除二叉搜索树中的节点](#)

进入正题：

题目中说要转换为一棵高度平衡二叉搜索树。为什么强调要平衡呢?

因为只要给我们一个有序数组，如果强调平衡，都可以以线性结构来构造二叉搜索树。

例如 有序数组[-10，-3，0，5，9] 就可以构造成这样的二叉搜索树，如图。



上图中，是符合二叉搜索树的特性吧，如果要这么做的话，是不是本题意义就不大了，所以才强调是平衡二叉搜索树。

其实数组构造二叉树，构成平衡树是自然而然的事情，因为大家默认都是从数组中间位置取值作为节点元素，一般不会随机取。**所以想构成不平衡的二叉树是自找麻烦**。

在二叉树：构造二叉树登场！和二叉树：构造一棵最大的二叉树中其实已经讲过了，如果根据数组构造一棵二叉树。

本质就是寻找分割点，分割点作为当前节点，然后递归左区间和右区间。

本题其实要比二叉树：构造二叉树登场！ 和 二叉树：构造一棵最大的二叉树简单一些，因为有序数组构造二叉搜索树，寻找分割点就比较容易了。

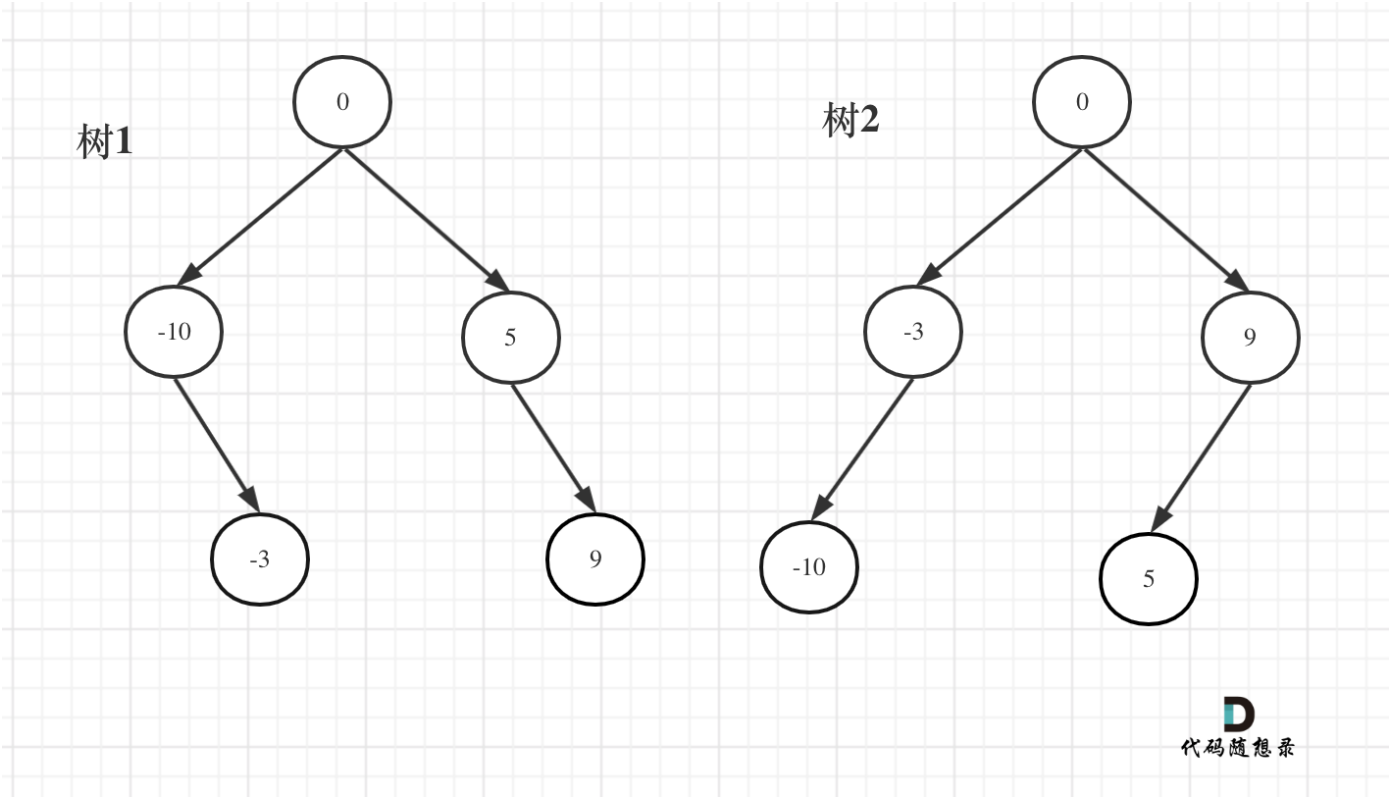
分割点就是数组中间位置的节点。

那么为问题来了，如果数组长度为偶数，中间节点有两个，取哪一个?

取哪一个都可以，只不过构成了不同的平衡二叉搜索树。

例如：输入：[-10,-3,0,5,9]

如下两棵树，都是这个数组的平衡二叉搜索树：



如果要分割的数组长度为偶数的时候，中间元素为两个，是取左边元素 就是树1，取右边元素就是树2。

这也是题目中强调答案不是唯一的原因。 理解这一点，这道题目算是理解到位了。

递归

递归三部曲：

- 确定递归函数返回值及其参数

删除二叉树节点，增加二叉树节点，都是用递归函数的返回值来完成，这样是比较方便的。

相信大家如果仔细看了二叉树：搜索树中的插入操作和二叉树：搜索树中的删除操作，一定会对递归函数返回值的作用深有感触。

那么本题要构造二叉树，依然用递归函数的返回值来构造中节点的左右孩子。

再来看参数，首先是传入数组，然后就是左下标left和右下标right，我们在二叉树：构造二叉树登场！中提过，在构造二叉树的时候尽量不要重新定义左右区间数组，而是用下标来操作原数组。

所以代码如下：

```
// 左闭右闭区间[left, right]
TreeNode* traversal(vector<int>& nums, int left, int right)
```

这里注意，**我这里定义的是左闭右闭区间，在不断分割的过程中，也会坚持左闭右闭的区间，这又涉及到我们讲过的循环不变量。**

在二叉树：构造二叉树登场！，35.搜索插入位置 和59.螺旋矩阵II都详细讲过循环不变量。

- 确定递归终止条件

这里定义的是左闭右闭的区间，所以当区间 left > right的时候，就是空节点了。

代码如下：

```
if (left > right) return nullptr;
```

- 确定单层递归的逻辑

首先取数组中间元素的位置，不难写出`int mid = (left + right) / 2;`，**这么写其实有一个问题，就是数值越界，例如left和right都是最大int，这么操作就越界了，在二分法中尤其需要注意！**

所以可以这么写：`int mid = left + ((right - left) / 2);`

但本题leetcode的测试数据并不会越界，所以怎么写都可以。但需要有这个意识！

取了中间位置，就开始以中间位置的元素构造节点，代码：`TreeNode* root = new TreeNode(nums[mid]);`。

接着划分区间，root的左孩子接住下一层左区间的构造节点，右孩子接住下一层右区间构造的节点。

最后返回root节点，单层递归整体代码如下：

```
int mid = left + ((right - left) / 2);
TreeNode* root = new TreeNode(nums[mid]);
root->left = traversal(nums, left, mid - 1);
root->right = traversal(nums, mid + 1, right);
return root;
```

这里`int mid = left + ((right - left) / 2);`的写法相当于是如果数组长度为偶数，中间位置有两个元素，取靠左边的。

- 递归整体代码如下：

```
class Solution {
private:
    TreeNode* traversal(vector<int>& nums, int left, int right) {
        if (left > right) return nullptr;
        int mid = left + ((right - left) / 2);
        TreeNode* root = new TreeNode(nums[mid]);
        root->left = traversal(nums, left, mid - 1);
        root->right = traversal(nums, mid + 1, right);
        return root;
    }
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        TreeNode* root = traversal(nums, 0, nums.size() - 1);
```

```
        return root;
    }
};
```

注意：在调用traversal的时候传入的left和right为什么是0和nums.size() - 1，因为定义的区间为左闭右闭。

迭代法

迭代法可以通过三个队列来模拟，一个队列放遍历的节点，一个队列放左区间下标，一个队列放右区间下标。

模拟的就是不断分割的过程，C++代码如下：（我已经详细注释）

```
class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        if (nums.size() == 0) return nullptr;

        TreeNode* root = new TreeNode(0);    // 初始根节点
        queue<TreeNode*> nodeQue;            // 放遍历的节点
        queue<int> leftQue;                  // 保存左区间下标
        queue<int> rightQue;                 // 保存右区间下标
        nodeQue.push(root);                  // 根节点入队列
        leftQue.push(0);                     // 0为左区间下标初始位置
        rightQue.push(nums.size() - 1);      // nums.size() - 1为右区间下标初始位置

        while (!nodeQue.empty()) {
            TreeNode* curNode = nodeQue.front();
            nodeQue.pop();
            int left = leftQue.front(); leftQue.pop();
            int right = rightQue.front(); rightQue.pop();
            int mid = left + ((right - left) / 2);

            curNode->val = nums[mid];        // 将mid对应的元素给中间节点

            if (left <= mid - 1) {           // 处理左区间
                curNode->left = new TreeNode(0);
                nodeQue.push(curNode->left);
                leftQue.push(left);
                rightQue.push(mid - 1);
            }

            if (right >= mid + 1) {          // 处理右区间
                curNode->right = new TreeNode(0);
                nodeQue.push(curNode->right);
                leftQue.push(mid + 1);
                rightQue.push(right);
            }
        }
        return root;
```

```
    }
};
```

总结

在二叉树：构造二叉树登场！ 和 二叉树：构造一棵最大的二叉树之后，我们顺理成章的应该构造一下二叉搜索树了，一不小心还是一棵平衡二叉搜索树。

其实思路也是一样的，不断中间分割，然后递归处理左区间，右区间，也可以说是分治。

此时相信大家应该对通过递归函数的返回值来增删二叉树很熟悉了，这也是常规操作。

在定义区间的过程中我们又一次强调了循环不变量的重要性。

最后依然给出迭代的方法，其实就是模拟取中间元素，然后不断分割去构造二叉树的过程。

33.把二叉搜索树转换为累加树

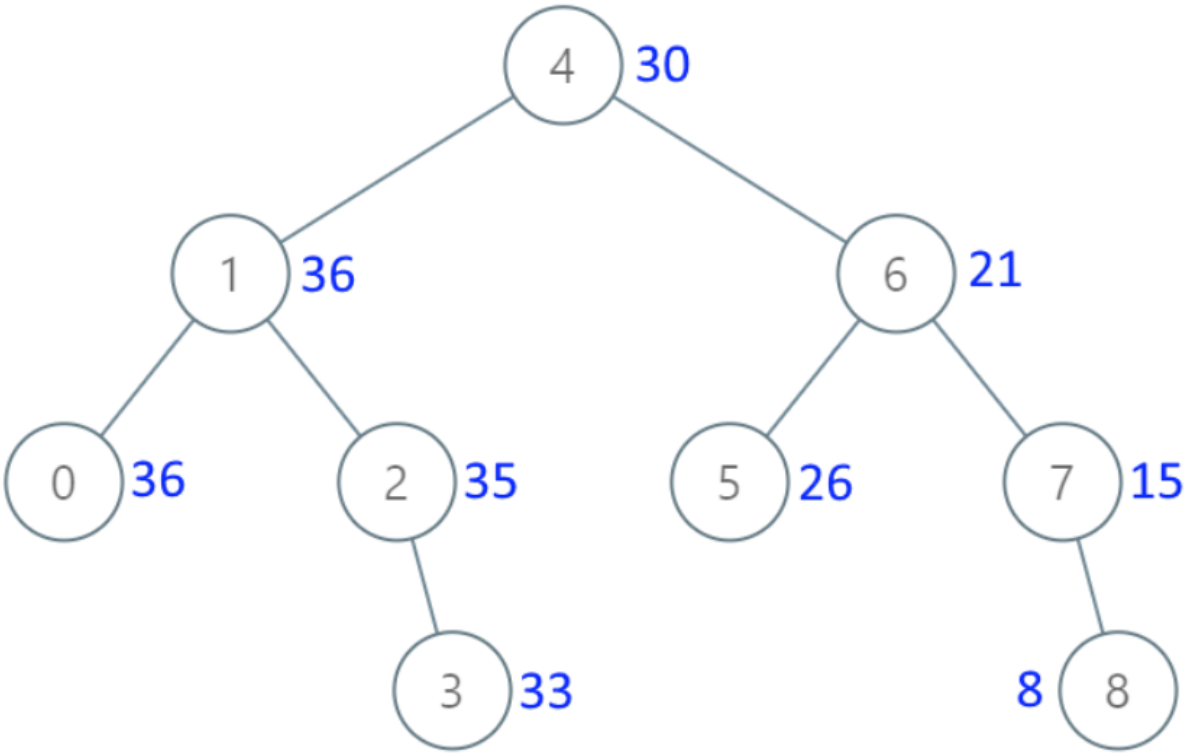
力扣题目链接

给出二叉 搜索 树的根节点，该树的节点值各不相同，请你将其转换为累加树（Greater Sum Tree），使每个节点 node 的新值等于原树中大于或等于 node.val 的值之和。

提醒一下，二叉搜索树满足下列约束条件：

节点的左子树仅包含键 小于 节点键的节点。
节点的右子树仅包含键 大于 节点键的节点。
左右子树也必须是二叉搜索树。

示例 1：



- 输入：[4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
- 输出：[30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]

示例 2：

- 输入：root = [0,null,1]
- 输出：[1,null,1]

示例 3：

- 输入：root = [1,0,2]
- 输出：[3,3,2]

示例 4：

- 输入：root = [3,2,4,1]
- 输出：[7,9,4,10]

提示：

- 树中的节点数介于 0 和 104 之间。
- 每个节点的值介于 -104 和 104 之间。
- 树中的所有值 互不相同 。
- 给定的树为二叉搜索树。

算法公开课

[《代码随想录》算法视频公开课](#)：[普大喜奔！二叉树章节已全部更完啦！ | LeetCode：538.把二叉搜索树转换为累加树](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

一看到累加树，相信很多小伙伴都会疑惑：如何累加？遇到一个节点，然后再遍历其他节点累加？怎么一想这么麻烦呢。

然后再发现这是一棵二叉搜索树，二叉搜索树啊，这是有序的啊。

那么有序的元素如何求累加呢？

其实这就是一棵树，大家可能看起来有点别扭，换一个角度来看，这就是一个有序数组[2, 5, 13]，求从后到前的累加数组，也就是[20, 18, 13]，是不是感觉这就简单了。

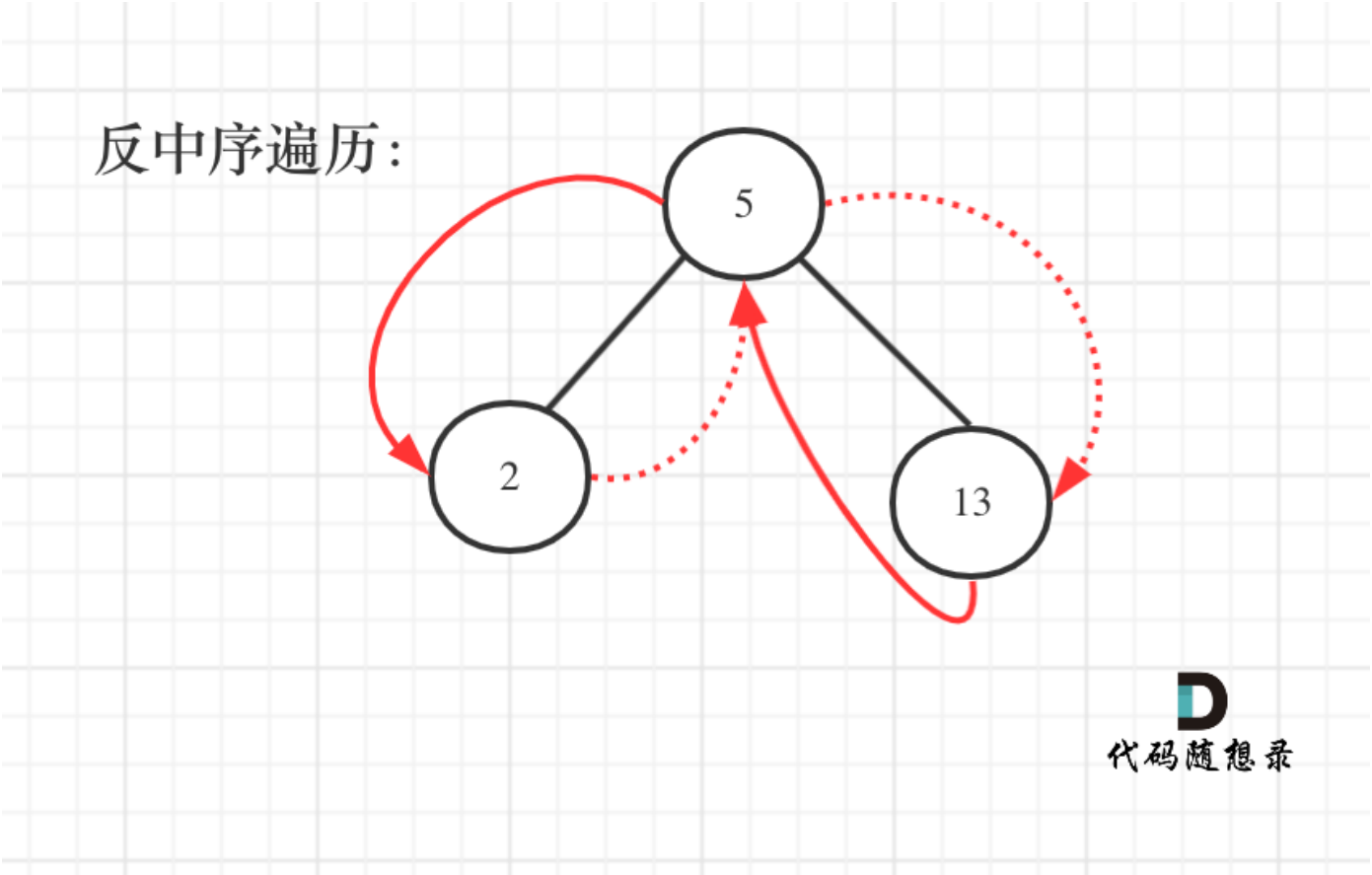
为什么变成数组就是感觉简单了呢？

因为数组大家都知道怎么遍历啊，从后向前，挨个累加就完事了，这换成了二叉搜索树，看起来就别扭了一些是不是。

那么知道如何遍历这个二叉树，也就迎刃而解了，**从树中可以看出累加的顺序是右中左，所以我们需要反中序遍历这个二叉树，然后顺序累加就可以了。**

递归

遍历顺序如图所示：



本题依然需要一个pre指针记录当前遍历节点cur的前一个节点，这样才方便做累加。

pre指针的使用技巧，我们在二叉树：搜索树的最小绝对差和二叉树：我的众数是多少？ 都提到了，这是常用的操作手段。

- 递归函数参数以及返回值

这里很明确了，不需要递归函数的返回值做什么操作了，要遍历整棵树。

同时需要定义一个全局变量pre，用来保存cur节点的前一个节点的数值，定义为int型就可以了。

代码如下：

```
int pre = 0; // 记录前一个节点的数值
void traversal(TreeNode* cur)
```

- 确定终止条件

遇空就终止。

```
if (cur == NULL) return;
```

- 确定单层递归的逻辑

注意**要右中左来遍历二叉树**， 中节点的处理逻辑就是让cur的数值加上前一个节点的数值。

代码如下：

```
traversal(cur->right);  // 右
cur->val += pre;        // 中
pre = cur->val;
traversal(cur->left);   // 左
```

递归法整体代码如下：

```
class Solution {
private:
    int pre = 0; // 记录前一个节点的数值
    void traversal(TreeNode* cur) { // 右中左遍历
        if (cur == NULL) return;
        traversal(cur->right);
        cur->val += pre;
        pre = cur->val;
        traversal(cur->left);
    }
public:
    TreeNode* convertBST(TreeNode* root) {
        pre = 0;
        traversal(root);
        return root;
    }
};
```

迭代法

迭代法其实就是中序模板题了，在二叉树：前中后序迭代法和二叉树：前中后序统一方式迭代法可以选一种自己习惯的写法。

这里我给出其中的一种，代码如下：

```
class Solution {
private:
    int pre; // 记录前一个节点的数值
    void traversal(TreeNode* root) {
        stack<TreeNode*> st;
        TreeNode* cur = root;
        while (cur != NULL || !st.empty()) {
            if (cur != NULL) {
                st.push(cur);
                cur = cur->right;   // 右
            } else {
                cur = st.top();     // 中
                st.pop();
                cur->val += pre;
                pre = cur->val;
                cur = cur->left;    // 左
            }
        }
    }
public:
    TreeNode* convertBST(TreeNode* root) {
        pre = 0;
        traversal(root);
        return root;
    }
};
```

总结

经历了前面各种二叉树增删改查的洗礼之后，这道题目应该比较简单了。

好了，二叉树已经接近尾声了，接下来就是要对二叉树来一个大总结了。

---

34. 二叉树：总结篇！（需要掌握的二叉树技能都在这里了）

> 力扣二叉树大总结！

不知不觉二叉树已经和我们度过了**三十三天**，「代码随想录」里已经发了**三十三篇二叉树的文章**，详细讲解了**30+二叉树经典题目**，一直坚持下来的录友们一定会二叉树有深刻理解了。

在每一道二叉树的题目中，我都使用了**递归三部曲**来分析题目，相信大家以后看到二叉树，看到递归，都会想：返回值、参数是什么？终止条件是什么？单层逻辑是什么？

而且**几乎每一道题目我都给出对应的迭代法**，可以用来进一步提高自己。

下面Carl把分析过的题目分门别类，可以帮助新录友循序渐进学习二叉树，也方便老录友面试前快速复习，看到一个标题，就回想一下对应的解题思路，这样很快就可以系统性的复习一遍二叉树了。

公众号的发文顺序，就是循序渐进的，所以如下分类基本就是按照文章发文顺序来的，我再做一个系统性的分类。

二叉树的理论基础

- 关于二叉树，你该了解这些！：二叉树的种类、存储方式、遍历方式、定义方式

二叉树的遍历方式

- 深度优先遍历
  - 二叉树：前中后序递归法：递归三部曲初次亮相
  - 二叉树：前中后序迭代法（一）：通过栈模拟递归
  - 二叉树：前中后序迭代法（二）统一风格
- 广度优先遍历
  - 二叉树的层序遍历：通过队列模拟

求二叉树的属性

- 二叉树：是否对称
  - 递归：后序，比较的是根节点的左子树与右子树是不是相互翻转
  - 迭代：使用队列/栈将两个节点顺序放入容器中进行比较
- 二叉树：求最大深度
  - 递归：后序，求根节点最大高度就是最大深度，通过递归函数的返回值做计算树的高度
  - 迭代：层序遍历
- 二叉树：求最小深度
  - 递归：后序，求根节点最小高度就是最小深度，注意最小深度的定义
  - 迭代：层序遍历
- 二叉树：求有多少个节点
  - 递归：后序，通过递归函数的返回值计算节点数量
  - 迭代：层序遍历
- 二叉树：是否平衡
  - 递归：后序，注意后序求高度和前序求深度，递归过程判断高度差

    - 迭代：效率很低，不推荐
- 二叉树：找所有路径
    - 递归：前序，方便让父节点指向子节点，涉及回溯处理根节点到叶子的所有路径
    - 迭代：一个栈模拟递归，一个栈来存放对应的遍历路径
- 二叉树：递归中如何隐藏着回溯
    - 详解二叉树：找所有路径中递归如何隐藏着回溯
- 二叉树：求左叶子之和
    - 递归：后序，必须三层约束条件，才能判断是否是左叶子。
    - 迭代：直接模拟后序遍历
- 二叉树：求左下角的值
    - 递归：顺序无所谓，优先左孩子搜索，同时找深度最大的叶子节点。
    - 迭代：层序遍历找最后一行最左边
- 二叉树：求路径总和
    - 递归：顺序无所谓，递归函数返回值为bool类型是为了搜索一条边，没有返回值是搜索整棵树。
    - 迭代：栈里元素不仅要记录节点指针，还要记录从头结点到该节点的路径数值总和

二叉树的修改与构造

- 翻转二叉树
    - 递归：前序，交换左右孩子
    - 迭代：直接模拟前序遍历
- 构造二叉树
    - 递归：前序，重点在于找分割点，分左右区间构造
    - 迭代：比较复杂，意义不大
- 构造最大的二叉树
    - 递归：前序，分割点为数组最大值，分左右区间构造
    - 迭代：比较复杂，意义不大
- 合并两个二叉树
    - 递归：前序，同时操作两个树的节点，注意合并的规则
    - 迭代：使用队列，类似层序遍历

求二叉搜索树的属性

- 二叉搜索树中的搜索
    - 递归：二叉搜索树的递归是有方向的
    - 迭代：因为有方向，所以迭代法很简单
- 是不是二叉搜索树
    - 递归：中序，相当于变成了判断一个序列是不是递增的

    - 迭代：模拟中序，逻辑相同
- 求二叉搜索树的最小绝对差
    - 递归：中序，双指针操作
    - 迭代：模拟中序，逻辑相同
- 求二叉搜索树的众数
    - 递归：中序，清空结果集的技巧，遍历一遍便可求众数集合
    - 二叉搜索树转成累加树
    - 递归：中序，双指针操作累加
    - 迭代：模拟中序，逻辑相同

二叉树公共祖先问题

- 二叉树的公共祖先问题
    - 递归：后序，回溯，找到左子树出现目标值，右子树节点目标值的节点。
    - 迭代：不适合模拟回溯
- 二叉搜索树的公共祖先问题
    - 递归：顺序无所谓，如果节点的数值在目标区间就是最近公共祖先
    - 迭代：按序遍历

二叉搜索树的修改与构造

- 二叉搜索树中的插入操作
    - 递归：顺序无所谓，通过递归函数返回值添加节点
    - 迭代：按序遍历，需要记录插入父节点，这样才能做插入操作
- 二叉搜索树中的删除操作
    - 递归：前序，想清楚删除非叶子节点的情况
    - 迭代：有序遍历，较复杂
- 修剪二叉搜索树
    - 递归：前序，通过递归函数返回值删除节点
    - 迭代：有序遍历，较复杂
- 构造二叉搜索树
    - 递归：前序，数组中间节点分割
    - 迭代：较复杂，通过三个队列来模拟

阶段总结

大家以上题目都做过了，也一定要看如下阶段小结。

每周小结都会对大家的疑问做统一解答，并且对每周的内容进行拓展和补充，所以一定要看，将细碎知识点一网打尽！

- 本周小结！（二叉树系列一）
- 本周小结！（二叉树系列二）
- 本周小结！（二叉树系列三）
- 本周小结！（二叉树系列四）

最后总结

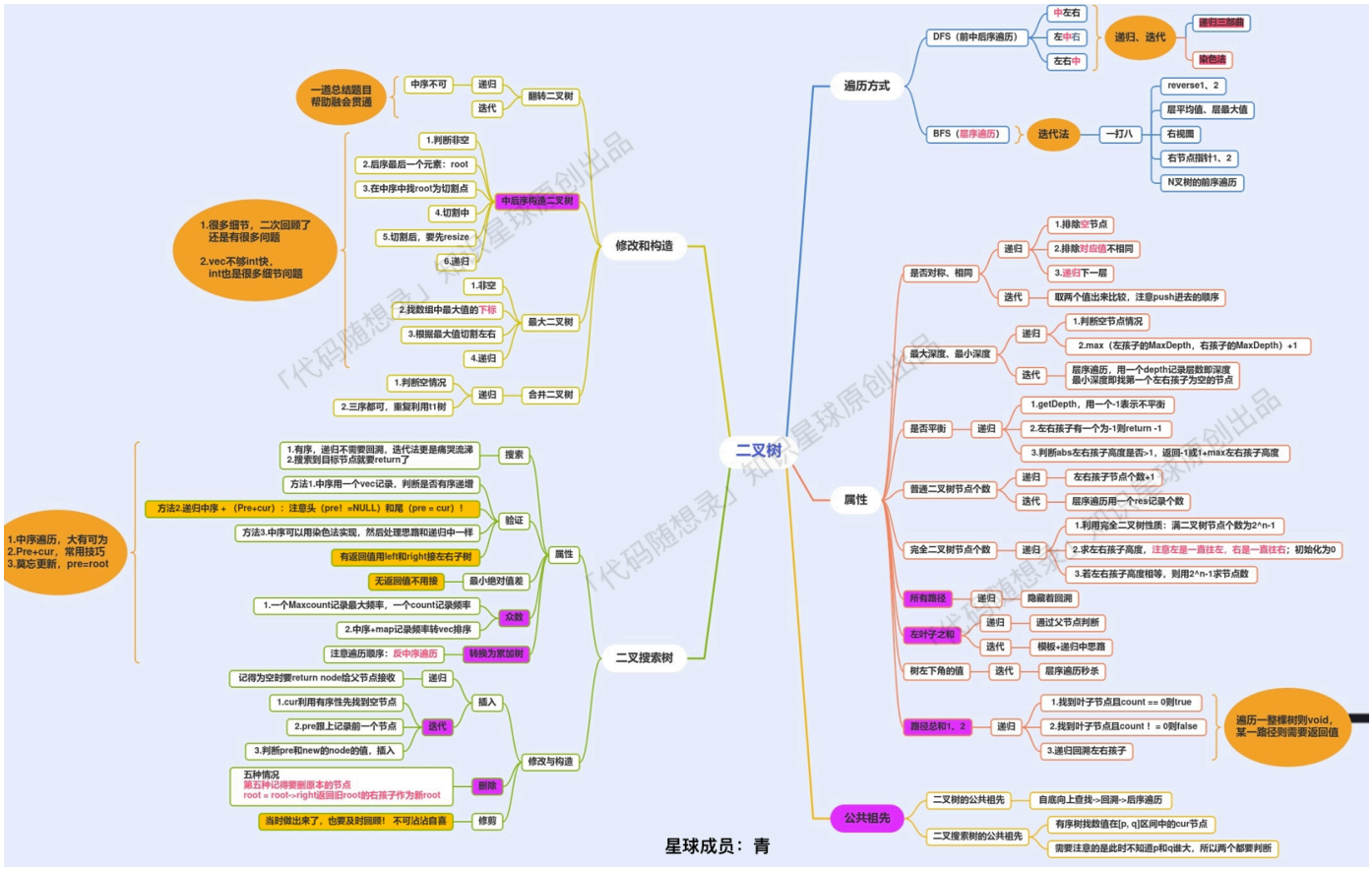
在二叉树题目选择什么遍历顺序是不少同学头疼的事情，我们做了这么多二叉树的题目了，Carl给大家大体分分类。

- 涉及到二叉树的构造，无论普通二叉树还是二叉搜索树一定前序，都是先构造中节点。
- 求普通二叉树的属性，一般是后序，一般要通过递归函数的返回值做计算。
- 求二叉搜索树的属性，一定是中序了，要不白瞎了有序性了。

注意在普通二叉树的属性中，我用的是一般为后序，例如单纯求深度就用前序，二叉树：找所有路径也用了前序，这是为了方便让父节点指向子节点。

所以求普通二叉树的属性还是要具体问题具体分析。

二叉树专题汇聚为一张图：



这个图是 代码随想录知识星球 成员：青，所画，总结的非常好，分享给大家。

最后，二叉树系列就这么完美结束了，估计这应该是最长的系列了，感谢大家33天的坚持与陪伴，接下来我们又要开始新的系列了「回溯算法」！

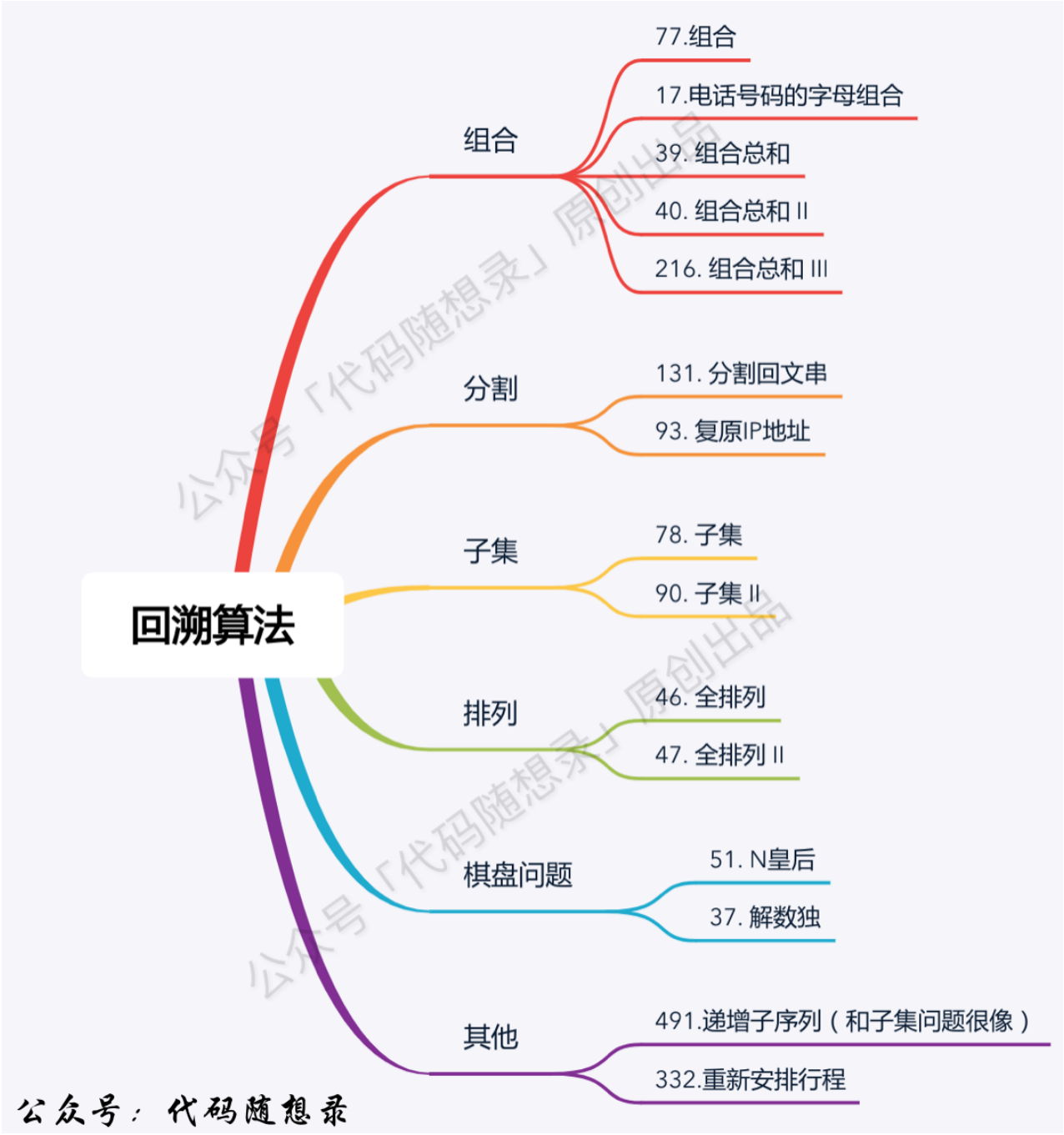
《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 回溯算法理论基础

题目分类



算法公开课

[《代码随想录》算法视频公开课](#)：[带你学透回溯算法（理论篇）](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

理论基础

什么是回溯法

回溯法也可以叫做回溯搜索法，它是一种搜索的方式。

在二叉树系列中，我们已经不止一次，提到了回溯，例如二叉树：以为使用了递归，其实还隐藏着回溯。

回溯是递归的副产品，只要有递归就会有回溯。

所以以下讲解中，回溯函数也就是递归函数，指的都是一个函数。

回溯法的效率

回溯法的性能如何呢，这里要和大家说清楚了，**虽然回溯法很难，很不好理解，但是回溯法并不是什么高效的算法**。

因为回溯的本质是穷举，穷举所有可能，然后选出我们想要的答案，如果想让回溯法高效一些，可以加一些剪枝的操作，但也改不了回溯法就是穷举的本质。

那么既然回溯法并不高效为什么还要用它呢?

因为没得选，一些问题能暴力搜出来就不错了，撑死了再剪枝一下，还没有更高效的解法。

此时大家应该好奇了，都什么问题，这么牛逼，只能暴力搜索。

回溯法解决的问题

回溯法，一般可以解决如下几种问题：

- 组合问题：N个数里面按一定规则找出k个数的集合
- 切割问题：一个字符串按一定规则有几种切割方式
- 子集问题：一个N个数的集合里有多少符合条件的子集
- 排列问题：N个数按一定规则全排列，有几种排列方式
- 棋盘问题：N皇后，解数独等等

相信大家看着这些之后会发现，每个问题，都不简单！

另外，会有一些同学可能分不清什么是组合，什么是排列?

组合是不强调元素顺序的，排列是强调元素顺序。

例如：{1, 2} 和 {2, 1} 在组合上，就是一个集合，因为不强调顺序，而要是排列的话，{1, 2} 和 {2, 1} 就是两个集合了。

记住组合无序，排列有序，就可以了。

如何理解回溯法

回溯法解决的问题都可以抽象为树形结构，是的，我指的是所有回溯法的问题都可以抽象为树形结构！

因为回溯法解决的都是在集合中递归查找子集，**集合的大小就构成了树的宽度，递归的深度，都构成的树的深度。**

递归就要有终止条件，所以必然是一棵高度有限的树（N叉树）。

这块可能初学者还不太理解，后面的回溯算法解决的所有题目中，我都会强调这一点并画图举相应的例子，现在有一个印象就行。

回溯法模板

这里给出Carl总结的回溯算法模板。

在讲二叉树的递归中我们说了递归三部曲，这里我再给大家列出回溯三部曲。

- 回溯函数模板返回值以及参数

在回溯算法中，我的习惯是函数起名字为backtracking，这个起名大家随意。

回溯算法中函数返回值一般为void。

再来看一下参数，因为回溯算法需要的参数可不像二叉树递归的时候那么容易一次性确定下来，所以一般是先写逻辑，然后需要什么参数，就填什么参数。

但后面的回溯题目的讲解中，为了方便大家理解，我在一开始就帮大家把参数确定下来。

回溯函数伪代码如下：

```
void backtracking(参数)
```

- 回溯函数终止条件

既然是树形结构，那么我们在讲解二叉树的递归的时候，就知道遍历树形结构一定要有终止条件。

所以回溯也有要终止条件。

什么时候达到了终止条件，树中就可以看出，一般来说搜到叶子节点了，也就找到了满足条件的一条答案，把这个答案存放起来，并结束本层递归。

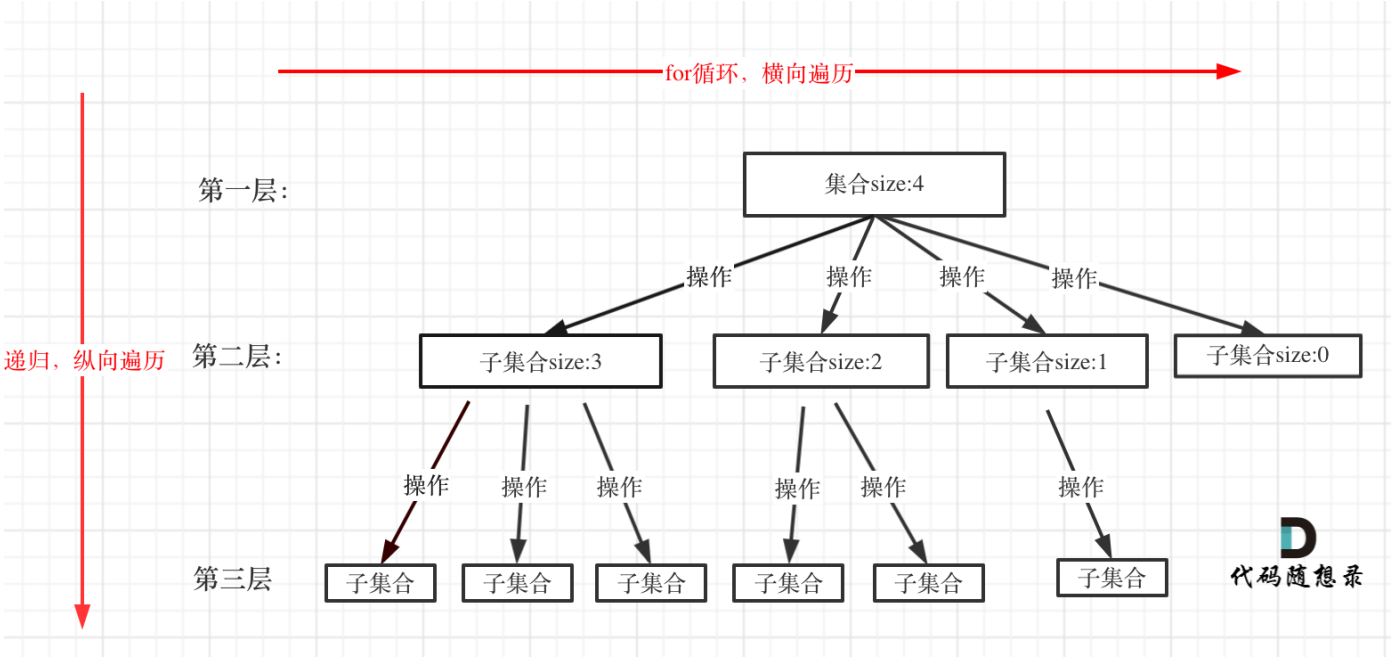
所以回溯函数终止条件伪代码如下：

```
if (终止条件) {
    存放结果;
    return;
}
```

- 回溯搜索的遍历过程

在上面我们提到了，回溯法一般是在集合中递归搜索，集合的大小构成了树的宽度，递归的深度构成的树的深度。

如图：



注意图中，我特意举例集合大小和孩子的数量是相等的！

回溯函数遍历过程伪代码如下：

```
for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
    处理节点;
    backtracking(路径，选择列表); // 递归
    回溯，撤销处理结果
}
```

for循环就是遍历集合区间，可以理解一个节点有多少个孩子，这个for循环就执行多少次。

backtracking这里自己调用自己，实现递归。

大家可以从图中看出**for循环可以理解是横向遍历，backtracking（递归）就是纵向遍历**，这样就把这棵树全遍历完了，一般来说，搜索叶子节点就是找的其中一个结果了。

分析完过程，回溯算法模板框架如下：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

这份模板很重要，后面做回溯法的题目都靠它了！

如果从来没有学过回溯算法的录友们，看到这里会有点懵，后面开始讲解具体题目的时候就会好一些了，已经做过回溯法题目的录友，看到这里应该会感同身受了。

总结

本篇我们讲解了，什么是回溯算法，知道了回溯和递归是相辅相成的。

接着提到了回溯法的效率，回溯法其实就是暴力查找，并不是什么高效的算法。

然后列出了回溯法可以解决几类问题，可以看出每一类问题都不简单。

最后我们讲到回溯法解决的问题都可以抽象为树形结构（N叉树），并给出了回溯法的模板。

今天是回溯算法的第一天，按照惯例Carl都是先概述一波，然后在开始讲解具体题目，没有接触过回溯法的同学刚学起来有点看不懂很正常，后面和具体题目结合起来会好一些。

---

2. 组合

力扣题目链接

给定两个整数 n 和 k，返回 1 ... n 中所有可能的 k 个数的组合。

示例:
输入: n = 4, k = 2
输出:
[
  [2,4],
  [3,4],
  [2,3],
  [1,2],
  [1,3],
  [1,4],
]

算法公开课

《代码随想录》算法视频公开课：带你学透回溯算法-组合问题（对应力扣题目：77.组合），组合问题的剪枝操作，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题是回溯法的经典题目。

直接的解法当然是使用for循环，例如示例中k为2，很容易想到 用两个for循环，这样就可以输出 和示例中一样的结果。

代码如下：

```
int n = 4;
for (int i = 1; i <= n; i++) {
    for (int j = i + 1; j <= n; j++) {
        cout << i << " " << j << endl;
    }
}
```

输入：n = 100, k = 3
那么就三层for循环，代码如下：

```
int n = 100;
for (int i = 1; i <= n; i++) {
    for (int j = i + 1; j <= n; j++) {
        for (int u = j + 1; u <= n; n++) {
            cout << i << " " << j << " " << u << endl;
        }
    }
}
```

如果n为100，k为50呢，那就50层for循环，是不是开始窒息。

此时就会发现虽然想暴力搜索，但是用for循环嵌套连暴力都写不出来！

咋整？

回溯搜索法来了，虽然回溯法也是暴力，但至少能写出来，不像for循环嵌套k层让人绝望。

那么回溯法怎么暴力搜呢？

上面我们说了**要解决 n为100，k为50的情况，暴力写法需要嵌套50层for循环，那么回溯法就用递归来解决嵌套层数的问题。**

递归来做层叠嵌套（可以理解是开k层for循环），**每一次的递归中嵌套一个for循环，那么递归就可以用于解决多层嵌套循环的问题了。**

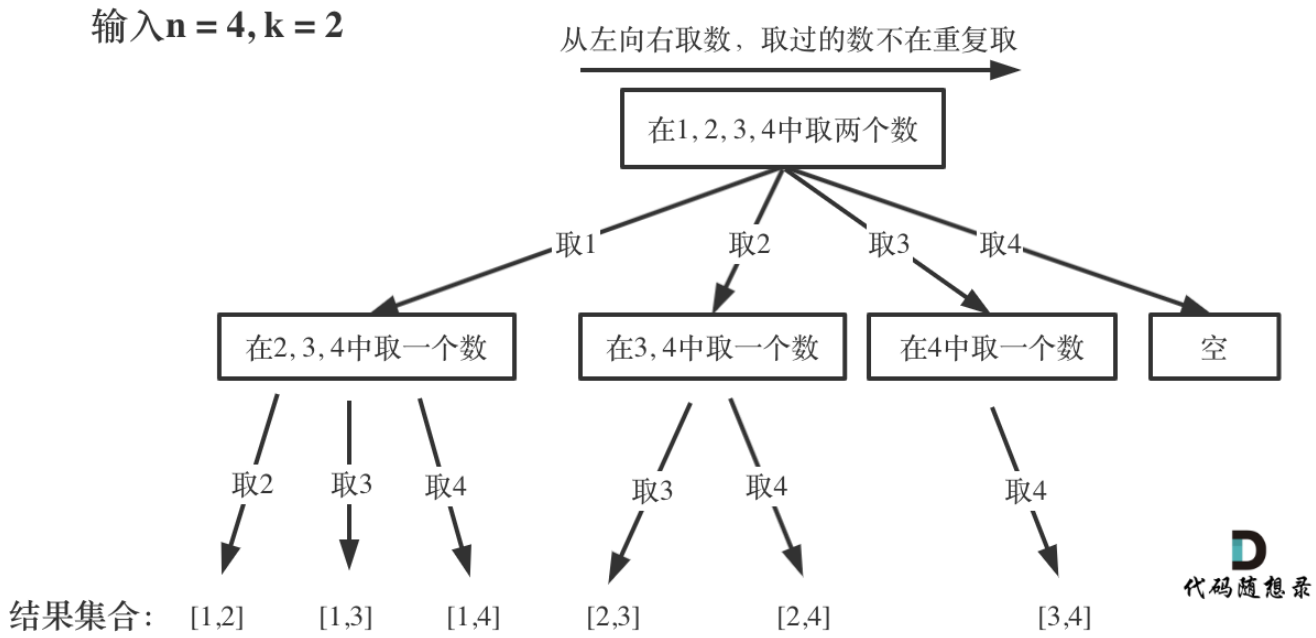
此时递归的层数大家应该知道了，例如：n为100，k为50的情况下，就是递归50层。

一些同学本来对递归就懵，回溯法中递归还要嵌套for循环，可能就直接晕倒了！

如果脑洞模拟回溯搜索的过程，绝对可以让人窒息，所以需要抽象图形结构来进一步理解。

我们在关于回溯算法，你该了解这些！中说到回溯法解决的问题都可以抽象为树形结构（N叉树），用树形结构来理解回溯就容易多了。

那么我把组合问题抽象为如下树形结构：



可以看出这棵树，一开始集合是 1，2，3，4， 从左向右取数，取过的数，不再重复取。

第一次取1，集合变为2，3，4 ，因为k为2，我们只需要再取一个数就可以了，分别取2，3，4，得到集合[1,2] [1,3] [1,4]，以此类推。

每次从集合中选取元素，可选择的范围随着选择的进行而收缩，调整可选择的范围。

图中可以发现n相当于树的宽度，k相当于树的深度。

那么如何在这个树上遍历，然后收集到我们要的结果集呢?

图中每次搜索到了叶子节点，我们就找到了一个结果。

相当于只需要把达到叶子节点的结果收集起来，就可以求得 n个数中k个数的组合集合。

在关于回溯算法，你该了解这些！ 中我们提到了回溯法三部曲，那么我们按照回溯法三部曲开始正式讲解代码了。

回溯法三部曲

- 递归函数的返回值以及参数

在这里要定义两个全局变量，一个用来存放符合条件单一结果，一个用来存放符合条件结果的集合。

代码如下：

```
vector<vector<int>> result; // 存放符合条件结果的集合
vector<int> path; // 用来存放符合条件结果
```

其实不定义这两个全局变量也是可以的，把这两个变量放进递归函数的参数里，但函数里参数太多影响可读性，所以我定义全局变量了。

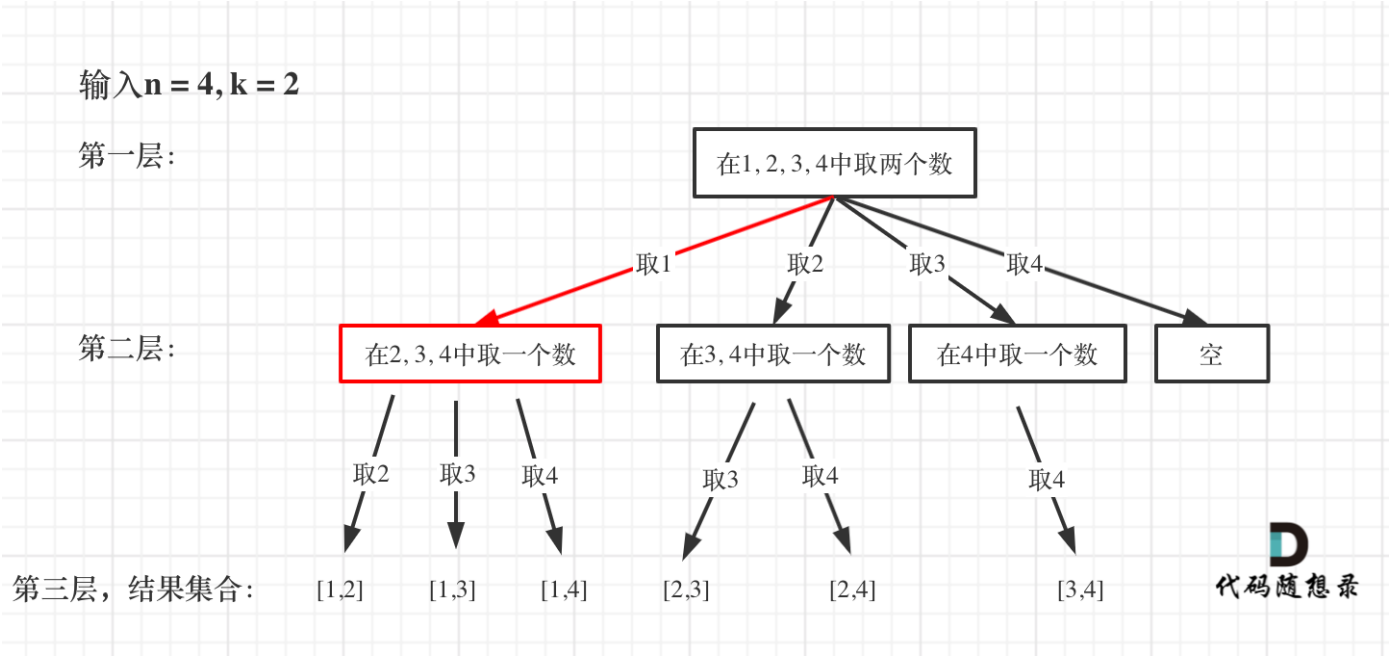
函数里一定有两个参数，既然是集合n里面取k个数，那么n和k是两个int型的参数。

然后还需要一个参数，为int型变量startIndex，这个参数用来记录本层递归的中，集合从哪里开始遍历（集合就是[1,...,n] ）。

为什么要有这个startIndex呢?

建议在77.组合视频讲解中，07:36的时候开始听，startIndex 就是防止出现重复的组合。

从下图中红线部分可以看出，在集合[1,2,3,4]取1之后，下一层递归，就要在[2,3,4]中取数了，那么下一层递归如何知道从[2,3,4]中取数呢，靠的就是startIndex。



所以需要startIndex来记录下一层递归，搜索的起始位置。

那么整体代码如下：

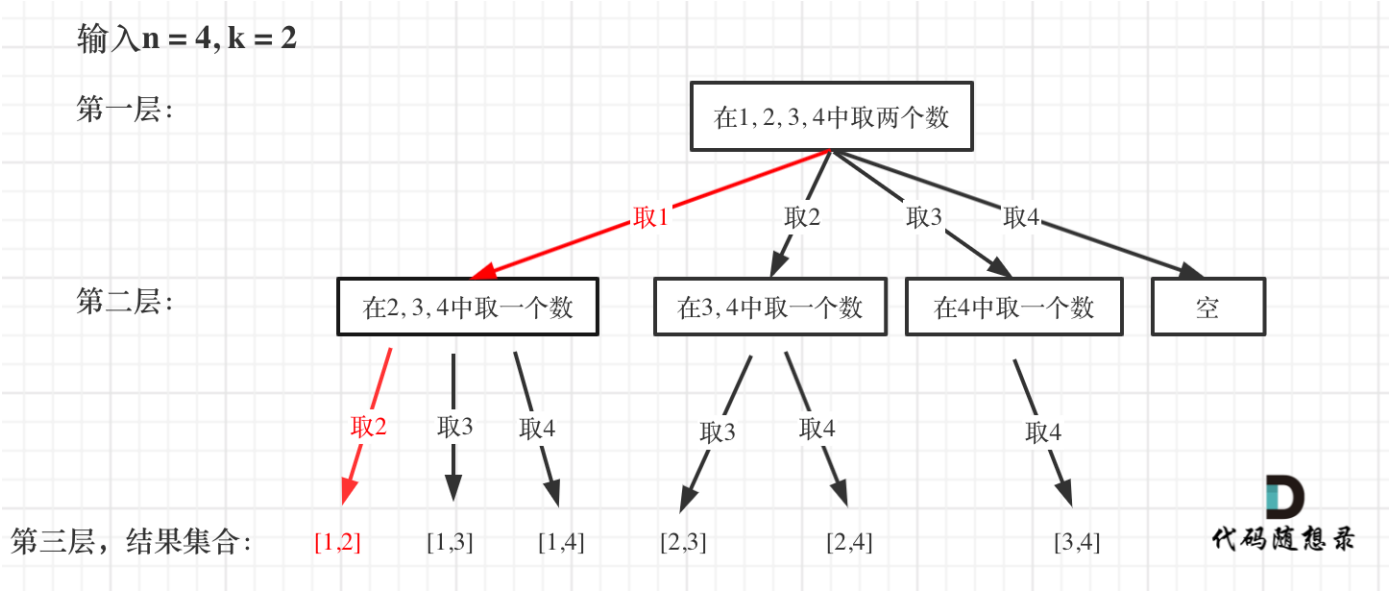
```
vector<vector<int>> result; // 存放符合条件结果的集合
vector<int> path; // 用来存放符合条件单一结果
void backtracking(int n, int k, int startIndex)
```

- 回溯函数终止条件

什么时候到达所谓的叶子节点了呢?

path这个数组的大小如果达到k，说明我们找到了一个子集大小为k的组合了，在图中path存的就是根节点到叶子节点的路径。

如图红色部分：



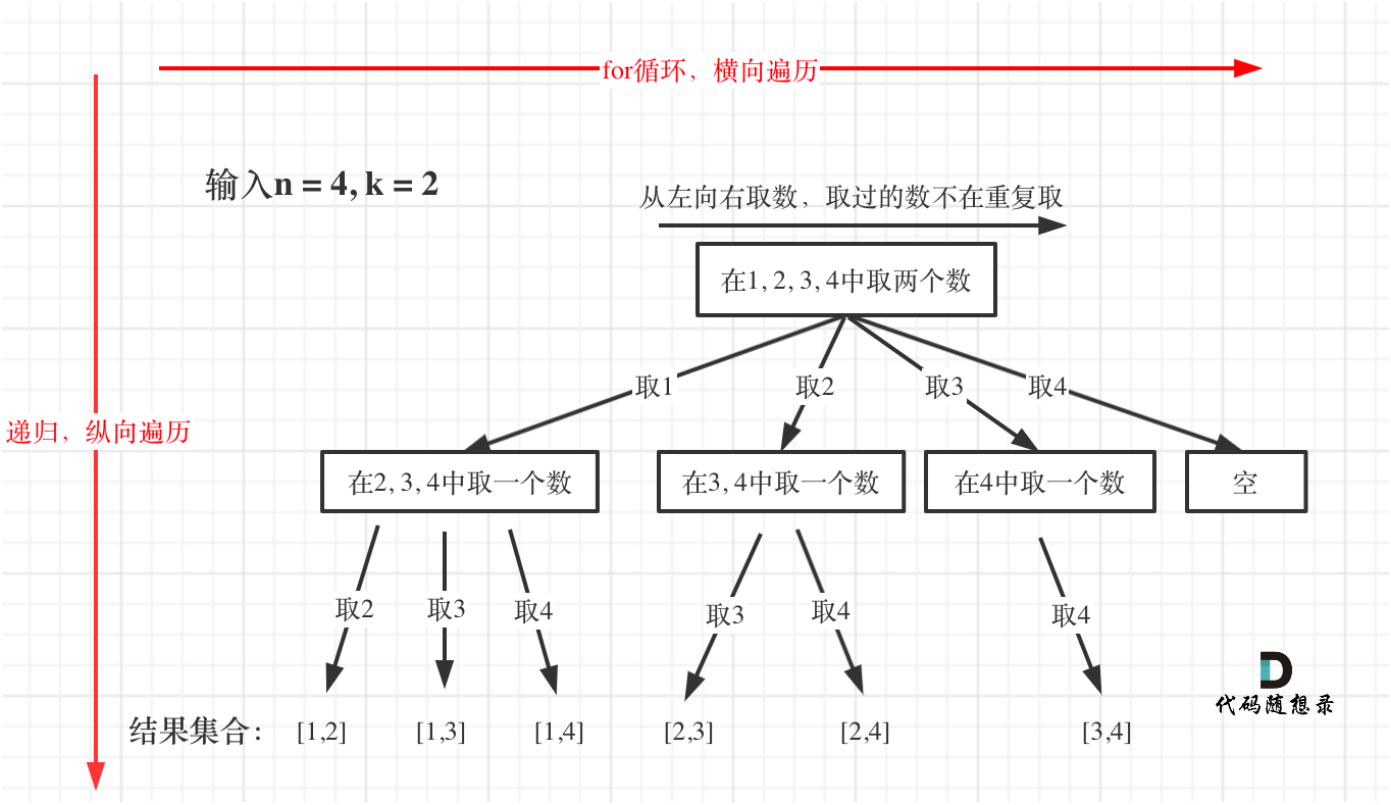
此时用result二维数组，把path保存起来，并终止本层递归。

所以终止条件代码如下：

```
if (path.size() == k) {
    result.push_back(path);
    return;
}
```

- 单层搜索的过程

回溯法的搜索过程就是一个树型结构的遍历过程，在如下图中，可以看出for循环用来横向遍历，递归的过程是纵向遍历。



如此我们才遍历完图中的这棵树。

for循环每次从startIndex开始遍历，然后用path保存取到的节点i。

代码如下：

```
for (int i = startIndex; i <= n; i++) { // 控制树的横向遍历
    path.push_back(i); // 处理节点
    backtracking(n, k, i + 1); // 递归：控制树的纵向遍历，注意下一层搜索要从i+1开始
    path.pop_back(); // 回溯，撤销处理的节点
}
```

可以看出backtracking（递归函数）通过不断调用自己一直往深处遍历，总会遇到叶子节点，遇到了叶子节点就要返回。

backtracking的下面部分就是回溯的操作了，撤销本次处理的结果。

关键地方都讲完了，组合问题C++完整代码如下：

```
class Solution {
private:
    vector<vector<int>> result; // 存放符合条件结果的集合
    vector<int> path; // 用来存放符合条件结果
    void backtracking(int n, int k, int startIndex) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i <= n; i++) {
            path.push_back(i); // 处理节点
            backtracking(n, k, i + 1); // 递归
            path.pop_back(); // 回溯，撤销处理的节点
        }
    }
public:
    vector<vector<int>> combine(int n, int k) {
        result.clear(); // 可以不写
        path.clear();   // 可以不写
        backtracking(n, k, 1);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

还记得我们在[关于回溯算法，你该了解这些！]中给出的回溯法模板么？

如下：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

对比一下本题的代码，是不是发现有点像！ 所以有了这个模板，就有解题的大体方向，不至于毫无头绪。

总结

组合问题是回溯法解决的经典问题，我们开始的时候给大家列举一个很形象的例子，就是n为100，k为50的话，直接想法就需要50层for循环。

从而引出了回溯法就是解决这种k层for循环嵌套的问题。

然后进一步把回溯法的搜索过程抽象为树形结构，可以直观的看出搜索的过程。

接着用回溯法三部曲，逐步分析了函数参数、终止条件和单层搜索的过程。

剪枝优化

我们说过，回溯法虽然是暴力搜索，但也有时候可以有点剪枝优化一下的。

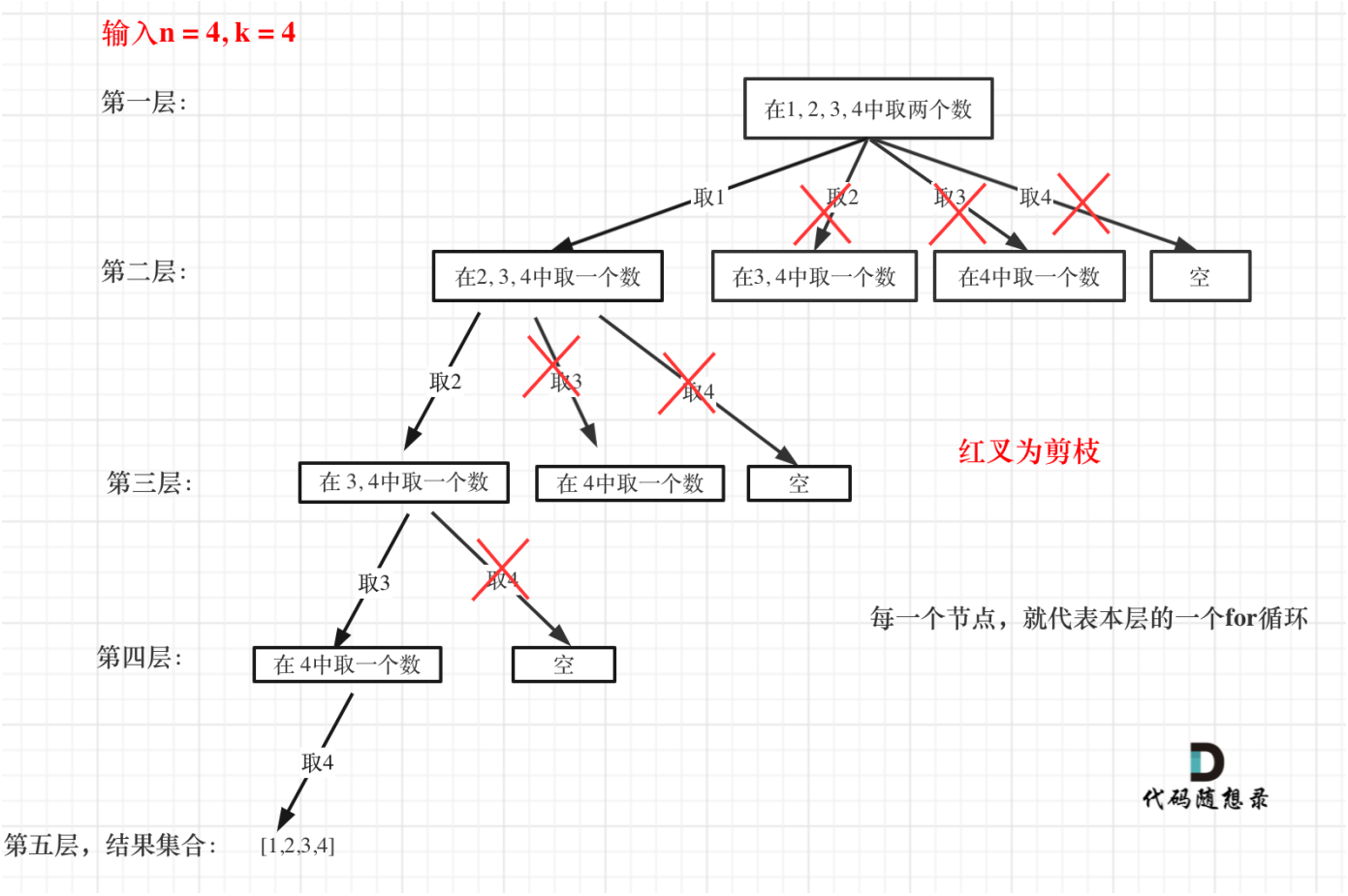
在遍历的过程中有如下代码：

```
for (int i = startIndex; i <= n; i++) {
    path.push_back(i);
    backtracking(n, k, i + 1);
    path.pop_back();
}
```

这个遍历的范围是可以剪枝优化的，怎么优化呢?

来举一个例子，n = 4，k = 4的话，那么第一层for循环的时候，从元素2开始的遍历都没有意义了。 在第二层for循环，从元素3开始的遍历都没有意义了。

这么说有点抽象，如图所示：



图中每一个节点（图中为矩形），就代表本层的一个for循环，那么每一层的for循环从第二个数开始遍历的话，都没有意义，都是无效遍历。

所以，可以剪枝的地方就在递归中每一层的for循环所选择的起始位置。

如果for循环选择的起始位置之后的元素个数 已经不足 我们需要的元素个数了，那么就没有必要搜索了。

注意代码中i，就是for循环里选择的起始位置。

```
for (int i = startIndex; i <= n; i++) {
```

接下来看一下优化过程如下：

1. 已经选择的元素个数：path.size();
2. 还需要的元素个数为: k - path.size();
3. 在集合n中至多要从该起始位置 : n - (k - path.size()) + 1，开始遍历

为什么有个+1呢，因为包括起始位置，我们要是一个左闭的集合。

举个例子，n = 4，k = 3， 目前已经选取的元素为0（path.size为0），n - (k - 0) + 1 即 4 - (3 - 0) + 1 = 2。

从2开始搜索都是合理的，可以是组合[2, 3, 4]。

这里大家想不懂的话，建议也举一个例子，就知道是不是要+1了。

所以优化之后的for循环是：

```
for (int i = startIndex; i <= n - (k - path.size()) + 1; i++) // i为本次搜索的起始位置
```

优化后整体代码如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(int n, int k, int startIndex) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i <= n - (k - path.size()) + 1; i++) { // 优化的地方
            path.push_back(i); // 处理节点
            backtracking(n, k, i + 1);
            path.pop_back(); // 回溯，撤销处理的节点
        }
    }
public:

    vector<vector<int>> combine(int n, int k) {
        backtracking(n, k, 1);
        return result;
    }
};
```

剪枝总结

本篇我们准对求组合问题的回溯法代码做了剪枝优化，这个优化如果不画图的话，其实不好理解，也不好讲清楚。

所以我依然是把整个回溯过程抽象为一棵树形结构，然后可以直观的看出，剪枝究竟是剪的哪里。

3.组合优化

算法公开课

《代码随想录》算法视频公开课：组合问题的剪枝操作，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

在回溯算法：求组合问题！中，我们通过回溯搜索法，解决了n个数中求k个数的组合问题。

文中的回溯法是可以剪枝优化的，本篇我们继续来看一下题目77. 组合。

链接：https://leetcode.cn/problems/combinations/

看本篇之前，需要先看回溯算法：求组合问题！。

大家先回忆一下[77. 组合]给出的回溯法的代码：

```
class Solution {
private:
    vector<vector<int>> result; // 存放符合条件结果的集合
    vector<int> path; // 用来存放符合条件结果
    void backtracking(int n, int k, int startIndex) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i <= n; i++) {
            path.push_back(i); // 处理节点
            backtracking(n, k, i + 1); // 递归
            path.pop_back(); // 回溯，撤销处理的节点
        }
    }
public:
    vector<vector<int>> combine(int n, int k) {
        result.clear(); // 可以不写
        path.clear();   // 可以不写
        backtracking(n, k, 1);
        return result;
    }
};
```

剪枝优化

我们说过，回溯法虽然是暴力搜索，但也有时候可以有点剪枝优化一下的。

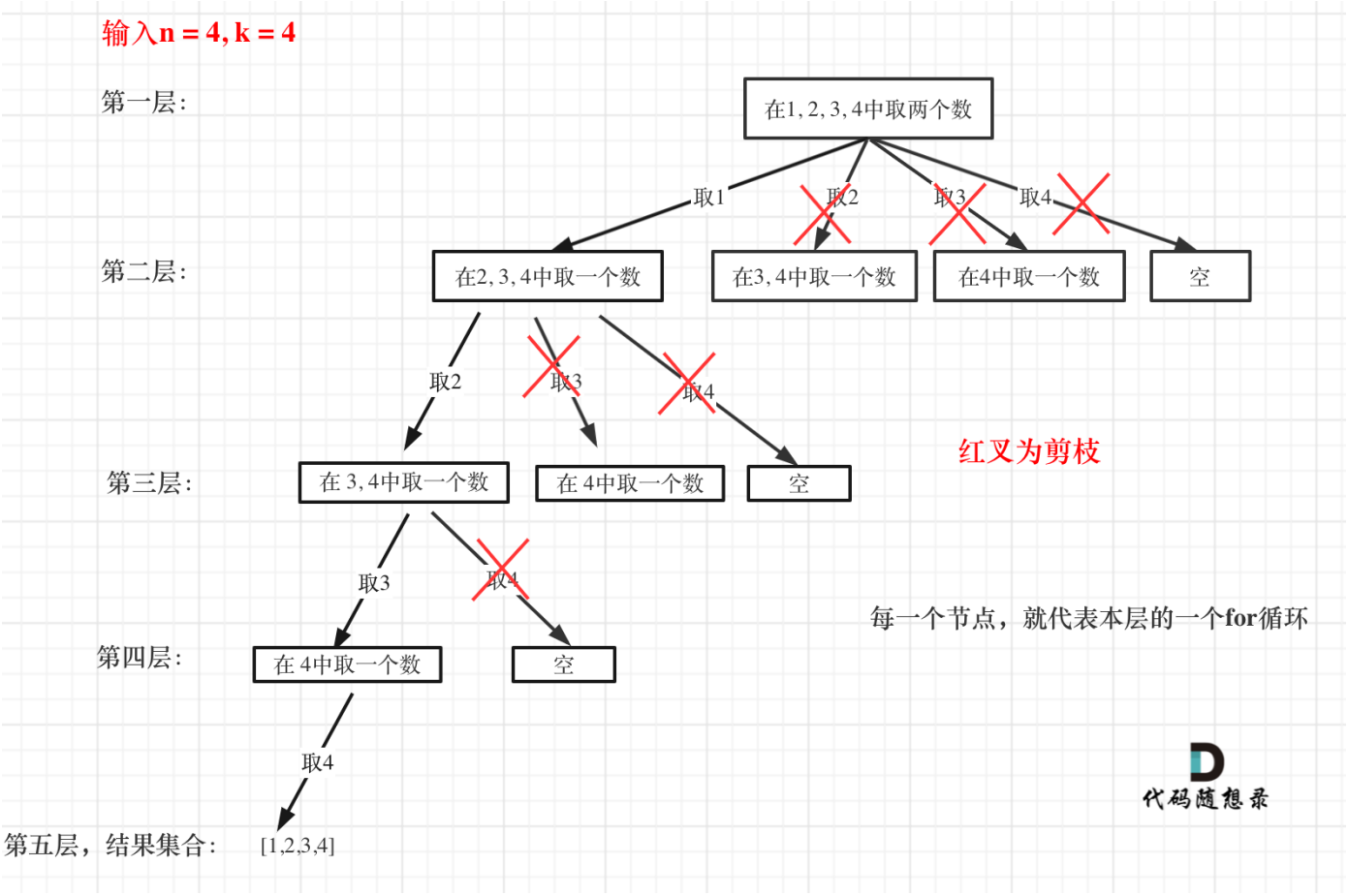
在遍历的过程中有如下代码：

```
for (int i = startIndex; i <= n; i++) {
    path.push_back(i);
    backtracking(n, k, i + 1);
    path.pop_back();
}
```

这个遍历的范围是可以剪枝优化的，怎么优化呢?

来举一个例子，n = 4，k = 4的话，那么第一层for循环的时候，从元素2开始的遍历都没有意义了。 在第二层for循环，从元素3开始的遍历都没有意义了。

这么说有点抽象，如图所示：



图中每一个节点（图中为矩形），就代表本层的一个for循环，那么每一层的for循环从第二个数开始遍历的话，都没有意义，都是无效遍历。

所以，可以剪枝的地方就在递归中每一层的for循环所选择的起始位置。

如果for循环选择的起始位置之后的元素个数 已经不足 我们需要的元素个数了，那么就没有必要搜索了。

注意代码中i，就是for循环里选择的起始位置。

```
for (int i = startIndex; i <= n; i++) {
```

接下来看一下优化过程如下：

1. 已经选择的元素个数：path.size();
2. 所需需要的元素个数为: k - path.size();
3. 列表中剩余元素 （n-i） >= 所需需要的元素个数（k - path.size()）
4. 在集合n中至多要从该起始位置 : i <= n - (k - path.size()) + 1，开始遍历

为什么有个+1呢，因为包括起始位置，我们要是一个左闭的集合。

举个例子，n = 4，k = 3， 目前已经选取的元素为0（path.size为0），n - (k - 0) + 1 即 4 - (3 - 0) + 1 = 2。

从2开始搜索都是合理的，可以是组合[2, 3, 4]。

这里大家想不懂的话，建议也举一个例子，就知道是不是要+1了。

所以优化之后的for循环是：

```
for (int i = startIndex; i <= n - (k - path.size()) + 1; i++) // i为本次搜索的起始位置
```

优化后整体代码如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(int n, int k, int startIndex) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i <= n - (k - path.size()) + 1; i++) { // 优化的地方
            path.push_back(i); // 处理节点
            backtracking(n, k, i + 1);
            path.pop_back(); // 回溯，撤销处理的节点
        }
    }
public:

    vector<vector<int>> combine(int n, int k) {
        backtracking(n, k, 1);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

总结

本篇我们针对求组合问题的回溯法代码做了剪枝优化，这个优化如果不画图的话，其实不好理解，也不好讲清楚。

所以我依然是把整个回溯过程抽象为一棵树形结构，然后可以直观的看出，剪枝究竟是剪的哪里。

就酱，学到了就帮Carl转发一下吧，让更多的同学知道这里！

> 别看本篇选的是组合总和III，而不是组合总和，本题和上一篇77.组合相比难度刚刚好！

4.组合总和III

力扣题目链接

找出所有相加之和为 n 的 k 个数的组合。组合中只允许含有 1 - 9 的正整数，并且每种组合中不存在重复的数字。

说明：

- 所有数字都是正整数。
- 解集不能包含重复的组合。

示例 1:
输入: k = 3, n = 7
输出: [[1,2,4]]

示例 2:
输入: k = 3, n = 9
输出: [[1,2,6], [1,3,5], [2,3,4]]

算法公开课

《代码随想录》算法视频公开课：和组合问题有啥区别？回溯算法如何剪枝？ | LeetCode：216.组合总和III，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题就是在[1,2,3,4,5,6,7,8,9]这个集合中找到和为n的k个数的组合。

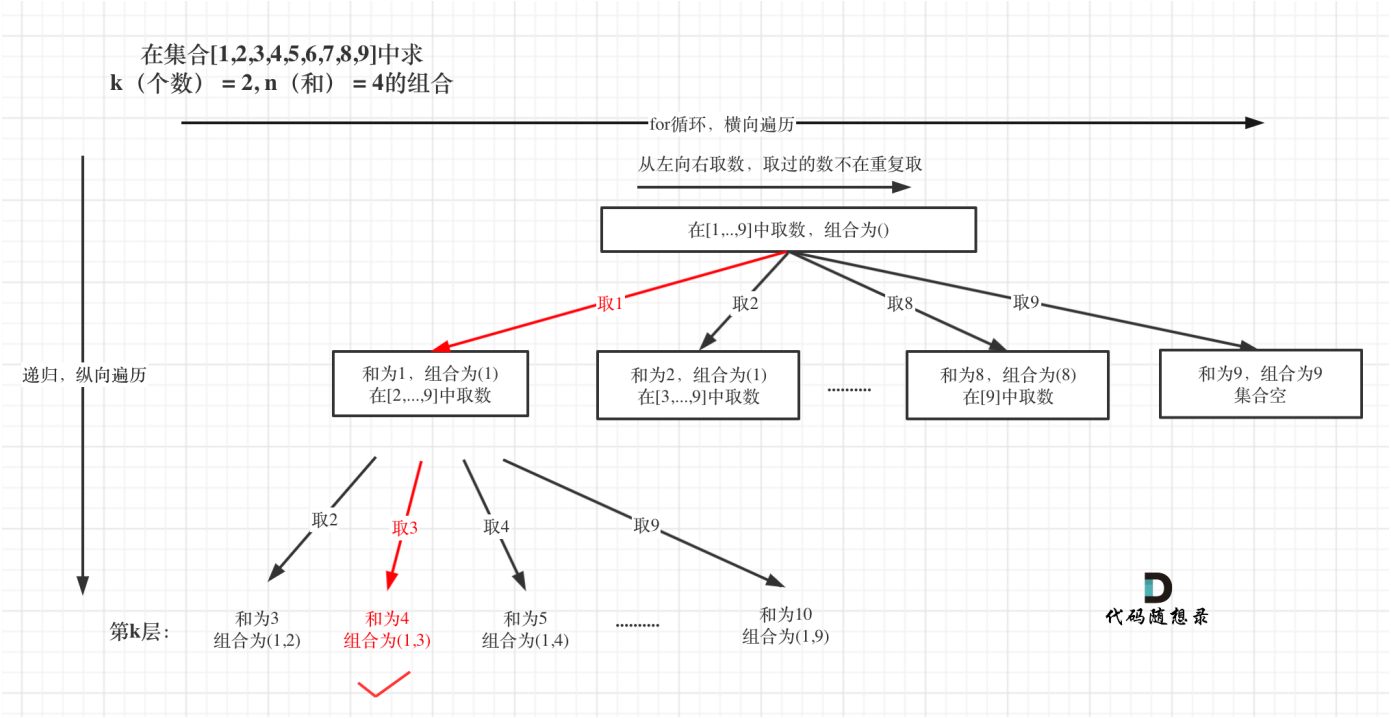
相对于77. 组合，无非就是多了一个限制，本题是要找到和为n的k个数的组合，而整个集合已经是固定的了[1,...,9]。

想到这一点了，做过77. 组合之后，本题是简单一些了。

本题k相当于树的深度，9（因为整个集合就是9个数）就是树的宽度。

例如 k = 2，n = 4的话，就是在集合[1,2,3,4,5,6,7,8,9]中求 k（个数） = 2, n（和） = 4的组合。

选取过程如图：



图中，可以看出，只有最后取到集合（1，3）和为4 符合条件。

回溯三部曲

- **确定递归函数参数**

和77. 组合一样，依然需要一维数组path来存放符合条件的结果，二维数组result来存放结果集。

这里我依然定义path 和 result为全局变量。

至于为什么取名为path？从上面树形结构中，可以看出，结果其实就是一条根节点到叶子节点的路径。

```
vector<vector<int>> result; // 存放结果集
vector<int> path; // 符合条件的结果
```

接下来还需要如下参数：

- targetSum（int）目标和，也就是题目中的n。
- k（int）就是题目中要求k个数的集合。
- sum（int）为已经收集的元素的总和，也就是path里元素的总和。
- startIndex（int）为下一层for循环搜索的起始位置。

所以代码如下：

```
vector<vector<int>> result;
vector<int> path;
void backtracking(int targetSum, int k, int sum, int startIndex)
```

其实这里sum这个参数也可以省略，每次targetSum减去选取的元素数值，然后判断如果targetSum为0了，说明收集到符合条件的结果了，我这里为了直观便于理解，还是加一个sum参数。

还要强调一下，回溯法中递归函数参数很难一次性确定下来，一般先写逻辑，需要啥参数了，填什么参数。

- 确定终止条件

什么时候终止呢?

在上面已经说了，k其实就已经限制树的深度，因为就取k个元素，树再往下深了没有意义。

所以如果path.size() 和 k相等了，就终止。

如果此时path里收集到的元素和（sum） 和targetSum（就是题目描述的n）相同了，就用result收集当前的结果。

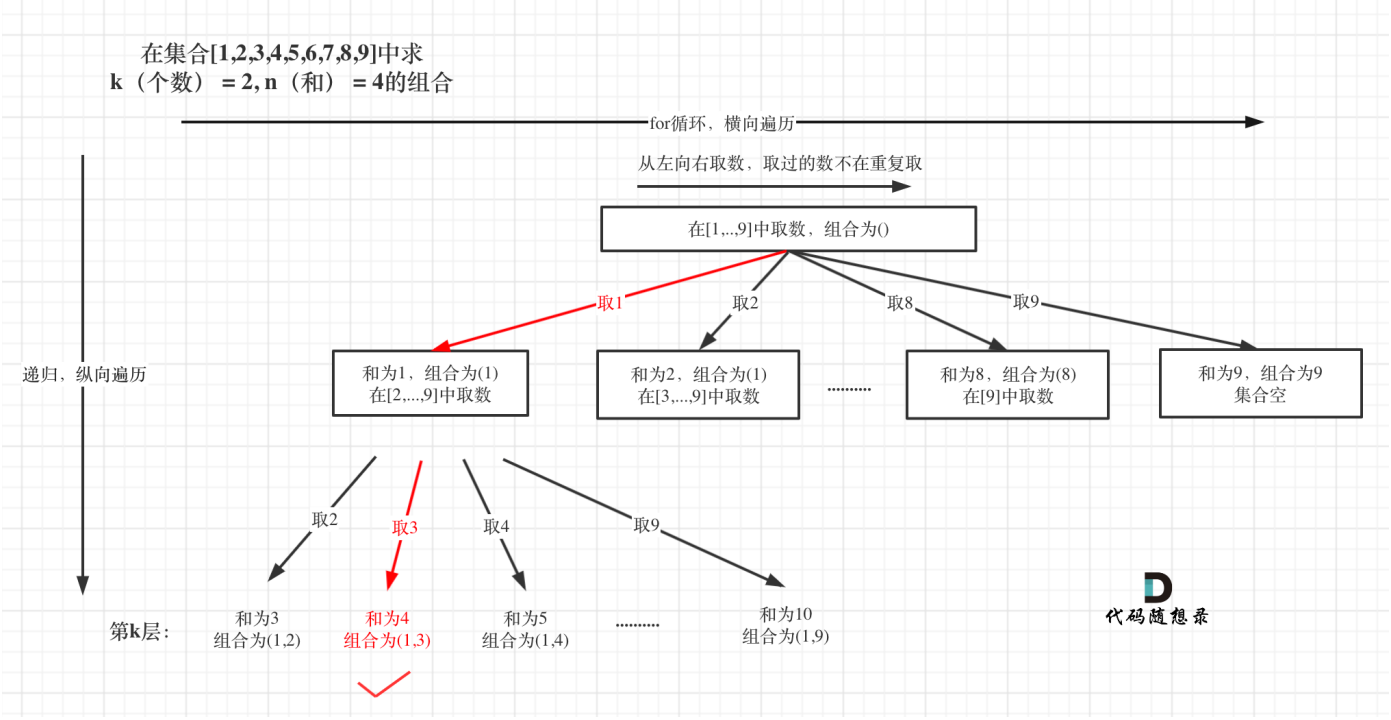
所以 终止代码如下：

```
if (path.size() == k) {
    if (sum == targetSum) result.push_back(path);
    return; // 如果path.size() == k 但sum != targetSum 直接返回
}
```

- **单层搜索过程**

本题和77. 组合区别之一就是集合固定的就是9个数[1,...,9]，所以for循环固定i<=9

如图：



处理过程就是 path收集每次选取的元素，相当于树型结构里的边，sum来统计path里元素的总和。

代码如下：

```
for (int i = startIndex; i <= 9; i++) {
    sum += i;
    path.push_back(i);
    backtracking(targetSum, k, sum, i + 1); // 注意i+1调整startIndex
    sum -= i; // 回溯
    path.pop_back(); // 回溯
}
```

别忘了处理过程 和 回溯过程是一一对应的，处理有加，回溯就要有减！

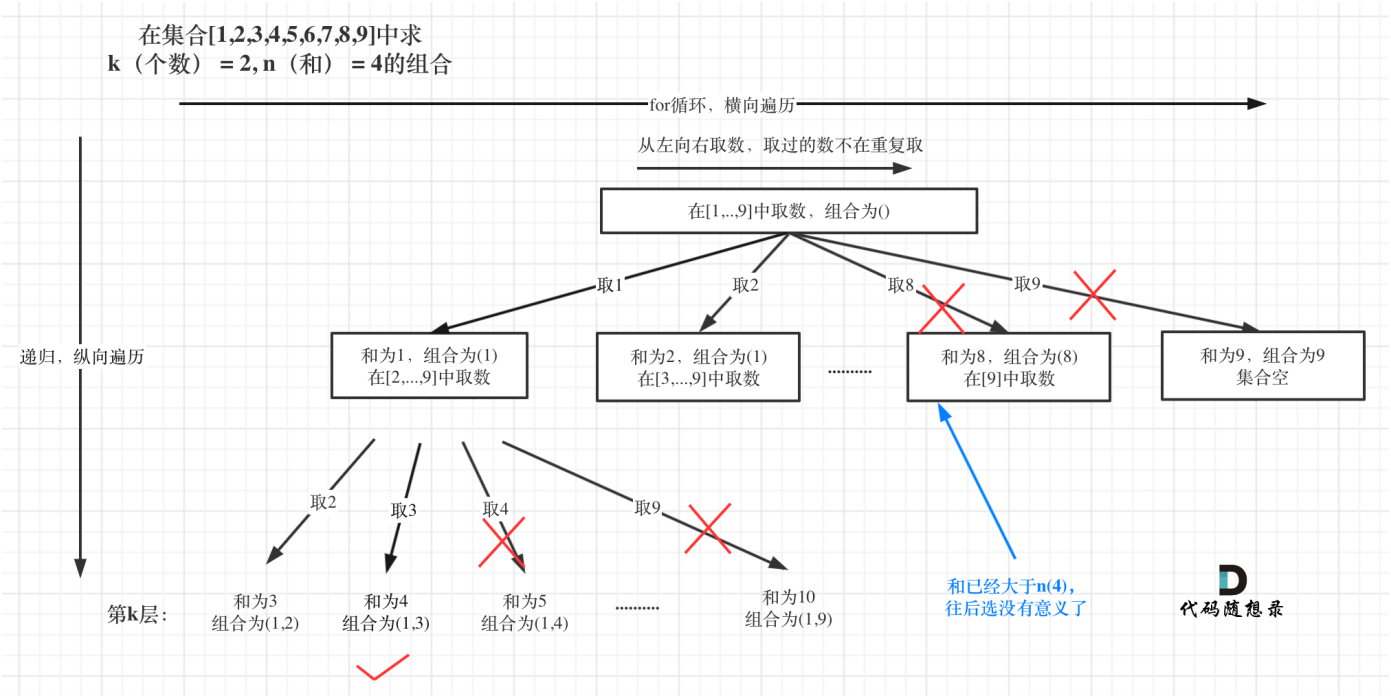
参照关于回溯算法，你该了解这些！中的模板，不难写出如下C++代码：

```
class Solution {
private:
    vector<vector<int>> result; // 存放结果集
    vector<int> path; // 符合条件的结果
    // targetSum：目标和，也就是题目中的n。
    // k：题目中要求k个数的集合。
    // sum：已经收集的元素的总和，也就是path里元素的总和。
    // startIndex：下一层for循环搜索的起始位置。
    void backtracking(int targetSum, int k, int sum, int startIndex) {
        if (path.size() == k) {
            if (sum == targetSum) result.push_back(path);
            return; // 如果path.size() == k 但sum != targetSum 直接返回
        }
        for (int i = startIndex; i <= 9; i++) {
            sum += i; // 处理
            path.push_back(i); // 处理
            backtracking(targetSum, k, sum, i + 1); // 注意i+1调整startIndex
            sum -= i; // 回溯
            path.pop_back(); // 回溯
        }
    }

public:
    vector<vector<int>> combinationSum3(int k, int n) {
        result.clear(); // 可以不加
        path.clear();   // 可以不加
        backtracking(n, k, 0, 1);
        return result;
    }
};
```

剪枝

这道题目，剪枝操作其实是很容易想到了，想必大家看上面的树形图的时候已经想到了。

如图：



已选元素总和如果已经大于n（图中数值为4）了，那么往后遍历就没有意义了，直接剪掉。

那么剪枝的地方可以放在递归函数开始的地方，剪枝代码如下：

```
if (sum > targetSum) { // 剪枝操作
    return;
}
```

当然这个剪枝也可以放在 调用递归之前，即放在这里，只不过要记得 要回溯操作给做了。

```
for (int i = startIndex; i <= 9 - (k - path.size()) + 1; i++) { // 剪枝
    sum += i; // 处理
    path.push_back(i); // 处理
    if (sum > targetSum) { // 剪枝操作
        sum -= i; // 剪枝之前先把回溯做了
        path.pop_back(); // 剪枝之前先把回溯做了
        return;
    }
    backtracking(targetSum, k, sum, i + 1); // 注意i+1调整startIndex
    sum -= i; // 回溯
    path.pop_back(); // 回溯
}
```

和回溯算法：组合问题再剪剪枝 一样，for循环的范围也可以剪枝，i <= 9 - (k - path.size()) + 1就可以了。

最后C++代码如下：

```
class Solution {
private:
    vector<vector<int>> result; // 存放结果集
    vector<int> path; // 符合条件的结果
    void backtracking(int targetSum, int k, int sum, int startIndex) {
        if (sum > targetSum) { // 剪枝操作
            return; // 如果path.size() == k 但sum != targetSum 直接返回
        }
        if (path.size() == k) {
            if (sum == targetSum) result.push_back(path);
            return;
        }
        for (int i = startIndex; i <= 9 - (k - path.size()) + 1; i++) { // 剪枝
            sum += i; // 处理
            path.push_back(i); // 处理
            backtracking(targetSum, k, sum, i + 1); // 注意i+1调整startIndex
            sum -= i; // 回溯
            path.pop_back(); // 回溯
        }
    }

public:
    vector<vector<int>> combinationSum3(int k, int n) {
        result.clear(); // 可以不加
        path.clear();   // 可以不加
        backtracking(n, k, 0, 1);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

总结

开篇就介绍了本题与77.组合的区别，相对来说加了元素总和的限制，如果做完77.组合再做本题在合适不过。

分析完区别，依然把问题抽象为树形结构，按照回溯三部曲进行讲解，最后给出剪枝的优化。

相信做完本题，大家对组合问题应该有初步了解了。

5.电话号码的字母组合

力扣题目链接

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

示例:

- 输入：“23”
- 输出：["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].

说明：尽管上面的答案是按字典序排列的，但是你可以任意选择答案输出的顺序。

算法公开课

《代码随想录》算法视频公开课：：还得用回溯算法！ | LeetCode：17.电话号码的字母组合，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

从示例上来说，输入"23"，最直接的想法就是两层for循环遍历了吧，正好把组合的情况都输出了。

如果输入"233"呢，那么就三层for循环，如果"2333"呢，就四层for循环.....

大家应该感觉出和77.组合遇到的一样的问题，就是这for循环的层数如何写出来，此时又是回溯法登场的时候了。

理解本题后，要解决如下三个问题：

1. 数字和字母如何映射
2. 两个字母就两个for循环，三个字符我就三个for循环，以此类推，然后发现代码根本写不出来
3. 输入1 * #按键等等异常情况

数字和字母如何映射

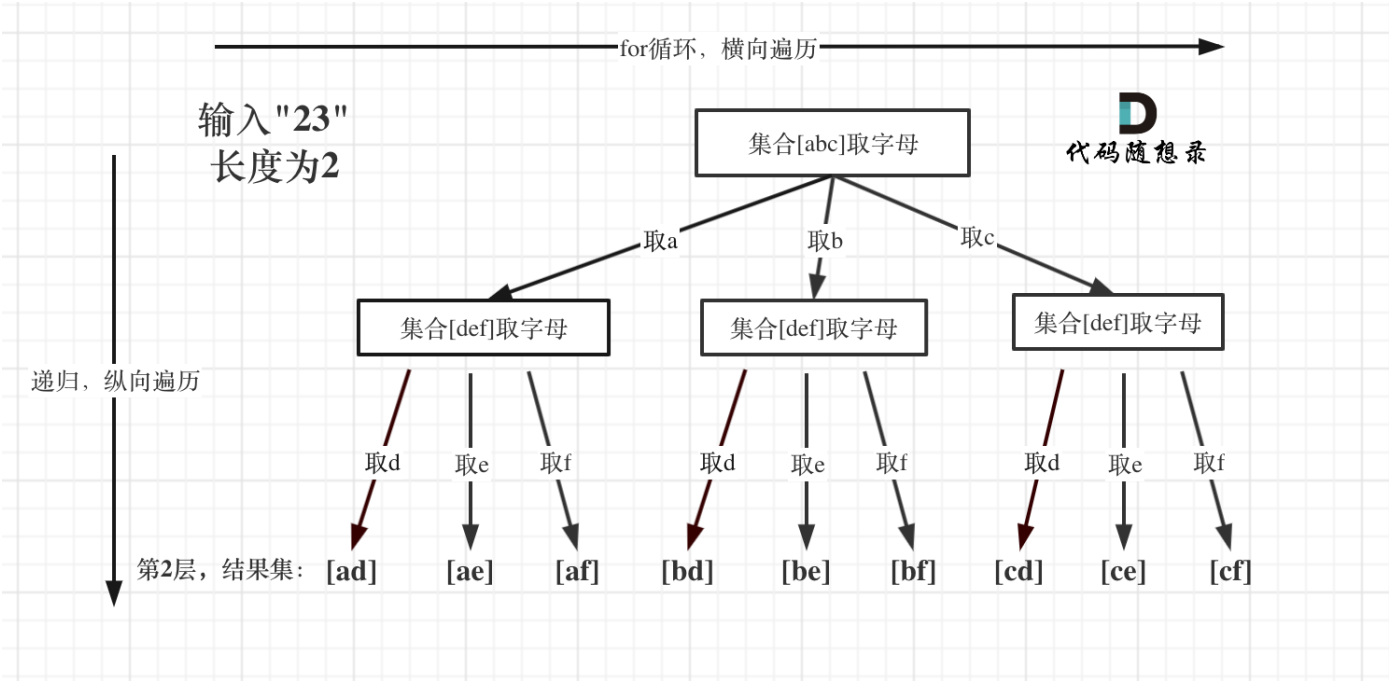
可以使用map或者定义一个二维数组，例如：string letterMap[10]，来做映射，我这里定义一个二维数组，代码如下：

```
const string letterMap[10] = {
    "", // 0
    "", // 1
    "abc", // 2
    "def", // 3
    "ghi", // 4
    "jkl", // 5
    "mno", // 6
    "pqrs", // 7
    "tuv", // 8
    "wxyz", // 9
};
```

回溯法来解决n个for循环的问题

对于回溯法还不了解的同学看这篇：[关于回溯算法，你该了解这些！]

例如：输入："23"，抽象为树形结构，如图所示：



图中可以看出遍历的深度，就是输入"23"的长度，而叶子节点就是我们要收集的结果，输出["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]。

回溯三部曲：

- 确定回溯函数参数

首先需要一个字符串s来收集叶子节点的结果，然后用一个字符串数组result保存起来，这两个变量我依然定义为全局。

再来看参数，参数指定是有题目中给的string digits，然后还要有一个参数就是int型的index。

注意这个index可不是 77.组合和216.组合总和III中的startIndex了。

这个index是记录遍历第几个数字了，就是用来遍历digits的（题目中给出数字字符串），同时index也表示树的深度。

代码如下：

```
vector<string> result;
string s;
void backtracking(const string& digits, int index)
```

- 确定终止条件

例如输入用例"23"，两个数字，那么根节点往下递归两层就可以了，叶子节点就是要收集的结果集。

那么终止条件就是如果index 等于 输入的数字个数（digits.size）了（本来index就是用来遍历digits的）。

然后收集结果，结束本层递归。

代码如下：

```
if (index == digits.size()) {
    result.push_back(s);
    return;
}
```

- 确定单层遍历逻辑

首先要取index指向的数字，并找到对应的字符集（手机键盘的字符集）。

然后for循环来处理这个字符集，代码如下：

```
int digit = digits[index] - '0';        // 将index指向的数字转为int
string letters = letterMap[digit];      // 取数字对应的字符集
for (int i = 0; i < letters.size(); i++) {
    s.push_back(letters[i]);            // 处理
    backtracking(digits, index + 1);    // 递归，注意index+1，一下层要处理下一个数字了
    s.pop_back();                       // 回溯
}
```

注意这里for循环，可不像是在回溯算法：求组合问题！和回溯算法：求组合总和！中从startIndex开始遍历的。

因为本题每一个数字代表的是不同集合，也就是求不同集合之间的组合，而77. 组合和216.组合总和III都是求同一个集合中的组合！

注意：输入1 * #按键等等异常情况

代码中最好考虑这些异常情况，但题目的测试数据中应该没有异常情况的数据，所以我就没有加了。

但是要知道会有这些异常，如果是现场面试中，一定要考虑到！

关键地方都讲完了，按照关于回溯算法，你该了解这些！中的回溯法模板，不难写出如下C++代码：

```
// 版本一
class Solution {
private:
    const string letterMap[10] = {
        "", // 0
        "", // 1
        "abc", // 2
        "def", // 3
        "ghi", // 4
        "jkl", // 5
        "mno", // 6
        "pqrs", // 7
        "tuv", // 8
        "wxyz", // 9
    };
public:
    vector<string> result;
    string s;
    void backtracking(const string& digits, int index) {
        if (index == digits.size()) {
            result.push_back(s);
            return;
        }
        int digit = digits[index] - '0';        // 将index指向的数字转为int
        string letters = letterMap[digit];      // 取数字对应的字符集
        for (int i = 0; i < letters.size(); i++) {
            s.push_back(letters[i]);            // 处理
            backtracking(digits, index + 1);    // 递归，注意index+1，一下层要处理下一个数字
了
            s.pop_back();                       // 回溯
        }
    }
    vector<string> letterCombinations(string digits) {
        s.clear();
        result.clear();
        if (digits.size() == 0) {
            return result;
        }
        backtracking(digits, 0);
        return result;
    }
};
```

- 时间复杂度: O(3^m * 4^n)，其中 m 是对应四个字母的数字个数，n 是对应三个字母的数字个数
- 空间复杂度: O(3^m * 4^n)

一些写法，是把回溯的过程放在递归函数里了，例如如下代码，我可以写成这样：（注意注释中不一样的地方）

```
// 版本二
class Solution {
private:
        const string letterMap[10] = {
            "", // 0
            "", // 1
            "abc", // 2
            "def", // 3
            "ghi", // 4
            "jkl", // 5
            "mno", // 6
            "pqrs", // 7
            "tuv", // 8
            "wxyz", // 9
        };
public:
    vector<string> result;
    void getCombinations(const string& digits, int index, const string& s) { // 注意参数的不同
        if (index == digits.size()) {
            result.push_back(s);
            return;
        }
        int digit = digits[index] - '0';
        string letters = letterMap[digit];
        for (int i = 0; i < letters.size(); i++) {
            getCombinations(digits, index + 1, s + letters[i]);  // 注意这里的不同
        }
    }
    vector<string> letterCombinations(string digits) {
        result.clear();
        if (digits.size() == 0) {
            return result;
        }
        getCombinations(digits, 0, "");
        return result;

    }
};
```

我不建议把回溯藏在递归的参数里这种写法，很不直观，我在二叉树：以为使用了递归，其实还隐藏着回溯这篇文章中也深度分析了，回溯隐藏在了哪里。

所以大家可以按照版本一来写就可以了。

总结

本篇将题目的三个要点一一列出，并重点强调了和前面讲解过的77. 组合和216.组合总和III的区别，本题是多个集合求组合，所以在回溯的搜索过程中，都有一些细节需要注意的。

其实本题不算难，但也处处是细节，大家还要自己亲自动手写一写。

---

6. 本周小结！（回溯算法系列一）

周一

本周我们正式开始了回溯算法系列，那么首先当然是概述。

在关于回溯算法，你该了解这些！中介绍了什么是回溯，回溯法的效率，回溯法解决的问题以及回溯法模板。

回溯是递归的副产品，只要有递归就会有回溯。

回溯法就是暴力搜索，并不是什么高效的算法，最多在剪枝一下。

回溯算法能解决如下问题：

- 组合问题：N个数里面按一定规则找出k个数的集合
- 排列问题：N个数按一定规则全排列，有几种排列方式
- 切割问题：一个字符串按一定规则有几种切割方式
- 子集问题：一个N个数的集合里有多少符合条件的子集
- 棋盘问题：N皇后，解数独等等

是不是感觉回溯算法有点厉害了。

回溯法确实不好理解，所以需要把回溯法抽象为一个图形来理解就容易多了，每一道回溯法的题目都可以抽象为树形结构。

针对很多同学都写不好回溯，我在关于回溯算法，你该了解这些！用回溯三部曲，分析了回溯算法，并给出了回溯法的模板。

这个模板会伴随整个回溯法系列！

周二

在回溯算法：求组合问题！中，我们开始用回溯法解决第一道题目，组合问题。

我在文中开始的时候给大家列举k层for循环例子，进而得出都是同样是暴力解法，为什么要用回溯法。

此时大家应该深有体会回溯法的魅力，用递归控制for循环嵌套的数量！

本题我把回溯问题抽象为树形结构，可以直观的看出其搜索的过程：**for循环横向遍历，递归纵向遍历，回溯不断调整结果集**。

周三

针对回溯算法：求组合问题！还可以做剪枝的操作。

在回溯算法：组合问题再剪剪枝中把回溯法代码做了剪枝优化，在文中我依然把问题抽象为一个树形结构，大家可以一目了然剪的究竟是哪里。

剪枝精髓是：for循环在寻找起点的时候要有一个范围，如果这个起点到集合终止之间的元素已经不够 题目要求的k个元素了，就没有必要搜索了。

周四

在回溯算法：求组合总和！中，相当于 回溯算法：求组合问题！加了一个元素总和的限制。

整体思路还是一样的，本题的剪枝会好想一些，即：**已选元素总和如果已经大于n（题中要求的和）了，那么往后遍历就没有意义了，直接剪掉。**

在本题中，依然还可以有一个剪枝，就是回溯算法：组合问题再剪剪枝中提到的，对for循环选择的起始范围的剪枝。

所以，剪枝的代码，可以把for循环，加上 `i <= 9 - (k - path.size()) + 1` 的限制！

组合总和问题还有一些花样，下周还会介绍到。

周五

在回溯算法：电话号码的字母组合中，开始用多个集合来求组合，还是熟悉的模板题目，但是有一些细节。

例如这里for循环，可不像是在 回溯算法：求组合问题！和回溯算法：求组合总和！中从startIndex开始遍历的。

因为本题每一个数字代表的是不同集合，也就是求不同集合之间的组合，而回溯算法：求组合问题！和回溯算法：求组合总和！都是是求同一个集合中的组合！

如果大家在现场面试的时候，一定要注意各种输入异常的情况，例如本题输入1 * #按键。

其实本题不算难，但也处处是细节，还是要反复琢磨。

周六

因为之前链表系列没有写总结，虽然链表系列已经是两个月前的事情，但还是有必要补一下。

所以给出链表：总结篇！，这里对之前链表理论基础和经典题目进行了总结。

同时对链表：环找到了，那入口呢？中求环入口的问题又进行了补充证明，可以说把环形链表的方方面面都讲的很通透了，大家如果没有做过环形链表的题目一定要去做一做。

总结

相信通过这一周对回溯法的学习，大家已经掌握其题本套路了，也不会对回溯法那么畏惧了。

回溯法抽象为树形结构后，其遍历过程就是：**for循环横向遍历，递归纵向遍历，回溯不断调整结果集。**

这个是我做了很多回溯的题目，不断摸索其规律才总结出来的。

对于回溯法的整体框架，网上搜的文章这块一般都说不清楚，按照天上掉下来的代码对着讲解，不知道究竟是怎么来的，也不知道为什么要这么写。

所以，录友们刚开始学回溯法，起跑姿势就很标准了，哈哈。

下周依然是回溯法，难度又要上升一个台阶了。

最后祝录友们周末愉快！

如果感觉「代码随想录」不错，就分享给身边的同学朋友吧，一起来学习算法！

---

7. 组合总和

力扣题目链接

给定一个无重复元素的数组 candidates 和一个目标数 target ，找出 candidates 中所有可以使数字和为 target 的组合。

candidates 中的数字可以无限制重复被选取。

说明：

- 所有数字（包括 target）都是正整数。
- 解集不能包含重复的组合。

示例 1：

- 输入：candidates = [2,3,6,7], target = 7,
- 所求解集为：
  [
  [7],
  [2,2,3]
  ]

示例 2：

- 输入：candidates = [2,3,5], target = 8,
- 所求解集为：
  [
  [2,2,2,2],
  [2,3,3],
  [3,5]
  ]

算法公开课

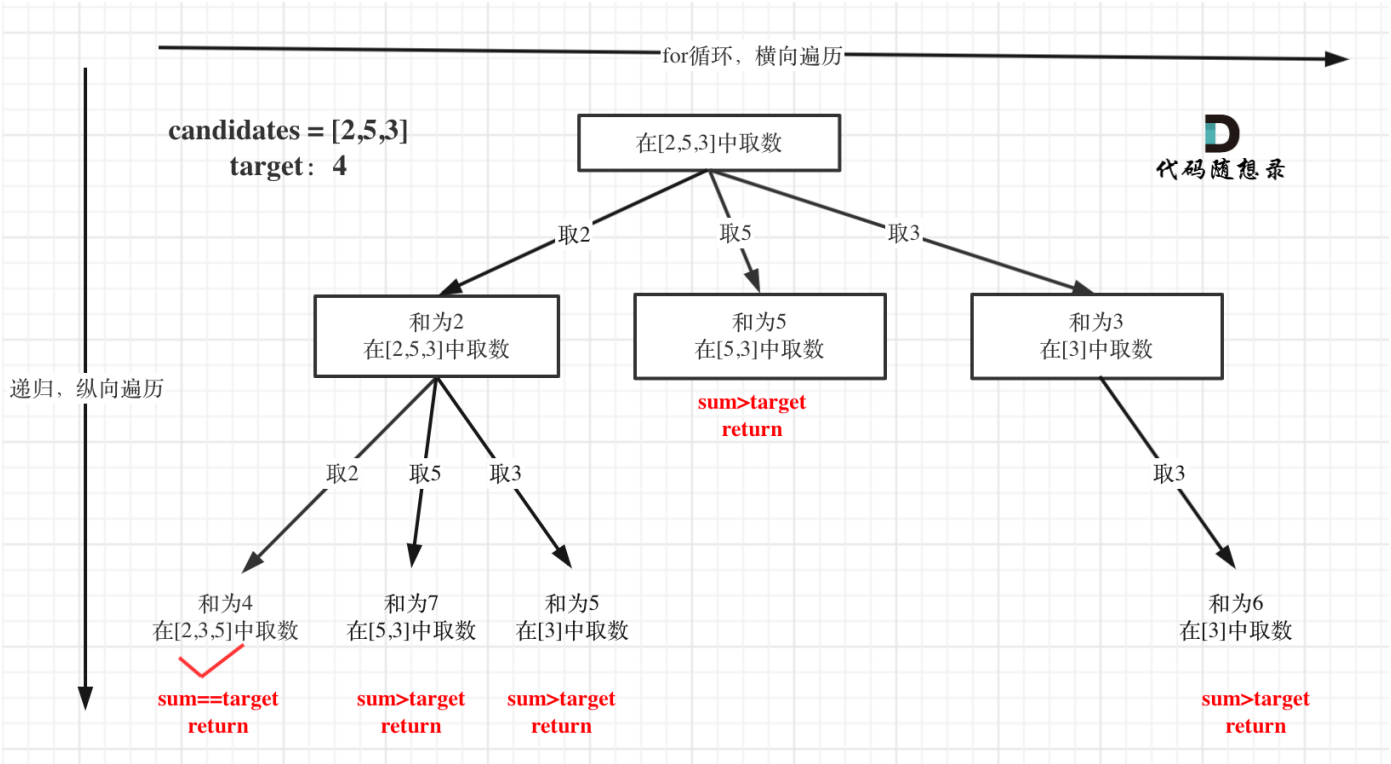
《代码随想录》算法视频公开课：Leetcode:39. 组合总和讲解，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

题目中的**无限制重复被选取，吓得我赶紧想想 出现0 可咋办**，然后看到下面提示：1 <= candidates[i] <= 200，我就放心了。

本题和77.组合，216.组合总和III的区别是：本题没有数量要求，可以无限重复，但是有总和的限制，所以间接的也是有个数的限制。

本题搜索的过程抽象成树形结构如下：



注意图中叶子节点的返回条件，因为本题没有组合数量要求，仅仅是总和的限制，所以递归没有层数的限制，只要选取的元素总和超过target，就返回！

而在77.组合和216.组合总和III 中都可以知道要递归K层，因为要取k个元素的组合。

回溯三部曲

- 递归函数参数

这里依然是定义两个全局变量，二维数组result存放结果集，数组path存放符合条件的结果。（这两个变量可以作为函数参数传入）

首先是题目中给出的参数，集合candidates, 和目标值target。

此外我还定义了int型的sum变量来统计单一结果path里的总和，其实这个sum也可以不用，用target做相应的减法就可以了，最后如何target==0就说明找到符合的结果了，但为了代码逻辑清晰，我依然用了sum。

本题还需要startIndex来控制for循环的起始位置，对于组合问题，什么时候需要startIndex呢？

我举过例子，如果是一个集合来求组合的话，就需要startIndex，例如：77.组合，216.组合总和III。

如果是多个集合取组合，各个集合之间相互不影响，那么就不用startIndex，例如：17.电话号码的字母组合

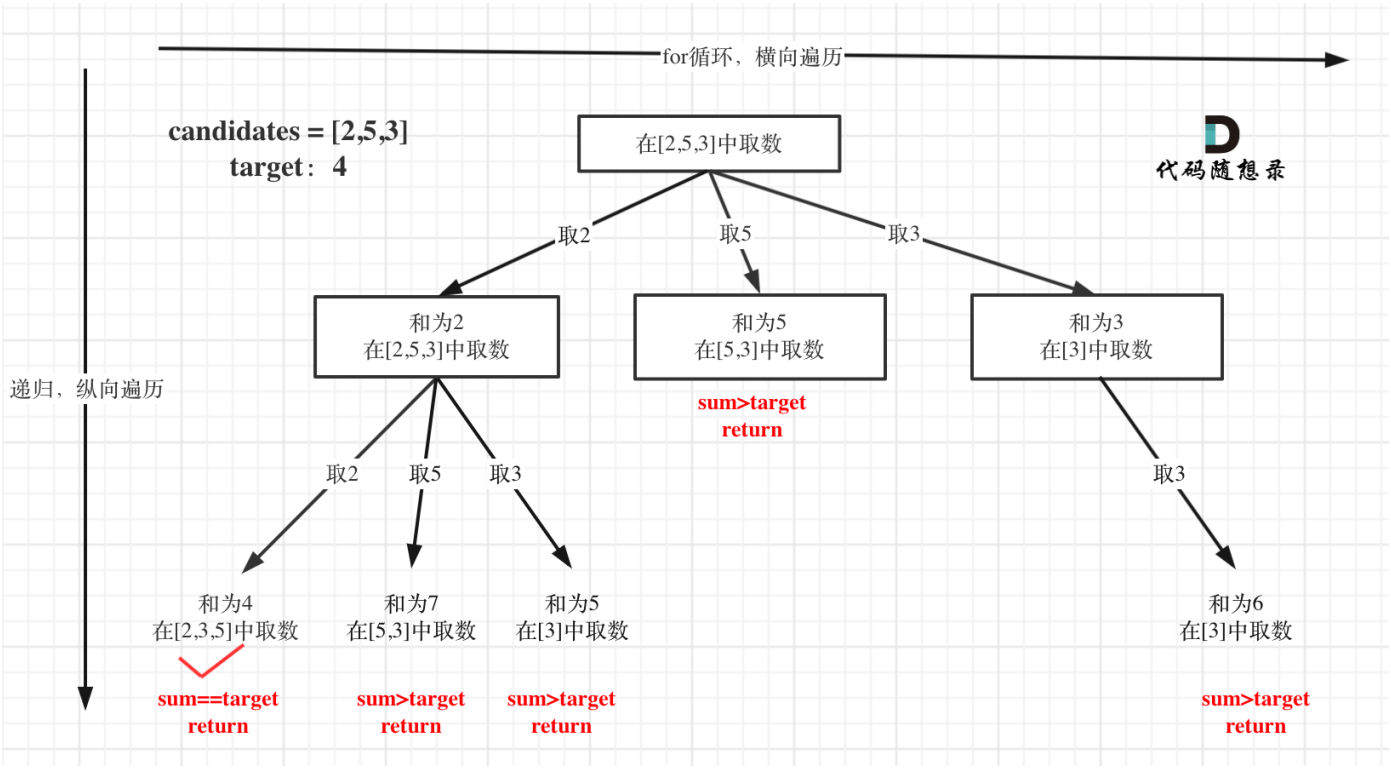
注意以上我只是说求组合的情况，如果是排列问题，又是另一套分析的套路，后面我再讲解排列的时候就重点介绍。

代码如下：

```
vector<vector<int>> result;
vector<int> path;
void backtracking(vector<int>& candidates, int target, int sum, int startIndex)
```

- 递归终止条件

在如下树形结构中：



从叶子节点可以清晰看到，终止只有两种情况，sum大于target和sum等于target。

sum等于target的时候，需要收集结果，代码如下：

```
if (sum > target) {
    return;
}
if (sum == target) {
    result.push_back(path);
    return;
}
```

- 单层搜索的逻辑

单层for循环依然是从startIndex开始，搜索candidates集合。

注意本题和77.组合、216.组合总和III的一个区别是：本题元素为可重复选取的。

如何重复选取呢，看代码，注释部分：

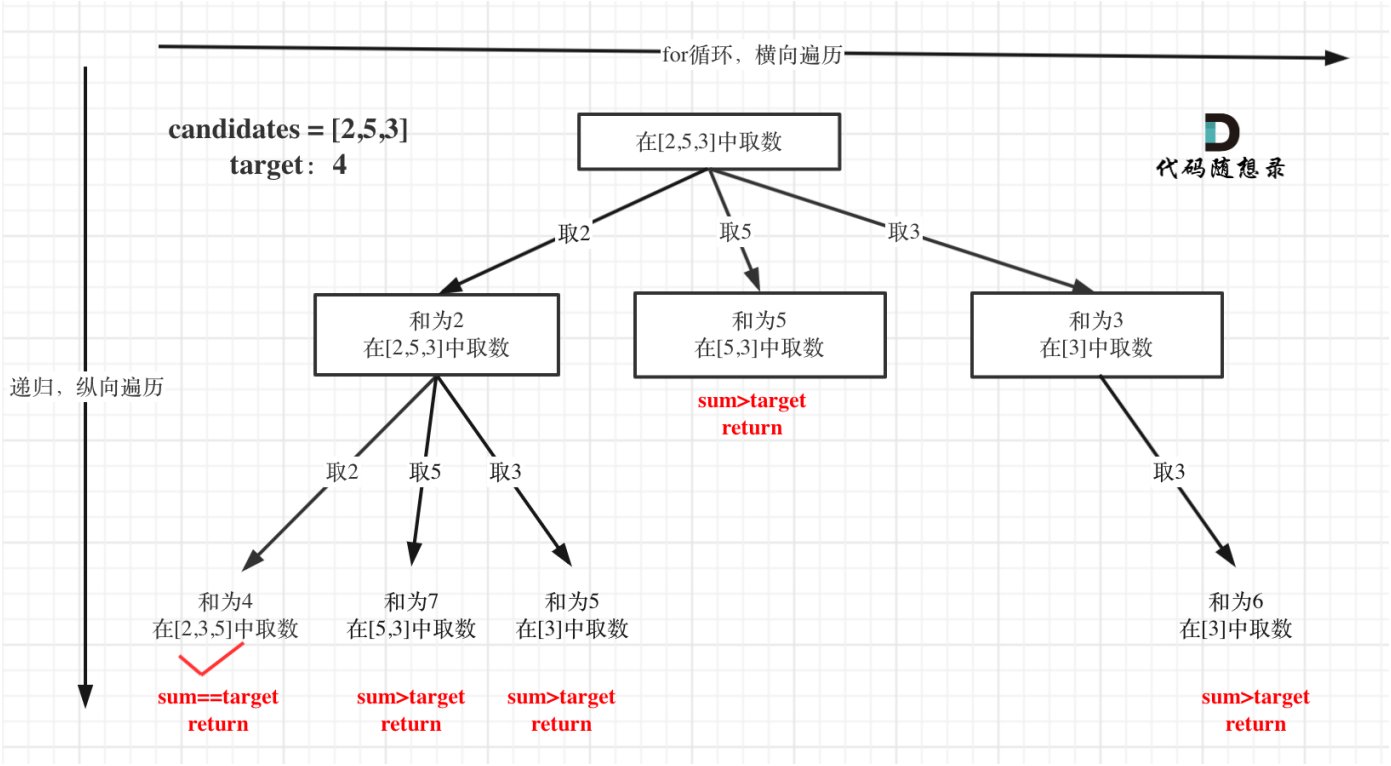
```
for (int i = startIndex; i < candidates.size(); i++) {
    sum += candidates[i];
    path.push_back(candidates[i]);
    backtracking(candidates, target, sum, i); // 关键点:不用i+1了，表示可以重复读取当前的数
    sum -= candidates[i];    // 回溯
    path.pop_back();         // 回溯
}
```

按照关于回溯算法，你该了解这些！中给出的模板，不难写出如下C++完整代码：

```
// 版本一
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex) {
        if (sum > target) {
            return;
        }
        if (sum == target) {
            result.push_back(path);
            return;
        }

        for (int i = startIndex; i < candidates.size(); i++) {
            sum += candidates[i];
            path.push_back(candidates[i]);
            backtracking(candidates, target, sum, i); // 不用i+1了，表示可以重复读取当前的数
            sum -= candidates[i];
            path.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        result.clear();
        path.clear();
        backtracking(candidates, target, 0, 0);
        return result;
    }
};
```

剪枝优化

在这个树形结构中：



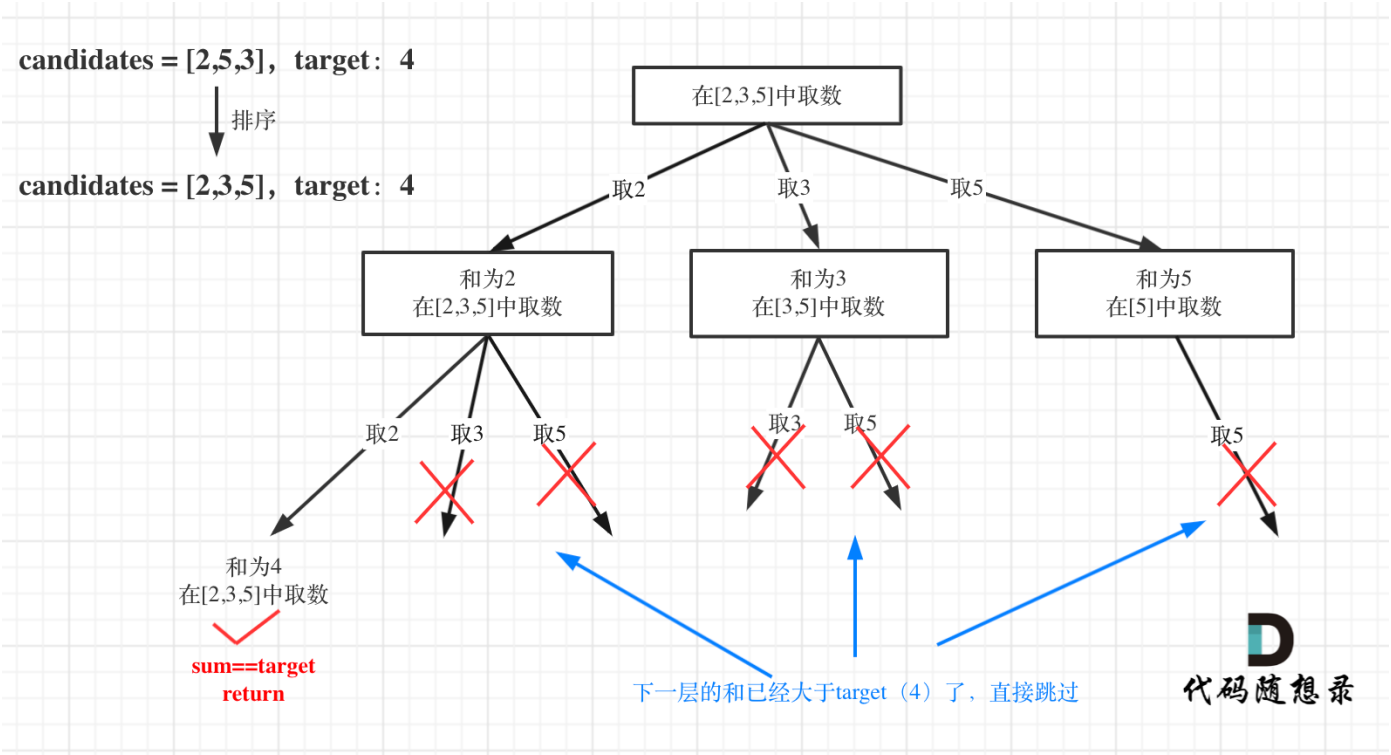
以及上面的版本一的代码大家可以看到，对于sum已经大于target的情况，其实是依然进入了下一层递归，只是下一层递归结束判断的时候，会判断sum > target的话就返回。

其实如果已经知道下一层的sum会大于target，就没有必要进入下一层递归了。

那么可以在for循环的搜索范围上做做文章了。

对总集合排序之后，如果下一层的sum（就是本层的 sum + candidates[i]）已经大于target，就可以结束本轮for循环的遍历。

如图：



for循环剪枝代码如下：

```
for (int i = startIndex; i < candidates.size() && sum + candidates[i] <= target; i++)
```

整体代码如下：（注意注释的部分）

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex) {
        if (sum == target) {
            result.push_back(path);
            return;
        }

        // 如果 sum + candidates[i] > target 就终止遍历
        for (int i = startIndex; i < candidates.size() && sum + candidates[i] <=
target; i++) {
            sum += candidates[i];
            path.push_back(candidates[i]);
            backtracking(candidates, target, sum, i);
            sum -= candidates[i];
            path.pop_back();

        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        result.clear();
        path.clear();
        sort(candidates.begin(), candidates.end()); // 需要排序
        backtracking(candidates, target, 0, 0);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)，注意这只是复杂度的上界，因为剪枝的存在，真实的时间复杂度远小于此
- 空间复杂度: O(target)

总结

本题和我们之前讲过的77.组合、216.组合总和III有两点不同：

- 组合没有数量要求
- 元素可无限重复选取

针对这两个问题，我都做了详细的分析。

并且给出了对于组合问题，什么时候用startIndex，什么时候不用，并用17.电话号码的字母组合做了对比。

最后还给出了本题的剪枝优化，这个优化如果是初学者的话并不容易想到。

在求和问题中，排序之后加剪枝是常见的套路！

可以看出我写的文章都会大量引用之前的文章，就是要不断作对比，分析其差异，然后给出代码解决的方法，这样才能彻底理解题目的本质与难点。

> 这篇可以说是全网把组合问题如何去重，讲的最清晰的了！

8.组合总和II

力扣题目链接

给定一个数组 candidates 和一个目标数 target ，找出 candidates 中所有可以使数字和为 target 的组合。

candidates 中的每个数字在每个组合中只能使用一次。

说明：
所有数字（包括目标数）都是正整数。解集不能包含重复的组合。

- 示例 1:
- 输入: candidates = [10,1,2,7,6,1,5], target = 8,
- 所求解集为:

```
[
  [1, 7],
  [1, 2, 5],
  [2, 6],
  [1, 1, 6]
]
```

- 示例 2:
- 输入: candidates = [2,5,2,1,2], target = 5,
- 所求解集为:

```
[
  [1,2,2],
  [5]
]
```

算法公开课

《代码随想录》算法视频公开课：回溯算法中的去重，树层去重树枝去重，你弄清楚了没？| LeetCode:40.组合总和II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目和39.组合总和如下区别：

1. 本题candidates 中的每个数字在每个组合中只能使用一次。
2. 本题数组candidates的元素是有重复的，而39.组合总和是无重复元素的数组candidates

最后本题和39.组合总和要求一样，解集不能包含重复的组合。

本题的难点在于区别2中：集合（数组candidates）有重复元素，但还不能有重复的组合。

一些同学可能想了：我把所有组合求出来，再用set或者map去重，这么做很容易超时！

所以要在搜索的过程中就去掉重复组合。

很多同学在去重的问题上想不明白，其实很多题解也没有讲清楚，反正代码是能过的，感觉是那么回事，稀里糊涂的先把题目过了。

这个去重为什么很难理解呢，**所谓去重，其实就是使用过的元素不能重复选取。** 这么一说好像很简单！

都知道组合问题可以抽象为树形结构，那么“使用过”在这个树形结构上是有两个维度的，一个维度是同一树枝上使用过，一个维度是同一树层上使用过。**没有理解这两个层面上的“使用过” 是造成大家没有彻底理解去重的根本原因。**

那么问题来了，我们是要同一树层上使用过，还是同一树枝上使用过呢?

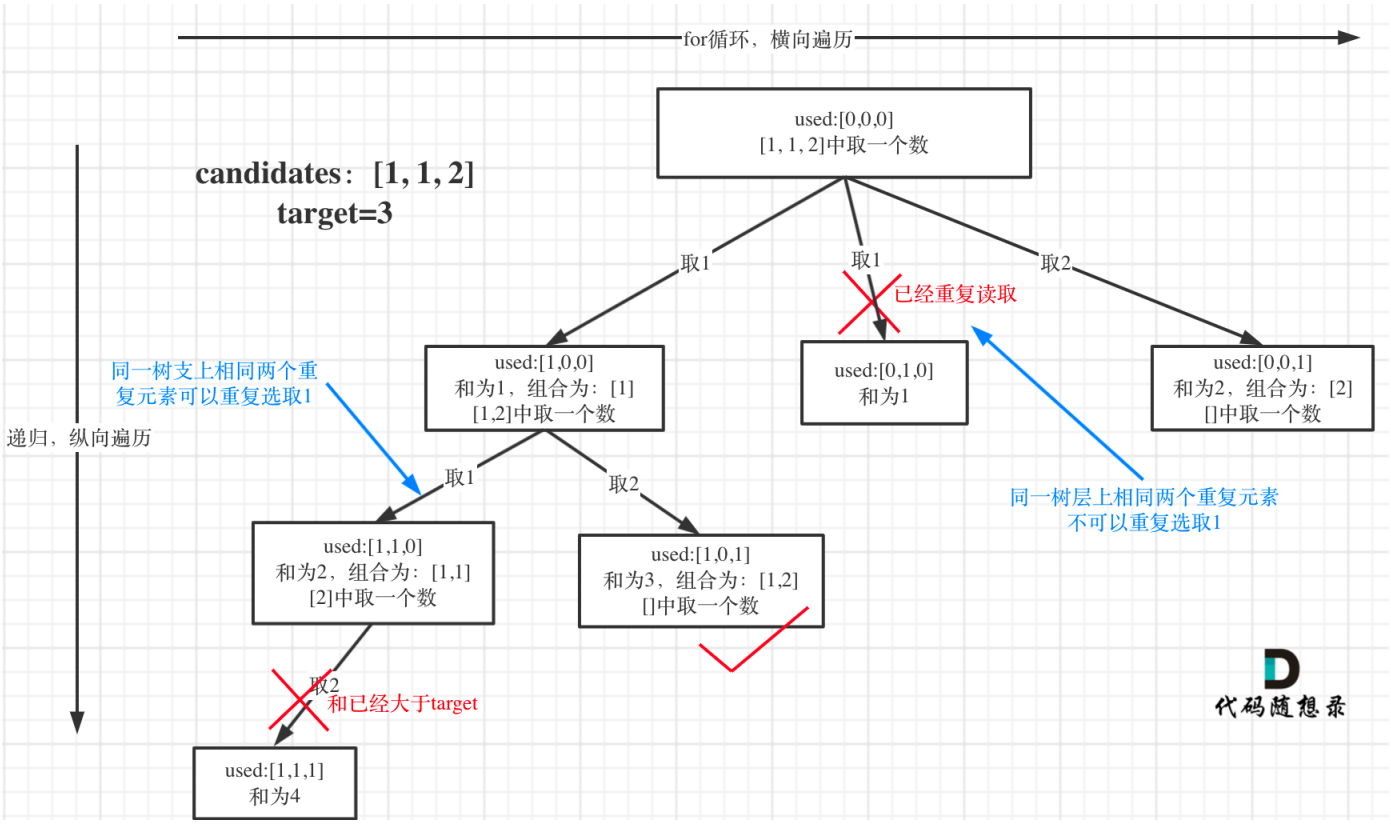
回看一下题目，元素在同一个组合内是可以重复的，怎么重复都没事，但两个组合不能相同。

所以我们要去重的是同一树层上的“使用过”，同一树枝上的都是一个组合里的元素，不用去重。

为了理解去重我们来举一个例子，candidates = [1, 1, 2], target = 3，（方便起见candidates已经排序了）

强调一下，树层去重的话，需要对数组排序！

选择过程树形结构如图所示：



可以看到图中，每个节点相对于 39.组合总和我多加了used数组，这个used数组下面会重点介绍。

回溯三部曲

- **递归函数参数**

与39.组合总和套路相同，此题还需要加一个bool型数组used，用来记录同一树枝上的元素是否使用过。

这个集合去重的重任就是used来完成的。

代码如下：

```
vector<vector<int>> result; // 存放组合集合
vector<int> path;           // 符合条件的组合
void backtracking(vector<int>& candidates, int target, int sum, int startIndex,
vector<bool>& used) {
```

- **递归终止条件**

与39.组合总和相同，终止条件为 `sum > target` 和 `sum == target`。

代码如下：

```
if (sum > target) { // 这个条件其实可以省略
    return;
}
if (sum == target) {
    result.push_back(path);
    return;
}
```

`sum > target` 这个条件其实可以省略，因为在递归单层遍历的时候，会有剪枝的操作，下面会介绍到。

- **单层搜索的逻辑**

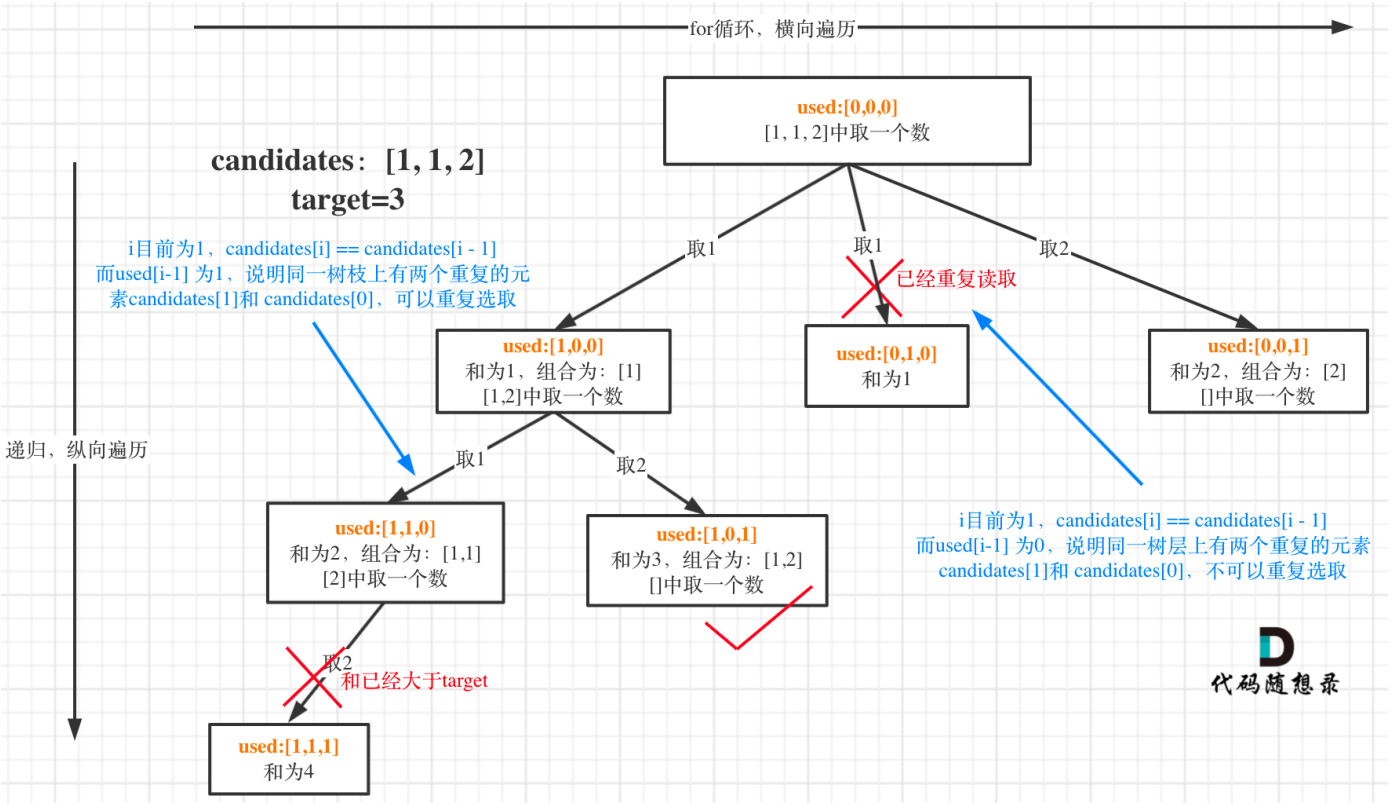
这里与39.组合总和最大的不同就是要去重了。

前面我们提到：要去重的是“同一树层上的使用过”，如何判断同一树层上元素（相同的元素）是否使用过了呢。

如果`candidates[i] == candidates[i - 1]`并且`used[i - 1] == false`，就说明：前一个树枝，使用了candidates[i - 1]，也就是说同一树层使用过candidates[i - 1]。

此时for循环里就应该做continue的操作。

这块比较抽象，如图：

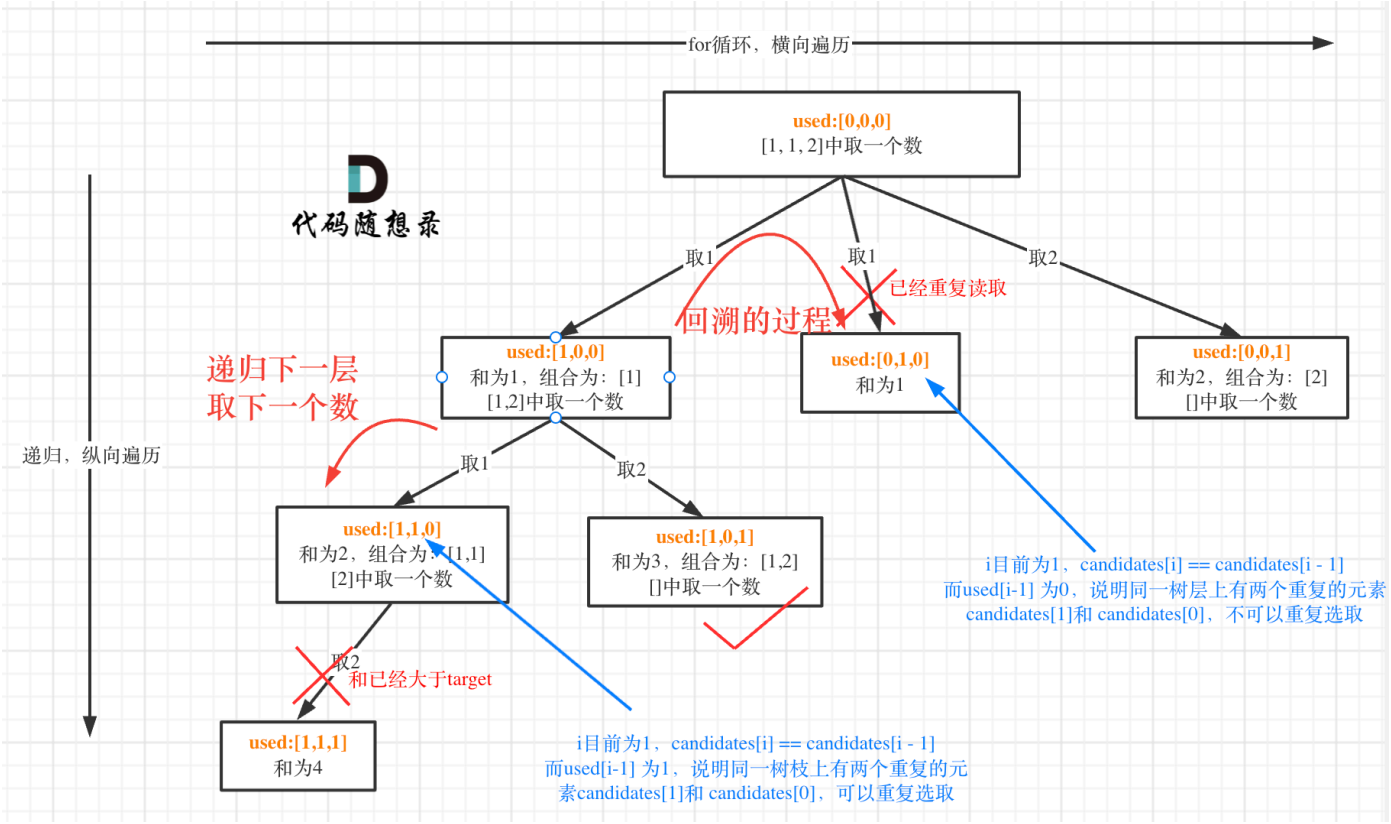


我在图中将used的变化用橘黄色标注上，可以看出在candidates[i] == candidates[i - 1]相同的情况下：

- used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
- used[i - 1] == false，说明同一树层candidates[i - 1]使用过

可能有的录友想，为什么 used[i - 1] == false 就是同一树层呢，因为同一树层，used[i - 1] == false 才能表示，当前取的 candidates[i] 是从 candidates[i - 1] 回溯而来的。

而 used[i - 1] == true，说明是进入下一层递归，去下一个数，所以是树枝上，如图所示：



这块去重的逻辑很抽象，网上搜的题解基本没有能讲清楚的，如果大家之前思考过这个问题或者刷过这道题目，看到这里一定会感觉通透了很多！

那么单层搜索的逻辑代码如下：

```
for (int i = startIndex; i < candidates.size() && sum + candidates[i] <= target; i++) {
    // used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
    // used[i - 1] == false，说明同一树层candidates[i - 1]使用过
    // 要对同一树层使用过的元素进行跳过
    if (i > 0 && candidates[i] == candidates[i - 1] && used[i - 1] == false) {
        continue;
    }
    sum += candidates[i];
    path.push_back(candidates[i]);
    used[i] = true;
    backtracking(candidates, target, sum, i + 1, used); // 和39.组合总和的区别1：这里是i+1，每个数字在每个组合中只能使用一次
    used[i] = false;
    sum -= candidates[i];
    path.pop_back();
}
```

注意sum + candidates[i] <= target为剪枝操作，在39.组合总和有讲解过！

回溯三部曲分析完了，整体C++代码如下：

```
class Solution {
private:
```

```
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex,
vector<bool>& used) {
        if (sum == target) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i < candidates.size() && sum + candidates[i] <=
target; i++) {
            // used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
            // used[i - 1] == false，说明同一树层candidates[i - 1]使用过
            // 要对同一树层使用过的元素进行跳过
            if (i > 0 && candidates[i] == candidates[i - 1] && used[i - 1] == false) {
                continue;
            }
            sum += candidates[i];
            path.push_back(candidates[i]);
            used[i] = true;
            backtracking(candidates, target, sum, i + 1, used); // 和39.组合总和的区别1，这
里是i+1，每个数字在每个组合中只能使用一次
            used[i] = false;
            sum -= candidates[i];
            path.pop_back();
        }
    }

public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        vector<bool> used(candidates.size(), false);
        path.clear();
        result.clear();
        // 首先把给candidates排序，让其相同的元素都挨在一起。
        sort(candidates.begin(), candidates.end());
        backtracking(candidates, target, 0, 0, used);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

补充

这里直接用startIndex来去重也是可以的， 就不用used数组了。

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex) {
        if (sum == target) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i < candidates.size() && sum + candidates[i] <=
target; i++) {
            // 要对同一树层使用过的元素进行跳过
            if (i > startIndex && candidates[i] == candidates[i - 1]) {
                continue;
            }
            sum += candidates[i];
            path.push_back(candidates[i]);
            backtracking(candidates, target, sum, i + 1); // 和39.组合总和的区别1，这里是
i+1，每个数字在每个组合中只能使用一次
            sum -= candidates[i];
            path.pop_back();
        }
    }

public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        path.clear();
        result.clear();
        // 首先把给candidates排序，让其相同的元素都挨在一起。
        sort(candidates.begin(), candidates.end());
        backtracking(candidates, target, 0, 0);
        return result;
    }
};
```

总结

本题同样是求组合总和，但就是因为其数组candidates有重复元素，而要求不能有重复的组合，所以相对于39.组合总和难度提升了不少。

关键是去重的逻辑，代码很简单，网上一搜一大把，但几乎没有能把这块代码含义讲明白的，基本都是给出代码，然后说这就是去重了，究竟怎么个去重法也是模棱两可。

所以Carl有必要把去重的这块彻彻底底的给大家讲清楚，**就连“树层去重”和“树枝去重”都是我自创的词汇，希望对大家理解有帮助！**

> 切割问题其实是一种组合问题！

9.分割回文串

[力扣题目链接]

给定一个字符串 s，将 s 分割成一些子串，使每个子串都是回文串。

返回 s 所有可能的分割方案。

示例:
输入: "aab"
输出:
[
  ["aa","b"],
  ["a","a","b"]
]

算法公开课

[《代码随想录》算法视频公开课]：[131.分割回文串]，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题这涉及到两个关键问题：

1. 切割问题，有不同的切割方式
2. 判断回文

相信这里不同的切割方式可以搞懵很多同学了。

这种题目，想用for循环暴力解法，可能都不那么容易写出来，所以要换一种暴力的方式，就是回溯。

一些同学可能想不清楚 回溯究竟是如何切割字符串呢？

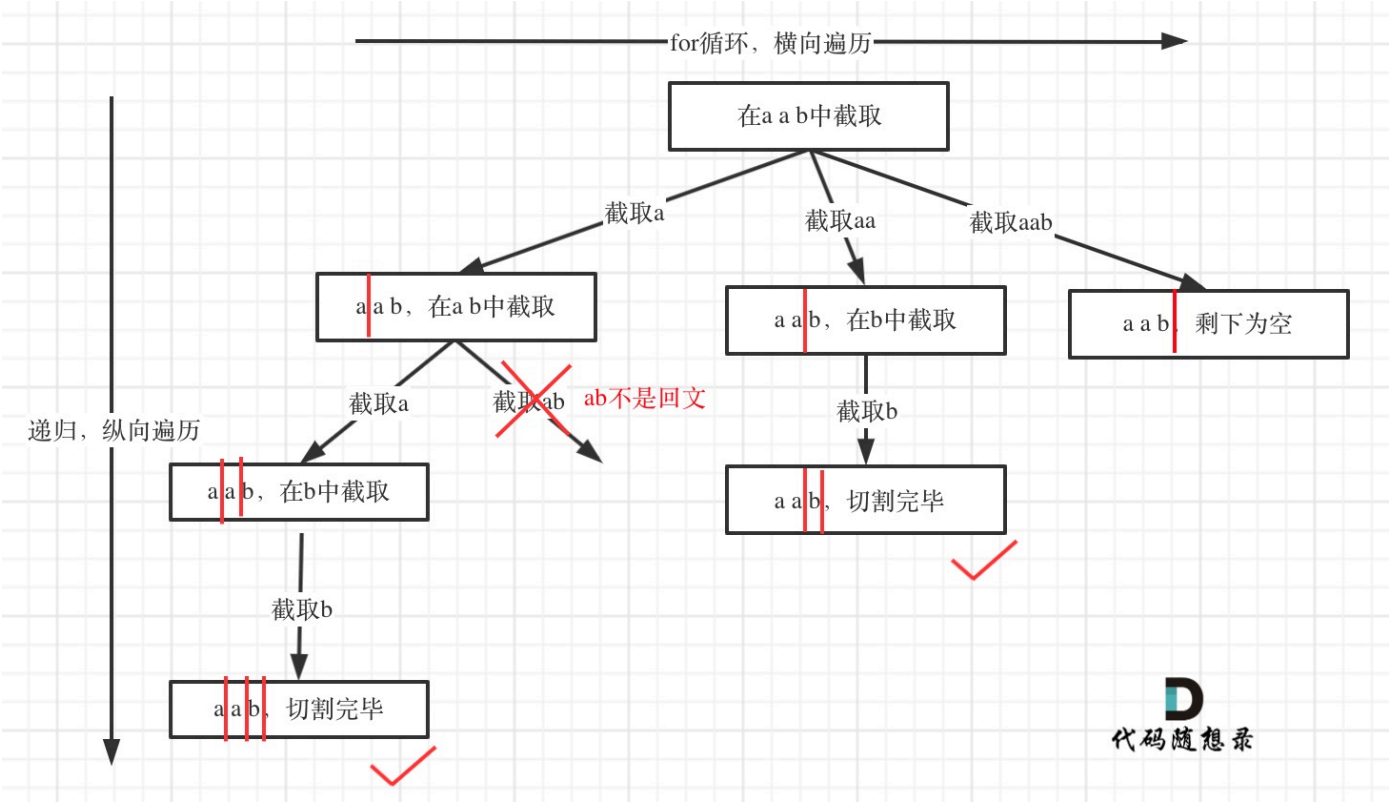
我们来分析一下切割，**其实切割问题类似组合问题**。

例如对于字符串abcdef：

- 组合问题：选取一个a之后，在bcdef中再去选取第二个，选取b之后在cdef中再选取第三个.....。
- 切割问题：切割一个a之后，在bcdef中再去切割第二段，切割b之后在cdef中再切割第三段.....。

感受出来了不？

所以切割问题，也可以抽象为一棵树形结构，如图：



递归用来纵向遍历，for循环用来横向遍历，切割线（就是图中的红线）切割到字符串的结尾位置，说明找到了一个切割方法。

此时可以发现，切割问题的回溯搜索的过程和组合问题的回溯搜索的过程是差不多的。

回溯三部曲

- 递归函数参数

全局变量数组path存放切割后回文的子串，二维数组result存放结果集。（这两个参数可以放到函数参数里）

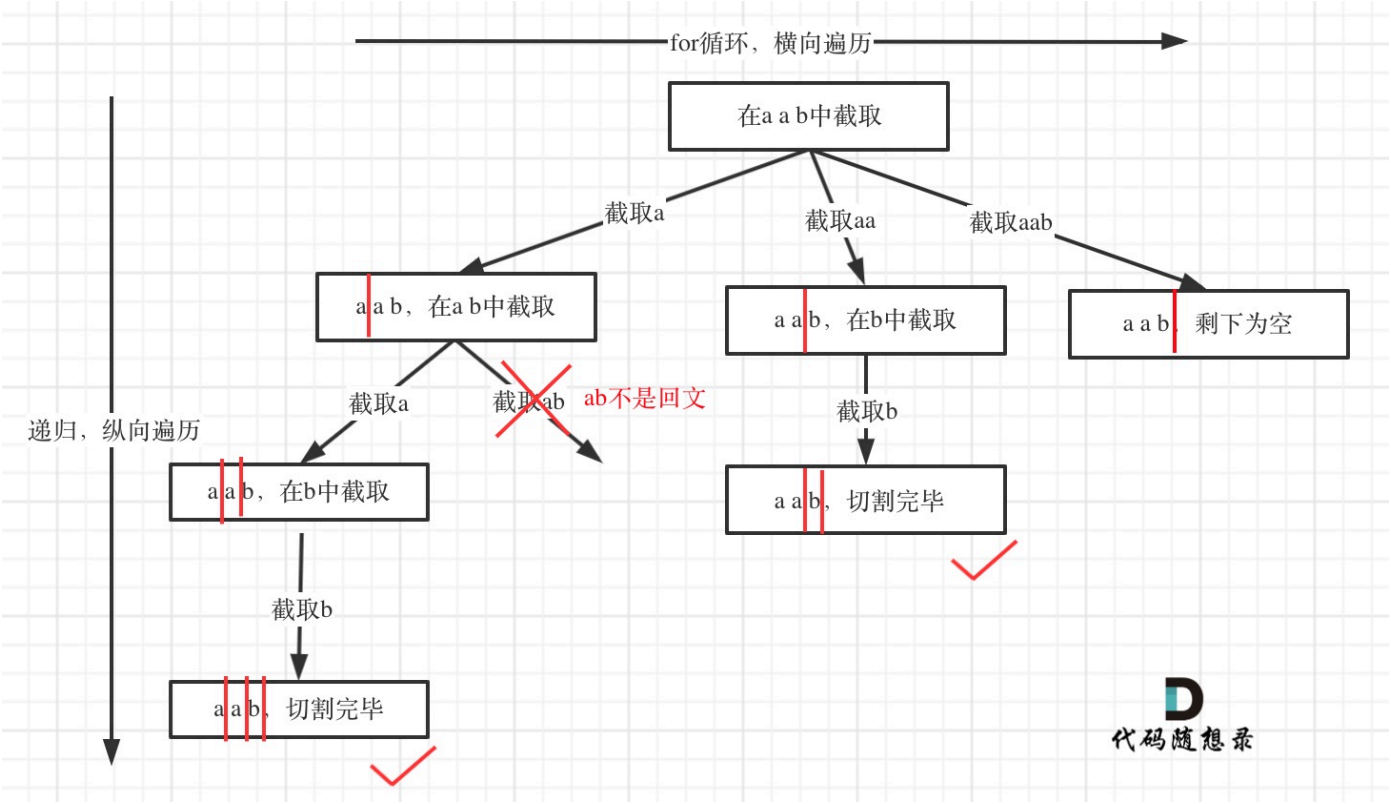
本题递归函数参数还需要startIndex，因为切割过的地方，不能重复切割，和组合问题也是保持一致的。

在回溯算法：求组合总和（二）中我们深入探讨了组合问题什么时候需要startIndex，什么时候不需要startIndex。

代码如下：

```
vector<vector<string>> result;
vector<string> path; // 放已经回文的子串
void backtracking (const string& s, int startIndex) {
```

- 递归函数终止条件



从树形结构的图中可以看出：切割线切到了字符串最后面，说明找到了一种切割方法，此时就是本层递归的终止条件。

那么在代码里什么是切割线呢?

在处理组合问题的时候，递归参数需要传入startIndex，表示下一轮递归遍历的起始位置，这个startIndex就是切割线。

所以终止条件代码如下：

```
void backtracking (const string& s, int startIndex) {
    // 如果起始位置已经大于s的大小，说明已经找到了一组分割方案了
    if (startIndex >= s.size()) {
        result.push_back(path);
        return;
    }
}
```

- 单层搜索的逻辑

来看看在递归循环中如何截取子串呢?

在`for (int i = startIndex; i < s.size(); i++)`循环中，我们 定义了起始位置startIndex，那么 [startIndex, i] 就是要截取的子串。

首先判断这个子串是不是回文，如果是回文，就加入在`vector<string> path`中，path用来记录切割过的回文子串。

代码如下：

```
for (int i = startIndex; i < s.size(); i++) {
    if (isPalindrome(s, startIndex, i)) { // 是回文子串
        // 获取[startIndex,i]在s中的子串
        string str = s.substr(startIndex, i - startIndex + 1);
        path.push_back(str);
    } else {                // 如果不是则直接跳过
        continue;
    }
    backtracking(s, i + 1); // 寻找i+1为起始位置的子串
    path.pop_back();        // 回溯过程，弹出本次已经添加的子串
}
```

注意切割过的位置，不能重复切割，所以，backtracking(s, i + 1); 传入下一层的起始位置为i + 1。

判断回文子串

最后我们看一下回文子串要如何判断了，判断一个字符串是否是回文。

可以使用双指针法，一个指针从前向后，一个指针从后向前，如果前后指针所指向的元素是相等的，就是回文字符串了。

那么判断回文的C++代码如下：

```
bool isPalindrome(const string& s, int start, int end) {
    for (int i = start, j = end; i < j; i++, j--) {
        if (s[i] != s[j]) {
            return false;
        }
    }
    return true;
}
```

如果大家对双指针法有生疏了，传送门：双指针法：总结篇！

此时关键代码已经讲解完毕，整体代码如下（详细注释了）

根据Carl给出的回溯算法模板：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

不难写出如下代码：

```
class Solution {
private:
    vector<vector<string>> result;
    vector<string> path; // 放已经回文的子串
    void backtracking (const string& s, int startIndex) {
        // 如果起始位置已经大于s的大小，说明已经找到了一组分割方案了
        if (startIndex >= s.size()) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i < s.size(); i++) {
            if (isPalindrome(s, startIndex, i)) {   // 是回文子串
                // 获取[startIndex,i]在s中的子串
                string str = s.substr(startIndex, i - startIndex + 1);
                path.push_back(str);
            } else {                                // 不是回文，跳过
                continue;
            }
            backtracking(s, i + 1); // 寻找i+1为起始位置的子串
            path.pop_back(); // 回溯过程，弹出本次已经添加的子串
        }
    }
    bool isPalindrome(const string& s, int start, int end) {
        for (int i = start, j = end; i < j; i++, j--) {
            if (s[i] != s[j]) {
                return false;
            }
        }
        return true;
    }
public:
    vector<vector<string>> partition(string s) {
```

```
        result.clear();
        path.clear();
        backtracking(s, 0);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n^2)

优化

上面的代码还存在一定的优化空间, 在于如何更高效的计算一个子字符串是否是回文字串。上述代码`isPalindrome`函数运用双指针的方法来判定对于一个字符串`s`, 给定起始下标和终止下标, 截取出的子字符串是否是回文字串。但是其中有一定的重复计算存在:

例如给定字符串`"abcde"`, 在已知`"bcd"`不是回文字串时, 不再需要去双指针操作`"abcde"`而可以直接判定它一定不是回文字串。

具体来说, 给定一个字符串`s`, 长度为`n`, 它成为回文字串的充分必要条件是`s[0] == s[n-1]`且`s[1:n-1]`是回文字串。

大家如果熟悉动态规划这种算法的话, 我们可以高效地事先一次性计算出, 针对一个字符串`s`, 它的任何子串是否是回文字串, 然后在我们的回溯函数中直接查询即可, 省去了双指针移动判定这一步骤.

具体参考代码如下:

```
class Solution {
private:
    vector<vector<string>> result;
    vector<string> path; // 放已经回文的子串
    vector<vector<bool>> isPalindrome; // 放事先计算好的是否回文子串的结果
    void backtracking (const string& s, int startIndex) {
        // 如果起始位置已经大于s的大小，说明已经找到了一组分割方案了
        if (startIndex >= s.size()) {
            result.push_back(path);
            return;
        }
        for (int i = startIndex; i < s.size(); i++) {
            if (isPalindrome[startIndex][i]) {   // 是回文子串
                // 获取[startIndex,i]在s中的子串
                string str = s.substr(startIndex, i - startIndex + 1);
                path.push_back(str);
            } else {                                    // 不是回文，跳过
                continue;
            }
            backtracking(s, i + 1); // 寻找i+1为起始位置的子串
            path.pop_back(); // 回溯过程，弹出本次已经添加的子串
        }
    }
```

```
    void computePalindrome(const string& s) {
        // isPalindrome[i][j] 代表 s[i:j](双边包括)是否是回文字串
        isPalindrome.resize(s.size(), vector<bool>(s.size(), false)); // 根据字符串s, 刷新布尔矩阵的大小
        for (int i = s.size() - 1; i >= 0; i--) {
            // 需要倒序计算, 保证在i行时, i+1行已经计算好了
            for (int j = i; j < s.size(); j++) {
                if (j == i) {isPalindrome[i][j] = true;}
                else if (j - i == 1) {isPalindrome[i][j] = (s[i] == s[j]);}
                else {isPalindrome[i][j] = (s[i] == s[j] && isPalindrome[i+1][j-1]);}
            }
        }
public:
    vector<vector<string>> partition(string s) {
        result.clear();
        path.clear();
        computePalindrome(s);
        backtracking(s, 0);
        return result;
    }
};
```

总结

这道题目在leetcode上是中等，但可以说是hard的题目了，但是代码其实就是按照模板的样子来的。

那么难究竟难在什么地方呢?

我列出如下几个难点：

- 切割问题可以抽象为组合问题
- 如何模拟那些切割线
- 切割问题中递归如何终止
- 在递归循环中如何截取子串
- 如何判断回文

我们平时在做难题的时候，总结出来难究竟难在哪里也是一种需要锻炼的能力。

一些同学可能遇到题目比较难，但是不知道题目难在哪里，反正就是很难。其实这样还是思维不够清晰，这种总结的能力需要多接触多锻炼。

本题我相信很多同学主要卡在了第一个难点上：就是不知道如何切割，甚至知道要用回溯法，也不知道如何用。也就是没有体会到按照求组合问题的套路就可以解决切割。

如果意识到这一点，算是重大突破了。接下来就可以对着模板照葫芦画瓢。

但接下来如何模拟切割线，如何终止，如何截取子串，其实都不好想，最后判断回文算是最简单的了。

关于模拟切割线，其实就是index是上一层已经确定了的分割线，i是这一层试图寻找的新分割线

除了这些难点，**本题还有细节，例如：切割过的地方不能重复切割所以递归函数需要传入i + 1**。

所以本题应该是一道hard题目了。

可能刷过这道题目的录友都没感受到自己原来克服了这么多难点，就把这道题目AC了，这应该叫做无招胜有招，人码合一，哈哈哈。

10.复原IP地址

[力扣题目链接]

给定一个只包含数字的字符串，复原它并返回所有可能的 IP 地址格式。

有效的 IP 地址 正好由四个整数（每个整数位于 0 到 255 之间组成，且不能含有前导 0），整数之间用 '.' 分隔。

例如："0.1.2.201" 和 "192.168.1.1" 是 有效的 IP 地址，但是 "0.011.255.245"、"192.168.1.312" 和 "192.168@1.1" 是 无效的 IP 地址。

示例 1：

- 输入：s = "25525511135"
- 输出：["255.255.11.135","255.255.111.35"]

示例 2：

- 输入：s = "0000"
- 输出：["0.0.0.0"]

示例 3：

- 输入：s = "1111"
- 输出：["1.1.1.1"]

示例 4：

- 输入：s = "010010"
- 输出：["0.10.0.10","0.100.1.0"]

示例 5：

- 输入：s = "101023"
- 输出：["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]

提示：

- 0 <= s.length <= 3000
- s 仅由数字组成

算法公开课

[《代码随想录》算法视频公开课](#)：[回溯算法如何分割字符串并判断是合法IP？ | LeetCode：93.复原IP地址](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

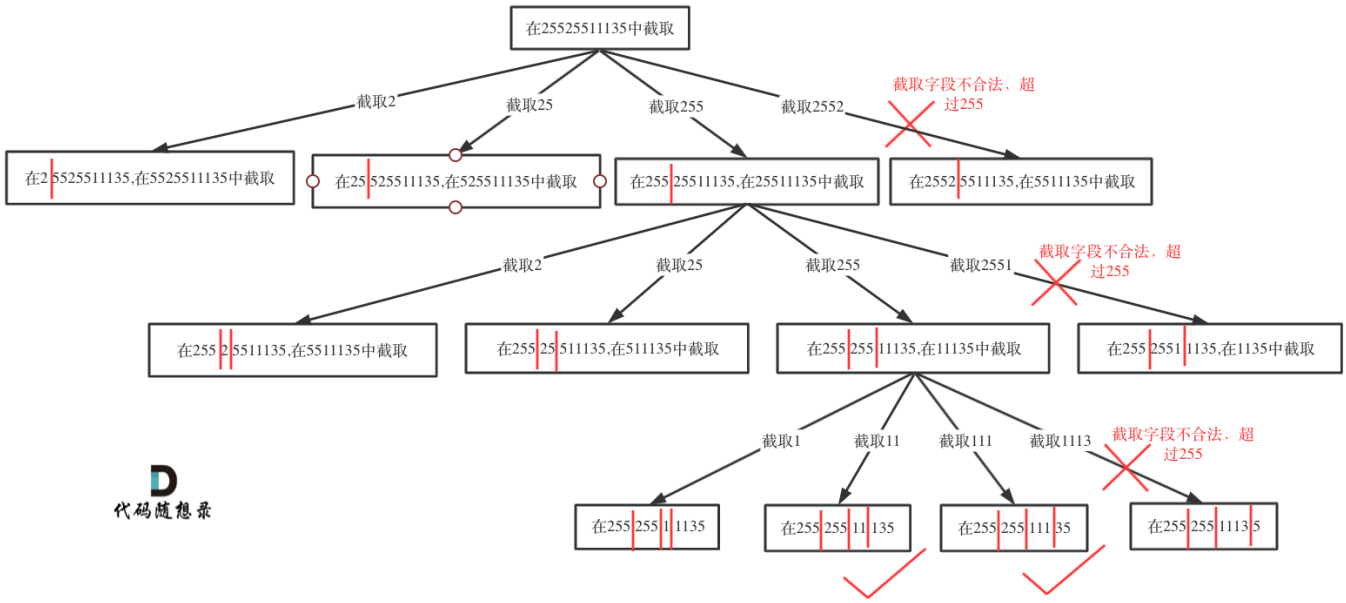
思路

做这道题目之前，最好先把131.分割回文串这个做了。

这道题目相信大家刚看的时候，应该会一脸茫然。

其实只要意识到这是切割问题，**切割问题就可以使用回溯搜索法把所有可能性搜出来**，和刚做过的131.分割回文串就十分类似了。

切割问题可以抽象为树型结构，如图：



回溯三部曲

- 递归参数

在131.分割回文串中我们就提到切割问题类似组合问题。

startIndex一定是需要的，因为不能重复分割，记录下一层递归分割的起始位置。

本题我们还需要一个变量pointNum，记录添加逗点的数量。

所以代码如下：

```
vector<string> result;// 记录结果
// startIndex: 搜索的起始位置，pointNum:添加逗点的数量
void backtracking(string& s, int startIndex, int pointNum) {
```

- 递归终止条件

终止条件和131.分割回文串情况就不同了，本题明确要求只会分成4段，所以不能用切割线切到最后作为终止条件，而是分割的段数作为终止条件。

pointNum表示逗点数量，pointNum为3说明字符串分成了4段了。

然后验证一下第四段是否合法，如果合法就加入到结果集里

代码如下：

```
if (pointNum == 3) { // 逗点数量为3时，分隔结束
    // 判断第四段子字符串是否合法，如果合法就放进result中
    if (isValid(s, startIndex, s.size() - 1)) {
        result.push_back(s);
    }
    return;
}
```

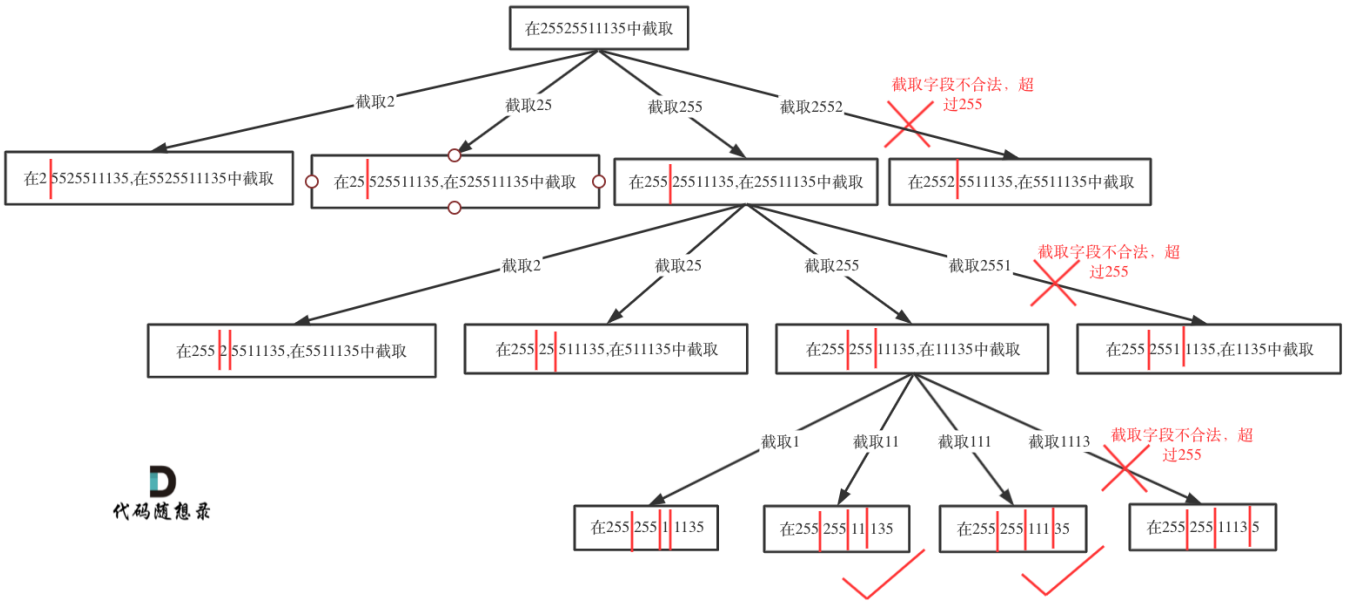
- 单层搜索的逻辑

在131.分割回文串中已经讲过在循环遍历中如何截取子串。

在`for (int i = startIndex; i < s.size(); i++)`循环中 [startIndex, i] 这个区间就是截取的子串，需要判断这个子串是否合法。

如果合法就在字符串后面加上符号`.`表示已经分割。

如果不合法就结束本层循环，如图中剪掉的分支：



然后就是递归和回溯的过程：

递归调用时，下一层递归的startIndex要从i+2开始（因为需要在字符串中加入了分隔符`.`），同时记录分割符的数量pointNum 要 +1。

回溯的时候，就将刚刚加入的分隔符`.`删掉就可以了，pointNum也要-1。

代码如下：

```
for (int i = startIndex; i < s.size(); i++) {
    if (isValid(s, startIndex, i)) { // 判断 [startIndex,i] 这个区间的子串是否合法
        s.insert(s.begin() + i + 1 , '.');  // 在i的后面插入一个逗点
        pointNum++;
        backtracking(s, i + 2, pointNum);    // 插入逗点之后下一个子串的起始位置为i+2
        pointNum--;                          // 回溯
        s.erase(s.begin() + i + 1);          // 回溯删掉逗点
    } else break; // 不合法，直接结束本层循环
}
```

判断子串是否合法

最后就是在写一个判断段位是否是有效段位了。

主要考虑到如下三点：

- 段位以0为开头的数字不合法
- 段位里有非正整数字符不合法
- 段位如果大于255了不合法

代码如下：

```
// 判断字符串s在左闭又闭区间[start, end]所组成的数字是否合法
bool isValid(const string& s, int start, int end) {
    if (start > end) {
        return false;
    }
    if (s[start] == '0' && start != end) { // 0开头的数字不合法
            return false;
    }
    int num = 0;
    for (int i = start; i <= end; i++) {
        if (s[i] > '9' || s[i] < '0') { // 遇到非数字字符不合法
            return false;
        }
        num = num * 10 + (s[i] - '0');
        if (num > 255) { // 如果大于255了不合法
            return false;
        }
    }
    return true;
}
```

根据关于回溯算法，你该了解这些！给出的回溯算法模板：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

可以写出如下回溯算法C++代码：

```
class Solution {
private:
    vector<string> result;// 记录结果
    // startIndex: 搜索的起始位置，pointNum:添加逗点的数量
    void backtracking(string& s, int startIndex, int pointNum) {
        if (pointNum == 3) { // 逗点数量为3时，分隔结束
            // 判断第四段子字符串是否合法，如果合法就放进result中
            if (isValid(s, startIndex, s.size() - 1)) {
                result.push_back(s);
            }
            return;
        }
        for (int i = startIndex; i < s.size(); i++) {
            if (isValid(s, startIndex, i)) { // 判断 [startIndex,i] 这个区间的子串是否合法
                s.insert(s.begin() + i + 1 , '.');  // 在i的后面插入一个逗点
                pointNum++;
                backtracking(s, i + 2, pointNum);   // 插入逗点之后下一个子串的起始位置为i+2
                pointNum--;                         // 回溯
                s.erase(s.begin() + i + 1);         // 回溯删掉逗点
            } else break; // 不合法，直接结束本层循环
        }
    }
    // 判断字符串s在左闭又闭区间[start, end]所组成的数字是否合法
    bool isValid(const string& s, int start, int end) {
        if (start > end) {
            return false;
        }
        if (s[start] == '0' && start != end) { // 0开头的数字不合法
                return false;
        }
        int num = 0;
        for (int i = start; i <= end; i++) {
            if (s[i] > '9' || s[i] < '0') { // 遇到非数字字符不合法
```

```
                return false;
            }
            num = num * 10 + (s[i] - '0');
            if (num > 255) { // 如果大于255了不合法
                return false;
            }
        }
        return true;
    }
public:
    vector<string> restoreIpAddresses(string s) {
        result.clear();
        if (s.size() < 4 || s.size() > 12) return result; // 算是剪枝了
        backtracking(s, 0, 0);
        return result;
    }
};
```

- 时间复杂度: O(3^4)，IP地址最多包含4个数字，每个数字最多有3种可能的分割方式，则搜索树的最大深度为4，每个节点最多有3个子节点。
- 空间复杂度: O(n)

总结

在131.分割回文串中我列举的分割字符串的难点，本题都覆盖了。

而且本题还需要操作字符串添加逗号作为分隔符，并验证区间的合法性。

可以说是131.分割回文串的加强版。

在本文的树形结构图中，我已经把详细的分析思路都画了出来，相信大家看了之后一定会思路清晰不少！

11.子集

力扣题目链接

给定一组不含重复元素的整数数组 nums，返回该数组所有可能的子集（幂集）。

说明：解集不能包含重复的子集。

示例:
输入: nums = [1,2,3]
输出:
[
  [3],

```
  [1],
  [2],
  [1,2,3],
  [1,3],
  [2,3],
  [1,2],
  []
]
```

算法公开课

[《代码随想录》算法视频公开课](#)：[回溯算法解决子集问题，树上节点都是目标集和！ | LeetCode：78.子集](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

求子集问题和[77.组合](#)和[131.分割回文串](#)又不一样了。

如果把 子集问题、组合问题、分割问题都抽象为一棵树的话，**那么组合问题和分割问题都是收集树的叶子节点，而子集问题是找树的所有节点！**

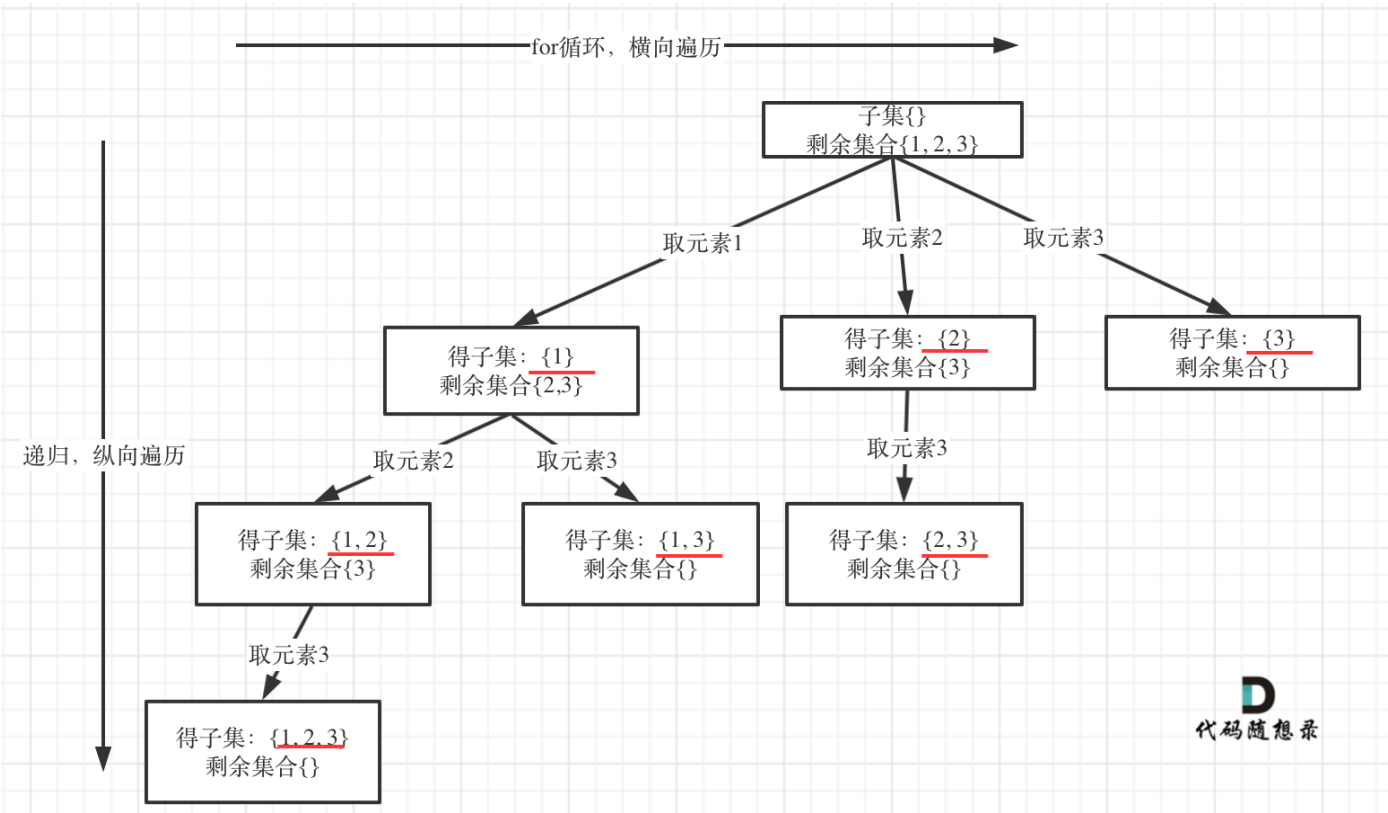
其实子集也是一种组合问题，因为它的集合是无序的，子集{1,2} 和 子集{2,1}是一样的。

那么既然是无序，取过的元素不会重复取，写回溯算法的时候，for就要从startIndex开始，而不是从0开始！

有同学问了，什么时候for可以从0开始呢?

求排列问题的时候，就要从0开始，因为集合是有序的，{1, 2} 和{2, 1}是两个集合，排列问题我们后续的文章就会讲到的。

以示例中nums = [1,2,3]为例把求子集抽象为树型结构，如下：



从图中红线部分，可以看出**遍历这个树的时候，把所有节点都记录下来，就是要求的子集集合。**

回溯三部曲

- 递归函数参数

全局变量数组path为子集收集元素，二维数组result存放子集组合。（也可以放到递归函数参数里）

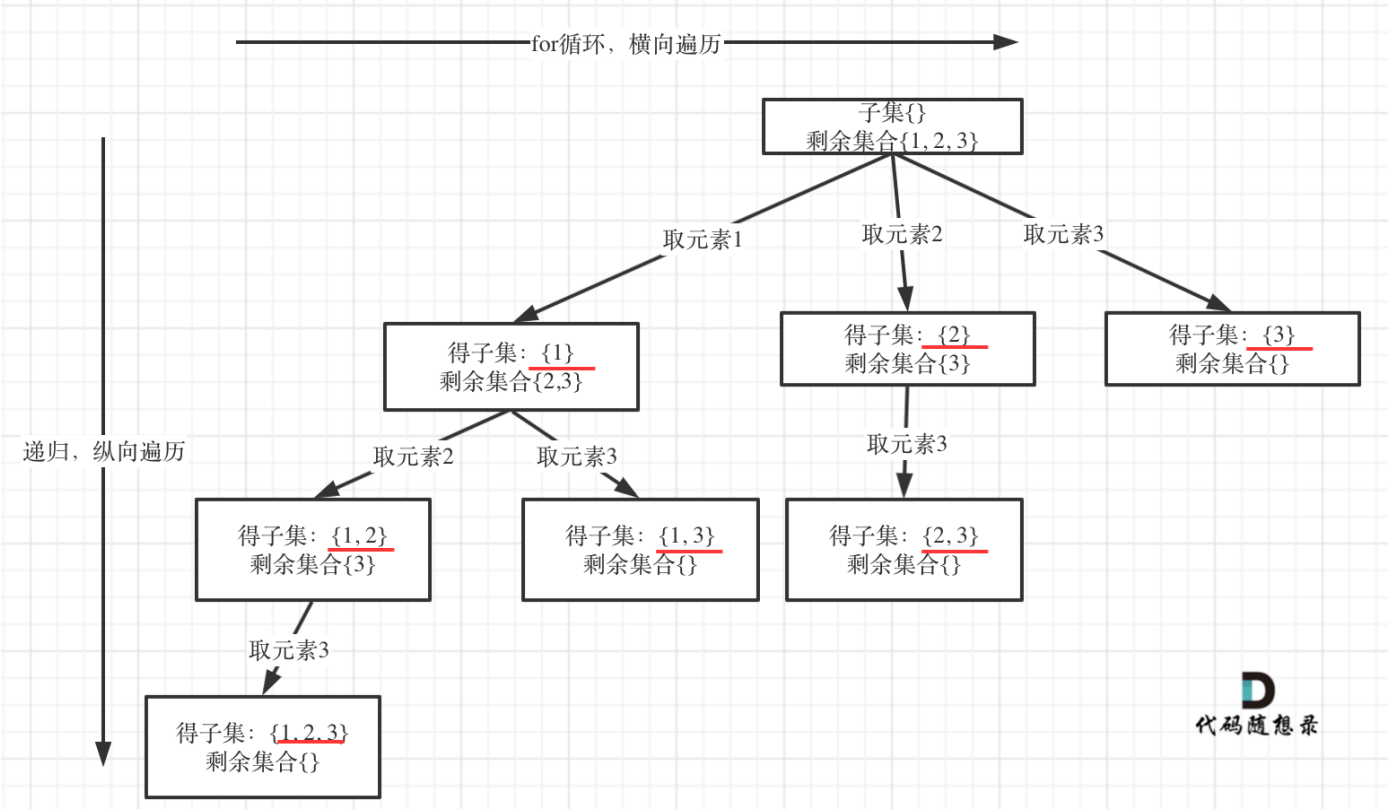
递归函数参数在上面讲到了，需要startIndex。

代码如下：

```
vector<vector<int>> result;
vector<int> path;
void backtracking(vector<int>& nums, int startIndex) {
```

递归终止条件

从图中可以看出：



剩余集合为空的时候，就是叶子节点。

那么什么时候剩余集合为空呢?

就是startIndex已经大于数组的长度了，就终止了，因为没有元素可取了，代码如下:

```
if (startIndex >= nums.size()) {
    return;
}
```

其实可以不需要加终止条件，因为startIndex >= nums.size()，本层for循环本来也结束了。

- 单层搜索逻辑

求取子集问题，不需要任何剪枝！因为子集就是要遍历整棵树。

那么单层递归逻辑代码如下：

```
for (int i = startIndex; i < nums.size(); i++) {
    path.push_back(nums[i]);    // 子集收集元素
    backtracking(nums, i + 1);  // 注意从i+1开始，元素不重复取
    path.pop_back();            // 回溯
}
```

根据关于回溯算法，你该了解这些！给出的回溯算法模板：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

可以写出如下回溯算法C++代码：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        result.push_back(path); // 收集子集，要放在终止添加的上面，否则会漏掉自己
        if (startIndex >= nums.size()) { // 终止条件可以不加
            return;
        }
        for (int i = startIndex; i < nums.size(); i++) {
            path.push_back(nums[i]);
            backtracking(nums, i + 1);
            path.pop_back();
        }
    }
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        result.clear();
        path.clear();
        backtracking(nums, 0);
```

```
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

在注释中，可以发现可以不写终止条件，因为本来我们就要遍历整棵树。

有的同学可能担心不写终止条件会不会无限递归?

并不会，因为每次递归的下一层就是从i+1开始的。

总结

相信大家经过了

- 组合问题：
  - 77.组合
  - 回溯算法：组合问题再剪剪枝
  - 216.组合总和III
  - 17.电话号码的字母组合
  - 39.组合总和
  - 40.组合总和II
- 分割问题：
  - 131.分割回文串
  - 93.复原IP地址

洗礼之后，发现子集问题还真的有点简单了，其实这就是一道标准的模板题。

但是要清楚子集问题和组合问题、分割问题的的区别，**子集是收集树形结构中树的所有节点的结果。**

而组合问题、分割问题是收集树形结构中叶子节点的结果。

---

12. 本周小结！（回溯算法系列二）

> 例行每周小结

周一

在回溯算法：求组合总和（二）中讲解的组合总和问题，和以前的组合问题还都不一样。

本题和回溯算法：求组合问题！，回溯算法：求组合总和！和区别是：本题没有数量要求，可以无限重复，但是有总和的限制，所以间接的也是有个数的限制。

不少录友都是看到可以重复选择，就义无反顾的把startIndex去掉了。

本题还需要startIndex来控制for循环的起始位置，对于组合问题，什么时候需要startIndex呢？

我举过例子，如果是一个集合来求组合的话，就需要startIndex，例如：回溯算法：求组合问题！，回溯算法：求组合总和！。

如果是多个集合取组合，各个集合之间相互不影响，那么就不用startIndex，例如：回溯算法：电话号码的字母组合

注意以上我只是说求组合的情况，如果是排列问题，又是另一套分析的套路，后面我再讲解排列的时候就重点介绍。

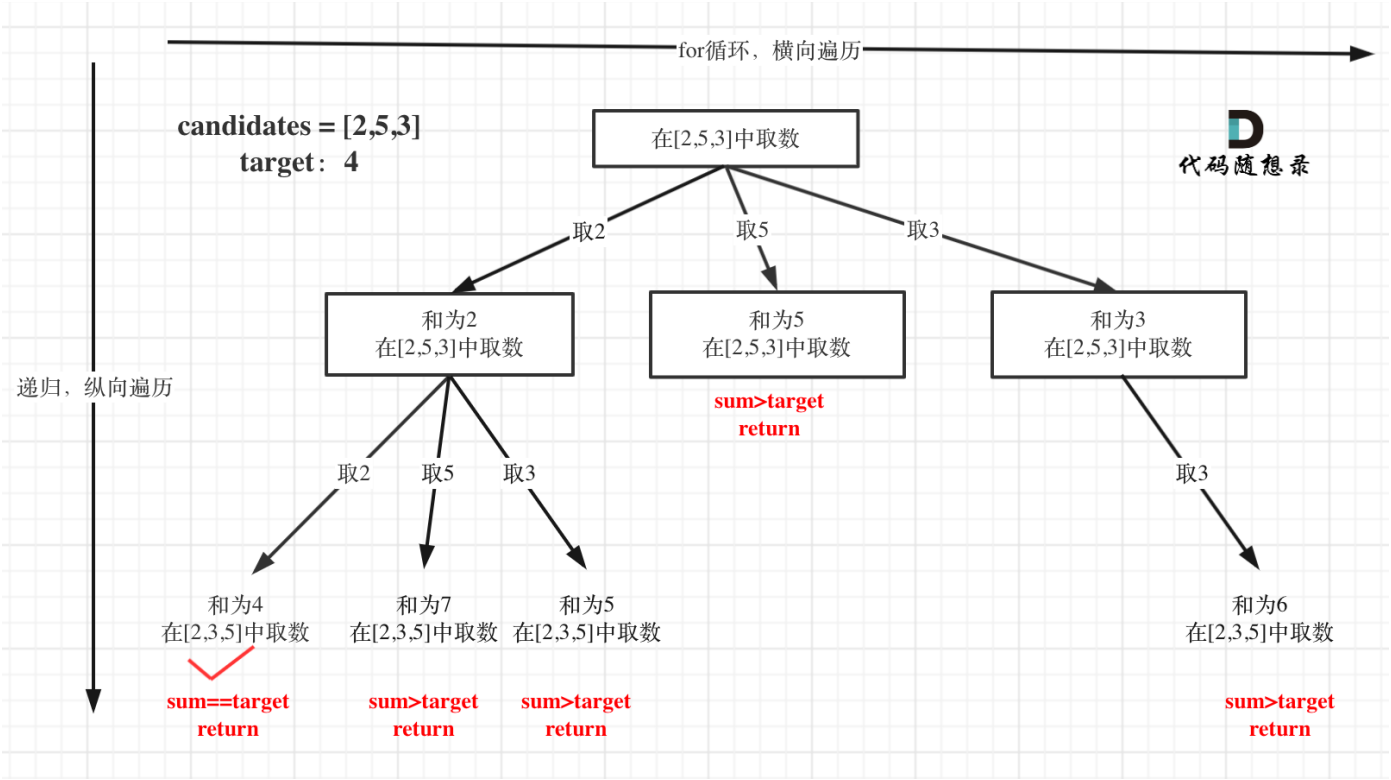
最后还给出了本题的剪枝优化，如下：

```
for (int i = startIndex; i < candidates.size() && sum + candidates[i] <= target; i++)
```

这个优化如果是初学者的话并不容易想到。

在求和问题中，排序之后加剪枝是常见的套路！

在回溯算法：求组合总和（二）第一个树形结构没有画出startIndex的作用，**这里这里纠正一下，准确的树形结构如图所示：**



周二

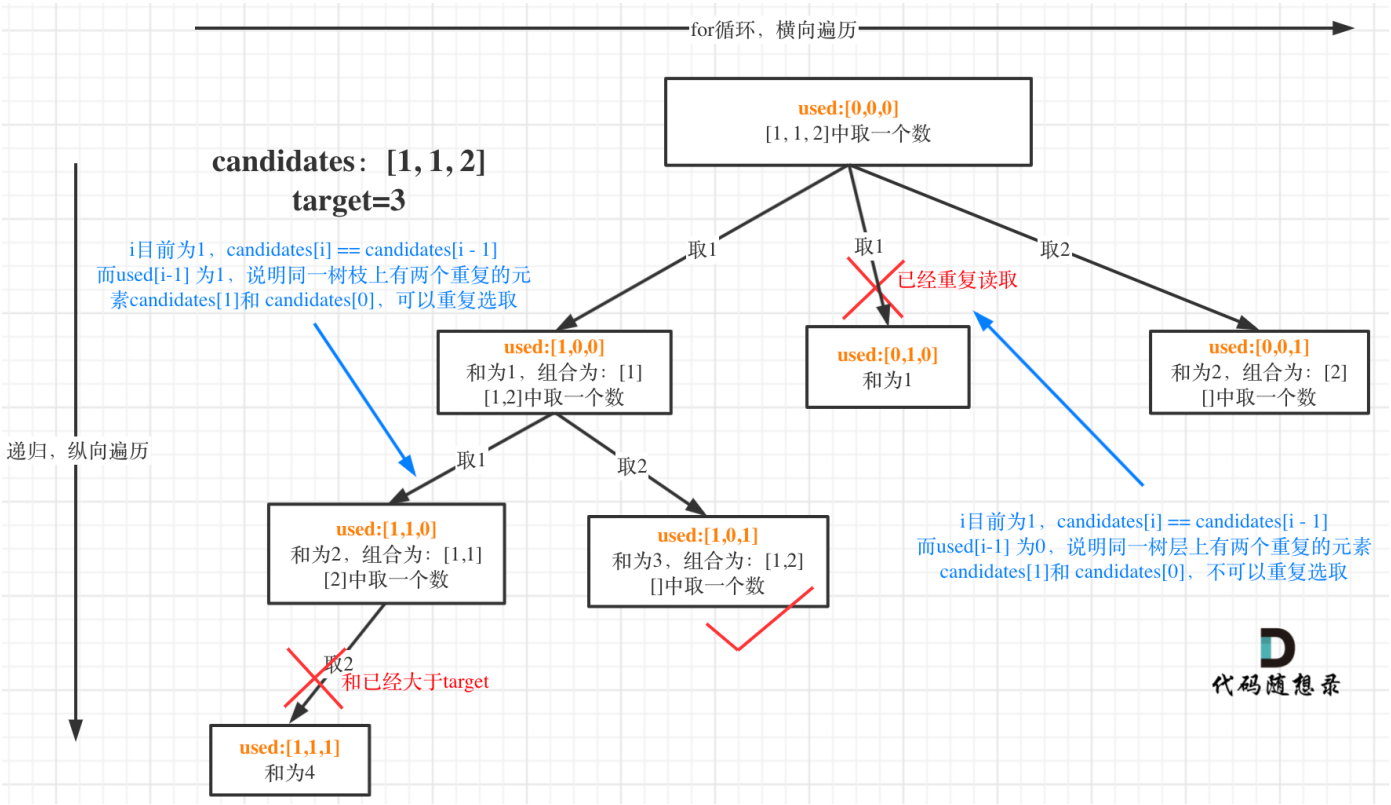
在回溯算法：求组合总和（三）中依旧讲解组合总和问题，本题集合元素会有重复，但要求解集不能包含重复的组合。

所以难就难在去重问题上了。

这个去重问题，相信做过的录友都知道有多么的晦涩难懂。网上的题解一般就说“去掉重复”，但说不清怎么个去重，代码一甩就完事了。

为了讲解这个去重问题，**我自创了两个词汇，“树枝去重”和“树层去重”。**

都知道组合问题可以抽象为树形结构，那么“使用过”在这个树形结构上是有两个维度的，一个维度是同一树枝上“使用过”，一个维度是同一树层上“使用过”。**没有理解这两个层面上的“使用过” 是造成大家没有彻底理解去重的根本原因。**



我在图中将used的变化用橘黄色标注上，可以看出在candidates[i] == candidates[i - 1]相同的情况下：

- used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
- used[i - 1] == false，说明同一树层candidates[i - 1]使用过

这块去重的逻辑很抽象，网上搜的题解基本没有能讲清楚的，如果大家之前思考过这个问题或者刷过这道题目，看到这里一定会感觉通透了很多！

对于去重，其实排列问题也是一样的道理，后面我会讲到。

周三

在回溯算法：分割回文串中，我们开始讲解切割问题，虽然最后代码看起来好像是一道模板题，但是从分析到学会套用这个模板，是比较难的。

我列出如下几个难点：

- 切割问题其实类似组合问题
- 如何模拟那些切割线
- 切割问题中递归如何终止
- 在递归循环中如何截取子串

- 如何判断回文

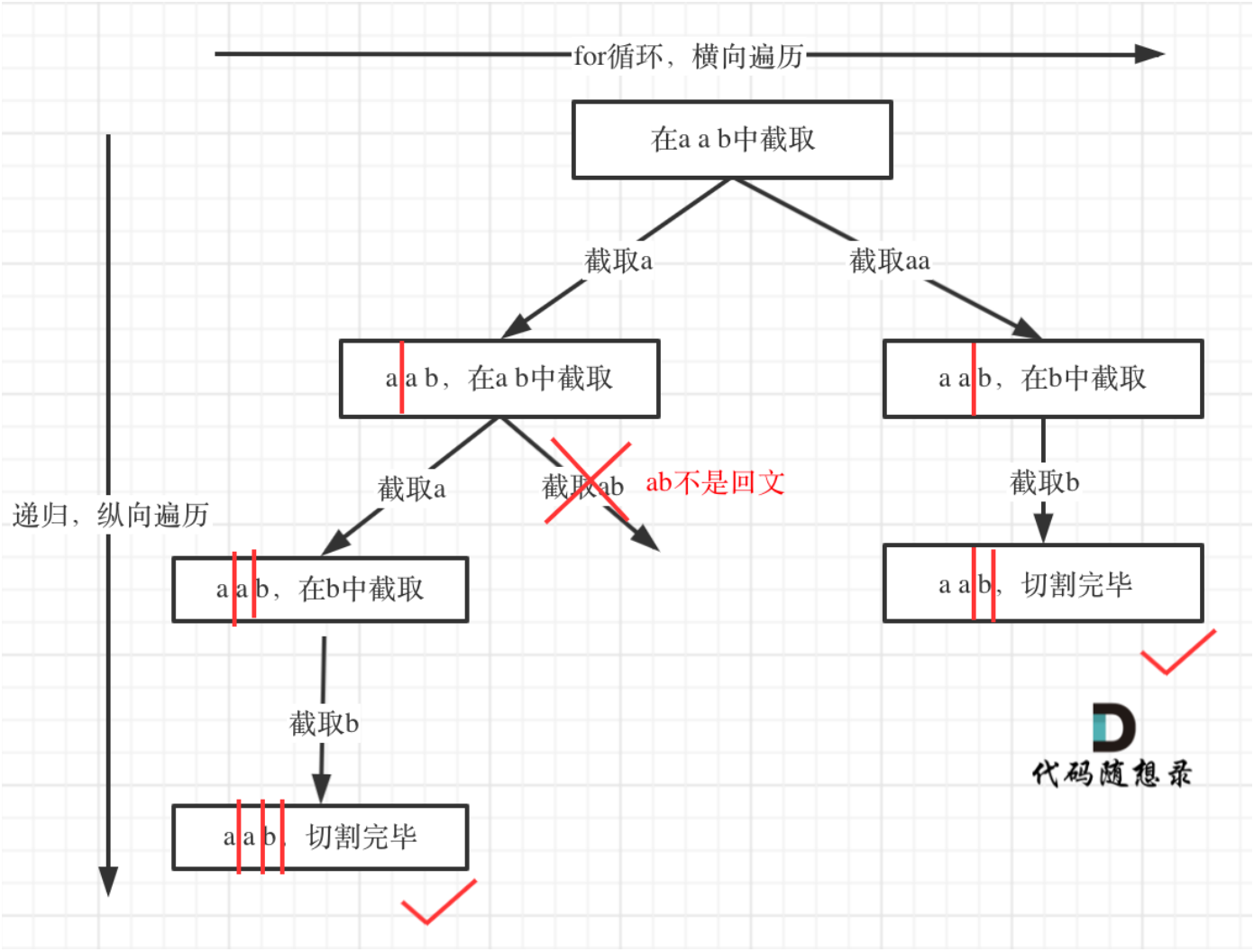
如果想到了**用求解组合问题的思路来解决 切割问题本题就成功一大半了**，接下来就可以对着模板照葫芦画瓢。

但后序如何模拟切割线，如何终止，如何截取子串，其实都不好想，最后判断回文算是最简单的了。

除了这些难点，**本题还有细节，例如：切割过的地方不能重复切割所以递归函数需要传入i + 1。**

所以本题应该是一个道hard题目了。

本题的树形结构中，和代码的逻辑有一个小出入，已经判断不是回文的子串就不会进入递归了，纠正如下：

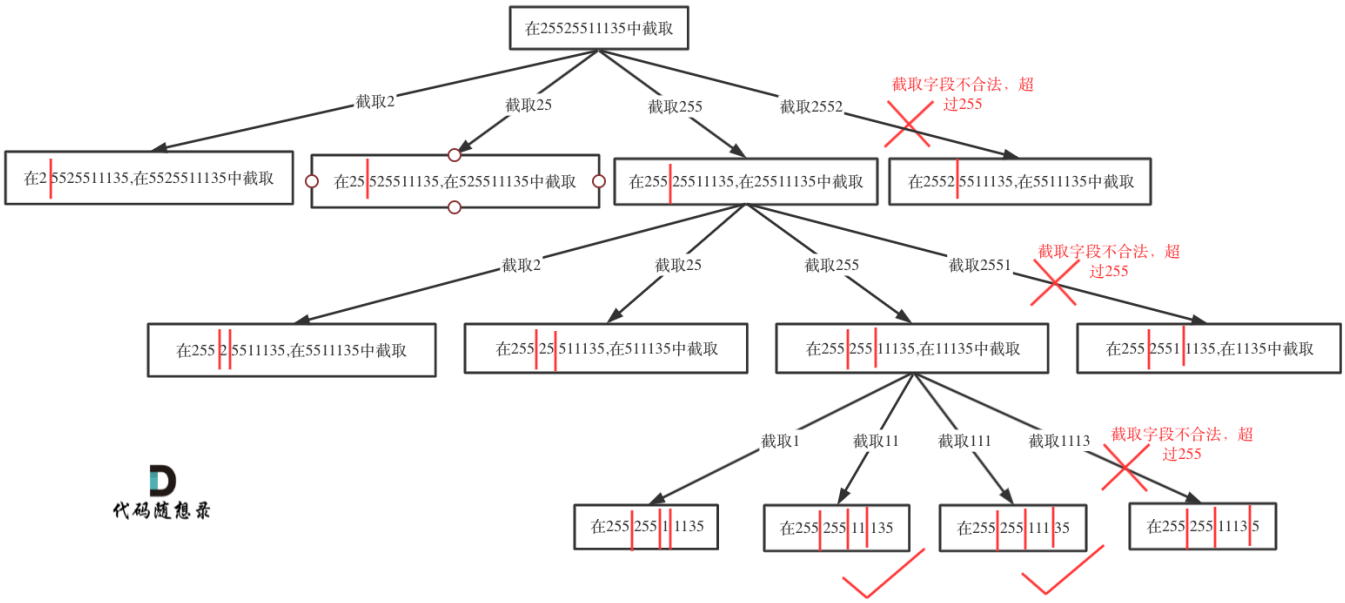


周四

如果没有做过回溯算法：分割回文串的话，回溯算法：复原IP地址这道题目应该是比较难的。

复原IP照回溯算法：分割回文串就多了一些限制，例如只能分四段，而且还是更改字符串，插入逗点。

树形图如下：



在本文的树形结构图中，我已经把详细的分析思路都画了出来，相信大家看了之后一定会思路清晰不少！

本题还可以有一个剪枝，合法ip长度为12，如果s的长度超过了12就不是有效IP地址，直接返回！

代码如下：

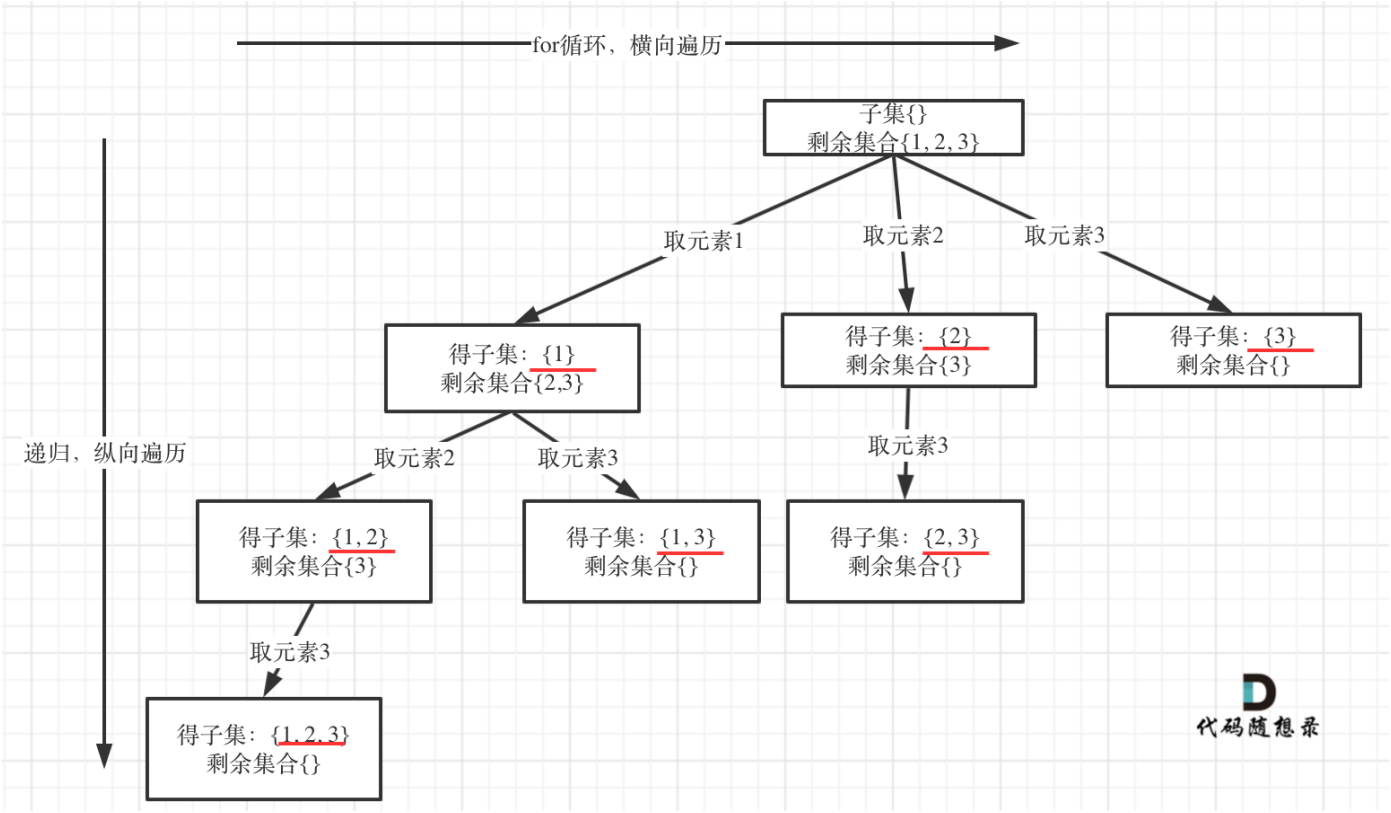
```
if (s.size() > 12) return result; // 剪枝
```

我之前给出的C++代码没有加这个限制，也没有超时，因为在第四段超过长度之后，就会截止了，所以就算给出特别长的字符串，搜索的范围也是有限的（递归只会到第三层），及时就会返回了。

周五

在回溯算法：求子集问题！中讲解了子集问题，**在树形结构中子集问题是要收集所有节点的结果，而组合问题是收集叶子节点的结果。**

如图：



认清这个本质之后，今天的题目就是一道模板题了。

其实可以不需要加终止条件，因为startIndex >= nums.size()，本层for循环本来也结束了，本来我们就要遍历整棵树。

有的同学可能担心不写终止条件会不会无限递归?

并不会，因为每次递归的下一层就是从i+1开始的。

如果要写终止条件，注意：`result.push_back(path);`要放在终止条件的上面，如下：

```
result.push_back(path); // 收集子集，要放在终止添加的上面，否则会漏掉自己
if (startIndex >= nums.size()) { // 终止条件可以不加
    return;
}
```

周六

早起的哈希表系列没有总结，所以哈希表：总结篇！（每逢总结必经典）如约而至。

可能之前大家做过很多哈希表的题目，但是没有串成线，总结篇来帮你串成线，捋顺哈希表的整个脉络。

大家对什么时候各种set与map比较疑惑，想深入了解红黑树，哈希之类的。

如果真的只是想清楚什么时候使用各种set与map，不用看那么多，把关于哈希表，你该了解这些！看了就够了。

总结

本周我们依次介绍了组合问题，分割问题以及子集问题，子集问题还没有讲完，下周还会继续。

我讲解每一种问题，都会和其他问题作对比，做分析，所以只要跟着细心琢磨相信对回溯又有新的认识。

最近这两天题目有点难度，刚刚开始学回溯算法的话，按照现在这个每天一题的速度来，确实有点快，学起来吃力非常正常，这些题目都是我当初学了好几个月才整明白的，哈哈。

所以大家能跟上的话，已经很优秀了！

还有一些录友会很关心leetcode上的耗时统计。

这个是很不准确的，相同的代码多提交几次，大家就知道怎么回事了。

leetcode上的计时应该是以4ms为单位，有的多提交几次，多个4ms就多击败50%，所以比较夸张，如果程序运行是几百ms的级别，可以看看leetcode上的耗时，因为它的误差10几ms对最终影响不大。

所以我的题解基本不会写击败百分之多少多少，没啥意义，时间复杂度分析清楚了就可以了，至于回溯算法不用分析时间复杂度了，都是一样的爆搜，就看谁剪枝厉害了。

一些录友表示最近回溯算法看的实在是有点懵，回溯算法确实是晦涩难懂，可能视频的话更直观一些，我最近应该会在B站（同名：「代码随想录」）出回溯算法的视频，大家也可以看视频在回顾一波。

就酱，又是充实的一周，做好本周总结，迎接下一周，冲！

13.子集II

力扣题目链接

给定一个可能包含重复元素的整数数组 nums，返回该数组所有可能的子集（幂集）。

说明：解集不能包含重复的子集。

示例:

- 输入: [1,2,2]
- 输出:
  [
  [2],
  [1],
  [1,2,2],
  [2,2],
  [1,2],
  []
  ]

算法公开课

《代码随想录》算法视频公开课：回溯算法解决子集问题，如何去重？ | LeetCode：90.子集II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

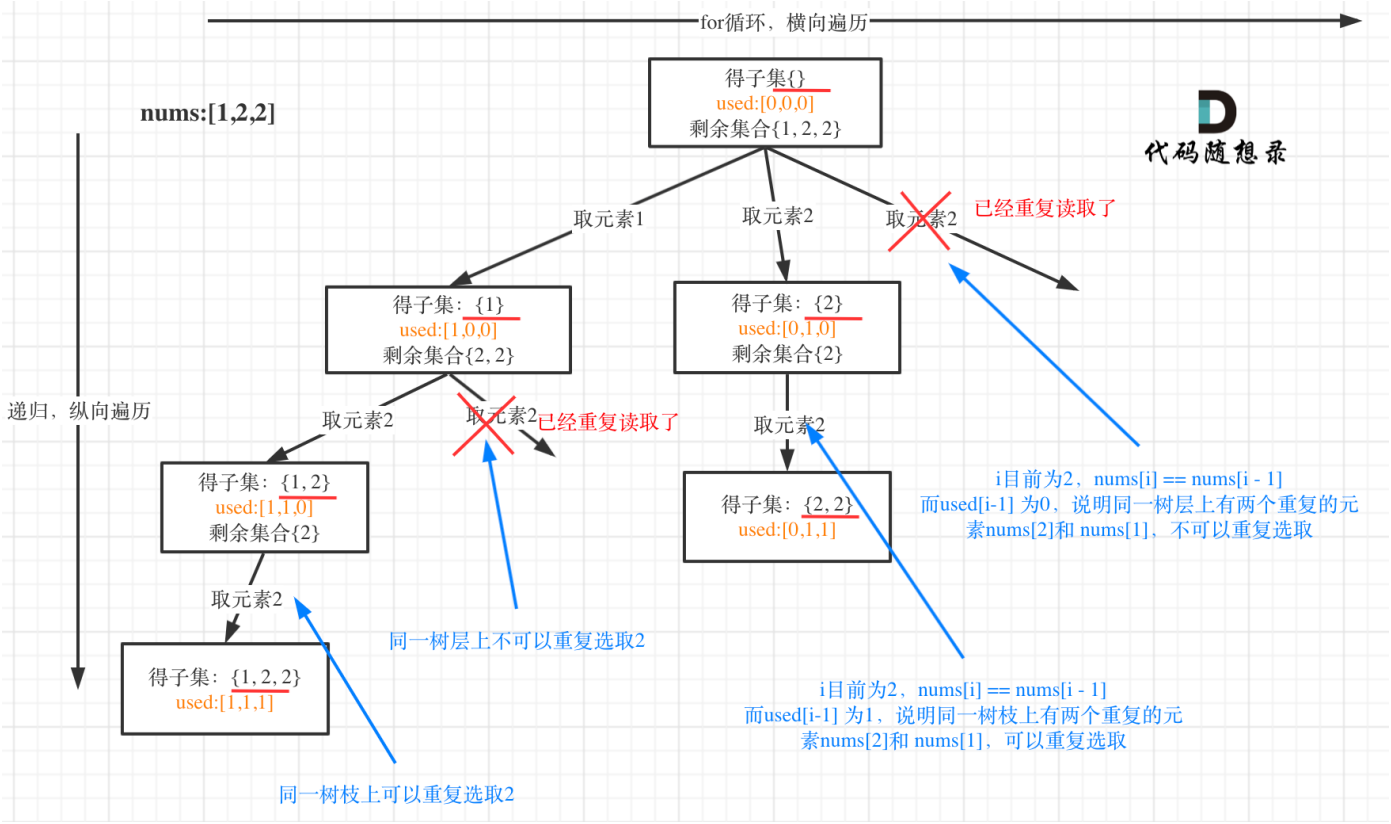
做本题之前一定要先做78.子集。

这道题目和78.子集区别就是集合里有重复元素了，而且求取的子集要去重。

那么关于回溯算法中的去重问题，**在[40.组合总和II]中已经详细讲解过了，和本题是一个套路。**

剧透一下，后期要讲解的排列问题里去重也是这个套路，所以理解“树层去重”和“树枝去重”非常重要。

用示例中的[1, 2, 2] 来举例，如图所示：（**注意去重需要先对集合排序**）



从图中可以看出，同一树层上重复取2 就要过滤掉，同一树枝上就可以重复取2，因为同一树枝上元素的集合才是唯一子集！

本题就是其实就是回溯算法：求子集问题！的基础上加上了去重，去重我们在回溯算法：求组合总和（三）也讲过了，所以我就直接给出代码了：

C++代码如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex, vector<bool>& used) {
        result.push_back(path);
        for (int i = startIndex; i < nums.size(); i++) {
            // used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
            // used[i - 1] == false，说明同一树层candidates[i - 1]使用过
            // 而我们要对同一树层使用过的元素进行跳过
            if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == false) {
                continue;
            }
            path.push_back(nums[i]);
            used[i] = true;
            backtracking(nums, i + 1, used);
```

```
            used[i] = false;
            path.pop_back();
        }
    }

public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        result.clear();
        path.clear();
        vector<bool> used(nums.size(), false);
        sort(nums.begin(), nums.end()); // 去重需要排序
        backtracking(nums, 0, used);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

使用set去重的版本。

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        result.push_back(path);
        unordered_set<int> uset;
        for (int i = startIndex; i < nums.size(); i++) {
            if (uset.find(nums[i]) != uset.end()) {
                continue;
            }
            uset.insert(nums[i]);
            path.push_back(nums[i]);
            backtracking(nums, i + 1);
            path.pop_back();
        }
    }

public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        result.clear();
        path.clear();
        sort(nums.begin(), nums.end()); // 去重需要排序
        backtracking(nums, 0);
        return result;
    }
};
```

补充

本题也可以不使用used数组来去重，因为递归的时候下一个startIndex是i+1而不是0。

如果要是全排列的话，每次要从0开始遍历，为了跳过已入栈的元素，需要使用used。

代码如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        result.push_back(path);
        for (int i = startIndex; i < nums.size(); i++) {
            // 而我们要对同一树层使用过的元素进行跳过
            if (i > startIndex && nums[i] == nums[i - 1] ) { // 注意这里使用i >
startIndex
                continue;
            }
            path.push_back(nums[i]);
            backtracking(nums, i + 1);
            path.pop_back();
        }
    }

public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        result.clear();
        path.clear();
        sort(nums.begin(), nums.end()); // 去重需要排序
        backtracking(nums, 0);
        return result;
    }
};
```

总结

其实这道题目的知识点，我们之前都讲过了，如果之前讲过的子集问题和去重问题都掌握的好，这道题目应该分分钟AC。

当然本题去重的逻辑，也可以这么写

```
if (i > startIndex && nums[i] == nums[i - 1] ) {
  continue;
}
```

> 和子集问题有点像，但又处处是陷阱

14.递增子序列

力扣题目链接

给定一个整型数组, 你的任务是找到所有该数组的递增子序列，递增子序列的长度至少是2。

示例:

- 输入: [4, 6, 7, 7]
- 输出: [[4, 6], [4, 7], [4, 6, 7], [4, 6, 7, 7], [6, 7], [6, 7, 7], [7,7], [4,7,7]]

说明:

- 给定数组的长度不会超过15。
- 数组中的整数范围是 [-100,100]。
- 给定数组中可能包含重复数字，相等的数字应该被视为递增的一种情况。

算法公开课

《代码随想录》算法视频公开课：回溯算法精讲，树层去重与树枝去重 | LeetCode：491.递增子序列，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这个递增子序列比较像是取有序的子集。而且本题也要求不能有相同的递增子序列。

这又是子集，又是去重，是不是不由自主的想起了刚刚讲过的90.子集II。

就是因为太像了，更要注意差别所在，要不就掉坑里了！

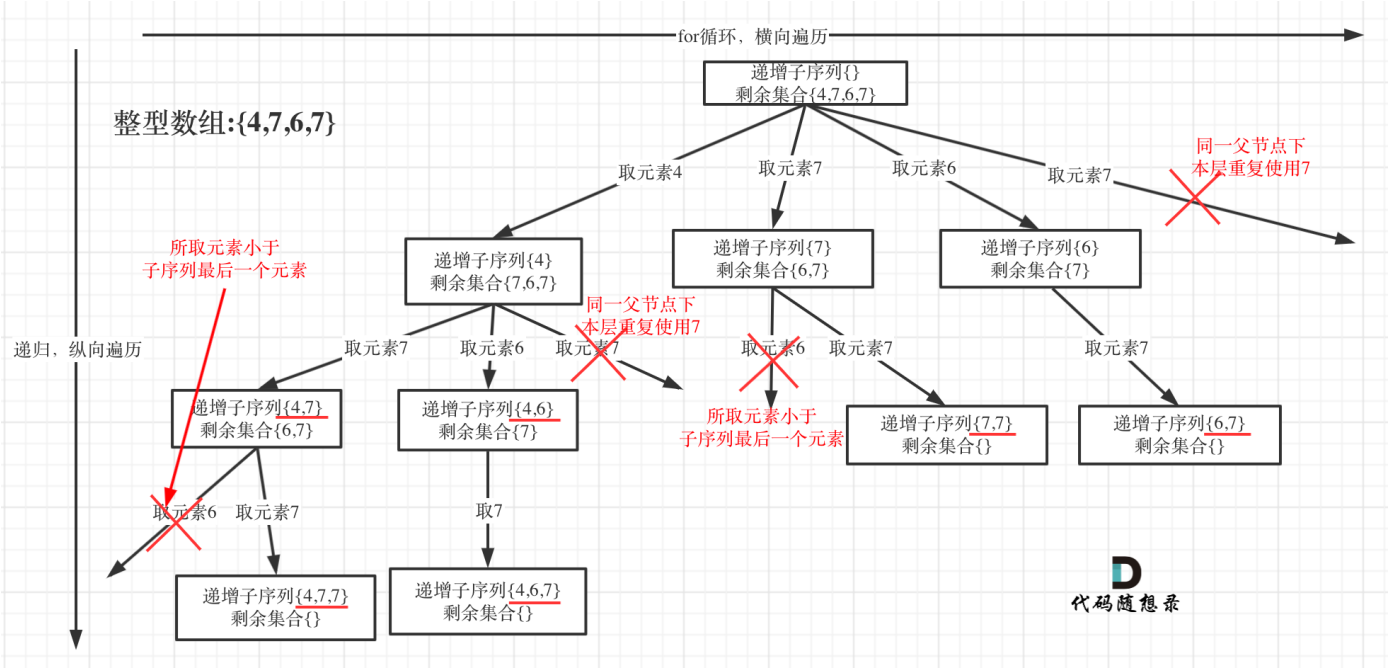
在90.子集II中我们是通过排序，再加一个标记数组来达到去重的目的。

而本题求自增子序列，是不能对原数组进行排序的，排完序的数组都是自增子序列了。

所以不能使用之前的去重逻辑！

本题给出的示例，还是一个有序数组 [4, 6, 7, 7]，这更容易误导大家按照排序的思路去做了。

为了有鲜明的对比，我用[4, 7, 6, 7]这个数组来举例，抽象为树形结构如图：



回溯三部曲

- 递归函数参数

本题求子序列，很明显一个元素不能重复使用，所以需要startIndex，调整下一层递归的起始位置。

代码如下：

```
vector<vector<int>> result;
vector<int> path;
void backtracking(vector<int>& nums, int startIndex)
```

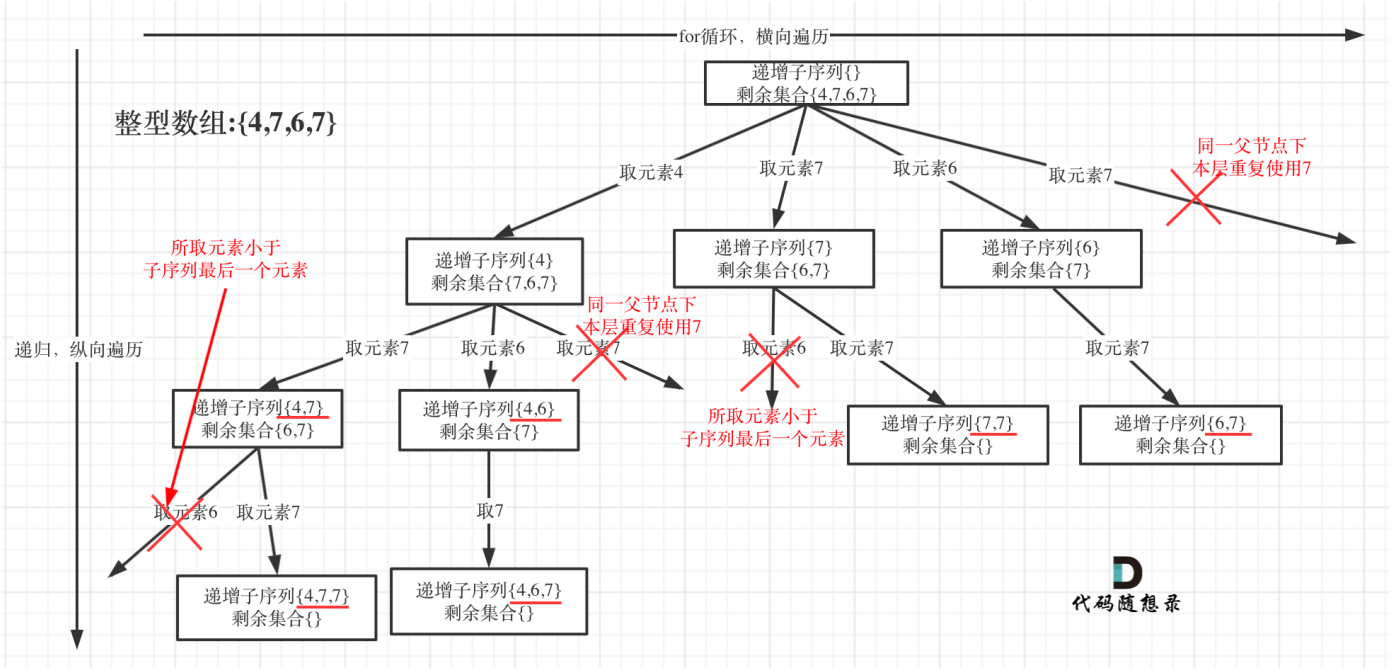
- 终止条件

本题其实类似求子集问题，也是要遍历树形结构找每一个节点，所以和回溯算法：求子集问题！一样，可以不加终止条件，startIndex每次都会加1，并不会无限递归。

但本题收集结果有所不同，题目要求递增子序列大小至少为2，所以代码如下：

```
if (path.size() > 1) {
    result.push_back(path);
    // 注意这里不要加return，因为要取树上的所有节点
}
```

- 单层搜索逻辑



在图中可以看出，**同一父节点下的同层上使用过的元素就不能再使用了**

那么单层搜索代码如下：

```
unordered_set<int> uset; // 使用set来对本层元素进行去重
for (int i = startIndex; i < nums.size(); i++) {
    if ((!path.empty() && nums[i] < path.back())
            || uset.find(nums[i]) != uset.end()) {
            continue;
    }
    uset.insert(nums[i]); // 记录这个元素在本层用过了，本层后面不能再用了
    path.push_back(nums[i]);
    backtracking(nums, i + 1);
    path.pop_back();
}
```

对于已经习惯写回溯的同学，看到递归函数上面的`uset.insert(nums[i]);`，下面却没有对应的pop之类的操作，应该很不习惯吧，哈哈

这也是需要注意的点，`unordered_set<int> uset;`是记录本层元素是否重复使用，新的一层uset都会重新定义（清空），所以要知道uset只负责本层！

最后整体C++代码如下：

```
// 版本一
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        if (path.size() > 1) {
            result.push_back(path);
            // 注意这里不要加return，要取树上的节点
```

```
        }
        unordered_set<int> uset; // 使用set对本层元素进行去重
        for (int i = startIndex; i < nums.size(); i++) {
            if ((!path.empty() && nums[i] < path.back())
                    || uset.find(nums[i]) != uset.end()) {
                    continue;
            }
            uset.insert(nums[i]); // 记录这个元素在本层用过了，本层后面不能再用了
            path.push_back(nums[i]);
            backtracking(nums, i + 1);
            path.pop_back();
        }
    }
public:
    vector<vector<int>> findSubsequences(vector<int>& nums) {
        result.clear();
        path.clear();
        backtracking(nums, 0);
        return result;
    }
};
```

- 时间复杂度: O(n * 2^n)
- 空间复杂度: O(n)

优化

以上代码用我用了`unordered_set<int>`来记录本层元素是否重复使用。

其实用数组来做哈希，效率就高了很多。

注意题目中说了，数值范围[-100,100]，所以完全可以用数组来做哈希。

程序运行的时候对unordered_set 频繁的insert，unordered_set需要做哈希映射（也就是把key通过hash function映射为唯一的哈希值）相对费时间，而且每次重新定义set，insert的时候其底层的符号表也要做相应的扩充，也是费事的。

那么优化后的代码如下：

```
// 版本二
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        if (path.size() > 1) {
            result.push_back(path);
        }
        int used[201] = {0}; // 这里使用数组来进行去重操作，题目说数值范围[-100, 100]
        for (int i = startIndex; i < nums.size(); i++) {
```

```
            if ((!path.empty() && nums[i] < path.back())
                    || used[nums[i] + 100] == 1) {
                    continue;
            }
            used[nums[i] + 100] = 1; // 记录这个元素在本层用过了，本层后面不能再用了
            path.push_back(nums[i]);
            backtracking(nums, i + 1);
            path.pop_back();
        }
    }
public:
    vector<vector<int>> findSubsequences(vector<int>& nums) {
        result.clear();
        path.clear();
        backtracking(nums, 0);
        return result;
    }
};
```

这份代码在leetcode上提交，要比版本一耗时要好的多。

所以正如在哈希表：总结篇！（每逢总结必经典）中说的那样，数组，set，map都可以做哈希表，而且数组干的活，map和set都能干，但如果数值范围小的话能用数组尽量用数组。

总结

本题题解清一色都说是深度优先搜索，但我更倾向于说它用回溯法，而且本题我也是完全使用回溯法的逻辑来分析的。

相信大家在本题中处处都能看到是回溯算法：求子集问题（二）的身影，但处处又都是陷阱。

对于养成思维定式或者套模板套嗨了的同学，这道题起到了很好的警醒作用。更重要的是拓展了大家的思路！

15.全排列

力扣题目链接

给定一个 没有重复 数字的序列，返回其所有可能的全排列。

示例:

- 输入: [1,2,3]
- 输出:
  [
  [1,2,3],
  [1,3,2],
  [2,1,3],
  [2,3,1],

[3,1,2],
[3,2,1]
]

算法公开课

《代码随想录》算法视频公开课：组合与排列的区别，回溯算法求解的时候，有何不同？| LeetCode：46.全排列，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

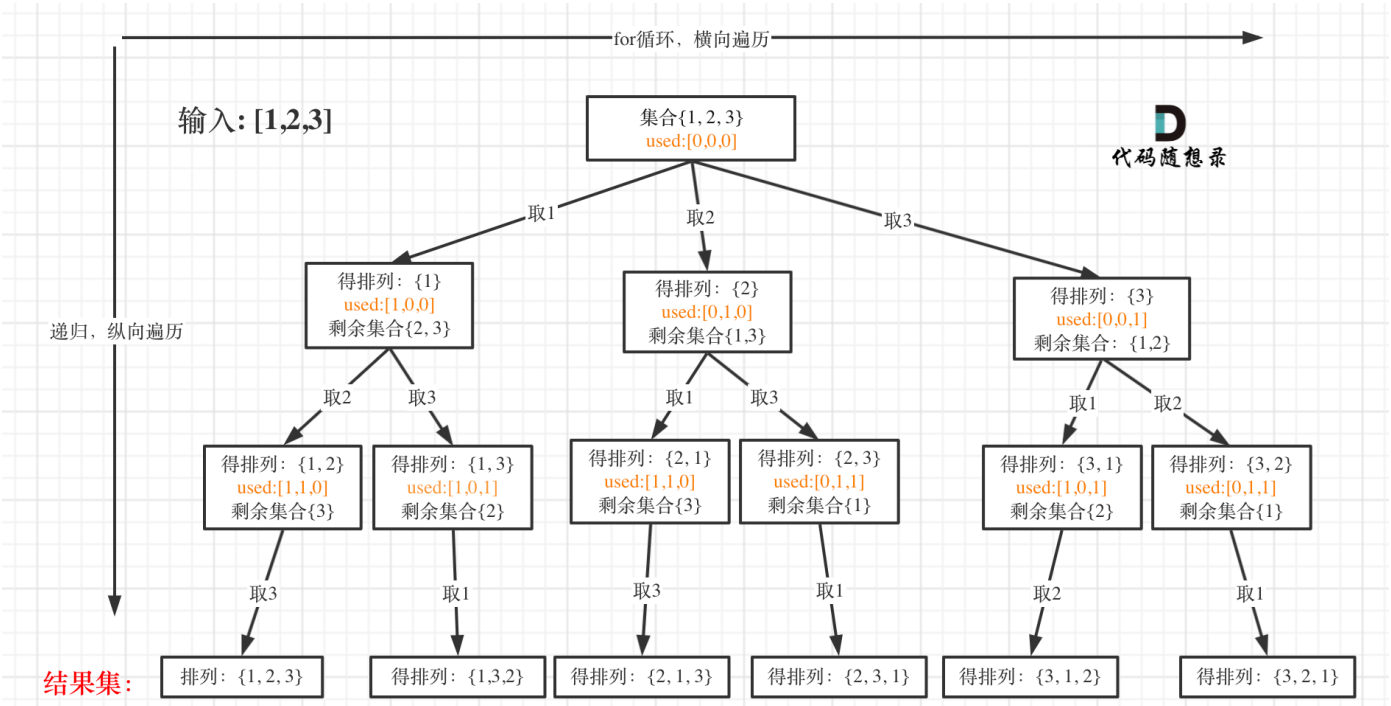
此时我们已经学习了77.组合问题、 131.分割回文串和78.子集问题，接下来看一看排列问题。

相信这个排列问题就算是让你用for循环暴力把结果搜索出来，这个暴力也不是很好写。

所以正如我们在关于回溯算法，你该了解这些！所讲的为什么回溯法是暴力搜索，效率这么低，还要用它?

因为一些问题能暴力搜出来就已经很不错了！

我以[1,2,3]为例，抽象成树形结构如下：



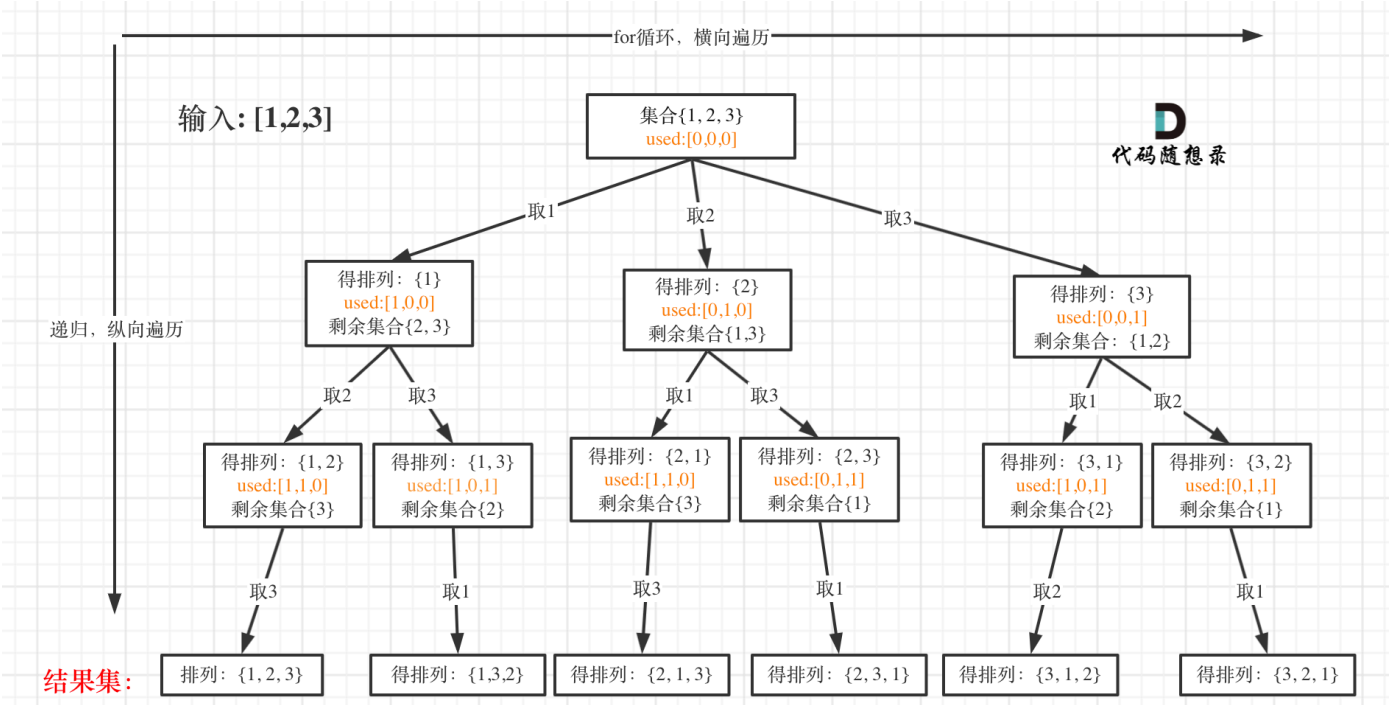
回溯三部曲

- 递归函数参数

首先排列是有序的，也就是说 [1,2] 和 [2,1] 是两个集合，这和之前分析的子集以及组合所不同的地方。

可以看出元素1在[1,2]中已经使用过了，但是在[2,1]中还要在使用一次1，所以处理排列问题就不用使用startIndex了。

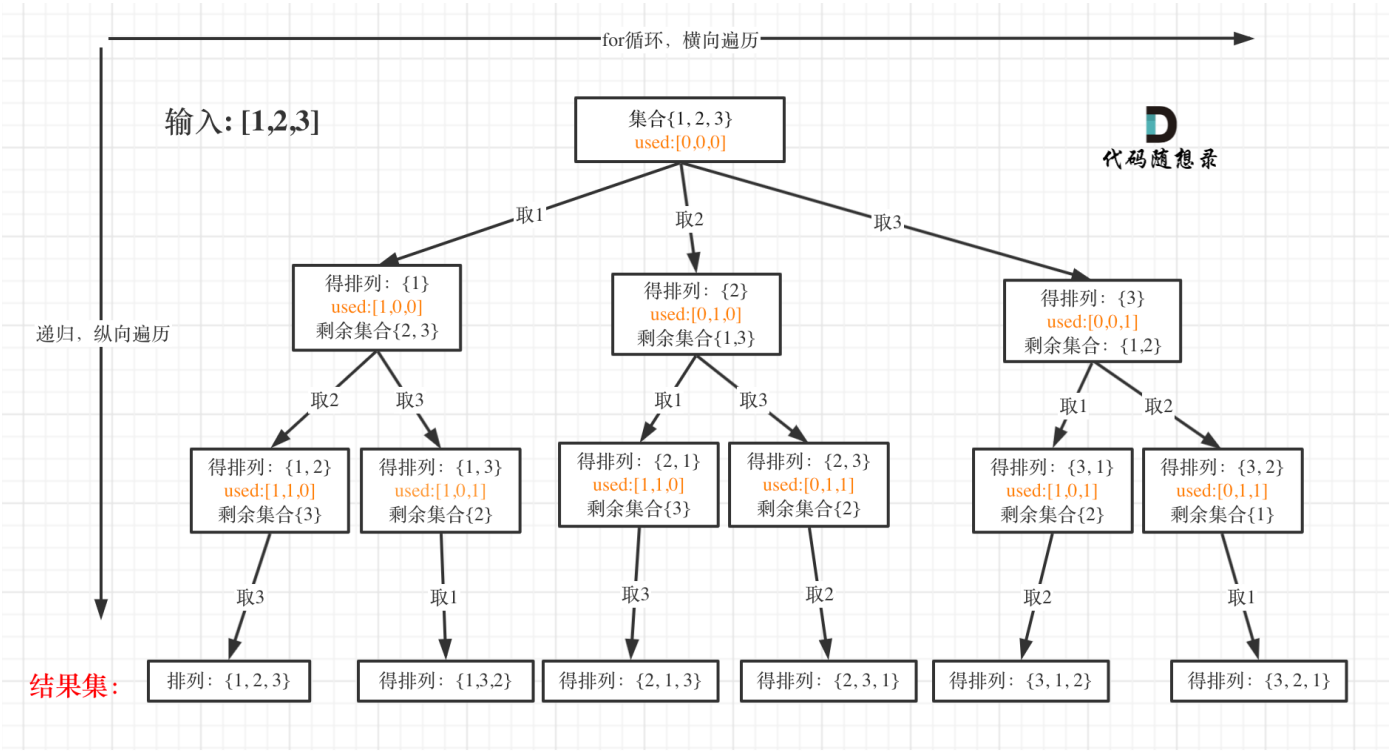
但排列问题需要一个used数组，标记已经选择的元素，如图橘黄色部分所示:



代码如下：

```
vector<vector<int>> result;
vector<int> path;
void backtracking (vector<int>& nums, vector<bool>& used)
```

- 递归终止条件



可以看出叶子节点，就是收割结果的地方。

那么什么时候，算是到达叶子节点呢?

当收集元素的数组path的大小达到和nums数组一样大的时候，说明找到了一个全排列，也表示到达了叶子节点。

代码如下：

```
// 此时说明找到了一组
if (path.size() == nums.size()) {
    result.push_back(path);
    return;
}
```

- 单层搜索的逻辑

这里和77.组合问题、131.切割问题和78.子集问题最大的不同就是for循环里不用startIndex了。

因为排列问题，每次都要从头开始搜索，例如元素1在[1,2]中已经使用过了，但是在[2,1]中还要再使用一次1。

而used数组，其实就是记录此时path里都有哪些元素使用了，一个排列里一个元素只能使用一次。

代码如下：

```
for (int i = 0; i < nums.size(); i++) {
    if (used[i] == true) continue; // path里已经收录的元素，直接跳过
    used[i] = true;
    path.push_back(nums[i]);
    backtracking(nums, used);
    path.pop_back();
    used[i] = false;
}
```

整体C++代码如下：

```
class Solution {
public:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking (vector<int>& nums, vector<bool>& used) {
        // 此时说明找到了一组
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }
        for (int i = 0; i < nums.size(); i++) {
            if (used[i] == true) continue; // path里已经收录的元素，直接跳过
            used[i] = true;
            path.push_back(nums[i]);
            backtracking(nums, used);
            path.pop_back();
            used[i] = false;
        }
    }
    vector<vector<int>> permute(vector<int>& nums) {
        result.clear();
```

```
        path.clear();
        vector<bool> used(nums.size(), false);
        backtracking(nums, used);
        return result;
    }
};
```

- 时间复杂度: O(n!)
- 空间复杂度: O(n)

总结

大家此时可以感受出排列问题的不同：

- 每层都是从0开始搜索而不是startIndex
- 需要used数组记录path里都放了哪些元素了

排列问题是回溯算法解决的经典题目，大家可以好好体会体会。

16.全排列 II

力扣题目链接

给定一个可包含重复数字的序列 nums ，按任意顺序 返回所有不重复的全排列。

示例 1：

- 输入：nums = [1,1,2]
- 输出：
  [[1,1,2],
  [1,2,1],
  [2,1,1]]

示例 2：

- 输入：nums = [1,2,3]
- 输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

提示：

- 1 <= nums.length <= 8
- -10 <= nums[i] <= 10

算法公开课

《代码随想录》算法视频公开课：回溯算法求解全排列，如何去重？ | LeetCode：47.全排列 II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目和46.全排列的区别在与**给定一个可包含重复数字的序列，要返回所有不重复的全排列。**

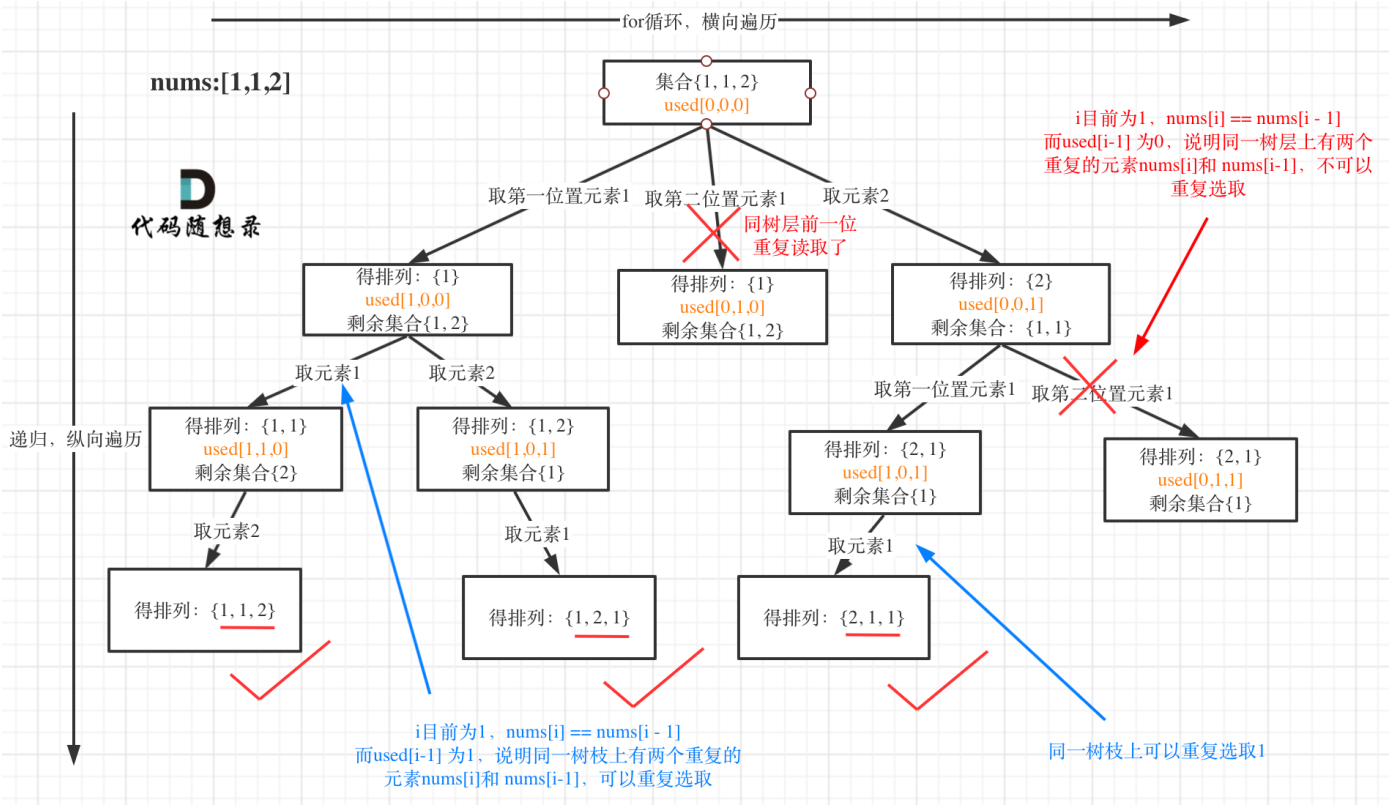
这里又涉及到去重了。

在40.组合总和II 、90.子集II我们分别详细讲解了组合问题和子集问题如何去重。

那么排列问题其实也是一样的套路。

还要强调的是去重一定要对元素进行排序，这样我们才方便通过相邻的节点来判断是否重复使用了。

我以示例中的 [1,1,2]为例 （为了方便举例，已经排序）抽象为一棵树，去重过程如图：



图中我们对同一树层，前一位（也就是nums[i-1]）如果使用过，那么就进行去重。

一般来说：组合问题和排列问题是在树形结构的叶子节点上收集结果，而子集问题就是取树上所有节点的结果。

在46.全排列中已经详细讲解了排列问题的写法，在40.组合总和II 、90.子集II中详细讲解了去重的写法，所以这次我就不用回溯三部曲分析了，直接给出代码，如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking (vector<int>& nums, vector<bool>& used) {
        // 此时说明找到了一组
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
```

```
        }
        for (int i = 0; i < nums.size(); i++) {
            // used[i - 1] == true，说明同一树枝nums[i - 1]使用过
            // used[i - 1] == false，说明同一树层nums[i - 1]使用过
            // 如果同一树层nums[i - 1]使用过则直接跳过
            if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == false) {
                continue;
            }
            if (used[i] == false) {
                used[i] = true;
                path.push_back(nums[i]);
                backtracking(nums, used);
                path.pop_back();
                used[i] = false;
            }
        }
    }
public:
    vector<vector<int>> permuteUnique(vector<int>& nums) {
        result.clear();
        path.clear();
        sort(nums.begin(), nums.end()); // 排序
        vector<bool> used(nums.size(), false);
        backtracking(nums, used);
        return result;
    }
};

// 时间复杂度：最差情况所有元素都是唯一的。复杂度和全排列1都是 O(n! * n) 对于 n 个元素一共有 n! 中排列方案。而对于每一个答案，我们需要 O(n) 去复制最终放到 result 数组
// 空间复杂度：O(n) 回溯树的深度取决于我们有多少个元素
```

- 时间复杂度: O(n! * n)
- 空间复杂度: O(n)

拓展

大家发现，去重最为关键的代码为：

```
if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == false) {
    continue;
}
```

如果改成 `used[i - 1] == true`， 也是正确的!，去重代码如下：

```
if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == true) {
    continue;
}
```

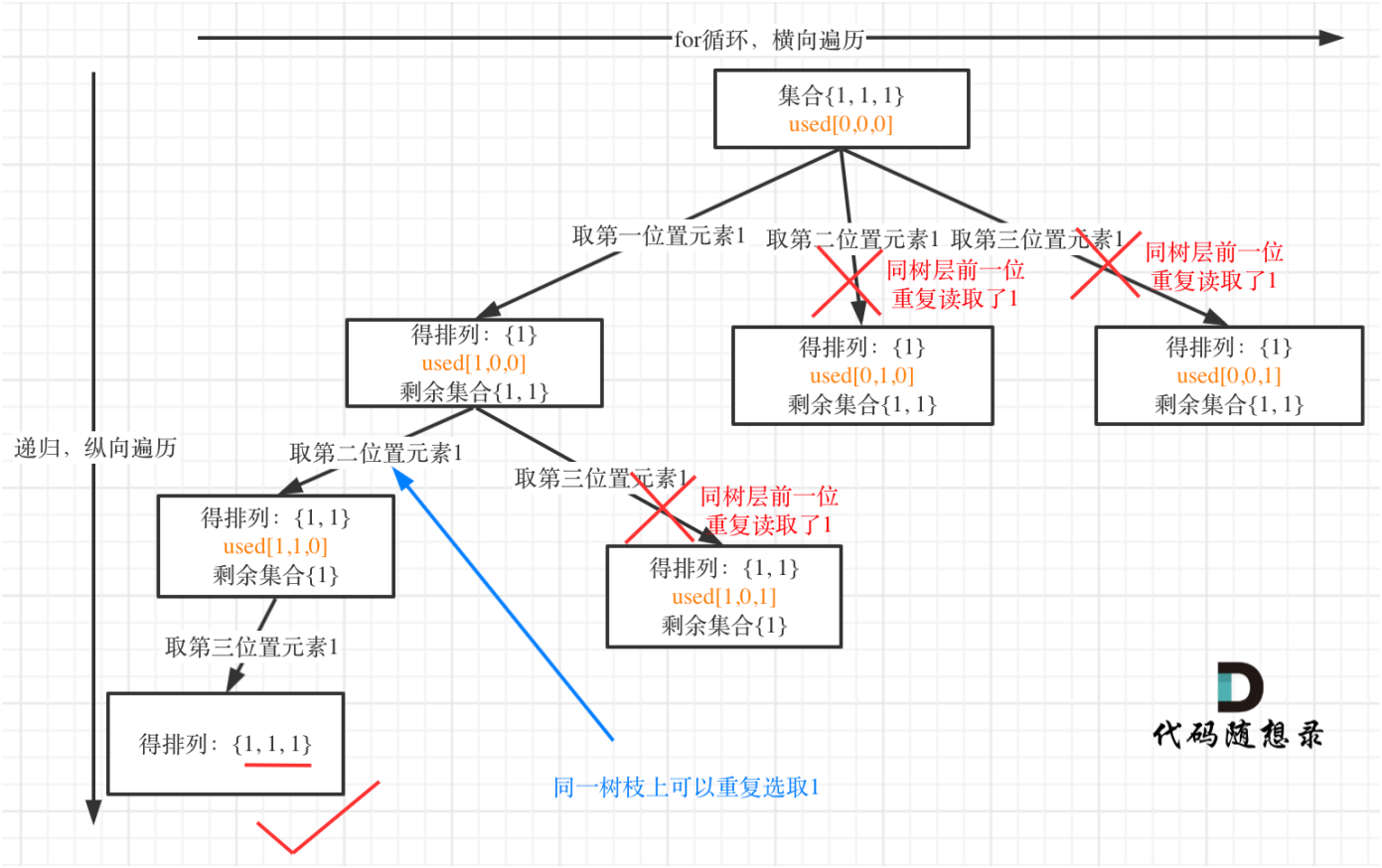
这是为什么呢，就是上面我刚说的，如果要对树层中前一位去重，就用`used[i - 1] == false`，如果要对树枝前一位去重用`used[i - 1] == true`。

对于排列问题，树层上去重和树枝上去重，都是可以的，但是树层上去重效率更高！

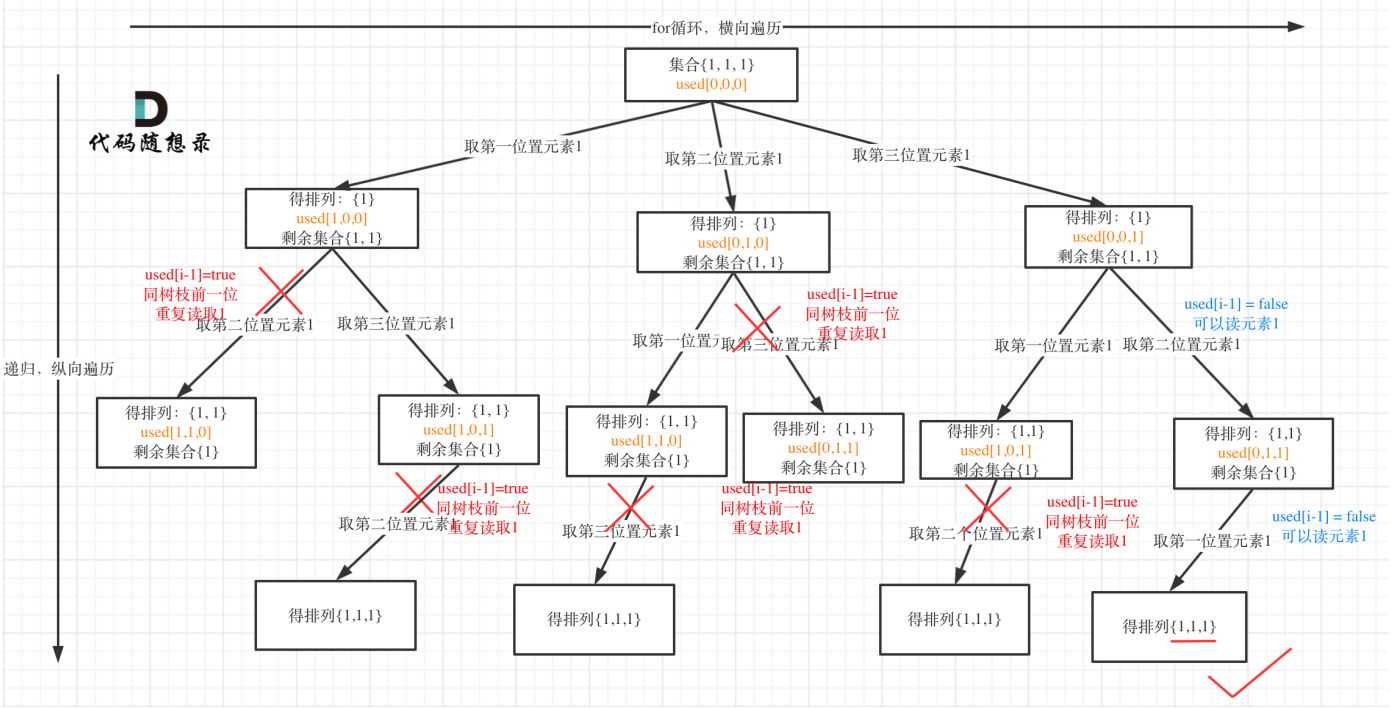
这么说是不是有点抽象？

来来来，我就用输入: [1,1,1] 来举一个例子。

树层上去重(used[i - 1] == false)，的树形结构如下：



树枝上去重（used[i - 1] == true）的树型结构如下：



大家应该很清晰的看到，树层上对前一位去重非常彻底，效率很高，树枝上对前一位去重虽然最后可以得到答案，但是做了很多无用搜索。

总结

这道题其实还是用了我们之前讲过的去重思路，但有意思的是，去重的代码中，这么写：

```
if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == false) {
    continue;
}
```

和这么写：

```
if (i > 0 && nums[i] == nums[i - 1] && used[i - 1] == true) {
    continue;
}
```

都是可以的，这也是很多同学做这道题目困惑的地方，知道`used[i - 1] == false`也行而`used[i - 1] == true`也行，但是就想不明白为啥。

所以我通过举[1,1,1]的例子，把这两个去重的逻辑分别抽象成树形结构，大家可以一目了然：为什么两种写法都可以以及哪一种效率更高！

这里可能大家又有疑惑，既然`used[i - 1] == false`也行而`used[i - 1] == true`也行，那为什么还要写这个条件呢?

直接这样写 不就完事了?

```
if (i > 0 && nums[i] == nums[i - 1]) {
    continue;
}
```

其实并不行，一定要加上 `used[i - 1] == false`或者`used[i - 1] == true`，因为 used[i - 1] 要一直是 true 或者一直是false 才可以，而不是 一会是true 一会又是false。 所以这个条件要写上。

是不是豁然开朗了！！

---

17. 本周小结！（回溯算法系列三）

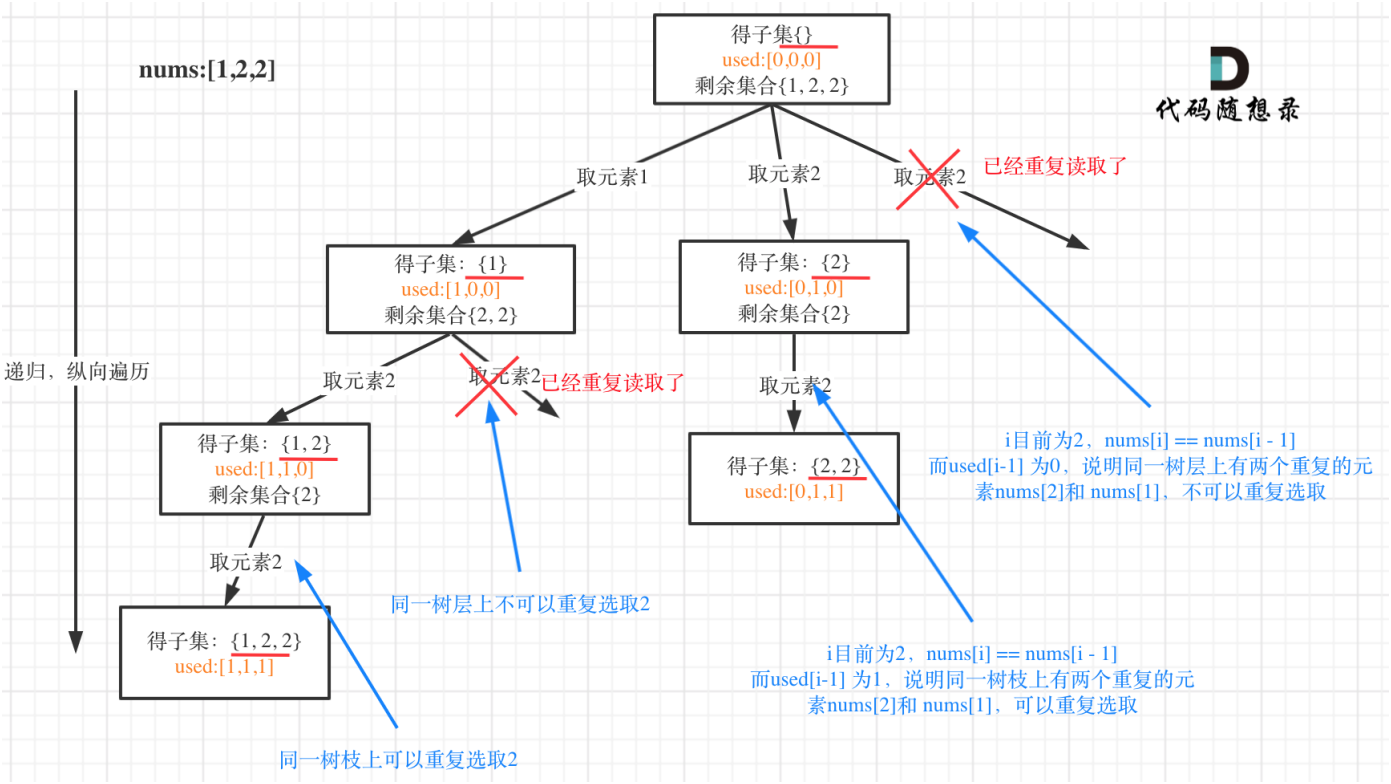
周一

在回溯算法：求子集问题（二）中，开始针对子集问题进行去重。

本题就是回溯算法：求子集问题！的基础上加上了去重，去重我们在回溯算法：求组合总和（三）也讲过了。

所以本题对大家应该并不难。

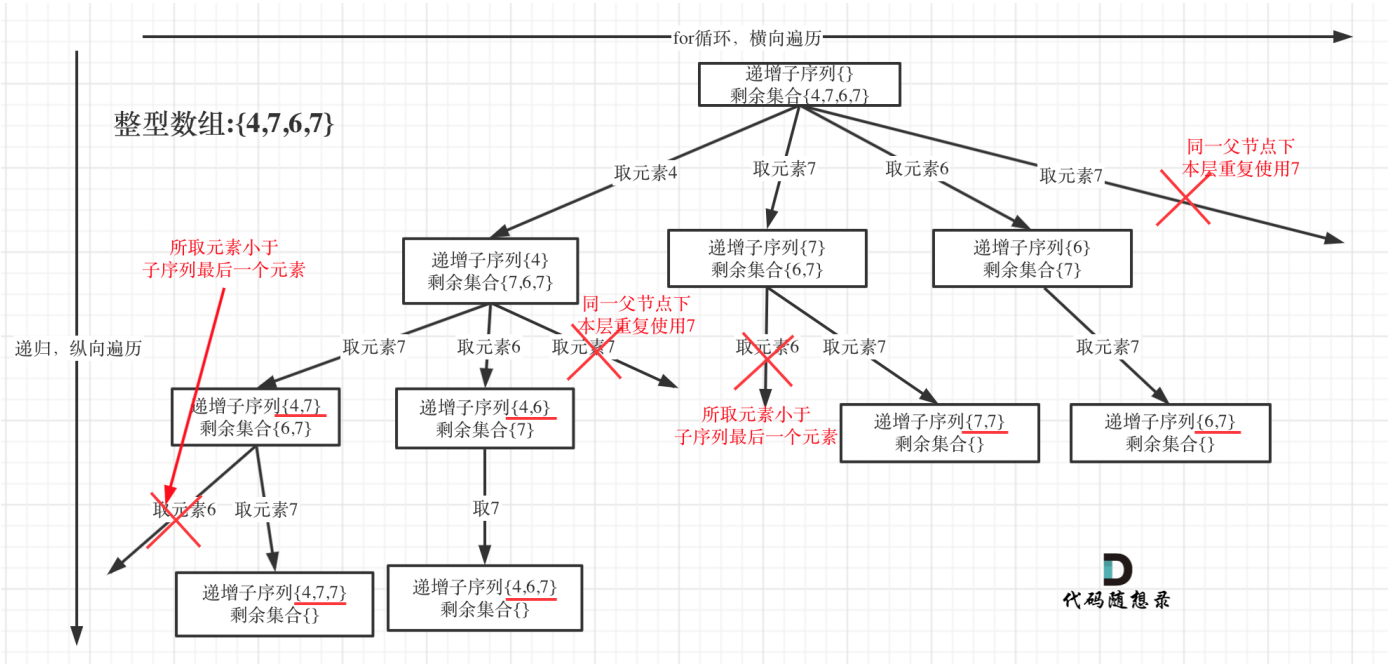
树形结构如下：



周二

在回溯算法：递增子序列中，处处都能看到子集的身影，但处处是陷阱，值得好好琢磨琢磨！

树形结构如下：



[回溯算法：递增子序列](#)留言区大家有很多疑问，主要还是和[回溯算法：求子集问题（二）](#)混合在了一起。

详细在[本周小结！（回溯算法系列三）续集](#)中给出了介绍！

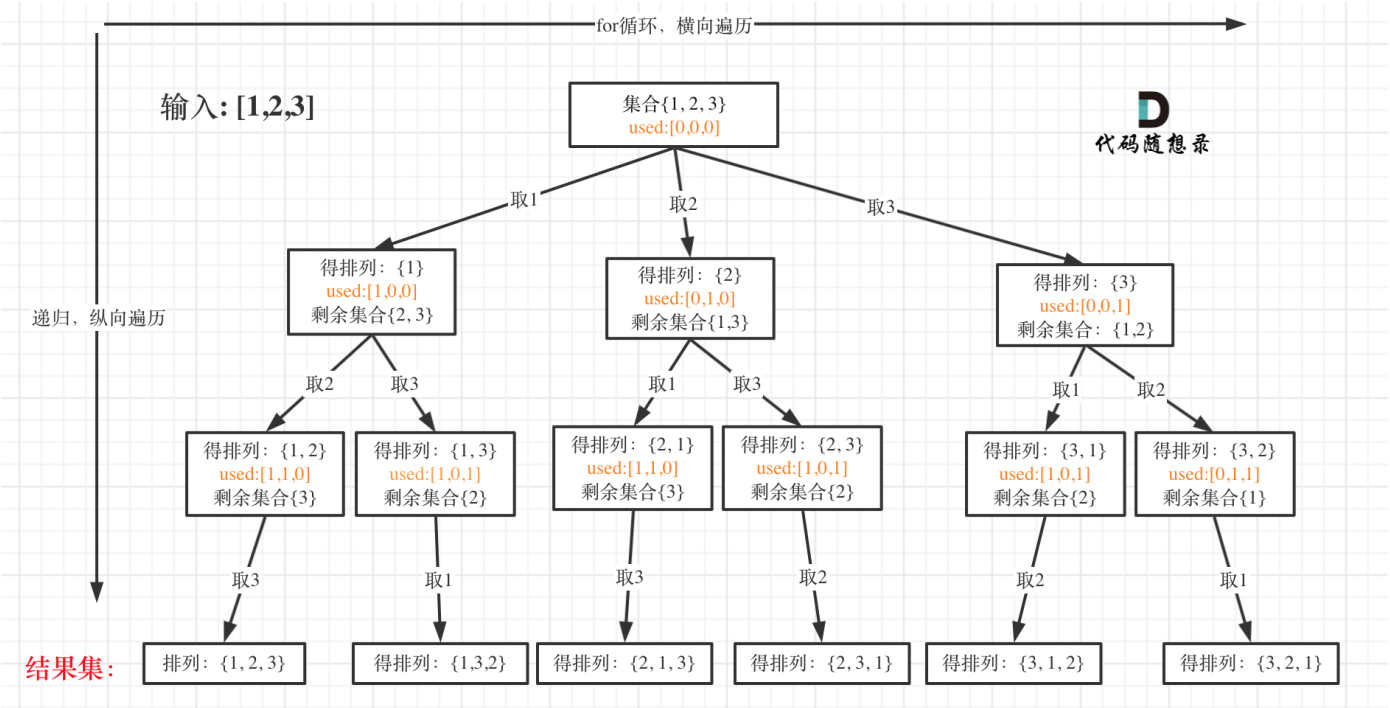
周三

我们已经分析了组合问题，分割问题，子集问题，那么[回溯算法：排列问题！](#)又不一样了。

排列是有序的，也就是说[1,2] 和[2,1] 是两个集合，这和之前分析的子集以及组合所不同的地方。

可以看出元素1在[1,2]中已经使用过了，但是在[2,1]中还要在使用一次1，所以处理排列问题就不用使用startIndex了。

如图：



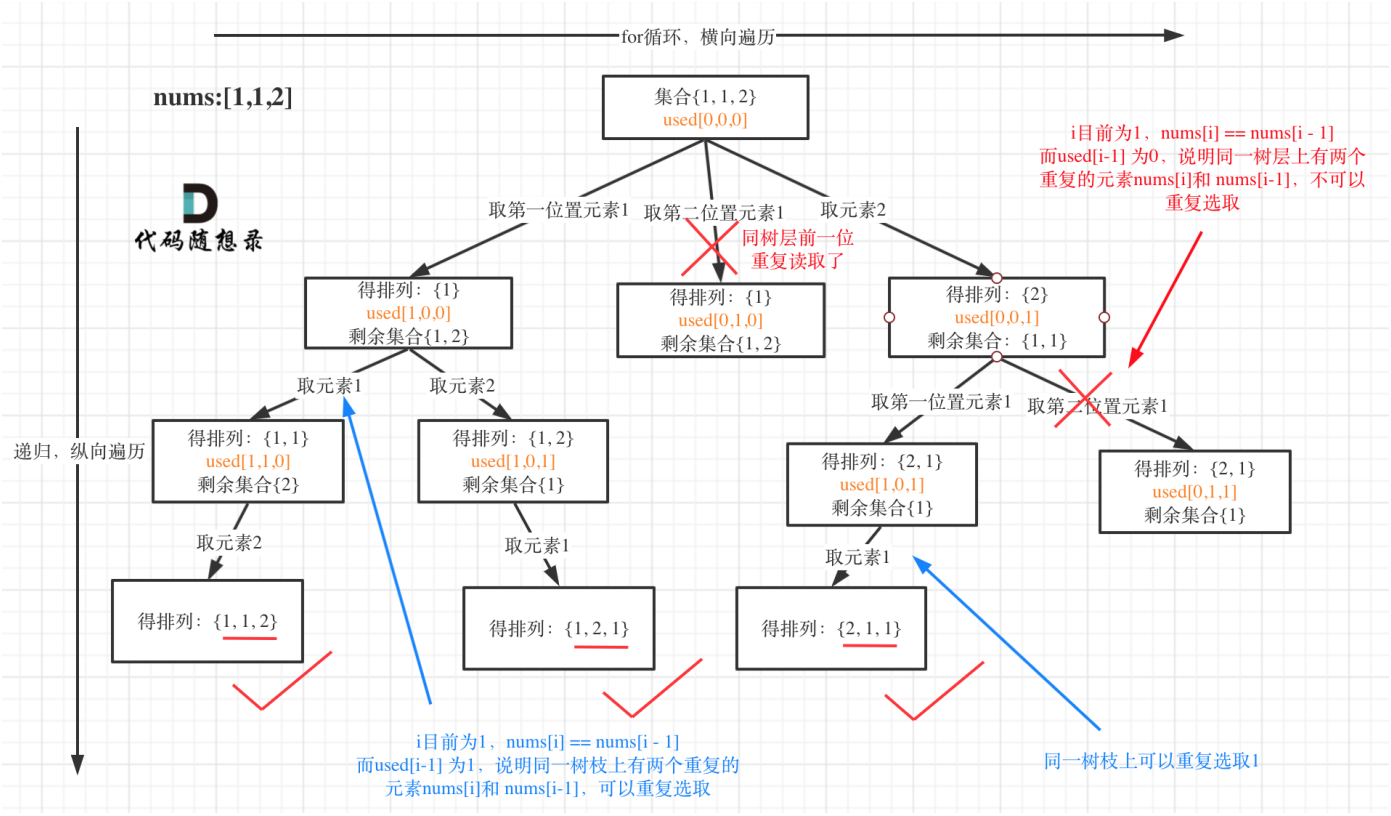
大家此时可以感受出排列问题的不同：

- 每层都是从0开始搜索而不是startIndex
- 需要used数组记录path里都放了哪些元素了

周四

排列问题也要去重了，在回溯算法：排列问题（二）中又一次强调了“树层去重”和“树枝去重”。

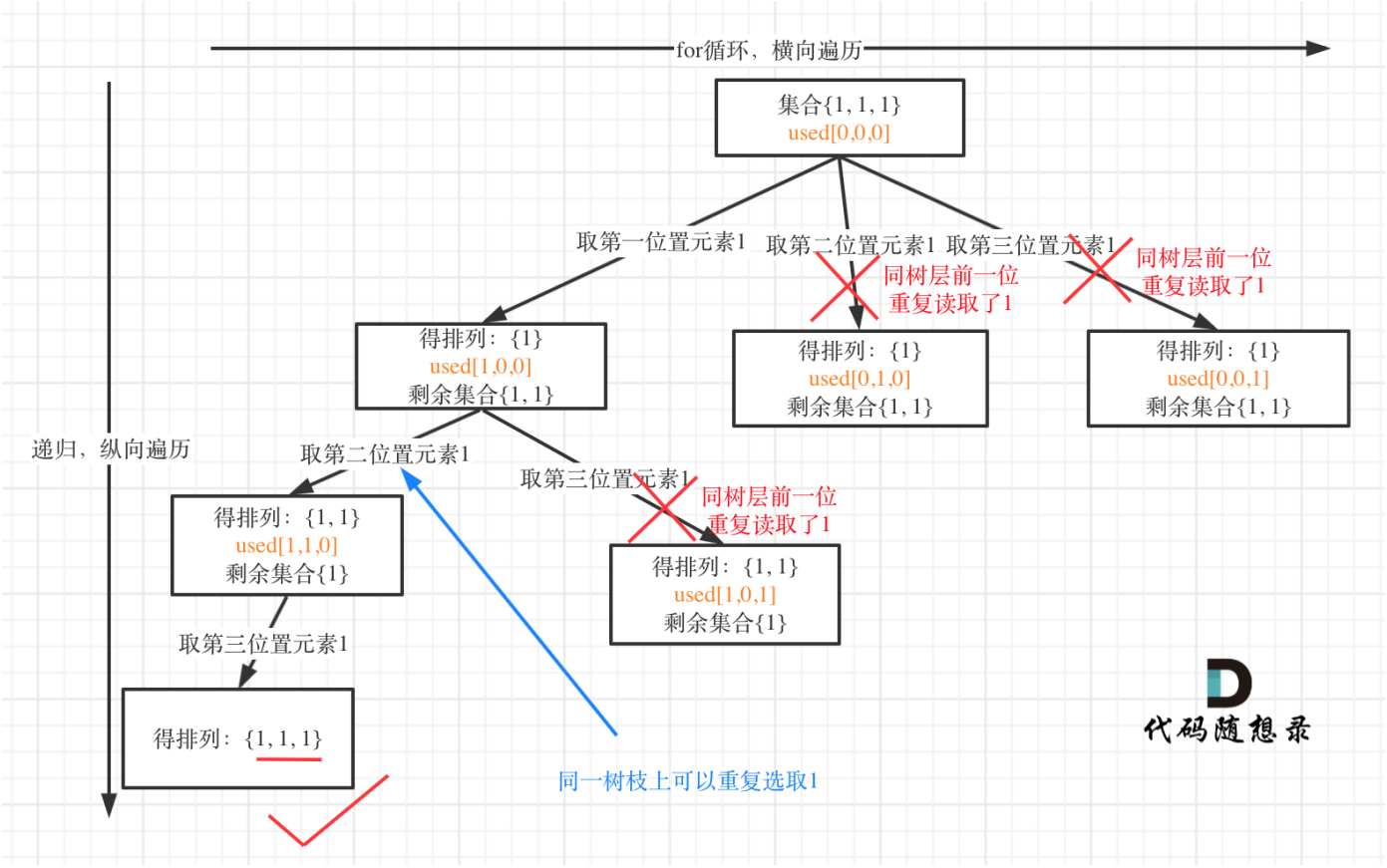
树形结构如下：



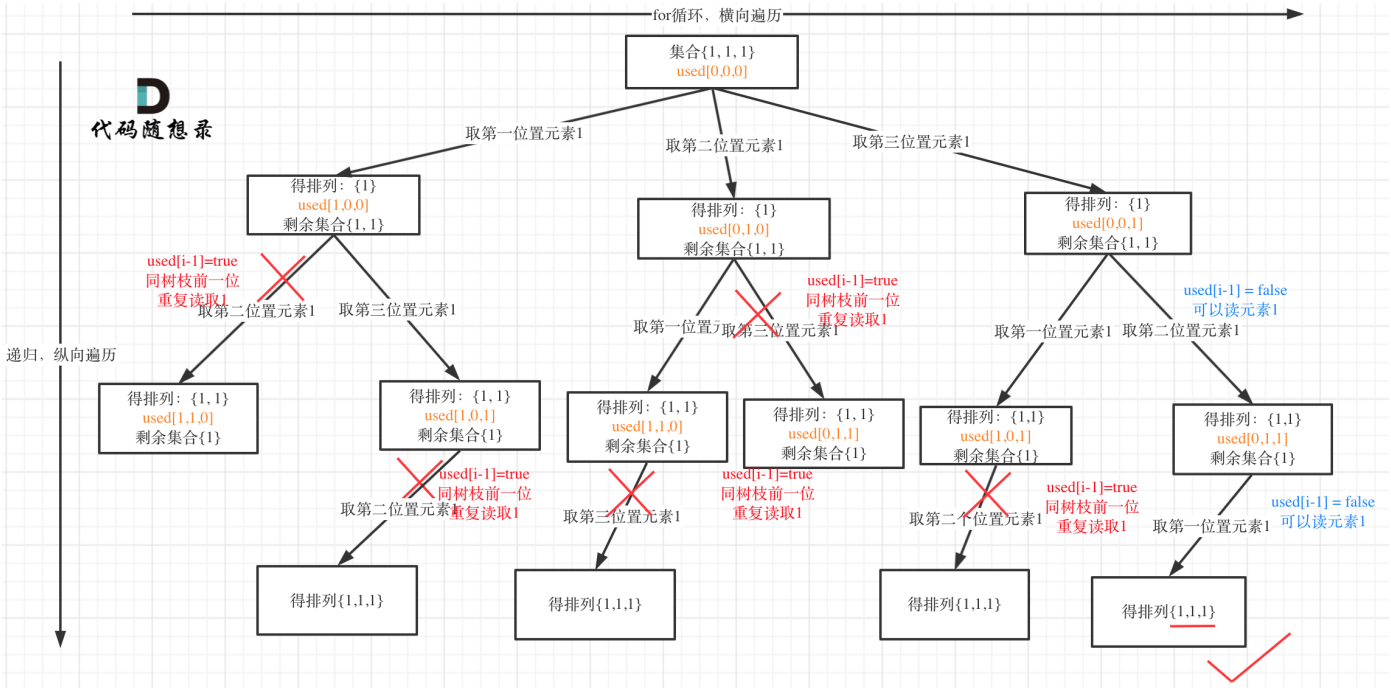
这道题目神奇的地方就是used[i - 1] == false也可以，used[i - 1] == true也可以！

我就用输入: [1,1,1] 来举一个例子。

树层上去重(used[i - 1] == false)，的树形结构如下：



树枝上去重（used[i - 1] == true）的树型结构如下：



可以清晰的看到使用(used[i - 1] == false)，即树层去重，效率更高！

性能分析

之前并没有分析各个问题的时间复杂度和空间复杂度，这次来说一说。

这块网上的资料鱼龙混杂，一些所谓的经典面试书籍根本不讲回溯算法，算法书籍对这块也避而不谈，感觉就像是算法里模糊的边界。

所以这块就说一说我个人理解，对内容持开放态度，集思广益，欢迎大家来讨论！

子集问题分析：

- 时间复杂度：$O(n × 2^n)$，因为每一个元素的状态无外乎取与不取，所以时间复杂度为$O(2^n)$，构造每一组子集都需要填进数组，又有需要$O(n)$，最终时间复杂度：$O(n × 2^n)$。
- 空间复杂度：$O(n)$，递归深度为n，所以系统栈所用空间为$O(n)$，每一层递归所用的空间都是常数级别，注意代码里的result和path都是全局变量，就算是放在参数里，传的也是引用，并不会新申请内存空间，最终空间复杂度为$O(n)$。

排列问题分析：

- 时间复杂度：$O(n!)$，这个可以从排列的树形图中很明显发现，每一层节点为n，第二层每一个分支都延伸了n-1个分支，再往下又是n-2个分支，所以一直到叶子节点一共就是 n * n-1 * n-2 * ..... 1 = n!。每个叶子节点都会有一个构造全排列填进数组的操作（对应的代码：`result.push_back(path)`），该操作的复杂度为$O(n)$。所以，最终时间复杂度为：n * n!，简化为$O(n!)$。
- 空间复杂度：$O(n)$，和子集问题同理。

组合问题分析：

- 时间复杂度：$O(n × 2^n)$，组合问题其实就是一种子集的问题，所以组合问题最坏的情况，也不会超过子集问题的时间复杂度。
- 空间复杂度：$O(n)$，和子集问题同理。

一般说道回溯算法的复杂度，都说是指数级别的时间复杂度，这也算是一个概括吧！

总结

本周我们对子集问题进行了去重，然后介绍了和子集问题非常像的递增子序列，如果还保持惯性思维，这道题就可以掉坑里。

接着介绍了排列问题！，以及对排列问题如何进行去重。

最后我补充了子集问题，排列问题和组合问题的性能分析，给大家提供了回溯算法复杂度的分析思路。

---

18. 回溯算法去重问题的另一种写法

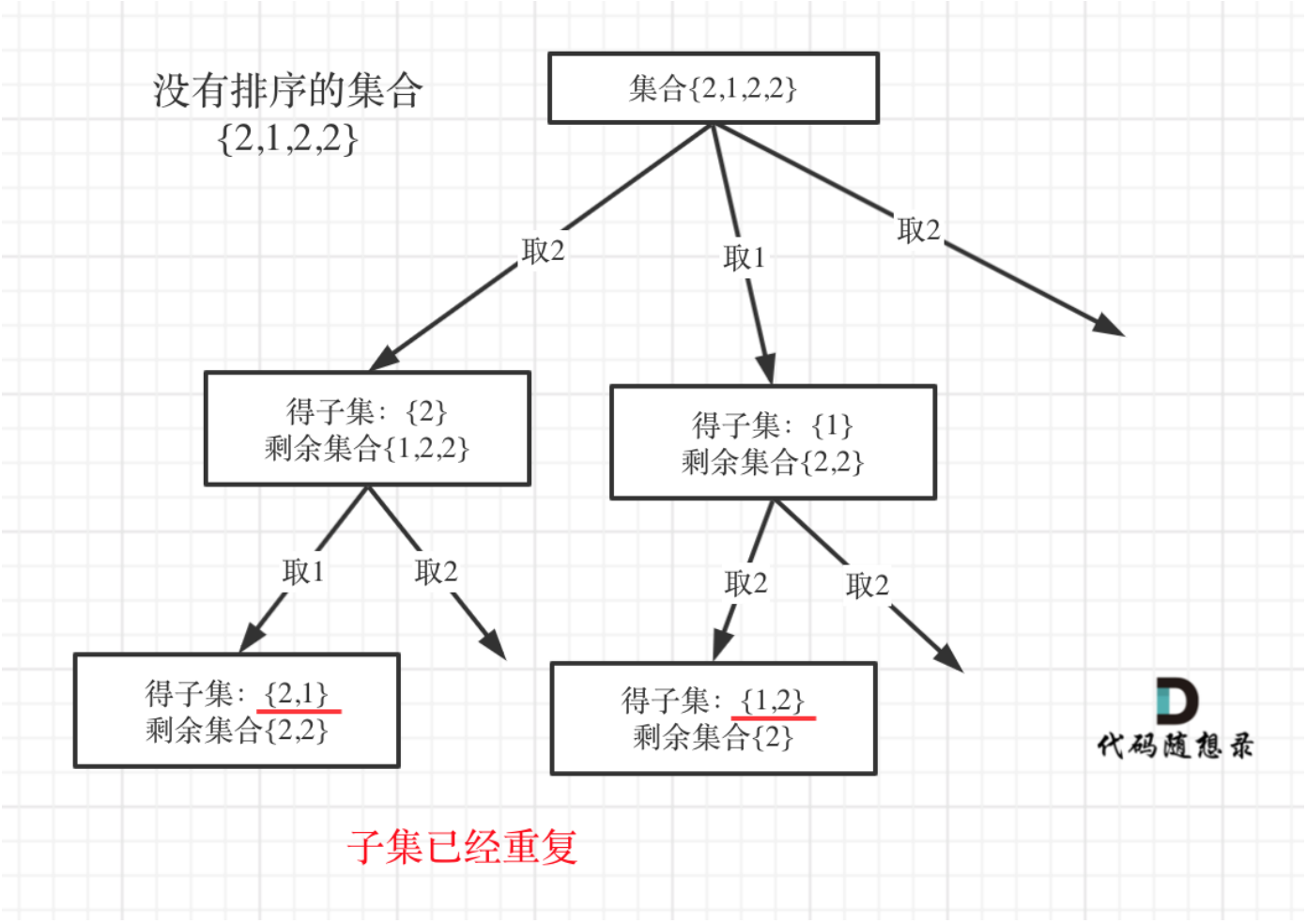
> 在 本周小结！（回溯算法系列三） 中一位录友对 整棵树的本层和同一节点的本层有疑问，也让我重新思考了一下，发现这里确实有问题，所以专门写一篇来纠正，感谢录友们的积极交流哈！

接下来我再把这块再讲一下。

在回溯算法：求子集问题（二）中的去重和 回溯算法：递增子序列中的去重 都是 同一父节点下本层的去重。

回溯算法：求子集问题（二）也可以使用set针对同一父节点本层去重，但子集问题一定要排序，为什么呢?

我用没有排序的集合{2,1,2,2}来举例子画一个图，如图：



图中，大家就很明显的看到，子集重复了。

那么下面我针对回溯算法：求子集问题（二） 给出使用set来对本层去重的代码实现。

子集II

used数组去重版本： 回溯算法：求子集问题（二）

使用set去重的版本如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex, vector<bool>& used) {
        result.push_back(path);
        unordered_set<int> uset; // 定义set对同一节点下的本层去重
        for (int i = startIndex; i < nums.size(); i++) {
            if (uset.find(nums[i]) != uset.end()) { // 如果发现出现过就pass
                continue;
            }
            uset.insert(nums[i]); // set跟新元素
            path.push_back(nums[i]);
            backtracking(nums, i + 1, used);
            path.pop_back();
```

```
        }
    }

public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        result.clear();
        path.clear();
        vector<bool> used(nums.size(), false);
        sort(nums.begin(), nums.end()); // 去重需要排序
        backtracking(nums, 0, used);
        return result;
    }
};
```

针对留言区录友们的疑问，我再补充一些常见的错误写法，

错误写法一

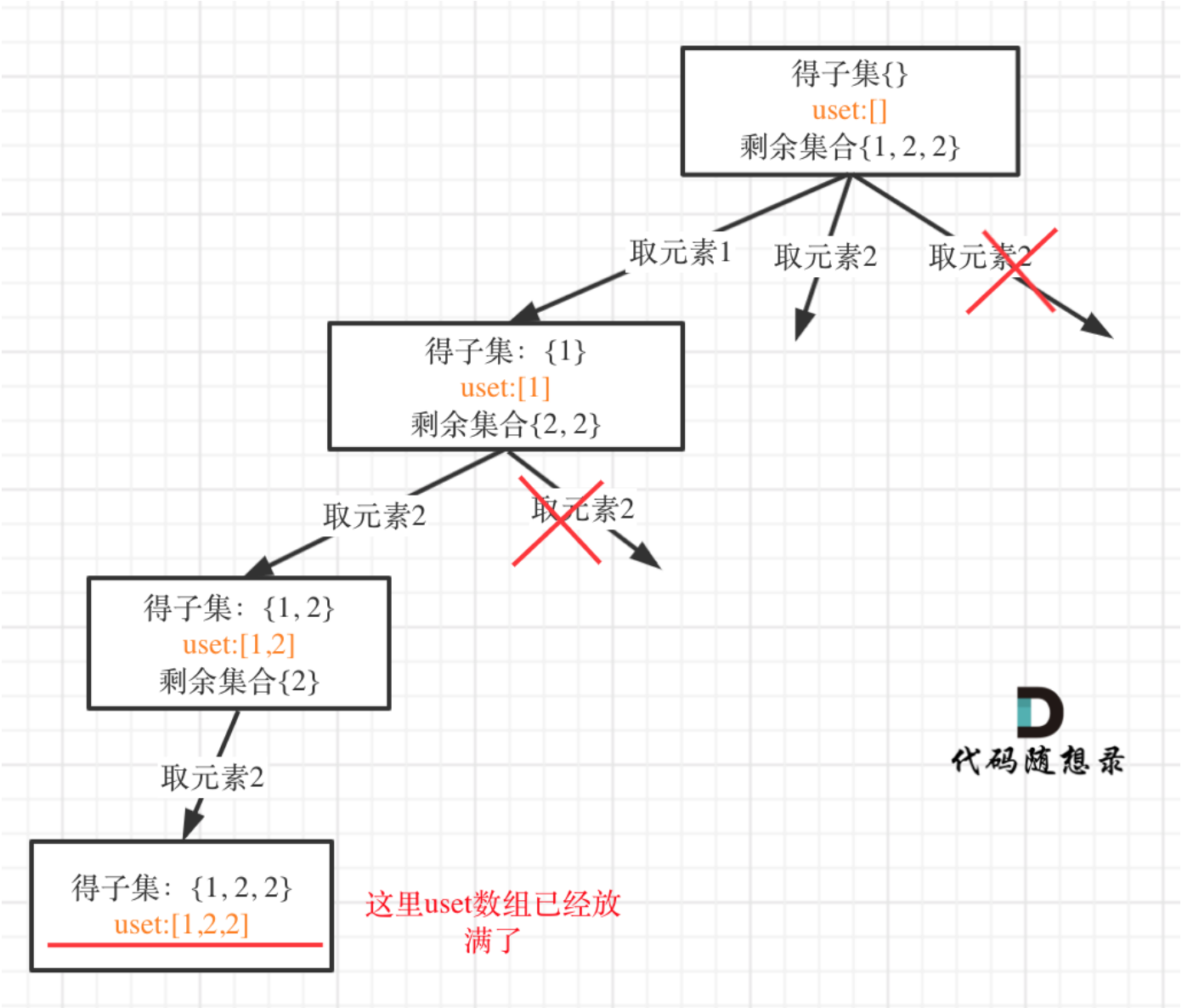
把uset定义放到类成员位置，然后模拟回溯的样子 insert一次，erase一次。

例如：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    unordered_set<int> uset; // 把uset定义放到类成员位置
    void backtracking(vector<int>& nums, int startIndex, vector<bool>& used) {
        result.push_back(path);

        for (int i = startIndex; i < nums.size(); i++) {
            if (uset.find(nums[i]) != uset.end()) {
                continue;
            }
            uset.insert(nums[i]);   // 递归之前insert
            path.push_back(nums[i]);
            backtracking(nums, i + 1, used);
            path.pop_back();
            uset.erase(nums[i]);    // 回溯再erase
        }
    }
```

在树形结构中，**如果把unordered_set uset放在类成员的位置（相当于全局变量），就把树枝的情况都记录了，不是单纯的控制某一节点下的同一层了。**

如图：



可以看出一旦把unordered_set uset放在类成员位置，它控制的就是整棵树，包括树枝。

所以这么写不行！

错误写法二

有同学把 unordered_set uset; 放到类成员位置，然后每次进入单层的时候用uset.clear()。

代码如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    unordered_set<int> uset; // 把uset定义放到类成员位置
    void backtracking(vector<int>& nums, int startIndex, vector<bool>& used) {
        result.push_back(path);
        uset.clear(); // 到每一层的时候，清空uset
        for (int i = startIndex; i < nums.size(); i++) {
            if (uset.find(nums[i]) != uset.end()) {
```

```
                continue;
            }
            uset.insert(nums[i]); // set记录元素
            path.push_back(nums[i]);
            backtracking(nums, i + 1, used);
            path.pop_back();
        }
    }
```

uset已经是全局变量，本层的uset记录了一个元素，然后进入下一层之后这个uset（和上一层是同一个uset）就被清空了，也就是说，层与层之间的uset是同一个，那么就会相互影响。

所以这么写依然不行！

组合问题和排列问题，其实也可以使用set来对同一节点下本层去重，下面我都分别给出实现代码。

组合总和 II

使用used数组去重版本：回溯算法：求组合总和（三）

使用set去重的版本如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex) {
        if (sum == target) {
            result.push_back(path);
            return;
        }
        unordered_set<int> uset; // 控制某一节点下的同一层元素不能重复
        for (int i = startIndex; i < candidates.size() && sum + candidates[i] <=
target; i++) {
            if (uset.find(candidates[i]) != uset.end()) {
                continue;
            }
            uset.insert(candidates[i]); // 记录元素
            sum += candidates[i];
            path.push_back(candidates[i]);
            backtracking(candidates, target, sum, i + 1);
            sum -= candidates[i];
            path.pop_back();
        }
    }

public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        path.clear();
        result.clear();
```

```
        sort(candidates.begin(), candidates.end());
        backtracking(candidates, target, 0, 0);
        return result;
    }
};
```

全排列 II

使用used数组去重版本：回溯算法：排列问题（二）

使用set去重的版本如下：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking (vector<int>& nums, vector<bool>& used) {
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }
        unordered_set<int> uset; // 控制某一节点下的同一层元素不能重复
        for (int i = 0; i < nums.size(); i++) {
            if (uset.find(nums[i]) != uset.end()) {
                continue;
            }
            if (used[i] == false) {
                uset.insert(nums[i]); // 记录元素
                used[i] = true;
                path.push_back(nums[i]);
                backtracking(nums, used);
                path.pop_back();
                used[i] = false;
            }
        }
    }
public:
    vector<vector<int>> permuteUnique(vector<int>& nums) {
        result.clear();
        path.clear();
        sort(nums.begin(), nums.end()); // 排序
        vector<bool> used(nums.size(), false);
        backtracking(nums, used);
        return result;
    }
};
```

两种写法的性能分析

需要注意的是：**使用set去重的版本相对于used数组的版本效率都要低很多**，大家在leetcode上提交，能明显发现。

原因在回溯算法：递增子序列中也分析过，主要是因为程序运行的时候对unordered_set 频繁的insert，unordered_set需要做哈希映射（也就是把key通过hash function映射为唯一的哈希值）相对费时间，而且insert的时候其底层的符号表也要做相应的扩充，也是费时的。

而使用used数组在时间复杂度上几乎没有额外负担！

使用set去重，不仅时间复杂度高了，空间复杂度也高了，在本周小结！（回溯算法系列三）中分析过，组合，子集，排列问题的空间复杂度都是O(n)，但如果使用set去重，空间复杂度就变成了O(n^2)，因为每一层递归都有一个set集合，系统栈空间是n，每一个空间都有set集合。

那有同学可能疑惑 用used数组也是占用O(n)的空间啊?

used数组可是全局变量，每层与每层之间公用一个used数组，所以空间复杂度是O(n + n)，最终空间复杂度还是O(n)。

总结

本篇本打算是对本周小结！（回溯算法系列三）的一个点做一下纠正，没想到又写出来这么多！

这个点都源于一位录友的疑问，然后我思考总结了一下，就写着这一篇，所以还是得多交流啊！

如果大家对「代码随想录」文章有什么疑问，尽管打卡留言的时候提出来哈，或者在交流群里提问。

其实这就是相互学习的过程，交流一波之后都对题目理解的更深刻了，我如果发现文中有问题，都会在评论区或者下一篇文章中即时修正，保证不会给大家带跑偏！

> 这也可以用回溯法？ 其实深搜和回溯也是相辅相成的，毕竟都用递归。

19.重新安排行程

力扣题目链接

给定一个机票的字符串二维数组 [from, to]，子数组中的两个成员分别表示飞机出发和降落的机场地点，对该行程进行重新规划排序。所有这些机票都属于一个从 JFK（肯尼迪国际机场）出发的先生，所以该行程必须从 JFK 开始。

提示：

- 如果存在多种有效的行程，请你按字符自然排序返回最小的行程组合。例如，行程 ["JFK", "LGA"] 与 ["JFK", "LGB"] 相比就更小，排序更靠前
- 所有的机场都用三个大写字母表示（机场代码）。
- 假定所有机票至少存在一种合理的行程。
- 所有的机票必须都用一次 且 只能用一次。

示例 1：

- 输入：[["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]
- 输出：["JFK", "MUC", "LHR", "SFO", "SJC"]

示例 2：

- 输入：[["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]
- 输出：["JFK","ATL","JFK","SFO","ATL","SFO"]
- 解释：另一种有效的行程是 ["JFK","SFO","ATL","JFK","ATL","SFO"]。但是它自然排序更大更靠后。

算法公开课

《代码随想录》算法视频公开课：带你学透回溯算法（理论篇），相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目还是很难的，之前我们用回溯法解决了如下问题：组合问题，分割问题，子集问题，排列问题。

直觉上来看 这道题和回溯法没有什么关系，更像是图论中的深度优先搜索。

实际上确实是深搜，但这是深搜中使用了回溯的例子，在查找路径的时候，如果不回溯，怎么能查到目标路径呢。

所以我倾向于说本题应该使用回溯法，那么我也用回溯法的思路来讲解本题，其实深搜一般都使用了回溯法的思路，在图论系列中我会再详细讲解深搜。

这里就是先给大家拓展一下，原来回溯法还可以这么玩！

这道题目有几个难点：

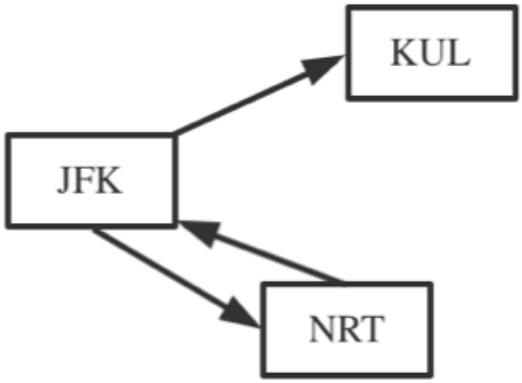
1. 一个行程中，如果航班处理不好容易变成一个圈，成为死循环
2. 有多种解法，字母序靠前排在前面，让很多同学望而退步，如何该记录映射关系呢？
3. 使用回溯法（也可以说深搜） 的话，那么终止条件是什么呢?
4. 搜索的过程中，如何遍历一个机场所对应的所有机场。

针对以上问题我来逐一解答！

如何理解死循环

对于死循环，我来举一个有重复机场的例子：

航班信息：[["JFK","KUL"],["JFK","NRT"],["NRT","JFK"]]



输出：["JFK","NRT","JFK","KUL"]

为什么要举这个例子呢，就是告诉大家，出发机场和到达机场也会重复的，**如果在解题的过程中没有对集合元素处理好，就会死循环。**

该记录映射关系

有多种解法，字母序靠前排在前面，让很多同学望而退步，如何该记录映射关系呢？

一个机场映射多个机场，机场之间要靠字母序排列，一个机场映射多个机场，可以使用std::unordered_map，如果让多个机场之间再有顺序的话，就是用std::map 或者std::multimap 或者 std::multiset。

如果对map 和 set 的实现机制不太了解，也不清楚为什么 map、multimap就是有序的同学，可以看这篇文章关于哈希表，你该了解这些！。

这样存放映射关系可以定义为 `unordered_map<string, multiset<string>> targets` 或者 `unordered_map<string, map<string, int>> targets`。

含义如下：

unordered_map<string, multiset> targets：unordered_map<出发机场, 到达机场的集合> targets

unordered_map<string, map<string, int>> targets：unordered_map<出发机场, map<到达机场, 航班次数>> targets

这两个结构，我选择了后者，因为如果使用`unordered_map<string, multiset<string>> targets`遍历multiset的时候，不能删除元素，一旦删除元素，迭代器就失效了。

再说一下为什么一定要增删元素呢，正如开篇我给出的图中所示，出发机场和到达机场是会重复的，搜索的过程没及时删除目的机场就会死循环。

所以搜索的过程中就是要不断的删multiset里的元素，那么推荐使用`unordered_map<string, map<string, int>> targets`。

在遍历`unordered_map<出发机场, map<到达机场, 航班次数>> targets`的过程中，**可以使用"航班次数"这个字段的数字做相应的增减，来标记到达机场是否使用过了。**

如果“航班次数”大于零，说明目的地还可以飞，如果“航班次数”等于零说明目的地不能飞了，而不用对集合做删除元素或者增加元素的操作。

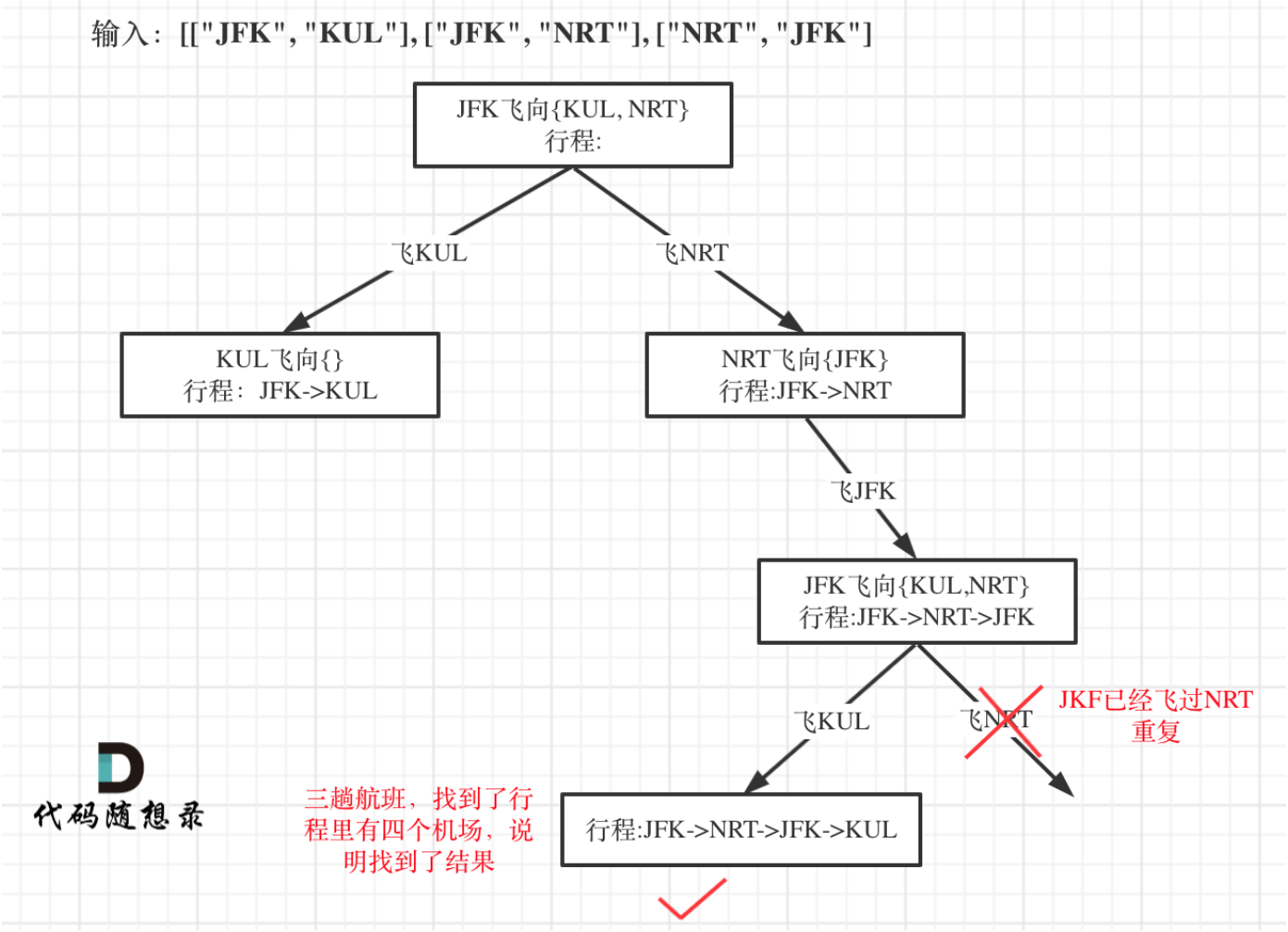
相当于说我不删，我就做一个标记！

回溯法

这道题目我使用回溯法，那么下面按照我总结的回溯模板来：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

本题以输入：[["JFK", "KUL"], ["JFK", "NRT"], ["NRT", "JFK"]为例，抽象为树形结构如下：



开始回溯三部曲讲解：

- 递归函数参数

在讲解映射关系的时候，已经讲过了，使用`unordered_map<string, map<string, int>> targets;`来记录航班的映射关系，我定义为全局变量。

当然把参数放进函数里传进去也是可以的，我是尽量控制函数里参数的长度。

参数里还需要ticketNum，表示有多少个航班（终止条件会用上）。

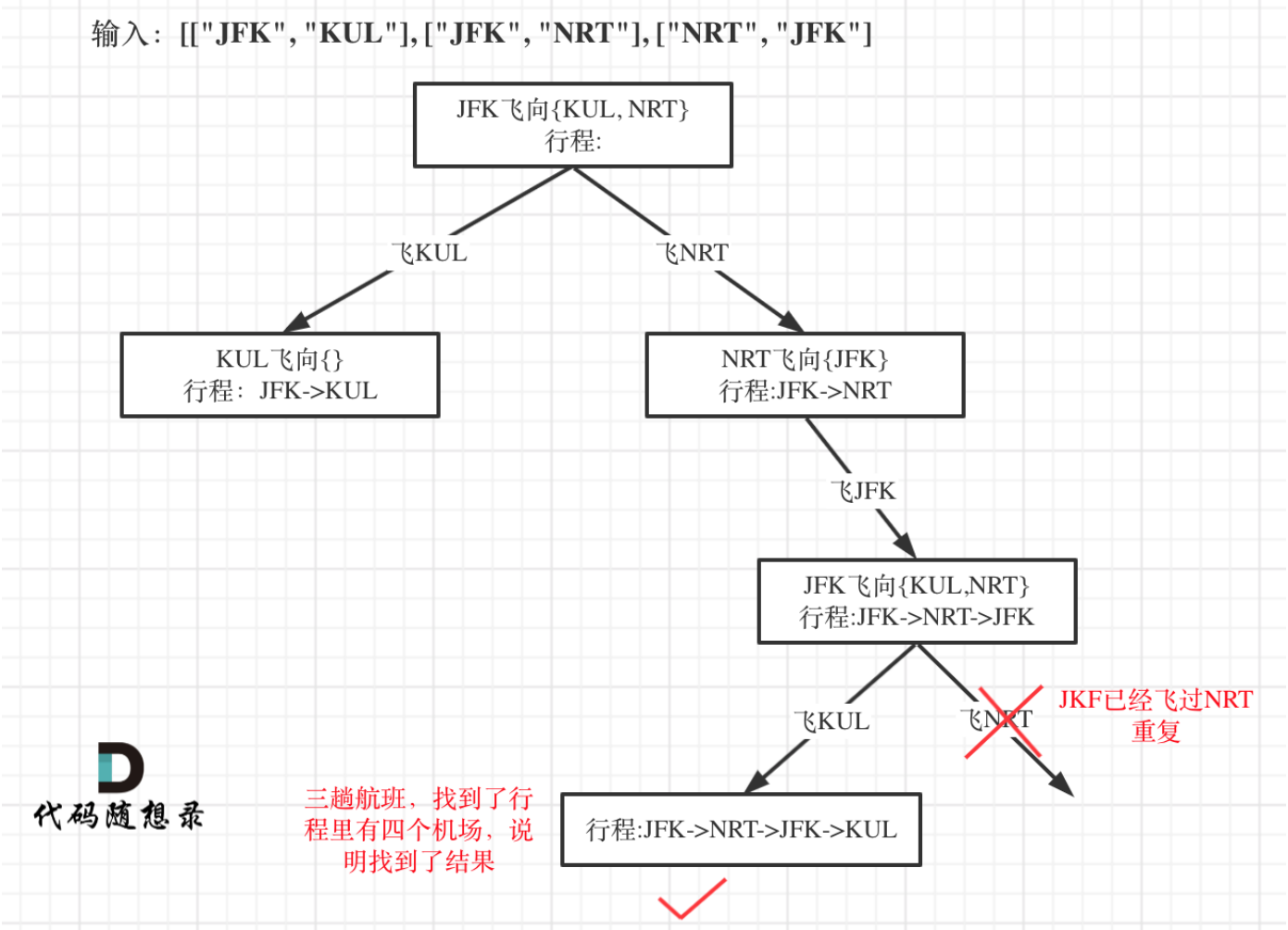
代码如下：

```
// unordered_map<出发机场, map<到达机场, 航班次数>> targets
unordered_map<string, map<string, int>> targets;
bool backtracking(int ticketNum, vector<string>& result) {
```

注意函数返回值我用的是bool！

我们之前讲解回溯算法的时候，一般函数返回值都是void，这次为什么是bool呢？

因为我们只需要找到一个行程，就是在树形结构中唯一的一条通向叶子节点的路线，如图：



所以找到了这个叶子节点了直接返回，这个递归函数的返回值问题我们在讲解二叉树的系列的时候，在这篇二叉树：递归函数究竟什么时候需要返回值，什么时候不要返回值？详细介绍过。

当然本题的targets和result都需要初始化，代码如下：

```
for (const vector<string>& vec : tickets) {
    targets[vec[0]][vec[1]]++; // 记录映射关系
}
result.push_back("JFK"); // 起始机场
```

- 递归终止条件

拿题目中的示例为例，输入: [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]] ，这是有4个航班，那么只要找出一种行程，行程里的机场个数是5就可以了。

所以终止条件是：我们回溯遍历的过程中，遇到的机场个数，如果达到了（航班数量+1），那么我们就找到了一个行程，把所有航班串在一起了。

代码如下：

```
if (result.size() == ticketNum + 1) {
    return true;
}
```

已经看习惯回溯法代码的同学，到叶子节点了习惯性的想要收集结果，但发现并不需要，本题的result相当于 回溯算法：求组合总和！ 中的path，也就是本题的result就是记录路径的（就一条），在如下单层搜索的逻辑中result就添加元素了。

- 单层搜索的逻辑

回溯的过程中，如何遍历一个机场所对应的所有机场呢?

这里刚刚说过，在选择映射函数的时候，不能选择`unordered_map<string, multiset<string>> targets`，因为一旦有元素增删multiset的迭代器就会失效，当然可能有牛逼的容器删除元素迭代器不会失效，这里就不在讨论了。

可以说本题既要找到一个对数据进行排序的容器，而且还要容易增删元素，迭代器还不能失效。

所以我选择了`unordered_map<string, map<string, int>> targets` 来做机场之间的映射。

遍历过程如下：

```
for (pair<const string, int>& target : targets[result[result.size() - 1]]) {
    if (target.second > 0 ) { // 记录到达机场是否飞过了
        result.push_back(target.first);
        target.second--;
        if (backtracking(ticketNum, result)) return true;
        result.pop_back();
        target.second++;
    }
}
```

可以看出 通过`unordered_map<string, map<string, int>> targets`里的int字段来判断 这个集合里的机场是否使用过，这样避免了直接去删元素。

分析完毕，此时完整C++代码如下：

```
class Solution {
private:
// unordered_map<出发机场, map<到达机场, 航班次数>> targets
unordered_map<string, map<string, int>> targets;
bool backtracking(int ticketNum, vector<string>& result) {
    if (result.size() == ticketNum + 1) {
        return true;
    }
    for (pair<const string, int>& target : targets[result[result.size() - 1]]) {
        if (target.second > 0 ) { // 记录到达机场是否飞过了
            result.push_back(target.first);
            target.second--;
            if (backtracking(ticketNum, result)) return true;
            result.pop_back();
            target.second++;
        }
    }
    return false;
```

```
}
public:
    vector<string> findItinerary(vector<vector<string>>& tickets) {
        targets.clear();
        vector<string> result;
        for (const vector<string>& vec : tickets) {
            targets[vec[0]][vec[1]]++; // 记录映射关系
        }
        result.push_back("JFK"); // 起始机场
        backtracking(tickets.size(), result);
        return result;
    }
};
```

一波分析之后，可以看出我就是按照回溯算法的模板来的。

代码中

```
for (pair<const string, int>& target : targets[result[result.size() - 1]])
```

pair里要有const，因为map中的key是不可修改的，所以是`pair<const string, int>`。

如果不加const，也可以复制一份pair，例如这么写：

```
for (pair<string, int>target : targets[result[result.size() - 1]])
```

总结

本题其实可以算是一道hard的题目了，关于本题的难点我在文中已经列出了。

如果单纯的回溯搜索（深搜）并不难，难还难在容器的选择和使用上。

本题其实是一道深度优先搜索的题目，但是我完全使用回溯法的思路来讲解这道题题目，**算是给大家拓展一下思维方式，其实深搜和回溯也是分不开的，毕竟最终都是用递归。**

如果最终代码，发现照着回溯法模板画的话好像也能画出来，但难就难如何知道可以使用回溯，以及如果套进去，所以我再写了这么长的一篇来详细讲解。

就酱，很多录友表示和「代码随想录」相见恨晚，那么帮Carl宣传一波吧，让更多同学知道这里！

---

20. N皇后

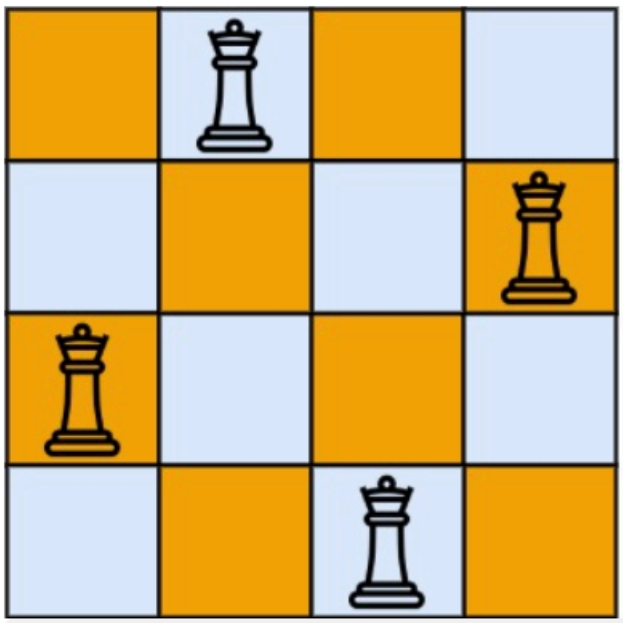
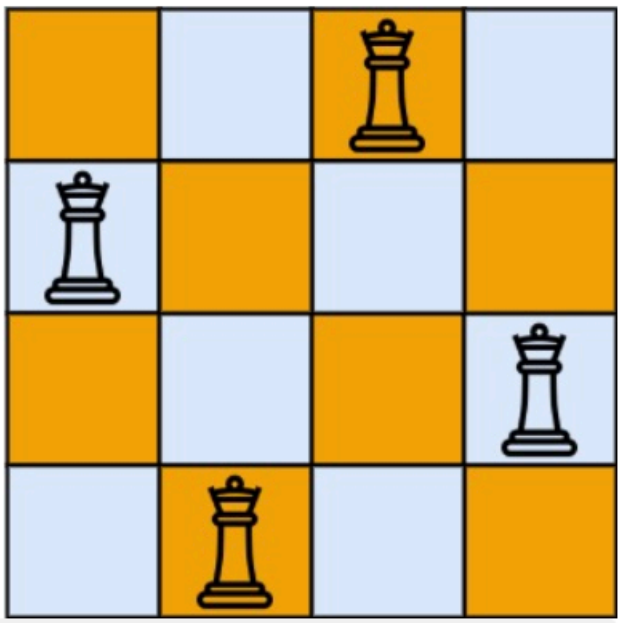
[力扣题目链接]

n 皇后问题 研究的是如何将 n 个皇后放置在 n×n 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 n ，返回所有不同的 n 皇后问题 的解决方案。

每一种解法包含一个不同的 n 皇后问题 的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1：

- 输入：n = 4
- 输出：[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
- 解释：如上图所示，4 皇后问题存在两个不同的解法。

示例 2：

- 输入：n = 1
- 输出：[["Q"]]

算法公开课

[《代码随想录》算法视频公开课](#)：[这就是传说中的N皇后？ 回溯算法安排！ | LeetCode：51.N皇后](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

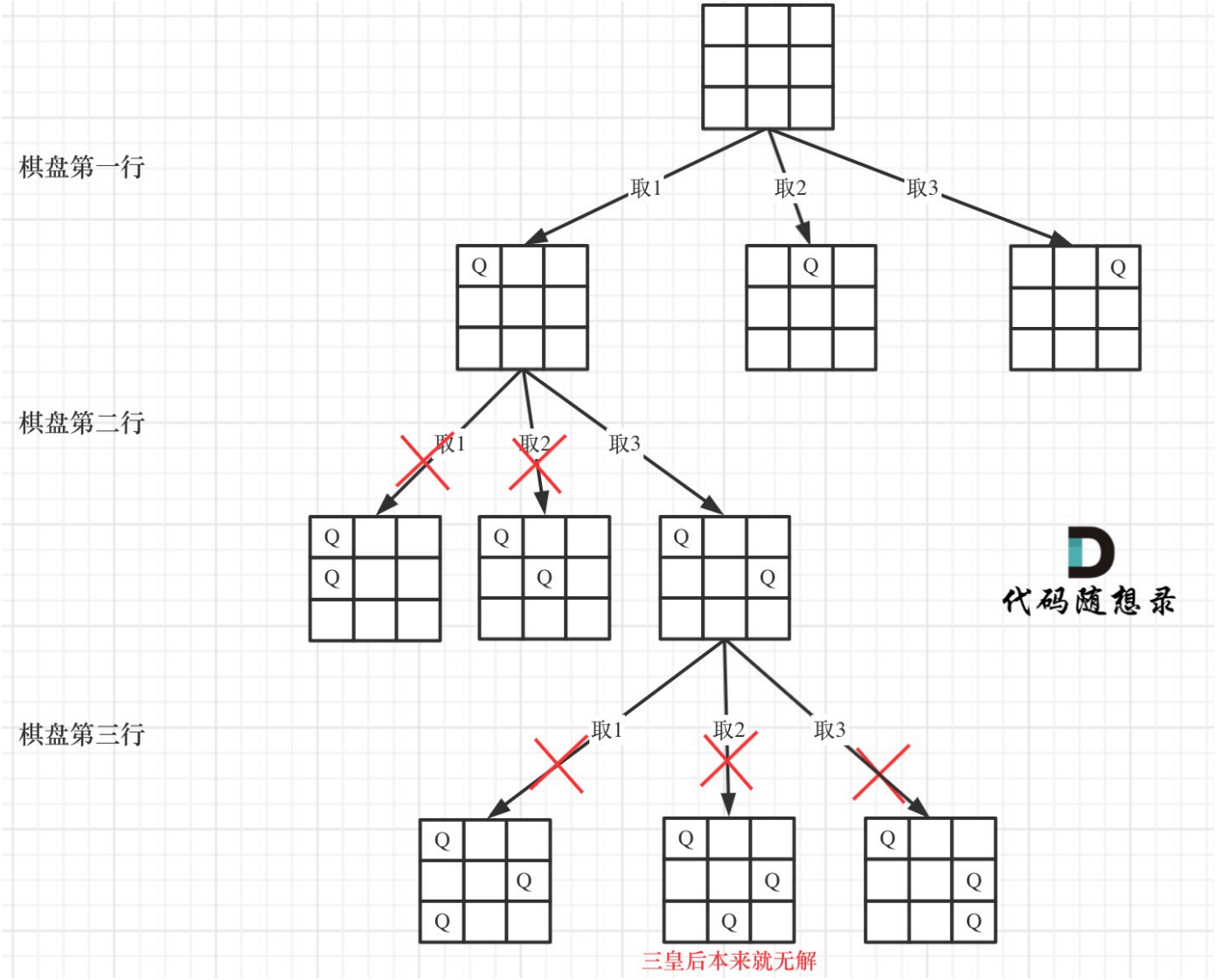
都知道n皇后问题是回溯算法解决的经典问题，但是用回溯解决多了组合、切割、子集、排列问题之后，遇到这种二维矩阵还会有点不知所措。

首先来看一下皇后们的约束条件：

1. 不能同行
2. 不能同列
3. 不能同斜线

确定完约束条件，来看看究竟要怎么去搜索皇后们的位置，其实搜索皇后的位置，可以抽象为一棵树。

下面我用一个 3 * 3 的棋盘，将搜索过程抽象为一棵树，如图：



从图中，可以看出，二维矩阵中矩阵的高就是这棵树的高度，矩阵的宽就是树形结构中每一个节点的宽度。

那么我们用皇后们的约束条件，来回溯搜索这棵树，**只要搜索到了树的叶子节点，说明就找到了皇后们的合理位置了。**

回溯三部曲

按照我总结的如下回溯模板，我们来依次分析：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }
    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

- 递归函数参数

我依然是定义全局变量二维数组result来记录最终结果。

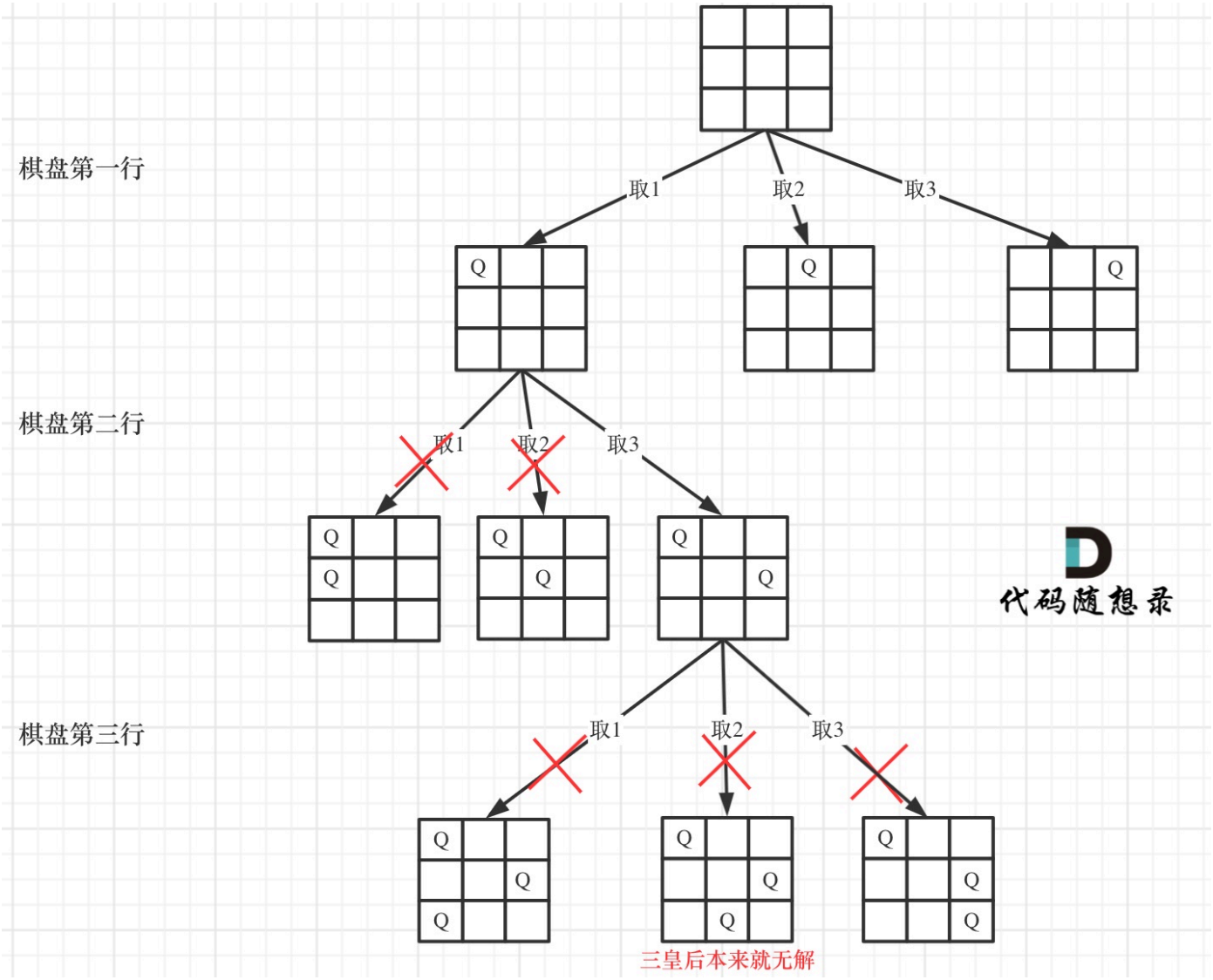
参数n是棋盘的大小，然后用row来记录当前遍历到棋盘的第几层了。

代码如下：

```
vector<vector<string>> result;
void backtracking(int n, int row, vector<string>& chessboard) {
```

- 递归终止条件

在如下树形结构中：



可以看出，当递归到棋盘最底层（也就是叶子节点）的时候，就可以收集结果并返回了。

代码如下：

```
if (row == n) {
    result.push_back(chessboard);
    return;
}
```

- 单层搜索的逻辑

递归深度就是row控制棋盘的行，每一层里for循环的col控制棋盘的列，一行一列，确定了放置皇后的位置。

每次都是要从新的一行的起始位置开始搜，所以都是从0开始。

代码如下：

```
for (int col = 0; col < n; col++) {
    if (isValid(row, col, chessboard, n)) { // 验证合法就可以放
        chessboard[row][col] = 'Q'; // 放置皇后
        backtracking(n, row + 1, chessboard);
        chessboard[row][col] = '.'; // 回溯，撤销皇后
    }
}
```

- 验证棋盘是否合法

按照如下标准去重：

1. 不能同行
2. 不能同列
3. 不能同斜线 （45度和135度角）

代码如下：

```
bool isValid(int row, int col, vector<string>& chessboard, int n) {
    // 检查列
    for (int i = 0; i < row; i++) { // 这是一个剪枝
        if (chessboard[i][col] == 'Q') {
            return false;
        }
    }
    // 检查 45度角是否有皇后
    for (int i = row - 1, j = col - 1; i >=0 && j >= 0; i--, j--) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    // 检查 135度角是否有皇后
    for(int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    return true;
}
```

在这份代码中，细心的同学可以发现为什么没有在同行进行检查呢?

因为在单层搜索的过程中，每一层递归，只会选for循环（也就是同一行）里的一个元素，所以不用去重了。

那么按照这个模板不难写出如下C++代码：

```
class Solution {
private:
vector<vector<string>> result;
// n 为输入的棋盘大小
// row 是当前递归到棋盘的第几行了
void backtracking(int n, int row, vector<string>& chessboard) {
    if (row == n) {
        result.push_back(chessboard);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isValid(row, col, chessboard, n)) { // 验证合法就可以放
            chessboard[row][col] = 'Q'; // 放置皇后
            backtracking(n, row + 1, chessboard);
            chessboard[row][col] = '.'; // 回溯，撤销皇后
        }
    }
}
bool isValid(int row, int col, vector<string>& chessboard, int n) {
    // 检查列
    for (int i = 0; i < row; i++) { // 这是一个剪枝
        if (chessboard[i][col] == 'Q') {
            return false;
        }
    }
    // 检查 45度角是否有皇后
    for (int i = row - 1, j = col - 1; i >=0 && j >= 0; i--, j--) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    // 检查 135度角是否有皇后
    for(int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    return true;
}
public:
    vector<vector<string>> solveNQueens(int n) {
        result.clear();
        std::vector<std::string> chessboard(n, std::string(n, '.'));
        backtracking(n, 0, chessboard);
        return result;
```

```
    }
};
```

- 时间复杂度: O(n!)
- 空间复杂度: O(n)

可以看出，除了验证棋盘合法性的代码，省下来部分就是按照回溯法模板来的。

总结

本题是我们解决棋盘问题的第一道题目。

如果从来没有接触过N皇后问题的同学看着这样的题会感觉无从下手，可能知道要用回溯法，但也不知道该怎么去搜。

这里我明确给出了棋盘的宽度就是for循环的长度，递归的深度就是棋盘的高度，这样就可以套进回溯法的模板里了。

大家可以在仔细体会体会！

> 如果对回溯法理论还不清楚的同学，可以先看这个视频视频来了！！带你学透回溯算法（理论篇）

21. 解数独

力扣题目链接

编写一个程序，通过填充空格来解决数独问题。

一个数独的解法需遵循如下规则：
数字 1-9 在每一行只能出现一次。
数字 1-9 在每一列只能出现一次。
数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。
空白格用 '.' 表示。

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

一个数独。

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

答案被标成红色。

提示：

- 给定的数独序列只包含数字 1-9 和字符 '.' 。
- 你可以假设给定的数独只有唯一解。
- 给定数独永远是 9x9 形式的。

算法公开课

[《代码随想录》算法视频公开课](#)：[回溯算法二维递归？解数独不过如此！ | LeetCode：37. 解数独](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

棋盘搜索问题可以使用回溯法暴力搜索，只不过这次我们要做的是**二维递归**。

怎么做二维递归呢？

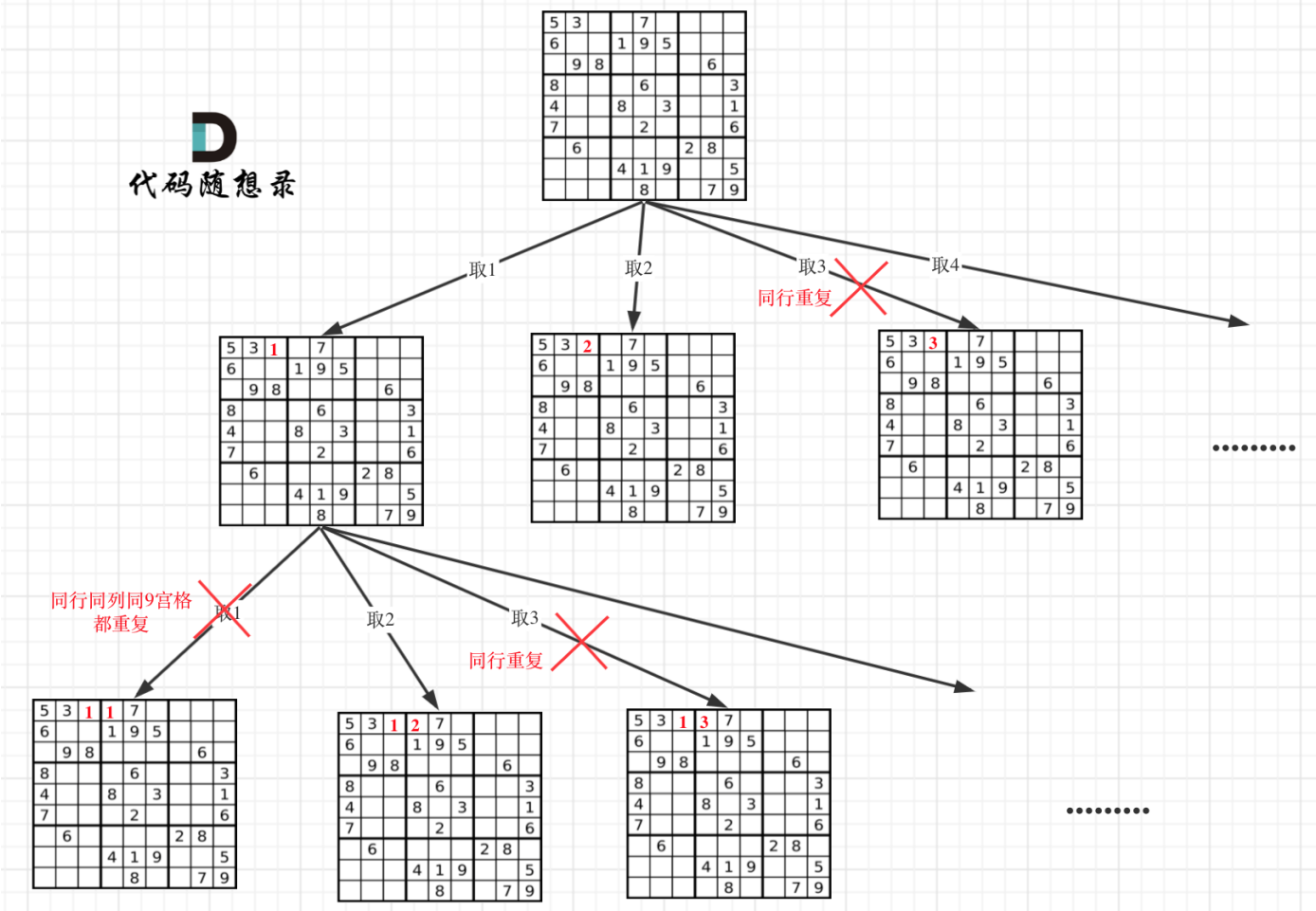
大家已经跟着「代码随想录」刷过了如下回溯法题目，例如：[77.组合（组合问题）](#)，[131.分割回文串（分割问题）](#)，[78.子集（子集问题）](#)，[46.全排列（排列问题）](#)，以及[51.N皇后（N皇后问题）](#)，其实这些题目都是一维递归。

如果以上这几道题目没有做过的话，不建议上来就做这道题哈！

[N皇后问题](#)是因为每一行每一列只放一个皇后，只需要一层for循环遍历一行，递归来遍历列，然后一行一列确定皇后的唯一位置。

本题就不一样了，**本题中棋盘的每一个位置都要放一个数字（而N皇后是一行只放一个皇后），并检查数字是否合法，解数独的树形结构要比N皇后更宽更深。**

因为这个树形结构太大了，我抽取一部分，如图所示：



回溯三部曲

- 递归函数以及参数

递归函数的返回值需要是bool类型，为什么呢?

因为解数独找到一个符合的条件（就在树的叶子节点上）立刻就返回，相当于找从根节点到叶子节点一条唯一路径，所以需要使用bool返回值。

代码如下：

```
bool backtracking(vector<vector<char>>& board)
```

- 递归终止条件

本题递归不用终止条件，解数独是要遍历整个树形结构寻找可能的叶子节点就立刻返回。

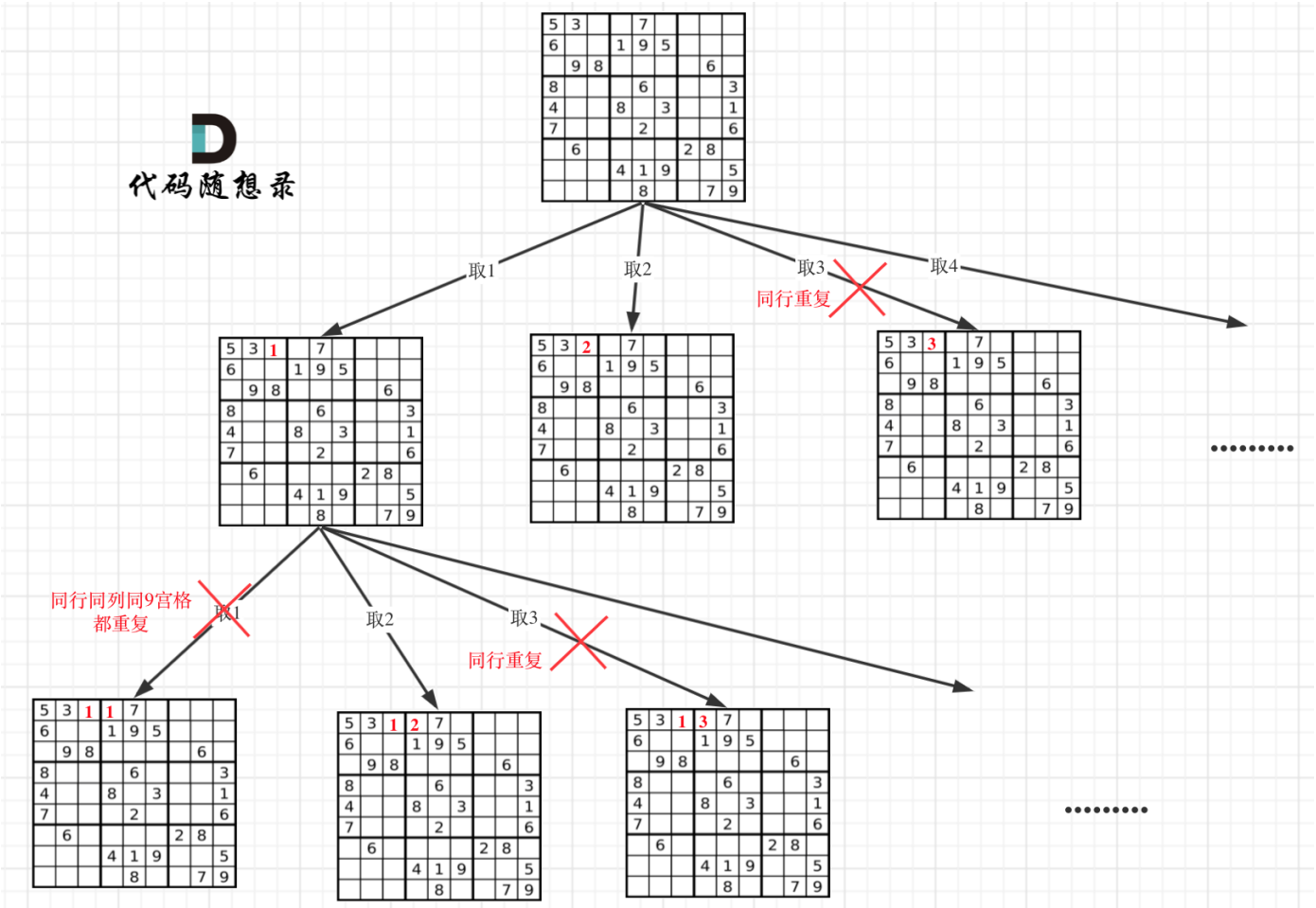
不用终止条件会不会死循环?

递归的下一层的棋盘一定比上一层的棋盘多一个数，等数填满了棋盘自然就终止（填满当然好了，说明找到结果了），所以不需要终止条件!

那么有没有永远填不满的情况呢?

这个问题我在递归单层搜索逻辑里再来讲!

- 递归单层搜索逻辑



在树形图中可以看出我们需要的是一个二维的递归（也就是两个for循环嵌套着递归）

一个for循环遍历棋盘的行，一个for循环遍历棋盘的列，一行一列确定下来之后，递归遍历这个位置放9个数字的可能性！

代码如下：（**详细看注释**）

```
bool backtracking(vector<vector<char>>& board) {
    for (int i = 0; i < board.size(); i++) {         // 遍历行
        for (int j = 0; j < board[0].size(); j++) { // 遍历列
            if (board[i][j] != '.') continue;
            for (char k = '1'; k <= '9'; k++) {      // (i, j) 这个位置放k是否合适
                if (isValid(i, j, k, board)) {
                    board[i][j] = k;                 // 放置k
                    if (backtracking(board)) return true; // 如果找到合适一组立刻返回
                    board[i][j] = '.';               // 回溯，撤销k
                }
            }
            return false;                            // 9个数都试完了，都不行，那么就返回false
        }
    }
    return true; // 遍历完没有返回false，说明找到了合适棋盘位置了
}
```

注意这里return false的地方，这里放return false 是有讲究的。

因为如果一行一列确定下来了，这里尝试了9个数都不行，说明这个棋盘找不到解决数独问题的解！

那么会直接返回， **这也就是为什么没有终止条件也不会永远填不满棋盘而无限递归下去！**

判断棋盘是否合法

判断棋盘是否合法有如下三个维度：

- 同行是否重复
- 同列是否重复
- 9宫格里是否重复

代码如下：

```
bool isValid(int row, int col, char val, vector<vector<char>>& board) {
    for (int i = 0; i < 9; i++) { // 判断行里是否重复
        if (board[row][i] == val) {
            return false;
        }
    }
    for (int j = 0; j < 9; j++) { // 判断列里是否重复
        if (board[j][col] == val) {
            return false;
        }
```

```
    }
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;
    for (int i = startRow; i < startRow + 3; i++) { // 判断9方格里是否重复
        for (int j = startCol; j < startCol + 3; j++) {
            if (board[i][j] == val ) {
                return false;
            }
        }
    }
    return true;
}
```

最后整体C++代码如下：

```
class Solution {
private:
bool backtracking(vector<vector<char>>& board) {
    for (int i = 0; i < board.size(); i++) {        // 遍历行
        for (int j = 0; j < board[0].size(); j++) { // 遍历列
            if (board[i][j] == '.') {
                for (char k = '1'; k <= '9'; k++) {     // (i, j) 这个位置放k是否合适
                    if (isValid(i, j, k, board)) {
                        board[i][j] = k;                // 放置k
                        if (backtracking(board)) return true; // 如果找到合适一组立刻返回
                        board[i][j] = '.';              // 回溯，撤销k
                    }
                }
                return false;  // 9个数都试完了，都不行，那么就返回false
            }
        }
    }
    return true; // 遍历完没有返回false，说明找到了合适棋盘位置了
}
bool isValid(int row, int col, char val, vector<vector<char>>& board) {
    for (int i = 0; i < 9; i++) { // 判断行里是否重复
        if (board[row][i] == val) {
            return false;
        }
    }
    for (int j = 0; j < 9; j++) { // 判断列里是否重复
        if (board[j][col] == val) {
            return false;
        }
    }
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;
    for (int i = startRow; i < startRow + 3; i++) { // 判断9方格里是否重复
        for (int j = startCol; j < startCol + 3; j++) {
```

```
                if (board[i][j] == val ) {
                    return false;
                }
            }
        }
        return true;
    }
public:
    void solveSudoku(vector<vector<char>>& board) {
        backtracking(board);
    }
};
```

总结

解数独可以说是非常难的题目了，如果还一直停留在单层递归的逻辑中，这道题目可以让大家瞬间崩溃。

所以我在开篇就提到了**二维递归**，这也是我自创词汇，希望可以帮助大家理解解数独的搜索过程。

一波分析之后，再看代码会发现其实也不难，唯一难点就是理解**二维递归**的思维逻辑。

这样，解数独这么难的问题，也被我们攻克了。

恭喜一路上坚持打卡的录友们，回溯算法已经接近尾声了，接下来就是要一波总结了。

> 20张树形结构图、14道精选回溯题目，21篇回溯法精讲文章，由浅入深，一气呵成，这是全网最强回溯算法总结！

22. 回溯总结篇

回溯法理论基础

转眼间「代码随想录」里已经分享连续讲解了21天的回溯算法，是时候做一个大总结了，本篇高能，需要花费很大的精力来看！

关于回溯算法理论基础，我录了一期B站视频带你学透回溯算法（理论篇）如果对回溯算法还不了解的话，可以看一下。

在关于回溯算法，你该了解这些！中我们详细的介绍了回溯算法的理论知识，不同于教科书般的讲解，这里介绍的回溯法的效率，解决的问题以及模板都是在刷题的过程中非常实用！

回溯是递归的副产品，只要有递归就会有回溯，所以回溯法也经常和二叉树遍历，深度优先搜索混在一起，因为这两种方式都是用了递归。

回溯法就是暴力搜索，并不是什么高效的算法，最多再剪枝一下。

回溯算法能解决如下问题：

- 组合问题：N个数里面按一定规则找出k个数的集合

- 排列问题：N个数按一定规则全排列，有几种排列方式
- 切割问题：一个字符串按一定规则有几种切割方式
- 子集问题：一个N个数的集合里有多少符合条件的子集
- 棋盘问题：N皇后，解数独等等

我在回溯算法系列讲解中就按照这个顺序给大家讲解，可以说深入浅出，步步到位。

回溯法确实不好理解，所以需要把回溯法抽象为一个图形来理解就容易多了，**在后面的每一道回溯法的题目我都将遍历过程抽象为树形结构方便大家的理解。**

在关于回溯算法，你该了解这些！还用了回溯三部曲来分析回溯算法，并给出了回溯法的模板：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

事实证明这个模板会伴随整个回溯法系列！

组合问题

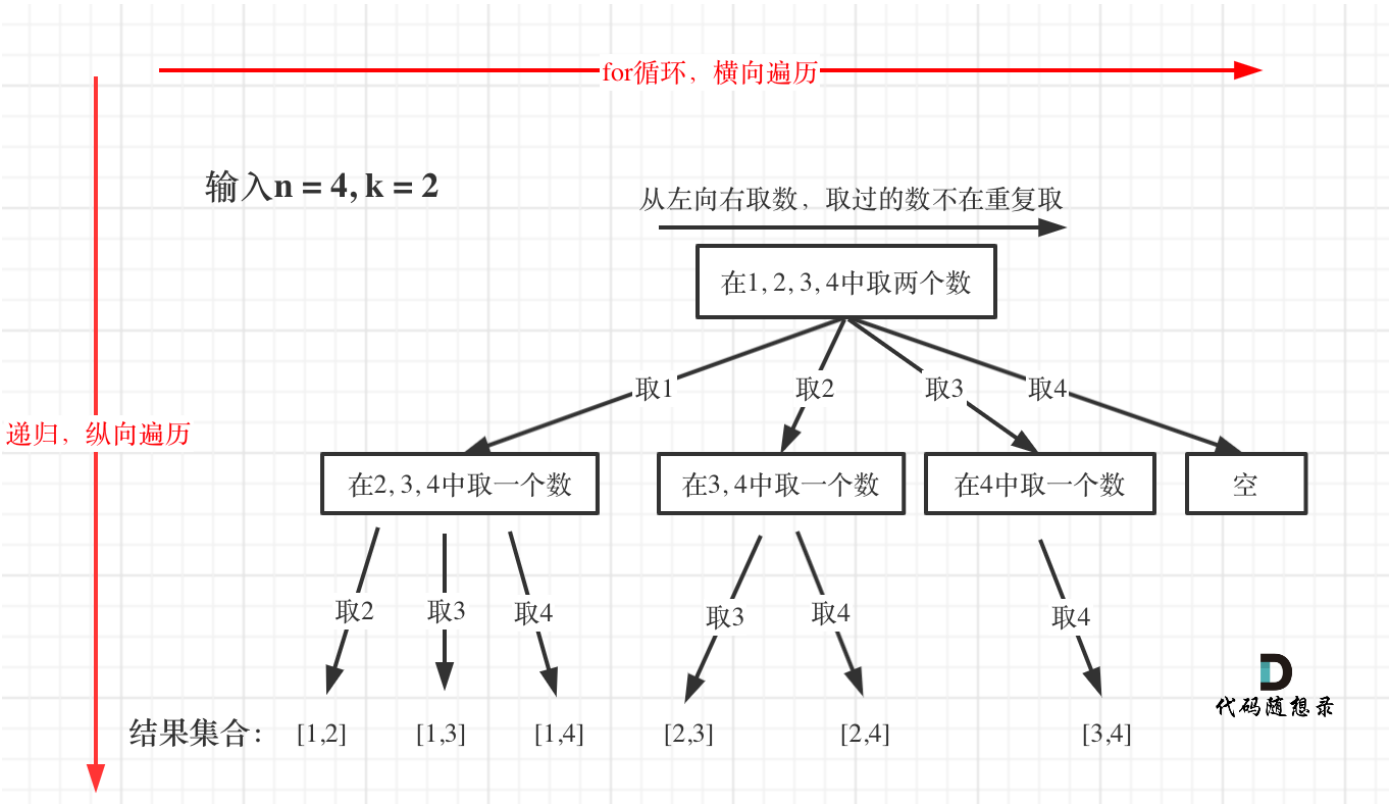
组合问题

在回溯算法：求组合问题！中，我们开始用回溯法解决第一道题目：组合问题。

我在文中开始的时候给大家列举k层for循环例子，进而得出都是同样是暴力解法，为什么要用回溯法！

此时大家应该深有体会回溯法的魅力，用递归控制for循环嵌套的数量！

本题我把回溯问题抽象为树形结构，如题：

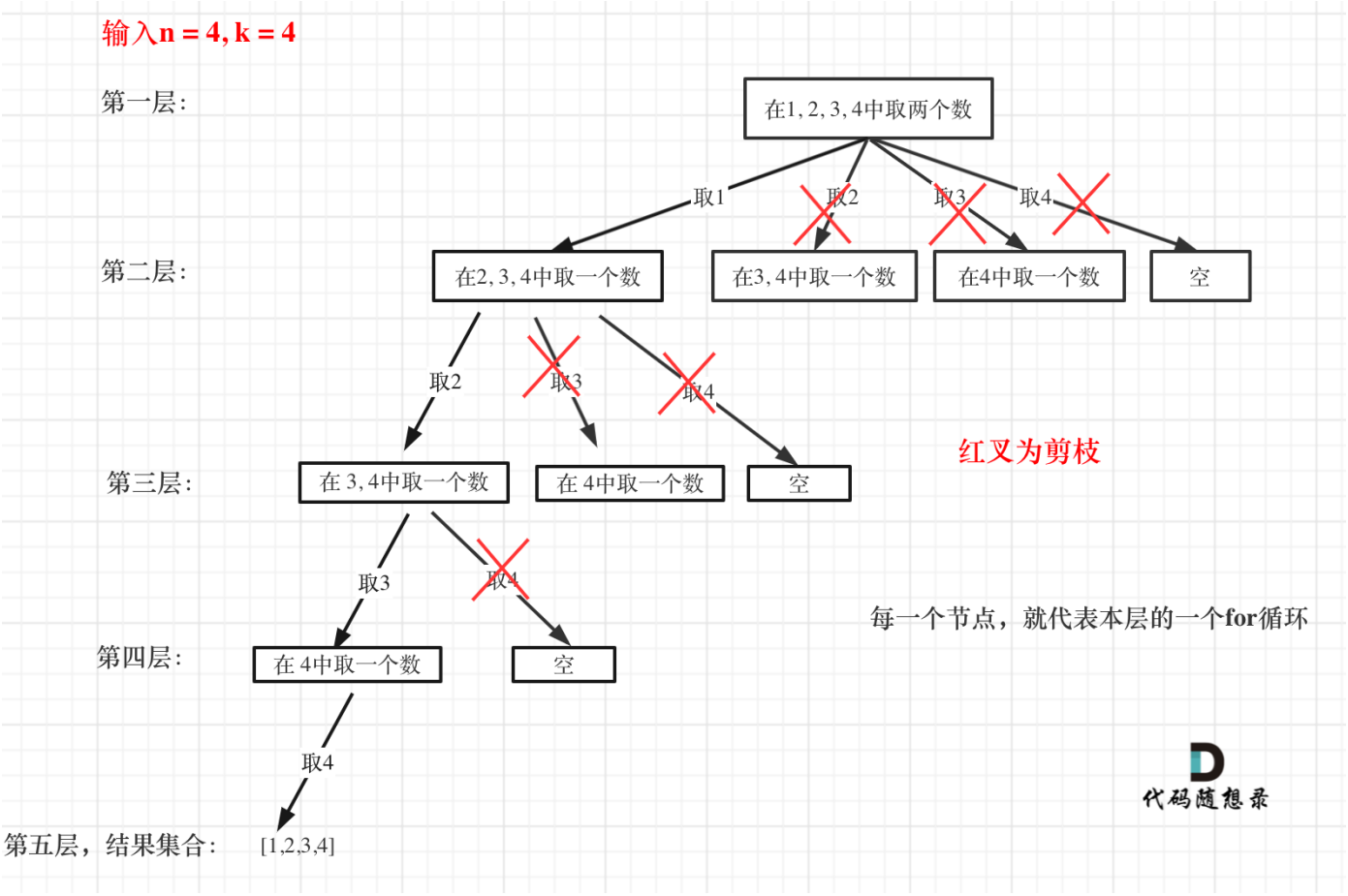


可以直观的看出其搜索的过程：**for循环横向遍历，递归纵向遍历，回溯不断调整结果集**，这个理念贯穿整个回溯法系列，也是我做了很多回溯的题目，不断摸索其规律才总结出来的。

对于回溯法的整体框架，网上搜的文章这块都说不清楚，按照天上掉下来的代码对着讲解，不知道究竟是怎么来的，也不知道为什么要这么写。

所以，录友们刚开始学回溯法，起跑姿势就很标准了！

优化回溯算法只有剪枝一种方法，在回溯算法：组合问题再剪剪枝中把回溯法代码做了剪枝优化，树形结构如图：



大家可以一目了然剪的究竟是哪里。

回溯算法：求组合问题！**剪枝精髓是：for循环在寻找起点的时候要有一个范围，如果这个起点到集合终止之间的元素已经不够题目要求的k个元素了，就没有必要搜索了。**

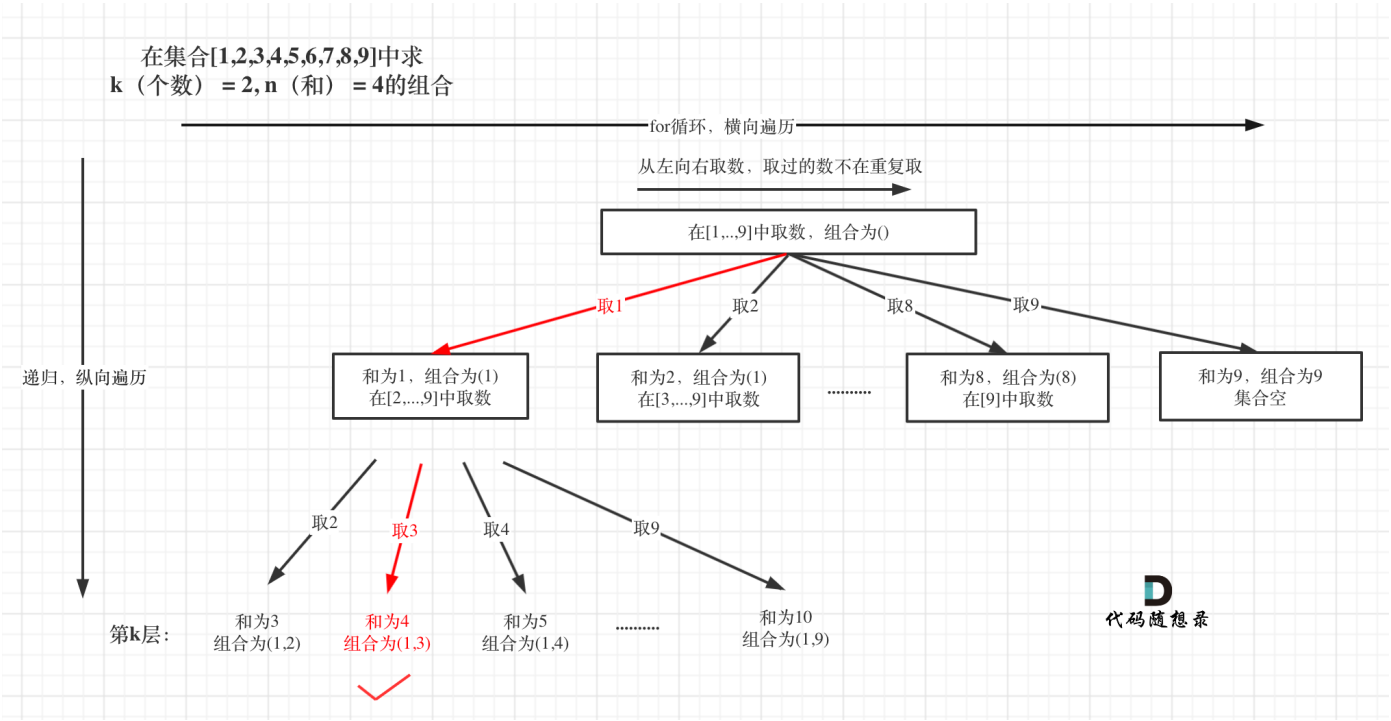
在for循环上做剪枝操作是回溯法剪枝的常见套路！ 后面的题目还会经常用到。

组合总和

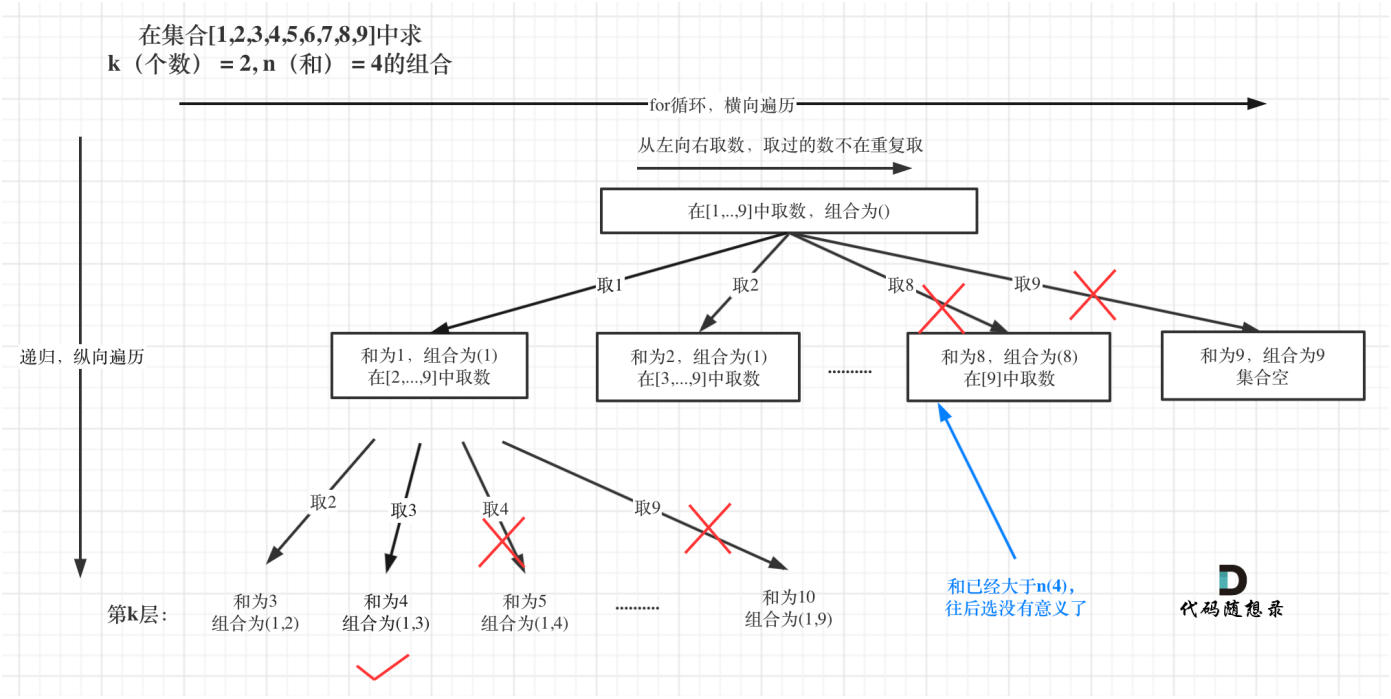
组合总和（一）

在回溯算法：求组合总和！中，相当于 回溯算法：求组合问题！加了一个元素总和的限制。

树形结构如图：



整体思路还是一样的，本题的剪枝会好想一些，即：**已选元素总和如果已经大于n（题中要求的和）了，那么往后遍历就没有意义了，直接剪掉**，如图：



在本题中，依然还可以有一个剪枝，就是回溯算法：组合问题再剪剪枝中提到的，对for循环选择的起始范围的剪枝。

所以剪枝的代码可以在for循环加上 `i <= 9 - (k - path.size()) + 1` 的限制！

组合总和（二）

在回溯算法：求组合总和（二）中讲解的组合总和问题，和回溯算法：求组合问题！，回溯算法：求组合总和！和区别是：本题没有数量要求，可以无限重复，但是有总和的限制，所以间接的也是有个数的限制。

不少同学都是看到可以重复选择，就义无反顾的把startIndex去掉了。

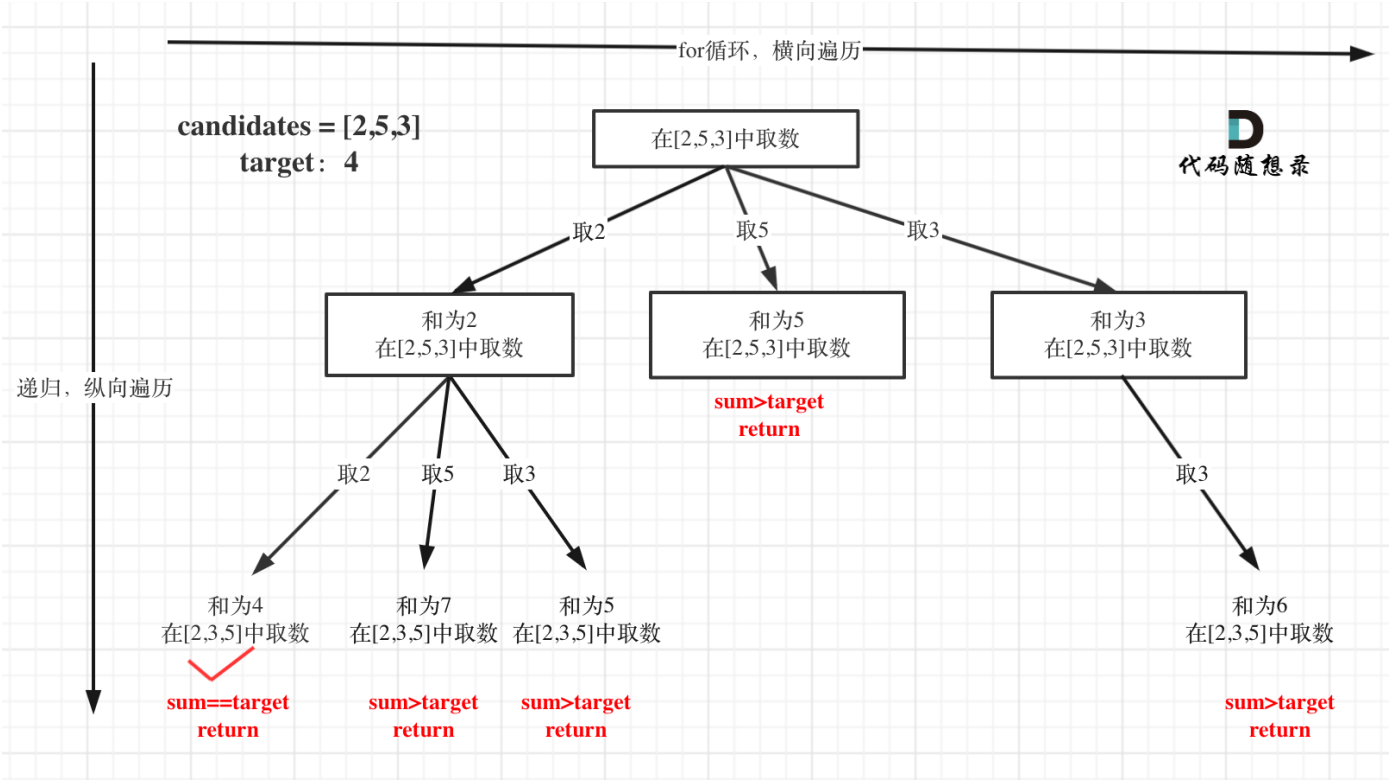
本题还需要startIndex来控制for循环的起始位置，对于组合问题，什么时候需要startIndex呢?

我举过例子，如果是一个集合来求组合的话，就需要startIndex，例如：回溯算法：求组合问题！，回溯算法：求组合总和！。

如果是多个集合取组合，各个集合之间相互不影响，那么就不用startIndex，例如：回溯算法：电话号码的字母组合

注意以上我只是说求组合的情况，如果是排列问题，又是另一套分析的套路。

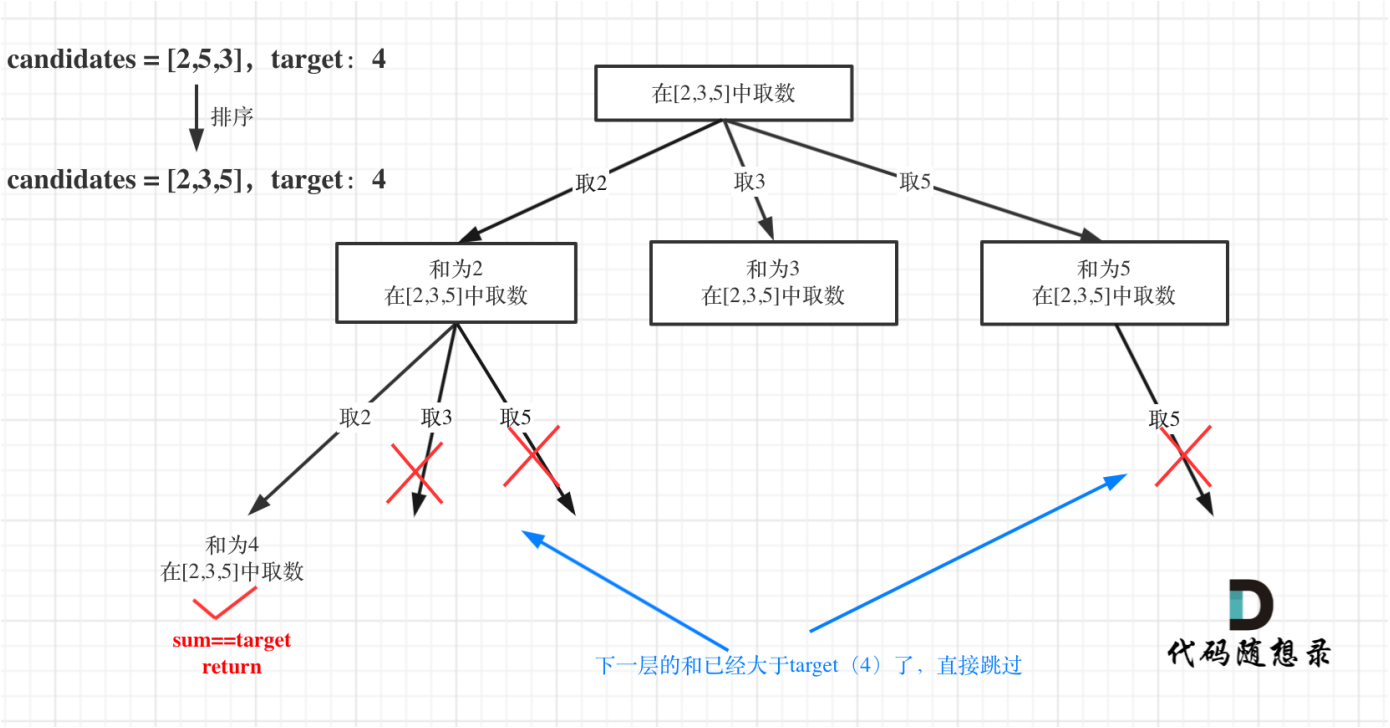
树形结构如下：



最后还给出了本题的剪枝优化，如下：

```
for (int i = startIndex; i < candidates.size() && sum + candidates[i] <= target; i++)
```

优化后树形结构如下：



组合总和（三）

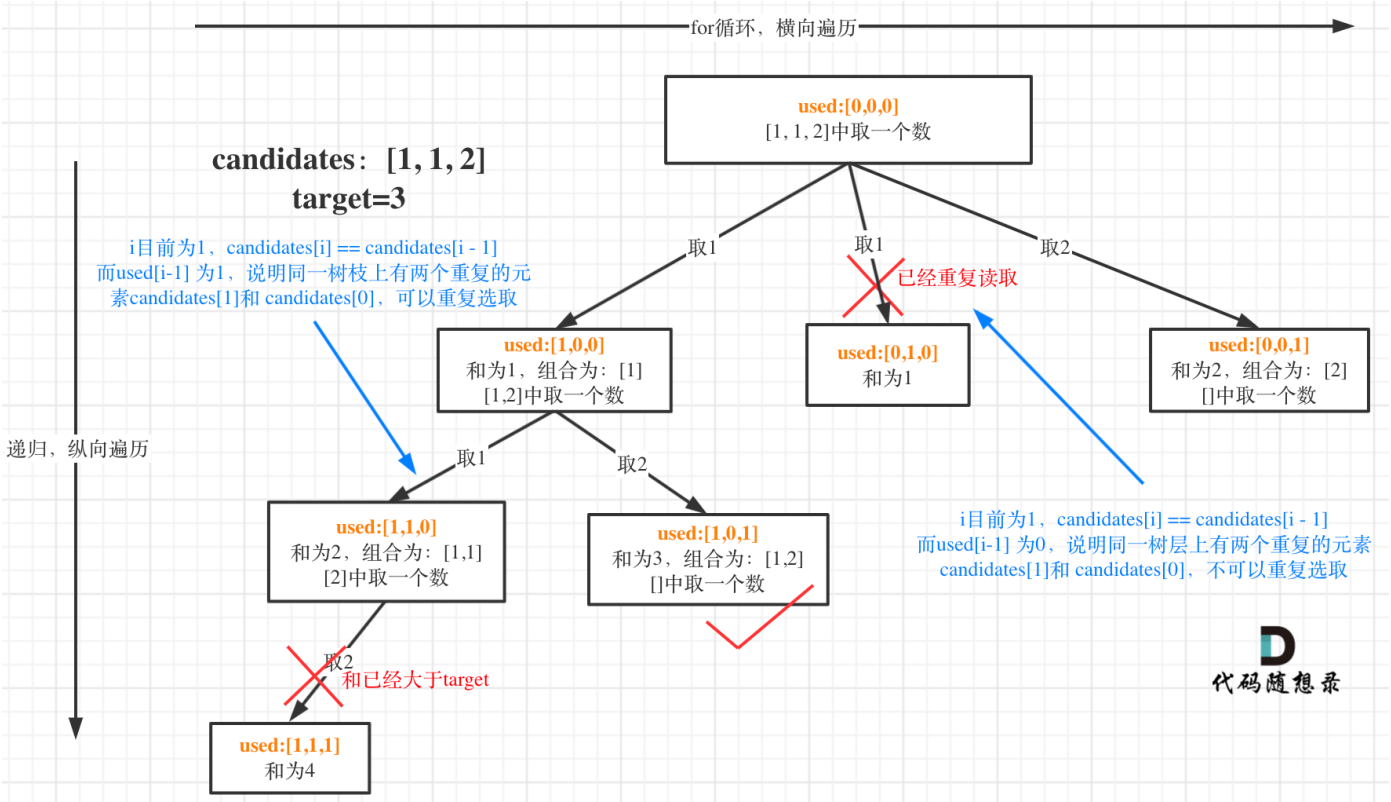
在回溯算法：求组合总和（三）中集合元素会有重复，但要求解集不能包含重复的组合。

所以难就难在去重问题上了。

这个去重问题，相信做过的录友都知道有多么的晦涩难懂。网上的题解一般就说“去掉重复”，但说不清怎么个去重，代码一甩就完事了。

为了讲解这个去重问题，**Carl自创了两个词汇，“树枝去重”和“树层去重”。**

都知道组合问题可以抽象为树形结构，那么“使用过”在这个树形结构上是有两个维度的，一个维度是同一树枝上“使用过”，一个维度是同一树层上“使用过”。**没有理解这两个层面上的“使用过” 是造成大家没有彻底理解去重的根本原因。**



我在图中将used的变化用橘黄色标注上，**可以看出在candidates[i] == candidates[i - 1]相同的情况下：**

- used[i - 1] == true，说明同一树枝candidates[i - 1]使用过
- used[i - 1] == false，说明同一树层candidates[i - 1]使用过

这块去重的逻辑很抽象，网上搜的题解基本没有能讲清楚的，如果大家之前思考过这个问题或者刷过这道题目，看到这里一定会感觉通透了很多！

对于去重，其实排列和子集问题也是一样的道理。

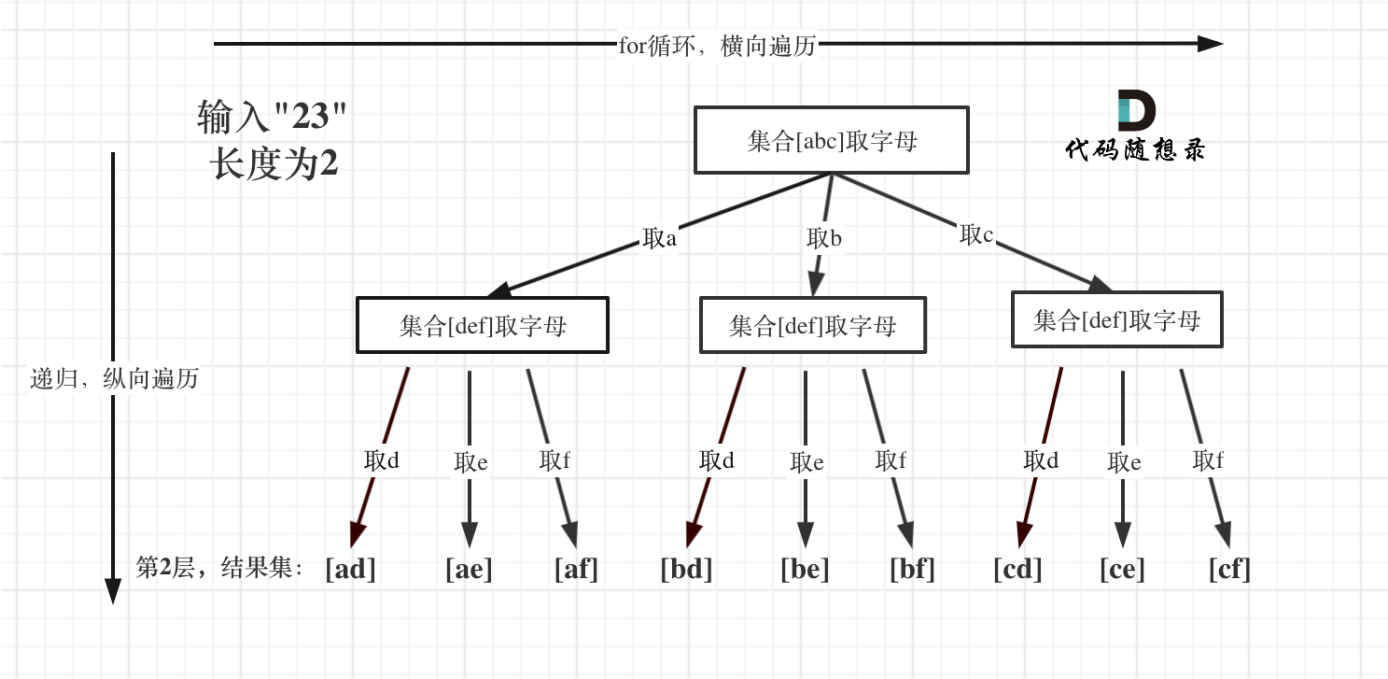
多个集合求组合

在回溯算法：电话号码的字母组合中，开始用多个集合来求组合，还是熟悉的模板题目，但是有一些细节。

例如这里for循环，可不像是在 回溯算法：求组合问题！和回溯算法：求组合总和！中从startIndex开始遍历的。

因为本题每一个数字代表的是不同集合，也就是求不同集合之间的组合，而回溯算法：求组合问题！和回溯算法：求组合总和！都是是求同一个集合中的组合！

树形结构如下：



如果大家在现场面试的时候，一定要注意各种输入异常的情况，例如本题输入1 * #按键。

其实本题不算难，但也处处是细节，还是要反复琢磨。

切割问题

在回溯算法：分割回文串中，我们开始讲解切割问题，虽然最后代码看起来好像是一道模板题，但是从分析到学会套用这个模板，是比较难的。

我列出如下几个难点：

- 切割问题其实类似组合问题
- 如何模拟那些切割线
- 切割问题中递归如何终止
- 在递归循环中如何截取子串
- 如何判断回文

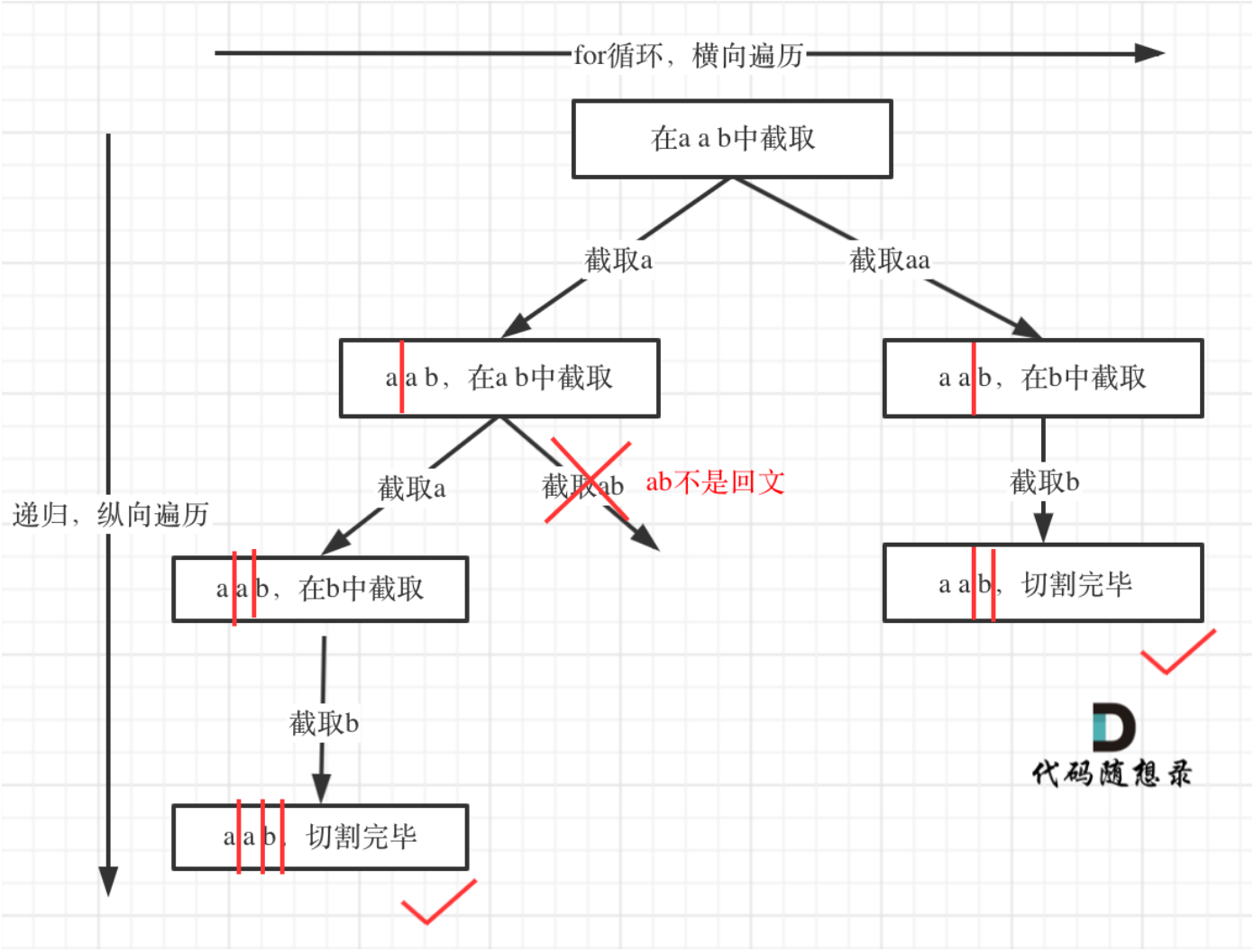
如果想到了**用求解组合问题的思路来解决 切割问题本题就成功一大半了**，接下来就可以对着模板照葫芦画瓢。

但后序如何模拟切割线，如何终止，如何截取子串，其实都不好想，最后判断回文算是最简单的了。

所以本题应该是一个道hard题目了。

除了这些难点，**本题还有细节，例如：切割过的地方不能重复切割所以递归函数需要传入i + 1。**

树形结构如下：

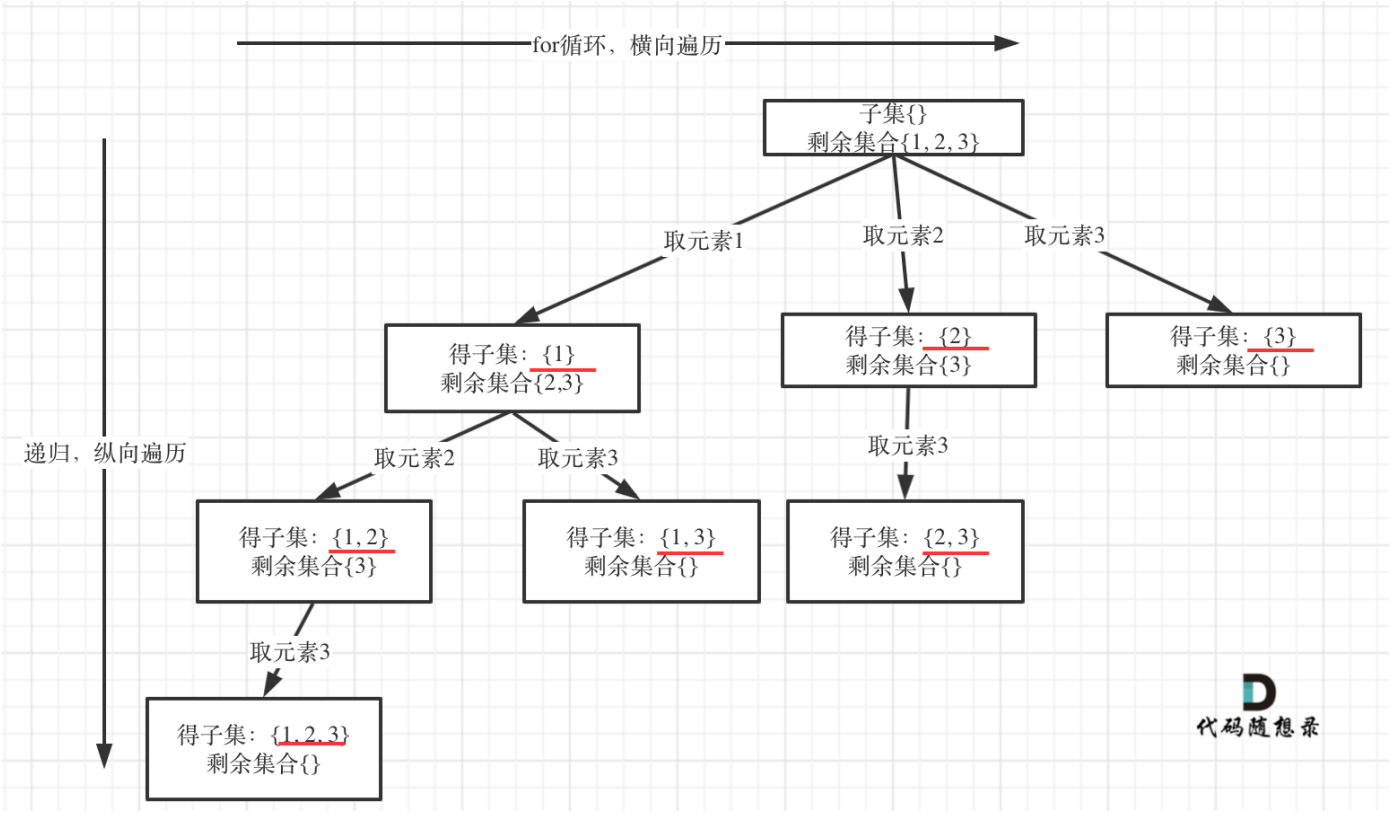


子集问题

子集问题（一）

在回溯算法：求子集问题！中讲解了子集问题，**在树形结构中子集问题是要收集所有节点的结果，而组合问题是收集叶子节点的结果。**

如图：



认清这个本质之后，今天的题目就是一道模板题了。

本题其实可以不需要加终止条件，因为startIndex >= nums.size()，本层for循环本来也结束了，本来我们就要遍历整棵树。

有的同学可能担心不写终止条件会不会无限递归?

并不会，因为每次递归的下一层就是从i+1开始的。

如果要写终止条件，注意：`result.push_back(path);`要放在终止条件的上面，如下：

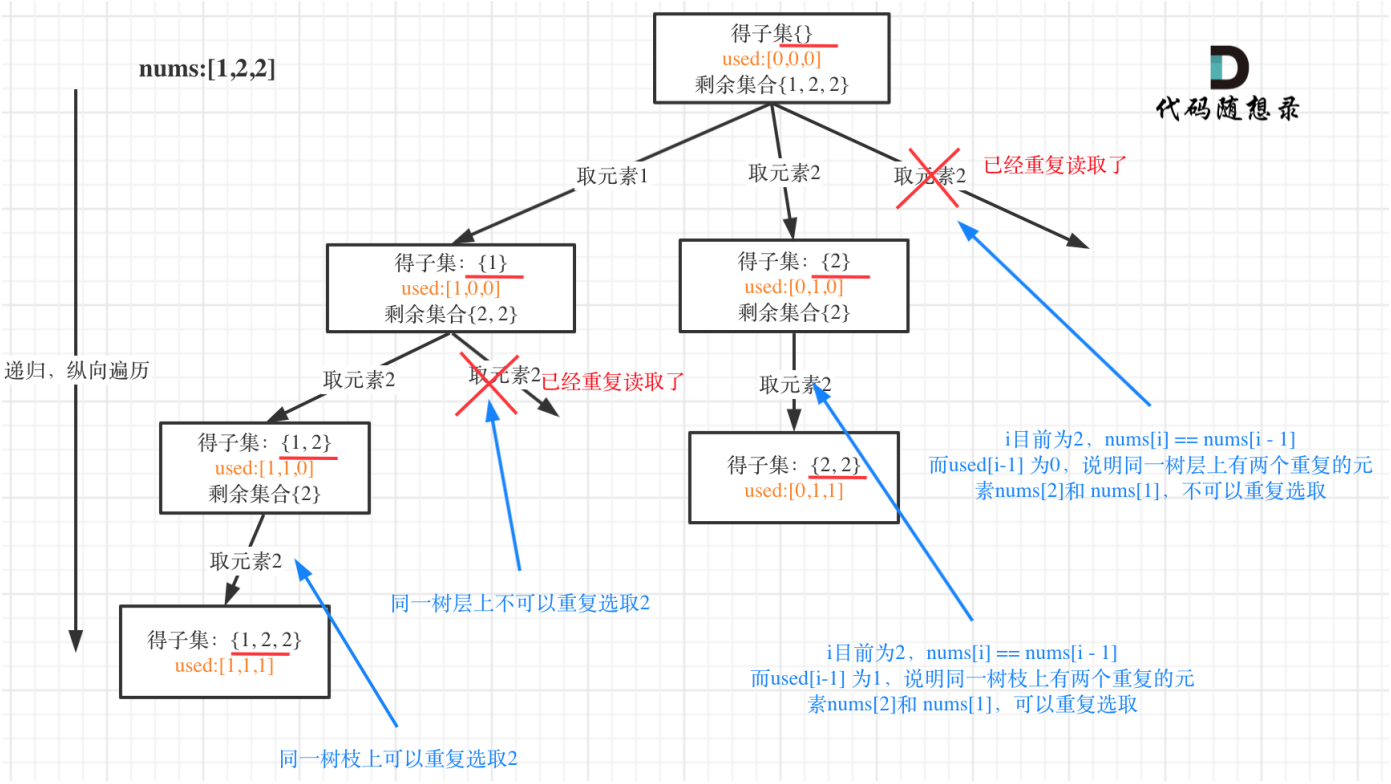
```
result.push_back(path); // 收集子集，要放在终止添加的上面，否则会漏掉结果
if (startIndex >= nums.size()) { // 终止条件可以不加
    return;
}
```

子集问题（二）

在回溯算法：求子集问题（二）中，开始针对子集问题进行去重。

本题就是回溯算法：求子集问题！的基础上加上了去重，去重我们在回溯算法：求组合总和（三）也讲过了，一样的套路。

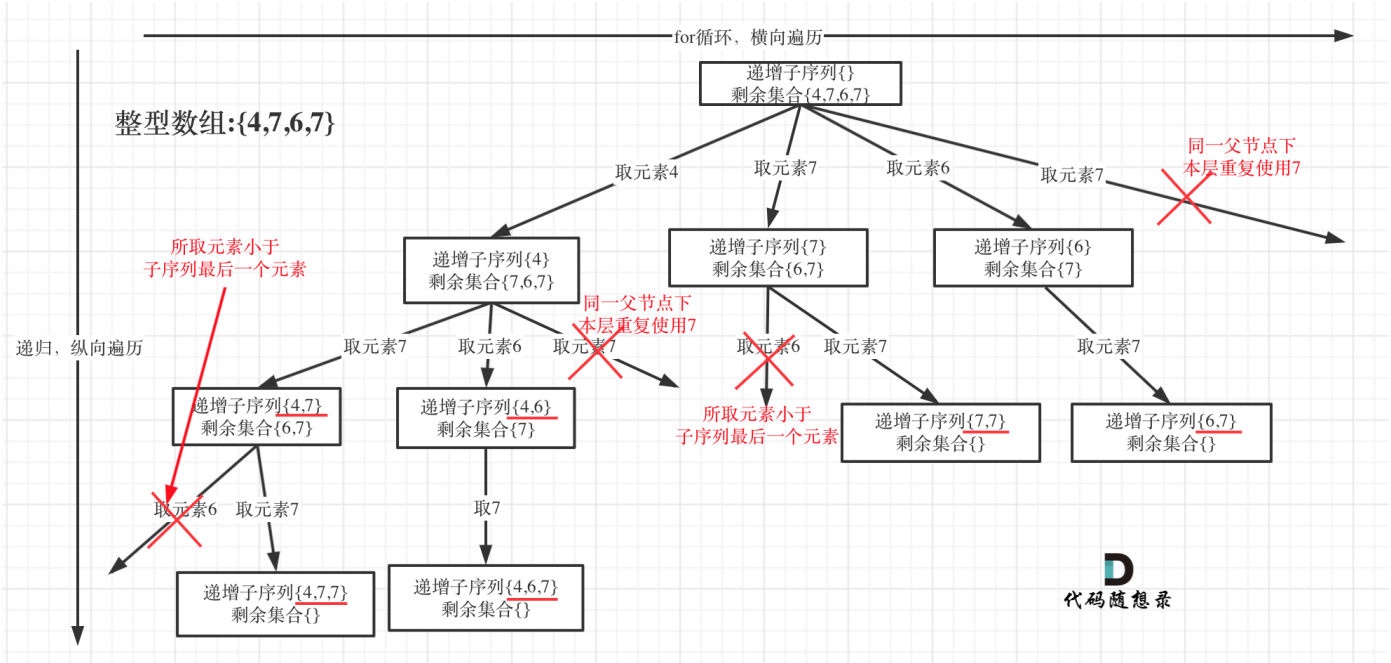
树形结构如下：



递增子序列

在回溯算法：递增子序列中，处处都能看到子集的身影，但处处是陷阱，值得好好琢磨琢磨！

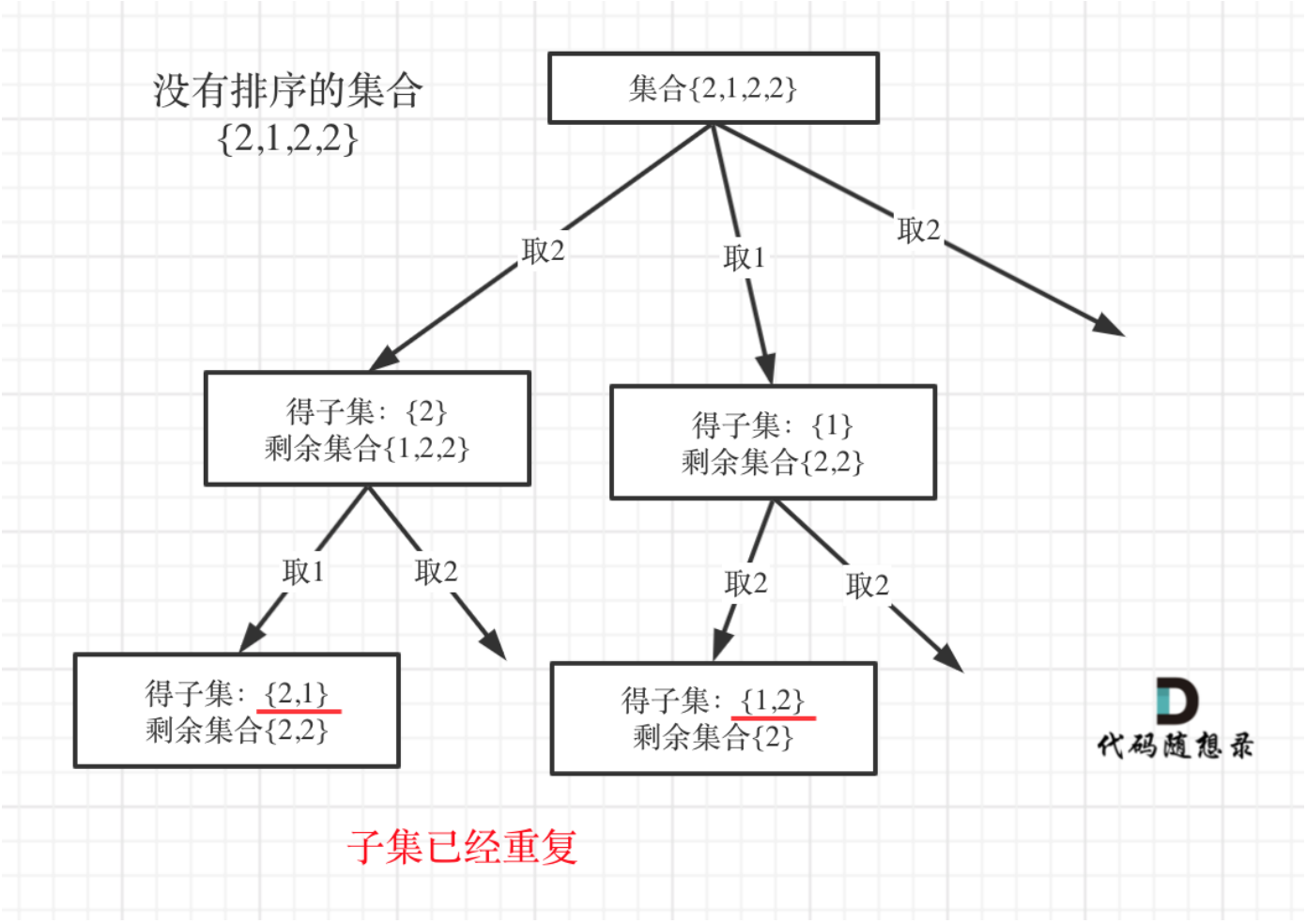
树形结构如下：



很多同学都会把这道题目和回溯算法：求子集问题（二）混在一起。

回溯算法：求子集问题（二）也可以使用set针对同一父节点本层去重，但子集问题一定要排序，为什么呢？

我用没有排序的集合{2,1,2,2}来举个例子画一个图，如下：



相信这个图胜过千言万语的解释了。

排列问题

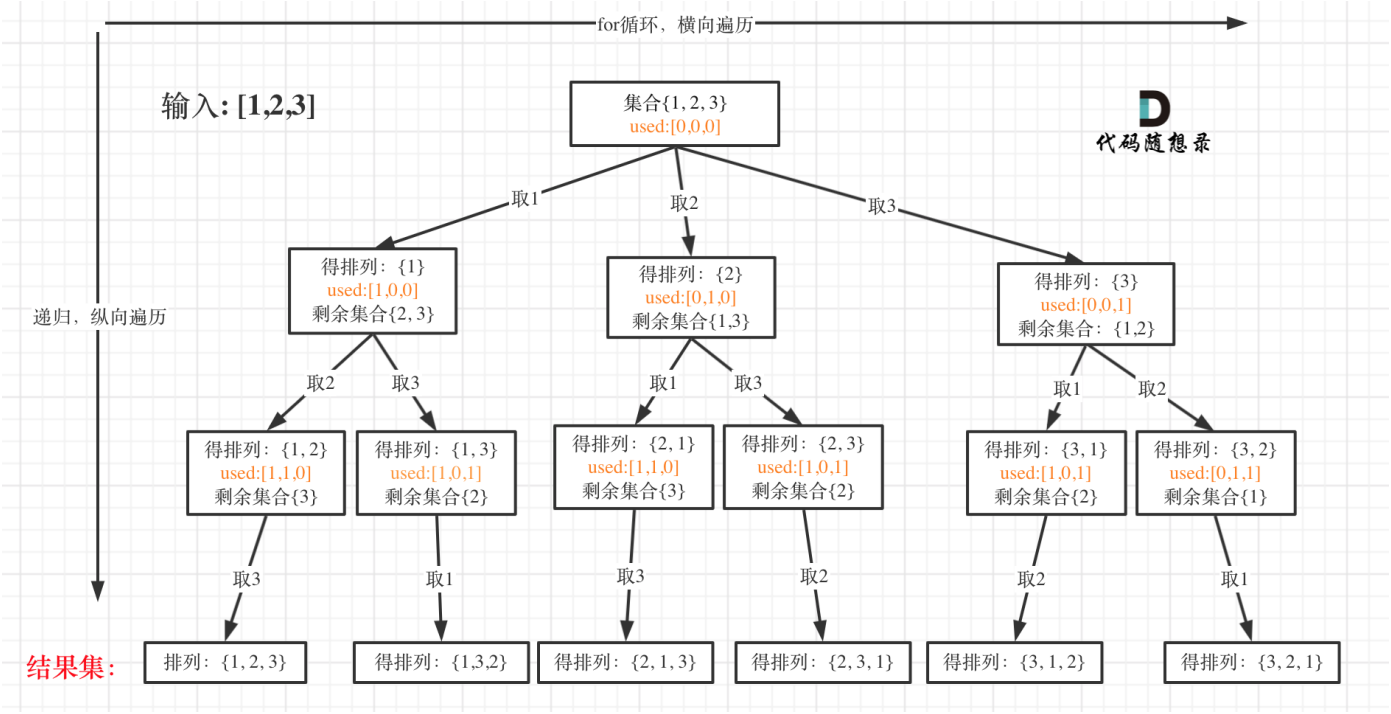
排列问题（一）

回溯算法：排列问题！ 又不一样了。

排列是有序的，也就是说 [1,2] 和 [2,1] 是两个集合，这和之前分析的子集以及组合所不同的地方。

可以看出元素1在[1,2]中已经使用过了，但是在[2,1]中还要在使用一次1，所以处理排列问题就不用使用startIndex了。

如图：



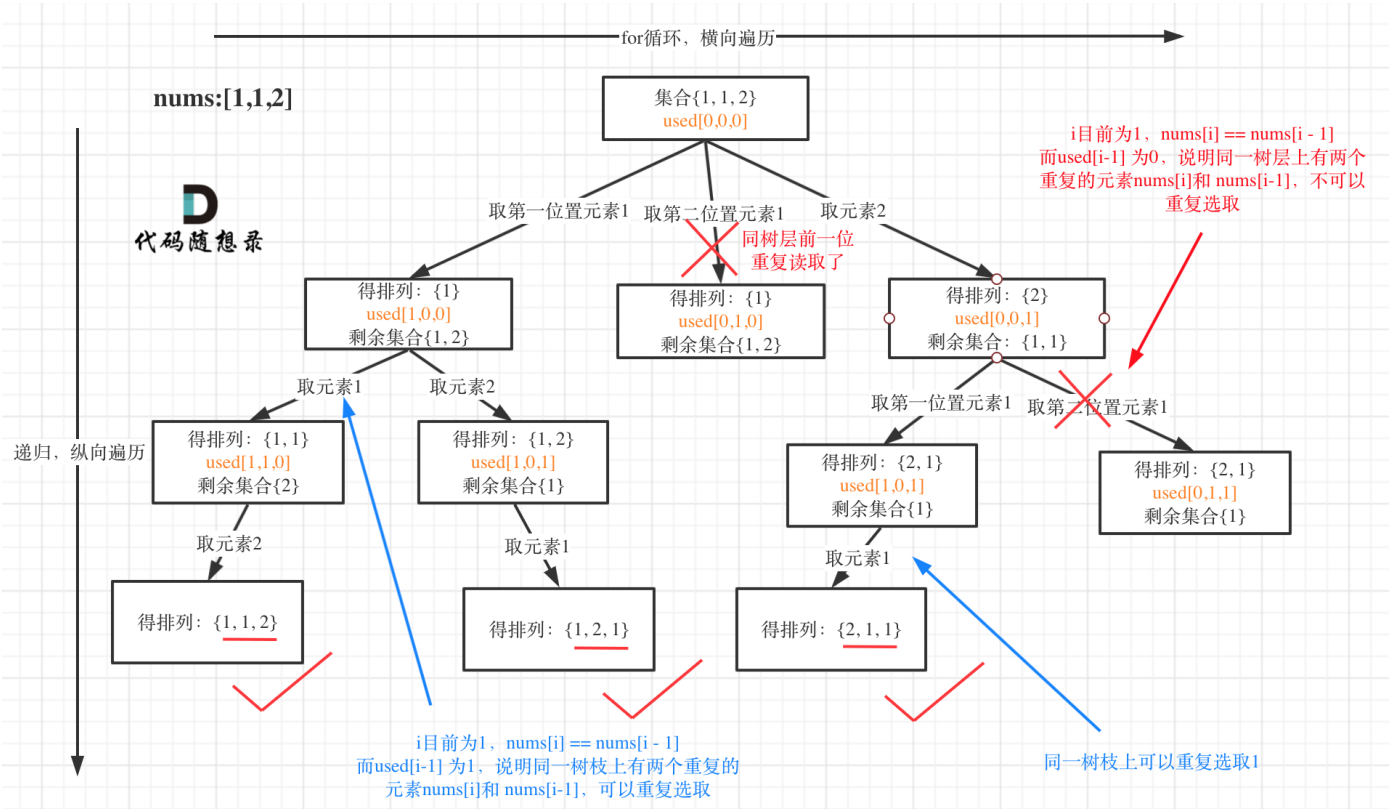
大家此时可以感受出排列问题的不同：

- 每层都是从0开始搜索而不是startIndex
- 需要used数组记录path里都放了哪些元素了

排列问题（二）

排列问题也要去重了，在回溯算法：排列问题（二）中又一次强调了“树层去重”和“树枝去重”。

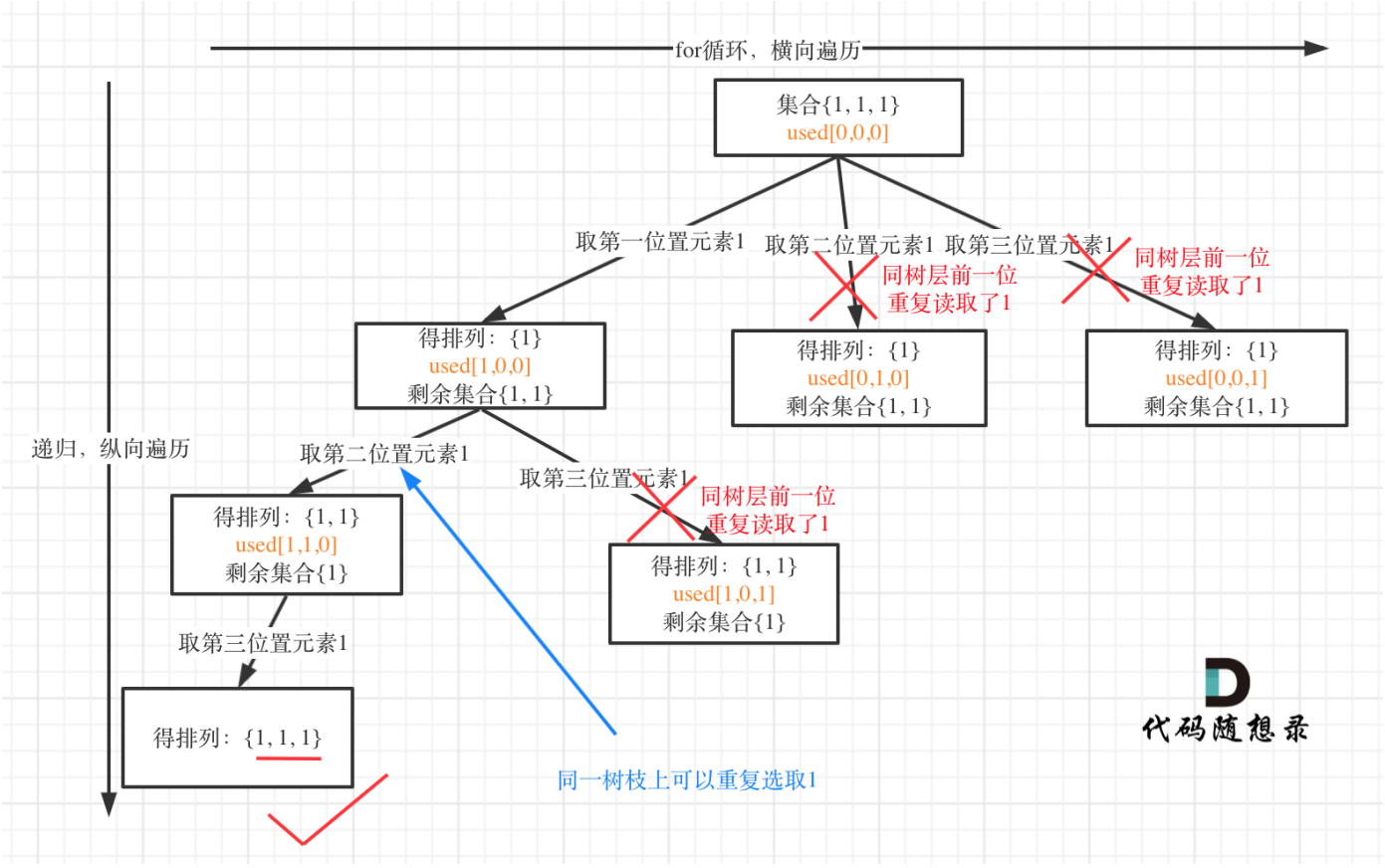
树形结构如下：



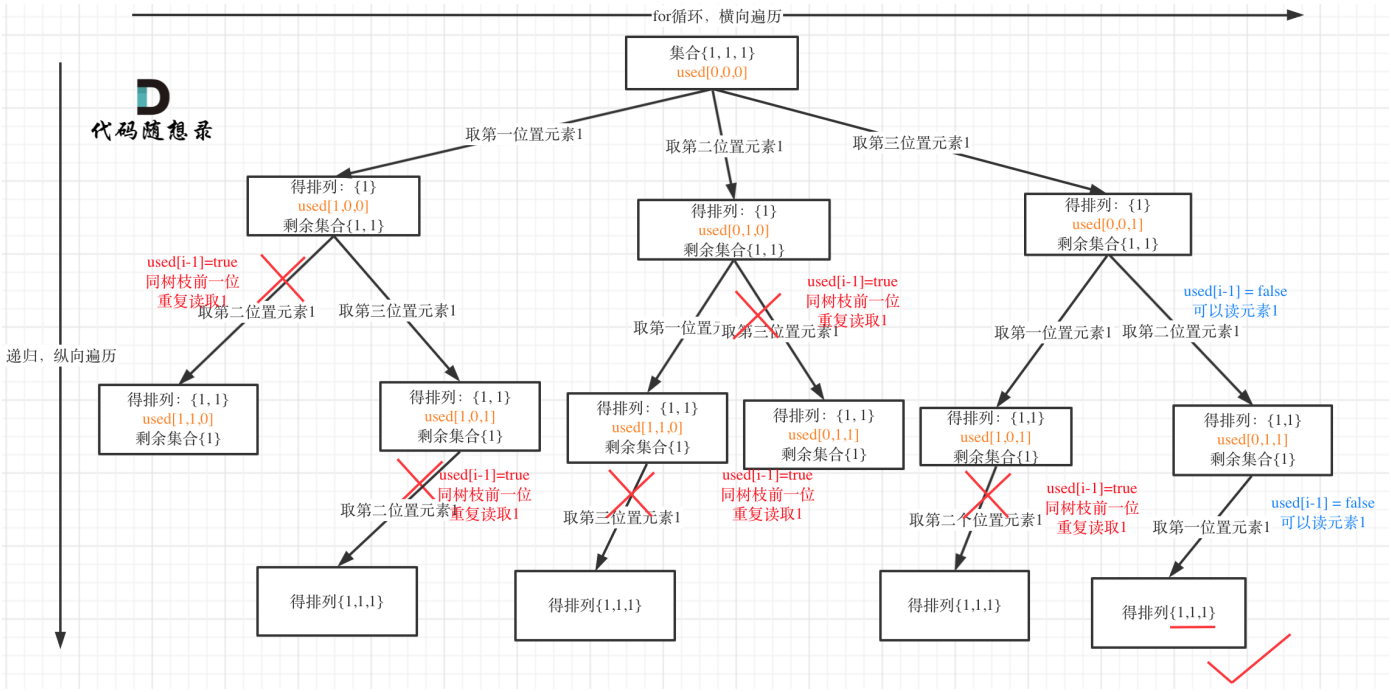
这道题目神奇的地方就是used[i - 1] == false也可以，used[i - 1] == true也可以！

我就用输入: [1,1,1] 来举一个例子。

树层上去重(used[i - 1] == false)，的树形结构如下：



树枝上去重（used[i - 1] == true）的树型结构如下：



可以清晰的看到使用(used[i - 1] == false)，即树层去重，效率更高！

本题used数组即是记录path里都放了哪些元素，同时也用来去重，一举两得。

去重问题

以上我都是统一使用used数组来去重的，其实使用set也可以用来去重！

在本周小结！（回溯算法系列三）续集中给出了子集、组合、排列问题使用set来去重的解法以及具体代码，并纠正一些同学的常见错误写法。

同时详细分析了 使用used数组去重 和 使用set去重 两种写法的性能差异：

使用set去重的版本相对于used数组的版本效率都要低很多，大家在leetcode上提交，能明显发现。

原因在回溯算法：递增子序列中也分析过，主要是因为程序运行的时候对unordered_set 频繁的insert，unordered_set需要做哈希映射（也就是把key通过hash function映射为唯一的哈希值）相对费时间，而且insert的时候其底层的符号表也要做相应的扩充，也是费时的。

而使用used数组在时间复杂度上几乎没有额外负担！

使用set去重，不仅时间复杂度高了，空间复杂度也高了，在本周小结！（回溯算法系列三）中分析过，组合，子集，排列问题的空间复杂度都是O(n)，但如果使用set去重，空间复杂度就变成了O(n^2)，因为每一层递归都有一个set集合，系统栈空间是n，每一个空间都有set集合。

那有同学可能疑惑 用used数组也是占用O(n)的空间啊?

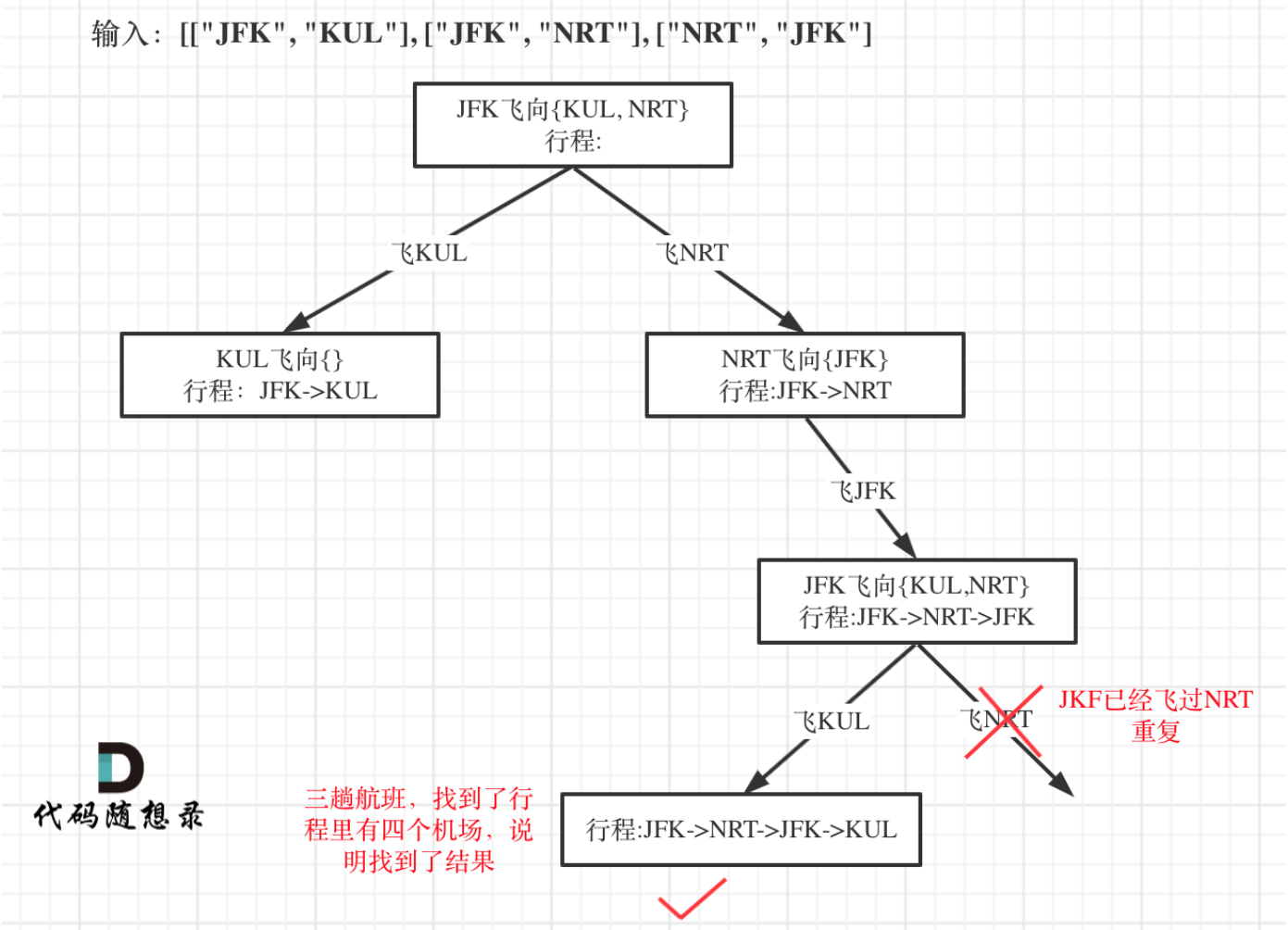
used数组可是全局变量，每层与每层之间公用一个used数组，所以空间复杂度是O(n + n)，最终空间复杂度还是O(n)。

重新安排行程（图论额外拓展）

之前说过，有递归的地方就有回溯，深度优先搜索也是用递归来实现的，所以往往伴随着回溯。

在回溯算法：重新安排行程其实也算是图论里深搜的题目，但是我用回溯法的套路来讲解这道题目，算是给大家拓展一下思路，原来回溯法还可以这么玩！

以输入：[["JFK", "KUL"], ["JFK", "NRT"], ["NRT", "JFK"]为例，抽象为树形结构如下：



本题可以算是一道hard的题目了，关于本题的难点我在文中已经详细列出。

如果单纯的回溯搜索（深搜）并不难，难还难在容器的选择和使用上！

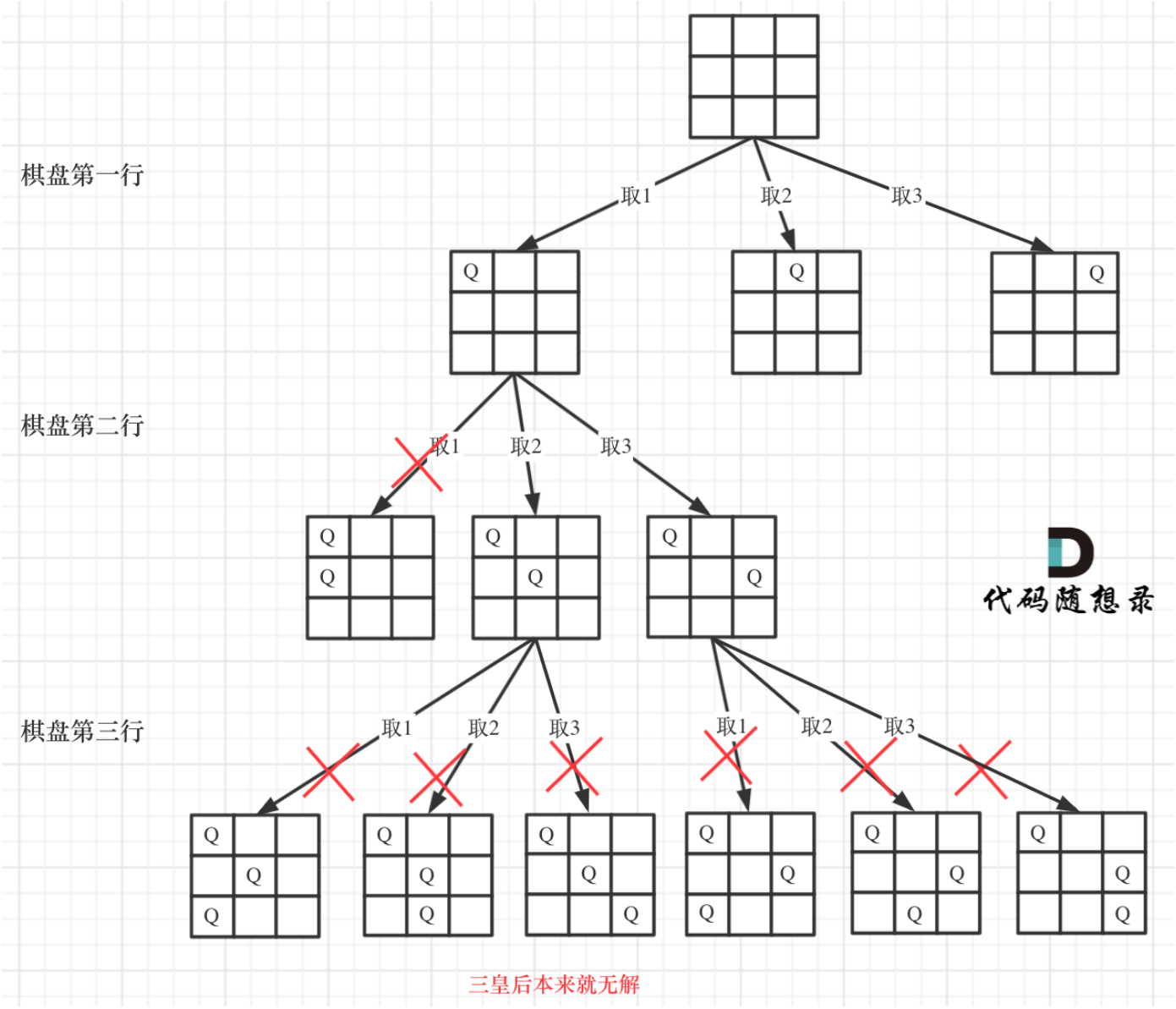
本题其实是一道深度优先搜索的题目，但是我完全使用回溯法的思路来讲解这道题题目，**算是给大家拓展一下思维方式，其实深搜和回溯也是分不开的，毕竟最终都是用递归。**

棋盘问题

N皇后问题

在回溯算法：N皇后问题中终于迎来了传说中的N皇后。

下面我用一个3 * 3 的棋盘，将搜索过程抽象为一棵树，如图：



从图中，可以看出，二维矩阵中矩阵的高就是这棵树的高度，矩阵的宽就是树形结构中每一个节点的宽度。

那么我们用皇后们的约束条件，来回溯搜索这棵树，**只要搜索到了树的叶子节点，说明就找到了皇后们的合理位置了。**

如果从来没有接触过N皇后问题的同学看着这样的题会感觉无从下手，可能知道要用回溯法，但也不知道该怎么去搜。

这里我明确给出了棋盘的宽度就是for循环的长度，递归的深度就是棋盘的高度，这样就可以套进回溯法的模板里了。

相信看完本篇回溯算法：N皇后问题也没那么难了，传说已经不是传说了，哈哈。

解数独问题

在回溯算法：解数独中要征服回溯法的最后一道山峰。

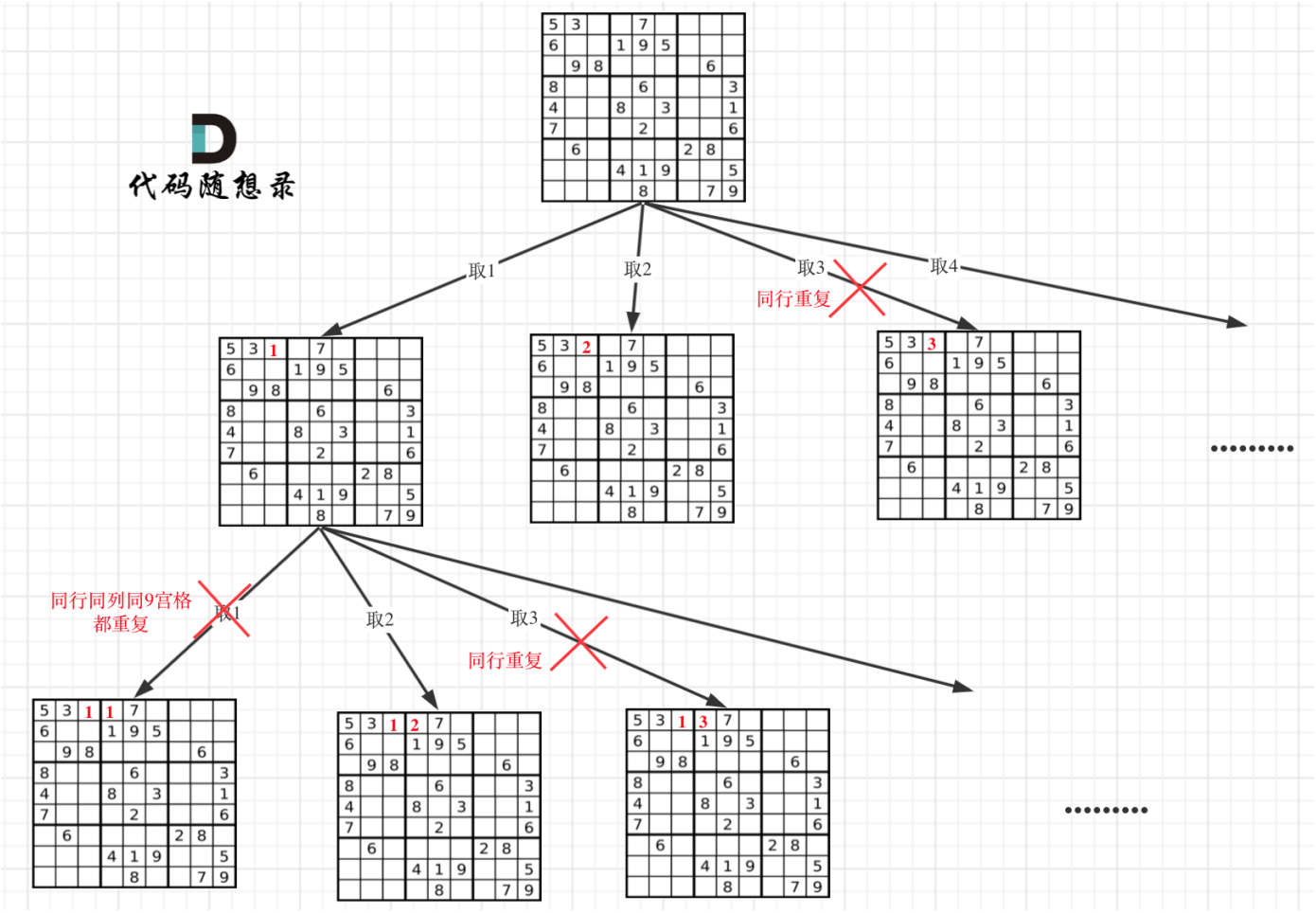
解数独应该是棋盘很难的题目了，比N皇后还要复杂一些，但只要理解 “二维递归”这个过程，其实发现就没那么难了。

大家已经跟着「代码随想录」刷过了如下回溯法题目，例如：77.组合（组合问题），131.分割回文串（分割问题），78.子集（子集问题），46.全排列（排列问题），以及51.N皇后（N皇后问题），其实这些题目都是一维递归。

其中N皇后问题是因为每一行每一列只放一个皇后，只需要一层for循环遍历一行，递归来遍历列，然后一行一列确定皇后的唯一位置。

本题就不一样了，**本题中棋盘的每一个位置都要放一个数字，并检查数字是否合法，解数独的树形结构要比N皇后更宽更深**。

因为这个树形结构太大了，我抽取一部分，如图所示：



解数独可以说是非常难的题目了，如果还一直停留在一维递归的逻辑中，这道题目可以让大家瞬间崩溃。

所以我在回溯算法：解数独中开篇就提到了二维递归，这也是我自创词汇，希望可以帮助大家理解解数独的搜索过程。

一波分析之后，在看代码会发现其实也不难，唯一难点就是理解**二维递归**的思维逻辑。

这样，解数独这么难的问题也被我们攻克了。

性能分析

关于回溯算法的复杂度分析在网上的资料鱼龙混杂，一些所谓的经典面试书籍不讲回溯算法，算法书籍对这块也避而不谈，感觉就像是算法里模糊的边界。

所以这块就说一说我个人理解，对内容持开放态度，集思广益，欢迎大家来讨论！

以下在计算空间复杂度的时候我都把系统栈（不是数据结构里的栈）所占空间算进去。

子集问题分析：

- 时间复杂度：O(2^n)，因为每一个元素的状态无外乎取与不取，所以时间复杂度为O(2^n)
- 空间复杂度：O(n)，递归深度为n，所以系统栈所用空间为O(n)，每一层递归所用的空间都是常数级别，注意代码里的result和path都是全局变量，就算是放在参数里，传的也是引用，并不会新申请内存空间，最终空间复杂度为O(n)

排列问题分析：

- 时间复杂度：O(n!)，这个可以从排列的树形图中很明显发现，每一层节点为n，第二层每一个分支都延伸了n-1个分支，再往下又是n-2个分支，所以一直到叶子节点一共就是 n * n-1 * n-2 * ..... 1 = n!。
- 空间复杂度：O(n)，和子集问题同理。

组合问题分析：

- 时间复杂度：O(2^n)，组合问题其实就是一种子集的问题，所以组合问题最坏的情况，也不会超过子集问题的时间复杂度。
- 空间复杂度：O(n)，和子集问题同理。

N皇后问题分析：

- 时间复杂度：O(n!)，其实如果看树形图的话，直觉上是O(n^n)，但皇后之间不能见面所以在搜索的过程中是有剪枝的，最差也就是O（n!），n!表示n * (n-1) * .... * 1。
- 空间复杂度：O(n)，和子集问题同理。

解数独问题分析：

- 时间复杂度：O(9^m) , m是'.'的数目。
- 空间复杂度：O(n^2)，递归的深度是n^2

一般说道回溯算法的复杂度，都说是指数级别的时间复杂度，这也算是一个概括吧！

总结

[「代码随想录」](#)**历时21天，14道经典题目分析，20张树形图，21篇回溯法精讲文章，从组合到切割，从子集到排列，从棋盘问题到最后的复杂度分析**，至此收尾了。

这里的每一种问题，讲解的时候我都会和其他问题作对比，做分析，确保每一个问题都讲的通透。

可以说方方面面都详细介绍到了。

例如：

- 如何理解回溯法的搜索过程?
- 什么时候用startIndex，什么时候不用?
- 如何去重？如何理解“树枝去重”与“树层去重”?
- 去重的几种方法?
- 如何理解二维递归?

这里的每一个问题，网上几乎找不到能讲清楚的文章，这也是直击回溯算法本质的问题。

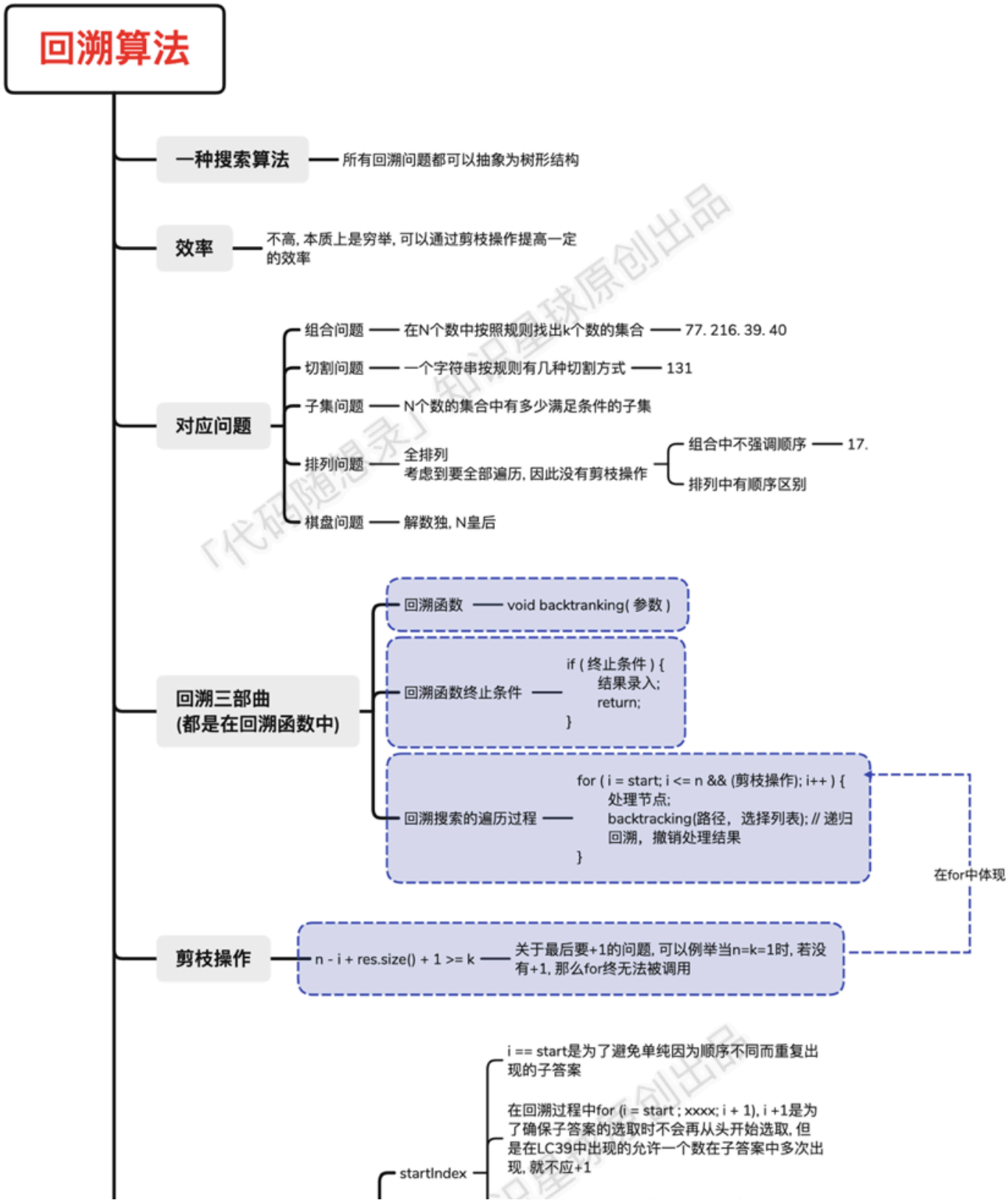
相信一路坚持下来的录友们，对回溯算法已经都深刻的认识。

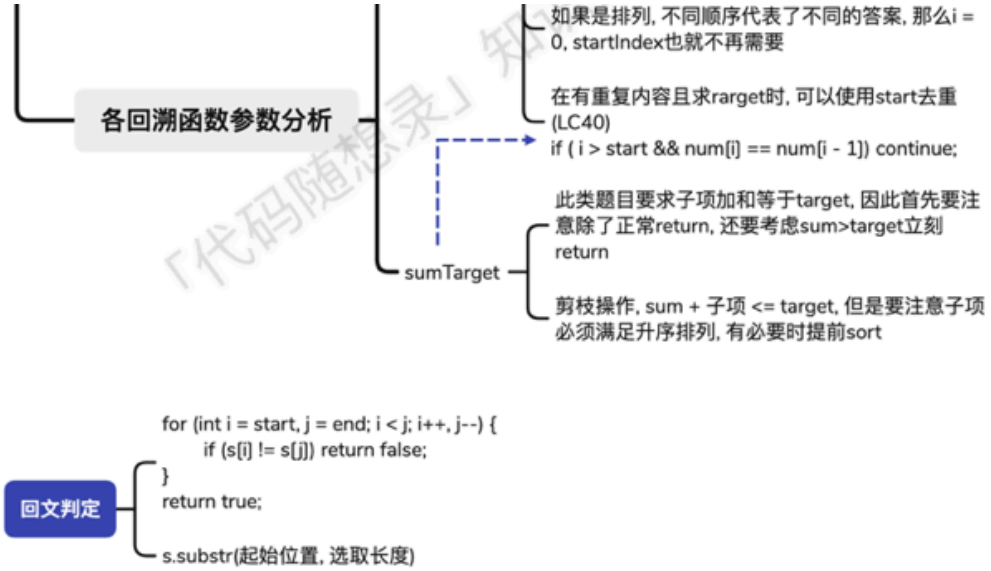
此时回溯算法系列就要正式告一段落了。

录友们可以回顾一下这21天，每天的打卡，每天在交流群里和大家探讨代码，最终换来的都是不知不觉的成长。

同样也感谢录友们的坚持，这也是我持续写作的动力，**正是因为大家的积极参与，我才知道这件事件是非常有意义的。**

回溯专题汇聚为一张图：





莫非毛

这个图是 代码随想录知识星球 成员：莫非毛，所画，总结的非常好，分享给大家。

回溯算法系列正式结束，新的系列终将开始，录友们准备开启新的征程！

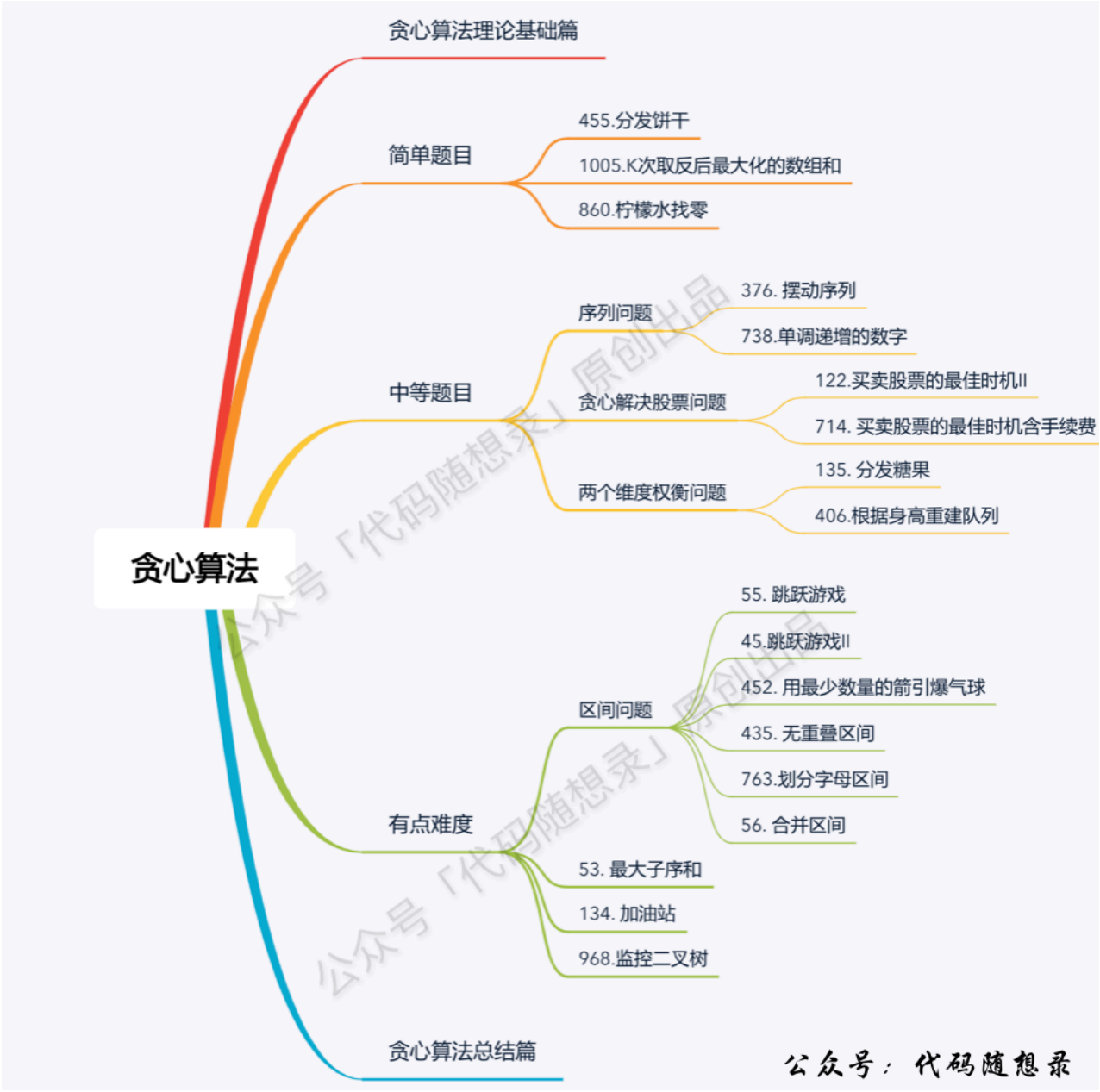
《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 关于贪心算法，你该了解这些！

题目分类大纲如下：



什么是贪心

贪心的本质是选择每一阶段的局部最优，从而达到全局最优。

这么说有点抽象，来举一个例子：

例如，有一堆钞票，你可以拿走十张，如果想达到最大的金额，你要怎么拿？

指定每次拿最大的，最终结果就是拿走最大数额的钱。

每次拿最大的就是局部最优，最后拿走最大数额的钱就是推出全局最优。

再举一个例子如果是 有一堆盒子，你有一个背包体积为n，如何把背包尽可能装满，如果还每次选最大的盒子，就不行了。这时候就需要动态规划。动态规划的问题在下一个系列会详细讲解。

贪心的套路（什么时候用贪心）

很多同学做贪心的题目的时候，想不出来是贪心，想知道有没有什么套路可以一看就看出来是贪心。

说实话贪心算法并没有固定的套路。

所以唯一的难点就是如何通过局部最优，推出整体最优。

那么如何能看出局部最优是否能推出整体最优呢？有没有什么固定策略或者套路呢？

不好意思，也没有！ 靠自己手动模拟，如果模拟可行，就可以试一试贪心策略，如果不可行，可能需要动态规划。

有同学问了如何验证可不可以用贪心算法呢？

最好用的策略就是举反例，如果想不到反例，那么就试一试贪心吧。

可有有同学认为手动模拟，举例子得出的结论不靠谱，想要严格的数学证明。

一般数学证明有如下两种方法：

- 数学归纳法
- 反证法

看教课书上讲解贪心可以是一堆公式，估计大家连看都不想看，所以数学证明就不在我要讲解的范围内了，大家感兴趣可以自行查找资料。

面试中基本不会让面试者现场证明贪心的合理性，代码写出来跑过测试用例即可，或者自己能自圆其说理由就行了。

举一个不太恰当的例子：我要用一下1+1 = 2，但我要先证明1+1 为什么等于2。严谨是严谨了，但没必要。

虽然这个例子很极端，但可以表达这么个意思：**刷题或者面试的时候，手动模拟一下感觉可以局部最优推出整体最优，而且想不到反例，那么就试一试贪心。**

例如刚刚举的拿钞票的例子，就是模拟一下每次拿做大的，最后就能拿到最多的钱，这还要数学证明的话，其实就不在算法面试的范围内了，可以看看专业的数学书籍！

所以这也是为什么很多同学通过（accept）了贪心的题目，但都不知道自己用了贪心算法，**因为贪心有时候就是常识性的推导，所以会认为本应该就这么做！**

那么刷题的时候什么时候真的需要数学推导呢？

例如这道题目：链表：环找到了，那入口呢？，这道题不用数学推导一下，就找不出环的起始位置，想试一下就不知道怎么试，这种题目确实需要数学简单推导一下。

贪心一般解题步骤

贪心算法一般分为如下四步：

- 将问题分解为若干个子问题
- 找出适合的贪心策略
- 求解每一个子问题的最优解
- 将局部最优解堆叠成全局最优解

这个四步其实过于理论化了，我们平时在做贪心类的题目 很难去按照这四步去思考，真是有点“鸡肋”。

做题的时候，只要想清楚 局部最优 是什么，如果推导出全局最优，其实就够了。

总结

本篇给出了什么是贪心以及大家关心的贪心算法固定套路。

不好意思了，贪心没有套路，说白了就是常识性推导加上举反例。

最后给出贪心的一般解题步骤，大家可以发现这个解题步骤也是比较抽象的，不像是二叉树，回溯算法，给出了那么具体的解题套路和模板。

2.分发饼干

力扣题目链接

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 i，都有一个胃口值 g[i]，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j，都有一个尺寸 s[j] 。如果 s[j] >= g[i]，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

示例 1:

- 输入: g = [1,2,3], s = [1,1]
- 输出: 1
  解释:你有三个孩子和两块小饼干，3 个孩子的胃口值分别是：1,2,3。虽然你有两块小饼干，由于他们的尺寸都是 1，你只能让胃口值是 1 的孩子满足。所以你应该输出 1。

示例 2:

- 输入: g = [1,2], s = [1,2,3]
- 输出: 2
- 解释:你有两个孩子和三块小饼干，2 个孩子的胃口值分别是 1,2。你拥有的饼干数量和尺寸都足以让所有孩子满足。所以你应该输出 2.

提示：

- 1 <= g.length <= 3 * 10^4
- 0 <= s.length <= 3 * 10^4
- 1 <= g[i], s[j] <= 2^31 - 1

算法公开课

《代码随想录》算法视频公开课：贪心算法，你想先喂哪个小孩？ | LeetCode：455.分发饼干，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

为了满足更多的小孩，就不要造成饼干尺寸的浪费。

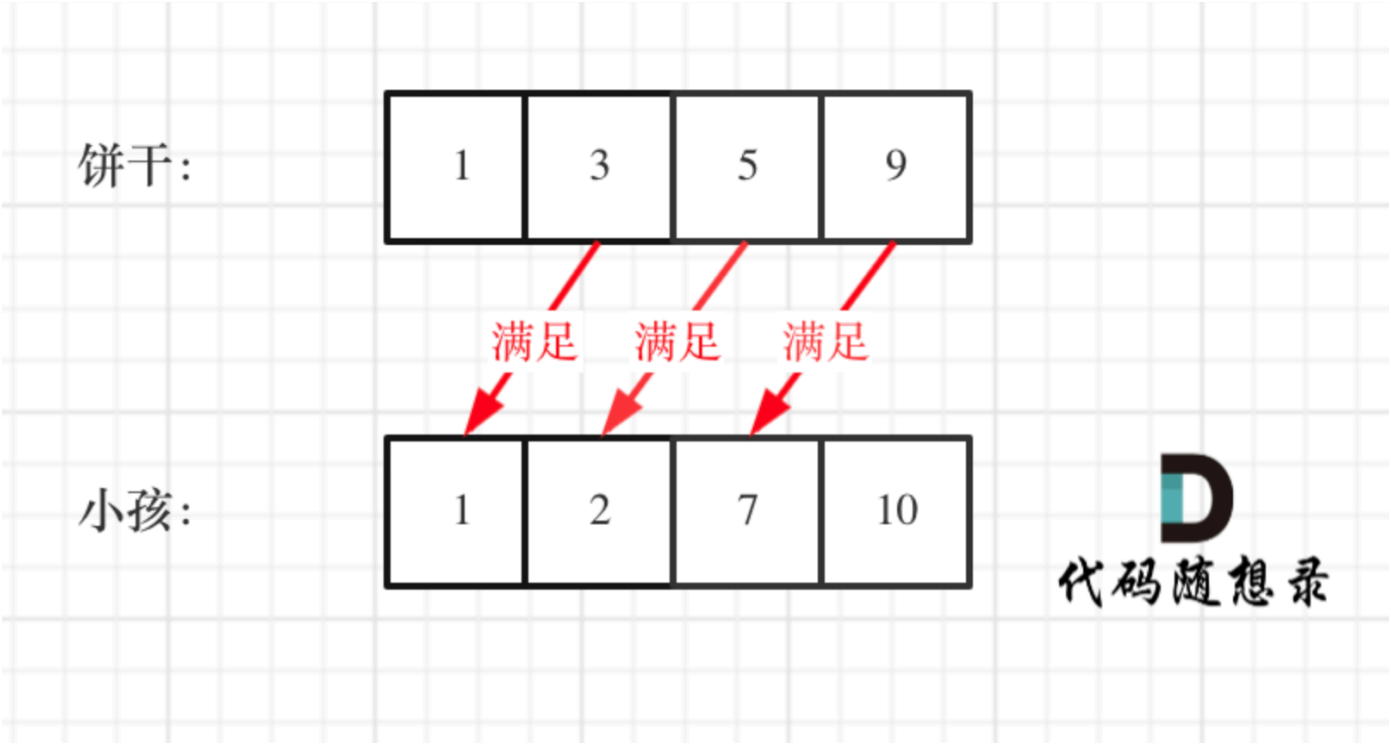
大尺寸的饼干既可以满足胃口大的孩子也可以满足胃口小的孩子，那么就应该优先满足胃口大的。

这里的局部最优就是大饼干喂给胃口大的，充分利用饼干尺寸喂饱一个，全局最优就是喂饱尽可能多的小孩。

可以尝试使用贪心策略，先将饼干数组和小孩数组排序。

然后从后向前遍历小孩数组，用大饼干优先满足胃口大的，并统计满足小孩数量。

如图：



这个例子可以看出饼干 9 只有喂给胃口为 7 的小孩，这样才是整体最优解，并想不出反例，那么就可以撸代码了。

C++代码整体如下：

```
// 版本一
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int index = s.size() - 1; // 饼干数组的下标
        int result = 0;
        for (int i = g.size() - 1; i >= 0; i--) { // 遍历胃口
            if (index >= 0 && s[index] >= g[i]) { // 遍历饼干
                result++;
                index--;
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(nlogn)
- 空间复杂度：O(1)

从代码中可以看出我用了一个 index 来控制饼干数组的遍历，遍历饼干并没有再起一个 for 循环，而是采用自减的方式，这也是常用的技巧。

有的同学看到要遍历两个数组，就想到用两个 for 循环，那样逻辑其实就复杂了。

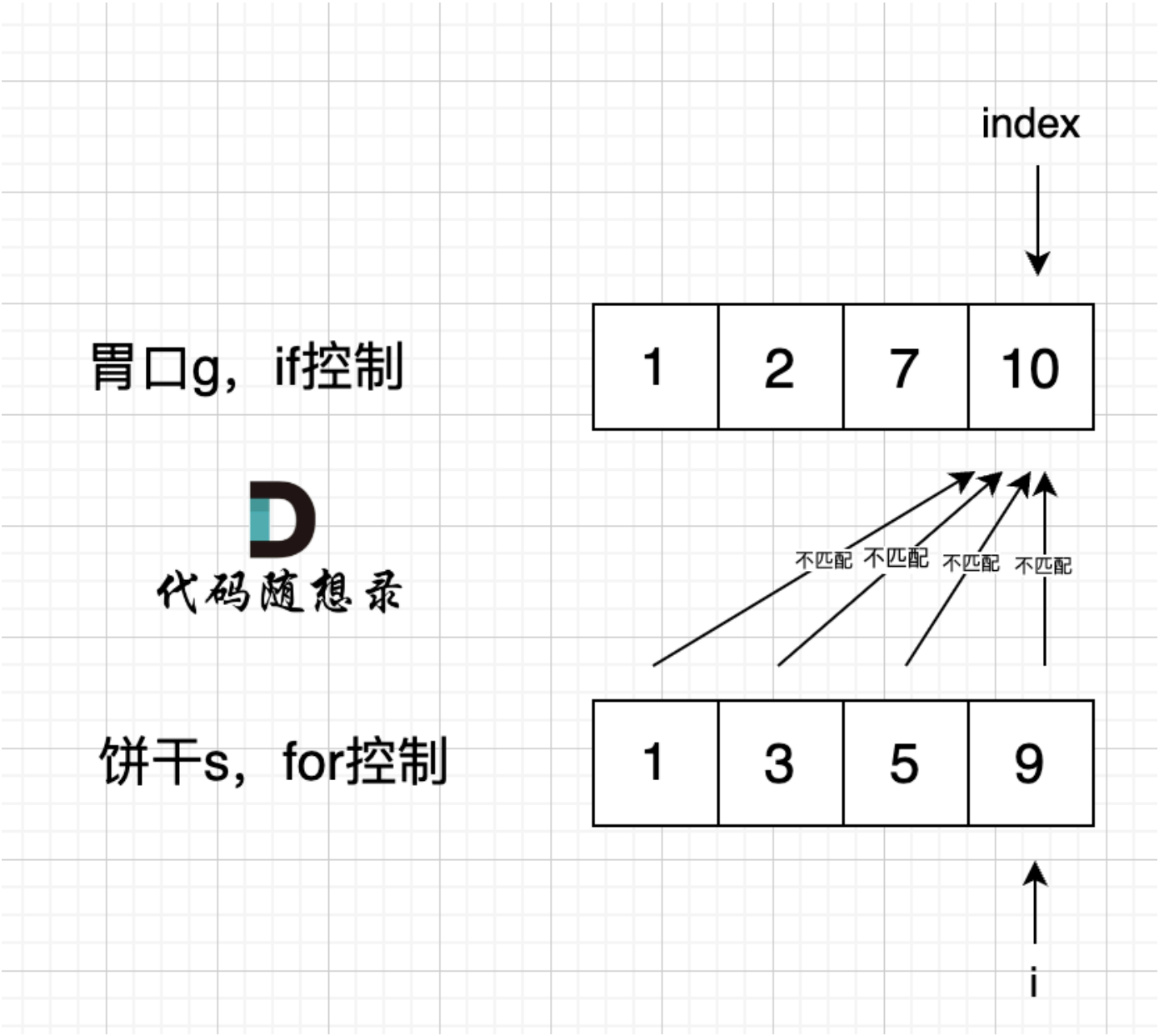
注意事项

注意版本一的代码中，可以看出来，是先遍历的胃口，在遍历的饼干，那么可不可以 先遍历 饼干，在遍历胃口呢?

其实是不可以的。

外面的 for 是里的下标 i 是固定移动的，而 if 里面的下标 index 是符合条件才移动的。

如果 for 控制的是饼干， if 控制胃口，就是出现如下情况：



if 里的 index 指向 胃口 10， for 里的 i 指向饼干 9，因为 饼干 9 满足不了 胃口 10，所以 i 持续向前移动，而 index 走不到 `s[index] >= g[i]` 的逻辑，所以 index 不会移动，那么当 i 持续向前移动，最后所有的饼干都匹配不上。

所以 一定要 for 控制 胃口，里面的 if 控制饼干。

其他思路

也可以换一个思路，小饼干先喂饱小胃口

代码如下：

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(),g.end());
        sort(s.begin(),s.end());
        int index = 0;
        for(int i = 0; i < s.size(); i++) { // 饼干
            if(index < g.size() && g[index] <= s[i]){ // 胃口
                index++;
            }
        }
        return index;
    }
};
```

- 时间复杂度：O(nlogn)
- 空间复杂度：O(1)

细心的录友可以发现，这种写法，两个循环的顺序改变了，先遍历的饼干，在遍历的胃口，这是因为遍历顺序变了，我们是从小到大遍历。

理由在上面 “注意事项”中 已经讲过。

总结

这道题是贪心很好的一道入门题目，思路还是比较容易想到的。

文中详细介绍了思考的过程，**想清楚局部最优，想清楚全局最优，感觉局部最优是可以推出全局最优，并想不出反例，那么就试一试贪心。**

> 本周讲解了贪心理论基础，以及第一道贪心的题目：贪心算法：分发饼干，可能会给大家一种贪心算法比较简单的错觉，好了，接下来几天的题目难度要上来了，哈哈。

3. 摆动序列

力扣题目链接

如果连续数字之间的差严格地在正数和负数之间交替，则数字序列称为摆动序列。第一个差（如果存在的话）可能是正数或负数。少于两个元素的序列也是摆动序列。

例如， [1,7,4,9,2,5] 是一个摆动序列，因为差值 (6,-3,5,-7,3) 是正负交替出现的。相反, [1,4,7,2,5] 和 [1,7,4,5,5] 不是摆动序列，第一个序列是因为它的前两个差值都是正数，第二个序列是因为它的最后一个差值为零。

给定一个整数序列，返回作为摆动序列的最长子序列的长度。 通过从原始序列中删除一些（也可以不删除）元素来获得子序列，剩下的元素保持其原始顺序。

示例 1:

- 输入: [1,7,4,9,2,5]
- 输出: 6
- 解释: 整个序列均为摆动序列。

示例 2:

- 输入: [1,17,5,10,13,15,10,5,16,8]
- 输出: 7
- 解释: 这个序列包含几个长度为 7 摆动序列，其中一个可为[1,17,10,13,10,16,8]。

示例 3:

- 输入: [1,2,3,4,5,6,7,8,9]
- 输出: 2

算法公开课

《代码随想录》算法视频公开课：贪心算法，寻找摆动有细节！| LeetCode：376.摆动序列，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

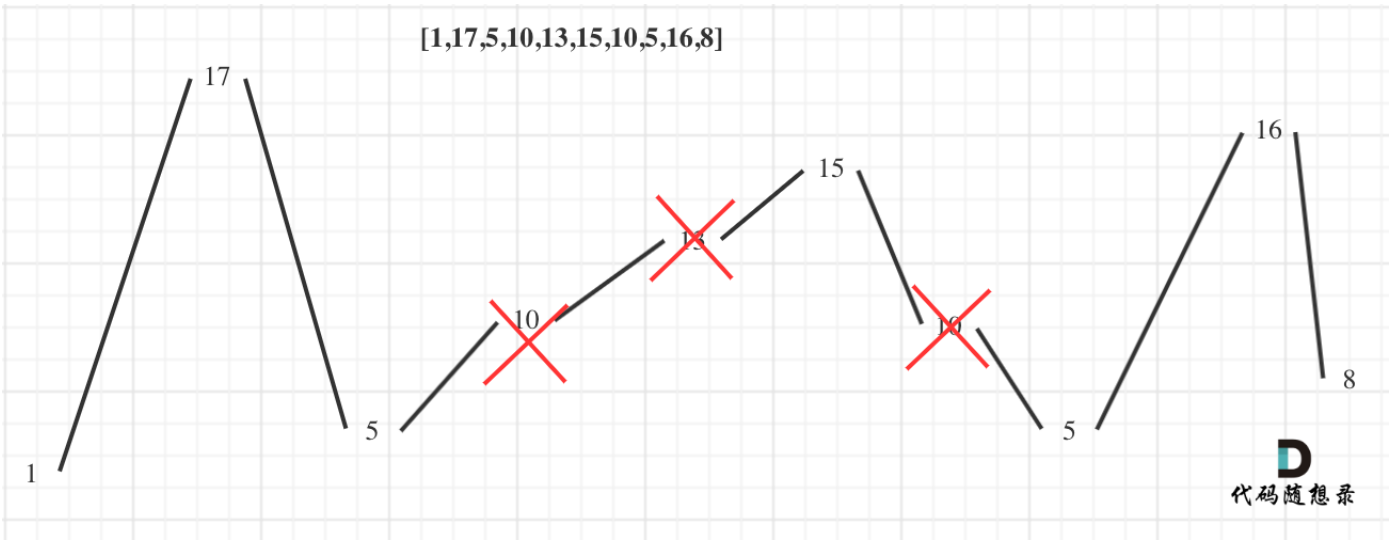
思路 1（贪心解法）

本题要求通过从原始序列中删除一些（也可以不删除）元素来获得子序列，剩下的元素保持其原始顺序。

相信这么一说吓退不少同学，这要求最大摆动序列又可以修改数组，这得如何修改呢?

来分析一下，要求删除元素使其达到最大摆动序列，应该删除什么元素呢?

用示例二来举例，如图所示：



局部最优：删除单调坡度上的节点（不包括单调坡度两端的节点），那么这个坡度就可以有两个局部峰值。

整体最优：整个序列有最多的局部峰值，从而达到最长摆动序列。

局部最优推出全局最优，并举不出反例，那么试试贪心！

（为方便表述，以下说的峰值都是指局部峰值）

实际操作上，其实连删除的操作都不用做，因为题目要求的是最长摆动子序列的长度，所以只需要统计数组的峰值数量就可以了（相当于是删除单一坡度上的节点，然后统计长度）

这就是贪心所贪的地方，让峰值尽可能的保持峰值，然后删除单一坡度上的节点

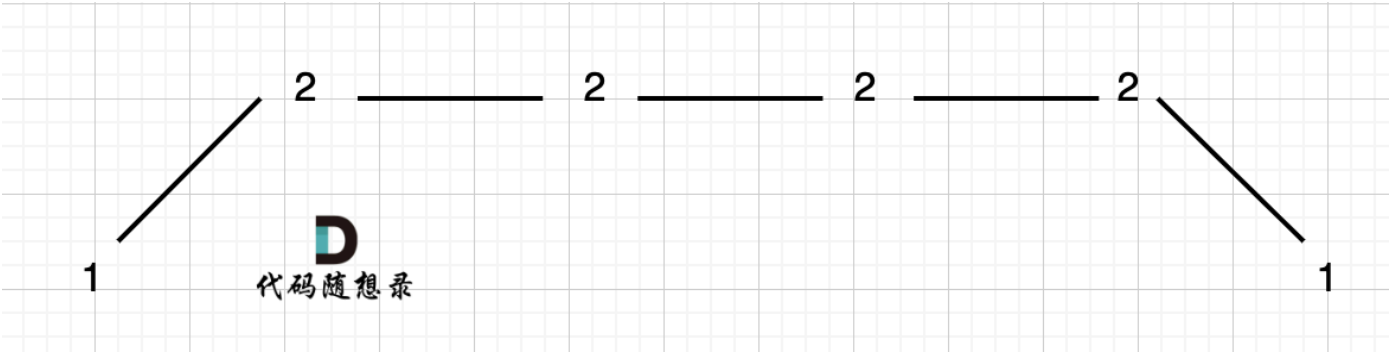
在计算是否有峰值的时候，大家知道遍历的下标 i ，计算 prediff（nums[i] - nums[i-1]） 和 curdiff（nums[i+1] - nums[i]），如果`prediff < 0 && curdiff > 0` 或者 `prediff > 0 && curdiff < 0` 此时就有波动就需要统计。

这是我们思考本题的一个大题思路，但本题要考虑三种情况：

1. 情况一：上下坡中有平坡
2. 情况二：数组首尾两端
3. 情况三：单调坡中有平坡

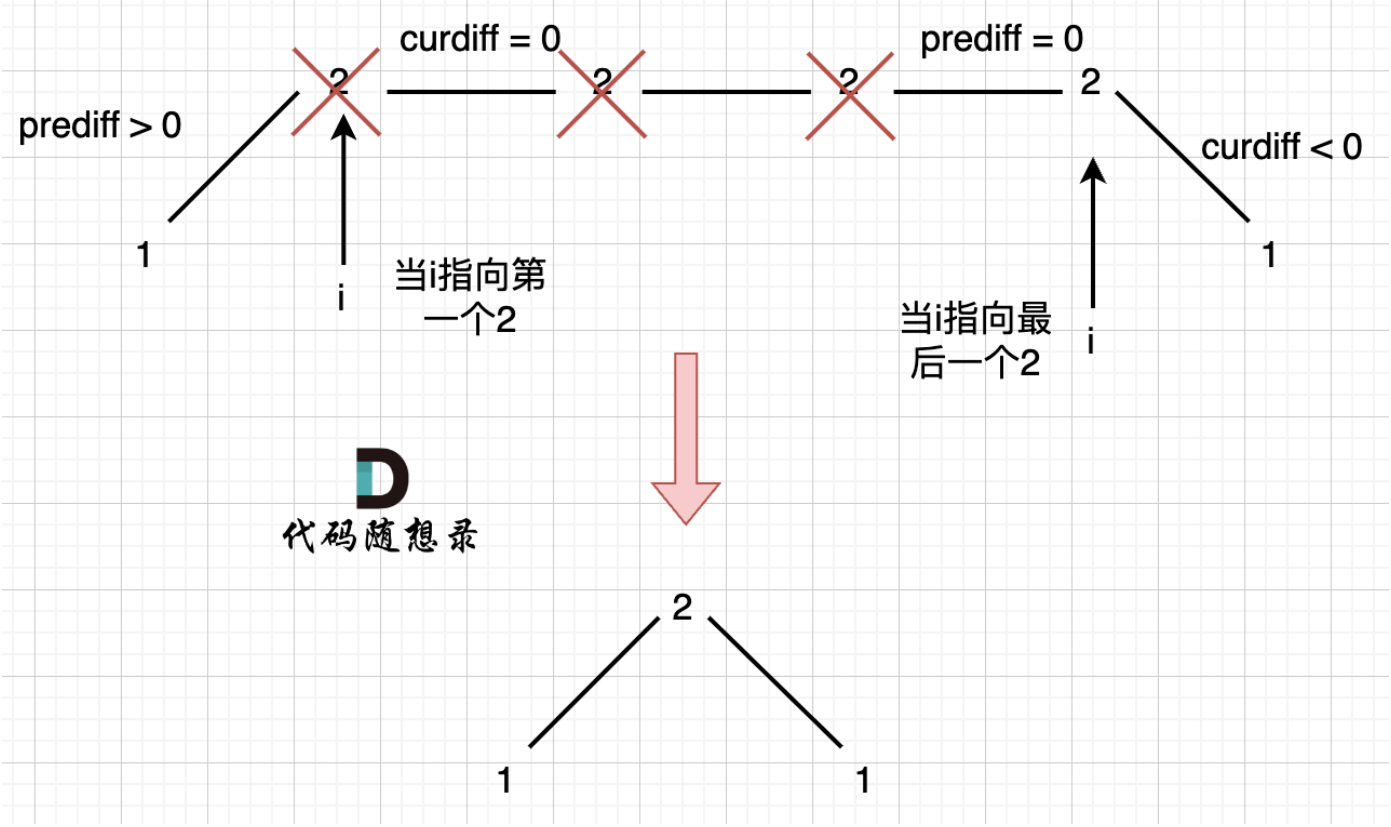
情况一：上下坡中有平坡

例如 [1,2,2,2,1]这样的数组，如图：



它的摇摆序列长度是多少呢？ **其实是长度是 3**，也就是我们在删除的时候 要不删除左面的三个 2，要不就删除右边的三个 2。

如图，可以统一规则，删除左边的三个 2：



在图中，当 i 指向第一个 2 的时候，`prediff > 0 && curdiff = 0` ，当 i 指向最后一个 2 的时候 `prediff = 0 && curdiff < 0`。

如果我们采用，删左面三个 2 的规则，那么 当 `prediff = 0 && curdiff < 0` 也要记录一个峰值，因为他是把之前相同的元素都删掉留下的峰值。

所以我们记录峰值的条件应该是：`(preDiff <= 0 && curDiff > 0) || (preDiff >= 0 && curDiff < 0)`，为什么这里允许 prediff == 0 ，就是为了 上面我说的这种情况。

情况二：数组首尾两端

所以本题统计峰值的时候，数组最左面和最右面如何统计呢?

题目中说了，如果只有两个不同的元素，那摆动序列也是 2。

例如序列[2,5]，如果靠统计差值来计算峰值个数就需要考虑数组最左面和最右面的特殊情况。

因为我们在计算 prediff（nums[i] - nums[i-1]） 和 curdiff（nums[i+1] - nums[i]）的时候，至少需要三个数字才能计算，而数组只有两个数字。

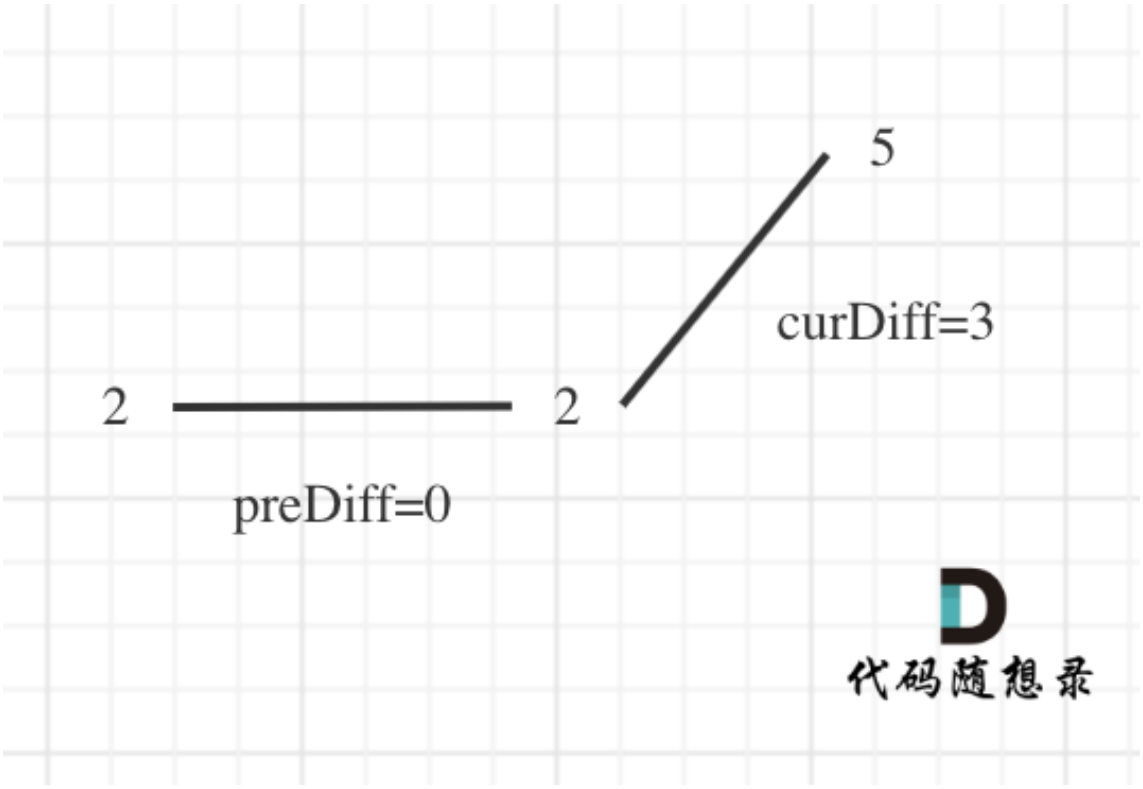
这里我们可以写死，就是 如果只有两个元素，且元素不同，那么结果为 2。

不写死的话，如何和我们的判断规则结合在一起呢?

可以假设，数组最前面还有一个数字，那这个数字应该是什么呢?

之前我们在 讨论 情况一：相同数字连续 的时候， prediff = 0 ，curdiff < 0 或者 >0 也记为波谷。

那么为了规则统一，针对序列[2,5]，可以假设为[2,2,5]，这样它就有坡度了即 preDiff = 0，如图：



针对以上情形，result 初始为 1（默认最右面有一个峰值），此时 curDiff > 0 && preDiff <= 0，那么 result++（计算了左面的峰值），最后得到的 result 就是 2（峰值个数为 2 即摆动序列长度为 2）

经过以上分析后，我们可以写出如下代码：

```
// 版本一
class Solution {
public:
    int wiggleMaxLength(vector<int>& nums) {
        if (nums.size() <= 1) return nums.size();
        int curDiff = 0; // 当前一对差值
        int preDiff = 0; // 前一对差值
        int result = 1;  // 记录峰值个数，序列默认序列最右边有一个峰值
        for (int i = 0; i < nums.size() - 1; i++) {
            curDiff = nums[i + 1] - nums[i];
            // 出现峰值
            if ((preDiff <= 0 && curDiff > 0) || (preDiff >= 0 && curDiff < 0)) {
                result++;
            }
            preDiff = curDiff;
        }
        return result;
    }
};
```

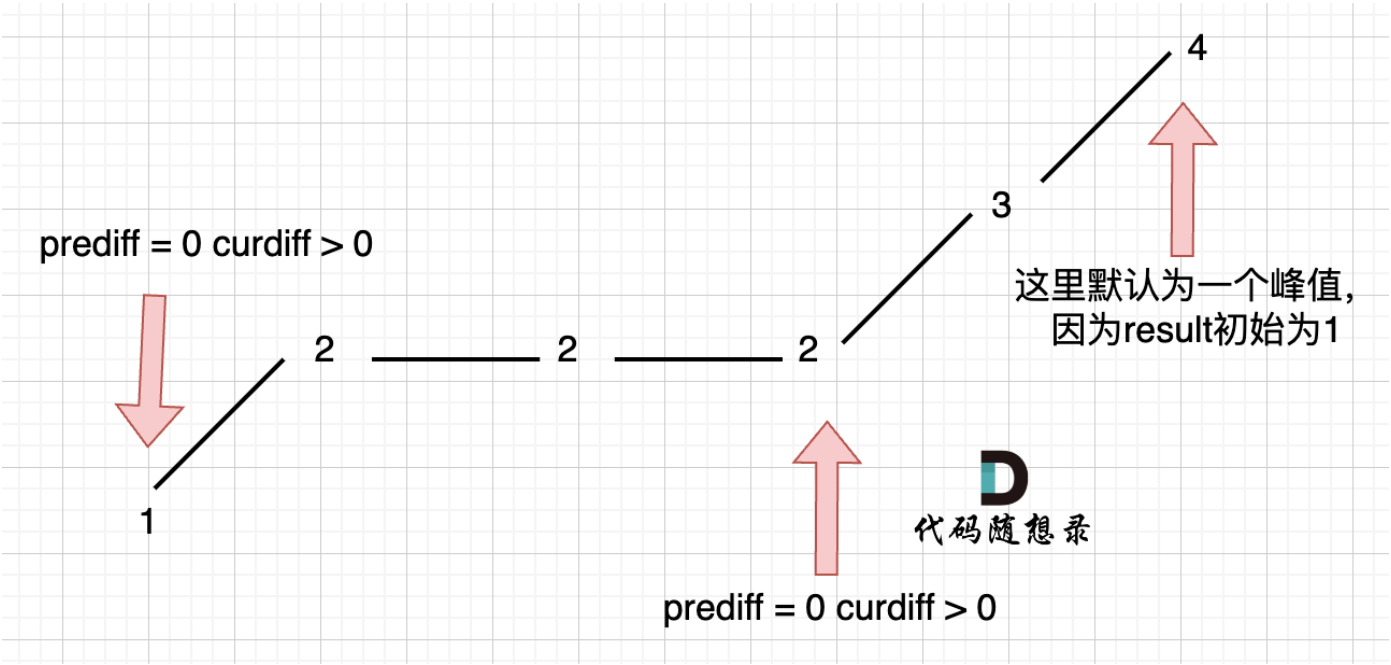
- 时间复杂度：O(n)
- 空间复杂度：O(1)

此时大家是不是发现 以上代码提交也不能通过本题?

所以此时我们要讨论情况三！

情况三：单调坡度有平坡

在版本一中，我们忽略了一种情况，即 如果在一个单调坡度上有平坡，例如[1,2,2,2,3,4]，如图：



图中，我们可以看出，版本一的代码在三个地方记录峰值，但其实结果因为是 2，因为 单调中的平坡 不能算峰值（即摆动）。

之所以版本一会出问题，是因为我们实时更新了 prediff。

那么我们应该什么时候更新 prediff 呢?

我们只需要在 这个坡度 摆动变化的时候，更新 prediff 就行，这样 prediff 在 单调区间有平坡的时候 就不会发生变化，造成我们的误判。

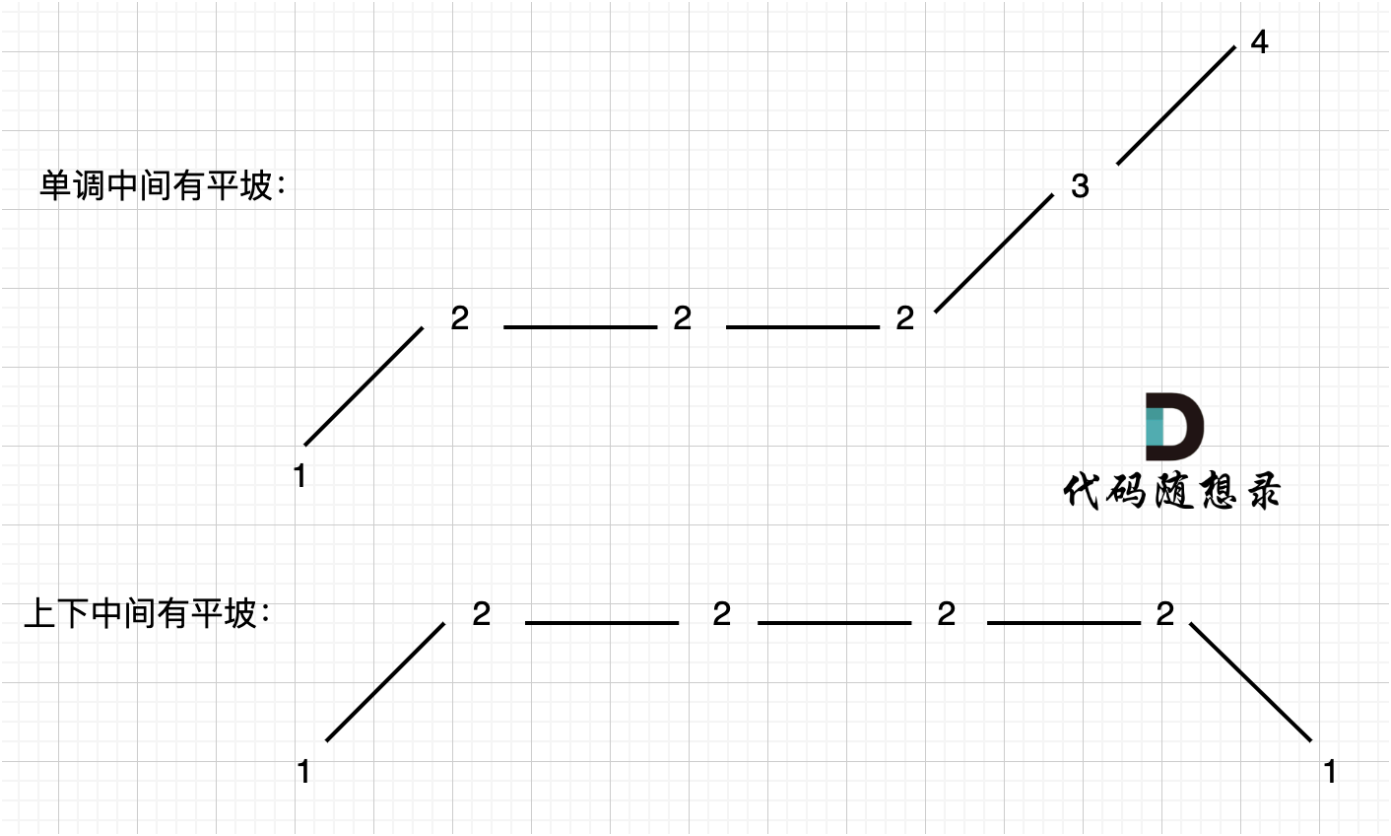
所以本题的最终代码为：

```
// 版本二
class Solution {
public:
    int wiggleMaxLength(vector<int>& nums) {
        if (nums.size() <= 1) return nums.size();
        int curDiff = 0; // 当前一对差值
        int preDiff = 0; // 前一对差值
        int result = 1;  // 记录峰值个数，序列默认序列最右边有一个峰值
        for (int i = 0; i < nums.size() - 1; i++) {
            curDiff = nums[i + 1] - nums[i];
            // 出现峰值
            if ((preDiff <= 0 && curDiff > 0) || (preDiff >= 0 && curDiff < 0)) {
                result++;
                preDiff = curDiff; // 注意这里，只在摆动变化的时候更新prediff
            }
```

```
        }
        return result;
    }
};
```

其实本题看起来好像简单，但需要考虑的情况还是很复杂的，而且很难一次性想到位。

本题异常情况的本质，就是要考虑平坡， 平坡分两种，一个是 上下中间有平坡，一个是单调有平坡，如图：



思路 2（动态规划）

考虑用动态规划的思想来解决这个问题。

很容易可以发现，对于我们当前考虑的这个数，要么是作为山峰（即 nums[i] > nums[i-1]），要么是作为山谷（即 nums[i] < nums[i - 1]）。

- 设 dp 状态`dp[i][0]`，表示考虑前 i 个数，第 i 个数作为山峰的摆动子序列的最长长度
- 设 dp 状态`dp[i][1]`，表示考虑前 i 个数，第 i 个数作为山谷的摆动子序列的最长长度

则转移方程为：

- `dp[i][0] = max(dp[i][0], dp[j][1] + 1)`，其中`0 < j < i`且`nums[j] < nums[i]`，表示将 nums[i]接到前面某个山谷后面，作为山峰。
- `dp[i][1] = max(dp[i][1], dp[j][0] + 1)`，其中`0 < j < i`且`nums[j] > nums[i]`，表示将 nums[i]接到前面某个山峰后面，作为山谷。

初始状态：

由于一个数可以接到前面的某个数后面，也可以以自身为子序列的起点，所以初始状态为：`dp[0][0] = dp[0][1] = 1`。

C++代码如下：

```
class Solution {
public:
    int dp[1005][2];
    int wiggleMaxLength(vector<int>& nums) {
        memset(dp, 0, sizeof dp);
        dp[0][0] = dp[0][1] = 1;
        for (int i = 1; i < nums.size(); ++i) {
            dp[i][0] = dp[i][1] = 1;
            for (int j = 0; j < i; ++j) {
                if (nums[j] > nums[i]) dp[i][1] = max(dp[i][1], dp[j][0] + 1);
            }
            for (int j = 0; j < i; ++j) {
                if (nums[j] < nums[i]) dp[i][0] = max(dp[i][0], dp[j][1] + 1);
            }
        }
        return max(dp[nums.size() - 1][0], dp[nums.size() - 1][1]);
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(n)

进阶

可以用两棵线段树来维护区间的最大值

- 每次更新`dp[i][0]`，则在`tree1`的`nums[i]`位置值更新为`dp[i][0]`
- 每次更新`dp[i][1]`，则在`tree2`的`nums[i]`位置值更新为`dp[i][1]`
- 则 dp 转移方程中就没有必要 j 从 0 遍历到 i-1，可以直接在线段树中查询指定区间的值即可。

时间复杂度：O(nlog n)

空间复杂度：O(n)

4. 最大子序和

力扣题目链接

给定一个整数数组 nums ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

示例:

- 输入: [-2,1,-3,4,-1,2,1,-5,4]
- 输出: 6

- 解释：连续子数组 [4,-1,2,1] 的和最大，为 6。

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法的巧妙需要慢慢体会！LeetCode：53. 最大子序和](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

暴力解法

暴力解法的思路，第一层 for 就是设置起始位置，第二层 for 循环遍历数组寻找最大值

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int result = INT32_MIN;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) { // 设置起始位置
            count = 0;
            for (int j = i; j < nums.size(); j++) { // 每次从起始位置i开始遍历寻找最大值
                count += nums[j];
                result = count > result ? count : result;
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

以上暴力的解法 C++勉强可以过，其他语言就不确定了。

贪心解法

贪心贪的是哪里呢?

如果 -2 1 在一起，计算起点的时候，一定是从 1 开始计算，因为负数只会拉低总和，这就是贪心贪的地方！

局部最优：当前“连续和”为负数的时候立刻放弃，从下一个元素重新计算“连续和”，因为负数加上下一个元素 “连续和”只会越来越小。

全局最优：选取最大“连续和”

局部最优的情况下，并记录最大的“连续和”，可以推出全局最优。

从代码角度上来讲：遍历 nums，从头开始用 count 累积，如果 count 一旦加上 nums[i]变为负数，那么就应该从 nums[i+1]开始从 0 累积 count 了，因为已经变为负数的 count，只会拖累总和。

这相当于是暴力解法中的不断调整最大子序和区间的起始位置。

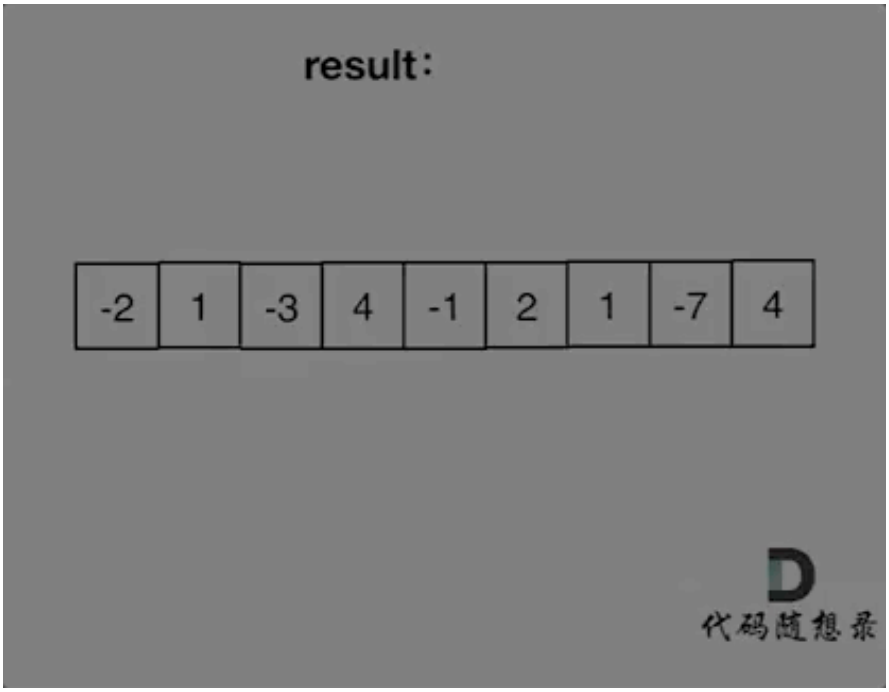
那有同学问了，区间终止位置不用调整么？ 如何才能得到最大“连续和”呢?

区间的终止位置，其实就是如果 count 取到最大值了，及时记录下来了。例如如下代码：

```
if (count > result) result = count;
```

这样相当于是用 result 记录最大子序和区间和（变相的算是调整了终止位置）。

如动画所示：



红色的起始位置就是贪心每次取 count 为正数的时候，开始一个区间的统计。

那么不难写出如下 C++代码（关键地方已经注释）

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int result = INT32_MIN;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) {
            count += nums[i];
            if (count > result) { // 取区间累计的最大值（相当于不断确定最大子序终止位置）
                result = count;
            }
            if (count <= 0) count = 0; // 相当于重置最大子序起始位置，因为遇到负数一定是拉低总和
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

当然题目没有说如果数组为空，应该返回什么，所以数组为空的话返回啥都可以了。

常见误区

误区一：

不少同学认为 如果输入用例都是-1，或者 都是负数，这个贪心算法跑出来的结果是 0， 这是**又一次证明脑洞模拟不靠谱的经典案例**，建议大家把代码运行一下试一试，就知道了，也会理解 为什么 result 要初始化为最小负数了。

误区二：

大家在使用贪心算法求解本题，经常陷入的误区，就是分不清，是遇到 负数就选择起始位置，还是连续和为负选择起始位置。

在动画演示用，大家可以发现， 4，遇到 -1 的时候，我们依然累加了，为什么呢?

因为和为 3，只要连续和还是正数就会 对后面的元素 起到增大总和的作用。 所以只要连续和为正数我们就保留。

这里也会有录友疑惑，那 4 + -1 之后 不就变小了吗？ 会不会错过 4 成为最大连续和的可能性?

其实并不会，因为还有一个变量 result 一直在更新 最大的连续和，只要有更大的连续和出现，result 就更新了，那么 result 已经把 4 更新了，后面 连续和变成 3，也不会对最后结果有影响。

动态规划

当然本题还可以用动态规划来做，在代码随想录动态规划章节我会详细介绍，如果大家想在想看，可以直接跳转：[动态规划版本详解]

那么先给出我的 dp 代码如下，有时间的录友可以提前做一做：

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        vector<int> dp(nums.size(), 0); // dp[i]表示包括i之前的最大连续子序列和
        dp[0] = nums[0];
        int result = dp[0];
        for (int i = 1; i < nums.size(); i++) {
            dp[i] = max(dp[i - 1] + nums[i], nums[i]); // 状态转移公式
            if (dp[i] > result) result = dp[i]; // result 保存dp[i]的最大值
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

总结

本题的贪心思路其实并不好想，这也进一步验证了，别看贪心理论很直白，有时候看似是常识，但贪心的题目一点都不简单！

后续将介绍的贪心题目都挺难的，所以贪心很有意思，别小看贪心！

---

5. 本周小结！（贪心算法系列一）

周一

本周正式开始了贪心算法，在关于贪心算法，你该了解这些！中，我们介绍了什么是贪心以及贪心的套路。

贪心的本质是选择每一阶段的局部最优，从而达到全局最优。

有没有啥套路呢?

不好意思，贪心没套路，就刷题而言，如果感觉好像局部最优可以推出全局最优，然后想不到反例，那就试一试贪心吧！

而严格的数据证明一般有如下两种：

- 数学归纳法
- 反证法

数学就不在讲解范围内了，感兴趣的同学可以自己去查一查资料。

正是因为贪心算法有时候会感觉这是常识，本就应该这么做！ 所以大家经常看到网上有人说这是一道贪心题目，有人说这不是。

这里说一下我的依据：**如果找到局部最优，然后推出整体最优，那么就是贪心**，大家可以参考哈。

周二

在贪心算法：分发饼干中讲解了贪心算法的第一道题目。

这道题目很明显能看出来是用贪心，也是入门好题。

我在文中给出**局部最优：大饼干喂给胃口大的，充分利用饼干尺寸喂饱一个，全局最优：喂饱尽可能多的小孩。**

很多录友都是用小饼干优先先喂饱小胃口的。

后来我想一想，虽然结果是一样的，但是大家的这个思考方式更好一些。

因为用小饼干优先喂饱小胃口的 这样可以尽量保证最后省下来的是大饼干（虽然题目没有这个要求）！

所以还是小饼干优先先喂饱小胃口更好一些，也比较直观。

一些录友不清楚贪心算法：分发饼干中时间复杂度是怎么来的?

就是快排O(nlog n)，遍历O(n)，加一起就是还是O(nlogn)。

周三

接下来就要上一点难度了，要不然大家会误以为贪心算法就是常识判断一下就行了。

在贪心算法：摆动序列中，需要计算最长摇摆序列。

其实就是让序列有尽可能多的局部峰值。

局部最优：删除单调坡度上的节点（不包括单调坡度两端的节点），那么这个坡度就可以有两个局部峰值。

整体最优：整个序列有最多的局部峰值，从而达到最长摆动序列。

在计算峰值的时候，还是有一些代码技巧的，例如序列两端的峰值如何处理。

这些技巧，其实还是要多看多用才会掌握。

周四

在贪心算法：最大子序和中，详细讲解了用贪心的方式来求最大子序列和，其实这道题目是一道动态规划的题目。

贪心的思路为局部最优：当前“连续和”为负数的时候立刻放弃，从下一个元素重新计算“连续和”，因为负数加上下一个元素 “连续和”只会越来越小。从而推出全局最优：选取最大“连续和”

代码很简单，但是思路却比较难。还需要反复琢磨。

针对贪心算法：最大子序和文章中给出的贪心代码如下；

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int result = INT32_MIN;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) {
            count += nums[i];
            if (count > result) { // 取区间累计的最大值（相当于不断确定最大子序终止位置）
                result = count;
            }
            if (count <= 0) count = 0; // 相当于重置最大子序起始位置，因为遇到负数一定是拉低总和
        }
        return result;
    }
};
```

不少同学都来问，如果数组全是负数这个代码就有问题了，如果数组里有int最小值这个代码就有问题了。

大家不要脑洞模拟哈，可以亲自构造一些测试数据试一试，就发现其实没有问题。

数组都为负数，result记录的就是最小的负数，如果数组里有int最小值，那么最终result就是int最小值。

总结

本周我们讲解了贪心算法的理论基础，了解了贪心本质：局部最优推出全局最优。

然后讲解了第一道题目分发饼干，还是比较基础的，可能会给大家一种贪心算法比较简单的错觉，因为贪心有时候接近于常识。

其实我还准备一些简单的贪心题目，甚至网上很多都质疑这些题目是不是贪心算法。这些题目我没有立刻发出来，因为真的会让大家感觉贪心过于简单，而忽略了贪心的本质：局部最优和全局最优两个关键点。

所以我在贪心系列难度会有所交替，难的题目在于拓展思路，简单的题目在于分析清楚其贪心的本质，后续我还会发一些简单的题目来做贪心的分析。

在摆动序列中大家就初步感受到贪心没那么简单了。

本周最后是最大子序和，这道题目要用贪心的方式做出来，就比较有难度，都知道负数加上正数之后会变小，但是这道题目依然会让很多人搞混淆，其关键在于：**不能让“连续和”为负数的时候加上下一个元素，而不是 不让“连续和”加上一个负数**。这块真的需要仔细体会！

6.买卖股票的最佳时机 II

力扣题目链接

给定一个数组，它的第 i 个元素是一支给定股票第 i 天的价格。

设计一个算法来计算你所能获取的最大利润。你可以尽可能地完成更多的交易（多次买卖一支股票）。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

- 输入: [7,1,5,3,6,4]
- 输出: 7
- 解释: 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5-1 = 4。随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出, 这笔交易所能获得利润 = 6-3 = 3 。

示例 2:

- 输入: [1,2,3,4,5]
- 输出: 4
- 解释: 在第 1 天（股票价格 = 1）的时候买入，在第 5 天 （股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5-1 = 4 。注意你不能在第 1 天和第 2 天接连购买股票，之后再将它们卖出。因为这样属于同时参与了多笔交易，你必须在再次购买前出售掉之前的股票。

示例 3:

- 输入: [7,6,4,3,1]
- 输出: 0
- 解释: 在这种情况下, 没有交易完成, 所以最大利润为 0。

提示：

- 1 <= prices.length <= 3 * 10 ^ 4

- 0 <= prices[i] <= 10 ^ 4

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法也能解决股票问题！LeetCode：122.买卖股票最佳时机 II](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题首先要清楚两点：

- 只有一只股票！
- 当前只有买股票或者卖股票的操作

想获得利润至少要两天为一个交易单元。

贪心算法

这道题目可能我们只会想，选一个低的买入，再选个高的卖，再选一个低的买入.....循环反复。

如果想到其实最终利润是可以分解的，那么本题就很容易了！

如何分解呢?

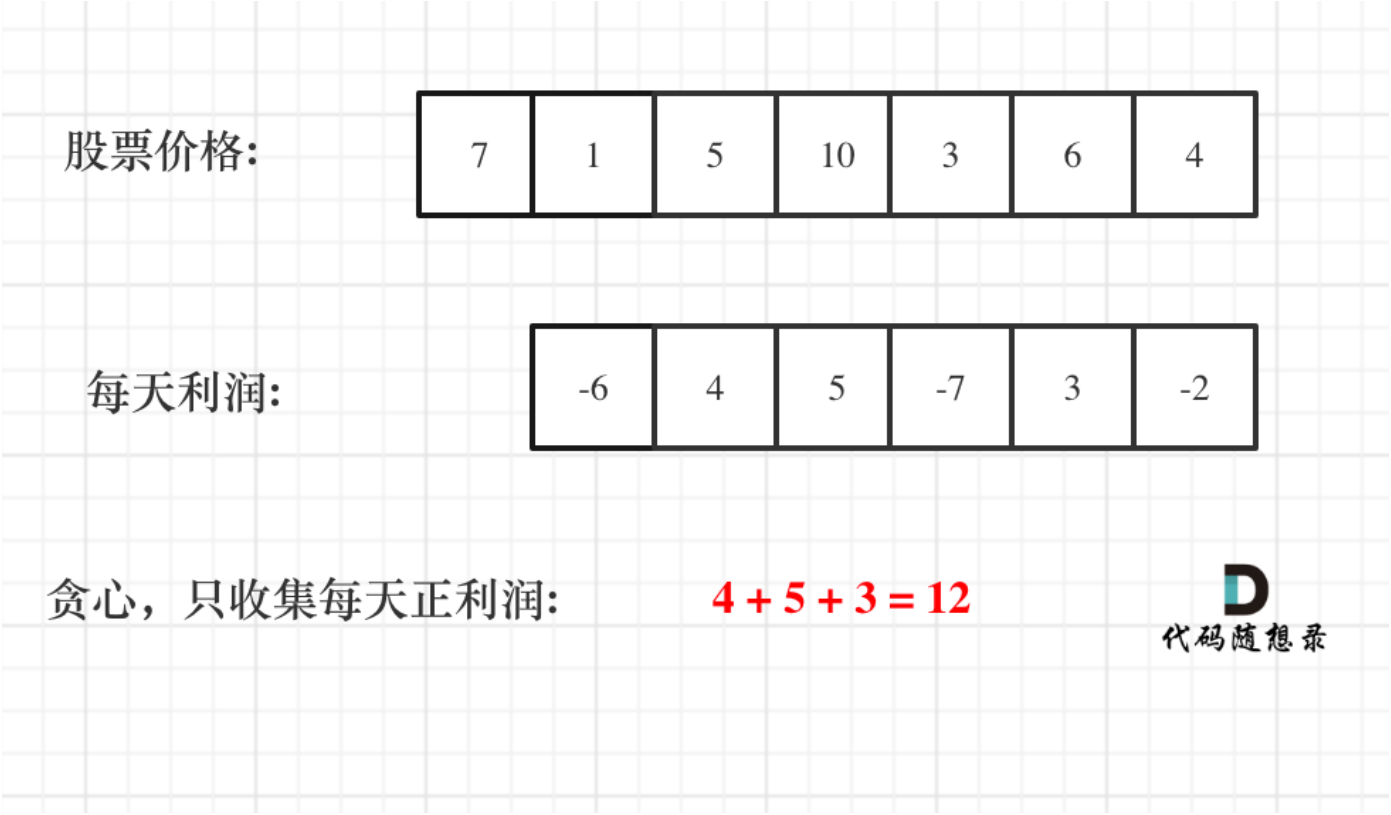
假如第 0 天买入，第 3 天卖出，那么利润为：prices[3] - prices[0]。

相当于(prices[3] - prices[2]) + (prices[2] - prices[1]) + (prices[1] - prices[0])。

此时就是把利润分解为每天为单位的维度，而不是从 0 天到第 3 天整体去考虑！

那么根据 prices 可以得到每天的利润序列：(prices[i] - prices[i - 1]).....(prices[1] - prices[0])。

如图：



一些同学陷入：第一天怎么就没有利润呢，第一天到底算不算的困惑中。

第一天当然没有利润，至少要第二天才会有利润，所以利润的序列比股票序列少一天！

从图中可以发现，其实我们需要收集每天的正利润就可以，**收集正利润的区间，就是股票买卖的区间，而我们只需要关注最终利润，不需要记录区间**。

那么只收集正利润就是贪心所贪的地方！

局部最优：收集每天的正利润，全局最优：求得最大利润。

局部最优可以推出全局最优，找不出反例，试一试贪心！

对应 C++代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int result = 0;
        for (int i = 1; i < prices.size(); i++) {
            result += max(prices[i] - prices[i - 1], 0);
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

动态规划

动态规划将在下一个系列详细讲解，本题解先给出我的 C++代码（带详细注释），想先学习的话，可以看本篇：[122.买卖股票的最佳时机II（动态规划）]

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // dp[i][1]第i天持有的最多现金
        // dp[i][0]第i天持有股票后的最多现金
        int n = prices.size();
        vector<vector<int>> dp(n, vector<int>(2, 0));
        dp[0][0] -= prices[0]; // 持股票
        for (int i = 1; i < n; i++) {
            // 第i天持股票所剩最多现金 = max(第i-1天持股票所剩现金, 第i-1天持现金-买第i天的股票)
            dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
            // 第i天持有最多现金 = max(第i-1天持有的最多现金, 第i-1天持有股票的最多现金+第i天卖出股票)
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i]);
        }
        return max(dp[n - 1][0], dp[n - 1][1]);
    }
};
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(n)$

总结

股票问题其实是一个系列的，属于动态规划的范畴，因为目前在讲解贪心系列，所以股票问题会在之后的动态规划系列中详细讲解。

可以看出有时候，贪心往往比动态规划更巧妙，更好用，所以别小看了贪心算法。

本题中理解利润拆分是关键点！ 不要整块的去看，而是把整体利润拆为每天的利润。

一旦想到这里了，很自然就会想到贪心了，即：只收集每天的正利润，最后稳稳的就是最大利润了。

---

7. 跳跃游戏

[力扣题目链接]

给定一个非负整数数组，你最初位于数组的第一个位置。

数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个位置。

示例 1:

- 输入: [2,3,1,1,4]
- 输出: true
- 解释: 我们可以先跳 1 步，从位置 0 到达 位置 1, 然后再从位置 1 跳 3 步到达最后一个位置。

示例 2:

- 输入: [3,2,1,0,4]
- 输出: false
- 解释: 无论怎样，你总会到达索引为 3 的位置。但该位置的最大跳跃长度是 0 ， 所以你永远不可能到达最后一个位置。

算法公开课

[《代码随想录》算法视频公开课](): [贪心算法，怎么跳跃不重要，关键在覆盖范围 | LeetCode：55.跳跃游戏](), 相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

刚看到本题一开始可能想：当前位置元素如果是 3，我究竟是跳一步呢，还是两步呢，还是三步呢，究竟跳几步才是最优呢?

其实跳几步无所谓，关键在于可跳的覆盖范围!

不一定非要明确一次究竟跳几步，每次取最大的跳跃步数，这个就是可以跳跃的覆盖范围。

这个范围内，别管是怎么跳的，反正一定可以跳过来。

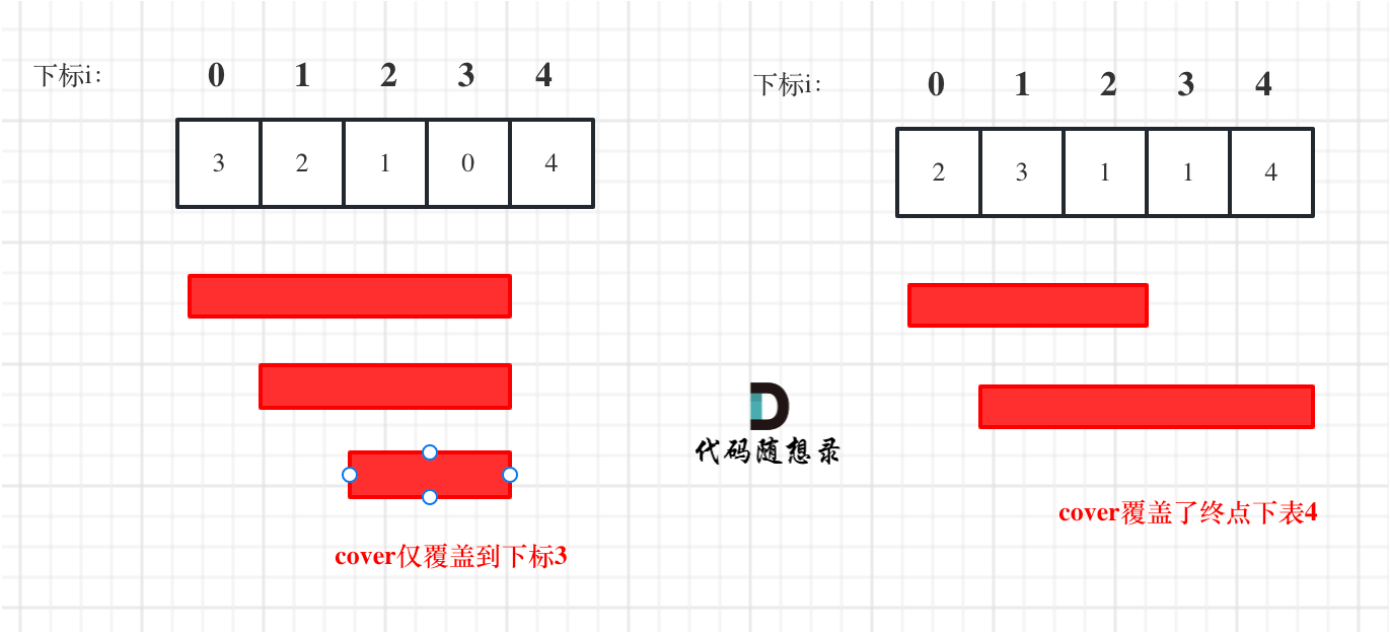
那么这个问题就转化为跳跃覆盖范围究竟可不可以覆盖到终点!

每次移动取最大跳跃步数（得到最大的覆盖范围），每移动一个单位，就更新最大覆盖范围。

贪心算法局部最优解：每次取最大跳跃步数（取最大覆盖范围），整体最优解：最后得到整体最大覆盖范围，看是否能到终点。

局部最优推出全局最优，找不出反例，试试贪心!

如图：



i 每次移动只能在 cover 的范围内移动，每移动一个元素，cover 得到该元素数值（新的覆盖范围）的补充，让 i 继续移动下去。

而 cover 每次只取 max(该元素数值补充后的范围, cover 本身范围)。

如果 cover 大于等于了终点下标，直接 return true 就可以了。

C++代码如下：

```
class Solution {
public:
    bool canJump(vector<int>& nums) {
        int cover = 0;
        if (nums.size() == 1) return true; // 只有一个元素，就是能达到
        for (int i = 0; i <= cover; i++) { // 注意这里是小于等于cover
            cover = max(i + nums[i], cover);
            if (cover >= nums.size() - 1) return true; // 说明可以覆盖到终点了
        }
        return false;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

总结

这道题目关键点在于：不用拘泥于每次究竟跳几步，而是看覆盖范围，覆盖范围内一定是可以跳过来的，不用管是怎么跳的。

大家可以看出思路想出来了，代码还是非常简单的。

一些同学可能感觉，我在讲贪心系列的时候，题目和题目之间貌似没有什么联系？

是真的就是没什么联系，因为贪心无套路！没有个整体的贪心框架解决一系列问题，只能是接触各种类型的题目锻炼自己的贪心思维！

> 相对于贪心算法：跳跃游戏难了不少，做好心里准备！

8.跳跃游戏 II

力扣题目链接

给定一个非负整数数组，你最初位于数组的第一个位置。

数组中的每个元素代表你在该位置可以跳跃的最大长度。

你的目标是使用最少的跳跃次数到达数组的最后一个位置。

示例:

- 输入: [2,3,1,1,4]
- 输出: 2
- 解释: 跳到最后一个位置的最小跳跃数是 2。从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置。

说明:
假设你总是可以到达数组的最后一个位置。

算法公开课

《代码随想录》算法视频公开课：贪心算法，最少跳几步还得看覆盖范围 | LeetCode： 45.跳跃游戏 II，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题相对于55.跳跃游戏还是难了不少。

但思路是相似的，还是要看最大覆盖范围。

本题要计算最小步数，那么就要想清楚什么时候步数才一定要加一呢?

贪心的思路，局部最优：当前可移动距离尽可能多走，如果还没到终点，步数再加一。整体最优：一步尽可能多走，从而达到最小步数。

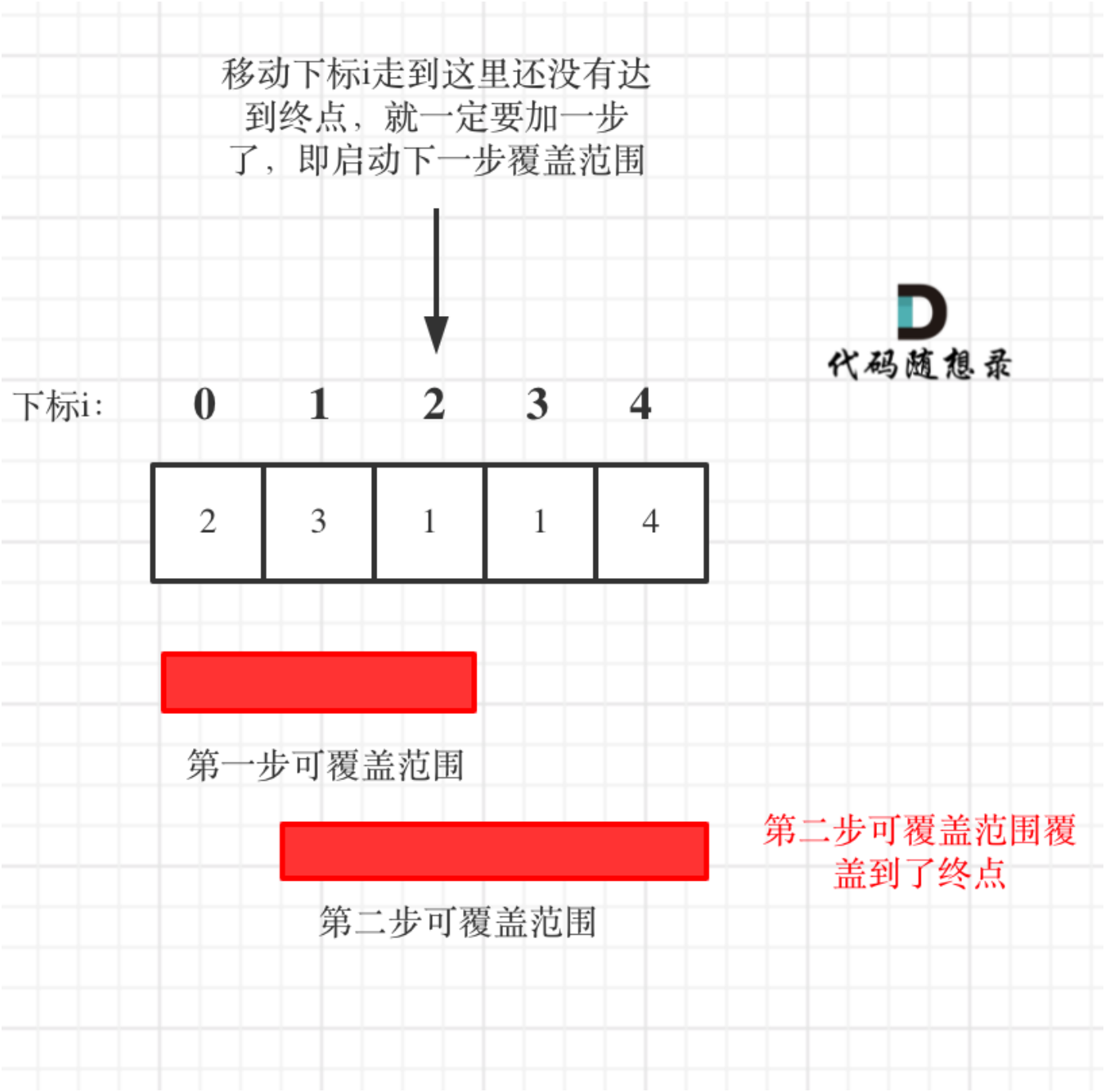
思路虽然是这样，但在写代码的时候还不能真的能跳多远就跳多远，那样就不知道下一步最远能跳到哪里了。

所以真正解题的时候，要从覆盖范围出发，不管怎么跳，覆盖范围内一定是可以跳到的，以最小的步数增加覆盖范围，覆盖范围一旦覆盖了终点，得到的就是最小步数！

这里需要统计两个覆盖范围，当前这一步的最大覆盖和下一步最大覆盖。

如果移动下标达到了当前这一步的最大覆盖最远距离了，还没有到终点的话，那么就必须再走一步来增加覆盖范围，直到覆盖范围覆盖了终点。

如图：



图中覆盖范围的意义在于，只要红色的区域，最多两步一定可以到！（不用管具体怎么跳，反正一定可以跳到）

方法一

从图中可以看出来，就是移动下标达到了当前覆盖的最远距离下标时，步数就要加一，来增加覆盖距离。最后的步数就是最少步数。

这里还是有个特殊情况需要考虑，当移动下标达到了当前覆盖的最远距离下标时

- 如果当前覆盖最远距离下标不是是集合终点，步数就加一，还需要继续走。
- 如果当前覆盖最远距离下标就是是集合终点，步数不用加一，因为不能再往后走了。

C++代码如下：（详细注释）

```
// 版本一
class Solution {
public:
    int jump(vector<int>& nums) {
        if (nums.size() == 1) return 0;
        int curDistance = 0;    // 当前覆盖最远距离下标
        int ans = 0;            // 记录走的最大步数
        int nextDistance = 0;   // 下一步覆盖最远距离下标
        for (int i = 0; i < nums.size(); i++) {
            nextDistance = max(nums[i] + i, nextDistance);  // 更新下一步覆盖最远距离下标
            if (i == curDistance) {                         // 遇到当前覆盖最远距离下标
                ans++;                                  // 需要走下一步
                curDistance = nextDistance;             // 更新当前覆盖最远距离下标（相当于加油了）
                if (nextDistance >= nums.size() - 1) break;  // 当前覆盖最远距到达集合终点，不用做ans++操作了，直接结束
            }
        }
        return ans;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

方法二

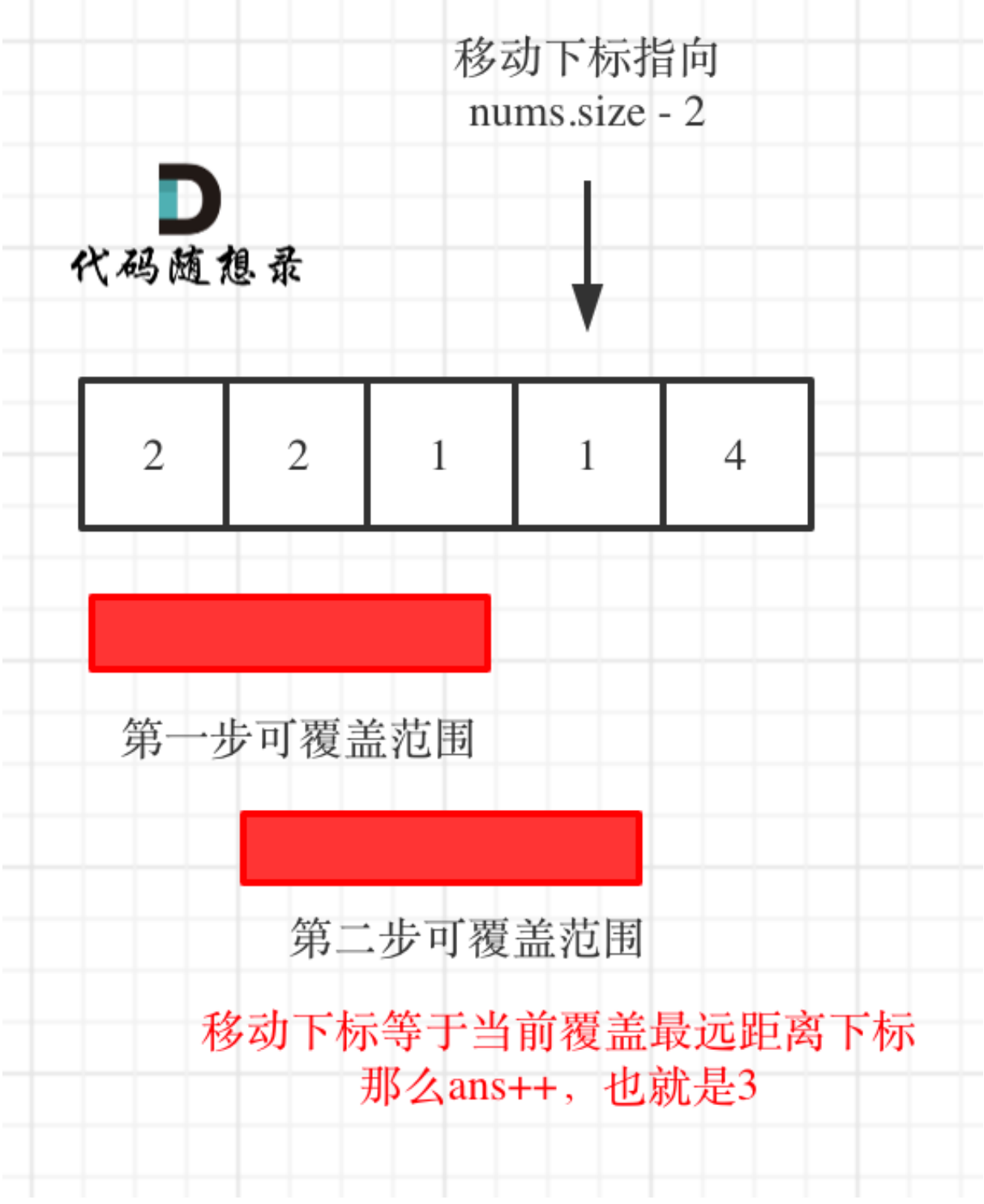
依然是贪心，思路和方法一差不多，代码可以简洁一些。

针对于方法一的特殊情况，可以统一处理，即：移动下标只要遇到当前覆盖最远距离的下标，直接步数加一，不考虑是不是终点的情况。

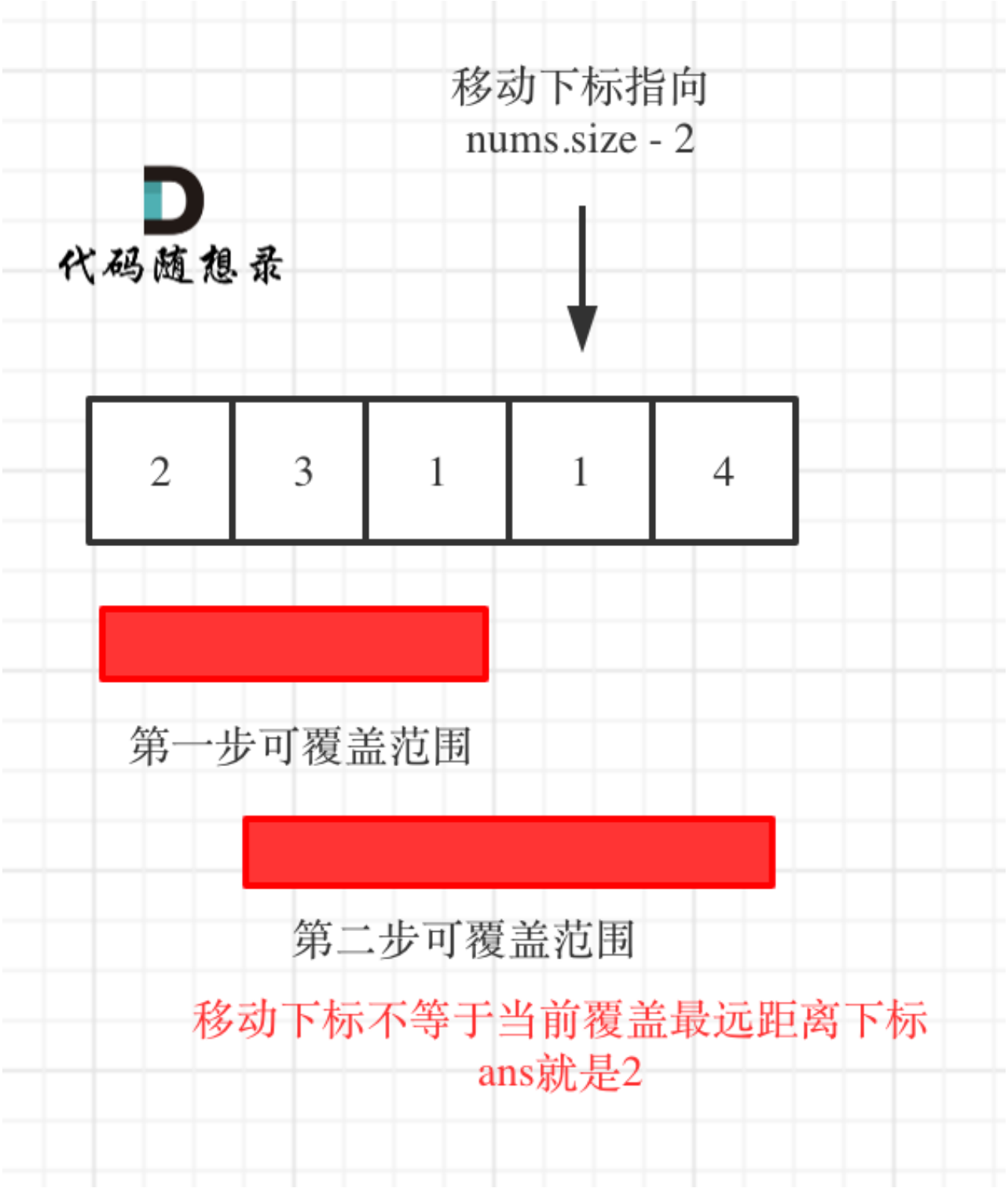
想要达到这样的效果，只要让移动下标，最大只能移动到 nums.size - 2 的地方就可以了。

因为当移动下标指向 nums.size - 2 时：

- 如果移动下标等于当前覆盖最大距离下标， 需要再走一步（即 ans++），因为最后一步一定是可以到的终点。（题目假设总是可以到达数组的最后一个位置），如图：



- 如果移动下标不等于当前覆盖最大距离下标，说明当前覆盖最远距离就可以直接达到终点了，不需要再走一步。如图：



代码如下：

```
// 版本二
class Solution {
public:
    int jump(vector<int>& nums) {
        int curDistance = 0;    // 当前覆盖的最远距离下标
        int ans = 0;            // 记录走的最大步数
        int nextDistance = 0;   // 下一步覆盖的最远距离下标
        for (int i = 0; i < nums.size() - 1; i++) { // 注意这里是小于nums.size() - 1，这是关键所在
```

```
            nextDistance = max(nums[i] + i, nextDistance); // 更新下一步覆盖的最远距离下标
            if (i == curDistance) {                 // 遇到当前覆盖的最远距离下标
                curDistance = nextDistance;         // 更新当前覆盖的最远距离下标
                ans++;
            }
        }
        return ans;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

可以看出版本二的代码相对于版本一简化了不少！

其精髓在于控制移动下标 i 只移动到 nums.size() - 2 的位置，所以移动下标只要遇到当前覆盖最远距离的下标，直接步数加一，不用考虑别的了。

总结

相信大家可以发现，这道题目相当于55.跳跃游戏难了不止一点。

但代码又十分简单，贪心就是这么巧妙。

理解本题的关键在于：**以最小的步数增加最大的覆盖范围，直到覆盖范围覆盖了终点**，这个范围内最小步数一定可以跳到，不用管具体是怎么跳的，不纠结于一步究竟跳一个单位还是两个单位。

9.K次取反后最大化的数组和

力扣题目链接

给定一个整数数组 A，我们只能用以下方法修改该数组：我们选择某个索引 i 并将 A[i] 替换为 -A[i]，然后总共重复这个过程 K 次。（我们可以多次选择同一个索引 i。）

以这种方式修改数组后，返回数组可能的最大和。

示例 1：

- 输入：A = [4,2,3], K = 1
- 输出：5
- 解释：选择索引 (1,) ，然后 A 变为 [4,-2,3]。

示例 2：

- 输入：A = [3,-1,0,2], K = 3

- 输出：6
- 解释：选择索引 (1, 2, 2) ，然后 A 变为 [3,1,0,2]。

示例 3：

- 输入：A = [2,-3,-1,5,-4], K = 2
- 输出：13
- 解释：选择索引 (1, 4) ，然后 A 变为 [2,3,-1,5,4]。

提示：

- 1 <= A.length <= 10000
- 1 <= K <= 10000
- -100 <= A[i] <= 100

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法，这不就是常识？还能叫贪心？LeetCode：1005.K次取反后最大化的数组和](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题思路其实比较好想了，如何可以让数组和最大呢?

贪心的思路，局部最优：让绝对值大的负数变为正数，当前数值达到最大，整体最优：整个数组和达到最大。

局部最优可以推出全局最优。

那么如果将负数都转变为正数了，K依然大于0，此时的问题是一个有序正整数序列，如何转变K次正负，让 数组和达到最大。

那么又是一个贪心：局部最优：只找数值最小的正整数进行反转，当前数值和可以达到最大（例如正整数数组{5, 3, 1}，反转1 得到-1 比 反转5得到的-5 大多了），全局最优：整个 数组和 达到最大。

虽然这道题目大家做的时候，可能都不会去想什么贪心算法，一鼓作气，就AC了。

我这里其实是为了给大家展现出来 经常被大家忽略的贪心思路，这么一道简单题，就用了两次贪心！

那么本题的解题步骤为：

- 第一步：将数组按照绝对值大小从大到小排序，**注意要按照绝对值的大小**
- 第二步：从前向后遍历，遇到负数将其变为正数，同时K--
- 第三步：如果K还大于0，那么反复转变数值最小的元素，将K用完
- 第四步：求和

对应C++代码如下：

```
class Solution {
static bool cmp(int a, int b) {
    return abs(a) > abs(b);
}
```

```
public:
    int largestSumAfterKNegations(vector<int>& A, int K) {
        sort(A.begin(), A.end(), cmp);        // 第一步
        for (int i = 0; i < A.size(); i++) { // 第二步
            if (A[i] < 0 && K > 0) {
                A[i] *= -1;
                K--;
            }
        }
        if (K % 2 == 1) A[A.size() - 1] *= -1; // 第三步
        int result = 0;
        for (int a : A) result += a;         // 第四步
        return result;
    }
};
```

- 时间复杂度: O(nlogn)
- 空间复杂度: O(1)

总结

贪心的题目如果简单起来，会让人简单到开始怀疑：本来不就应该这么做么？这也算是算法？我认为这不是贪心？

本题其实很简单，不会贪心算法的同学都可以做出来，但是我还是全程用贪心的思路来讲解。

因为贪心的思考方式一定要有！

如果没有贪心的思考方式（局部最优，全局最优），很容易陷入贪心简单题凭感觉做，贪心难题直接不会做，其实这样就锻炼不了贪心的思考方式了。

所以明知道是贪心简单题，也要靠贪心的思考方式来解题，这样对培养解题感觉很有帮助。

---

10. 本周小结！（贪心算法系列二）

周一

一说到股票问题，一般都会想到动态规划，其实有时候贪心更有效！

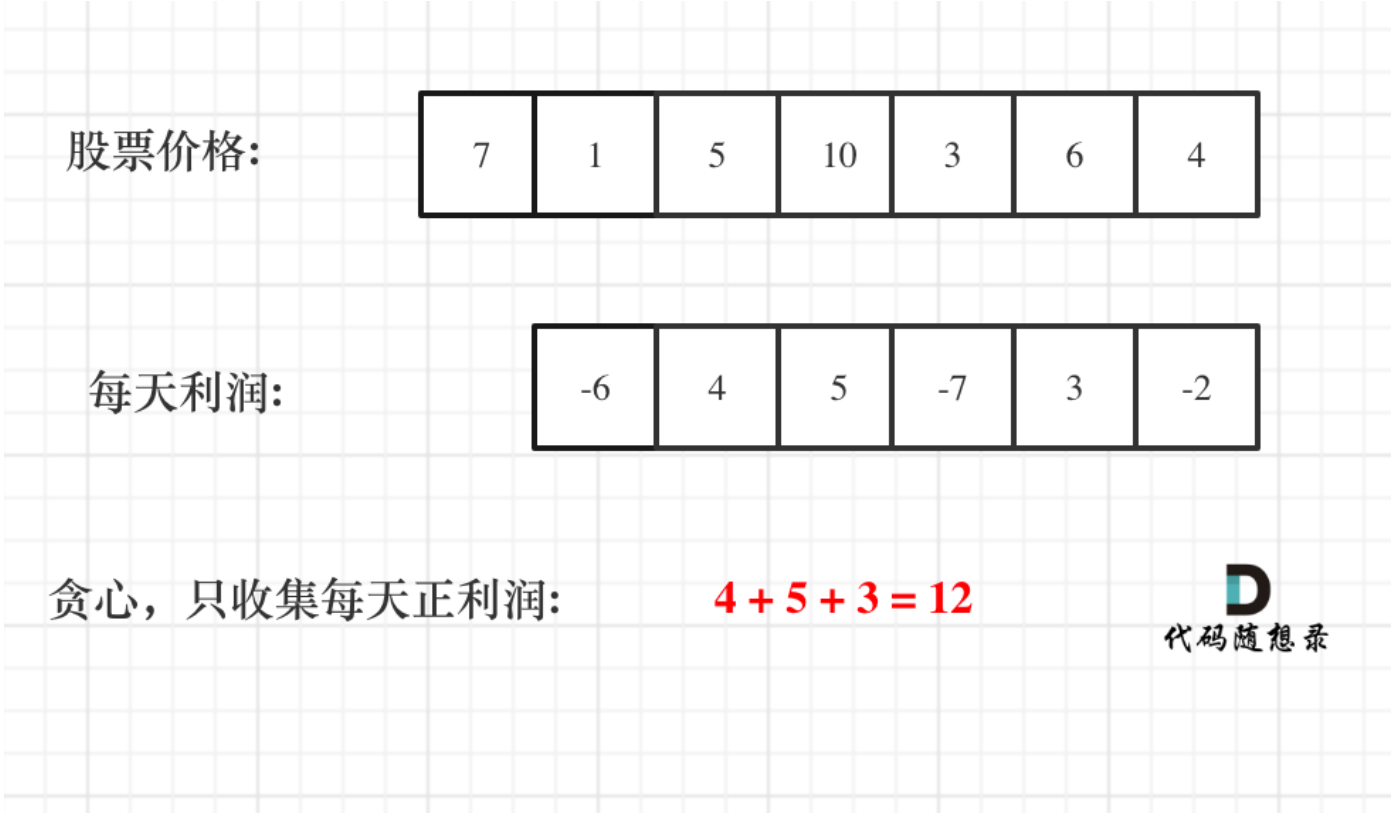
在贪心算法：买卖股票的最佳时机II中，讲到只能多次买卖一支股票，如何获取最大利润。

这道题目理解利润拆分是关键点！ 不要整块的去看，而是把整体利润拆为每天的利润，就很容易想到贪心了。

局部最优：只收集每天的正利润，全局最优：得到最大利润。

如果正利润连续上了，相当于连续持有股票，而本题并不需要计算具体的区间。

如图：



周二

在贪心算法：跳跃游戏中是给你一个数组看能否跳到终点。

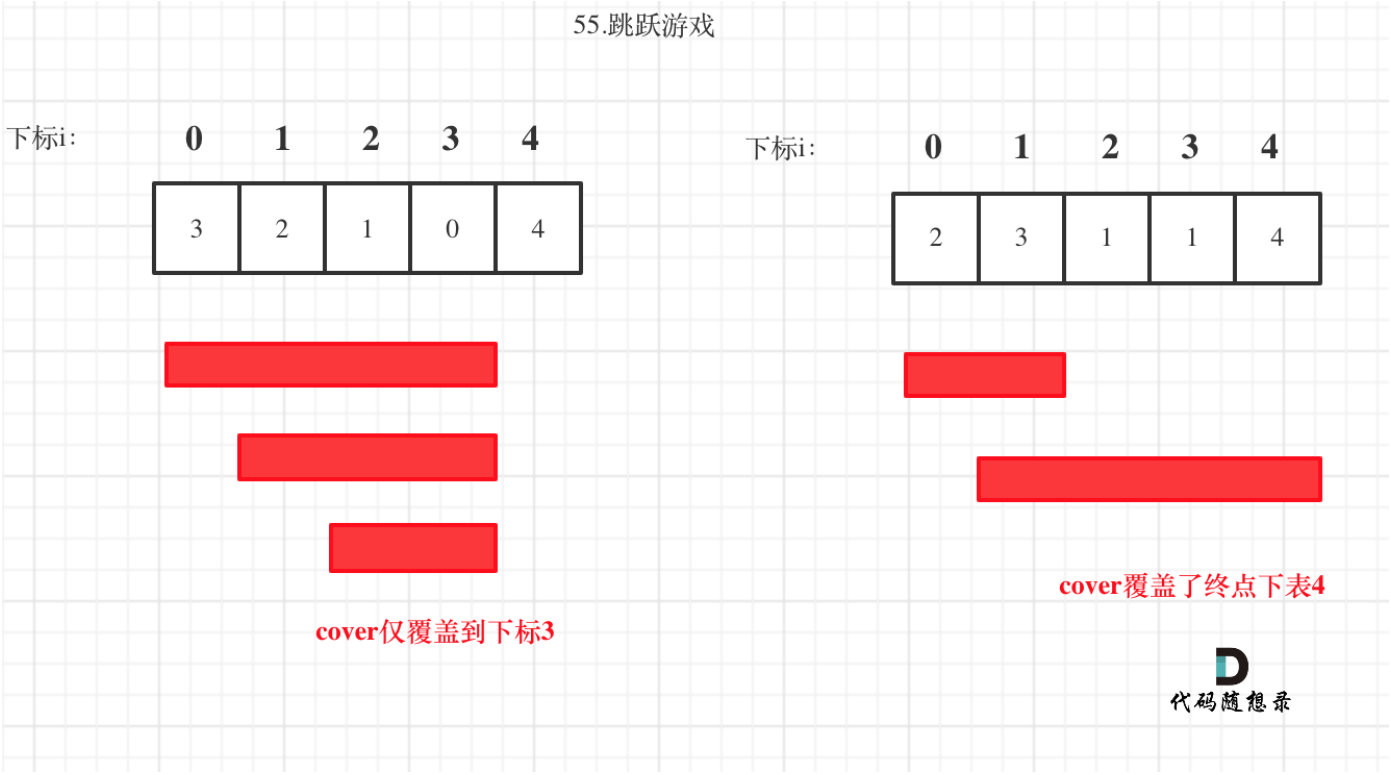
本题贪心的关键是：**不用拘泥于每次究竟跳几步，而是看覆盖范围，覆盖范围内一定是可以跳过来的，不用管是怎么跳的。**

那么这个问题就转化为跳跃覆盖范围究竟可不可以覆盖到终点！

贪心算法局部最优解：移动下标每次取最大跳跃步数（取最大覆盖范围），整体最优解：最后得到整体最大覆盖范围，看是否能到终点

如果覆盖范围覆盖到了终点，就表示一定可以跳过去。

如图：



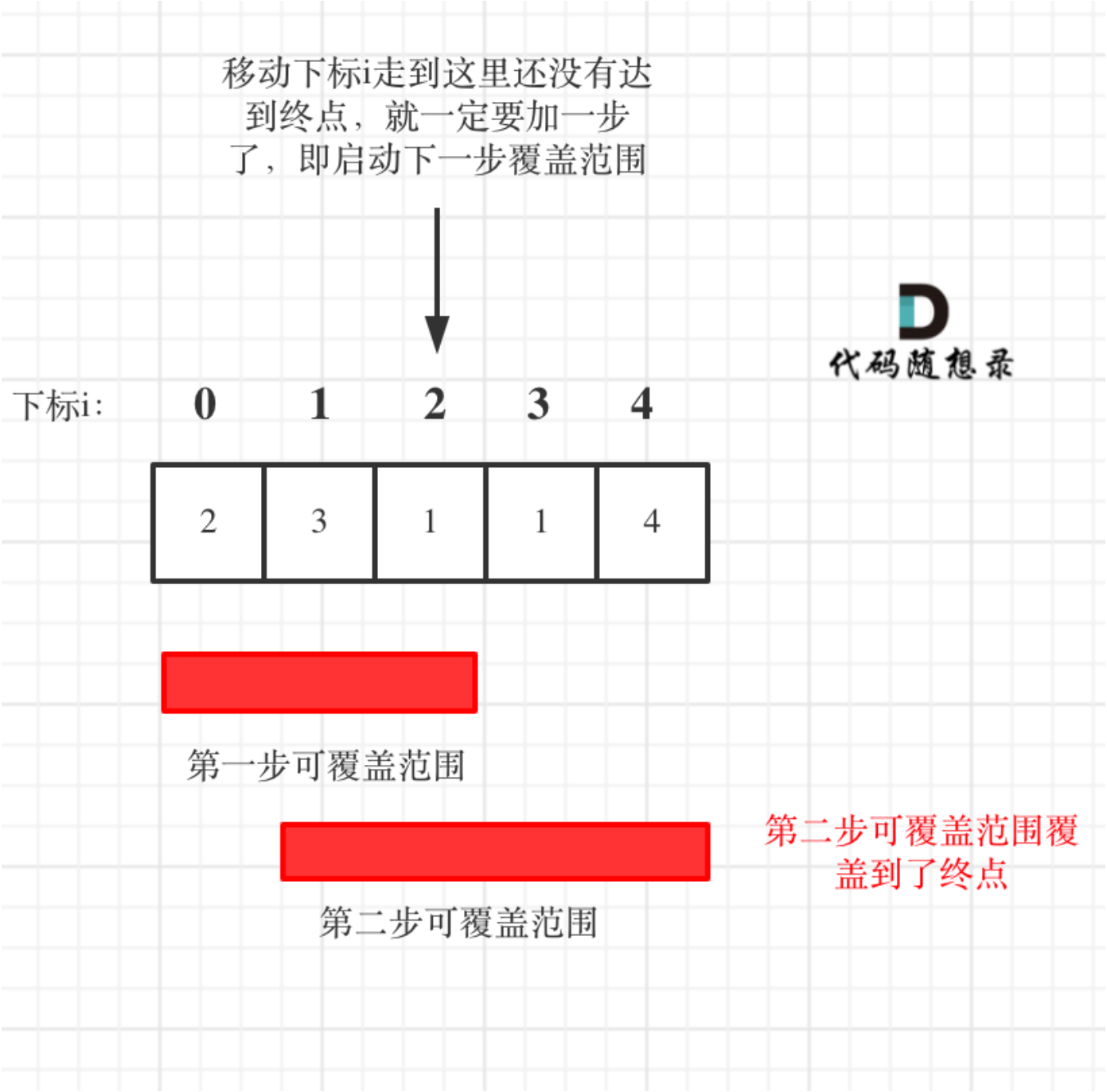
周三

这道题目：贪心算法：跳跃游戏II可就有点难了。

本题解题关键在于：**以最小的步数增加最大的覆盖范围，直到覆盖范围覆盖了终点。**

那么局部最优：求当前这步的最大覆盖，那么尽可能多走，到达覆盖范围的终点，只需要一步。整体最优：达到终点，步数最少。

如图：



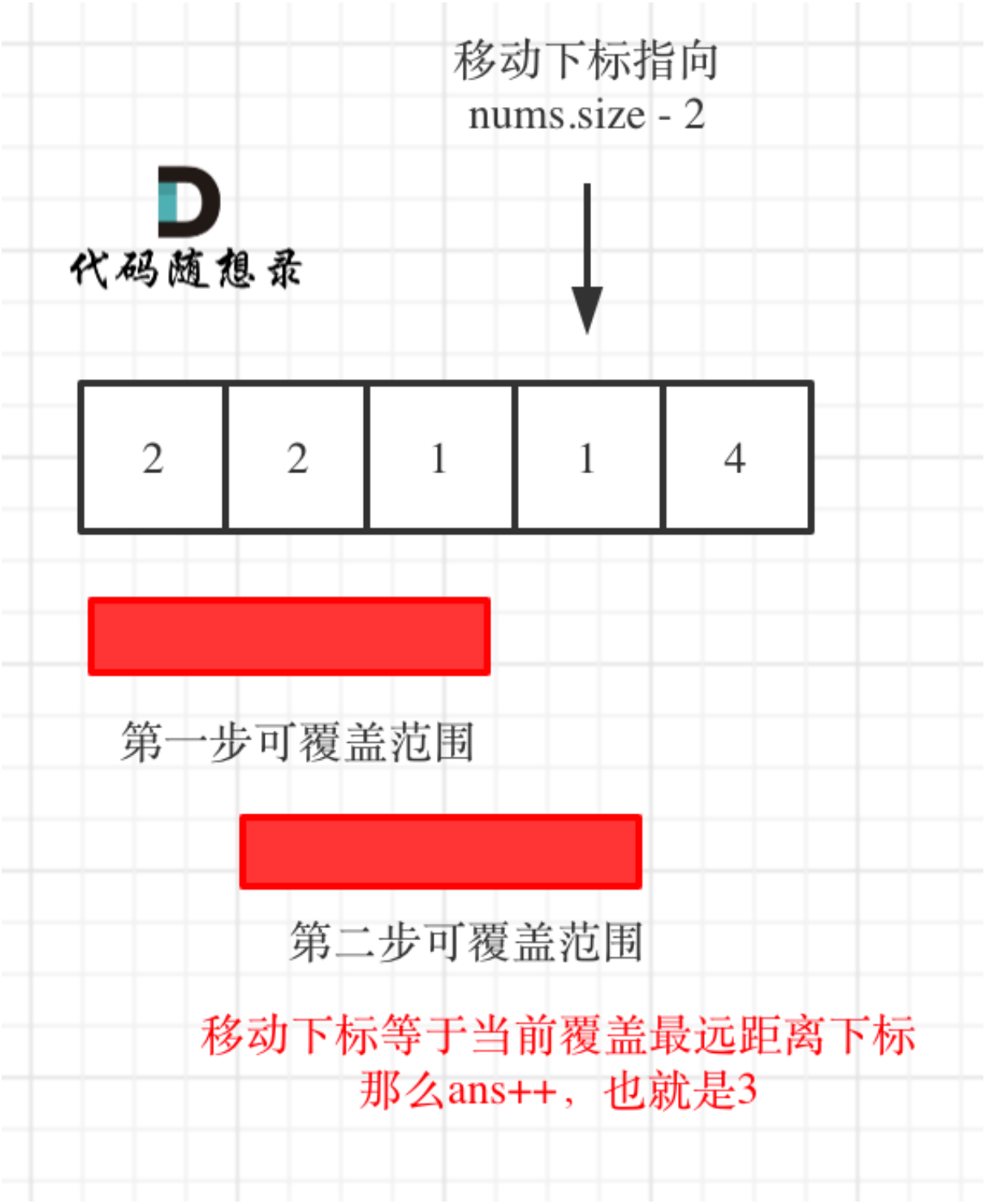
注意：**图中的移动下标是到当前这步覆盖的最远距离（下标2的位置），此时没有到终点，只能增加第二步来扩大覆盖范围。**

在贪心算法：跳跃游戏II中我给出了两个版本的代码。

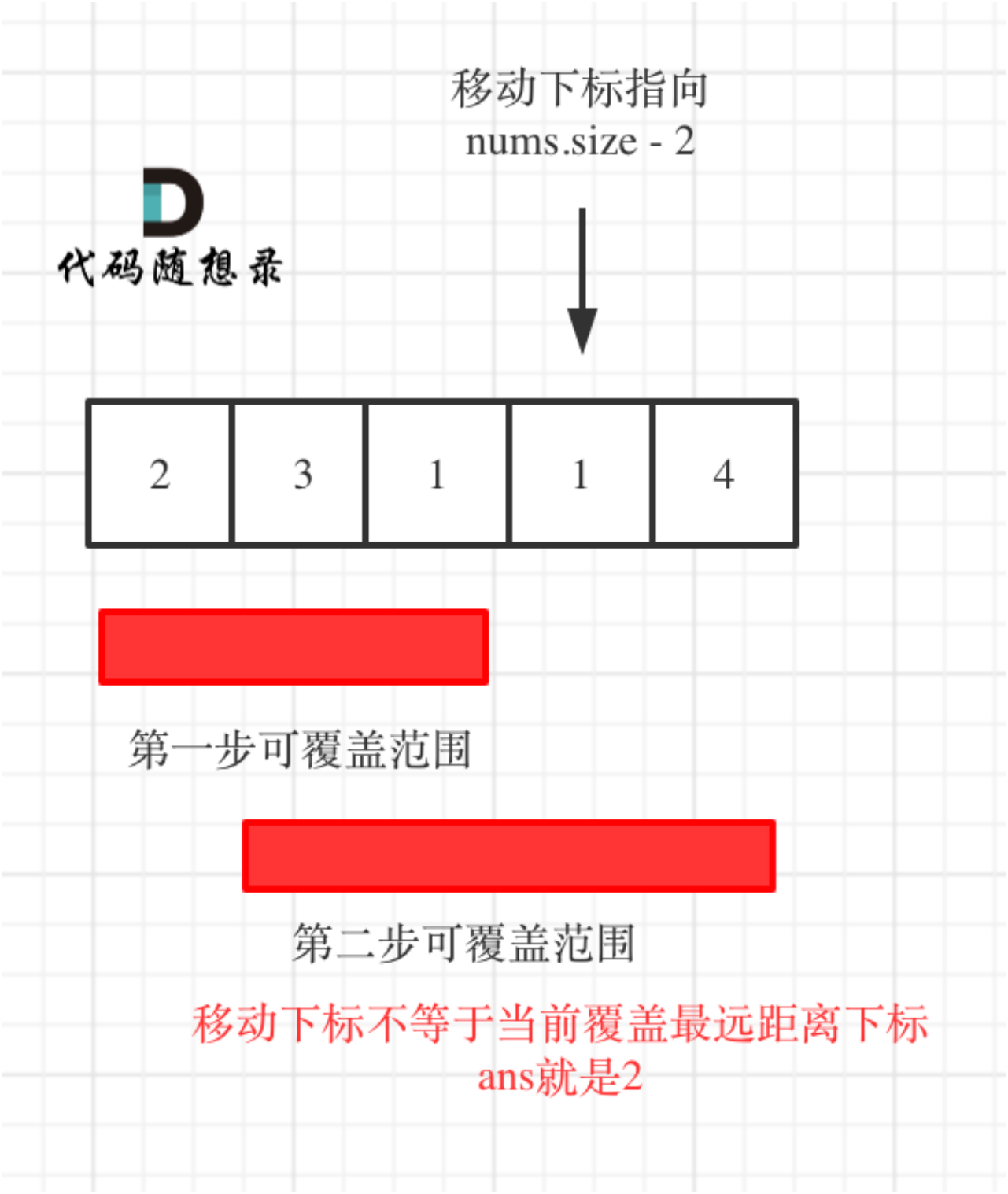
其实本质都是超过当前覆盖范围，步数就加一，但版本一需要考虑当前覆盖最远距离下标是不是数组终点的情况。

而版本二就比较统一的，超过范围，步数就加一，但在移动下标的范围了做了文章。

即如果覆盖最远距离下标是倒数第二点：直接加一就行，默认一定可以到终点。如图：



如果覆盖最远距离下标不是倒数第二点，说明本次覆盖已经到终点了。如图：



有的录友认为版本一好理解，有的录友认为版本二好理解，其实掌握一种就可以了，也不用非要比拼一下代码的简洁性，简洁程度都差不多了。

我个人倾向于版本一的写法，思路清晰一点，版本二会有点绕。

周四

这道题目：贪心算法：K次取反后最大化的数组和就比较简单了，哈哈，用简单题来讲一讲贪心的思想。

这里其实用了两次贪心！

第一次贪心：局部最优：让绝对值大的负数变为正数，当前数值达到最大，整体最优：整个数组和达到最大。

处理之后，如果K依然大于0，此时的问题是一个有序正整数序列，如何转变K次正负，让 数组和 达到最大。

第二次贪心：局部最优：只找数值最小的正整数进行反转，当前数值可以达到最大（例如正整数数组{5, 3, 1}，反转1 得到-1 比 反转5得到的-5 大多了），全局最优：整个 数组和 达到最大。

贪心算法：K次取反后最大化的数组和中的代码，最后while处理K的时候，其实直接判断奇偶数就可以了，文中给出的方式太粗暴了，哈哈，Carl大意了。

例外一位录友留言给出一个很好的建议，因为文中是使用快排，仔细看题，**题目中限定了数据范围是正负一百，所以可以使用桶排序**，这样时间复杂度就可以优化为$O(n)$了。但可能代码要复杂一些了。

总结

大家会发现本周的代码其实都简单，但思路却很巧妙，并不容易写出来。

如果是第一次接触的话，其实很难想出来，就是接触过之后就会了，所以大家不用感觉自己想不出来而烦躁，哈哈。

相信此时大家现在对贪心算法又有一个新的认识了，加油💪

---

11. 加油站

力扣题目链接

在一条环路上有 N 个加油站，其中第 i 个加油站有汽油 gas[i] 升。

你有一辆油箱容量无限的的汽车，从第 i 个加油站开往第 i+1 个加油站需要消耗汽油 cost[i] 升。你从其中的一个加油站出发，开始时油箱为空。

如果你可以绕环路行驶一周，则返回出发时加油站的编号，否则返回 -1。

说明:

- 如果题目有解，该答案即为唯一答案。
- 输入数组均为非空数组，且长度相同。
- 输入数组中的元素均为非负数。

示例 1:
输入:

- gas = [1,2,3,4,5]
- cost = [3,4,5,1,2]

输出: 3
解释:

- 从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油

- 开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油
- 开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油
- 开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油
- 开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油
- 开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。
- 因此，3 可为起始索引。

示例 2:
输入:

- gas = [2,3,4]
- cost = [3,4,3]
- 输出: -1
- 解释:
  你不能从 0 号或 1 号加油站出发，因为没有足够的汽油可以让你行驶到下一个加油站。我们从 2 号加油站出发，可以获得 4 升汽油。 此时油箱有 = 0 + 4 = 4 升汽油。开往 0 号加油站，此时油箱有 4 - 3 + 2 = 3 升汽油。开往 1 号加油站，此时油箱有 3 - 3 + 3 = 3 升汽油。你无法返回 2 号加油站，因为返程需要消耗 4 升汽油，但是你的油箱只有 3 升汽油。因此，无论怎样，你都不可能绕环路行驶一周。

算法公开课

[《代码随想录》算法视频公开课](): [贪心算法，得这么加油才能跑完全程！LeetCode： 134.加油站](), 相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

暴力方法

暴力的方法很明显就是O(n^2)的，遍历每一个加油站为起点的情况，模拟一圈。

如果跑了一圈，中途没有断油，而且最后油量大于等于0，说明这个起点是ok的。

暴力的方法思路比较简单，但代码写起来也不是很容易，关键是要模拟跑一圈的过程。

for循环适合模拟从头到尾的遍历，而while循环适合模拟环形遍历，要善于使用while！

C++代码如下：

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        for (int i = 0; i < cost.size(); i++) {
            int rest = gas[i] - cost[i]; // 记录剩余油量
            int index = (i + 1) % cost.size();
            while (rest > 0 && index != i) { // 模拟以i为起点行驶一圈（如果有rest==0，那么答案就不唯一了）
                rest += gas[index] - cost[index];
                index = (index + 1) % cost.size();
```

```
            }
            // 如果以i为起点跑一圈，剩余油量>=0，返回该起始位置
            if (rest >= 0 && index == i) return i;
        }
        return -1;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

贪心算法（方法一）

直接从全局进行贪心选择，情况如下：

- 情况一：如果gas的总和小于cost总和，那么无论从哪里出发，一定是跑不了一圈的
- 情况二：rest[i] = gas[i]-cost[i]为一天剩下的油，i从0开始计算累加到最后一站，如果累加没有出现负数，说明从0出发，油就没有断过，那么0就是起点。
- 情况三：如果累加的最小值是负数，汽车就要从非0节点出发，从后向前，看哪个节点能把这个负数填平，能把这个负数填平的节点就是出发节点。

C++代码如下：

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int curSum = 0;
        int min = INT_MAX; // 从起点出发，油箱里的油量最小值
        for (int i = 0; i < gas.size(); i++) {
            int rest = gas[i] - cost[i];
            curSum += rest;
            if (curSum < min) {
                min = curSum;
            }
        }
        if (curSum < 0) return -1;  // 情况1
        if (min >= 0) return 0;     // 情况2
                                    // 情况3
        for (int i = gas.size() - 1; i >= 0; i--) {
            int rest = gas[i] - cost[i];
            min += rest;
            if (min >= 0) {
                return i;
            }
        }
        return -1;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

其实我不认为这种方式是贪心算法，因为没有找出局部最优，而是直接从全局最优的角度上思考问题。

但这种解法又说不出是什么方法，这就是一个从全局角度选取最优解的模拟操作。

所以对于本解法是贪心，我持保留意见！

但不管怎么说，解法毕竟还是巧妙的，不用过于执着于其名字称呼。

贪心算法（方法二）

可以换一个思路，首先如果总油量减去总消耗大于等于零那么一定可以跑完一圈，说明 各个站点的加油站 剩油量rest[i]相加一定是大于等于零的。

每个加油站的剩余量rest[i]为gas[i] - cost[i]。

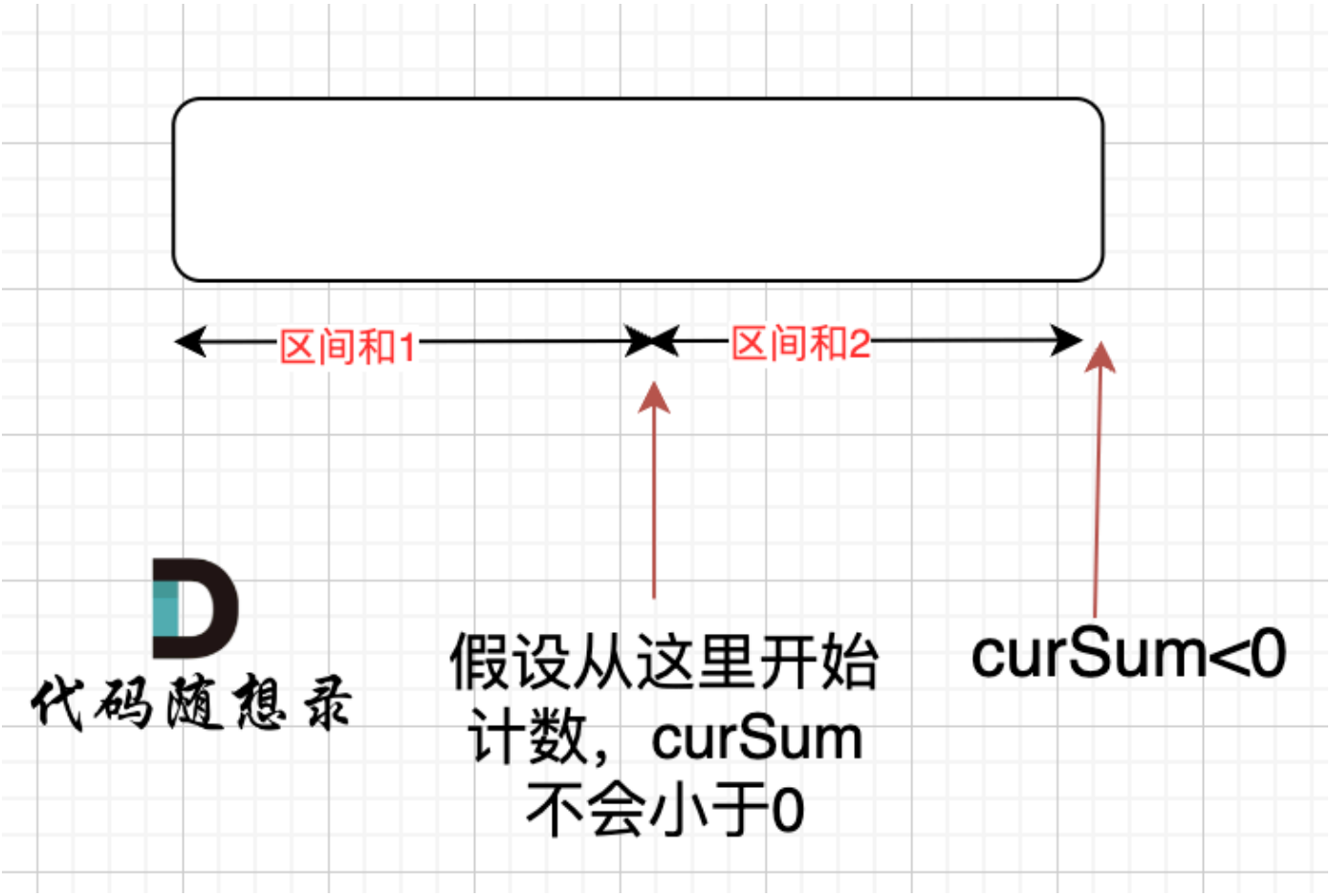
i从0开始累加rest[i]，和记为curSum，一旦curSum小于零，说明[0, i]区间都不能作为起始位置，因为这个区间选择任何一个位置作为起点，到i这里都会断油，那么起始位置从i+1算起，再从0计算curSum。

如图：

那么为什么一旦[0，i] 区间和为负数，起始位置就可以是i+1呢，i+1后面就不会出现更大的负数?

如果出现更大的负数，就是更新i，那么起始位置又变成新的i+1了。

那有没有可能 [0，i] 区间 选某一个作为起点，累加到 i这里 curSum是不会小于零呢？ 如图：



如果 curSum<0 说明 区间和1 + 区间和2 < 0， 那么 假设从上图中的位置开始计数curSum不会小于0的话，就是区间和2>0。

区间和1 + 区间和2 < 0 同时 区间和2>0，只能说明区间和1 < 0， 那么就会从假设的箭头初就开始从新选择其实位置了。

那么局部最优：当前累加rest[i]的和curSum一旦小于0，起始位置至少要是i+1，因为从i之前开始一定不行。全局最优：找到可以跑一圈的起始位置。

局部最优可以推出全局最优，找不出反例，试试贪心！

C++代码如下：

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int curSum = 0;
        int totalSum = 0;
        int start = 0;
        for (int i = 0; i < gas.size(); i++) {
            curSum += gas[i] - cost[i];
            totalSum += gas[i] - cost[i];
            if (curSum < 0) {   // 当前累加rest[i]和 curSum一旦小于0
                start = i + 1;  // 起始位置更新为i+1
                curSum = 0;     // curSum从0开始
            }
        }
```

```
            if (totalSum < 0) return -1; // 说明怎么走都不可能跑一圈了
            return start;
        }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

说这种解法为贪心算法，才是有理有据的，因为全局最优解是根据局部最优推导出来的。

总结

对于本题首先给出了暴力解法，暴力解法模拟跑一圈的过程其实比较考验代码技巧的，要对while使用的很熟练。

然后给出了两种贪心算法，对于第一种贪心方法，其实我认为就是一种直接从全局选取最优的模拟操作，思路还是很巧妙的，值得学习一下。

对于第二种贪心方法，才真正体现出贪心的精髓，用局部最优可以推出全局最优，进而求得起始位置。

---

12. 分发糖果

力扣题目链接

老师想给孩子们分发糖果，有 N 个孩子站成了一条直线，老师会根据每个孩子的表现，预先给他们评分。

你需要按照以下要求，帮助老师给这些孩子分发糖果：

- 每个孩子至少分配到 1 个糖果。
- 相邻的孩子中，评分高的孩子必须获得更多的糖果。

那么这样下来，老师至少需要准备多少颗糖果呢?

示例 1:

- 输入: [1,0,2]
- 输出: 5
- 解释: 你可以分别给这三个孩子分发 2、1、2 颗糖果。

示例 2:

- 输入: [1,2,2]
- 输出: 4
- 解释: 你可以分别给这三个孩子分发 1、2、1 颗糖果。第三个孩子只得到 1 颗糖果，这已满足上述两个条件。

算法公开课

[《代码随想录》算法视频公开课]：[贪心算法，两者兼顾很容易顾此失彼！LeetCode：135.分发糖果]，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

这道题目一定是要确定一边之后，再确定另一边，例如比较每一个孩子的左边，然后再比较右边，**如果两边一起考虑一定会顾此失彼。**

先确定右边评分大于左边的情况（也就是从前向后遍历）

此时局部最优：只要右边评分比左边大，右边的孩子就多一个糖果，全局最优：相邻的孩子中，评分高的右孩子获得比左边孩子更多的糖果

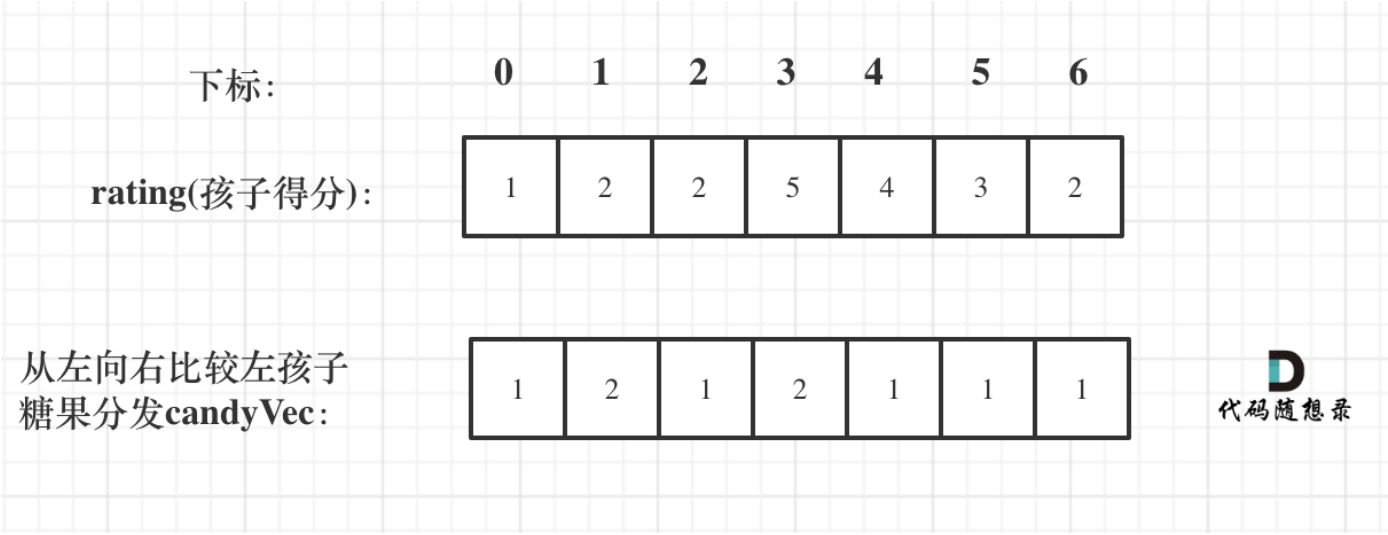
局部最优可以推出全局最优。

如果ratings[i] > ratings[i - 1] 那么[i]的糖 一定要比[i - 1]的糖多一个，所以贪心：candyVec[i] = candyVec[i - 1] + 1

代码如下：

```
// 从前向后
for (int i = 1; i < ratings.size(); i++) {
    if (ratings[i] > ratings[i - 1]) candyVec[i] = candyVec[i - 1] + 1;
}
```

如图：

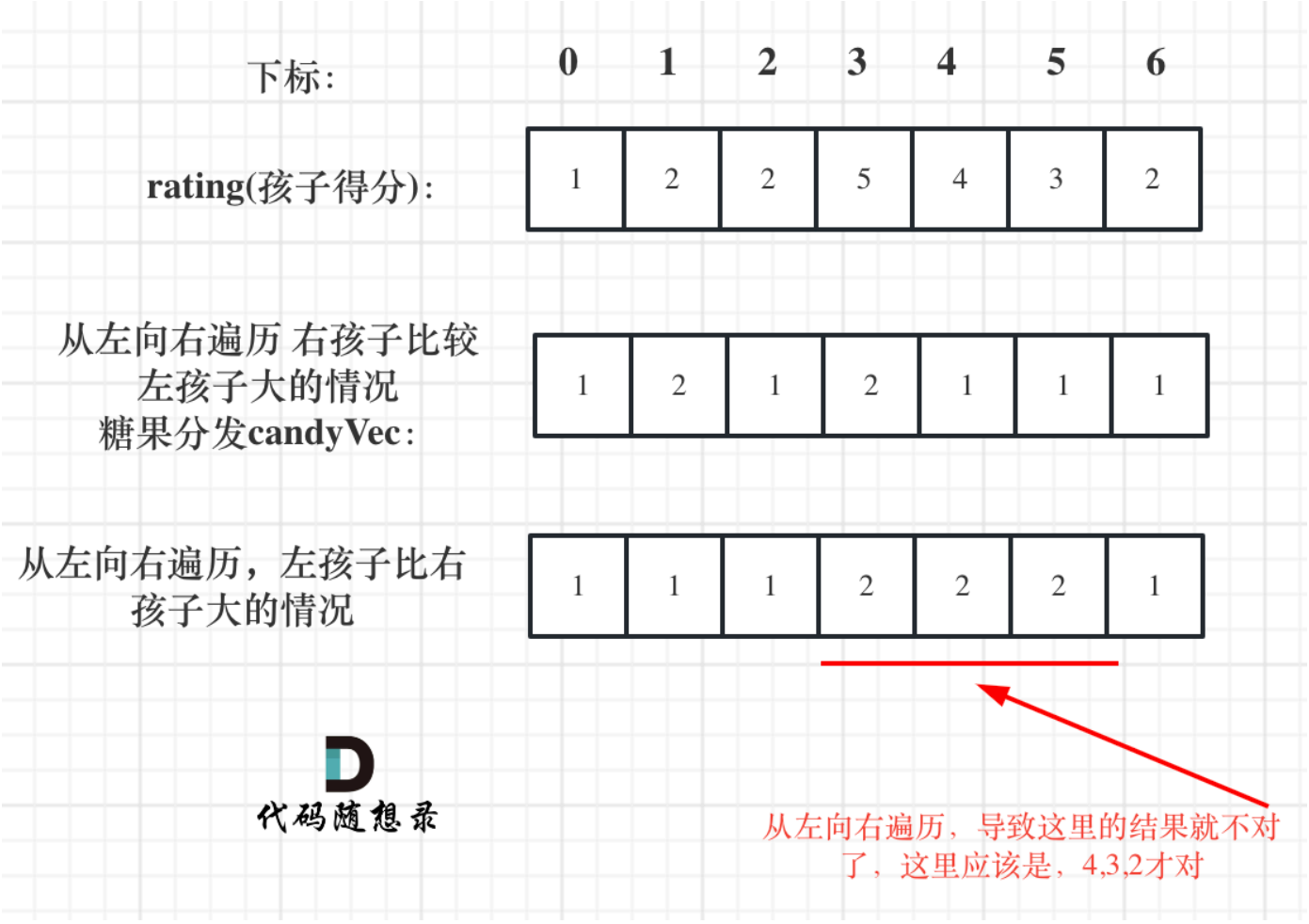


再确定左孩子大于右孩子的情况（从后向前遍历）

遍历顺序这里有同学可能会有疑问，为什么不能从前向后遍历呢?

因为 rating[5]与rating[4]的比较 要利用上 rating[5]与rating[6]的比较结果，所以 要从后向前遍历。

如果从前向后遍历，rating[5]与rating[4]的比较 就不能用上 rating[5]与rating[6]的比较结果了 。如图：



所以确定左孩子大于右孩子的情况一定要从后向前遍历！

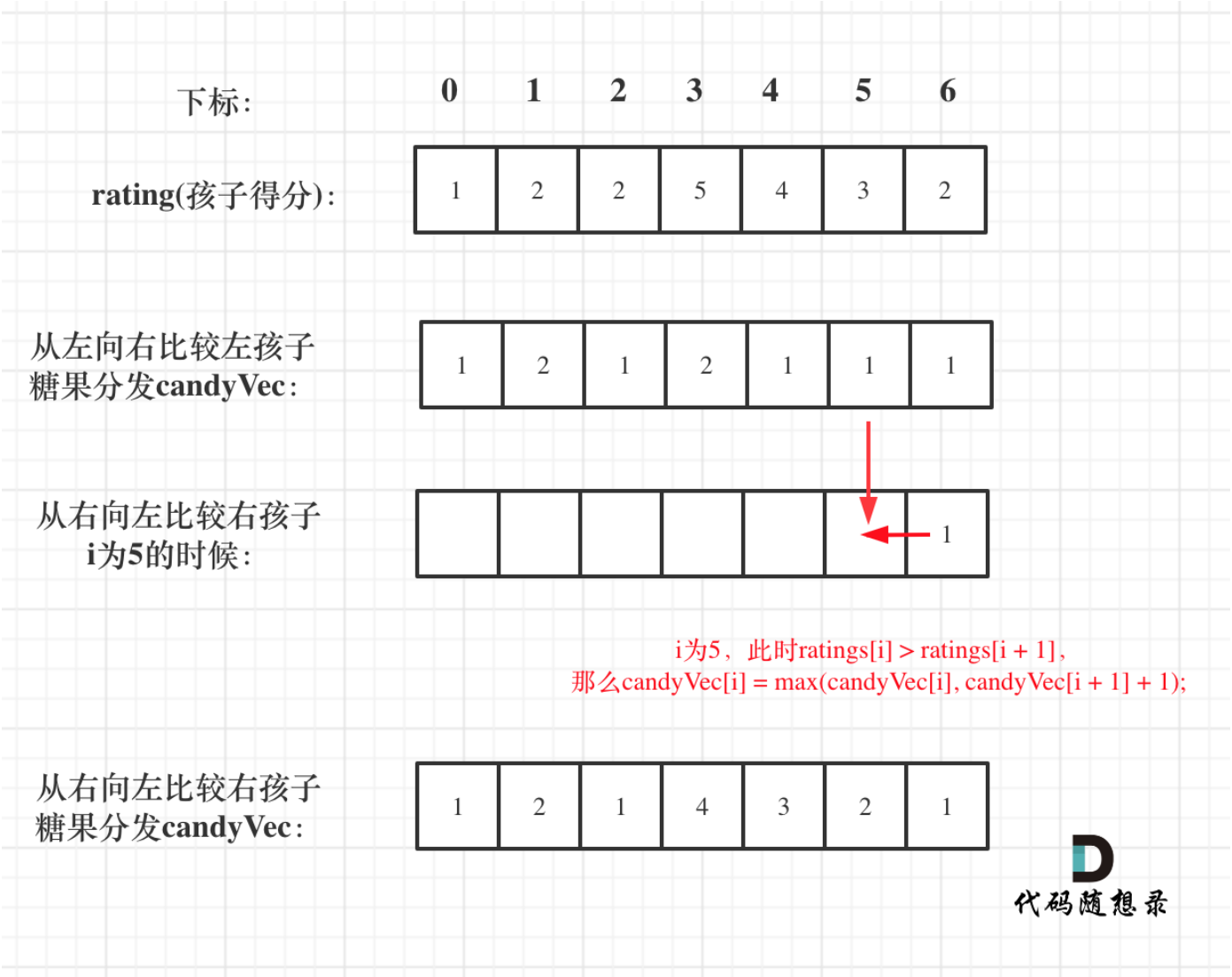
如果 ratings[i] > ratings[i + 1]，此时candyVec[i]（第i个小孩的糖果数量）就有两个选择了，一个是candyVec[i + 1] + 1（从右边这个加1得到的糖果数量），一个是candyVec[i]（之前比较右孩子大于左孩子得到的糖果数量）。

那么又要贪心了，局部最优：取candyVec[i + 1] + 1 和 candyVec[i] 最大的糖果数量，保证第i个小孩的糖果数量既大于左边的也大于右边的。全局最优：相邻的孩子中，评分高的孩子获得更多的糖果。

局部最优可以推出全局最优。

所以就取candyVec[i + 1] + 1 和 candyVec[i] 最大的糖果数量，**candyVec[i]只有取最大的才能既保持对左边candyVec[i - 1]的糖果多，也比右边candyVec[i + 1]的糖果多。**

如图：



所以该过程代码如下：

```
// 从后向前
for (int i = ratings.size() - 2; i >= 0; i--) {
    if (ratings[i] > ratings[i + 1] ) {
        candyVec[i] = max(candyVec[i], candyVec[i + 1] + 1);
    }
}
```

整体代码如下：

```
class Solution {
public:
    int candy(vector<int>& ratings) {
        vector<int> candyVec(ratings.size(), 1);
        // 从前向后
        for (int i = 1; i < ratings.size(); i++) {
            if (ratings[i] > ratings[i - 1]) candyVec[i] = candyVec[i - 1] + 1;
        }
        // 从后向前
        for (int i = ratings.size() - 2; i >= 0; i--) {
```

```
            if (ratings[i] > ratings[i + 1] ) {
                candyVec[i] = max(candyVec[i], candyVec[i + 1] + 1);
            }
        }
        // 统计结果
        int result = 0;
        for (int i = 0; i < candyVec.size(); i++) result += candyVec[i];
        return result;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

总结

这在leetcode上是一道困难的题目，其难点就在于贪心的策略，如果在考虑局部的时候想两边兼顾，就会顾此失彼。

那么本题我采用了两次贪心的策略：

- 一次是从左到右遍历，只比较右边孩子评分比左边大的情况。
- 一次是从右到左遍历，只比较左边孩子评分比右边大的情况。

这样从局部最优推出了全局最优，即：相邻的孩子中，评分高的孩子获得更多的糖果。

13.柠檬水找零

力扣题目链接

在柠檬水摊上，每一杯柠檬水的售价为 5 美元。

顾客排队购买你的产品，（按账单 bills 支付的顺序）一次购买一杯。

每位顾客只买一杯柠檬水，然后向你付 5 美元、10 美元或 20 美元。你必须给每个顾客正确找零，也就是说净交易是每位顾客向你支付 5 美元。

注意，一开始你手头没有任何零钱。

如果你能给每位顾客正确找零，返回 true ，否则返回 false 。

示例 1：

- 输入：[5,5,5,10,20]
- 输出：true

- 解释：
    - 前 3 位顾客那里，我们按顺序收取 3 张 5 美元的钞票。
    - 第 4 位顾客那里，我们收取一张 10 美元的钞票，并返还 5 美元。
    - 第 5 位顾客那里，我们找还一张 10 美元的钞票和一张 5 美元的钞票。
    - 由于所有客户都得到了正确的找零，所以我们输出 true。

示例 2：

- 输入：[5,5,10]
- 输出：true

示例 3：

- 输入：[10,10]
- 输出：false

示例 4：

- 输入：[5,5,10,10,20]
- 输出：false
- 解释：
    - 前 2 位顾客那里，我们按顺序收取 2 张 5 美元的钞票。
    - 对于接下来的 2 位顾客，我们收取一张 10 美元的钞票，然后返还 5 美元。
    - 对于最后一位顾客，我们无法退回 15 美元，因为我们现在只有两张 10 美元的钞票。
    - 由于不是每位顾客都得到了正确的找零，所以答案是 false。

提示：

- 0 <= bills.length <= 10000
- bills[i] 不是 5 就是 10 或是 20

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法，看上去复杂，其实逻辑都是固定的！LeetCode：860.柠檬水找零](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

这是前几天的leetcode每日一题，感觉不错，给大家讲一下。

这道题目刚一看，可能会有点懵，这要怎么找零才能保证完成全部账单的找零呢?

但仔细一琢磨就会发现，可供我们做判断的空间非常少！

只需要维护三种金额的数量，5，10和20。

有如下三种情况：

- 情况一：账单是5，直接收下。

- 情况二：账单是10，消耗一个5，增加一个10
- 情况三：账单是20，优先消耗一个10和一个5，如果不够，再消耗三个5

此时大家就发现 情况一，情况二，都是固定策略，都不用我们来做分析了，而唯一不确定的其实在情况三。

而情况三逻辑也不复杂甚至感觉纯模拟就可以了，其实情况三这里是有贪心的。

账单是20的情况，为什么要优先消耗一个10和一个5呢?

因为美元10只能给账单20找零，而美元5可以给账单10和账单20找零，美元5更万能!

所以局部最优：遇到账单20，优先消耗美元10，完成本次找零。全局最优：完成全部账单的找零。

局部最优可以推出全局最优，并找不出反例，那么就试试贪心算法!

C++代码如下：

```
class Solution {
public:
    bool lemonadeChange(vector<int>& bills) {
        int five = 0, ten = 0, twenty = 0;
        for (int bill : bills) {
            // 情况一
            if (bill == 5) five++;
            // 情况二
            if (bill == 10) {
                if (five <= 0) return false;
                ten++;
                five--;
            }
            // 情况三
            if (bill == 20) {
                // 优先消耗10美元，因为5美元的找零用处更大，能多留着就多留着
                if (five > 0 && ten > 0) {
                    five--;
                    ten--;
                    twenty++; // 其实这行代码可以删了，因为记录20已经没有意义了，不会用20来找零
                } else if (five >= 3) {
                    five -= 3;
                    twenty++; // 同理，这行代码也可以删了
                } else return false;
            }
        }
        return true;
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(1)

总结

咋眼一看好像很复杂，分析清楚之后，会发现逻辑其实非常固定。

这道题目可以告诉大家，遇到感觉没有思路的题目，可以静下心来把能遇到的情况分析一下，只要分析到具体情况了，一下子就豁然开朗了。

如果一直陷入想从整体上寻找找零方案，就会把自己陷进去，各种情况一交叉，只会越想越复杂了。

14.根据身高重建队列

力扣题目链接

假设有打乱顺序的一群人站成一个队列，数组 people 表示队列中一些人的属性（不一定按顺序）。每个 people[i] = [hi, ki] 表示第 i 个人的身高为 hi ，前面 正好 有 ki 个身高大于或等于 hi 的人。

请你重新构造并返回输入数组 people 所表示的队列。返回的队列应该格式化为数组 queue ，其中 queue[j] = [hj, kj] 是队列中第 j 个人的属性（queue[0] 是排在队列前面的人）。

示例 1：

- 输入：people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
- 输出：[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
- 解释：
    - 编号为 0 的人身高为 5 ，没有身高更高或者相同的人排在他前面。
    - 编号为 1 的人身高为 7 ，没有身高更高或者相同的人排在他前面。
    - 编号为 2 的人身高为 5 ，有 2 个身高更高或者相同的人排在他前面，即编号为 0 和 1 的人。
    - 编号为 3 的人身高为 6 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
    - 编号为 4 的人身高为 4 ，有 4 个身高更高或者相同的人排在他前面，即编号为 0、1、2、3 的人。
    - 编号为 5 的人身高为 7 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
    - 因此 [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] 是重新构造后的队列。

示例 2：

- 输入：people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]
- 输出：[[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]

提示：

- 1 <= people.length <= 2000
- 0 <= hi <= 10^6
- 0 <= ki < people.length

题目数据确保队列可以被重建

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法，不要两边一起贪，会顾此失彼 | LeetCode：406.根据身高重建队列](#)，**相信结合视频在看本篇题解，更有助于大家对本题的理解。**

思路

本题有两个维度，h和k，看到这种题目一定要想如何确定一个维度，然后再按照另一个维度重新排列。

其实如果大家认真做了135. 分发糖果，就会发现和此题有点点的像。

在135. 分发糖果我就强调过一次，遇到两个维度权衡的时候，一定要先确定一个维度，再确定另一个维度。

如果两个维度一起考虑一定会顾此失彼。

对于本题相信大家困惑的点是先确定k还是先确定h呢，也就是究竟先按h排序呢，还是先按照k排序呢？

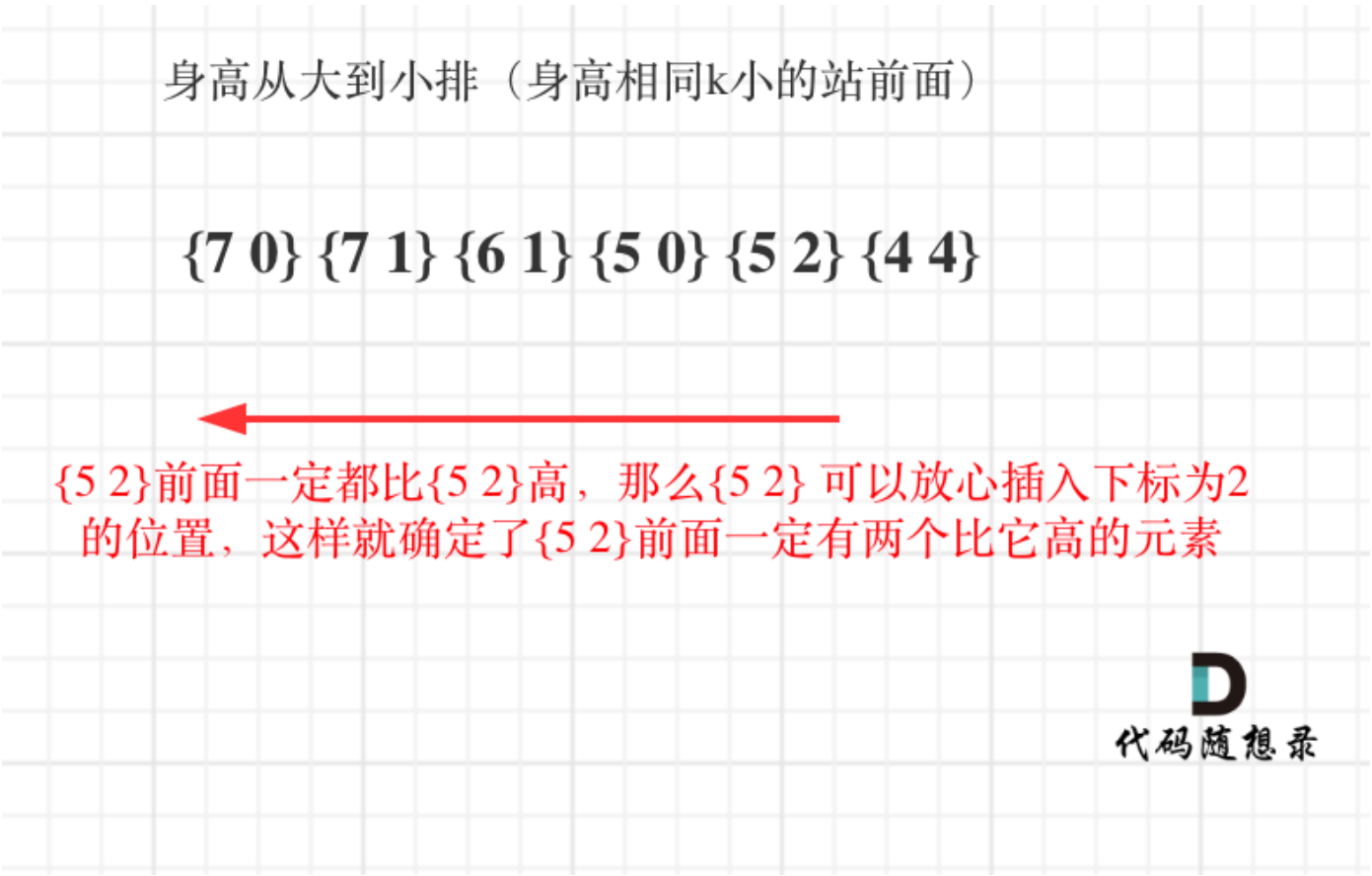
如果按照k来从小到大排序，排完之后，会发现k的排列并不符合条件，身高也不符合条件，两个维度哪一个都没确定下来。

那么按照身高h来排序呢，身高一定是从大到小排（身高相同的话则k小的站前面），让高个子在前面。

此时我们可以确定一个维度了，就是身高，前面的节点一定都比本节点高！

那么只需要按照k为下标重新插入队列就可以了，为什么呢？

以图中{5,2} 为例：



按照身高排序之后，优先按身高高的people的k来插入，后序插入节点也不会影响前面已经插入的节点，最终按照k的规则完成了队列。

所以在按照身高从大到小排序后：

局部最优：优先按身高高的people的k来插入。插入操作过后的people满足队列属性

全局最优：最后都做完插入操作，整个队列满足题目队列属性

局部最优可推出全局最优，找不出反例，那就试试贪心。

一些同学可能也会疑惑，你怎么知道局部最优就可以推出全局最优呢？ 有数学证明么？

在贪心系列开篇词关于贪心算法，你该了解这些！中，我已经讲过了这个问题了。

刷题或者面试的时候，手动模拟一下感觉可以局部最优推出整体最优，而且想不到反例，那么就试一试贪心，至于严格的数学证明，就不在讨论范围内了。

如果没有读过关于贪心算法，你该了解这些！的同学建议读一下，相信对贪心就有初步的了解了。

回归本题，整个插入过程如下：

排序完的people：
[[7,0], [7,1], [6,1], [5,0], [5,2]，[4,4]]

插入的过程：

- 插入[7,0]：[[7,0]]
- 插入[7,1]：[[7,0],[7,1]]
- 插入[6,1]：[[7,0],[6,1],[7,1]]
- 插入[5,0]：[[5,0],[7,0],[6,1],[7,1]]
- 插入[5,2]：[[5,0],[7,0],[5,2],[6,1],[7,1]]
- 插入[4,4]：[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

此时就按照题目的要求完成了重新排列。

C++代码如下：

```
// 版本一
class Solution {
public:
    static bool cmp(const vector<int>& a, const vector<int>& b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        vector<vector<int>> que;
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1];
            que.insert(que.begin() + position, people[i]);
        }
        return que;
    }
};
```

- 时间复杂度：O(nlog n + n^2)
- 空间复杂度：O(n)

但使用vector是非常费时的，C++中vector（可以理解是一个动态数组，底层是普通数组实现的）如果插入元素大于预先普通数组大小，vector底部会有一个扩容的操作，即申请两倍于原先普通数组的大小，然后把数据拷贝到另一个更大的数组上。

所以使用vector（动态数组）来insert，是费时的，插入再拷贝的话，单纯一个插入的操作就是O(n^2)了，甚至可能拷贝好几次，就不止O(n^2)了。

改成链表之后，C++代码如下：

```
// 版本二
class Solution {
public:
    // 身高从大到小排（身高相同k小的站前面）
    static bool cmp(const vector<int>& a, const vector<int>& b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        list<vector<int>> que; // list底层是链表实现，插入效率比vector高的多
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1]; // 插入到下标为position的位置
            std::list<vector<int>>::iterator it = que.begin();
            while (position--) { // 寻找在插入位置
                it++;
            }
            que.insert(it, people[i]);
        }
        return vector<vector<int>>(que.begin(), que.end());
    }
};
```

- 时间复杂度：O(nlog n + n^2)
- 空间复杂度：O(n)

大家可以把两个版本的代码提交一下试试，就可以发现其差别了！

关于本题使用数组还是使用链表的性能差异，我在贪心算法：根据身高重建队列（续集）中详细讲解了一波

总结

关于出现两个维度一起考虑的情况，我们已经做过两道题目了，另一道就是135. 分发糖果。

其技巧都是确定一边然后贪心另一边，两边一起考虑，就会顾此失彼。

这道题目可以说比135. 分发糖果难不少，其贪心的策略也是比较巧妙。

最后我给出了两个版本的代码，可以明显看是使用C++中的list（底层链表实现）比vector（数组）效率高得多。

对使用某一种语言容器的使用，特性的选择都会不同程度上影响效率。

所以很多人都说写算法题用什么语言都可以，主要体现在算法思维上，其实我是同意的但也不同意。

对于看别人题解的同学，题解用什么语言其实影响不大，只要题解把所使用语言特性优化的点讲出来，大家都可以看懂，并使用自己语言的时候注意一下。

对于写题解的同学，刷题用什么语言影响就非常大，如果自己语言没有学好而强调算法和编程语言没关系，其实是会误伤别人的。

这也是我为什么统一使用C++写题解的原因

---

15. 本周小结！（贪心算法系列三）

对于贪心，大多数同学都会感觉，不就是常识嘛，这算啥算法，那么本周的题目就可以带大家初步领略一下贪心的巧妙，贪心算法往往妙的出其不意。

周一

在贪心算法：加油站中给出每一个加油站的汽油和开到这个加油站的消耗，问汽车能不能开一圈。

这道题目咋眼一看，感觉是一道模拟题，模拟一下汽车从每一个节点出发看看能不能开一圈，时间复杂度是O(n^2)。

即使用模拟这种情况，也挺考察代码技巧的。

for循环适合模拟从头到尾的遍历，而while循环适合模拟环形遍历，对于本题的场景要善于使用while！

如果代码功力不到位，就模拟这种情况，可能写的也会很费劲。

本题的贪心解法，我给出两种解法。

对于解法一，其实我并不认为这是贪心，因为没有找出局部最优，而是直接从全局最优的角度上思考问题，但思路很巧妙，值得学习一下。

对于解法二，贪心的局部最优：当前累加rest[j]的和curSum一旦小于0，起始位置至少要是j+1，因为从j开始一定不行。全局最优：找到可以跑一圈的起始位置。

这里是可以从局部最优推出全局最优的，想不出反例，那就试试贪心。

解法二就体现出贪心的精髓，同时大家也会发现，虽然贪心是常识，有些常识并不容易，甚至很难！

周二

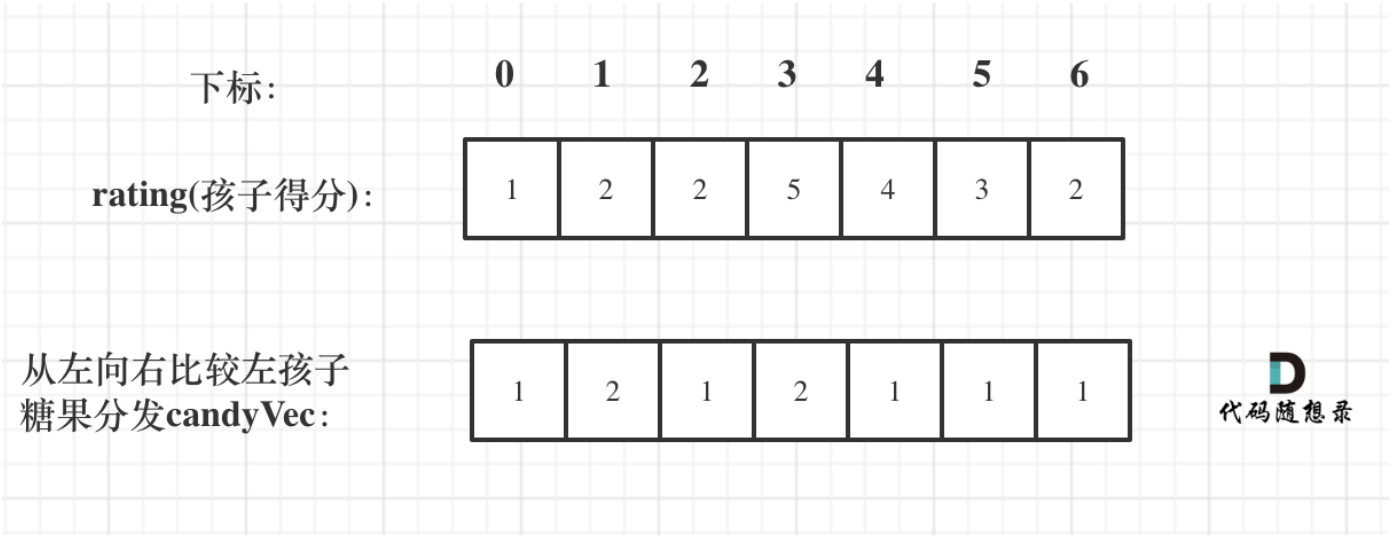
在贪心算法：分发糖果中我们第一次接触了需要考虑两个维度的情况。

例如这道题，是先考虑左边呢，还是考虑右边呢？

先考虑哪一边都可以！ 就别两边一起考虑，那样就把自己陷进去了。

先贪心一边，局部最优：只要右边评分比左边大，右边的孩子就多一个糖果，全局最优：相邻的孩子中，评分高的右孩子获得比左边孩子更多的糖果

如图：



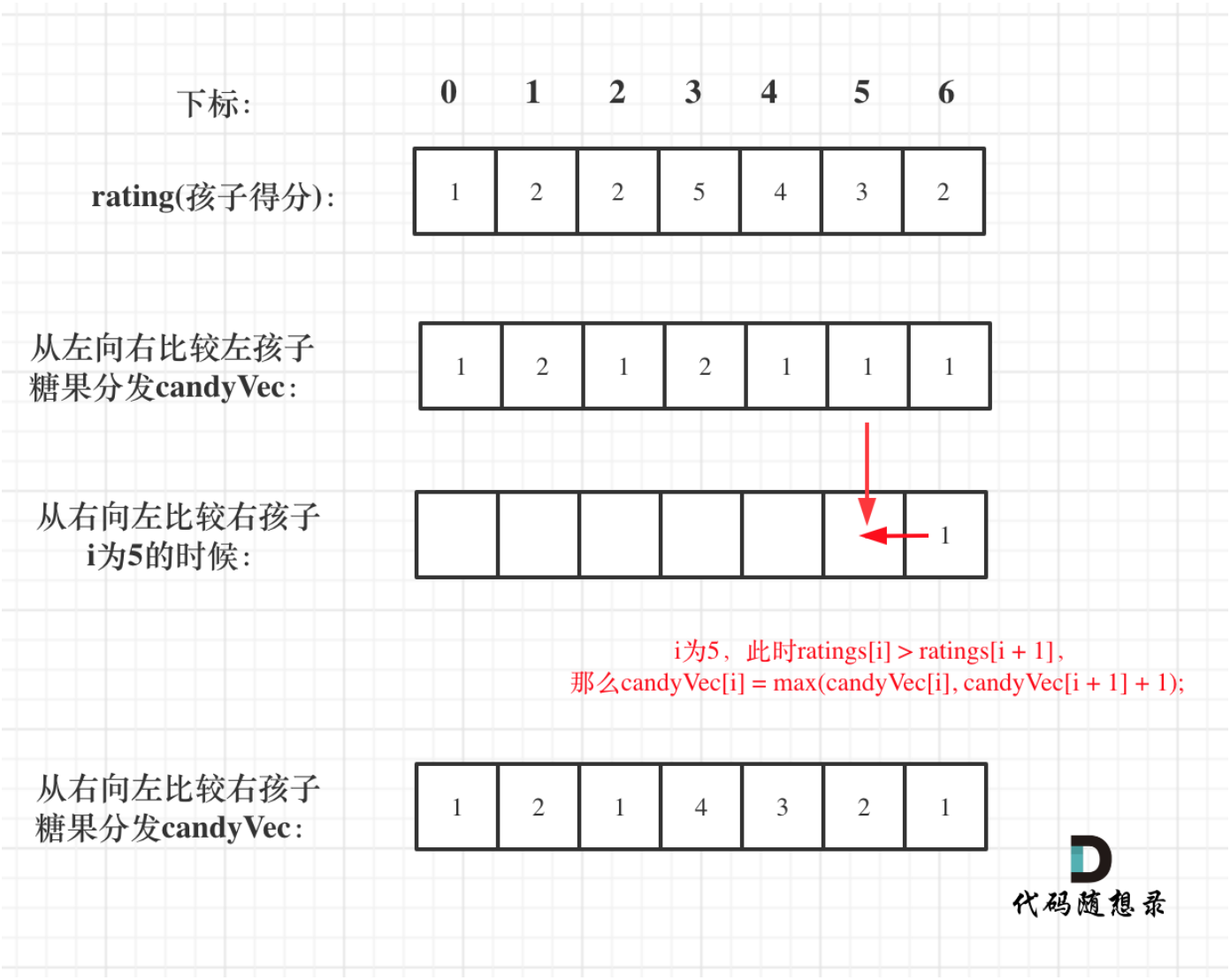
接着在贪心另一边，左孩子大于右孩子，左孩子的糖果就要比右孩子多。

此时candyVec[i]（第i个小孩的糖果数量，左孩子）就有两个选择了，一个是candyVec[i + 1] + 1（从右孩子这个加1得到的糖果数量），一个是candyVec[i]（之前比较右孩子大于左孩子得到的糖果数量）。

那么第二次贪心的局部最优：取candyVec[i + 1] + 1 和 candyVec[i] 最大的糖果数量，保证第i个小孩的糖果数量即大于左边的也大于右边的。全局最优：相邻的孩子中，评分高的孩子获得更多的糖果。

局部最优可以推出全局最优。

如图：



周三

在贪心算法：柠檬水找零中我们模拟了买柠檬水找零的过程。

这道题目刚一看，可能会有点懵，这要怎么找零才能保证完整全部账单的找零呢?

但仔细一琢磨就会发现，可供我们做判断的空间非常少！

美元10只能给账单20找零，而美元5可以给账单10和账单20找零，美元5更万能！

局部最优：遇到账单20，优先消耗美元10，完成本次找零。全局最优：完成全部账单的找零。

局部最优可以推出全局最优。

所以把能遇到的情况分析一下，只要分析到具体情况了，一下子就豁然开朗了。

这道题目其实是一道简单题，但如果一开始就想从整体上寻找找零方案，就会把自己陷进去，各种情况一交叉，只会越想越复杂了。

周四

在贪心算法：根据身高重建队列中，我们再一次遇到了需要考虑两个维度的情况。

之前我们已经做过一道类似的了就是贪心算法：分发糖果，但本题比分发糖果难不少！

贪心算法：根据身高重建队列中依然是要确定一边，然后在考虑另一边，两边一起考虑一定会蒙圈。

那么本题先确定k还是先确定h呢，也就是究竟先按h排序呢，还先按照k排序呢？

这里其实很考察大家的思考过程，如果按照k来从小到大排序，排完之后，会发现k的排列并不符合条件，身高也不符合条件，两个维度哪一个都没确定下来。

所以先从大到小按照h排个序，再来贪心k。

此时局部最优：优先按身高高的people的k来插入。插入操作过后的people满足队列属性。全局最优：最后都做完插入操作，整个队列满足题目队列属性。

局部最优可以推出全局最优，找不出反例，那么就来贪心。

总结

「代码随想录」里已经讲了十一道贪心题目了，大家可以发现在每一道题目的讲解中，我都是把什么是局部最优，和什么是全局最优说清楚。

虽然有时候感觉贪心就是常识，但如果真正是常识性的题目，其实是模拟题，就不是贪心算法了！例如贪心算法：加油站中的贪心方法一，其实我就认为不是贪心算法，而是直接从全局最优的角度上来模拟，因为方法里没有体现局部最优的过程。

而且大家也会发现，贪心并没有想象中的那么简单，贪心往往妙的出其不意，触不及防！哈哈

---

16. 贪心算法：根据身高重建队列（续集）

在讲解贪心算法：根据身高重建队列中，我们提到了使用vector（C++中的动态数组）来进行insert操作是费时的。

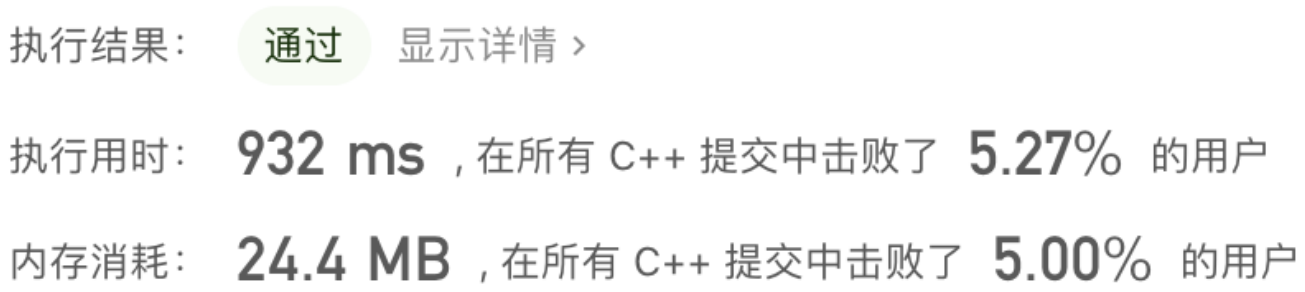
这里专门写一篇文章来详细说一说这个问题。

使用vector的代码如下：

```
// 版本一，使用vector（动态数组）
class Solution {
public:
    static bool cmp(const vector<int> a, const vector<int> b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        vector<vector<int>> que;
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1];
```

```
            que.insert(que.begin() + position, people[i]);
        }
        return que;
    }
};
```

耗时如下：



其直观上来看数组的insert操作是O(n)的，整体代码的时间复杂度是O(n^2)。

这么一分析好像和版本二链表实现的时间复杂度是一样的啊，为什么提交之后效率会差距这么大呢?

```
// 版本二，使用list（链表）
class Solution {
public:
    // 身高从大到小排（身高相同k小的站前面）
    static bool cmp(const vector<int> a, const vector<int> b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        list<vector<int>> que; // list底层是链表实现，插入效率比vector高的多
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1]; // 插入到下标为position的位置
            std::list<vector<int>>::iterator it = que.begin();
            while (position--) { // 寻找在插入位置
                it++;
            }
            que.insert(it, people[i]);
        }
        return vector<vector<int>>(que.begin(), que.end());
    }
};
```

耗时如下：

执行结果： 通过 显示详情 >

执行用时： 172 ms ，在所有 C++ 提交中击败了 75.26% 的用户

内存消耗： 25.5 MB ，在所有 C++ 提交中击败了 5.00% 的用户

大家都知道对于普通数组，一旦定义了大小就不能改变，例如int a[10];，这个数组a至多只能放10个元素，改不了的。

对于动态数组，就是可以不用关心初始时候的大小，可以随意往里放数据，那么耗时的原因就在于动态数组的底层实现。

动态数组为什么可以不受初始大小的限制，可以随意push_back数据呢?

首先vector的底层实现也是普通数组。

vector的大小有两个维度一个是size一个是capicity，size就是我们平时用来遍历vector时候用的，例如：

```
for (int i = 0; i < vec.size(); i++) {

}
```

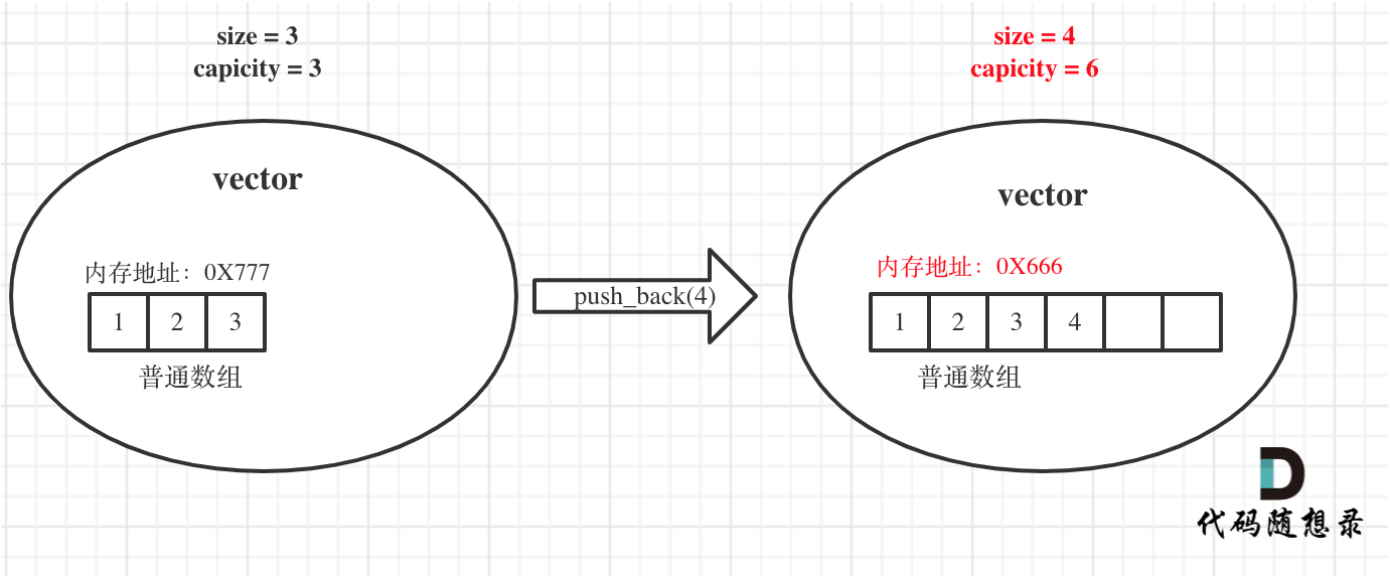
而capicity是vector底层数组（就是普通数组）的大小，capicity可不一定就是size。

当insert数据的时候，如果已经大于capicity，capicity会成倍扩容，但对外暴漏的size其实仅仅是+1。

那么既然vector底层实现是普通数组，怎么扩容的?

就是重新申请一个二倍于原数组大小的数组，然后把数据都拷贝过去，并释放原数组内存。（对，就是这么原始粗暴的方法！）

举一个例子，如图：



原vector中的size和capicity相同都是3，初始化为1 2 3，此时要push_back一个元素4。

那么底层其实就要申请一个大小为6的普通数组，并且把原元素拷贝过去，释放原数组内存，**注意图中底层数组的内存起始地址已经变了。**

同时也注意此时capicity和size的变化，关键的地方我都标红了。

而在贪心算法：根据身高重建队列中，我们使用vector来做insert的操作，此时大家可会发现，**虽然表面上复杂度是O(n^2)，但是其底层都不知道额外做了多少次全量拷贝了，所以算上vector的底层拷贝，整体时间复杂度可以认为是O(n^2 + t × n)级别的，t是底层拷贝的次数。**

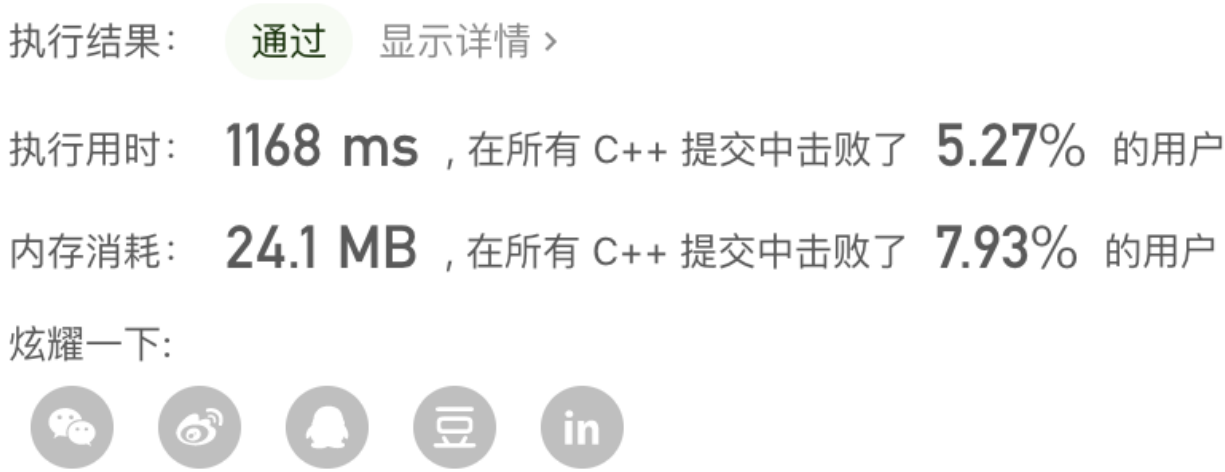
那么是不是可以直接确定好vector的大小，不让它在动态扩容了，例如在贪心算法：根据身高重建队列中已经给出了有people.size这么多的人，可以定义好一个固定大小的vector，这样我们就可以控制vector，不让它底层动态扩容。

这种方法需要自己模拟插入的操作，不仅没有直接调用insert接口那么方便，需要手动模拟插入操作，而且效率也不高！

手动模拟的过程其实不是很简单的，需要很多细节，我粗略写了一个版本，如下：

```
// 版本三
// 使用vector，但不让它动态扩容
class Solution {
public:
    static bool cmp(const vector<int> a, const vector<int> b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        vector<vector<int>> que(people.size(), vector<int>(2, -1));
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1];
            if (position == que.size() - 1) que[position] = people[i];
            else { // 将插入位置后面的元素整体向后移
                for (int j = que.size() - 2; j >= position; j--) que[j + 1] = que[j];
                que[position] = people[i];
            }
        }
        return que;
    }
};
```

耗时如下：



这份代码就是不让vector动态扩容，全程我们自己模拟insert的操作，大家也可以直观的看出是一个O(n^2)的方法了。

但这份代码在leetcode上统计的耗时甚至比版本一的还高，我们都不让它动态扩容了，为什么耗时更高了呢?

一方面是leetcode的耗时统计本来就不太准，忽高忽低的，只能测个大概。

另一方面：可能是就算避免的vector的底层扩容，但这个固定大小的数组，每次向后移动元素赋值的次数比方法一中移动赋值的次数要多很多。

因为方法一中一开始数组是很小的，插入操作，向后移动元素次数比较少，即使有偶尔的扩容操作。而方法三每次都是按照最大数组规模向后移动元素的。

所以对于两种使用数组的方法一和方法三，也不好确定谁优，但一定都没有使用方法二链表的效率高！

一波分析之后，对于贪心算法：根据身高重建队列，大家就安心使用链表吧！别折腾了，哈哈，相当于我替大家折腾了一下。

总结

大家应该发现了，编程语言中一个普通容器的insert，delete的使用，都可能对写出来的算法的有很大影响！

如果抛开语言谈算法，除非从来不用代码写算法纯分析，**否则的话，语言功底不到位O(n)的算法可以写出O(n^2)的性能**，哈哈。

相信在这里学习算法的录友们，都是想在软件行业长远发展的，都是要从事编程的工作，那么一定要深耕好一门编程语言，这个非常重要！

---

17. 用最少数量的箭引爆气球

力扣题目链接

在二维空间中有许多球形的气球。对于每个气球，提供的输入是水平方向上，气球直径的开始和结束坐标。由于它是水平的，所以纵坐标并不重要，因此只要知道开始和结束的横坐标就足够了。开始坐标总是小于结束坐标。

一支弓箭可以沿着 x 轴从不同点完全垂直地射出。在坐标 x 处射出一支箭，若有一个气球的直径的开始和结束坐标为 xstart，xend， 且满足 xstart ≤ x ≤ xend，则该气球会被引爆。可以射出的弓箭的数量没有限制。 弓箭一旦被射出之后，可以无限地前进。我们想找到使得所有气球全部被引爆，所需的弓箭的最小数量。

给你一个数组 points ，其中 points [i] = [xstart,xend] ，返回引爆所有气球所必须射出的最小弓箭数。

示例 1：

- 输入：points = [[10,16],[2,8],[1,6],[7,12]]
- 输出：2
- 解释：对于该样例，x = 6 可以射爆 [2,8],[1,6] 两个气球，以及 x = 11 射爆另外两个气球

示例 2：

- 输入：points = [[1,2],[3,4],[5,6],[7,8]]
- 输出：4

示例 3：

- 输入：points = [[1,2],[2,3],[3,4],[4,5]]
- 输出：2

示例 4：

- 输入：points = [[1,2]]
- 输出：1

示例 5：

- 输入：points = [[2,3],[2,3]]
- 输出：1

提示：

- 0 <= points.length <= 10^4
- points[i].length == 2
- -2^31 <= xstart < xend <= 2^31 - 1

算法公开课

《代码随想录》算法视频公开课：贪心算法，判断重叠区间问题 | LeetCode：452.用最少数量的箭引爆气球，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

如何使用最少的弓箭呢?

直觉上来看，貌似只射重叠最多的气球，用的弓箭一定最少，那么有没有当前重叠了三个气球，我射两个，留下一个和后面的一起射这样弓箭用的更少的情况呢?

尝试一下举反例，发现没有这种情况。

那么就试一试贪心吧！局部最优：当气球出现重叠，一起射，所用弓箭最少。全局最优：把所有气球射爆所用弓箭最少。

算法确定下来了，那么如何模拟气球射爆的过程呢？是在数组中移除元素还是做标记呢？

如果真实的模拟射气球的过程，应该射一个，气球数组就remove一个元素，这样最直观，毕竟气球被射了。

但仔细思考一下就发现：如果把气球排序之后，从前到后遍历气球，被射过的气球仅仅跳过就行了，没有必要让气球数组remove气球，只要记录一下箭的数量就可以了。

以上为思考过程，已经确定下来使用贪心了，那么开始解题。

为了让气球尽可能的重叠，需要对数组进行排序。

那么按照气球起始位置排序，还是按照气球终止位置排序呢？

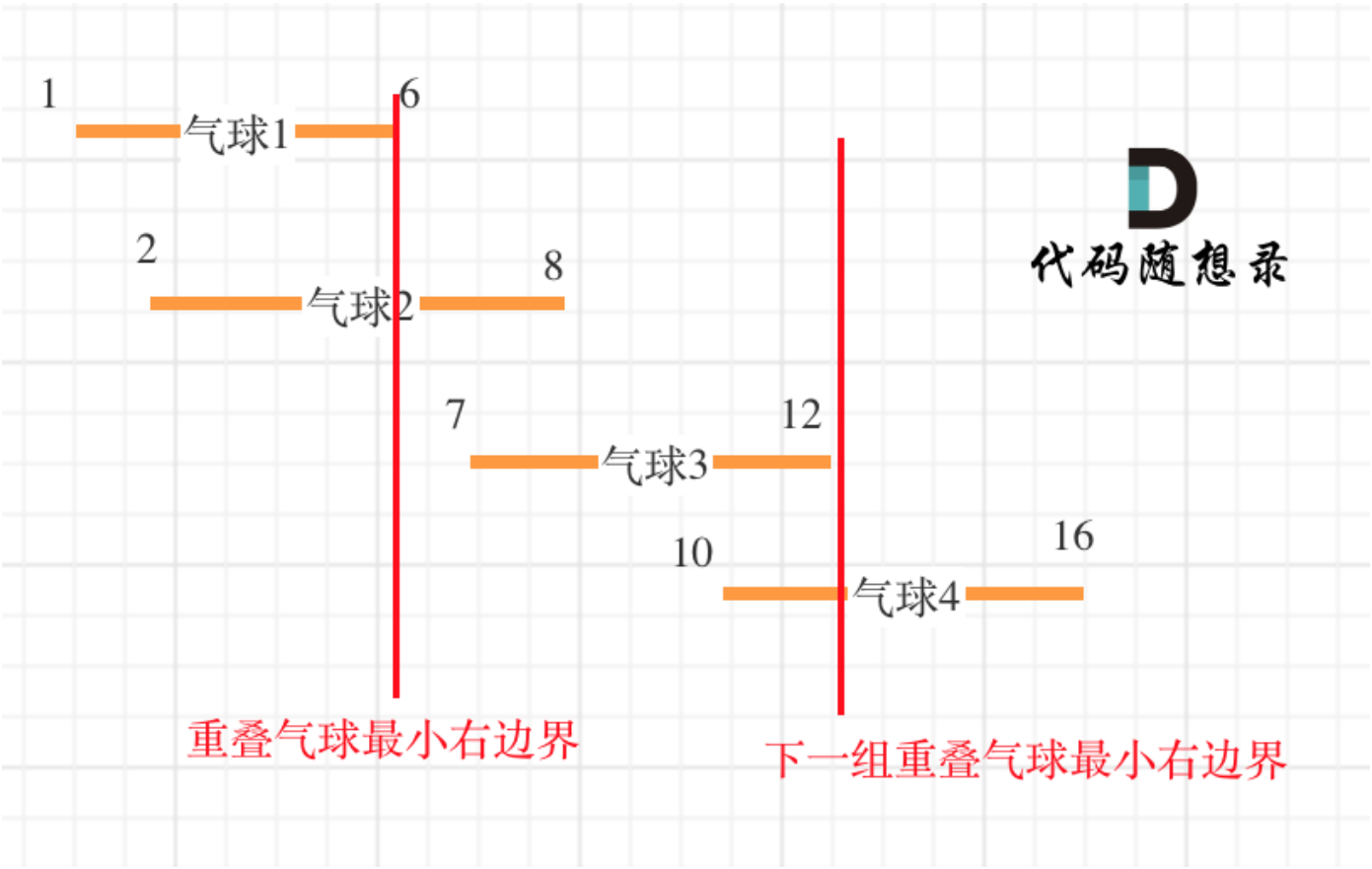
其实都可以！只不过对应的遍历顺序不同，我就按照气球的起始位置排序了。

既然按照起始位置排序，那么就从前向后遍历气球数组，靠左尽可能让气球重复。

从前向后遍历遇到重叠的气球了怎么办？

如果气球重叠了，重叠气球中右边边界的最小值 之前的区间一定需要一个弓箭。

以题目示例： [[10,16],[2,8],[1,6],[7,12]]为例，如图：（方便起见，已经排序）



可以看出首先第一组重叠气球，一定是需要一个箭，气球3，的左边界大于了 第一组重叠气球的最小右边界，所以再需要一支箭来射气球3了。

C++代码如下：

```
class Solution {
private:
    static bool cmp(const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0];
    }
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.size() == 0) return 0;
        sort(points.begin(), points.end(), cmp);

        int result = 1; // points 不为空至少需要一支箭
        for (int i = 1; i < points.size(); i++) {
            if (points[i][0] > points[i - 1][1]) {  // 气球i和气球i-1不挨着，注意这里不是>=
                result++; // 需要一支箭
            }
            else {  // 气球i和气球i-1挨着
                points[i][1] = min(points[i - 1][1], points[i][1]); // 更新重叠气球最小右边界

            }
        }
        return result;
    }
};
```

- 时间复杂度：O(nlog n)，因为有一个快排
- 空间复杂度：O(1)，有一个快排，最差情况(倒序)时，需要n次递归调用。因此确实需要O(n)的栈空间

可以看出代码并不复杂。

注意事项

注意题目中说的是：满足 xstart ≤ x ≤ xend，则该气球会被引爆。那么说明两个气球挨在一起不重叠也可以一起射爆，

所以代码中 `if (points[i][0] > points[i - 1][1])` 不能是>=

总结

这道题目贪心的思路很简单也很直接，就是重复的一起射了，但本题我认为是有难度的。

就算思路都想好了，模拟射气球的过程，很多同学真的要去模拟了，实时把气球从数组中移走，这么写的话就复杂了。

而且寻找重复的气球，寻找重叠气球最小右边界，其实都有代码技巧。

贪心题目有时候就是这样，看起来很简单，思路很直接，但是一写代码就感觉贼复杂无从下手。

这里其实是需要代码功底的，那代码功底怎么练？

多看多写多总结！

---

18. 无重叠区间

力扣题目链接

给定一个区间的集合，找到需要移除区间的最小数量，使剩余区间互不重叠。

注意:
可以认为区间的终点总是大于它的起点。
区间 [1,2] 和 [2,3] 的边界相互“接触”，但没有相互重叠。

示例 1:

- 输入: [[1,2], [2,3], [3,4], [1,3]]
- 输出: 1
- 解释: 移除 [1,3] 后，剩下的区间没有重叠。

示例 2:

- 输入: [[1,2], [1,2], [1,2]]
- 输出: 2
- 解释: 你需要移除两个 [1,2] 来使剩下的区间没有重叠。

示例 3:

- 输入: [[1,2], [2,3]]
- 输出: 0
- 解释: 你不需要移除任何区间，因为它们已经是无重叠的了。

算法公开课

《代码随想录》算法视频公开课：贪心算法，依然是判断重叠区间 | LeetCode：435.无重叠区间，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

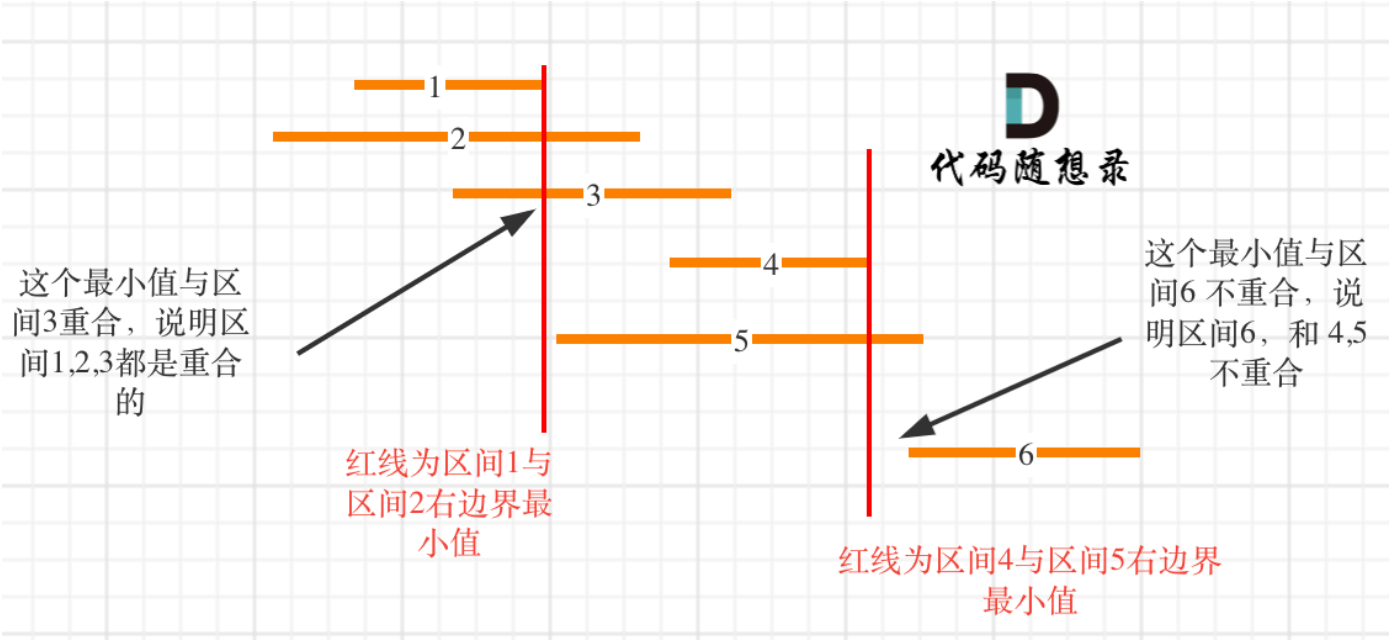
相信很多同学看到这道题目都冥冥之中感觉要排序，但是究竟是按照右边界排序，还是按照左边界排序呢?

其实都可以。主要就是为了让区间尽可能的重叠。

我来按照右边界排序，从左向右记录非交叉区间的个数。最后用区间总数减去非交叉区间的个数就是需要移除的区间个数了。

此时问题就是要求非交叉区间的最大个数。

这里记录非交叉区间的个数还是有技巧的，如图：



区间，1，2，3，4，5，6都按照右边界排好序。

当确定区间 1 和 区间2 重叠后，如何确定是否与 区间3 也重贴呢?

就是取 区间1 和 区间2 右边界的最小值，因为这个最小值之前的部分一定是 区间1 和区间2 的重合部分，如果这个最小值也触达到区间3，那么说明 区间 1，2，3都是重合的。

接下来就是找大于区间1结束位置的区间，是从区间4开始。**那有同学问了为什么不从区间5开始？别忘了已经是按照右边界排序的了**。

区间4结束之后，再找到区间6，所以一共记录非交叉区间的个数是三个。

总共区间个数为6，减去非交叉区间的个数3。移除区间的最小数量就是3。

C++代码如下：

```
class Solution {
public:
    // 按照区间右边界排序
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[1] < b[1];
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);
        int count = 1; // 记录非交叉区间的个数
        int end = intervals[0][1]; // 记录区间分割点
        for (int i = 1; i < intervals.size(); i++) {
            if (end <= intervals[i][0]) {
                end = intervals[i][1];
                count++;
            }
        }
        return intervals.size() - count;
    }
```

```
};
```

- 时间复杂度：O(nlog n)，有一个快排
- 空间复杂度：O(n)，有一个快排，最差情况(倒序)时，需要n次递归调用。因此确实需要O(n)的栈空间

大家此时会发现如此复杂的一个问题，代码实现却这么简单！

补充

补充（1）

左边界排序可不可以呢?

也是可以的，只不过 左边界排序我们就是直接求 重叠的区间，count为记录重叠区间数。

```
class Solution {
public:
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0]; // 改为左边界排序
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);
        int count = 0; // 注意这里从0开始，因为是记录重叠区间
        int end = intervals[0][1]; // 记录区间分割点
        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] >= end)  end = intervals[i][1]; // 无重叠的情况
            else { // 重叠情况
                end = min(end, intervals[i][1]);
                count++;
            }
        }
        return count;
    }
};
```

其实代码还可以精简一下， 用 intervals[i][1] 替代 end变量，只判断 重叠情况就好

```
class Solution {
public:
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0]; // 改为左边界排序
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);
        int count = 0; // 注意这里从0开始，因为是记录重叠区间
        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < intervals[i - 1][1]) { //重叠情况
```

```
                intervals[i][1] = min(intervals[i - 1][1], intervals[i][1]);
                count++;
            }
        }
        return count;
    }
};
```

补充（2）

本题其实和452.用最少数量的箭引爆气球非常像，弓箭的数量就相当于是非交叉区间的数量，只要把弓箭那道题目代码里射爆气球的判断条件加个等号（认为[0，1][1，2]不是相邻区间），然后用总区间数减去弓箭数量 就是要移除的区间数量了。

把452.用最少数量的箭引爆气球代码稍做修改，就可以AC本题。

```
class Solution {
public:
    // 按照区间右边界排序
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[1] < b[1]; // 右边界排序
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);

        int result = 1; // points 不为空至少需要一支箭
        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] >= intervals[i - 1][1]) {
                result++; // 需要一支箭
            }
            else {  // 气球i和气球i-1挨着
                intervals[i][1] = min(intervals[i - 1][1], intervals[i][1]); // 更新重叠
气球最小右边界
            }
        }
        return intervals.size() - result;
    }
};
```

这里按照 左边界排序，或者按照右边界排序，都可以AC，原理是一样的。

```
class Solution {
public:
    // 按照区间左边界排序
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0]; // 左边界排序
```

```
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);

        int result = 1; // points 不为空至少需要一支箭
        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] >= intervals[i - 1][1]) {
                result++; // 需要一支箭
            }
            else {  // 气球i和气球i-1挨着
                intervals[i][1] = min(intervals[i - 1][1], intervals[i][1]); // 更新重叠
气球最小右边界
            }
        }
        return intervals.size() - result;
    }
};
```

19.划分字母区间

[力扣题目链接]

字符串 S 由小写字母组成。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。返回一个表示每个字符串片段的长度的列表。

示例：

- 输入：S = "ababcbacadefegdehijhklij"
- 输出：[9,7,8]
  解释：
  划分结果为 "ababcbaca", "defegde", "hijhklij"。
  每个字母最多出现在一个片段中。
  像 "ababcbacadefegde", "hijhklij" 的划分是错误的，因为划分的片段数较少。

提示：

- S的长度在[1, 500]之间。
- S只包含小写字母 'a' 到 'z' 。

算法公开课

《代码随想录》算法视频公开课：贪心算法，寻找最远的出现位置！ LeetCode：763.划分字母区间，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

一想到分割字符串就想到了回溯，但本题其实不用回溯去暴力搜索。

题目要求同一字母最多出现在一个片段中，那么如何把同一个字母的都圈在同一个区间里呢?

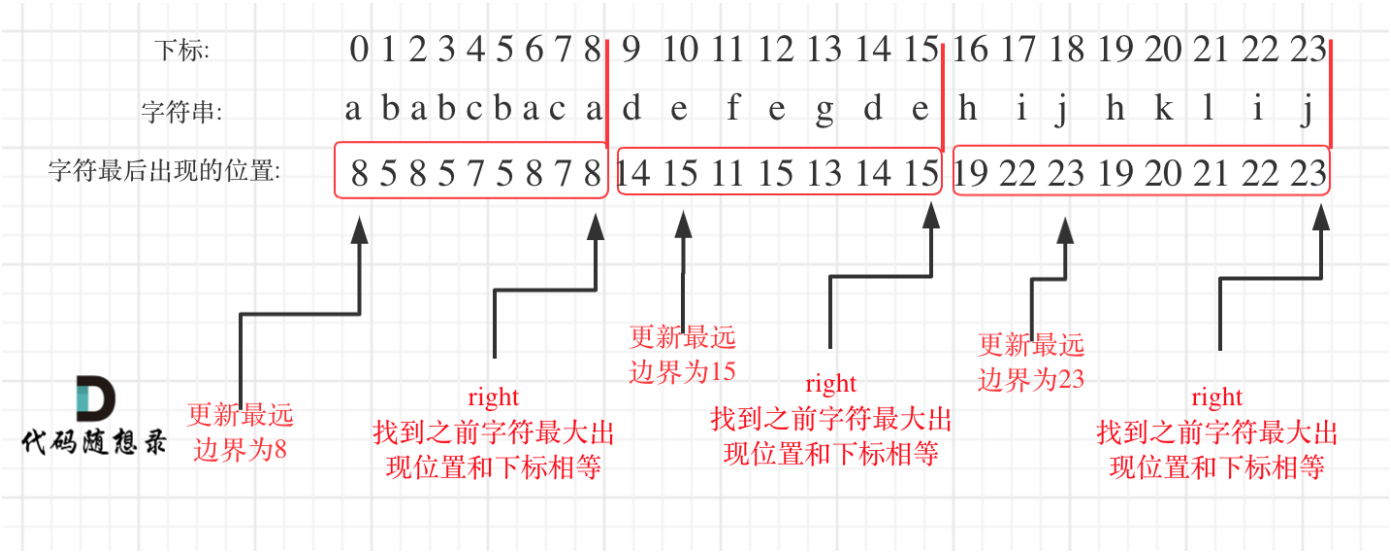
如果没有接触过这种题目的话，还挺有难度的。

在遍历的过程中相当于是要找每一个字母的边界，**如果找到之前遍历过的所有字母的最远边界，说明这个边界就是分割点了**。此时前面出现过所有字母，最远也就到这个边界了。

可以分为如下两步：

- 统计每一个字符最后出现的位置
- 从头遍历字符，并更新字符的最远出现下标，如果找到字符最远出现位置下标和当前下标相等了，则找到了分割点

如图：



明白原理之后，代码并不复杂，如下：

```
class Solution {
public:
    vector<int> partitionLabels(string S) {
        int hash[27] = {0}; // i为字符，hash[i]为字符出现的最后位置
        for (int i = 0; i < S.size(); i++) { // 统计每一个字符最后出现的位置
            hash[S[i] - 'a'] = i;
        }
        vector<int> result;
        int left = 0;
        int right = 0;
        for (int i = 0; i < S.size(); i++) {
            right = max(right, hash[S[i] - 'a']); // 找到字符出现的最远边界
            if (i == right) {
                result.push_back(right - left + 1);
                left = i + 1;
            }
```

```
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)，使用的hash数组是固定大小

总结

这道题目leetcode标记为贪心算法，说实话，我没有感受到贪心，找不出局部最优推出全局最优的过程。就是用最远出现距离模拟了圈字符的行为。

但这道题目的思路是很巧妙的，所以有必要介绍给大家做一做，感受一下。

补充

这里提供一种与452.用最少数量的箭引爆气球、435.无重叠区间相同的思路。

统计字符串中所有字符的起始和结束位置，记录这些区间(实际上也就是435.无重叠区间题目里的输入)，**将区间按左边界从小到大排序，找到边界将区间划分成组，互不重叠。找到的边界就是答案。**

```
class Solution {
public:
    static bool cmp(vector<int> &a, vector<int> &b) {
        return a[0] < b[0];
    }
    // 记录每个字母出现的区间
    vector<vector<int>> countLabels(string s) {
        vector<vector<int>> hash(26, vector<int>(2, INT_MIN));
        vector<vector<int>> hash_filter;
        for (int i = 0; i < s.size(); ++i) {
            if (hash[s[i] - 'a'][0] == INT_MIN) {
                hash[s[i] - 'a'][0] = i;
            }
            hash[s[i] - 'a'][1] = i;
        }
        // 去除字符串中未出现的字母所占用区间
        for (int i = 0; i < hash.size(); ++i) {
            if (hash[i][0] != INT_MIN) {
                hash_filter.push_back(hash[i]);
            }
        }
        return hash_filter;
    }
    vector<int> partitionLabels(string s) {
        vector<int> res;
        // 这一步得到的 hash 即为无重叠区间题意中的输入样例格式：区间列表
        // 只不过现在我们要求的是区间分割点
```

```
        vector<vector<int>> hash = countLabels(s);
        // 按照左边界从小到大排序
        sort(hash.begin(), hash.end(), cmp);
        // 记录最大右边界
        int rightBoard = hash[0][1];
        int leftBoard = 0;
        for (int i = 1; i < hash.size(); ++i) {
            // 由于字符串一定能分割，因此，
            // 一旦下一区间左边界大于当前右边界，即可认为出现分割点
            if (hash[i][0] > rightBoard) {
                res.push_back(rightBoard - leftBoard + 1);
                leftBoard = hash[i][0];
            }
            rightBoard = max(rightBoard, hash[i][1]);
        }
        // 最右端
        res.push_back(rightBoard - leftBoard + 1);
        return res;
    }
};
```

---

20. 合并区间

[力扣题目链接]

给出一个区间的集合，请合并所有重叠的区间。

示例 1:

- 输入: intervals = [[1,3],[2,6],[8,10],[15,18]]
- 输出: [[1,6],[8,10],[15,18]]
- 解释: 区间 [1,3] 和 [2,6] 重叠, 将它们合并为 [1,6].

示例 2:

- 输入: intervals = [[1,4],[4,5]]
- 输出: [[1,5]]
- 解释: 区间 [1,4] 和 [4,5] 可被视为重叠区间。
- 注意：输入类型已于2019年4月15日更改。 请重置默认代码定义以获取新方法签名。

算法公开课

[《代码随想录》算法视频公开课]：[贪心算法，合并区间有细节！LeetCode：56.合并区间]，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题的本质其实还是判断重叠区间问题。

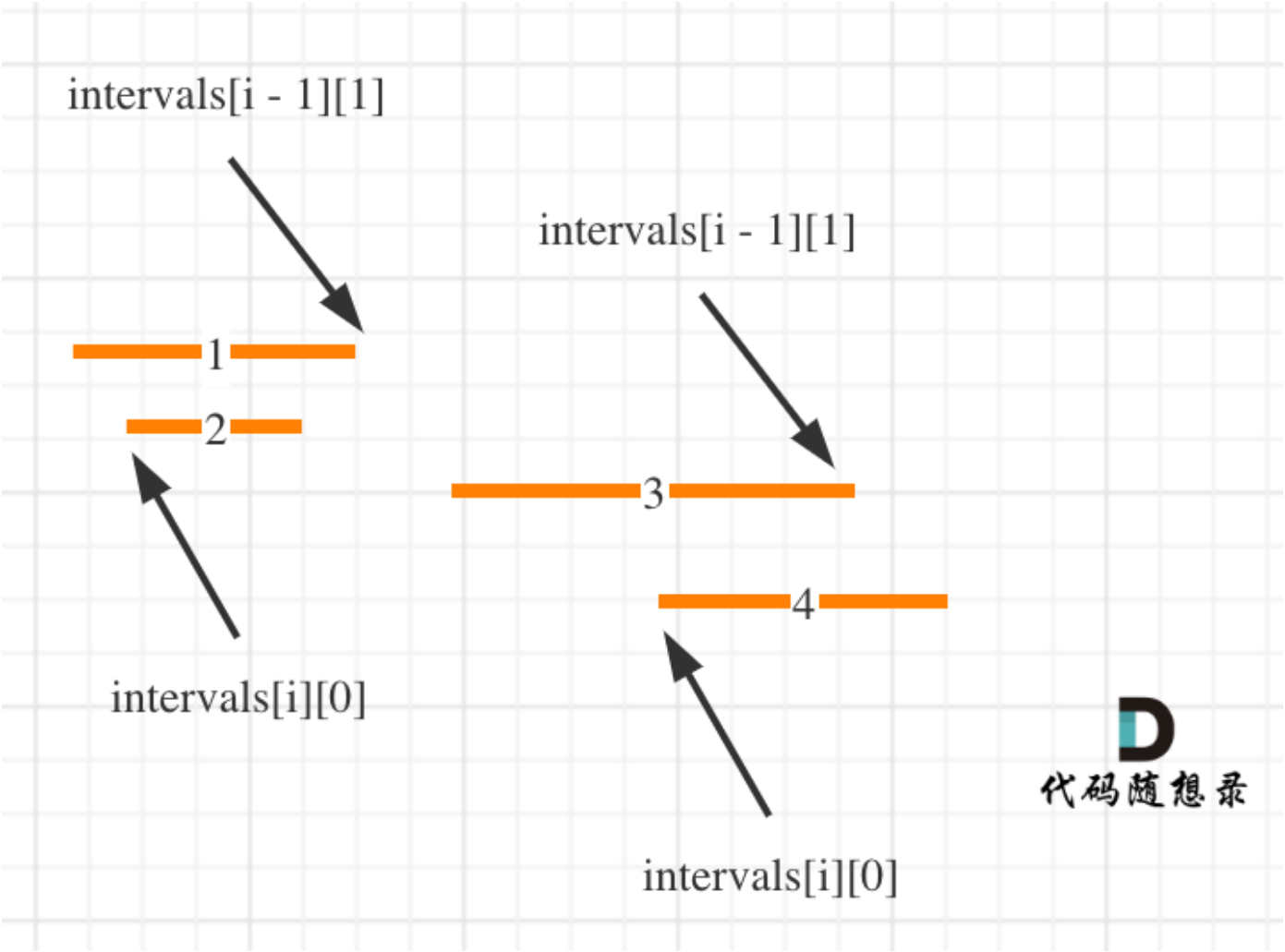
大家如果认真做题的话，话发现和我们刚刚讲过的452. 用最少数量的箭引爆气球 和 435. 无重叠区间 都是一个套路。

这几道题都是判断区间重叠，区别就是判断区间重叠后的逻辑，本题是判断区间重贴后要进行区间合并。

所以一样的套路，先排序，让所有的相邻区间尽可能的重叠在一起，按左边界，或者右边界排序都可以，处理逻辑稍有不同。

按照左边界从小到大排序之后，如果 `intervals[i][0] <= intervals[i - 1][1]` 即intervals[i]的左边界 <= intervals[i - 1]的右边界，则一定有重叠。（本题相邻区间也算重贴，所以是<=）

这么说有点抽象，看图：（**注意图中区间都是按照左边界排序之后了**）



知道如何判断重复之后，剩下的就是合并了，如何去模拟合并区间呢?

其实就是用合并区间后左边界和右边界，作为一个新的区间，加入到result数组里就可以了。如果没有合并就把原区间加入到result数组。

C++代码如下：

```
class Solution {
public:
```

```
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        vector<vector<int>> result;
        if (intervals.size() == 0) return result; // 区间集合为空直接返回
        // 排序的参数使用了lambda表达式
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b){return a[0] < b[0];});

        // 第一个区间就可以放进结果集里，后面如果重叠，在result上直接合并
        result.push_back(intervals[0]);

        for (int i = 1; i < intervals.size(); i++) {
            if (result.back()[1] >= intervals[i][0]) { // 发现重叠区间
                // 合并区间，只更新右边界就好，因为result.back()的左边界一定是最小值，因为我们按照
左边界排序的
                result.back()[1] = max(result.back()[1], intervals[i][1]);
            } else {
                result.push_back(intervals[i]); // 区间不重叠
            }
        }
        return result;
    }
};
```

- 时间复杂度: O(nlogn)
- 空间复杂度: O(logn)，排序需要的空间开销

---

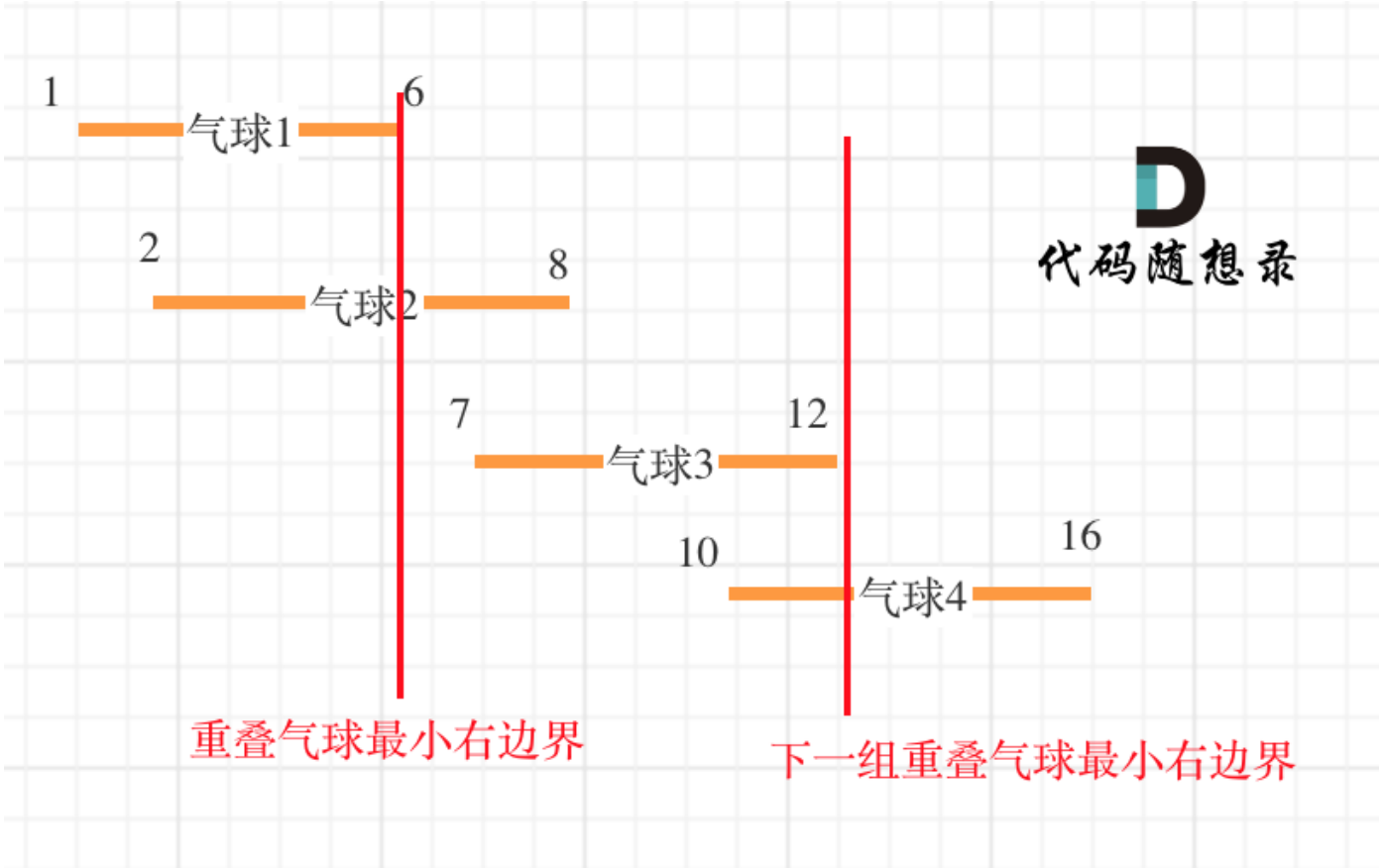
21. 本周小结！（贪心算法系列四）

周一

在贪心算法：用最少数量的箭引爆气球中，我们开始讲解了重叠区间问题，用最少的弓箭射爆所有气球，其本质就是找到最大的重叠区间。

按照左边界进行排序后，如果气球重叠了，重叠气球中右边边界的最小值 之前的区间一定需要一个弓箭

如图：



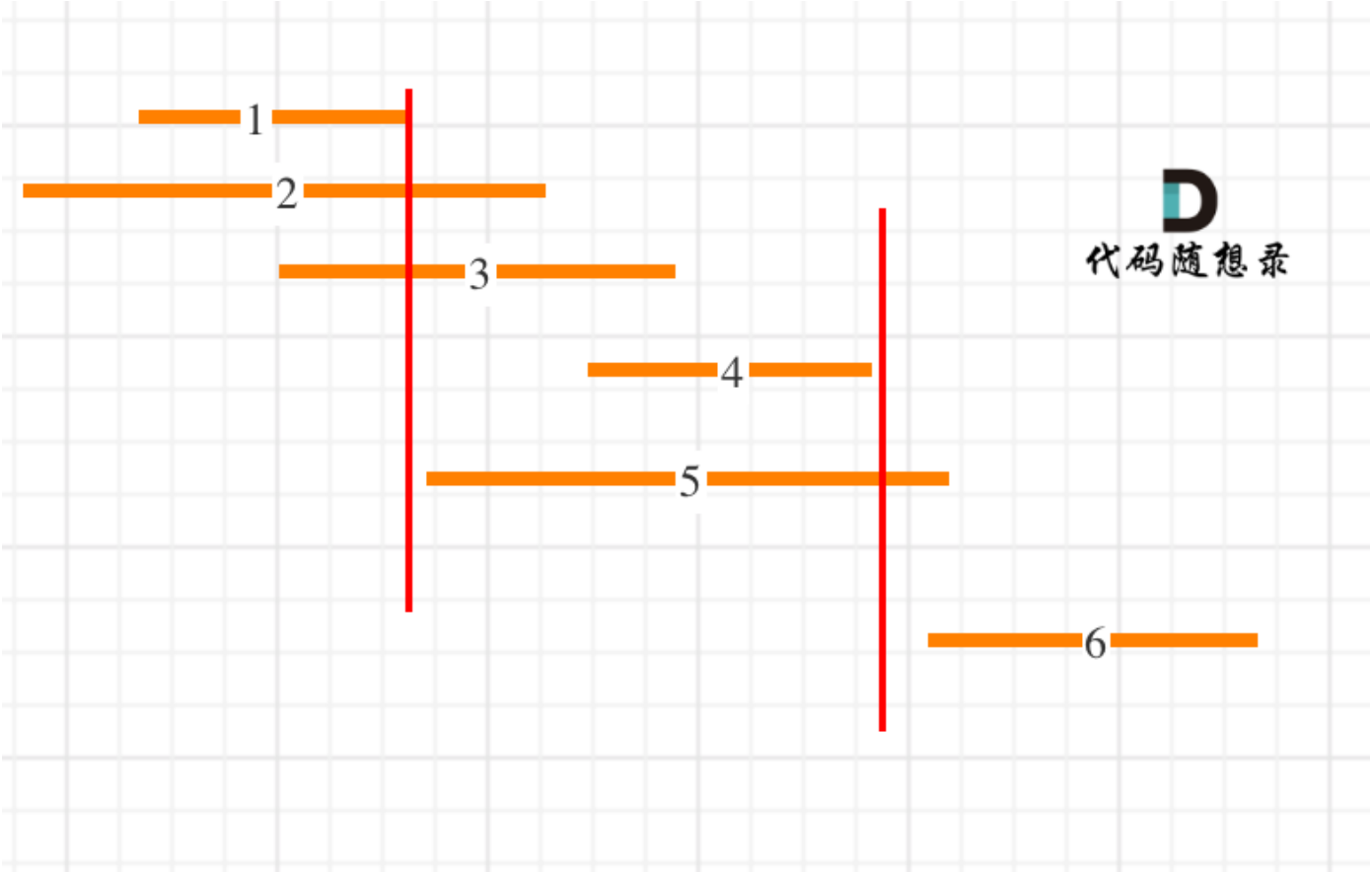
模拟射气球的过程，很多同学真的要去模拟了，实时把气球从数组中移走，这么写的话就复杂了，从前向后遍历重复的只要跳过就可以的。

周二

在贪心算法：无重叠区间中要去掉最少的区间，来让所有区间没有重叠。

我来按照右边界排序，从左向右记录非交叉区间的个数。最后用区间总数减去非交叉区间的个数就是需要移除的区间个数了。

如图：



细心的同学就发现了，此题和 贪心算法：用最少数量的箭引爆气球非常像。

弓箭的数量就相当于是非交叉区间的数量，只要把弓箭那道题目代码里射爆气球的判断条件加个等号（认为[0，1] [1，2]不是相邻区间），然后用总区间数减去弓箭数量 就是要移除的区间数量了。

把贪心算法：用最少数量的箭引爆气球代码稍做修改，就可以AC本题。

修改后的C++代码如下：

```
class Solution {
public:
    // 按照区间左边界从大到小排序
    static bool cmp (const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0];
    }
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.size() == 0) return 0;
        sort(intervals.begin(), intervals.end(), cmp);

        int result = 1;
        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] >= intervals[i - 1][1]) { // 需要要把> 改成 >= 就可以了
                result++; // 需要一支箭
            }
            else {
                intervals[i][1] = min(intervals[i - 1][1], intervals[i][1]); // 更新重叠
气球最小右边界
            }
```

```
        }
        return intervals.size() - result;
    }
};
```

周三

贪心算法：划分字母区间中我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。

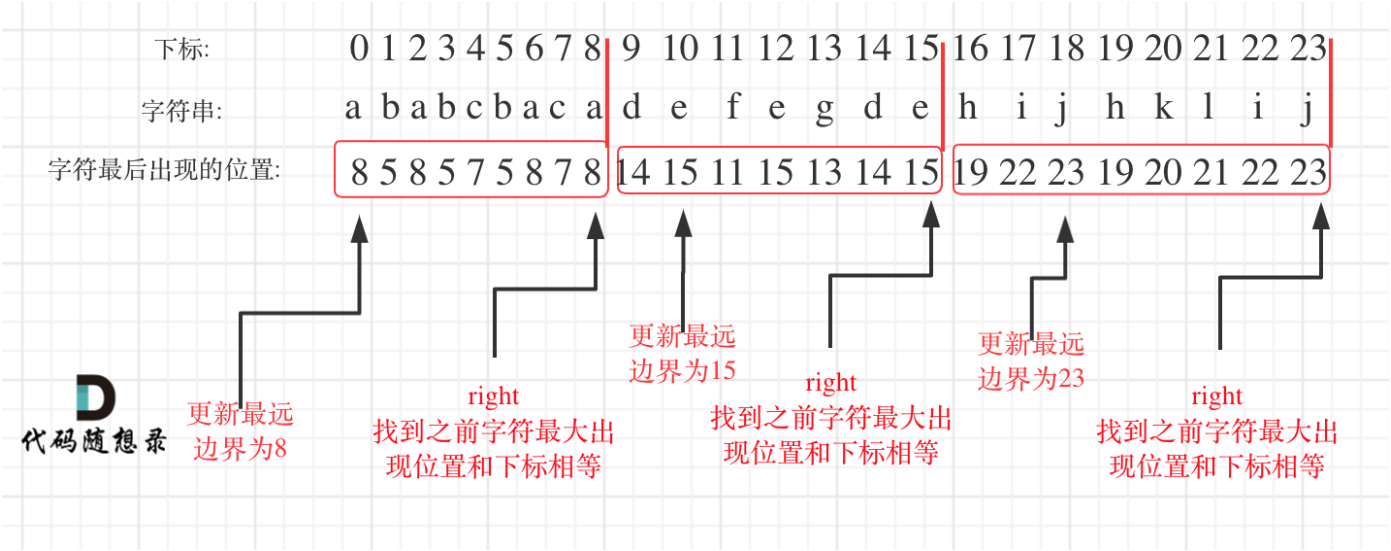
这道题目leetcode上标的是贪心，其实我不认为是贪心，因为没感受到局部最优和全局最优的关系。

但不影响这是一道好题，思路很不错，**通过字符出现最远距离取并集的方法，把出现过的字符都圈到一个区间里。**

解题过程分如下两步：

- 统计每一个字符最后出现的位置
- 从头遍历字符，并更新字符的最远出现下标，如果找到字符最远出现位置下标和当前下标相等了，则找到了分割点

如图：



周四

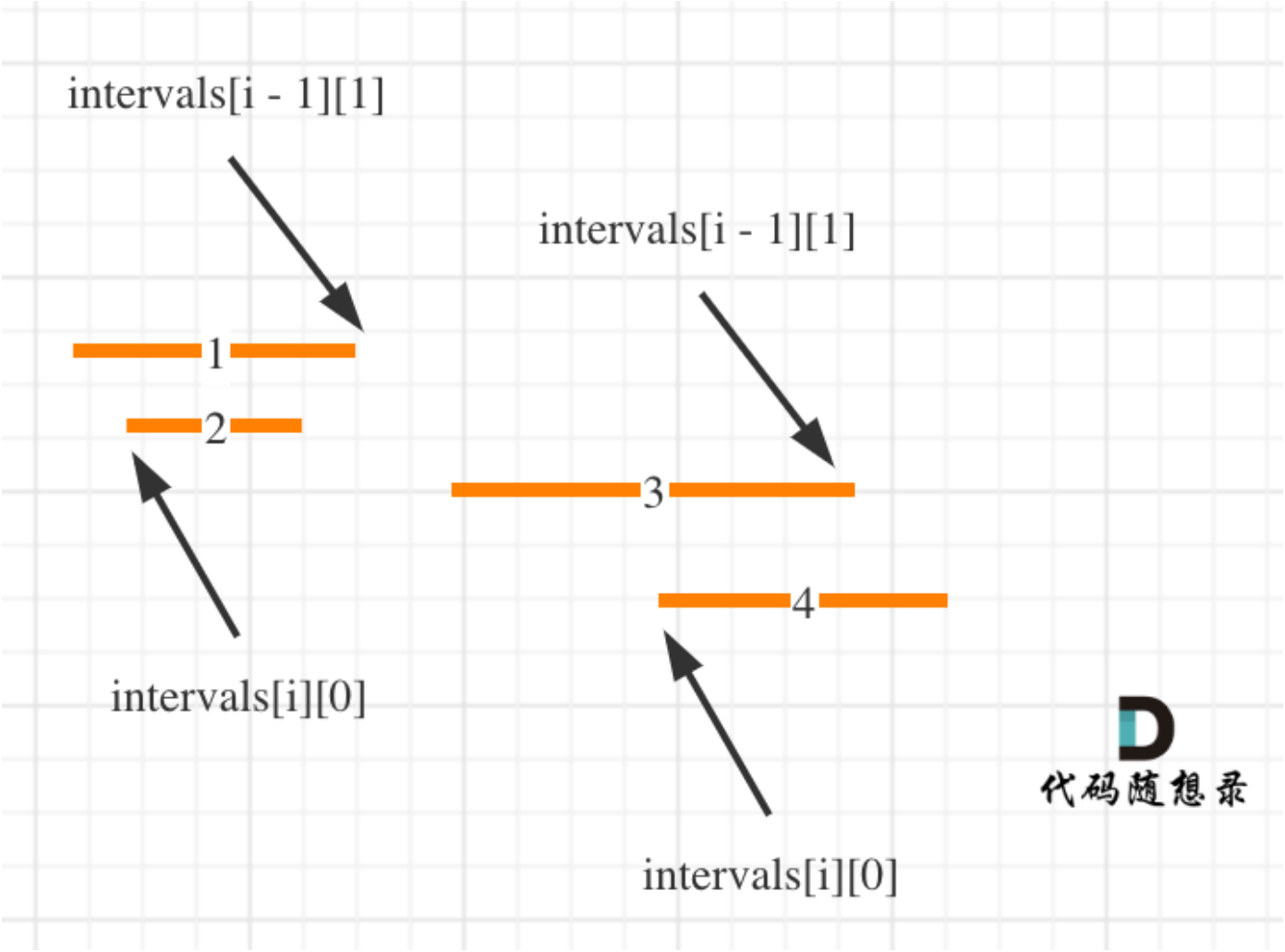
贪心算法：合并区间中要合并所有重叠的区间。

相信如果录友们前几天区间问题的题目认真练习了，今天题目就应该算简单一些了。

按照左边界排序，排序之后局部最优：每次合并都取最大的右边界，这样就可以合并更多的区间了，整体最优：合并所有重叠的区间。

具体操作：按照左边界从小到大排序之后，如果 intervals[i][0] < intervals[i - 1][1] 即intervals[i]左边界 < intervals[i - 1]右边界，则一定有重复，因为intervals[i]的左边界一定是大于等于intervals[i - 1]的左边界。

如图：



总结

本周的主题就是用贪心算法来解决区间问题，经过本周的学习，大家应该对区间的各种合并分割有一定程度的了解了。

其实很多区间的合并操作看起来都是常识，其实贪心算法有时候就是常识，哈哈，但也别小看了贪心算法。

在贪心算法：合并区间中就说过，对于贪心算法，很多同学都是：「如果能凭常识直接做出来，就会感觉不到自己用了贪心, 一旦第一直觉想不出来, 可能就一直想不出来了」。

所以还是要多看多做多练习！

「代码随想录」里总结的都是经典题目，大家跟着练就节省了不少选择题目的时间了。

22.单调递增的数字

力扣题目链接

给定一个非负整数 N，找出小于或等于 N 的最大的整数，同时这个整数需要满足其各个位数上的数字是单调递增。

（当且仅当每个相邻位数上的数字 x 和 y 满足 x <= y 时，我们称这个整数是单调递增的。）

示例 1:

- 输入: N = 10
- 输出: 9

示例 2:

- 输入: N = 1234
- 输出: 1234

示例 3:

- 输入: N = 332
- 输出: 299

说明: N 是在 [0, 10^9] 范围内的一个整数。

算法公开课

[《代码随想录》算法视频公开课](): [贪心算法，思路不难想，但代码不好写！LeetCode:738.单调自增的数字](), 相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

暴力解法

题意很简单，那么首先想的就是暴力解法了，来我替大家暴力一波，结果自然是超时！

代码如下：

```
class Solution {
private:
    // 判断一个数字的各位上是否是递增
    bool checkNum(int num) {
        int max = 10;
        while (num) {
            int t = num % 10;
            if (max >= t) max = t;
            else return false;
            num = num / 10;
        }
        return true;
    }
public:
    int monotoneIncreasingDigits(int N) {
        for (int i = N; i > 0; i--) { // 从大到小遍历
            if (checkNum(i)) return i;
        }
        return 0;
```

```
    }
};
```

- 时间复杂度：O(n × m) m为n的数字长度
- 空间复杂度：O(1)

贪心算法

题目要求小于等于N的最大单调递增的整数，那么拿一个两位的数字来举例。

例如：98，一旦出现strNum[i - 1] > strNum[i]的情况（非单调递增），首先想让strNum[i - 1]--，然后strNum[i]给为9，这样这个整数就是89，即小于98的最大的单调递增整数。

这一点如果想清楚了，这道题就好办了。

此时是从前向后遍历还是从后向前遍历呢?

从前向后遍历的话，遇到strNum[i - 1] > strNum[i]的情况，让strNum[i - 1]减一，但此时如果strNum[i - 1]减一了，可能又小于strNum[i - 2]。

这么说有点抽象，举个例子，数字：332，从前向后遍历的话，那么就把变成了329，此时2又小于了第一位的3了，真正的结果应该是299。

那么从后向前遍历，就可以重复利用上次比较得出的结果了，从后向前遍历332的数值变化为：332 -> 329 -> 299

确定了遍历顺序之后，那么此时局部最优就可以推出全局，找不出反例，试试贪心。

C++代码如下：

```
class Solution {
public:
    int monotoneIncreasingDigits(int N) {
        string strNum = to_string(N);
        // flag用来标记赋值9从哪里开始
        // 设置为这个默认值，为了防止第二个for循环在flag没有被赋值的情况下执行
        int flag = strNum.size();
        for (int i = strNum.size() - 1; i > 0; i--) {
            if (strNum[i - 1] > strNum[i] ) {
                flag = i;
                strNum[i - 1]--;
            }
        }
        for (int i = flag; i < strNum.size(); i++) {
            strNum[i] = '9';
        }
        return stoi(strNum);
    }
};
```

- 时间复杂度：O(n)，n 为数字长度

- 空间复杂度：O(n)，需要一个字符串，转化为字符串操作更方便

总结

本题只要想清楚个例，例如98，一旦出现strNum[i - 1] > strNum[i]的情况（非单调递增），首先想让strNum[i - 1]减一，strNum[i]赋值9，这样这个整数就是89。就可以很自然想到对应的贪心解法了。

想到了贪心，还要考虑遍历顺序，只有从后向前遍历才能重复利用上次比较的结果。

最后代码实现的时候，也需要一些技巧，例如用一个flag来标记从哪里开始赋值9。

23.监控二叉树

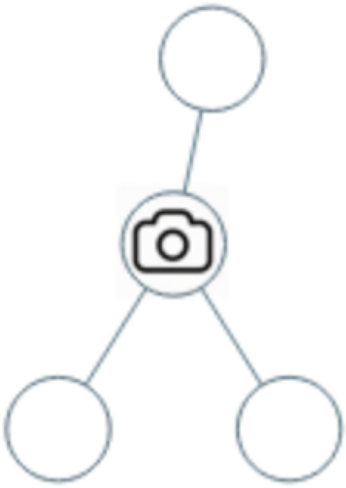
力扣题目链接

给定一个二叉树，我们在树的节点上安装摄像头。

节点上的每个摄影头都可以监视其父对象、自身及其直接子对象。

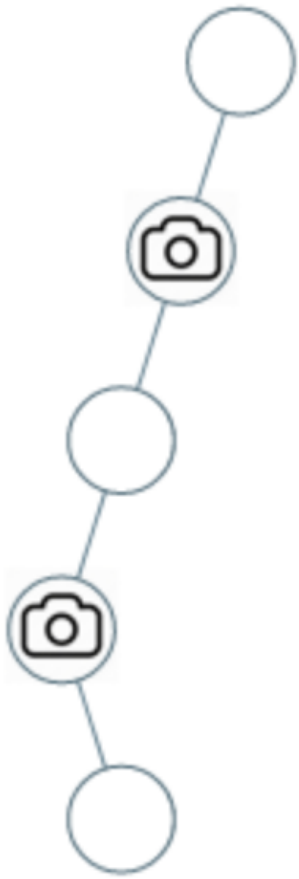
计算监控树的所有节点所需的最小摄像头数量。

示例 1：



- 输入：[0,0,null,0,0]
- 输出：1
- 解释：如图所示，一台摄像头足以监控所有节点。

示例 2：



- 输入：[0,0,null,0,null,0,null,null,0]
- 输出：2
- 解释：需要至少两个摄像头来监视树的所有节点。 上图显示了摄像头放置的有效位置之一。

提示：

- 给定树的节点数的范围是 [1, 1000]。
- 每个节点的值都是 0。

算法公开课

[《代码随想录》算法视频公开课](#)：[贪心算法，二叉树与贪心的结合，有点难..... LeetCode:968.监督二叉树](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

这道题目首先要想，如何放置，才能让摄像头最小的呢?

从题目中示例，其实可以得到启发，**我们发现题目示例中的摄像头都没有放在叶子节点上！**

这是很重要的一个线索，摄像头可以覆盖上中下三层，如果把摄像头放在叶子节点上，就浪费的一层的覆盖。

所以把摄像头放在叶子节点的父节点位置，才能充分利用摄像头的覆盖面积。

那么有同学可能问了，为什么不从头结点开始看起呢，为啥要从叶子节点看呢?

因为头结点放不放摄像头也就省下一个摄像头， 叶子节点放不放摄像头省下了的摄像头数量是指数阶别的。

所以我们要从下往上看，局部最优：让叶子节点的父节点安摄像头，所用摄像头最少，整体最优：全部摄像头数量所用最少！

局部最优推出全局最优，找不出反例，那么就按照贪心来！

此时，大体思路就是从低到上，先给叶子节点父节点放个摄像头，然后隔两个节点放一个摄像头，直至到二叉树头结点。

此时这道题目还有两个难点：

1. 二叉树的遍历
2. 如何隔两个节点放一个摄像头

确定遍历顺序

在二叉树中如何从低向上推导呢?

可以使用后序遍历也就是左右中的顺序，这样就可以在回溯的过程中从下到上进行推导了。

后序遍历代码如下：

```
int traversal(TreeNode* cur) {

    // 空节点，该节点有覆盖
    if (终止条件) return ;

    int left = traversal(cur->left);    // 左
    int right = traversal(cur->right);  // 右

    逻辑处理                            // 中
    return ;
}
```

注意在以上代码中我们取了左孩子的返回值，右孩子的返回值，即left 和 right， 以后推导中间节点的状态

如何隔两个节点放一个摄像头

此时需要状态转移的公式，大家不要和动态的状态转移公式混到一起，本题状态转移没有择优的过程，就是单纯的状态转移！

来看看这个状态应该如何转移，先来看看每个节点可能有几种状态：

有如下三种：

- 该节点无覆盖
- 本节点有摄像头
- 本节点有覆盖

我们分别有三个数字来表示：

- 0：该节点无覆盖
- 1：本节点有摄像头
- 2：本节点有覆盖

大家应该找不出第四个节点的状态了。

一些同学可能会想有没有第四种状态：本节点无摄像头，其实无摄像头就是 无覆盖 或者 有覆盖的状态，所以一共还是三个状态。

因为在遍历树的过程中，就会遇到空节点，那么问题来了，空节点究竟是哪一种状态呢？ 空节点表示无覆盖？ 表示有摄像头？还是有覆盖呢？

回归本质，为了让摄像头数量最少，我们要尽量让叶子节点的父节点安装摄像头，这样才能摄像头的数量最少。

那么空节点不能是无覆盖的状态，这样叶子节点就要放摄像头了，空节点也不能是有摄像头的状态，这样叶子节点的父节点就没有必要放摄像头了，而是可以把摄像头放在叶子节点的爷爷节点上。

所以空节点的状态只能是有覆盖，这样就可以在叶子节点的父节点放摄像头了

接下来就是递推关系。

那么递归的终止条件应该是遇到了空节点，此时应该返回2（有覆盖），原因上面已经解释过了。

代码如下：

```
// 空节点，该节点有覆盖
if (cur == NULL) return 2;
```

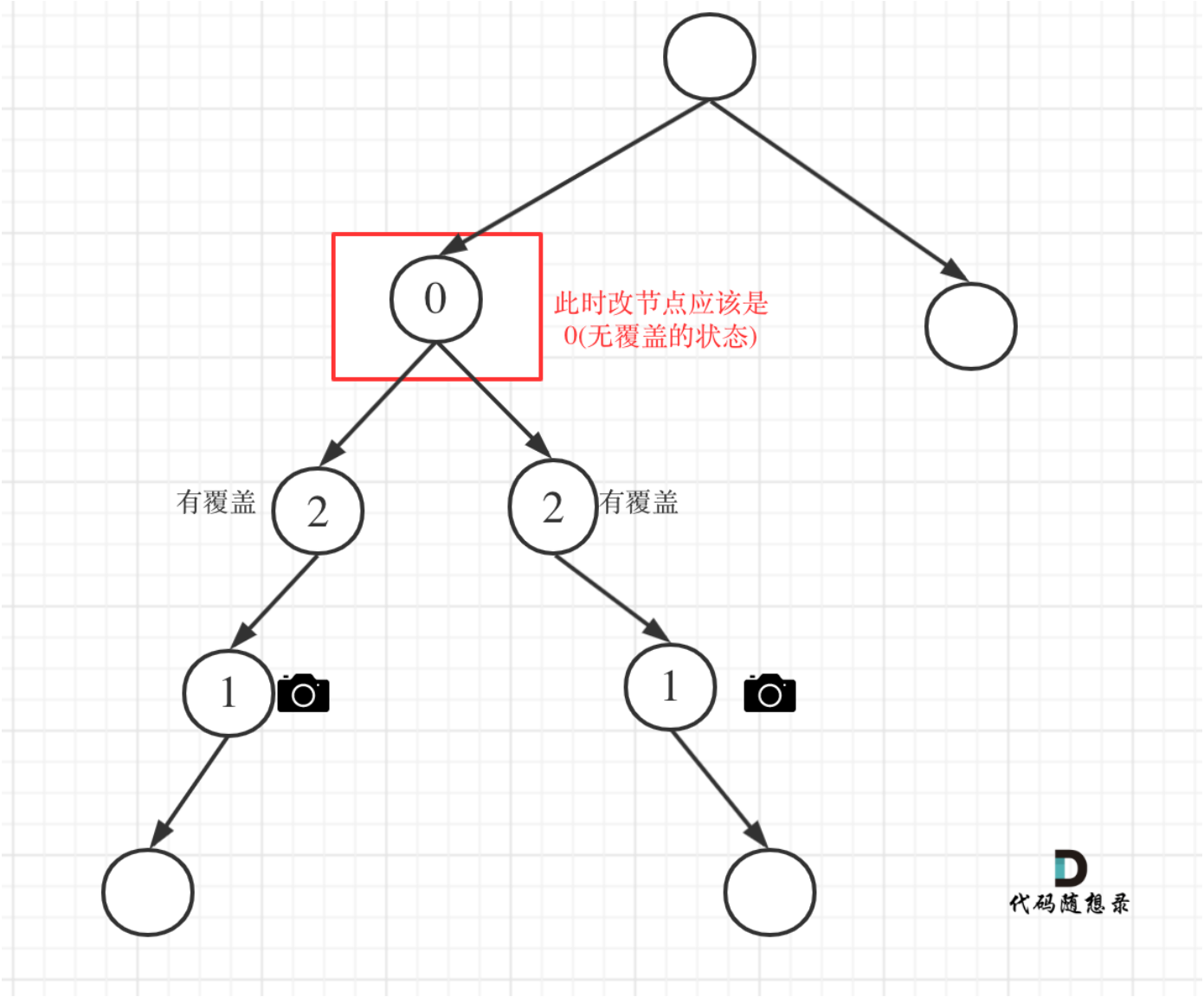
递归的函数，以及终止条件已经确定了，再来看单层逻辑处理。

主要有如下四类情况：

- 情况1：左右节点都有覆盖

左孩子有覆盖，右孩子有覆盖，那么此时中间节点应该就是无覆盖的状态了。

如图：



代码如下：

```
// 左右节点都有覆盖
if (left == 2 && right == 2) return 0;
```

- 情况2：左右节点至少有一个无覆盖的情况

如果是以下情况，则中间节点（父节点）应该放摄像头：

- left == 0 && right == 0 左右节点无覆盖
- left == 1 && right == 0 左节点有摄像头，右节点无覆盖
- left == 0 && right == 1 左节点有无覆盖，右节点摄像头
- left == 0 && right == 2 左节点无覆盖，右节点覆盖
- left == 2 && right == 0 左节点覆盖，右节点无覆盖

这个不难理解，毕竟有一个孩子没有覆盖，父节点就应该放摄像头。

此时摄像头的数量要加一，并且return 1，代表中间节点放摄像头。

代码如下：

```
if (left == 0 || right == 0) {
    result++;
    return 1;
}
```

- 情况3：左右节点至少有一个有摄像头

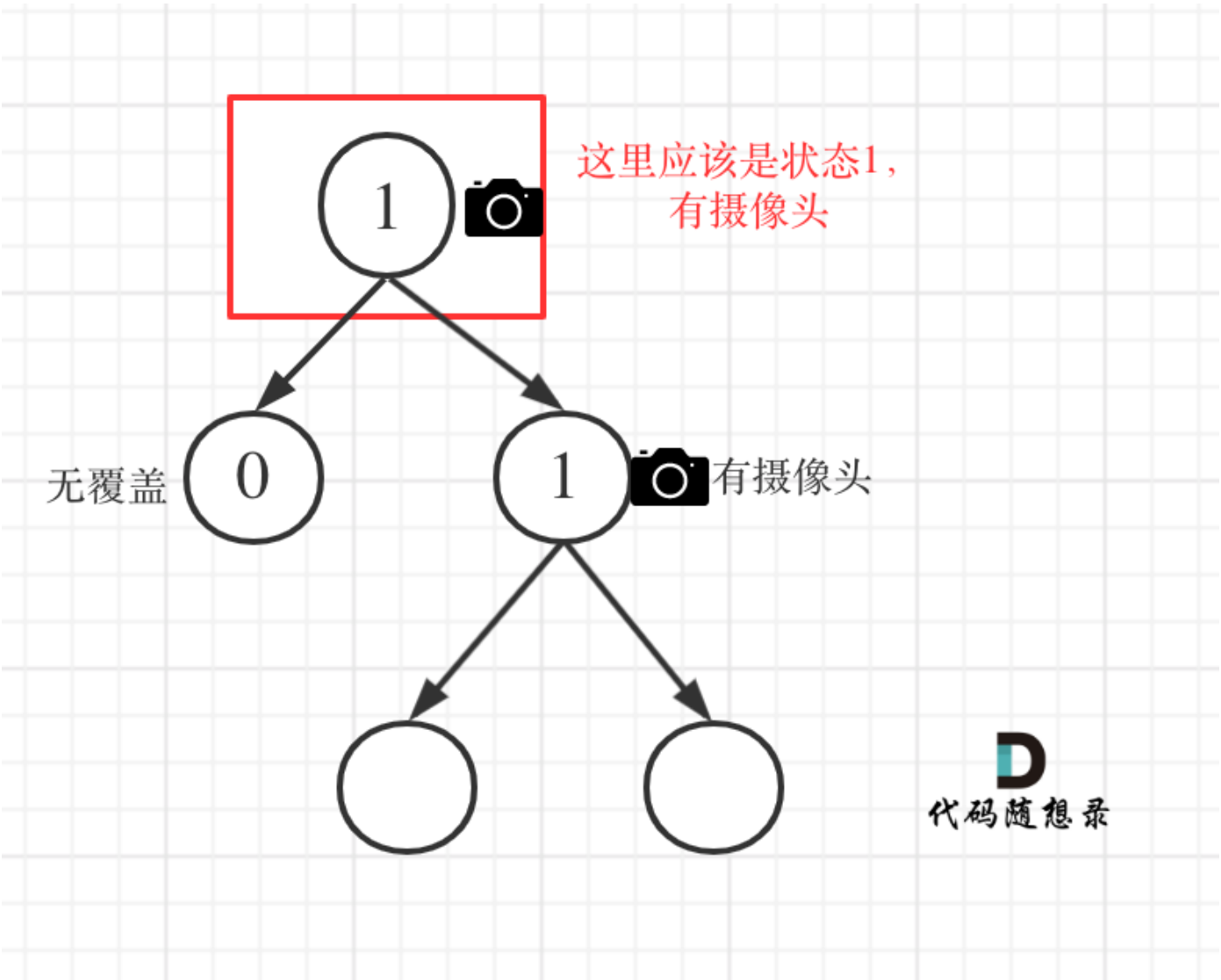
如果是以下情况，其实就是 左右孩子节点有一个有摄像头了，那么其父节点就应该是2（覆盖的状态）

- left == 1 && right == 2 左节点有摄像头，右节点有覆盖
- left == 2 && right == 1 左节点有覆盖，右节点有摄像头
- left == 1 && right == 1 左右节点都有摄像头

代码如下：

```
if (left == 1 || right == 1) return 2;
```

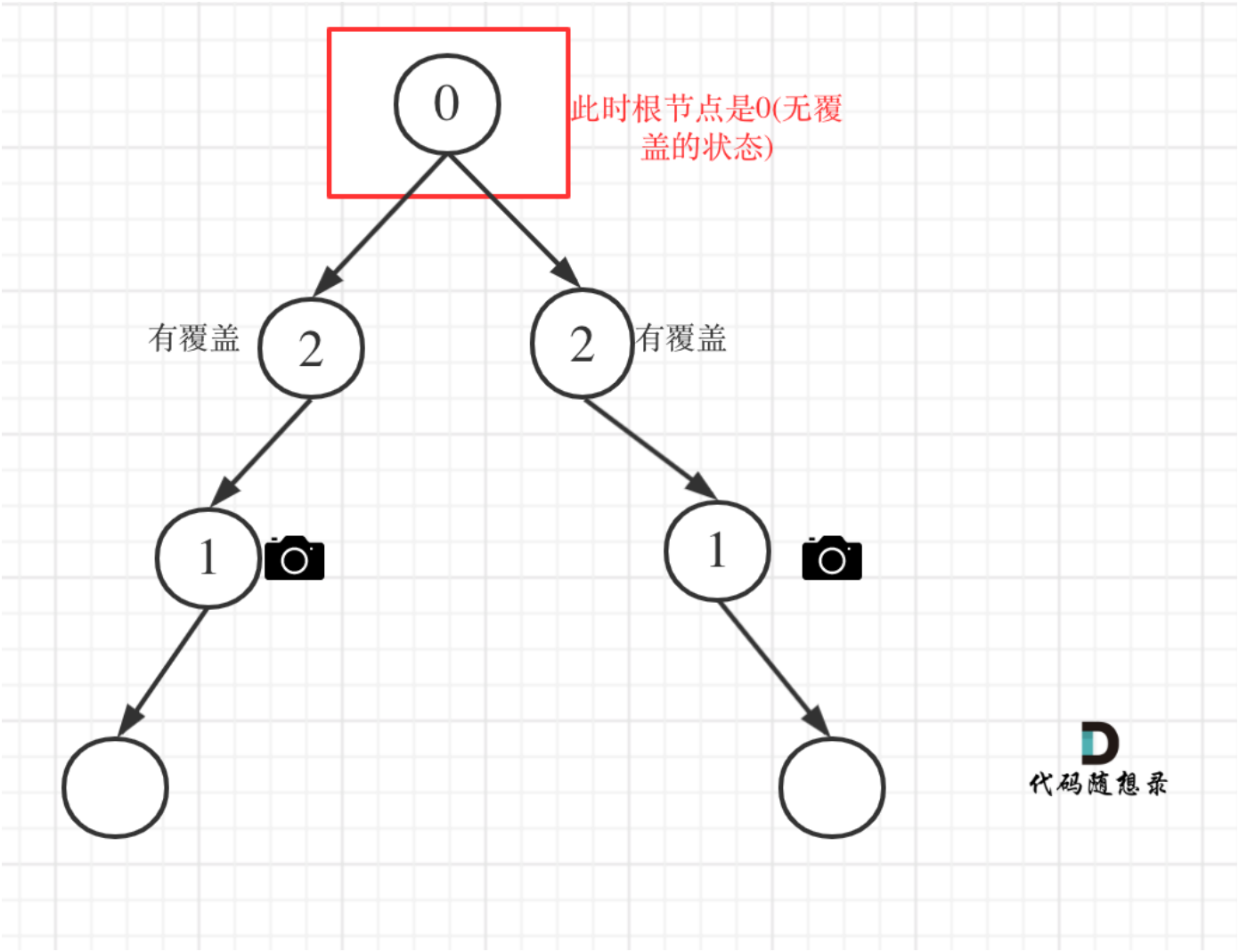
从这个代码中，可以看出，如果left == 1, right == 0 怎么办？其实这种条件在情况2中已经判断过了，如图：



这种情况也是大多数同学容易迷惑的情况。

4. 情况4：头结点没有覆盖

以上都处理完了，递归结束之后，可能头结点 还有一个无覆盖的情况，如图：



所以递归结束之后，还要判断根节点，如果没有覆盖，result++，代码如下：

```
int minCameraCover(TreeNode* root) {
    result = 0;
    if (traversal(root) == 0) { // root 无覆盖
        result++;
    }
    return result;
}
```

以上四种情况我们分析完了，代码也差不多了，整体代码如下：

（以下我的代码注释很详细，为了把情况说清楚，特别把每种情况列出来。）

C++代码如下：

```
// 版本一
class Solution {
private:
```

```
    int result;
    int traversal(TreeNode* cur) {

        // 空节点，该节点有覆盖
        if (cur == NULL) return 2;

        int left = traversal(cur->left);    // 左
        int right = traversal(cur->right);  // 右

        // 情况1
        // 左右节点都有覆盖
        if (left == 2 && right == 2) return 0;

        // 情况2
        // left == 0 && right == 0 左右节点无覆盖
        // left == 1 && right == 0 左节点有摄像头，右节点无覆盖
        // left == 0 && right == 1 左节点有无覆盖，右节点摄像头
        // left == 0 && right == 2 左节点无覆盖，右节点覆盖
        // left == 2 && right == 0 左节点覆盖，右节点无覆盖
        if (left == 0 || right == 0) {
            result++;
            return 1;
        }

        // 情况3
        // left == 1 && right == 2 左节点有摄像头，右节点有覆盖
        // left == 2 && right == 1 左节点有覆盖，右节点有摄像头
        // left == 1 && right == 1 左右节点都有摄像头
        // 其他情况前段代码均已覆盖
        if (left == 1 || right == 1) return 2;

        // 以上代码我没有使用else，主要是为了把各个分支条件展现出来，这样代码有助于读者理解
        // 这个 return -1 逻辑不会走到这里。
        return -1;
    }

public:
    int minCameraCover(TreeNode* root) {
        result = 0;
        // 情况4
        if (traversal(root) == 0) { // root 无覆盖
            result++;
        }
        return result;
    }
};
```

在以上代码的基础上，再进行精简，代码如下：

```
// 版本二
class Solution {
private:
    int result;
    int traversal(TreeNode* cur) {
        if (cur == NULL) return 2;
        int left = traversal(cur->left);    // 左
        int right = traversal(cur->right);  // 右
        if (left == 2 && right == 2) return 0;
        else if (left == 0 || right == 0) {
            result++;
            return 1;
        } else return 2;
    }
public:
    int minCameraCover(TreeNode* root) {
        result = 0;
        if (traversal(root) == 0) { // root 无覆盖
            result++;
        }
        return result;
    }
};
```

- 时间复杂度: O(n)，需要遍历二叉树上的每个节点
- 空间复杂度: O(n)

大家可能会惊讶，居然可以这么简短，**其实就是在版本一的基础上，使用else把一些情况直接覆盖掉了**。

在网上关于这道题解可以搜到很多这种神级别的代码，但都没讲不清楚，如果直接看代码的话，指定越看越晕，**所以建议大家对着版本一的代码一步一步来，版本二中看不中用！**。

总结

本题的难点首先是要想到贪心的思路，然后就是遍历和状态推导。

在二叉树上进行状态推导，其实难度就上了一个台阶了，需要对二叉树的操作非常娴熟。

这道题目是名副其实的hard，大家感受感受。

24. 贪心算法总结篇

我刚刚开始讲解贪心系列的时候就说了，贪心系列并不打算严格的从简单到困难这么个顺序来讲解。

因为贪心的简单题可能往往过于简单甚至感觉不到贪心，如果我连续几天讲解简单的贪心，估计录友们一定会不耐烦了，会感觉贪心有啥好学的。

但贪心的难题又真的有点难，所以我是简单困难交错着讲的，这样大家就感觉难度适中，而且贪心也没有什么框架和套路，所以对刷题顺序要求没有那么高。

但在贪心系列，我发的题目难度会整体呈现一个阶梯状上升，细心的录友们应该有所体会。

在刚刚讲过的回溯系列中，大家可以发现我是严格按照框架难度顺序循序渐进讲解的，**和贪心又不一样，因为回溯法如果题目顺序没选好，刷题效果会非常差！**

同样回溯系列也不允许简单困难交替着来，因为前后题目都是有因果关系的，**相信跟着刷过回溯系列的录友们都会明白我的良苦用心，哈哈。**

每个系列都有每个系列的特点，我都会根据特点有所调整，大家看我每天的推送的题目，都不是随便找一个到就推送的，都是先有整体规划，然后反复斟酌具体题目的结果。

那么在贪心总结篇里，我按难易程度以及题目类型大体归个类。

贪心大总结正式开始：

贪心理论基础

在贪心系列开篇词关于贪心算法，你该了解这些！中，我们就讲解了大家对贪心的普遍疑惑。

1. 贪心很简单，就是常识?

跟着一起刷题的录友们就会发现，贪心思路往往很巧妙，并不简单。

2. 贪心有没有固定的套路?

贪心无套路，也没有框架之类的，需要多看多练培养感觉才能想到贪心的思路。

3. 究竟什么题目是贪心呢?

Carl个人认为：如果找出局部最优并可以推出全局最优，就是贪心，如果局部最优都没找出来，就不是贪心，可能是单纯的模拟。（并不是权威解读，一家之辞哈）

但我们也不用过于强调什么题目是贪心，什么不是贪心，那就太学术了，毕竟学会解题就行了。

4. 如何知道局部最优推出全局最优，有数学证明么?

在做贪心题的过程中，如果再来一个数据证明，其实没有必要，手动模拟一下，如果找不出反例，就试试贪心。面试中，代码写出来跑过测试用例即可，或者自己能自圆其说理由就行了

就像是 要用一下 1 + 1 = 2，没有必要再证明一下 1 + 1 究竟为什么等于 2。（例子极端了点，但是这个道理）

相信大家读完关于贪心算法，你该了解这些！，就对贪心有了一个基本的认识了。

贪心简单题

以下三道题目就是简单题，大家会发现贪心感觉就是常识。是的，如下三道题目，就是靠常识，但我都具体分析了局部最优是什么，全局最优是什么，贪心也要贪的有理有据！

- 贪心算法：分发饼干
- 贪心算法：K次取反后最大化的数组和

- 贪心算法：柠檬水找零

贪心中等题

贪心中等题，靠常识可能就有点想不出来了。开始初现贪心算法的难度与巧妙之处。

- 贪心算法：摆动序列
- 贪心算法：单调递增的数字

贪心解决股票问题

大家都知道股票系列问题是动规的专长，其实用贪心也可以解决，而且还不止就这两道题目，但这两道比较典型，我就拿来单独说一说

- 贪心算法：买卖股票的最佳时机II
- 贪心算法：买卖股票的最佳时机含手续费 本题使用贪心算法比较绕，建议后面学习动态规划章节的时候，理解动规就好

两个维度权衡问题

在出现两个维度相互影响的情况时，两边一起考虑一定会顾此失彼，要先确定一个维度，再确定另一个一个维度。

- 贪心算法：分发糖果
- 贪心算法：根据身高重建队列

在讲解本题的过程中，还强调了编程语言的重要性，模拟插队的时候，使用C++中的list（链表）替代了vector(动态数组)，效率会高很多。

所以在贪心算法：根据身高重建队列（续集）详细讲解了，为什么用list（链表）更快！

大家也要掌握自己所用的编程语言，理解其内部实现机制，这样才能写出高效的算法！

贪心难题

这里的题目如果没有接触过，其实是很难想到的，甚至接触过，也一时想不出来，所以题目不要做一遍，要多练！

贪心解决区间问题

关于区间问题，大家应该印象深刻，有一周我们专门讲解的区间问题，各种覆盖各种去重。

- 贪心算法：跳跃游戏
- 贪心算法：跳跃游戏II
- 贪心算法：用最少数量的箭引爆气球
- 贪心算法：无重叠区间
- 贪心算法：划分字母区间
- 贪心算法：合并区间

其他难题

贪心算法：最大子序和 其实是动态规划的题目，但贪心性能更优，很多同学也是第一次发现贪心能比动规更优的题目。

贪心算法：加油站可能以为是一道模拟题，但就算模拟其实也不简单，需要把while用的很娴熟。但其实是可以使用贪心给时间复杂度降低一个数量级。

最后贪心系列压轴题目贪心算法：我要监控二叉树！，不仅贪心的思路不好想，而且需要对二叉树的操作特别娴熟，这就是典型的交叉类难题了。

贪心每周总结

周总结里会对每周的题目中大家的疑问、相关难点或者笔误之类的进行复盘和总结。

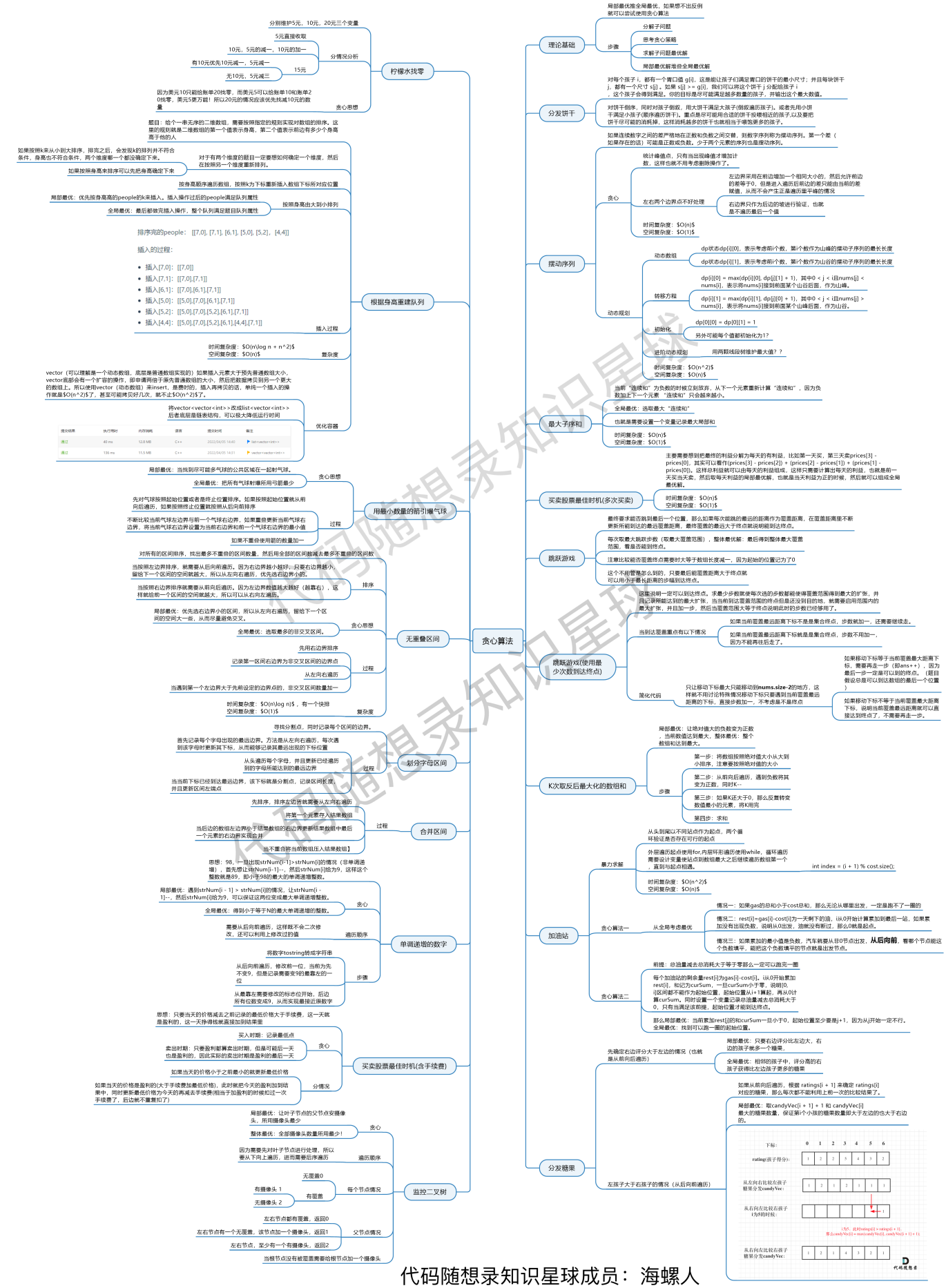
如果大家发现文章哪里有问题，那么在周总结里或者文章评论区一定进行了修正，保证不会因为我的笔误或者理解问题而误导大家，哈哈。

所以周总结一定要看！

- 本周小结！（贪心算法系列一）
- 本周小结！（贪心算法系列二）
- 本周小结！（贪心算法系列三）
- 本周小结！（贪心算法系列四）

总结

贪心专题汇聚为一张图：



代码随想录知识星球成员：海螺人

这个图是 代码随想录知识星球 成员：海螺人所画，总结的非常好，分享给大家。

很多没有接触过贪心的同学都会感觉贪心有啥可学的，但只要跟着「代码随想录」坚持下来之后，就会发现，贪心是一种很重要的算法思维而且并不简单，贪心往往妙的出其不意，触不及防！

回想一下我们刚刚开始讲解贪心的时候，大家会发现自己在坚持中进步了很多！

这也是「代码随想录」的初衷，只要一路坚持下来，不仅基础扎实，而且进步也是飞速的。

在这十八道贪心经典题目中，大家可以发现在每一道题目的讲解中，我都是把什么是局部最优，和什么是全局最优说清楚。

这也是我认为判断这是一道贪心题目的依据，如果找不出局部最优，那可能就是一道模拟题。

不知不觉又一个系列结束了，同时也是2020年的结束。

一个系列的结束，又是一个新系列的开始，我们将在明年第一个工作日正式开始动态规划，来不及解释了，录友们上车别掉队，我们又要开始新的征程！

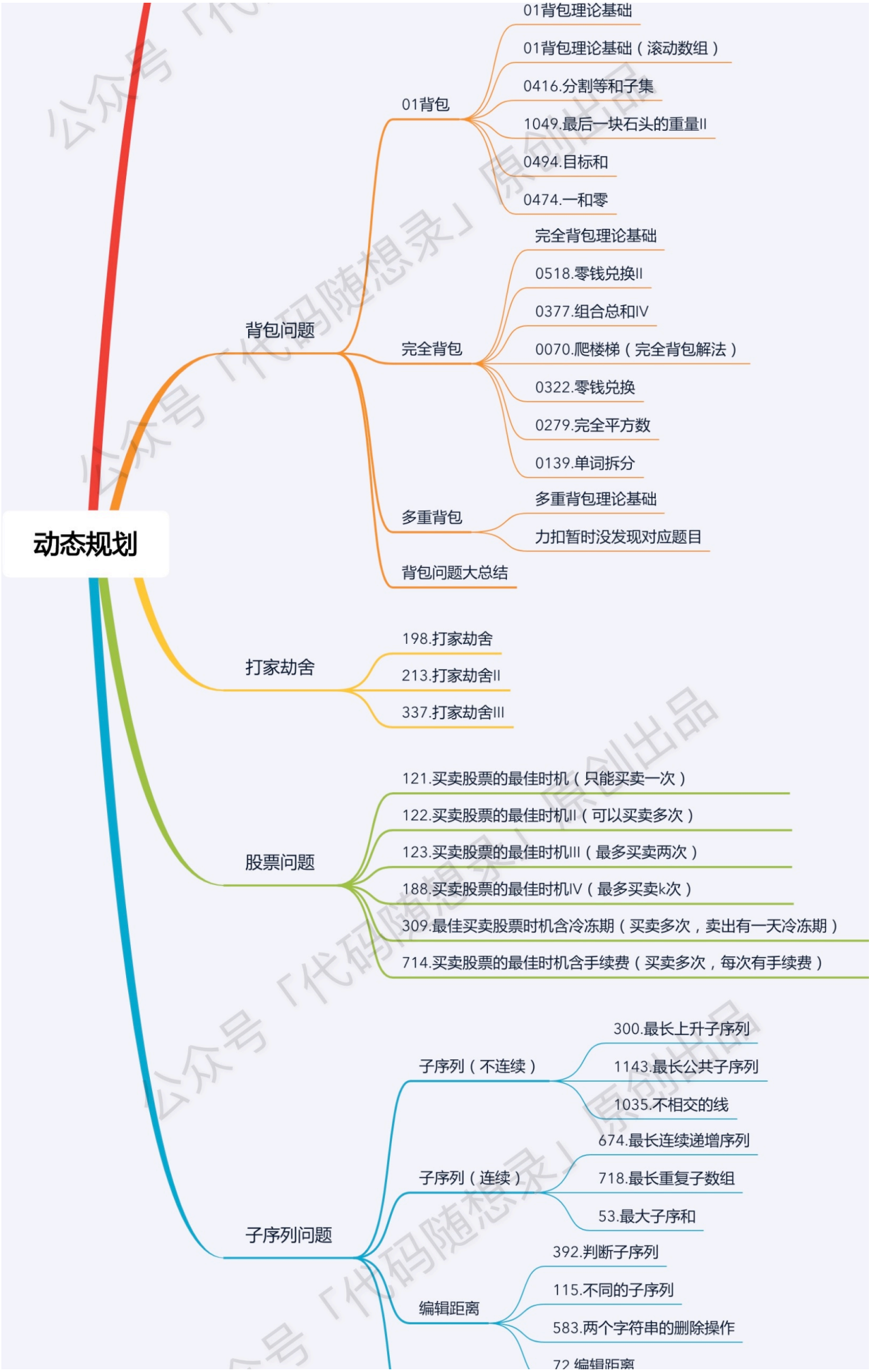
《代码随想录》作者：程序员Carl

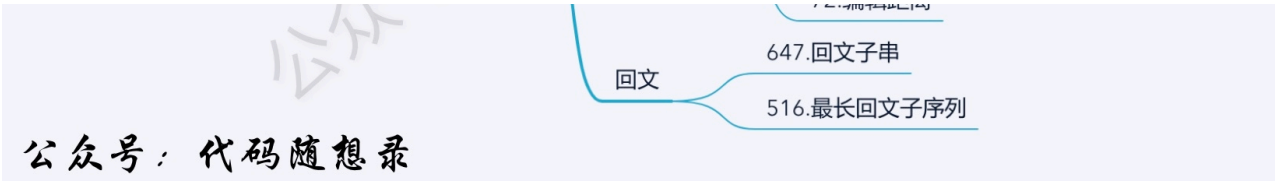
- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 动态规划理论基础

动态规划刷题大纲


动态规划
背包问题
01背包
01背包理论基础
01背包理论基础（滚动数组）
0416.分割等和子集
1049.最后一块石头的重量II
0494.目标和
0474.一和零
完全背包
完全背包理论基础
0518.零钱兑换II
0377.组合总和IV
0070.爬楼梯（完全背包解法）
0322.零钱兑换
0279.完全平方数
0139.单词拆分
多重背包
多重背包理论基础
力扣暂时没发现对应题目
背包问题大总结
打家劫舍
198.打家劫舍
213.打家劫舍II
337.打家劫舍III
股票问题
121.买卖股票的最佳时机（只能买卖一次）
122.买卖股票的最佳时机II（可以买卖多次）
123.买卖股票的最佳时机III（最多买卖两次）
188.买卖股票的最佳时机IV（最多买卖k次）
309.最佳买卖股票时机含冷冻期（买卖多次，卖出有一天冷冻期）
714.买卖股票的最佳时机含手续费（买卖多次，每次有手续费）
子序列问题
子序列（不连续）
300.最长上升子序列
1143.最长公共子序列
1035.不相交的线
子序列（连续）
674.最长连续递增序列
718.最长重复子数组
53.最大子序和
编辑距离
392.判断子序列
115.不同的子序列
583.两个字符串的删除操作
72.编辑距离



算法公开课

[《代码随想录》算法视频公开课](#)：[动态规划理论基础](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

什么是动态规划

动态规划，英文：Dynamic Programming，简称DP，如果某一问题有很多重叠子问题，使用动态规划是最有效的。

所以动态规划中每一个状态一定是由上一个状态推导出来的，**这一点就区分于贪心**，贪心没有状态推导，而是从局部直接选最优的，

在[关于贪心算法，你该了解这些！](#)中我举了一个背包问题的例子。

例如：有N件物品和一个最多能背重量为W 的背包。第i件物品的重量是weight[i]，得到的价值是value[i] 。**每件物品只能用一次**，求解将哪些物品装入背包里物品价值总和最大。

动态规划中dp[j]是由dp[j-weight[i]]推导出来的，然后取max(dp[j], dp[j - weight[i]] + value[i])。

但如果是贪心呢，每次拿物品选一个最大的或者最小的就完事了，和上一个状态没有关系。

所以贪心解决不了动态规划的问题。

其实大家也不用死扣动规和贪心的理论区别，后面做做题目自然就知道了。

而且很多讲解动态规划的文章都会讲最优子结构啊和重叠子问题啊这些，这些东西都是教科书的上定义，晦涩难懂而且不实用。

大家知道动规是由前一个状态推导出来的，而贪心是局部直接选最优的，对于刷题来说就够用了。

上述提到的背包问题，后序会详细讲解。

动态规划的解题步骤

做动规题目的时候，很多同学会陷入一个误区，就是以为把状态转移公式背下来，照葫芦画瓢改改，就开始写代码，甚至把题目AC之后，都不太清楚dp[i]表示的是什么。

这就是一种朦胧的状态，然后就把题给过了，遇到稍稍难一点的，可能直接就不会了，然后看题解，然后继续照葫芦画瓢陷入这种恶性循环中。

状态转移公式（递推公式）是很重要，但动规不仅仅只有递推公式。

对于动态规划问题，我将拆解为如下五步曲，这五步都搞清楚了，才能说把动态规划真的掌握了！

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序

5. 举例推导dp数组

一些同学可能想为什么要先确定递推公式，然后在考虑初始化呢?

因为一些情况是递推公式决定了dp数组要如何初始化!

后面的讲解中我都是围绕着这五点来进行讲解。

可能刷过动态规划题目的同学可能都知道递推公式的重要性，感觉确定了递推公式这道题目就解出来了。

其实 确定递推公式 仅仅是解题里的一步而已!

一些同学知道递推公式，但搞不清楚dp数组应该如何初始化，或者正确的遍历顺序，以至于记下来公式，但写的程序怎么改都通过不了。

后序的讲解的大家就会慢慢感受到这五步的重要性了。

动态规划应该如何debug

相信动规的题目，很大部分同学都是这样做的。

看一下题解，感觉看懂了，然后照葫芦画瓢，如果能正好画对了，万事大吉，一旦要是没通过，就怎么改都通过不了，对 dp数组的初始化，递推公式，遍历顺序，处于一种黑盒的理解状态。

写动规题目，代码出问题很正常!

找问题的最好方式就是把dp数组打印出来，看看究竟是不是按照自己思路推导的!

一些同学对于dp的学习是黑盒的状态，就是不清楚dp数组的含义，不懂为什么这么初始化，递推公式背下来了，遍历顺序靠习惯就是这么写的，然后一鼓作气写出代码，如果代码能通过万事大吉，通过不了的话就凭感觉改一改。

这是一个很不好的习惯!

做动规的题目，写代码之前一定要把状态转移在dp数组的上具体情况模拟一遍，心中有数，确定最后推出的是想要的结果。

然后再写代码，如果代码没通过就打印dp数组，看看是不是和自己预先推导的哪里不一样。

如果打印出来和自己预先模拟推导是一样的，那么就是自己的递归公式、初始化或者遍历顺序有问题了。

如果和自己预先模拟推导的不一样，那么就是代码实现细节有问题。

这样才是一个完整的思考过程，而不是一旦代码出问题，就毫无头绪的东改改西改改，最后过不了，或者说是稀里糊涂的过了。

这也是我为什么在动规五步曲里强调推导dp数组的重要性。

举个例子哈：在「代码随想录」刷题小分队微信群里，一些录友可能代码通过不了，会把代码抛到讨论群里问：我这里代码都已经和题解一模一样了，为什么通过不了呢?

发出这样的问题之前，其实可以自己先思考这三个问题：

- 这道题目我举例推导状态转移公式了么?
- 我打印dp数组的日志了么?
- 打印出来了dp数组和我想的一样么?

如果这灵魂三问自己都做到了，基本上这道题目也就解决了，或者更清晰的知道自己究竟是哪一点不明白，是状态转移不明白，还是实现代码不知道该怎么写，还是不理解遍历dp数组的顺序。

然后在问问题，目的性就很强了，群里的小伙伴也可以快速知道提问者的疑惑了。

注意这里不是说不让大家问问题哈， 而是说问问题之前要有自己的思考，问题要问到点子上！

大家工作之后就会发现，特别是大厂，问问题是一个专业活，是的，问问题也要体现出专业！

如果问同事很不专业的问题，同事们会懒的回答，领导也会认为你缺乏思考能力，这对职场发展是很不利的。

所以大家在刷题的时候，就锻炼自己养成专业提问的好习惯。

总结

这一篇是动态规划的整体概述，讲解了什么是动态规划，动态规划的解题步骤，以及如何debug。

动态规划是一个很大的领域，今天这一篇讲解的内容是整个动态规划系列中都会使用到的一些理论基础。

在后序讲解中针对某一具体问题，还会讲解其对应的理论基础，例如背包问题中的01背包，leetcode上的题目都是01背包的应用，而没有纯01背包的问题，那么就需要在把对应的理论知识讲解一下。

大家会发现，我讲解的理论基础并不是教科书上各种动态规划的定义，错综复杂的公式。

这里理论基础篇已经是非常偏实用的了，每个知识点都是在解题实战中非常有用的内容，大家要重视起来哈。

今天我们开始新的征程了，你准备好了么？

---

2. 斐波那契数

力扣题目链接

斐波那契数，通常用 F(n) 表示，形成的序列称为 斐波那契数列 。该数列由 0 和 1 开始，后面的每一项数字都是前面两项数字的和。也就是：
F(0) = 0，F(1) = 1
F(n) = F(n - 1) + F(n - 2)，其中 n > 1
给你n ，请计算 F(n) 。

示例 1：

- 输入：2
- 输出：1
- 解释：F(2) = F(1) + F(0) = 1 + 0 = 1

示例 2：

- 输入：3
- 输出：2
- 解释：F(3) = F(2) + F(1) = 1 + 1 = 2

示例 3：

- 输入：4
- 输出：3
- 解释：F(4) = F(3) + F(2) = 2 + 1 = 3

提示：

- 0 <= n <= 30

算法公开课

《代码随想录》算法视频公开课：手把手带你入门动态规划 | leetcode：509.斐波那契数，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

斐波那契数列大家应该非常熟悉不过了，非常适合作为动规第一道题目来练练手。

因为这道题目比较简单，可能一些同学并不需要做什么分析，直接顺手一写就过了。

但「代码随想录」的风格是：简单题目是用来加深对解题方法论的理解的。

通过这道题目让大家可以初步认识到，按照动规五部曲是如何解题的。

对于动规，如果没有方法论的话，可能简单题目可以顺手一写就过，难一点就不知道如何下手了。

所以我总结的动规五部曲，是要用来贯穿整个动态规划系列的，就像之前讲过二叉树系列的递归三部曲，回溯法系列的回溯三部曲一样。后面慢慢大家就会体会到，动规五部曲方法的重要性。

动态规划

动规五部曲：

这里我们要用一个一维dp数组来保存递归的结果

1. 确定dp数组以及下标的含义

dp[i]的定义为：第i个数的斐波那契数值是dp[i]

2. 确定递推公式

为什么这是一道非常简单的入门题目呢?

因为题目已经把递推公式直接给我们了：状态转移方程 dp[i] = dp[i - 1] + dp[i - 2];

3. dp数组如何初始化

题目中把如何初始化也直接给我们了，如下：

```
dp[0] = 0;
dp[1] = 1;
```

4. 确定遍历顺序

从递归公式dp[i] = dp[i - 1] + dp[i - 2];中可以看出，dp[i]是依赖 dp[i - 1] 和 dp[i - 2]，那么遍历的顺序一定是从前到后遍历的

5. 举例推导dp数组

按照这个递推公式dp[i] = dp[i - 1] + dp[i - 2]，我们来推导一下，当N为10的时候，dp数组应该是如下的数列：

0 1 1 2 3 5 8 13 21 34 55

如果代码写出来，发现结果不对，就把dp数组打印出来看看和我们推导的数列是不是一致的。

以上我们用动规的方法分析完了，C++代码如下：

```
class Solution {
public:
    int fib(int N) {
        if (N <= 1) return N;
        vector<int> dp(N + 1);
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= N; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[N];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

当然可以发现，我们只需要维护两个数值就可以了，不需要记录整个序列。

代码如下：

```
class Solution {
public:
    int fib(int N) {
        if (N <= 1) return N;
        int dp[2];
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= N; i++) {
            int sum = dp[0] + dp[1];
            dp[0] = dp[1];
            dp[1] = sum;
        }
        return dp[1];
    }
};
```

- 时间复杂度：O(n)

- 空间复杂度：O(1)

递归解法

本题还可以使用递归解法来做

代码如下：

```
class Solution {
public:
    int fib(int N) {
        if (N < 2) return N;
        return fib(N - 1) + fib(N - 2);
    }
};
```

- 时间复杂度：O(2^n)
- 空间复杂度：O(n)，算上了编程语言中实现递归的系统栈所占空间

这个递归的时间复杂度大家画一下树形图就知道了，如果不清晰的同学，可以看这篇：通过一道面试题目，讲一讲递归算法的时间复杂度！

总结

斐波那契数列这道题目是非常基础的题目，我在后面的动态规划的讲解中将会多次提到斐波那契数列！

这里我严格按照关于动态规划，你该了解这些！中的动规五部曲来分析了这道题目，一些分析步骤可能同学感觉没有必要搞的这么复杂，代码其实上来就可以撸出来。

但我还是强调一下，简单题是用来掌握方法论的，动规五部曲将在接下来的动态规划讲解中发挥重要作用，敬请期待！

就酱，循序渐进学算法，认准「代码随想录」！

---

3. 爬楼梯

力扣题目链接

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢?

注意：给定 n 是一个正整数。

示例 1：

- 输入： 2
- 输出： 2

- 解释： 有两种方法可以爬到楼顶。
    - 1 阶 + 1 阶
    - 2 阶

示例 2：

- 输入： 3
- 输出： 3
- 解释： 有三种方法可以爬到楼顶。
    - 1 阶 + 1 阶 + 1 阶
    - 1 阶 + 2 阶
    - 2 阶 + 1 阶

算法公开课

[《代码随想录》算法视频公开课](#)：[带你学透动态规划-爬楼梯|LeetCode:70.爬楼梯）](#)，相信结合视频在看本篇题解，更有助于大家对本题的理解。

思路

本题大家如果没有接触过的话，会感觉比较难，多举几个例子，就可以发现其规律。

爬到第一层楼梯有一种方法，爬到二层楼梯有两种方法。

那么第一层楼梯再跨两步就到第三层 ，第二层楼梯再跨一步就到第三层。

所以到第三层楼梯的状态可以由第二层楼梯 和 到第一层楼梯状态推导出来，那么就可以想到动态规划了。

我们来分析一下，动规五部曲：

定义一个一维数组来记录不同楼层的状态

1. 确定dp数组以及下标的含义

dp[i]： 爬到第i层楼梯，有dp[i]种方法

2. 确定递推公式

如何可以推出dp[i]呢?

从dp[i]的定义可以看出，dp[i] 可以有两个方向推出来。

首先是dp[i - 1]，上i-1层楼梯，有dp[i - 1]种方法，那么再一步跳一个台阶不就是dp[i]了么。

还有就是dp[i - 2]，上i-2层楼梯，有dp[i - 2]种方法，那么再一步跳两个台阶不就是dp[i]了么。

那么dp[i]就是 dp[i - 1]与dp[i - 2]之和!

所以dp[i] = dp[i - 1] + dp[i - 2] 。

在推导dp[i]的时候，一定要时刻想着dp[i]的定义，否则容易跑偏。

这体现出确定dp数组以及下标的含义的重要性!

3. dp数组如何初始化

再回顾一下dp[i]的定义：爬到第i层楼梯，有dp[i]种方法。

那么i为0，dp[i]应该是多少呢，这个可以有很多解释，但基本都是直接奔着答案去解释的。

例如强行安慰自己爬到第0层，也有一种方法，什么都不做也就是一种方法即：dp[0] = 1，相当于直接站在楼顶。

但总有点牵强的成分。

那还这么理解呢：我就认为跑到第0层，方法就是0啊，一步只能走一个台阶或者两个台阶，然而楼层是0，直接站楼顶上了，就是不用方法，dp[0]就应该是0.

其实这么争论下去没有意义，大部分解释说dp[0]应该为1的理由其实是因为dp[0]=1的话在递推的过程中i从2开始遍历本题就能过，然后就往结果上靠去解释dp[0] = 1。

从dp数组定义的角度上来说，dp[0] = 0 也能说得通。

需要注意的是：题目中说了n是一个正整数，题目根本就没说n有为0的情况。

所以本题其实就不应该讨论dp[0]的初始化！

我相信dp[1] = 1，dp[2] = 2，这个初始化大家应该都没有争议的。

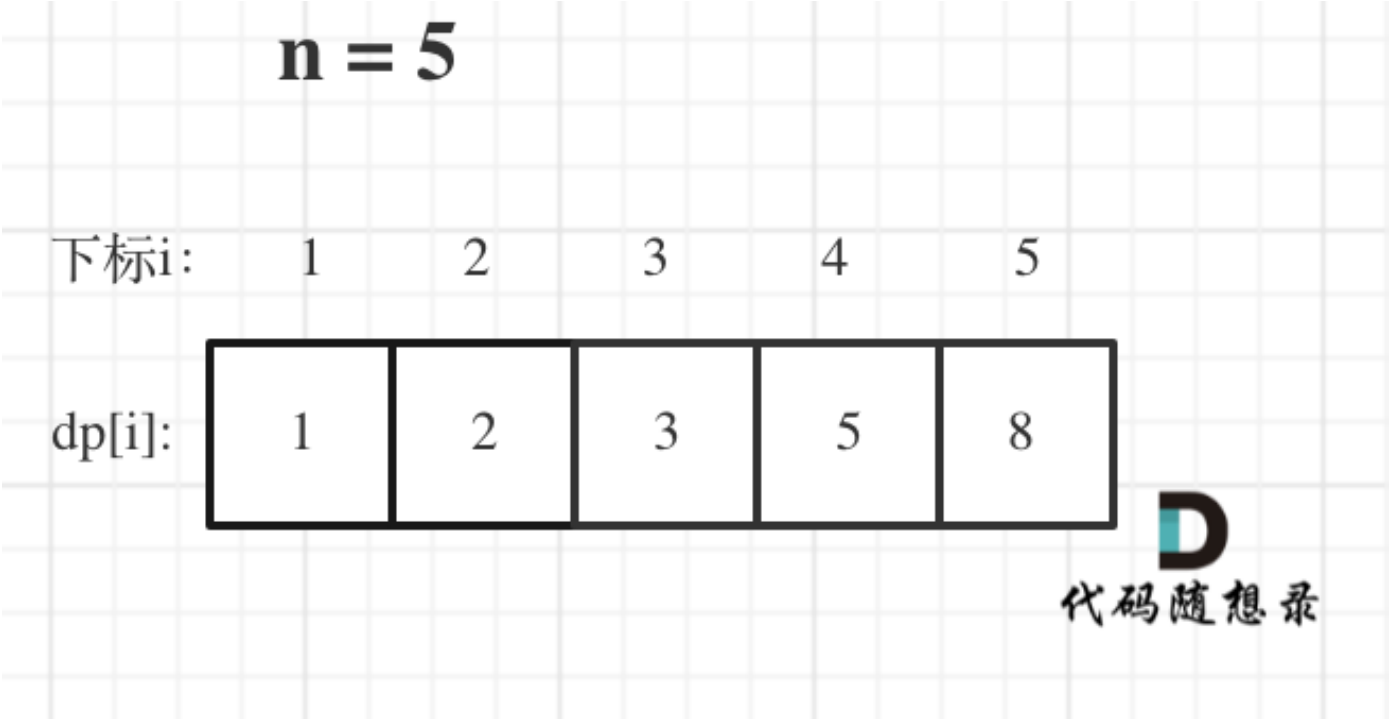
所以我的原则是：不考虑dp[0]如何初始化，只初始化dp[1] = 1，dp[2] = 2，然后从i = 3开始递推，这样才符合dp[i]的定义。

4. 确定遍历顺序

从递推公式dp[i] = dp[i - 1] + dp[i - 2];中可以看出，遍历顺序一定是从前向后遍历的

5. 举例推导dp数组

举例当n为5的时候，dp table（dp数组）应该是这样的



如果代码出问题了，就把dp table 打印出来，看看究竟是不是和自己推导的一样。

此时大家应该发现了，这不就是斐波那契数列么！

唯一的区别是，没有讨论dp[0]应该是什么，因为dp[0]在本题没有意义！

以上五部分析完之后，C++代码如下：

```
// 版本一
class Solution {
public:
    int climbStairs(int n) {
        if (n <= 1) return n; // 因为下面直接对dp[2]操作了，防止空指针
        vector<int> dp(n + 1);
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) { // 注意i是从3开始的
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
};
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(n)$

当然依然也可以，优化一下空间复杂度，代码如下：

```
// 版本二
class Solution {
public:
    int climbStairs(int n) {
        if (n <= 1) return n;
        int dp[3];
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
            int sum = dp[1] + dp[2];
            dp[1] = dp[2];
            dp[2] = sum;
        }
        return dp[2];
    }
};
```

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

后面将讲解的很多动规的题目其实都是当前状态依赖前两个，或者前三个状态，都可以做空间上的优化，**但我个人认为面试中能写出版本一就够了哈，清晰明了，如果面试官要求进一步优化空间的话，我们再去优化。**

因为版本一才能体现出动规的思想精髓，递推的状态变化。

拓展

这道题目还可以继续深化，就是一步一个台阶，两个台阶，三个台阶，直到 m个台阶，有多少种方法爬到n阶楼顶。

这又有难度了，这其实是一个完全背包问题，但力扣上没有这种题目，所以后续我在讲解背包问题的时候，今天这道题还会从背包问题的角度上来再讲一遍。 如果想提前看一下，可以看这篇:70.爬楼梯完全背包版本

这里我先给出我的实现代码：

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) { // 把m换成2，就可以AC爬楼梯这道题
                if (i - j >= 0) dp[i] += dp[i - j];
            }
        }
        return dp[n];
    }
};
```

代码中m表示最多可以爬m个台阶。

以上代码不能运行哈，我主要是为了体现只要把m换成2，粘过去，就可以AC爬楼梯这道题，不信你就粘一下试试。

此时我就发现一个绝佳的大厂面试题，第一道题就是单纯的爬楼梯，然后看候选人的代码实现，如果把dp[0]的定义成1了，就可以发难了，为什么dp[0]一定要初始化为1，此时可能候选人就要强行给dp[0]应该是1找各种理由。那这就是一个考察点了，对dp[i]的定义理解的不深入。

然后可以继续发难，如果一步一个台阶，两个台阶，三个台阶，直到 m个台阶，有多少种方法爬到n阶楼顶。这道题目leetcode上并没有原题，绝对是考察候选人算法能力的绝佳好题。

这一连套问下来，候选人算法能力如何，面试官心里就有数了。

其实大厂面试最喜欢的问题就是这种简单题，然后慢慢变化，在小细节上考察候选人。

总结

这道题目和动态规划：斐波那契数题目基本是一样的，但是会发现本题相比动态规划：斐波那契数难多了，为什么呢?

关键是 动态规划：斐波那契数 题目描述就已经把动规五部曲里的递归公式和如何初始化都给出来了，剩下几部曲也自然而然的推出来了。

而本题，就需要逐个分析了，大家现在应该初步感受出关于动态规划，你该了解这些！里给出的动规五部曲了。

简单题是用来掌握方法论的，例如昨天斐波那契的题目够简单了吧，但昨天和今天可以使用一套方法分析出来的，这就是方法论！

所以不要轻视简单题，那种凭感觉就刷过去了，其实和没掌握区别不大，只有掌握方法论并说清一二三，才能触类旁通，举一反三哈！

就酱，循序渐进学算法，认准「代码随想录」！

---

4. 使用最小花费爬楼梯

力扣题目链接

旧题目描述：

数组的每个下标作为一个阶梯，第 i 个阶梯对应着一个非负数的体力花费值 cost[i]（下标从 0 开始）。

每当你爬上一个阶梯你都要花费对应的体力值，一旦支付了相应的体力值，你就可以选择向上爬一个阶梯或者爬两个阶梯。

请你找出达到楼层顶部的最低花费。在开始时，你可以选择从下标为 0 或 1 的元素作为初始阶梯。

示例 1：

- 输入：cost = [10, 15, 20]
- 输出：15
- 解释：最低花费是从 cost[1] 开始，然后走两步即可到阶梯顶，一共花费 15 。

示例 2：

- 输入：cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
- 输出：6
- 解释：最低花费方式是从 cost[0] 开始，逐个经过那些 1 ，跳过 cost[3] ，一共花费 6 。

提示：

- cost 的长度范围是 [2, 1000]。
- cost[i] 将会是一个整型数据，范围为 [0, 999] 。

算法公开课

《代码随想录》算法视频公开课：：动态规划开更了！ | LeetCode：746. 使用最小花费爬楼梯，相信结合视频再看本篇题解，更有助于大家对本题的理解。

本题之前的题目描述是很模糊的，看不出来，第一步需要花费体力值，最后一步不用花费，还是说 第一步不花费体力值，最后一步花费。

后来力扣改了题目描述，**新题目描述**：

给你一个整数数组 cost ，其中 cost[i] 是从楼梯第 i 个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个或者两个台阶。

你可以选择从下标为 0 或下标为 1 的台阶开始爬楼梯。

请你计算并返回达到楼梯顶部的最低花费。

示例 1：

```
输入：cost = [10,15,20]
输出：15
解释：你将从下标为 1 的台阶开始。
- 支付 15 ，向上爬两个台阶，到达楼梯顶部。
总花费为 15 。
```

示例 2：

```
输入：cost = [1,100,1,1,1,100,1,1,100,1]
输出：6
解释：你将从下标为 0 的台阶开始。
- 支付 1 ，向上爬两个台阶，到达下标为 2 的台阶。
- 支付 1 ，向上爬两个台阶，到达下标为 4 的台阶。
- 支付 1 ，向上爬两个台阶，到达下标为 6 的台阶。
- 支付 1 ，向上爬一个台阶，到达下标为 7 的台阶。
- 支付 1 ，向上爬两个台阶，到达下标为 9 的台阶。
- 支付 1 ，向上爬一个台阶，到达楼梯顶部。
总花费为 6 。
```

思路

（在力扣修改了题目描述下，我又重新修改了题解）

修改之后的题意就比较明确了，题目中说 “你可以选择从下标为 0 或下标为 1 的台阶开始爬楼梯” 也就是相当于 跳到 下标 0 或者 下标 1 是不花费体力的， 从 下标 0 下标1 开始跳就要花费体力了。

1. 确定dp数组以及下标的含义

使用动态规划，就要有一个数组来记录状态，本题只需要一个一维数组dp[i]就可以了。

dp[i]的定义：到达第i台阶所花费的最少体力为dp[i]。

对于dp数组的定义，大家一定要清晰！

2. 确定递推公式

可以有两个途径得到dp[i]，一个是dp[i-1] 一个是dp[i-2]。

dp[i - 1] 跳到 dp[i] 需要花费 dp[i - 1] + cost[i - 1]。

dp[i - 2] 跳到 dp[i] 需要花费 dp[i - 2] + cost[i - 2]。

那么究竟是选从dp[i - 1]跳还是从dp[i - 2]跳呢?

一定是选最小的，所以dp[i] = min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);

3. dp数组如何初始化

看一下递归公式，dp[i]由dp[i - 1]，dp[i - 2]推出，既然初始化所有的dp[i]是不可能的，那么只初始化dp[0]和dp[1]就够了，其他的最终都是dp[0]dp[1]推出。

那么 dp[0] 应该是多少呢？ 根据dp数组的定义，到达第0台阶所花费的最小体力为dp[0]，那么有同学可能想，那 dp[0] 应该是 cost[0]，例如 cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1] 的话，dp[0] 就是 cost[0] 应该是1。

这里就要说明本题力扣为什么改题意，而且修改题意之后 就清晰很多的原因了。

新题目描述中明确说了 “你可以选择从下标为 0 或下标为 1 的台阶开始爬楼梯。” 也就是说 到达 第 0 个台阶是不花费的，但从 第0 个台阶 往上跳的话，需要花费 cost[0]。

所以初始化 dp[0] = 0，dp[1] = 0;

4. 确定遍历顺序

最后一步，递归公式有了，初始化有了，如何遍历呢?

本题的遍历顺序其实比较简单，简单到很多同学都忽略了思考这一步直接就把代码写出来了。

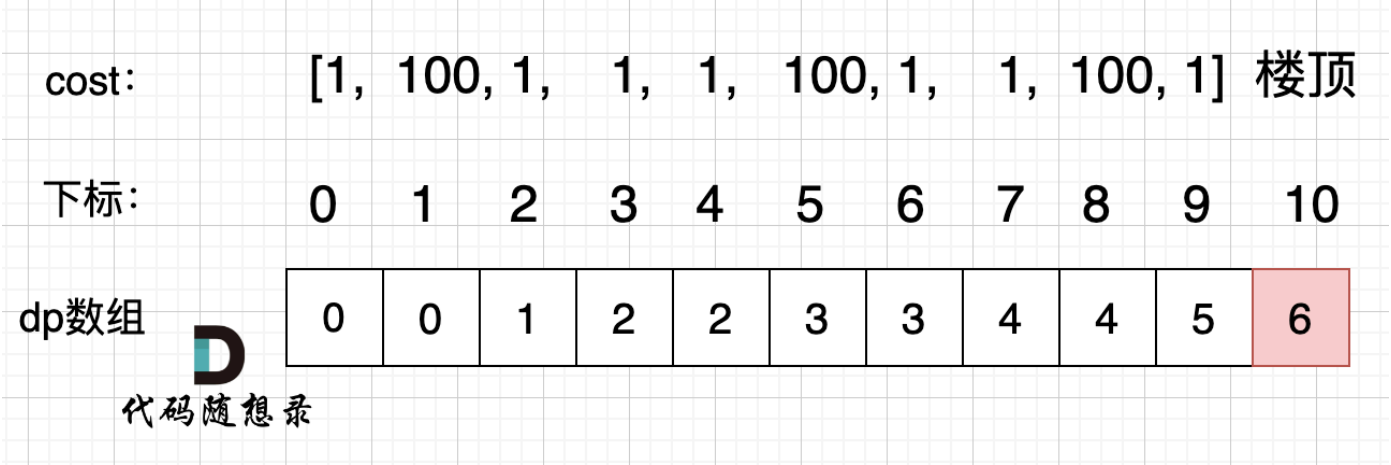
因为是模拟台阶，而且dp[i]由dp[i-1]dp[i-2]推出，所以是从前到后遍历cost数组就可以了。

> **但是稍稍有点难度的动态规划，其遍历顺序并不容易确定下来。**
> 例如：01背包，都知道两个for循环，一个for遍历物品嵌套一个for遍历背包容量，那么为什么不是一个for遍历背包容量嵌套一个for遍历物品呢？ 以及在使用一维dp数组的时候遍历背包容量为什么要倒序呢?

这些都与遍历顺序息息相关。当然背包问题后续「代码随想录」都会重点讲解的!

5. 举例推导dp数组

拿示例2：cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1] ，来模拟一下dp数组的状态变化，如下：



如果大家代码写出来有问题，就把dp数组打印出来，看看和如上推导的是不是一样的。

以上分析完毕，整体C++代码如下：

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        vector<int> dp(cost.size() + 1);
        dp[0] = 0; // 默认第一步都是不花费体力的
        dp[1] = 0;
        for (int i = 2; i <= cost.size(); i++) {
            dp[i] = min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);
        }
        return dp[cost.size()];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

还可以优化空间复杂度，因为dp[i]就是由前两位推出来的，那么也不用dp数组了，C++代码如下：

```
// 版本二
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        int dp0 = 0;
        int dp1 = 0;
        for (int i = 2; i <= cost.size(); i++) {
            int dpi = min(dp1 + cost[i - 1], dp0 + cost[i - 2]);
            dp0 = dp1; // 记录一下前两位
            dp1 = dpi;
        }
        return dp1;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

当然如果在面试中，能写出版本一就行，除非面试官额外要求 空间复杂度，那么再去思考版本二，因为版本二还是有点绕。版本一才是正常思路。

拓展

旧力扣描述，如果按照 第一步是花费的，最后一步不花费，那么代码是这么写的，提交也可以通过

```
// 版本一
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
```

```
        vector<int> dp(cost.size());
        dp[0] = cost[0]; // 第一步有花费
        dp[1] = cost[1];
        for (int i = 2; i < cost.size(); i++) {
            dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];
        }
        // 注意最后一步可以理解为不用花费，所以取倒数第一步，第二步的最少值
        return min(dp[cost.size() - 1], dp[cost.size() - 2]);
    }
};
```

当然如果对 动态规划 理解不够深入的话，拓展内容就别看了，容易越看越懵。

总结

大家可以发现这道题目相对于 昨天的动态规划：爬楼梯又难了一点，但整体思路是一样的。

从动态规划：斐波那契数到 动态规划：爬楼梯再到今天这道题目，录友们感受到循序渐进的梯度了嘛。

每个系列开始的时候，都有录友和我反馈说题目太简单了，赶紧上难度，但也有录友和我说有点难了，快跟不上了。

其实我选的题目都是有目的性的，就算是简单题，也是为了练习方法论，然后难度都是梯度上来的，一环扣一环。

但我也可以随便选来一道难题讲呗，这其实是最省事的，不用管什么题目顺序，看心情找一道就讲。

难的是把题目按梯度排好，循序渐进，再按照统一方法论把这些都串起来，所以大家不要催我哈，按照我的节奏一步一步来就行了。

---

5. 本周小结！（动态规划系列一）

这周我们正式开始动态规划的学习！

周一

在关于动态规划，你该了解这些！中我们讲解了动态规划的基础知识。

首先讲一下动规和贪心的区别，其实大家不用太强调理论上的区别，做做题，就感受出来了。

然后我们讲了动规的五部曲：

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

后序我们在讲解动规的题目时候，都离不开这五步！

本周都是简单题目，大家可能会感觉 按照这五部来好麻烦，凭感觉随手一写，直接就过，越到后面越会感觉，凭感觉这个事还是不靠谱的，哈哈。

最后我们讲了动态规划题目应该如何debug，相信一些录友做动规的题目，一旦报错也是凭感觉来改。

其实只要把dp数组打印出来，哪里有问题一目了然！

如果代码写出来了，一直AC不了，灵魂三问：

1. 这道题目我举例推导状态转移公式了么？
2. 我打印dp数组的日志了么？
3. 打印出来了dp数组和我想的一样么？

哈哈，专治各种代码写出来了但AC不了的疑难杂症。

周二

这道题目动态规划：斐波那契数是当之无愧的动规入门题。

简单题，我们就是用来了解方法论的，用动规五部曲走一遍，题目其实已经把递推公式，和dp数组如何初始化都给我们了。

周三

动态规划：爬楼梯 这道题目其实就是斐波那契数列。

但正常思考过程应该是推导完递推公式之后，发现这是斐波那契，而不是上来就知道这是斐波那契。

在这道题目的第三步，确认dp数组如何初始化，其实就可以看出来，对dp[i]定义理解的深度。

dp[0]其实就是一个无意义的存在，不用去初始化dp[0]。

有的题解是把dp[0]初始化为1，然后遍历的时候i从2开始遍历，这样是可以解题的，然后强行解释一波dp[0]应该等于1的含义。

一个严谨的思考过程，应该是初始化dp[1] = 1，dp[2] = 2，然后i从3开始遍历，代码如下：

```
dp[1] = 1;
dp[2] = 2;
for (int i = 3; i <= n; i++) { // 注意i是从3开始的
    dp[i] = dp[i - 1] + dp[i - 2];
}
```

这个可以是面试的一个小问题，哈哈，考察候选人对dp[i]定义的理解程度。

这道题目还可以继续深化，就是一步一个台阶，两个台阶，三个台阶，直到 m个台阶，有多少种方法爬到n阶楼顶。

这又有难度了，这其实是一个完全背包问题，但力扣上没有这种题目，所以后续我在讲解背包问题的时候，今天这道题还会拿从背包问题的角度上来再讲一遍。

这里我先给出我的实现代码：

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) { // 把m换成2，就可以AC爬楼梯这道题
                if (i - j >= 0) dp[i] += dp[i - j];
            }
        }
        return dp[n];
    }
};
```

代码中m表示最多可以爬m个台阶。

以上代码不能运行哈，我主要是为了体现只要把m换成2，粘过去，就可以AC爬楼梯这道题，不信你就粘一下试试，哈哈。

此时我就发现一个绝佳的大厂面试题，第一道题就是单纯的爬楼梯，然后看候选人的代码实现，如果把dp[0]的定义成1了，就可以发难了，为什么dp[0]一定要初始化为1，此时可能候选人就要强行给dp[0]应该是1找各种理由。那这就是一个考察点了，对dp[i]的定义理解的不深入。

然后可以继续发难，如果一步一个台阶，两个台阶，三个台阶，直到 m个台阶，有多少种方法爬到n阶楼顶。这道题目leetcode上并没有原题，绝对是考察候选人算法能力的绝佳好题。

这一连套问下来，候选人算法能力如何，面试官心里就有数了。

其实大厂面试最喜欢问题的就是这种简单题，然后慢慢变化，在小细节上考察候选人。

这道绝佳的面试题我没有用过，如果录友们有面试别人的需求，就把这个套路拿去吧，哈哈哈。

我在通过一道面试题目，讲一讲递归算法的时间复杂度！中，以我自己面试别人的真实经历，通过求x的n次方 这么简单的题目，就可以考察候选人对算法性能以及递归的理解深度，录友们可以看看，绝对有收获！

周四

这道题目动态规划：使用最小花费爬楼梯就是在爬台阶的基础上加了一个花费，

这道题描述也确实有点魔幻。

题目描述为：每当你爬上一个阶梯你都要花费对应的体力值，一旦支付了相应的体力值，你就可以选择向上爬一个阶梯或者爬两个阶梯。

示例1：

输入：cost = [10, 15, 20]
输出：15

从题目描述可以看出：要不是第一步不需要花费体力，要不就是第最后一步不需要花费体力，我个人理解：题意说的其实是第一步是要支付费用的！。因为是当你爬上一个台阶就要花费对应的体力值！

所以我定义的dp[i]意思是也是第一步是要花费体力的，最后一步不用花费体力了，因为已经支付了。

之后一些录友在留言区说 可以定义dp[i]为:第一步是不花费体力，最后一步是花费体力的。

所以代码也可以这么写：

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        vector<int> dp(cost.size() + 1);
        dp[0] = 0; // 默认第一步都是不花费体力的
        dp[1] = 0;
        for (int i = 2; i <= cost.size(); i++) {
            dp[i] = min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);
        }
        return dp[cost.size()];
    }
};
```

这么写看上去比较顺，但是就是感觉和题目描述的不太符。哈哈，也没有必要这么细扣题意了，大家只要知道，题目的意思反正就是要不是第一步不花费，要不是最后一步不花费，都可以。

总结

本周题目简单一些，也非常合适初学者来练练手。

下周开始上难度了哈，然后大下周就开始讲解背包问题，好戏还在后面，录友们跟上哈。

学算法，认准「代码随想录」就够了，Carl带你打怪升级！

6.不同路径

力扣题目链接

一个机器人位于一个 m x n 网格的左上角 （起始点在下图中标记为 “Start” ）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish” ）。

问总共有多少条不同的路径?

示例 1：

示例 1：

- 输入：m = 3, n = 7
- 输出：28

示例 2：

- 输入：m = 2, n = 3
- 输出：3

解释： 从左上角开始，总共有 3 条路径可以到达右下角。

1. 向右 -> 向右 -> 向下
2. 向右 -> 向下 -> 向右
3. 向下 -> 向右 -> 向右

示例 3：

- 输入：m = 7, n = 3
- 输出：28

示例 4：

- 输入：m = 3, n = 3
- 输出：6

提示：

- 1 <= m, n <= 100
- 题目数据保证答案小于等于 2 * 10^9

算法公开课

[《代码随想录》算法视频公开课](#)：[动态规划中如何初始化很重要！| LeetCode：62.不同路径](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

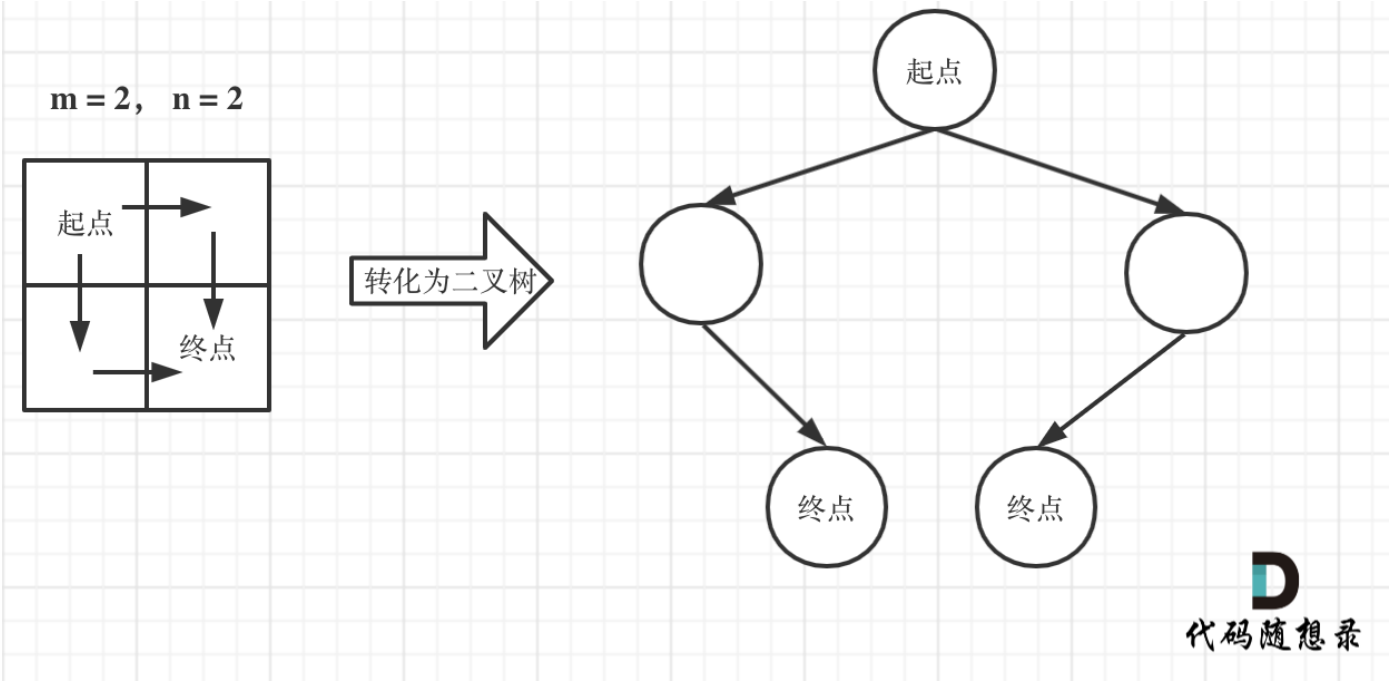
思路

深搜

这道题目，刚一看最直观的想法就是用图论里的深搜，来枚举出来有多少种路径。

注意题目中说机器人每次只能向下或者向右移动一步，那么其实**机器人走过的路径可以抽象为一棵二叉树，而叶子节点就是终点！**

如图举例：



此时问题就可以转化为求二叉树叶子节点的个数，代码如下：

```
class Solution {
private:
    int dfs(int i, int j, int m, int n) {
        if (i > m || j > n) return 0; // 越界了
        if (i == m && j == n) return 1; // 找到一种方法，相当于找到了叶子节点
        return dfs(i + 1, j, m, n) + dfs(i, j + 1, m, n);
    }
public:
    int uniquePaths(int m, int n) {
        return dfs(1, 1, m, n);
    }
};
```

大家如果提交了代码就会发现超时了！

来分析一下时间复杂度，这个深搜的算法，其实就是要遍历整个二叉树。

这棵树的深度其实就是m+n-1（深度按从1开始计算）。

那二叉树的节点个数就是 2^(m + n - 1) - 1。可以理解深搜的算法就是遍历了整个满二叉树（其实没有遍历整个满二叉树，只是近似而已）

所以上面深搜代码的时间复杂度为O(2^(m + n - 1) - 1)，可以看出，这是指数级别的时间复杂度，是非常大的。

动态规划

机器人从(0 , 0) 位置出发，到(m - 1, n - 1)终点。

按照动规五部曲来分析：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j] ： 表示从（0 ，0）出发，到(i, j) 有dp[i][j]条不同的路径。

2. 确定递推公式

想要求dp[i][j]，只能有两个方向来推导出来，即dp[i - 1][j] 和 dp[i][j - 1]。

此时在回顾一下 dp[i - 1][j] 表示啥，是从(0, 0)的位置到(i - 1, j)有几条路径，dp[i][j - 1]同理。

那么很自然，dp[i][j] = dp[i - 1][j] + dp[i][j - 1]，因为dp[i][j]只有这两个方向过来。

3. dp数组的初始化

如何初始化呢，首先dp[i][0]一定都是1，因为从(0, 0)的位置到(i, 0)的路径只有一条，那么dp[0][j]也同理。

所以初始化代码为：

```
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;
```

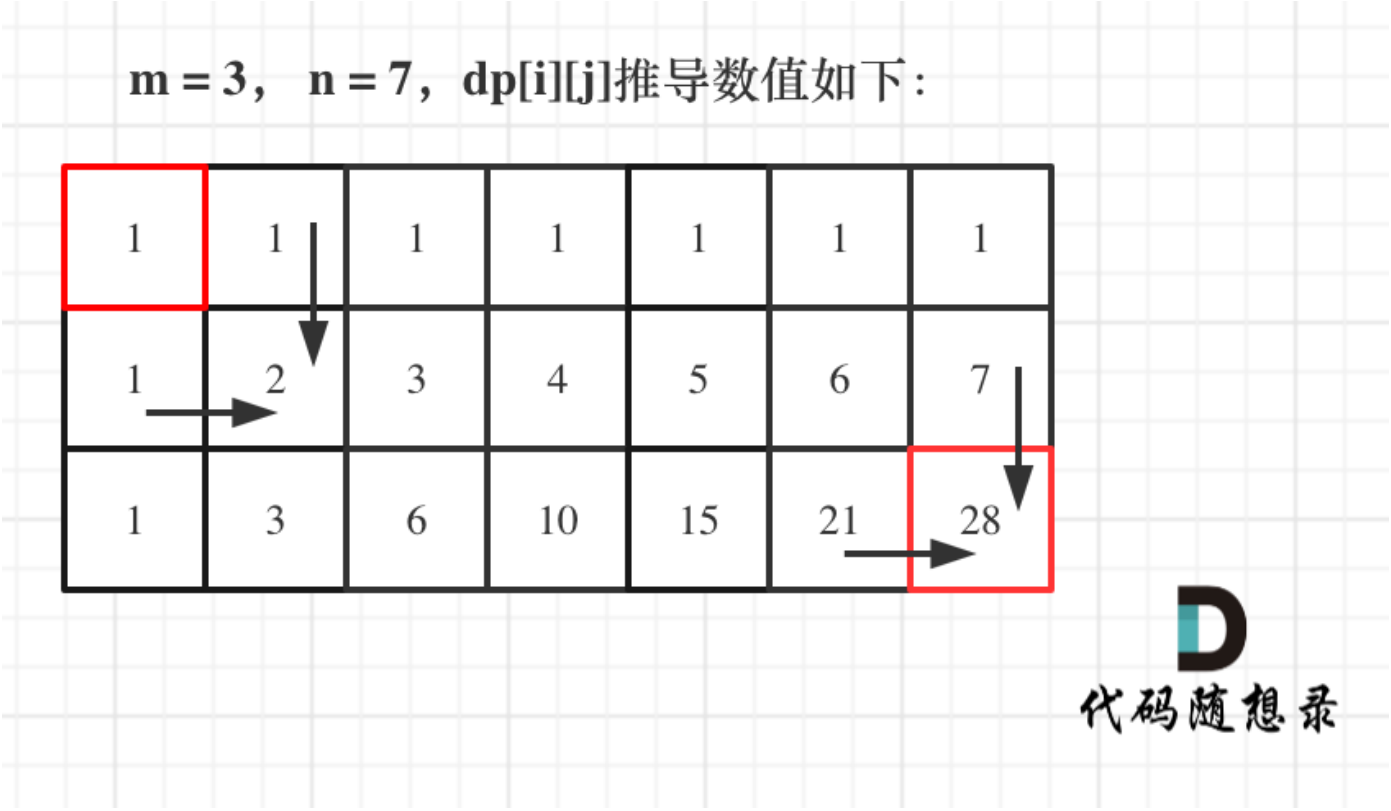
4. 确定遍历顺序

这里要看一下递推公式dp[i][j] = dp[i - 1][j] + dp[i][j - 1]，dp[i][j]都是从其上方和左方推导而来，那么从左到右一层一层遍历就可以了。

这样就可以保证推导dp[i][j]的时候，dp[i - 1][j] 和 dp[i][j - 1]一定是有数值的。

5. 举例推导dp数组

如图所示：



以上动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, 0));
        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
        return dp[m - 1][n - 1];
    }
};
```

- 时间复杂度：O(m × n)
- 空间复杂度：O(m × n)

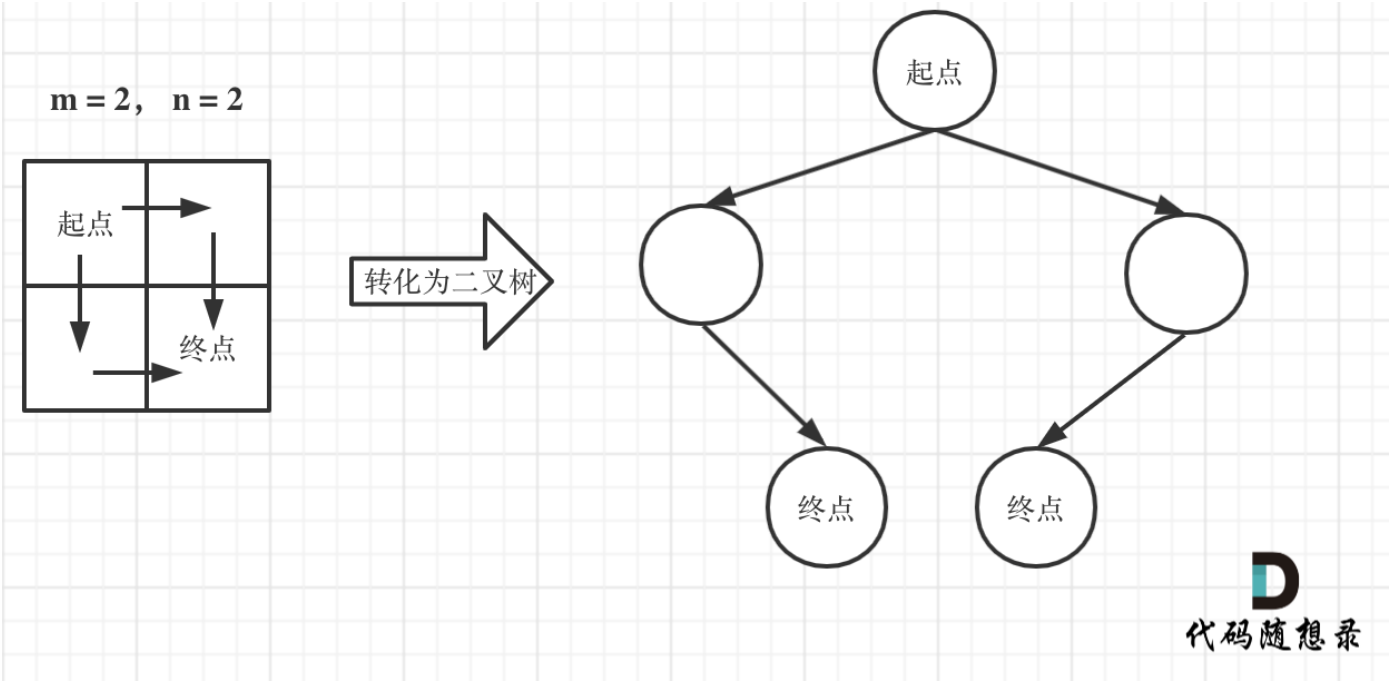
其实用一个一维数组（也可以理解是滚动数组）就可以了，但是不利于理解，可以优化点空间，建议先理解了二维，在理解一维，C++代码如下：

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n);
        for (int i = 0; i < n; i++) dp[i] = 1;
        for (int j = 1; j < m; j++) {
            for (int i = 1; i < n; i++) {
                dp[i] += dp[i - 1];
            }
        }
        return dp[n - 1];
    }
};
```

- 时间复杂度：O(m × n)
- 空间复杂度：O(n)

数论方法

在这个图中，可以看出一共m，n的话，无论怎么走，走到终点都需要 m + n - 2 步。

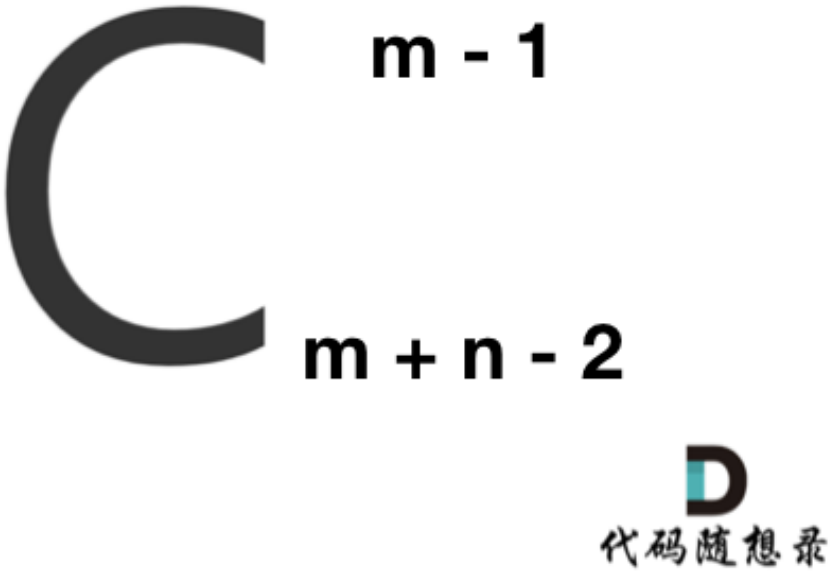


在这m + n - 2 步中，一定有 m - 1 步是要向下走的，不用管什么时候向下走。

那么有几种走法呢？ 可以转化为，给你m + n - 2个不同的数，随便取m - 1个数，有几种取法。

那么这就是一个组合问题了。

那么答案，如图所示：



求组合的时候，要防止两个int相乘溢出！ 所以不能把算式的分子都算出来，分母都算出来再做除法。

例如如下代码是不行的。

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        int numerator = 1, denominator = 1;
        int count = m - 1;
        int t = m + n - 2;
        while (count--) numerator *= (t--); // 计算分子，此时分子就会溢出
        for (int i = 1; i <= m - 1; i++) denominator *= i; // 计算分母
        return numerator / denominator;
    }
};
```

需要在计算分子的时候，不断除以分母，代码如下：

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        long long numerator = 1; // 分子
        int denominator = m - 1; // 分母
        int count = m - 1;
        int t = m + n - 2;
        while (count--) {
            numerator *= (t--);
            while (denominator != 0 && numerator % denominator == 0) {
```

```
                numerator /= denominator;
                denominator--;
            }
        }
        return numerator;
    }
};
```

- 时间复杂度：O(m)
- 空间复杂度：O(1)

计算组合问题的代码还是有难度的，特别是处理溢出的情况！

总结

本文分别给出了深搜，动规，数论三种方法。

深搜当然是超时了，顺便分析了一下使用深搜的时间复杂度，就可以看出为什么超时了。

然后在给出动规的方法，依然是使用动规五部曲，这次我们就要考虑如何正确的初始化了，初始化和遍历顺序其实也很重要！

---

7. 不同路径 II

力扣题目链接

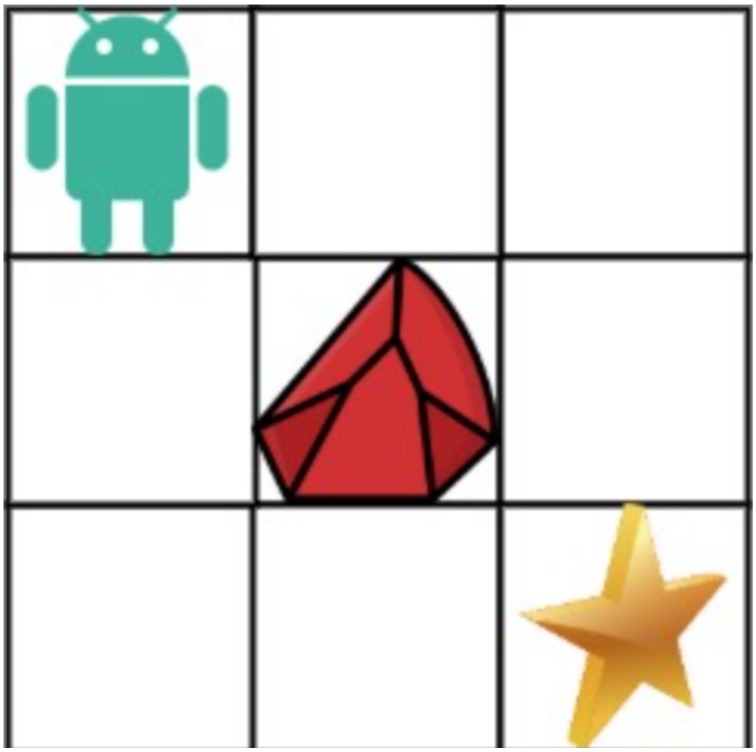
一个机器人位于一个 m x n 网格的左上角 （起始点在下图中标记为“Start” ）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。

现在考虑网格中有障碍物。那么从左上角到右下角将会有多少条不同的路径?

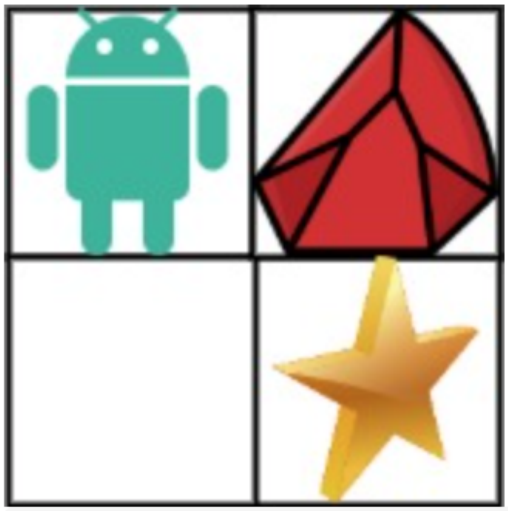
网格中的障碍物和空位置分别用 1 和 0 来表示。

示例 1：



- 输入：obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
- 输出：2
  解释：
- 3x3 网格的正中间有一个障碍物。
- 从左上角到右下角一共有 2 条不同的路径：
  1. 向右 -> 向右 -> 向下 -> 向下
  2. 向下 -> 向下 -> 向右 -> 向右

示例 2：



- 输入：obstacleGrid = [[0,1],[0,0]]
- 输出：1

提示：

- m == obstacleGrid.length
- n == obstacleGrid[i].length
- 1 <= m, n <= 100
- obstacleGrid[i][j] 为 0 或 1

算法公开课

《代码随想录》算法视频公开课：动态规划，这次遇到障碍了 | LeetCode：63. 不同路径 II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题相对于62.不同路径 就是有了障碍。

第一次接触这种题目的同学可能会有点懵，这有障碍了，应该怎么算呢?

62.不同路径中我们已经详细分析了没有障碍的情况，有障碍的话，其实就是标记对应的dp table（dp数组）保持初始值(0)就可以了。

动规五部曲：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：表示从（0，0）出发，到(i, j) 有dp[i][j]条不同的路径。

2. 确定递推公式

递推公式和62.不同路径一样，dp[i][j] = dp[i - 1][j] + dp[i][j - 1]。

但这里需要注意一点，因为有了障碍，(i, j)如果就是障碍的话应该就保持初始状态（初始状态为0）。

所以代码为：

```
if (obstacleGrid[i][j] == 0) { // 当(i, j)没有障碍的时候，再推导dp[i][j]
    dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
}
```

3. dp数组如何初始化

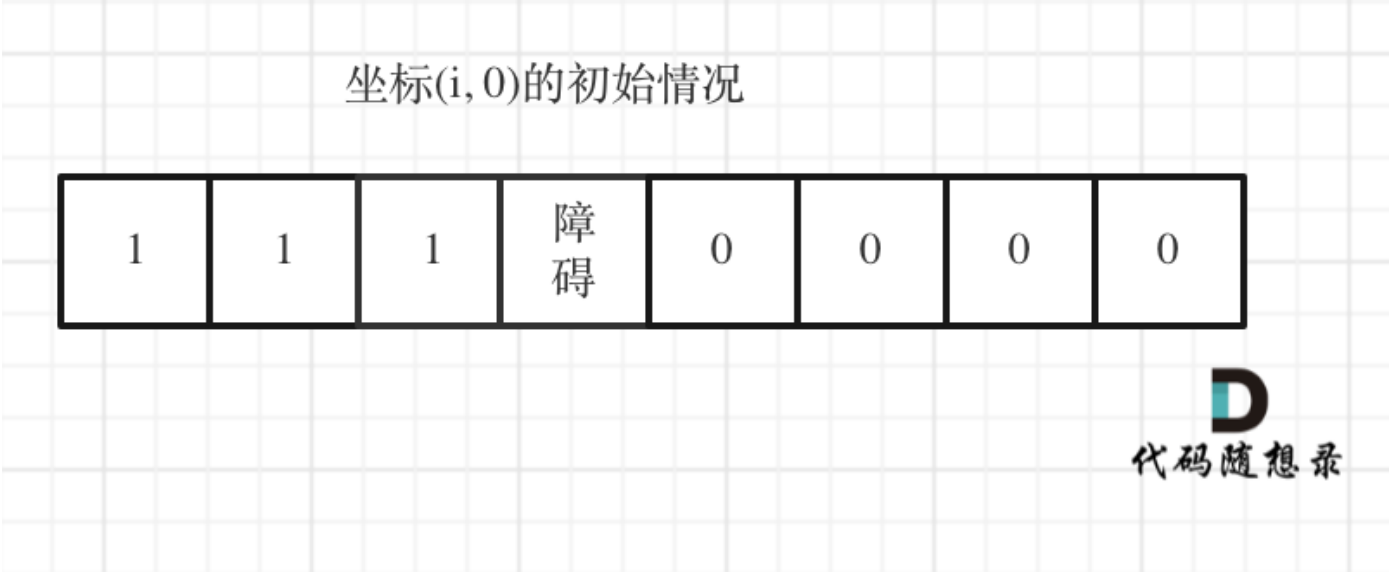
在62.不同路径不同路径中我们给出如下的初始化：

```
vector<vector<int>> dp(m, vector<int>(n, 0)); // 初始值为0
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;
```

因为从(0, 0)的位置到(i, 0)的路径只有一条，所以dp[i][0]一定为1，dp[0][j]也同理。

但如果(i, 0) 这条边有了障碍之后，障碍之后（包括障碍）都是走不到的位置了，所以障碍之后的dp[i][0]应该还是初始值0。

如图：



下标(0, j)的初始化情况同理。

所以本题初始化代码为：

```
vector<vector<int>> dp(m, vector<int>(n, 0));
for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1;
for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
```

注意代码里for循环的终止条件，一旦遇到obstacleGrid[i][0] == 1的情况就停止dp[i][0]的赋值1的操作，dp[0][j]同理

4. 确定遍历顺序

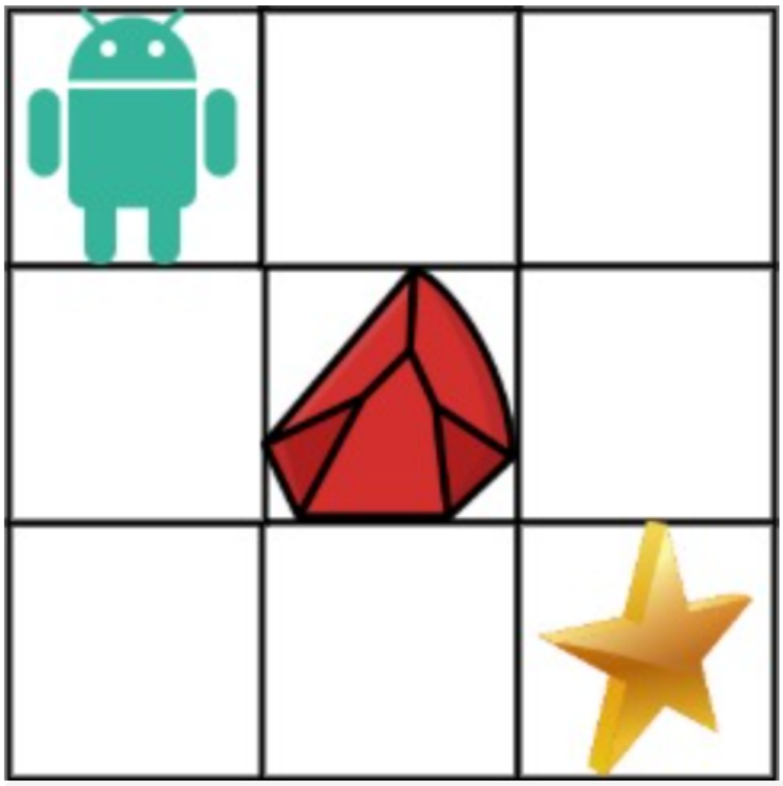
从递归公式dp[i][j] = dp[i - 1][j] + dp[i][j - 1] 中可以看出，一定是从左到右一层一层遍历，这样保证推导dp[i][j]的时候，dp[i - 1][j] 和 dp[i][j - 1]一定是有数值。

代码如下：

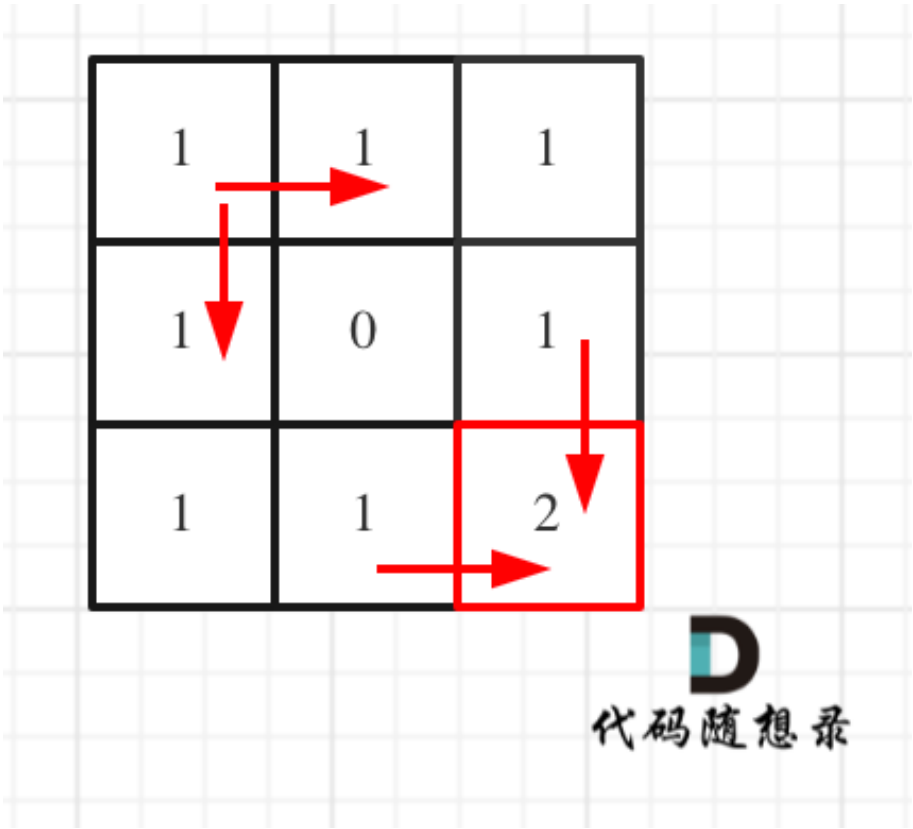
```
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        if (obstacleGrid[i][j] == 1) continue;
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
}
```

5. 举例推导dp数组

拿示例1来举例如题：



对应的dp table 如图：



如果这个图看不懂，建议再理解一下递归公式，然后照着文章中说的遍历顺序，自己推导一下！

动规五部分分析完毕，对应C++代码如下：

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
```

```
        int m = obstacleGrid.size();
        int n = obstacleGrid[0].size();
  if (obstacleGrid[m - 1][n - 1] == 1 || obstacleGrid[0][0] == 1) //如果在起点或终点出现了
障碍，直接返回0
            return 0;
        vector<vector<int>> dp(m, vector<int>(n, 0));
        for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1;
        for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (obstacleGrid[i][j] == 1) continue;
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
        return dp[m - 1][n - 1];
    }
};
```

- 时间复杂度：O(n × m)，n、m 分别为obstacleGrid 长度和宽度
- 空间复杂度：O(n × m)

同样我们给出空间优化版本：

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
        if (obstacleGrid[0][0] == 1)
            return 0;
        vector<int> dp(obstacleGrid[0].size());
        for (int j = 0; j < dp.size(); ++j)
            if (obstacleGrid[0][j] == 1)
                dp[j] = 0;
            else if (j == 0)
                dp[j] = 1;
            else
                dp[j] = dp[j-1];

        for (int i = 1; i < obstacleGrid.size(); ++i)
            for (int j = 0; j < dp.size(); ++j){
                if (obstacleGrid[i][j] == 1)
                    dp[j] = 0;
                else if (j != 0)
                    dp[j] = dp[j] + dp[j-1];
            }
        return dp.back();
    }
};
```

- 时间复杂度：O(n × m)，n、m 分别为obstacleGrid 长度和宽度

- 空间复杂度：O(m)

总结

本题是62.不同路径的障碍版，整体思路大体一致。

但就算是做过62.不同路径，在做本题也会有感觉遇到障碍无从下手。

其实只要考虑到，遇到障碍dp[i][j]保持0就可以了。

也有一些小细节，例如：初始化的部分，很容易忽略了障碍之后应该都是0的情况。

---

8. 整数拆分

力扣题目链接

给定一个正整数 n，将其拆分为至少两个正整数的和，并使这些整数的乘积最大化。 返回你可以获得的最大乘积。

示例 1:

- 输入: 2
- 输出: 1
- 解释: 2 = 1 + 1, 1 × 1 = 1。

示例 2:

- 输入: 10
- 输出: 36
- 解释: 10 = 3 + 3 + 4, 3 × 3 × 4 = 36。
- 说明: 你可以假设 n 不小于 2 且不大于 58。

算法公开课

《代码随想录》算法视频公开课：动态规划，本题关键在于理解递推公式！| LeetCode：343. 整数拆分，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

看到这道题目，都会想拆成两个呢，还是三个呢，还是四个....

我们来看一下如何使用动规来解决。

动态规划

动规五部曲，分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i]：分拆数字i，可以得到的最大乘积为dp[i]。

dp[i]的定义将贯彻整个解题过程，下面哪一步想不懂了，就想想dp[i]究竟表示的是啥！

2. 确定递推公式

可以想 dp[i]最大乘积是怎么得到的呢?

其实可以从1遍历j，然后有两种渠道得到dp[i].

一个是j * (i - j) 直接相乘。

一个是j * dp[i - j]，相当于是拆分(i - j)，对这个拆分不理解的话，可以回想dp数组的定义。

那有同学问了，j怎么就不拆分呢?

j是从1开始遍历，拆分j的情况，在遍历j的过程中其实都计算过了。那么从1遍历j，比较(i - j) * j和dp[i - j] * j 取最大的。递推公式：dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));

也可以这么理解，j * (i - j) 是单纯的把整数拆分为两个数相乘，而j * dp[i - j]是拆分成两个以及两个以上的个数相乘。

如果定义dp[i - j] * dp[j] 也是默认将一个数强制拆成4份以及4份以上了。

所以递推公式：dp[i] = max({dp[i], (i - j) * j, dp[i - j] * j});

那么在取最大值的时候，为什么还要比较dp[i]呢?

因为在递推公式推导的过程中，每次计算dp[i]，取最大的而已。

3. dp的初始化

不少同学应该疑惑，dp[0] dp[1]应该初始化多少呢?

有的题解里会给出dp[0] = 1，dp[1] = 1的初始化，但解释比较牵强，主要还是因为这么初始化可以把题目过了。

严格从dp[i]的定义来说，dp[0] dp[1] 就不应该初始化，也就是没有意义的数值。

拆分0和拆分1的最大乘积是多少?

这是无解的。

这里我只初始化dp[2] = 1，从dp[i]的定义来说，拆分数字2，得到的最大乘积是1，这个没有任何异议！

4. 确定遍历顺序

确定遍历顺序，先来看看递归公式：dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));

dp[i] 是依靠 dp[i - j]的状态，所以遍历i一定是从前向后遍历，先有dp[i - j]再有dp[i]。

所以遍历顺序为：

```
for (int i = 3; i <= n ; i++) {
    for (int j = 1; j < i - 1; j++) {
        dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));
    }
}
```

注意 枚举j的时候，是从1开始的。从0开始的话，那么让拆分一个数拆个0，求最大乘积就没有意义了。

j的结束条件是 j < i - 1 ，其实 j < i 也是可以的，不过可以节省一步，例如让j = i - 1，的话，其实在 j = 1的时候，这一步就已经拆出来了，重复计算，所以 j < i - 1

至于 i是从3开始，这样dp[i - j]就是dp[2]正好可以通过我们初始化的数值求出来。

更优化一步，可以这样：

```
for (int i = 3; i <= n ; i++) {
    for (int j = 1; j <= i / 2; j++) {
        dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));
    }
}
```

因为拆分一个数n 使之乘积最大，那么一定是拆分成m个近似相同的子数相乘才是最大的。

例如 6 拆成 3 * 3， 10 拆成 3 * 3 * 4。 100的话 也是拆成m个近似数组的子数 相乘才是最大的。

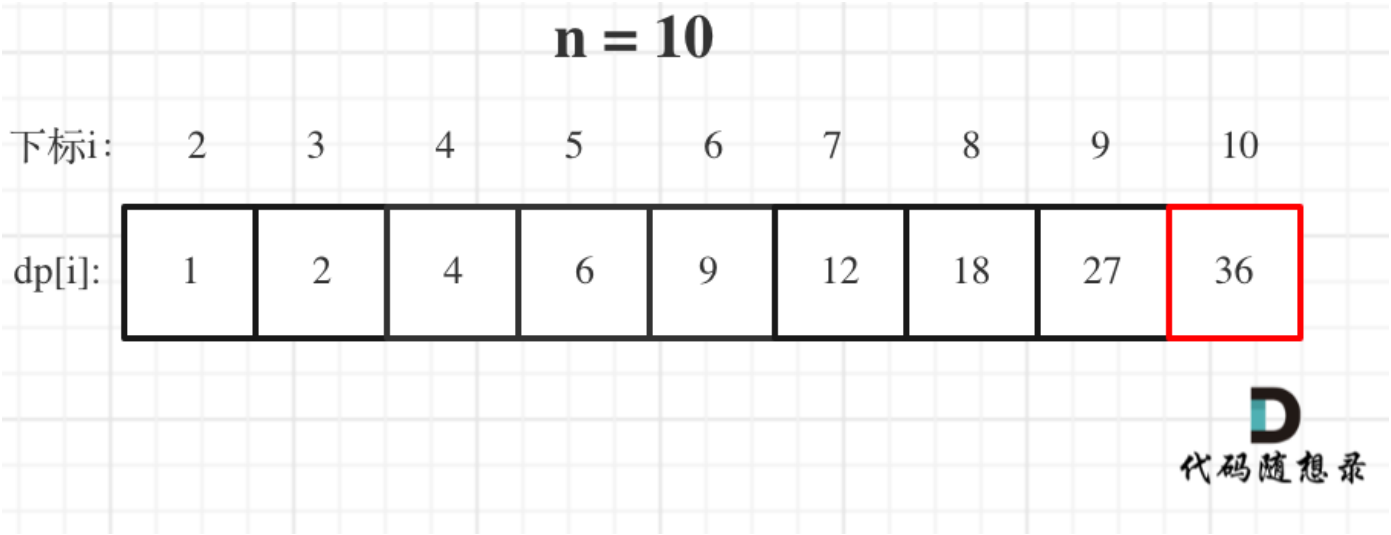
只不过我们不知道m究竟是多少而已，但可以明确的是m一定大于等于2，既然m大于等于2，也就是 最差也应该是拆成两个相同的 可能是最大值。

那么 j 遍历，只需要遍历到 n/2 就可以，后面就没有必要遍历了，一定不是最大值。

至于 “拆分一个数n 使之乘积最大，那么一定是拆分成m个近似相同的子数相乘才是最大的” 这个我就不去做数学证明了，感兴趣的同学，可以自己证明。

5. 举例推导dp数组

举例当n为10 的时候，dp数组里的数值，如下：



以上动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    int integerBreak(int n) {
        vector<int> dp(n + 1);
        dp[2] = 1;
        for (int i = 3; i <= n ; i++) {
            for (int j = 1; j <= i / 2; j++) {
                dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));
            }
        }
        return dp[n];
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(n)

贪心

本题也可以用贪心，每次拆成n个3，如果剩下是4，则保留4，然后相乘，**但是这个结论需要数学证明其合理性！**

我没有证明，而是直接用了结论。感兴趣的同学可以自己再去研究研究数学证明哈。

给出我的C++代码如下：

```
class Solution {
public:
    int integerBreak(int n) {
        if (n == 2) return 1;
        if (n == 3) return 2;
        if (n == 4) return 4;
        int result = 1;
        while (n > 4) {
            result *= 3;
            n -= 3;
        }
        result *= n;
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

总结

本题掌握其动规的方法，就可以了，贪心的解法确实简单，但需要有数学证明，如果能自圆其说也是可以的。

其实这道题目的递推公式并不好想，而且初始化的地方也很有讲究，我在写本题的时候一开始写的代码是这样的：

```
class Solution {
public:
    int integerBreak(int n) {
        if (n <= 3) return 1 * (n - 1);
        vector<int> dp(n + 1, 0);
        dp[1] = 1;
        dp[2] = 2;
        dp[3] = 3;
        for (int i = 4; i <= n ; i++) {
            for (int j = 1; j <= i / 2; j++) {
                dp[i] = max(dp[i], dp[i - j] * dp[j]);
            }
        }
        return dp[n];
    }
};
```

这个代码也是可以过的！

在解释递推公式的时候，也可以解释通，dp[i] 就等于 拆解i - j的最大乘积 * 拆解j的最大乘积。 看起来没毛病！

但是在解释初始化的时候，就发现自相矛盾了，dp[1]为什么一定是1呢？根据dp[i]的定义，dp[2]也不应该是2啊。

但如果递归公式是 dp[i] = max(dp[i], dp[i - j] * dp[j]);，就一定要这么初始化。递推公式没毛病，但初始化解释不通！

虽然代码在初始位置有一个判断if (n <= 3) return 1 * (n - 1);，保证n<=3 结果是正确的，但代码后面又要给dp[1]赋值1 和 dp[2] 赋值 2，**这其实就是自相矛盾的代码，违背了dp[i]的定义！**

我举这个例子，其实就说做题的严谨性，上面这个代码也可以AC，大体上一看好像也没有毛病，递推公式也说得过去，但是仅仅是恰巧过了而已。

9.不同的二叉搜索树

力扣题目链接

给定一个整数 n，求以 1 ... n 为节点组成的二叉搜索树有多少种?

示例:

输入： 3
输出： 5
解释：
给定 *n* = 3，一共有 5 种不同结构的二叉搜索树：

算法公开课

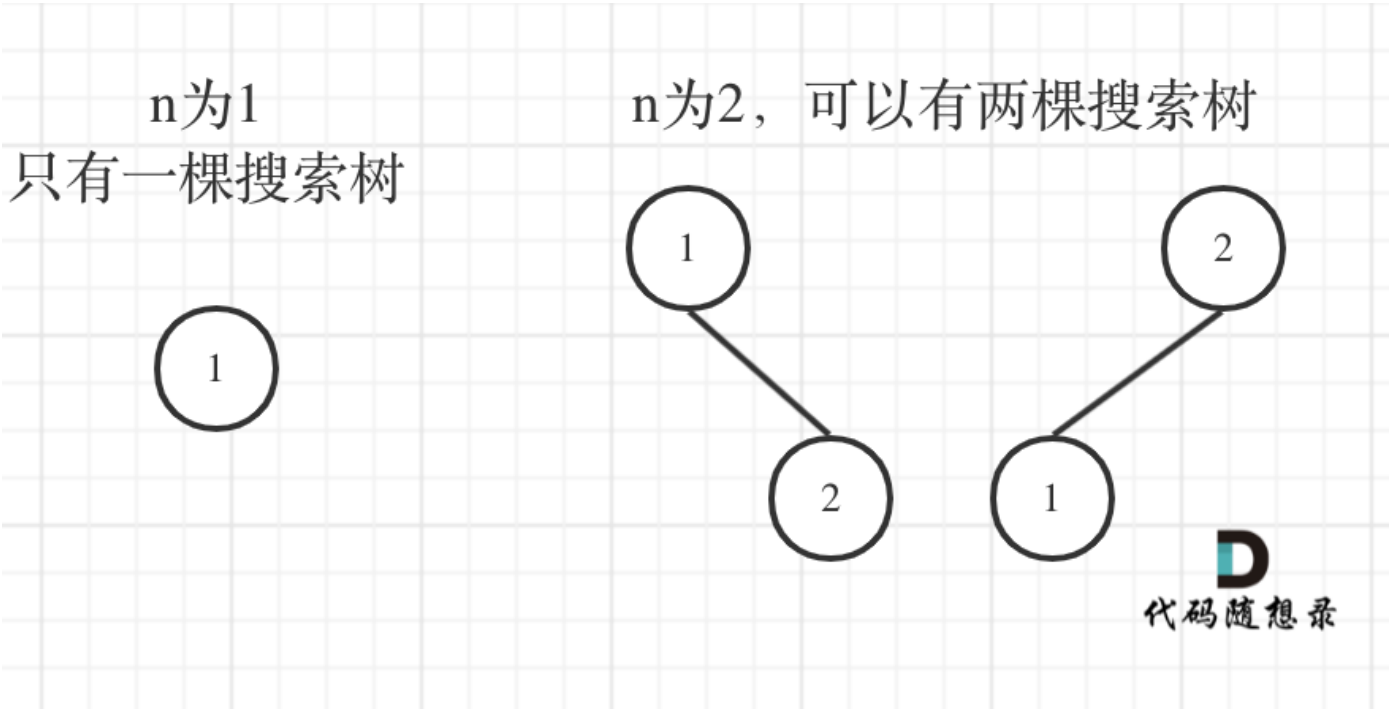
[《代码随想录》算法视频公开课](#)：[动态规划找到子状态之间的关系很重要！| LeetCode：96.不同的二叉搜索树](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

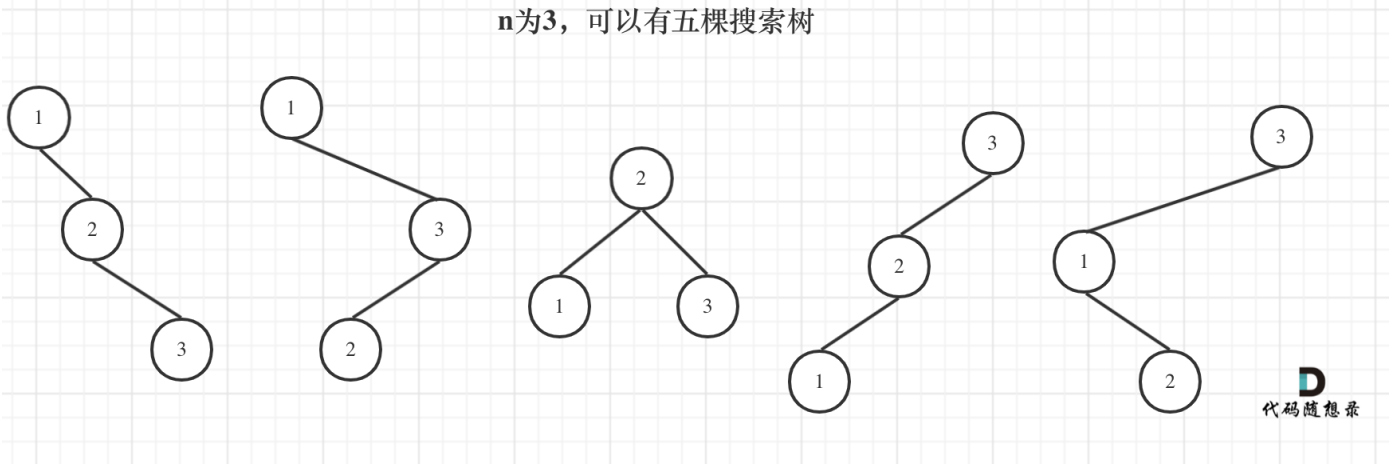
这道题目描述很简短，但估计大部分同学看完都是懵懵的状态，这得怎么统计呢?

关于什么是二叉搜索树，我们之前在讲解二叉树专题的时候已经详细讲解过了，也可以看看这篇[二叉树：二叉搜索树登场！](#)再回顾一波。

了解了二叉搜索树之后，我们应该先举几个例子，画画图，看看有没有什么规律，如图：



n为1的时候有一棵树，n为2有两棵树，这个是很直观的。



来看看n为3的时候，有哪几种情况。

当1为头结点的时候，其右子树有两个节点，看这两个节点的布局，是不是和 n 为2的时候两棵树的布局是一样的啊！

（可能有同学问了，这布局不一样啊，节点数值都不一样。别忘了我们就是求不同树的数量，并不用把搜索树都列出来，所以不用关心其具体数值的差异）

当3为头结点的时候，其左子树有两个节点，看这两个节点的布局，是不是和n为2的时候两棵树的布局也是一样的啊！

当2为头结点的时候，其左右子树都只有一个节点，布局是不是和n为1的时候只有一棵树的布局也是一样的啊！

发现到这里，其实我们就找到了重叠子问题了，其实也就是发现可以通过dp[1] 和 dp[2] 来推导出来dp[3]的某种方式。

思考到这里，这道题目就有眉目了。

dp[3]，就是 元素1为头结点搜索树的数量 + 元素2为头结点搜索树的数量 + 元素3为头结点搜索树的数量

元素1为头结点搜索树的数量 = 右子树有2个元素的搜索树数量 * 左子树有0个元素的搜索树数量

元素2为头结点搜索树的数量 = 右子树有1个元素的搜索树数量 * 左子树有1个元素的搜索树数量

元素3为头结点搜索树的数量 = 右子树有0个元素的搜索树数量 * 左子树有2个元素的搜索树数量

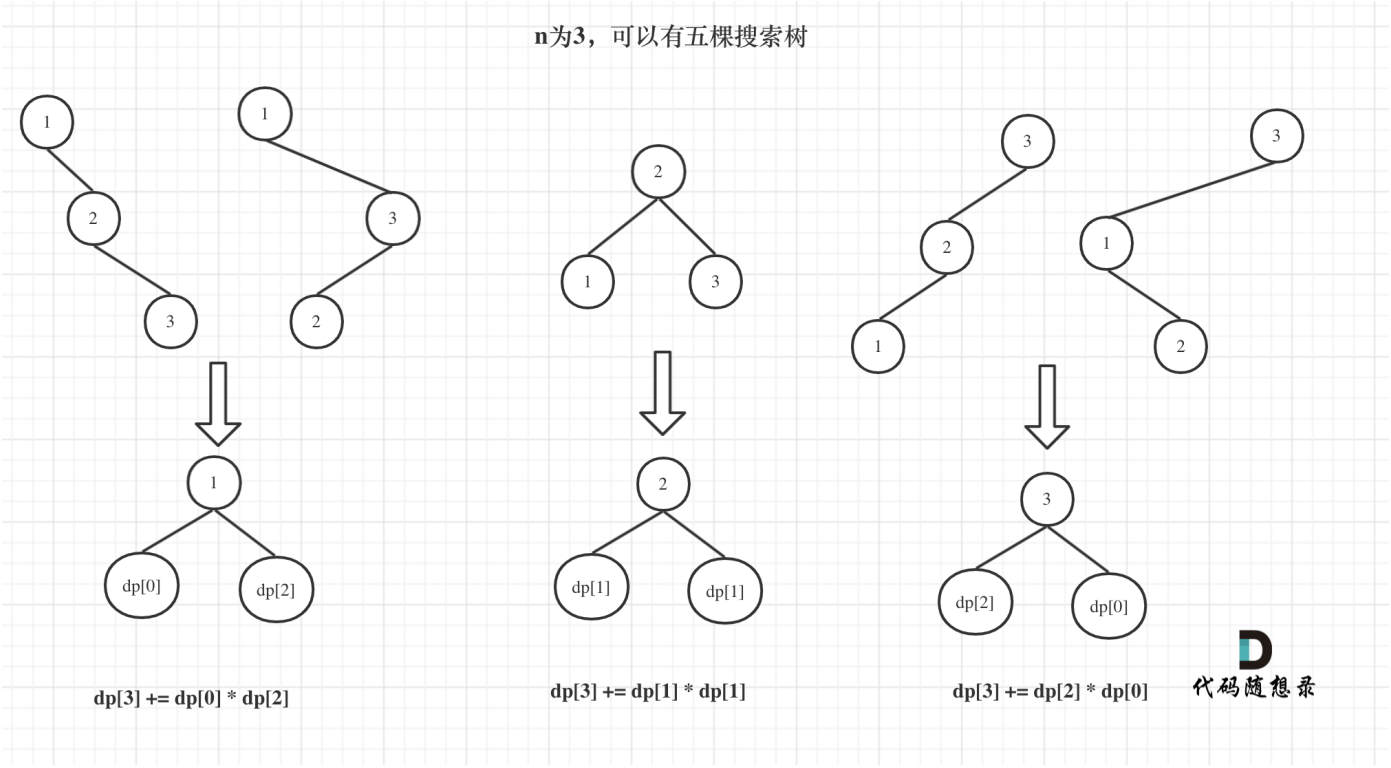
有2个元素的搜索树数量就是dp[2]。

有1个元素的搜索树数量就是dp[1]。

有0个元素的搜索树数量就是dp[0]。

所以dp[3] = dp[2] * dp[0] + dp[1] * dp[1] + dp[0] * dp[2]

如图所示：



此时我们已经找到递推关系了，那么可以用动规五部曲再系统分析一遍。

1. 确定dp数组（dp table）以及下标的含义

dp[i]： 1到i为节点组成的二叉搜索树的个数为dp[i]。

也可以理解是i个不同元素节点组成的二叉搜索树的个数为dp[i] ，都是一样的。

以下分析如果想不清楚，就来回想一下dp[i]的定义

2. 确定递推公式

在上面的分析中，其实已经看出其递推关系， dp[i] += dp[以j为头结点左子树节点数量] * dp[以j为头结点右子树节点数量]

j相当于是头结点的元素，从1遍历到i为止。

所以递推公式：dp[i] += dp[j - 1] * dp[i - j]; ，j-1 为j为头结点左子树节点数量，i-j 为以j为头结点右子树节点数量

3. dp数组如何初始化

初始化，只需要初始化dp[0]就可以了，推导的基础，都是dp[0]。

那么dp[0]应该是多少呢?

从定义上来讲，空节点也是一棵二叉树，也是一棵二叉搜索树，这是可以说得通的。

从递归公式上来讲，dp[以j为头结点左子树节点数量] * dp[以j为头结点右子树节点数量] 中以j为头结点左子树节点数量为0，也需要dp[以j为头结点左子树节点数量] = 1， 否则乘法的结果就都变成0了。

所以初始化dp[0] = 1

4. 确定遍历顺序

首先一定是遍历节点数，从递归公式：dp[i] += dp[j - 1] * dp[i - j]可以看出，节点数为i的状态是依靠 i之前节点数的状态。

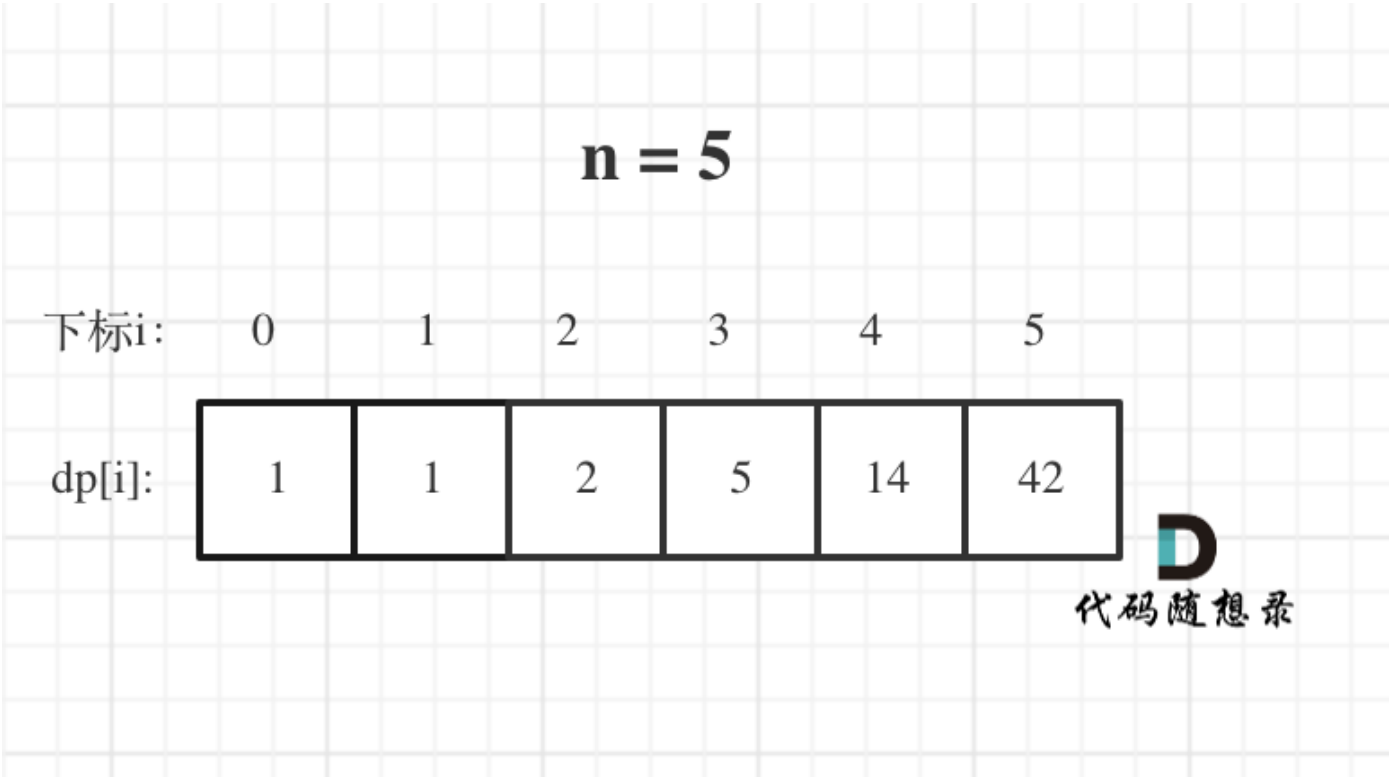
那么遍历i里面每一个数作为头结点的状态，用j来遍历。

代码如下：

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        dp[i] += dp[j - 1] * dp[i - j];
    }
}
```

5. 举例推导dp数组

n为5时候的dp数组状态如图：



当然如果自己画图举例的话，基本举例到n为3就可以了，n为4的时候，画图已经比较麻烦了。

我这里列到了n为5的情况，是为了方便大家 debug代码的时候，把dp数组打出来，看看哪里有问题。

综上分析完毕，C++代码如下：

```
class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1);
        dp[0] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                dp[i] += dp[j - 1] * dp[i - j];
            }
        }
        return dp[n];
    }
};
```

- 时间复杂度：$O(n^2)$
- 空间复杂度：$O(n)$

大家应该发现了，我们分析了这么多，最后代码却如此简单！

总结

这道题目虽然在力扣上标记是中等难度，但可以算是困难了！

首先这道题想到用动规的方法来解决，就不太好想，需要举例，画图，分析，才能找到递推的关系。

然后难点就是确定递推公式了，如果把递推公式想清楚了，遍历顺序和初始化，就是自然而然的事情了。

可以看出我依然还是用动规五部曲来进行分析，会把题目的方方面面都覆盖到！

而且具体这五部分析是我自己平时总结的经验，找不出来第二个的，可能过一阵子 其他题解也会有动规五部曲了，哈哈。

当时我在用动规五部曲讲解斐波那契的时候，一些录友和我反应，感觉讲复杂了。

其实当时我一直强调简单题是用来练习方法论的，并不能因为简单我就代码一甩，简单解释一下就完事了。

可能当时一些同学不理解，现在大家应该感受方法论的重要性了，加油💪

---

10. 本周小结！（动态规划系列二）

周一

动态规划：不同路径中求从出发点到终点有几种路径，只能向下或者向右移动一步。

我们提供了三种方法，但重点讲解的还是动规，也是需要重点掌握的。

dp[i][j]定义：表示从（0，0）出发，到(i, j) 有dp[i][j]条不同的路径。

本题在初始化的时候需要点思考了，即：

dp[i][0]一定都是1，因为从(0, 0)的位置到(i, 0)的路径只有一条，那么dp[0][j]也同理。

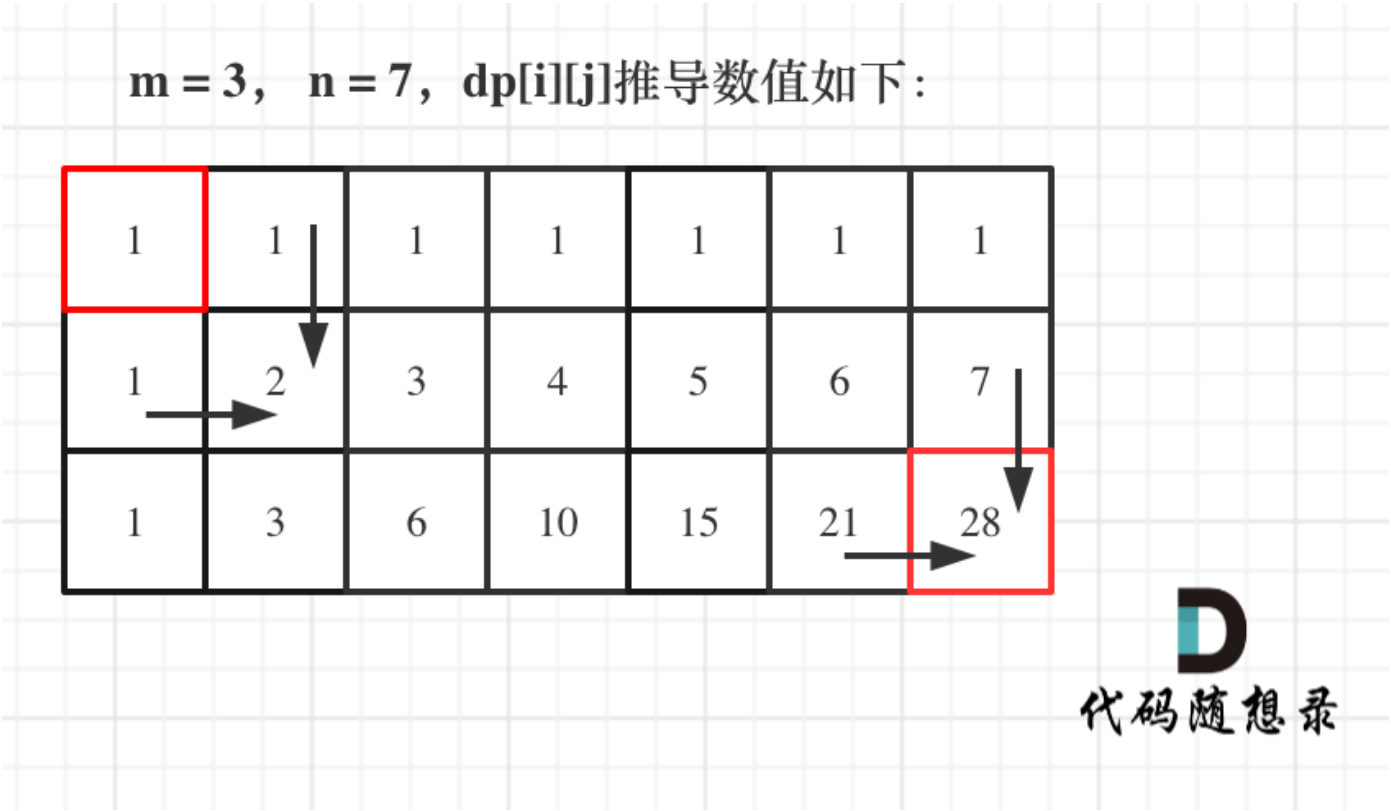
所以初始化为：

```
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;
```

这里已经不像之前做过的题目，随便赋个0就行的。

遍历顺序以及递推公式：

```
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
}
```



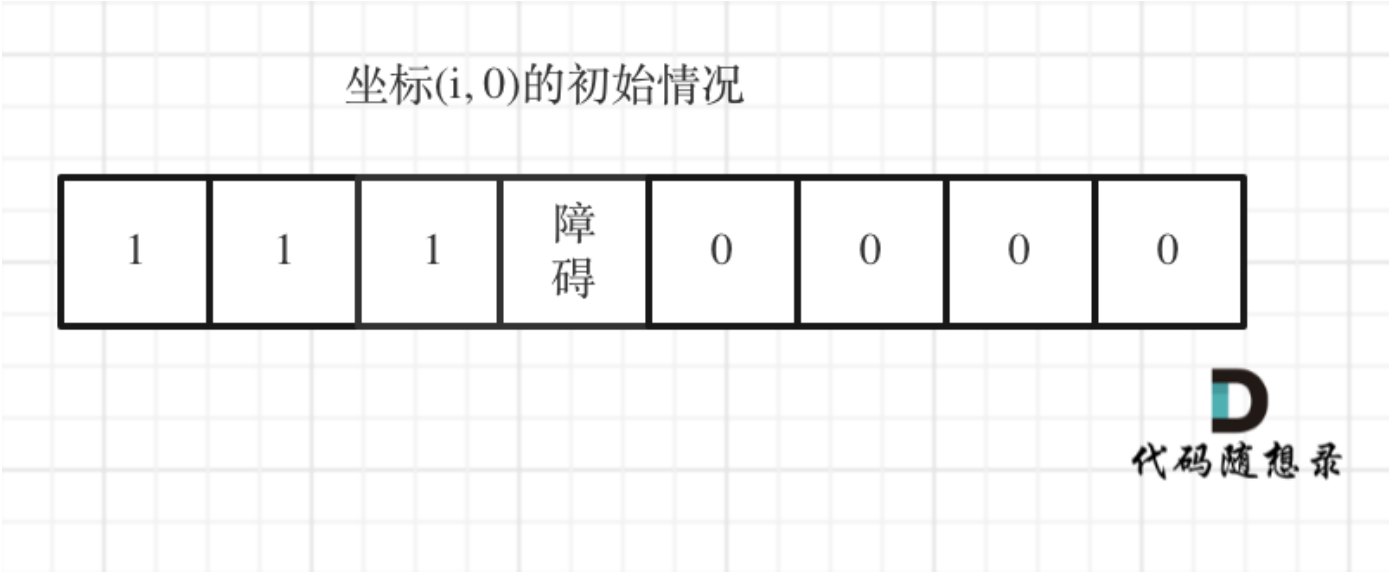
周二

动态规划：不同路径还不够，要有障碍！ 相对于动态规划：不同路径添加了障碍。

dp[i][j]定义依然是：表示从（0 ，0）出发，到(i, j) 有dp[i][j]条不同的路径。

本题难点在于初始化，如果(i, 0) 这条边有了障碍之后，障碍之后（包括障碍）都是走不到的位置了，所以障碍之后的dp[i][0]应该还是初始值0。

如图：



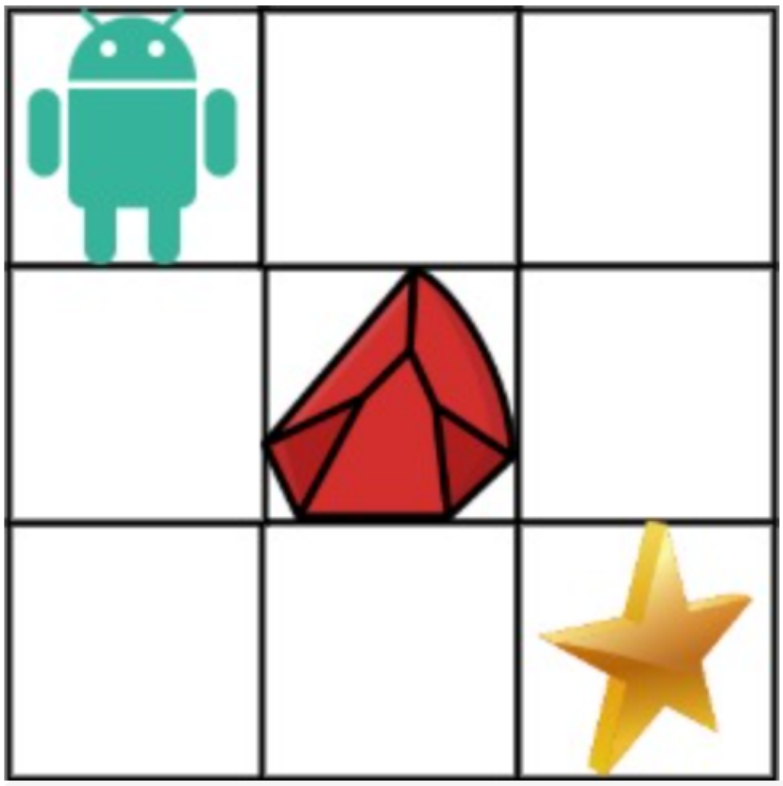
这里难住了不少同学，代码如下：

```
vector<vector<int>> dp(m, vector<int>(n, 0));
for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1;
for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
```

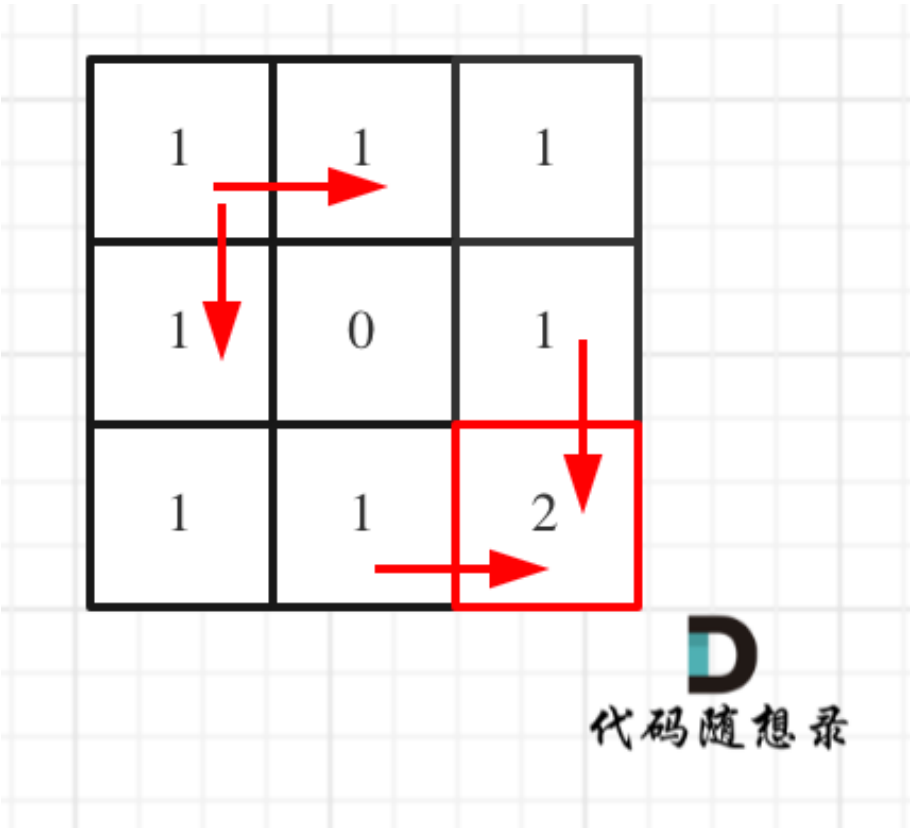
递推公式只要考虑一下障碍，就不赋值了就可以了，如下：

```
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        if (obstacleGrid[i][j] == 1) continue;
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
}
```

拿示例1来举例如题：



对应的dp table 如图：



周三

动态规划：整数拆分，你要怎么拆？给出一个整数，问有多少种拆分的方法。

这道题目就有点难度了，题目中dp我也给出了两种方法，但通过两种方法的比较可以看出，对dp数组定义的理解，以及dp数组初始化的重要性。

dp[i]定义：分拆数字i，可以得到的最大乘积为dp[i]。

本题中dp[i]的初始化其实也很有考究，严格从dp[i]的定义来说，dp[0] dp[1] 就不应该初始化，也就是没有意义的数值。

拆分0和拆分1的最大乘积是多少？

这是无解的。

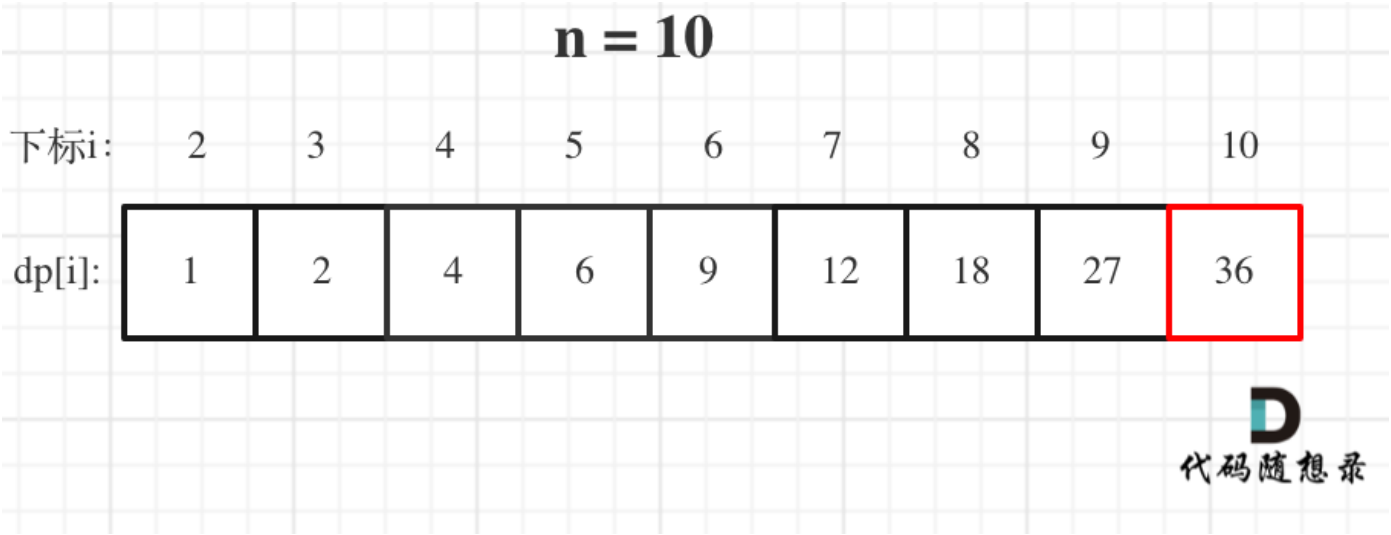
所以题解里我只初始化dp[2] = 1，从dp[i]的定义来说，拆分数字2，得到的最大乘积是1，这个没有任何异议！

```
vector<int> dp(n + 1);
dp[2] = 1;
```

遍历顺序以及递推公式：

```
for (int i = 3; i <= n ; i++) {
    for (int j = 1; j < i - 1; j++) {
        dp[i] = max(dp[i], max((i - j) * j, dp[i - j] * j));
    }
}
```

举例当n为10 的时候，dp数组里的数值，如下：



一些录友可能对为什么没有拆分j没有想清楚。

其实可以模拟一下哈，拆分j的情况，在遍历j的过程中dp[i - j]其实都计算过了。

例如 i= 10，j = 5，i-j = 5，如果把j拆分为 2 和 3，其实在j = 2 的时候，i-j= 8 ，拆分i-j的时候就可以拆出来一个3了。

或者也可以理解j是拆分i的第一个整数。

动态规划：整数拆分，你要怎么拆？总结里，我也给出了递推公式dp[i] = max(dp[i], dp[i - j] * dp[j])这种写法。

对于这种写法，一位录友总结的很好，意思就是：如果递推公式是dp[i-j] * dp[j]，这样就相当于强制把一个数至少拆分成四份。

dp[i-j]至少是两个数的乘积，dp[j]又至少是两个数的乘积，但其实3以下的数，数的本身比任何它的拆分乘积都要大了，所以文章中初始化的时候才要特殊处理。

周四

动态规划：不同的二叉搜索树给出n个不同的节点求能组成多少个不同二叉搜索树。

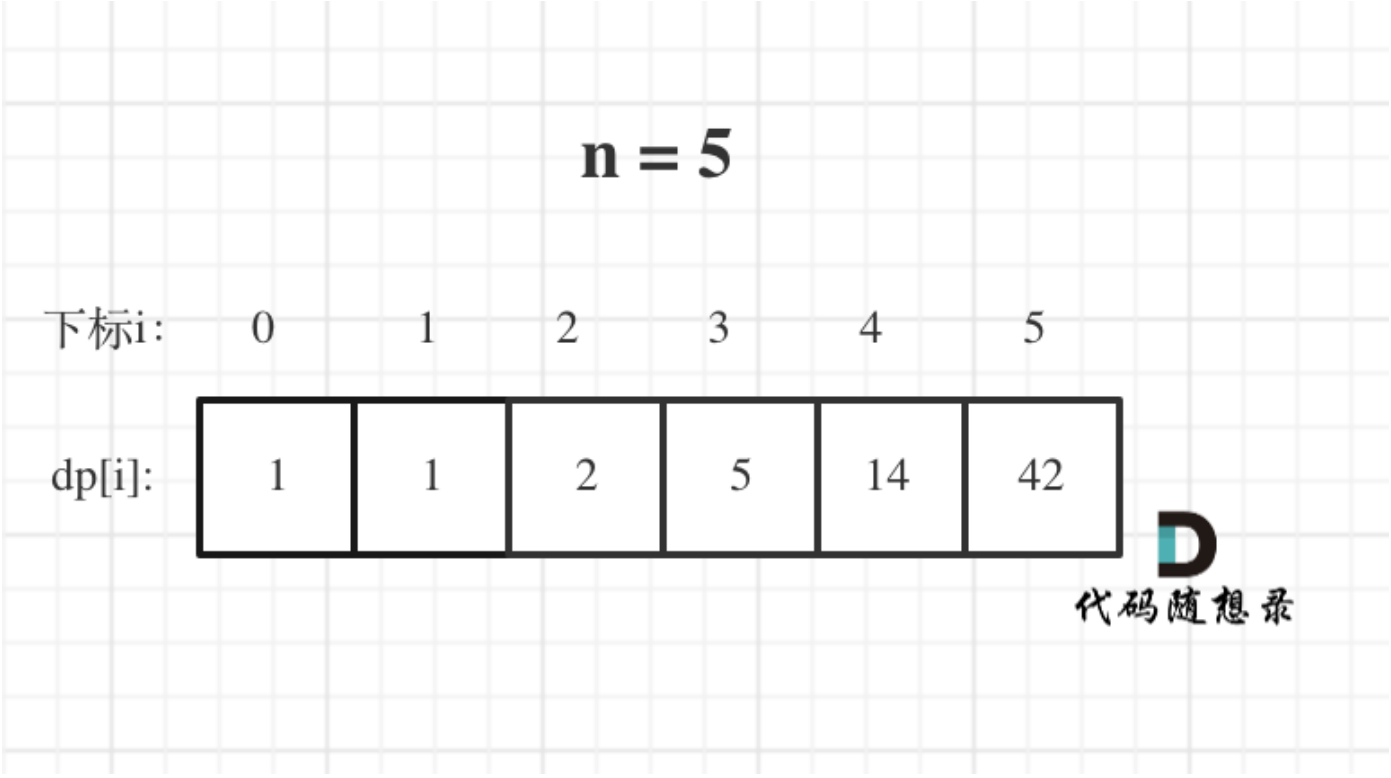
这道题目还是比较难的，想到用动态规划的方法就很不容易了！

dp[i]定义： 1到i为节点组成的二叉搜索树的个数为dp[i]。

递推公式：dp[i] += dp[j - 1] * dp[i - j]; ，j-1 为j为头结点左子树节点数量，i-j 为以j为头结点右子树节点数量

dp数组如何初始化：只需要初始化dp[0]就可以了，推导的基础，都是dp[0]。

n为5时候的dp数组状态如图：



总结

本周题目已经开始点难度了，特别是动态规划：不同的二叉搜索树这道题目，明显感觉阅读量很低，可能是因为确实有点难吧。

我现在也陷入了纠结，题目一简单，就会有录友和我反馈说题目太简单了，题目一难，阅读量就特别低。

我也好难那，哈哈哈。

但我还会坚持规划好的路线，难度循序渐进，并以面试经典题目为准，该简单的时候就是简单，同时也不会因为阅读量低就放弃有难度的题目！。

录友们看到这是不是得给个Carl点个赞啊[让我看看]。

预告，我们下周正式开始讲解背包问题，经典的不能再经典，也是比较难的一类动态规划的题目了，录友们上车抓稳咯。

11. 动态规划：01背包理论基础（一）

算法公开课

[《代码随想录》算法视频公开课](#)：[带你学透0-1背包问题！](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

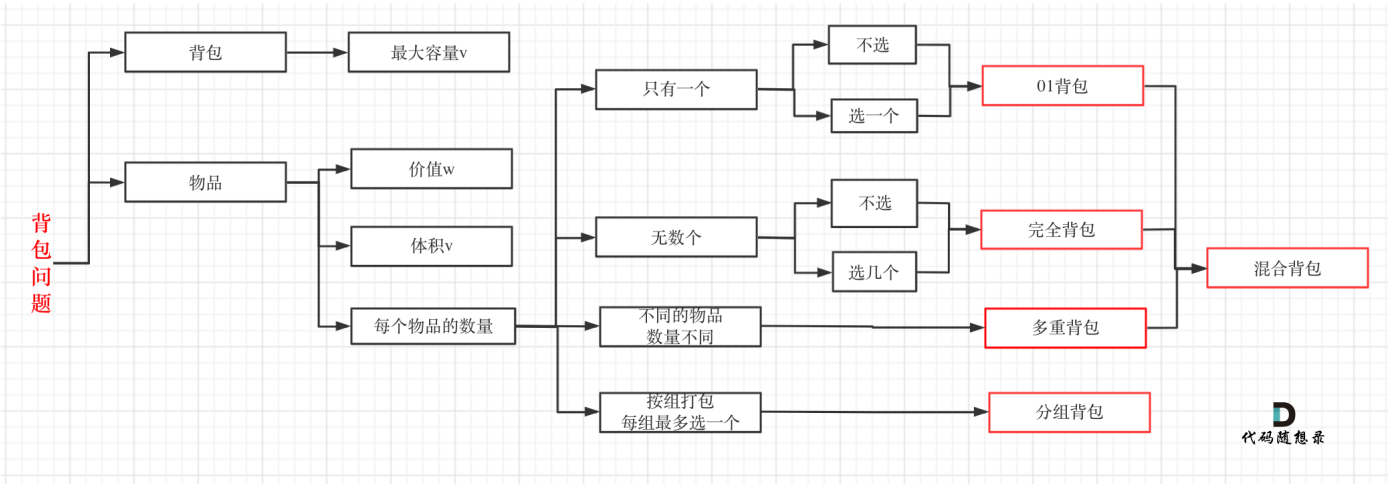
这周我们正式开始讲解背包问题！

背包问题的经典资料当然是：背包九讲。在公众号「代码随想录」后台回复：背包九讲，就可以获得背包九讲的pdf。

但说实话，背包九讲对于小白来说确实不太友好，看起来还是有点费劲的，而且都是伪代码理解起来也吃力。

对于面试的话，其实掌握01背包，和完全背包，就够用了，最多可以再来一个多重背包。

如果这几种背包，分不清，我这里画了一个图，如下：



至于背包九讲其其他背包，面试几乎不会问，都是竞赛级别的了，leetcode上连多重背包的题目都没有，所以题库也告诉我们，01背包和完全背包就够用了。

而完全背包又是也是01背包稍作变化而来，即：完全背包的物品数量是无限的。

所以背包问题的理论基础重中之重是01背包，一定要理解透！

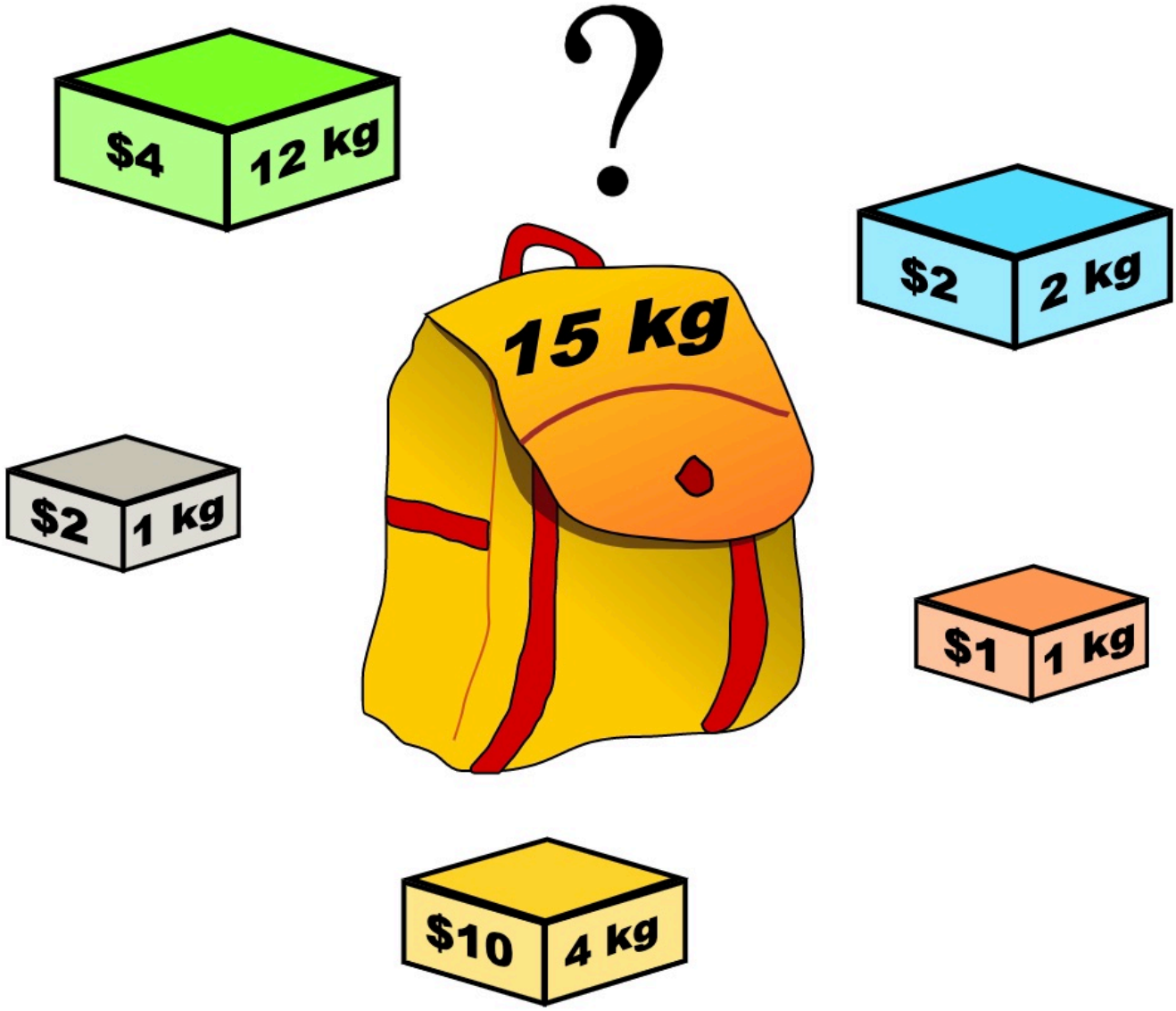
leetcode上没有纯01背包的问题，都是01背包应用方面的题目，也就是需要转化为01背包问题。

所以我先通过纯01背包问题，把01背包原理讲清楚，后续再讲解leetcode题目的时候，重点就是讲解如何转化为01背包问题了。

之前可能有些录友已经可以熟练写出背包了，但只要把这个文章仔细看完，相信你会意外收获！

01 背包

有n件物品和一个最多能背重量为w 的背包。第i件物品的重量是weight[i]，得到的价值是value[i] 。**每件物品只能用一次**，求解将哪些物品装入背包里物品价值总和最大。



这是标准的背包问题，以至于很多同学看了这个自然就会想到背包，甚至都不知道暴力的解法应该怎么解了。

这样其实是没有从底向上去思考，而是习惯性想到了背包，那么暴力的解法应该是怎么样的呢?

每一件物品其实只有两个状态，取或者不取，所以可以使用回溯法搜索出所有的情况，那么时间复杂度就是$o(2^n)$，这里的n表示物品数量。

所以暴力的解法是指数级别的时间复杂度。进而才需要动态规划的解法来进行优化!

在下面的讲解中，我举一个例子:

背包最大重量为4。

物品为:

	重量	价值
物品0	1	15
物品1	3	20
物品2	4	30

问背包能背的物品最大价值是多少?

以下讲解和图示中出现的数字都是以这个例子为例。

二维dp数组01背包

依然动规五部曲分析一波。

1. 确定dp数组以及下标的含义

对于背包问题，有一种写法， 是使用二维数组，即**dp[i][j] 表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。**

只看这个二维数组的定义，大家一定会有点懵，看下面这个图：

要时刻记着这个dp数组的含义，下面的一些步骤都围绕这dp数组的含义进行的，如果哪里看懵了，就来回顾一下i代表什么，j又代表什么。

2. 确定递推公式

再回顾一下dp[i][j]的含义：从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。

那么可以有两个方向推出来dp[i][j],

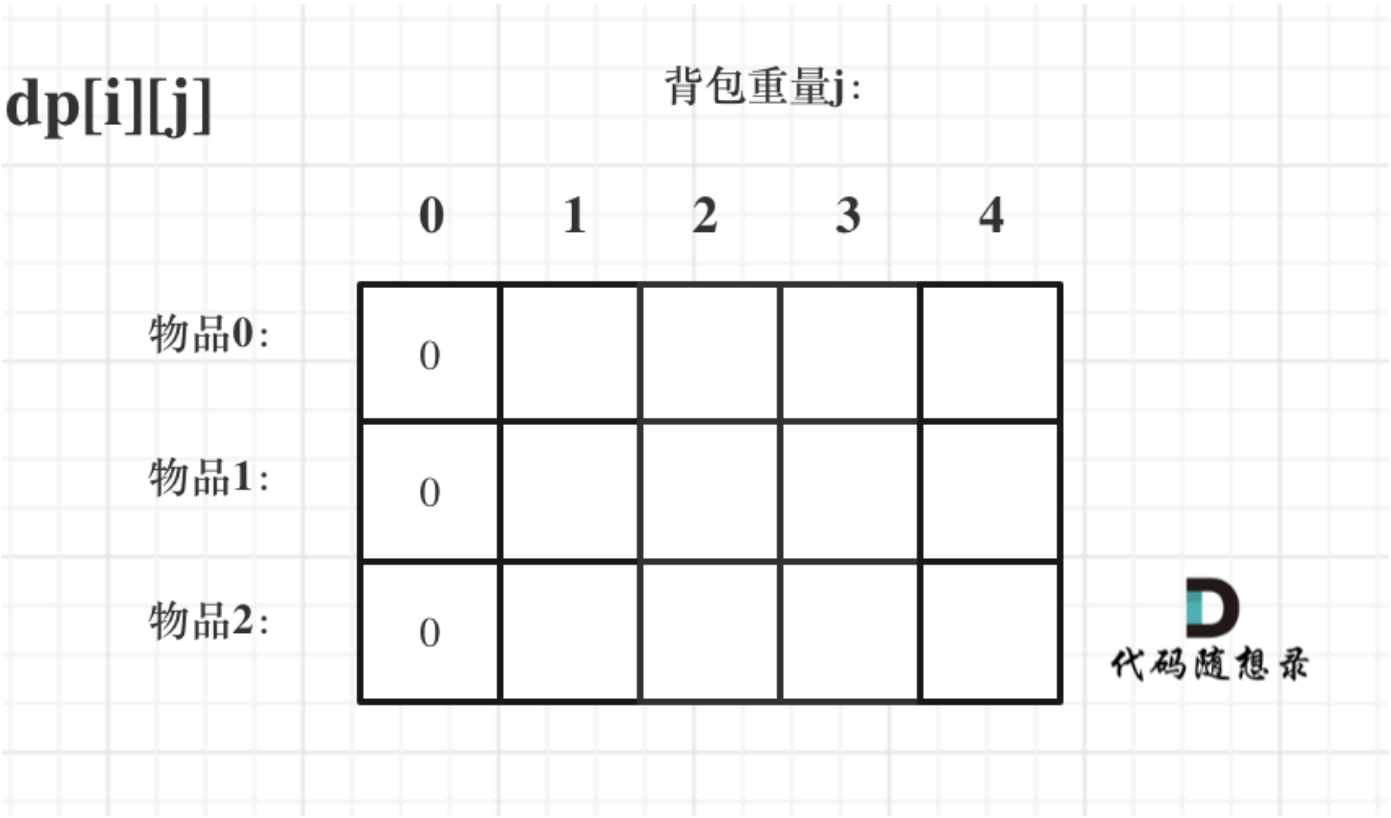
- **不放物品i**：由dp[i - 1][j]推出，即背包容量为j，里面不放物品i的最大价值，此时dp[i][j]就是dp[i - 1][j]。(其实就是当物品i的重量大于背包j的重量时，物品i无法放进背包中，所以背包内的价值依然和前面相同。)
- **放物品i**：由dp[i - 1][j - weight[i]]推出，dp[i - 1][j - weight[i]] 为背包容量为j - weight[i]的时候不放物品i的最大价值，那么dp[i - 1][j - weight[i]] + value[i] （物品i的价值），就是背包放物品i得到的最大价值

所以递归公式： dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

3. dp数组如何初始化

关于初始化，一定要和dp数组的定义吻合，否则到递推公式的时候就会越来越乱。

首先从dp[i][j]的定义出发，如果背包容量j为0的话，即dp[i][0]，无论是选取哪些物品，背包价值总和一定为0。如图：



在看其他情况。

状态转移方程 dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]); 可以看出i 是由 i-1 推导出来，那么i为0的时候就一定要初始化。

dp[0][j]，即：i为0，存放编号0的物品的时候，各个容量的背包所能存放的最大价值。

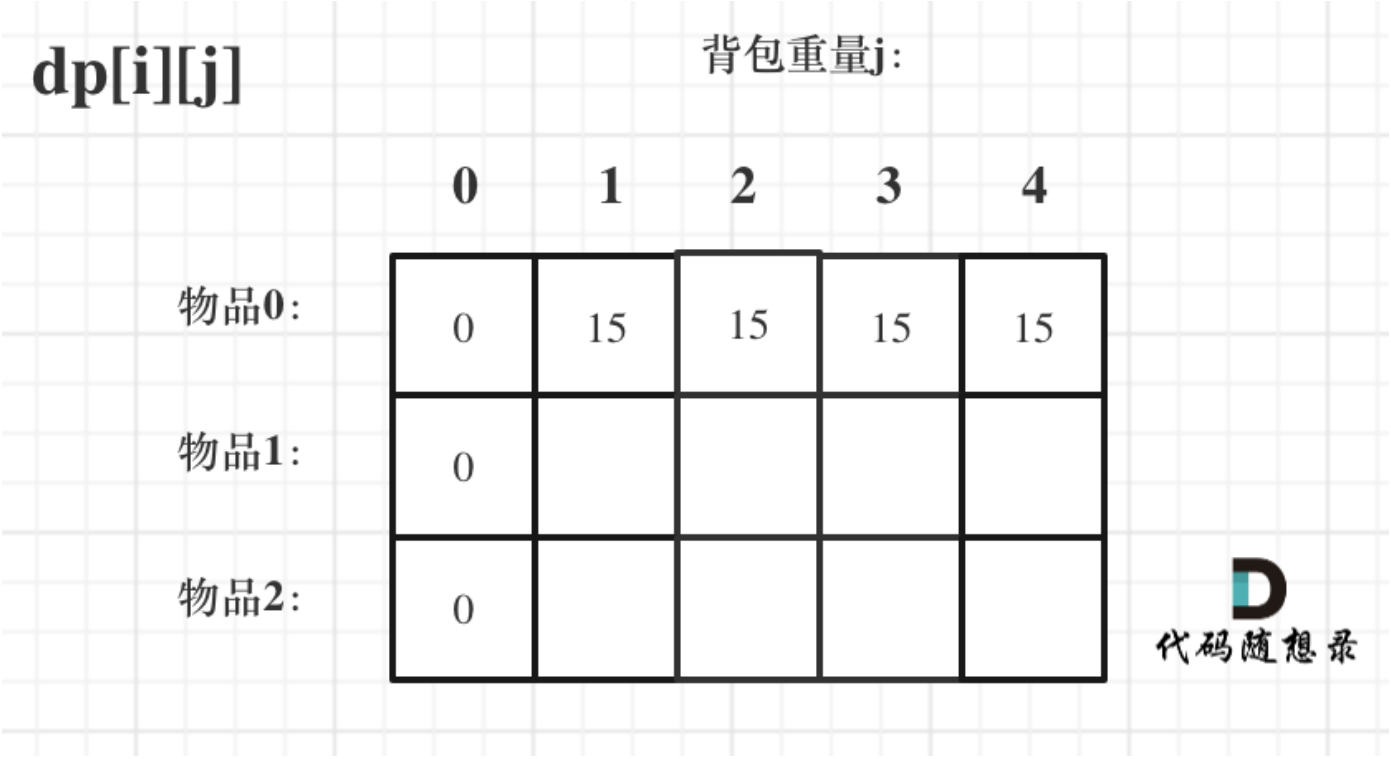
那么很明显当 j < weight[0]的时候，dp[0][j] 应该是 0，因为背包容量比编号0的物品重量还小。

当j >= weight[0]时，dp[0][j] 应该是value[0]，因为背包容量放足够放编号0物品。

代码初始化如下：

```
for (int j = 0 ; j < weight[0]; j++) {  // 当然这一步，如果把dp数组预先初始化为0了，这一步就可以省略，但很多同学应该没有想清楚这一点。
    dp[0][j] = 0;
}
// 正序遍历
for (int j = weight[0]; j <= bagweight; j++) {
    dp[0][j] = value[0];
}
```

此时dp数组初始化情况如图所示：



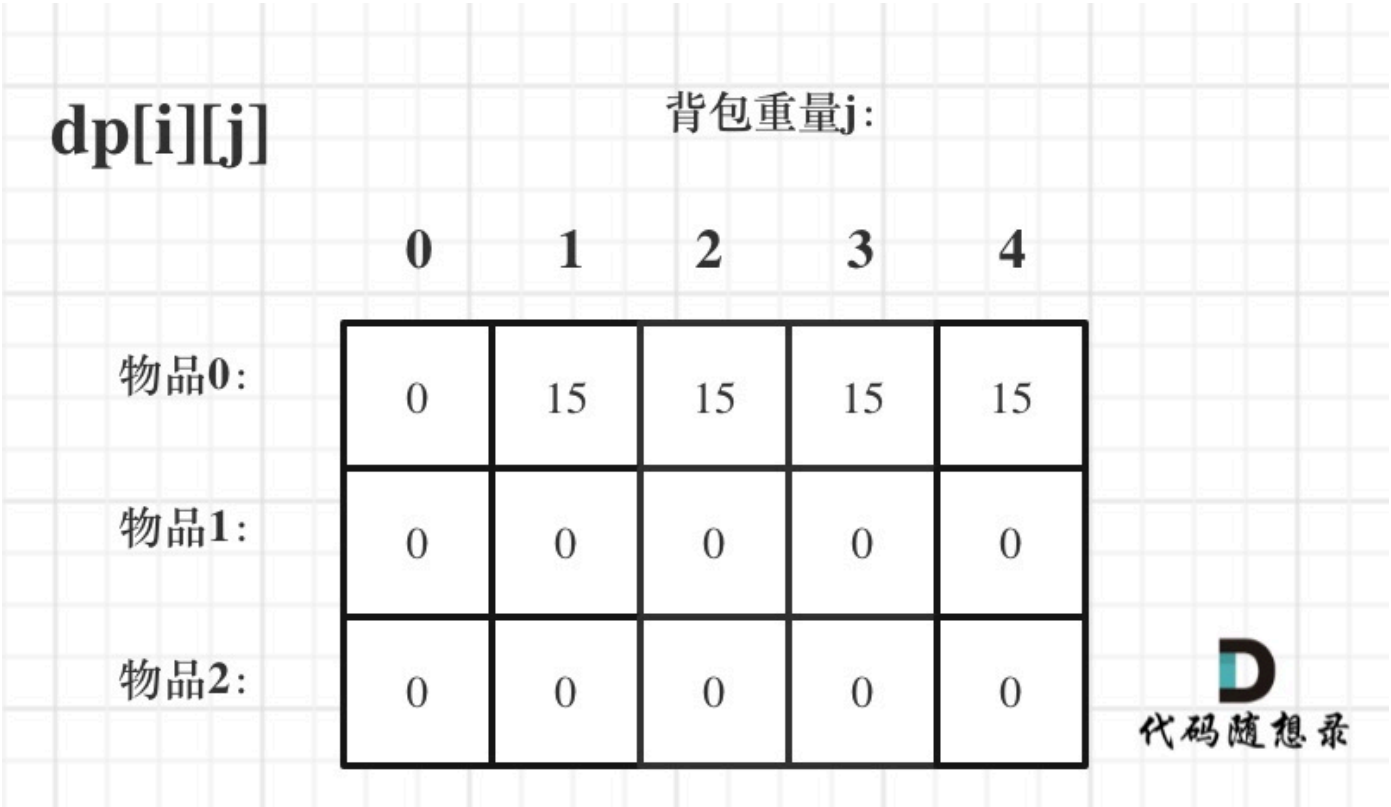
dp[0][j] 和 dp[i][0] 都已经初始化了，那么其他下标应该初始化多少呢?

其实从递归公式： dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]); 可以看出dp[i][j] 是由左上方数值推导出来了，那么 其他下标初始为什么数值都可以，因为都会被覆盖。

初始-1，初始-2，初始100，都可以!

但只不过一开始就统一把dp数组统一初始为0，更方便一些。

如图：



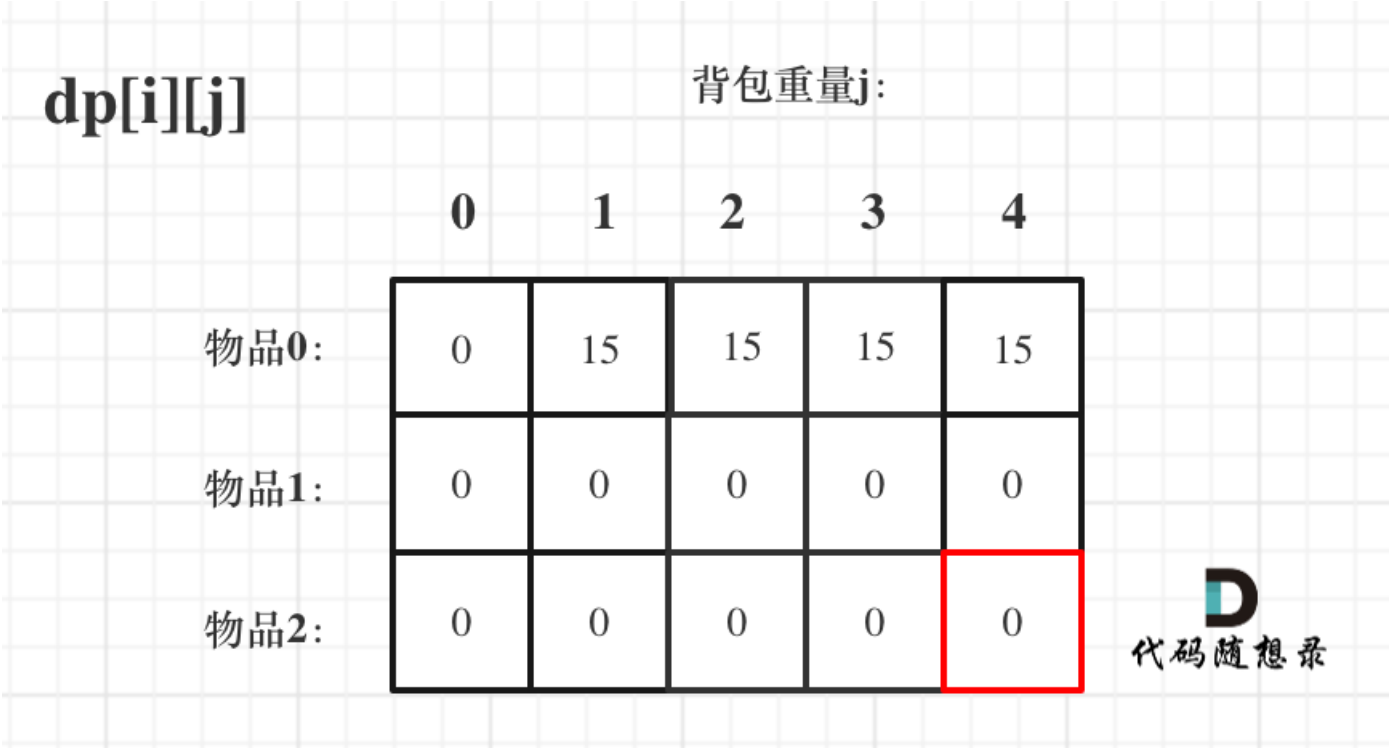
最后初始化代码如下：

```
// 初始化 dp
vector<vector<int>> dp(weight.size(), vector<int>(bagweight + 1, 0));
for (int j = weight[0]; j <= bagweight; j++) {
    dp[0][j] = value[0];
}
```

费了这么大的功夫，才把如何初始化讲清楚，相信不少同学平时初始化dp数组是凭感觉来的，但有时候感觉是不靠谱的。

4. 确定遍历顺序

在如下图中，可以看出，有两个遍历的维度：物品与背包重量



那么问题来了，**先遍历 物品还是先遍历背包重量呢?**

其实都可以！！ 但是先遍历物品更好理解。

那么我先给出先遍历物品，然后遍历背包重量的代码。

```
// weight数组的大小 就是物品个数
for(int i = 1; i < weight.size(); i++) { // 遍历物品
    for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

    }
}
```

先遍历背包，再遍历物品，也是可以的！（注意我这里使用的二维dp数组）

例如这样：

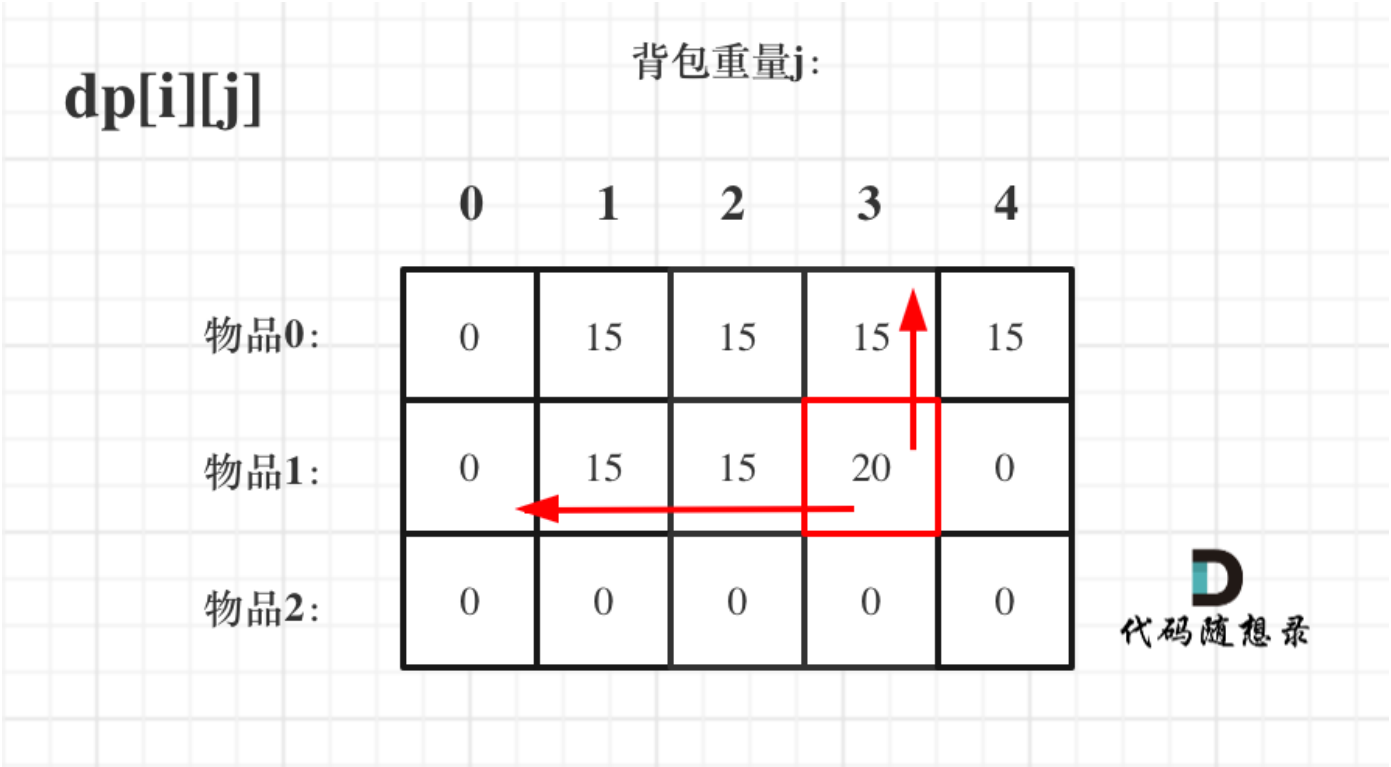
```
// weight数组的大小 就是物品个数
for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
    for(int i = 1; i < weight.size(); i++) { // 遍历物品
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);
    }
}
```

为什么也是可以的呢?

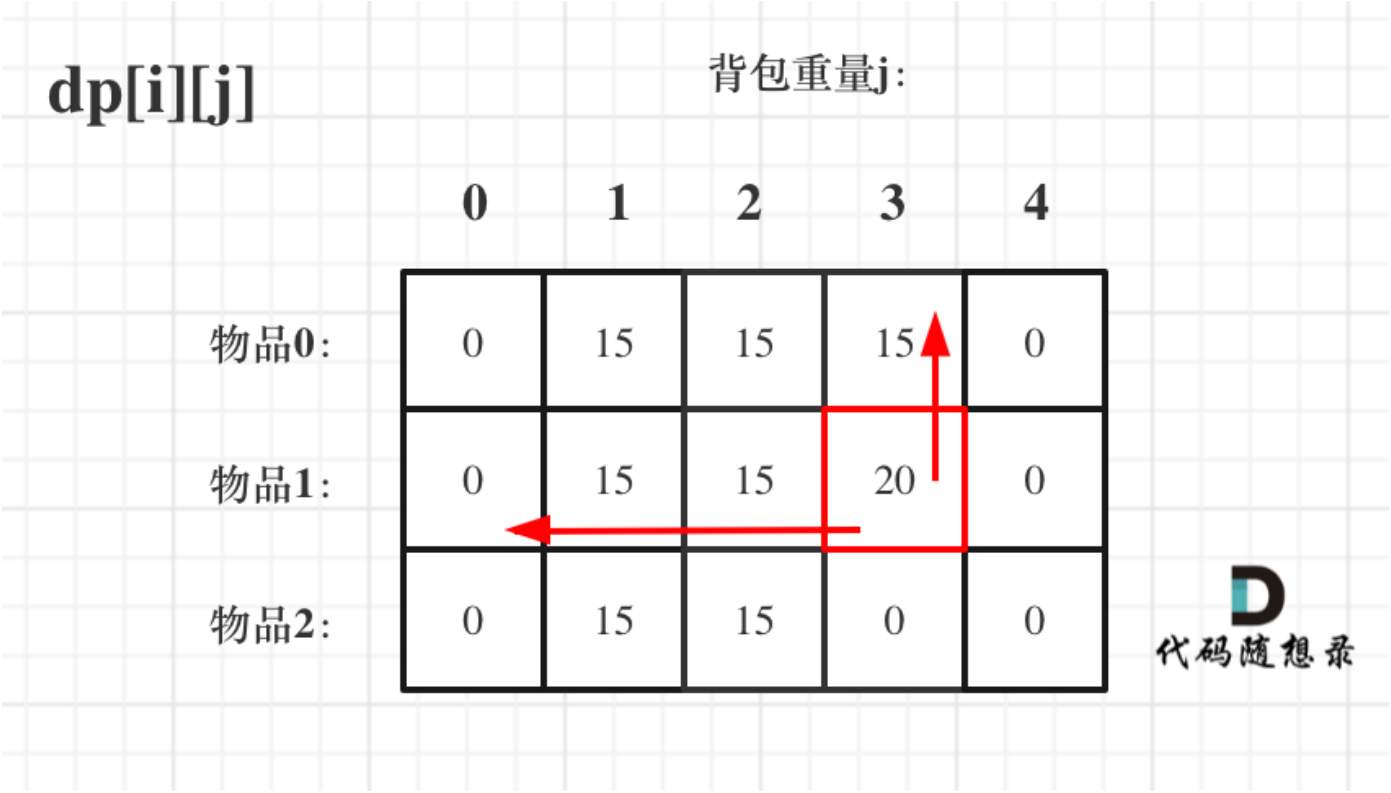
要理解递归的本质和递推的方向。

dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]); 递归公式中可以看出dp[i][j]是靠dp[i-1][j]和dp[i - 1][j - weight[i]]推导出来的。

dp[i-1][j]和dp[i - 1][j - weight[i]] 都在dp[i][j]的左上角方向（包括正上方向），那么先遍历物品，再遍历背包的过程如图所示：



再来看看先遍历背包，再遍历物品呢，如图：



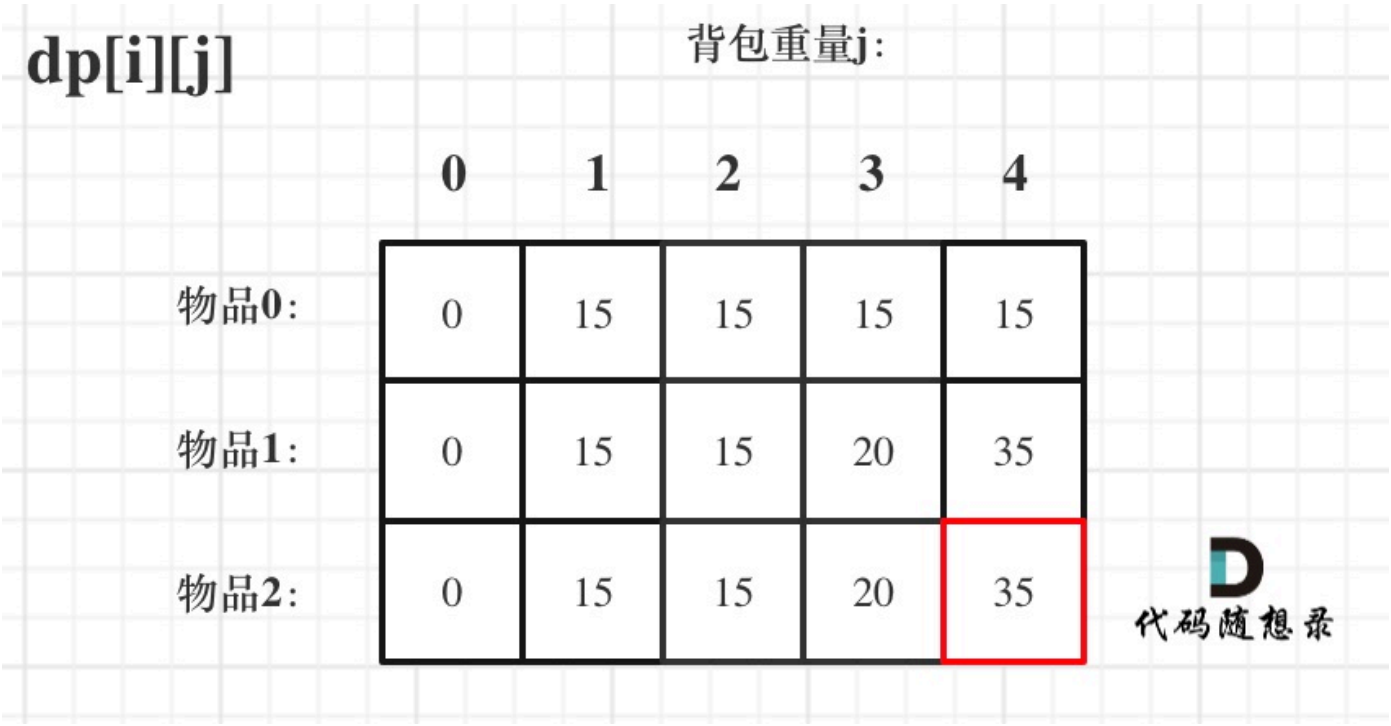
大家可以看出，虽然两个for循环遍历的次序不同，但是dp[i][j]所需要的数据就是左上角，根本不影响dp[i][j]公式的推导！

但先遍历物品再遍历背包这个顺序更好理解。

其实背包问题里，两个for循环的先后循序是非常有讲究的，理解遍历顺序其实比理解推导公式难多了。

5. 举例推导dp数组

来看一下对应的dp数组的数值，如图：



最终结果就是dp[2][4]。

建议大家此时自己在纸上推导一遍，看看dp数组里每一个数值是不是这样的。

做动态规划的题目，最好的过程就是自己在纸上举一个例子把对应的dp数组的数值推导一下，然后在动手写代码！

很多同学做dp题目，遇到各种问题，然后凭感觉东改改西改改，怎么改都不对，或者稀里糊涂就改过了。

主要就是自己没有动手推导一下dp数组的演变过程，如果推导明白了，代码写出来就算有问题，只要把dp数组打印出来，对比一下和自己推导的有什么差异，很快就可以发现问题了。

```
void test_2_wei_bag_problem1() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    int bagweight = 4;

    // 二维数组
    vector<vector<int>> dp(weight.size(), vector<int>(bagweight + 1, 0));

    // 初始化
    for (int j = weight[0]; j <= bagweight; j++) {
        dp[0][j] = value[0];
    }

    // weight数组的大小 就是物品个数
    for(int i = 1; i < weight.size(); i++) { // 遍历物品
        for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
            if (j < weight[i]) dp[i][j] = dp[i - 1][j];
            else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

        }
    }

    cout << dp[weight.size() - 1][bagweight] << endl;
}

int main() {
    test_2_wei_bag_problem1();
}
```

总结

讲了这么多才刚刚把二维dp的01背包讲完，**这里大家其实可以发现最简单的是推导公式了，推导公式估计看一遍就记下来了，但难就难在如何初始化和遍历顺序上。**

可能有的同学并没有注意到初始化 和 遍历顺序的重要性，我们后面做力扣上背包面试题目的时候，大家就会感受出来了。

下一篇 还是理论基础，我们再来讲一维dp数组实现的01背包（滚动数组），分析一下和二维有什么区别，在初始化和遍历顺序上又有什么差异，敬请期待！

12. 动态规划：01背包理论基础（二）

算法公开课

[《代码随想录》算法视频公开课](#)：[带你学透0-1背包问题！（滚动数组）](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

昨天[动态规划：关于01背包问题，你该了解这些！](#)中是用二维dp数组来讲解01背包。

今天我们就来说一说滚动数组，其实在前面的题目中我们已经用到过滚动数组了，就是把二维dp降为一维dp，一些录友当时还表示比较困惑。

那么我们通过01背包，来彻底讲一讲滚动数组！

接下来还是用如下这个例子来进行讲解

背包最大重量为4。

物品为：

	重量	价值
物品0	1	15
物品1	3	20
物品2	4	30

问背包能背的物品最大价值是多少?

一维dp数组（滚动数组）

对于背包问题其实状态都是可以压缩的。

在使用二维数组的时候，递推公式：dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

其实可以发现如果把dp[i - 1]那一层拷贝到dp[i]上，表达式完全可以是：dp[i][j] = max(dp[i][j], dp[i][j - weight[i]] + value[i]);

与其把dp[i - 1]这一层拷贝到dp[i]上，不如只用一个一维数组了，只用dp[j]（一维数组，也可以理解是一个滚动数组）。

这就是滚动数组的由来，需要满足的条件是上一层可以重复利用，直接拷贝到当前层。

读到这里估计大家都忘了 dp[i][j]里的i和j表达的是什么了，i是物品，j是背包容量。

dp[i][j] 表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。

一定要时刻记住这里i和j的含义，要不然很容易看懵了。

动规五部曲分析如下：

1. 确定dp数组的定义

在一维dp数组中，dp[j]表示：容量为j的背包，所背的物品价值可以最大为dp[j]。

2. 一维dp数组的递推公式

dp[j]为 容量为j的背包所背的最大价值，那么如何推导dp[j]呢?

dp[j]可以通过dp[j - weight[i]]推导出来，dp[j - weight[i]]表示容量为j - weight[i]的背包所背的最大价值。

dp[j - weight[i]] + value[i] 表示 容量为 j - 物品i重量 的背包 加上 物品i的价值。（也就是容量为j的背包，放入物品i了之后的价值即：dp[j]）

此时dp[j]有两个选择，一个是取自己dp[j] 相当于 二维dp数组中的dp[i-1][j]，即不放物品i，一个是取dp[j - weight[i]] + value[i]，即放物品i，指定是取最大的，毕竟是求最大价值，

所以递归公式为：

```
dp[j] = max(dp[j], dp[j – weight[i]] + value[i]);
```

可以看出相对于二维dp数组的写法，就是把dp[i][j]中i的维度去掉了。

3. 一维dp数组如何初始化

关于初始化，一定要和dp数组的定义吻合，否则到递推公式的时候就会越来越乱。

dp[j]表示：容量为j的背包，所背的物品价值可以最大为dp[j]，那么dp[0]就应该是0，因为背包容量为0所背的物品的最大价值就是0。

那么dp数组除了下标0的位置，初始为0，其他下标应该初始化多少呢?

看一下递归公式：dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

dp数组在推导的时候一定是取价值最大的数，如果题目给的价值都是正整数那么非0下标都初始化为0就可以了。

这样才能让dp数组在递归公式的过程中取的最大的价值，而不是被初始值覆盖了。

那么我假设物品价值都是大于0的，所以dp数组初始化的时候，都初始为0就可以了。

4. 一维dp数组遍历顺序

代码如下：

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j – weight[i]] + value[i]);

    }
}
```

这里大家发现和二维dp的写法中，遍历背包的顺序是不一样的!

二维dp遍历的时候，背包容量是从小到大，而一维dp遍历的时候，背包是从大到小。

为什么呢?

倒序遍历是为了保证物品i只被放入一次！。但如果一旦正序遍历了，那么物品0就会被重复加入多次！

举一个例子：物品0的重量weight[0] = 1，价值value[0] = 15

如果正序遍历

dp[1] = dp[1 - weight[0]] + value[0] = 15

dp[2] = dp[2 - weight[0]] + value[0] = 30

此时dp[2]就已经是30了，意味着物品0，被放入了两次，所以不能正序遍历。

为什么倒序遍历，就可以保证物品只放入一次呢?

倒序就是先算dp[2]

dp[2] = dp[2 - weight[0]] + value[0] = 15 （dp数组已经都初始化为0）

dp[1] = dp[1 - weight[0]] + value[0] = 15

所以从后往前循环，每次取得状态不会和之前取得状态重合，这样每种物品就只取一次了。

那么问题又来了，为什么二维dp数组历的时候不用倒序呢?

因为对于二维dp，dp[i][j]都是通过上一层即dp[i - 1][j]计算而来，本层的dp[i][j]并不会被覆盖！

（如何这里读不懂，大家就要动手试一试了，空想还是不靠谱的，实践出真知！）

再来看看两个嵌套for循环的顺序，代码中是先遍历物品嵌套遍历背包容量，那可不可以先遍历背包容量嵌套遍历物品呢?

不可以！

因为一维dp的写法，背包容量一定是要倒序遍历（原因上面已经讲了），如果遍历背包容量放在上一层，那么每个dp[j]就只会放入一个物品，即：背包里只放入了一个物品。

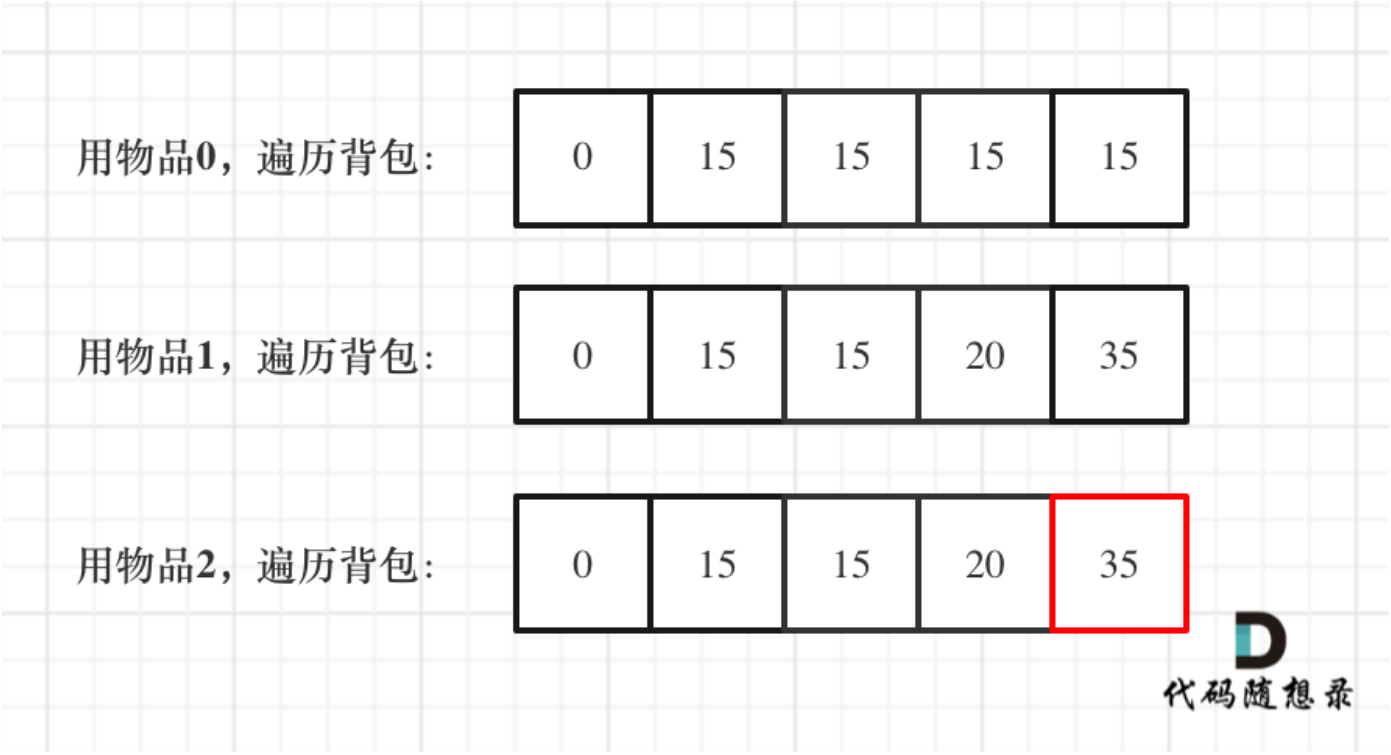
倒序遍历的原因是，本质上还是一个对二维数组的遍历，并且右下角的值依赖上一层左上角的值，因此需要保证左边的值仍然是上一层的，从右向左覆盖。

（这里如果读不懂，就再回想一下dp[j]的定义，或者就把两个for循环顺序颠倒一下试试！）

所以一维dp数组的背包在遍历顺序上和二维其实是有很大差异的！，这一点大家一定要注意。

5. 举例推导dp数组

一维dp，分别用物品0，物品1，物品2 来遍历背包，最终得到结果如下：




```
void test_1_wei_bag_problem() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    int bagWeight = 4;

    // 初始化
    vector<int> dp(bagWeight + 1, 0);
    for(int i = 0; i < weight.size(); i++) { // 遍历物品
        for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    cout << dp[bagWeight] << endl;
}

int main() {
    test_1_wei_bag_problem();
}
```

可以看出，一维dp 的01背包，要比二维简洁的多！ 初始化 和 遍历顺序相对简单了。

所以我倾向于使用一维dp数组的写法，比较直观简洁，而且空间复杂度还降了一个数量级！

在后面背包问题的讲解中，我都直接使用一维dp数组来进行推导。

总结

以上的讲解可以开发一道面试题目（毕竟力扣上没原题）。

就是本文中的题目，要求先实现一个纯二维的01背包，如果写出来了，然后再问为什么两个for循环的嵌套顺序这么写？反过来写行不行？再讲一讲初始化的逻辑。

然后要求实现一个一维数组的01背包，最后再问，一维数组的01背包，两个for循环的顺序反过来写行不行？为什么？

注意以上问题都是在候选人把代码写出来的情况下才问的。

就是纯01背包的题目，都不用考01背包应用类的题目就可以看出候选人对算法的理解程度了。

相信大家读完这篇文章，应该对以上问题都有了答案！

此时01背包理论基础就讲完了，我用了两篇文章把01背包的dp数组定义、递推公式、初始化、遍历顺序从二维数组到一维数组统统深度剖析了一遍，没有放过任何难点。

大家可以发现其实信息量还是挺大的。

如果把动态规划：关于01背包问题，你该了解这些！和本篇的内容都理解了，后面我们在做01背包的题目，就会发现非常简单了。

不用再凭感觉或者记忆去写背包，而是有自己的思考，了解其本质，代码的方方面面都在自己的掌控之中。

即使代码没有通过，也会有自己的逻辑去debug，这样就思维清晰了。

接下来就要开始用这两天的理论基础去做力扣上的背包面试题目了，录友们握紧扶手，我们要上高速啦！

---

13. 分割等和子集

力扣题目链接

题目难易：中等

给定一个只包含正整数的非空数组。是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

注意:
每个数组中的元素不会超过 100
数组的大小不会超过 200

示例 1:

- 输入: [1, 5, 11, 5]
- 输出: true
- 解释: 数组可以分割成 [1, 5, 5] 和 [11].

示例 2:

- 输入: [1, 2, 3, 5]
- 输出: false

- 解释: 数组不能分割成两个元素和相等的子集.

提示：

- 1 <= nums.length <= 200
- 1 <= nums[i] <= 100

算法公开课

《代码随想录》算法视频公开课：动态规划之背包问题，这个包能装满吗？ | LeetCode：416.分割等和子集，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目初步看，和如下两题几乎是一样的，大家可以用回溯法，解决如下两题

- 698.划分为k个相等的子集
- 473.火柴拼正方形

这道题目是要找是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

那么只要找到集合里能够出现 sum / 2 的子集总和，就算是可以分割成两个相同元素和子集了。

本题是可以用回溯暴力搜索出所有答案的，但最后超时了，也不想再优化了，放弃回溯，直接上01背包吧。

如果对01背包不够了解，建议仔细看完如下两篇：

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

01背包问题

背包问题，大家都知道，有N件物品和一个最多能背重量为W 的背包。第i件物品的重量是weight[i]，得到的价值是value[i] 。每件物品只能用一次，求解将哪些物品装入背包里物品价值总和最大。

背包问题有多种背包方式，常见的有：01背包、完全背包、多重背包、分组背包和混合背包等等。

要注意题目描述中商品是不是可以重复放入。

即一个商品如果可以重复多次放入是完全背包，而只能放入一次是01背包，写法还是不一样的。

要明确本题中我们要使用的是01背包，因为元素我们只能用一次。

回归主题：首先，本题要求集合里能否出现总和为 sum / 2 的子集。

那么来一一对应一下本题，看看背包问题如何来解决。

只有确定了如下四点，才能把01背包问题套到本题上来。

- 背包的体积为sum / 2
- 背包要放入的商品（集合里的元素）重量为 元素的数值，价值也为元素的数值
- 背包如果正好装满，说明找到了总和为 sum / 2 的子集。
- 背包中每一个元素是不可重复放入。

以上分析完，我们就可以套用01背包，来解决这个问题了。

动规五部曲分析如下：

1. 确定dp数组以及下标的含义

01背包中，dp[j] 表示： 容量为j的背包，所背的物品价值最大可以为dp[j]。

本题中每一个元素的数值既是重量，也是价值。

套到本题，dp[j]表示 背包总容量（所能装的总重量）是j，放进物品后，背的最大重量为dp[j]。

那么如果背包容量为target， dp[target]就是装满 背包之后的重量，所以 当 dp[target] == target 的时候，背包就装满了。

有录友可能想，那还有装不满的时候?

拿输入数组 [1, 5, 11, 5]，举例， dp[7] 只能等于 6，因为 只能放进 1 和 5。

而dp[6] 就可以等于6了，放进1 和 5，那么dp[6] == 6，说明背包装满了。

2. 确定递推公式

01背包的递推公式为：dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

本题，相当于背包里放入数值，那么物品i的重量是nums[i]，其价值也是nums[i]。

所以递推公式：dp[j] = max(dp[j], dp[j - nums[i]] + nums[i]);

3. dp数组如何初始化

在01背包，一维dp如何初始化，已经讲过，

从dp[j]的定义来看，首先dp[0]一定是0。

如果题目给的价值都是正整数那么非0下标都初始化为0就可以了，如果题目给的价值有负数，那么非0下标就要初始化为负无穷。

这样才能让dp数组在递推的过程中取得最大的价值，而不是被初始值覆盖了。

本题题目中 只包含正整数的非空数组，所以非0下标的元素初始化为0就可以了。

代码如下：

```
// 题目中说：每个数组中的元素不会超过 100，数组的大小不会超过 200
// 总和不会大于20000，背包最大只需要其中一半，所以10001大小就可以了
vector<int> dp(10001, 0);
```

4. 确定遍历顺序

在动态规划：关于01背包问题，你该了解这些！（滚动数组）中就已经说明：如果使用一维dp数组，物品遍历的for循环放在外层，遍历背包的for循环放在内层，且内层for循环倒序遍历！

代码如下：

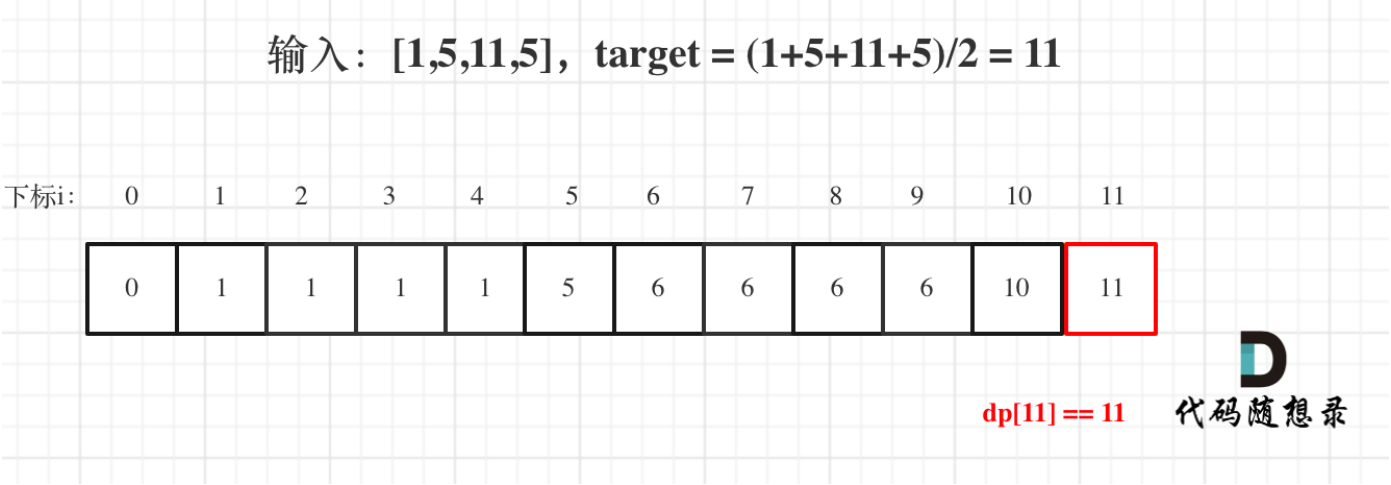
```
// 开始 01背包
for(int i = 0; i < nums.size(); i++) {
    for(int j = target; j >= nums[i]; j--) { // 每一个元素一定是不可重复放入，所以从大到小遍历
        dp[j] = max(dp[j], dp[j - nums[i]] + nums[i]);
    }
}
```

5. 举例推导dp数组

dp[j]的数值一定是小于等于j的。

如果dp[j] == j 说明，集合中的子集总和正好可以凑成总和j，理解这一点很重要。

用例1，输入[1,5,11,5] 为例，如图：



最后dp[11] == 11，说明可以将这个数组分割成两个子集，使得两个子集的元素和相等。

综上分析完毕，C++代码如下：

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int sum = 0;

        // dp[i]中的i表示背包内总和
        // 题目中说：每个数组中的元素不会超过 100，数组的大小不会超过 200
        // 总和不会大于20000，背包最大只需要其中一半，所以10001大小就可以了
        vector<int> dp(10001, 0);
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i];
        }
        // 也可以使用库函数一步求和
        // int sum = accumulate(nums.begin(), nums.end(), 0);
        if (sum % 2 == 1) return false;
        int target = sum / 2;

        // 开始 01背包
        for(int i = 0; i < nums.size(); i++) {
```

```
            for(int j = target; j >= nums[i]; j--) { // 每一个元素一定是不可重复放入，所以从大到小遍历
                dp[j] = max(dp[j], dp[j - nums[i]] + nums[i]);
            }
        }
        // 集合中的元素正好可以凑成总和target
        if (dp[target] == target) return true;
        return false;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(n)，虽然dp数组大小为一个常数，但是大常数

总结

这道题目就是一道01背包应用类的题目，需要我们拆解题目，然后套入01背包的场景。

01背包相对于本题，主要要理解，题目中物品是nums[i]，重量是nums[i]，价值也是nums[i]，背包体积是sum/2。

看代码的话，就可以发现，基本就是按照01背包的写法来的。

14.最后一块石头的重量II

[力扣题目链接]

题目难度：中等

有一堆石头，每块石头的重量都是正整数。

每一回合，从中选出任意两块石头，然后将它们一起粉碎。假设石头的重量分别为 x 和 y，且 x <= y。那么粉碎的可能结果如下：

如果 x == y，那么两块石头都会被完全粉碎；

如果 x != y，那么重量为 x 的石头将会完全粉碎，而重量为 y 的石头新重量为 y-x。

最后，最多只会剩下一块石头。返回此石头最小的可能重量。如果没有石头剩下，就返回 0。

示例：

- 输入：[2,7,4,1,8,1]
- 输出：1

解释：

- 组合 2 和 4，得到 2，所以数组转化为 [2,7,1,8,1]，
- 组合 7 和 8，得到 1，所以数组转化为 [2,1,1,1]，
- 组合 2 和 1，得到 1，所以数组转化为 [1,1,1]，

- 组合 1 和 1，得到 0，所以数组转化为 [1]，这就是最优值。

提示：

- 1 <= stones.length <= 30
- 1 <= stones[i] <= 1000

算法公开课

《代码随想录》算法视频公开课：这个背包最多能装多少？LeetCode：1049.最后一块石头的重量II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

如果对背包问题不都熟悉先看这两篇：

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

本题其实就是尽量让石头分成重量相同的两堆，相撞之后剩下的石头最小，**这样就化解成01背包问题了**。

是不是感觉和昨天讲解的416. 分割等和子集非常像了。

本题物品的重量为stones[i]，物品的价值也为stones[i]。

对应着01背包里的物品重量weight[i]和 物品价值value[i]。

接下来进行动规五步曲：

1. 确定dp数组以及下标的含义

dp[j]表示容量（这里说容量更形象，其实就是重量）为j的背包，最多可以背最大重量为dp[j]。

可以回忆一下01背包中，dp[j]的含义，容量为j的背包，最多可以装的价值为 dp[j]。

相对于 01背包，本题中，石头的重量是 stones[i]，石头的价值也是 stones[i] ，可以 “最多可以装的价值为 dp[j]” == “最多可以背的重量为dp[j]”

2. 确定递推公式

01背包的递推公式为：dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

本题则是：**dp[j] = max(dp[j], dp[j - stones[i]] + stones[i]);**

一些同学可能看到这dp[j - stones[i]] + stones[i]中 又有- stones[i] 又有+stones[i]，看着有点晕乎。

大家可以再去看 dp[j]的含义。

3. dp数组如何初始化

既然 dp[j]中的j表示容量，那么最大容量（重量）是多少呢，就是所有石头的重量和。

因为提示中给出1 <= stones.length <= 30，1 <= stones[i] <= 1000，所以最大重量就是30 * 1000 。

而我们要求的target其实只是最大重量的一半，所以dp数组开到15000大小就可以了。

当然也可以把石头遍历一遍，计算出石头总重量 然后除2，得到dp数组的大小。

我这里就直接用15000了。

接下来就是如何初始化dp[j]呢，因为重量都不会是负数，所以dp[j]都初始化为0就可以了，这样在递归公式dp[j] = max(dp[j], dp[j - stones[i]] + stones[i]);中dp[j]才不会初始值所覆盖。

代码为：

```
vector<int> dp(15001, 0);
```

4. 确定遍历顺序

在动态规划：关于01背包问题，你该了解这些！（滚动数组）中就已经说明：如果使用一维dp数组，物品遍历的for循环放在外层，遍历背包的for循环放在内层，且内层for循环倒序遍历！

代码如下：

```
for (int i = 0; i < stones.size(); i++) { // 遍历物品
    for (int j = target; j >= stones[i]; j--) { // 遍历背包
        dp[j] = max(dp[j], dp[j - stones[i]] + stones[i]);
    }
}
```

5. 举例推导dp数组

举例，输入：[2,4,1,1]，此时target = (2 + 4 + 1 + 1)/2 = 4 ，dp数组状态图如下：

输入：[2,4,1,1]，target = 4

最后dp[target]里是容量为target的背包所能背的最大重量。

那么分成两堆石头，一堆石头的总重量是dp[target]，另一堆就是sum - dp[target]。

在计算target的时候，target = sum / 2 因为是向下取整，所以sum - dp[target] 一定是大于等于dp[target]的。

那么相撞之后剩下的最小石头重量就是 (sum - dp[target]) - dp[target]。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int lastStoneWeightII(vector<int>& stones) {
        vector<int> dp(15001, 0);
        int sum = 0;
        for (int i = 0; i < stones.size(); i++) sum += stones[i];
        int target = sum / 2;
        for (int i = 0; i < stones.size(); i++) { // 遍历物品
            for (int j = target; j >= stones[i]; j--) { // 遍历背包
                dp[j] = max(dp[j], dp[j - stones[i]] + stones[i]);
            }
        }
```

```
        return sum - dp[target] - dp[target];
    }
};
```

- 时间复杂度：O(m × n) , m是石头总重量（准确的说是总重量的一半），n为石头块数
- 空间复杂度：O(m)

总结

本题其实和416. 分割等和子集几乎是一样的，只是最后对dp[target]的处理方式不同。

416. 分割等和子集相当于是求背包是否正好装满，而本题是求背包最多能装多少。

---

15. 本周小结！（动态规划系列三）

本周我们正式开始讲解背包问题，也是动规里非常重要的一类问题。

背包问题其实有很多细节，如果了解个大概，然后也能一气呵成把代码写出来，但稍稍变变花样可能会陷入迷茫了。

开始回顾一下本周的内容吧！

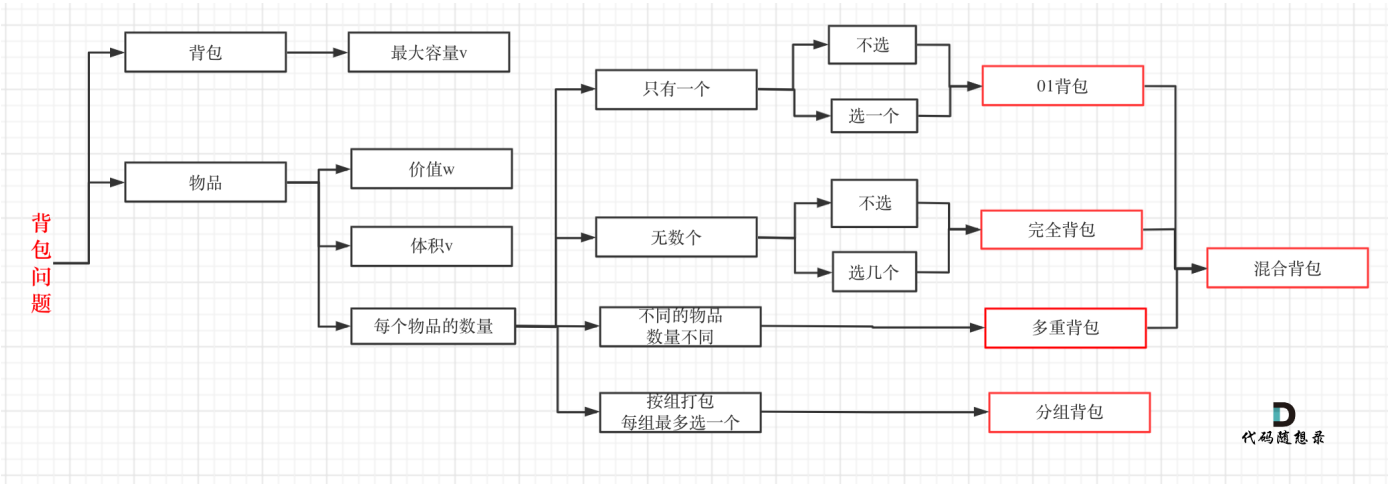
周一

动态规划：关于01背包问题，你该了解这些！ 中，我们开始介绍了背包问题。

首先对于背包的所有问题中，01背包是最最基础的，其他背包也是在01背包的基础上稍作变化。

所以我才花费这么大精力去讲解01背包。

关于其他几种常用的背包，大家看这张图就了然于胸了：



本文用动规五部曲详细讲解了01背包的二维dp数组的实现方法，大家其实可以发现最简单的是推导公式了，推导公式估计看一遍就记下来了，但难就难在确定初始化和遍历顺序上。

1. 确定dp数组以及下标的含义

dp[i][j] 表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。

2. 确定递推公式

dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

3. dp数组如何初始化

```
// 初始化 dp
vector<vector<int>> dp(weight.size() + 1, vector<int>(bagWeight + 1, 0));
for (int j = bagWeight; j >= weight[0]; j--) {
    dp[0][j] = dp[0][j - weight[0]] + value[0];
}
```

4. 确定遍历顺序

01背包二维dp数组在遍历顺序上，外层遍历物品 ，内层遍历背包容量 和 外层遍历背包容量 ，内层遍历物品 都是可以的！

但是先遍历物品更好理解。代码如下：

```
// weight数组的大小 就是物品个数
for(int i = 1; i < weight.size(); i++) { // 遍历物品
    for(int j = 0; j <= bagWeight; j++) { // 遍历背包容量
        if (j < weight[i]) dp[i][j] = dp[i - 1][j]; // 这个是为了展现dp数组里元素的变化
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);

    }
}
```

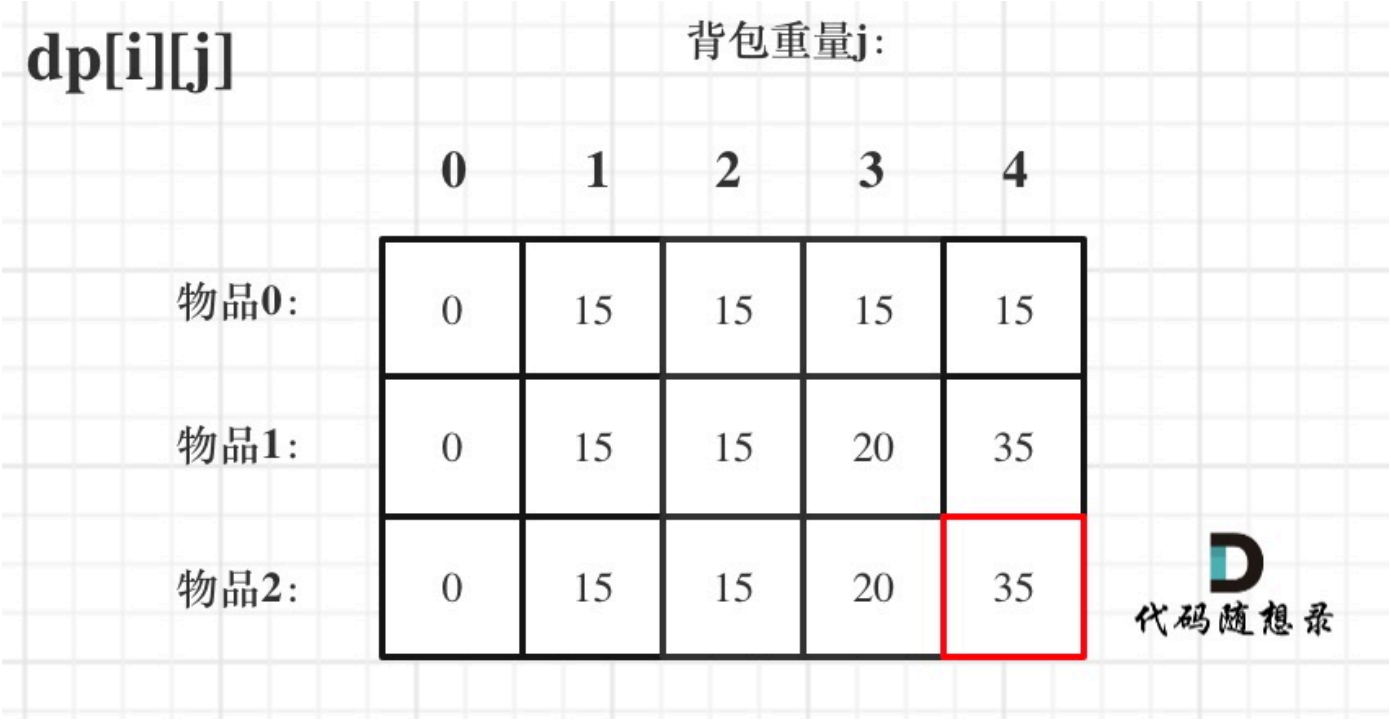
5. 举例推导dp数组

背包最大重量为4。

物品为：

	重量	价值
物品0	1	15
物品1	3	20
物品2	4	30

来看一下对应的dp数组的数值，如图：



最终结果就是dp[2][4]。

周二

动态规划：关于01背包问题，你该了解这些！（滚动数组） 中把01背包的一维dp数组（滚动数组）实现详细讲解了一遍。

分析一下和二维dp数组有什么区别，在初始化和遍历顺序上又有什么差异?

最后总结了一道朴实无华的背包面试题。

要求候选人先实现一个纯二维的01背包，如果写出来了，然后再问为什么两个for循环的嵌套顺序这么写？反过来写行不行？再讲一讲初始化的逻辑。

然后要求实现一个一维数组的01背包，最后再问，一维数组的01背包，两个for循环的顺序反过来写行不行？为什么?

这几个问题就可以考察出候选人的算法功底了。

01背包一维数组分析如下：

1. 确定dp数组的定义

在一维dp数组中，dp[j]表示：容量为j的背包，所背的物品价值可以最大为dp[j]。

2. 一维dp数组的递推公式

```
dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
```

3. 一维dp数组如何初始化

如果物品价值都是大于0的，所以dp数组初始化的时候，都初始为0就可以了。

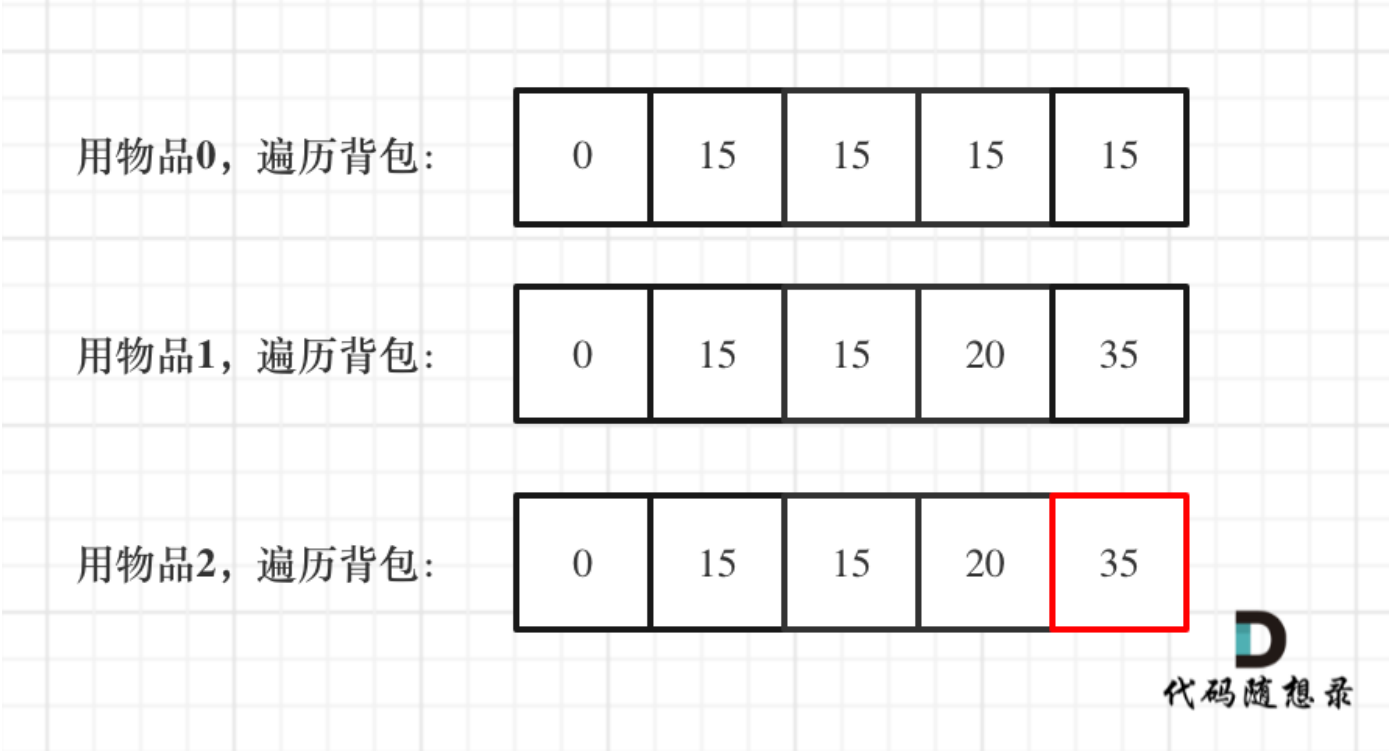
4. 一维dp数组遍历顺序

代码如下：

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

    }
}
```

5. 举例推导dp数组

一维dp，分别用物品0，物品1，物品2 来遍历背包，最终得到结果如下：



周三

动态规划：416. 分割等和子集中我们开始用01背包来解决问题。

只有确定了如下四点，才能把01背包问题套到本题上来。

- 背包的体积为sum / 2
- 背包要放入的商品（集合里的元素）重量为 元素的数值，价值也为元素的数值
- 背包如何正好装满，说明找到了总和为 sum / 2 的子集。
- 背包中每一个元素是不可重复放入。

接下来就是一个完整的01背包问题，大家应该可以轻松做出了。

周四

动态规划：1049. 最后一块石头的重量 II这道题目其实和动态规划：416. 分割等和子集是非常像的。

本题其实就是尽量让石头分成重量相同的两堆，相撞之后剩下的石头最小，这样就化解成01背包问题了。

动态规划：416. 分割等和子集相当于是求背包是否正好装满，而本题是求背包最多能装多少。

这两道题目是对dp[target]的处理方式不同。这也考验的对dp[i]定义的理解。

总结

总体来说，本周信息量还是比较大的，特别对于对动态规划还不够了解的同学。

但如果坚持下来把，我在文章中列出的每一个问题，都仔细思考，消化为自己的知识，那么进步一定是飞速的。

有的同学可能看了看背包递推公式，上来就能撸它几道题目，然后背包问题就这么过去了，其实这样是很不牢固的。

就像是我们讲解01背包的时候，花了那么大力气才把每一个细节都讲清楚，这里其实是基础，后面的背包问题怎么变，基础比较牢固自然会有自己的一套思考过程。

16.目标和

力扣题目链接

难度：中等

给定一个非负整数数组，a1, a2, ..., an, 和一个目标数，S。现在你有两个符号 + 和 -。对于数组中的任意一个整数，你都可以从 + 或 -中选择一个符号添加在前面。

返回可以使最终数组和为目标数 S 的所有添加符号的方法数。

示例：

- 输入：nums: [1, 1, 1, 1, 1], S: 3
- 输出：5

解释：

- -1+1+1+1+1 = 3
- +1-1+1+1+1 = 3
- +1+1-1+1+1 = 3
- +1+1+1-1+1 = 3
- +1+1+1+1-1 = 3

一共有5种方法让最终目标和为3。

提示：

- 数组非空，且长度不会超过 20 。
- 初始的数组的和不会超过 1000 。
- 保证返回的最终结果能被 32 位整数存下。

算法公开课

《代码随想录》算法视频公开课：装满背包有多少种方法？ | LeetCode：494.目标和，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

如果对背包问题不都熟悉先看这两篇：

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

如果跟着「代码随想录」一起学过回溯算法系列的录友，看到这道题，应该有一种直觉，就是感觉好像回溯法可以爆搜出来。

事实确实如此，下面我也会给出相应的代码，只不过会超时，哈哈。

这道题目咋眼一看和动态规划背包啥的也没啥关系。

本题要如何使表达式结果为target，

既然为target，那么就一定有 left组合 - right组合 = target。

left + right = sum，而sum是固定的。right = sum - left

公式来了， left - (sum - left) = target 推导出 left = (target + sum)/2 。

target是固定的，sum是固定的，left就可以求出来。

此时问题就是在集合nums中找出和为left的组合。

回溯算法

在回溯算法系列中，一起学过这道题目回溯算法：39. 组合总和的录友应该感觉很熟悉，这不就是组合总和问题么？

此时可以套组合总和的回溯法代码，几乎不用改动。

当然，也可以转变成序列区间选+ 或者 -，使用回溯法，那就是另一个解法。

我也把代码给出来吧，大家可以了解一下，回溯的解法，以下是本题转变为组合总和问题的回溯法代码：

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& candidates, int target, int sum, int startIndex) {
        if (sum == target) {
            result.push_back(path);
        }
        // 如果 sum + candidates[i] > target 就终止遍历
        for (int i = startIndex; i < candidates.size() && sum + candidates[i] <=
target; i++) {
            sum += candidates[i];
            path.push_back(candidates[i]);
            backtracking(candidates, target, sum, i + 1);
```

```
            sum -= candidates[i];
            path.pop_back();

        }
    }
public:
    int findTargetSumWays(vector<int>& nums, int S) {
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) sum += nums[i];
        if (S > sum) return 0; // 此时没有方案
        if ((S + sum) % 2) return 0; // 此时没有方案，两个int相加的时候要各位小心数值溢出的问题
        int bagSize = (S + sum) / 2; // 转变为组合总和问题，bagsize就是要求的和

        // 以下为回溯法代码
        result.clear();
        path.clear();
        sort(nums.begin(), nums.end()); // 需要排序
        backtracking(nums, bagSize, 0, 0);
        return result.size();
    }
};
```

当然以上代码超时了。

也可以使用记忆化回溯，但这里我就不在回溯上下功夫了，直接看动规吧

动态规划

如何转化为01背包问题呢。

假设加法的总和为x，那么减法对应的总和就是sum - x。

所以我们要求的是 x - (sum - x) = target

x = (target + sum) / 2

此时问题就转化为，装满容量为x的背包，有几种方法。

这里的x，就是bagSize，也就是我们后面要求的背包容量。

大家看到(target + sum) / 2 应该担心计算的过程中向下取整有没有影响。

这么担心就对了，例如sum 是5，S是2的话其实就是无解的，所以：

```
 (C++代码中，输入的S 就是题目描述的 target)
if ((S + sum) % 2 == 1) return 0; // 此时没有方案
```

同时如果 S的绝对值已经大于sum，那么也是没有方案的。

```
 (C++代码中，输入的S 就是题目描述的 target)
if (abs(S) > sum) return 0; // 此时没有方案
```

再回归到01背包问题，为什么是01背包呢?

因为每个物品（题目中的1）只用一次!

这次和之前遇到的背包问题不一样了，之前都是求容量为j的背包，最多能装多少。

本题则是装满有几种方法。其实这就是一个组合问题了。

1. 确定dp数组以及下标的含义

dp[j] 表示：填满j（包括j）这么大容积的包，有dp[j]种方法

其实也可以使用二维dp数组来求解本题，dp[i][j]：使用 下标为[0, i]的nums[i]能够凑满j（包括j）这么大容量的包，有dp[i][j]种方法。

下面我都是统一使用一维数组进行讲解， 二维降为一维（滚动数组），其实就是上一层拷贝下来，这个我在动态规划：关于01背包问题，你该了解这些！（滚动数组）也有介绍。

2. 确定递推公式

有哪些来源可以推出dp[j]呢?

只要搞到nums[i]，凑成dp[j]就有dp[j - nums[i]] 种方法。

例如：dp[j]，j 为5，

- 已经有一个1（nums[i]） 的话，有 dp[4]种方法 凑成 容量为5的背包。
- 已经有一个2（nums[i]） 的话，有 dp[3]种方法 凑成 容量为5的背包。
- 已经有一个3（nums[i]） 的话，有 dp[2]中方法 凑成 容量为5的背包
- 已经有一个4（nums[i]） 的话，有 dp[1]中方法 凑成 容量为5的背包
- 已经有一个5 （nums[i]）的话，有 dp[0]中方法 凑成 容量为5的背包

那么凑整dp[5]有多少方法呢，也就是把 所有的 dp[j - nums[i]] 累加起来。

所以求组合类问题的公式，都是类似这种：

```
dp[j] += dp[j - nums[i]]
```

这个公式在后面在讲解背包解决排列组合问题的时候还会用到!

3. dp数组如何初始化

从递推公式可以看出，在初始化的时候dp[0] 一定要初始化为1，因为dp[0]是在公式中一切递推结果的起源，如果dp[0]是0的话，递推结果将都是0。

这里有录友可能认为从dp数组定义来说 dp[0] 应该是0，也有录友认为dp[0]应该是1。

其实不要硬去解释它的含义，咱就把 dp[0]的情况带入本题看看应该等于多少。

如果数组[0] ，target = 0，那么 bagSize = (target + sum) / 2 = 0。 dp[0]也应该是1， 也就是说给数组里的元素 0 前面无论放加法还是减法，都是 1 种方法。

所以本题我们应该初始化 dp[0] 为 1。

可能有同学想了，那 如果是 数组[0,0,0,0,0] target = 0 呢。

其实 此时最终的dp[0] = 32，也就是这五个零 子集的所有组合情况，但此dp[0]非彼dp[0]，dp[0]能算出32，其基础是因为dp[0] = 1 累加起来的。

dp[j]其他下标对应的数值也应该初始化为0，从递推公式也可以看出，dp[j]要保证是0的初始值，才能正确的由dp[j - nums[i]]推导出来。

4. 确定遍历顺序

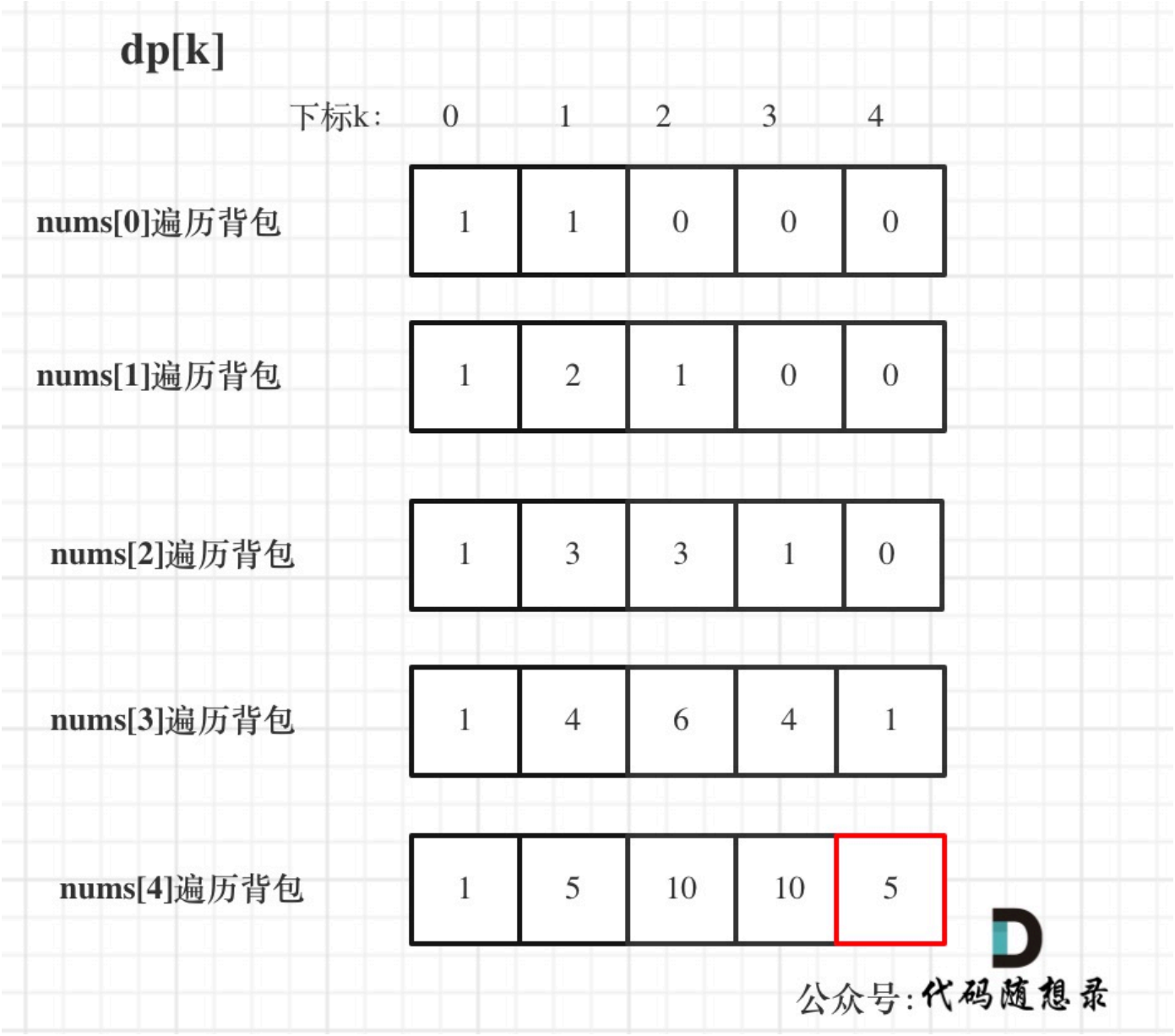
在动态规划：关于01背包问题，你该了解这些！（滚动数组）中，我们讲过对于01背包问题一维dp的遍历，nums放在外循环，target在内循环，且内循环倒序。

5. 举例推导dp数组

输入：nums: [1, 1, 1, 1, 1], S: 3

bagSize = (S + sum) / 2 = (3 + 5) / 2 = 4

dp数组状态变化如下：



C++代码如下：

```
class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int S) {
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) sum += nums[i];
        if (abs(S) > sum) return 0; // 此时没有方案
        if ((S + sum) % 2 == 1) return 0; // 此时没有方案
        int bagSize = (S + sum) / 2;
        vector<int> dp(bagSize + 1, 0);
        dp[0] = 1;
        for (int i = 0; i < nums.size(); i++) {
            for (int j = bagSize; j >= nums[i]; j--) {
                dp[j] += dp[j - nums[i]];
            }
        }
        return dp[bagSize];
    }
};
```

- 时间复杂度：O(n × m)，n为正数个数，m为背包容量
- 空间复杂度：O(m)，m为背包容量

总结

此时 大家应该不禁想起，我们之前讲过的回溯算法：39. 组合总和是不是应该也可以用dp来做啊?

是的，如果仅仅是求个数的话，就可以用dp，但回溯算法：39. 组合总和要求的是把所有组合列出来，还是要使用回溯法爆搜的。

本题还是有点难度，大家也可以记住，在求装满背包有几种方法的情况下，递推公式一般为：

```
dp[j] += dp[j - nums[i]];
```

后面我们在讲解完全背包的时候，还会用到这个递推公式!

17.一和零

力扣题目链接

给你一个二进制字符串数组 strs 和两个整数 m 和 n 。

请你找出并返回 strs 的最大子集的大小，该子集中 最多 有 m 个 0 和 n 个 1 。

如果 x 的所有元素也是 y 的元素，集合 x 是集合 y 的 子集 。

示例 1：

- 输入：strs = ["10", "0001", "111001", "1", "0"], m = 5, n = 3
- 输出：4
- 解释：最多有 5 个 0 和 3 个 1 的最大子集是 {"10","0001","1","0"} ，因此答案是 4 。
  其他满足题意但较小的子集包括 {"0001","1"} 和 {"10","1","0"} 。{"111001"} 不满足题意，因为它含 4 个 1 ，大于 n 的值 3 。

示例 2：

- 输入：strs = ["10", "0", "1"], m = 1, n = 1
- 输出：2
- 解释：最大的子集是 {"0", "1"} ，所以答案是 2 。

提示：

- 1 <= strs.length <= 600
- 1 <= strs[i].length <= 100
- strs[i] 仅由 '0' 和 '1' 组成
- 1 <= m, n <= 100

算法公开课

《代码随想录》算法视频公开课：装满这个背包最多用多少个物品？ | LeetCode：474.一和零，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

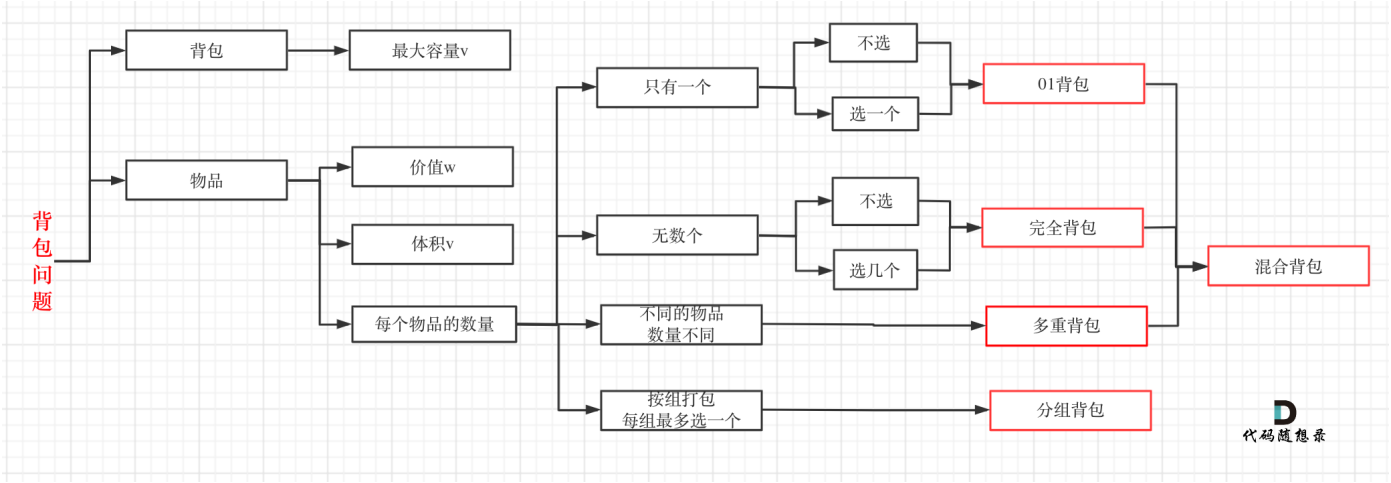
如果对背包问题不都熟悉先看这两篇：

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

这道题目，还是比较难的，也有点像程序员自己给自己出个脑筋急转弯，程序员何苦为难程序员呢。

来说题，本题不少同学会认为是多重背包，一些题解也是这么写的。

其实本题并不是多重背包，再来看一下这个图，捋清几种背包的关系



多重背包是每个物品，数量不同的情况。

本题中strs 数组里的元素就是物品，每个物品都是一个！

而m 和 n相当于是一个背包，两个维度的背包。

理解成多重背包的同学主要是把m和n混淆为物品了，感觉这是不同数量的物品，所以以为是多重背包。

但本题其实是01背包问题！

只不过这个背包有两个维度，一个是m 一个是n，而不同长度的字符串就是不同大小的待装物品。

开始动规五部曲：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：最多有i个0和j个1的strs的最大子集的大小为dp[i][j]。

2. 确定递推公式

dp[i][j] 可以由前一个strs里的字符串推导出来，strs里的字符串有zeroNum个0，oneNum个1。

dp[i][j] 就可以是 dp[i - zeroNum][j - oneNum] + 1。

然后我们在遍历的过程中，取dp[i][j]的最大值。

所以递推公式：dp[i][j] = max(dp[i][j], dp[i - zeroNum][j - oneNum] + 1);

此时大家可以回想一下01背包的递推公式：dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

对比一下就会发现，字符串的zeroNum和oneNum相当于物品的重量（weight[i]），字符串本身的个数相当于物品的价值（value[i]）。

这就是一个典型的01背包！ 只不过物品的重量有了两个维度而已。

3. dp数组如何初始化

在动态规划：关于01背包问题，你该了解这些！（滚动数组）中已经讲解了，01背包的dp数组初始化为0就可以。

因为物品价值不会是负数，初始为0，保证递推的时候dp[i][j]不会被初始值覆盖。

4. 确定遍历顺序

在动态规划：关于01背包问题，你该了解这些！（滚动数组）中，我们讲到了01背包为什么一定是外层for循环遍历物品，内层for循环遍历背包容量且从后向前遍历！

那么本题也是，物品就是strs里的字符串，背包容量就是题目描述中的m和n。

代码如下：

```
for (string str : strs) { // 遍历物品
    int oneNum = 0, zeroNum = 0;
    for (char c : str) {
        if (c == '0') zeroNum++;
        else oneNum++;
    }
    for (int i = m; i >= zeroNum; i--) { // 遍历背包容量且从后向前遍历!
        for (int j = n; j >= oneNum; j--) {
            dp[i][j] = max(dp[i][j], dp[i - zeroNum][j - oneNum] + 1);
        }
    }
}
```

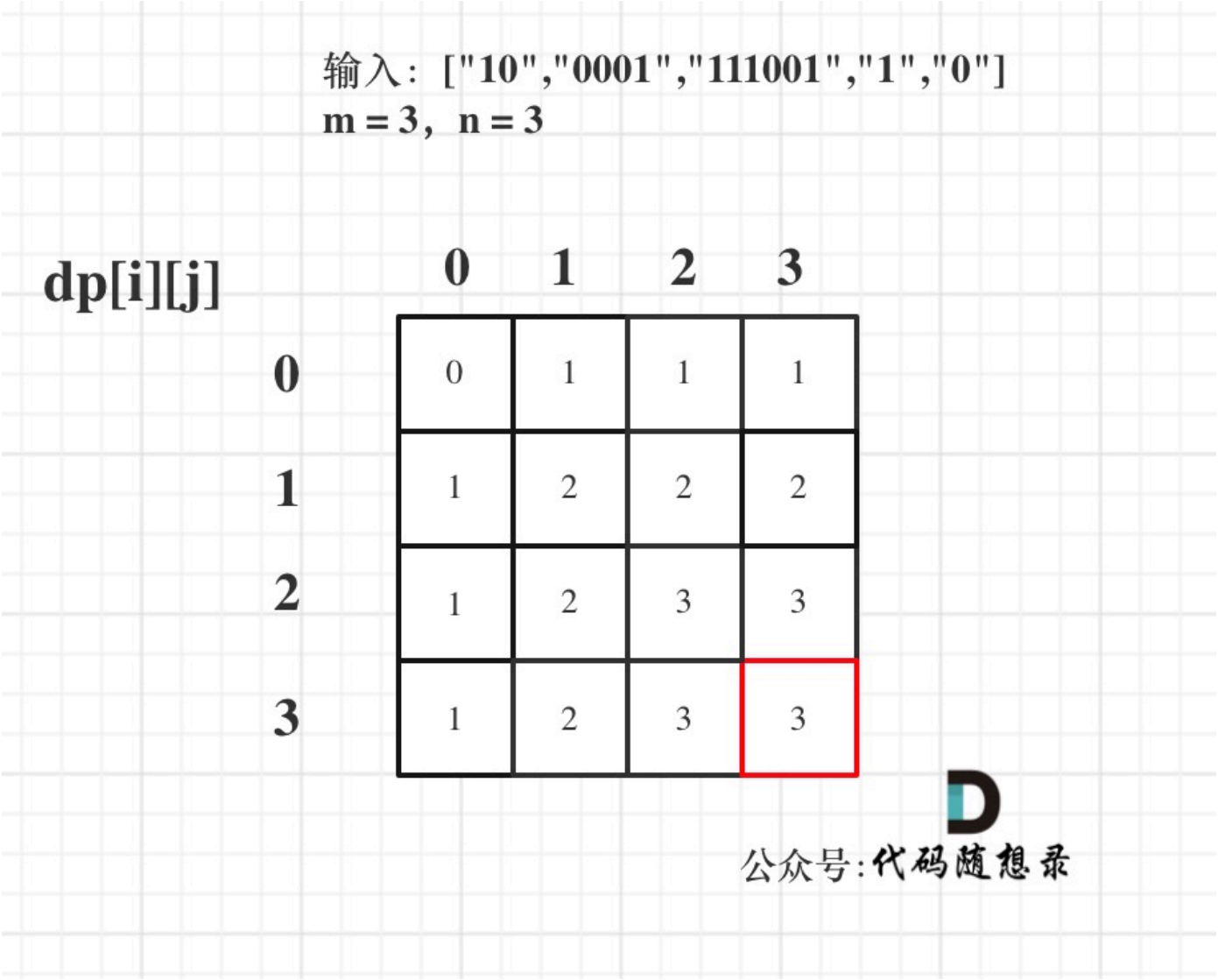
有同学可能想，那个遍历背包容量的两层for循环先后循序有没有什么讲究?

没讲究，都是物品重量的一个维度，先遍历哪个都行!

5. 举例推导dp数组

以输入：["10","0001","111001","1","0"]，m = 3，n = 3为例

最后dp数组的状态如下所示：



以上动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    int findMaxForm(vector<string>& strs, int m, int n) {
        vector<vector<int>> dp(m + 1, vector<int> (n + 1, 0)); // 默认初始化0
        for (string str : strs) { // 遍历物品
            int oneNum = 0, zeroNum = 0;
            for (char c : str) {
                if (c == '0') zeroNum++;
                else oneNum++;
            }
            for (int i = m; i >= zeroNum; i--) { // 遍历背包容量且从后向前遍历!
                for (int j = n; j >= oneNum; j--) {
                    dp[i][j] = max(dp[i][j], dp[i - zeroNum][j - oneNum] + 1);
                }
            }
        }
        return dp[m][n];
    }
};
```

- 时间复杂度: O(kmn)，k 为strs的长度
- 空间复杂度: O(mn)

总结

不少同学刷过这道题，可能没有总结这究竟是什么背包。

此时我们讲解了0-1背包的多种应用，

- 纯 0 - 1 背包 是求 给定背包容量 装满背包 的最大价值是多少。
- 416. 分割等和子集 是求 给定背包容量，能不能装满这个背包。
- 1049. 最后一块石头的重量 II 是求 给定背包容量，尽可能装，最多能装多少
- 494. 目标和 是求 给定背包容量，装满背包有多少种方法。
- 本题是求 给定背包容量，装满背包最多有多少个物品。

所以在代码随想录中所列举的题目，都是 0-1背包不同维度上的应用，大家可以细心体会！

---

18. 动态规划：完全背包理论基础

算法公开课

《代码随想录》算法视频公开课：带你学透完全背包问题！，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

完全背包

有N件物品和一个最多能背重量为W的背包。第i件物品的重量是weight[i]，得到的价值是value[i] 。**每件物品都有无限个（也就是可以放入背包多次）**，求解将哪些物品装入背包里物品价值总和最大。

完全背包和01背包问题唯一不同的地方就是，每种物品有无限件。

同样leetcode上没有纯完全背包问题，都是需要完全背包的各种应用，需要转化成完全背包问题，所以我这里还是以纯完全背包问题进行讲解理论和原理。

在下面的讲解中，我依然举这个例子：

背包最大重量为4。

物品为：

	重量	价值
物品0	1	15
物品1	3	20
物品2	4	30

每件商品都有无限个！

问背包能背的物品最大价值是多少?

01背包和完全背包唯一不同就是体现在遍历顺序上，所以本文就不去做动规五部曲了，我们直接针对遍历顺序经行分析！

关于01背包我如下两篇已经进行深入分析了：

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

首先再回顾一下01背包的核心代码

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

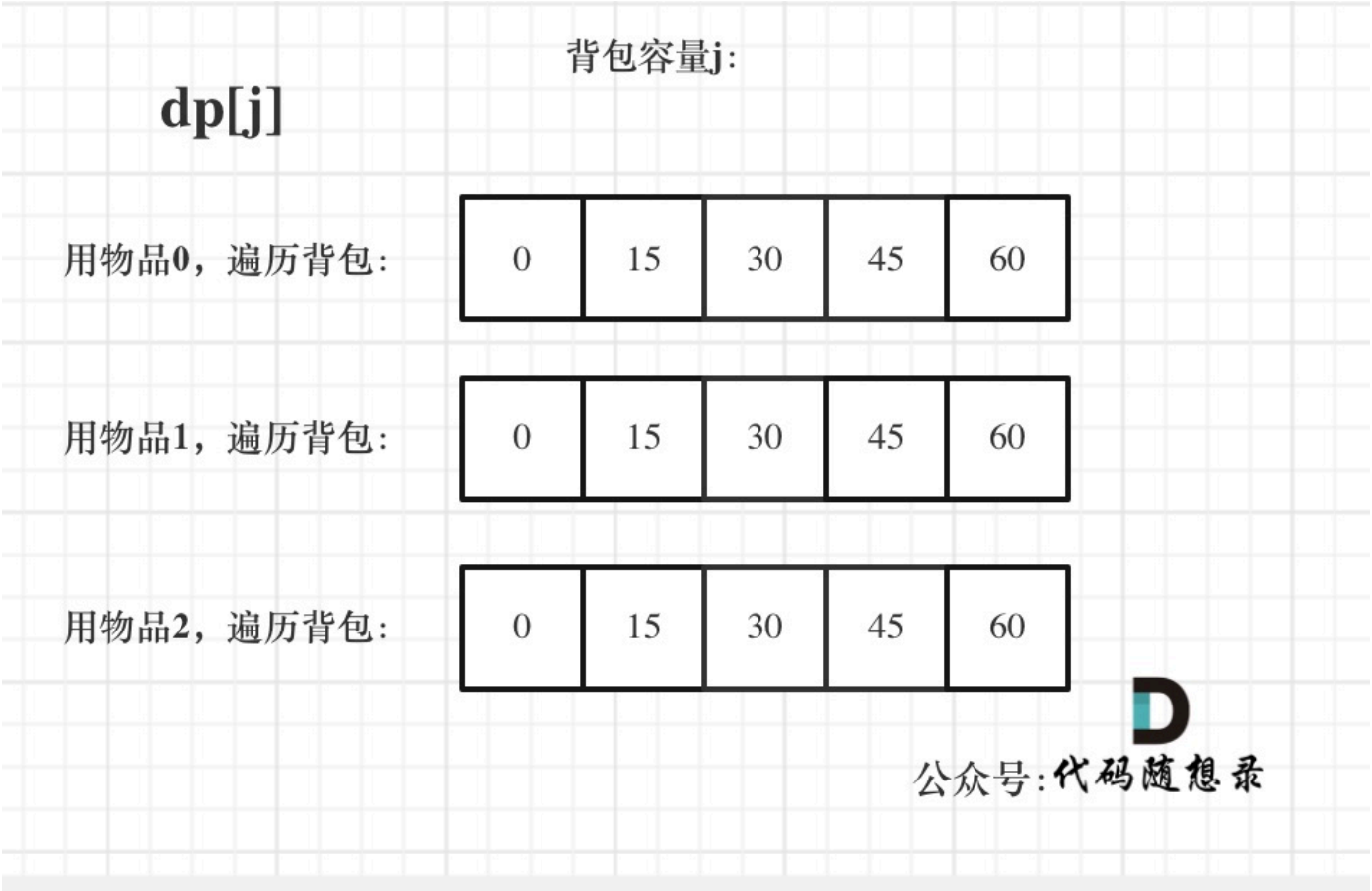
我们知道01背包内嵌的循环是从大到小遍历，为了保证每个物品仅被添加一次。

而完全背包的物品是可以添加多次的，所以要从小到大去遍历，即：

```
// 先遍历物品，再遍历背包
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = weight[i]; j <= bagWeight ; j++) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

    }
}
```

至于为什么，我在动态规划：关于01背包问题，你该了解这些！（滚动数组）中也做了讲解。

dp状态图如下：



相信很多同学看网上的文章，关于完全背包介绍基本就到为止了。

其实还有一个很重要的问题，为什么遍历物品在外层循环，遍历背包容量在内层循环?

这个问题很多题解关于这里都是轻描淡写就略过了，大家都默认 遍历物品在外层，遍历背包容量在内层，好像本应该如此一样，那么为什么呢?

难道就不能遍历背包容量在外层，遍历物品在内层?

看过这两篇的话:

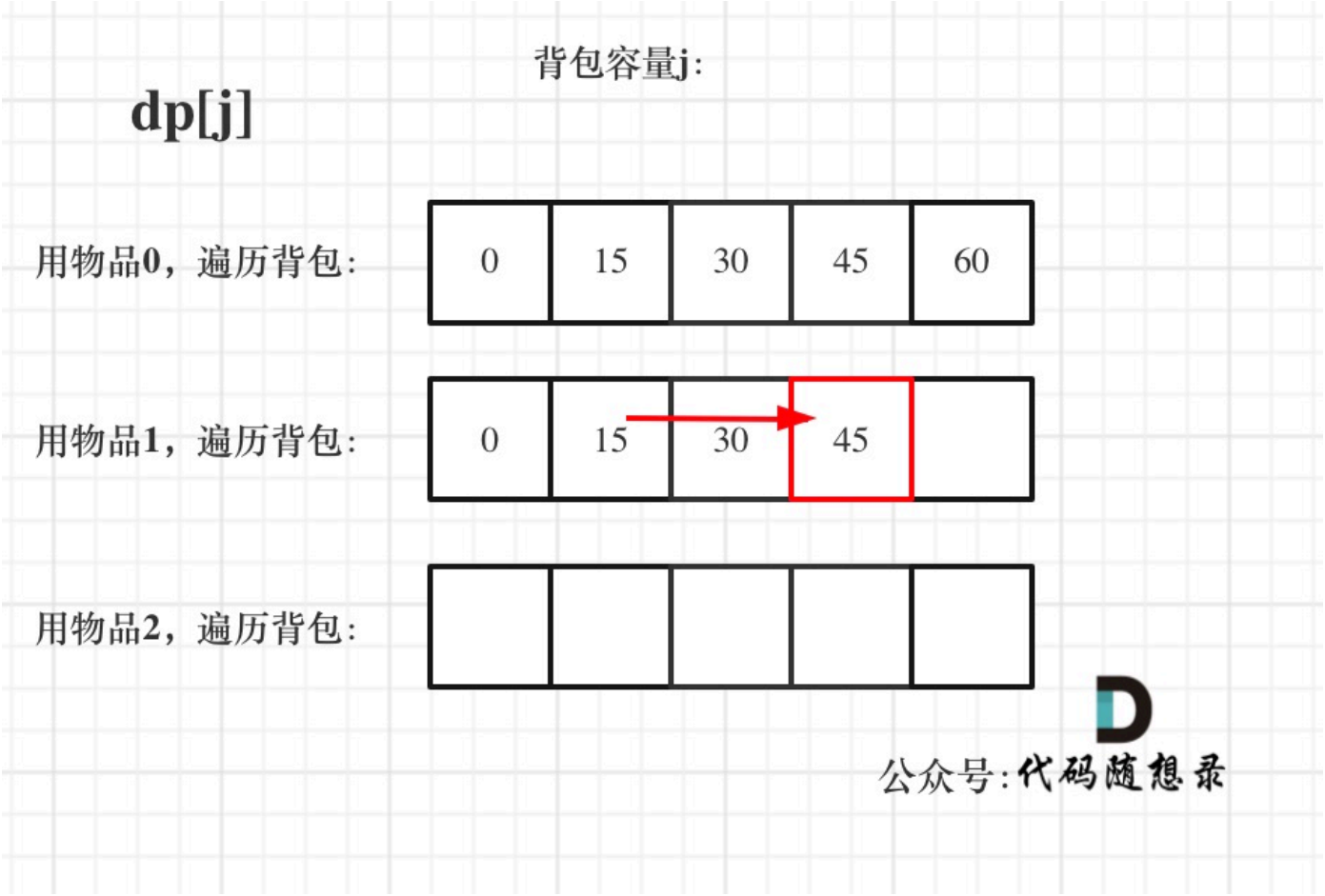
- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）

就知道了，01背包中二维dp数组的两个for遍历的先后循序是可以颠倒了，一维dp数组的两个for循环先后循序一定是先遍历物品，再遍历背包容量。

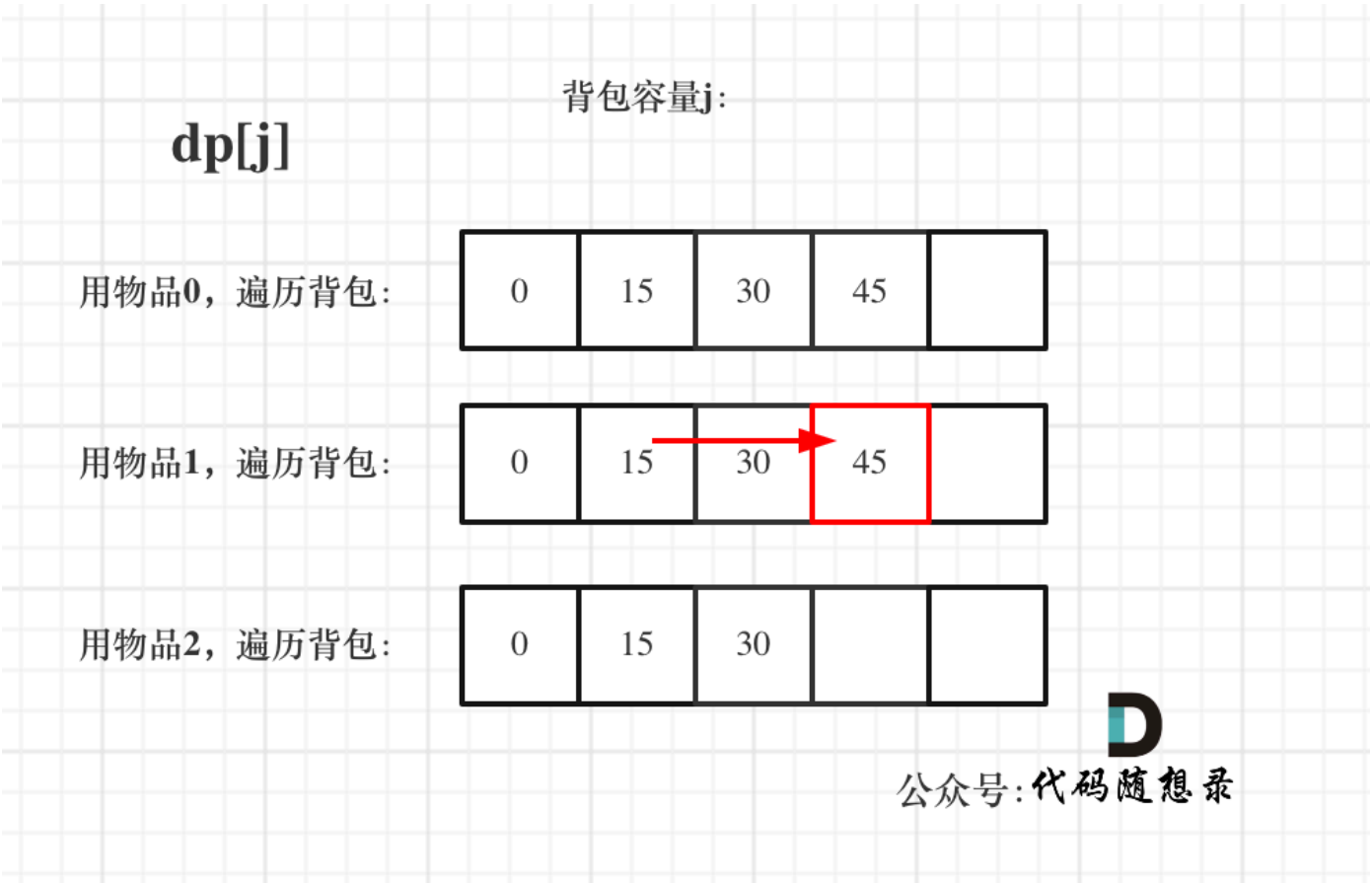
在完全背包中，对于一维dp数组来说，其实两个for循环嵌套顺序是无所谓的!

因为dp[j] 是根据 下标j之前所对应的dp[j]计算出来的。 只要保证下标j之前的dp[j]都是经过计算的就可以了。

遍历物品在外层循环，遍历背包容量在内层循环，状态如图:



遍历背包容量在外层循环，遍历物品在内层循环，状态如图:



看了这两个图，大家就会理解，完全背包中，两个for循环的先后循序，都不影响计算dp[j]所需要的值（这个值就是下标j之前所对应的dp[j]）。

先遍历背包在遍历物品，代码如下：

```
// 先遍历背包，再遍历物品
for(int j = 0; j <= bagWeight; j++) { // 遍历背包容量
    for(int i = 0; i < weight.size(); i++) { // 遍历物品
        if (j - weight[i] >= 0) dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
    cout << endl;
}
```

完整的C++测试代码如下：

```
// 先遍历物品，在遍历背包
void test_CompletePack() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    int bagWeight = 4;
    vector<int> dp(bagWeight + 1, 0);
    for(int i = 0; i < weight.size(); i++) { // 遍历物品
        for(int j = weight[i]; j <= bagWeight; j++) { // 遍历背包容量
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    cout << dp[bagWeight] << endl;
}
int main() {
    test_CompletePack();
}
```

```
// 先遍历背包，再遍历物品
void test_CompletePack() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    int bagWeight = 4;

    vector<int> dp(bagWeight + 1, 0);

    for(int j = 0; j <= bagWeight; j++) { // 遍历背包容量
        for(int i = 0; i < weight.size(); i++) { // 遍历物品
            if (j - weight[i] >= 0) dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    cout << dp[bagWeight] << endl;
}
```

```
int main() {
    test_CompletePack();
}
```

总结

细心的同学可能发现，**全文我说的都是对于纯完全背包问题，其for循环的先后循环是可以颠倒的！**

但如果题目稍稍有点变化，就会体现在遍历顺序上。

如果问装满背包有几种方式的话？ 那么两个for循环的先后顺序就有很大区别了，而leetcode上的题目都是这种稍有变化的类型。

这个区别，我将在后面讲解具体leetcode题目中给大家介绍，因为这块如果不结合具题目，单纯的介绍原理估计很多同学会越看越懵！

别急，下一篇就是了！哈哈

最后，**又可以出一道面试题了，就是纯完全背包，要求先用二维dp数组实现，然后再用一维dp数组实现，最后再问，两个for循环的先后是否可以颠倒？为什么？**
这个简单的完全背包问题，估计就可以难住不少候选人了。

19.零钱兑换II

力扣题目链接

给定不同面额的硬币和一个总金额。写出函数来计算可以凑成总金额的硬币组合数。假设每一种面额的硬币有无限个。

示例 1:

- 输入: amount = 5, coins = [1, 2, 5]
- 输出: 4

解释: 有四种方式可以凑成总金额:

- 5=5
- 5=2+2+1
- 5=2+1+1+1
- 5=1+1+1+1+1

示例 2:

- 输入: amount = 3, coins = [2]
- 输出: 0
- 解释: 只用面额2的硬币不能凑成总金额3。

示例 3:

- 输入: amount = 10, coins = [10]
- 输出: 1

注意，你可以假设：

- 0 <= amount (总金额) <= 5000
- 1 <= coin (硬币面额) <= 5000
- 硬币种类不超过 500 种
- 结果符合 32 位符号整数

算法公开课

《代码随想录》算法视频公开课：装满背包有多少种方法？组合与排列有讲究！ | LeetCode：518.零钱兑换II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这是一道典型的背包问题，一看到钱币数量不限，就知道这是一个完全背包。

对完全背包还不了解的同学，可以看这篇：动态规划：关于完全背包，你该了解这些！

但本题和纯完全背包不一样，**纯完全背包是凑成背包最大价值是多少，而本题是要求凑成总金额的物品组合个数！**

注意题目描述中是凑成总金额的硬币组合数，为什么强调是组合数呢?

例如示例一：

5 = 2 + 2 + 1

5 = 2 + 1 + 2

这是一种组合，都是 2 2 1。

如果问的是排列数，那么上面就是两种排列了。

组合不强调元素之间的顺序，排列强调元素之间的顺序。 其实这一点我们在讲解回溯算法专题的时候就讲过了哈。

那我为什么要介绍这些呢，因为这和下文讲解遍历顺序息息相关!

回归本题，动规五步曲来分析如下：

1. 确定dp数组以及下标的含义

dp[j]：凑成总金额j的货币组合数为dp[j]

2. 确定递推公式

dp[j] 就是所有的dp[j - coins[i]]（考虑coins[i]的情况）相加。

所以递推公式：dp[j] += dp[j - coins[i]];

这个递推公式大家应该不陌生了，我在讲解01背包题目的时候在这篇494. 目标和中就讲解了，求装满背包有几种方法，公式都是：dp[j] += dp[j - nums[i]];

3. dp数组如何初始化

首先dp[0]一定要为1，dp[0] = 1是 递归公式的基础。如果dp[0] = 0 的话，后面所有推导出来的值都是0了。

那么 dp[0] = 1 有没有含义，其实既可以说 凑成总金额0的货币组合数为1，也可以说 凑成总金额0的货币组合数为0，好像都没有毛病。

但题目描述中，也没明确说 amount = 0 的情况，结果应该是多少。

这里我认为题目描述还是要说明一下，因为后台测试数据是默认，amount = 0 的情况，组合数为1的。

下标非0的dp[j]初始化为0，这样累计加dp[j - coins[i]]的时候才不会影响真正的dp[j]

dp[0]=1还说明了一种情况：如果正好选了coins[i]后，也就是j-coins[i] == 0的情况表示这个硬币刚好能选，此时dp[0]为1表示只选coins[i]存在这样的一种选法。

4. 确定遍历顺序

本题中我们是外层for循环遍历物品（钱币），内层for遍历背包（金钱总额），还是外层for遍历背包（金钱总额），内层for循环遍历物品（钱币）呢?

我在动态规划：关于完全背包，你该了解这些！中讲解了完全背包的两个for循环的先后顺序都是可以的。

但本题就不行了！

因为纯完全背包求得装满背包的最大价值是多少，和凑成总和的元素有没有顺序没关系，即：有顺序也行，没有顺序也行！

而本题要求凑成总和的组合数，元素之间明确要求没有顺序。

所以纯完全背包是能凑成总和就行，不用管怎么凑的。

本题是求凑出来的方案个数，且每个方案个数是为组合数。

那么本题，两个for循环的先后顺序可就有说法了。

我们先来看 外层for循环遍历物品（钱币），内层for遍历背包（金钱总额）的情况。

代码如下：

```
for (int i = 0; i < coins.size(); i++) { // 遍历物品
    for (int j = coins[i]; j <= amount; j++) { // 遍历背包容量
        dp[j] += dp[j - coins[i]];
    }
}
```

假设：coins[0] = 1，coins[1] = 5。

那么就是先把1加入计算，然后再把5加入计算，得到的方法数量只有{1, 5}这种情况。而不会出现{5, 1}的情况。

所以这种遍历顺序中dp[j]里计算的是组合数！

如果把两个for交换顺序，代码如下：

```
for (int j = 0; j <= amount; j++) { // 遍历背包容量
    for (int i = 0; i < coins.size(); i++) { // 遍历物品
        if (j - coins[i] >= 0) dp[j] += dp[j - coins[i]];
    }
}
```

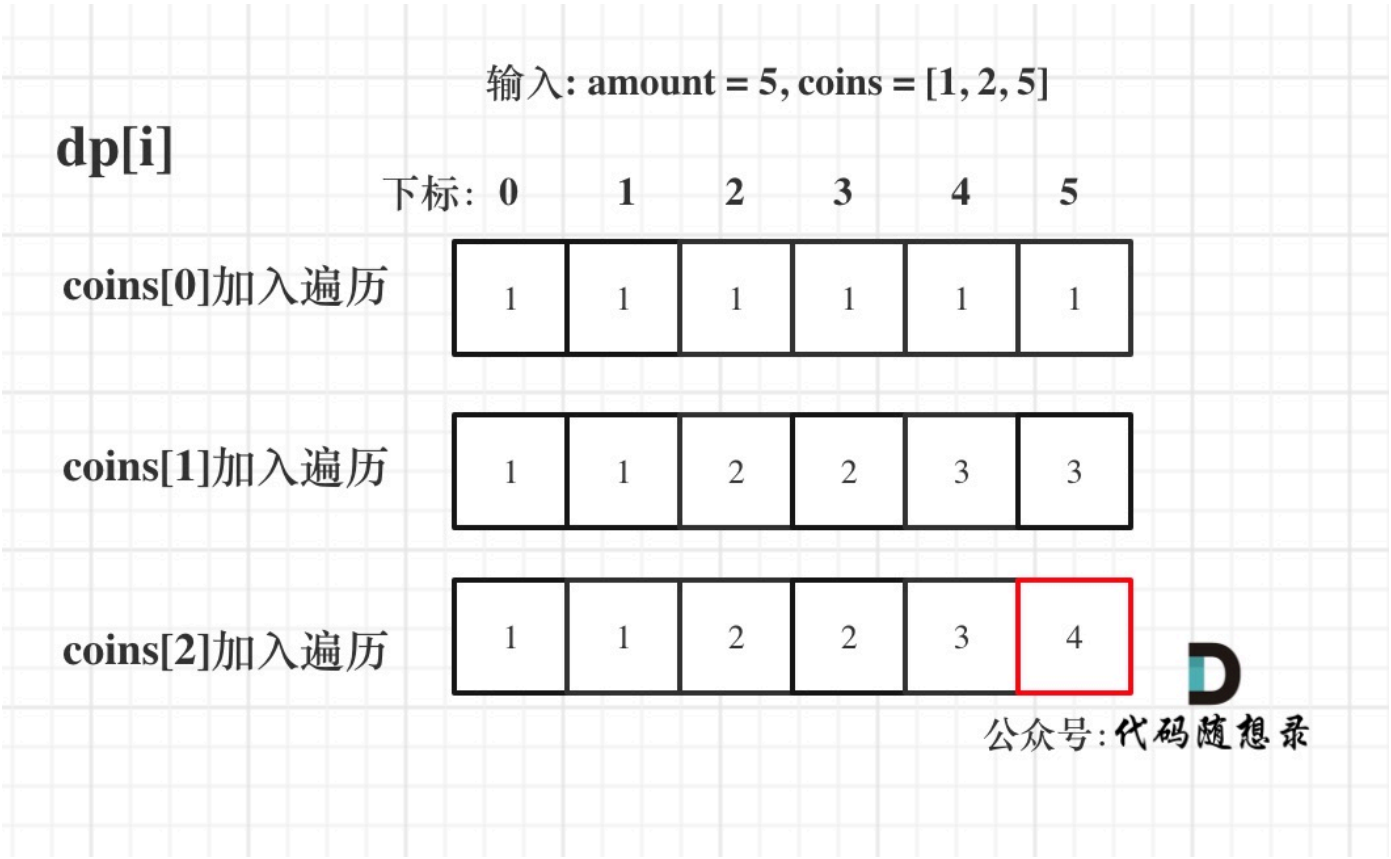
背包容量的每一个值，都是经过 1 和 5 的计算，包含了{1, 5} 和 {5, 1}两种情况。

此时dp[j]里算出来的就是排列数！

可能这里很多同学还不是很理解，**建议动手把这两种方案的dp数组数值变化打印出来，对比看一看！（实践出真知）**

5. 举例推导dp数组

输入: amount = 5, coins = [1, 2, 5] ，dp状态图如下：



最后红色框dp[amount]为最终结果。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int change(int amount, vector<int>& coins) {
        vector<int> dp(amount + 1, 0);
        dp[0] = 1;
        for (int i = 0; i < coins.size(); i++) { // 遍历物品
            for (int j = coins[i]; j <= amount; j++) { // 遍历背包
                dp[j] += dp[j - coins[i]];
            }
        }
        return dp[amount];
    }
};
```

- 时间复杂度: O(mn)，其中 m 是amount，n 是 coins 的长度
- 空间复杂度: O(m)

是不是发现代码如此精简，哈哈

总结

本题的递推公式，其实我们在494. 目标和中就已经讲过了，**而难点在于遍历顺序！**

在求装满背包有几种方案的时候，认清遍历顺序是非常关键的。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

可能说到排列数录友们已经有点懵了，后面Carl还会安排求排列数的题目，到时候在对比一下，大家就会发现神奇所在！

---

20. 本周小结！（动态规划系列四）

周一

动态规划：目标和！要求在数列之间加入+ 或者 -，使其和为S。

所有数的总和为sum，假设加法的总和为x，那么可以推出x = (S + sum) / 2。

S 和 sum都是固定的，那此时问题就转化为01背包问题（数列中的数只能使用一次）：给你一些物品（数字），装满背包（就是x）有几种方法。

1. 确定dp数组以及下标的含义

dp[j] 表示：填满j（包括j）这么大容积的包，有dp[j]种方法

2. 确定递推公式

dp[j] += dp[j - nums[i]]

注意：求装满背包有几种方法类似的题目，递推公式基本都是这样的。

3. dp数组如何初始化

dp[0] 初始化为1 ， dp[j]其他下标对应的数值应该初始化为0。

4. 确定遍历顺序

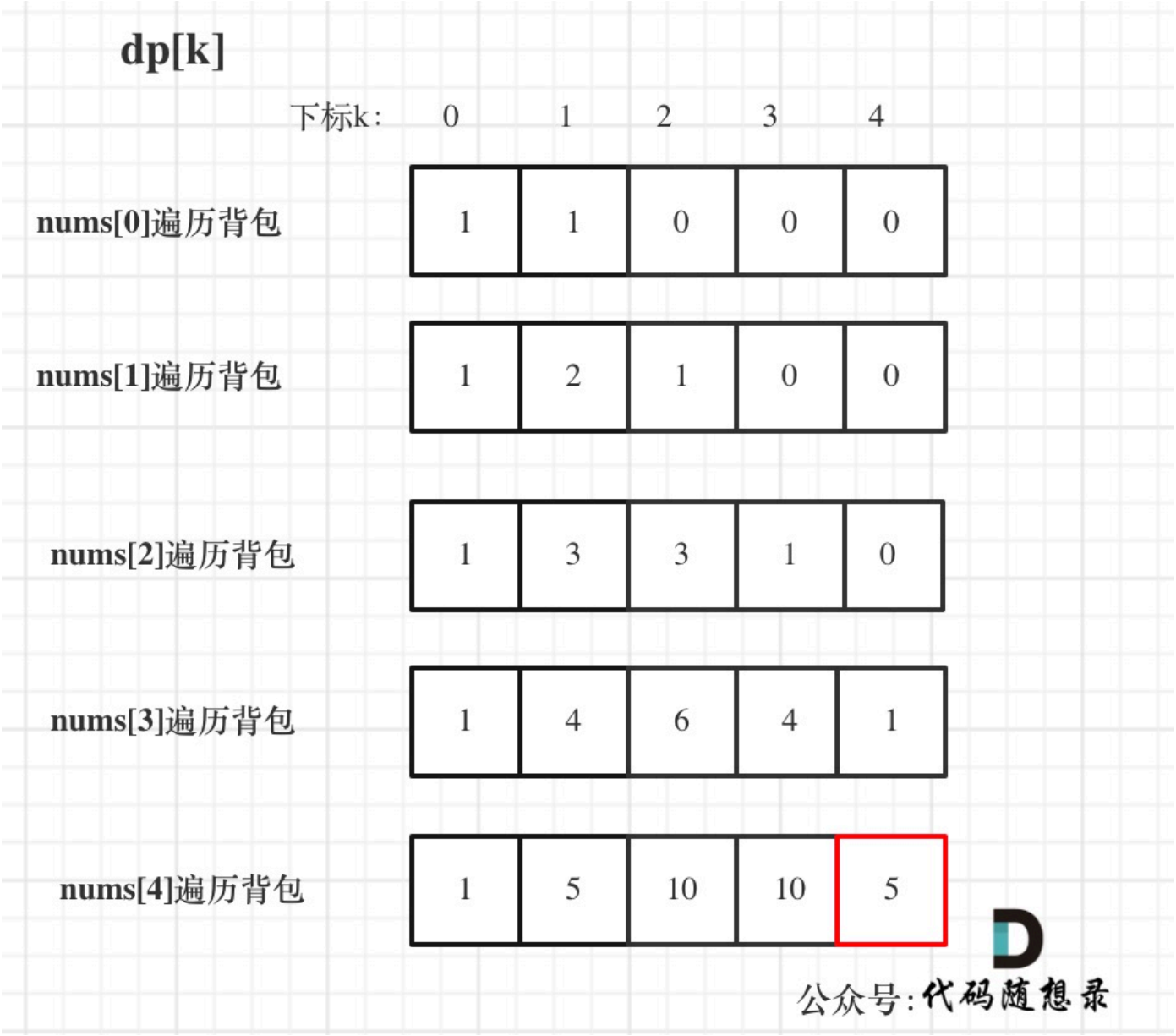
01背包问题一维dp的遍历，nums放在外循环，target在内循环，且内循环倒序。

5. 举例推导dp数组

输入：nums: [1, 1, 1, 1, 1], S: 3

bagSize = (S + sum) / 2 = (3 + 5) / 2 = 4

dp数组状态变化如下：



周二

这道题目动态规划：一和零！算有点难度。

不少同学都以为是多重背包，其实这是一道标准的01背包。

这不过这个背包有两个维度，一个是m 一个是n，而不同长度的字符串就是不同大小的待装物品。

所以这是一个二维01背包！

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：最多有i个0和j个1的strs的最大子集的大小为dp[i][j]。

2. 确定递推公式

dp[i][j] = max(dp[i][j], dp[i - zeroNum][j - oneNum] + 1);

字符串集合中的一个字符串0的数量为zeroNum，1的数量为oneNum。

3. dp数组如何初始化

因为物品价值不会是负数，初始为0，保证递推的时候dp[i][j]不会被初始值覆盖。

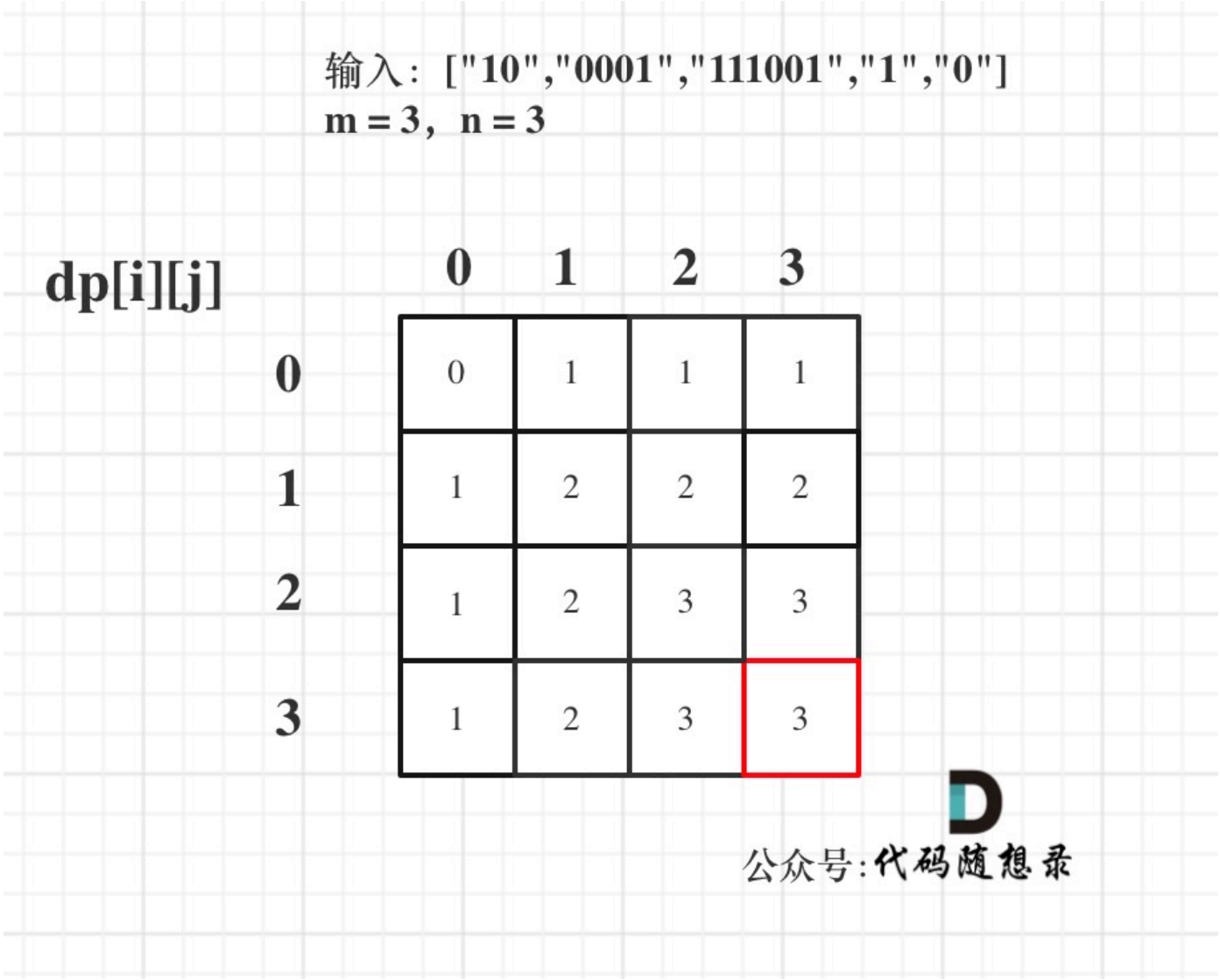
4. 确定遍历顺序

01背包一定是外层for循环遍历物品，内层for循环遍历背包容量且从后向前遍历！

5. 举例推导dp数组

以输入：["10","0001","111001","1","0"]，m = 3，n = 3为例

最后dp数组的状态如下所示：



周三

此时01背包我们就讲完了，正式开始完全背包。

在动态规划：关于完全背包，你该了解这些！中我们讲解了完全背包的理论基础。

其实完全背包和01背包区别就是完全背包的物品是无限数量。

递推公式也是一样的，但难点在于遍历顺序上！

完全背包的物品是可以添加多次的，所以遍历背包容量要从小到大去遍历，即：

```
// 先遍历物品，再遍历背包
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = weight[i]; j < bagWeight ; j++) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

    }
}
```

基本网上题的题解介绍到这里就到此为止了。

那么为什么要先遍历物品，在遍历背包呢？（灵魂拷问）

其实对于纯完全背包，先遍历物品，再遍历背包 与 先遍历背包，再遍历物品都是可以的。我在文中动态规划：关于完全背包，你该了解这些！也给出了详细的解释。

这个细节是很多同学忽略掉的点，其实也不算细节了，**相信不少同学在写背包的时候，两层for循环的先后循序搞不清楚，靠感觉来的。**

所以理解究竟是先遍历啥，后遍历啥非常重要，这也体现出遍历顺序的重要性！

在文中，我也强调了是对纯完全背包，两个for循环先后循序无所谓，那么题目稍有变化，可就有所谓了。

周四

在动态规划：给你一些零钱，你要怎么凑？中就是给你一堆零钱（零钱个数无限），为凑成amount的组合数有几种。

注意这里组合数和排列数的区别！

看到无限零钱个数就知道是完全背包，

但本题不是纯完全背包了（求是否能装满背包），而是求装满背包有几种方法。

这里在遍历顺序上可就有说法了。

- 如果求组合数就是外层for循环遍历物品，内层for遍历背包。
- 如果求排列数就是外层for遍历背包，内层for循环遍历物品。

这里同学们需要理解一波，我在文中也给出了详细的解释，下周我们将介绍求排列数的完全背包题目来加深对这个遍历顺序的理解。

总结

相信通过本周的学习，大家已经初步感受到遍历顺序的重要性！

很多对动规理解不深入的同学都会感觉：动规嘛，就是把递推公式推出来其他都easy了。

其实这是一种错觉，或者说对动规理解的不够深入！

我在动规专题开篇介绍关于动态规划，你该了解这些！中就强调了 **递推公式仅仅是 动规五部曲里的一小部分， dp数组的定义、初始化、遍历顺序，哪一点没有搞透的话，即使知道递推公式，遇到稍稍难一点的动规题目立刻会感觉写不出来了。**

此时相信大家对动规五部曲也有更深的理解了，同样也验证了Carl之前讲过的：**简单题是用来学习方法论的，而遇到难题才体现出方法论的重要性！**

---

21. 组合总和 IV

力扣题目链接

难度：中等

给定一个由正整数组成且不存在重复数字的数组，找出和为给定目标正整数的组合的个数。

示例:

- nums = [1, 2, 3]
- target = 4

所有可能的组合为：
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
(2, 1, 1)
(2, 2)
(3, 1)

请注意，顺序不同的序列被视作不同的组合。

因此输出为 7。

算法公开课

《代码随想录》算法视频公开课：装满背包有几种方法？求排列数？ | LeetCode：377.组合总和IV，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

对完全背包还不了解的同学，可以看这篇：动态规划：关于完全背包，你该了解这些！

本题题目描述说是求组合，但又说是可以元素相同顺序不同的组合算两个组合，**其实就是求排列！**

弄清什么是组合，什么是排列很重要。

组合不强调顺序，(1,5)和(5,1)是同一个组合。

排列强调顺序，(1,5)和(5,1)是两个不同的排列。

大家在公众号里学习回溯算法专题的时候，一定做过这两道题目回溯算法：39.组合总和和回溯算法：40.组合总和II会感觉这两题和本题很像！

但其本质是本题求的是排列总和，而且仅仅是求排列总和的个数，并不是把所有的排列都列出来。

如果本题要把排列都列出来的话，只能使用回溯算法爆搜。

动规五部曲分析如下：

1. 确定dp数组以及下标的含义

dp[i]: 凑成目标正整数为i的排列个数为dp[i]

2. 确定递推公式

dp[i]（考虑nums[j]）可以由 dp[i - nums[j]]（不考虑nums[j]） 推导出来。

因为只要得到nums[j]，排列个数dp[i - nums[j]]，就是dp[i]的一部分。

在动态规划：494.目标和 和 动态规划：518.零钱兑换II中我们已经讲过了，求装满背包有几种方法，递推公式一般都是dp[i] += dp[i - nums[j]];

本题也一样。

3. dp数组如何初始化

因为递推公式dp[i] += dp[i - nums[j]]的缘故，dp[0]要初始化为1，这样递归其他dp[i]的时候才会有数值基础。

至于dp[0] = 1 有没有意义呢?

其实没有意义，所以我也不去强行解释它的意义了，因为题目中也说了：给定目标值是正整数！ 所以dp[0] = 1是没有意义的，仅仅是为了推导递推公式。

至于非0下标的dp[i]应该初始为多少呢?

初始化为0，这样才不会影响dp[i]累加所有的dp[i - nums[j]]。

4. 确定遍历顺序

个数可以不限使用，说明这是一个完全背包。

得到的集合是排列，说明需要考虑元素之间的顺序。

本题要求的是排列，那么这个for循环嵌套的顺序可以有说法了。

在动态规划：518.零钱兑换II 中就已经讲过了。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

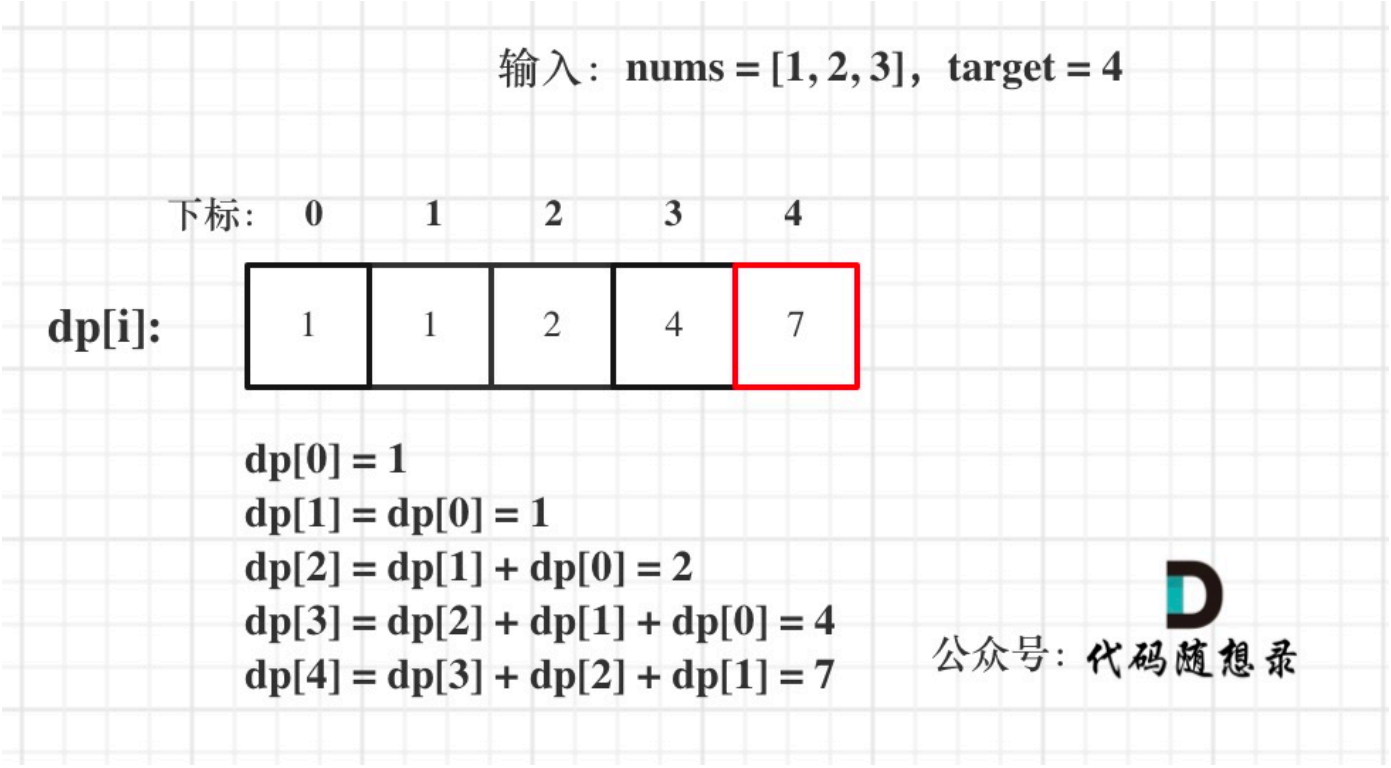
如果求排列数就是外层for遍历背包，内层for循环遍历物品。

如果把遍历nums（物品）放在外循环，遍历target的作为内循环的话，举一个例子：计算dp[4]的时候，结果集只有 {1,3} 这样的集合，不会有{3,1}这样的集合，因为nums遍历放在外层，3只能出现在1后面！

所以本题遍历顺序最终遍历顺序：**target（背包）放在外循环，将nums（物品）放在内循环，内循环从前到后遍历。**

5. 举例来推导dp数组

我们再来用示例中的例子推导一下：



如果代码运行处的结果不是想要的结果，就把dp[i]都打出来，看看和我们推导的一不一样。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        vector<int> dp(target + 1, 0);
        dp[0] = 1;
        for (int i = 0; i <= target; i++) { // 遍历背包
            for (int j = 0; j < nums.size(); j++) { // 遍历物品
                if (i - nums[j] >= 0 && dp[i] < INT_MAX - dp[i - nums[j]]) {
                    dp[i] += dp[i - nums[j]];
                }
            }
        }
        return dp[target];
    }
};
```

- 时间复杂度: O(target * n)，其中 n 为 nums 的长度
- 空间复杂度: O(target)

C++测试用例有两个数相加超过int的数据，所以需要在if里加上dp[i] < INT_MAX - dp[i - num]。

但java就不用考虑这个限制，java里的int也是四个字节吧，也有可能leetcode后台对不同语言的测试数据不一样。

总结

求装满背包有几种方法，递归公式都是一样的，没有什么差别，但关键在于遍历顺序！

本题与动态规划：518.零钱兑换II就是一个鲜明的对比，一个是求排列，一个是求组合，遍历顺序完全不同。

如果对遍历顺序没有深度理解的话，做这种完全背包的题目会很懵逼，即使题目刷过了可能也不太清楚具体是怎么过的。

此时大家应该对动态规划中的遍历顺序又有更深的理解了。

---

22. 爬楼梯(进阶版)

力扣题目链接

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢?

注意：给定 n 是一个正整数。

示例 1：
输入： 2
输出： 2
解释： 有两种方法可以爬到楼顶。

1. 1 阶 + 1 阶
2. 2 阶

示例 2：
输入： 3
输出： 3
解释： 有三种方法可以爬到楼顶。

1. 1 阶 + 1 阶 + 1 阶
2. 1 阶 + 2 阶
3. 2 阶 + 1 阶

思路

之前讲这道题目的时候，因为还没有讲背包问题，所以就只是讲了一下爬楼梯最直接的动规方法（斐波那契）。

这次终于讲到了背包问题，我选择带录友们再爬一次楼梯！

这道题目 我们在动态规划：爬楼梯 中已经讲过一次了，原题其实是一道简单动规的题目。

既然这么简单为什么还要讲呢，其实本题稍加改动就是一道面试好题。

改为：一步一个台阶，两个台阶，三个台阶，.....，直到 m个台阶。问有多少种不同的方法可以爬到楼顶呢?

1阶，2阶，.... m阶就是物品，楼顶就是背包。

每一阶可以重复使用，例如跳了1阶，还可以继续跳1阶。

问跳到楼顶有几种方法其实就是问装满背包有几种方法。

此时大家应该发现这就是一个完全背包问题了！

和昨天的题目动态规划：377. 组合总和 IV基本就是一道题了。

动规五部曲分析如下：

1. 确定dp数组以及下标的含义

dp[i]：爬到有i个台阶的楼顶，有dp[i]种方法。

2. 确定递推公式

在动态规划：494.目标和 、 动态规划：518.零钱兑换II、动态规划：377. 组合总和 IV中我们都讲过了，求装满背包有几种方法，递推公式一般都是dp[i] += dp[i - nums[j]];

本题呢，dp[i]有几种来源，dp[i - 1]，dp[i - 2]，dp[i - 3] 等等，即：dp[i - j]

那么递推公式为：dp[i] += dp[i - j]

3. dp数组如何初始化

既然递归公式是 dp[i] += dp[i - j]，那么dp[0] 一定为1，dp[0]是递归中一切数值的基础所在，如果dp[0]是0的话，其他数值都是0了。

下标非0的dp[i]初始化为0，因为dp[i]是靠dp[i-j]累计上来的，dp[i]本身为0这样才不会影响结果

4. 确定遍历顺序

这是背包里求排列问题，即：**1、2 步 和 2、1 步都是上三个台阶，但是这两种方法不一样！**

所以需将target放在外循环，将nums放在内循环。

每一步可以走多次，这是完全背包，内循环需要从前向后遍历。

5. 举例来推导dp数组

介于本题和动态规划：377. 组合总和 IV几乎是一样的，这里我就不再重复举例了。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1;
        for (int i = 1; i <= n; i++) { // 遍历背包
            for (int j = 1; j <= m; j++) { // 遍历物品
                if (i - j >= 0) dp[i] += dp[i - j];
            }
        }
        return dp[n];
    }
};
```

- 时间复杂度: O(nm)
- 空间复杂度: O(n)

代码中m表示最多可以爬m个台阶，代码中把m改成2就是本题70.爬楼梯可以AC的代码了。

总结

本题看起来是一道简单题目，稍稍进阶一下其实就是一个完全背包！

如果我来面试的话，我就会先给候选人出一个 本题原题，看其表现，如果顺利写出来，进而在要求每次可以爬[1 - m]个台阶应该怎么写。

顺便再考察一下两个for循环的嵌套顺序，为什么target放外面，nums放里面。

这就能考察对背包问题本质的掌握程度，候选人是不是刷题背公式，一眼就看出来了。

这么一连套下来，如果候选人都能答出来，相信任何一位面试官都是非常满意的。

本题代码不长，题目也很普通，但稍稍一进阶就可以考察完全背包，而且题目进阶的内容在leetcode上并没有原题，一定程度上就可以排除掉刷题党了，简直是面试题目的绝佳选择！

---

23. 零钱兑换

[力扣题目链接]

给定不同面额的硬币 coins 和一个总金额 amount。编写一个函数来计算可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1。

你可以认为每种硬币的数量是无限的。

示例 1：

- 输入：coins = [1, 2, 5], amount = 11
- 输出：3
- 解释：11 = 5 + 5 + 1

示例 2：

- 输入：coins = [2], amount = 3
- 输出：-1

示例 3：

- 输入：coins = [1], amount = 0
- 输出：0

示例 4：

- 输入：coins = [1], amount = 1

- 输出：1

示例 5：

- 输入：coins = [1], amount = 2
- 输出：2

提示：

- 1 <= coins.length <= 12
- 1 <= coins[i] <= 2^31 - 1
- 0 <= amount <= 10^4

算法公开课

《代码随想录》算法视频公开课：装满背包最少的物品件数是多少？| LeetCode：322.零钱兑换，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

在动态规划：518.零钱兑换II中我们已经兑换一次零钱了，这次又要兑换，套路不一样！

题目中说每种硬币的数量是无限的，可以看出是典型的完全背包问题。

动规五部曲分析如下：

1. 确定dp数组以及下标的含义

dp[j]：凑足总额为j所需钱币的最少个数为dp[j]

2. 确定递推公式

凑足总额为j - coins[i]的最少个数为dp[j - coins[i]]，那么只需要加上一个钱币coins[i]即dp[j - coins[i]] + 1就是dp[j]（考虑coins[i]）

所以dp[j] 要取所有 dp[j - coins[i]] + 1 中最小的。

递推公式：dp[j] = min(dp[j - coins[i]] + 1, dp[j]);

3. dp数组如何初始化

首先凑足总金额为0所需钱币的个数一定是0，那么dp[0] = 0;

其他下标对应的数值呢?

考虑到递推公式的特性，dp[j]必须初始化为一个最大的数，否则就会在min(dp[j - coins[i]] + 1, dp[j])比较的过程中被初始值覆盖。

所以下标非0的元素都是应该是最大值。

代码如下：

```
vector<int> dp(amount + 1, INT_MAX);
dp[0] = 0;
```

4. 确定遍历顺序

本题求钱币最小个数，**那么钱币有顺序和没有顺序都可以，都不影响钱币的最小个数。**

所以本题并不强调集合是组合还是排列。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

在动态规划专题我们讲过了求组合数是动态规划：518.零钱兑换II，求排列数是动态规划：377. 组合总和 IV。

所以本题的两个for循环的关系是：外层for循环遍历物品，内层for遍历背包或者外层for遍历背包，内层for循环遍历物品都是可以的！

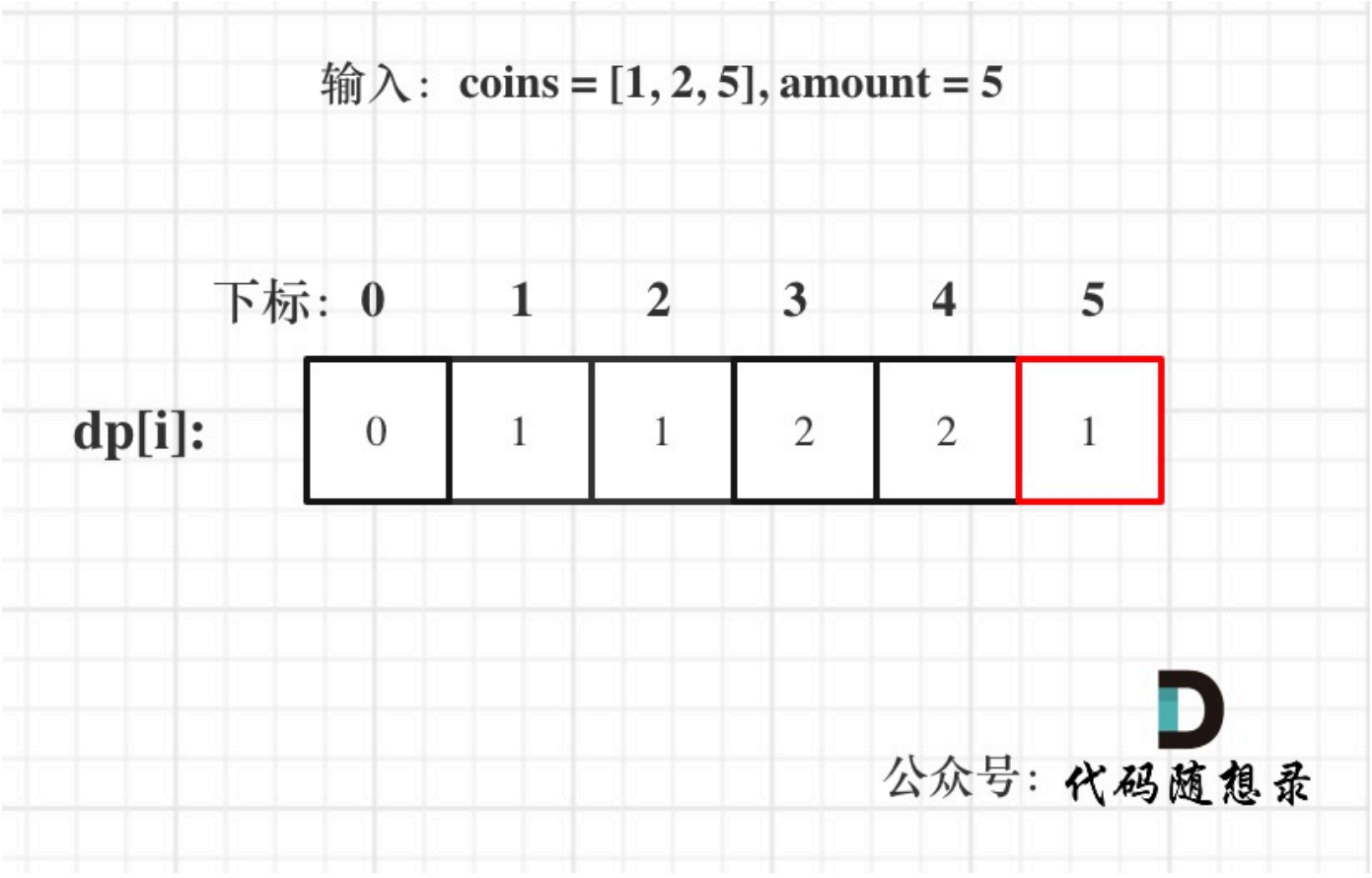
那么我采用coins放在外循环，target在内循环的方式。

本题钱币数量可以无限使用，那么是完全背包。所以遍历的内循环是正序

综上所述，遍历顺序为：coins（物品）放在外循环，target（背包）在内循环。且内循环正序。

5. 举例推导dp数组

以输入：coins = [1, 2, 5], amount = 5为例



dp[amount]为最终结果。

以上分析完毕，C++ 代码如下：

```
// 版本一
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i < coins.size(); i++) { // 遍历物品
            for (int j = coins[i]; j <= amount; j++) { // 遍历背包
                if (dp[j - coins[i]] != INT_MAX) { // 如果dp[j - coins[i]]是初始值则跳过
                    dp[j] = min(dp[j - coins[i]] + 1, dp[j]);
                }
            }
        }
        if (dp[amount] == INT_MAX) return -1;
        return dp[amount];
    }
};
```

- 时间复杂度: O(n * amount)，其中 n 为 coins 的长度
- 空间复杂度: O(amount)

对于遍历方式遍历背包放在外循环，遍历物品放在内循环也是可以的，我就直接给出代码了

```
// 版本二
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {  // 遍历背包
            for (int j = 0; j < coins.size(); j++) { // 遍历物品
                if (i - coins[j] >= 0 && dp[i - coins[j]] != INT_MAX ) {
                    dp[i] = min(dp[i - coins[j]] + 1, dp[i]);
                }
            }
        }
        if (dp[amount] == INT_MAX) return -1;
        return dp[amount];
    }
};
```

- 同上

总结

细心的同学看网上的题解，**可能看一篇是遍历背包的for循环放外面，看一篇又是遍历背包的for循环放里面，看多了都看晕了**，到底两个for循环应该是什么先后关系。

能把遍历顺序讲明白的文章几乎找不到！

这也是大多数同学学习动态规划的苦恼所在，有的时候递推公式很简单，难在遍历顺序上！

但最终又可以稀里糊涂的把题目过了，也不知道为什么这样可以过，反正就是过了，哈哈

那么这篇文章就把遍历顺序分析的清清楚楚。

动态规划：518.零钱兑换II中求的是组合数，动态规划：377. 组合总和 IV中求的是排列数。

而本题是要求最少硬币数量，硬币是组合数还是排列数都无所谓！所以两个for循环先后顺序怎样都可以！

这也是我为什么要先讲518.零钱兑换II 然后再讲本题即：322.零钱兑换，这是Carl的良苦用心那。

相信大家看完之后，对背包问题中的遍历顺序有更深的理解了。

24.完全平方数

力扣题目链接

给定正整数 n，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n。你需要让组成和的完全平方数的个数最少。

给你一个整数 n ，返回和为 n 的完全平方数的 最少数量 。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例 1：

- 输入：n = 12
- 输出：3
- 解释：12 = 4 + 4 + 4

示例 2：

- 输入：n = 13
- 输出：2
- 解释：13 = 4 + 9

提示：

- 1 <= n <= 10^4

算法公开课

《代码随想录》算法视频公开课：换汤不换药！ | LeetCode：279.完全平方数，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

可能刚看这种题感觉没啥思路，又平方和的，又最小数的。

我来把题目翻译一下：完全平方数就是物品（可以无限件使用），凑个正整数n就是背包，问凑满这个背包最少有多少物品?

感受出来了没，这么浓厚的完全背包氛围，而且和昨天的题目动态规划：322. 零钱兑换就是一样一样的！

动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[j]：和为j的完全平方数的最少数量为dp[j]

2. 确定递推公式

dp[j] 可以由dp[j - i * i]推出， dp[j - i * i] + 1 便可以凑成dp[j]。

此时我们要选择最小的dp[j]，所以递推公式：dp[j] = min(dp[j - i * i] + 1, dp[j]);

3. dp数组如何初始化

dp[0]表示 和为0的完全平方数的最小数量，那么dp[0]一定是0。

有同学问题，那0 * 0 也算是一种啊，为啥dp[0] 就是 0呢?

看题目描述，找到若干个完全平方数（比如 1, 4, 9, 16, ...），题目描述中可没说要从0开始，dp[0]=0完全是为了递推公式。

非0下标的dp[j]应该是多少呢?

从递归公式dp[j] = min(dp[j - i * i] + 1, dp[j]);中可以看出每次dp[j]都要选最小的，**所以非0下标的dp[j]一定要初始为最大值，这样dp[j]在递推的时候才不会被初始值覆盖**。

4. 确定遍历顺序

我们知道这是完全背包，

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

在动态规划：322. 零钱兑换中我们就深入探讨了这个问题，本题也是一样的，是求最小数！

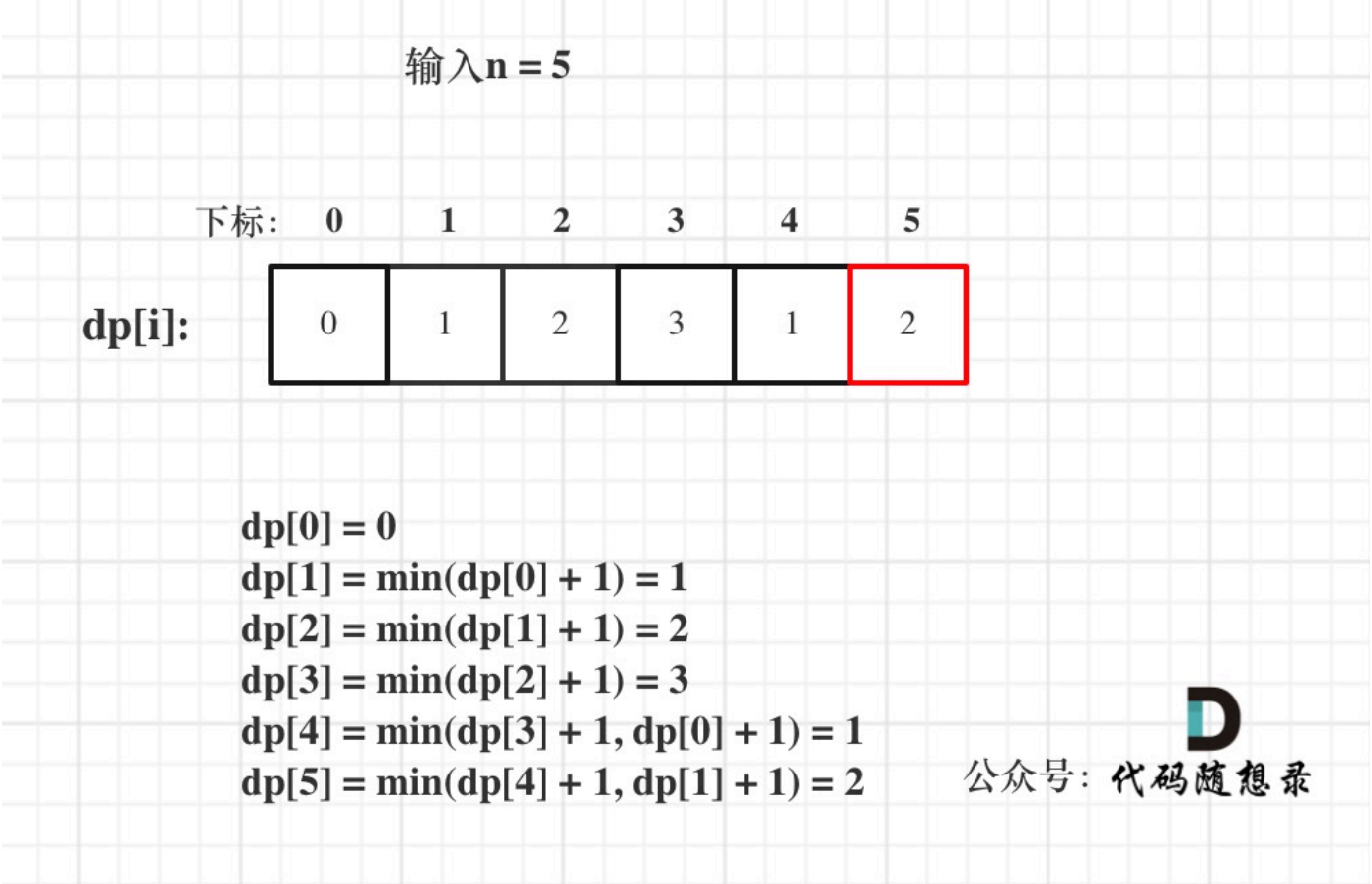
所以本题外层for遍历背包，内层for遍历物品，还是外层for遍历物品，内层for遍历背包，都是可以的！

我这里先给出外层遍历背包，内层遍历物品的代码：

```
vector<int> dp(n + 1, INT_MAX);
dp[0] = 0;
for (int i = 0; i <= n; i++) { // 遍历背包
    for (int j = 1; j * j <= i; j++) { // 遍历物品
        dp[i] = min(dp[i - j * j] + 1, dp[i]);
    }
}
```

5. 举例推导dp数组

已输入n为5例，dp状态图如下：



dp[0] = 0
dp[1] = min(dp[0] + 1) = 1
dp[2] = min(dp[1] + 1) = 2
dp[3] = min(dp[2] + 1) = 3
dp[4] = min(dp[3] + 1, dp[0] + 1) = 1
dp[5] = min(dp[4] + 1, dp[1] + 1) = 2

最后的dp[n]为最终结果。

以上动规五部曲分析完毕C++代码如下：

```
// 版本一
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i <= n; i++) { // 遍历背包
            for (int j = 1; j * j <= i; j++) { // 遍历物品
                dp[i] = min(dp[i - j * j] + 1, dp[i]);
            }
        }
        return dp[n];
    }
};
```

- 时间复杂度: O(n * √n)
- 空间复杂度: O(n)

同样我在给出先遍历物品，在遍历背包的代码，一样的可以AC的。

```
// 版本二
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 1; i * i <= n; i++) { // 遍历物品
            for (int j = i * i; j <= n; j++) { // 遍历背包
                dp[j] = min(dp[j - i * i] + 1, dp[j]);
            }
        }
        return dp[n];
    }
};
```

- 同上

总结

如果大家认真做了昨天的题目动态规划：322. 零钱兑换，今天这道就非常简单了，一样的套路一样的味道。

但如果没有按照「代码随想录」的题目顺序来做的话，做动态规划或者做背包问题，上来就做这道题，那还是挺难的！

经过前面的训练这道题已经是简单题了，哈哈哈

---

25. 本周小结！（动态规划系列五）

周一

动态规划：377. 组合总和 Ⅳ中给定一个由正整数组成且不存在重复数字的数组，找出和为给定目标正整数的组合的个数（顺序不同的序列被视作不同的组合）。

题目面试虽然是组合，但又强调顺序不同的序列被视作不同的组合，其实这道题目求的是排列数！

递归公式：dp[i] += dp[i - nums[j]];

这个和前上周讲的组合问题又不一样，关键就体现在遍历顺序上！

在动态规划：518.零钱兑换II 中就已经讲过了。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

如果把遍历nums（物品）放在外循环，遍历target的作为内循环的话，举一个例子：计算dp[4]的时候，结果集只有 {1,3} 这样的集合，不会有{3,1}这样的集合，因为nums遍历放在外层，3只能出现在1后面！

所以本题遍历顺序最终遍历顺序：**target（背包）放在外循环，将nums（物品）放在内循环，内循环从前到后遍历**。

```
class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        vector<int> dp(target + 1, 0);
        dp[0] = 1;
        for (int i = 0; i <= target; i++) { // 遍历背包
            for (int j = 0; j < nums.size(); j++) { // 遍历物品
                if (i - nums[j] >= 0 && dp[i] < INT_MAX - dp[i - nums[j]]) {
                    dp[i] += dp[i - nums[j]];
                }
            }
        }
        return dp[target];
    }
};
```

周二

爬楼梯之前我们已经做过了，就是斐波那契数列，很好解，但动态规划：70. 爬楼梯进阶版（完全背包）中我们进阶了一下。

改为：每次可以爬 1 、 2、.....、m 个台阶。问有多少种不同的方法可以爬到楼顶呢?

1阶，2阶，.... m阶就是物品，楼顶就是背包。

每一阶可以重复使用，例如跳了1阶，还可以继续跳1阶。

问跳到楼顶有几种方法其实就是问装满背包有几种方法。

此时大家应该发现这就是一个完全背包问题了！

和昨天的题目动态规划：377. 组合总和 IV基本就是一道题了，遍历顺序也是一样一样的！

代码如下：

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1;
        for (int i = 1; i <= n; i++) { // 遍历背包
            for (int j = 1; j <= m; j++) { // 遍历物品
                if (i - j >= 0) dp[i] += dp[i - j];
            }
        }
```

```
        return dp[n];
    }
};
```

代码中m表示最多可以爬m个台阶，代码中把m改成2就是本题70.爬楼梯可以AC的代码了。

周三

动态规划：322.零钱兑换给定不同面额的硬币 coins 和一个总金额 amount。编写一个函数来计算可以凑成总金额所需的最少的硬币个数（每种硬币的数量是无限的）。

这里我们都知道这是完全背包。

递归公式：dp[j] = min(dp[j - coins[i]] + 1, dp[j]);

关键看遍历顺序。

本题求钱币最小个数，**那么钱币有顺序和没有顺序都可以，都不影响钱币的最小个数**。

所以本题并不强调集合是组合还是排列。

那么本题的两个for循环的关系是：外层for循环遍历物品，内层for遍历背包或者外层for遍历背包，内层for循环遍历物品都是可以的！

外层for循环遍历物品，内层for遍历背包：

```
// 版本一
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i < coins.size(); i++) { // 遍历物品
            for (int j = coins[i]; j <= amount; j++) { // 遍历背包
                if (dp[j - coins[i]] != INT_MAX) { // 如果dp[j - coins[i]]是初始值则跳过
                    dp[j] = min(dp[j - coins[i]] + 1, dp[j]);
                }
            }
        }
        if (dp[amount] == INT_MAX) return -1;
        return dp[amount];
    }
};
```

外层for遍历背包，内层for循环遍历物品：

```
// 版本二
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
```

```
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {  // 遍历背包
            for (int j = 0; j < coins.size(); j++) { // 遍历物品
                if (i - coins[j] >= 0 && dp[i - coins[j]] != INT_MAX ) {
                    dp[i] = min(dp[i - coins[j]] + 1, dp[i]);
                }
            }
        }
        if (dp[amount] == INT_MAX) return -1;
        return dp[amount];
    }
};
```

周四

[动态规划：279.完全平方数](#)给定正整数 n，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n。你需要让组成和的完全平方数的个数最少（平方数可以重复使用）。

如果按顺序把前面的文章都看了，这道题目就是简单题了。 dp[i]的定义，递推公式，初始化，遍历顺序，都是和[动态规划：322. 零钱兑换](#) 一样一样的。

要是没有前面的基础上来做这道题，那这道题目就有点难度了。

这也体现了刷题顺序的重要性。

先遍历背包，再遍历物品：

```
// 版本一
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i <= n; i++) { // 遍历背包
            for (int j = 1; j * j <= i; j++) { // 遍历物品
                dp[i] = min(dp[i - j * j] + 1, dp[i]);
            }
        }
        return dp[n];
    }
};
```

先遍历物品，再遍历背包：

```
// 版本二
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, INT_MAX);
```

```
        dp[0] = 0;
        for (int i = 1; i * i <= n; i++) { // 遍历物品
            for (int j = 1; j <= n; j++) { // 遍历背包
                if (j - i * i >= 0) {
                    dp[j] = min(dp[j - i * i] + 1, dp[j]);
                }
            }
        }
        return dp[n];
    }
};
```

总结

本周的主题其实就是背包问题中的遍历顺序！

我这里做一下总结：

求组合数：动态规划：518.零钱兑换II
求排列数：动态规划：377. 组合总和 IV、动态规划：70. 爬楼梯进阶版（完全背包）
求最小数：动态规划：322. 零钱兑换、动态规划：279.完全平方数

此时我们就已经把完全背包的遍历顺序研究的透透的了！

26.单词拆分

力扣题目链接

给定一个非空字符串 s 和一个包含非空单词的列表 wordDict，判定 s 是否可以被空格拆分为一个或多个在字典中出现的单词。

说明：

拆分时可以重复使用字典中的单词。

你可以假设字典中没有重复的单词。

示例 1：

- 输入: s = "leetcode", wordDict = ["leet", "code"]
- 输出: true
- 解释: 返回 true 因为 "leetcode" 可以被拆分成 "leet code"。

示例 2：

- 输入: s = "applepenapple", wordDict = ["apple", "pen"]
- 输出: true
- 解释: 返回 true 因为 "applepenapple" 可以被拆分成 "apple pen apple"。

- 注意你可以重复使用字典中的单词。

示例 3：

- 输入: s = "catsandog", wordDict = ["cats", "dog", "sand", "and", "cat"]
- 输出: false

算法公开课

《代码随想录》算法视频公开课：你的背包如何装满？ | LeetCode：139.单词拆分，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

看到这道题目的时候，大家应该回想起我们之前讲解回溯法专题的时候，讲过的一道题目回溯算法：分割回文串，就是枚举字符串的所有分割情况。

回溯算法：分割回文串：是枚举分割后的所有子串，判断是否回文。

本道是枚举分割所有字符串，判断是否在字典里出现过。

那么这里我也给出回溯法C++代码：

```
class Solution {
private:
    bool backtracking (const string& s, const unordered_set<string>& wordSet, int
startIndex) {
        if (startIndex >= s.size()) {
            return true;
        }
        for (int i = startIndex; i < s.size(); i++) {
            string word = s.substr(startIndex, i - startIndex + 1);
            if (wordSet.find(word) != wordSet.end() && backtracking(s, wordSet, i + 1))
{
                return true;
            }
        }
        return false;
    }
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
        return backtracking(s, wordSet, 0);
    }
};
```

- 时间复杂度：O(2^n)，因为每一个单词都有两个状态，切割和不切割
- 空间复杂度：O(n)，算法递归系统调用栈的空间

那么以上代码很明显要超时了，超时的数据如下：

```
"aaaa
```

背包问题

单词就是物品，字符串s就是背包，单词能否组成字符串s，就是问物品能不能把背包装满。

拆分时可以重复使用字典中的单词，说明就是一个完全背包！

动规五部曲分析如下：

1. 确定dp数组以及下标的含义

dp[i]：字符串长度为i的话，dp[i]为true，表示可以拆分为一个或多个在字典中出现的单词。

2. 确定递推公式

如果确定dp[j] 是true，且 [j, i] 这个区间的子串出现在字典里，那么dp[i]一定是true。（j < i ）。

所以递推公式是 if([j, i] 这个区间的子串出现在字典里 && dp[j]是true) 那么 dp[i] = true。

3. dp数组如何初始化

从递推公式中可以看出，dp[i] 的状态依靠 dp[j]是否为true，那么dp[0]就是递推的根基，dp[0]一定要为true，否则递推下去后面都都是false了。

那么dp[0]有没有意义呢?

dp[0]表示如果字符串为空的话，说明出现在字典里。

但题目中说了“给定一个非空字符串 s” 所以测试数据中不会出现i为0的情况，那么dp[0]初始为true完全就是为了推导公式。

下标非0的dp[i]初始化为false，只要没有被覆盖说明都是不可拆分为一个或多个在字典中出现的单词。

4. 确定遍历顺序

题目中说是拆分为一个或多个在字典中出现的单词，所以这是完全背包。

还要讨论两层for循环的前后顺序。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

我在这里做一个总结：

求组合数：动态规划：518.零钱兑换II
求排列数：动态规划：377. 组合总和 IV、动态规划：70. 爬楼梯进阶版（完全背包）
求最小数：动态规划：322. 零钱兑换、动态规划：279.完全平方数

而本题其实我们求的是排列数，为什么呢。 拿 s = "applepenapple", wordDict = ["apple", "pen"] 举例。

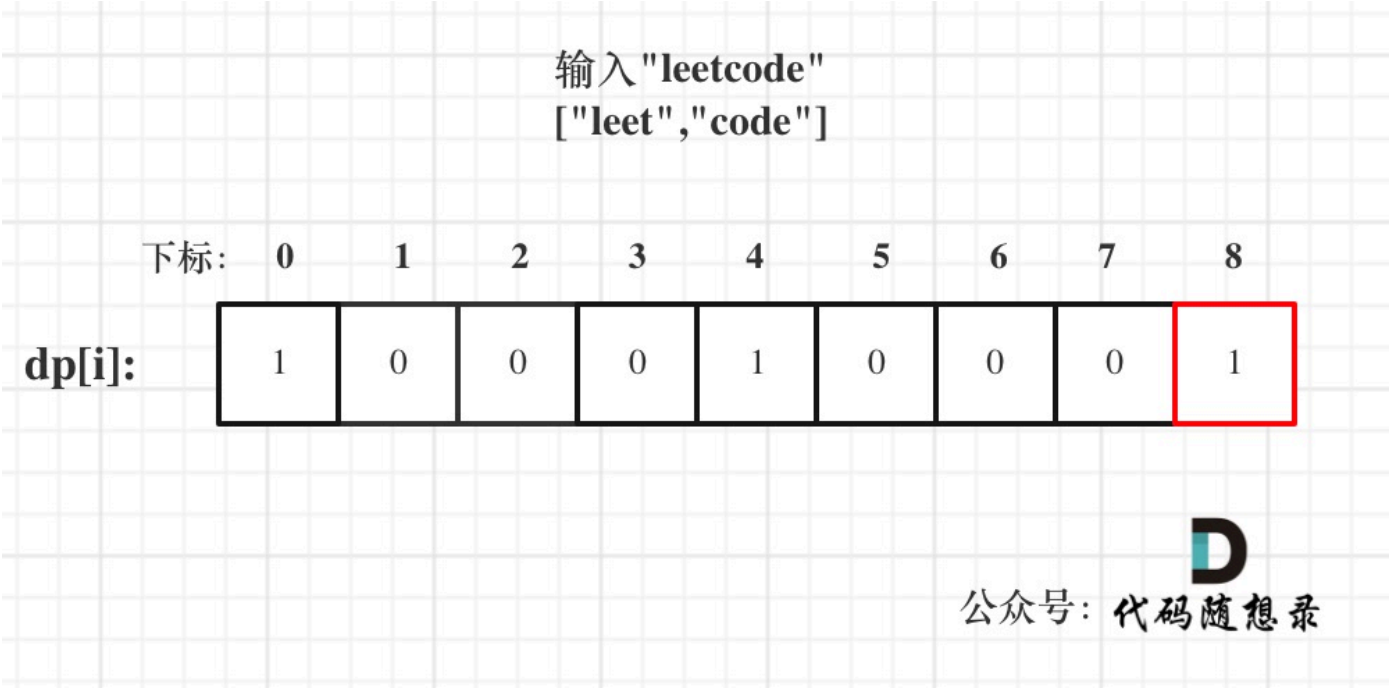
"apple", "pen" 是物品，那么我们要求 物品的组合一定是 "apple" + "pen" + "apple" 才能组成 "applepenapple"。

"apple" + "apple" + "pen" 或者 "pen" + "apple" + "apple" 是不可以的，那么我们就是强调物品之间顺序。

所以说，本题一定是 先遍历 背包，再遍历物品。

5. 举例推导dp[i]

以输入: s = "leetcode", wordDict = ["leet", "code"]为例，dp状态如图：



dp[s.size()]就是最终结果。

动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
        vector<bool> dp(s.size() + 1, false);
        dp[0] = true;
        for (int i = 1; i <= s.size(); i++) {   // 遍历背包
            for (int j = 0; j < i; j++) {       // 遍历物品
                string word = s.substr(j, i - j); //substr(起始位置，截取的个数)
                if (wordSet.find(word) != wordSet.end() && dp[j]) {
                    dp[i] = true;
                }
            }
        }
        return dp[s.size()];
    }
};
```

- 时间复杂度：O(n^3)，因为substr返回子串的副本是O(n)的复杂度（这里的n是substring的长度）
- 空间复杂度：O(n)

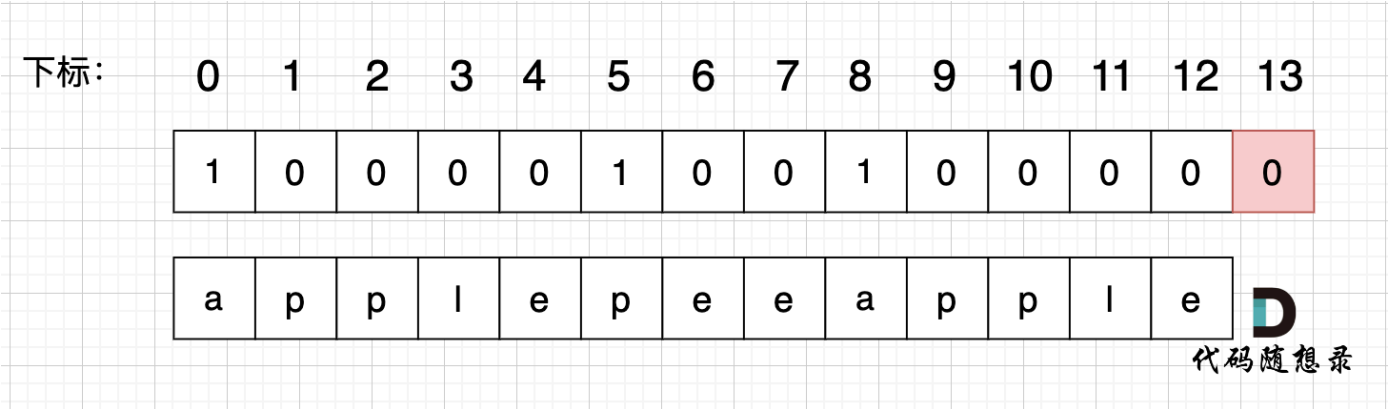
拓展

关于遍历顺序，再给大家讲一下为什么 先遍历物品再遍历背包不行。

这里可以给出先遍历物品再遍历背包的代码：

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
        vector<bool> dp(s.size() + 1, false);
        dp[0] = true;
        for (int j = 0; j < wordDict.size(); j++) { // 物品
            for (int i = wordDict[j].size(); i <= s.size(); i++) { // 背包
                string word = s.substr(i - wordDict[j].size(), wordDict[j].size());
                // cout << word << endl;
                if ( word == wordDict[j] && dp[i - wordDict[j].size()]) {
                    dp[i] = true;
                }
                // for (int k = 0; k <= s.size(); k++) cout << dp[k] << " "; //这里打印
dp数组的情况
                // cout << endl;
            }
        }
        return dp[s.size()];

    }
};
```

使用用例：s = "applepenapple", wordDict = ["apple", "pen"]，对应的dp数组状态如下：



最后dp[s.size()] = 0 即 dp[13] = 0 ，而不是1，因为先用 "apple" 去遍历的时候，dp[8]并没有被赋值为1 （还没用"pen"），所以 dp[13]也不能变成1。

除非是先用 "apple" 遍历一遍，再用 "pen" 遍历，此时 dp[8]已经是1，最后再用 "apple" 去遍历，dp[13]才能是1。

如果大家对这里不理解，建议可以把我上面给的代码，拿去力扣上跑一跑，把dp数组打印出来，对着递推公式一步一步去看，思路就清晰了。

总结

本题和我们之前讲解回溯专题的回溯算法：分割回文串非常像，所以我也给出了对应的回溯解法。

稍加分析，便可知道本题是完全背包，是求能否组成背包，而且这里要求物品是要有顺序的。

27. 动态规划：关于多重背包，你该了解这些！

之前我们已经系统的讲解了01背包和完全背包，如果没有看过的录友，建议先把如下三篇文章仔细阅读一波。

- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）
- 动态规划：关于完全背包，你该了解这些！

这次我们再来说一说多重背包

多重背包

对于多重背包，我在力扣上还没发现对应的题目，所以这里就做一下简单介绍，大家大概了解一下。

有N种物品和一个容量为V 的背包。第i种物品最多有Mi件可用，每件耗费的空间是Ci ，价值是Wi 。求解将哪些物品装入背包可使这些物品的耗费的空间 总和不超过背包容量，且价值总和最大。

多重背包和01背包是非常像的， 为什么和01背包像呢?

每件物品最多有Mi件可用，把Mi件摊开，其实就是一个01背包问题了。

例如：

背包最大重量为10。

物品为：

	重量	价值	数量
物品0	1	15	2
物品1	3	20	3
物品2	4	30	2

问背包能背的物品最大价值是多少?

和如下情况有区别么?

	重量	价值	数量
物品0	1	15	1
物品0	1	15	1
物品1	3	20	1
物品1	3	20	1
物品1	3	20	1
物品2	4	30	1
物品2	4	30	1

毫无区别，这就转成了一个01背包问题了，且每个物品只用一次。

这种方式来实现多重背包的代码如下：

```
void test_multi_pack() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    vector<int> nums = {2, 3, 2};
    int bagWeight = 10;
    for (int i = 0; i < nums.size(); i++) {
        while (nums[i] > 1) { // nums[i]保留到1，把其他物品都展开
            weight.push_back(weight[i]);
            value.push_back(value[i]);
            nums[i]--;
        }
    }

    vector<int> dp(bagWeight + 1, 0);
    for(int i = 0; i < weight.size(); i++) { // 遍历物品
        for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
        for (int j = 0; j <= bagWeight; j++) {
            cout << dp[j] << " ";
        }
        cout << endl;
    }
    cout << dp[bagWeight] << endl;

}
int main() {
    test_multi_pack();
}
```

- 时间复杂度：O(m × n × k)，m：物品种类个数，n背包容量，k单类物品数量

也有另一种实现方式，就是把每种商品遍历的个数放在01背包里面在遍历一遍。

代码如下：（详看注释）

```
void test_multi_pack() {
    vector<int> weight = {1, 3, 4};
    vector<int> value = {15, 20, 30};
    vector<int> nums = {2, 3, 2};
    int bagWeight = 10;
    vector<int> dp(bagWeight + 1, 0);


    for(int i = 0; i < weight.size(); i++) { // 遍历物品
        for(int j = bagWeight; j >= weight[i]; j--) { // 遍历背包容量
            // 以上为01背包，然后加一个遍历个数
            for (int k = 1; k <= nums[i] && (j - k * weight[i]) >= 0; k++) { // 遍历个数
                dp[j] = max(dp[j], dp[j - k * weight[i]] + k * value[i]);
            }
        }
        // 打印一下dp数组
        for (int j = 0; j <= bagWeight; j++) {
            cout << dp[j] << " ";
        }
        cout << endl;
    }
    cout << dp[bagWeight] << endl;
}
int main() {
    test_multi_pack();
}
```

- 时间复杂度：O(m × n × k)，m：物品种类个数，n背包容量，k单类物品数量

从代码里可以看出是01背包里面在加一个for循环遍历一个每种商品的数量。 和01背包还是如出一辙的。

当然还有那种二进制优化的方法，其实就是把每种物品的数量，打包成一个个独立的包。

和以上在循环遍历上有所不同，因为是分拆为各个包最后可以组成一个完整背包，具体原理我就不做过多解释了，大家了解一下就行，面试的话基本不会考完这个深度了，感兴趣可以自己深入研究一波。

总结

多重背包在面试中基本不会出现，力扣上也没有对应的题目，大家对多重背包的掌握程度知道它是一种01背包，并能在01背包的基础上写出对应代码就可以了。

至于背包九讲里面还有混合背包，二维费用背包，分组背包等等这些，大家感兴趣可以自己去学习学习，这里也不做介绍了，面试也不会考。

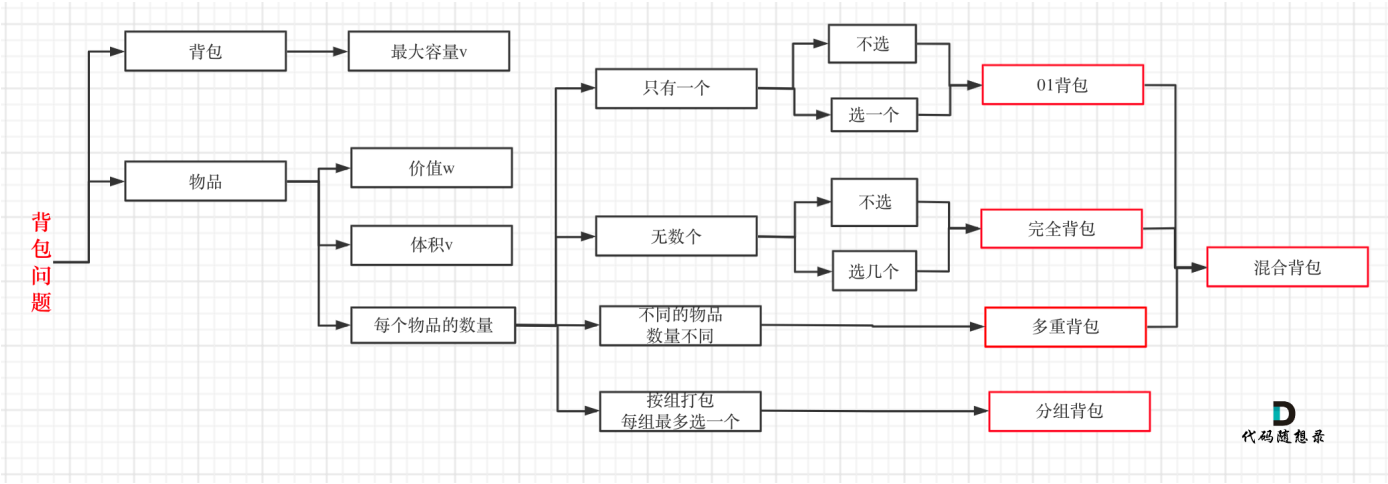
---

28. 听说背包问题很难？ 这篇总结篇来拯救你了

年前我们已经把背包问题都讲完了，那么现在我们要对背包问题进行总结一番。

背包问题是动态规划里的非常重要的一部分，所以我把背包问题单独总结一下，等动态规划专题更新完之后，我们还会在整体总结一波动态规划。

关于这几种常见的背包，其关系如下：



通过这个图，可以很清晰分清这几种常见背包之间的关系。

在讲解背包问题的时候，我们都是按照如下五部来逐步分析，相信大家也体会到，把这五部都搞透了，算是对动规来理解深入了。

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

其实这五部里哪一步都很关键，但确定递推公式和确定遍历顺序都具有规律性和代表性，所以下面我从这两点来对背包问题做一做总结。

背包递推公式

问能否能装满背包（或者最多装多少）：dp[j] = max(dp[j], dp[j - nums[i]] + nums[i]); ，对应题目如下：

- 动态规划：416.分割等和子集
- 动态规划：1049.最后一块石头的重量 II

问装满背包有几种方法：dp[j] += dp[j - nums[i]] ，对应题目如下：

- 动态规划：494.目标和
- 动态规划：518. 零钱兑换 II
- 动态规划：377.组合总和IV
- 动态规划：70. 爬楼梯进阶版（完全背包）

问背包装满最大价值：dp[j] = max(dp[j], dp[j - weight[i]] + value[i]); ，对应题目如下：

- 动态规划：474.一和零

问装满背包所有物品的最小个数：dp[j] = min(dp[j - coins[i]] + 1, dp[j]); ，对应题目如下：

- 动态规划：322.零钱兑换
- 动态规划：279.完全平方数

遍历顺序

01背包

在动态规划：关于01背包问题，你该了解这些！中我们讲解二维dp数组01背包先遍历物品还是先遍历背包都是可以的，且第二层for循环是从小到大遍历。

和动态规划：关于01背包问题，你该了解这些！（滚动数组）中，我们讲解一维dp数组01背包只能先遍历物品再遍历背包容量，且第二层for循环是从大到小遍历。

一维dp数组的背包在遍历顺序上和二维dp数组实现的01背包其实是有很大差异的，大家需要注意！

完全背包

说完01背包，再看看完全背包。

在动态规划：关于完全背包，你该了解这些！中，讲解了纯完全背包的一维dp数组实现，先遍历物品还是先遍历背包都是可以的，且第二层for循环是从小到大遍历。

但是仅仅是纯完全背包的遍历顺序是这样的，题目稍有变化，两个for循环的先后顺序就不一样了。

如果求组合数就是外层for循环遍历物品，内层for遍历背包。

如果求排列数就是外层for遍历背包，内层for循环遍历物品。

相关题目如下：

- 求组合数：动态规划：518.零钱兑换II
- 求排列数：动态规划：377. 组合总和 IV、动态规划：70. 爬楼梯进阶版（完全背包）

如果求最小数，那么两层for循环的先后顺序就无所谓了，相关题目如下：

- 求最小数：动态规划：322. 零钱兑换、动态规划：279.完全平方数

对于背包问题，其实递推公式算是容易的，难是难在遍历顺序上，如果把遍历顺序搞透，才算是真正理解了。

总结

这篇背包问题总结篇是对背包问题的高度概括，讲最关键的两部：递推公式和遍历顺序，结合力扣上的题目全都抽象出来了。

而且每一个点，我都给出了对应的力扣题目。

最后如果你想了解多重背包，可以看这篇动态规划：关于多重背包，你该了解这些！，力扣上还没有多重背包的题目，也不是面试考察的重点。

如果把我本篇总结出来的内容都掌握的话，可以说对背包问题理解的就很深刻了，用来对付面试中的背包问题绰绰有余！

背包问题总结：

这个图是 代码随想录知识星球 成员：海螺人，所画结的非常好，分享给大家。

29.打家劫舍

力扣题目链接

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你 不触动警报装置的情况下 ，一夜之内能够偷窃到的最高金额。

- 示例 1：
- 输入：[1,2,3,1]
- 输出：4

解释：偷窃 1 号房屋 (金额 = 1) ，然后偷窃 3 号房屋 (金额 = 3)。
偷窃到的最高金额 = 1 + 3 = 4 。

- 示例 2：
- 输入：[2,7,9,3,1]
- 输出：12
  解释：偷窃 1 号房屋 (金额 = 2), 偷窃 3 号房屋 (金额 = 9)，接着偷窃 5 号房屋 (金额 = 1)。
  偷窃到的最高金额 = 2 + 9 + 1 = 12 。

提示：

- 0 <= nums.length <= 100
- 0 <= nums[i] <= 400

算法公开课

《代码随想录》算法视频公开课：动态规划，偷不偷这个房间呢？ | LeetCode：198.打家劫舍，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

大家如果刚接触这样的题目，会有点困惑，当前的状态我是偷还是不偷呢?

仔细一想，当前房屋偷与不偷取决于 前一个房屋和前两个房屋是否被偷了。

所以这里就更感觉到，当前状态和前面状态会有一种依赖关系，那么这种依赖关系都是动规的递推公式。

当然以上是大概思路，打家劫舍是dp解决的经典问题，接下来我们来动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i]：考虑下标i（包括i）以内的房屋，最多可以偷窃的金额为dp[i]。

2. 确定递推公式

决定dp[i]的因素就是第i房间偷还是不偷。

如果偷第i房间，那么dp[i] = dp[i - 2] + nums[i] ，即：第i-1房一定是不考虑的，找出 下标i-2（包括i-2）以内的房屋，最多可以偷窃的金额为dp[i-2] 加上第i房间偷到的钱。

如果不偷第i房间，那么dp[i] = dp[i - 1]，即考 虑i-1房，（**注意这里是考虑，并不是一定要偷i-1房，这是很多同学容易混淆的点**）

然后dp[i]取最大值，即dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);

3. dp数组如何初始化

从递推公式dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);可以看出，递推公式的基础就是dp[0] 和 dp[1]

从dp[i]的定义上来讲，dp[0] 一定是 nums[0]，dp[1]就是nums[0]和nums[1]的最大值即：dp[1] = max(nums[0], nums[1]);

代码如下：

```
vector<int> dp(nums.size());
dp[0] = nums[0];
dp[1] = max(nums[0], nums[1]);
```

4. 确定遍历顺序

dp[i] 是根据dp[i - 2] 和 dp[i - 1] 推导出来的，那么一定是从前到后遍历！

代码如下：

```
for (int i = 2; i < nums.size(); i++) {
    dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);
}
```

5. 举例推导dp数组

以示例二，输入[2,7,9,3,1]为例。

红框dp[nums.size() - 1]为结果。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int rob(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        if (nums.size() == 1) return nums[0];
        vector<int> dp(nums.size());
        dp[0] = nums[0];
        dp[1] = max(nums[0], nums[1]);
        for (int i = 2; i < nums.size(); i++) {
            dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);
        }
        return dp[nums.size() - 1];
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

总结

打家劫舍是DP解决的经典题目，这道题也是打家劫舍入门级题目，后面我们还会变种方式来打劫的。

30.打家劫舍II

力扣题目链接

你是一个专业的小偷，计划偷窃沿街的房屋，每间房内都藏有一定的现金。这个地方所有的房屋都 围成一圈 ，这意味着第一个房屋和最后一个房屋是紧挨着的。同时，相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警 。

给定一个代表每个房屋存放金额的非负整数数组，计算你 在不触动警报装置的情况下 ，能够偷窃到的最高金额。

示例 1：

- 输入：nums = [2,3,2]

- 输出：3
- 解释：你不能先偷窃 1 号房屋（金额 = 2），然后偷窃 3 号房屋（金额 = 2），因为他们是相邻的。
- 示例 2：
- 输入：nums = [1,2,3,1]
- 输出：4
- 解释：你可以先偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3）。偷窃到的最高金额 = 1 + 3 = 4 。
- 示例 3：
- 输入：nums = [0]
- 输出：0

提示：

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 1000

算法公开课

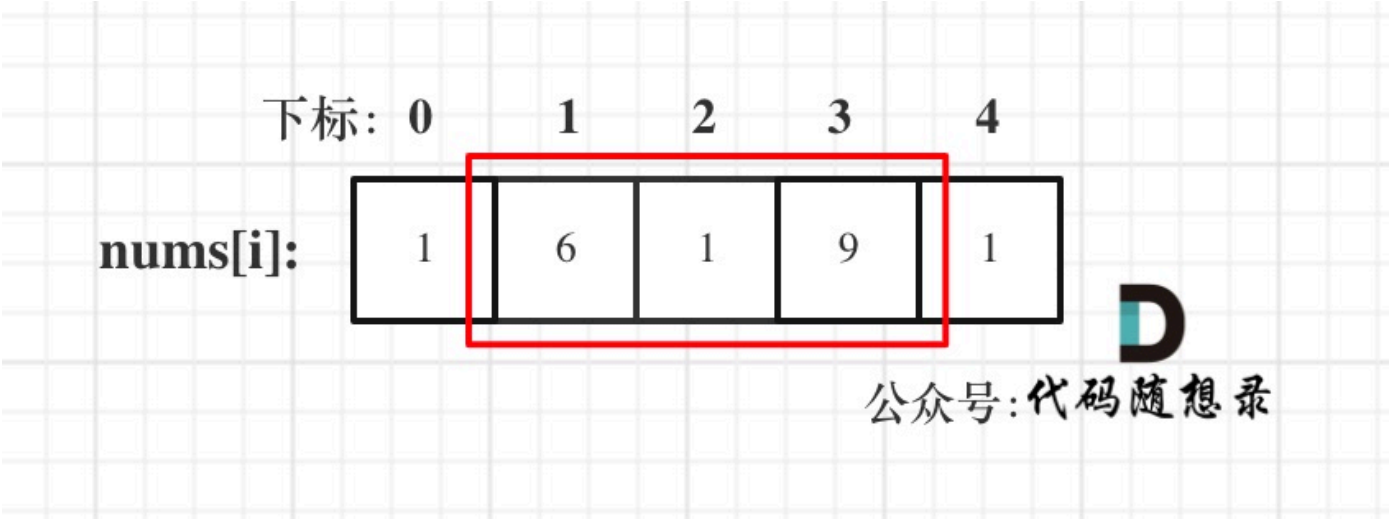
《代码随想录》算法视频公开课：动态规划，房间连成环了那还偷不偷呢？ | LeetCode：213.打家劫舍II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

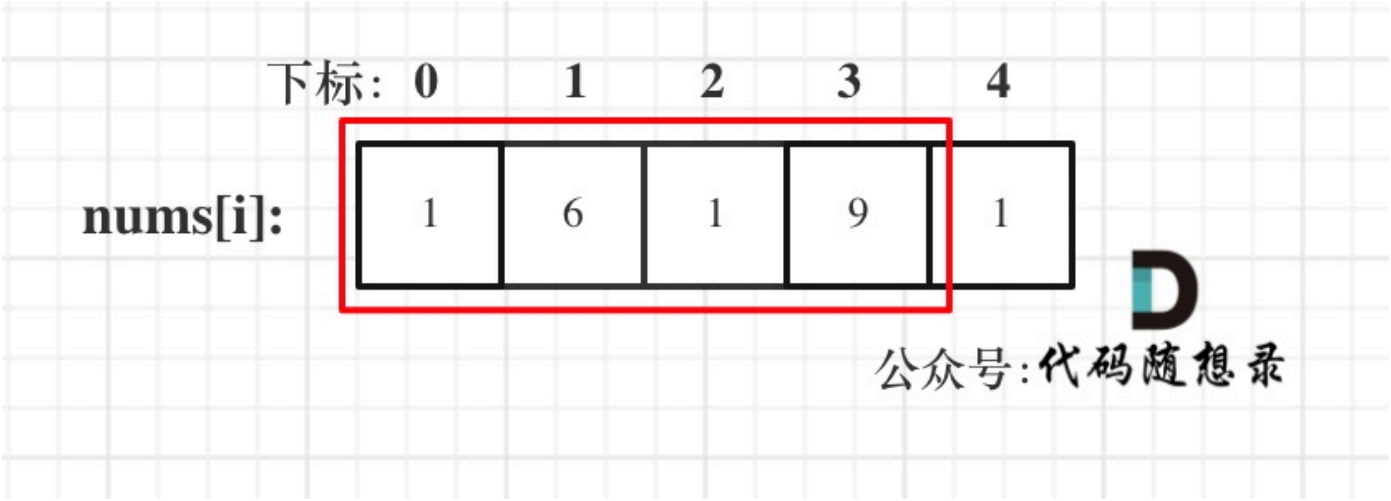
这道题目和198.打家劫舍是差不多的，唯一区别就是成环了。

对于一个数组，成环的话主要有如下三种情况：

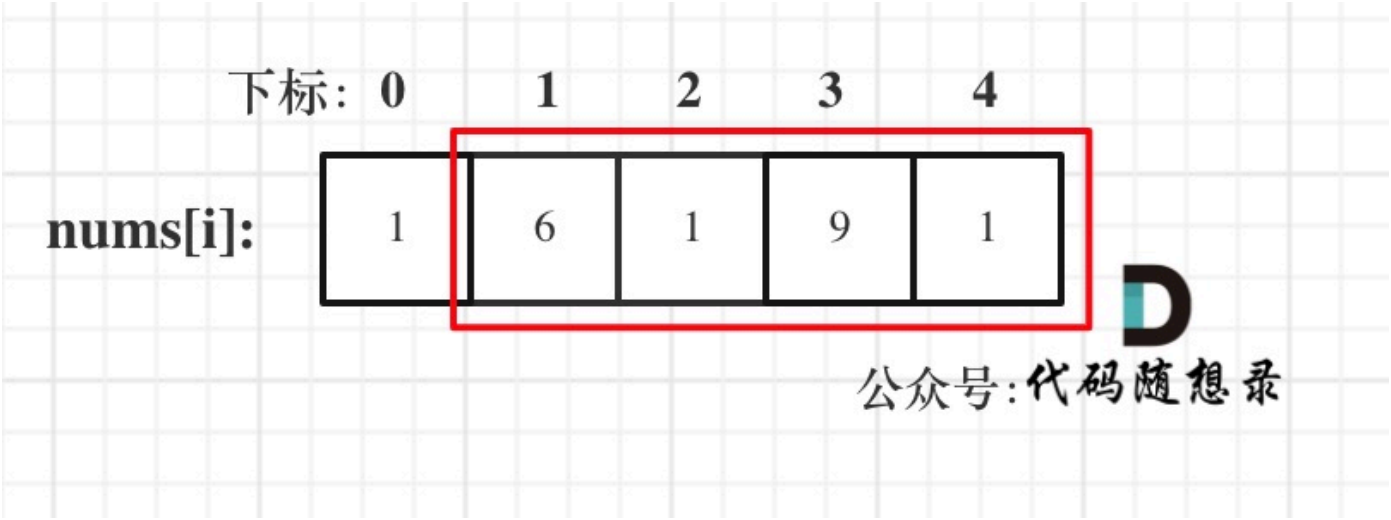
- 情况一：考虑不包含首尾元素



- 情况二：考虑包含首元素，不包含尾元素



- 情况三：考虑包含尾元素，不包含首元素



注意我这里用的是"考虑"，例如情况三，虽然是考虑包含尾元素，但不一定要选尾部元素！ 对于情况三，取nums[1] 和 nums[3]就是最大的。

而情况二 和 情况三 都包含了情况一了，所以只考虑情况二和情况三就可以了。

分析到这里，本题其实比较简单了。 剩下的和198.打家劫舍就是一样的了。

代码如下：

```
// 注意注释中的情况二情况三，以及把198.打家劫舍的代码抽离出来了
class Solution {
public:
    int rob(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        if (nums.size() == 1) return nums[0];
        int result1 = robRange(nums, 0, nums.size() - 2); // 情况二
        int result2 = robRange(nums, 1, nums.size() - 1); // 情况三
        return max(result1, result2);
    }
    // 198.打家劫舍的逻辑
    int robRange(vector<int>& nums, int start, int end) {
        if (end == start) return nums[start];
```

```
        vector<int> dp(nums.size());
        dp[start] = nums[start];
        dp[start + 1] = max(nums[start], nums[start + 1]);
        for (int i = start + 2; i <= end; i++) {
            dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);
        }
        return dp[end];
    }
};
```

- 时间复杂度: O(n)
- 空间复杂度: O(n)

总结

成环之后还是难了一些的， 不少题解没有把“考虑房间”和“偷房间”说清楚。

这就导致大家会有这样的困惑：情况三怎么就包含了情况一了呢？ 本文图中最后一间房不能偷啊，偷了一定不是最优结果。

所以我在本文重点强调了情况一二三是“考虑”的范围，而具体房间偷与不偷交给递推公式去抉择。

这样大家就不难理解情况二和情况三包含了情况一了。

31.打家劫舍 III

力扣题目链接

在上次打劫完一条街道之后和一圈房屋后，小偷又发现了一个新的可行窃的地区。这个地区只有一个入口，我们称之为“根”。 除了“根”之外，每栋房子有且只有一个“父”房子与之相连。一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。 如果两个直接相连的房子在同一天晚上被打劫，房屋将自动报警。

计算在不触动警报的情况下，小偷一晚能够盗取的最高金额。

示例 1：

```
输入：[3,2,3,null,3,null,1]

     3
    / \
   2   3
    \   \
     3   1

输出：7
解释：小偷一晚能够盗取的最高金额 = 3 + 3 + 1 = 7.
```

示例 2：

```
输入：[3,4,5,1,3,null,1]

     3
    / \
   4   5
  / \   \
 1   3   1

输出：9
解释：小偷一晚能够盗取的最高金额 = 4 + 5 = 9.
```

算法公开课

《代码随想录》算法视频公开课：动态规划，房间连成树了，偷不偷呢？| LeetCode：337.打家劫舍3，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目和 198.打家劫舍，213.打家劫舍II也是如出一辙，只不过这个换成了树。

如果对树的遍历不够熟悉的话，那本题就有难度了。

对于树的话，首先就要想到遍历方式，前中后序（深度优先搜索）还是层序遍历（广度优先搜索）。

本题一定是要后序遍历，因为通过递归函数的返回值来做下一步计算。

与198.打家劫舍，213.打家劫舍II一样，关键是要讨论当前节点抢还是不抢。

如果抢了当前节点，两个孩子就不能动，如果没抢当前节点，就可以考虑抢左右孩子（**注意这里说的是“考虑”**）

暴力递归

代码如下：

```
class Solution {
public:
    int rob(TreeNode* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right == NULL) return root->val;
        // 偷父节点
        int val1 = root->val;
        if (root->left) val1 += rob(root->left->left) + rob(root->left->right); // 跳过root->left，相当于不考虑左孩子了
        if (root->right) val1 += rob(root->right->left) + rob(root->right->right); // 跳过root->right，相当于不考虑右孩子了
        // 不偷父节点
        int val2 = rob(root->left) + rob(root->right); // 考虑root的左右孩子
        return max(val1, val2);
    }
};
```

- 时间复杂度：O(n^2)，这个时间复杂度不太标准，也不容易准确化，例如越往下的节点重复计算次数就越多
- 空间复杂度：O(log n)，算上递推系统栈的空间

当然以上代码超时了，这个递归的过程中其实是有重复计算了。

我们计算了root的四个孙子（左右孩子的孩子）为头结点的子树的情况，又计算了root的左右孩子为头结点的子树的情况，计算左右孩子的时候其实又把孙子计算了一遍。

记忆化递推

所以可以使用一个map把计算过的结果保存一下，这样如果计算过孙子了，那么计算孩子的时候可以复用孙子节点的结果。

代码如下：

```
class Solution {
public:
    unordered_map<TreeNode* , int> umap; // 记录计算过的结果
    int rob(TreeNode* root) {
        if (root == NULL) return 0;
```

```
        if (root->left == NULL && root->right == NULL) return root->val;
        if (umap[root]) return umap[root]; // 如果umap里已经有记录则直接返回
        // 偷父节点
        int val1 = root->val;
        if (root->left) val1 += rob(root->left->left) + rob(root->left->right); // 跳过
root->left
        if (root->right) val1 += rob(root->right->left) + rob(root->right->right); // 跳
过root->right
        // 不偷父节点
        int val2 = rob(root->left) + rob(root->right); // 考虑root的左右孩子
        umap[root] = max(val1, val2); // umap记录一下结果
        return max(val1, val2);
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(log n)，算上递推系统栈的空间

动态规划

在上面两种方法，其实对一个节点 偷与不偷得到的最大金钱都没有做记录，而是需要实时计算。

而动态规划其实就是使用状态转移容器来记录状态的变化，这里可以使用一个长度为2的数组，记录当前节点偷与不偷所得到的的最大金钱。

这道题目算是树形dp的入门题目，因为是在树上进行状态转移，我们在讲解二叉树的时候说过递归三部曲，那么下面我以递归三部曲为框架，其中融合动规五部曲的内容来进行讲解。

1. 确定递归函数的参数和返回值

这里我们要求一个节点 偷与不偷的两个状态所得到的金钱，那么返回值就是一个长度为2的数组。

参数为当前节点，代码如下：

```
vector<int> robTree(TreeNode* cur) {
```

其实这里的返回数组就是dp数组。

所以dp数组（dp table）以及下标的含义：下标为0记录不偷该节点所得到的的最大金钱，下标为1记录偷该节点所得到的的最大金钱。

所以本题dp数组就是一个长度为2的数组！

那么有同学可能疑惑，长度为2的数组怎么标记树中每个节点的状态呢？

别忘了在递归的过程中，系统栈会保存每一层递归的参数。

如果还不理解的话，就接着往下看，看到代码就理解了哈。

2. 确定终止条件

在遍历的过程中，如果遇到空节点的话，很明显，无论偷还是不偷都是0，所以就返回

```
if (cur == NULL) return vector<int>{0, 0};
```

这也相当于dp数组的初始化

3. 确定遍历顺序

首先明确的是使用后序遍历。 因为要通过递归函数的返回值来做下一步计算。

通过递归左节点，得到左节点偷与不偷的金钱。

通过递归右节点，得到右节点偷与不偷的金钱。

代码如下：

```
// 下标0：不偷，下标1：偷
vector<int> left = robTree(cur->left); // 左
vector<int> right = robTree(cur->right); // 右
// 中
```

4. 确定单层递归的逻辑

如果是偷当前节点，那么左右孩子就不能偷，val1 = cur->val + left[0] + right[0]；（**如果对下标含义不理解就再回顾一下dp数组的含义**）

如果不偷当前节点，那么左右孩子就可以偷，至于到底偷不偷一定是选一个最大的，所以：val2 = max(left[0], left[1]) + max(right[0], right[1]);

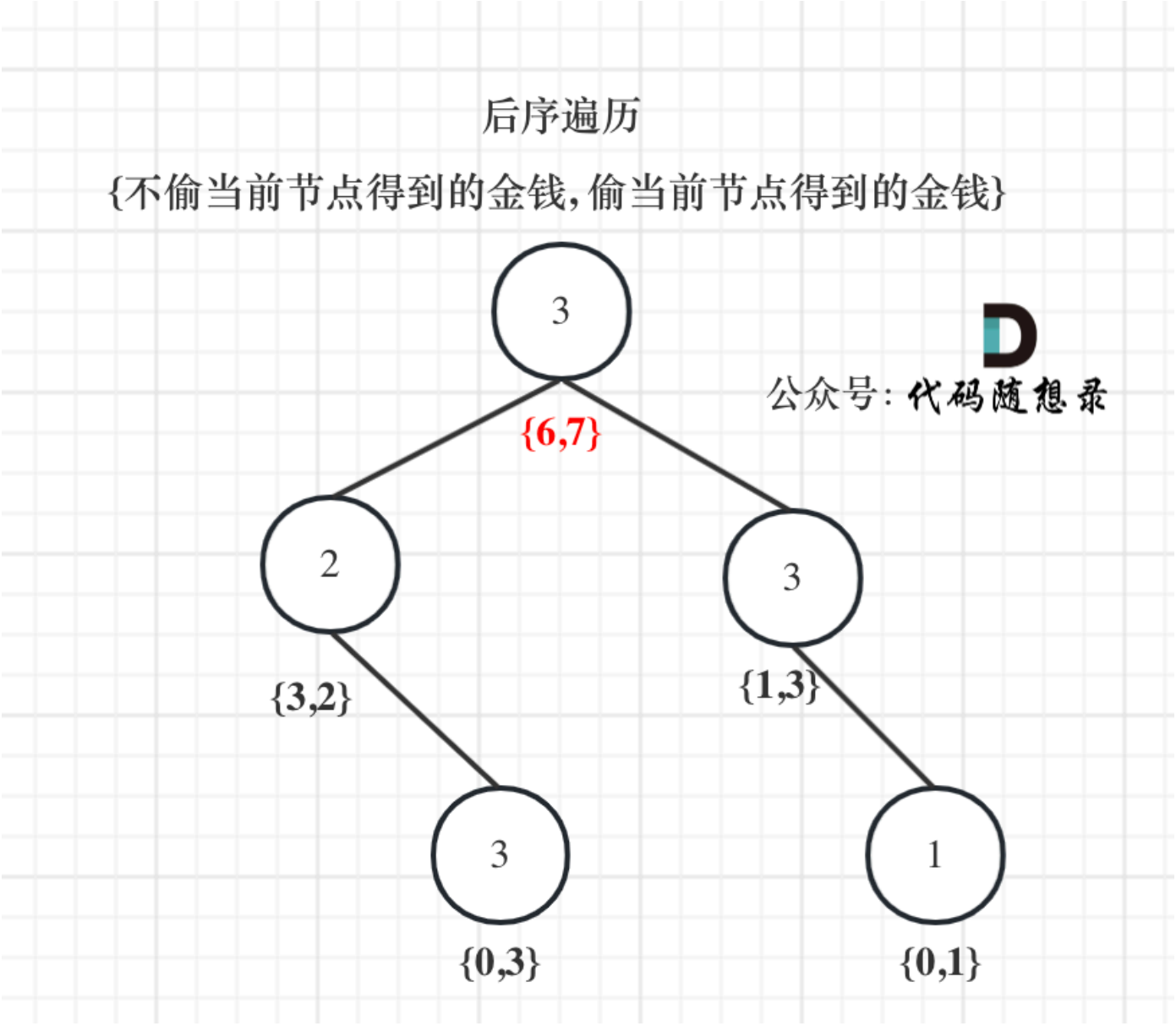
最后当前节点的状态就是{val2, val1}; 即：{不偷当前节点得到的最大金钱，偷当前节点得到的最大金钱}

代码如下：

```
vector<int> left = robTree(cur->left); // 左
vector<int> right = robTree(cur->right); // 右

// 偷cur
int val1 = cur->val + left[0] + right[0];
// 不偷cur
int val2 = max(left[0], left[1]) + max(right[0], right[1]);
return {val2, val1};
```

5. 举例推导dp数组

以示例1为例，dp数组状态如下：（**注意用后序遍历的方式推导**）



最后头结点就是 取下标0 和 下标1的最大值就是偷得的最大金钱。

递归三部曲与动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    int rob(TreeNode* root) {
        vector<int> result = robTree(root);
        return max(result[0], result[1]);
    }
    // 长度为2的数组，0：不偷，1：偷
    vector<int> robTree(TreeNode* cur) {
        if (cur == NULL) return vector<int>{0, 0};
        vector<int> left = robTree(cur->left);
        vector<int> right = robTree(cur->right);
        // 偷cur，那么就不能偷左右节点。
        int val1 = cur->val + left[0] + right[0];
        // 不偷cur，那么可以偷也可以不偷左右节点，则取较大的情况
        int val2 = max(left[0], left[1]) + max(right[0], right[1]);
```

```
        return {val2, val1};
    }
};
```

- 时间复杂度：O(n)，每个节点只遍历了一次
- 空间复杂度：O(log n)，算上递推系统栈的空间

总结

这道题是树形DP的入门题目，通过这道题目大家应该也了解了，所谓树形DP就是在树上进行递归公式的推导。

所以树形DP也没有那么神秘！

只不过平时我们习惯了在一维数组或者二维数组上推导公式，一下子换成了树，就需要对树的遍历方式足够了解！

大家还记不记得我在讲解贪心专题的时候，讲到这道题目：贪心算法：我要监控二叉树！，这也是贪心算法在树上的应用。**那我也可以把这个算法起一个名字，叫做树形贪心**，哈哈哈

“树形贪心”词汇从此诞生，来自「代码随想录」

---

32. 买卖股票的最佳时机

力扣题目链接

给定一个数组 prices ，它的第 i 个元素 prices[i] 表示一支给定股票第 i 天的价格。

你只能选择 某一天 买入这只股票，并选择在 未来的某一个不同的日子 卖出该股票。设计一个算法来计算你所能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 0 。

- 示例 1：
- 输入：[7,1,5,3,6,4]
- 输出：5
  解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 。
  注意利润不能是 7-1 = 6, 因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。
- 示例 2：
- 输入：prices = [7,6,4,3,1]
- 输出：0
  解释：在这种情况下, 没有交易完成, 所以最大利润为 0。

算法公开课

《代码随想录》算法视频公开课：动态规划之 LeetCode：121.买卖股票的最佳时机1，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

暴力

这道题目最直观的想法，就是暴力，找最优间距了。

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int result = 0;
        for (int i = 0; i < prices.size(); i++) {
            for (int j = i + 1; j < prices.size(); j++){
                result = max(result, prices[j] - prices[i]);
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

当然该方法超时了。

贪心

因为股票就买卖一次，那么贪心的想法很自然就是取最左最小值，取最右最大值，那么得到的差值就是最大利润。

C++代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int low = INT_MAX;
        int result = 0;
        for (int i = 0; i < prices.size(); i++) {
            low = min(low, prices[i]);  // 取最左最小价格
            result = max(result, prices[i] - low); // 直接取最大区间利润
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

动态规划

动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][0] 表示第i天持有股票所得最多现金 ，**这里可能有同学疑惑，本题中只能买卖一次，持有股票之后哪还有现金呢？**

其实一开始现金是0，那么加入第i天买入股票现金就是 -prices[i]， 这是一个负数。

dp[i][1] 表示第i天不持有股票所得最多现金

注意这里说的是“持有”，“持有”不代表就是当天“买入”！也有可能是昨天就买入了，今天保持持有的状态

很多同学把“持有”和“买入”没区分清楚。

在下面递推公式分析中，我会进一步讲解。

2. 确定递推公式

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是买入今天的股票后所得现金即：-prices[i]

那么dp[i][0]应该选所得现金最大的，所以dp[i][0] = max(dp[i - 1][0], -prices[i]);

如果第i天不持有股票即dp[i][1]， 也可以由两个状态推出来

- 第i-1天就不持有股票，那么就保持现状，所得现金就是昨天不持有股票的所得现金 即：dp[i - 1][1]
- 第i天卖出股票，所得现金就是按照今天股票价格卖出后所得现金即：prices[i] + dp[i - 1][0]

同样dp[i][1]取最大的，dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);

这样递推公式我们就分析完了

3. dp数组如何初始化

由递推公式 dp[i][0] = max(dp[i - 1][0], -prices[i]); 和 dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);可以看出

其基础都是要从dp[0][0]和dp[0][1]推导出来。

那么dp[0][0]表示第0天持有股票，此时的持有股票就一定是买入股票了，因为不可能有前一天推出来，所以dp[0][0] -= prices[0];

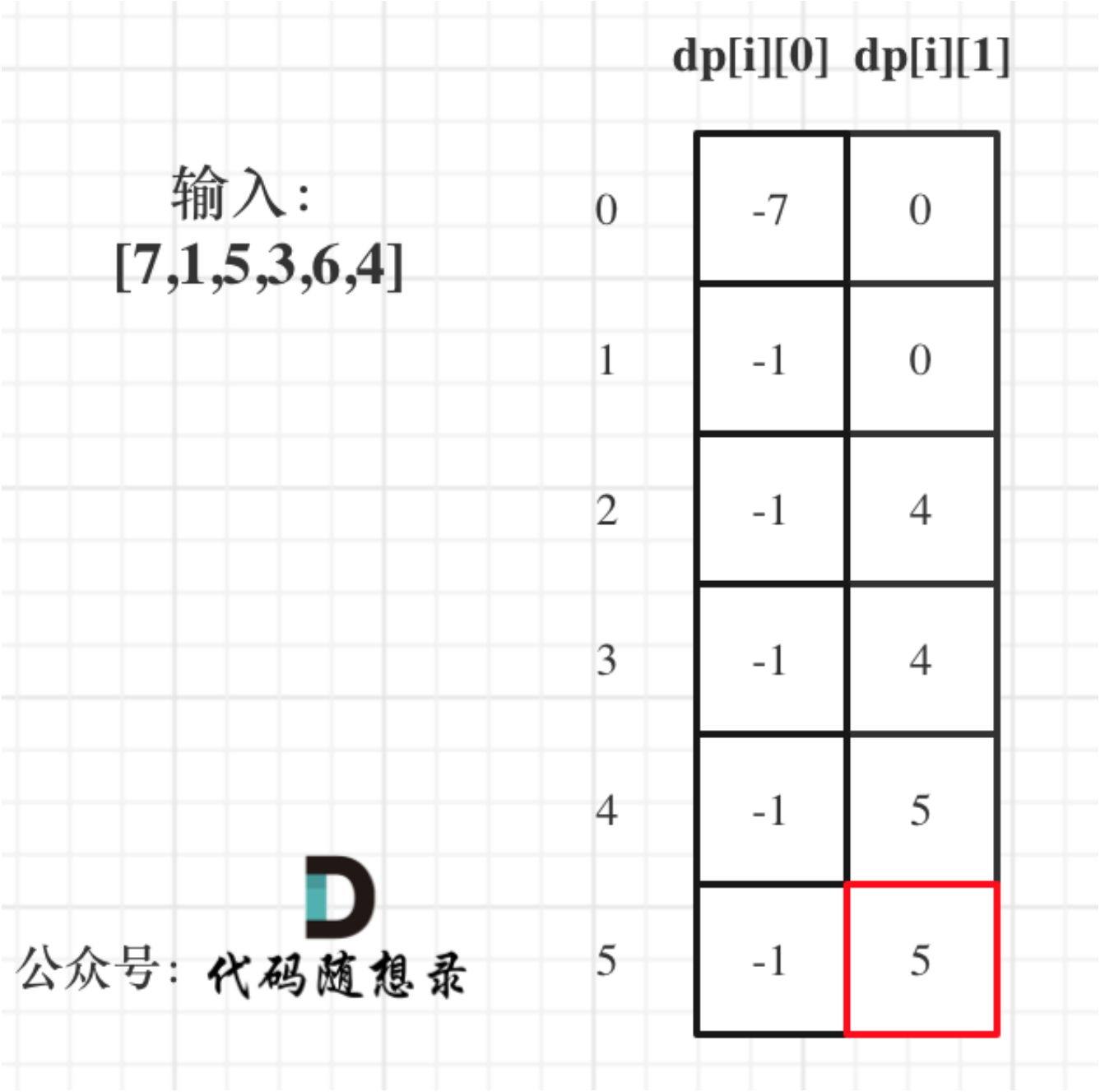
dp[0][1]表示第0天不持有股票，不持有股票那么现金就是0，所以dp[0][1] = 0;

4. 确定遍历顺序

从递推公式可以看出dp[i]都是由dp[i - 1]推导出来的，那么一定是从前向后遍历。

5. 举例推导dp数组

以示例1，输入：[7,1,5,3,6,4]为例，dp数组状态如下：



dp[5][1]就是最终结果。

为什么不是dp[5][0]呢?

因为本题中不持有股票状态所得金钱一定比持有股票状态得到的多!

以上分析完毕，C++代码如下：

```
// 版本一
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        if (len == 0) return 0;
        vector<vector<int>> dp(len, vector<int>(2));
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
```

```
        for (int i = 1; i < len; i++) {
            dp[i][0] = max(dp[i - 1][0], -prices[i]);
            dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);
        }
        return dp[len - 1][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

从递推公式可以看出，dp[i]只是依赖于dp[i - 1]的状态。

```
dp[i][0] = max(dp[i - 1][0], -prices[i]);
dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);
```

那么我们只需要记录 当前天的dp状态和前一天的dp状态就可以了，可以使用滚动数组来节省空间，代码如下：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(2, vector<int>(2)); // 注意这里只开辟了一个2 * 2大小的二维数组
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i % 2][0] = max(dp[(i - 1) % 2][0], -prices[i]);
            dp[i % 2][1] = max(dp[(i - 1) % 2][1], prices[i] + dp[(i - 1) % 2][0]);
        }
        return dp[(len - 1) % 2][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

这里能写出版本一就可以了，版本二虽然原理都一样，但是想直接写出版本二还是有点麻烦，容易自己给自己找bug。

所以建议是先写出版本一，然后在版本一的基础上优化成版本二，而不是直接就写出版本二。

---

33. 本周小结！（动态规划系列六）

本周我们主要讲解了打家劫舍系列，这个系列也是dp解决的经典问题，那么来看看我们收获了哪些呢，一起来回顾一下吧。

周一

动态规划：开始打家劫舍！中就是给一个数组相邻之间不能连着偷，如何偷才能得到最大金钱。

1. 确定dp数组含义

dp[i]：考虑下标i（包括i）以内的房屋，最多可以偷窃的金额为dp[i]。

2. 确定递推公式

dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);

3. dp数组如何初始化

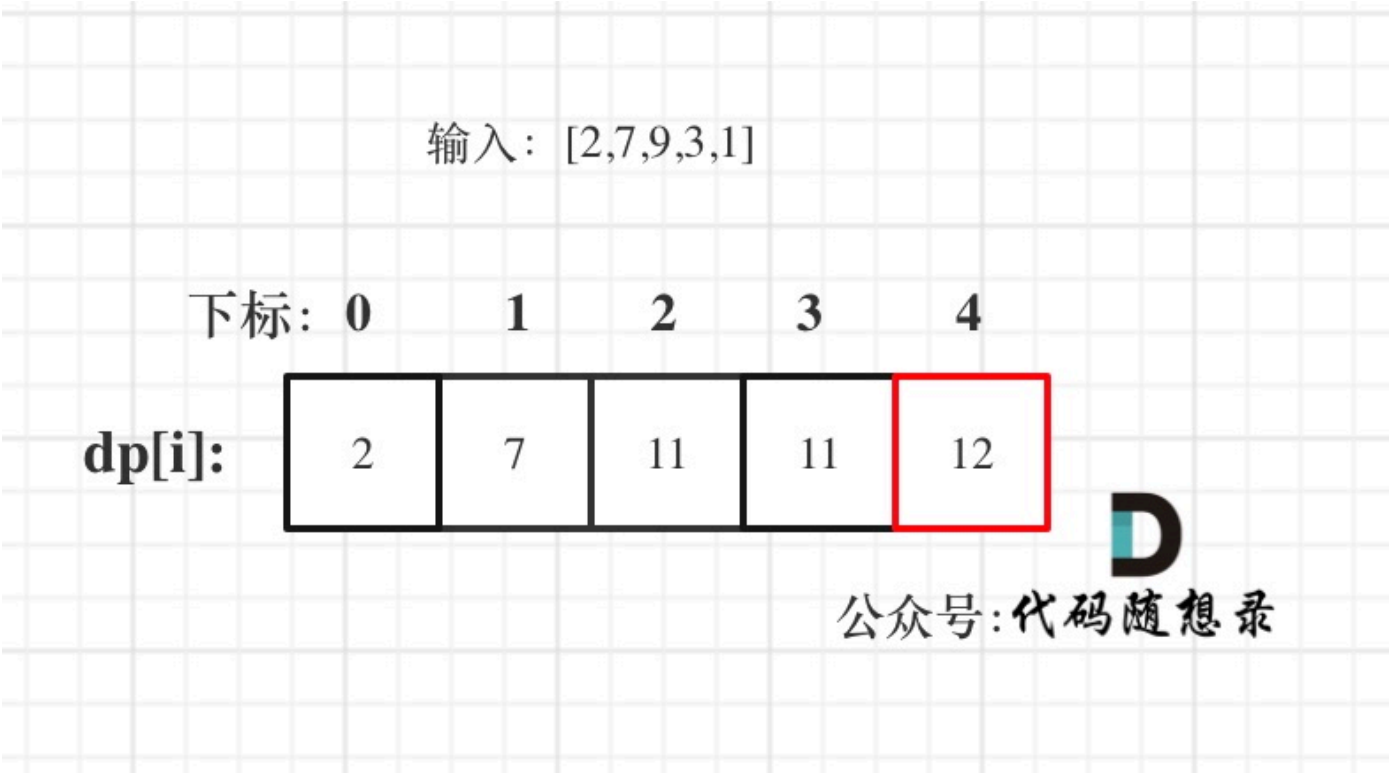
```
vector<int> dp(nums.size());
dp[0] = nums[0];
dp[1] = max(nums[0], nums[1]);
```

4. 确定遍历顺序

从前到后遍历

5. 举例推导dp数组

以示例二，输入[2,7,9,3,1]为例。



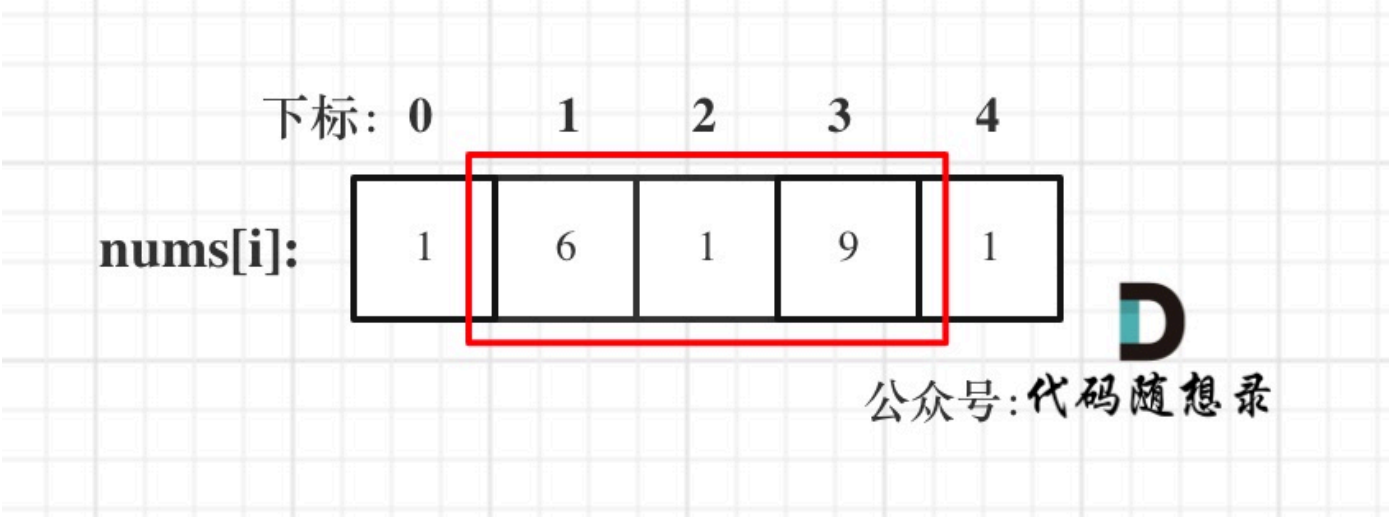
红框dp[nums.size() - 1]为结果。

周二

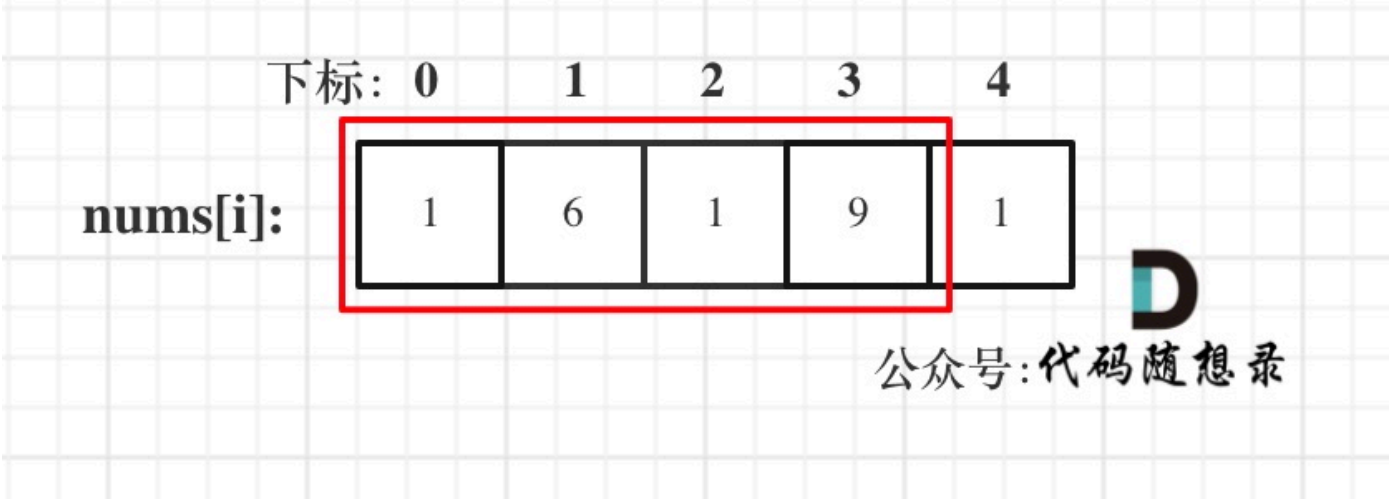
动态规划：继续打家劫舍！就是数组成环了，然后相邻的不能连着偷。

这里主要考虑清楚三种情况：

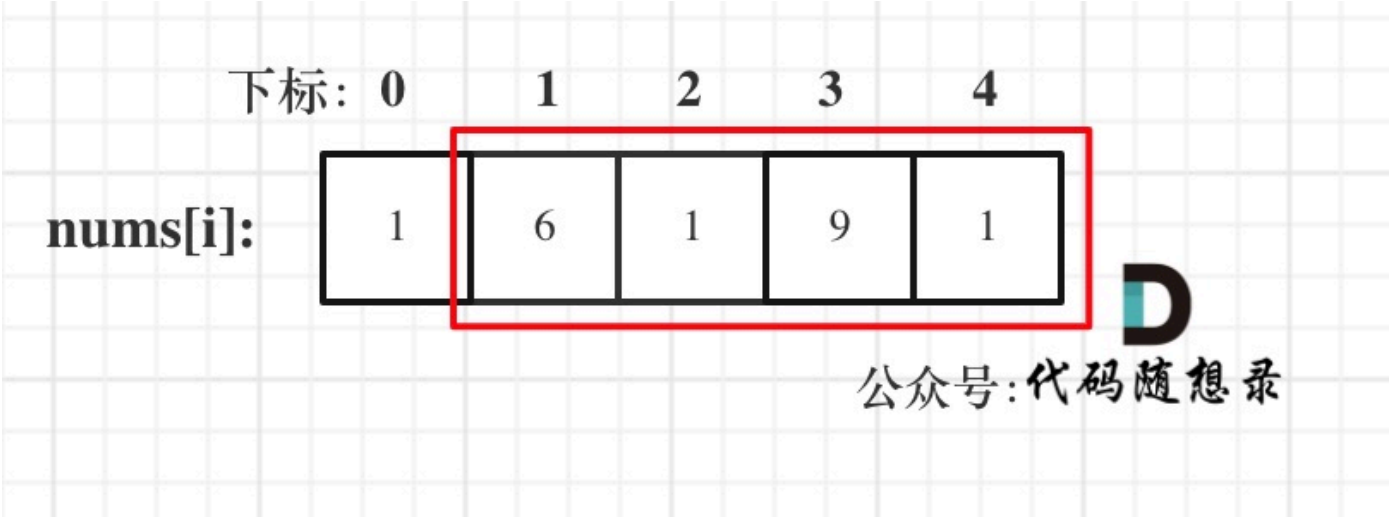
- 情况一：考虑不包含首尾元素



- 情况二：考虑包含首元素，不包含尾元素



- 情况三：考虑包含尾元素，不包含首元素



需要注意的是，**“考虑” 不等于 “偷”**，例如情况三，虽然是考虑包含尾元素，但不一定要选尾部元素！对于情况三，取nums[1] 和 nums[3]就是最大的。

所以情况二 和 情况三 都包含了情况一了，**所以只考虑情况二和情况三就可以了**。

成环之后还是难了一些的， 不少题解没有把“考虑房间”和“偷房间”说清楚。

这就导致大家会有这样的困惑：“情况三怎么就包含了情况一了呢？本文图中最后一间房不能偷啊，偷了一定不是最优结果”。

所以我在本文重点强调了情况一二三是“考虑”的范围，而具体房间偷与不偷交给递推公式去抉择。

剩下的就和动态规划：开始打家劫舍！是一个逻辑了。

周三

动态规划：还要打家劫舍！这次是在一棵二叉树上打家劫舍了，条件还是一样的，相临的不能偷。

这道题目是树形DP的入门题目，其实树形DP其实就是在树上进行递推公式的推导，没有什么神秘的。

这道题目我给出了暴力的解法：

```
class Solution {
public:
    int rob(TreeNode* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right == NULL) return root->val;
        // 偷父节点
        int val1 = root->val;
        if (root->left) val1 += rob(root->left->left) + rob(root->left->right); // 跳过root->left，相当于不考虑左孩子了
        if (root->right) val1 += rob(root->right->left) + rob(root->right->right); // 跳过root->right，相当于不考虑右孩子了
        // 不偷父节点
        int val2 = rob(root->left) + rob(root->right); // 考虑root的左右孩子
        return max(val1, val2);
    }
};
```

当然超时了，因为我们计算了root的四个孙子（左右孩子的孩子）为头结点的子树的情况，又计算了root的左右孩子为头结点的子树的情况，计算左右孩子的时候其实又把孙子计算了一遍。

那么使用一个map把计算过的结果保存一下，这样如果计算过孙子了，那么计算孩子的时候可以复用孙子节点的结果。

代码如下：

```
class Solution {
public:
    unordered_map<TreeNode* , int> umap; // 记录计算过的结果
    int rob(TreeNode* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right == NULL) return root->val;
        if (umap[root]) return umap[root]; // 如果umap里已经有记录则直接返回
        // 偷父节点
        int val1 = root->val;
        if (root->left) val1 += rob(root->left->left) + rob(root->left->right); // 跳过root->left
```

```
        if (root->right) val1 += rob(root->right->left) + rob(root->right->right); // 跳过root->right
        // 不偷父节点
        int val2 = rob(root->left) + rob(root->right); // 考虑root的左右孩子
        umap[root] = max(val1, val2); // umap记录一下结果
        return max(val1, val2);
    }
};
```

最后我们还是给出动态规划的解法。

因为是在树上进行状态转移，我们在讲解二叉树的时候说过递归三部曲，那么下面我以递归三部曲为框架，其中融合动规五部曲的内容来进行讲解。

1. 确定递归函数的参数和返回值

```
vector<int> robTree(TreeNode* cur) {
```

dp数组含义：下标为0记录不偷该节点所得到的的最大金钱，下标为1记录偷该节点所得到的的最大金钱。

所以本题dp数组就是一个长度为2的数组！

那么有同学可能疑惑，长度为2的数组怎么标记树中每个节点的状态呢？

别忘了在递归的过程中，系统栈会保存每一层递归的参数。

2. 确定终止条件

在遍历的过程中，如果遇到空间点的话，很明显，无论偷还是不偷都是0，所以就返回

```
if (cur == NULL) return vector<int>{0, 0};
```

3. 确定遍历顺序

采用后序遍历，代码如下：

```
// 下标0：不偷，下标1：偷
vector<int> left = robTree(cur->left); // 左
vector<int> right = robTree(cur->right); // 右
// 中
```

4. 确定单层递归的逻辑

如果是偷当前节点，那么左右孩子就不能偷，val1 = cur->val + left[0] + right[0];

如果不偷当前节点，那么左右孩子就可以偷，至于到底偷不偷一定是选一个最大的，所以：val2 = max(left[0], left[1]) + max(right[0], right[1]);

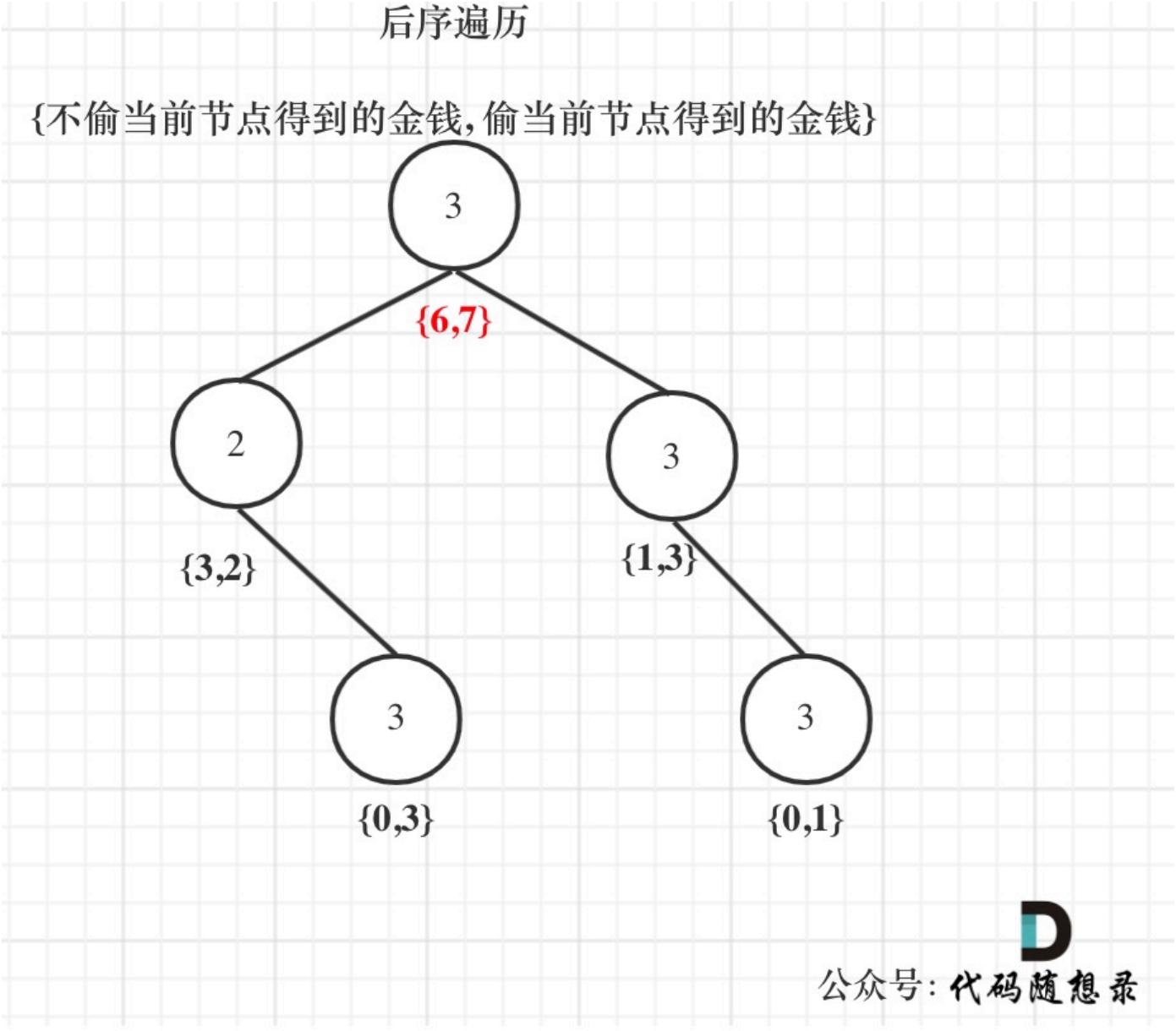
最后当前节点的状态就是{val2, val1}; 即：{不偷当前节点得到的最大金钱，偷当前节点得到的最大金钱}

代码如下：

```
vector<int> left = robTree(cur->left); // 左
vector<int> right = robTree(cur->right); // 右

// 偷cur
int val1 = cur->val + left[0] + right[0];
// 不偷cur
int val2 = max(left[0], left[1]) + max(right[0], right[1]);
return {val2, val1};
```

5. 举例推导dp数组

以示例1为例，dp数组状态如下：（**注意用后序遍历的方式推导**）



最后头结点就是 取下标0 和 下标1的最大值就是偷得的最大金钱。

树形DP为什么比较难呢?

因为平时我们习惯了在一维数组或者二维数组上推导公式，一下子换成了树，就需要对树的遍历方式足够了解！

大家还记不记得我在讲解贪心专题的时候，讲到这道题目：贪心算法：我要监控二叉树！，这也是贪心算法在树上的应用。**那我也可以把这个算法起一个名字，叫做树形贪心**，哈哈哈

“树形贪心”词汇从此诞生，来自「代码随想录」

周四

动态规划：买卖股票的最佳时机 一段时间，只能买卖一次，问最大收益。

这里我给出了三种解法：

暴力解法代码：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int result = 0;
        for (int i = 0; i < prices.size(); i++) {
            for (int j = i + 1; j < prices.size(); j++){
                result = max(result, prices[j] - prices[i]);
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

贪心解法代码如下：

因为股票就买卖一次，那么贪心的想法很自然就是取最左最小值，取最右最大值，那么得到的差值就是最大利润。

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int low = INT_MAX;
        int result = 0;
        for (int i = 0; i < prices.size(); i++) {
            low = min(low, prices[i]);  // 取最左最小价格
            result = max(result, prices[i] - low); // 直接取最大区间利润
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

动规解法，版本一，代码如下：

```
// 版本一
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(len, vector<int>(2));
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i][0] = max(dp[i - 1][0], -prices[i]);
            dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);
        }
        return dp[len - 1][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

从递推公式可以看出，dp[i]只是依赖于dp[i - 1]的状态。

那么我们只需要记录 当前天的dp状态和前一天的dp状态就可以了，可以使用滚动数组来节省空间，代码如下：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(2, vector<int>(2)); // 注意这里只开辟了一个2 * 2大小的二维数组
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i % 2][0] = max(dp[(i - 1) % 2][0], -prices[i]);
            dp[i % 2][1] = max(dp[(i - 1) % 2][1], prices[i] + dp[(i - 1) % 2][0]);
        }
        return dp[(len - 1) % 2][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

建议先写出版本一，然后在版本一的基础上优化成版本二，而不是直接就写出版本二。

总结

刚刚结束了背包问题，本周主要讲解打家劫舍系列。

劫舍系列简单来说就是 数组上连续元素二选一，成环之后连续元素二选一，在树上连续元素二选一，所能得到的最大价值。

那么这里每一种情况 我在文章中都做了详细的介绍。

周四我们开始讲解股票系列了，大家应该预测到了，我们下周的主题就是股票！ 哈哈哈，多么浮躁的一个系列！敬请期待吧！

代码随想录温馨提醒：投资有风险，入市需谨慎！

34.买卖股票的最佳时机II

力扣题目链接

给定一个数组，它的第 i 个元素是一支给定股票第 i 天的价格。

设计一个算法来计算你所能获取的最大利润。你可以尽可能地完成更多的交易（多次买卖一支股票）。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

- 示例 1:
- 输入: [7,1,5,3,6,4]
- 输出: 7
  解释: 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5-1 = 4。随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出, 这笔交易所能获得利润 = 6-3 = 3 。
- 示例 2:
- 输入: [1,2,3,4,5]
- 输出: 4
  解释: 在第 1 天（股票价格 = 1）的时候买入，在第 5 天 （股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5-1 = 4 。注意你不能在第 1 天和第 2 天接连购买股票，之后再将它们卖出。因为这样属于同时参与了多笔交易，你必须在再次购买前出售掉之前的股票。
- 示例 3:
- 输入: [7,6,4,3,1]
- 输出: 0
  解释: 在这种情况下, 没有交易完成, 所以最大利润为 0。

提示：

- 1 <= prices.length <= 3 * 10 ^ 4
- 0 <= prices[i] <= 10 ^ 4

算法公开课

《代码随想录》算法视频公开课：动态规划，股票问题第二弹 | LeetCode：122.买卖股票的最佳时机II，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题我们在讲解贪心专题的时候就已经讲解过了贪心算法：买卖股票的最佳时机II，只不过没有深入讲解动态规划的解法，那么这次我们再好好分析一下动规的解法。

本题和121. 买卖股票的最佳时机的唯一区别是本题股票可以买卖多次了（注意只有一只股票，所以再次购买前要出售掉之前的股票）

在动规五部曲中，这个区别主要是体现在递推公式上，其他都和121. 买卖股票的最佳时机一样一样的。

所以我们重点讲一讲递推公式。

这里重申一下dp数组的含义：

- dp[i][0] 表示第i天持有股票所得现金。
- dp[i][1] 表示第i天不持有股票所得最多现金

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是昨天不持有股票的所得现金减去 今天的股票价格 即：dp[i - 1][1] - prices[i]

注意这里和121. 买卖股票的最佳时机唯一不同的地方，就是推导dp[i][0]的时候，第i天买入股票的情况。

在121. 买卖股票的最佳时机中，因为股票全程只能买卖一次，所以如果买入股票，那么第i天持有股票即dp[i][0]一定就是 -prices[i]。

而本题，因为一只股票可以买卖多次，所以当第i天买入股票的时候，所持有的现金可能有之前买卖过的利润。

那么第i天持有股票即dp[i][0]，如果是第i天买入股票，所得现金就是昨天不持有股票的所得现金 减去 今天的股票价格 即：dp[i - 1][1] - prices[i]。

再来看看如果第i天不持有股票即dp[i][1]的情况， 依然可以由两个状态推出来

- 第i-1天就不持有股票，那么就保持现状，所得现金就是昨天不持有股票的所得现金 即：dp[i - 1][1]
- 第i天卖出股票，所得现金就是按照今天股票价格卖出后所得现金即：prices[i] + dp[i - 1][0]

注意这里和121. 买卖股票的最佳时机就是一样的逻辑，卖出股票收获利润（可能是负值）天经地义！

代码如下：（注意代码中的注释，标记了和121.买卖股票的最佳时机唯一不同的地方）

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(len, vector<int>(2, 0));
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]); // 注意这里是和121. 买卖股票的最佳时机唯一不同的地方。
```

```
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i]);
        }
        return dp[len - 1][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

大家可以本题和121. 买卖股票的最佳时机的代码几乎一样，唯一的区别在：

```
dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
```

这正是因为本题的股票可以买卖多次！ 所以买入股票的时候，可能会有之前买卖的利润即：dp[i - 1][1]，所以dp[i - 1][1] - prices[i]。

想到到这一点，对这两道题理解的就比较深刻了。

这里我依然给出滚动数组的版本，C++代码如下：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(2, vector<int>(2)); // 注意这里只开辟了一个2 * 2大小的二维数组
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i % 2][0] = max(dp[(i - 1) % 2][0], dp[(i - 1) % 2][1] - prices[i]);
            dp[i % 2][1] = max(dp[(i - 1) % 2][1], prices[i] + dp[(i - 1) % 2][0]);
        }
        return dp[(len - 1) % 2][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

35.买卖股票的最佳时机III

力扣题目链接

给定一个数组，它的第 i 个元素是一支给定的股票在第 i 天的价格。

设计一个算法来计算你所能获取的最大利润。你最多可以完成 两笔 交易。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

- 示例 1:

- 输入：prices = [3,3,5,0,0,3,1,4]
- 输出：6
  解释：在第 4 天（股票价格 = 0）的时候买入，在第 6 天（股票价格 = 3）的时候卖出，这笔交易所能获得利润 = 3-0 = 3 。随后，在第 7 天（股票价格 = 1）的时候买入，在第 8 天 （股票价格 = 4）的时候卖出，这笔交易所能获得利润 = 4-1 = 3。
- 示例 2：
- 输入：prices = [1,2,3,4,5]
- 输出：4
  解释：在第 1 天（股票价格 = 1）的时候买入，在第 5 天 （股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5-1 = 4。注意你不能在第 1 天和第 2 天接连购买股票，之后再将它们卖出。因为这样属于同时参与了多笔交易，你必须在再次购买前出售掉之前的股票。
- 示例 3：
- 输入：prices = [7,6,4,3,1]
- 输出：0
  解释：在这个情况下, 没有交易完成, 所以最大利润为0。
- 示例 4：
- 输入：prices = [1]
  输出：0

提示：

- 1 <= prices.length <= 10^5
- 0 <= prices[i] <= 10^5

算法公开课

《代码随想录》算法视频公开课：动态规划，股票至多买卖两次，怎么求？ | LeetCode：123.买卖股票最佳时机III，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目相对 121.买卖股票的最佳时机 和 122.买卖股票的最佳时机II 难了不少。

关键在于至多买卖两次，这意味着可以买卖一次，可以买卖两次，也可以不买卖。

接来下我用动态规划五部曲详细分析一下：

1. 确定dp数组以及下标的含义

一天一共就有五个状态，

0. 没有操作 （其实我们也可以不设置这个状态）
1. 第一次持有股票
2. 第一次不持有股票
3. 第二次持有股票
4. 第二次不持有股票

dp[i][j]中 i表示第i天，j为 [0 - 4] 五个状态，dp[i][j]表示第i天状态j所剩最大现金。

需要注意：dp[i][1]，**表示的是第i天，买入股票的状态，并不是说一定要第i天买入股票，这是很多同学容易陷入的误区**。

例如 dp[i][1] ，并不是说 第i天一定买入股票，有可能 第 i-1天 就买入了，那么 dp[i][1] 延续买入股票的这个状态。

2. 确定递推公式

达到dp[i][1]状态，有两个具体操作：

- 操作一：第i天买入股票了，那么dp[i][1] = dp[i-1][0] - prices[i]
- 操作二：第i天没有操作，而是沿用前一天买入的状态，即：dp[i][1] = dp[i - 1][1]

那么dp[i][1]究竟选 dp[i-1][0] - prices[i]，还是dp[i - 1][1]呢?

一定是选最大的，所以 dp[i][1] = max(dp[i-1][0] - prices[i], dp[i - 1][1]);

同理dp[i][2]也有两个操作：

- 操作一：第i天卖出股票了，那么dp[i][2] = dp[i - 1][1] + prices[i]
- 操作二：第i天没有操作，沿用前一天卖出股票的状态，即：dp[i][2] = dp[i - 1][2]

所以dp[i][2] = max(dp[i - 1][1] + prices[i], dp[i - 1][2])

同理可推出剩下状态部分：

dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);

dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);

3. dp数组如何初始化

第0天没有操作，这个最容易想到，就是0，即：dp[0][0] = 0;

第0天做第一次买入的操作，dp[0][1] = -prices[0];

第0天做第一次卖出的操作，这个初始值应该是多少呢?

此时还没有买入，怎么就卖出呢？ 其实大家可以理解当天买入，当天卖出，所以dp[0][2] = 0;

第0天第二次买入操作，初始值应该是多少呢？应该不少同学疑惑，第一次还没买入呢，怎么初始化第二次买入呢?

第二次买入依赖于第一次卖出的状态，其实相当于第0天第一次买入了，第一次卖出了，然后再买入一次（第二次买入），那么现在手头上没有现金，只要买入，现金就做相应的减少。

所以第二次买入操作，初始化为：dp[0][3] = -prices[0];

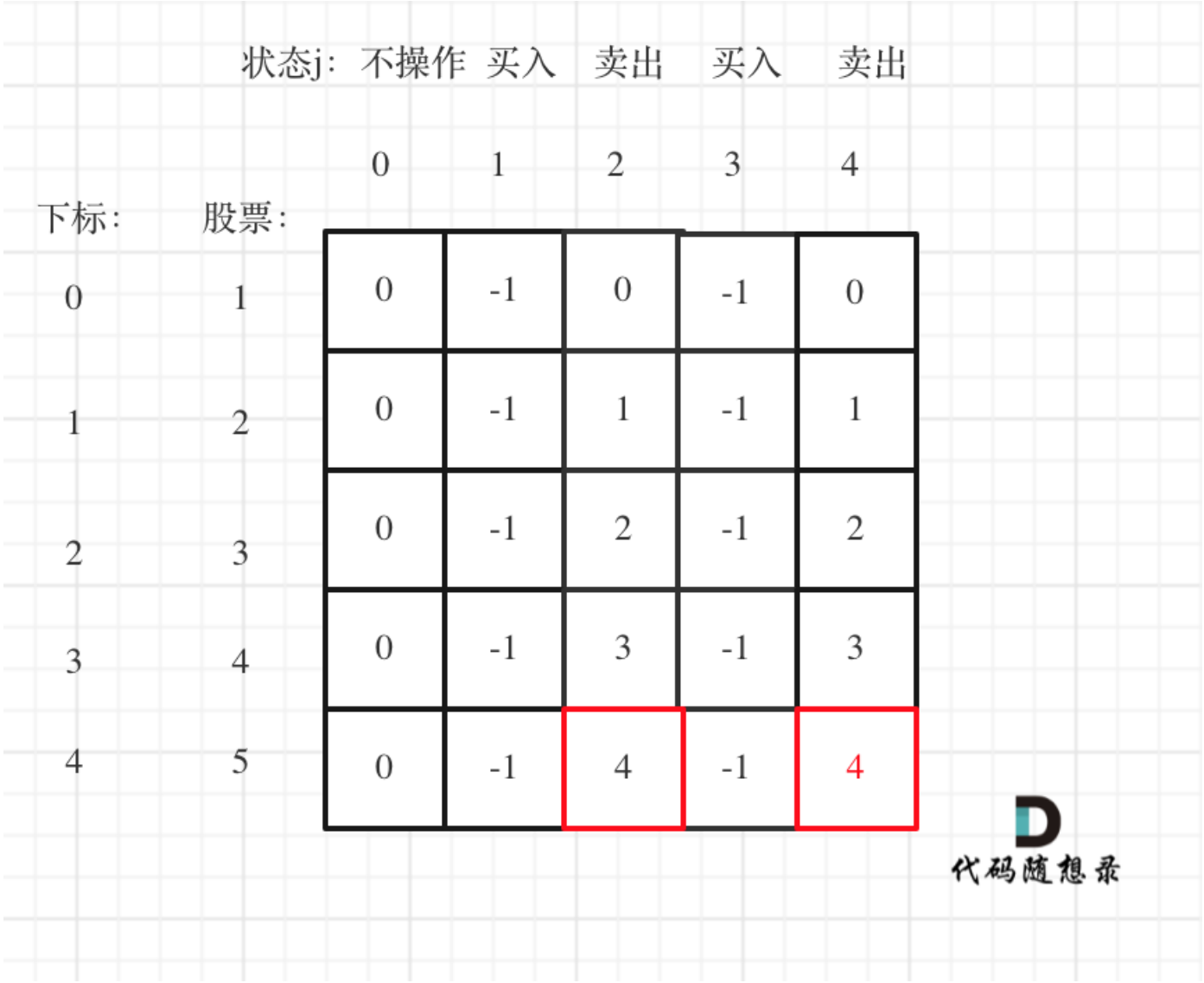
同理第二次卖出初始化dp[0][4] = 0;

4. 确定遍历顺序

从递归公式其实已经可以看出，一定是从前向后遍历，因为dp[i]，依靠dp[i - 1]的数值。

5. 举例推导dp数组

以输入[1,2,3,4,5]为例



大家可以看到红色框为最后两次卖出的状态。

现在最大的时候一定是卖出的状态，而两次卖出的状态现金最大一定是最后一次卖出。如果想不明白的录友也可以这么理解：如果第一次卖出已经是最大值了，那么我们可以在当天立刻买入再立刻卖出。所以dp[4][4]已经包含了dp[4][2]的情况。也就是说第二次卖出手里所剩的钱一定是最多的。

所以最终最大利润是dp[4][4]

以上五部都分析完了，不难写出如下代码：

```
// 版本一
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        if (prices.size() == 0) return 0;
        vector<vector<int>> dp(prices.size(), vector<int>(5, 0));
        dp[0][1] = -prices[0];
        dp[0][3] = -prices[0];
        for (int i = 1; i < prices.size(); i++) {
            dp[i][0] = dp[i - 1][0];
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] - prices[i]);
```

```
            dp[i][2] = max(dp[i - 1][2], dp[i - 1][1] + prices[i]);
            dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);
            dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);
        }
        return dp[prices.size() - 1][4];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n × 5)

当然，大家可以看到力扣官方题解里的一种优化空间写法，我这里给出对应的C++版本：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        if (prices.size() == 0) return 0;
        vector<int> dp(5, 0);
        dp[1] = -prices[0];
        dp[3] = -prices[0];
        for (int i = 1; i < prices.size(); i++) {
            dp[1] = max(dp[1], dp[0] - prices[i]);
            dp[2] = max(dp[2], dp[1] + prices[i]);
            dp[3] = max(dp[3], dp[2] - prices[i]);
            dp[4] = max(dp[4], dp[3] + prices[i]);
        }
        return dp[4];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

大家会发现dp[2]利用的是当天的dp[1]。 但结果也是对的。

我来简单解释一下：

dp[1] = max(dp[1], dp[0] - prices[i]); 如果dp[1]取dp[1]，即保持买入股票的状态，那么 dp[2] = max(dp[2], dp[1] + prices[i]);中dp[1] + prices[i] 就是今天卖出。

如果dp[1]取dp[0] - prices[i]，今天买入股票，那么dp[2] = max(dp[2], dp[1] + prices[i]);中的dp[1] + prices[i]相当于是今天再卖出股票，一买一卖收益为0，对所得现金没有影响。相当于今天买入股票又卖出股票，等于没有操作，保持昨天卖出股票的状态了。

这种写法看上去简单，其实思路很绕，不建议大家这么写，这么思考，很容易把自己绕进去！

对于本题，把版本一的写法研究明白，足以！

拓展

其实我们可以不设置，‘0. 没有操作’ 这个状态，因为没有操作，手上的现金自然就是0， 正如我们在 121.买卖股票的最佳时机 和 122.买卖股票的最佳时机II 也没有设置这一状态是一样的。

代码如下：

```
// 版本三
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        if (prices.size() == 0) return 0;
        vector<vector<int>> dp(prices.size(), vector<int>(5, 0));
        dp[0][1] = -prices[0];
        dp[0][3] = -prices[0];
        for (int i = 1; i < prices.size(); i++) {
            dp[i][1] = max(dp[i - 1][1], 0 - prices[i]);
            dp[i][2] = max(dp[i - 1][2], dp[i - 1][1] + prices[i]);
            dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);
            dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);
        }
        return dp[prices.size() - 1][4];
    }
};
```

36.买卖股票的最佳时机IV

力扣题目链接

给定一个整数数组 prices ，它的第 i 个元素 prices[i] 是一支给定的股票在第 i 天的价格。

设计一个算法来计算你所能获取的最大利润。你最多可以完成 k 笔交易。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

- 示例 1：
- 输入：k = 2, prices = [2,4,1]
- 输出：2
  解释：在第 1 天 (股票价格 = 2) 的时候买入，在第 2 天 (股票价格 = 4) 的时候卖出，这笔交易所能获得利润 = 4-2 = 2。
- 示例 2：
- 输入：k = 2, prices = [3,2,6,5,0,3]
- 输出：7
  解释：在第 2 天 (股票价格 = 2) 的时候买入，在第 3 天 (股票价格 = 6) 的时候卖出, 这笔交易所能获得利润 = 6-2 = 4。随后，在第 5 天 (股票价格 = 0) 的时候买入，在第 6 天 (股票价格 = 3) 的时候卖出, 这笔交易所能获得利润 = 3-0 = 3 。

提示：

- 0 <= k <= 100

- 0 <= prices.length <= 1000
- 0 <= prices[i] <= 1000

算法公开课

《代码随想录》算法视频公开课：动态规划来决定最佳时机，至多可以买卖K次！ | LeetCode：188.买卖股票最佳时机4，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题目可以说是动态规划：123.买卖股票的最佳时机III的进阶版，这里要求至多有k次交易。

动规五部曲，分析如下：

1. 确定dp数组以及下标的含义

在动态规划：123.买卖股票的最佳时机III中，我是定义了一个二维dp数组，本题其实依然可以用一个二维dp数组。

使用二维数组 dp[i][j] ： 第i天的状态为j，所剩下的最大现金是dp[i][j]

j的状态表示为：

- 0 表示不操作
- 1 第一次买入
- 2 第一次卖出
- 3 第二次买入
- 4 第二次卖出
- .....

大家应该发现规律了吧 ，除了0以外，偶数就是卖出，奇数就是买入。

题目要求是至多有K笔交易，那么j的范围就定义为 2 * k + 1 就可以了。

所以二维dp数组的C++定义为：

```
vector<vector<int>> dp(prices.size(), vector<int>(2 * k + 1, 0));
```

2. 确定递推公式

还要强调一下：dp[i][1]，**表示的是第i天，买入股票的状态，并不是说一定要第i天买入股票，这是很多同学容易陷入的误区。**

达到dp[i][1]状态，有两个具体操作：

- 操作一：第i天买入股票了，那么dp[i][1] = dp[i - 1][0] - prices[i]
- 操作二：第i天没有操作，而是沿用前一天买入的状态，即：dp[i][1] = dp[i - 1][1]

选最大的，所以 dp[i][1] = max(dp[i - 1][0] - prices[i], dp[i - 1][1]);

同理dp[i][2]也有两个操作：

- 操作一：第i天卖出股票了，那么dp[i][2] = dp[i - 1][1] + prices[i]

- 操作二：第i天没有操作，沿用前一天卖出股票的状态，即：dp[i][2] = dp[i - 1][2]

所以dp[i][2] = max(dp[i - 1][1] + prices[i], dp[i - 1][2])

同理可以类比剩下的状态，代码如下：

```
for (int j = 0; j < 2 * k - 1; j += 2) {
    dp[i][j + 1] = max(dp[i - 1][j + 1], dp[i - 1][j] - prices[i]);
    dp[i][j + 2] = max(dp[i - 1][j + 2], dp[i - 1][j + 1] + prices[i]);
}
```

本题和动态规划：123.买卖股票的最佳时机III最大的区别就是这里要类比j为奇数是买，偶数是卖的状态。

3. dp数组如何初始化

第0天没有操作，这个最容易想到，就是0，即：dp[0][0] = 0;

第0天做第一次买入的操作，dp[0][1] = -prices[0];

第0天做第一次卖出的操作，这个初始值应该是多少呢?

此时还没有买入，怎么就卖出呢？ 其实大家可以理解当天买入，当天卖出，所以dp[0][2] = 0;

第0天第二次买入操作，初始值应该是多少呢？应该不少同学疑惑，第一次还没买入呢，怎么初始化第二次买入呢?

第二次买入依赖于第一次卖出的状态，其实相当于第0天第一次买入了，第一次卖出了，然后在买入一次（第二次买入），那么现在手头上没有现金，只要买入，现金就做相应的减少。

所以第二次买入操作，初始化为：dp[0][3] = -prices[0];

第二次卖出初始化dp[0][4] = 0;

所以同理可以推出dp[0][j]当j为奇数的时候都初始化为 -prices[0]

代码如下：

```
for (int j = 1; j < 2 * k; j += 2) {
    dp[0][j] = -prices[0];
}
```

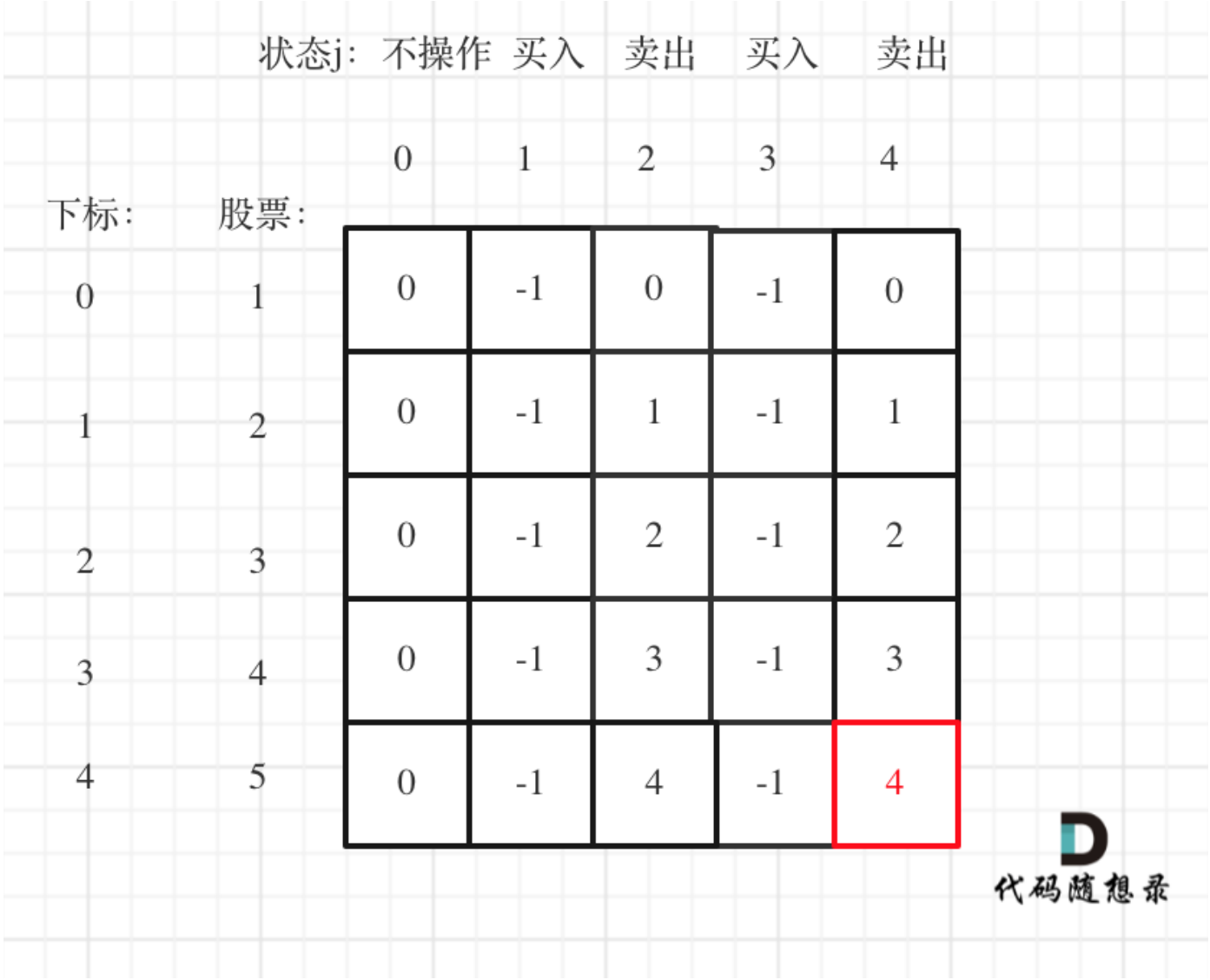
在初始化的地方同样要类比j为偶数是卖、奇数是买的状态。

4. 确定遍历顺序

从递归公式其实已经可以看出，一定是从前向后遍历，因为dp[i]，依靠dp[i - 1]的数值。

5. 举例推导dp数组

以输入[1,2,3,4,5]，k=2为例。



最后一次卖出，一定是利润最大的，dp[prices.size() - 1][2 * k]即红色部分就是最后求解。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int maxProfit(int k, vector<int>& prices) {

        if (prices.size() == 0) return 0;
        vector<vector<int>> dp(prices.size(), vector<int>(2 * k + 1, 0));
        for (int j = 1; j < 2 * k; j += 2) {
            dp[0][j] = -prices[0];
        }
        for (int i = 1;i < prices.size(); i++) {
            for (int j = 0; j < 2 * k - 1; j += 2) {
                dp[i][j + 1] = max(dp[i - 1][j + 1], dp[i - 1][j] - prices[i]);
                dp[i][j + 2] = max(dp[i - 1][j + 2], dp[i - 1][j + 1] + prices[i]);
            }
        }
        return dp[prices.size() - 1][2 * k];
    }
```

```
};
```

- 时间复杂度: O(n * k)，其中 n 为 prices 的长度
- 空间复杂度: O(n * k)

当然有的解法是定义一个三维数组dp[i][j][k]，第i天，第j次买卖，k表示买还是卖的状态，从定义上来讲是比较直观。

但感觉三维数组操作起来有些麻烦，我是直接用二维数组来模拟三维数组的情况，代码看起来也清爽一些。

37.最佳买卖股票时机含冷冻期

力扣题目链接

给定一个整数数组，其中第 i 个元素代表了第 i 天的股票价格 。

设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：

- 你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。
- 卖出股票后，你无法在第二天买入股票 (即冷冻期为 1 天)。

示例:

- 输入: [1,2,3,0,2]
- 输出: 3
- 解释: 对应的交易状态为: [买入, 卖出, 冷冻期, 买入, 卖出]

算法公开课

《代码随想录》算法视频公开课：动态规划来决定最佳时机，这次有冷冻期！| LeetCode：309.买卖股票的最佳时机含冷冻期，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

相对于动态规划：122.买卖股票的最佳时机II，本题加上了一个冷冻期

在动态规划：122.买卖股票的最佳时机II 中有两个状态，持有股票后的最多现金，和不持有股票的最多现金。

动规五部曲，分析如下：

1. 确定dp数组以及下标的含义

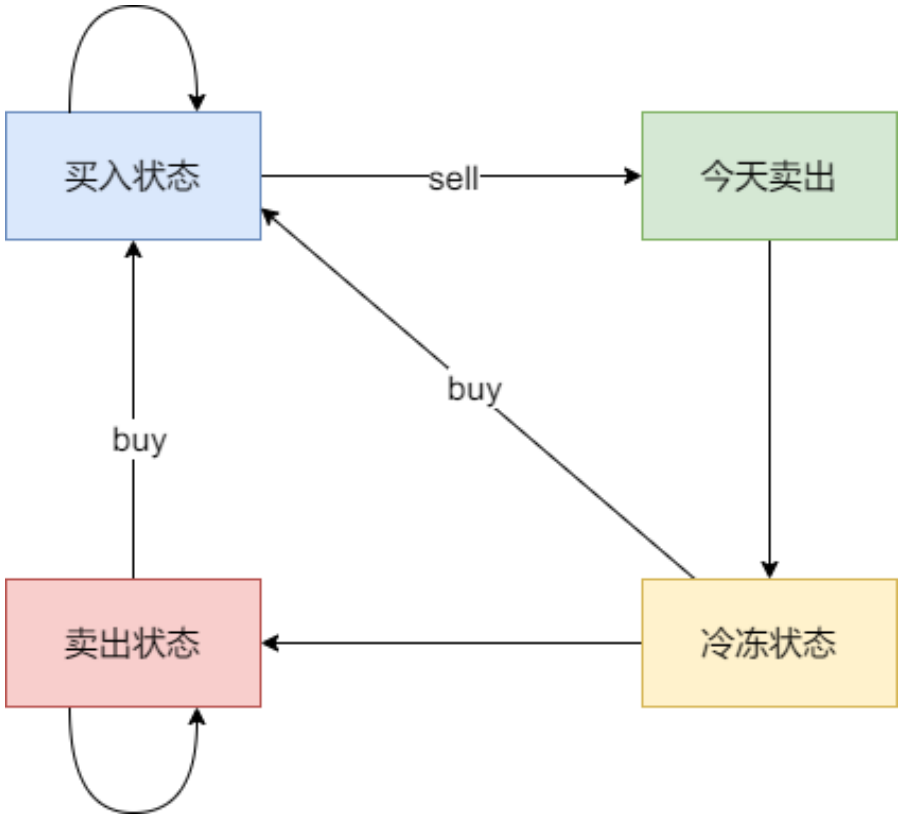
dp[i][j]，第i天状态为j，所剩的最多现金为dp[i][j]。

其实本题很多同学搞的比较懵，是因为出现冷冻期之后，状态其实是比较复杂度，例如今天买入股票、今天卖出股票、今天是冷冻期，都是不能操作股票的。

具体可以区分出如下四个状态：

- 状态一：持有股票状态（今天买入股票，或者是之前就买入了股票然后没有操作，一直持有）
- 不持有股票状态，这里就有两种卖出股票状态

    - 状态二：保持卖出股票的状态（两天前就卖出了股票，度过一天冷冻期。或者是前一天就是卖出股票状态，一直没操作）
    - 状态三：今天卖出股票
- 状态四：今天为冷冻期状态，但冷冻期状态不可持续，只有一天！



j的状态为：

- 0：状态一
- 1：状态二
- 2：状态三
- 3：状态四

很多题解为什么讲的比较模糊，是因为把这四个状态合并成三个状态了，其实就是把状态二和状态四合并在一起了。

从代码上来看确实可以合并，但从逻辑上分析合并之后就很难理解了，所以我下面的讲解是按照这四个状态来的，把每一个状态分析清楚。

如果大家按照代码随想录顺序来刷的话，会发现 买卖股票最佳时机 1，2，3，4 的题目讲解中

- 动态规划：121.买卖股票的最佳时机
- 动态规划：122.买卖股票的最佳时机II
- 动态规划：123.买卖股票的最佳时机III
- 动态规划：188.买卖股票的最佳时机IV

「今天卖出股票」我是没有单独列出一个状态的归类为「不持有股票的状态」，而本题为什么要单独列出「今天卖出股票」 一个状态呢?

因为本题我们有冷冻期，而冷冻期的前一天，只能是 「今天卖出股票」状态，如果是 「不持有股票状态」那么就很模糊，因为不一定是 卖出股票的操作。

如果没有按照 代码随想录 顺序去刷的录友，可能看这里的讲解 会有点困惑，建议把代码随想录本篇之前股票内容的讲解都看一下，领会一下每天 状态的设置。

注意这里的每一个状态，例如状态一，是持有股票股票状态并不是说今天一定就买入股票，而是说保持买入股票的状态即：可能是前几天买入的，之后一直没操作，所以保持买入股票的状态。

1. 确定递推公式

达到买入股票状态（状态一）即：dp[i][0]，有两个具体操作：

- 操作一：前一天就是持有股票状态（状态一），dp[i][0] = dp[i - 1][0]
- 操作二：今天买入了，有两种情况
  - 前一天是冷冻期（状态四），dp[i - 1][3] - prices[i]
  - 前一天是保持卖出股票的状态（状态二），dp[i - 1][1] - prices[i]

那么dp[i][0] = max(dp[i - 1][0], dp[i - 1][3] - prices[i], dp[i - 1][1] - prices[i]);

达到保持卖出股票状态（状态二）即：dp[i][1]，有两个具体操作：

- 操作一：前一天就是状态二
- 操作二：前一天是冷冻期（状态四）

dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);

达到今天就卖出股票状态（状态三），即：dp[i][2]，只有一个操作：

昨天一定是持有股票状态（状态一），今天卖出

即：dp[i][2] = dp[i - 1][0] + prices[i];

达到冷冻期状态（状态四），即：dp[i][3]，只有一个操作：

昨天卖出了股票（状态三）

dp[i][3] = dp[i - 1][2];

综上分析，递推代码如下：

```
dp[i][0] = max(dp[i - 1][0], max(dp[i - 1][3], dp[i - 1][1]) - prices[i]);
dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);
dp[i][2] = dp[i - 1][0] + prices[i];
dp[i][3] = dp[i - 1][2];
```

3. dp数组如何初始化

这里主要讨论一下第0天如何初始化。

如果是持有股票状态（状态一）那么：dp[0][0] = -prices[0]，一定是当天买入股票。

保持卖出股票状态（状态二），这里其实从 「状态二」的定义来说，很难明确应该初始多少，这种情况我们就看递推公式需要我们给他初始成什么数值。

如果i为1，第1天买入股票，那么递归公式中需要计算 dp[i - 1][1] - prices[i] ，即 dp[0][1] - prices[1]，那么大家感受一下 dp[0][1] （即第0天的状态二）应该初始成多少，只能初始为0。想一想如果初始为其他数值，是我们第1天买入股票后 手里还剩的现金数量是不是就不对了。

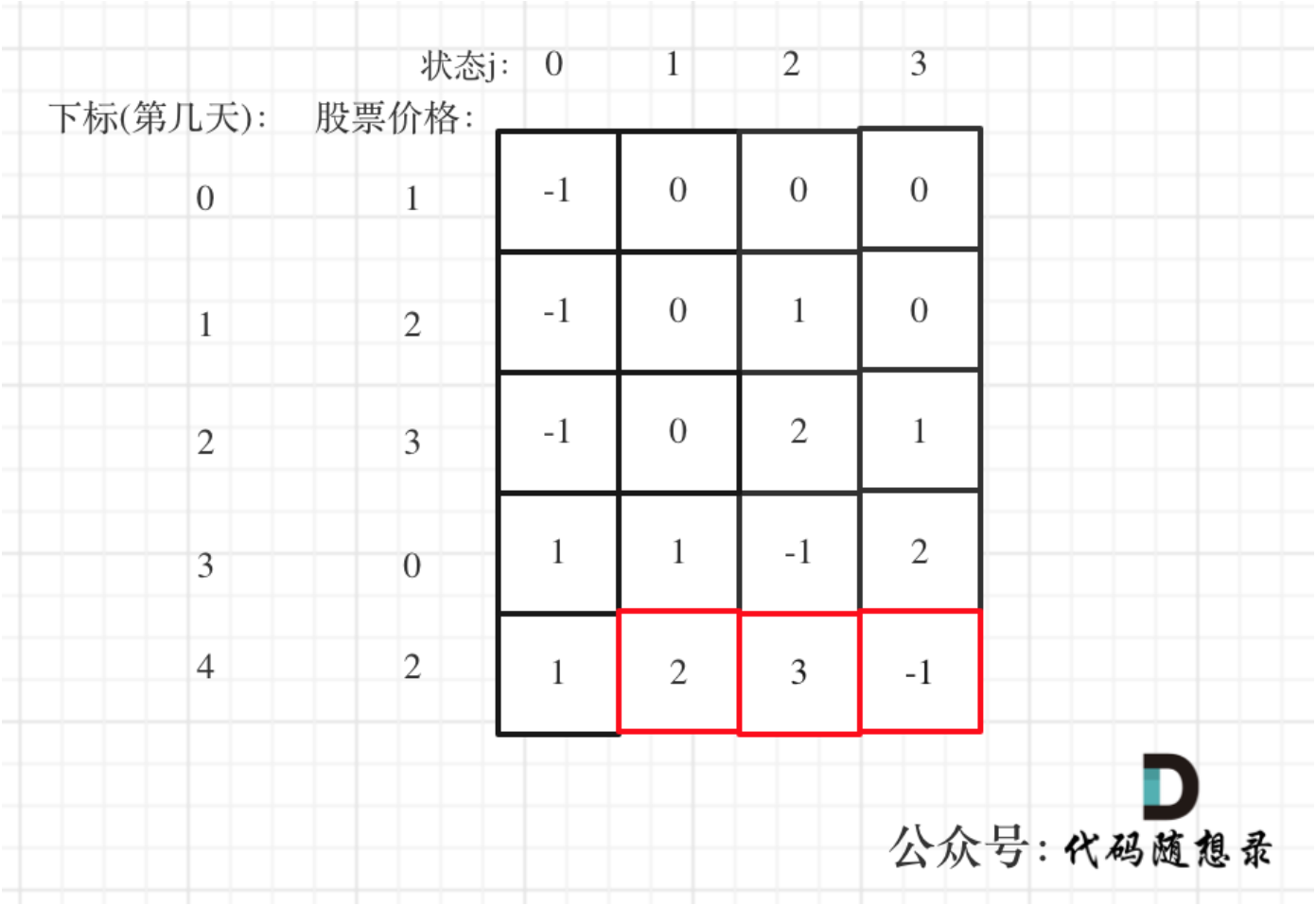
今天卖出了股票（状态三），同上分析，dp[0][2]初始化为0，dp[0][3]也初始为0。

4. 确定遍历顺序

从递归公式上可以看出，dp[i] 依赖于 dp[i-1]，所以是从前向后遍历。

5. 举例推导dp数组

以 [1,2,3,0,2] 为例，dp数组如下：



最后结果是取 状态二，状态三，和状态四的最大值，不少同学会把状态四忘了，状态四是冷冻期，最后一天如果是冷冻期也可能是最大值。

代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        if (n == 0) return 0;
        vector<vector<int>> dp(n, vector<int>(4, 0));
        dp[0][0] -= prices[0]; // 持股票
        for (int i = 1; i < n; i++) {
```

```
            dp[i][0] = max(dp[i - 1][0], max(dp[i - 1][3] - prices[i], dp[i - 1][1] -
prices[i]));
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);
            dp[i][2] = dp[i - 1][0] + prices[i];
            dp[i][3] = dp[i - 1][2];
        }
        return max(dp[n - 1][3], max(dp[n - 1][1], dp[n - 1][2]));
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

当然，空间复杂度可以优化，定义一个dp[2][4]大小的数组就可以了，就保存前一天的当前的状态，感兴趣的同学可以自己去写一写，思路是一样的。

总结

这次把冷冻期这道题目，讲的很透彻了，细分为四个状态，其状态转移也十分清晰，建议大家都按照四个状态来分析，如果只划分三个状态确实很容易给自己绕进去。

---

38. 本周小结！（动态规划系列七）

本周的主题就是股票系列，来一起回顾一下吧

周一

动态规划：买卖股票的最佳时机II中股票可以买卖多次了！

这也是和121. 买卖股票的最佳时机的唯一区别（注意只有一只股票，所以再次购买前要出售掉之前的股票）

重点在于递推公式的不同。

在回顾一下dp数组的含义：

- dp[i][0] 表示第i天持有股票所得现金。
- dp[i][1] 表示第i天不持有股票所得最多现金

递推公式：

```
dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i]);
```

大家可以发现本题和121. 买卖股票的最佳时机的代码几乎一样，唯一的区别在：

```
dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
```

这正是因为本题的股票可以买卖多次！ 所以买入股票的时候，可能会有之前买卖的利润即：dp[i - 1][1]，所以dp[i - 1][1] - prices[i]。

周二

动态规划：买卖股票的最佳时机III中最多只能完成两笔交易。

这意味着可以买卖一次，可以买卖两次，也可以不买卖。

1. 确定dp数组以及下标的含义

一天一共就有五个状态，

0. 没有操作
1. 第一次买入
2. 第一次卖出
3. 第二次买入
4. 第二次卖出

dp[i][j]中 i表示第i天，j为 [0 - 4] 五个状态，dp[i][j]表示第i天状态j所剩最大现金。

2. 确定递推公式

需要注意：dp[i][1]，**表示的是第i天，买入股票的状态，并不是说一定要第i天买入股票，这是很多同学容易陷入的误区。**

```
dp[i][1] = max(dp[i-1][0] - prices[i], dp[i - 1][1]);
dp[i][2] = max(dp[i - 1][1] + prices[i], dp[i - 1][2]);
dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);
dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);
```

3. dp数组如何初始化

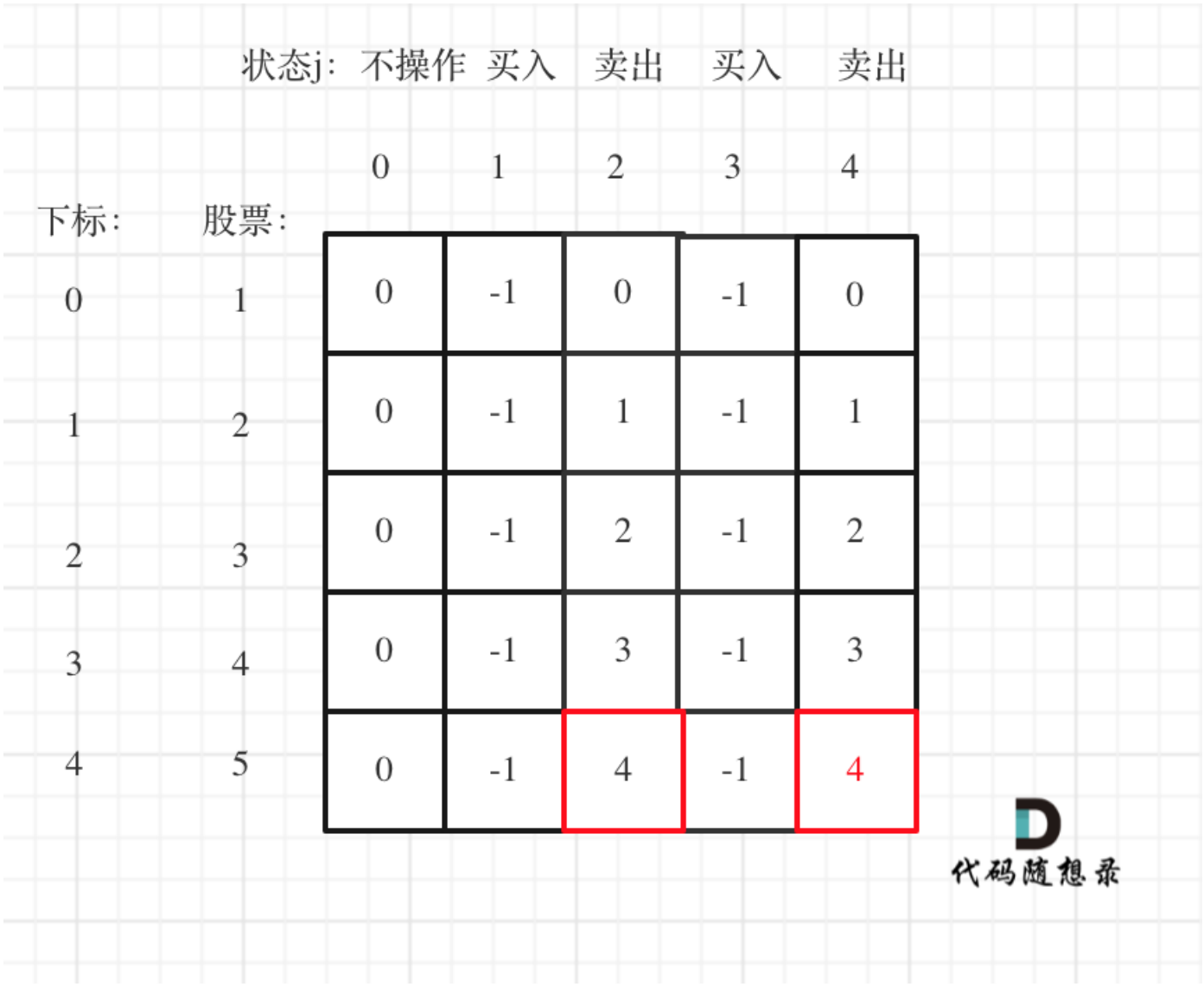
dp[0][0] = 0;
dp[0][1] = -prices[0];
dp[0][2] = 0;
dp[0][3] = -prices[0];
dp[0][4] = 0;

4. 确定遍历顺序

从递归公式其实已经可以看出，一定是从前向后遍历，因为dp[i]，依靠dp[i - 1]的数值。

5. 举例推导dp数组

以输入[1,2,3,4,5]为例



可以看到红色框为最后两次卖出的状态。

现在最大的时候一定是卖出的状态，而两次卖出的状态现金最大一定是最后一次卖出。

所以最终最大利润是dp[4][4]

周三

动态规划：买卖股票的最佳时机IV最多可以完成 k 笔交易。

相对于上一道动态规划：123.买卖股票的最佳时机III，本题需要通过前两次的交易，来类比前k次的交易

1. 确定dp数组以及下标的含义

使用二维数组 dp[i][j]： 第i天的状态为j，所剩下的最大现金是dp[i][j]

j的状态表示为：

- 0 表示不操作
- 1 第一次买入
- 2 第一次卖出
- 3 第二次买入

- 4 第二次卖出
- .....

除了0以外，偶数就是卖出，奇数就是买入。

2. 确定递推公式

还要强调一下：dp[i][1]，**表示的是第i天，买入股票的状态，并不是说一定要第i天买入股票，这是很多同学容易陷入的误区。**

```
for (int j = 0; j < 2 * k - 1; j += 2) {
    dp[i][j + 1] = max(dp[i - 1][j + 1], dp[i - 1][j] - prices[i]);
    dp[i][j + 2] = max(dp[i - 1][j + 2], dp[i - 1][j + 1] + prices[i]);
}
```

本题和动态规划：123.买卖股票的最佳时机III最大的区别就是这里要类比j为奇数是买，偶数是卖的状态。

3. dp数组如何初始化

dp[0][j]当j为奇数的时候都初始化为 -prices[0]

代码如下：

```
for (int j = 1; j < 2 * k; j += 2) {
    dp[0][j] = -prices[0];
}
```

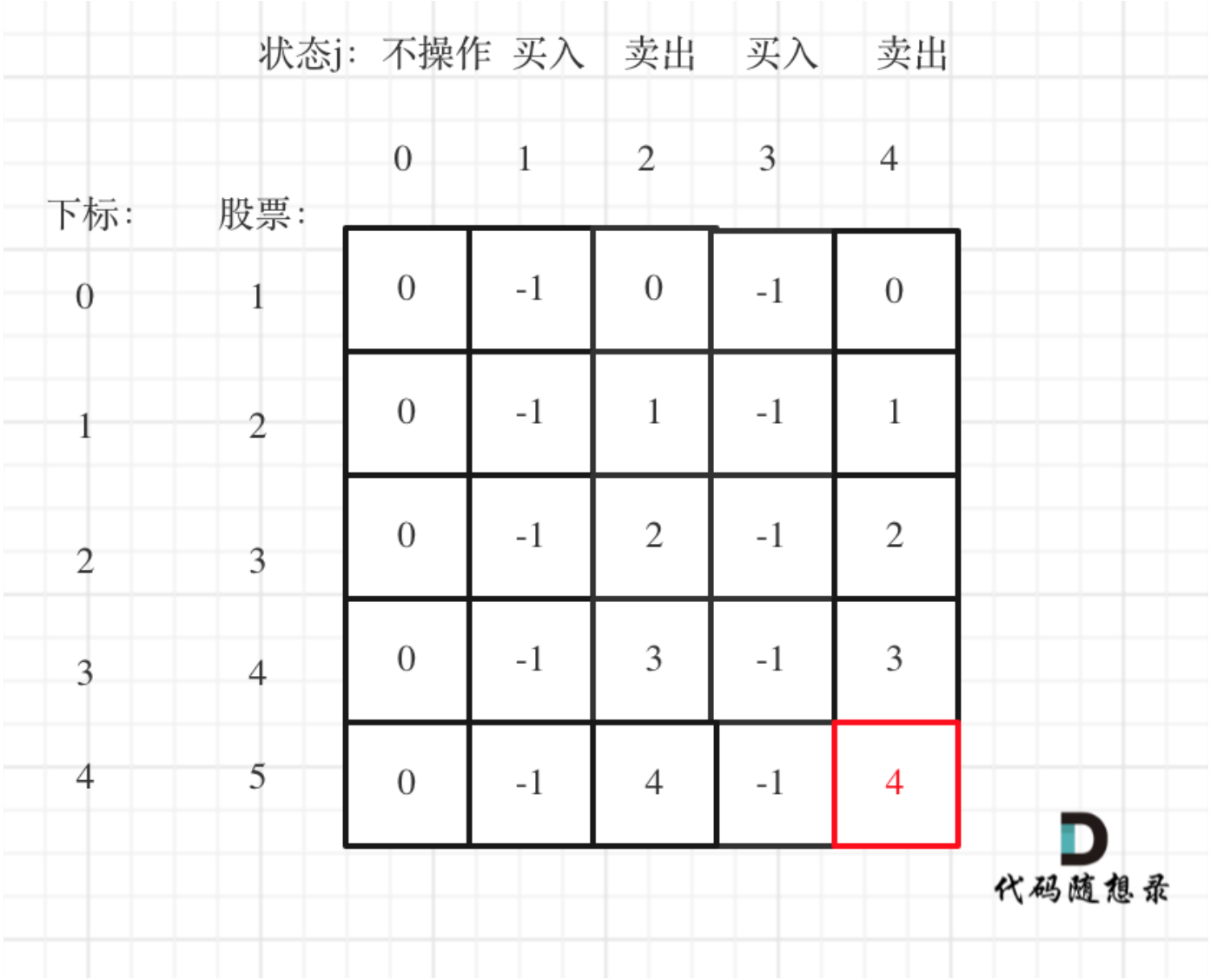
在初始化的地方同样要类比j为奇数是买、偶数是卖的状态。

4. 确定遍历顺序

从递归公式其实已经可以看出，一定是从前向后遍历，因为dp[i]，依靠dp[i - 1]的数值。

5. 举例推导dp数组

以输入[1,2,3,4,5]，k=2为例。



最后一次卖出，一定是利润最大的，dp[prices.size() - 1][2 * k]即红色部分就是最后求解。

周四

动态规划：最佳买卖股票时机含冷冻期尽可能地完成更多的交易（多次买卖一支股票），但有冷冻期，冷冻期为1天

相对于动态规划：122.买卖股票的最佳时机II，本题加上了一个冷冻期

本题则需要第三个状态：不持有股票（冷冻期）的最多现金。

动规五部曲，分析如下：

1. 确定dp数组以及下标的含义

dp[i][j]，第i天状态为j，所剩的最多现金为dp[i][j]。

j的状态为：

- 0：持有股票后的最多现金
- 1：不持有股票（能购买）的最多现金
- 2：不持有股票（冷冻期）的最多现金

2. 确定递推公式

```
dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
dp[i][1] = max(dp[i - 1][1], dp[i - 1][2]);
dp[i][2] = dp[i - 1][0] + prices[i];
```

3. dp数组如何初始化

可以统一都初始为0了。

代码如下：

```
vector<vector<int>> dp(n, vector<int>(3, 0));
```

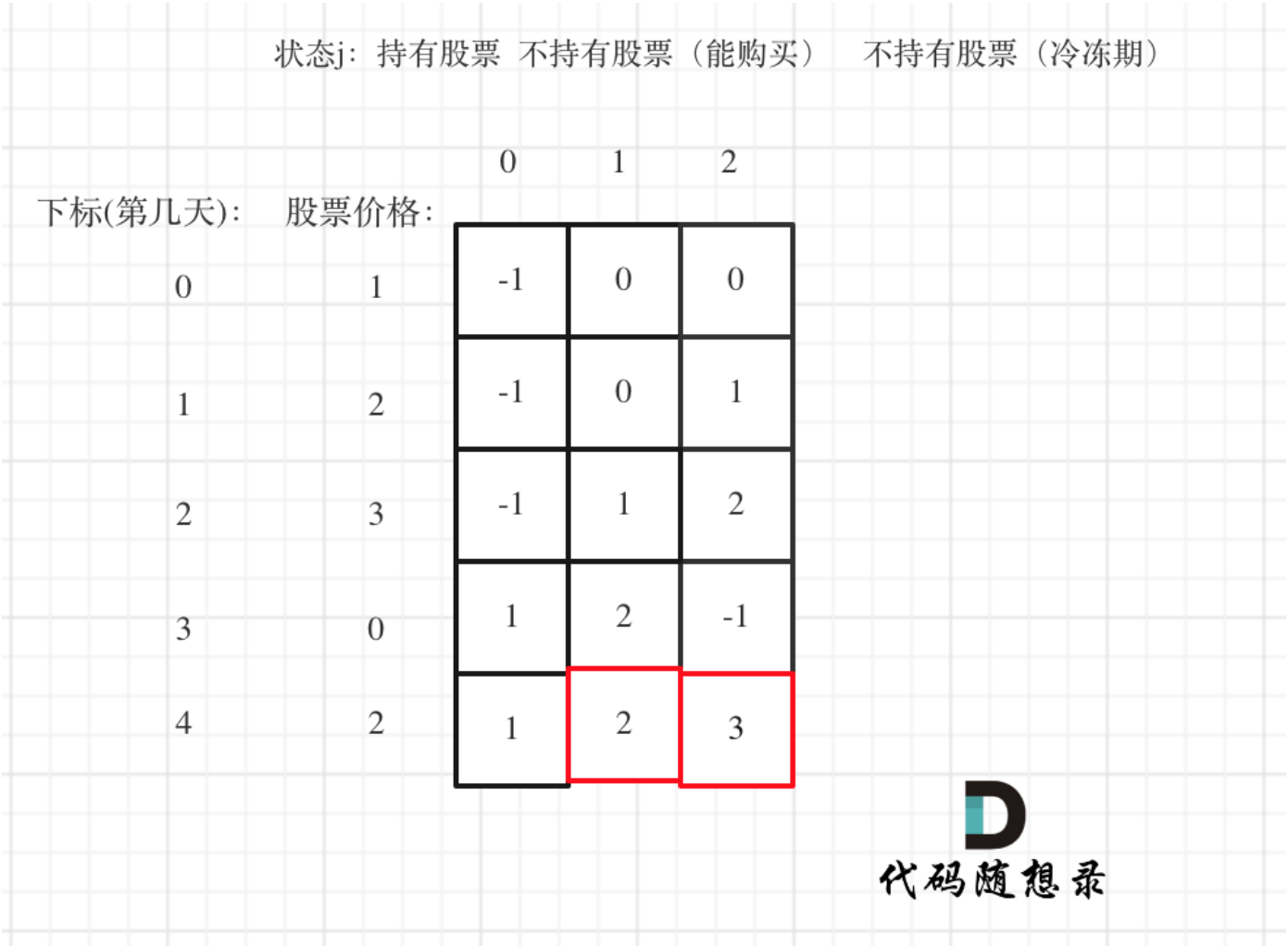
初始化其实很有讲究，很多同学可能是稀里糊涂的全都初始化0，反正就可以通过，但没有想清楚，为什么都初始化为0。

4. 确定遍历顺序

从递归公式上可以看出，dp[i] 依赖于 dp[i-1]，所以是从前向后遍历。

5. 举例推导dp数组

以 [1,2,3,0,2] 为例，dp数组如下：



最后两个状态 不持有股票（能购买） 和 不持有股票（冷冻期）都有可能最后结果，取最大的。

总结

下周还会有一篇股票系列的文章，**股票系列后面我也会单独写一篇总结，来高度概括一下，这样大家会对股票问题就有一个整体性的理解了**。

39.买卖股票的最佳时机含手续费

力扣题目链接

给定一个整数数组 prices，其中第 i 个元素代表了第 i 天的股票价格 ； 非负整数 fee 代表了交易股票的手续费用。

你可以无限次地完成交易，但是你每笔交易都需要付手续费。如果你已经购买了一个股票，在卖出它之前你就不能再继续购买股票了。

返回获得利润的最大值。

注意：这里的一笔交易指买入持有并卖出股票的整个过程，每笔交易你只需要为支付一次手续费。

示例 1:

- 输入: prices = [1, 3, 2, 8, 4, 9], fee = 2
- 输出: 8

解释: 能够达到的最大利润:

- 在此处买入 prices[0] = 1
- 在此处卖出 prices[3] = 8
- 在此处买入 prices[4] = 4
- 在此处卖出 prices[5] = 9
- 总利润: ((8 - 1) - 2) + ((9 - 4) - 2) = 8.

注意:

- 0 < prices.length <= 50000.
- 0 < prices[i] < 50000.
- 0 <= fee < 50000.

算法公开课

《代码随想录》算法视频公开课：动态规划来决定最佳时机，这次含手续费！ | LeetCode：714.买卖股票的最佳时机含手续费，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题贪心解法：贪心算法：买卖股票的最佳时机含手续费

性能是：

- 时间复杂度：O(n)
- 空间复杂度：O(1)

本题使用贪心算法并不好理解，也很容易出错，那么我们再来看看是使用动规的方法如何解题。

相对于动态规划：122.买卖股票的最佳时机II，本题只需要在计算卖出操作的时候减去手续费就可以了，代码几乎是一样的。

唯一差别在于递推公式部分，所以本篇也就不按照动规五部曲详细讲解了，主要讲解一下递推公式部分。

这里重申一下dp数组的含义：

dp[i][0] 表示第i天持有股票所省最多现金。
dp[i][1] 表示第i天不持有股票所得最多现金

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是昨天不持有股票的所得现金减去 今天的股票价格 即：dp[i - 1][1] - prices[i]

所以：dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);

在来看看如果第i天不持有股票即dp[i][1]的情况， 依然可以由两个状态推出来

- 第i-1天就不持有股票，那么就保持现状，所得现金就是昨天不持有股票的所得现金 即：dp[i - 1][1]
- 第i天卖出股票，所得现金就是按照今天股票价格卖出后所得现金，**注意这里需要有手续费了**即：dp[i - 1][0] + prices[i] - fee

所以：dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i] - fee);

本题和动态规划：122.买卖股票的最佳时机II的区别就是这里需要多一个减去手续费的操作。

以上分析完毕，C++代码如下：

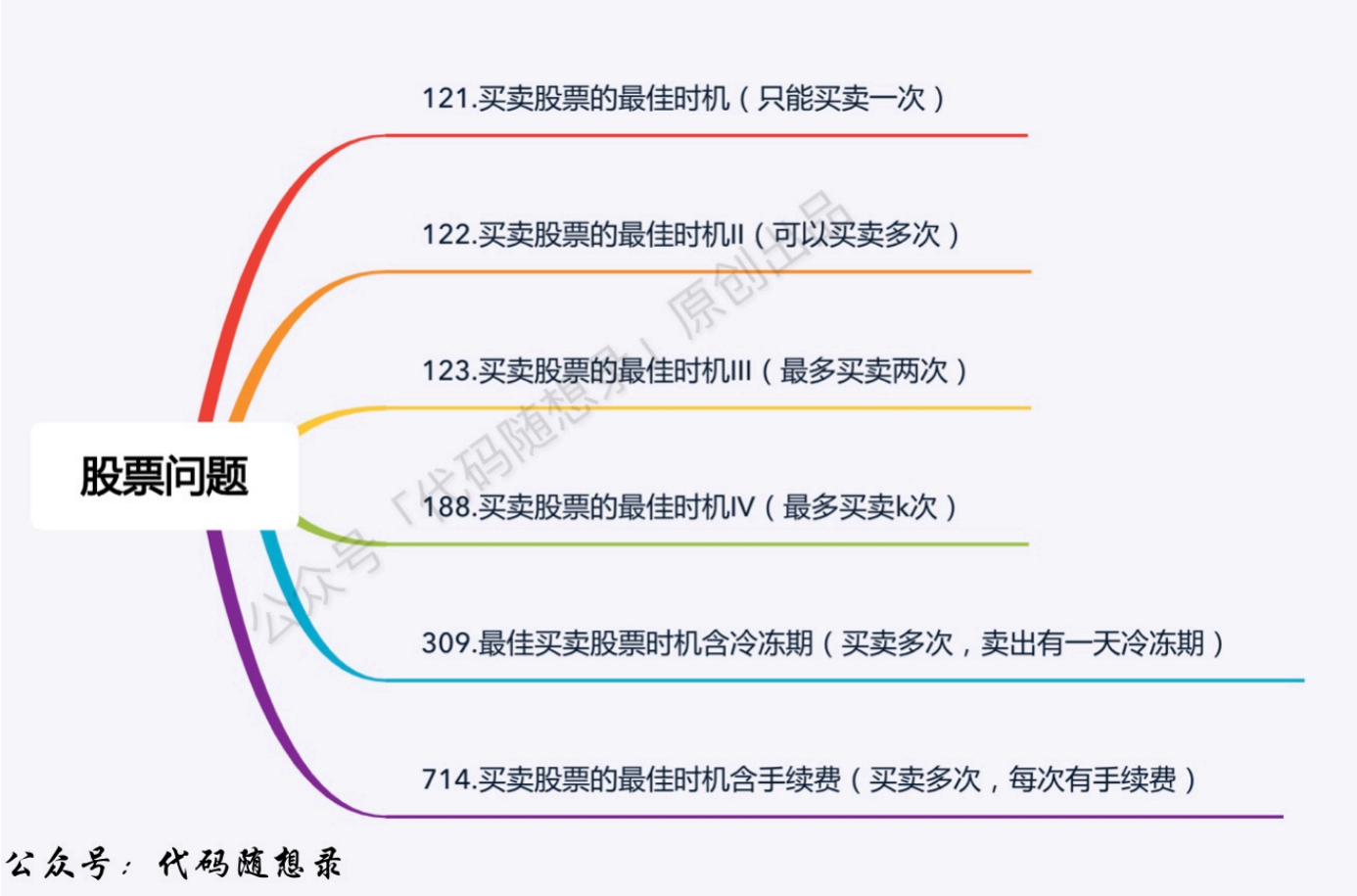
```
class Solution {
public:
    int maxProfit(vector<int>& prices, int fee) {
        int n = prices.size();
        vector<vector<int>> dp(n, vector<int>(2, 0));
        dp[0][0] -= prices[0]; // 持股票
        for (int i = 1; i < n; i++) {
            dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i] - fee);
        }
        return max(dp[n - 1][0], dp[n - 1][1]);
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

---

40. 股票问题总结篇!

之前我们已经把力扣上股票系列的题目都讲过的，但没有来一篇股票总结，来帮大家高屋建瓴，所以总结篇这就来了！



- 动态规划：121.买卖股票的最佳时机
- 动态规划：122.买卖股票的最佳时机II
- 动态规划：123.买卖股票的最佳时机III
- 动态规划：188.买卖股票的最佳时机IV
- 动态规划：309.最佳买卖股票时机含冷冻期
- 动态规划：714.买卖股票的最佳时机含手续费

卖股票的最佳时机

动态规划：121.买卖股票的最佳时机，**股票只能买卖一次，问最大利润。**

【贪心解法】

取最左最小值，取最右最大值，那么得到的差值就是最大利润，代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int low = INT_MAX;
        int result = 0;
        for (int i = 0; i < prices.size(); i++) {
            low = min(low, prices[i]);  // 取最左最小价格
            result = max(result, prices[i] - low); // 直接取最大区间利润
        }
        return result;
    }
};
```

【动态规划】

- dp[i][0] 表示第i天持有股票所得现金。
- dp[i][1] 表示第i天不持有股票所得现金。

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是买入今天的股票后所得现金即：-prices[i]
  所以dp[i][0] = max(dp[i - 1][0], -prices[i]);

如果第i天不持有股票即dp[i][1]， 也可以由两个状态推出来

- 第i-1天就不持有股票，那么就保持现状，所得现金就是昨天不持有股票的所得现金 即：dp[i - 1][1]
- 第i天卖出股票，所得现金就是按照今天股票佳价格卖出后所得现金即：prices[i] + dp[i - 1][0]
  所以dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);

代码如下：

```
// 版本一
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        if (len == 0) return 0;
        vector<vector<int>> dp(len, vector<int>(2));
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i][0] = max(dp[i - 1][0], -prices[i]);
            dp[i][1] = max(dp[i - 1][1], prices[i] + dp[i - 1][0]);
        }
        return dp[len - 1][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

使用滚动数组，代码如下：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(2, vector<int>(2)); // 注意这里只开辟了一个2 * 2大小的二维数组
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i % 2][0] = max(dp[(i - 1) % 2][0], -prices[i]);
            dp[i % 2][1] = max(dp[(i - 1) % 2][1], prices[i] + dp[(i - 1) % 2][0]);
        }
        return dp[(len - 1) % 2][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

买卖股票的最佳时机II

动态规划：122.买卖股票的最佳时机II可以多次买卖股票，问最大收益。

【贪心解法】

收集每天的正利润便可，代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int result = 0;
        for (int i = 1; i < prices.size(); i++) {
            result += max(prices[i] - prices[i - 1], 0);
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

【动态规划】

dp数组定义：

- dp[i][0] 表示第i天持有股票所得现金
- dp[i][1] 表示第i天不持有股票所得最多现金

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是昨天不持有股票的所得现金减去 今天的股票价格 即：dp[i - 1][1] - prices[i]

注意这里和121. 买卖股票的最佳时机唯一不同的地方，就是推导dp[i][0]的时候，第i天买入股票的情况。

在121. 买卖股票的最佳时机中，因为股票全程只能买卖一次，所以如果买入股票，那么第i天持有股票即dp[i][0]一定就是 -prices[i]。

而本题，因为一只股票可以买卖多次，所以当第i天买入股票的时候，所持有的现金可能有之前买卖过的利润。

代码如下：（注意代码中的注释，标记了和121.买卖股票的最佳时机唯一不同的地方）

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int len = prices.size();
        vector<vector<int>> dp(len, vector<int>(2, 0));
        dp[0][0] -= prices[0];
        dp[0][1] = 0;
        for (int i = 1; i < len; i++) {
            dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]); // 注意这里是和121. 买卖股票的最佳时机唯一不同的地方。
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i]);
        }
        return dp[len - 1][1];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

买卖股票的最佳时机III

动态规划：123.买卖股票的最佳时机III最多买卖两次，问最大收益。

【动态规划】

一天一共就有五个状态，

0. 没有操作
1. 第一次买入
2. 第一次卖出
3. 第二次买入
4. 第二次卖出

dp[i][j]中 i表示第i天，j为 [0 - 4] 五个状态，dp[i][j]表示第i天状态j所剩最大现金。

达到dp[i][1]状态，有两个具体操作：

- 操作一：第i天买入股票了，那么dp[i][1] = dp[i-1][0] - prices[i]
- 操作二：第i天没有操作，而是沿用前一天买入的状态，即：dp[i][1] = dp[i - 1][1]

dp[i][1] = max(dp[i-1][0] - prices[i], dp[i - 1][1]);

同理dp[i][2]也有两个操作：

- 操作一：第i天卖出股票了，那么dp[i][2] = dp[i - 1][1] + prices[i]
- 操作二：第i天没有操作，沿用前一天卖出股票的状态，即：dp[i][2] = dp[i - 1][2]

所以dp[i][2] = max(dp[i - 1][1] + prices[i], dp[i - 1][2])

同理可推出剩下状态部分：

dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);

dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);

代码如下：

```
// 版本一
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        if (prices.size() == 0) return 0;
        vector<vector<int>> dp(prices.size(), vector<int>(5, 0));
        dp[0][1] = -prices[0];
        dp[0][3] = -prices[0];
        for (int i = 1; i < prices.size(); i++) {
            dp[i][0] = dp[i - 1][0];
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] - prices[i]);
            dp[i][2] = max(dp[i - 1][2], dp[i - 1][1] + prices[i]);
            dp[i][3] = max(dp[i - 1][3], dp[i - 1][2] - prices[i]);
            dp[i][4] = max(dp[i - 1][4], dp[i - 1][3] + prices[i]);
        }
        return dp[prices.size() - 1][4];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n × 5)

当然，大家可以看到力扣官方题解里的一种优化空间写法，我这里给出对应的C++版本：

```
// 版本二
class Solution {
public:
    int maxProfit(vector<int>& prices) {
```

```
        if (prices.size() == 0) return 0;
        vector<int> dp(5, 0);
        dp[1] = -prices[0];
        dp[3] = -prices[0];
        for (int i = 1; i < prices.size(); i++) {
            dp[1] = max(dp[1], dp[0] - prices[i]);
            dp[2] = max(dp[2], dp[1] + prices[i]);
            dp[3] = max(dp[3], dp[2] - prices[i]);
            dp[4] = max(dp[4], dp[3] + prices[i]);
        }
        return dp[4];
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

这种写法看上去简单，其实思路很绕，不建议大家这么写，这么思考，很容易把自己绕进去！ 对于本题，把版本一的写法研究明白，足以！

买卖股票的最佳时机IV

动态规划：188.买卖股票的最佳时机IV 最多买卖k笔交易，问最大收益。

使用二维数组 dp[i][j]： 第i天的状态为j，所剩下的最大现金是dp[i][j]

j的状态表示为：

- 0 表示不操作
- 1 第一次买入
- 2 第一次卖出
- 3 第二次买入
- 4 第二次卖出
- .....

除了0以外，偶数就是卖出，奇数就是买入。

2. 确定递推公式

达到dp[i][1]状态，有两个具体操作：

- 操作一：第i天买入股票了，那么dp[i][1] = dp[i - 1][0] - prices[i]
- 操作二：第i天没有操作，而是沿用前一天买入的状态，即：dp[i][1] = dp[i - 1][1]

dp[i][1] = max(dp[i - 1][0] - prices[i], dp[i - 1][1]);

同理dp[i][2]也有两个操作：

- 操作一：第i天卖出股票了，那么dp[i][2] = dp[i - 1][1] + prices[i]
- 操作二：第i天没有操作，沿用前一天卖出股票的状态，即：dp[i][2] = dp[i - 1][2]

dp[i][2] = max(dp[i - 1][1] + prices[i], dp[i - 1][2])

同理可以类比剩下的状态，代码如下：

```
for (int j = 0; j < 2 * k - 1; j += 2) {
    dp[i][j + 1] = max(dp[i - 1][j + 1], dp[i - 1][j] - prices[i]);
    dp[i][j + 2] = max(dp[i - 1][j + 2], dp[i - 1][j + 1] + prices[i]);
}
```

整体代码如下：

```
class Solution {
public:
    int maxProfit(int k, vector<int>& prices) {

        if (prices.size() == 0) return 0;
        vector<vector<int>> dp(prices.size(), vector<int>(2 * k + 1, 0));
        for (int j = 1; j < 2 * k; j += 2) {
            dp[0][j] = -prices[0];
        }
        for (int i = 1;i < prices.size(); i++) {
            for (int j = 0; j < 2 * k - 1; j += 2) {
                dp[i][j + 1] = max(dp[i - 1][j + 1], dp[i - 1][j] - prices[i]);
                dp[i][j + 2] = max(dp[i - 1][j + 2], dp[i - 1][j + 1] + prices[i]);
            }
        }
        return dp[prices.size() - 1][2 * k];
    }
};
```

当然有的解法是定义一个三维数组dp[i][j][k]，第i天，第j次买卖，k表示买还是卖的状态，从定义上来讲是比较直观。但感觉三维数组操作起来有些麻烦，直接用二维数组来模拟三维数组的情况，代码看起来也清爽一些。

最佳买卖股票时机含冷冻期

动态规划：309.最佳买卖股票时机含冷冻期可以多次买卖但每次卖出有冷冻期1天。

相对于动态规划：122.买卖股票的最佳时机II，本题加上了一个冷冻期。

在动态规划：122.买卖股票的最佳时机II 中有两个状态，持有股票后的最多现金，和不持有股票的最多现金。本题则可以花费为四个状态

dp[i][j]：第i天状态为j，所剩的最多现金为dp[i][j]。

具体可以区分出如下四个状态：

- 状态一：买入股票状态（今天买入股票，或者是之前就买入了股票然后没有操作）
- 卖出股票状态，这里就有两种卖出股票状态
    - 状态二：两天前就卖出了股票，度过了冷冻期，一直没操作，今天保持卖出股票状态

    - 状态三：今天卖出了股票
- 状态四：今天为冷冻期状态，但冷冻期状态不可持续，只有一天！

达到买入股票状态（状态一）即：dp[i][0]，有两个具体操作：

- 操作一：前一天就是持有股票状态（状态一），dp[i][0] = dp[i - 1][0]
- 操作二：今天买入了，有两种情况
    - 前一天是冷冻期（状态四），dp[i - 1][3] - prices[i]
    - 前一天是保持卖出股票状态（状态二），dp[i - 1][1] - prices[i]

所以操作二取最大值，即：max(dp[i - 1][3], dp[i - 1][1]) - prices[i]

那么dp[i][0] = max(dp[i - 1][0], max(dp[i - 1][3], dp[i - 1][1]) - prices[i]);

达到保持卖出股票状态（状态二）即：dp[i][1]，有两个具体操作：

- 操作一：前一天就是状态二
- 操作二：前一天是冷冻期（状态四）

dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);

达到今天就卖出股票状态（状态三），即：dp[i][2]，只有一个操作：

- 操作一：昨天一定是买入股票状态（状态一），今天卖出

即：dp[i][2] = dp[i - 1][0] + prices[i];

达到冷冻期状态（状态四），即：dp[i][3]，只有一个操作：

- 操作一：昨天卖出了股票（状态三）

p[i][3] = dp[i - 1][2];

综上分析，递推代码如下：

```
dp[i][0] = max(dp[i - 1][0], max(dp[i - 1][3]- prices[i], dp[i - 1][1]) - prices[i];
dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);
dp[i][2] = dp[i - 1][0] + prices[i];
dp[i][3] = dp[i - 1][2];
```

整体代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        if (n == 0) return 0;
        vector<vector<int>> dp(n, vector<int>(4, 0));
        dp[0][0] -= prices[0]; // 持股票
        for (int i = 1; i < n; i++) {
            dp[i][0] = max(dp[i - 1][0], max(dp[i - 1][3], dp[i - 1][1]) - prices[i]);
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][3]);
```

```
            dp[i][2] = dp[i - 1][0] + prices[i];
            dp[i][3] = dp[i - 1][2];
        }
        return max(dp[n - 1][3],max(dp[n - 1][1], dp[n - 1][2]));
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

买卖股票的最佳时机含手续费

动态规划：714.买卖股票的最佳时机含手续费 可以多次买卖，但每次有手续费。

相对于动态规划：122.买卖股票的最佳时机II，本题只需要在计算卖出操作的时候减去手续费就可以了，代码几乎是一样的。

唯一差别在于递推公式部分，所以本篇也就不按照动规五部曲详细讲解了，主要讲解一下递推公式部分。

这里重申一下dp数组的含义：

dp[i][0] 表示第i天持有股票所省最多现金。
dp[i][1] 表示第i天不持有股票所得最多现金

如果第i天持有股票即dp[i][0]， 那么可以由两个状态推出来

- 第i-1天就持有股票，那么就保持现状，所得现金就是昨天持有股票的所得现金 即：dp[i - 1][0]
- 第i天买入股票，所得现金就是昨天不持有股票的所得现金减去 今天的股票价格 即：dp[i - 1][1] - prices[i]

所以：dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);

在来看看如果第i天不持有股票即dp[i][1]的情况， 依然可以由两个状态推出来

- 第i-1天就不持有股票，那么就保持现状，所得现金就是昨天不持有股票的所得现金 即：dp[i - 1][1]
- 第i天卖出股票，所得现金就是按照今天股票价格卖出后所得现金，**注意这里需要有手续费了**即：dp[i - 1][0] + prices[i] - fee

所以：dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i] - fee);

本题和动态规划：122.买卖股票的最佳时机II的区别就是这里需要多一个减去手续费的操作。

以上分析完毕，代码如下：

```
class Solution {
public:
    int maxProfit(vector<int>& prices, int fee) {
        int n = prices.size();
        vector<vector<int>> dp(n, vector<int>(2, 0));
        dp[0][0] -= prices[0]; // 持股票
        for (int i = 1; i < n; i++) {
            dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - prices[i]);
            dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + prices[i] - fee);
        }
        return max(dp[n - 1][0], dp[n - 1][1]);
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

总结

至此，股票系列正式剧终，全部讲解完毕！

从买卖一次到买卖多次，从最多买卖两次到最多买卖k次，从冷冻期再到手续费，最后再来一个股票大总结，可以说对股票系列完美收官了。

「代码随想录」值得推荐给身边每一位学习算法的朋友同学们，关注后都会发现相见恨晚！

41.最长递增子序列

力扣题目链接

给你一个整数数组 nums ，找到其中最长严格递增子序列的长度。

子序列是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，[3,6,2,7] 是数组 [0,3,1,6,2,2,7] 的子序列。

示例 1：

- 输入：nums = [10,9,2,5,3,7,101,18]
- 输出：4
- 解释：最长递增子序列是 [2,3,7,101]，因此长度为 4 。

示例 2：

- 输入：nums = [0,1,0,3,2,3]
- 输出：4

示例 3：

- 输入：nums = [7,7,7,7,7,7,7]

- 输出：1

提示：

- 1 <= nums.length <= 2500
- -10^4 <= nums[i] <= 104

算法公开课

[《代码随想录》算法视频公开课](#)：：[动态规划之子序列问题，元素不连续！ | LeetCode：300.最长递增子序列](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

思路

首先通过本题大家要明确什么是子序列，“子序列是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序”。

本题也是代码随想录中子序列问题的第一题，如果没接触过这种题目的话，本题还是很难的，甚至想暴力去搜索也不知道怎么搜。
子序列问题是动态规划解决的经典问题，当前下标i的递增子序列长度，其实和i之前的下表j的子序列长度有关系，那又是什么样的关系呢。

接下来，我们依然用动规五部曲来详细分析一波：

1. dp[i]的定义

本题中，正确定义dp数组的含义十分重要。

dp[i]表示i之前包括i的以nums[i]结尾的最长递增子序列的长度

为什么一定表示 “以nums[i]结尾的最长递增子序”，因为我们在 做 递增比较的时候，如果比较 nums[j] 和 nums[i] 的大小，那么两个递增子序列一定分别以nums[j]为结尾 和 nums[i]为结尾， 要不然这个比较就没有意义了，不是尾部元素的比较那么 如何算递增呢。

2. 状态转移方程

位置i的最长升序子序列等于j从0到i-1各个位置的最长升序子序列 + 1 的最大值。

所以：if (nums[i] > nums[j]) dp[i] = max(dp[i], dp[j] + 1);

注意这里不是要dp[i] 与 dp[j] + 1进行比较，而是我们要取dp[j] + 1的最大值。

3. dp[i]的初始化

每一个i，对应的dp[i]（即最长递增子序列）起始大小至少都是1.

4. 确定遍历顺序

dp[i] 是有0到i-1各个位置的最长递增子序列 推导而来，那么遍历i一定是从前向后遍历。

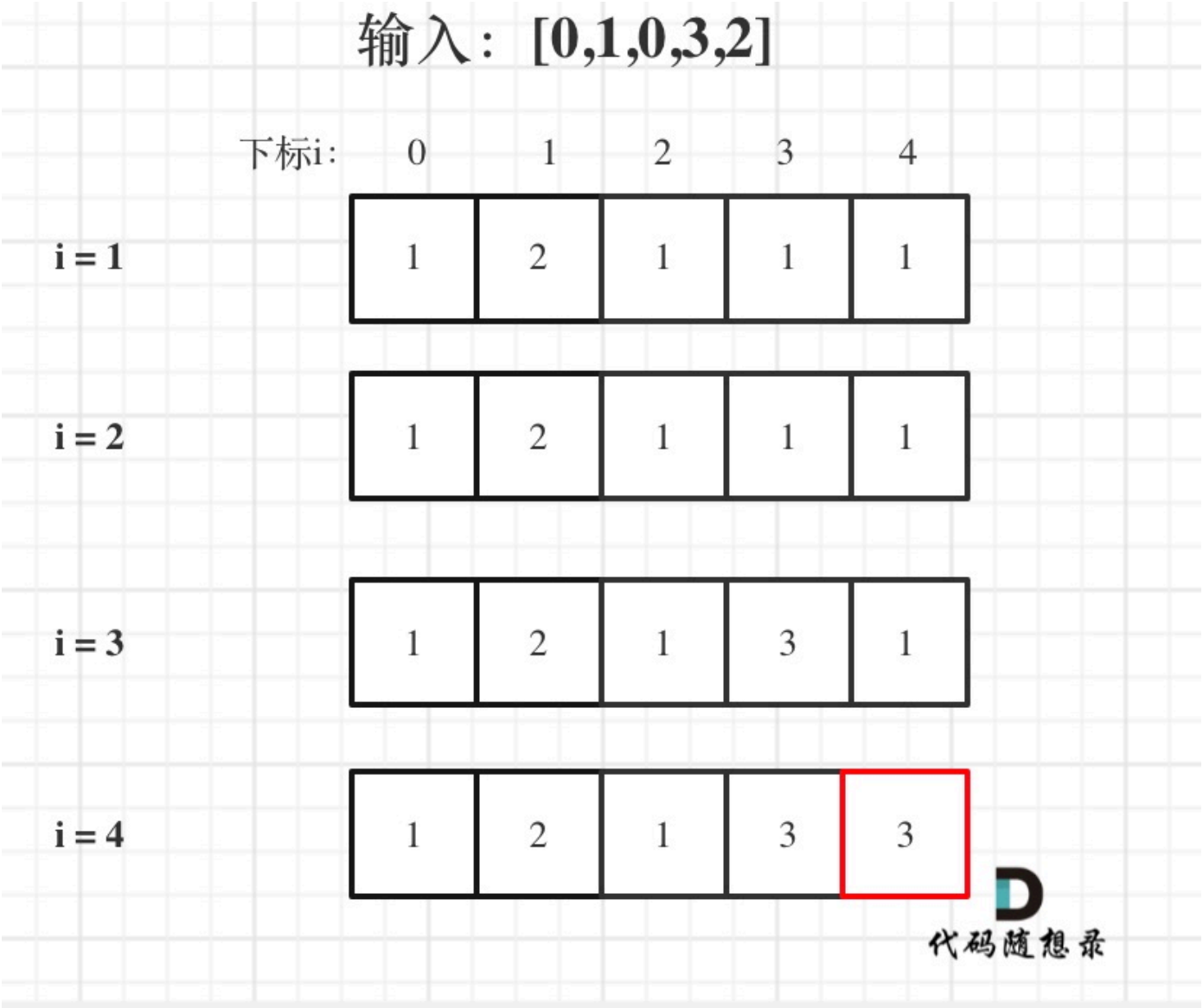
j其实就是遍历0到i-1，那么是从前到后，还是从后到前遍历都无所谓，只要吧 0 到 i-1 的元素都遍历了就行了。 所以默认习惯 从前向后遍历。

遍历i的循环在外层，遍历j则在内层，代码如下：

```
for (int i = 1; i < nums.size(); i++) {
    for (int j = 0; j < i; j++) {
        if (nums[i] > nums[j]) dp[i] = max(dp[i], dp[j] + 1);
    }
    if (dp[i] > result) result = dp[i]; // 取长的子序列
}
```

5. 举例推导dp数组

输入：[0,1,0,3,2]，dp数组的变化如下：



如果代码写出来，但一直AC不了，那么就把dp数组打印出来，看看对不对！

以上五部分析完毕，C++代码如下：

```
class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        if (nums.size() <= 1) return nums.size();
        vector<int> dp(nums.size(), 1);
```

```
        int result = 0;
        for (int i = 1; i < nums.size(); i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] > nums[j]) dp[i] = max(dp[i], dp[j] + 1);
            }
            if (dp[i] > result) result = dp[i]; // 取长的子序列
        }
        return result;
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(n)

总结

本题最关键的是要想到dp[i]由哪些状态可以推出来，并取最大值，那么很自然就能想到递推公式：dp[i] = max(dp[i], dp[j] + 1);

子序列问题是动态规划的一个重要系列，本题算是入门题目，好戏刚刚开始！

---

42. 最长连续递增序列

力扣题目链接

给定一个未经排序的整数数组，找到最长且 连续递增的子序列，并返回该序列的长度。

连续递增的子序列 可以由两个下标 l 和 r（l < r）确定，如果对于每个 l <= i < r，都有 nums[i] < nums[i + 1] ，那么子序列 [nums[l], nums[l + 1], ..., nums[r - 1], nums[r]] 就是连续递增子序列。

示例 1：

- 输入：nums = [1,3,5,4,7]
- 输出：3
- 解释：最长连续递增序列是 [1,3,5], 长度为3。尽管 [1,3,5,7] 也是升序的子序列, 但它不是连续的，因为 5 和 7 在原数组里被 4 隔开。

示例 2：

- 输入：nums = [2,2,2,2,2]
- 输出：1
- 解释：最长连续递增序列是 [2], 长度为1。

提示：

- 0 <= nums.length <= 10^4
- -10^9 <= nums[i] <= 10^9

算法公开课

[《代码随想录》算法视频公开课](#)：[动态规划之子序列问题，重点在于连续！ | LeetCode：674.最长连续递增序列](#)，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题相对于昨天的[动态规划：300.最长递增子序列](#)最大的区别在于“连续”。

本题要求的是最长**连续**递增序列

动态规划

动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i]：以下标i为结尾的连续递增的子序列长度为dp[i]。

注意这里的定义，一定是以下标i为结尾，并不是说一定以下标0为起始位置。

2. 确定递推公式

如果 nums[i] > nums[i - 1]，那么以 i 为结尾的连续递增的子序列长度 一定等于 以i - 1为结尾的连续递增的子序列长度 + 1 。

即：dp[i] = dp[i - 1] + 1;

注意这里就体现出和[动态规划：300.最长递增子序列](#)的区别！

因为本题要求连续递增子序列，所以就只要比较nums[i]与nums[i - 1]，而不用去比较nums[j]与nums[i] （j是在0到i之间遍历）。

既然不用j了，那么也不用两层for循环，本题一层for循环就行，比较nums[i] 和 nums[i - 1]。

这里大家要好好体会一下！

3. dp数组如何初始化

以下标i为结尾的连续递增的子序列长度最少也应该是1，即就是nums[i]这一个元素。

所以dp[i]应该初始1;

4. 确定遍历顺序

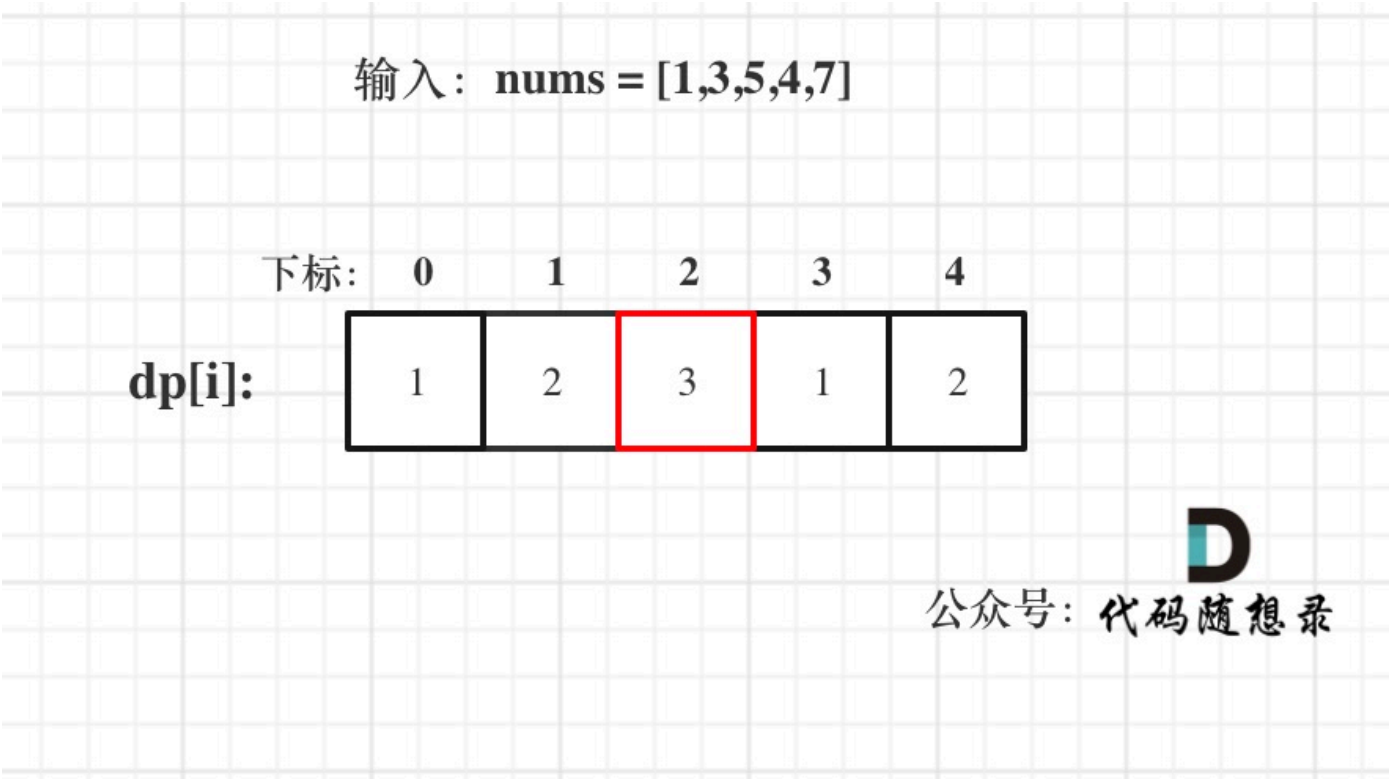
从递推公式上可以看出， dp[i + 1]依赖dp[i]，所以一定是从前向后遍历。

本文在确定递推公式的时候也说明了为什么本题只需要一层for循环，代码如下：

```
for (int i = 1; i < nums.size(); i++) {
    if (nums[i] > nums[i - 1]) { // 连续记录
        dp[i] = dp[i - 1] + 1;
    }
}
```

5. 举例推导dp数组

已输入nums = [1,3,5,4,7]为例，dp数组状态如下：



注意这里要取dp[i]里的最大值，所以dp[2]才是结果！

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int findLengthOfLCIS(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        int result = 1;
        vector<int> dp(nums.size() ,1);
        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] > nums[i - 1]) { // 连续记录
                dp[i] = dp[i - 1] + 1;
            }
            if (dp[i] > result) result = dp[i];
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

贪心

这道题目也可以用贪心来做，也就是遇到nums[i] > nums[i - 1]的情况，count就++，否则count为1，记录count的最大值就可以了。

代码如下：

```
class Solution {
public:
    int findLengthOfLCIS(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        int result = 1; // 连续子序列最少也是1
        int count = 1;
        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] > nums[i - 1]) { // 连续记录
                count++;
            } else { // 不连续，count从头开始
                count = 1;
            }
            if (count > result) result = count;
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

总结

本题也是动规里子序列问题的经典题目，但也可以用贪心来做，大家也会发现贪心好像更简单一点，而且空间复杂度仅是O(1)。

在动规分析中，关键是要理解和动态规划：300.最长递增子序列的区别。

要联动起来，才能理解递增子序列怎么求，递增连续子序列又要怎么求。

概括来说：不连续递增子序列的跟前0-i 个状态有关，连续递增的子序列只跟前一个状态有关

本篇我也把区别所在之处重点介绍了，关键在递推公式和遍历方法上，大家可以仔细体会一波！

43. 最长重复子数组

力扣题目链接

给两个整数数组 A 和 B ，返回两个数组中公共的、长度最长的子数组的长度。

示例：

输入：

- A: [1,2,3,2,1]
- B: [3,2,1,4,7]
- 输出：3
- 解释：长度最长的公共子数组是 [3, 2, 1] 。

提示：

- 1 <= len(A), len(B) <= 1000
- 0 <= A[i], B[i] < 100

算法公开课

《代码随想录》算法视频公开课：动态规划之子序列问题，想清楚DP数组的定义 | LeetCode：718.最长重复子数组，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

注意题目中说的子数组，其实就是连续子序列。

要求两个数组中最长重复子数组，如果是暴力的解法 只需要先两层for循环确定两个数组起始位置，然后再来一个循环可以是for或者while，来从两个起始位置开始比较，取得重复子数组的长度。

本题其实是动规解决的经典题目，我们只要想到 用二维数组可以记录两个字符串的所有比较情况，这样就比较好推递推公式了。
动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：以下标i - 1为结尾的A，和以下标j - 1为结尾的B，最长重复子数组长度为dp[i][j]。（**特别注意**：“以下标i - 1为结尾的A” 标明一定是 以A[i-1]为结尾的字符串 ）

此时细心的同学应该发现，那dp[0][0]是什么含义呢？总不能是以下标-1为结尾的A数组吧。

其实dp[i][j]的定义也就决定着，我们在遍历dp[i][j]的时候i 和 j都要从1开始。

那有同学问了，我就定义dp[i][j]为 以下标i为结尾的A，和以下标j 为结尾的B，最长重复子数组长度。不行么？

行倒是行！ 但实现起来就麻烦一点，需要单独处理初始化部分，在本题解下面的拓展内容里，我给出了 第二种 dp数组的定义方式所对应的代码和讲解，大家比较一下就了解了。

2. 确定递推公式

根据dp[i][j]的定义，dp[i][j]的状态只能由dp[i - 1][j - 1]推导出来。

即当A[i - 1] 和B[j - 1]相等的时候，dp[i][j] = dp[i - 1][j - 1] + 1;

根据递推公式可以看出，遍历i 和 j 要从1开始！

3. dp数组如何初始化

根据dp[i][j]的定义，dp[i][0] 和dp[0][j]其实都是没有意义的！

但dp[i][0] 和dp[0][j]要初始值，因为 为了方便递归公式dp[i][j] = dp[i - 1][j - 1] + 1;

所以dp[i][0] 和dp[0][j]初始化为0。

举个例子A[0]如果和B[0]相同的话，dp[1][1] = dp[0][0] + 1，只有dp[0][0]初始为0，正好符合递推公式逐步累加起来。

4. 确定遍历顺序

外层for循环遍历A，内层for循环遍历B。

那又有同学问了，外层for循环遍历B，内层for循环遍历A。不行么？

也行，一样的，我这里就用外层for循环遍历A，内层for循环遍历B了。

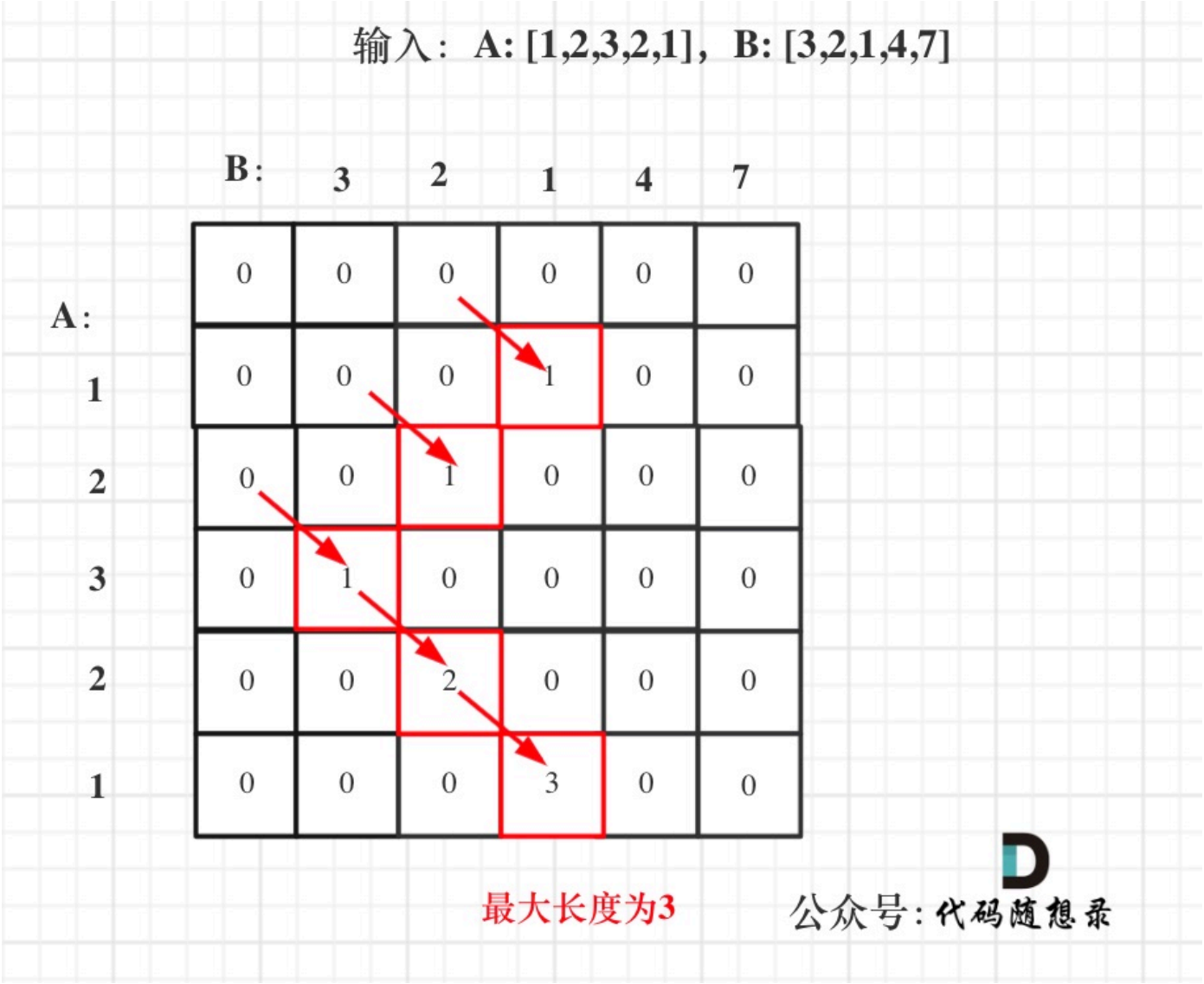
同时题目要求长度最长的子数组的长度。所以在遍历的时候顺便把dp[i][j]的最大值记录下来。

代码如下：

```
for (int i = 1; i <= nums1.size(); i++) {
    for (int j = 1; j <= nums2.size(); j++) {
        if (nums1[i - 1] == nums2[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1] + 1;
        }
        if (dp[i][j] > result) result = dp[i][j];
    }
}
```

5. 举例推导dp数组

拿示例1中，A: [1,2,3,2,1]，B: [3,2,1,4,7]为例，画一个dp数组的状态变化，如下：



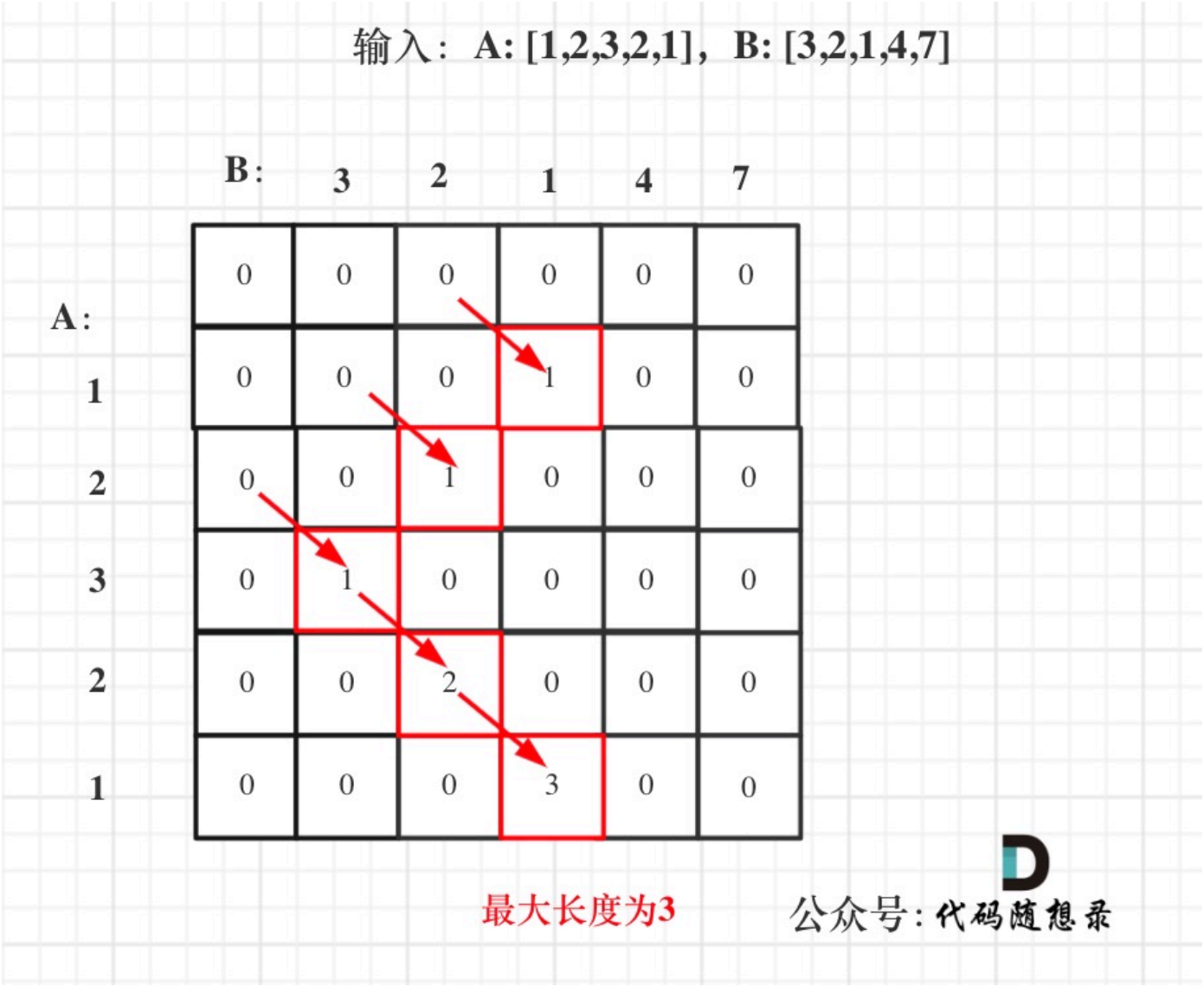
以上五部曲分析完毕，C++代码如下：

```
// 版本一
class Solution {
public:
    int findLength(vector<int>& nums1, vector<int>& nums2) {
        vector<vector<int>> dp (nums1.size() + 1, vector<int>(nums2.size() + 1, 0));
        int result = 0;
        for (int i = 1; i <= nums1.size(); i++) {
            for (int j = 1; j <= nums2.size(); j++) {
                if (nums1[i - 1] == nums2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                }
                if (dp[i][j] > result) result = dp[i][j];
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n × m)，n 为A长度，m为B长度
- 空间复杂度：O(n × m)

滚动数组

在如下图中：



我们可以看出dp[i][j]都是由dp[i - 1][j - 1]推出。那么压缩为一维数组，也就是dp[j]都是由dp[j - 1]推出。

也就是相当于可以把上一层dp[i - 1][j]拷贝到下一层dp[i][j]来继续用。

此时遍历B数组的时候，就要从后向前遍历，这样避免重复覆盖。

```
// 版本二
class Solution {
public:
    int findLength(vector<int>& A, vector<int>& B) {
        vector<int> dp(vector<int>(B.size() + 1, 0));
        int result = 0;
        for (int i = 1; i <= A.size(); i++) {
            for (int j = B.size(); j > 0; j--) {
```

```
                if (A[i - 1] == B[j - 1]) {
                    dp[j] = dp[j - 1] + 1;
                } else dp[j] = 0; // 注意这里不相等的时候要有赋0的操作
                if (dp[j] > result) result = dp[j];
            }
        }
        return result;
    }
};
```

- 时间复杂度：$O(n × m)$，n 为A长度，m为B长度
- 空间复杂度：$O(m)$

拓展

前面讲了 dp数组为什么定义：以下标i - 1为结尾的A，和以下标j - 1为结尾的B，最长重复子数组长度为dp[i][j]。

我就定义dp[i][j]为 以下标i为结尾的A，和以下标j 为结尾的B，最长重复子数组长度。不行么？

当然可以，就是实现起来麻烦一些。

如果定义 dp[i][j]为 以下标i为结尾的A，和以下标j 为结尾的B，那么 第一行和第一列毕竟要进行初始化，如果 nums1[i] 与 nums2[0] 相同的话，对应的 dp[i][0]就要初始为1， 因为此时最长重复子数组为1。 nums2[j] 与 nums1[0]相同的话，同理。

所以代码如下：

```
// 版本三
class Solution {
public:
    int findLength(vector<int>& nums1, vector<int>& nums2) {
        vector<vector<int>> dp (nums1.size() + 1, vector<int>(nums2.size() + 1, 0));
        int result = 0;

        // 要对第一行，第一列经行初始化
        for (int i = 0; i < nums1.size(); i++) if (nums1[i] == nums2[0]) dp[i][0] = 1;
        for (int j = 0; j < nums2.size(); j++) if (nums1[0] == nums2[j]) dp[0][j] = 1;

        for (int i = 0; i < nums1.size(); i++) {
            for (int j = 0; j < nums2.size(); j++) {
                if (nums1[i] == nums2[j] && i > 0 && j > 0) { // 防止 i-1 出现负数
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                }
                if (dp[i][j] > result) result = dp[i][j];
            }
        }
        return result;
    }
};
```

大家会发现 这种写法 一定要多写一段初始化的过程。

而且为了让 `if (dp[i][j] > result) result = dp[i][j];` 收集到全部结果，两层for训练一定从0开始遍历，这样需要加上 `&& i > 0 && j > 0` 的判断。

对于基础不牢的小白来说，在推导出转移方程后可能疑惑上述代码为什么要从`i=0,j=0`遍历而不是从`i=1,j=1`开始遍历，原因在于这里如果不是从`i=0,j=0`位置开始遍历，会漏掉如下样例结果：

```
nums1 = [70,39,25,40,7]
nums2 = [52,20,67,5,31]
```

当然，如果你愿意也可以使用如下代码，与上面那个c++是同一思路：

```
class Solution {
    public int findLength(int[] nums1, int[] nums2) {
        int len1 = nums1.length;
        int len2 = nums2.length;
        int[][] result = new int[len1][len2];

        int maxresult = Integer.MIN_VALUE;

        for(int i=0;i<len1;i++){
            if(nums1[i] == nums2[0])
                result[i][0] = 1;
            if(maxresult<result[i][0])
                    maxresult = result[i][0];
        }

        for(int j=0;j<len2;j++){
            if(nums1[0] == nums2[j])
                result[0][j] = 1;
            if(maxresult<result[0][j])
                maxresult = result[0][j];
        }

        for(int i=1;i<len1;i++){
            for(int j=1;j<len2;j++){

                if(nums1[i]==nums2[j])
                    result[i][j] = result[i-1][j-1]+1;

                if(maxresult<result[i][j])
                    maxresult = result[i][j];

            }

        }

        return maxresult;
```

```
    }
}
```

对于小白来说一定要明确dp数组中初始化的数据是什么

整体而言相对于版本一来说还是多写了不少代码。而且逻辑上也复杂了一些。 优势就是dp数组的定义，更直观一点。

44.最长公共子序列

力扣题目链接

给定两个字符串 text1 和 text2，返回这两个字符串的最长公共子序列的长度。

一个字符串的 子序列 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

例如，"ace" 是 "abcde" 的子序列，但 "aec" 不是 "abcde" 的子序列。两个字符串的「公共子序列」是这两个字符串所共同拥有的子序列。

若这两个字符串没有公共子序列，则返回 0。

示例 1:

- 输入：text1 = "abcde", text2 = "ace"
- 输出：3
- 解释：最长公共子序列是 "ace"，它的长度为 3。

示例 2:

- 输入：text1 = "abc", text2 = "abc"
- 输出：3
- 解释：最长公共子序列是 "abc"，它的长度为 3。

示例 3:

- 输入：text1 = "abc", text2 = "def"
- 输出：0
- 解释：两个字符串没有公共子序列，返回 0。

提示:

- 1 <= text1.length <= 1000
- 1 <= text2.length <= 1000
  输入的字符串只含有小写英文字符。

算法公开课

《代码随想录》算法视频公开课：动态规划子序列问题经典题目 | LeetCode：1143.最长公共子序列，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

本题和动态规划：718. 最长重复子数组区别在于这里不要求是连续的了，但要有相对顺序，即："ace" 是 "abcde" 的子序列，但 "aec" 不是 "abcde" 的子序列。

继续动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：长度为[0, i - 1]的字符串text1与长度为[0, j - 1]的字符串text2的最长公共子序列为dp[i][j]

有同学会问：为什么要定义长度为[0, i - 1]的字符串text1，定义为长度为[0, i]的字符串text1不香么？

这样定义是为了后面代码实现方便，如果非要定义为长度为[0, i]的字符串text1也可以，我在 动态规划：718. 最长重复子数组 中的「拓展」里 详细讲解了区别所在，其实就是简化了dp数组第一行和第一列的初始化逻辑。

2. 确定递推公式

主要就是两大情况： text1[i - 1] 与 text2[j - 1]相同，text1[i - 1] 与 text2[j - 1]不相同

如果text1[i - 1] 与 text2[j - 1]相同，那么找到了一个公共元素，所以dp[i][j] = dp[i - 1][j - 1] + 1;

如果text1[i - 1] 与 text2[j - 1]不相同，那就看看text1[0, i - 2]与text2[0, j - 1]的最长公共子序列 和 text1[0, i - 1]与text2[0, j - 2]的最长公共子序列，取最大的。

即：dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

代码如下：

```
if (text1[i - 1] == text2[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1] + 1;
} else {
    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
}
```

3. dp数组如何初始化

先看看dp[i][0]应该是多少呢?

test1[0, i-1]和空串的最长公共子序列自然是0，所以dp[i][0] = 0;

同理dp[0][j]也是0。

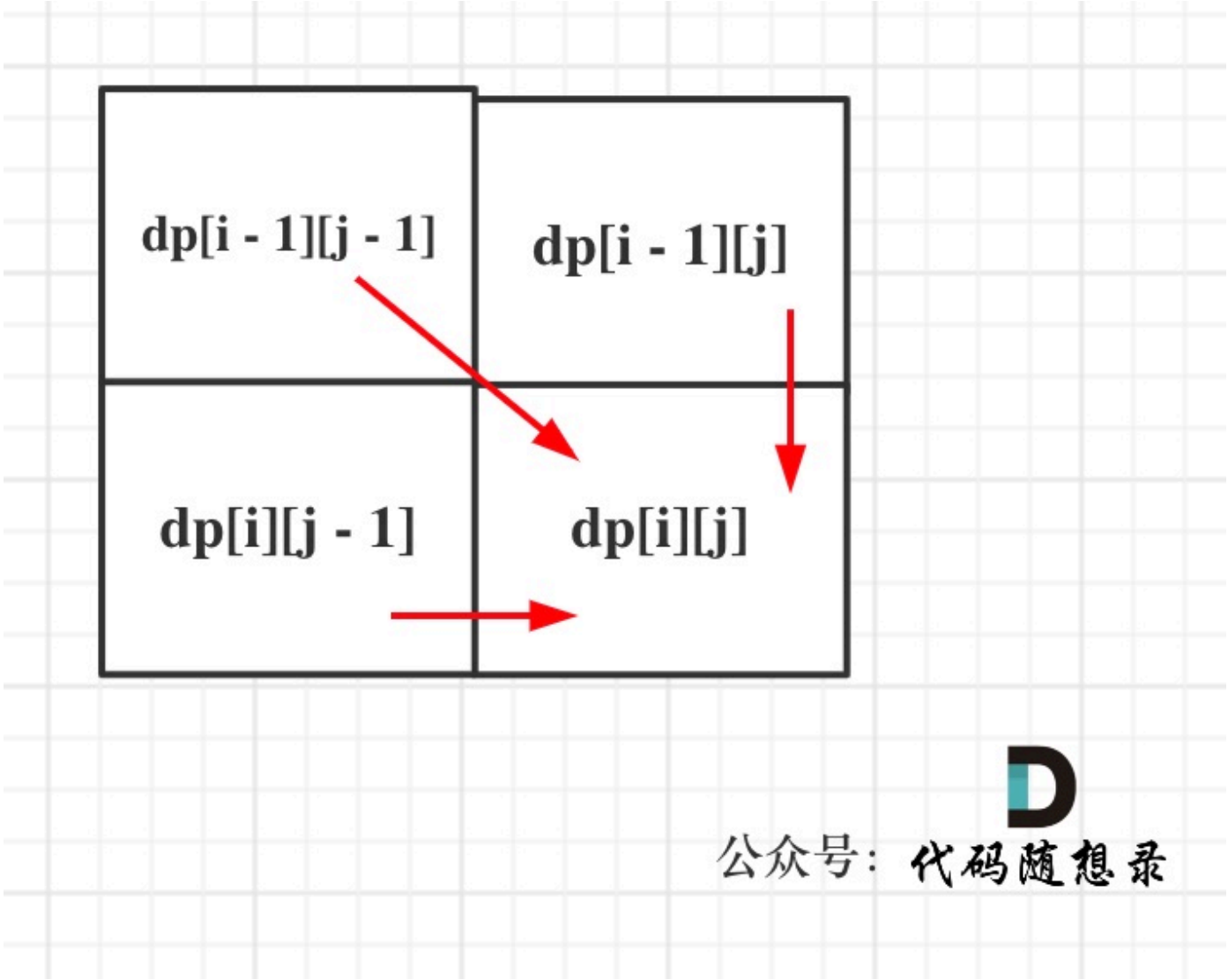
其他下标都是随着递推公式逐步覆盖，初始为多少都可以，那么就统一初始为0。

代码：

```
vector<vector<int>> dp(text1.size() + 1, vector<int>(text2.size() + 1, 0));
```

4. 确定遍历顺序

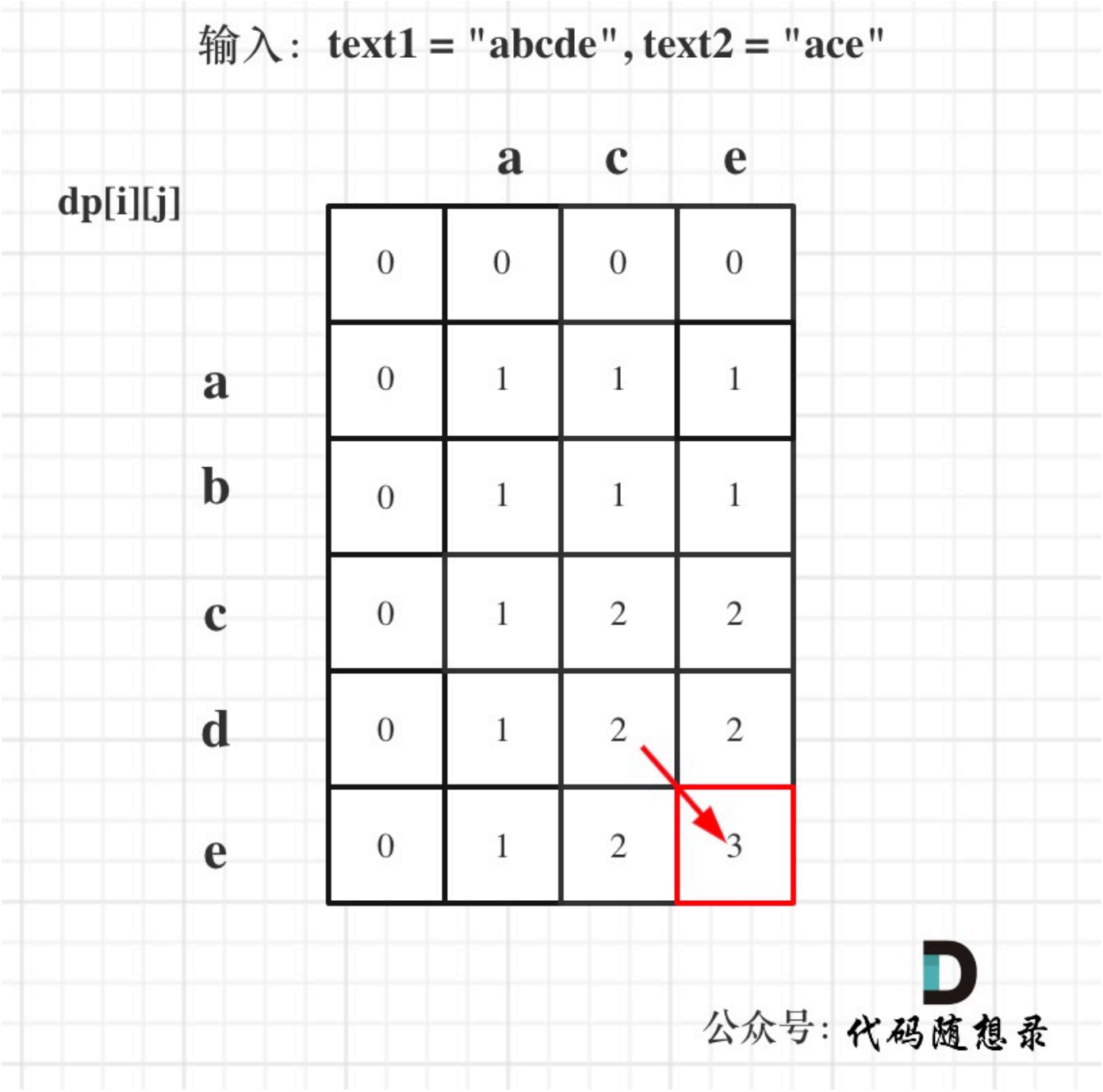
从递推公式，可以看出，有三个方向可以推出dp[i][j]，如图：



那么为了在递推的过程中，这三个方向都是经过计算的数值，所以要从前向后，从上到下来遍历这个矩阵。

5. 举例推导dp数组

以输入：text1 = "abcde", text2 = "ace" 为例，dp状态如图：



最后红框dp[text1.size()][text2.size()]为最终结果

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        vector<vector<int>> dp(text1.size() + 1, vector<int>(text2.size() + 1, 0));
        for (int i = 1; i <= text1.size(); i++) {
            for (int j = 1; j <= text2.size(); j++) {
                if (text1[i - 1] == text2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
```

```
                }
            }
            return dp[text1.size()][text2.size()];
        }
};
```

- 时间复杂度: O(n * m)，其中 n 和 m 分别为 text1 和 text2 的长度
- 空间复杂度: O(n * m)

45.不相交的线

力扣题目链接

我们在两条独立的水平线上按给定的顺序写下 A 和 B 中的整数。

现在，我们可以绘制一些连接两个数字 A[i] 和 B[j] 的直线，只要 A[i] == B[j]，且我们绘制的直线不与任何其他连线（非水平线）相交。

以这种方法绘制线条，并返回我们可以绘制的最大连线数。

示例 1：

```
输入：A = [1,4,2], B = [1,2,4]
输出：2
解释：
我们可以画出两条不交叉的线，如上图所示。
我们无法画出第三条不相交的直线，因为从 A[1]=4 到 B[2]=4 的直线将与从 A[2]=2 到 B[1]=2 的直线相交。
```

示例 2：

```
输入：A = [2,5,1,2,5], B = [10,5,2,1,5,2]
输出：3
```

示例 3：

```
输入：A = [1,3,7,1,7,5], B = [1,9,2,5,1]
输出：2
```

算法公开课

[《代码随想录》算法视频公开课](#)：[动态规划之子序列问题，换汤不换药 | LeetCode：1035.不相交的线](#)，**相信结合视频再看本篇题解，更有助于大家对本题的理解。**

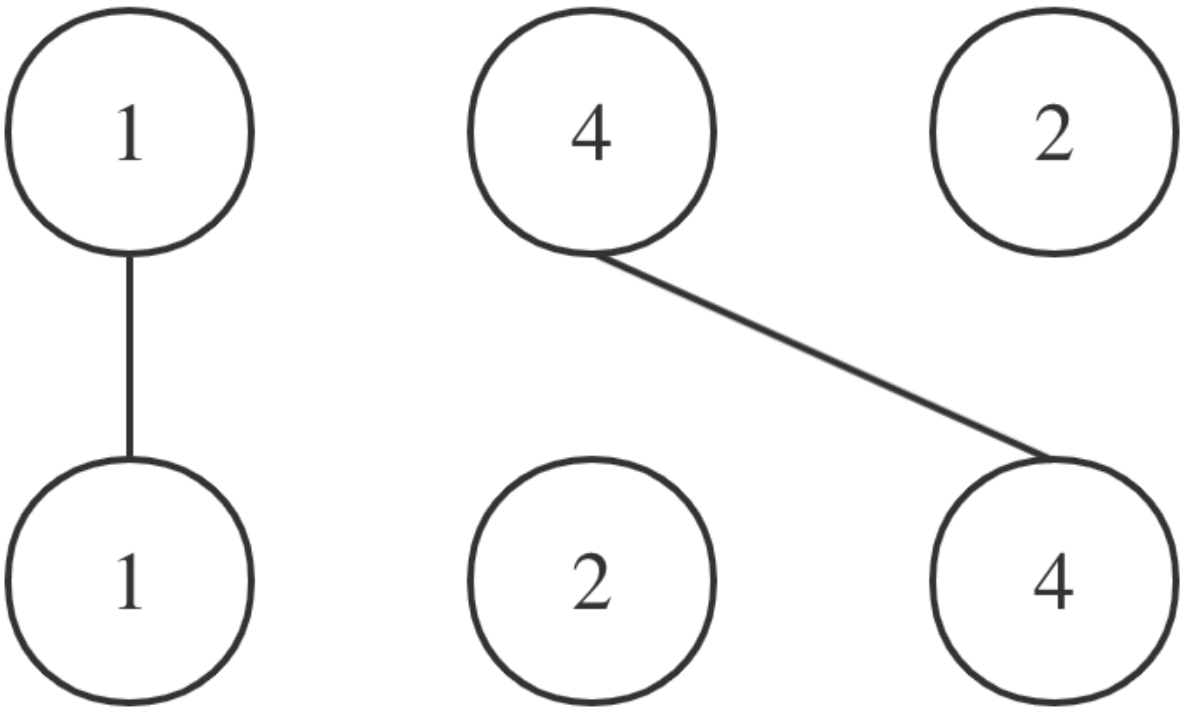
思路

相信不少录友看到这道题目都没啥思路，我们来逐步分析一下。

绘制一些连接两个数字 A[i] 和 B[j] 的直线，只要 A[i] == B[j]，且直线不能相交！

直线不能相交，这就是说明在字符串A中 找到一个与字符串B相同的子序列，且这个子序列不能改变相对顺序，只要相对顺序不改变，链接相同数字的直线就不会相交。

拿示例一A = [1,4,2], B = [1,2,4]为例，相交情况如图：



其实也就是说A和B的最长公共子序列是[1,4]，长度为2。 这个公共子序列指的是相对顺序不变（即数字4在字符串A中数字1的后面，那么数字4也应该在字符串B数字1的后面）

这么分析完之后，大家可以发现：**本题说是求绘制的最大连线数，其实就是求两个字符串的最长公共子序列的长度！**

那么本题就和我们刚刚讲过的这道题目动态规划：1143.最长公共子序列就是一样一样的了。

一样到什么程度呢？ 把字符串名字改一下，其他代码都不用改，直接copy过来就行了。

其实本题就是求最长公共子序列的长度，介于我们刚刚讲过动态规划：1143.最长公共子序列，所以本题我就不再做动规五部曲分析了。

如果大家有点遗忘了最长公共子序列，就再看一下这篇：动态规划：1143.最长公共子序列

本题代码如下：

```
class Solution {
public:
    int maxUncrossedLines(vector<int>& A, vector<int>& B) {
        vector<vector<int>> dp(A.size() + 1, vector<int>(B.size() + 1, 0));
        for (int i = 1; i <= A.size(); i++) {
            for (int j = 1; j <= B.size(); j++) {
                if (A[i - 1] == B[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
```

```
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[A.size()][B.size()];
    }
};
```

- 时间复杂度: O(n * m)
- 空间复杂度: O(n * m)

总结

看到代码大家也可以发现其实就是求两个字符串的最长公共子序列，但如果没有做过1143.最长公共子序列，本题其实还有很有难度的。

这是Carl为什么要先讲1143.最长公共子序列再讲本题，大家会发现一个正确的刷题顺序对算法学习是非常重要的！

这也是Carl做了很多题目（包括ACM和力扣）才总结出来的规律，大家仔细体会一下哈。

46. 最大子序和

力扣题目链接

给定一个整数数组 nums ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

示例:

- 输入: [-2,1,-3,4,-1,2,1,-5,4]
- 输出: 6
- 解释: 连续子数组 [4,-1,2,1] 的和最大，为 6。

算法公开课

《代码随想录》算法视频公开课：看起来复杂，其实是简单动态规划 | LeetCode：53.最大子序和，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

这道题之前我们在讲解贪心专题的时候用贪心算法解决过一次，贪心算法：最大子序和。

这次我们用动态规划的思路再来分析一次。

动规五部曲如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i]：包括下标i（以nums[i]为结尾）的最大连续子序列和为dp[i]。

2. 确定递推公式

dp[i]只有两个方向可以推出来：

- dp[i - 1] + nums[i]，即：nums[i]加入当前连续子序列和
- nums[i]，即：从头开始计算当前连续子序列和

一定是取最大的，所以dp[i] = max(dp[i - 1] + nums[i], nums[i]);

3. dp数组如何初始化

从递推公式可以看出来dp[i]是依赖于dp[i - 1]的状态，dp[0]就是递推公式的基础。

dp[0]应该是多少呢?

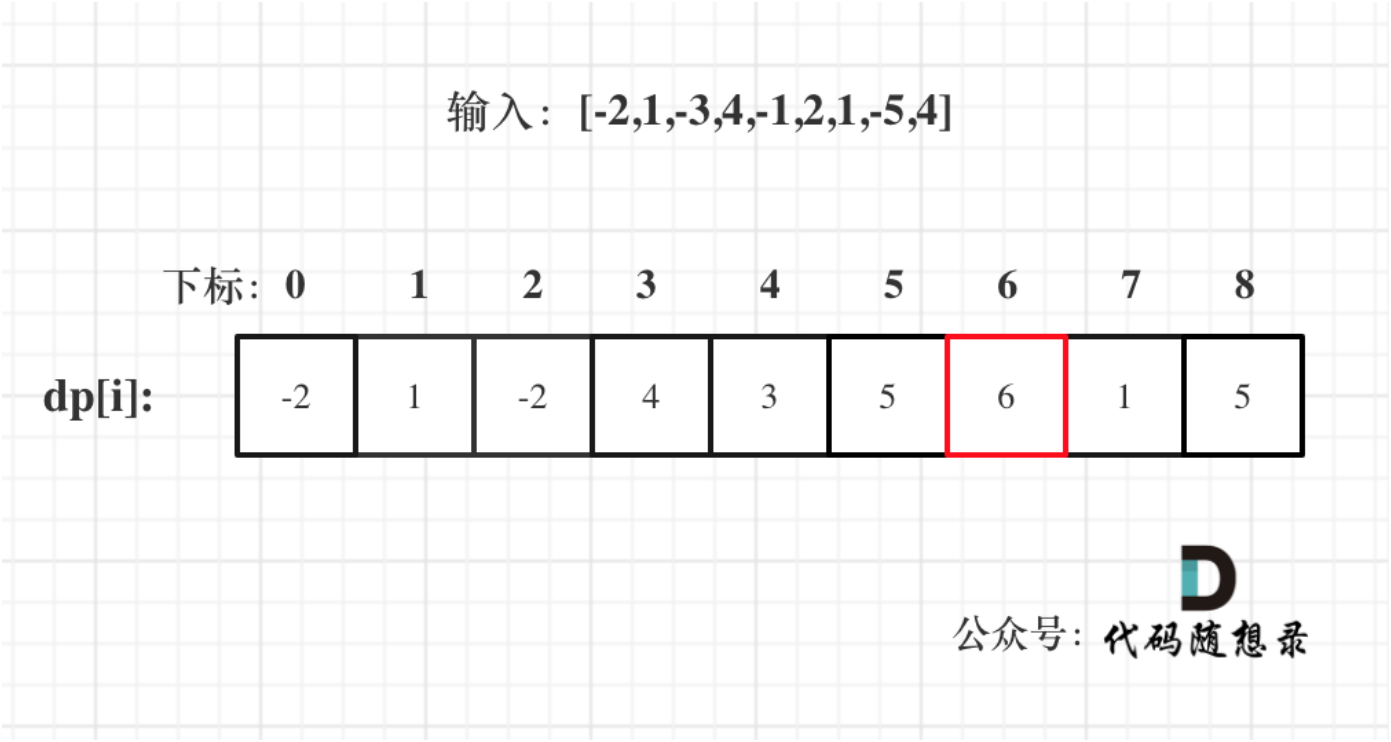
根据dp[i]的定义，很明显dp[0]应为nums[0]即dp[0] = nums[0]。

4. 确定遍历顺序

递推公式中dp[i]依赖于dp[i - 1]的状态，需要从前向后遍历。

5. 举例推导dp数组

以示例一为例，输入：nums = [-2,1,-3,4,-1,2,1,-5,4]，对应的dp状态如下：



注意最后的结果可不是dp[nums.size() - 1]！ ，而是dp[6]。

在回顾一下dp[i]的定义：包括下标i之前的最大连续子序列和为dp[i]。

那么我们要找最大的连续子序列，就应该找每一个i为终点的连续最大子序列。

所以在递推公式的时候，可以直接选出最大的dp[i]。

以上动规五部曲分析完毕，完整代码如下：

```
class Solution {
public:
```

```
    int maxSubArray(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        vector<int> dp(nums.size());
        dp[0] = nums[0];
        int result = dp[0];
        for (int i = 1; i < nums.size(); i++) {
            dp[i] = max(dp[i - 1] + nums[i], nums[i]); // 状态转移公式
            if (dp[i] > result) result = dp[i]; // result 保存dp[i]的最大值
        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

总结

这道题目用贪心也很巧妙，但有一点绕，需要仔细想一想，如果想回顾一下贪心就看这里吧：贪心算法：最大子序和

动规的解法还是很直接的。

47.判断子序列

力扣题目链接

给定字符串 s 和 t ，判断 s 是否为 t 的子序列。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

示例 1：

- 输入：s = "abc", t = "ahbgdc"
- 输出：true

示例 2：

- 输入：s = "axc", t = "ahbgdc"
- 输出：false

提示：

- 0 <= s.length <= 100
- 0 <= t.length <= 10^4

两个字符串都只由小写字符组成。

算法公开课

《代码随想录》算法视频公开课：动态规划，用相似思路解决复杂问题 | LeetCode：392.判断子序列，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

（这道题也可以用双指针的思路来实现，时间复杂度也是O(n)）

这道题应该算是编辑距离的入门题目，因为从题意中我们也可以发现，只需要计算删除的情况，不用考虑增加和替换的情况。

所以掌握本题的动态规划解法是对后面要讲解的编辑距离的题目打下基础。

动态规划五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j] 表示以下标i-1为结尾的字符串s，和以下标j-1为结尾的字符串t，相同子序列的长度为dp[i][j]。

注意这里是判断s是否为t的子序列。即t的长度是大于等于s的。

有同学问了，为啥要表示下标i-1为结尾的字符串呢，为啥不表示下标i为结尾的字符串呢?

为什么这么定义我在 718. 最长重复子数组 中做了详细的讲解。

其实用i来表示也可以！

但我统一以下标i-1为结尾的字符串来计算，这样在下面的递归公式中会容易理解一些，如果还有疑惑，可以继续往下看。

2. 确定递推公式

在确定递推公式的时候，首先要考虑如下两种操作，整理如下：

- if (s[i - 1] == t[j - 1])
  - t中找到了一个字符在s中也出现了
- if (s[i - 1] != t[j - 1])
  - 相当于t要删除元素，继续匹配

if (s[i - 1] == t[j - 1])，那么dp[i][j] = dp[i - 1][j - 1] + 1;，因为找到了一个相同的字符，相同子序列长度自然要在dp[i-1][j-1]的基础上加1（**如果不理解，在回看一下dp[i][j]的定义**）

if (s[i - 1] != t[j - 1])，此时相当于t要删除元素，t如果把当前元素t[j - 1]删除，那么dp[i][j] 的数值就是 看s[i - 1]与 t[j - 2]的比较结果了，即：dp[i][j] = dp[i][j - 1];

其实这里 大家可以发现和 1143.最长公共子序列 的递推公式基本那就是一样的，区别就是 本题 如果删元素一定是字符串t，而 1143.最长公共子序列 是两个字符串都可以删元素。

3. dp数组如何初始化

从递推公式可以看出dp[i][j]都是依赖于dp[i - 1][j - 1] 和 dp[i][j - 1]，所以dp[0][0]和dp[i][0]是一定要初始化的。

这里大家已经可以发现，在定义dp[i][j]含义的时候为什么要**表示以下标i-1为结尾的字符串s，和以下标j-1为结尾的字符串t，相同子序列的长度为dp[i][j]。**

因为这样的定义在dp二维矩阵中可以留出初始化的区间，如图：

如果要是定义的dp[i][j]是以下标i为结尾的字符串s和以下标j为结尾的字符串t，初始化就比较麻烦了。

dp[i][0] 表示以下标i-1为结尾的字符串，与空字符串的相同子序列长度，所以为0. dp[0][j]同理。

```
vector<vector<int>> dp(s.size() + 1, vector<int>(t.size() + 1, 0));
```

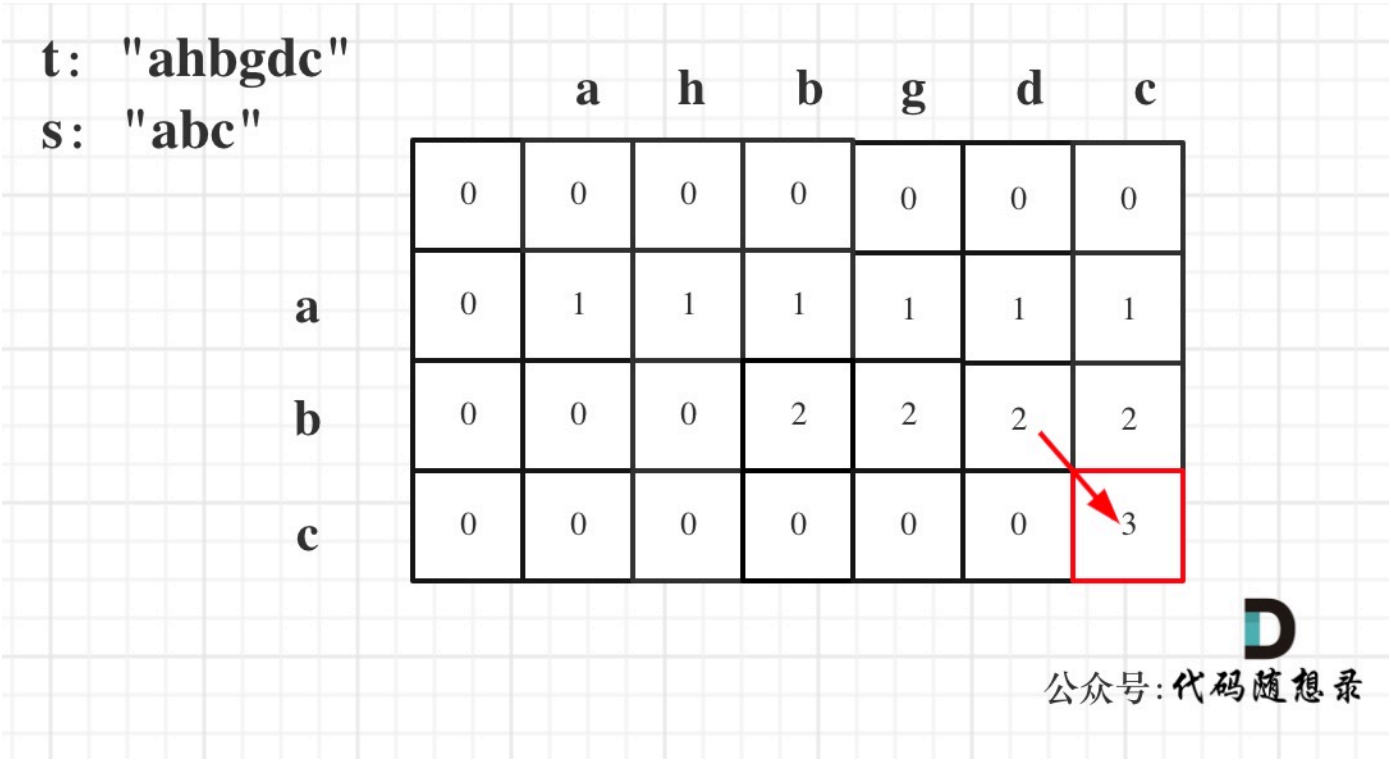
4. 确定遍历顺序

同理从递推公式可以看出dp[i][j]都是依赖于dp[i - 1][j - 1] 和 dp[i][j - 1]，那么遍历顺序也应该是从上到下，从左到右

如图所示：

5. 举例推导dp数组

以示例一为例，输入：s = "abc", t = "ahbgdc"，dp状态转移图如下：



dp[i][j]表示以下标i-1为结尾的字符串s和以下标j-1为结尾的字符串t 相同子序列的长度，所以如果dp[s.size()][t.size()] 与 字符串s的长度相同说明：s与t的最长相同子序列就是s，那么s 就是 t 的子序列。

图中dp[s.size()][t.size()] = 3， 而s.size() 也为3。所以s是t 的子序列，返回true。

动规五部曲分析完毕，C++代码如下：

```
class Solution {
public:
    bool isSubsequence(string s, string t) {
        vector<vector<int>> dp(s.size() + 1, vector<int>(t.size() + 1, 0));
        for (int i = 1; i <= s.size(); i++) {
            for (int j = 1; j <= t.size(); j++) {
                if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
                else dp[i][j] = dp[i][j - 1];
```

```
                }
            }
            if (dp[s.size()][t.size()] == s.size()) return true;
            return false;
        }
};
```

- 时间复杂度：O(n × m)
- 空间复杂度：O(n × m)

总结

这道题目算是编辑距离的入门题目（毕竟这里只是涉及到减法），也是动态规划解决的经典题型。

这一类题都是题目读上去感觉很复杂，模拟一下也发现很复杂，用动规分析完了也感觉很复杂，但是最终代码却很简短。

在之前的题目讲解中，我们讲了 1143.最长公共子序列，大家会发现 本题和 1143.最长公共子序列 的相似之处。

编辑距离的题目最能体现出动规精髓和巧妙之处，大家可以好好体会一下。

48.不同的子序列

力扣题目链接

给定一个字符串 s 和一个字符串 t ，计算在 s 的子序列中 t 出现的个数。

字符串的一个 子序列 是指，通过删除一些（也可以不删除）字符且不干扰剩余字符相对位置所组成的新字符串。（例如，"ACE" 是 "ABCDE" 的一个子序列，而 "AEC" 不是）

题目数据保证答案符合 32 位带符号整数范围。

示例 1：

```
输入：s = "rabbbit", t = "rabbit"
输出：3
解释：
如下图所示，有 3 种可以从 s 中得到 "rabbit" 的方案。
(上箭头符号 ^ 表示选取的字母)
rabbbit
^^^^ ^^
rabbbit
^^ ^^^^
rabbbit
^^^ ^^^
```

示例 2：

```
输入：s = "babgbag", t = "bag"
输出：5
解释：
如下图所示，有 5 种可以从 s 中得到 "bag" 的方案。
(上箭头符号 ^ 表示选取的字母)
babgbag
^^ ^
babgbag
^^    ^
babgbag
^    ^^
babgbag
  ^  ^^
babgbag
    ^^^
```

提示：

- 0 <= s.length, t.length <= 1000
- s 和 t 由英文字母组成

思路

这道题目如果不是子序列，而是要求连续序列的，那就可以考虑用KMP。

这道题目相对于72. 编辑距离，简单了不少，因为本题相当于只有删除操作，不用考虑替换增加之类的。

但相对于刚讲过的动态规划：392.判断子序列就有难度了，这道题目双指针法可就做不了了，来看看动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：以i-1为结尾的s子序列中出现以j-1为结尾的t的个数为dp[i][j]。

为什么i-1，j-1 这么定义我在 718. 最长重复子数组 中做了详细的讲解。

2. 确定递推公式

这一类问题，基本是要分析两种情况

- s[i - 1] 与 t[j - 1]相等
- s[i - 1] 与 t[j - 1] 不相等

当s[i - 1] 与 t[j - 1]相等时，dp[i][j]可以有两部分组成。

一部分是用s[i - 1]来匹配，那么个数为dp[i - 1][j - 1]。即不需要考虑当前s子串和t子串的最后一位字母，所以只需要 dp[i-1][j-1]。

一部分是不用s[i - 1]来匹配，个数为dp[i - 1][j]。

这里可能有录友不明白了，为什么还要考虑 不用s[i - 1]来匹配，都相同了指定要匹配啊。

例如： s：bagg 和 t：bag ，s[3] 和 t[2]是相同的，但是字符串s也可以不用s[3]来匹配，即用s[0]s[1]s[2]组成的bag。

当然也可以用s[3]来匹配，即：s[0]s[1]s[3]组成的bag。

所以当s[i - 1] 与 t[j - 1]相等时，dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];

当s[i - 1] 与 t[j - 1]不相等时，dp[i][j]只有一部分组成，不用s[i - 1]来匹配（就是模拟在s中删除这个元素），即：dp[i - 1][j]

所以递推公式为：dp[i][j] = dp[i - 1][j];

这里可能有录友还疑惑，为什么只考虑 “不用s[i - 1]来匹配” 这种情况， 不考虑 “不用t[j - 1]来匹配” 的情况呢。

这里大家要明确，我们求的是 s 中有多少个 t，而不是 求t中有多少个s，所以只考虑 s中删除元素的情况，即 不用s[i - 1]来匹配 的情况。

3. dp数组如何初始化

从递推公式dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]; 和 dp[i][j] = dp[i - 1][j]; 中可以看出dp[i][j] 是从上方和左上方推导而来，如图：，那么 dp[i][0] 和dp[0][j]是一定要初始化的。

每次当初始化的时候，都要回顾一下dp[i][j]的定义，不要凭感觉初始化。

dp[i][0]表示什么呢?

dp[i][0] 表示：以i-1为结尾的s可以随便删除元素，出现空字符串的个数。

那么dp[i][0]一定都是1，因为也就是把以i-1为结尾的s，删除所有元素，出现空字符串的个数就是1。

再来看dp[0][j]，dp[0][j]：空字符串s可以随便删除元素，出现以j-1为结尾的字符串t的个数。

那么dp[0][j]一定都是0，s如论如何也变成不了t。

最后就要看一个特殊位置了，即：dp[0][0] 应该是多少。

dp[0][0]应该是1，空字符串s，可以删除0个元素，变成空字符串t。

初始化分析完毕，代码如下：

```
vector<vector<long long>> dp(s.size() + 1, vector<long long>(t.size() + 1));
for (int i = 0; i <= s.size(); i++) dp[i][0] = 1;
for (int j = 1; j <= t.size(); j++) dp[0][j] = 0; // 其实这行代码可以和dp数组初始化的时候放在一起，但我为了凸显初始化的逻辑，所以还是加上了。
```

4. 确定遍历顺序

从递推公式dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]; 和 dp[i][j] = dp[i - 1][j]; 中可以看出dp[i][j]都是根据左上方和正上方推出来的。

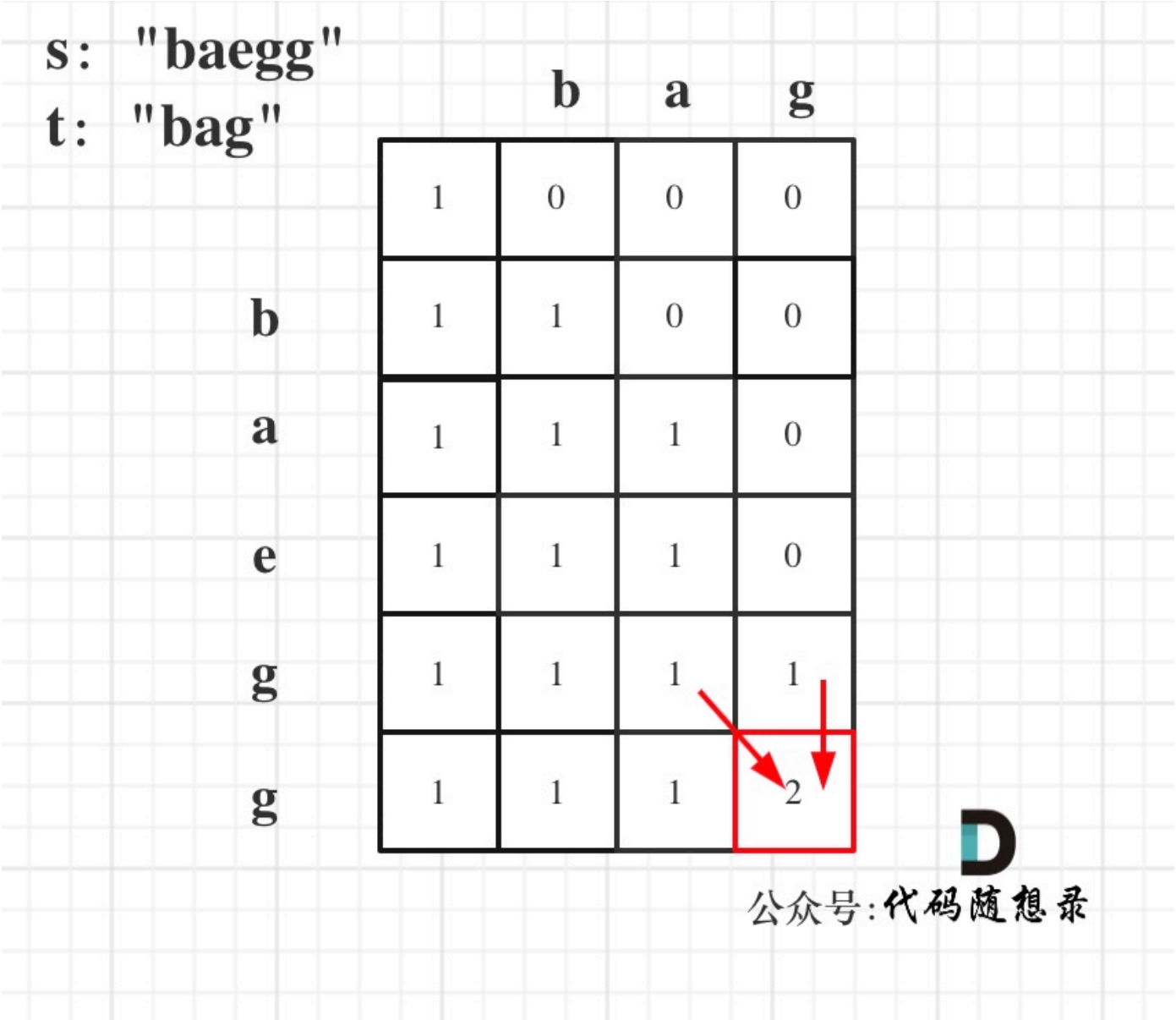
所以遍历的时候一定是从上到下，从左到右，这样保证dp[i][j]可以根据之前计算出来的数值进行计算。

代码如下：

```
for (int i = 1; i <= s.size(); i++) {
    for (int j = 1; j <= t.size(); j++) {
        if (s[i - 1] == t[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
        } else {
            dp[i][j] = dp[i - 1][j];
        }
    }
}
```

5. 举例推导dp数组

以s："baegg"，t："bag"为例，推导dp数组状态如下：



如果写出来的代码怎么改都通过不了，不妨把dp数组打印出来，看一看，是不是这样的。

动规五部曲分析完毕，代码如下：

```
class Solution {
public:
    int numDistinct(string s, string t) {
        vector<vector<uint64_t>> dp(s.size() + 1, vector<uint64_t>(t.size() + 1));
        for (int i = 0; i < s.size(); i++) dp[i][0] = 1;
        for (int j = 1; j < t.size(); j++) dp[0][j] = 0;
        for (int i = 1; i <= s.size(); i++) {
            for (int j = 1; j <= t.size(); j++) {
                if (s[i - 1] == t[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }
```

```
        return dp[s.size()][t.size()];
    }
};
```

- 时间复杂度: O(n * m)
- 空间复杂度: O(n * m)

49. 两个字符串的删除操作

力扣题目链接

给定两个单词 word1 和 word2，找到使得 word1 和 word2 相同所需的最小步数，每步可以删除任意一个字符串中的一个字符。

示例：

- 输入: "sea", "eat"
- 输出: 2
- 解释: 第一步将"sea"变为"ea"，第二步将"eat"变为"ea"

算法公开课

《代码随想录》算法视频公开课：LeetCode：583.两个字符串的删除操，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

动态规划一

本题和动态规划：115.不同的子序列相比，其实就是两个字符串都可以删除了，情况虽说复杂一些，但整体思路是不变的。

这次是两个字符串可以相互删了，这种题目也知道用动态规划的思路来解，动规五部曲，分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：以i-1为结尾的字符串word1，和以j-1位结尾的字符串word2，想要达到相等，所需要删除元素的最少次数。

这里dp数组的定义有点点绕，大家要撸清思路。

2. 确定递推公式

- 当word1[i - 1] 与 word2[j - 1]相同的时候
- 当word1[i - 1] 与 word2[j - 1]不相同的时候

当word1[i - 1] 与 word2[j - 1]相同的时候，dp[i][j] = dp[i - 1][j - 1];

当word1[i - 1] 与 word2[j - 1]不相同的时候，有三种情况：

情况一：删word1[i - 1]，最少操作次数为dp[i - 1][j] + 1

情况二：删word2[j - 1]，最少操作次数为dp[i][j - 1] + 1

情况三：同时删word1[i - 1]和word2[j - 1]，操作的最少次数为dp[i - 1][j - 1] + 2

那最后当然是取最小值，所以当word1[i - 1] 与 word2[j - 1]不相同的时候，递推公式：dp[i][j] = min({dp[i - 1][j - 1] + 2, dp[i - 1][j] + 1, dp[i][j - 1] + 1});

因为 dp[i][j - 1] + 1 = dp[i - 1][j - 1] + 2，所以递推公式可简化为：dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1);

这里可能不少录友有点迷糊，从字面上理解 就是 当 同时删word1[i - 1]和word2[j - 1]，dp[i][j-1] 本来就不考虑word2[j - 1]了，那么我在删 word1[i - 1]，是不是就达到两个元素都删除的效果，即 dp[i][j-1] + 1。

3. dp数组如何初始化

从递推公式中，可以看出来，dp[i][0] 和 dp[0][j]是一定要初始化的。

dp[i][0]：word2为空字符串，以i-1为结尾的字符串word1要删除多少个元素，才能和word2相同呢，很明显dp[i][0] = i。

dp[0][j]的话同理，所以代码如下：

```
vector<vector<int>> dp(word1.size() + 1, vector<int>(word2.size() + 1));
for (int i = 0; i <= word1.size(); i++) dp[i][0] = i;
for (int j = 0; j <= word2.size(); j++) dp[0][j] = j;
```

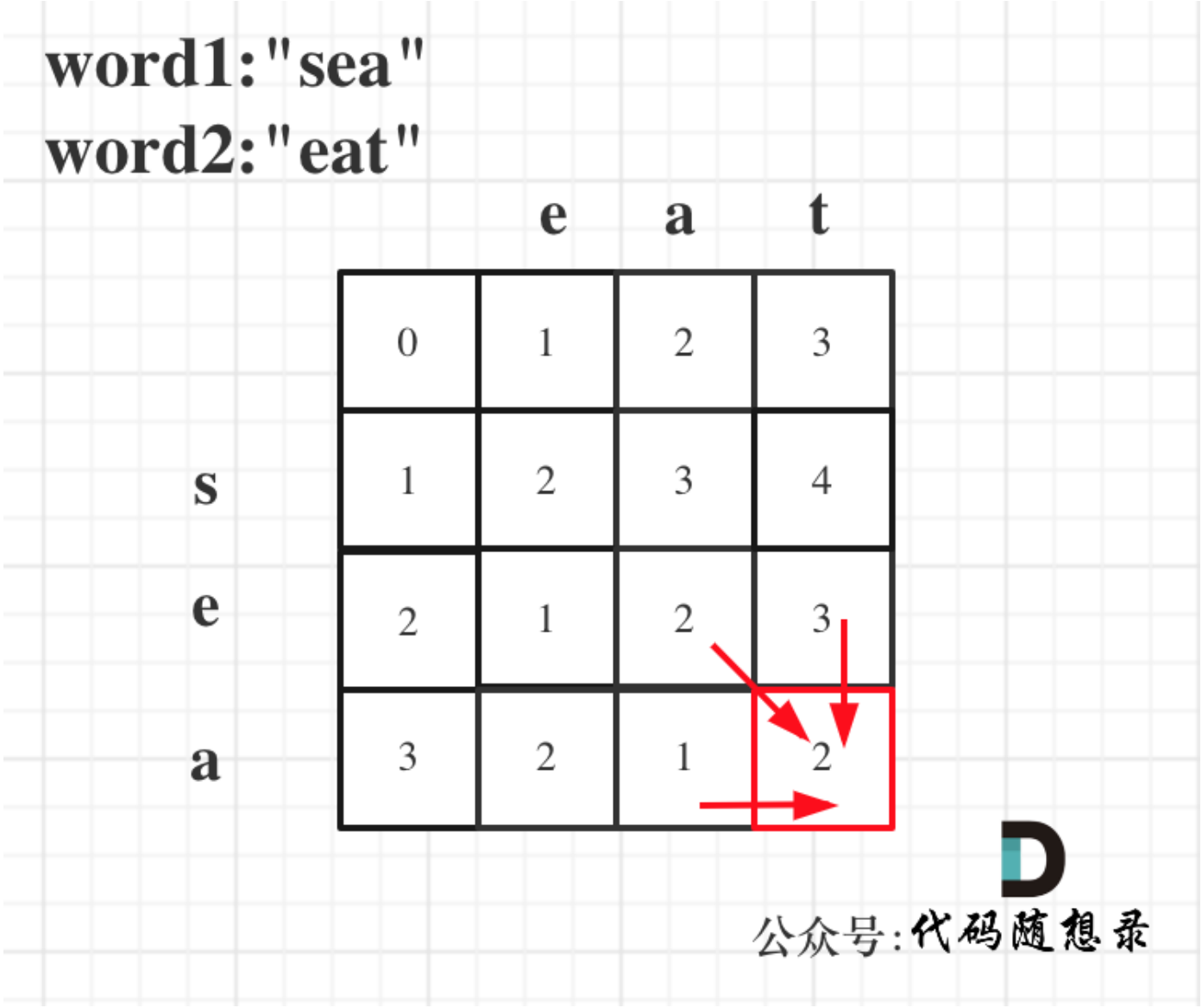
4. 确定遍历顺序

从递推公式 dp[i][j] = min(dp[i - 1][j - 1] + 2, min(dp[i - 1][j], dp[i][j - 1]) + 1); 和dp[i][j] = dp[i - 1][j - 1]可以看出dp[i][j]都是根据左上方、正上方、正左方推出来的。

所以遍历的时候一定是从上到下，从左到右，这样保证dp[i][j]可以根据之前计算出来的数值进行计算。

5. 举例推导dp数组

以word1:"sea"，word2:"eat"为例，推导dp数组状态图如下：



以上分析完毕，代码如下：

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        vector<vector<int>> dp(word1.size() + 1, vector<int>(word2.size() + 1));
        for (int i = 0; i <= word1.size(); i++) dp[i][0] = i;
        for (int j = 0; j <= word2.size(); j++) dp[0][j] = j;
        for (int i = 1; i <= word1.size(); i++) {
            for (int j = 1; j <= word2.size(); j++) {
                if (word1[i - 1] == word2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1);
                }
            }
        }
        return dp[word1.size()][word2.size()];
    }
};
```

- 时间复杂度: O(n * m)
- 空间复杂度: O(n * m)

动态规划二

本题和动态规划：1143.最长公共子序列基本相同，只要求出两个字符串的最长公共子序列长度即可，那么除了最长公共子序列之外的字符都是必须删除的，最后用两个字符串的总长度减去两个最长公共子序列的长度就是删除的最少步数。

代码如下：

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        vector<vector<int>> dp(word1.size()+1, vector<int>(word2.size()+1, 0));
        for (int i=1; i<=word1.size(); i++){
            for (int j=1; j<=word2.size(); j++){
                if (word1[i-1] == word2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
                else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
        return word1.size()+word2.size()-dp[word1.size()][word2.size()]*2;
    }
};
```

- 时间复杂度: O(n * m)
- 空间复杂度: O(n * m)

50. 编辑距离

力扣题目链接

给你两个单词 word1 和 word2，请你计算出将 word1 转换成 word2 所使用的最少操作数 。

你可以对一个单词进行如下三种操作：

- 插入一个字符
- 删除一个字符
- 替换一个字符
- 示例 1：
- 输入：word1 = "horse", word2 = "ros"
- 输出：3

- 解释：
  horse -> rorse (将 'h' 替换为 'r')
  rorse -> rose (删除 'r')
  rose -> ros (删除 'e')
- 示例 2：
- 输入：word1 = "intention", word2 = "execution"
- 输出：5
- 解释：
  intention -> inention (删除 't')
  inention -> enention (将 'i' 替换为 'e')
  enention -> exention (将 'n' 替换为 'x')
  exention -> exection (将 'n' 替换为 'c')
  exection -> execution (插入 'u')

提示：

- 0 <= word1.length, word2.length <= 500
- word1 和 word2 由小写英文字母组成

算法公开课

《代码随想录》算法视频公开课：动态规划终极绝杀！ LeetCode：72.编辑距离，相信结合视频再看本篇题解，更有助于大家对本题的理解。

思路

编辑距离终于来了，这道题目如果大家没有了解动态规划的话，会感觉超级复杂。

编辑距离是用动规来解决的经典题目，这道题目看上去好像很复杂，但用动规可以很巧妙的算出最少编辑距离。

接下来我依然使用动规五部曲，对本题做一个详细的分析：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j] 表示以下标i-1为结尾的字符串word1，和以下标j-1为结尾的字符串word2，最近编辑距离为dp[i][j]。

有同学问了，为啥要表示下标i-1为结尾的字符串呢，为啥不表示下标i为结尾的字符串呢?

为什么这么定义我在 718. 最长重复子数组 中做了详细的讲解。

其实用i来表示也可以！ 用i-1就是为了方便后面dp数组初始化的。

2. 确定递推公式

在确定递推公式的时候，首先要考虑清楚编辑的几种操作，整理如下：

```
if (word1[i - 1] == word2[j - 1])
    不操作
if (word1[i - 1] != word2[j - 1])
    增
    删
    换
```

也就是如上4种情况。

`if (word1[i - 1] == word2[j - 1])` 那么说明不用任何编辑，`dp[i][j]` 就应该是 `dp[i - 1][j - 1]`，即`dp[i][j] = dp[i - 1][j - 1];`

此时可能有同学有点不明白，为啥要即`dp[i][j] = dp[i - 1][j - 1]`呢?

那么就在回顾上面讲过的`dp[i][j]`的定义，`word1[i - 1]` 与 `word2[j - 1]`相等了，那么就不用编辑了，以下标i-2为结尾的字符串word1和以下标j-2为结尾的字符串`word2`的最近编辑距离`dp[i - 1][j - 1]`就是 `dp[i][j]`了。

在下面的讲解中，如果哪里看不懂，就回想一下`dp[i][j]`的定义，就明白了。

在整个动规的过程中，最为关键就是正确理解`dp[i][j]`的定义!

`if (word1[i - 1] != word2[j - 1])`，此时就需要编辑了，如何编辑呢?

- 操作一：word1删除一个元素，那么就是以下标i - 2为结尾的word1 与 j-1为结尾的word2的最近编辑距离 再加上一个操作。

即 `dp[i][j] = dp[i - 1][j] + 1;`

- 操作二：word2删除一个元素，那么就是以下标i - 1为结尾的word1 与 j-2为结尾的word2的最近编辑距离 再加上一个操作。

即 `dp[i][j] = dp[i][j - 1] + 1;`

这里有同学发现了，怎么都是删除元素，添加元素去哪了。

word2添加一个元素，相当于word1删除一个元素，例如 `word1 = "ad" ，word2 = "a"`，`word1`删除元素`'d'` 和 `word2`添加一个元素`'d'`，变成`word1="a", word2="ad"`， 最终的操作数是一样！ dp数组如下图所示意的:

```
           a                         a     d
   +-----+-----+             +-----+-----+-----+
   |  0  |  1  |             |  0  |  1  |  2  |
   +-----+-----+   ===>      +-----+-----+-----+
 a |  1  |  0  |           a |  1  |  0  |  1  |
   +-----+-----+             +-----+-----+-----+
 d |  2  |  1  |
   +-----+-----+
```

操作三：替换元素，`word1`替换`word1[i - 1]`，使其与`word2[j - 1]`相同，此时不用增删加元素。

可以回顾一下，`if (word1[i - 1] == word2[j - 1])`的时候我们的操作 是`dp[i][j] = dp[i - 1][j - 1]` 对吧。

那么只需要一次替换的操作，就可以让 word1[i - 1] 和 word2[j - 1] 相同。

所以`dp[i][j] = dp[i - 1][j - 1] + 1;`

综上，当`if (word1[i - 1] != word2[j - 1])`时取最小的，即：`dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;`

递归公式代码如下：

```
if (word1[i - 1] == word2[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1];
}
else {
    dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;
}
```

---

3. dp数组如何初始化

再回顾一下dp[i][j]的定义：

dp[i][j] 表示以下标i-1为结尾的字符串word1，和以下标j-1为结尾的字符串word2，最近编辑距离为dp[i][j]。

那么dp[i][0] 和 dp[0][j] 表示什么呢?

dp[i][0]：以下标i-1为结尾的字符串word1，和空字符串word2，最近编辑距离为dp[i][0]。

那么dp[i][0]就应该是i，对word1里的元素全部做删除操作，即：dp[i][0] = i;

同理dp[0][j] = j;

所以C++代码如下：

```
for (int i = 0; i <= word1.size(); i++) dp[i][0] = i;
for (int j = 0; j <= word2.size(); j++) dp[0][j] = j;
```

4. 确定遍历顺序

从如下四个递推公式：

- `dp[i][j] = dp[i - 1][j - 1]`
- `dp[i][j] = dp[i - 1][j - 1] + 1`
- `dp[i][j] = dp[i][j - 1] + 1`
- `dp[i][j] = dp[i - 1][j] + 1`

可以看出dp[i][j]是依赖左方，上方和左上方元素的，如图：

所以在dp矩阵中一定是从左到右从上到下去遍历。

代码如下：

```
for (int i = 1; i <= word1.size(); i++) {
    for (int j = 1; j <= word2.size(); j++) {
        if (word1[i - 1] == word2[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        }
        else {
            dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;
        }
    }
}
```

5. 举例推导dp数组

以示例1为例，输入：`word1 = "horse", word2 = "ros"`为例，dp矩阵状态图如下：

以上动规五部分析完毕，C++代码如下：

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        vector<vector<int>> dp(word1.size() + 1, vector<int>(word2.size() + 1, 0));
        for (int i = 0; i <= word1.size(); i++) dp[i][0] = i;
        for (int j = 0; j <= word2.size(); j++) dp[0][j] = j;
        for (int i = 1; i <= word1.size(); i++) {
            for (int j = 1; j <= word2.size(); j++) {
                if (word1[i - 1] == word2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
                else {
                    dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;
                }
            }
        }
        return dp[word1.size()][word2.size()];
    }
};
```

- 时间复杂度: O(n * m)
- 空间复杂度: O(n * m)

51. 动态规划之编辑距离总结篇

本周我们讲了动态规划之终极绝杀：编辑距离，为什么叫做终极绝杀呢?

细心的录友应该知道，我们在前三篇动态规划的文章就一直为 编辑距离 这道题目做铺垫。

判断子序列

动态规划：392.判断子序列 给定字符串 s 和 t ，判断 s 是否为 t 的子序列。

这道题目 其实是可以用双指针或者贪心的的，但是我在开篇的时候就说了这是编辑距离的入门题目，因为从题意中我们也可以发现，只需要计算删除的情况，不用考虑增加和替换的情况。

- if (s[i - 1] == t[j - 1])
  - t中找到了一个字符在s中也出现了
- if (s[i - 1] != t[j - 1])
  - 相当于t要删除元素，继续匹配

状态转移方程：

```
if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
else dp[i][j] = dp[i][j - 1];
```

不同的子序列

动态规划：115.不同的子序列 给定一个字符串 s 和一个字符串 t ，计算在 s 的子序列中 t 出现的个数。

本题虽然也只有删除操作，不用考虑替换增加之类的，但相对于动态规划：392.判断子序列就有难度了，这道题目双指针法可就做不了。

当s[i - 1] 与 t[j - 1]相等时，dp[i][j]可以有两部分组成。

一部分是用s[i - 1]来匹配，那么个数为dp[i - 1][j - 1]。

一部分是不用s[i - 1]来匹配，个数为dp[i - 1][j]。

这里可能有同学不明白了，为什么还要考虑 不用s[i - 1]来匹配，都相同了指定要匹配啊。

例如： s：bagg 和 t：bag ，s[3] 和 t[2]是相同的，但是字符串s也可以不用s[3]来匹配，即用s[0]s[1]s[2]组成的bag。

当然也可以用s[3]来匹配，即：s[0]s[1]s[3]组成的bag。

所以当s[i - 1] 与 t[j - 1]相等时，dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];

当s[i - 1] 与 t[j - 1]不相等时，dp[i][j]只有一部分组成，不用s[i - 1]来匹配，即：dp[i - 1][j]

所以递推公式为：dp[i][j] = dp[i - 1][j];

状态转移方程：

```
if (s[i - 1] == t[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
} else {
    dp[i][j] = dp[i - 1][j];
}
```

两个字符串的删除操作

动态规划：583.两个字符串的删除操作给定两个单词 word1 和 word2，找到使得 word1 和 word2 相同所需的最小步数，每步可以删除任意一个字符串中的一个字符。

本题和动态规划：115.不同的子序列相比，其实就是两个字符串可以都可以删除了，情况虽说复杂一些，但整体思路是不变的。

- 当word1[i - 1] 与 word2[j - 1]相同的时候
- 当word1[i - 1] 与 word2[j - 1]不相同的时候

当word1[i - 1] 与 word2[j - 1]相同的时候，dp[i][j] = dp[i - 1][j - 1];

当word1[i - 1] 与 word2[j - 1]不相同的时候，有三种情况：

情况一：删word1[i - 1]，最少操作次数为dp[i - 1][j] + 1

情况二：删word2[j - 1]，最少操作次数为dp[i][j - 1] + 1

情况三：同时删word1[i - 1]和word2[j - 1]，操作的最少次数为dp[i - 1][j - 1] + 2

那最后当然是取最小值，所以当word1[i - 1] 与 word2[j - 1]不相同的时候，递推公式：dp[i][j] = min({dp[i - 1][j - 1] + 2, dp[i - 1][j] + 1, dp[i][j - 1] + 1});

状态转移方程：

```
if (word1[i - 1] == word2[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1];
} else {
    dp[i][j] = min({dp[i - 1][j - 1] + 2, dp[i - 1][j] + 1, dp[i][j - 1] + 1});
}
```

编辑距离

动态规划：72.编辑距离 给你两个单词 word1 和 word2，请你计算出将 word1 转换成 word2 所使用的最少操作数 。

编辑距离终于来了，**有了前面三道题目的铺垫，应该有思路了**，本题是两个字符串可以增删改，比 动态规划：判断子序列，动态规划：不同的子序列，动态规划：两个字符串的删除操作都要复杂的多。

在确定递推公式的时候，首先要考虑清楚编辑的几种操作，整理如下：

- if (word1[i - 1] == word2[j - 1])
  - 不操作
- if (word1[i - 1] != word2[j - 1])
  - 增
  - 删
  - 换

也就是如上四种情况。

if (word1[i - 1] == word2[j - 1]) 那么说明不用任何编辑，dp[i][j] 就应该是 dp[i - 1][j - 1]，即dp[i][j] = dp[i - 1][j - 1];

此时可能有同学有点不明白，为啥要即dp[i][j] = dp[i - 1][j - 1]呢?

那么就在回顾上面讲过的dp[i][j]的定义，word1[i - 1] 与 word2[j - 1]相等了，那么就不用编辑了，以下标i-2为结尾的字符串word1和以下标j-2为结尾的字符串word2的最近编辑距离dp[i - 1][j - 1] 就是 dp[i][j]了。

在下面的讲解中，如果哪里看不懂，就回想一下dp[i][j]的定义，就明白了。

在整个动规的过程中，最为关键就是正确理解dp[i][j]的定义！

if (word1[i - 1] != word2[j - 1])，此时就需要编辑了，如何编辑呢?

操作一：word1增加一个元素，使其word1[i - 1]与word2[j - 1]相同，那么就是以下标i-2为结尾的word1 与 i-1为结尾的word2的最近编辑距离 加上一个增加元素的操作。

即 dp[i][j] = dp[i - 1][j] + 1;

操作二：word2添加一个元素，使其word1[i - 1]与word2[j - 1]相同，那么就是以下标i-1为结尾的word1 与 j-2为结尾的word2的最近编辑距离 加上一个增加元素的操作。

即 dp[i][j] = dp[i][j - 1] + 1;

这里有同学发现了，怎么都是添加元素，删除元素去哪了。

word2添加一个元素，相当于word1删除一个元素，例如 word1 = "ad" ，word2 = "a"，word2添加一个元素d，也就是相当于word1删除一个元素d，操作数是一样！

操作三：替换元素，word1替换word1[i - 1]，使其与word2[j - 1]相同，此时不用增加元素，那么以下标i-2为结尾的word1 与 j-2为结尾的word2的最近编辑距离 加上一个替换元素的操作。

即 dp[i][j] = dp[i - 1][j - 1] + 1;

综上，当 if (word1[i - 1] != word2[j - 1]) 时取最小的，即：dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;

递归公式代码如下：

```
if (word1[i - 1] == word2[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1];
}
else {
    dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;
}
```

总结

心思的录友应该会发现我用了三道题做铺垫，才最后引出了动态规划：72.编辑距离 ，Carl的良苦用心呀，你们体会到了嘛！

52. 回文子串

力扣题目链接

给定一个字符串，你的任务是计算这个字符串中有多少个回文子串。

具有不同开始位置或结束位置的子串，即使是由相同的字符组成，也会被视作不同的子串。

示例 1：

- 输入："abc"
- 输出：3
- 解释：三个回文子串: "a", "b", "c"

示例 2：

- 输入："aaa"
- 输出：6
- 解释：6个回文子串: "a", "a", "a", "aa", "aa", "aaa"

提示：输入的字符串长度不会超过 1000 。

思路

暴力解法

两层for循环，遍历区间起始位置和终止位置，然后还需要一层遍历判断这个区间是不是回文。所以时间复杂度：O(n^3)

动态规划

动规五部曲：

1. 确定dp数组（dp table）以及下标的含义

如果大家做了很多这种子序列相关的题目，在定义dp数组的时候 很自然就会想题目求什么，我们就如何定义dp数组。

绝大多数题目确实是这样，不过本题如果我们定义，dp[i] 为 下标i结尾的字符串有 dp[i]个回文串的话，我们会发现很难找到递归关系。

dp[i] 和 dp[i-1] ，dp[i + 1] 看上去都没啥关系。

所以我们要看回文串的性质。 如图：

我们在判断字符串S是否是回文，那么如果我们知道 s[1]，s[2]，s[3] 这个子串是回文的，那么只需要比较 s[0]和s[4]这两个元素是否相同，如果相同的话，这个字符串s 就是回文串。

那么此时我们是不是能找到一种递归关系，也就是判断一个子字符串（字符串的下表范围[i,j]）是否回文，依赖于，子字符串（下表范围[i + 1, j - 1]） 是否是回文。

所以为了明确这种递归关系，我们的dp数组是要定义成一位二维dp数组。

布尔类型的dp[i][j]：表示区间范围[i,j] （注意是左闭右闭）的子串是否是回文子串，如果是dp[i][j]为true，否则为false。

2. 确定递推公式

在确定递推公式时，就要分析如下几种情况。

整体上是两种，就是s[i]与s[j]相等，s[i]与s[j]不相等这两种。

当s[i]与s[j]不相等，那没啥好说的了，dp[i][j]一定是false。

当s[i]与s[j]相等时，这就复杂一些了，有如下三种情况

- 情况一：下标i 与 j相同，同一个字符例如a，当然是回文子串
- 情况二：下标i 与 j相差为1，例如aa，也是回文子串
- 情况三：下标：i 与 j相差大于1的时候，例如cabac，此时s[i]与s[j]已经相同了，我们看i到j区间是不是回文子串就看aba是不是回文就可以了，那么aba的区间就是 i+1 与 j-1区间，这个区间是不是回文就看dp[i + 1][j - 1]是否为true。

以上三种情况分析完了，那么递归公式如下：

```
if (s[i] == s[j]) {
    if (j - i <= 1) { // 情况一 和 情况二
        result++;
        dp[i][j] = true;
    } else if (dp[i + 1][j - 1]) { // 情况三
        result++;
        dp[i][j] = true;
    }
}
```

result就是统计回文子串的数量。

注意这里我没有列出当s[i]与s[j]不相等的时候，因为在下面dp[i][j]初始化的时候，就初始为false。

3. dp数组如何初始化

dp[i][j]可以初始化为true么？ 当然不行，怎能刚开始就全都匹配上了。

所以dp[i][j]初始化为false。

4. 确定遍历顺序

遍历顺序可有有点讲究了。

首先从递推公式中可以看出，情况三是根据dp[i + 1][j - 1]是否为true，在对dp[i][j]进行赋值true的。

dp[i + 1][j - 1] 在 dp[i][j]的左下角，如图：

如果这矩阵是从上到下，从左到右遍历，那么会用到没有计算过的dp[i + 1][j - 1]，也就是根据不确定是不是回文的区间[i+1,j-1]，来判断了[i,j]是不是回文，那结果一定是不对的。

所以一定要从下到上，从左到右遍历，这样保证dp[i + 1][j - 1]都是经过计算的。

有的代码实现是优先遍历列，然后遍历行，其实也是一个道理，都是为了保证dp[i + 1][j - 1]都是经过计算的。

代码如下：

```
for (int i = s.size() - 1; i >= 0; i--) {  // 注意遍历顺序
    for (int j = i; j < s.size(); j++) {
        if (s[i] == s[j]) {
            if (j - i <= 1) { // 情况一 和 情况二
                result++;
                dp[i][j] = true;
            } else if (dp[i + 1][j - 1]) { // 情况三
                result++;
                dp[i][j] = true;
            }
        }
    }
}
```

5. 举例推导dp数组

举例，输入："aaa"，dp[i][j]状态如下：

图中有6个true，所以就是有6个回文子串。

注意因为dp[i][j]的定义，所以j一定是大于等于i的，那么在填充dp[i][j]的时候一定是只填充右上半部分。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int countSubstrings(string s) {
        vector<vector<bool>> dp(s.size(), vector<bool>(s.size(), false));
        int result = 0;
        for (int i = s.size() - 1; i >= 0; i--) {  // 注意遍历顺序
            for (int j = i; j < s.size(); j++) {
                if (s[i] == s[j]) {
                    if (j - i <= 1) { // 情况一 和 情况二
                        result++;
                        dp[i][j] = true;
                    } else if (dp[i + 1][j - 1]) { // 情况三
                        result++;
                        dp[i][j] = true;
                    }
                }
            }
        }
        return result;
    }
};
```

以上代码是为了凸显情况一二三，当然是可以简洁一下的，如下：

```
class Solution {
```

```
public:
    int countSubstrings(string s) {
        vector<vector<bool>> dp(s.size(), vector<bool>(s.size(), false));
        int result = 0;
        for (int i = s.size() - 1; i >= 0; i--) {
            for (int j = i; j < s.size(); j++) {
                if (s[i] == s[j] && (j - i <= 1 || dp[i + 1][j - 1])) {
                    result++;
                    dp[i][j] = true;
                }
            }
        }
        return result;
    }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(n^2)

双指针法

动态规划的空间复杂度是偏高的，我们再看一下双指针法。

首先确定回文串，就是找中心然后向两边扩散看是不是对称的就可以了。

在遍历中心点的时候，要注意中心点有两种情况。

一个元素可以作为中心点，两个元素也可以作为中心点。

那么有人同学问了，三个元素还可以做中心点呢。其实三个元素就可以由一个元素左右添加元素得到，四个元素则可以由两个元素左右添加元素得到。

所以我们在计算的时候，要注意一个元素为中心点和两个元素为中心点的情况。

这两种情况可以放在一起计算，但分别计算思路更清晰，我倾向于分别计算，代码如下：

```
class Solution {
public:
    int countSubstrings(string s) {
        int result = 0;
        for (int i = 0; i < s.size(); i++) {
            result += extend(s, i, i, s.size()); // 以i为中心
            result += extend(s, i, i + 1, s.size()); // 以i和i+1为中心
        }
        return result;
    }
    int extend(const string& s, int i, int j, int n) {
        int res = 0;
        while (i >= 0 && j < n && s[i] == s[j]) {
            i--;
            j++;
```

```
                res++;
            }
            return res;
        }
};
```

- 时间复杂度：O(n^2)
- 空间复杂度：O(1)

53.最长回文子序列

力扣题目链接

给定一个字符串 s ，找到其中最长的回文子序列，并返回该序列的长度。可以假设 s 的最大长度为 1000 。

示例 1:
输入: "bbbab"
输出: 4
一个可能的最长回文子序列为 "bbbb"。

示例 2:
输入:"cbbd"
输出: 2
一个可能的最长回文子序列为 "bb"。

提示：

- 1 <= s.length <= 1000
- s 只包含小写英文字母

思路

我们刚刚做过了 动态规划：回文子串，求的是回文子串，而本题要求的是回文子序列， 要搞清楚这两者之间的区别。

回文子串是要连续的，回文子序列可不是连续的！ 回文子串，回文子序列都是动态规划经典题目。

回文子串，可以做这两题：

- 647.回文子串
- 5.最长回文子串

思路其实是差不多的，但本题要比求回文子串简单一点，因为情况少了一点。

动规五部曲分析如下：

1. 确定dp数组（dp table）以及下标的含义

dp[i][j]：字符串s在[i, j]范围内最长的回文子序列的长度为dp[i][j]。

2. 确定递推公式

在判断回文子串的题目中，关键逻辑就是看s[i]与s[j]是否相同。

如果s[i]与s[j]相同，那么dp[i][j] = dp[i + 1][j - 1] + 2;

如图：

（如果这里看不懂，回忆一下dp[i][j]的定义）

如果s[i]与s[j]不相同，说明s[i]和s[j]的同时加入 并不能增加[i,j]区间回文子序列的长度，那么分别加入s[i]、s[j]看看哪一个可以组成最长的回文子序列。

加入s[j]的回文子序列长度为dp[i + 1][j]。

加入s[i]的回文子序列长度为dp[i][j - 1]。

那么dp[i][j]一定是取最大的，即：dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);

代码如下：

```
if (s[i] == s[j]) {
    dp[i][j] = dp[i + 1][j - 1] + 2;
} else {
    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
}
```

3. dp数组如何初始化

首先要考虑当i 和j 相同的情况，从递推公式：dp[i][j] = dp[i + 1][j - 1] + 2; 可以看出 递推公式是计算不到 i 和j相同时候的情况。

所以需要手动初始化一下，当i与j相同，那么dp[i][j]一定是等于1的，即：一个字符的回文子序列长度就是1。

其他情况dp[i][j]初始为0就行，这样递推公式：dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]); 中dp[i][j]才不会被初始值覆盖。

```
vector<vector<int>> dp(s.size(), vector<int>(s.size(), 0));
for (int i = 0; i < s.size(); i++) dp[i][i] = 1;
```

4. 确定遍历顺序

从递归公式中，可以看出，dp[i][j] 依赖于 dp[i + 1][j - 1] ，dp[i + 1][j] 和 dp[i][j - 1]，如图：

所以遍历i的时候一定要从下到上遍历，这样才能保证下一行的数据是经过计算的。

j的话，可以正常从左向右遍历。

代码如下：

```
for (int i = s.size() - 1; i >= 0; i--) {
    for (int j = i + 1; j < s.size(); j++) {
        if (s[i] == s[j]) {
            dp[i][j] = dp[i + 1][j - 1] + 2;
        } else {
            dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
}
```

5. 举例推导dp数组

输入s:"cbbd" 为例，dp数组状态如图：

红色框即：dp[0][s.size() - 1]; 为最终结果。

以上分析完毕，C++代码如下：

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        vector<vector<int>> dp(s.size(), vector<int>(s.size(), 0));
        for (int i = 0; i < s.size(); i++) dp[i][i] = 1;
        for (int i = s.size() - 1; i >= 0; i--) {
            for (int j = i + 1; j < s.size(); j++) {
                if (s[i] == s[j]) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[0][s.size() - 1];
    }
};
```

- 时间复杂度: O(n^2)
- 空间复杂度: O(n^2)

54. 规划最强总结篇！

如今动态规划已经讲解了42道经典题目，共50篇文章，是时候做一篇总结了。

关于动态规划，在专题第一篇关于动态规划，你该了解这些！就说了动规五部曲，**而且强调了五部对解动规题目至关重要！**

这是Carl做过一百多道动规题目总结出来的经验结晶啊，如果大家跟着「代码随想哦」刷过动规专题，一定会对这动规五部曲的作用感受极其深刻。

动规五部曲分别为：

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

动规专题刚开始的时候，讲的题目比较简单，不少录友和我反应：这么简单的题目 讲的复杂了，不用那么多步骤分析，想出递推公式直接就AC这道题目了。

Carl的观点一直都是 简单题是用来 巩固方法论的。 简单题目是可以靠感觉，但后面稍稍难一点的题目，估计感觉就不好使了。

在动规专题讲解中，也充分体现出，这动规五部曲的重要性。

还有不少录友对动规的理解是：递推公式是才是最难最重要的，只要想出递归公式，其他都好办。

其实这么想的同学基本对动规理解的不到位的。

动规五部曲里，哪一部没想清楚，这道题目基本就做不出来，即使做出来了也没有想清楚，而是朦朦胧胧的就把题目过了。

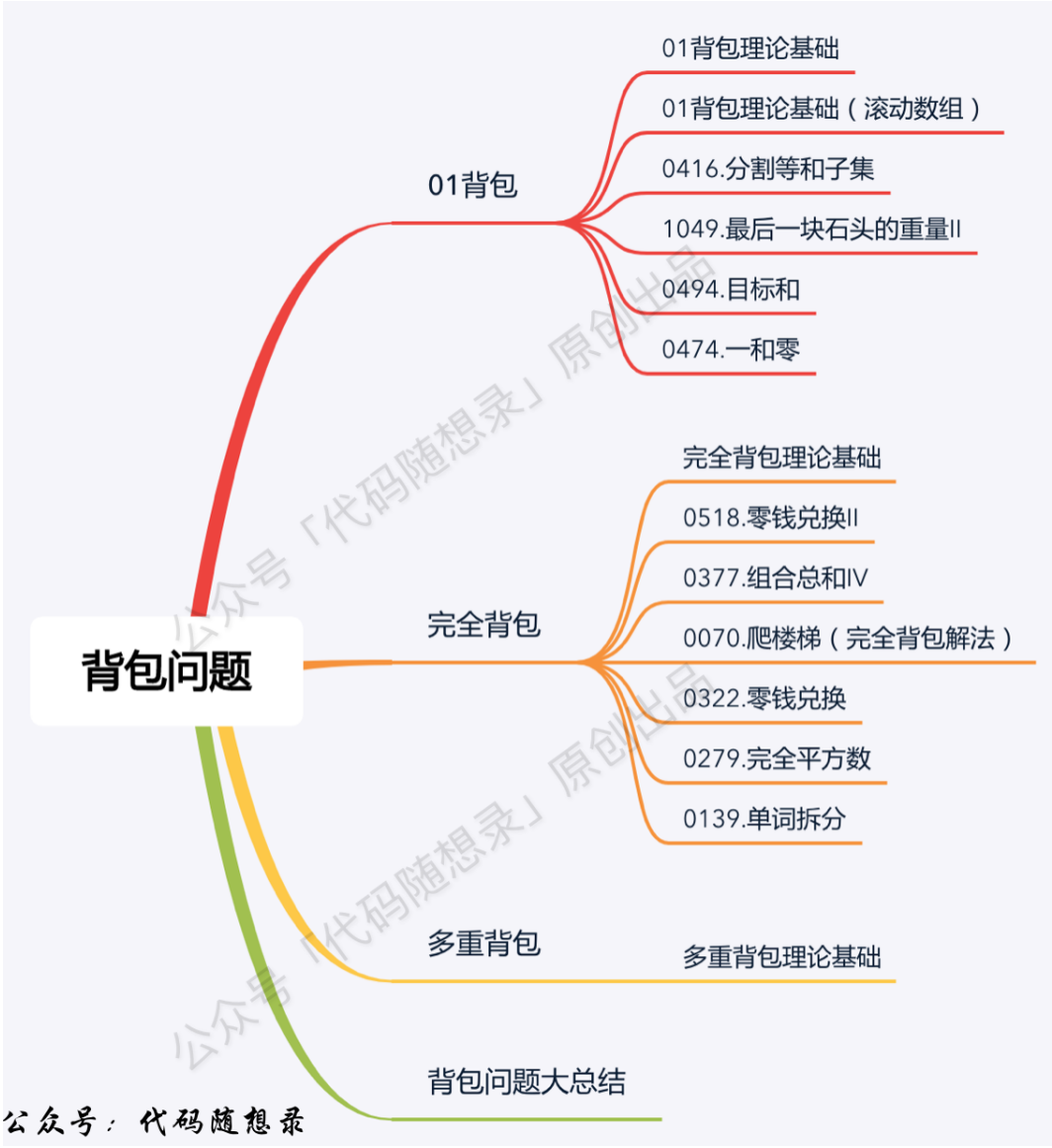
- 如果想不清楚dp数组的具体含义，递归公式从何谈起，甚至初始化的时候就写错了。
- 例如动态规划：不同路径还不够，要有障碍！ 在这道题目中，初始化才是重头戏
- 如果看过背包系列，特别是完全背包，那么两层for循环先后顺序绝对可以搞懵很多人，反而递归公式是简单的。
- 至于推导dp数组的重要性，动规专题里几乎每篇Carl都反复强调，当程序结果不对的时候，一定要自己推导公式，看看和程序打印的日志是否一样。

好啦，我们再一起回顾一下，动态规划专题中我们都讲了哪些内容。

动态规划基础

- 关于动态规划，你该了解这些！
- 动态规划：斐波那契数
- 动态规划：爬楼梯
- 动态规划：使用最小花费爬楼梯
- 动态规划：不同路径
- 动态规划：不同路径还不够，要有障碍！
- 动态规划：整数拆分，你要怎么拆?
- 动态规划：不同的二叉搜索树

背包问题系列



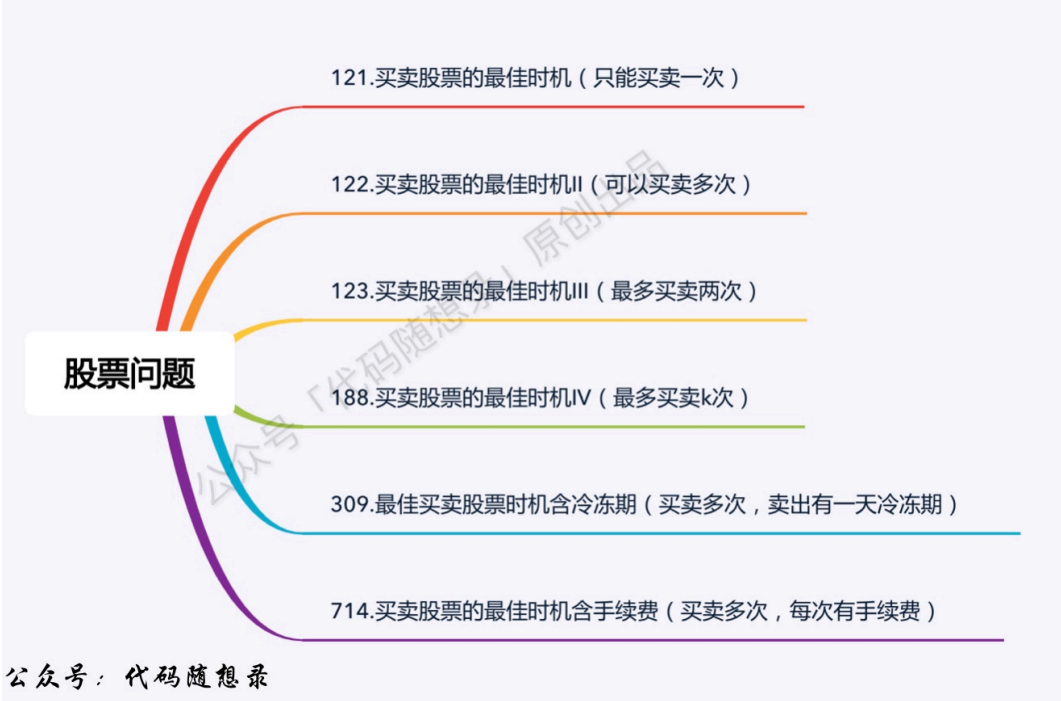
- 动态规划：关于01背包问题，你该了解这些！
- 动态规划：关于01背包问题，你该了解这些！（滚动数组）
- 动态规划：分割等和子集可以用01背包！
- 动态规划：最后一块石头的重量 II
- 动态规划：目标和！
- 动态规划：一和零！
- 动态规划：关于完全背包，你该了解这些！
- 动态规划：给你一些零钱，你要怎么凑？
- 动态规划：Carl称它为排列总和！
- 动态规划：以前我没得选，现在我选择再爬一次！
- 动态规划： 给我个机会，我再兑换一次零钱
- 动态规划：一样的套路，再求一次完全平方数
- 动态规划：单词拆分
- 动态规划：关于多重背包，你该了解这些！

- 听说背包问题很难？ 这篇总结篇来拯救你了

打家劫舍系列

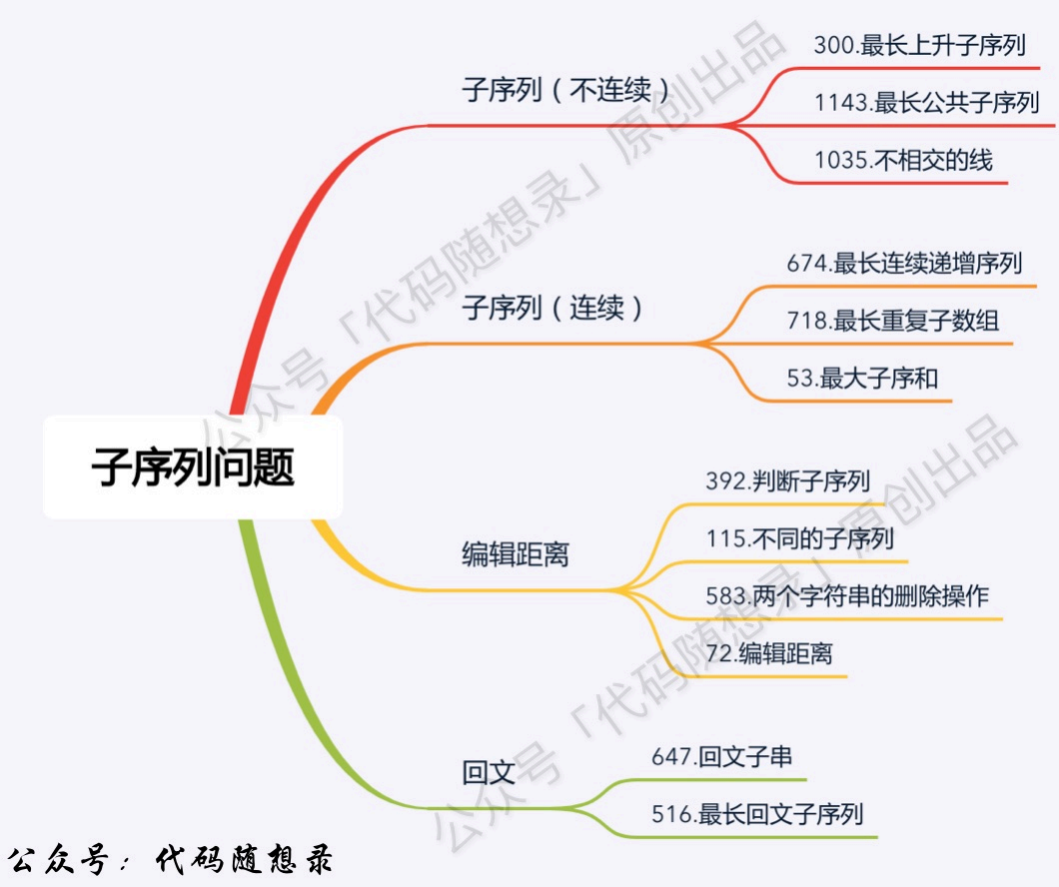
- 动态规划：开始打家劫舍！
- 动态规划：继续打家劫舍！
- 动态规划：还要打家劫舍！

股票系列



- 动态规划：买卖股票的最佳时机
- 动态规划：本周我们都讲了这些（系列六）
- 动态规划：买卖股票的最佳时机II
- 动态规划：买卖股票的最佳时机III
- 动态规划：买卖股票的最佳时机IV
- 动态规划：最佳买卖股票时机含冷冻期
- 动态规划：本周我们都讲了这些（系列七）
- 动态规划：买卖股票的最佳时机含手续费
- 动态规划：股票系列总结篇

子序列系列



- 动态规划：最长递增子序列
- 动态规划：最长连续递增序列
- 动态规划：最长重复子数组
- 动态规划：最长公共子序列
- 动态规划：不相交的线
- 动态规划：最大子序和
- 动态规划：判断子序列
- 动态规划：不同的子序列
- 动态规划：两个字符串的删除操作
- 动态规划：编辑距离
- 为了绝杀编辑距离，我做了三步铺垫，你都知道么？
- 动态规划：回文子串
- 动态规划：最长回文子序列

动规结束语

关于动规，还有 树形DP（打家劫舍系列里有一道），数位DP，区间DP，概率型DP，博弈型DP，状态压缩dp等等等，这些我就不去做讲解了，面试中出现的概率非常低。

能把本篇中列举的题目都研究通透的话，你的动规水平就已经非常高了。 对付面试已经足够！

这个图是 代码随想录知识星球 成员：青，所画，总结的非常好，分享给大家。

这应该是全网对动规最深刻的讲解系列了。

其实大家去网上搜一搜也可以发现，能把动态规划讲清楚的资料挺少的，因为动规确实很难！要给别人讲清楚更难！

《剑指offer》上 动规的题目很少，经典的算法书籍《算法4》 没有讲 动规，而《算法导论》讲的动规基本属于劝退级别的。

讲清楚一道题容易，讲清楚两道题也容易，但把整个动态规划的各个分支讲清楚，每道题目讲通透，并用一套方法论把整个动规贯彻始终就非常难了。

所以Carl花费的这么大精力，把自己对动规算法理解 一五一十的全部分享给了录友们，帮助大家少走弯路，加油！

《代码随想录》作者：程序员Carl

- 代码随想录官网（网站持续更新优化内容，建议直接看网站）：www.programmercarl.com
- 代码随想录Github开源地址
- 代码随想录算法公开课，代码随想录的全部内容将由我（程序员Carl）视频讲解并开免费开放给大家。
- 《代码随想录》已经出版。
- 代码随想录知识星球 上万录友在这里学习
- 代码随想录算法训练营 帮助录友高效刷完代码随想录。
- 微信公众号：代码随想录
- 组队刷题，可以添加代码随想录官方微信
- ACM模式练习，推荐：卡码网

特别提示：PDF仅提供C++语言版本同时PDF中很多动图无法加载，其他编程语言版本和查看动图可以移步至代码随想录官方网站查看。

1. 每日温度

力扣题目链接

请根据每日 气温 列表，重新生成一个列表。对应位置的输出为：要想观测到更高的气温，至少需要等待的天数。如果气温在这之后都不会升高，请在该位置用 0 来代替。

例如，给定一个列表 temperatures = [73, 74, 75, 71, 69, 72, 76, 73]，你的输出应该是 [1, 1, 4, 2, 1, 1, 0, 0]。

提示：气温 列表长度的范围是 [1, 30000]。每个气温的值的均为华氏度，都是在 [30, 100] 范围内的整数。

思路

首先想到的当然是暴力解法，两层for循环，把至少需要等待的天数就搜出来了。时间复杂度是O(n^2)

那么接下来在来看看使用单调栈的解法。

那有同学就问了，我怎么能想到用单调栈呢？ 什么时候用单调栈呢？

通常是一维数组，要寻找任一个元素的右边或者左边第一个比自己大或者小的元素的位置，此时我们就要想到可以用单调栈了。时间复杂度为O(n)。

例如本题其实就是找找到一个元素右边第一个比自己大的元素，此时就应该想到用单调栈了。

那么单调栈的原理是什么呢？为什么时间复杂度是O(n)就可以找到每一个元素的右边第一个比它大的元素位置呢?

单调栈的本质是空间换时间，因为在遍历的过程中需要用一个栈来记录右边第一个比当前元素高的元素，优点是整个数组只需要遍历一次。

更直白来说，就是用一个栈来记录我们遍历过的元素，因为我们遍历数组的时候，我们不知道之前都遍历了哪些元素，以至于遍历一个元素找不到是不是之前遍历过一个更小的，所以我们需要用一个容器（这里用单调栈）来记录我们遍历过的元素。

在使用单调栈的时候首先要明确如下几点：

1. 单调栈里存放的元素是什么?

单调栈里只需要存放元素的下标i就可以了，如果需要使用对应的元素，直接T[i]就可以获取。

2. 单调栈里元素是递增呢？ 还是递减呢?

注意以下讲解中，顺序的描述为 从栈头到栈底的顺序，因为单纯的说从左到右或者从前到后，不说栈头朝哪个方向的话，大家一定比较懵。

这里我们要使用递增循序（再强调一下是指从栈头到栈底的顺序），因为只有递增的时候，栈里要加入一个元素i的时候，才知道栈顶元素在数组中右面第一个比栈顶元素大的元素是i。

即：如果求一个元素右边第一个更大元素，单调栈就是递增的，如果求一个元素右边第一个更小元素，单调栈就是递减的。

文字描述理解起来有点费劲，接下来我画了一系列的图，来讲解单调栈的工作过程，大家再去思考，本题为什么是递增栈。

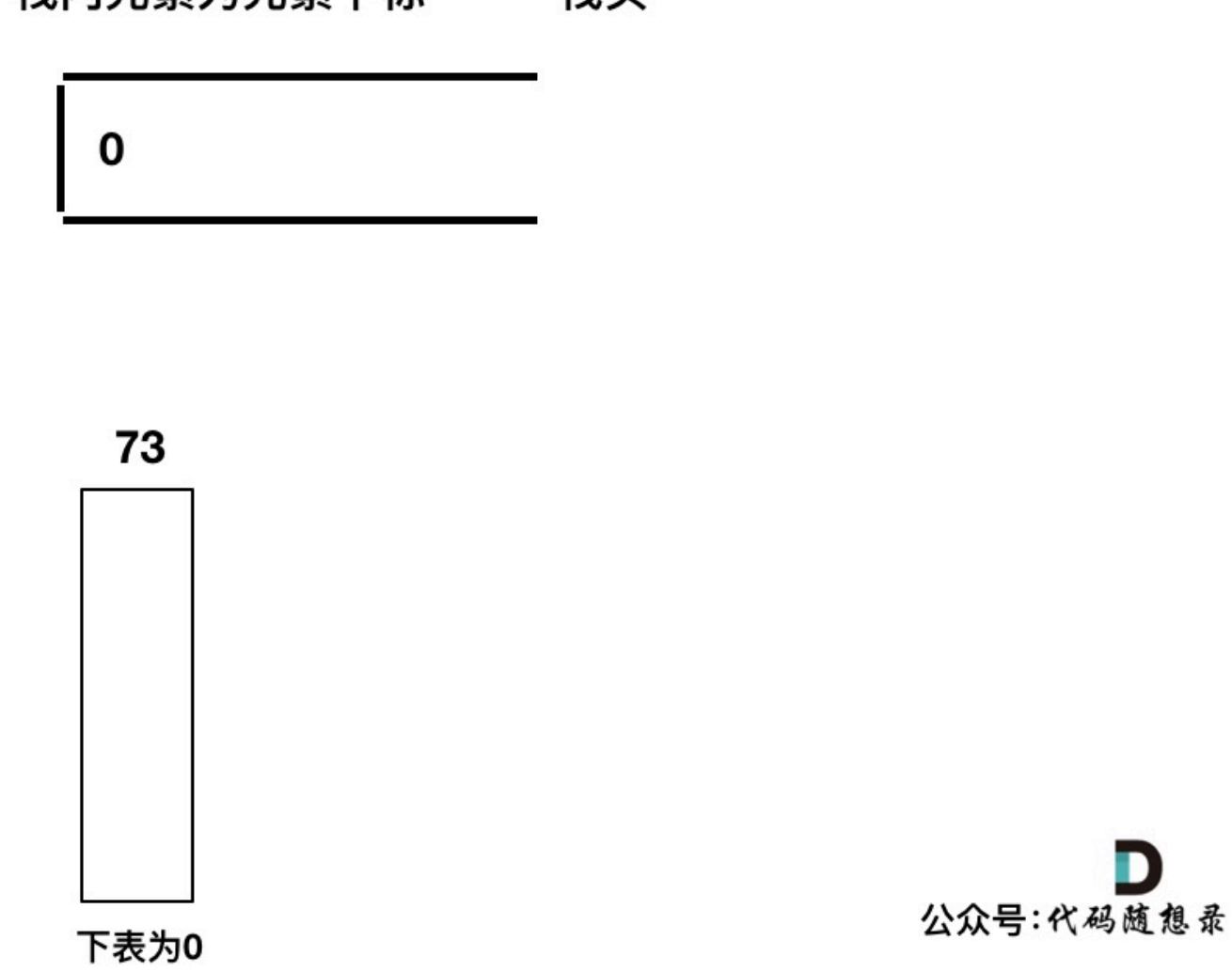
使用单调栈主要有三个判断条件。

- 当前遍历的元素T[i]小于栈顶元素T[st.top()]的情况
- 当前遍历的元素T[i]等于栈顶元素T[st.top()]的情况
- 当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况

把这三种情况分析清楚了，也就理解透彻了。

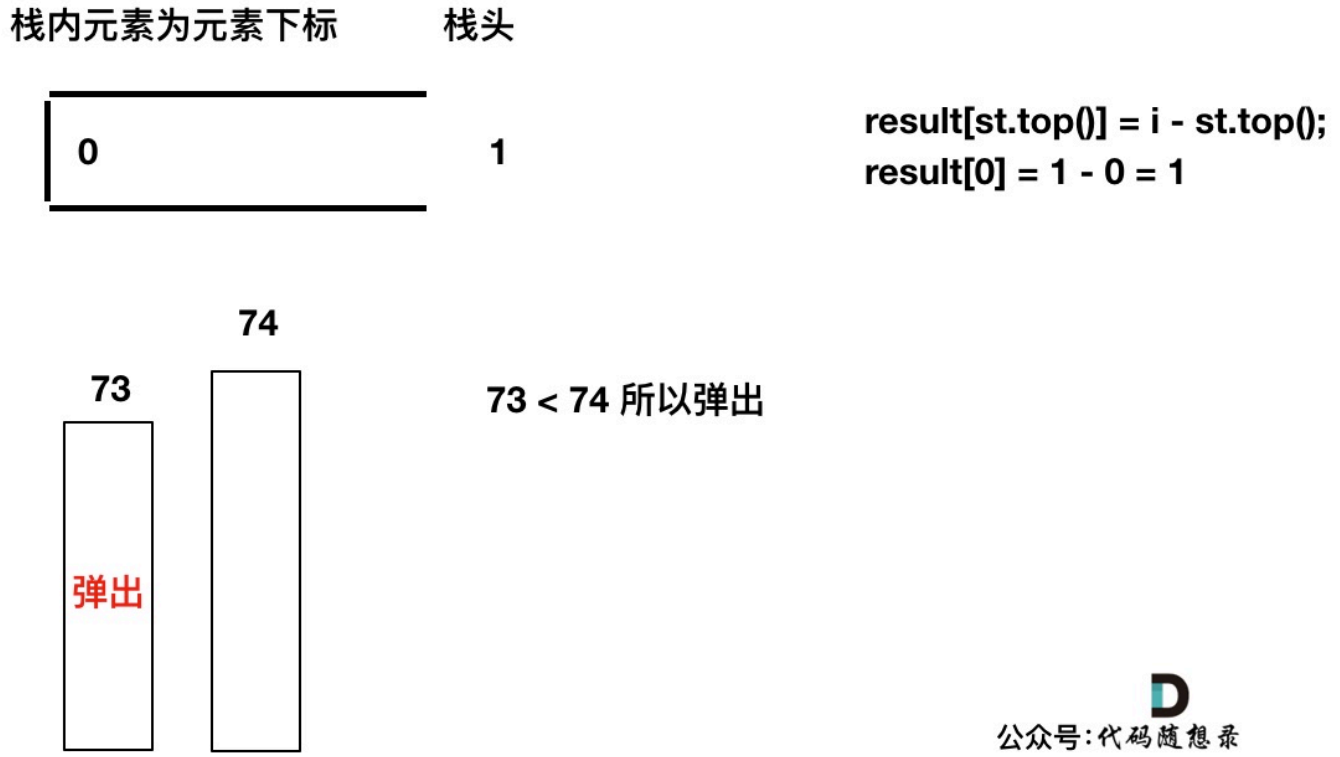
接下来我们用temperatures = [73, 74, 75, 71, 71, 72, 76, 73]为例来逐步分析，输出应该是 [1, 1, 4, 2, 1, 1, 0, 0]。

首先先将第一个遍历元素加入单调栈

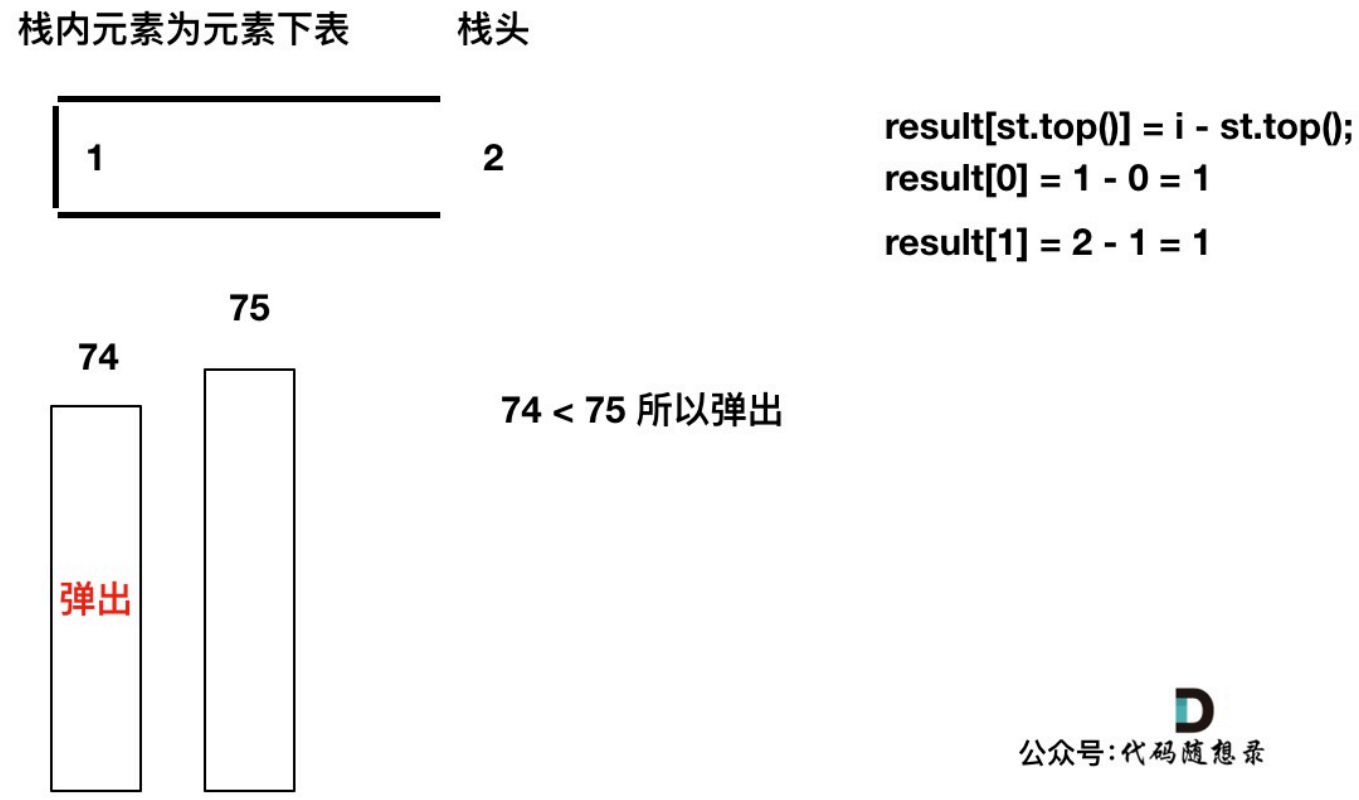


加入T[1] = 74，因为T[1] > T[0]（当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况）。

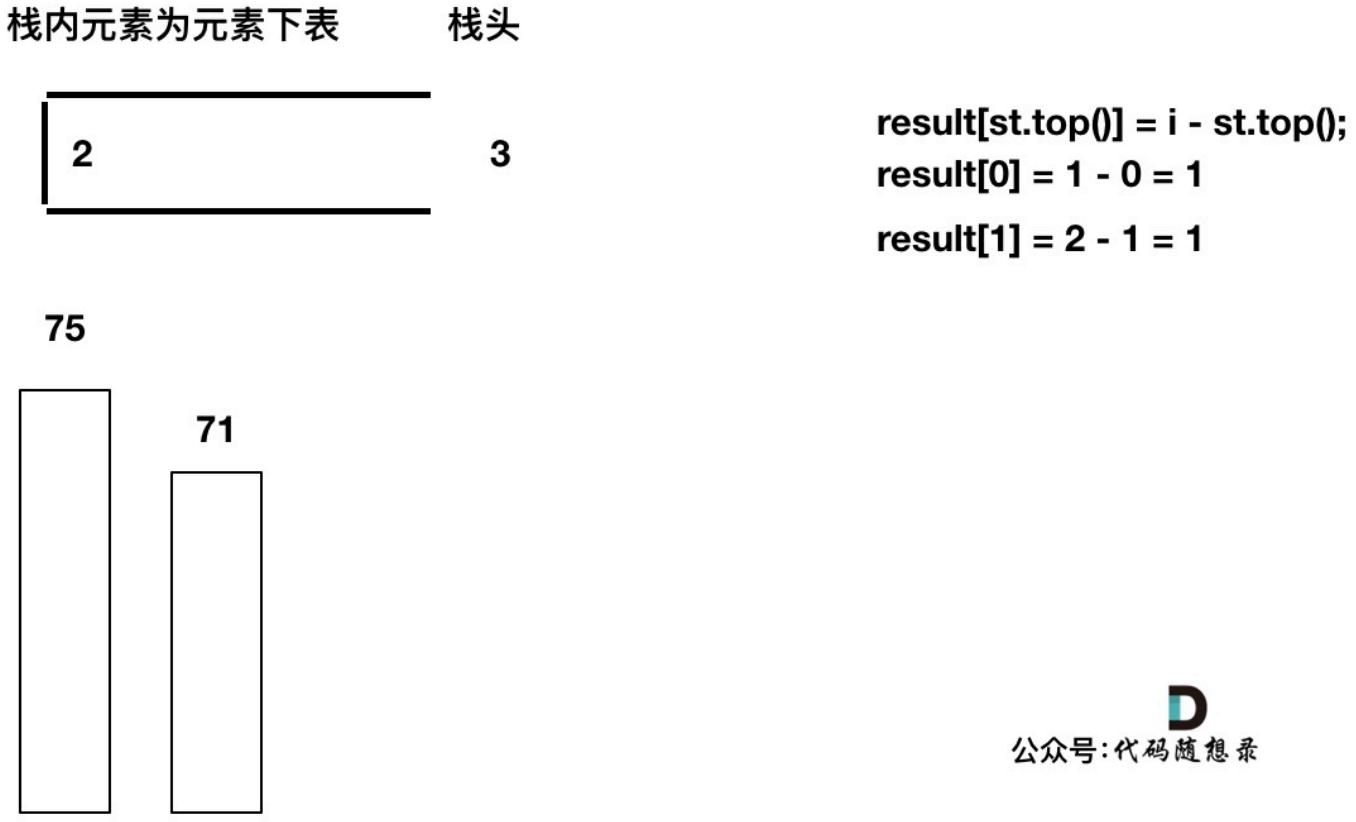
我们要保持一个递增单调栈（从栈头到栈底），所以将T[0]弹出，T[1]加入，此时result数组可以记录了，result[0] = 1，即T[0]右面第一个比T[0]大的元素是T[1]。



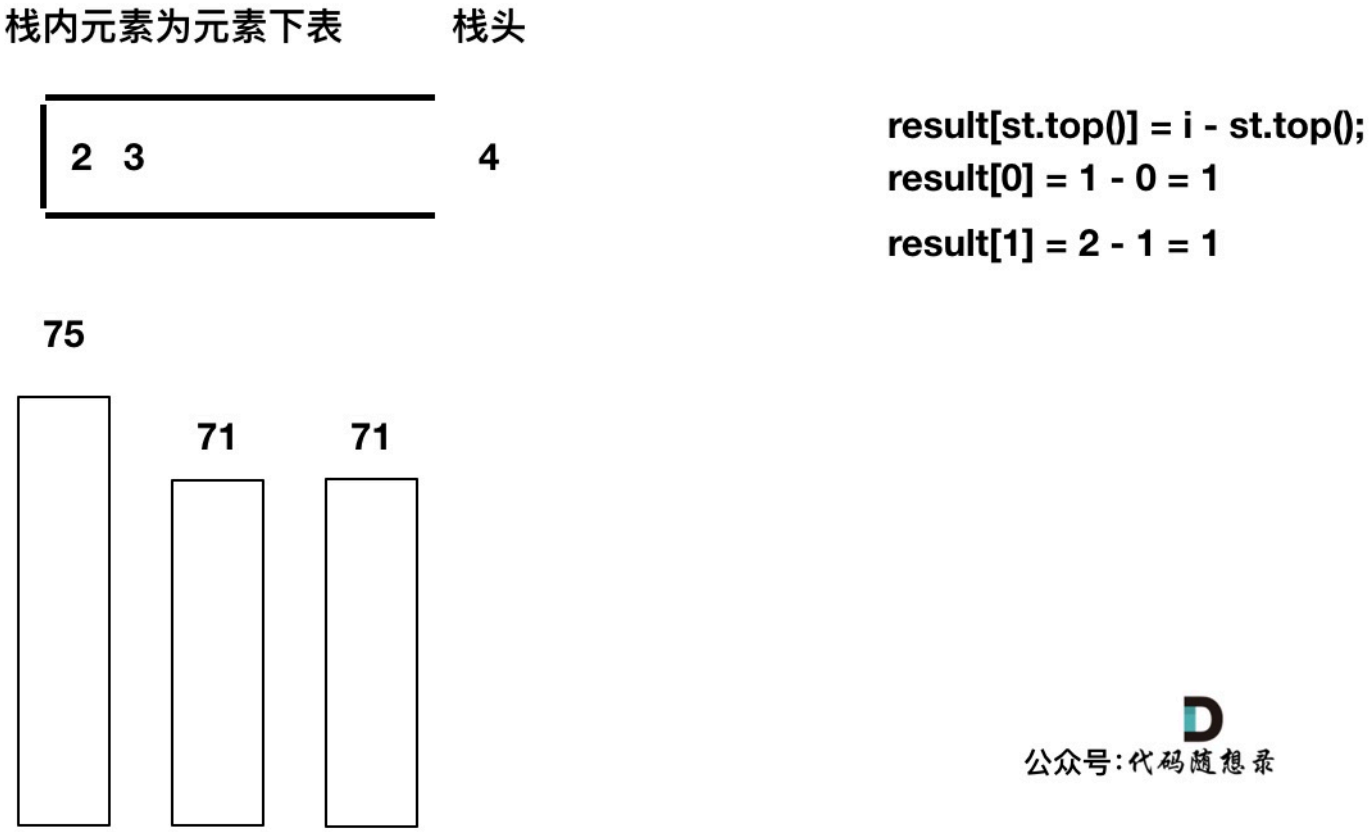
加入T[2]，同理，T[1]弹出



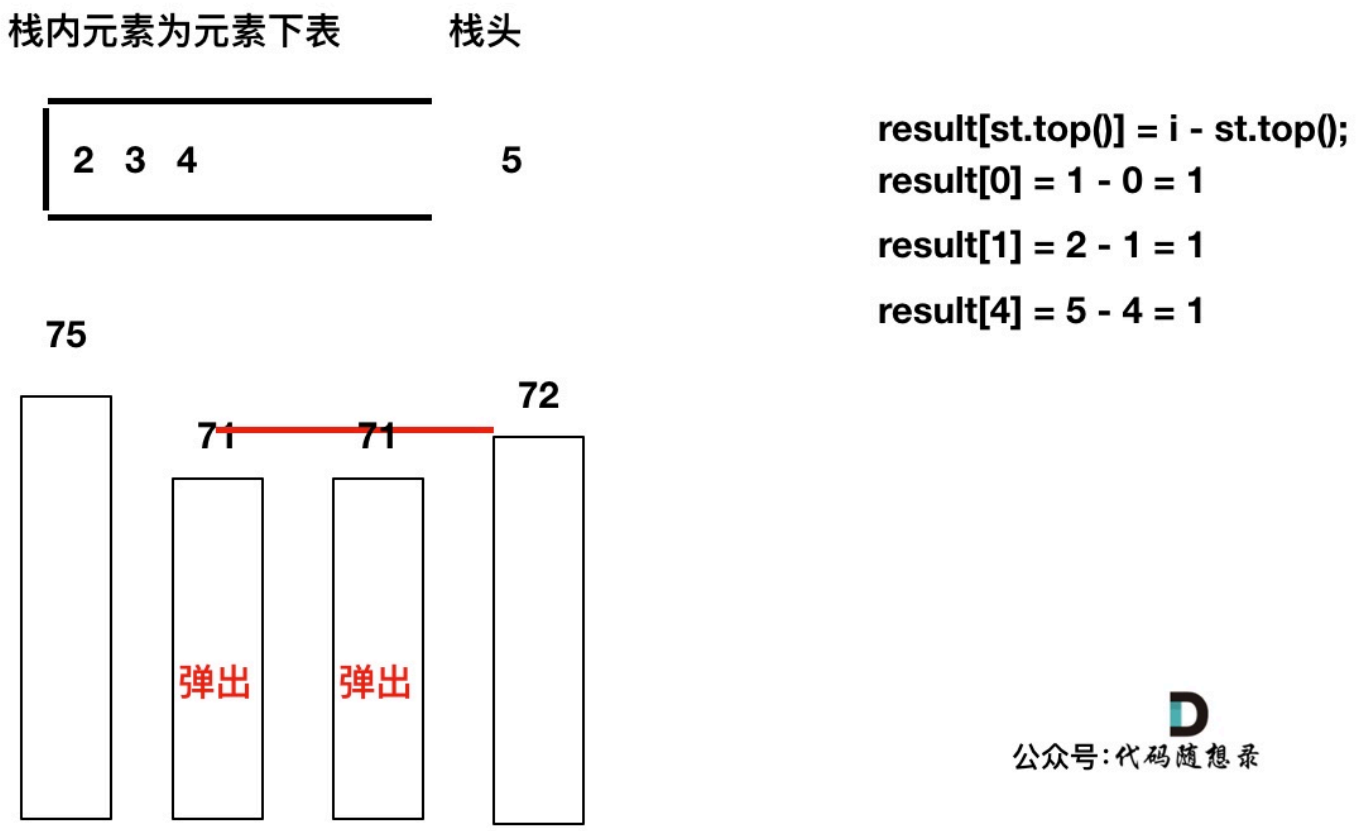
加入T[3]，T[3] < T[2] （当前遍历的元素T[i]小于栈顶元素T[st.top()]的情况），加T[3]加入单调栈。



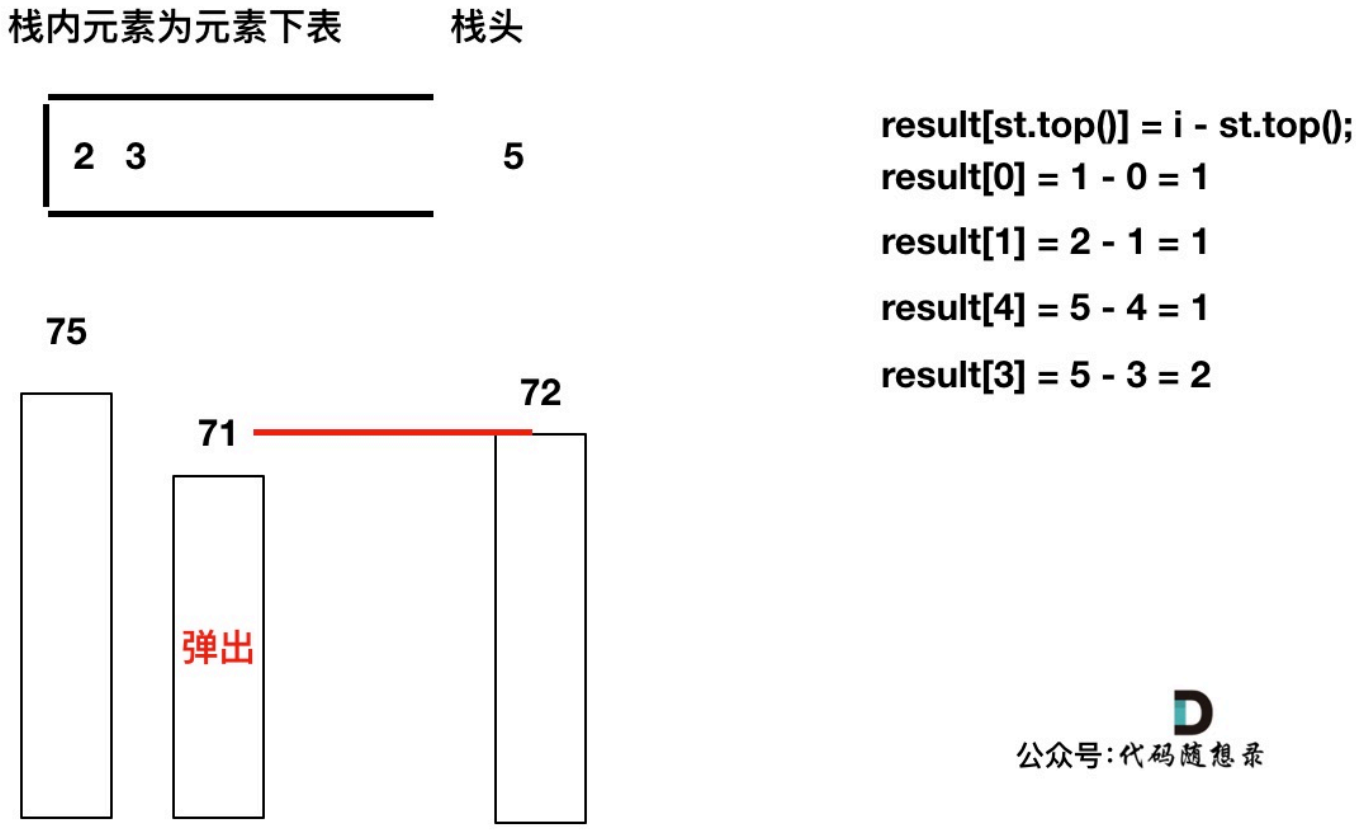
加入T[4]，T[4] == T[3] （当前遍历的元素T[i]等于栈顶元素T[st.top()]的情况），此时依然要加入栈，不用计算距离，因为我们要求的是右面第一个大于本元素的位置，而不是大于等于！



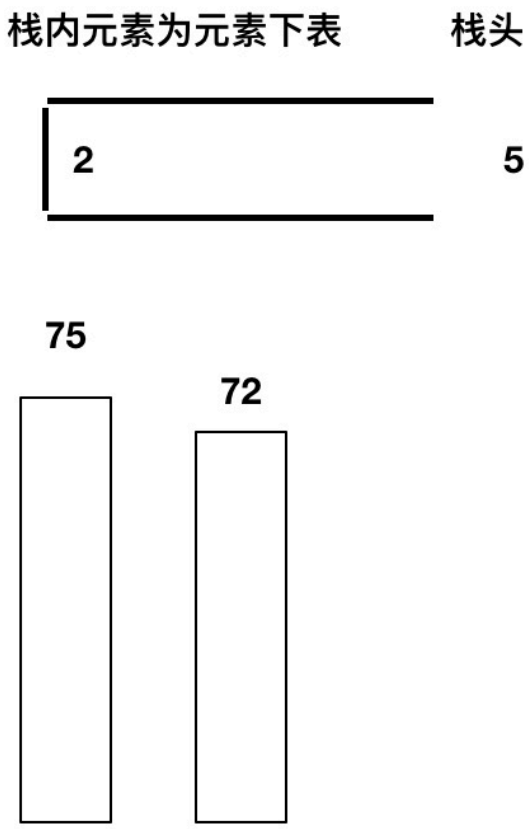
加入T[5]，T[5] > T[4] （当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况），将T[4]弹出，同时计算距离，更新result



T[4]弹出之后， T[5] > T[3] （当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况），将T[3]继续弹出，同时计算距离，更新result



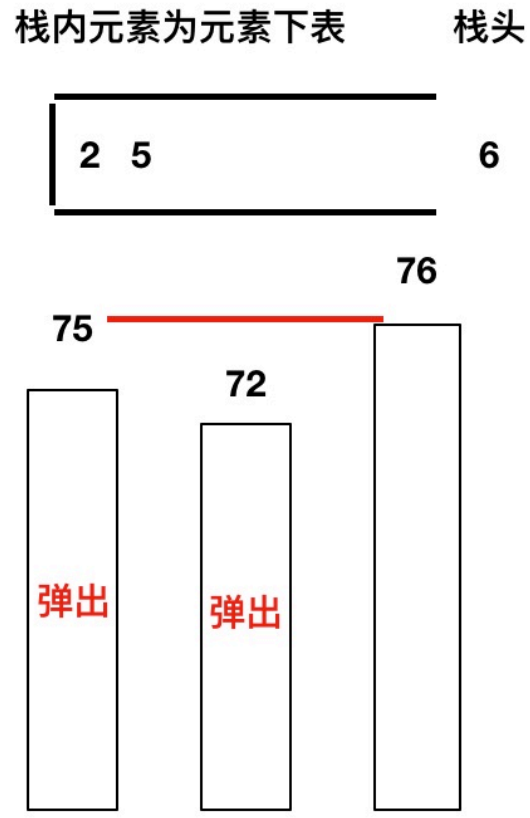
直到发现T[5]小于T[st.top()]，终止弹出，将T[5]加入单调栈



result[st.top()] = i - st.top();
result[0] = 1 - 0 = 1
result[1] = 2 - 1 = 1
result[4] = 5 - 4 = 1
result[3] = 5 - 3 = 2

公众号:代码随想录

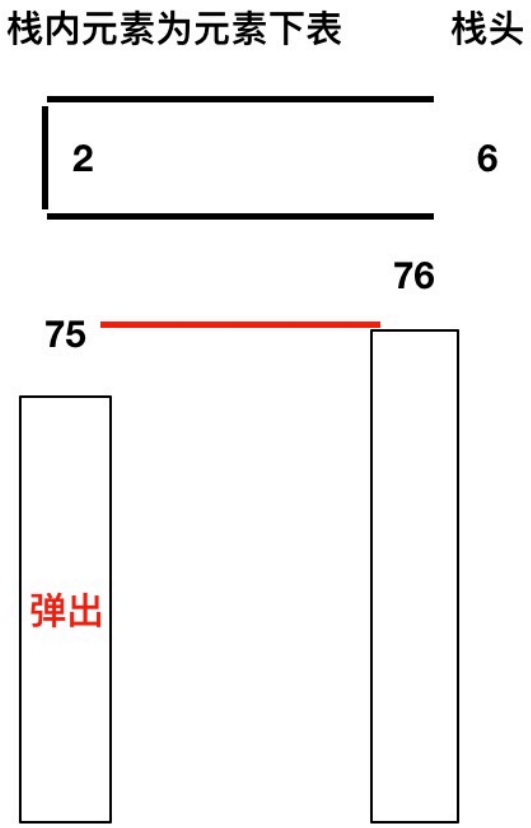
加入T[6]，同理，需要将栈里的T[5]，T[2]弹出



result[st.top()] = i - st.top();
result[0] = 1 - 0 = 1
result[1] = 2 - 1 = 1
result[4] = 5 - 4 = 1
result[3] = 5 - 3 = 2
result[5] = 6 - 5 = 1

公众号:代码随想录

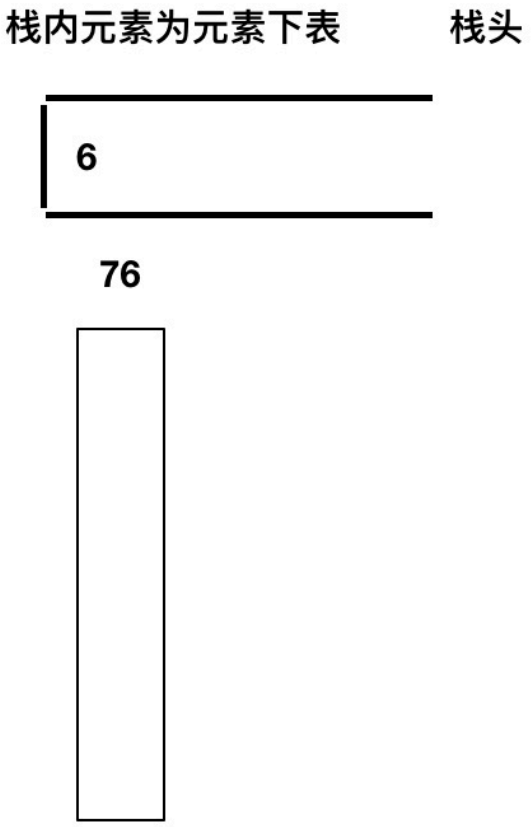
同理，继续弹出



result[st.top()] = i - st.top();
result[0] = 1 - 0 = 1
result[1] = 2 - 1 = 1
result[4] = 5 - 4 = 1
result[3] = 5 - 3 = 2
result[5] = 6 - 5 = 1
result[2] = 6 - 2 = 4

公众号:代码随想录

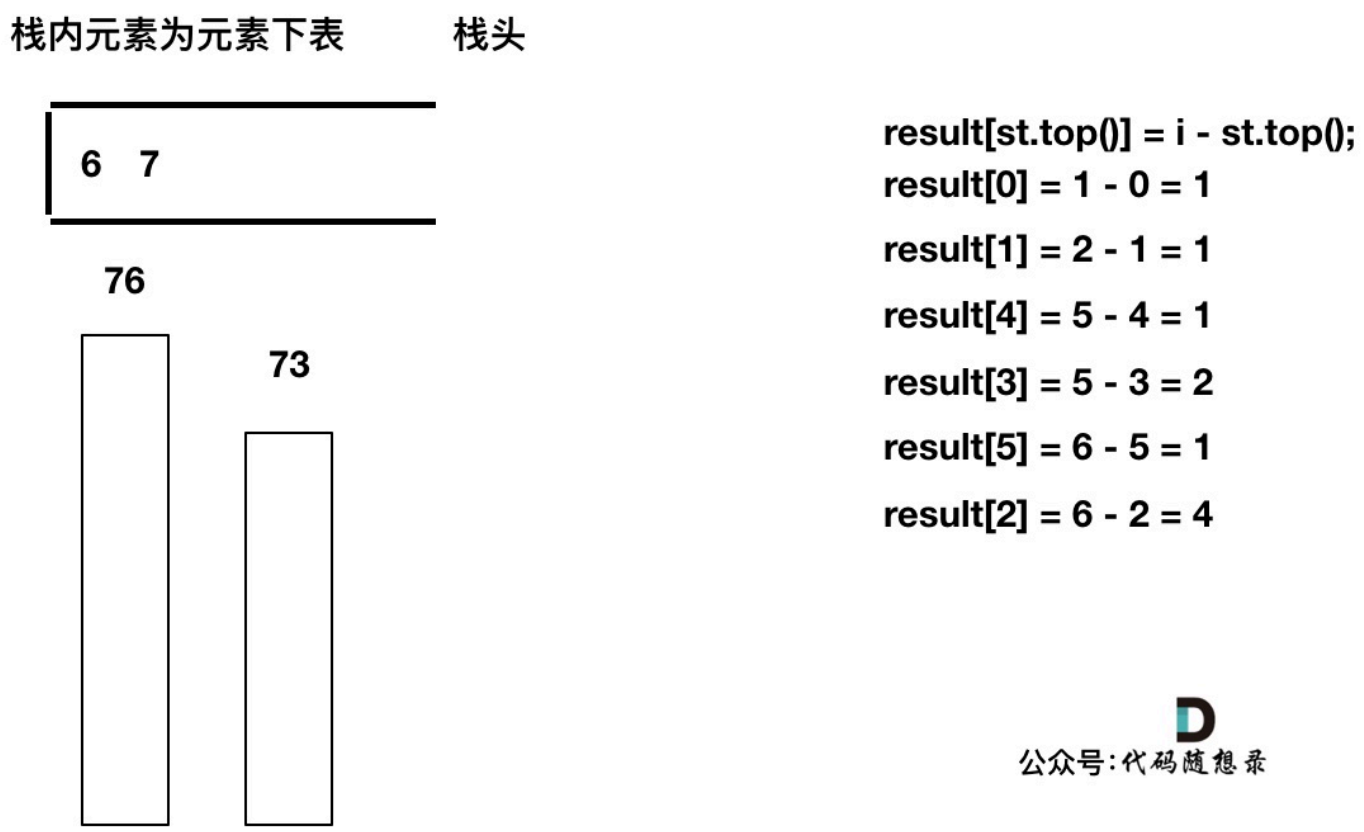
此时栈里只剩下了T[6]



result[st.top()] = i - st.top();
result[0] = 1 - 0 = 1
result[1] = 2 - 1 = 1
result[4] = 5 - 4 = 1
result[3] = 5 - 3 = 2
result[5] = 6 - 5 = 1
result[2] = 6 - 2 = 4

公众号:代码随想录

加入T[7]， T[7] < T[6] 直接入栈，这就是最后的情况，result数组也更新完了。



此时有同学可能就疑惑了，那result[6] , result[7]怎么没更新啊，元素也一直在栈里。

其实定义result数组的时候，就应该直接初始化为0，如果result没有更新，说明这个元素右面没有更大的了，也就是为0。

以上在图解的时候，已经把，这三种情况都做了详细的分析。

- 情况一：当前遍历的元素T[i]小于栈顶元素T[st.top()]的情况
- 情况二：当前遍历的元素T[i]等于栈顶元素T[st.top()]的情况
- 情况三：当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况

通过以上过程，大家可以自己再模拟一遍，就会发现：只有单调栈递增（从栈口到栈底顺序），就是求右边第一个比自己大的，单调栈递减的话，就是求右边第一个比自己小的。

C++代码如下：

```
// 版本一
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& T) {
        // 递增栈
        stack<int> st;
        vector<int> result(T.size(), 0);
        st.push(0);
        for (int i = 1; i < T.size(); i++) {
            if (T[i] < T[st.top()]) {                       // 情况一
                st.push(i);
```

```
            } else if (T[i] == T[st.top()]) {                // 情况二
                st.push(i);
            } else {
                while (!st.empty() && T[i] > T[st.top()]) { // 情况三
                    result[st.top()] = i - st.top();
                    st.pop();
                }
                st.push(i);
            }
        }
        return result;
    }
};
```

建议一开始 都把每种情况分析好，不要上来看简短的代码，关键逻辑都被隐藏了。

精简代码如下：

```
// 版本二
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& T) {
        stack<int> st; // 递增栈
        vector<int> result(T.size(), 0);
        for (int i = 0; i < T.size(); i++) {
            while (!st.empty() && T[i] > T[st.top()]) { // 注意栈不能为空
                result[st.top()] = i - st.top();
                st.pop();
            }
            st.push(i);

        }
        return result;
    }
};
```

- 时间复杂度：O(n)
- 空间复杂度：O(n)

精简的代码是直接把情况一二三都合并到了一起，其实这种代码精简是精简，但思路不是很清晰。

建议大家把情况一二三想清楚了，先写出版本一的代码，然后在其基础上在做精简！

2.下一个更大元素 I

力扣题目链接

给你两个 没有重复元素 的数组 nums1 和 nums2 ，其中nums1 是 nums2 的子集。

请你找出 nums1 中每个元素在 nums2 中的下一个比其大的值。

nums1 中数字 x 的下一个更大元素是指 x 在 nums2 中对应位置的右边的第一个比 x 大的元素。如果不存在，对应位置输出 -1 。

示例 1:

输入: nums1 = [4,1,2], nums2 = [1,3,4,2].
输出: [-1,3,-1]
解释:
对于 num1 中的数字 4 ，你无法在第二个数组中找到下一个更大的数字，因此输出 -1 。
对于 num1 中的数字 1 ，第二个数组中数字1右边的下一个较大数字是 3 。
对于 num1 中的数字 2 ，第二个数组中没有下一个更大的数字，因此输出 -1 。

示例 2:
输入: nums1 = [2,4], nums2 = [1,2,3,4].
输出: [3,-1]
解释:
对于 num1 中的数字 2 ，第二个数组中的下一个较大数字是 3 。
对于 num1 中的数字 4 ，第二个数组中没有下一个更大的数字，因此输出-1 。

提示：

- 1 <= nums1.length <= nums2.length <= 1000
- 0 <= nums1[i], nums2[i] <= 10^4
- nums1和nums2中所有整数 互不相同
- nums1 中的所有整数同样出现在 nums2 中

思路

做本题之前，建议先做一下739. 每日温度

在739. 每日温度中是求每个元素下一个比当前元素大的元素的位置。

本题则是说nums1 是 nums2的子集，找nums1中的元素在nums2中下一个比当前元素大的元素。

看上去和739. 每日温度 就如出一辙了。

几乎是一样的，但是这么绕了一下，其实还上升了一点难度。

需要对单调栈使用的更熟练一些，才能顺利的把本题写出来。

从题目示例中我们可以看出最后是要求nums1的每个元素在nums2中下一个比当前元素大的元素，那么就要定义一个和nums1一样大小的数组result来存放结果。

一些同学可能看到两个数组都已经懵了，不知道要定一个一个多大的result数组来存放结果了。

这么定义这个result数组初始化应该为多少呢?

题目说如果不存在对应位置就输出 -1 ，所以result数组如果某位置没有被赋值，那么就应该是是-1，所以就初始化为-1。

在遍历nums2的过程中，我们要判断nums2[i]是否在nums1中出现过，因为最后是要根据nums1元素的下标来更新result数组。

注意题目中说是两个没有重复元素 的数组 nums1 和 nums2。

没有重复元素，我们就可以用map来做映射了。根据数值快速找到下标，还可以判断nums2[i]是否在nums1中出现过。

C++中，当我们要使用集合来解决哈希问题的时候，优先使用unordered_set，因为它的查询和增删效率是最优的。我在关于哈希表，你该了解这些！中也做了详细的解释。

那么预处理代码如下:

```
unordered_map<int, int> umap; // key:下标元素，value：下标
for (int i = 0; i < nums1.size(); i++) {
    umap[nums1[i]] = i;
}
```

使用单调栈，首先要想单调栈是从大到小还是从小到大。

本题和739. 每日温度是一样的。

栈头到栈底的顺序，要从小到大，也就是保持栈里的元素为递增顺序。只要保持递增，才能找到右边第一个比自己大的元素。

可能这里有一些同学不理解，那么可以自己尝试一下用递减栈，能不能求出来。**其实递减栈就是求右边第一个比自己小的元素了。**

接下来就要分析如下三种情况，一定要分析清楚。

1. 情况一：当前遍历的元素T[i]小于栈顶元素T[st.top()]的情况

此时满足递增栈（栈头到栈底的顺序），所以直接入栈。

2. 情况二：当前遍历的元素T[i]等于栈顶元素T[st.top()]的情况

如果相等的话，依然直接入栈，因为我们要求的是右边第一个比自己大的元素，而不是大于等于！

3. 情况三：当前遍历的元素T[i]大于栈顶元素T[st.top()]的情况

此时如果入栈就不满足递增栈了，这也是找到右边第一个比自己大的元素的时候。

判断栈顶元素是否在nums1里出现过，（注意栈里的元素是nums2的元素），如果出现过，开始记录结果。

记录结果这块逻辑有一点小绕，要清楚，此时栈顶元素在nums2数组中右面第一个大的元素是nums2[i]（即当前遍历元素）。

代码如下：

```
while (!st.empty() && nums2[i] > nums2[st.top()]) {
    if (umap.count(nums2[st.top()]) > 0) { // 看map里是否存在这个元素
        int index = umap[nums2[st.top()]]; // 根据map找到nums2[st.top()] 在 nums1中的下标
        result[index] = nums2[i];
    }
    st.pop();
}
st.push(i);
```

以上分析完毕，C++代码如下：（其实本题代码和 739. 每日温度 是基本差不多的）

```
// 版本一
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        stack<int> st;
        vector<int> result(nums1.size(), -1);
        if (nums1.size() == 0) return result;

        unordered_map<int, int> umap; // key:下标元素，value：下标
        for (int i = 0; i < nums1.size(); i++) {
            umap[nums1[i]] = i;
        }
        st.push(0);
        for (int i = 1; i < nums2.size(); i++) {
            if (nums2[i] < nums2[st.top()]) {           // 情况一
                st.push(i);
            } else if (nums2[i] == nums2[st.top()]) {   // 情况二
                st.push(i);
            } else {                                    // 情况三
                while (!st.empty() && nums2[i] > nums2[st.top()]) {
                    if (umap.count(nums2[st.top()]) > 0) { // 看map里是否存在这个元素
                        int index = umap[nums2[st.top()]]; // 根据map找到nums2[st.top()]
在 nums1中的下标
                        result[index] = nums2[i];
                    }
                    st.pop();
                }
                st.push(i);
            }
        }
        return result;
    }
};
```

针对版本一，进行代码精简后，代码如下：

```
// 版本二
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        stack<int> st;
        vector<int> result(nums1.size(), -1);
        if (nums1.size() == 0) return result;

        unordered_map<int, int> umap; // key:下标元素，value：下标
        for (int i = 0; i < nums1.size(); i++) {
            umap[nums1[i]] = i;
```

```
        }
        st.push(0);
        for (int i = 1; i < nums2.size(); i++) {
            while (!st.empty() && nums2[i] > nums2[st.top()]) {
                if (umap.count(nums2[st.top()]) > 0) { // 看map里是否存在这个元素
                    int index = umap[nums2[st.top()]]; // 根据map找到nums2[st.top()] 在
nums1中的下标
                    result[index] = nums2[i];
                }
                st.pop();
            }
            st.push(i);
        }
        return result;
    }
};
```

精简的代码是直接把情况一二三都合并到了一起，其实这种代码精简是精简，但思路不是很清晰。

建议大家把情况一二三想清楚了，先写出版本一的代码，然后在其基础上在做精简！

3.下一个更大元素II

[力扣题目链接]

给定一个循环数组（最后一个元素的下一个元素是数组的第一个元素），输出每个元素的下一个更大元素。数字 x 的下一个更大的元素是按数组遍历顺序，这个数字之后的第一个比它更大的数，这意味着你应该循环地搜索它的下一个更大的数。如果不存在，则输出 -1。

示例 1:

- 输入: [1,2,1]
- 输出: [2,-1,2]
- 解释: 第一个 1 的下一个更大的数是 2；数字 2 找不到下一个更大的数；第二个 1 的下一个最大的数需要循环搜索，结果也是 2。

提示:

- 1 <= nums.length <= 10^4
- -10^9 <= nums[i] <= 10^9

思路

做本题之前建议先做739. 每日温度 和 496.下一个更大元素 I。

这道题和739. 每日温度也几乎如出一辙。

不过，本题要循环数组了。

关于单调栈的讲解我在题解739. 每日温度中已经详细讲解了。

本篇我侧重与说一说，如何处理循环数组。

相信不少同学看到这道题，就想那我直接把两个数组拼接在一起，然后使用单调栈求下一个最大值不就行了！

确实可以！

将两个nums数组拼接在一起，使用单调栈计算出每一个元素的下一个最大值，最后再把结果集即result数组resize到原数组大小就可以了。

代码如下：

```
// 版本一
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        // 拼接一个新的nums
        vector<int> nums1(nums.begin(), nums.end());
        nums.insert(nums.end(), nums1.begin(), nums1.end());
        // 用新的nums大小来初始化result
        vector<int> result(nums.size(), -1);
        if (nums.size() == 0) return result;

        // 开始单调栈
        stack<int> st;
        st.push(0);
        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] < nums[st.top()]) st.push(i);
            else if (nums[i] == nums[st.top()]) st.push(i);
            else {
                while (!st.empty() && nums[i] > nums[st.top()]) {
                    result[st.top()] = nums[i];
                    st.pop();
                }
                st.push(i);
            }
        }
        // 最后再把结果集即result数组resize到原数组大小
        result.resize(nums.size() / 2);
        return result;
    }
};
```

这种写法确实比较直观，但做了很多无用操作，例如修改了nums数组，而且最后还要把result数组resize回去。

resize倒是不费时间，是O(1)的操作，但扩充nums数组相当于多了一个O(n)的操作。

其实也可以不扩充nums，而是在遍历的过程中模拟走了两边nums。

代码如下：

```
// 版本二
class Solution {
```

```
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        vector<int> result(nums.size(), -1);
        if (nums.size() == 0) return result;
        stack<int> st;
        st.push(0);
        for (int i = 1; i < nums.size() * 2; i++) {
            // 模拟遍历两边nums，注意一下都是用i % nums.size()来操作
            if (nums[i % nums.size()] < nums[st.top()]) st.push(i % nums.size());
            else if (nums[i % nums.size()] == nums[st.top()]) st.push(i % nums.size());
            else {
                while (!st.empty() && nums[i % nums.size()] > nums[st.top()]) {
                    result[st.top()] = nums[i % nums.size()];
                    st.pop();
                }
                st.push(i % nums.size());
            }
        }
        return result;
    }
};
```

可以版本二不仅代码精简了，也比版本一少做了无用功！

最后在给出 单调栈的精简版本，即三种情况都做了合并的操作。

```
// 版本二
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        vector<int> result(nums.size(), -1);
        if (nums.size() == 0) return result;
        stack<int> st;
        for (int i = 0; i < nums.size() * 2; i++) {
            // 模拟遍历两边nums，注意一下都是用i % nums.size()来操作
            while (!st.empty() && nums[i % nums.size()] > nums[st.top()]) {
                result[st.top()] = nums[i % nums.size()];
                st.pop();
            }
            st.push(i % nums.size());
        }
        return result;
    }
};
```

> 这个图就是大厂面试经典题目，接雨水！ 最常青藤的一道题，面试官百出不厌！

4. 接雨水

力扣题目链接

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：

- 输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]
- 输出：6
- 解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2：

- 输入：height = [4,2,0,3,2,5]
- 输出：9

思路

接雨水问题在面试中还是常见题目的，有必要好好讲一讲。

本文深度讲解如下三种方法：

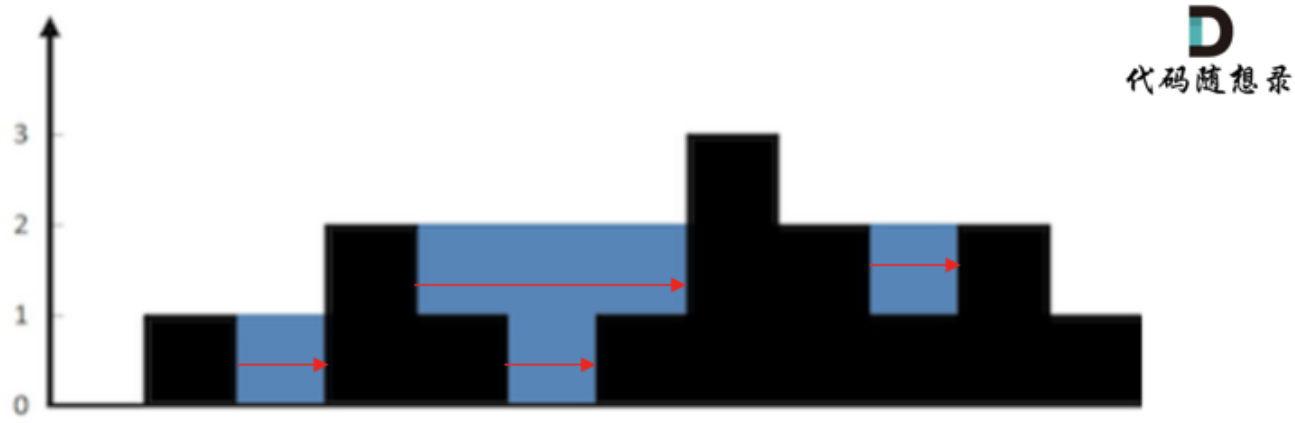
- 双指针法
- 动态规划
- 单调栈

暴力解法

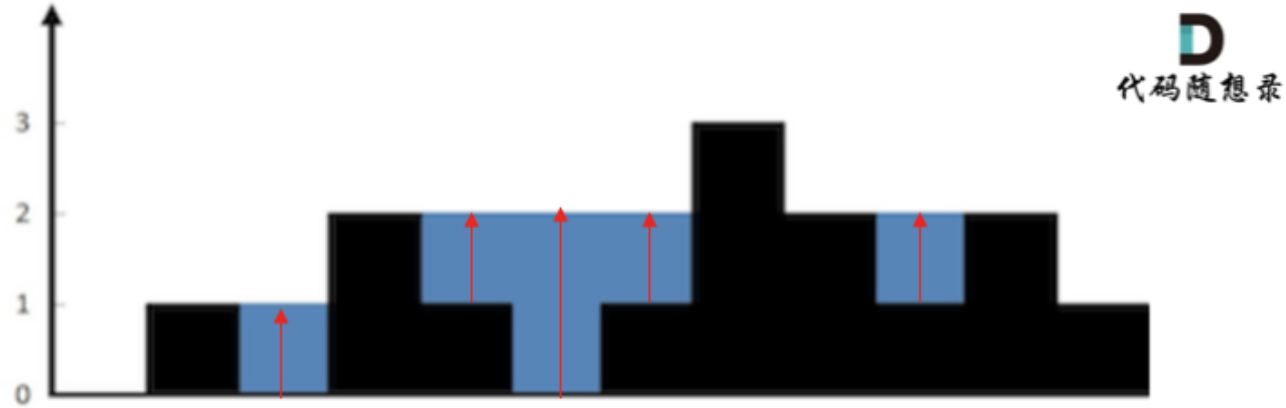
本题暴力解法也是也是使用双指针。

首先要明确，要按照行来计算，还是按照列来计算。

按照行来计算如图：



按照列来计算如图：



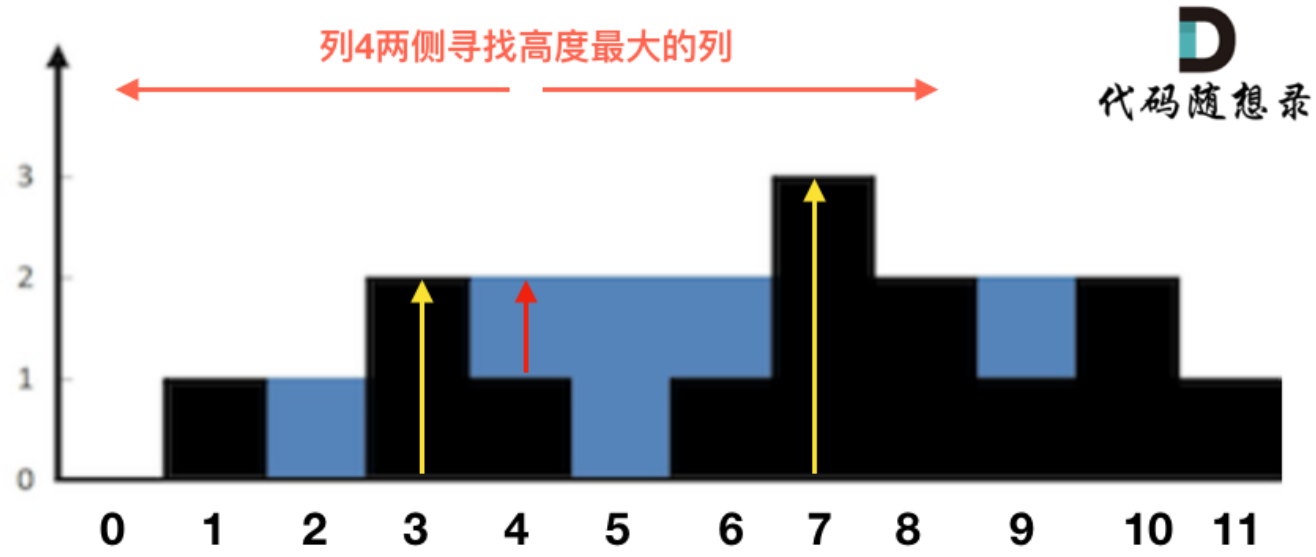
一些同学在实现的时候，很容易一会按照行来计算一会按照列来计算，这样就会越写越乱。

我个人倾向于按照列来计算，比较容易理解，接下来看一下按照列如何计算。

首先，**如果按照列来计算的话，宽度一定是1了，我们再把每一列的雨水的高度求出来就可以了。**

可以看出每一列雨水的高度，取决于，该列 左侧最高的柱子和右侧最高的柱子中最矮的那个柱子的高度。

这句话可以有点绕，来举一个理解，例如求列4的雨水高度，如图：



列4 左侧最高的柱子是列3，高度为2（以下用lHeight表示）。

列4 右侧最高的柱子是列7，高度为3（以下用rHeight表示）。

列4 柱子的高度为1（以下用height表示）

那么列4的雨水高度为 列3和列7的高度最小值减列4高度，即： min(lHeight, rHeight) - height。

列4的雨水高度求出来了，宽度为1，相乘就是列4的雨水体积了。

此时求出了列4的雨水体积。

一样的方法，只要从头遍历一遍所有的列，然后求出每一列雨水的体积，相加之后就是总雨水的体积了。

首先从头遍历所有的列，并且**要注意第一个柱子和最后一个柱子不接雨水**，代码如下：

```
for (int i = 0; i < height.size(); i++) {
    // 第一个柱子和最后一个柱子不接雨水
    if (i == 0 || i == height.size() - 1) continue;
}
```

在for循环中求左右两边最高柱子，代码如下：

```
int rHeight = height[i]; // 记录右边柱子的最高高度
int lHeight = height[i]; // 记录左边柱子的最高高度
for (int r = i + 1; r < height.size(); r++) {
    if (height[r] > rHeight) rHeight = height[r];
}
for (int l = i - 1; l >= 0; l--) {
    if (height[l] > lHeight) lHeight = height[l];
}
```

最后，计算该列的雨水高度，代码如下：

```
int h = min(lHeight, rHeight) - height[i];
if (h > 0) sum += h; // 注意只有h大于零的时候，在统计到总和中
```

整体代码如下：

```
class Solution {
public:
    int trap(vector<int>& height) {
        int sum = 0;
        for (int i = 0; i < height.size(); i++) {
            // 第一个柱子和最后一个柱子不接雨水
            if (i == 0 || i == height.size() - 1) continue;

            int rHeight = height[i]; // 记录右边柱子的最高高度
            int lHeight = height[i]; // 记录左边柱子的最高高度
            for (int r = i + 1; r < height.size(); r++) {
                if (height[r] > rHeight) rHeight = height[r];
            }
            for (int l = i - 1; l >= 0; l--) {
                if (height[l] > lHeight) lHeight = height[l];
            }
            int h = min(lHeight, rHeight) - height[i];
            if (h > 0) sum += h;
        }
        return sum;
    }
};
```

因为每次遍历列的时候，还要向两边寻找最高的列，所以时间复杂度为O(n^2)，空间复杂度为O(1)。

力扣后面修改了后台测试数据，所以以上暴力解法超时了。

双指针优化

在暴力解法中，我们可以看到只要记录左边柱子的最高高度 和 右边柱子的最高高度，就可以计算当前位置的雨水面积，这就是通过列来计算。

当前列雨水面积：min(左边柱子的最高高度，记录右边柱子的最高高度) - 当前柱子高度。

为了得到两边的最高高度，使用了双指针来遍历，每到一个柱子都向两边遍历一遍，这其实是有重复计算的。我们把每一个位置的左边最高高度记录在一个数组上（maxLeft），右边最高高度记录在一个数组上（maxRight），这样就避免了重复计算。

当前位置，左边的最高高度是前一个位置的左边最高高度和本高度的最大值。

即从左向右遍历：maxLeft[i] = max(height[i], maxLeft[i - 1]);

从右向左遍历：maxRight[i] = max(height[i], maxRight[i + 1]);

代码如下：

```cpp
class Solution {
public:
    int trap(vector<int>& height) {
        if (height.size() <= 2) return 0;
        vector<int> maxLeft(height.size(), 0);
        vector<int> maxRight(height.size(), 0);
        int size = maxRight.size();

        // 记录每个柱子左边柱子最大高度
        maxLeft[0] = height[0];
        for (int i = 1; i < size; i++) {
            maxLeft[i] = max(height[i], maxLeft[i - 1]);
        }
        // 记录每个柱子右边柱子最大高度
        maxRight[size - 1] = height[size - 1];
        for (int i = size - 2; i >= 0; i--) {
            maxRight[i] = max(height[i], maxRight[i + 1]);
        }
        // 求和
        int sum = 0;
        for (int i = 0; i < size; i++) {
            int count = min(maxLeft[i], maxRight[i]) - height[i];
            if (count > 0) sum += count;
        }
        return sum;
    }
};
```

单调栈解法

关于单调栈的理论基础，单调栈适合解决什么问题，单调栈的工作过程，大家可以先看这题讲解 739. 每日温度。

单调栈就是保持栈内元素有序。和栈与队列：单调队列一样，需要我们自己维持顺序，没有现成的容器可以用。

通常是一维数组，要寻找任一个元素的右边或者左边第一个比自己大或者小的元素的位置，此时我们就要想到可以用单调栈了。

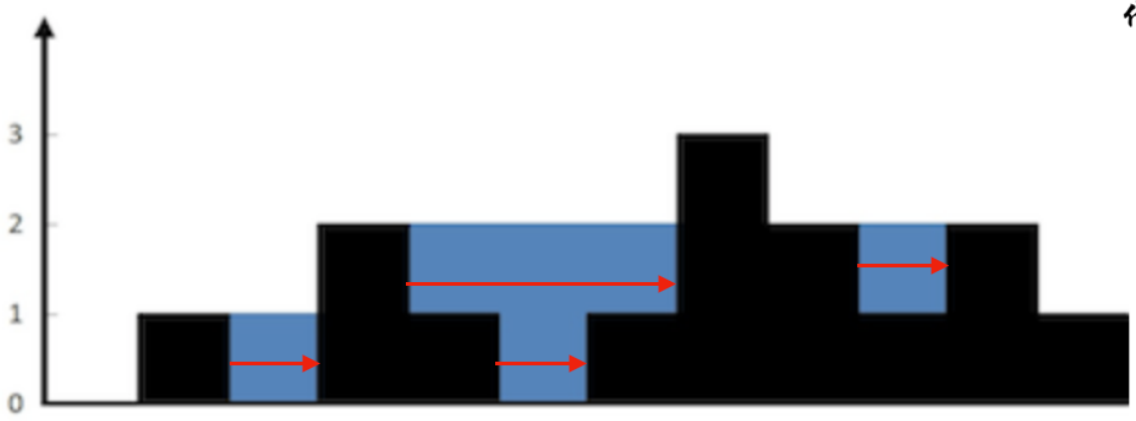
而接雨水这道题目，我们正需要寻找一个元素，右边最大元素以及左边最大元素，来计算雨水面积。

准备工作

那么本题使用单调栈有如下几个问题：

1. 首先单调栈是按照行方向来计算雨水，如图：

按照行计算



上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。 **感谢 Marcos** 贡献此图。

知道这一点，后面的就可以理解了。

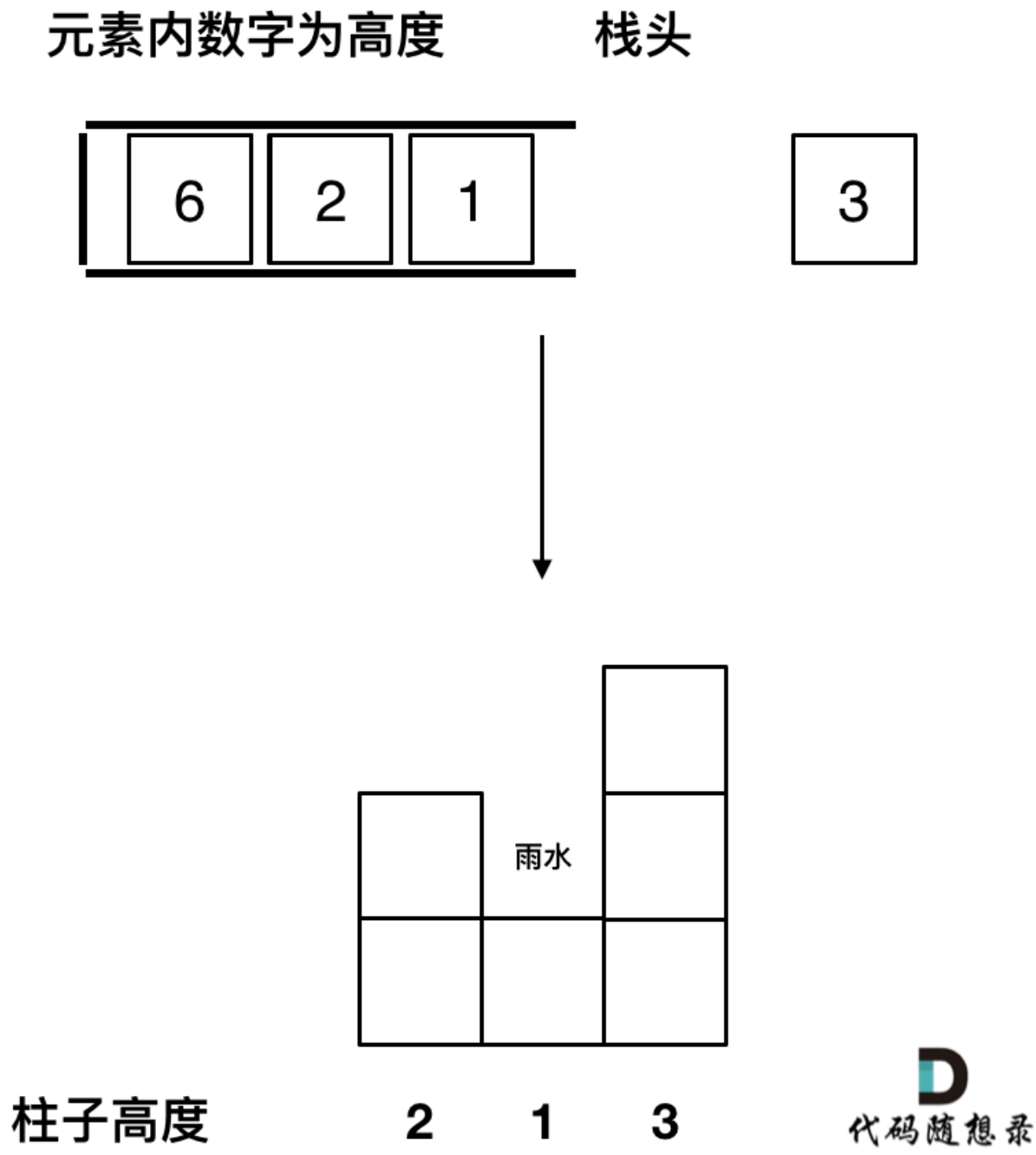
2. 使用单调栈内元素的顺序

从大到小还是从小到大呢?

从栈头（元素从栈头弹出）到栈底的顺序应该是从小到大的顺序。

因为一旦发现添加的柱子高度大于栈头元素了，此时就出现凹槽了，栈头元素就是凹槽底部的柱子，栈头第二个元素就是凹槽左边的柱子，而添加的元素就是凹槽右边的柱子。

如图：



关于单调栈的顺序给大家一个总结： 739. 每日温度 中求一个元素右边第一个更大元素，单调栈就是递增的，84.柱状图中最大的矩形求一个元素右边第一个更小元素，单调栈就是递减的。

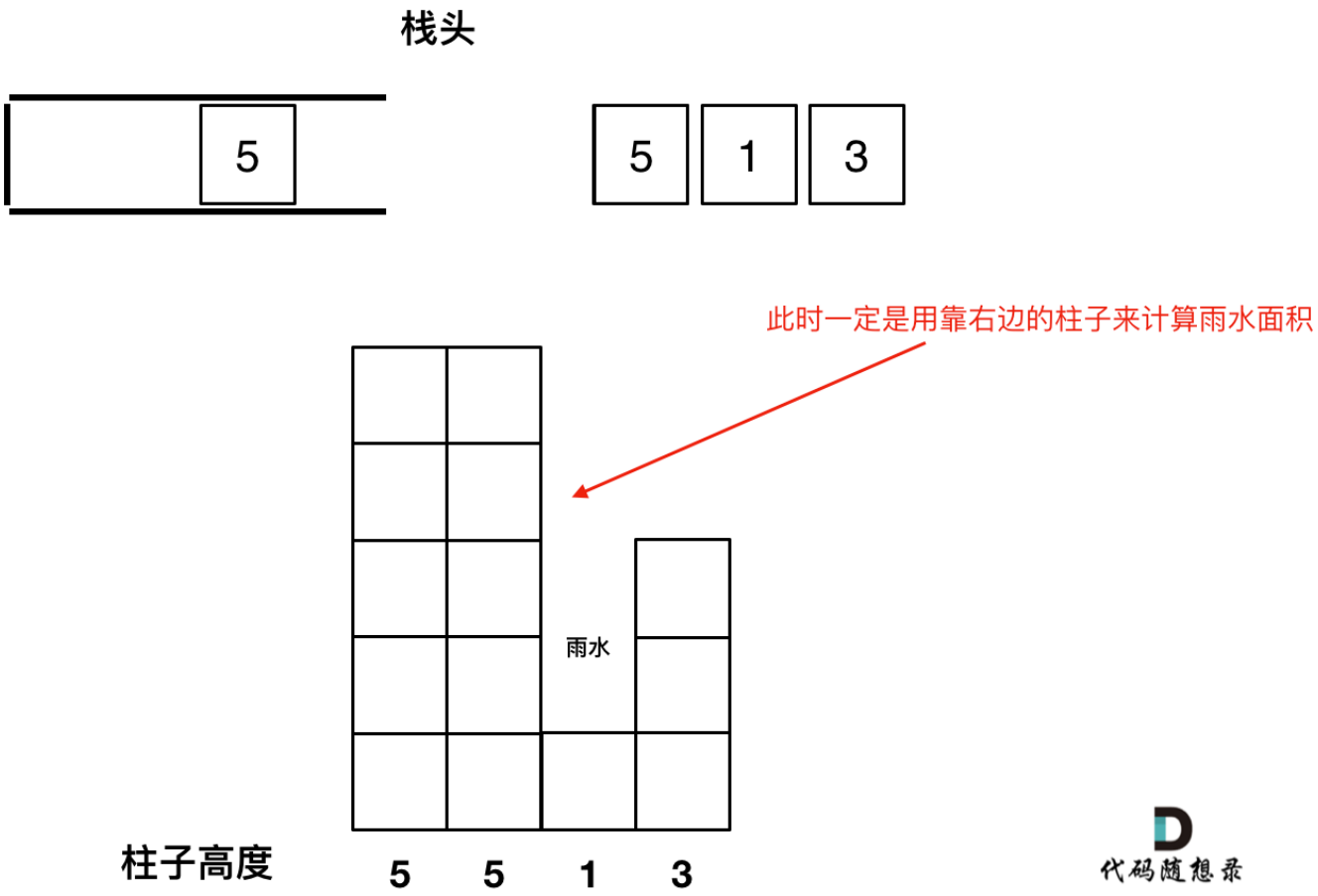
3. 遇到相同高度的柱子怎么办。

遇到相同的元素，更新栈内下标，就是将栈里元素（旧下标）弹出，将新元素（新下标）加入栈中。

例如 5 5 1 3 这种情况。如果添加第二个5的时候就应该将第一个5的下标弹出，把第二个5添加到栈中。

因为我们要求宽度的时候 如果遇到相同高度的柱子，需要使用最右边的柱子来计算宽度。

如图所示：



4. 栈里要保存什么数值

使用单调栈，也是通过 长 * 宽 来计算雨水面积的。

长就是通过柱子的高度来计算，宽是通过柱子之间的下标来计算，

那么栈里有没有必要存一个pair<int, int>类型的元素，保存柱子的高度和下标呢。

其实不用，栈里就存放下标就行，想要知道对应的高度，通过height[stack.top()] 就知道弹出的下标对应的高度了。

所以栈的定义如下：

```
stack<int> st; // 存着下标，计算的时候用下标对应的柱子高度
```

明确了如上几点，我们再来看处理逻辑。

单调栈处理逻辑

以下操作过程其实和 739. 每日温度 也是一样的，建议先做 739. 每日温度。

以下逻辑主要就是三种情况

- 情况一：当前遍历的元素（柱子）高度小于栈顶元素的高度 height[i] < height[st.top()]
- 情况二：当前遍历的元素（柱子）高度等于栈顶元素的高度 height[i] == height[st.top()]
- 情况三：当前遍历的元素（柱子）高度大于栈顶元素的高度 height[i] > height[st.top()]

先将下标0的柱子加入到栈中，`st.push(0);`。 栈中存放我们遍历过的元素，所以先将下标0加进来。

然后开始从下标1开始遍历所有的柱子，`for (int i = 1; i < height.size(); i++)`。

如果当前遍历的元素（柱子）高度小于栈顶元素的高度，就把这个元素加入栈中，因为栈里本来就要保持从小到大的顺序（从栈头到栈底）。

代码如下：

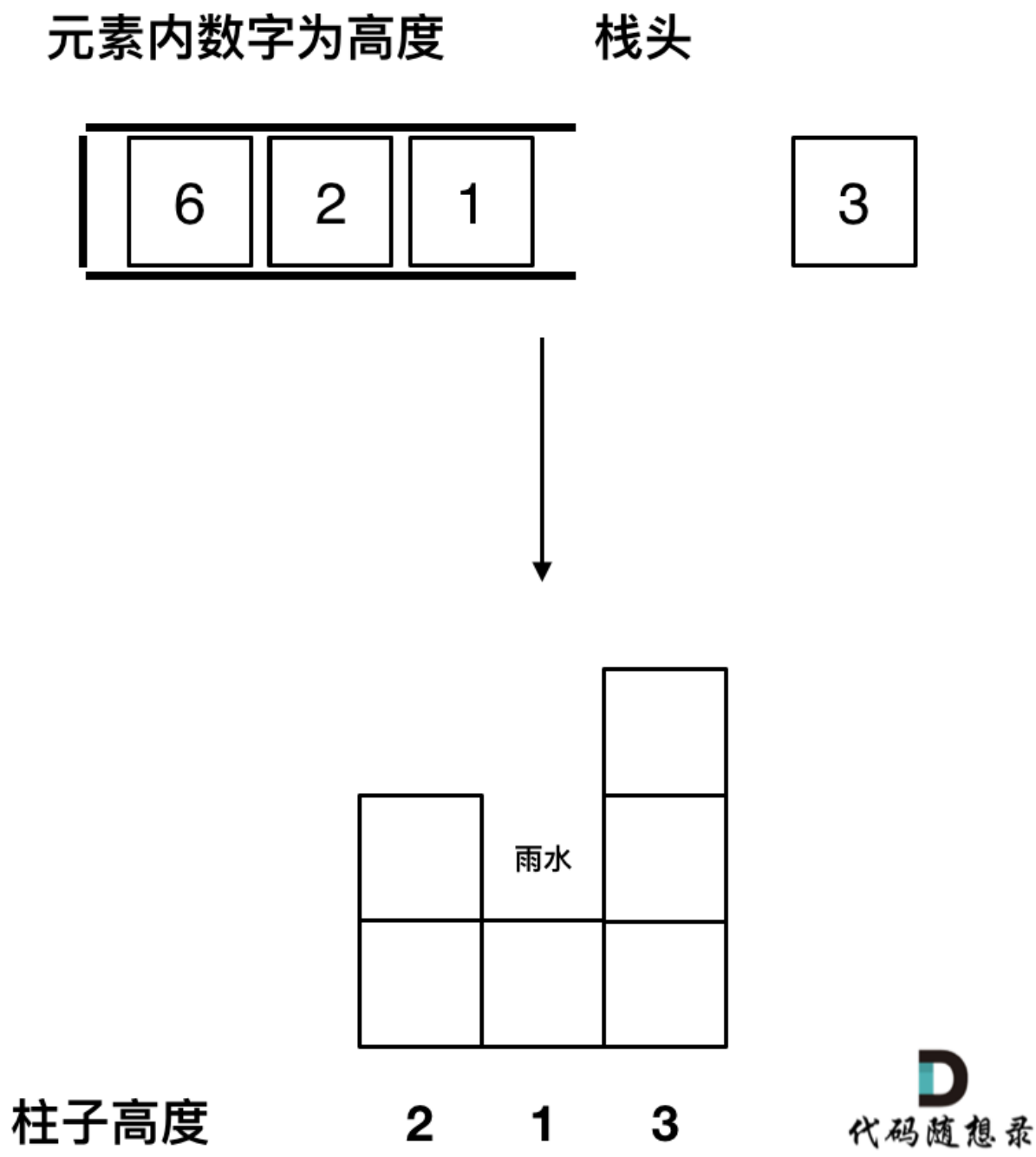
```
if (height[i] < height[st.top()])  st.push(i);
```

如果当前遍历的元素（柱子）高度等于栈顶元素的高度，要跟更新栈顶元素，因为遇到相相同高度的柱子，需要使用最右边的柱子来计算宽度。

代码如下：

```
if (height[i] == height[st.top()]) { // 例如 5 5 1 7 这种情况
  st.pop();
  st.push(i);
}
```

如果当前遍历的元素（柱子）高度大于栈顶元素的高度，此时就出现凹槽了，如图所示：



取栈顶元素，将栈顶元素弹出，这个就是凹槽的底部，也就是中间位置，下标记为mid，对应的高度为height[mid]（就是图中的高度1）。

此时的栈顶元素st.top()，就是凹槽的左边位置，下标为st.top()，对应的高度为height[st.top()]（就是图中的高度2）。

当前遍历的元素i，就是凹槽右边的位置，下标为i，对应的高度为height[i]（就是图中的高度3）。

此时大家应该可以发现其实就是**栈顶和栈顶的下一个元素以及要入栈的元素，三个元素来接水！**

那么雨水高度是 min(凹槽左边高度, 凹槽右边高度) - 凹槽底部高度，代码为：`int h = min(height[st.top()], height[i]) - height[mid];`

雨水的宽度是 凹槽右边的下标 - 凹槽左边的下标 - 1（因为只求中间宽度），代码为：`int w = i - st.top() - 1 ;`

当前凹槽雨水的体积就是：`h * w`。

求当前凹槽雨水的体积代码如下：

```
while (!st.empty() && height[i] > height[st.top()]) { // 注意这里是while，持续跟新栈顶元素
    int mid = st.top();
    st.pop();
    if (!st.empty()) {
        int h = min(height[st.top()], height[i]) - height[mid];
        int w = i - st.top() - 1; // 注意减一，只求中间宽度
        sum += h * w;
    }
}
```

关键部分讲完了，整体代码如下：

```
class Solution {
public:
    int trap(vector<int>& height) {
        if (height.size() <= 2) return 0; // 可以不加
        stack<int> st; // 存着下标，计算的时候用下标对应的柱子高度
        st.push(0);
        int sum = 0;
        for (int i = 1; i < height.size(); i++) {
            if (height[i] < height[st.top()]) {      // 情况一
                st.push(i);
            } if (height[i] == height[st.top()]) {  // 情况二
                st.pop(); // 其实这一句可以不加，效果是一样的，但处理相同的情况的思路却变了。
                st.push(i);
            } else {                                 // 情况三
                while (!st.empty() && height[i] > height[st.top()]) { // 注意这里是while
                    int mid = st.top();
                    st.pop();
                    if (!st.empty()) {
                        int h = min(height[st.top()], height[i]) - height[mid];
                        int w = i - st.top() - 1; // 注意减一，只求中间宽度
                        sum += h * w;
                    }
                }
                st.push(i);
            }
        }
        return sum;
    }
};
```

以上代码冗余了一些，但是思路是清晰的，下面我将代码精简一下，如下：

```
class Solution {
public:
    int trap(vector<int>& height) {
        stack<int> st;
        st.push(0);
        int sum = 0;
        for (int i = 1; i < height.size(); i++) {
            while (!st.empty() && height[i] > height[st.top()]) {
                int mid = st.top();
                st.pop();
                if (!st.empty()) {
                    int h = min(height[st.top()], height[i]) - height[mid];
                    int w = i - st.top() - 1;
                    sum += h * w;
                }
            }
            st.push(i);
        }
        return sum;
    }
};
```

精简之后的代码，大家就看不出去三种情况的处理了，貌似好像只处理的情况三，其实是把情况一和情况二融合了。 这样的代码不太利于理解。

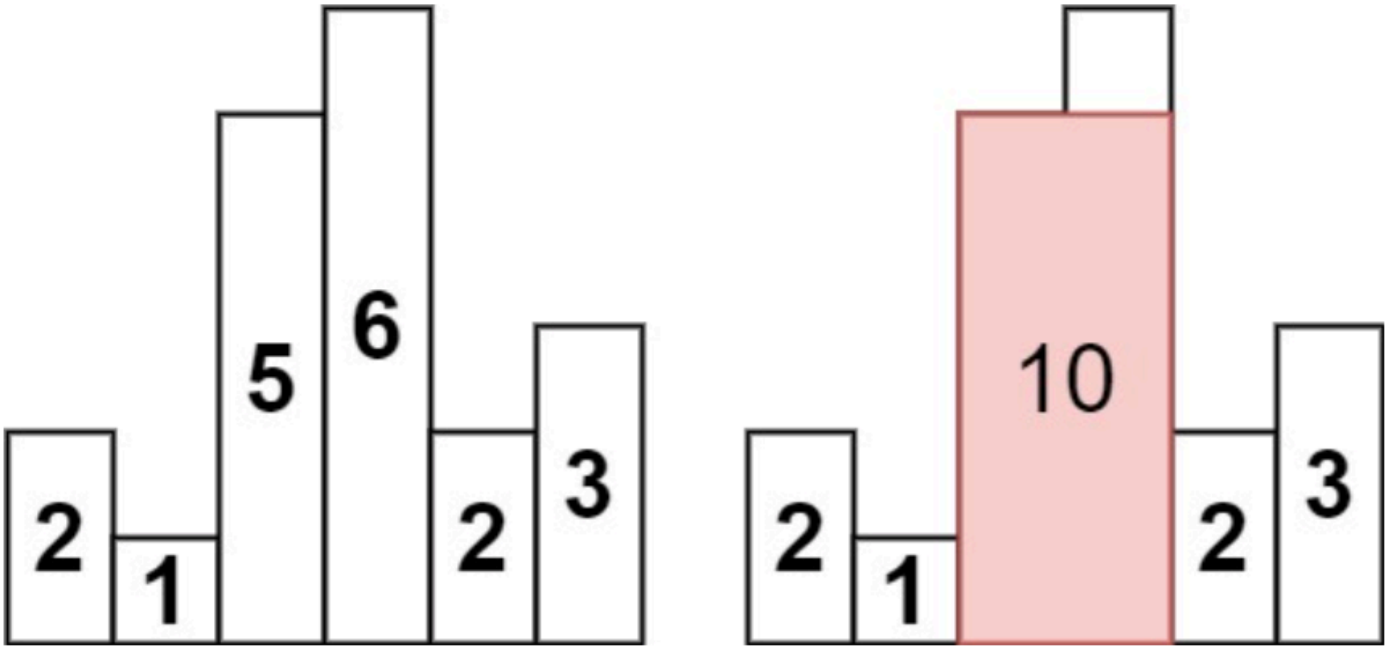
5.柱状图中最大的矩形

力扣题目链接

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1 。

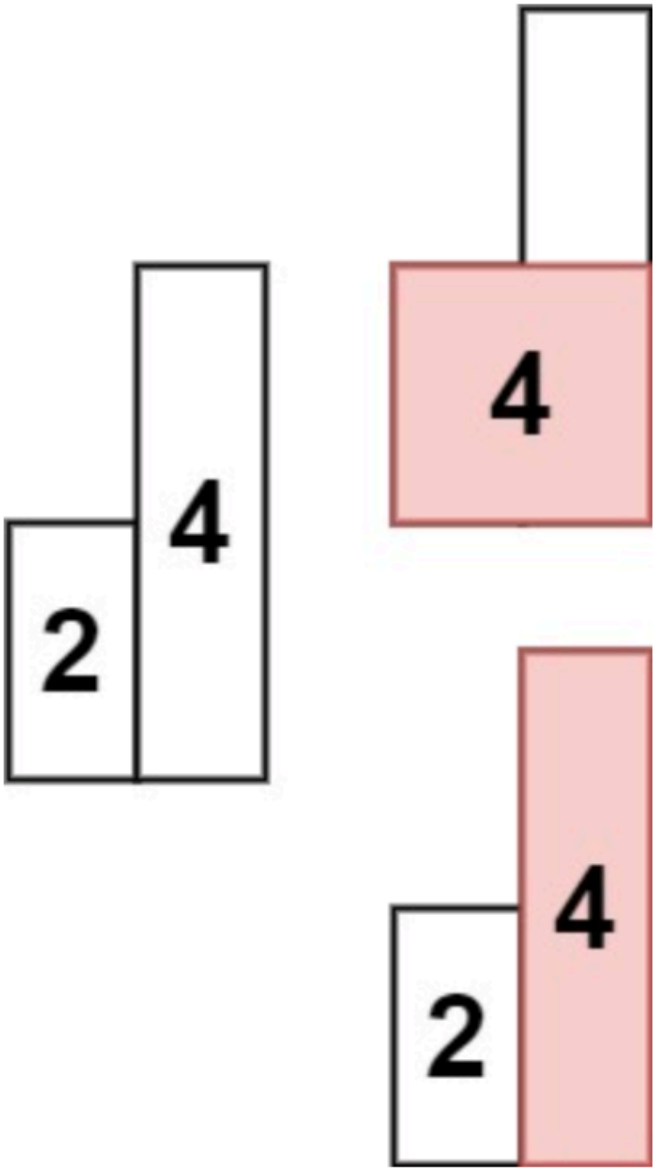
求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:



输入: `heights = [2,1,5,6,2,3]`
输出: `10`
解释: 最大的矩形为图中红色区域，面积为 `10`

示例 2：




```
输入：  heights = [2,4]
输出：  4
```

- 1 <= heights.length <=10^5
- 0 <= heights[i] <= 10^4

思路

本题和42. 接雨水，是遥相呼应的两道题目，建议都要仔细做一做，原理上有很多相同的地方，但细节上又有差异，更可以加深对单调栈的理解！

其实这两道题目先做那一道都可以，但我先写的42.接雨水的题解，所以如果没做过接雨水的话，建议先做一做接雨水，可以参考我的题解：42. 接雨水

我们先来看一下暴力解法的解法：

暴力解法

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int sum = 0;
        for (int i = 0; i < heights.size(); i++) {
            int left = i;
            int right = i;
            for (; left >= 0; left--) {
                if (heights[left] < heights[i]) break;
            }
            for (; right < heights.size(); right++) {
                if (heights[right] < heights[i]) break;
            }
            int w = right - left - 1;
            int h = heights[i];
            sum = max(sum, w * h);
        }
        return sum;
    }
};
```

如上代码并不能通过leetcode，超时了，因为时间复杂度是$O(n^2)$。

双指针解法

本题双指针的写法整体思路和42. 接雨水是一致的，但要比42. 接雨水难一些。

难就难在本题要记录记录每个柱子 左边第一个小于该柱子的下标，而不是左边第一个小于该柱子的高度。

所以需要循环查找，也就是下面在寻找的过程中使用了while，详细请看下面注释，整理思路在题解：42. 接雨水中已经介绍了。

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        vector<int> minLeftIndex(heights.size());
        vector<int> minRightIndex(heights.size());
        int size = heights.size();

        // 记录每个柱子 左边第一个小于该柱子的下标
        minLeftIndex[0] = -1; // 注意这里初始化，防止下面while死循环
        for (int i = 1; i < size; i++) {
            int t = i - 1;
            // 这里不是用if，而是不断向左寻找的过程
            while (t >= 0 && heights[t] >= heights[i]) t = minLeftIndex[t];
            minLeftIndex[i] = t;
        }
```

```
        // 记录每个柱子 右边第一个小于该柱子的下标
        minRightIndex[size - 1] = size; // 注意这里初始化，防止下面while死循环
        for (int i = size - 2; i >= 0; i--) {
            int t = i + 1;
            // 这里不是用if，而是不断向右寻找的过程
            while (t < size && heights[t] >= heights[i]) t = minRightIndex[t];
            minRightIndex[i] = t;
        }
        // 求和
        int result = 0;
        for (int i = 0; i < size; i++) {
            int sum = heights[i] * (minRightIndex[i] - minLeftIndex[i] - 1);
            result = max(sum, result);
        }
        return result;
    }
};
```

单调栈

本地单调栈的解法和接雨水的题目是遥相呼应的。

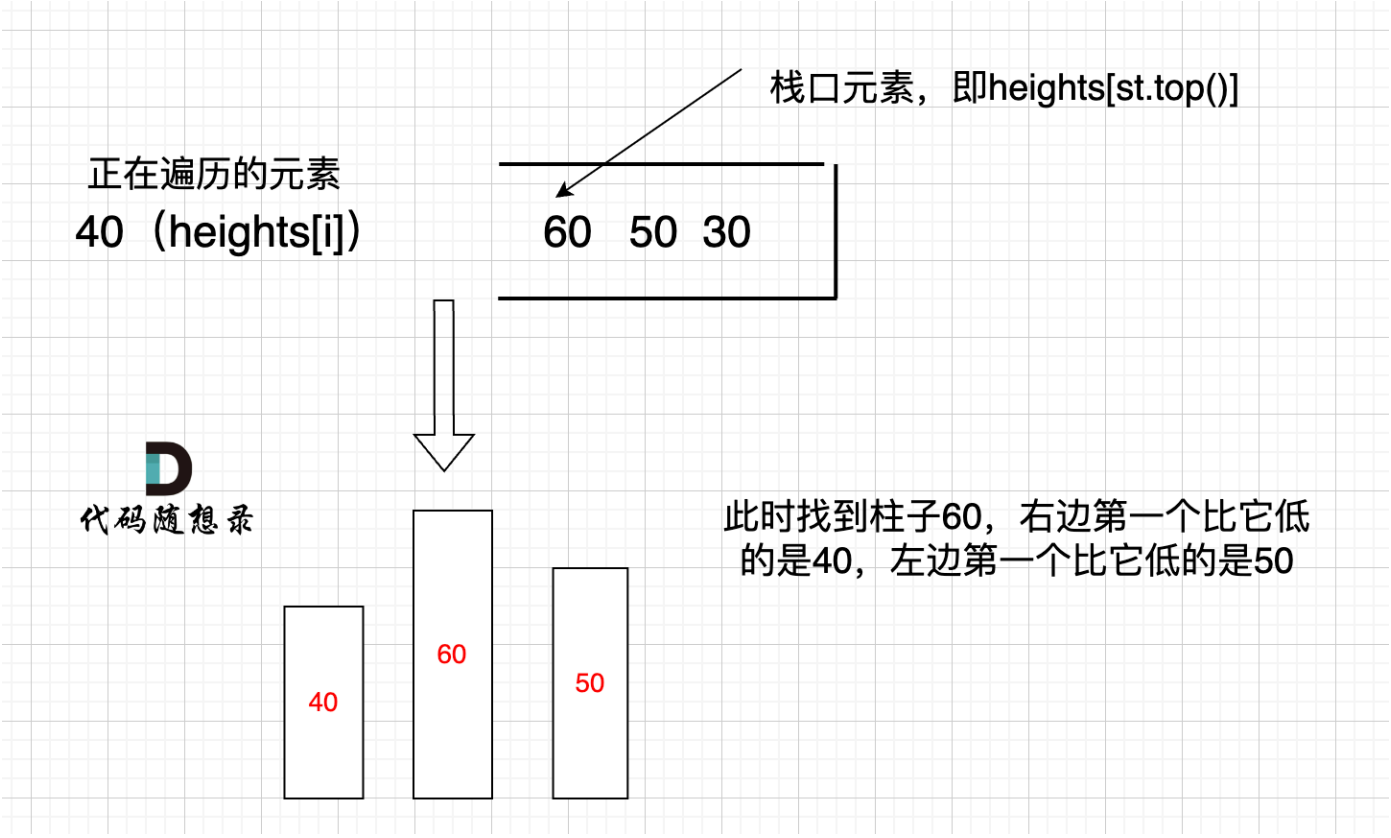
为什么这么说呢，42. 接雨水是找每个柱子左右两边第一个大于该柱子高度的柱子，而本题是找每个柱子左右两边第一个小于该柱子的柱子。

这里就涉及到了单调栈很重要的性质，就是单调栈里的顺序，是从小到大还是从大到小。

在题解42. 接雨水中我讲解了接雨水的单调栈从栈头（元素从栈头弹出）到栈底的顺序应该是从小到大的顺序。

那么因为本题是要找每个柱子左右两边第一个小于该柱子的柱子，所以从栈头（元素从栈头弹出）到栈底的顺序应该是从大到小的顺序！

我来举一个例子，如图：



只有栈里从大到小的顺序，才能保证栈顶元素找到左右两边第一个小于栈顶元素的柱子。

所以本题单调栈的顺序正好与接雨水反过来。

此时大家应该可以发现其实就是**栈顶和栈顶的下一个元素以及要入栈的三个元素组成了我们要求最大面积的高度和宽度**

理解这一点，对单调栈就掌握的比较到位了。

除了栈内元素顺序和接雨水不同，剩下的逻辑就都差不多了，在题解42. 接雨水我已经对单调栈的各个方面做了详细讲解，这里就不赘述了。

主要就是分析清楚如下三种情况：

- 情况一：当前遍历的元素heights[i]大于栈顶元素heights[st.top()]的情况
- 情况二：当前遍历的元素heights[i]等于栈顶元素heights[st.top()]的情况
- 情况三：当前遍历的元素heights[i]小于栈顶元素heights[st.top()]的情况

C++代码如下：

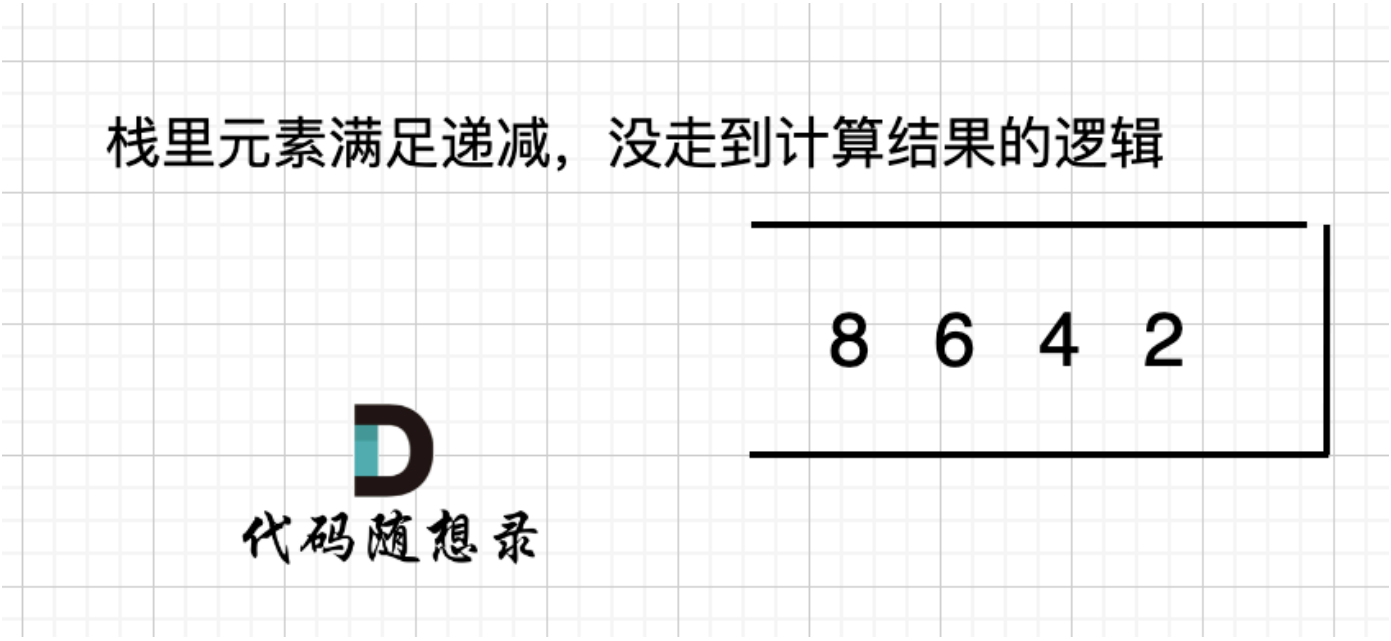
```
// 版本一
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int result = 0;
        stack<int> st;
        heights.insert(heights.begin(), 0); // 数组头部加入元素0
        heights.push_back(0); // 数组尾部加入元素0
        st.push(0);
```

```
        // 第一个元素已经入栈，从下标1开始
        for (int i = 1; i < heights.size(); i++) {
            if (heights[i] > heights[st.top()]) { // 情况一
                st.push(i);
            } else if (heights[i] == heights[st.top()]) { // 情况二
                st.pop(); // 这个可以加，可以不加，效果一样，思路不同
                st.push(i);
            } else { // 情况三
                while (!st.empty() && heights[i] < heights[st.top()]) { // 注意是while
                    int mid = st.top();
                    st.pop();
                    if (!st.empty()) {
                        int left = st.top();
                        int right = i;
                        int w = right - left - 1;
                        int h = heights[mid];
                        result = max(result, w * h);
                    }
                }
                st.push(i);
            }
        }
        return result;
    }
};
```

细心的录友会发现，我在 height数组上后，都加了一个元素0， 为什么这么做呢?

首先来说末尾为什么要加元素0?

如果数组本身就是升序的，例如[2,4,6,8]，那么入栈之后 都是单调递减，一直都没有走 情况三 计算结果的哪一步，所以最后输出的就是0了。 如图：



那么结尾加一个0，就会让栈里的所有元素，走到情况三的逻辑。

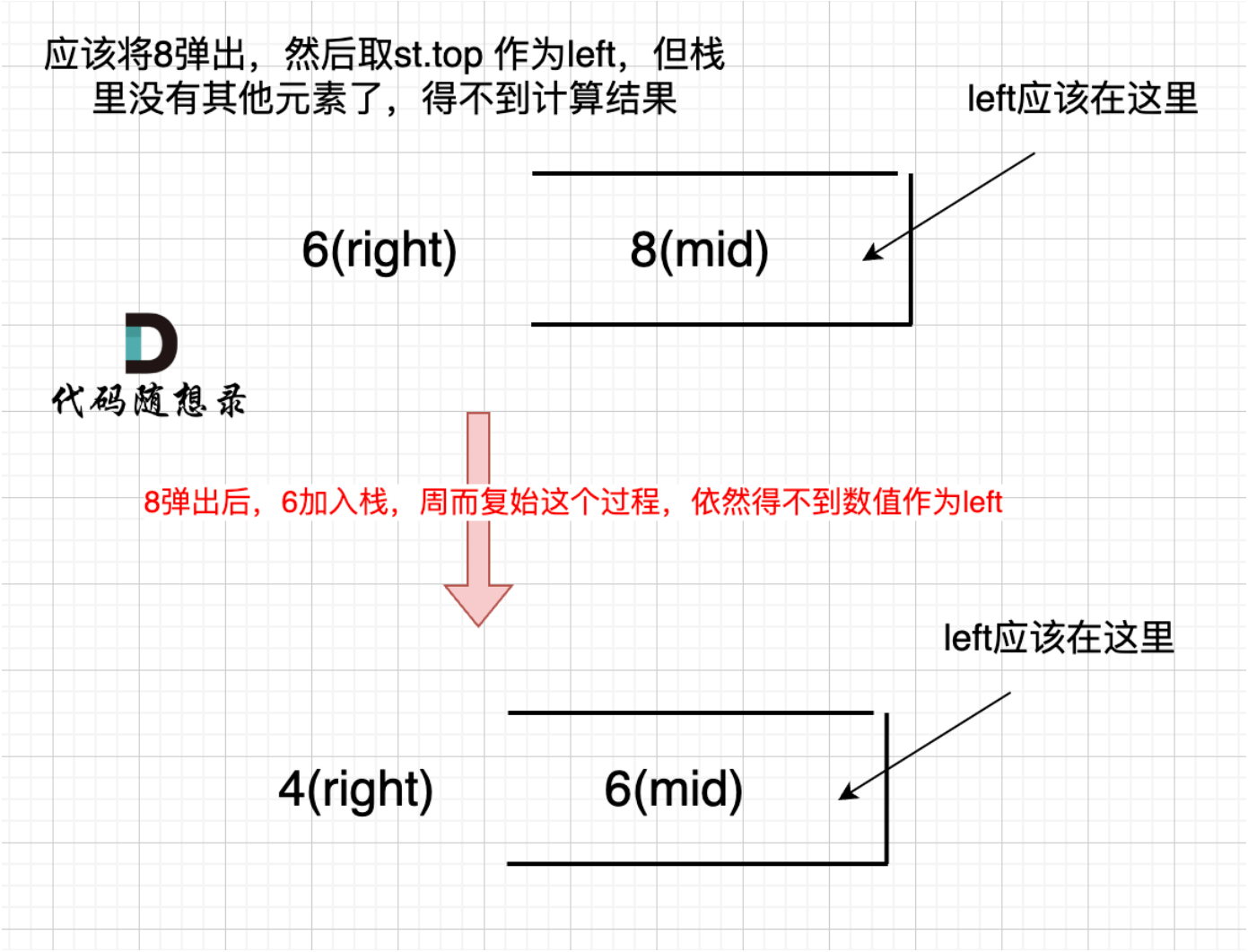
开头为什么要加元素0?

如果数组本身是降序的，例如 [8,6,4,2]，在 8 入栈后，6 开始与8 进行比较，此时我们得到 mid（8），rigt（6），但是得不到 left。

（mid、left，right 都是对应版本一里的逻辑）

因为 将 8 弹出之后，栈里没有元素了，那么为了避免空栈取值，直接跳过了计算结果的逻辑。

之后又将6 加入栈（此时8已经弹出了），然后 就是 4 与 栈口元素 8 进行比较，周而复始，那么计算的最后结果resutl就是0。 如图所示：



所以我们需要在 height数组前后各加一个元素0。

版本一代码精简之后：

```
// 版本二
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<int> st;
```

```
        heights.insert(heights.begin(), 0); // 数组头部加入元素0
        heights.push_back(0); // 数组尾部加入元素0
        st.push(0);
        int result = 0;
        for (int i = 1; i < heights.size(); i++) {
            while (heights[i] < heights[st.top()]) {
                int mid = st.top();
                st.pop();
                int w = i - st.top() - 1;
                int h = heights[mid];
                result = max(result, w * h);
            }
            st.push(i);
        }
        return result;
    }
};
```

这里我依然建议大家按部就班把版本一写出来，把情况一二三分析清楚，然后在精简代码到版本二。 直接看版本二容易忽略细节！